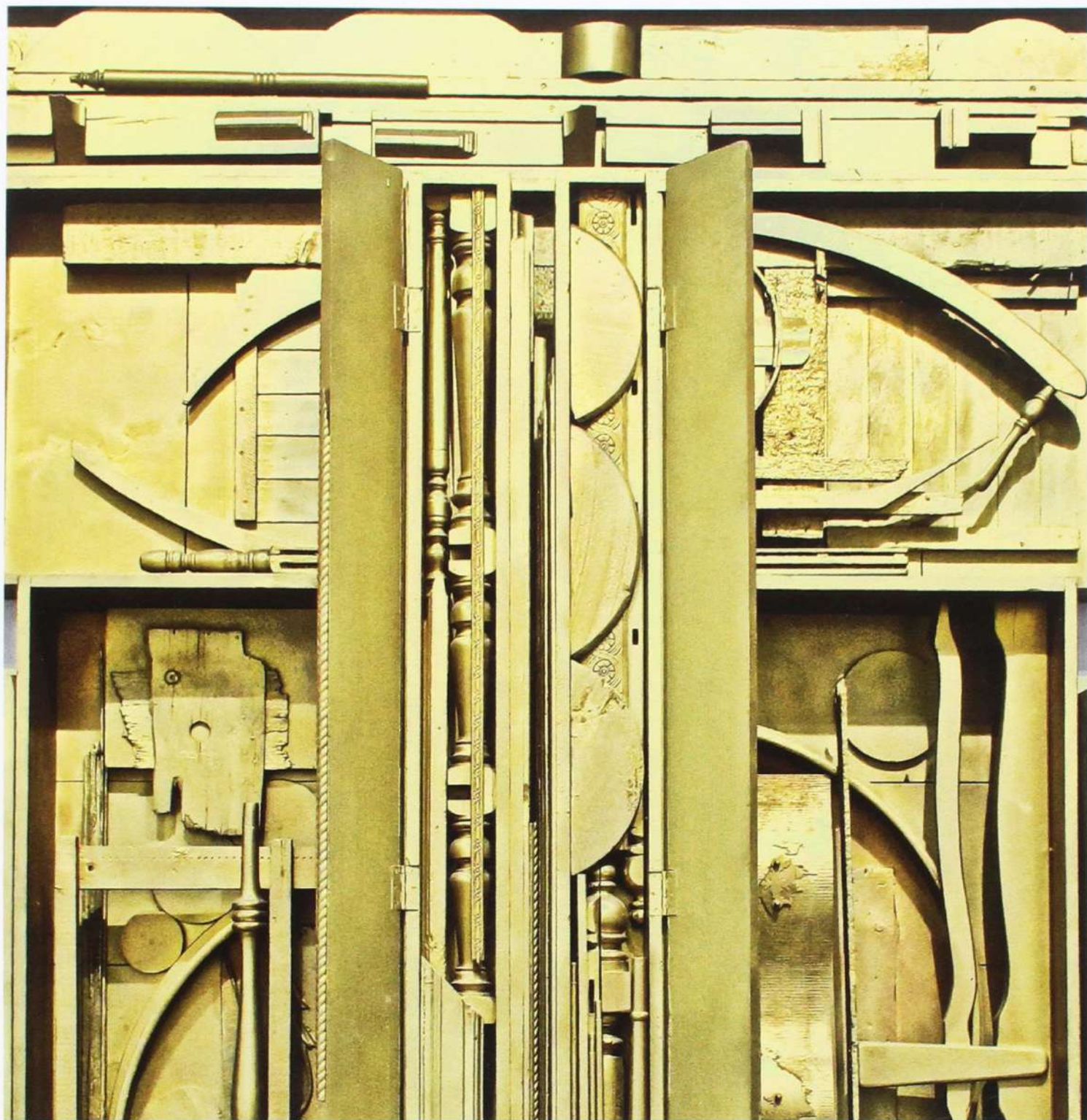




Sarah L. Harris David Money Harris

Sistemi digitali e architettura dei calcolatori

Progettare con tecnologia ARM

Edizione italiana a cura di Nello Scarabottolo

LIBRO MULTIMEDIALE 

ZANICHELLI

Indice generale

<i>Prefazione degli autori</i>	XIII
<i>Ringraziamenti</i>	XV

1.9 ■ Riassunto e anticipazione	27
Esercizi	29
Domande di valutazione	34

Capitolo 1 Da zero a uno

1.1 ■ La pianificazione del gioco	1
1.2 ■ L'arte di gestire la complessità	2
1.2.1 L'astrazione	2
1.2.2 La disciplina	3
1.2.3 Le tre -Y	3
1.3 ■ L'astrazione digitale	5
1.4 ■ I sistemi numerici	6
1.4.1 Numeri decimali	6
1.4.2 Numeri binari	7
1.4.3 Numeri esadecimali	8
1.4.4 Byte, nibble, word	9
1.4.5 Somma binaria	10
1.4.6 Numeri binari relativi	11
1.5 ■ Le porte logiche	14
1.5.1 La porta NOT	15
1.5.2 Buffer	15
1.5.3 La porta AND	15
1.5.4 La porta OR	15
1.5.5 Altre porte logiche a due ingressi	15
1.5.6 Porte a ingressi multipli	16
1.6 ■ Oltre l'astrazione digitale	16
1.6.1 Tensione	17
1.6.2 Livelli logici	17
1.6.3 Margini di rumore	17
1.6.4 Caratteristica di trasferimento DC	18
1.6.5 La disciplina statica	19
1.7 ■ I transistori CMOS*	20
1.7.1 Semiconduttori	20
1.7.2 I diodi	21
1.7.3 I condensatori	21
1.7.4 Transistori nMOS e pMOS	22
1.7.5 Porta NOT CMOS	24
1.7.6 Altre porte logiche CMOS	24
1.7.7 Porte di trasmissione	25
1.7.8 Logica pseudo-nMOS	26
1.8 ■ Consumo di potenza*	26

Capitolo 2 Progetto di reti logiche combinatorie

2.1 ■ Introduzione	35
2.2 ■ Espressioni booleane	37
2.2.1 Terminologia	38
2.2.2 Forma somma di prodotti	38
2.2.3 Forma prodotto di somme	39
2.3 ■ Algebra booleana	40
2.3.1 Postulati	40
2.3.2 Teoremi a una variabile	41
2.3.3 Teoremi di più variabili	41
2.3.4 Saranno veri i teoremi booleani?	43
2.3.5 Semplificare le espressioni	43
2.4 ■ Dalla logica alle porte	45
2.5 ■ Logica combinatoria su più di due livelli	47
2.5.1 Riduzione dell'hardware	48
2.5.2 Spingere le bolle	49
2.6 ■ Non solo 0 e 1, anche X e Z	50
2.6.1 Il valore illegale: X	50
2.6.2 Il valore fluttuante: Z	51
2.7 ■ Le mappe di Karnaugh	52
2.7.1 Pensare in cerchi	53
2.7.2 Minimizzazione logica con le mappe di Karnaugh	54
2.7.3 Indifferenze	57
2.7.4 Il quadro generale	57
2.8 ■ Blocchi costitutivi combinatori	58
2.8.1 Multiplexer	58
2.8.2 Decoder	61
2.9 ■ Temporizzazioni	62
2.9.1 Ritardi di propagazione e di contaminazione	62
2.9.2 Alee	65
2.10 ■ Riassunto	67
Esercizi	69
Domande di valutazione	72

Capitolo 3 Progetto di logica sequenziale

3.1	■ Introduzione	73
3.2	■ Latch e flip-flop	73
3.2.1	Latch SR	75
3.2.2	Latch D	76
3.2.3	Flip-flop D	77
3.2.4	Registro	78
3.2.5	Flip-flop con abilitazione	78
3.2.6	Flip-flop resettabile	78
3.2.7	Progetto di latch e flip-flop a livello di transistori	79
3.2.8	Per riassumere	80
3.3	■ Progetto di reti logiche sincrone	81
3.3.1	Alcune reti problematiche	81
3.3.2	Reti sequenziali sincrone	82
3.3.3	Reti sincrone e asincrone	84
3.4	■ Macchine a stati finiti	84
3.4.1	Esempio di progettazione di una FSM	85
3.4.2	Codifica degli stati	89
3.4.3	Macchine alla Moore e macchine alla Mealy	92
3.4.4	Fattorizzazione delle macchine a stati	94
3.4.5	Derivare una FSM da uno schema circuitale	95
3.4.6	Riassunto sulle FSM	98
3.5	■ Temporizzazione della logica sequenziale	98
3.5.1	La disciplina dinamica	99
3.5.2	Temporizzazione del sistema	100
3.5.3	Sfasamento del clock*	104
3.5.4	Metastabilità	107
3.5.5	Sincronizzatori	108
3.5.6	Formulazione del tempo di risoluzione*	110
3.6	■ Parallelismo	112
3.7	■ Riassunto	115
	Esercizi	116
	Domande di valutazione	120

Capitolo 4 Linguaggi di descrizione dell'hardware

4.1	■ Introduzione	121
4.1.1	Moduli	121

4.1.2	Origini dei linguaggi	122
4.1.3	Simulazione e sintesi	123
4.2	■ Logica combinatoria	125
4.2.1	Operatori a singolo bit	125
4.2.2	Commenti e spazio vuoto	127
4.2.3	Operatori di riduzione	127
4.2.4	Assegnamento condizionale	128
4.2.5	Variabili interne	129
4.2.6	Precedenza	131
4.2.7	Numeri	132
4.2.8	Z e X	132
4.2.9	Concatenazione di bit	134
4.2.10	Ritardi	134
4.3	■ Modellazione strutturale	135
4.4	■ Logica sequenziale	138
4.4.1	Registri	138
4.4.2	Registri resettabili	140
4.4.3	Registri con abilitazione	141
4.4.4	Registri multipli	142
4.4.5	Latch	143
4.5	■ Ancora logica combinatoria	143
4.5.1	Istruzione case	145
4.5.2	Istruzione if	145
4.5.3	Tabelle delle verità con indifferenze	147
4.5.4	Assegnamenti bloccanti e non bloccanti	147
4.6	■ Macchine a stati finiti	153
4.7	■ Tipi di dati*	157
4.7.1	SystemVerilog	157
4.7.2	VHDL	158
4.8	■ Moduli parametrici*	160
4.9	■ Testbench	163
4.10	■ Riassunto	167
	Esercizi	168
	Domande di valutazione	174

Capitolo 5 Blocchi costruttivi digitali

5.1	■ Introduzione	175
5.2	■ Circuiti aritmetici	175
5.2.1	Addizione	175
5.2.2	Sottrazione	181
5.2.3	Comparatori	182
5.2.4	ALU	183
5.2.5	Traslatori e rotatori	185
5.2.6	Moltiplicazione*	186

5.2.7	Divisione*	188
5.2.8	Lecture aggiuntive	189
5.3	■ Sistemi di numerazione	189
5.3.1	Numeri in virgola fissa	189
5.3.2	Numeri in virgola mobile*	190
5.4	■ Blocchi costruttivi sequenziali	193
5.4.1	Contatori	193
5.4.2	Registri a scorrimento	194
5.5	■ Componenti di memoria	196
5.5.1	Panoramica	197
5.5.2	Memoria ad accesso casuale dinamica	199
5.5.3	Memoria ad accesso casuale statica	199
5.5.4	Area e ritardo	199
5.5.5	Banchi di registri	200
5.5.6	Memorie a sola lettura	200
5.5.7	Reti logiche realizzate con componenti di memoria	202
5.5.8	Descrizione HDL delle memorie	202
5.6	■ Matrici logiche	204
5.6.1	Matrici logiche programmabili	204
5.6.2	Matrici di porte logiche programmabili sul campo	205
5.6.3	Realizzazione delle matrici di memoria*	209
5.7	■ Riassunto	210
	Esercizi	212
	Domande di valutazione	216

Capitolo 6 Architettura

6.1	■ Introduzione	217
6.2	■ Il linguaggio <i>assembly</i>	218
6.2.1	Istruzioni	218
6.2.2	Operandi: registri, memoria e costanti	220
6.3	■ Programmare	224
6.3.1	Istruzioni di elaborazione dati	224
6.3.2	Flag di condizione	226
6.3.3	Salto	227
6.3.4	Costrutti di selezione	229
6.3.5	Cicli	230
6.3.6	La memoria	232
6.3.7	Chiamate a sottoprogrammi	235
6.4	■ Linguaggio macchina	244
6.4.1	Istruzioni di elaborazione dati	244
6.4.2	Istruzioni di accesso a memoria	247

6.4.3	Istruzioni di salto	248
6.4.4	Modi di indirizzamento	250
6.4.5	Interpretare il linguaggio macchina	250
6.4.6	La potenza di un programma scritto in memoria	251

6.5	■ Compilare, assemblare e caricare*	252
6.5.1	La mappa di memoria	252
6.5.2	Compilazione	253
6.5.3	Assemblaggio	254
6.5.4	Collegamento	255
6.5.5	Caricamento	256
6.6	■ Qualche dettaglio	257
6.6.1	Caricamento di <i>literal</i>	257
6.6.2	NOP	258
6.6.3	Eccezioni	258
6.7	■ Evoluzione dell'architettura ARM	261
6.7.1	Set di istruzione <i>Thumb</i>	262
6.7.2	Istruzioni DSP	263
6.7.3	Istruzioni in virgola mobile	266
6.7.4	Istruzioni per il risparmio di potenza e per la sicurezza	267
6.7.5	Istruzioni SIMD	268
6.7.6	Architettura a 64 bit	269
6.8	■ Un'altra prospettiva: l'architettura x86	269
6.8.1	Registri x86	270
6.8.2	Operandi x86	271
6.8.3	Flag di stato	272
6.8.4	Istruzioni x86	272
6.8.5	Codifica delle istruzioni x86	272
6.8.6	Altre particolarità di x86	275
6.8.7	Il quadro generale	275
6.9	■ Riassunto	275
	Esercizi	276
	Domande di valutazione	282

Capitolo 7 Microarchitettura

7.1	■ Introduzione	283
7.1.1	Stato architetturale e set di istruzioni	283
7.1.2	Progettazione	284
7.1.3	Microarchitetture	285
7.2	■ Analisi delle prestazioni	286

7.3	■ Processore a ciclo singolo	287
7.3.1	Percorso dati a ciclo singolo	287
7.3.2	Unità di controllo a ciclo singolo	293
7.3.3	Istruzioni aggiuntive	296
7.3.4	Analisi delle prestazioni	298
7.4	■ Processore multi ciclo	300
7.4.1	Percorso dati multi ciclo	300
7.4.2	Unità di controllo multi ciclo	306
7.4.3	Analisi delle prestazioni	313
7.5	■ Processore pipeline	316
7.5.1	Percorso dati pipeline	318
7.5.2	Unità di controllo della pipeline	319
7.5.3	Dipendenze	321
7.5.4	Analisi delle prestazioni	328
7.6	■ Rappresentazione HDL*	330
7.6.1	Processore a ciclo singolo	331
7.6.2	Altri blocchi costruttivi	337
7.6.3	Testbench	339
7.7	■ Microarchitetture avanzate*	343
7.7.1	Pipeline lunghe	344
7.7.2	Micro operazioni	344
7.7.3	Previsione dei salti	345
7.7.4	Processori superscalari	347
7.7.5	Processore <i>out-of-order</i>	349
7.7.6	Ridenominazione dei registri	351
7.7.7	<i>Multithreading</i>	352
7.7.8	Multiprocessori	353
7.8	■ Uno sguardo al mondo reale: evoluzione dell'architettura ARM*	355
7.9	■ Riassunto	359
	Esercizi	361
	Domande di valutazione	364

Capitolo 8 Sistemi di memoria

8.1	■ Introduzione	365
8.2	■ Analisi delle prestazioni del sistema di memoria	368
8.3	■ Memoria cache	370
8.3.1	Quali dati devono essere memorizzati nella cache?	370
8.3.2	Come si verifica se un dato è in cache?	371
8.3.3	Quale dato viene sostituito?	378
8.3.4	Progetto di cache avanzate*	378
8.3.5	Evoluzione delle cache di ARM	382

8.4	■ Memoria virtuale	382
8.4.1	Traduzione dell'indirizzo	384
8.4.2	La tabella delle pagine	386
8.4.3	Il <i>Translation Lookaside Buffer</i> (TLB)	387
8.4.4	Protezione della memoria	388
8.4.5	Politiche di sostituzione*	389
8.4.6	Tabelle delle pagine multi livello*	389
8.5	■ Riassunto	391
	Epilogo	391
	Esercizi	392
	Domande di valutazione	396

Capitolo 9 Sistemi di ingresso/uscita



Capitolo disponibile online

9.1	■ Introduzione	1
9.2	■ I/O mappato in memoria	1
9.3	■ I/O nei sistemi embedded	3
9.3.1	Il sistema a singolo chip BCM2835	3
9.3.2	Driver di dispositivo	4
9.3.3	I/O digitali di uso generale	7
9.3.4	I/O seriale	9
9.3.5	Timer	18
9.3.6	I/O analogici	19
9.3.7	Interrupt	25
9.4	■ Altre periferiche di microcontrollori	26
9.4.1	Display di caratteri a cristalli liquidi	26
9.4.2	Monitor VGA	29
9.4.3	Comunicazioni wireless Bluetooth	33
9.4.4	Controllo di motori	35
9.5	■ Interfacce a bus	43
9.5.1	AHB-Lite	43
9.5.2	Esempio di interfaccia a memoria e periferica	44
9.6	■ Sistemi di I/O del PC	47
9.6.1	USB	48
9.6.2	PCI e PCI Express	48
9.6.3	Memoria DDR3	49
9.6.4	Interconnessione in rete	49
9.6.5	SATA	50
9.6.6	Interfacciamento a un PC	50
9.7	■ Riassunto	52

Appendice A Realizzazione dei sistemi digitali

A.1	■ Introduzione	397
A.2	■ La logica 74xx	397
A.2.1	Porte logiche	398
A.2.2	Altre funzioni	398
A.3	■ Logica programmabile	398
A.3.1	PROM	398
A.3.2	PLA	402
A.3.3	FPGA	402
A.4	■ Circuiti integrati specifici per un'applicazione	404
A.5	■ Data sheet	404
A.6	■ Famiglie logiche	408
A.7	■ Packaging e assemblaggio	411
A.8	■ Linee di trasmissione	414
A.8.1	Terminazione adattata	416
A.8.2	Terminazione aperta	417
A.8.3	Terminazione cortocircuitata	418
A.8.4	Terminazione non adattata	418
A.8.5	Quando serve usare i modelli delle linee di trasmissione	420
A.8.6	Corrette terminazioni delle linee di trasmissione	421
A.8.7	Espressione di Z_0^*	422
A.8.8	Espressione del coefficiente di riflessione*	423
A.8.9	Riassumendo	424
A.9	■ Aspetti economici	425

Appendice B Istruzioni ARM

B.1	■ Istruzioni di elaborazione dati	427
B.1.1	Istruzioni di moltiplicazione	427
B.2	■ Istruzioni di accesso a memoria	429
B.3	■ Istruzioni di salto	429
B.4	■ Istruzioni varie	430
B.5	■ Flag di condizione	430

Appendice C Programmazione in C

Capitolo disponibile online

C.1	■ Introduzione	1
C.2	■ Benvenuti al linguaggio C	2
C.2.1	Struttura di un programma C	3
C.2.2	Esecuzione di un programma C	3
C.3	■ Compilazione	4
C.3.1	Commenti	4
C.3.2	#define	5
C.3.3	#include	5
C.4	■ Variabili	6
C.4.1	Tipi di dati primitivi	7
C.4.2	Variabili globali e locali	8
C.4.3	Inizializzazione delle variabili	9
C.5	■ Operatori	10
C.6	■ Chiamate di funzione	12
C.7	■ Istruzioni di controllo del flusso di esecuzione	13
C.7.1	Istruzioni condizionali	13
C.7.2	Cicli	15
C.8	■ Altri tipi di dati	17
C.8.1	Puntatori	17
C.8.2	Array	18
C.8.3	Caratteri	22
C.8.4	Stringhe	23
C.8.5	Strutture	24
C.8.6	typedef	25
C.8.7	Allocazione dinamica della memoria*	26
C.8.8	Liste collegate*	27
C.9	■ Librerie standard	29
C.9.1	stdio	29
C.9.2	stdlib	33
C.9.3	math	34
C.9.4	string	35
C.10	■ Opzioni del compilatore e argomenti nella riga di comando	35
C.10.1	Compilare più file sorgente C	35
C.10.2	Opzioni del compilatore	35
C.10.3	Argomenti nella riga di comando	35
C.11	■ Errori frequenti	36
	Indice analitico	431

Prefazione degli autori

Questo libro è unico nel senso che presenta il progetto della logica digitale dal punto di vista dell'architettura dei calcolatori, partendo dagli 1 e dagli 0, procedendo fino al progetto di un microprocessore.

Riteniamo che costruire un microprocessore sia uno speciale rito di passaggio per gli studenti di ingegneria e di informatica. Il funzionamento interno di un processore può sembrare quasi magico agli occhi degli inesperti, ma diventa chiarissimo se spiegato con cura. Il progetto digitale di per sé è un argomento vasto e interessante. Programmare in linguaggio assembly rivela il linguaggio interno del processore. La microarchitettura è il ponte di collegamento che unisce il tutto.

Le prime due edizioni di questo testo, la cui popolarità va aumentando, trattavano l'architettura MIPS adeguandosi all'impostazione data dai diffusi testi di architettura dei calcolatori di Patterson e Hennessy. Il MIPS – una delle prime architetture RISC (*Reduced Instruction Set Computer*) – è chiaro e facile da comprendere e costruire. MIPS rimane un'architettura importante, e ha ricevuto nuova energia dopo il suo acquisto nel 2013 da parte della Imagination Technologies.

Negli ultimi due decenni, l'architettura ARM è esplosa in popolarità grazie alla sua efficienza e ricchezza. Sono stati prodotti più di 50 miliardi di processori ARM, e più del 75% della popolazione mondiale utilizza prodotti che contengono processori ARM. Al momento della stesura di questo libro, praticamente ogni telefono cellulare e ogni tablet contengono uno o più processori ARM e le previsioni dicono che migliaia di milioni di processori ARM controlleranno in futuro la cosiddetta *Internet of Things* (Internet delle cose). Molte aziende stanno costruendo sistemi ARM ad alte prestazioni per poter concorrere nel mercato dei server contro i processori Intel. Vista l'importanza commerciale e l'interesse da parte degli studenti, abbiamo deciso di sviluppare questa edizione ARM del nostro libro.

Da un punto di vista pedagogico, gli obiettivi di apprendimento dell'edizione MIPS e dell'edizione ARM sono identici. L'architettura ARM possiede un numero di funzioni, inclusi i metodi di indirizzamento e l'esecuzione condizionale, che contribuiscono all'efficacia del processore ma aumentano un po' il livello di complessità. Anche le microarchitetture sono molto simili: le differenze maggiori sono l'esecuzione condizionale e il program counter. Il capitolo sull'ingresso/uscita fornisce numerosi esempi utilizzando il Raspberry Pi, un diffusissimo calcolatore Linux su singola scheda basato su ARM.

Ci aspettiamo di offrire entrambe le edizioni MIPS e ARM finché il mercato lo richiede.

CARATTERISTICHE

Trattazione fianco a fianco di SystemVerilog e VHDL

I linguaggi di descrizione dell'hardware (HDL, *Hardware Description Languages*) sono il punto chiave delle moderne pratiche di progettazione digitale. Ma i progettisti si dividono quasi equamente tra due linguaggi dominanti: SystemVerilog e VHDL. In questo libro, gli HDL sono trattati nel Capitolo 4 dopo la presentazione della logica combinatoria e di quella sequenziale. Gli HDL vengono utilizzati successivamente nei Capitoli 5 e 7 per progettare

blocchi di costruttivi più complessi e interi processori. È comunque possibile tralasciare il Capitolo 4: i capitoli successivi rimangono utilizzabili anche per quei corsi che non affrontano lo studio degli HDL.

Questo libro è unico nella sua presentazione fianco a fianco di SystemVerilog e VHDL, che permette al lettore di apprendere entrambi i linguaggi. Il Capitolo 4 descrive i principi che si applicano a entrambi gli HDL, e successivamente presenta in colonne adiacenti la sintassi e gli esempi d'uso dei due linguaggi specifici. Questa doppia trattazione permette inoltre a un docente di scegliere uno dei due HDL, e al lettore di passare facilmente da uno all'altro, sia in aula sia nella pratica professionale.

Architettura e microarchitettura ARM

I Capitoli 6 e 7 offrono la prima esposizione dettagliata dell'architettura e della microarchitettura di ARM. ARM è un'architettura perfetta per un libro, in quanto è un'architettura reale presente in milioni di prodotti ogni anno, pur restando un'architettura agile e facile da imparare. Inoltre, data la sua popolarità nel mondo commerciale e degli hobbisti, esistono per questa architettura numerosi simulatori e strumenti di sviluppo. Tutti i materiali relativi alla tecnologia ARM sono stati riprodotti con permesso rilasciato dalla ARM Limited.

Prospettive del mondo reale

Oltre alla prospettiva del mondo reale offerta presentando l'architettura ARM, il Capitolo 6 illustra l'architettura dei processori Intel x86 per dare una prospettiva alternativa. Il Capitolo 9 (disponibile come risorsa aggiuntiva online) descrive anche le periferiche relative al calcolatore su scheda singola Raspberry Pi, una piattaforma basata su ARM molto diffusa. Queste prospettive sul mondo reale mostrano come i concetti esposti nei capitoli si traducano nei chip presenti in molti personal computer e in altri prodotti elettronici di largo consumo.

Panoramica accessibile di una microarchitettura avanzata

Il Capitolo 7 include una visione d'insieme sulle caratteristiche architetturali di una moderna microarchitettura ad alte prestazioni, come la predizione dei salti, l'esecuzione superscalare e *out-of-order*, il *multithreading*, i processori *multicore*. La trattazione è accessibile anche agli studenti dei primi anni e mostra come le microarchitetture esposte nel libro possano essere estese ai processori moderni.

Esercizi ricapitolativi e domande per colloqui

Il modo migliore per imparare la progettazione digitale è metterla in pratica. Ogni capitolo termina quindi con degli esercizi ricapitolativi per fare pratica sull'argomento. Gli esercizi sono seguiti da alcune domande di valutazione che i nostri colleghi dell'industria hanno posto a studenti candidatisi per posti di lavoro in questo campo. Queste domande forniscono un'anteprima utile dei tipi di problemi davanti ai quali vengono messi i candidati a un posto di lavoro durante il colloquio di assunzione. Le soluzioni agli esercizi sono disponibili sul sito del libro (online.universita.zanichelli.it/harris-arm).

LE RISORSE MULTIMEDIALI

All'indirizzo online.universita.zanichelli.it/harris-arm sono disponibili le seguenti risorse:

- le soluzioni degli esercizi con numero dispari (in lingua inglese);
- il Capitolo 9, sui sistemi I/O;
- l'appendice C, sulla programmazione in C;
- i materiali per il laboratorio (in lingua inglese);
- il codice HDL del processore ARM (in lingua inglese).

COME USARE GLI STRUMENTI SOFTWARE IN UN CORSO

Altera Quartus II

Quartus II Web Edition è una versione gratuita dello strumento professionale di progettazione di componenti FPGA Quartus II. Questo strumento permette agli studenti di costruire i propri progetti digitali come schemi circuitali o di utilizzare i linguaggi di descrizione hardware (HDL) SystemVerilog e VHDL. Una volta costruito il progetto, gli studenti possono lanciare una simulazione della struttura circuitale utilizzando ModelSimTM-Altera Starter Edition, disponibile all'interno di Altera Quartus II Web Edition. Quartus II Web Edition include anche uno strumento di sintesi logica che supporta sia SystemVerilog sia VHDL.

La differenza tra Web Edition e Subscription Edition è che la prima supporta solo un sottoinsieme dei componenti FPGA di Altera più diffusi. La differenza tra ModelSim-Altera Starter Edition e le versioni commerciali di ModelSim è che la Starter Edition offre prestazioni degradate per simulazioni di codici con più di 10 000 righe di HDL.

ARM Microcontroller Development Kit (MDK-ARM) di Keil

MDK-ARM della Keil è uno strumento per sviluppare programmi per il processore ARM, disponibile gratuitamente su web. Include un compilatore ARM C commerciale e un simulatore che permette agli studenti di scrivere programmi sia in linguaggio C sia in linguaggio assembly, di compilarli e di simularli.

RINGRAZIAMENTI

Ringraziamo il duro lavoro di Nate McFadden, Joe Hayton, Punithavathy Govindaradjane e di tutti gli altri componenti del gruppo di lavoro della Morgan Kaufmann, che ha reso possibile la realizzazione di questo volume. Apprezziamo moltissimo il lavoro di Duane Bibby, le cui vignette animano i capitoli.

Ringraziamo Matthew Watkins che ha contribuito al paragrafo sui multiprocessori eterogenei nel Capitolo 7. Abbiamo molto apprezzato il lavoro di Joshua Vasquez che ha sviluppato il codice per il Raspberry Pi nel Capitolo 9. Ringraziamo anche Josef Spjut e Ruye Wang, che hanno collaudato il materiale in classe.

Numerosi revisori hanno migliorato sostanzialmente questo libro. Tra questi ricordiamo Boyang Wang, John Barr, Jack V. Briner, Andrew C. Brown, Carl Baumgaertner, A. Utku Diril, Jim Frenzel, Jaeha Kim, Phillip King, James PinterLucke, Amir Roth, Z. Jerry Shi, James E. Stine, Luke Teysier, Peiyi Zhao, Zach Dodds, Nathaniel Guy, Aswin Krishna, Volnei Pedroni, Karl Wang, Ricardo Jasinski, Josef Spjut, Jörgen Lien, Sameer Sharma, John Nestor, Syed Manzoor, James Hoe, Srinivasa Vemuru, K. Joseph Hass, Jayantha Herath, Robert Mullins, Bruno Quoitin, Subramaniam Ganesan, Braden Phillips, John Oliver, Yahswant K. Malaiya, Mohammad Awedh, Zachary Kurmas, Donald Hung, e un revisore anonimo. Ringraziamo inoltre Khaled Benkrid e i suoi colleghi della ARM per aver esaminato attentamente i materiali relativi ad ARM.

Ringraziamo anche gli studenti dei nostri corsi allo Harvey Mudd College e a UNLV che ci hanno dato le loro utili opinioni sulle bozze di questo libro di testo. Sono degni di menzione Clinton Barnes, Matt Weiner, Carl Walsh, Andrew Carter, Casey Schilling, Alice Clifton, Chris Acon, e Stephen Brawner.

Un ultimo ma non per questo meno importante ringraziamento va alle nostre famiglie per l'amore e il supporto che ci hanno dato.

Da zero a uno

Capitolo

1

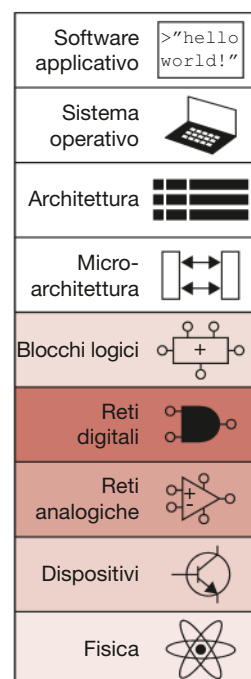
- 1.1 La pianificazione del gioco
- 1.2 L'arte di gestire la complessità
- 1.3 L'astrazione digitale
- 1.4 I sistemi numerici
- 1.5 Le porte logiche
- 1.6 Oltre l'astrazione digitale
- 1.7 I transistori CMOS*
- 1.8 Consumo di potenza*
- 1.9 Riassunto e anticipazione

1.1 ■ LA PIANIFICAZIONE DEL GIOCO

Durante gli ultimi trent'anni, i microprocessori hanno rivoluzionato il mondo in cui viviamo. Un calcolatore portatile al giorno d'oggi ha una capacità decisamente superiore rispetto a un grosso calcolatore (*mainframe*) delle dimensioni di una stanza di ieri. Un'automobile di lusso contiene circa 100 microprocessori. I progressi raggiunti nel campo dei microprocessori hanno portato alla nascita dei telefoni cellulari e di Internet, hanno migliorato incredibilmente le tecnologie mediche e hanno rivoluzionato il mondo militare. A livello mondiale, le vendite delle industrie di semiconduttori sono salite da 21 milioni di dollari nel 1985 a 306 milioni nel 2013, e i microprocessori rappresentano la maggior parte di queste vendite. Gli Autori sono convinti che i microprocessori non siano solo tecnicamente, economicamente e socialmente importanti, ma che rappresentino anche un'invenzione intrinsecamente affascinante. Una volta portata a termine la lettura di questo testo, il lettore sarà in grado di effettuare il progetto del proprio microprocessore e di costruirlo. Le competenze e le tecniche che il lettore apprenderà durante il percorso proposto lo prepareranno al progetto di molti altri sistemi digitali.

Si presuppone che il lettore abbia di base una familiarità con l'elettricità, alcune esperienze precedenti di programmazione e un interesse genuino nel comprendere cosa accade sotto la superficie di un calcolatore. Questo testo pone l'attenzione sul progetto dei sistemi digitali che operano con gli 1 e gli 0. Si inizia con le porte logiche digitali che accettano 1 e 0 in ingresso (*input*) e producono 1 e 0 in uscita (*output*). Successivamente si vede come combinare porte logiche in moduli più complessi come i sommatore e le memorie, e si studierà la programmazione in **linguaggio assembly** (assemblaggio, ma la traduzione non si usa), la lingua nativa dei microprocessori. Infine, si vedrà come unire le porte logiche per costruire un microprocessore che esegua questi programmi in linguaggio assembly.

Un grande vantaggio dei sistemi digitali è che gli elementi che li costituiscono sono piuttosto semplici: solo 1 e 0, e non richiedono una matematica



difficile o una profonda conoscenza della fisica. Invece, la sfida più grande di un progettista è combinare questi semplici elementi in sistemi complessi. Per il lettore, un microprocessore può essere la prima esperienza di costruzione di un sistema troppo complesso per essere compreso tutto in una volta. Ecco perché uno dei temi principali che verranno trattati nel testo è la gestione della complessità.

1.2 ■ L'ARTE DI GESTIRE LA COMPLESSITÀ

Una delle caratteristiche che distingue un ingegnere o un informatico da altri professionisti è l'approccio sistematico alla gestione della complessità. I moderni sistemi digitali sono costituiti da milioni o miliardi di transistori. Nessun uomo potrebbe comprendere questi sistemi semplicemente scrivendo le espressioni matematiche che descrivono il movimento degli elettroni in ogni transistore e risolvendole tutte allo stesso tempo. È quindi necessario imparare a gestire la complessità per comprendere come costruire un microprocessore senza essere oberati dai troppi dettagli.

1.2.1 L'astrazione

La tecnica fondamentale per imparare a controllare la complessità è l'**astrazione**, che consiste nel nascondere dettagli quando questi non sono importanti. Un sistema può essere visto da molti diversi livelli di astrazione. Per esempio, i politici americani astraggono il mondo in termini di città, province, regioni, e infine stati. Una provincia contiene più città e una regione contiene più province. Quando un politico si candida alla presidenza è più interessato alle dinamiche dello stato che andrà al voto nel suo complesso, piuttosto che ai voti delle singole province, quindi lo stato diventa il livello di astrazione per lui più utile.

La **Figura 1.1** illustra i diversi livelli di astrazione per un calcolatore elettronico insieme ai tipici elementi relativi a ogni livello. Al livello più basso di astrazione c'è la fisica, il moto degli elettroni, il cui comportamento è descritto dalla meccanica quantistica e dalle equazioni di Maxwell. Il sistema è costituito da **dispositivi elettronici** come i transistori (e prima di loro le valvole termoioniche). Questi dispositivi hanno punti di connessione ben definiti, chiamati **terminali**, e possono essere descritti in termini di rapporto tra tensione e corrente misurato a ogni terminale. Astraendosi al livello dei dispositivi è possibile ignorare i singoli elettroni. Il livello di astrazione successivo riguarda i circuiti analogici, nel quale i dispositivi vengono assemblati per creare componenti come gli amplificatori. I **circuiti analogici** hanno degli ingressi e delle uscite i cui livelli di tensione variano in modo continuo. I **circuiti digitali** come le porte logiche limitano i possibili valori di tensione a livelli discreti, che verranno utilizzati per indicare 0 e 1. Nel progetto logico si costruiscono strutture più complesse, come i sommatore e le memorie, a partire proprio dai circuiti digitali.

La **microarchitettura** unisce il livello di astrazione della logica e quello dell'architettura, che descrive un calcolatore dal punto di vista del programmatore. Per esempio, l'**architettura** della famiglia Intel x86, usata dai microprocessori presenti nella maggior parte dei *calcolatori portatili*, è definita tramite una serie di istruzioni e registri (contenitori temporanei di variabili) che il programmatore può utilizzare.

La microarchitettura richiede di combinare elementi logici per eseguire le istruzioni definite dall'architettura. Un'architettura particolare può essere realizzata da tante diverse microarchitetture con diversi rapporti prezzo/prestazioni. Per esempio, l'Intel core i7, l'Intel 80486 e l'AMD Athlon utilizzano tutti l'architettura x86 con diverse microarchitetture.

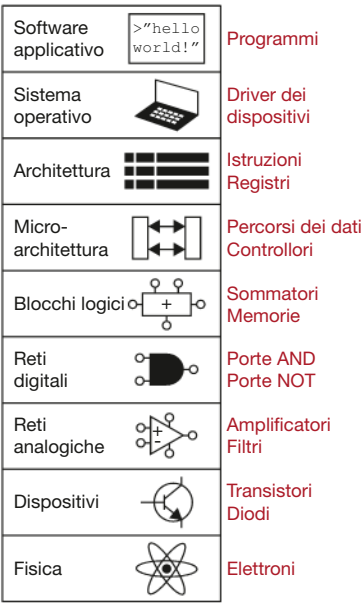


Figura 1.1
Livelli di astrazione in un calcolatore elettronico.

Spostandosi a livello del software, il sistema operativo si occupa dei dettagli a livello più basso, come le operazioni di lettura da un disco rigido o la gestione della memoria. Infine, il software applicativo utilizza le funzioni date dal sistema operativo per risolvere un problema dell'utente. Grazie al potere dell'astrazione, la nonna può navigare nel web senza dover conoscere le vibrazioni quantiche degli elettroni o l'organizzazione della memoria del calcolatore.

Questo testo si focalizza sui livelli di astrazione dai circuiti digitali fino all'architettura del calcolatore. Quando si lavora a un particolare livello di astrazione è utile sapere qualcosa dei livelli di astrazione immediatamente sopra e sotto quello su cui si sta lavorando. Per esempio, un informatico non può ottimizzare completamente un codice senza capire l'architettura per la quale il programma è stato scritto, così come un progettista di dispositivi elettronici non può ottimizzare il progetto di un transistor senza capire i circuiti nei quali il transistor verrà utilizzato. Una volta terminata la lettura di questo testo, il lettore dovrà essere in grado di riconoscere il livello di astrazione appropriato per risolvere un problema e valutare l'impatto delle proprie scelte di progetto sugli altri livelli di astrazione.

Ogni capitolo del libro inizia con l'icona dei livelli di astrazione che indica in rosso scuro l'argomento principale del capitolo, e in tonalità di rosso più chiare gli argomenti secondari.

1.2.2 La disciplina

La **disciplina** è l'atto di restringere intenzionalmente le scelte di progetto in modo tale da poter lavorare in maniera più produttiva a un livello più alto di astrazione. L'utilizzo di parti intercambiabili è un metodo ben noto di applicazione della disciplina. Uno dei primi esempi di parti intercambiabili è stata la produzione di fucili a pietra focaia. Fino ai primi anni del 1900, i fucili venivano assemblati a mano individualmente e le componenti, acquistate da differenti artigiani, venivano accuratamente lavorate e unite da un produttore di armi molto esperto. La disciplina delle parti intercambiabili ha rivoluzionato l'industria: limitando le componenti a una serie standardizzata con tolleranze ben definite, è stato possibile assemblare e produrre i fucili molto più velocemente e con minori requisiti di esperienza. In questo modo, il produttore di armi non doveva più preoccuparsi dei livelli più bassi di astrazione, come le forme specifiche di una particolare canna o del fusto del fucile.

In questo testo la disciplina digitale sarà molto importante. I circuiti digitali utilizzano valori di tensione discreti, mentre i circuiti analogici usano valori continui; questo fa dei circuiti digitali un sottoinsieme dei circuiti analogici e in un certo senso un gruppo con capacità minore rispetto alla classe dei circuiti analogici. Però, il progetto dei circuiti digitali è molto più semplice. Limitandosi ai circuiti digitali si possono facilmente combinare componenti in un sistema complesso che, in ultima istanza, ha prestazioni migliori rispetto a quelle dei circuiti costituiti da componenti analogici in molte applicazioni. Per fornire qualche esempio, le televisioni digitali, i compact disc (CD), i telefoni cellulari, che stanno rimpiazzando i loro predecessori analogici.

1.2.3 Le tre -Y

Oltre all'astrazione e alla disciplina, i progettisti utilizzano le **tre -Y** per gestire la complessità: la gerarchia, la modularità e la regolarità (dalla finale delle parole inglesi *hierarchy*, *modularity* e *regularity*). Questi principi si applicano sia ai sistemi software sia a quelli hardware.

- La **gerarchia** implica dividere un sistema in moduli e successivamente suddividere ulteriormente ognuno di questi moduli finché i pezzi che li compongono non siano facili da comprendere.

Il capitano Meriwether Lewis della Spedizione Lewis e Clark è stato uno dei primi sostenitori dell'intercambiabilità dei pezzi di un fucile. Nel 1806 spiegava:

Le armi di Drewyer e del Sergente Pryor erano entrambe fuori uso. La prima era stata riparata con un nuovo percussore perché l'originale si era guastato con l'uso; la seconda aveva la vite del cane rotta, ed era stata sostituita con una preparata per il percussore ad Harpers Ferry dove era stata costruita. Per via delle precauzioni prese per conservare questi percussori aggiuntivi, e queste parti di percussori, oltre all'ingenuità di John Shields, la maggior parte delle nostre armi sarebbero state completamente inutilizzabili; posso invece testimoniare che, fortunatamente, sono tutte perfettamente funzionanti.

Vedi Elliott Coues, ed., *The History of the Lewis and Clark Expedition...* (4 volumi), New York: Harper, 1893; ristampa, 3 volumi, New York: Dover, 3:817.

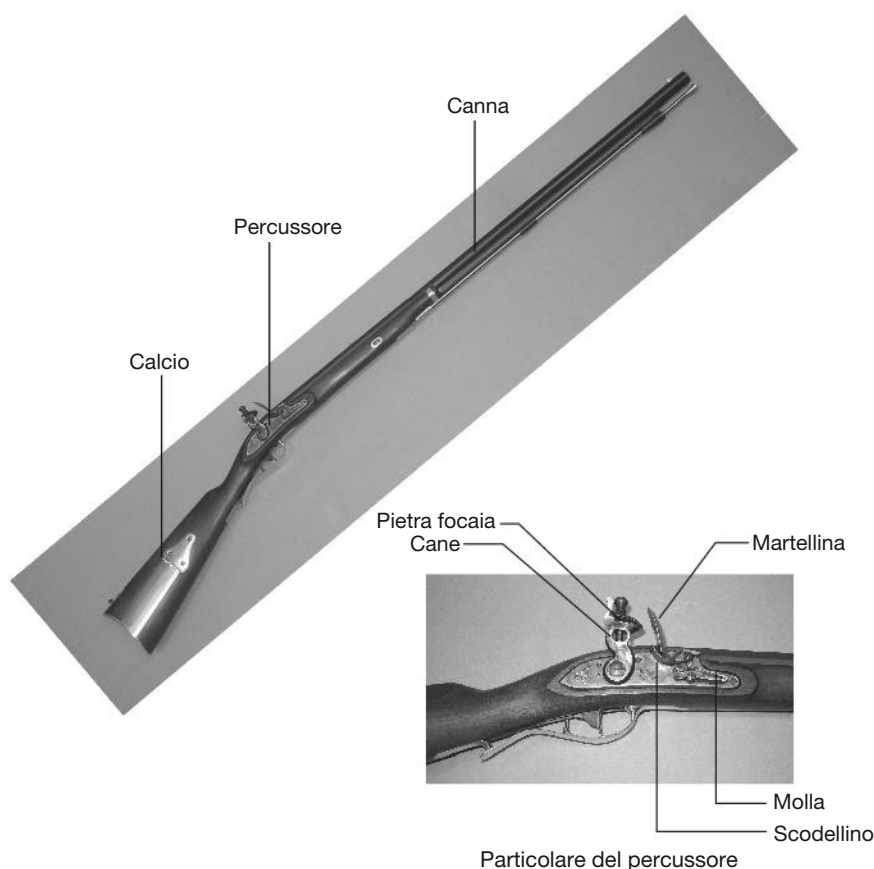
- La **modularità** implica che i moduli abbiano funzioni e interfacce ben definite, così da connettersi tra di loro in maniera semplice e senza effetti collaterali inaspettati.
- La **regolarità**, infine, cerca l'uniformità tra i moduli. I moduli più comuni vengono riutilizzati più volte, riducendo il numero di moduli diversi che devono essere progettati.

Per spiegare queste tre -Y si faccia nuovamente riferimento all'esempio della produzione di un fucile. Un fucile a pietra focaia era uno degli oggetti più complessi di utilizzo comune nei primi anni del '900. Applicando il principio della gerarchia, è possibile suddividerlo in più componenti, come mostra la **Figura 1.2**: la canna, il fusto e il percussore.

La canna è il lungo tubo di metallo attraverso il quale viene sparato il proiettile, il percussore è il meccanismo per lo sparo, mentre il fusto è la parte in legno che unisce le diverse componenti e assicura una presa stabile al cecchino. A sua volta poi, il percussore è composto dalla pietra focaia, dal cane, dalla martellina e dallo scodellino. Ognuna di queste componenti può essere descritta gerarchicamente sempre più nel dettaglio.

La modularità prevede che ogni componente abbia funzioni e interfaccia ben definite. Una funzione del fusto è quella di unire la canna e il percussore. La sua interfaccia consiste nella sua lunghezza e nella posizione delle viti di fissaggio. Nel progetto di un fucile modulare i fusti di diversi artigiani possono essere utilizzati con una stessa canna a patto che siano della lunghezza corretta e abbiano la stessa struttura di montaggio. Una funzione della canna è dare velocità al proiettile in modo che viaggi con maggiore precisione. La modularità impone che non ci siano effetti collaterali: il progetto del fusto non deve impedire il corretto funzionamento della canna.

Figura 1.2
Fucile a pietra focaia con particolare del percussore. (Immagine di Euroarms Italia; www.euroarms.net © 2006.)



La regolarità insegna che è meglio avere parti intercambiabili; grazie a questo principio, una canna danneggiata può essere sostituita da un'altra identica. Le canne possono così essere costruite in maniera più efficiente attraverso una catena di montaggio, invece che essere costruite a mano in maniera lenta e difficile. Si ritornerà su questi principi di gerarchia, modularità e regolarità durante l'intero testo.

1.3 ■ L'ASTRAZIONE DIGITALE

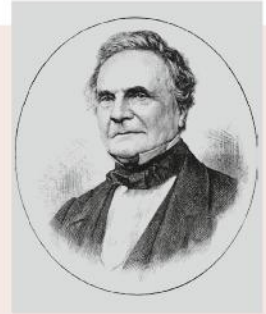
La maggior parte delle variabili fisiche è continua, come per esempio la tensione elettrica in un filo, la frequenza di un'oscillazione o la posizione di una massa. I sistemi digitali, al contrario, rappresentano informazioni con **variabili dal valore discreto**, cioè variabili con un numero finito di valori possibili.

Uno dei primi sistemi digitali che utilizzava le variabili con 10 diversi valori era il Motore Analitico di Charles Babbage. Babbage ci lavorò dal 1834 fino al 1871, progettando e tentando di costruire questo calcolatore meccanico. Il Motore Analitico utilizzava ingranaggi composti da 10 posizioni comprese tra 0 e 9, molto simili a quelli del contachilometri meccanico di un'automobile. La **Figura 1.3** mostra un prototipo del Motore Analitico in cui ogni riga elabora una cifra. Babbage scelse di utilizzare 25 righe di ingranaggi, così che la macchina avesse una precisione a 25 cifre. A differenza della macchina di Babbage, la maggior parte dei calcolatori elettronici utilizza una rappresentazione binaria, cioè a due valori, dal momento che è più facile distinguere tra due sole tensioni piuttosto che tra dieci, dove la tensione maggiore indica un 1 e la minore indica uno 0.

La **quantità di informazione** D associata a una variabile a valori discreti con N stati distinti è misurata in termini di **bit** come

$$D = \log_2 N \text{ bit} \quad (1.1)$$

Una variabile binaria trasporta $\log_2 2 = 1$ bit di informazione. Infatti, la parola bit è la contrazione di *binary digit* (cifra binaria). Ognuno degli ingranaggi di Babbage trasportava $\log_2 10 = 3.322$ bit di informazione, poiché poteva trovarsi in una delle $2^{3.322} = 10$ diverse posizioni previste. In linea teorica, un segnale continuo contiene una quantità infinita di informazioni, dal momento che può assumere un numero infinito di valori. Tuttavia, in pratica, il rumore e gli errori di misurazione limitano le informazioni, nella maggior parte dei segnali continui, da 10 fino a 16 bit. Se è necessario eseguire rapidamente la misurazione, il contenuto di informazioni può dover essere ulteriormente ridotto (per es. a 8 bit).



Charles Babbage, 1791-1871. Ha frequentato l'Università di Cambridge e si è sposato con Georgiana Whitmore nel 1814. Ha inventato il Motore Analitico, il primo calcolatore meccanico. Ha inventato anche la parte frontale delle locomotive a vapore (chiamata "scacciamucche") e le tariffe postali universali. Si divertiva a forzare serrature e odiava i musicisti da strada. (Immagine gentilmente concessa da Fourmilab Switzerland, www.fourmilab.ch.)

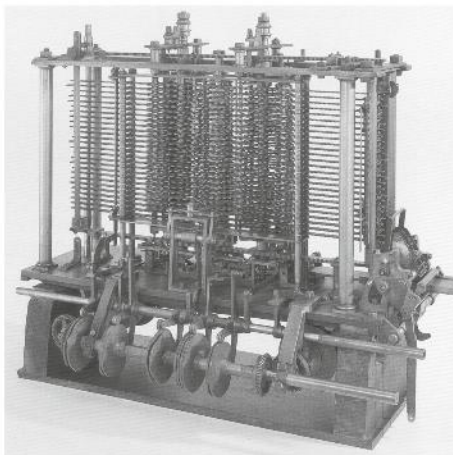


Figura 1.3
Il Motore Analitico di Babbage, in costruzione al momento della sua morte, nel 1871. (Immagine gentilmente concessa dal Science Museum/Science and Society Picture Library.)



George Boole, 1815-1864. Figlio di lavoratori senza possibilità di avere un'educazione formale, Boole ha studiato matematica da solo ed è diventato membro della facoltà del Queen's College in Irlanda. Ha scritto *An Investigation of the Laws of Thought* (1854), che introduce le variabili logiche e le tre operazioni logiche fondamentali: AND, OR e NOT. (Immagine gentilmente concessa dall'American Institute of Physics.)

Questo testo si focalizza sui circuiti digitali che utilizzano variabili binarie: gli 1 e gli 0. George Boole ha sviluppato una logica che opera su variabili binarie, nota come **logica Booleana**. Ognuna delle variabili di Boole può assumere solo uno dei due valori, VERO o FALSO. Solitamente, i calcolatori elettronici utilizzano una tensione elettrica positiva (alta) per rappresentare "1" e una tensione di zero volt (bassa) per rappresentare "0". In questo testo, i termini "1", VERO e ALTO verranno utilizzati come sinonimi, così come i termini "0", FALSO e BASSO.

La bellezza dell'astrazione è il fatto che i progettisti digitali possono focalizzarsi sugli 1 e sugli 0 ignorando se le variabili booleane siano rappresentate fisicamente con livelli di tensione, ingranaggi rotanti, o addirittura livelli di fluidi idraulici, così come un programmatore può lavorare senza aver bisogno di conoscere i dettagli più specifici dell'hardware del calcolatore su cui sta operando. D'altro canto, comprendere i dettagli dell'hardware permette al programmatore di ottimizzare il software per quel particolare calcolatore.

Un unico bit non trasporta molta informazione. Nella prossima sezione si esamina come sia possibile utilizzare gruppi (spesso denominati **stringhe**) di bit per rappresentare i numeri e, nei capitoli successivi, i gruppi di bit vengono utilizzati per rappresentare anche lettere e programmi.

1.4 ■ I SISTEMI NUMERICI

Si è abituati a lavorare con i numeri decimali; tuttavia, quando si ha a che fare con sistemi digitali che operano con 1 e 0, è spesso più utile utilizzare i numeri binari ed esadecimali. In questo paragrafo sono introdotti i diversi sistemi numerici che verranno utilizzati nel resto del testo.

1.4.1 Numeri decimali

Alle elementari, i bambini imparano a contare e a eseguire operazioni aritmetiche con i numeri **decimali**. Proprio come le dita delle mani, ci sono dieci cifre decimali: 0, 1, 2, ..., 9. Le cifre decimali vengono unite per formare numeri decimali più lunghi; ogni posizione di una cifra in un numero decimale ha 10 volte il peso della posizione alla sua destra. Da destra verso sinistra i pesi delle posizioni delle varie cifre sono 1, 10, 100, 1000 e così via. Per questo motivo ci si riferisce ai numeri decimali come ai numeri **in base 10**. Per prevenire una possibile confusione quando si lavora con più di una base, questa viene indicata al pedice dopo il numero. Per fare un esempio, la **Figura 1.4** mostra come il numero decimale 9742_{10} venga scritto come la somma di ognuna delle cifre che lo compongono, moltiplicata per il peso della posizione (cioè della colonna) corrispondente.

Un numero decimale a N cifre rappresenta uno dei 10^N valori: 0, 1, 2, 3, ..., $10^N - 1$. Questo insieme di valori viene definito **intervallo del numero**: per esempio, un numero decimale a tre cifre rappresenta uno dei 1000 valori nell'intervallo da 0 a 999.

Figura 1.4
Rappresentazione di un numero decimale.

$$\begin{array}{ccccccc}
 & & & & & & \text{colonna del 1} \\
 & & & & & & \text{colonna del 10} \\
 & & & & & & \text{colonna del 100} \\
 & & & & & & \text{colonna del 1000} \\
 9742_{10} & = & 9 \times 10^3 & + & 7 \times 10^2 & + & 4 \times 10^1 & + & 2 \times 10^0 \\
 & & \text{nove} & & \text{sette} & & \text{quattro} & & \text{due} \\
 & & 1000 & & 100 & & 10 & & 1
 \end{array}$$

1.4.2 Numeri binari

I bit rappresentano uno dei due valori 0 o 1, e vengono uniti l'uno con l'altro per formare numeri binari. Ogni posizione di un bit in un numero binario ha il doppio del peso della posizione precedente, quindi i numeri binari sono **numeri in base 2**. Nei numeri binari, i pesi delle posizioni (ovviamente ancora da destra a sinistra) sono 1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096, 8192, 16384, 32768, 65536 e così via. Se si lavora spesso con i numeri binari si risparmia tempo se si ricordano queste potenze fino a 2^{16} .

Come coi numeri decimali, un numero binario a N bit rappresenta uno di 2^N valori: 0, 1, 2, 3, ..., $2^N - 1$. La **Tabella 1.1** mostra i numeri binari a 1, 2, 3 e 4 bit e i loro equivalenti decimali.

Tabella 1.1 Numeri binari e corrispondenti numeri decimali.

Numeri binari a 1 bit	Numeri binari a 2 bit	Numeri binari a 3 bit	Numeri binari a 4 bit	Corrispondenti numeri decimali
0	00	000	0000	0
1	01	001	0001	1
	10	010	0010	2
	11	011	0011	3
		100	0100	4
		101	0101	5
		110	0110	6
		111	0111	7
			1000	8
			1001	9
			1010	10
			1011	11
			1100	12
			1101	13
			1110	14
			1111	15

ESEMPIO 1.1

Conversione da binario a decimale. Convertire il numero binario 10110_2 in decimale.

Soluzione La **Figura 1.5** mostra come effettuare la conversione.

colonna dell'1
colonna del 2
colonna del 4
colonna dell'8
colonna del 16

$$10110_2 = 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 22_{10}$$

un 16 nessun 8 un 4 un 2 nessun 1

Figura 1.5
Conversione di un numero binario in decimale.

ESEMPIO 1.2

Conversione da decimale a binario. Convertire il numero decimale 84_{10} in binario.

Soluzione Determinare per ogni posizione se il risultato binario ha un 1 o uno 0. Questa operazione può essere effettuata a partire da entrambe le estremità.

Partendo da sinistra, si inizi dalla potenza di due più alta che sia minore o uguale al numero decimale (in questo caso, 64). $84 \geq 64$, quindi la posizione del 64 deve avere valore 1, lasciando $84 - 64 = 20$. $20 \leq 32$, quindi la posizione del 32 deve avere valore 0. $20 \geq 16$, quindi la posizione del 16 deve avere valore 1, lasciando $20 - 16 = 4$. $4 < 8$, quindi la posizione dell'8 deve avere valore 0. $4 \geq 4$, quindi la posizione del 4 deve avere valore 1, lasciando $4 - 4 = 0$. Questo significa che sia la posizione del 2 sia quella dell'1 devono avere valore 0. Unendo tutte le cifre si ottiene $84_{10} = 1010100_2$.

Partendo invece da destra verso sinistra, si divide ripetutamente il numero espresso in base 10 per 2. Il resto di ogni divisione viene riportato in posizioni successive del numero in base 2 a partire dalla meno significativa. $84/2 = 42$, quindi nella posizione dell'1 si riporta il valore 0. $42/2 = 21$, ancora una volta il valore 0 nella posizione del 2. $21/2 = 10$ con resto di 1, da scrivere nella posizione del 4. $10/2 = 5$, quindi il valore 0 nella posizione dell'8. $5/2 = 2$ con resto di 1, da scrivere nella posizione del 16. $2/2 = 1$, quindi il valore 0 nella posizione del 32. Infine, $1/2 = 0$ con resto di 1, da scrivere nella posizione del 64. Come prima, $84_{10} = 1010100_2$.

1.4.3 Numeri esadecimali

Scrivere lunghe file di cifre binarie diventa noioso e aumenta le possibilità di commettere errori. Un gruppo di 4 bit rappresenta una di $2^4 = 16$ possibilità. Quindi, a volte è più utile lavorare **in base 16**; i numeri in base 16 sono chiamati **numeri esadecimali**, e utilizzano per rappresentare le 16 cifre necessarie le cifre decimali da 0 a 9 e le prime sei lettere dell'alfabeto dalla A alla F, come mostra la **Tabella 1.2**. Le posizioni delle cifre in base 16 pesano rispettivamente 1, 16, 16^2 (o 256), 16^3 (o 4096) e così via.

"Esadecimale", termine coniato da IBM nel 1963, deriva dal greco *hexi* (sei) e dal latino *decem* (dieci). Sarebbe stato più appropriato usare il latino *sexa* (sei) ma il termine risultante, sexadecimale, è sembrato troppo osé...

ESEMPIO 1.3

Conversione da esadecimale a binario e decimale. Convertire il numero esadecimale $2ED_{16}$ in binario e decimale.

Soluzione La conversione tra numeri esadecimali e numeri binari è semplice, dal momento che ogni cifra esadecimale corrisponde a quattro cifre binarie. $2_{16} = 0010_2$, $E_{16} = 1110_2$ e $D_{16} = 1101_2$, quindi $2ED_{16} = 001011101101_2$. La conversione a numero decimale, invece, richiede i calcoli mostrati nella **Figura 1.6**.

Tabella 1.2 La numerazione esadecimale.

Numerazione esadecimale	Equivalente decimale	Equivalente binario
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

$$\begin{array}{c}
 \text{colonna dell'1} \\
 \text{colonna del 16} \\
 \text{colonna del 256} \\
 2ED_{16} = 2 \times 16^2 + E \times 16^1 + D \times 16^0 = 749_{10} \\
 \begin{array}{ccc}
 \text{due} & \text{quattordici} & \text{tredici} \\
 256 & 16 & 1
 \end{array}
 \end{array}$$

Figura 1.6
Conversione di un numero
esadecimale in decimale.

ESEMPIO 1.4

Conversione da binario a esadecimale. Convertire il numero binario 1111010_2 in esadecimale.

Soluzione Ancora una volta, la conversione è semplice. Leggendo a partire da destra, i quattro bit meno significativi sono $1010_2 = A_{16}$. I bit successivi sono $111_2 = 7_{16}$. Quindi $1111010_2 = 7A_{16}$.

ESEMPIO 1.5

Conversione da decimale a esadecimale e binario. Convertire il numero decimale 333_{10} in esadecimale e binario.

Soluzione Come nel caso della conversione da decimale a binario, la conversione da decimale a esadecimale può essere effettuata a partire sia da destra sia da sinistra.

Partendo da sinistra, trovare la più grande potenza di 16 che sia minore o uguale al numero (in questo caso, 256). 256 sta una volta nel 333, quindi la posizione del 256 deve avere valore 1, lasciando $333 - 256 = 77$. Il 16 sta nel 77 quattro volte, quindi la posizione del 16 deve avere valore 4, lasciando $77 - 16 \times 4 = 13$. $13_{10} = D_{16}$, quindi la posizione dell'1 deve avere valore D. In conclusione, $333_{10} = 14D_{16}$. A questo punto è facile eseguire la conversione da esadecimale a binario, come illustrato nell'Esempio 1.3. $14D_{16} = 101001101_2$.

Partendo invece da destra, il numero decimale viene diviso ripetutamente per 16. Il resto viene riportato in posizioni successive del numero in base 16. $333/16 = 20$ con un resto di $13_{10} = D_{16}$ che viene riportato nella posizione dell'1. $20/16 = 1$ con resto di 4 nella posizione del 16. $1/16 = 0$ con resto di 1 da riportare nella posizione del 256. Ancora una volta, il risultato è $14D_{16}$.

1.4.4 Byte, nibble, word

Un insieme di 8 bit è chiamato un **byte**. Esso rappresenta una di $2^8 = 256$ possibilità e convenzionalmente è l'unità di misura utilizzata, al posto del bit, per misurare la grandezza degli oggetti immagazzinati nelle memorie dei calcolatori.

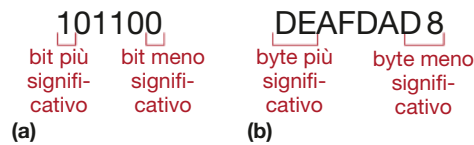
Un gruppo di 4 bit, o la metà di un byte, è chiamato invece **nibble**. Un nibble rappresenta una di $2^4 = 16$ possibilità. Una cifra esadecimale rappresenta un nibble e due cifre esadecimali rappresentano un byte intero. I nibble non sono più un'unità di misura in utilizzo, ma ciò non toglie che valga la pena conoscere il termine.

I microprocessori gestiscono dati in gruppi di bit chiamati **word** (spesso tradotti con "parole") la cui grandezza dipende dall'architettura del microprocessore. Quando questo capitolo è stato scritto, nel 2015, la maggior parte dei calcolatori utilizzava processori a 64 bit, il che significa che questi operavano con parole di 64 bit. Calcolatori più vecchi gestivano parole di 32. I microprocessori più semplici invece, specialmente quelli usati negli elettrodomestici, come per esempio i tostapane, utilizzano parole di 8 o 16 bit.

All'interno di un gruppo di bit, il bit che si trova nella colonna di peso 1 viene chiamato **bit meno significativo** (*lsb, least significant bit*) e il bit che si trova all'estremità opposta viene chiamato **bit più significativo** (*msb, most significant bit*), come mostra la **Figura 1.7(a)** per un numero binario di 6 bit.

Un **microprocessore** è un processore su singolo chip. Fino agli anni '70, i processori erano troppo complessi per essere realizzati su un unico chip: i processori dei grossi calcolatori (*mainframe*) erano costituiti da schede con molti chip. Intel ha introdotto il primo microprocessore a 4 bit, denominato 4004, nel 1971. Oggi anche i supercalcolatori più sofisticati sono costruiti utilizzando microprocessori. Nel resto del libro useremo quindi indifferentemente i termini microprocessore e processore.

Figura 1.7
Bit e byte meno e più significativi.



Allo stesso modo, all'interno di una parola, i byte sono identificati dal **byte meno significativo** (LSB, *Least Significant Byte*) fino al **byte più significativo** (MSB, *Most Significant Byte*), come mostra la **Figura 1.7(b)** per un numero a 4 byte scritto con 8 cifre esadecimali.

Per fortunata coincidenza, $2^{10} = 1024 \approx 10^3$. Il termine “kilo” (“migliaio” in greco) indica 2^{10} . Quindi, si può dire per approssimazione che 2^{10} byte equivalgono circa a un kilobyte (1 KB con la “K” maiuscola usata in informatica). Allo stesso modo, “mega” (“milione” in greco) indica $2^{20} \approx 10^6$ e “giga” (“miliardo”) indica $2^{30} \approx 10^9$. Se ci si ricorda che $2^{10} \approx 1$ migliaia, $2^{20} \approx 1$ milione e $2^{30} \approx 1$ miliardo, e quali sono i valori delle potenze di 2 fino a 2^9 , sarà più facile calcolare qualsiasi potenza di 2 a mente.

ESEMPIO 1.6

Calcolare le potenze di due. Trovare il valore approssimato di 2^{24} senza usare la calcolatrice.

Soluzione Per calcolare il valore, bisogna dividere l'esponente in un multiplo di 10 e nel resto.

$2^{24} = 2^{20} \times 2^4$. $2^{20} \approx 1$ milione, e $2^4 = 16$. Quindi, $2^{24} \approx 16$ milioni. Tecnicamente, $2^{24} = 16\,777\,216$ ma si può ragionevolmente approssimare con 16 milioni che è un risultato che si avvicina più che bene al valore reale.

1024 byte formano un **kilobyte** (KB) così come 1024 bit formano un **kilobit** (Kb o Kbit). Allo stesso modo, MB, Mb, GB e Gb rappresentano milioni e miliardi di byte e bit. La capacità di memoria è normalmente misurata in byte, mentre la velocità di comunicazione è solitamente misurata in bit al secondo (bit/sec). Per esempio, la velocità massima di un modem è solitamente di 56 kbit/sec.

1.4.5 Somma binaria

La somma binaria è un'operazione simile alla somma decimale, anche se più semplice, come mostra la **Figura 1.8**. Come per la somma decimale, se la somma di due cifre è maggiore di quanto si possa esprimere con una singola cifra, allora si ha riporto di un 1 nella colonna (posizione) successiva. La **Figura 1.8** confronta la somma tra numeri decimali con quella tra numeri binari. Nella colonna più a destra della **Figura 1.8(a)**, $7 + 9 = 16$, numero che non può essere espresso in una sola cifra decimale perché maggiore di 9. Quindi si scrive 6 nella colonna delle unità, e si riporta 1 nella colonna successiva, quella delle decine. Lo stesso meccanismo vale per le somme binarie: se la somma dei due numeri è maggiore di uno, si porta la cifra di peso 2 nella colonna successiva.

Si osservi l'esempio presentato nella **Figura 1.8(b)**, la somma $1 + 1 = 2_{10} = 10_2$ non può essere espressa con una singola cifra binaria. Quindi si scrive lo 0 nella prima colonna e si riporta la cifra del 2 (1) nella colonna successiva. Nella seconda colonna della stessa figura, l'esempio è la somma $1 + 1 + 1 = 3_{10} = 11_2$. Di nuovo, si indica un 1 nella colonna e si riporta la seconda cifra

Figura 1.8
Esempi di somme con riporti:
(a) decimale, (b) binaria.

Figure 1.8 shows two examples of addition with carries. Part (a) shows a decimal addition: 4277 plus 5499 equals 9776. A red arrow labeled 'riporti' points from the units column to the tens column. Part (b) shows a binary addition: 1011 plus 0011 equals 1110. A red arrow labeled 'riporti' points from the first column to the second column.

(1) nella colonna successiva. Per ovvie ragioni, il bit che viene riportato nella colonna successiva viene chiamato **bit di riporto** (*carry*).

ESEMPIO 1.7

Somma binaria. Calcolare $0111_2 + 0101_2$.

Soluzione La **Figura 1.9** mostra che il risultato di questa operazione è 1100_2 . I riporti sono indicati in rosso. È possibile fare la prova dell'operazione eseguendo la trasformazione in numeri decimali per poi eseguirne la somma: $0111_2 = 7_{10}$, $0101_2 = 5_{10}$, la somma è $12_{10} = 1100_2$.

I sistemi digitali operano praticamente sempre su un numero fisso di cifre. Se il risultato di una somma è troppo grande per essere espresso con le cifre a disposizione, si dice che la somma dà luogo a un **traboccamento** (*overflow*). Un numero a 4 bit, per esempio, copre l'intervallo $[0, 15]$. Una somma binaria a 4 bit dà quindi traboccamento se il risultato è maggiore di 15. In questo caso, il quinto bit viene ignorato, producendo un risultato scorretto nei rimanenti 4 bit. Il traboccamento può essere individuato se si controlla la presenza o meno di un riporto nella colonna più significativa.

ESEMPIO 1.8

Somma con traboccamento. Eseguire la somma $1101_2 + 0101_2$ e verificare se si presenta o meno un traboccamento.

Soluzione La **Figura 1.10** mostra che la somma è 10010_2 . Questo risultato supera l'intervallo di un numero binario a 4 bit. Se deve essere memorizzato in quattro bit, il bit più significativo viene scartato, lasciando il risultato scorretto 0010_2 . Naturalmente la cosa non succede se si usano numeri a 5 o più bit.

1.4.6 Numeri binari relativi

Fino ad ora sono stati considerati unicamente numeri binari non relativi (senza segno) che rappresentano solo quantità positive. Spesso si devono rappresentare sia numeri positivi sia numeri negativi, e per fare questo c'è bisogno di un sistema di numeri binari differente. Esistono diversi sistemi che possono essere usati per rappresentare i numeri binari negativi; i due più utilizzati sono chiamati **modulo e segno** e **segno e complemento a due**.

Numeri in modulo e segno

I numeri in modulo e segno sono facili da capire perché coincidono col modo in cui vengono tradizionalmente scritti i numeri (anche decimali) negativi, con un segno meno seguito dal modulo (cioè dal valore assoluto) del numero. Un numero in modulo e segno a N bit utilizza il bit più significativo per esprimere il segno e i rimanenti $N - 1$ bit come indicatori del modulo. Il bit di segno per un numero positivo vale 0 e per un numero negativo vale 1.

ESEMPIO 1.9

Numeri in modulo e segno. Scrivere 5 e -5 sotto forma di numeri in modulo e segno a 4 bit.

Soluzione Entrambi i numeri hanno un modulo pari a $5_{10} = 101_2$. Quindi $5_{10} = 0101_2$ e $-5_{10} = 1101_2$.

Sfortunatamente, il metodo per calcolare la somma binaria visto prima non funziona per i numeri espressi in modulo e segno: utilizzando tale metodo, infatti, $-5_{10} + 5_{10}$ sarebbe $1101_2 + 0101_2 = 10010_2$ invece di zero, quindi un risultato insensato.

$$\begin{array}{r} \text{111} \\ 0111 \\ + 0101 \\ \hline 1100 \end{array}$$

Figura 1.9
Esempio di somma binaria.

$$\begin{array}{r} \text{11 1} \\ 1101 \\ + 0101 \\ \hline 10010 \end{array}$$

Figura 1.10
Esempio di somma binaria con traboccamento.

Il razzo Ariane 5, costato 7 miliardi di dollari e lanciato il 4 giugno 1996, è uscito dalla traiettoria prevista dopo 40 secondi dal lancio, si è rotto ed è esploso. L'incidente è stato causato quando il calcolatore che controllava il razzo ha generato un traboccamento nei suoi dati a 16 bit e si è bloccato. Il codice era stato ampiamente collaudato sul razzo Ariane 4. Ma Ariane 5 aveva un motore più veloce che forniva valori numerici più grandi al calcolatore di controllo, generando così il traboccamento.



(Fotografia gentilmente concessa da ESA/CNES/ARIANESPACE-Service Optique CS6.)

Un numero in modulo e segno a N bit ha un intervallo di variabilità pari a $[-2^{N-1} + 1, 2^{N-1} - 1]$. I numeri in modulo e segno sono un po' strani dal momento che esiste una codifica per $+0$ e una diversa per -0 , ma entrambe indicano lo stesso numero 0: è facile intuire che diventa complicato avere due rappresentazioni differenti per lo stesso numero.

Numeri in complemento a due

I numeri in complemento a due sono identici ai numeri binari non relativi, eccezion fatta per la posizione del bit più significativo, il cui peso è di -2^{N-1} invece che 2^{N-1} . Questi numeri superano i limiti dei numeri in modulo e segno: lo zero ha un'unica rappresentazione ed è possibile effettuare la somma con il metodo usuale.

Nella rappresentazione in complemento a due, lo zero è scritto come al solito, ovvero una sequenza di bit tutti a zero: $00\dots00_2$. Il massimo numero positivo ha il bit più significativo a 0 e tutti gli altri a 1: $01\dots11_2 = 2^{N-1} - 1$. Il numero più negativo (cioè il minimo numero rappresentabile) ha il bit più significativo a 1 e tutti gli altri a 0: $10\dots00_2 = -2^{N-1}$. E il numero -1 è scritto con una sequenza di bit tutti a uno: $11\dots11_2$.

Si noti che nella rappresentazione in complemento a due i numeri positivi hanno 0 come bit più significativo, mentre i numeri negativi hanno 1, quindi il bit più significativo può essere considerato il bit di segno. Tuttavia, il numero nel suo complesso viene interpretato in maniera diversa nel caso di codifica in modulo e segno e di codifica in complemento a due.

Il segno di un numero in complemento a due può essere invertito grazie a un'operazione detta **calcolo del complemento a due**: consiste nell'invertire tutti i bit all'interno del numero, poi aggiungere 1. Questa operazione è molto utile per ricavare la rappresentazione di un numero negativo o per calcolarne il valore assoluto.

ESEMPIO 1.10

Rappresentazione in complemento a due di un numero negativo. Trovare la rappresentazione di -2_{10} come numero in complemento a due a 4 bit.

Soluzione Partendo dal numero positivo $+2_{10} = 0010_2$, per ottenere il corrispondente negativo si invertono tutti i bit e si aggiunge 1. Invertendo 0010_2 si ottiene 1101_2 e $1101_2 + 1 = 1110_2$. Quindi, la rappresentazione di -2_{10} è 1110_2 .

ESEMPIO 1.11

Valore dei numeri negativi in complemento a due. Trovare il valore decimale del numero in complemento a due 1001_2 .

Soluzione 1001_2 ha il bit più significativo a 1, il che significa che si tratta di un numero negativo. Per trovarne il valore assoluto, è necessario invertire i bit e aggiungere 1. Invertendo si ottiene $1001_2 = 0110_2$ e $0110_2 + 1 = 0111_2 = 7_{10}$. Quindi, $1001_2 = -7_{10}$.

I numeri in complemento a due hanno il vantaggio che la somma è eseguibile in maniera corretta utilizzando il metodo usuale sia per i numeri positivi sia per quelli negativi. Si ricordi soltanto che quando si sommano numeri a N bit il riporto dell' N -esimo bit (cioè il bit di posizione $N + 1$ all'interno del risultato) deve essere scartato.

ESEMPIO 1.12

Sommare numeri in complemento a due. Calcolare (a) $-2_{10} + 1_{10}$ e (b) $-7_{10} + 7_{10}$ utilizzando i numeri in complemento a due.

Soluzione (a) $-2_{10} + 1_{10} = 1110_2 + 0001_2 = 1111_2 = -1_{10}$. (b) $-7_{10} + 7_{10} = 1001_2 + 0111_2 = 10000_2$. Il quinto bit viene scartato, quindi il risultato corretto è 0000_2 .

La sottrazione viene effettuata effettuando il complemento a 2 del secondo numero e poi procedendo con la somma.

ESEMPIO 1.13

Sottrazione tra numeri in complemento a due. Calcolare (a) $5_{10} - 3_{10}$ e (b) $3_{10} - 5_{10}$ utilizzando numeri in complemento a due a 4 bit.

Soluzione (a) $3_{10} = 0011_2$. Calcolando il complemento a due si ottiene $-3_{10} = 1101_2$. Quindi si effettua la somma $5_{10} + (-3_{10}) = 0101_2 + 1101_2 = 0010_2 = 2_{10}$. Si noti che il riporto della posizione più significativa viene scartato perché il risultato deve essere espresso in 4 bit. (b) Calcolare il complemento a due di 5_{10} per ottenere $-5_{10} = 1011$. A questo punto sommare $3_{10} + (-5_{10}) = 0011_2 + 1011_2 = 1110_2 = -2_{10}$.

Il complemento a due di 0 si calcola invertendo tutti i bit (operazione che produce $11...111_2$) e aggiungendo successivamente un 1, che produce tutti 0, e scartando il bit di riporto nella posizione più significativa, quindi lo zero è sempre rappresentato da tutti zeri. A differenza della rappresentazione in modulo e segno, la rappresentazione in complemento a due non ha un -0 distinto: lo zero è considerato positivo perché il bit di segno è 0.

Proprio come i numeri non relativi, i numeri in complemento a due a N bit rappresentano uno di 2^N valori possibili, però tali valori sono distinti tra positivi e negativi. Per esempio, un numero non relativo a 4 bit rappresenta 16 valori compresi tra 0 e 15. Un numero in complemento a due a 4 bit rappresenta anch'esso 16 valori, ma questa volta da -8 a 7. Generalizzando, l'intervallo di un numero in complemento a due a N bit è dato da $[-2^{N-1}, 2^{N-1} - 1]$. Non stupisce che ci sia un numero negativo in più all'interno di questo intervallo piuttosto che un numero positivo, dal momento che non è presente il valore -0 . Il numero più negativo $10...000_2 = -2^{N-1}$ è anomalo, perché il suo complemento a due, che si trova invertendo i bit (e quindi producendo $01...111_2$) e aggiungendo 1, è ancora il numero stesso: $10...000_2$. In altre parole, questo numero negativo non ha una controparte positiva.

Sommare numeri a N bit entrambi positivi o entrambi negativi può causare un traboccamento se il risultato è maggiore di $2^{N-1} - 1$ o minore di -2^{N-1} . Invece, la somma di un numero positivo con un numero negativo non causa mai traboccamento.

Diversamente dai numeri senza segno, un riporto della colonna più significativa non indica un traboccamento, che si verifica invece se i due numeri che vengono sommati hanno lo stesso bit di segno e il risultato ha un bit di segno opposto.

ESEMPIO 1.14

Somma di numeri in complemento a due con traboccamento. Calcolare $4_{10} + 5_{10}$ usando numeri in complemento a due a 4 bit. L'operazione genera traboccamento?

Soluzione $4_{10} + 5_{10} = 0100_2 + 0101_2 = -7_{10}$. Il risultato supera l'intervallo dei numeri in complemento a due positivi a 4 bit, infatti produce un risultato negativo ovviamente sbagliato. Se il calcolo fosse stato eseguito con 5 bit o più, il risultato corretto sarebbe stato $01001_2 = 9_{10}$.

Quando un numero in complemento a due viene esteso a un numero maggiore di bit, il bit che dà il segno deve essere copiato in tutte le posizioni più significative. Questo processo viene chiamato **estensione del segno**. Per esempio, i numeri 3 e -3 si scrivono numeri in complemento a due a 4 bit rispettivamente 0011 e 1011 . Questi ultimi vengono estesi a 7 bit copiando il bit che dà il segno nella posizione dei tre nuovi bit per formare rispettivamente 0000011 e 1111101 .

Tabella 1.3 Intervallo di rappresentazione dei numeri a N bit.

Sistema	Range
Senza segno	$[0, 2^N - 1]$
Modulo e segno	$[-2^{N-1} + 1, 2^{N-1} - 1]$
Complemento a due	$[-2^{N-1}, 2^{N-1} - 1]$

Rappresentazioni numeriche a confronto

Le tre rappresentazioni di numeri binari più comunemente usate sono, come si è visto, i numeri senza segno, i numeri in complemento a due e i numeri in modulo e segno. La **Tabella 1.3** paragona l'intervallo di variabilità di numeri a N bit in ognuna di queste tre rappresentazioni. La rappresentazione in complemento a due è utile perché rappresenta sia i numeri positivi sia i numeri negativi e perché la somma si effettua sempre nel modo usuale indipendentemente dal segno degli addendi. La sottrazione si effettua cambiando segno al secondo numero (cioè calcolandone il complemento a due) e poi sommando. A meno che non venga specificato diversamente, si assume che i numeri binari con segno utilizzino sempre la rappresentazione in complemento a due.

La **Figura 1.11** mostra una retta dei numeri che riporta i valori dei numeri a 4 bit in ognuna delle rappresentazioni viste sinora. I numeri senza segno coprono l'intervallo $[0, 15]$ con valori binari crescenti. I numeri in complemento a due coprono l'intervallo $[-8, 7]$ con i numeri positivi $[0, 7]$ codificati come i corrispondenti senza segno e i numeri negativi $[-8, -1]$ caratterizzati dal fatto che un valore binario senza segno più grande rappresenta un numero più vicino a 0. Si ricordi che il numero anomalo 1000 rappresenta -8 e non possiede una controparte positiva. I numeri in modulo e segno coprono l'intervallo $[-7, 7]$. Il bit più significativo è il bit di segno, i numeri positivi nell'intervallo $[1, 7]$ condividono gli stessi codici dei numeri senza segno, i numeri negativi sono simmetrici ma hanno il bit di segno a 1, e il valore 0 viene rappresentato sia da 0000 sia da 1000. Perciò, i numeri in modulo e segno a N bit rappresentano solo $2^N - 1$ valori interi a causa delle due rappresentazioni dello 0.

1.5 ■ LE PORTE LOGICHE

Ora che si è visto come utilizzare le variabili binarie per rappresentare informazioni, è possibile analizzare i sistemi digitali che effettuano operazioni su queste variabili binarie. Le **porte logiche** (*logic gates*) sono semplici circuiti digitali che utilizzano uno o più ingressi binari per produrre un'uscita binaria. Le porte logiche vengono disegnate con un simbolo che mostra l'ingresso (o gli ingressi) e l'uscita. Solitamente, gli ingressi vengono disegnati a sinistra o in alto e l'uscita a destra o in basso. Tipicamente, i progettisti digitali utilizzano le lettere iniziali dell'alfabeto per gli ingressi della porta e la lettera Y per l'uscita. La relazione tra ingressi e uscita può essere descritta con una **tabella delle verità** o con una **espressione booleana**. Una tabella delle verità riporta l'elenco degli ingressi a sinistra e l'uscita a destra, e presenta una riga per ogni possibile combinazione dei valori di ingresso. Un'espressione booleana, invece, è un'espressione matematica che utilizza variabili binarie e operatori dell'algebra di Boole.

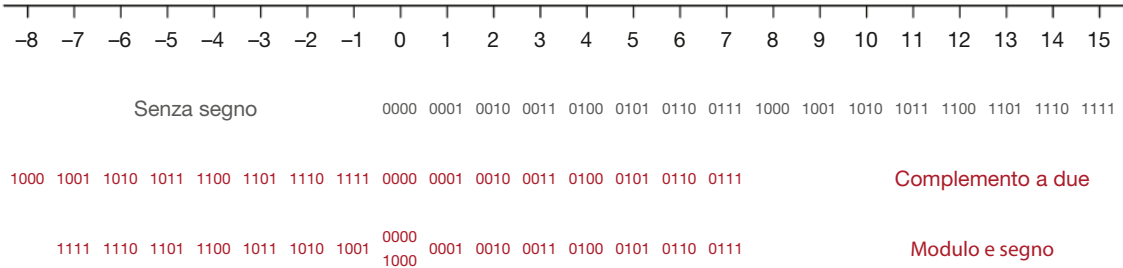


Figura 1.11 Retta dei numeri e codifiche binarie a 4 bit.

1.5.1 La porta NOT

Una porta NOT ha un ingresso, A , e un'uscita, Y , come mostra la **Figura 1.12**. L'uscita della porta NOT è l'esatto contrario del suo ingresso: se A è FALSO allora Y è VERO e, viceversa, se A è VERO allora Y è FALSO. Questa relazione è riassunta dalla tabella delle verità e dall'espressione booleana riportate in figura. Il trattino sopra la A nell'espressione booleana si pronuncia *NON*, quindi $Y = \bar{A}$ si legge "Y uguale NON A". La porta NOT (**negatore**) viene anche chiamata porta invertente (*inverter*).

Si fa presente che altri testi utilizzano una varietà di notazioni per NOT, come $Y = A'$, $Y = \neg A$, $Y = !A$ e $Y = \sim A$.

1.5.2 Buffer

L'altra porta logica a un solo ingresso viene chiamata **buffer**, come mostra la **Figura 1.13**. Questa porta logica riproduce semplicemente il valore di ingresso in uscita.

Dal punto di vista logico, un buffer non è diverso da un semplice filo, quindi all'apparenza potrebbe sembrare di poca utilità. Dal punto di vista elettrico, il buffer ha invece caratteristiche utili, come la possibilità di erogare grandi quantità di corrente, per esempio a un motore elettrico, o la possibilità di trasmettere velocemente il valore della sua uscita a molte porte logiche diverse. Questo è un esempio del perché sia necessario considerare diversi livelli di astrazione per comprendere totalmente un sistema: l'astrazione digitale nasconde l'utilità reale di un buffer.

Il triangolo è il simbolo che rappresenta un buffer. Un pallino all'uscita indica un'inversione (cioè una negazione), come si può osservare nel simbolo della porta NOT nella **Figura 1.12**.

1.5.3 La porta AND

Le porte logiche a due ingressi sono più interessanti. La porta AND, mostrata nella **Figura 1.14**, genera all'uscita Y il valore VERO se e soltanto se sia A sia B hanno valore VERO, altrimenti l'uscita vale FALSO. Per convenzione, le configurazioni di ingressi vengono riportate nell'ordine 00, 01, 10, 11, come se si stesse contando in numeri binari. L'espressione booleana per una porta AND può essere scritta in diversi modi: $Y = A \cdot B$, $Y = AB$, $Y = A \cap B$. Il simbolo $\cap$ è il simbolo di intersezione insiemistica ed è il preferito dai logici. In questo testo si preferisce utilizzare la notazione $Y = AB$.

1.5.4 La porta OR

La porta OR, mostrata nella **Figura 1.15**, produce all'uscita Y il valore VERO se A , oppure B , oppure entrambi gli ingressi hanno valore VERO. L'espressione booleana per una porta OR si può scrivere $Y = A + B$ o anche $Y = A \cup B$. Il simbolo $\cup$ è il simbolo di unione insiemistica, e ancora una volta è il preferito dai logici. I progettisti digitali normalmente utilizzano il segno $+$, $Y = A + B$ che significa "Y uguale A o B".

1.5.5 Altre porte logiche a due ingressi

La **Figura 1.16** mostra altre comuni porte logiche a due ingressi. La porta XOR (OR esclusivo, detto "ex-OR") dà uscita VERO se A oppure B , ma non entrambi, hanno valore VERO. L'operazione XOR è indicata da $\oplus$, un più con un cerchietto intorno. Una porta XOR a N ingressi viene anche chiamata "porta di parità" perché produce in uscita VERO se un numero dispari di ingressi è VERO.

Qualsiasi porta può essere seguita da un pallino per invertire le sue ope-

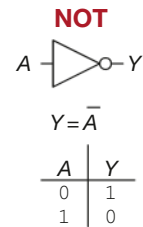


Figura 1.12
Porta NOT.

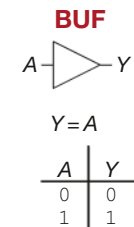


Figura 1.13
Buffer.

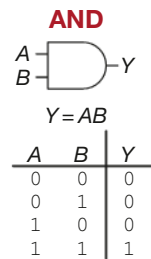


Figura 1.14
Porta AND.

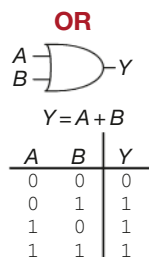


Figura 1.15
Porta OR.

Secondo Larry Wall, inventore del linguaggio di programmazione PERL, "le tre virtù principali di un programmatore sono Pigritia, Impazienza e Tracotanza".

Un modo buffo per ricordarsi il simbolo della porta OR è il fatto che il suo lato di ingresso è curvo come la bocca di un Pacman, quindi la porta OR è affamata di qualsiasi ingresso con valore VERO che riesca a trovare!

XOR

$Y = A \oplus B$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

NAND

$Y = \overline{AB}$

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

NOR

$Y = \overline{A + B}$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

Figura 1.16 Altre porte logiche a due ingressi.

razioni. La porta NAND esegue le operazioni NOT e AND. La sua uscita ha sempre valore VERO tranne quando entrambi gli ingressi sono VERO. La porta NOR esegue le operazioni NOT e OR. La sua uscita è VERO se né A né B sono VERO. Come in tutte le porte a due ingressi, le combinazioni di ingresso nella tabella delle verità vengono elencate in ordine numerico.

ESEMPIO 1.15

La porta XNOR. La **Figura 1.17** mostra il simbolo e l'espressione booleana per una porta XNOR a due ingressi, che esegue l'operazione negata rispetto a una porta XOR. Completare la tabella delle verità.

Soluzione La **Figura 1.18** mostra la tabella delle verità. L'uscita della porta XNOR è VERO se entrambi gli ingressi hanno valore FALSO o entrambi valore VERO. La porta XNOR a due ingressi viene chiamata anche "porta di uguaglianza", perché la sua uscita assume valore VERO quando gli ingressi sono uguali.

Figura 1.17
Porta XNOR.

XNOR

$Y = A \oplus B$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

Figura 1.18
Tabella delle verità della porta XNOR.

NOR3

$Y = \overline{A + B + C}$

A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

Figura 1.19
Porta NOR a tre ingressi.

1.5.6 Porte a ingressi multipli

Esistono molte funzioni booleane di tre o più ingressi. Le più comuni sono AND, OR, XOR, NAND, NOR e XNOR. Una porta AND con un numero *N* di ingressi produce un valore di uscita VERO quando tutti i valori di ingresso sono VERO. Invece una porta OR con un numero *N* di ingressi produce un valore di uscita VERO quando almeno uno dei suoi ingressi è VERO.

ESEMPIO 1.16

Porta NOR a tre ingressi. La **Figura 1.19** mostra il simbolo e l'espressione booleana per una porta NOR a tre ingressi. Completare la tabella delle verità.

Soluzione La **Figura 1.20** mostra la tabella della verità. L'uscita vale VERO se e solo se nessuno degli ingressi è VERO.

ESEMPIO 1.17

Porta AND a quattro ingressi. La **Figura 1.21** mostra il simbolo e l'espressione booleana per una porta AND a quattro ingressi. Scrivere la tabella delle verità.

Soluzione La **Figura 1.22** mostra la tabella delle verità. L'uscita vale VERO solo se tutti gli ingressi sono VERO.

1.6 ■ OLTRE L'ASTRAZIONE DIGITALE

Un sistema digitale utilizza variabili a valori discreti. Nonostante, le variabili sono rappresentate da quantità fisiche continue come la tensione elettrica di un filo, la posizione di un ingranaggio, o il livello di un fluido in un cilindro.

Perciò, un progettista deve scegliere un modo per mettere in relazione il fenomeno fisico continuo con il valore discreto.

Per esempio, si consideri la rappresentazione di un valore binario A con la tensione elettrica di un filo, nell'ipotesi che 0 volt (V) indichino che $A = 0$ e 5 V indichino che $A = 1$. Ogni sistema reale deve tollerare una certa quantità di rumore, quindi è ragionevole pensare che 4.97 V si dovranno interpretare come $A = 1$, come se fossero 5 V. Ma come si interpretano 4.3 V? Oppure 2.8 V? O peggio ancora 2.5 V?

1.6.1 Tensione

Si supponga che il livello di tensione più basso nel sistema sia di 0 V, anche chiamato “terra” o “massa” (*ground*, abbreviato GND). Il livello di tensione più alto nel sistema dipende dall'alimentatore e solitamente viene chiamato V_{DD} . Nella tecnologia degli anni '70 e '80, il valore di V_{DD} era generalmente di 5 V. Con l'evoluzione dei chip verso transistori sempre più piccoli, il livello di V_{DD} è sceso a 3.3 V, 2.5 V, 1.8 V, 1.5 V, 1.2 V, o anche più in basso per risparmiare energia ed evitare di sovraccaricare i transistori.

1.6.2 Livelli logici

Il processo di rappresentazione di una variabile binaria discreta mediante un fenomeno fisico continuo si effettua definendo i livelli logici, come mostra la **Figura 1.23**. La prima porta è chiamata generatore e la seconda ricevitore. L'uscita (output) del generatore è collegata all'ingresso del ricevitore. Il generatore produce un valore di uscita BASSO (0) nell'intervallo da 0 a V_{OL} , oppure ALTO (1) nell'intervallo da V_{OH} a V_{DD} . Se il ricevitore ottiene un ingresso compreso tra 0 e V_{IL} , lo considera un valore BASSO, mentre se ottiene un ingresso compreso tra V_{IH} e V_{DD} , lo considera un valore ALTO. Se per qualche motivo, come rumore o componenti guasti, l'ingresso del ricevitore rientrasse nella zona proibita tra V_{IL} e V_{IH} , il comportamento della porta sarebbe imprevedibile. V_{OH} , V_{OL} , V_{IH} e V_{IL} sono chiamati livelli logici alti e bassi di ingressi e uscite.

1.6.3 Margini di rumore

Perché l'uscita del generatore sia interpretata in maniera corretta all'ingresso del ricevitore, è necessario che sia $V_{OL} < V_{IL}$ e $V_{OH} > V_{IH}$. In questo modo, anche se l'uscita del generatore viene alterata da una certa quantità di rumore, l'ingresso del ricevitore è comunque in grado di riconoscere il livello logico corretto. Il margine di rumore (NM da *noise margin*) è la quantità di rumore che può essere aggiunta all'uscita nella peggiore delle ipotesi per far sì che

A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

Figura 1.20
Tabella delle verità della porta NOR a tre ingressi.

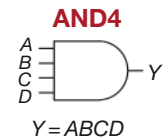


Figura 1.21
Porta AND a quattro ingressi.

A	B	C	D	Y
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

Figura 1.22
Tabella delle verità della porta AND a quattro ingressi.

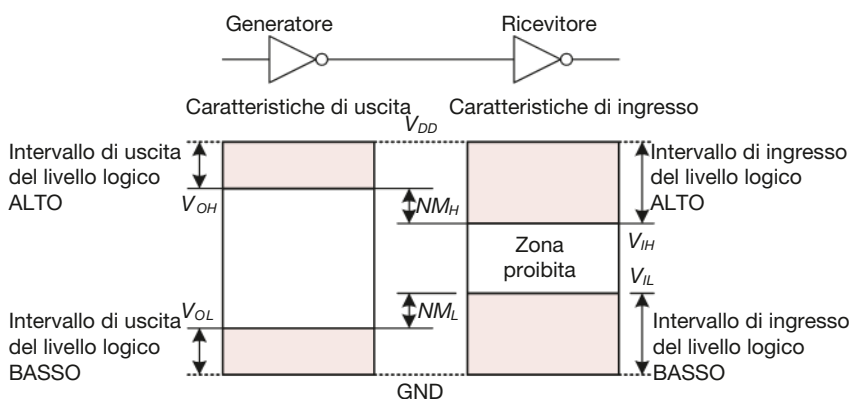


Figura 1.23 Livelli logici e margini di rumore.

V_{DD} sta per tensione al *drain* (letteralmente "pozzo") di un transistor metallo-ossido-semiconduttore, usato nella costruzione della maggior parte dei chip moderni. La tensione dell'alimentatore viene a volte indicata con V_{CC} che sta per tensione al *collector* (collettore) di un transistor a giunzione bipolare usato in tempi meno recenti per la costruzione di chip. La terra è a volte indicata V_{SS} perché è la tensione della source (letteralmente "sorgente") di un transistor metallo-ossido-semiconduttore. Per maggiori informazioni sui transistori vedi il paragrafo 1.7.



Figura 1.24
Rete di invertitori.

DC indica il comportamento che si ha quando una tensione d'ingresso viene mantenuta costante oppure cambia abbastanza lentamente da consentire al resto del sistema di tenersi al passo. L'origine del termine deriva dall'inglese *direct current* (corrente diretta), un modo per trasmettere potenza lungo una linea a tensione costante. Per contro, la risposta ai transitori (*transient response*) di una rete è il comportamento in presenza di tensioni di ingresso che variano velocemente. La risposta ai transitori è approfondita nel paragrafo 2.9.

il segnale riesca ugualmente a essere interpretato come un ingresso valido. Come si può vedere nella **Figura 1.23**, i margini di rumore basso e alto sono rispettivamente

$$NM_L = V_{IL} - V_{OL} \quad (1.2)$$

$$NM_H = V_{IH} - V_{OH} \quad (1.3)$$

ESEMPIO 1.18

Calcolo dei margini di rumore. Si consideri la rete di negatori di **Figura 1.24**. V_{O1} è la tensione di uscita del negatore I1 e V_{I2} la tensione di ingresso al negatore I2. Entrambi i negatori hanno le seguenti caratteristiche: $V_{DD} = 5$ V, $V_{IL} = 1.35$ V, $V_{IH} = 3.15$ V, $V_{OL} = 0.33$ V, $V_{OH} = 3.84$ V. Quali sono i margini di rumore alto e basso dei negatori? La rete può tollerare un rumore di 1 V tra V_{O1} e V_{I2} ?

Soluzione I margini di rumore degli invertitori sono $NM_L = V_{IL} - V_{OL} = (1.35$ V $- 0.33$ V) = 1.02 V, $NM_H = V_{OH} - V_{IH} = (3.84$ V $- 3.15$ V) = 0.69 V. La rete può tollerare un rumore di 1 V se l'uscita ha valore BASSO ($NM_L = 1.02$ V) ma non quando l'uscita ha valore ALTO ($NM_H = 0.69$ V). Quindi se l'invertitore che funge da generatore produce il suo peggior valore di tensione ALTO ($V_{O1} = V_{OH} = 3.84$ V) e il rumore causa una diminuzione di tensione di 1 V prima di raggiungere l'ingresso del ricevitore (3.84 V $- 1$ V = 2.84 V), il valore risultante è troppo basso per essere accettabile come valore di ingresso ALTO ($V_{IH} = 3.15$ V), quindi il ricevitore non è in grado di funzionare correttamente.

1.6.4 Caratteristica di trasferimento DC

Per comprendere i limiti dell'astrazione digitale, bisogna addentrarsi nel comportamento analogico di una porta. La **caratteristica di trasferimento DC** (*Direct Current*, "in corrente continua") di una porta descrive la tensione all'uscita come funzione delle tensioni in ingresso, quando tali tensioni vengono variate abbastanza lentamente da permettere all'uscita di adeguarsi. È chiamata caratteristica di trasferimento perché descrive la relazione tra le tensioni in ingresso e la tensione in uscita.

Un negatore ideale avrebbe una soglia di commutazione improvvisa a $V_{DD}/2$, come mostra la **Figura 1.25(a)**. Ogni volta che $V(A) < V_{DD}/2$, $V(Y) = V_{DD}$, mentre per ogni $V(A) > V_{DD}/2$, $V(Y) = 0$. In questo caso, $V_{IH} = V_{IL} = V_{DD}/2$, $V_{OH} = V_{DD}$ e $V_{OL} = 0$.

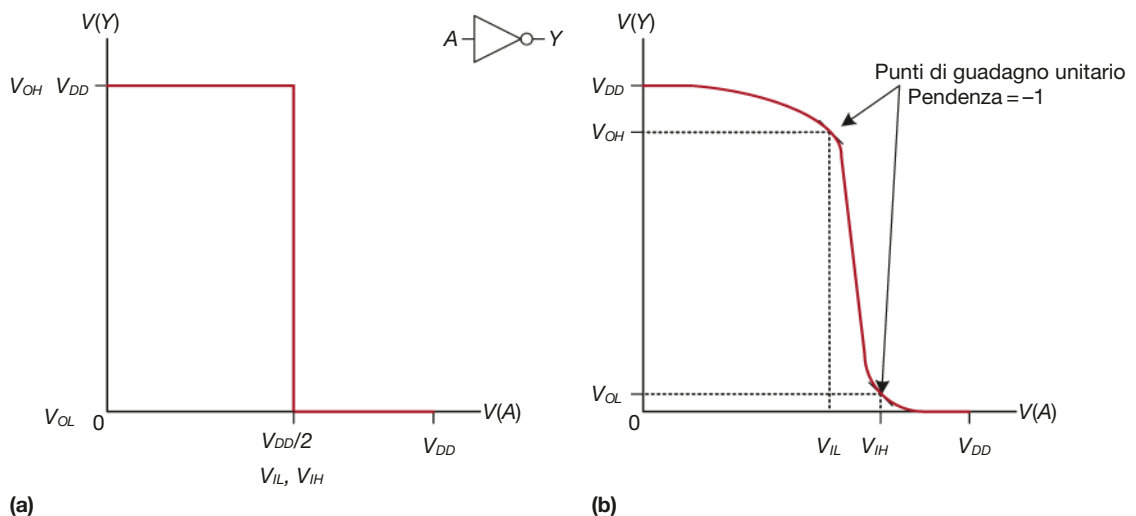


Figura 1.25 Caratteristica di trasferimento DC e livelli logici.

Un invertitore reale, invece, cambia più gradualmente la propria tensione di uscita tra i valori estremi, come mostra la **Figura 1.25(b)**. Quando la tensione di ingresso $V(A)$ è uguale a 0, quella di uscita è $V(Y) = V_{DD}$. Quando $V(A) = V_{DD}$, $V(Y) = 0$. Ma la transizione tra questi valori iniziale e finale è graduale e potrebbe non essere esattamente centrata a $V_{DD}/2$. Questo pone il problema di come definire i livelli logici.

Un punto ragionevole per scegliere i livelli logici è dove la pendenza della caratteristica di trasferimento $dV(Y)/dV(A)$ è pari a -1 . Questi due punti sono chiamati “punti di guadagno unitario”. Scegliere i livelli logici ai punti di guadagno unitario solitamente massimizza i margini di rumore. Se V_{IL} venisse ridotto, V_{OH} aumenterebbe di poco; al contrario, se si aumentasse V_{IL} , V_{OH} diminuirebbe velocemente.

1.6.5 La disciplina statica

Per evitare che gli ingressi cadano nella zona proibita, le porte logiche digitali vengono progettate per essere conformi alla **disciplina statica**. Tale disciplina richiede che, dati valori di ingresso validi dal punto di vista logico, ogni elemento del circuito produca uscite altrettanto valide. Per conformarsi alla disciplina statica i progettisti digitali sacrificano la libertà di utilizzare elementi circuitali analogici arbitrari in cambio della semplicità e della robustezza dei circuiti digitali. I progettisti aumentano il livello di astrazione da analogico a digitale per aumentare la produttività del progetto, nascondendo i dettagli che non sono necessari.

La scelta di V_{DD} e dei livelli logici è arbitraria, ma tutte le porte che comunicano devono avere livelli logici compatibili. Per questo, le porte vengono raggruppate in **famiglie logiche**, in modo che tutte le porte appartenenti a una famiglia logica obbediscano alla disciplina statica quando vengono utilizzate con altre porte della stessa famiglia. Le porte della stessa famiglia logica si incastrano come pezzi di Lego e, per fare ciò, utilizzano tensioni di alimentazione e livelli logici consistenti.

Le quattro grandi famiglie logiche maggiormente diffuse dagli anni '70 fino agli anni '90 sono: Logica Transistore-Transistore (*TTL*, *Transistor-Transistor Logic*), Logica Metallo-Ossido-Semiconduttore Complementare (*CMOS*, *Complementary Metal-Oxide-Semiconductor Logic*), Logica TTL a Bassa Tensione (*LVTTL*, *Low Voltage TTL Logic*) e Logica CMOS a Bassa Tensione (*LVC MOS*, *Low Voltage CMOS Logic*). I loro livelli logici sono messi a confronto nella **Tabella 1.4**. Da allora le famiglie logiche si sono moltiplicate utilizzando tensioni di alimentazione ancora più basse. Le famiglie logiche più diffuse sono analizzate in maggior dettaglio nell'Appendice A.6.

ESEMPIO 1.19

Compatibilità delle famiglie logiche. Quali delle famiglie logiche della **Tabella 1.4** possono comunicare affidabilmente tra loro?

Tabella 1.4 Livelli di tensione delle famiglie logiche a 5 Volt e a 3.3 Volt.

Famiglia logica	V_{DD}	V_{IL}	V_{IH}	V_{OL}	V_{OH}
TTL	5 (4.75-5.25)	0.8	2.0	0.4	2.4
CMOS	5 (4.5-6)	1.35	3.15	0.33	3.84
LVTTL	3.3 (3-6)	0.8	2.0	0.4	2.4
LVC MOS	3.3 (3-6)	0.9	1.8	0.36	2.7

Tabella 1.5 Compatibilità delle famiglie logiche.

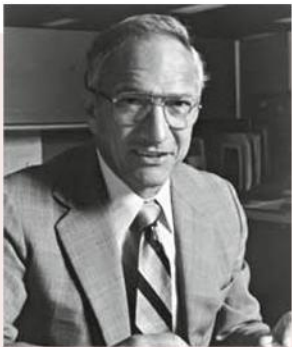
		Ricevitore			
		TTL	CMOS	LVTTL	LVC MOS
Generatore	TTL	OK	NO: $V_{OH} < V_{IH}$	FORSE ^a	FORSE ^a
	CMOS	OK	OK	FORSE ^a	FORSE ^a
	LVTTL	OK	NO: $V_{OH} < V_{IH}$	OK	OK
	LVC MOS	OK	NO: $V_{OH} < V_{IH}$	OK	OK

^a Solo se una tensione di 5 V non danneggia l'ingresso del ricevitore.

Soluzione La **Tabella 1.5** propone un elenco di famiglie logiche con livelli logici compatibili. Si noti che una famiglia logica a 5 V, come la TTL o la CMOS, può produrre tensioni di uscita ALTE fino a 5 V. Se questo segnale a 5 V fosse usato per pilotare un ingresso di una famiglia logica a 3 V, come la LVTTL o la LVC MOS, si potrebbe danneggiare il ricevitore, a meno che quest'ultimo non sia stato progettato specificatamente per essere compatibile con porte a 5 V.

1.7 ■ I TRANSISTORI CMOS*

*Questa sezione, così come altre sezioni contrassegnate da un * nel titolo, è opzionale e non necessaria per comprendere l'argomento principale del testo.*



Robert Noyce, 1927-1990. È nato a Burlington, nello Iowa. Si è laureato in fisica al Grinnell College e ha conseguito il dottorato di ricerca pure in fisica al MIT. Era soprannominato "Sindaco della Silicon Valley" per la sua profonda influenza sull'industria. È stato cofondatore della Fairchild Semiconductor nel 1957 e della Intel nel 1968. Molti ingegneri del suo gruppo di lavoro hanno in seguito fondato altre importanti industrie di semiconduttori.

Il Motore Analitico di Babbage fu costruito con degli ingranaggi, e i primi calcolatori elettrici utilizzavano relè e valvole. I calcolatori moderni utilizzano i transistori, perché sono più economici, più piccoli e più affidabili. I transistori sono interruttori controllati elettricamente che si accendono o si spengono quando una tensione, o una corrente, viene applicata al terminale di controllo. I due tipi principali di transistori sono i **transistori a giunzione bipolare** (transistori BJT, *Bipolar Junction Transistor*) e i **transistori metallo-ossido-semiconduttore a effetto di campo** (transistori MOSFET, *Metal-Oxide-Semiconductor Field Effect Transistor*, o più semplicemente MOS).

Nel 1958, Jack Kilby della Texas Instruments costruì il primo circuito integrato contenente due transistori. Nel 1959, Robert Noyce alla Fairchild Semiconductor brevettò un metodo di interconnessione multipla tra transistori su un singolo chip di silicio. All'epoca, un transistor costava all'incirca 10 dollari.

Grazie a più di quattro decenni di sviluppo dell'industria produttrice, gli ingegneri possono ora inserire all'incirca tre milioni di MOSFET su un chip di silicio di 1 cm², e questi transistori costano meno di un micro centesimo al pezzo. La quantità di transistori su un singolo chip e il loro costo continuano a migliorare di un ordine di grandezza ogni 8 anni. I MOSFET rappresentano oggi i blocchi costruttivi praticamente di tutti i sistemi digitali. In questa sezione si vuole superare l'astrazione digitale per vedere come le porte logiche vengono costruite a partire dai MOSFET.

1.7.1 Semiconduttori

I transistori MOS vengono costruiti a partire dal silicio, elemento predominante sia nelle rocce sia nella sabbia. Il silicio (Si) è un elemento chimico del IV gruppo, il che significa che ha quattro elettroni di valenza e forma legami con quattro atomi adiacenti, producendo come risultato un reticolo cristallino. La **Figura 1.26** mostra il reticolo in due dimensioni per questioni di grafica: in realtà il reticolo forma un cristallo cubico. Nella figura, ogni linea rappresenta un legame covalente. Di per sé, il silicio è uno scarso conduttore elettrico, perché tutti i suoi elettroni sono uniti in legami

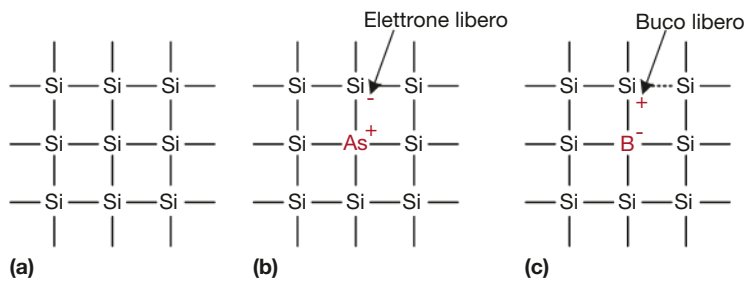


Figura 1.26
Reticolo cristallino del silicio
e atomi droganti.

covalenti. Tuttavia, può diventare un conduttore migliore se piccole quantità di impurità, chiamate “atomi droganti”, vengono aggiunte opportunamente. Se viene aggiunto un drogante appartenente al V gruppo, come per esempio l'arsenico (As), gli atomi droganti hanno un elettrone in più che non viene usato nei legami covalenti. Questo elettrone è quindi in grado di muoversi facilmente nel reticolo, lasciando un atomo drogante ionizzato (As^+) dietro di sé, come mostra la **Figura 1.26(b)**. L'elettrone trasporta una carica negativa, per questo l'arsenico viene chiamato un **drogante di tipo n**. Al contrario, se viene aggiunto un drogante del III gruppo, come per esempio il boro (B), gli atomi droganti mancano di un elettrone, come mostra la **Figura 1.26(c)**. Questo elettrone mancante viene chiamato un **buco**. Un elettrone di un atomo di silicio adiacente ha la possibilità di muoversi per riempire il legame mancante, formando un atomo drogante ionizzato (B^-) e lasciandosi dietro un buco nell'atomo di silicio. Allo stesso modo dell'elettrone, il buco può muoversi facilmente nel reticolo. Il buco è una mancanza di carica negativa e, in quanto tale, agisce come una particella di carica positiva. Per questa ragione il boro è chiamato un “drogante di tipo p”. Dal momento che la conduttività del silicio varia di diversi ordini di grandezza, a seconda della concentrazione di droganti, il silicio è definito **semiconduttore**.

1.7.2 I diodi

La giunzione tra un silicio di tipo n e uno di tipo p viene chiamata **diodo**. La regione di tipo p è chiamata **anodo**, mentre quella di tipo n **catodo**, come illustrato nella **Figura 1.27**. Quando la tensione dell'anodo supera quella del catodo, si dice che il diodo è “polarizzato direttamente” (*forward biased*) e la corrente passa all'interno del diodo dall'anodo al catodo. Quando la tensione dell'anodo è più bassa di quella del catodo, il diodo è “polarizzato inversamente” (*reverse biased*) e non passa corrente nel diodo. Il simbolo del diodo mostra in maniera intuitiva che la corrente può scorrere solo in una direzione.

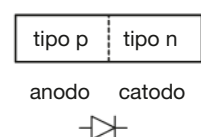


Figura 1.27
Struttura e simbolo elettrico
del diodo a giunzione p-n.

1.7.3 I condensatori

Un condensatore è costituito da due elementi conduttori, come due piastre metalliche, separati da un isolante. Quando una tensione V viene applicata a una delle due piastre, su questa si accumula una carica elettrica Q mentre sull'altra si accumula una carica di segno opposto $-Q$. La **capacità** C del condensatore (misurata in Farad) è uguale al rapporto tra la carica e la tensione: $C = Q/V$. La capacità è proporzionale alla grandezza delle piastre e inversamente proporzionale alla distanza presente tra loro. Il simbolo di un condensatore è riportato nella **Figura 1.28**.

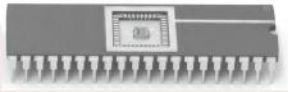
La capacità è importante perché caricare e scaricare le piastre richiede tempo ed energia. Una capacità maggiore indica che un circuito è più lento e che ha bisogno di più energia per operare. Velocità ed energia saranno argomenti ricorrenti in tutto questo testo.



Figura 1.28
Simbolo elettrico del condensatore.



I tecnici in una camera bianca della Intel indossano tute in Gore-Tex per evitare che particelle dei capelli, della pelle o degli indumenti possano contaminare i microscopici transistori sui wafer di silicio. (Fotografia © 2006, Intel Corporation. Con permesso di riproduzione.)



Un contenitore a due file parallele di piedini (*dual-inline package*, DIP) contiene al centro un chip (scarsamente visibile) collegato a 40 piedini, 20 per lato, mediante fili d'oro più sottili di un capello. (Fotografia di Kevin Mapp. © 2006 Harvey Mudd College.)

I contatti source e drain sono fisicamente simmetrici. Tuttavia, si assume per convenzione che le cariche elettriche fluiscano dalla source al drain. In un transistor nMOS le cariche elettriche sono trasportate dagli elettroni, che fluiscono dalla tensione negativa a quella positiva. In un transistor pMOS le cariche elettriche sono trasportate dai "buchi" (*holes*) che fluiscono dalla tensione positiva a quella negativa. Se si disegnano gli schemi elettrici con la tensione positiva in alto e quella negativa in basso, la source di cariche (negative) in un transistor nMOS è il contatto inferiore, mentre la source di cariche (positive) in un transistor pMOS è il contatto superiore.

1.7.4 Transistori nMOS e pMOS

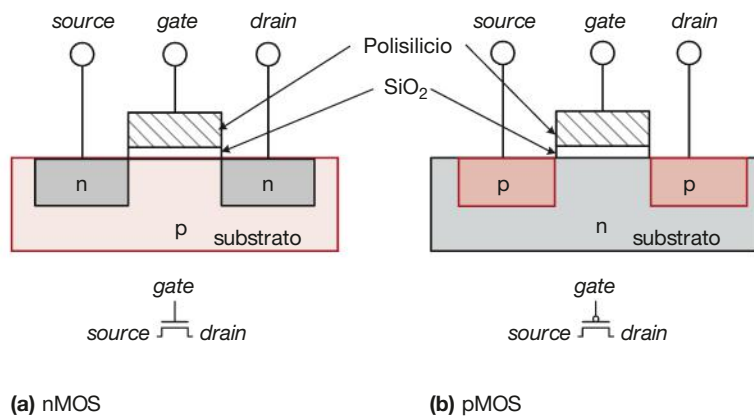
Un MOSFET può essere paragonato a un sandwich con diversi strati di materiali conduttori e isolanti. I MOSFET sono costruiti su dischi sottili di silicio (denominati **wafer**) la cui grandezza varia dai 15 ai 30 cm di diametro. Il processo di costruzione inizia da un wafer vergine e implica una sequenza di passi in cui i droganti vengono impiantati nel silicio, sottili strati di biossido di silicio e di silicio vengono fatti crescere, e viene depositato del metallo. Tra una fase e l'altra, il wafer viene mascherato in modo che i vari materiali appaiano solo dove necessario. Dal momento che i transistori hanno dimensioni dell'ordine della frazione di micron¹ e che il wafer viene lavorato come oggetto unico, il costo della produzione di miliardi di transistori è molto basso. Una volta completato il processo, il wafer iniziale viene tagliato in rettangoli chiamati *chip* o *dice* che contengono migliaia, milioni, o addirittura miliardi di transistori. Ogni chip viene successivamente collaudato e posizionato in un contenitore di plastica o ceramica con contatti metallici (*pin*, in italiano "piedini") che consentono di connetterlo ad altri in una scheda di circuito stampato.

Il sandwich MOSFET consiste di uno strato conduttore chiamato *gate*, posizionato sopra uno strato isolante di biossido di silicio (SiO_2), che a sua volta è posizionato sopra il wafer di silicio iniziale, detto substrato. Una volta, il gate veniva costruito in metallo, da cui il nome metallo-ossido-semiconduttore. Il processo manifatturiero moderno utilizza invece il silicio policristallino per il gate, perché non si scioglie durante le successive fasi di lavorazione, che sono ad alta temperatura. Il biossido di silicio è noto comunemente come vetro e viene spesso chiamato semplicemente ossido nell'industria dei semiconduttori. Il sandwich metallo-ossido-semiconduttore forma un condensatore nel quale un sottile strato di ossido isolante, chiamato dielettrico, separa il metallo dal semiconduttore.

Ci sono due tipi di MOSFET: gli nMOS e i pMOS (enne-MOS e pi-MOS). La **Figura 1.29** mostra una sezione di entrambi i tipi di MOSFET, ottenuta tagliando verticalmente un wafer e osservandolo lateralmente. I transistori di tipo n, chiamati nMOS, hanno regioni di droganti di tipo n adiacenti al gate chiamati *source* e *drain*, che vengono costruite su un substrato di semiconduttore di tipo p. I transistori pMOS sono semplicemente l'opposto, cioè consistono di *source* e *drain* di tipo p su un substrato di tipo n.

Un MOSFET si comporta come un interruttore controllato in tensione, nel quale la tensione al gate crea un campo elettrico che apre o chiude il collegamento tra source e drain. Il termine **transistore a effetto di campo** deriva proprio da questo suo principio di funzionamento.

Figura 1.29
Transistori nMOS e pMOS.



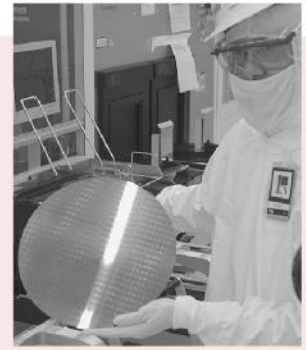
¹ 1 μm = 1 micron = 10^{-6} metri.

Si può analizzare il comportamento di un transistor nMOS, il cui substrato normalmente è collegato a terra, ovvero la tensione più bassa del sistema. Si consideri innanzitutto la situazione in cui il gate è anch'esso a tensione 0, come mostra la **Figura 1.30(a)**. I diodi tra source e drain e il substrato sono polarizzati inversamente, perché la tensione di source e drain non può essere negativa. Dunque, non c'è un percorso che permetta alla corrente di fluire tra source e drain e quindi il transistor è spento. Si consideri ora il caso in cui il *gate* viene portato a V_{DD} , come mostra la **Figura 1.30(b)**. Quando viene applicata una tensione positiva alla piastra superiore di un condensatore, si crea un campo elettrico che attrae carica positiva verso la piastra superiore e carica negativa verso quella inferiore. Se la tensione è sufficiente, viene attirata così tanta carica negativa verso la piastra opposta al gate che la regione si inverte da tipo p a tipo n. Questa regione di inversione viene chiamata **canale**. Ora il transistor ha un percorso elettrico da source di tipo n attraverso il canale di tipo n fino a drain anch'esso di tipo n, cosicché gli elettroni sono liberi di scorrere da source a drain e il transistor è acceso. La tensione al *gate* richiesta per accendere un transistor è chiamata **tensione di soglia**, V_t (da *threshold*) ed è tipicamente compresa tra 0,3 e 0,7 V.

I transistori pMOS lavorano in maniera opposta agli nMOS, come si può facilmente dedurre dal pallino presente nel loro simbolo, mostrato nella **Figura 1.31**. Il substrato viene tenuto a V_{DD} . Quando il gate è anch'esso a V_{DD} , il transistor è spento; quando invece il gate si trova a GND, il canale si inverte a tipo p e il transistor si accende.

Sfortunatamente, i MOSFET non sono interruttori perfetti. In particolare, un transistor nMOS trasmette facilmente degli 0, ma difficilmente degli 1. Nello specifico, quando il gate di un nMOS si trova a V_{DD} , il drain può commutare solo tra 0 e $V_{DD} - V_t$. Allo stesso modo, un transistor pMOS trasmette bene gli 1 ma non gli 0. Si vedrà successivamente che è possibile costruire delle porte logiche che utilizzano i transistori solo nella loro condizione migliore.

Un transistor nMOS necessita di un substrato di tipo p e, viceversa, un transistor pMOS utilizza un substrato di tipo n. Per costruire entrambi i tipi di transistori sullo stesso chip, il processo manifatturiero tipicamente inizia



Un tecnico tiene in mano un wafer da 12 pollici contenente centinaia di chip di microprocessori. (Fotografia © 2006, Intel Corporation. Con permesso di riproduzione.)

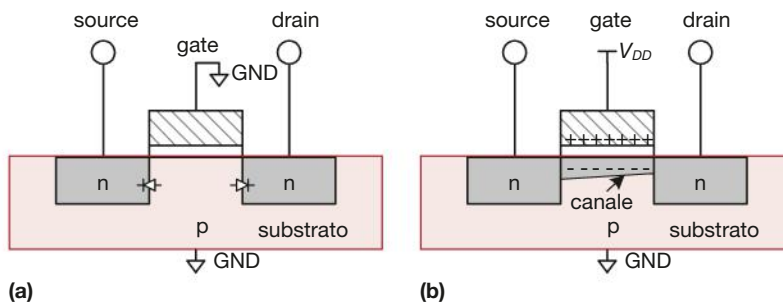


Figura 1.30
Funzionamento del transistor nMOS.

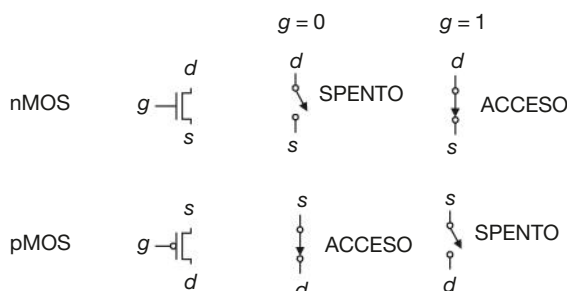
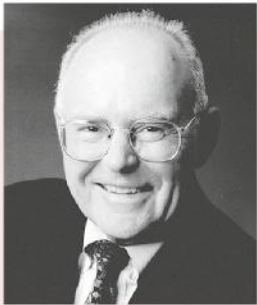


Figura 1.31
Modelli a interruttore dei transistori MOSFET.



Gordon Moore, 1929-. È nato a San Francisco. Si è laureato in chimica a Berkeley e ha conseguito il dottorato di ricerca pure in chimica a Caltech. Cofondatore della Intel nel 1968 insieme a Robert Noyce. Nel 1965 ha osservato che il numero di transistori in un chip raddoppia ogni anno. Questo andamento ha preso il nome di *legge di Moore*. Dal 1975, il numero di transistori in un chip è raddoppiato ogni due anni. Un corollario della legge di Moore è il raddoppio delle prestazioni dei microprocessori ogni 18-24 mesi. Anche le vendite di semiconduttori sono cresciute esponenzialmente. La legge di Moore ha governato l'incredibile avanzata dell'industria dei semiconduttori negli ultimi 50 anni, nei quali la dimensione dei transistori si è ridotta da più di 10 micron a soli 28 nanometri. Tuttavia questo processo mostra segni di rallentamento al di sotto dei 28 nanometri perché la costruzione di transistori molto più piccoli della lunghezza d'onda della luce è estremamente costosa. (Fotografia © 2006, Intel Corporation. Con permesso di riproduzione.)

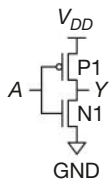


Figura 1.32
Schema elettrico della porta NOT.

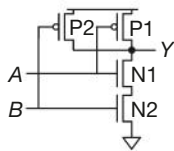


Figura 1.33
Schema elettrico della porta NAND a due ingressi.

con un wafer di tipo p, sul quale viene successivamente creata una regione di tipo n chiamata *well* (pozzo) dove devono essere collocati i transistori pMOS. Questi processi produttivi, che consentono la presenza di entrambi i tipi di transistori, danno luogo ai cosiddetti **MOS Complementari** o CMOS. I processi CMOS vengono utilizzati per la stragrande maggioranza dei transistori fabbricati al giorno d'oggi.

Per ricapitolare, i processi CMOS forniscono due tipi di interruttori controllati elettricamente, come mostra la Figura 1.31. La tensione al gate (*g*) regola il flusso di corrente tra source (*s*) e drain (*d*). Gli nMOS sono spenti quando il gate è 0 e accesi quando il gate è 1. Viceversa, i pMOS sono accesi quando il gate è 0 e spenti quando il gate è 1.

1.7.5 Porta NOT CMOS

La **Figura 1.32** mostra lo schema di una porta NOT costruita con transistori CMOS. Il triangolo in basso indica la terra (GND) mentre la barra piatta in alto indica V_{DD} ; queste due abbreviazioni sono omesse negli schemi successivi. Il transistore nMOS, N1, è connesso tra GND e l'uscita Y, mentre il transistore pMOS, P1, è connesso tra V_{DD} e l'uscita Y. I gate di entrambi i transistori sono controllati dall'ingresso, A.

Se $A = 0$, N1 è spento e P1 è acceso. Di conseguenza, Y è connesso a V_{DD} ma non a GND, e viene innalzato al valore logico 1. P1 trasmette bene un 1. Se invece $A = 1$, N1 è acceso e P1 è spento, e Y viene abbassato al valore logico 0. N1 trasmette correttamente uno 0. Se si esamina il comportamento del circuito tramite la tabella delle verità di Figura 1.12, si nota facilmente che il circuito altro non è che una porta NOT.

1.7.6 Altre porte logiche CMOS

La **Figura 1.33** mostra lo schema di una porta NAND a due ingressi. Negli schemi, i fili sono collegati tra loro negli incroci a tre vie. Negli incroci a quattro vie sono collegati solo se è disegnato un pallino nero. I transistori N1 e N2 sono connessi in serie; entrambi i transistori nMOS devono essere accesi per portare l'uscita a GND (*pull-down*). Al contrario, i transistori pMOS P1 e P2 sono in parallelo; è sufficiente che solo uno dei transistori pMOS sia acceso per portare l'uscita a V_{DD} (*pull-up*). La **Tabella 1.6** riporta il funzionamento delle reti di pull-down e pull-up e lo stato dell'uscita, dimostrando che la porta opera come un NAND. Per esempio, quando $A = 1$ e $B = 0$, N1 è acceso ma N2 è spento, bloccando così il percorso da Y a GND. P1 è spento ma P2 è acceso, creando così un percorso da V_{DD} a Y. Quindi, Y viene portato a 1.

La **Figura 1.34** mostra la forma generale utilizzata per costruire una qualsiasi porta logica invertente, come NOT, NAND, o NOR. I transistori nMOS passano facilmente gli 0, quindi viene inserita una rete di transistori nMOS di pull-down tra l'uscita e GND per portare l'uscita a 0. Viceversa, i transistori pMOS trasmettono bene gli 1, quindi viene inserita una rete di transistori pMOS di pull-up tra l'uscita e V_{DD} per portare l'uscita a 1. Le reti possono essere realizzate con transistori collegati in serie o in parallelo. Quando i transistori sono in parallelo, la rete è accesa se uno dei due transistori è acceso;

Tabella 1.6 Funzionamento della porta NAND.

A	B	Rete di pull-down	Rete di pull-up	Y
0	0	SPENTO	ACCESO	1
0	1	SPENTO	ACCESO	1
1	0	SPENTO	ACCESO	1
1	1	ACCESO	SPENTO	0

quando invece i transistori sono in serie, la rete è accesa solo se entrambi i transistori sono accesi. Il trattino che taglia la linea degli ingressi indica che la porta potrebbe avere ingressi multipli.

Se entrambe le reti pull-up e pull-down fossero accese allo stesso momento, ci sarebbe un corto circuito tra V_{DD} e GND. La tensione all'uscita della porta potrebbe trovarsi nella zona proibita e i transistori consumerebbero una grande quantità di energia, probabilmente abbastanza per bruciarsi. Viceversa, se entrambe le reti fossero spente allo stesso tempo, l'uscita non sarebbe connessa né a V_{DD} né a GND. In questo caso si dice che l'uscita "fluttua" perché il suo valore non è definito. Solitamente uscite fluttuanti non sono opportune, ma nel paragrafo 2.6 si vedrà come occasionalmente possono essere utilizzate per uno scopo ben preciso.

In una porta logica che funzioni correttamente, in ogni momento una delle reti deve essere accesa e l'altra spenta, cosicché l'uscita venga spinta in alto o in basso, senza che ci sia mai un corto circuito o un'uscita fluttuante. Per garantire questo effetto si può utilizzare la regola dei **complementi di conduzione**. Quando i transistori nMOS sono in serie, i transistori pMOS devono essere collegati in parallelo; viceversa, quando i transistori nMOS sono in parallelo, i pMOS devono essere in serie.

ESEMPIO 1.20

Schema di una porta NAND a tre ingressi. Disegnare lo schema di una porta NAND a tre ingressi utilizzando i transistori CMOS.

Soluzione La porta NAND deve dare in uscita 0 solo quando tutti e tre gli ingressi sono 1. Di conseguenza, la rete di pull-down deve avere tre transistori nMOS in serie. Per la regola dei complementi di conduzione, i transistori pMOS devono essere quindi in parallelo. Una porta di questo genere è descritta nella **Figura 1.35**; è possibile verificarne il comportamento controllando che la sua tabella delle verità sia corretta.

ESEMPIO 1.21

Schema di una porta NOR a due ingressi. Disegnare lo schema di una porta NOR a due ingressi utilizzando transistori CMOS.

Soluzione La porta NOR deve dare in uscita 0 se almeno uno dei due ingressi è 1. Di conseguenza, la rete di pull-down deve avere due transistori nMOS in parallelo. Per la regola dei complementi di conduzione, i transistori pMOS devono essere in serie. Una porta di questo tipo è mostrata nella **Figura 1.36**.

ESEMPIO 1.22

Schema di una porta AND a due ingressi. Disegnare lo schema di una porta AND a due ingressi.

Soluzione Non è possibile costruire una porta AND con una singola porta CMOS. È invece semplice la costruzione di porte NAND e NOT. Quindi, il modo migliore per costruire una porta AND utilizzando transistori CMOS è utilizzare una porta NAND seguita da una porta NOT, come mostrato nella **Figura 1.37**.

1.7.7 Porte di trasmissione

A volte, i progettisti trovano conveniente utilizzare un interruttore ideale che trasmetta bene sia gli 0 sia gli 1. Ma i transistori nMOS trasmettono meglio gli 0 mentre i transistori pMOS trasmettono meglio gli 1; di conseguenza, una combinazione parallela dei due è in grado di trasmettere bene entrambi

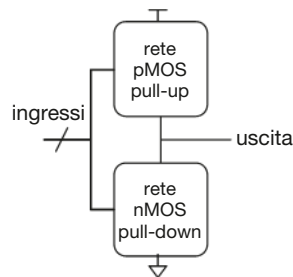


Figura 1.34
Struttura generale di una porta logica invertente.

I progettisti esperti sostengono che i dispositivi elettronici funzionano perché contengono un fumo magico. La conferma di questa teoria è data dal fatto che, se il fumo magico esce dal dispositivo, quello smette di funzionare...

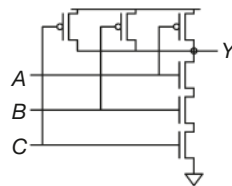


Figura 1.35
Schema elettrico della porta NAND a tre ingressi.

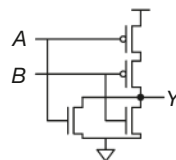


Figura 1.36
Schema elettrico della porta NOR a due ingressi.

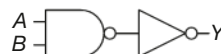


Figura 1.37
Struttura della porta AND a due ingressi.

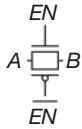


Figura 1.38
Porta di trasmissione.

i valori. La **Figura 1.38** mostra un circuito di questo tipo chiamato **porta di trasmissione** o “porta passante”. I due lati dell’interruttore sono chiamati *A* e *B* perché l’interruttore è bidirezionale e non ha un lato di ingresso e uno di uscita preferiti. I segnali di controllo sono chiamati **abilitazioni** (*enable*) *EN* ed $\overline{EN}$. Quando $EN = 0$ ed $\overline{EN} = 1$, entrambi i transistori sono spenti. Quindi, la porta di trasmissione è spenta o disabilitata, il che significa che *A* e *B* non sono collegati. Quando $EN = 1$ ed $\overline{EN} = 0$, la porta di trasmissione è accesa o abilitata, e qualsiasi valore logico è libero di passare tra *A* e *B*.

1.7.8 Logica pseudo-nMOS

Una porta NOR CMOS a *N* ingressi utilizza *N* transistori nMOS in parallelo e *N* transistori pMOS in serie. I transistori in serie sono più lenti di quelli in parallelo, proprio come i resistori in serie hanno resistenza maggiore di quelli in parallelo. Inoltre, i transistori pMOS sono più lenti di quelli nMOS, dal momento che i buchi non possono muoversi nel reticolo cristallino del silicio alla stessa velocità degli elettroni. Questo implica che i transistori nMOS in parallelo siano estremamente veloci, mentre i transistori pMOS in serie siano lenti, specialmente quando nella serie ve ne sono molti.

La logica pseudo-nMOS sostituisce la serie lenta di transistori pMOS con un singolo transistor pMOS “debole” che resta sempre acceso, come mostrato nella **Figura 1.39**. Questo transistor pMOS viene spesso chiamato **pull-up debole**. Le dimensioni fisiche di questo transistor sono definite in modo tale che sia in grado di spingere l’uscita *Y* debolmente verso ALTO, e questo accade solo quando nessuno dei transistori nMOS è acceso. Ma se almeno uno dei transistori nMOS è acceso, questo è in grado di sopraffare il pull-up debole e l’uscita *Y* viene portata abbastanza vicina a GND da produrre uno 0 logico.

Il vantaggio della logica pseudo-nMOS è che può essere utilizzata per costruire porte NOR veloci con molti ingressi. Per esempio, la **Figura 1.40** mostra una porta pseudo-nMOS NOR a quattro ingressi. Le porte pseudo-nMOS sono utili per alcune memorie e alcune matrici logiche (*logic array*), di cui si parlerà meglio nel Capitolo 5. Lo svantaggio invece è che esiste un percorso elettrico tra il V_{DD} e GND quando l’uscita è BASSA, perché sia il transistor debole pMOS sia uno o più degli nMOS sono accesi. Il percorso elettrico consuma energia continuamente, quindi la logica pseudo-nMOS deve essere utilizzata con parsimonia.

Il nome delle porte pseudo-nMOS è un’eredità degli anni ’70, quando i processi manifatturieri utilizzavano solo transistori nMOS. All’epoca veniva utilizzato un transistor nMOS debole per portare l’uscita al valore ALTO perché i transistori pMOS non erano disponibili.

1.8 ■ CONSUMO DI POTENZA*

Il **consumo di potenza** è la quantità di energia utilizzata per unità di tempo ed è un aspetto di grande importanza per i sistemi digitali. La vita di una batteria nei sistemi portatili come i cellulari e i personal computer è limitata dal consumo di potenza. Ma il consumo di potenza è importante anche per i sistemi collegati alla presa di corrente, perché l’elettricità costa e perché i sistemi si surriscaldano se viene utilizzata troppa energia.

I sistemi digitali consumano sia potenza **dinamica** sia **statica**. La potenza dinamica è quella utilizzata per caricare le capacità dei condensatori quando i segnali cambiano tra 0 e 1, mentre la potenza statica è quella consumata anche quando i segnali non cambiano e il sistema è inattivo.

Le porte logiche e i fili che le collegano hanno una capacità. L’energia prelevata dall’alimentatore e utilizzata per caricare la capacità *C* fino alla tensione

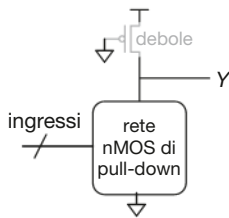


Figura 1.39
Generica porta pseudo-nMOS.

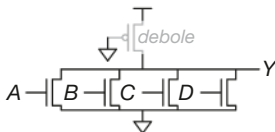


Figura 1.40
Porta NOR a quattro ingressi pseudo-nMOS.

V_{DD} è CV_{DD}^2 . Se la tensione sul condensatore cambia a una frequenza f (si intende f volte al secondo) questo si carica $f/2$ volte e si scarica $f/2$ volte al secondo. Quando il condensatore viene scaricato non viene prelevata energia dall'alimentatore, quindi il consumo di potenza dinamica è pari a

$$P_{\text{dinamica}} = \frac{1}{2} CV_{DD}^2 f \quad (1.4)$$

I dispositivi elettronici usano una certa quantità di corrente anche quando sono inattivi. Quando i transistori sono spenti, disperdono comunque una piccola quantità di corrente. Alcuni circuiti, come le porte pseudo-nMOS di cui si è parlato nel paragrafo 1.7.8, hanno un percorso da V_{DD} a GND attraverso il quale la corrente fluisce continuamente. La corrente statica totale, I_{DD} , viene anche chiamata **corrente di dispersione** o “corrente di alimentazione quiescente” tra V_{DD} e GND. Il consumo di potenza statica è proporzionale a questa corrente statica

$$P_{\text{statica}} = I_{DD} V_{DD} \quad (1.5)$$

ESEMPIO 1.23

Consumo di energia. Un telefono cellulare ha una batteria da 6 watt-ora (Wh) e opera a 1.2 V. Si supponga che, quando è in uso, il cellulare operi a 300 MHz, che la quantità media di capacità legata alle commutazioni dei segnali nei chip del telefono sia 10 nF (1 nanoFarad: 10^{-9} Farad) e che trasmetta 3 W di potenza dalla sua antenna. Quando il telefono non è in uso la potenza dinamica scende quasi a zero, perché l'elaborazione di segnali è spenta. Ma il telefono utilizza anche 40 mA di corrente statica, quando è in uso ma anche quando non lo è. Determinare la vita della batteria del telefono (a) se non viene utilizzato e (b) se viene utilizzato continuamente.

Soluzione La potenza statica è $P_{\text{statica}} = (0,040 \text{ A})(1.2 \text{ V}) = 48 \text{ mW}$. (a) Se il telefono non è in uso, questo è l'unico consumo di potenza, quindi la vita della batteria è pari a $(6 \text{ Wh})/(0.048 \text{ W}) = 125 \text{ ore}$ (circa 5 giorni). (b) Se il telefono è in uso, l'energia dinamica è $P_{\text{dinamica}} = (0.5)(10^{-8} \text{ F})(1.2 \text{ V})^2(3 \times 10^8 \text{ Hz}) = 2.16 \text{ W}$. Insieme alla potenza statica e di trasmissione, la potenza attiva totale è $2.16 \text{ W} + 0.048 \text{ W} + 3 \text{ W} = 5.2 \text{ W}$, quindi la vita della batteria è $6 \text{ Wh}/5.2 \text{ W} = 1.15 \text{ ore}$. Questo esempio è una semplificazione dell'effettivo funzionamento di un telefono cellulare, ma mostra gli aspetti chiave relativi al consumo di energia.

1.9 ■ RIASSUNTO E ANTICIPAZIONE

Ci sono 10 tipi di persone al mondo: quelle che sanno contare in binario e quelle che non lo sanno.

Questo capitolo ha introdotto i principi per comprendere e progettare sistemi complessi. Anche se il mondo reale è analogico, i progettisti digitali si abituano a utilizzare un insieme discreto di segnali possibili. In particolare, le variabili binarie hanno solo due stati possibili: 0 e 1, anche chiamati FALSO e VERO, o BASSO e ALTO. Le porte logiche producono uscite binarie a partire da uno o più ingressi binari. Alcune delle porte logiche più comuni sono:

- **NOT:** VERO quando l'ingresso è FALSO
- **AND:** VERO quando tutti gli ingressi sono VERI
- **OR:** VERO quando almeno un ingresso è VERO
- **XOR:** VERO quando un numero dispari di ingressi è VERO

Le porte logiche vengono comunemente costruite a partire dai transistori, che si comportano come interruttori controllati elettronicamente. I transistori nMOS si accendono quando il gate è 1, mentre i transistori pMOS si accendono quando il gate è 0.

Nei Capitoli dal 2 al 5, si prosegue con lo studio della logica digitale. Il Capitolo 2 si focalizza sulla **logica combinatoria**, nella quale le uscite dipendono solo dagli ingressi correnti. Le porte logiche già introdotte sono esempi di logica combinatoria. Il lettore imparerà a progettare circuiti che contengono più porte per costruire una relazione tra ingressi e uscite che sia descritta da una tabella della verità o da un'espressione booleana. Il Capitolo 3 si focalizza invece sulla **logica sequenziale**, nella quale le uscite non dipendono solo dagli ingressi presenti ma anche da quelli passati. I **registri** sono elementi sequenziali comuni, che ricordano i valori di ingressi precedenti. Gli **automi a stati finiti**, costruiti a partire dai registri e dalla logica combinatoria, sono metodi potenti per la costruzione di sistemi complessi. Si studierà anche la temporizzazione dei sistemi digitali per analizzare a quale velocità è in grado di operare il sistema. Il Capitolo 4 illustra i linguaggi di descrizione dell'hardware (HDL, *Hardware Description Languages*). Gli HDL sono imparentati con i linguaggi di programmazione convenzionale ma vengono utilizzati per simulare e costruire l'hardware piuttosto che il software. La maggior parte dei sistemi digitali oggi è progettata con HDL. SystemVerilog e VHDL sono i due linguaggi prevalenti, e vengono analizzati e messi a confronto in questo testo. Il Capitolo 5 studia altri blocchi combinatori e sequenziali come i sommatore, i moltiplicatori e le memorie.

Il Capitolo 6 è il capitolo di passaggio all'architettura dei calcolatori: descrive il processore ARM, un microprocessore industriale standard utilizzato in quasi tutti gli smartphone e i tablet e in molti altri dispositivi, dai flipper, alle automobili, ai server. L'architettura ARM è definita dai suoi registri e dal suo set di istruzioni in linguaggio assembly. Il lettore imparerà a scrivere programmi in linguaggio assembly per processori ARM in modo da comunicare col processore nella sua lingua nativa.

I Capitoli 7 e 8 coprono il salto tra logica digitale e architettura del calcolatore. Il Capitolo 7 analizza la microarchitettura, la disposizione dei blocchi digitali, come i sommatore e i registri, che sono necessari per costruire un processore. In questo capitolo, il lettore imparerà a costruire il proprio processore ARM. Imparerà infatti tre microarchitetture che presentano tre differenti rapporti costo/prestazioni. Le prestazioni del processore sono cresciute esponenzialmente, richiedendo sistemi di memoria ancora più sofisticati per soddisfare la domanda insaziabile di dati. Il Capitolo 8 si concentra sull'architettura dei sistemi di memoria. Il Capitolo 9 (disponibile come supplemento web, *vedi* la Prefazione) descrive come i calcolatori comunicano coi dispositivi periferici come monitor, radio Bluetooth e motori.

Esercizi

Esercizio 1.1 Spiegare con un paragrafo di testo almeno tre livelli di astrazione che sono usati da:

- (a) biologi che studiano il comportamento di una cellula
- (b) chimici che studiano la composizione della materia

Esercizio 1.2 Spiegare con un paragrafo di testo come le tecniche di gerarchia, modularità e regolarità possono essere usate da:

- (a) progettisti di automobili
- (b) uomini d'affari per gestire le proprie attività

Esercizio 1.3 Il signor Ben Imbrogliabit sta costruendo una casa. Spiegare come può usare i principi di gerarchia, modularità e regolarità per risparmiare tempo e denaro durante la costruzione.

Esercizio 1.4 Una tensione analogica è nell'intervallo 0-5 V. Se il suo valore può essere misurato con un'accuratezza di ± 50 mV, al massimo quanti bit di informazione sono associabili a tale valore?

Esercizio 1.5 Sulla parete di un'aula c'è un vecchio orologio con la lancetta dei minuti rotta.

- (a) Se si riescono a leggere le ore con un'approssimazione di 15 minuti, quanti bit di informazione l'orologio è in grado di associare all'orario?
- (b) Se si conosce anche se è mattina o pomeriggio, quanti ulteriori bit di informazione si hanno relativamente all'orario?

Esercizio 1.6 I Babilonesi hanno sviluppato il sistema di numerazione sessagesimale (base 60) circa 4000 anni fa. Quanti bit di informazione sono associati a una cifra sessagesimale? Come si scrive in sessagesimale il numero 4000_{10} ?

Esercizio 1.7 Quanti diversi numeri si possono rappresentare con 16 bit?

Esercizio 1.8 Qual è il più grande numero binario senza segno a 32 bit?

Esercizio 1.9 Qual è il più grande numero binario a 16 bit che può essere rappresentato utilizzando:

- (a) numeri senza segno?
- (b) numeri in complemento a 2?
- (c) numeri in modulo e segno?

Esercizio 1.10 Qual è il più grande numero binario a 32 bit che può essere rappresentato utilizzando:

- (a) numeri senza segno?
- (b) numeri in complemento a 2?
- (c) numeri in modulo e segno?

Esercizio 1.11 Qual è il più piccolo (cioè il più negativo) numero binario a 16 bit che può essere rappresentato utilizzando:

- (a) numeri senza segno?
- (b) numeri in complemento a 2?
- (c) numeri in modulo e segno?

Esercizio 1.12 Qual è il più piccolo (cioè il più negativo) numero binario a 32 bit che può essere rappresentato utilizzando:

- (a) numeri senza segno?
- (b) numeri in complemento a 2?
- (c) numeri in modulo e segno?

Esercizio 1.13 Convertire i seguenti numeri binari senza segno in numeri decimali. Mostrare il procedimento seguito.

- (a) 1010_2
- (b) 110110_2
- (c) 11110000_2
- (d) 000100010100111_2

Esercizio 1.14 Convertire i seguenti numeri binari senza segno in numeri decimali. Mostrare il procedimento seguito.

- (a) 1110_2
- (b) 100100_2
- (c) 11010111_2
- (d) 011101010100100_2

Esercizio 1.15 Ripetere l'Esercizio 1.13 convertendo i numeri binari in esadecimale.

Esercizio 1.16 Ripetere l'Esercizio 1.14 convertendo i numeri binari in esadecimale.

Esercizio 1.17 Convertire i seguenti numeri esadecimali in numeri decimali. Mostrare il procedimento seguito.

- (a) $A5_{16}$
- (b) $3B_{16}$
- (c) $FFFF_{16}$
- (d) $D000000_{16}$

Esercizio 1.18 Convertire i seguenti numeri esadecimali in numeri decimali. Mostrare il procedimento seguito.

- (a) $4E_{16}$
- (b) $7C_{16}$
- (c) $ED3A_{16}$
- (d) $403FB001_{16}$

Esercizio 1.19 Ripetere l'Esercizio 1.17 convertendo i numeri esadecimali in binario senza segno.

Esercizio 1.20 Ripetere l'Esercizio 1.18 convertendo i numeri esadecimali in binario senza segno.

Esercizio 1.21 Convertire i seguenti numeri binari in complemento a due in numeri decimali.

- (a) 1010_2
- (b) 110110_2
- (c) 01110000_2
- (d) 10011111_2

Esercizio 1.22 Convertire i seguenti numeri binari in complemento a due in numeri decimali.

- (a) 1110_2
- (b) 100011_2
- (c) 01001110_2
- (d) 10110101_2

Esercizio 1.23 Ripetere l'Esercizio 1.21 nell'ipotesi che i numeri binari siano rappresentati in modulo e segno invece che in complemento a due.

Esercizio 1.24 Ripetere l'Esercizio 1.22 nell'ipotesi che i numeri binari siano rappresentati in modulo e segno invece che in complemento a due.

Esercizio 1.25 Convertire i seguenti numeri decimali in numeri binari senza segno.

- (a) 42_{10}
- (b) 63_{10}
- (c) 229_{10}
- (d) 845_{10}

Esercizio 1.26 Convertire i seguenti numeri decimali in numeri binari senza segno.

- (a) 14_{10}
- (b) 52_{10}
- (c) 339_{10}
- (d) 711_{10}

Esercizio 1.27 Ripetere l'Esercizio 1.25 convertendo i numeri in esadecimale.

Esercizio 1.28 Ripetere l'Esercizio 1.26 convertendo i numeri in esadecimale.

Esercizio 1.29 Convertire i seguenti numeri decimali in numeri in complemento a due a 8 bit, indicando se il numero decimale genera traboccamento.

- (a) 42_{10}
- (b) -63_{10}
- (c) 124_{10}
- (d) -128_{10}
- (e) 133_{10}

Esercizio 1.30 Convertire i seguenti numeri decimali in numeri in complemento a due a 8 bit, indicando se il numero decimale genera traboccamento.

- (a) 24_{10}
- (b) -59_{10}
- (c) 128_{10}
- (d) -150_{10}
- (e) 127_{10}

Esercizio 1.31 Ripetere l'Esercizio 1.29 convertendo in numeri in modulo e segno a 8 bit.

Esercizio 1.32 Ripetere l'Esercizio 1.30 convertendo in numeri in modulo e segno a 8 bit.

Esercizio 1.33 Convertire i seguenti numeri a 4 bit in complemento a due in numeri in complemento a due a 8 bit.

- (a) 0101_2
- (b) 1010_2

Esercizio 1.34 Convertire i seguenti numeri a 4 bit in complemento a due in numeri in complemento a due a 8 bit.

- (a) 0111_2
- (b) 1001_2

Esercizio 1.35 Ripetere l'Esercizio 1.33 nell'ipotesi che i numeri a 4 bit siano rappresentati in modulo e segno invece che in complemento a due.

Esercizio 1.36 Ripetere l'Esercizio 1.34 nell'ipotesi che i numeri a 4 bit siano rappresentati in modulo e segno invece che in complemento a due.

Esercizio 1.37 La base 8 è detta ottale. Convertire i numeri dell'Esercizio 1.25 in ottale.

Esercizio 1.38 La base 8 è detta ottale. Convertire i numeri dell'Esercizio 1.26 in ottale.

Esercizio 1.39 Convertire i seguenti numeri ottali in numeri binari, decimali ed esadecimali.

- (a) 42_8
- (b) 63_8
- (c) 255_8
- (d) 3047_8

Esercizio 1.40 Convertire i seguenti numeri ottali in numeri binari, decimali ed esadecimali.

- (a) 23_8
- (b) 45_8
- (c) 371_8
- (d) 2560_8

Esercizio 1.41 Quanti numeri in complemento a due a 5 bit sono maggiori di 0? Quanti sono minori di 0? Come cambierebbero le risposte alle precedenti domande nel caso di numeri in modulo e segno?

Esercizio 1.42 Quanti numeri in complemento a due a 7 bit sono maggiori di 0? Quanti sono minori di 0? Come cambierebbero le risposte alle precedenti domande nel caso di numeri in modulo e segno?

Esercizio 1.43 Quanti byte ci sono in una parola di 32 bit? Quanti nibble nella stessa parola?

Esercizio 1.44 Quanti byte ci sono in una parola di 64 bit?

Esercizio 1.45 Si consideri un modem DSL che opera a 768 kbit/sec. Quanti byte può ricevere in un minuto?

Esercizio 1.46 Un'interfaccia USB 3.0 può trasmettere dati a 5 Gbit/sec. Quanti byte può trasmettere in un minuto?

Esercizio 1.47 I produttori di dischi rigidi usano il termine "megabyte" per indicare 10^6 byte e "gigabyte" per indicare 10^9 byte. Quanti GB effettivi di musica si possono memorizzare in un disco rigido da 50 GB?

Esercizio 1.48 Stimare il valore di 2^{31} senza usare la calcolatrice.

Esercizio 1.49 Una memoria per il microprocessore Pentium II è organizzata come matrice rettangolare di 2^8 righe per 2^9 colonne. Calcolare quanti bit contiene senza usare la calcolatrice.

Esercizio 1.50 Disegnare una retta dei numeri simile a quella della Figura 1.11 per numeri a 3 bit senza segno, in complemento a due e in modulo e segno.

Esercizio 1.51 Tracciare una retta dei numeri simile a quella della Figura 1.11 per numeri a 2 bit senza segno, in complemento a due e in modulo e segno.

Esercizio 1.52 Eseguire le seguenti somme di numeri binari senza segno. Indicare se si verifica traboccamento se il risultato è a 4 bit.

(a) $1001_2 + 0100_2$

(b) $1101_2 + 1011_2$

Esercizio 1.53 Eseguire le seguenti somme di numeri binari senza segno. Indicare se si verifica traboccamento se il risultato è a 8 bit.

(a) $10011001_2 + 01000100_2$

(b) $11010010_2 + 10110110_2$

Esercizio 1.54 Ripetere l'Esercizio 1.52 nell'ipotesi che i numeri binari siano rappresentati in complemento a due.

Esercizio 1.55 Ripetere l'Esercizio 1.53 nell'ipotesi che i numeri binari siano rappresentati in complemento a due.

Esercizio 1.56 Convertire le seguenti coppie di numeri decimali in numeri in complemento a due a 6 bit e sommarli. Indicare se si verifica traboccamento se il risultato è a 6 bit.

(a) $16_{10} + 9_{10}$

(b) $27_{10} + 31_{10}$

(c) $-4_{10} + 19_{10}$

(d) $3_{10} + -32_{10}$

(e) $-16_{10} + -9_{10}$

(f) $-27_{10} + -31_{10}$

Esercizio 1.57 Ripetere l'Esercizio 1.56 per le seguenti coppie di numeri decimali.

(a) $7_{10} + 13_{10}$

(b) $17_{10} + 25_{10}$

(c) $-26_{10} + 8_{10}$

(d) $31_{10} + -14_{10}$

(e) $-19_{10} + -22_{10}$

(f) $-2_{10} + -29_{10}$

Esercizio 1.58 Sommare le seguenti coppie di numeri esadecimali senza segno. Indicare se si verifica traboccamento se il risultato è a 8 bit (due cifre esadecimali).

(a) $7_{16} + 9_{16}$

(b) $13_{16} + 28_{16}$

(c) $AB_{16} + 3E_{16}$

(d) $8F_{16} + AD_{16}$

Esercizio 1.59 Sommare le seguenti coppie di numeri esadecimali senza segno. Indicare se si verifica traboccamento se il risultato è a 8 bit (due cifre esadecimali).

(a) $22_{16} + 8_{16}$

(b) $73_{16} + 2C_{16}$

(c) $7F_{16} + 7F_{16}$

(d) $C2_{16} + A4_{16}$

Esercizio 1.60 Convertire le seguenti coppie di numeri decimali in numeri in complemento a due a 5 bit ed effettuare le sottrazioni. Indicare se si verifica traboccamento se il risultato è a 5 bit.

(a) $9_{10} - 7_{10}$

(b) $12_{10} - 15_{10}$

(c) $-6_{10} - 11_{10}$

(d) $4_{10} - -8_{10}$

Esercizio 1.61 Convertire le seguenti coppie di numeri decimali in numeri in complemento a due a 6 bit ed effettuare le sottrazioni. Indicare se si verifica traboccamento se il risultato è a 6 bit.

(a) $18_{10} - 12_{10}$

(b) $30_{10} - 9_{10}$

(c) $-28_{10} - 3_{10}$

(d) $16_{10} - 21_{10}$

Esercizio 1.62 Nella codifica **eccesso B** di numeri binari a N bit, i numeri positivi e negativi sono rappresentati con il loro valore cui viene sommato l'eccesso B . Per esempio, nella codifica eccesso 15 a 5 bit, il numero 0 è rappresentato come 01111, il numero 1 come 10000, e così via. Le codifiche a eccesso sono usate nell'aritmetica in virgola mobile, che sarà discussa nel Capitolo 5. Si consideri una codifica eccesso 127_{10} di numeri a 8 bit.

- (a) Quale valore decimale è rappresentato dalla codifica 10000010₂?
- (b) Qual è la codifica binaria del numero 0?
- (c) Quali sono la codifica e il valore del massimo numero negativo?
- (d) Quali sono la codifica e il valore del massimo numero positivo?

Esercizio 1.63 Tracciare una retta dei numeri simile a quella della Figura 1.11 per numeri eccesso 3 a 3 bit (vedi l'Esercizio 1.62 per la definizione di codifica a eccesso).

Esercizio 1.64 Nella codifica BCD (*Binary Coded Decimal*, cifra decimale codificata in binario) si usano 4 bit per rappresentare le cifre decimali da 0 a 9. Per esempio, il numero decimale 37_{10} viene codificato in BCD come 00110111_{BCD}.

- (a) Codificare 289_{10} in BCD
- (b) Convertire 100101010001_{BCD} in decimale
- (c) Convertire 01101001_{BCD} in binario
- (d) Elencare i motivi dell'utilità della codifica BCD

Esercizio 1.65 Rispondete alle seguenti domande relative alla codifica BCD (vedi l'Esercizio 1.64 per la definizione di BCD).

- (a) Codificare 371_{10} in BCD
- (b) Convertire 000110000111_{BCD} in decimale
- (c) Convertire 10010101_{BCD} in binario
- (d) Elencare gli svantaggi della codifica BCD rispetto alla rappresentazione binaria dei numeri

Esercizio 1.66 Un disco volante si è schiantato nei campi di grano del Nebraska. L'indagine dell'FBI sul relitto ha portato alla scoperta di un manuale contenente un'espressione matematica nel sistema numerico dei Marziani: $325 + 42 = 411$. Ipotizzando che l'espressione sia giusta, quante dita delle mani si pensa possano avere i Marziani invece delle nostre dieci?

Esercizio 1.67 Ben Imbrogliabit e Alyssa Guastacomputer hanno una discussione. Ben dice: "Tutti i numeri interi maggiori di zero e multipli di sei hanno esattamente due uni nella loro rappresentazione binaria." Alyssa non è d'accordo. Lei dice: "No, però tutti questi numeri hanno un numero dispari di uni nella loro rappresentazione." Indicare se si è d'accordo con Ben, con Alyssa, con entrambi o con nessuno dei due e perché.

Esercizio 1.68 Ben Imbrogliabit e Alyssa Guastacomputer hanno un'altra discussione. Ben dice: "Per ottenere il complemento a due di un numero sottraggo 1 e poi inverte tutti i bit del risultato." Alyssa non è d'accordo. Lei dice: "No, io esamino ogni bit del numero a partire dal meno significativo. Quando trovo il primo uno, inverte tutti i bit successivi." Indicare se si è d'accordo con Ben, con Alyssa, con entrambi o con nessuno dei due e perché.

Esercizio 1.69 Scrivere in un qualsiasi linguaggio di programmazione (per es. C, Java, Perl) un programma per convertire numeri binari in decimale. L'utente deve poter inserire da tastiera un numero binario senza segno e il programma restituisce il corrispondente valore decimale.

Esercizio 1.70 Ripetere l'Esercizio 1.69 ma convertendo i numeri da una generica base b_1 a una generica base b_2 , indicate dall'utente. Gestire basi fino a 16, usando le dieci cifre decimali e le prime sei lettere dell'alfabeto (da A a F). L'utente deve poter inserire da tastiera b_1 , b_2 e il numero codificato in base b_1 da convertire, e il programma restituisce il corrispondente valore in base b_2 .

Esercizio 1.71 Rappresentare il simbolo elettrico, l'espressione booleana e la tabella delle verità per

- (a) una porta OR a tre ingressi
- (b) una porta OR esclusivo (XOR) a tre ingressi
- (c) una porta XNOR a quattro ingressi

Esercizio 1.72 Rappresentare il simbolo elettrico, l'espressione booleana e la tabella delle verità per

- (a) una porta OR a quattro ingressi
- (b) una porta XNOR a tre ingressi
- (c) una porta NAND a cinque ingressi

Esercizio 1.73 Una porta a maggioranza genera in uscita il valore VERO se e solo se più della metà dei suoi ingressi assume valore VERO. Completare la tabella delle verità della porta a maggioranza a tre ingressi rappresentata nella Figura 1.41.

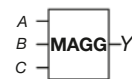


Figura 1.41 Porta a maggioranza a tre ingressi.

Esercizio 1.74 La porta AND-OR (AO) a tre ingressi rappresentata nella Figura 1.42 genera in uscita il valore VERO se sia A sia B assumono valore VERO, oppure se C assume valore VERO. Completare la tabella delle verità della porta.



Figura 1.42 Porta AND-OR a tre ingressi.

Esercizio 1.75 La porta OR-AND-INVERT (OAI) a tre ingressi rappresentata nella Figura 1.43 genera in uscita il valore VERO se C assume valore VERO e A oppure B assumono valore VERO. Negli altri casi genera in uscita il valore FALSO. Completare la tabella delle verità della porta.

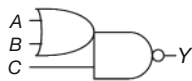


Figura 1.43 Porta OR-AND-INVERT a tre ingressi.

Esercizio 1.76 Ci sono 16 diverse tabelle delle verità per una funzione booleana di due variabili. Scrivere tutte le tabelle, associando a ciascuna una breve descrizione (come OR, NAND ecc.).

Esercizio 1.77 Quante diverse tabelle delle verità si possono costruire per una funzione booleana di N variabili?

Esercizio 1.78 Si possono assegnare dei livelli logici in modo tale che un dispositivo con la curva caratteristica di trasferimento rappresentata nella Figura 1.44 si comporti da negatore? Se sì, quali sono i livelli di ingresso e uscita bassi e alti (V_{IL} , V_{OL} , V_{IH} e V_{OH}) e i margini di rumore (NM_L e NM_H)? Se no, perché?

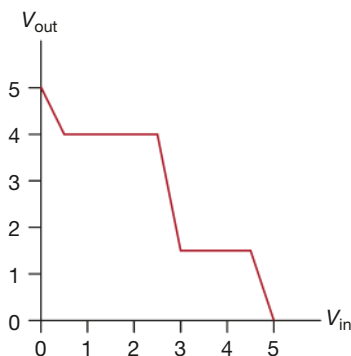


Figura 1.44 Curva caratteristica di trasferimento DC.

Esercizio 1.79 Ripetere l'Esercizio 1.78 per la curva caratteristica di trasferimento rappresentata nella Figura 1.45.

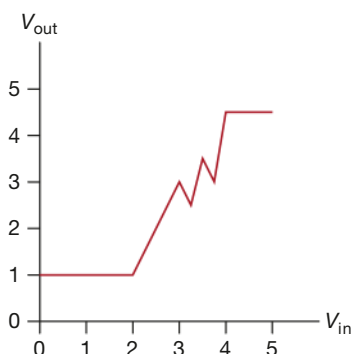


Figura 1.45 Curva caratteristica di trasferimento DC.

Esercizio 1.80 Si possono assegnare dei livelli logici in modo tale che un dispositivo con la curva caratteristica di trasferimento rappresentata nella Figura 1.46 si comporti da buffer? Se sì,

quali sono i livelli di ingresso e uscita bassi e alti (V_{IL} , V_{OL} , V_{IH} e V_{OH}) e i margini di rumore (NM_L e NM_H)? Se no, perché?

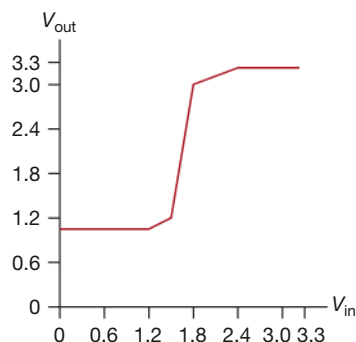


Figura 1.46 Curva caratteristica di trasferimento DC.

Esercizio 1.81 Ben Imbrogliabit ha inventato un circuito con la curva caratteristica di trasferimento rappresentata nella Figura 1.47 e vuole usarlo come buffer. Funziona? Perché? Ben vorrebbe poter dire che il suo circuito è compatibile con le logiche LVCMOS e LVTTL. Il circuito può ricevere correttamente i propri valori di ingresso da queste famiglie logiche? Può pilotare con la propria uscita queste famiglie logiche? Motivare le proprie risposte.

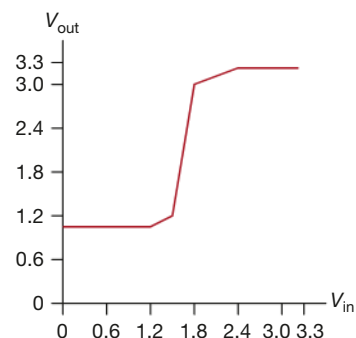


Figura 1.47 Curva caratteristica di trasferimento DC del buffer di Ben.

Esercizio 1.82 Ben Imbrogliabit ha trovato una porta logica a due ingressi la cui funzione di trasferimento è rappresentata nella Figura 1.48, dove A e B sono gli ingressi e Y l'uscita.

- Che tipo di porta logica ha trovato?
- Quali sono approssimativamente i livelli logici alti e bassi?

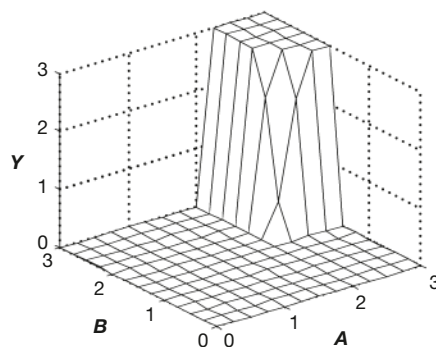


Figura 1.48 Curva caratteristica di trasferimento DC a due ingressi.

Esercizio 1.83 Ripetere l'Esercizio 1.82 in riferimento alla Figura 1.49.

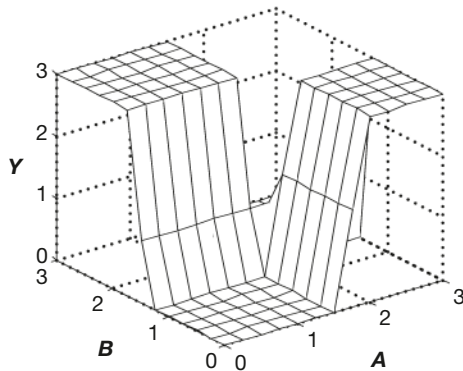


Figura 1.49 Curva caratteristica di trasferimento DC a due ingressi.

Esercizio 1.84 Disegnare la struttura delle seguenti porte logiche CMOS utilizzando il minimo numero di transistori:

- (a) porta NAND a quattro ingressi
- (b) porta OR-AND-INVERT a tre ingressi (vedi l'Esercizio 1.75)
- (c) porta AND-OR a tre ingressi (vedi l'Esercizio 1.74)

Esercizio 1.85 Disegnare la struttura delle seguenti porte logiche CMOS utilizzando il minimo numero di transistori:

- (a) porta NOR a tre ingressi
- (b) porta AND a tre ingressi
- (c) porta OR a due ingressi

Esercizio 1.86 Una porta a minoranza genera in uscita il valore VERO se e solo se meno della metà dei suoi ingressi assumono valore VERO, altrimenti genera il valore FALSO. Disegnare utilizzando il minimo numero di transistori la struttura della porta a minoranza a tre ingressi in tecnologia CMOS.

Esercizio 1.87 Scrivere la tabella delle verità della funzione booleana a due ingressi (A e B) associata alla porta logica della Figura 1.50. Che nome ha questa funzione?

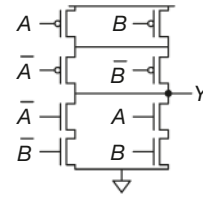


Figura 1.50 Schema elettrico misterioso.

Esercizio 1.88 Scrivere la tabella delle verità della funzione booleana a tre ingressi (A , B e C) associata alla porta logica della Figura 1.51.

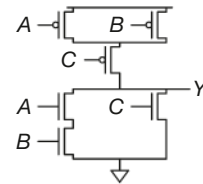


Figura 1.51 Schema elettrico misterioso.

Esercizio 1.89 Disegnare la struttura delle seguenti porte logiche a tre ingressi (A , B e C) utilizzando il minimo numero di transistori e solo porte logiche pseudo-nMOS.

- (a) porta NOR a tre ingressi
- (b) porta NAND a tre ingressi
- (c) porta AND a tre ingressi

Esercizio 1.90 La logica RTL (*Resistor-Transistor Logic*, logica a resistori e transistori) usa transistori nMOS per portare l'uscita delle porte logiche al valore BASSO, e un piccolo resistore per portarle al valore ALTO quando non è attivo nessuno dei percorsi elettrici verso massa. Una porta NOT RTL è rappresentata nella Figura 1.52. Disegnare la struttura di una porta NOR RTL a tre ingressi utilizzando il minimo numero di transistori.

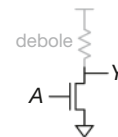


Figura 1.52 Porta NOT RTL.

Domande di valutazione

Queste domande sono state poste a candidati per un posto di lavoro nell'ambito della progettazione di sistemi digitali.

Domanda 1.1 Le chiediamo di disegnare lo schema a livello di transistori di una porta NOR CMOS a 4 ingressi.

Domanda 1.2 Il re ha ricevuto 64 monete d'oro come tasse, ma ha ragione di sospettare che una sia falsa, e la convoca per individuarla. Lei ha a disposizione una bilancia con due piatti su cui mettere le monete: quante pesate occorrono per trovare la moneta falsa, che è più leggera delle altre?

Domanda 1.3 Un professore, il suo assistente, un laureando in progettazione di sistemi digitali e una matricola devono passare su un ponte traballante in una notte oscura. Il ponte è così instabile che solo due persone alla volta possono passare. I quattro hanno una sola torcia elettrica, e il ponte è troppo lungo per poterla gettare da una parte all'altra, quindi deve essere riportata a mano per essere usata dagli altri. La matricola riesce a passare il ponte in 1 minuto, il laureando in 2 minuti, l'assistente in 5 minuti e il professore – sempre distratto dai suoi profondi pensieri – in 10 minuti. Saprebbe dirci qual è il minimo tempo necessario perché tutti abbiano oltrepassato il ponte?

Progetto di reti logiche combinatorie

Capitolo

2

- 2.1 Introduzione
- 2.2 Espressioni booleane
- 2.3 Algebra booleana
- 2.4 Dalla logica alle porte
- 2.5 Logica combinatoria su più di due livelli
- 2.6 Non solo 0 e 1, anche X e Z
- 2.7 Le mappe di Karnaugh
- 2.8 Blocchi costitutivi combinatori
- 2.9 Temporizzazioni
- 2.10 Riassunto

2.1 ■ INTRODUZIONE

Nel campo dell'elettronica digitale, un **circuito** è una rete elettrica che elabora variabili a valori discreti. Una rete può essere vista come una scatola nera (come mostra la **Figura 2.1**) che contiene:

- uno o più ingressi a valori discreti;
- una o più uscite a valori discreti;
- una specifica funzionale che descrive la relazione tra ingressi e uscite;
- una specifica di temporizzazione che descrive il ritardo tra il cambio degli ingressi e la risposta delle uscite.

Se si osserva l'interno di questa scatola nera, è possibile notare come le reti siano composte da nodi ed elementi. Un **elemento** è, a sua volta, una rete con ingressi, uscite e specifiche propri, mentre un **nodo** è un contatto elettrico la cui tensione trasmette una variabile a valore discreto. I nodi sono divisi in tre categorie: **ingressi**, **uscite** e **nodi interni**. Gli ingressi ricevono valori dal mondo esterno, mentre, viceversa, le uscite emettono valori all'esterno; infine, i contatti che non sono né ingressi né uscite vengono chiamati nodi interni. La **Figura 2.2** mostra

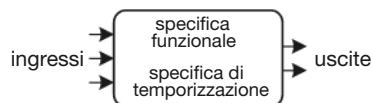


Figura 2.1 Rete logica come scatola nera con ingressi, uscite e specifiche.

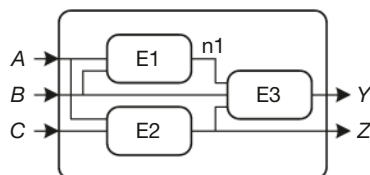


Figura 2.2 Elementi e nodi.

Software applicativo	
Sistema operativo	
Architettura	
Micro-architettura	
Blocchi logici	
Reti digitali	
Reti analogiche	
Dispositivi	
Fisica	

una rete composta da tre elementi, E1, E2 e E3, e sei nodi. I nodi A, B e C sono ingressi, Y e Z sono uscite, n1 è un nodo interno tra E1 ed E3.

Le reti digitali vengono divise in due categorie: reti **combinatorie** e reti **sequenziali**. Le uscite di una rete combinatoria dipendono esclusivamente dai valori presenti in quel momento agli ingressi; in altre parole, una rete combinatoria utilizza i valori presenti agli ingressi per calcolare i valori delle uscite (un esempio di rete combinatoria è una porta logica). Le uscite di una rete sequenziale, invece, dipendono sia dai valori presenti agli ingressi, sia dai valori precedenti; in altre parole, i valori delle uscite dipendono dalla **sequenza** dei valori degli ingressi. Una rete combinatoria è priva di **memoria**, mentre una rete sequenziale ne ha una. In questo capitolo ci si concentra sulle reti combinatorie, mentre le reti sequenziali verranno esaminate nel Capitolo 3.

La specifica funzionale di una rete combinatoria riporta i valori delle uscite, espressi in funzione dei valori presenti in quel momento agli ingressi. La specifica di temporizzazione relativa a una rete combinatoria, invece, consiste in un limite superiore e un limite inferiore sul ritardo tra ingressi e uscite. Per cominciare, si analizza la specifica funzionale, per poi riprendere la specifica di temporizzazione più avanti nel capitolo.

La **Figura 2.3** mostra una rete combinatoria con due ingressi e un'uscita. Sulla sinistra della figura sono raffigurati i due ingressi A e B, e sulla destra è mostrata l'uscita Y. L'indicazione LC, presente all'interno del riquadro, indica che la rete è realizzata utilizzando unicamente logica combinatoria. In questo esempio, la funzione F è una somma logica OR: $Y = F(A, B) = A + B$. Quindi si può dire che l'uscita Y è una funzione dei due ingressi A e B, e in particolare $Y = A \text{ OR } B$.

La **Figura 2.4** mostra due possibili **realizzazioni** relative alla rete logica combinatoria della **Figura 2.3**. Come mostrato più volte all'interno del testo, spesso esistono più realizzazioni diverse di una singola funzione. La scelta della realizzazione da utilizzare per una specifica rete dipende dai blocchi circuitali che si hanno a disposizione e dai requisiti di progetto. Spesso, tali requisiti includono area, velocità, potenza e tempo di progettazione.

La **Figura 2.5** mostra una rete combinatoria a molte uscite. Questa rete combinatoria particolare viene chiamata **sommatore** (o sommatore completo, dall'inglese *full adder*, contrapposto al **semisommatore** o *half adder*) e viene ulteriormente esaminata nel paragrafo 5.2.1. Le due espressioni specificano la funzione delle due uscite S e R_{out} espresse in funzione degli ingressi A, B e R_{in} .

Per semplificare la raffigurazione grafica delle reti, spesso viene utilizzata una singola linea con un trattino che la attraversa e un numero scritto al suo fianco per indicare un **bus** (cioè un insieme di segnali multipli). Il numero posto a fianco della barra specifica quanti segnali sono presenti nel bus. Per esempio, la **Figura 2.6(a)** rappresenta un blocco di logica combinatoria con tre ingressi e due uscite. Nel caso in cui il numero di bit non sia importante o sia facilmente deducibile dal contesto, il trattino potrebbe essere indicato anche senza numero. La **Figura 2.6(b)** mostra due blocchi di logica combinatoria con un numero arbitrario di uscite da un blocco che vengono utilizzate come ingressi per il secondo blocco.

Le regole della **composizione combinatoria** insegnano come combinare elementi di reti combinatorie di dimensioni limitate per creare una rete combinatoria più grande. Una rete è combinatoria se consiste di elementi circuitali interconnessi che presentano le seguenti caratteristiche:

- ogni elemento circuitale è di per sé combinatorio;

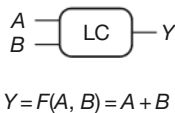


Figura 2.3
Rete logica combinatoria.

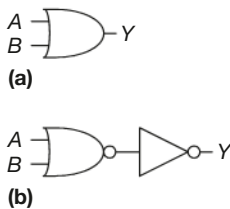


Figura 2.4
Due realizzazioni della funzione OR.

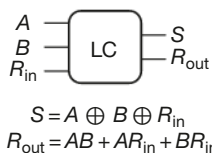


Figura 2.5
Rete combinatoria a molte uscite.

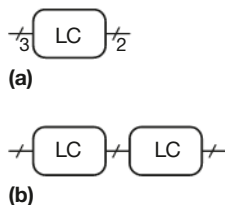


Figura 2.6
Notazione a trattino per segnali multipli.

- ogni nodo della rete è un ingresso per la rete oppure è connesso solamente a un terminale di uscita di un elemento della rete;
- la rete non contiene percorsi ciclici: ogni percorso che la attraversa passa attraverso ogni nodo al massimo una volta.

ESEMPIO 2.1

Reti combinatorie. Quale delle reti nella **Figura 2.7** è una rete combinatoria secondo le regole della composizione combinatoria?

Soluzione La rete (a) è combinatoria, perché è costruita con due circuiti combinatori (i negatori I1 e I2). Questa rete ha 3 nodi: n1, n2 e n3. n1 è un ingresso della rete e del negatore I1; n2 è un nodo interno che ha il ruolo di uscita di I1 e di ingresso di I2. n3, infine, è l'uscita della rete e di I2. La rete (b) non è combinatoria, perché ha un percorso ciclico: l'uscita dello XOR ritorna in uno dei suoi ingressi, quindi si crea un percorso ciclico che inizia al nodo n4, passa attraverso lo XOR fino a n5, che a sua volta torna a n4. La rete (c) è combinatoria, mentre la rete (d) non lo è perché il nodo n6 è connesso a entrambi i nodi di uscita I3 e I4. La rete (e) è combinatoria, perché costituita da due reti combinatorie connesse tra loro per formare una rete combinatoria più grande. Infine, la rete (f) non obbedisce alle regole della composizione combinatoria dato che possiede dei percorsi ciclici tra i due elementi. A seconda delle funzioni degli elementi, potrebbe essere una rete combinatoria oppure no.

Reti logiche di grandi dimensioni (come i microprocessori) possono essere molto complesse, ragion per cui verranno utilizzati i principi del Capitolo 1 per gestirne la complessità. Visualizzare una rete come una scatola nera con un'interfaccia e una funzione ben definite è un'applicazione del principio dell'astrazione e della modularità, così come la costruzione di una rete a partire da elementi circuitali più piccoli è un'applicazione del principio della gerarchia. Le regole della composizione combinatoria sono, infine, un'applicazione della regolarità.

La specifica funzionale di una rete combinatoria è solitamente espressa come una tabella delle verità o come un'espressione booleana. Nel prossimo paragrafo viene spiegato come derivare un'espressione booleana da una qualsiasi tabella delle verità e come utilizzare l'algebra booleana e le mappe di Karnaugh per semplificare le espressioni. Viene inoltre mostrato come costruire queste espressioni utilizzando le porte logiche e come analizzare la velocità dei circuiti risultanti.

2.2 ■ ESPRESSIONI BOOLEANE

Le espressioni booleane si basano su variabili che possono assumere i due valori VERO o FALSO, il che le rende perfette per descrivere la logica digitale. Questo paragrafo definisce alcuni termini utilizzati di frequente nelle espressioni

Le regole di composizione combinatoria sono sufficienti ma non necessarie. Alcune reti che non rispettano queste regole sono comunque combinatorie, se le loro uscite dipendono soltanto dai valori attuali agli ingressi. Tuttavia, stabilire se queste strane reti sono combinatorie o no è più difficile, per cui nel testo ci si limita alle regole di composizione combinatoria per la costruzione delle reti combinatorie.

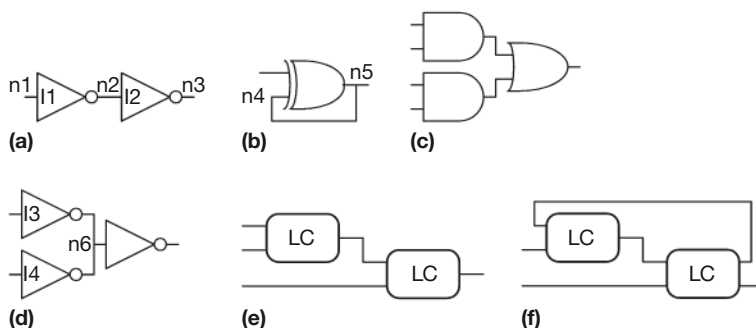


Figura 2.7
Esempi di reti.

sioni booleane, per poi mostrare come scrivere l'espressione booleana per una qualsiasi funzione logica, a partire dalla sua tabella delle verità.

2.2.1 Terminologia

Il **complemento** di una variabile A è il suo negato (o inverso) $\bar{A}$. La variabile, o il suo complemento, è denominata **letterale**: per esempio, A , $\bar{A}$, B e $\bar{B}$ sono dei letterali. A viene detta forma **diritta** della variabile, mentre $\bar{A}$ è la forma **negata** (complementare). L'AND di uno o più letterali è definito **prodotto logico** o **implicante**: $\bar{A}B$, $\bar{A}\bar{B}\bar{C}$, e B sono tutti implicanti di una funzione di almeno tre variabili. Un **mintermine** è il prodotto di tutti gli ingressi di una funzione, quindi $\bar{A}\bar{B}\bar{C}$ è un mintermine di una funzione delle tre variabili A , B e C , mentre $\bar{A}B$ non lo è perché non include C . Analogamente, l'OR di uno o più letterali è definito **somma logica** o **implicato**. Un **maxtermine**, invece, è la somma di tutti gli ingressi di una funzione, quindi $A + \bar{B} + C$ è un maxtermine di una funzione delle tre variabili A , B e C .

L'**ordine delle operazioni** è importante per interpretare correttamente le espressioni booleane: infatti $Y = A + BC$ potrebbe essere interpretato come $Y = (A \text{ OR } B) \text{ AND } C$ oppure come $Y = A \text{ OR } (B \text{ AND } C)$. Nelle espressioni booleane, l'operatore NOT ha la massima **precedenza**, seguito da AND e da OR; in altre parole, come nelle solite espressioni algebriche, prima si fanno i prodotti e poi le somme. Quindi l'espressione viene interpretata come $Y = A \text{ OR } (B \text{ AND } C)$. L'Espressione 2.1 è un altro esempio di ordine delle operazioni.

$$\bar{A}B + BC\bar{D} = ((\bar{A})B) + (BC(\bar{D})) \quad (2.1)$$

2.2.2 Forma somma di prodotti

Una tabella delle verità di un numero N di ingressi contiene 2^N righe, una per ognuno dei possibili valori degli ingressi. Ogni riga di una tabella delle verità è associata a un mintermine che è VERO per quella riga. La **Figura 2.8** mostra una tabella delle verità di due ingressi, A e B : ogni riga mostra il mintermine corrispondente. Per esempio, il mintermine associato alla prima riga è $\bar{A}\bar{B}$ perché $\bar{A}\bar{B}$ è vero quando $A = 0$ e $B = 0$. I mintermini vengono numerati a partire da 0; la riga più alta corrisponde al mintermine 0, m_0 , la riga successiva al mintermine 1, m_1 , e così via.

È possibile scrivere un'espressione booleana a partire da qualsiasi tabella delle verità tramite la somma di tutti i mintermini in corrispondenza dei quali l'uscita, Y , vale VERO. Per esempio, nella **Figura 2.8** c'è solo una riga (cerchiata in rosso nella figura) e quindi solo un mintermine, per cui l'uscita Y vale VERO. Di conseguenza, $Y = \bar{A}\bar{B}$. La **Figura 2.9** mostra una tabella delle verità con più di una riga nella quale l'uscita è VERO. Eseguendo la somma di tutti i mintermini cerchiati si ottiene $Y = \bar{A}\bar{B} + AB$. Questa espressione è chiamata **forma canonica somma di prodotti** di una funzione, perché è la somma (OR) dei prodotti (cioè degli AND che formano i mintermini). Anche se esistono diversi modi per scrivere la stessa espressione, come per esempio $Y = B\bar{A} + BA$, in questo testo i letterali presenti nei mintermini vengono scritti nello stesso ordine con cui appaiono nella tabella delle verità, così che ogni tabella delle verità abbia una sola espressione booleana.

La forma canonica della somma di prodotti può anche essere scritta in **notazione sigma**, utilizzando il simbolo di sommatoria, Σ . Con questa notazione, la funzione della Figura 2.9 viene indicata come:

$$F(A, B) = \Sigma(m_1, m_3) \quad (2.2)$$

oppure come

$$F(A, B) = \Sigma(1, 3)$$

A	B	Y	mintermine	nome del mintermine
0	0	0	$\bar{A}\bar{B}$	m_0
0	1	1	$\bar{A}B$	m_1
1	0	0	$A\bar{B}$	m_2
1	1	0	AB	m_3

Figura 2.8
Tabella delle verità e mintermini.

A	B	Y	mintermine	nome del mintermine
0	0	0	$\bar{A}\bar{B}$	m_0
0	1	1	$\bar{A}B$	m_1
1	0	0	$A\bar{B}$	m_2
1	1	1	AB	m_3

Figura 2.9
Tabella delle verità con più mintermini associati a uscita VERA.

Forma canonica è il modo rigoroso per indicare la forma standard. Si può usare per sottolineare la propria competenza agli interlocutori.

ESEMPIO 2.2

Forma somma di prodotti. Ben Imbrogliabit sta facendo un picnic e non se lo gusterà di certo se piove oppure se ci sono le formiche. Progettare una rete logica che produca un'uscita VERA solo se Ben si gusta il picnic.

Soluzione Per prima cosa serve definire gli ingressi e le uscite. Gli ingressi sono F e P , che indicano rispettivamente le Formiche e la Pioggia. F è VERO quando ci sono le Formiche e FALSO quando non ci sono; analogamente, P è VERO quando piove e FALSO quando c'è il sole. L'uscita è G , e indica se Ben si Gusta il picnic oppure no. G è VERO se Ben si diverte, e FALSO se Ben non si diverte. La **Figura 2.10** mostra la tabella delle verità dell'esperienza di Ben.

Utilizzando la forma somma di prodotti, l'espressione diventa: $G = \overline{F}\overline{P}$ oppure $G = \Sigma(0)$. È possibile realizzare l'espressione utilizzando due negatori e una porta AND a due ingressi, come mostra la **Figura 2.11(a)**. È facile riconoscere che questa tabella delle verità è uguale a quella, presente nel paragrafo 1.5.5, di una funzione NOR: $G = F \text{ NOR } P = \overline{F} + \overline{P}$. La **Figura 2.11(b)** mostra la realizzazione con una porta NOR. Nel paragrafo 2.3 viene mostrato come le due espressioni $\overline{F}\overline{P}$ e $\overline{F} + \overline{P}$ siano equivalenti.

La forma somma di prodotti consente di scrivere un'espressione booleana per qualsiasi tabella delle verità con un numero qualsiasi di variabili. La **Figura 2.12** mostra una generica tabella delle verità a tre ingressi. In questo caso, la forma somma di prodotti della funzione logica è uguale a:

$$Y = \overline{A}\overline{B}\overline{C} + \overline{A}B\overline{C} + \overline{A}BC$$

oppure a

$$Y = \Sigma(0, 4, 5)$$

Sfortunatamente, la somma di prodotti non genera necessariamente l'espressione più semplice. Nel paragrafo 2.3 si mostra come scrivere la medesima funzione utilizzando un numero minore di elementi.

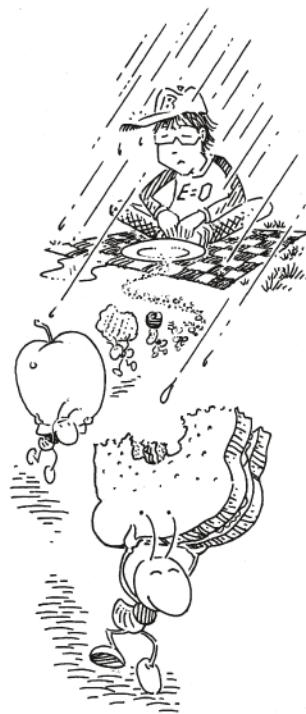
2.2.3 Forma prodotto di somme

Un modo alternativo per esprimere funzioni booleane è la **forma canonica prodotto di somme**. Ogni riga di una tabella delle verità corrisponde a un maxtermine che è FALSO per quella riga. Per esempio, il maxtermine per la prima riga di una tabella delle verità a due ingressi è $(A + B)$ perché $(A + B)$ è FALSO quando $A = 0$ e $B = 0$. È possibile scrivere un'espressione booleana a partire da qualsiasi tabella delle verità tramite il prodotto di tutti i maxtermini in corrispondenza dei quali l'uscita, Y , vale FALSO. La forma canonica prodotto di somme può anche essere scritta in **notazione pi greco**, utilizzando il simbolo di produttoria, Π .

ESEMPIO 2.3

Forma prodotto di somme. Scrivere un'espressione in forma prodotto di somme per la tabella delle verità della **Figura 2.13**.

Soluzione La tabella delle verità ha due righe in cui l'uscita è FALSA, il che significa che la funzione può essere scritta in forma prodotto di somme come $Y = (A + B)(\overline{A} + B)$ o, utilizzando la notazione pi greco, $Y = \Pi(M_0, M_2)$, oppure ancora come $Y = \Pi(0, 2)$. Il primo maxtermine, $(\overline{A} + B)$, garantisce che sia $Y = 0$ quando $A = 0$ e $B = 0$, perché ogni valore AND 0 è uguale a 0. Allo stesso modo il secondo maxtermine, $(A + B)$, garantisce che sia $Y = 0$ quando $A = 1$ e $B = 0$. La tabella delle verità della **Figura 2.13** è la stessa di quella della **Figura 2.9**, e mostra quindi come sia possibile scrivere la stessa funzione in più modi.



F	P	G
0	0	1
0	1	0
1	0	0
1	1	0

Figura 2.10
Tabella delle verità di Ben.

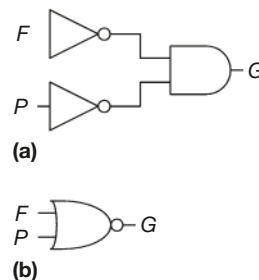


Figura 2.11
Rete logica di Ben.

A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

Figura 2.12
Generica tabella delle verità a tre ingressi.

A	B	Y	maxtermini	nome del maxtermine
0	0	0	$A + B$	M_0
0	1	1	$A + \overline{B}$	M_1
1	0	0	$\overline{A} + B$	M_2
1	1	1	$\overline{A} + \overline{B}$	M_3

Figura 2.13
Tabella delle verità con più maxtermini associati a uscita FALSA.

Come per l'Esempio 2.3, un'espressione booleana che descriva l'esperienza di Ben al picnic della Figura 2.10 può essere scritta in forma prodotto di somme, cerchiando le tre righe con uscita 0 per ottenere $G = (F + \overline{P})(\overline{F} + P)(\overline{F} + \overline{P})$ oppure $G = \Pi(1, 2, 3)$. Questa espressione è meno elegante rispetto all'espressione somma di prodotti, $G = \overline{F}\overline{P}$, tuttavia le due espressioni sono a livello logico equivalenti.

La somma di prodotti produce un'espressione più corta quando l'uscita è VERA solo su poche righe della tabella delle verità; analogamente il prodotto di somme è più semplice quando l'uscita è FALSA solo su poche righe della tabella della verità.

2.3 ■ ALGEBRA BOOLEANA

Nel paragrafo precedente è stato spiegato come ricavare un'espressione booleana a partire da una tabella delle verità. Tuttavia, non necessariamente quest'espressione corrisponde all'insieme minimo di porte logiche necessario per realizzare la funzione considerata. Proprio come si utilizza l'algebra per semplificare le espressioni matematiche, è possibile utilizzare l'**algebra booleana** per semplificare le espressioni booleane. Le regole dell'algebra booleana sono simili a quelle dell'algebra ordinaria, in alcuni casi addirittura più semplici, dal momento che le variabili hanno unicamente due possibili valori: 0 o 1.

L'algebra booleana si basa su un insieme di postulati che come tali vengono per definizione considerati corretti. I postulati non sono dimostrabili, nel senso che un assunto di partenza non può essere dimostrato. A partire da questi postulati è possibile invece dimostrare tutti i teoremi dell'algebra booleana.

Questi teoremi hanno un significato pratico enorme, perché consentono di semplificare le espressioni logiche per produrre reti più piccole e meno costose.

I postulati e i teoremi dell'algebra booleana obbediscono al **principio di dualità**: se i simboli 0 e 1 e gli operatori $\bullet$ (AND) e $+$ (OR) sono scambiati tra loro, l'affermazione rimane corretta. L'apostrofo (') viene usato nel testo per indicare la **forma duale** di un'affermazione.

2.3.1 Postulati

La **Tabella 2.1** riporta i postulati dell'algebra booleana. Questi cinque postulati e i loro duali definiscono le variabili booleane e i significati di NOT, AND e OR. Il postulato A1 afferma che una variabile booleana B è 0 se non è 1. La forma duale di questo postulato afferma che la variabile è 1 se non ha valore 0. Insieme, i postulati A1 e A1' indicano che si sta lavorando in campo booleano (o binario) di zeri e uni. I postulati A2 e A2' definiscono l'operazione NOT. I postulati da A3 a A5 definiscono l'operazione AND, mentre i loro duali (da A3' a A5') definiscono l'operazione OR.

Tabella 2.1 Postulati dell'algebra booleana.

Postulato		Forma duale		Nome
A1	$B = 0$ se $B \neq 1$	A1'	$B = 1$ se $B \neq 0$	Algebra binaria
A2	$\overline{0} = 1$	A2'	$\overline{1} = 0$	NOT
A3	$0 \bullet 0 = 0$	A3'	$1 + 1 = 1$	AND/OR
A4	$1 \bullet 1 = 1$	A4'	$0 + 0 = 0$	AND/OR
A5	$0 \bullet 1 = 1 \bullet 0 = 0$	A5'	$1 + 0 = 0 + 1 = 1$	AND/OR

2.3.2 Teoremi a una variabile

I teoremi da T1 a T5 nella **Tabella 2.2** descrivono come semplificare espressioni a una variabile.

Il **teorema dell'identità**, T1, afferma che per ogni variabile booleana B , $B \text{ AND } 1 = B$. La sua forma duale T1' afferma che $B \text{ OR } 0 = B$. A livello hardware, come mostra la **Figura 2.14**, T1 significa che se un ingresso di una porta AND a due ingressi è sempre 1, è possibile rimuovere la porta AND e sostituirla con un filo connesso alla variabile d'ingresso B . Allo stesso modo, T1' significa che se un ingresso di una porta OR a due ingressi è sempre 0, è possibile sostituire la porta con un filo connesso a B . In generale, le porte hanno un costo non solo economico, ma anche energetico e a livello di ritardo, quindi la sostituzione di una porta con un filo è molto vantaggiosa.

Il **teorema dell'elemento nullo**, T2, afferma che $B \text{ AND } 0$ è sempre uguale a 0: per questo motivo 0 è chiamato l'elemento nullo per un'operazione AND, perché annulla l'effetto di qualsiasi altro ingresso. La forma duale del teorema afferma che $B \text{ OR } 1$ è sempre uguale a 1 e quindi 1 è l'elemento nullo per l'operazione OR.

A livello hardware, come mostrato nella **Figura 2.15**, se l'ingresso di una porta AND è 0, è possibile sostituire la porta AND con un filo che viene tenuto BASSO (a 0). Allo stesso modo, se un ingresso di una porta OR è 1, è possibile sostituire la porta con un filo tenuto ALTO (a 1).

Il **teorema dell'idempotenza**, T3, afferma che una variabile AND sé stessa è uguale solo a sé stessa, e così anche una variabile OR sé stessa. Il nome del teorema deriva dalla sua radice latina: *idem* (stesso) e *potent* (potenza). Secondo questo teorema, le suddette operazioni danno come risultato il dato iniziale. La **Figura 2.16** mostra che l'idempotenza permette ancora una volta di sostituire una porta con un filo.

Il **teorema dell'involuzione**, T4, è un modo curioso di dire che negare due volte una variabile equivale al valore originario della variabile. Quindi due negatori in serie si cancellano, e possono essere sostituiti da un filo, come mostra la **Figura 2.17**. Il teorema in forma duale di T4 è sé stesso.

Il **teorema dei complementi**, T5 (**Figura 2.18**), afferma che in una variabile AND il suo complemento è 0 (perché uno dei due è necessariamente 0). Grazie al principio di dualità è possibile affermare anche che una variabile OR il suo complemento è uguale a 1 (dal momento che, ancora una volta, uno dei due è necessariamente un 1).

2.3.3 Teoremi di più variabili

I teoremi da T6 a T12 nella **Tabella 2.3** descrivono come semplificare espressioni che hanno a che fare con più di una variabile booleana.

I **teoremi della commutatività** e **dell'associatività**, T6 e T7, funzionano come nell'algebra tradizionale: per proprietà commutativa, l'ordine degli ingressi per una funzione AND o OR non ha effetto sul valore dell'uscita. Come

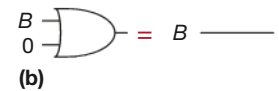
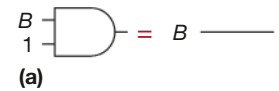


Figura 2.14
Realizzazione circuitale del teorema dell'identità: (a) T1, (b) T1'.

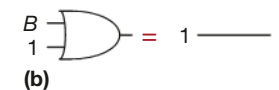
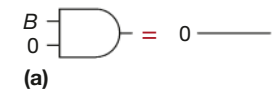


Figura 2.15
Realizzazione circuitale del teorema dell'elemento nullo: (a) T2, (b) T2'.

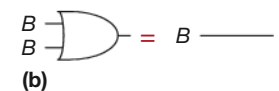
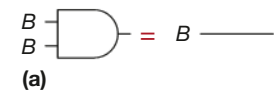


Figura 2.16
Realizzazione circuitale del teorema dell'idempotenza: (a) T3, (b) T3'.

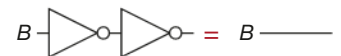


Figura 2.17
Realizzazione circuitale del teorema dell'involuzione: T4.

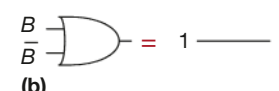
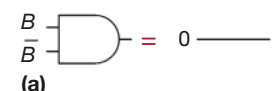


Figura 2.18
Realizzazione circuitale del teorema dei complementi: (a) T5, (b) T5'.

Tabella 2.2 Teoremi dell'algebra booleana a una variabile.

	Postulato		Forma duale	Nome
T1	$B \cdot 1 = B$	T1'	$B + 0 = B$	Identità
T2	$B \cdot 0 = 0$	T2'	$B + 1 = 1$	Nullo
T3	$B \cdot B = B$	T3'	$B + B = B$	Idempotenza
T4		$\bar{\bar{B}} = B$		Involuzione
T5	$B \cdot \bar{B} = 0$	T5'	$B + \bar{B} = 1$	Complementi

Tabella 2.3 Teoremi dell'algebra booleana a più variabili.

	Postulato		Forma duale		Nome
T6	$B \cdot C = C \cdot B$	T6'	$B + C = C + B$		Commutatività
T7	$(B \cdot C) \cdot D = B \cdot (C \cdot D)$	T7'	$(B + C) + D = B + (C + D)$		Associatività
T8	$(B \cdot C) + (B \cdot D) = B \cdot (C + D)$	T8'	$(B + C) \cdot (B + D) = B + (C \cdot D)$		Distributività
T9	$B \cdot (B + C) = B$	T9'	$B + (B \cdot C) = B$		Assorbimento
T10	$(B \cdot C) + (B \cdot \bar{C}) = B$	T10'	$(B + C) \cdot (B + \bar{C}) = B$		Combinazione
T11	$(B \cdot C) + (\bar{B} \cdot D) + (C \cdot D) = B \cdot C + \bar{B} \cdot D$	T11'	$(B + C) \cdot (\bar{B} + D) \cdot (C + D) = (B + C) \cdot (\bar{B} + D)$		Consenso
T12	$\overline{B_0 \cdot B_1 \cdot B_2 \dots} = (\bar{B}_0 + \bar{B}_1 + \bar{B}_2 \dots)$	T12'	$\overline{B_0 + B_1 + B_2 \dots} = (\bar{B}_0 \cdot \bar{B}_1 \cdot \bar{B}_2 \dots)$		Teorema di De Morgan



Augustus De Morgan, morto nel 1871. Matematico inglese nato in India. Cieco da un occhio. Il padre morì quando aveva 10 anni. A 16 anni ha frequentato il Trinity College di Cambridge, e a 22 anni è stato nominato Professore di Matematica alla neonata London University. Ha scritto molto su vari argomenti della matematica, inclusi logica, algebra e paradossi. Il cratere lunare De Morgan è stato così denominato in suo onore. Ha anche proposto un indovinello per la data della sua nascita: "avevo x anni di età nell'anno x^2 ."

in algebra, per proprietà associativa, come vengono raggruppati gli ingressi non ha effetto sul valore finale dell'uscita.

Il **teorema della distributività**, T8, è lo stesso che in algebra tradizionale, ma la sua forma duale non lo è. Secondo il teorema T8, AND distribuisce su OR, e secondo T8', OR distribuisce su AND. In algebra tradizionale esiste la proprietà distributiva della moltiplicazione sull'addizione, ma non viceversa e quindi $(B + C) \times (B + D) \neq B + (C \times D)$.

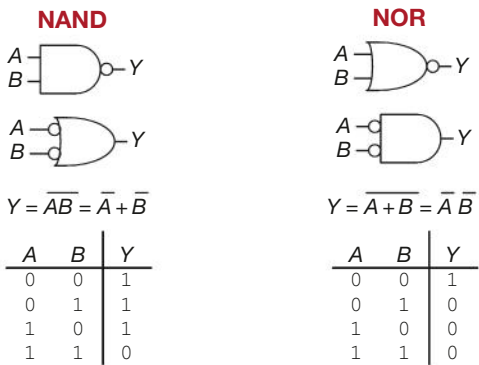
I **teoremi dell'assorbimento, della combinazione e del consenso** (da T9 a T11) permettono di eliminare variabili ridondanti.

Il **teorema di De Morgan**, T12, è uno strumento particolarmente utile per la progettazione digitale. Questo teorema spiega che il complemento del prodotto di tutti i termini di un'espressione è uguale alla somma dei complementi di ogni singolo termine.

Secondo il teorema di De Morgan, una porta NAND è equivalente a una porta OR con gli ingressi negati. Allo stesso modo, una porta NOR è uguale a una porta AND con gli ingressi negati. La **Figura 2.19** mostra le **porte equivalenti di De Morgan** per le porte NAND e NOR. I due simboli presenti in ogni figura vengono chiamati **duali**: essi sono equivalenti a livello logico e quindi intercambiabili.

Il circoletto di negazione viene chiamato **bolla**. Si può dire che "spingere una bolla" attraverso una porta fa sì che la bolla fuoriesca dall'altro lato e trasformi il corpo della porta da AND a OR, o viceversa. Per esempio, la porta NAND riportata nella Figura 2.19 è formata da una porta AND con una bolla presente sull'uscita. Se si spinge questa bolla a sinistra attraverso la porta si ottiene una porta OR con due bolle su entrambi gli ingressi. Le regole alla base dello spostamento di una bolla sono:

Figura 2.19
Porte equivalenti secondo De Morgan.



- spingere una bolla indietro (dall'uscita all'ingresso) o in avanti (dall'ingresso all'uscita) trasforma una porta AND in una porta OR e viceversa;
- se si spinge una bolla dall'uscita verso gli ingressi, questa viene trasferita a ognuno degli ingressi;
- se si spingono tutte le bolle degli ingressi di una porta verso la sua uscita, quest'ultima avrà una sola bolla.

Nel paragrafo 2.5.2 verranno utilizzate le bolle per aiutare nell'analisi delle reti.

ESEMPIO 2.4

Ricavare la forma prodotto di somme. La **Figura 2.20** mostra la tabella delle verità di una funzione booleana Y e del suo negato $\bar{Y}$. Utilizzando il teorema di De Morgan, ricavare la forma canonica prodotto di somme di Y dalla forma somma di prodotti di $\bar{Y}$.

Soluzione La **Figura 2.21** mostra i mintermini (cerchiati) contenuti nella funzione $\bar{Y}$. La forma canonica somma di prodotti di $\bar{Y}$ è uguale a:

$$\bar{Y} = \bar{A}\bar{B} + \bar{A}B \quad (2.4)$$

Negando entrambi i termini dell'uguaglianza e applicando due volte il teorema di De Morgan si ottiene:

$$\bar{\bar{Y}} = Y = \overline{\bar{A}\bar{B} + \bar{A}B} = (\overline{\bar{A}\bar{B}})(\overline{\bar{A}B}) = (A + B)(A + \bar{B}) \quad (2.5)$$

A	B	Y	$\bar{Y}$
0	0	0	1
0	1	0	1
1	0	1	0
1	1	1	0

Figura 2.20
Tabella delle verità con le uscite Y e $\bar{Y}$.

A	B	Y	$\bar{Y}$	mintermine
0	0	0	1	$\bar{A}\bar{B}$
0	1	0	1	$\bar{A}B$
1	0	1	0	$A\bar{B}$
1	1	1	0	AB

Figura 2.21
Tabella delle verità con i mintermini relativi a $\bar{Y}$.

2.3.4 Saranno veri i teoremi booleani?

Un lettore particolarmente curioso potrebbe domandarsi come sia possibile provare la validità di un teorema. Nell'algebra booleana è molto semplice dimostrare un teorema con un numero finito di variabili: è sufficiente verificare che il teorema sia valido per tutti i valori possibili di queste variabili. Questo metodo viene chiamato **induzione matematica perfetta** e può essere eseguito con una tabella delle verità.

ESEMPIO 2.5

Dimostrazione del teorema del consenso mediante induzione matematica perfetta. Dimostrare la validità del teorema del consenso, T11, riportato nella **Tabella 2.3**.

Soluzione Basta verificare entrambi i termini del teorema per tutte le otto possibili combinazioni di B , C e D . La tabella delle verità della **Figura 2.22** mostra tutte queste combinazioni. Dal momento che $BC + \bar{B}D + CD = BC + \bar{B}D$ per tutte le combinazioni, il teorema è dimostrato.

2.3.5 Semplificare le espressioni

I teoremi dell'algebra booleana sono utili per semplificare le espressioni booleane. Per esempio, si consideri l'espressione in forma somma di prodotti della tabella delle verità della **Figura 2.9**: $Y = \bar{A}B + AB$. Grazie al teorema T10,

B	C	D	$BC + \bar{B}D + CD$	$BC + \bar{B}D$
0	0	0	0	0
0	0	1	1	1
0	1	0	0	0
0	1	1	1	1
1	0	0	0	0
1	0	1	0	0
1	1	0	1	1
1	1	1	1	1

Figura 2.22
Tabella delle verità per la dimostrazione del teorema T11.

l'espressione può essere semplificata come $Y = B$. In questo caso, la semplificazione poteva essere facilmente dedotta guardando la tabella delle verità. Tuttavia, in generale sono necessari diversi passaggi per semplificare espressioni più complesse.

Il principio base per semplificare equazioni in forma somma di prodotti è combinare i termini utilizzando la relazione $PA + P\overline{A} = P$, dove P rappresenta un implicante qualsiasi. Quanto si può semplificare un'espressione? Si definisce un'espressione in somma di prodotti come **minima** se utilizza il minor numero possibile di implicanti. Se si confrontano espressioni con lo stesso numeri di implicanti, l'espressione minima è quella che usa il minor numero possibile di letterali.

Un implicante è detto **implicante primo** se non può essere combinato con nessun altro elemento all'interno dell'espressione per formare un nuovo implicante con un numero minore di letterali. In un'espressione minima, gli implicanti devono essere tutti implicanti primi, altrimenti è possibile combinarli ulteriormente per diminuire il numero di letterali.

ESEMPIO 2.6

Minimizzare un'espressione. Minimizzare l'Espressione 2.3: $Y = \overline{A}\overline{B}\overline{C} + A\overline{B}\overline{C} + A\overline{B}C$.

Soluzione Si parte dall'espressione iniziale e si applicano i teoremi booleani passo per passo, come mostra la **Tabella 2.4**.

Per verificare se a questo punto l'espressione è completamente semplificata, si può fare un esame più accurato. Partendo dall'espressione originale, si nota che i mintermini $\overline{A}\overline{B}\overline{C}$ e $A\overline{B}\overline{C}$ differiscono tra loro solo per la variabile A . È quindi possibile unire i mintermini per formare $\overline{B}\overline{C}$. Tuttavia, sempre tornando all'espressione iniziale, si nota che anche gli ultimi due mintermini $A\overline{B}\overline{C}$ e $A\overline{B}C$ sono diversi solo per un letterale (C e $\overline{C}$): utilizzando lo stesso metodo, sarebbe stato possibile combinare questi due mintermini per formare $A\overline{B}$. In questo caso si dice che gli implicanti $\overline{B}C$ e $A\overline{B}$ condividono il mintermine $A\overline{B}\overline{C}$.

L'unica possibilità per semplificare l'espressione è quindi semplificare solo una delle due coppie di mintermini, oppure è possibile semplificarle entrambe? Utilizzando il teorema dell'idempotenza è possibile duplicare i termini all'infinito: $B = B + B + B + B \dots$. Utilizzando questo teorema, l'espressione viene ulteriormente semplificata nei suoi due implicanti primi, $\overline{B}\overline{C} + A\overline{B}$, come mostra la **Tabella 2.5**.

Tabella 2.4 Semplificazione di un'espressione logica.

Passo	Espressione	Teorema applicato
	$\overline{A}\overline{B}\overline{C} + A\overline{B}\overline{C} + A\overline{B}C$	
1	$\overline{B}\overline{C}(\overline{A} + A) + A\overline{B}C$	T8: Distributività
2	$\overline{B}\overline{C}(1) + A\overline{B}C$	T5: Complementi
3	$\overline{B}\overline{C} + A\overline{B}C$	T1: Identità

Tabella 2.5 Migliore semplificazione di un'espressione logica.

Passo	Espressione	Teorema applicato
	$\overline{A}\overline{B}\overline{C} + A\overline{B}\overline{C} + A\overline{B}C$	
1	$\overline{A}\overline{B}\overline{C} + A\overline{B}\overline{C} + A\overline{B}\overline{C} + A\overline{B}C$	T3: Idempotenza
2	$\overline{B}\overline{C}(\overline{A} + A) + A\overline{B}(\overline{C} + C)$	T8: Distributività
3	$\overline{B}\overline{C}(1) + A\overline{B}(1)$	T5: Complementi
4	$\overline{B}\overline{C} + A\overline{B}$	T1: Identità

Anche se può sembrare controproducente, espandere un implicante (per es. trasformare AB in $ABC + AB\bar{C}$) a volte è un'operazione molto utile per minimizzare un'espressione. Così facendo è possibile ripetere uno dei mintermini espansi per combinarlo (condividerlo) con un altro mintermine.

Quanto detto fin qui mostra che semplificare completamente un'espressione booleana con l'utilizzo dei teoremi dell'algebra booleana è un processo complesso, che può causare diversi errori. A questo proposito, il paragrafo 2.7 descrive una tecnica metodica, basata sulle cosiddette mappe di Karnaugh, che facilita il processo di semplificazione delle espressioni.

Qual è lo scopo della semplificazione di un'espressione booleana se il risultato è equivalente, a livello logico, all'espressione di partenza? La semplificazione è importante perché riduce il numero di porte necessarie per eseguire a livello circuitale una funzione, rendendola più piccola, meno costosa e probabilmente anche più veloce. Nel prossimo paragrafo viene spiegato come realizzare le espressioni booleane con le porte logiche.

2.4 ■ DALLA LOGICA ALLE PORTE

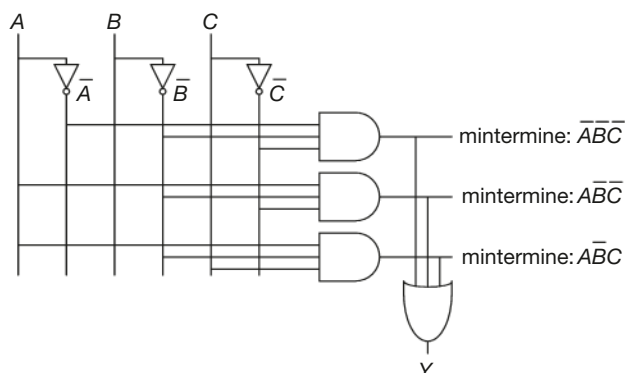
Uno schema circuitale è un diagramma di una rete digitale che ne mostra gli elementi e i fili che li connettono tra loro. Per esempio, lo schema della **Figura 2.23** mostra una possibile realizzazione circuitale della funzione logica, già utilizzata nell'Espressione 2.3:

$$Y = \bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}C + \bar{A}B\bar{C}$$

Se gli schemi vengono disegnati in maniera chiara e coerente, sono più facili da leggere ed è più facile eseguire il collaudo (*debug*) del circuito. Di seguito vengono elencate le linee guida generali da utilizzare quando si disegna uno schema.

- Gli ingressi vengono indicati a sinistra (o in alto) dello schema.
- Le uscite vengono indicate a destra (o in basso) dello schema.
- Le porte logiche, quando possibile, sono disegnate in modo che i segnali vadano da sinistra a destra.
- I fili dritti sono preferibili ai fili con troppi angoli: fili a zigzag richiedono uno sforzo mentale maggiore per seguirne il percorso, invece di concentrare l'attenzione a ciò che fa la rete.
- Fili che arrivano a una giunzione a T sono collegati tra loro.
- Un punto disegnato dove due fili si incrociano indica che quei fili sono collegati tra loro.
- Fili che si incrociano, ma che non presentano un punto disegnato all'incrocio, non sono collegati tra loro.

La **Figura 2.24** illustra le ultime tre linee guida da seguire.



I materiali di laboratorio che accompagnano questo testo (vedi la Prefazione) mostrano come usare strumenti CAD (*Computer Aided Design*, progettazione assistita da calcolatore) per progettare, simulare e collaudare reti logiche.

Figura 2.23
Rete logica per l'espressione
 $Y = \bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}C + \bar{A}B\bar{C}$.



Figura 2.24
Collegamenti dei fili.

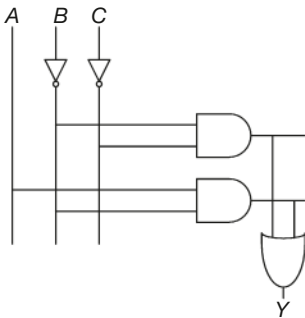


Figura 2.25
Rete logica per l'espressione
 $Y = \overline{B}C + AB$.

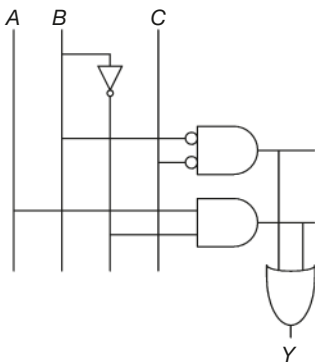


Figura 2.26
Rete logica che utilizza meno porte.

Una qualsiasi espressione booleana in forma somma di prodotti può essere tradotta in schema circuitale in maniera sistematica, come mostra la Figura 2.23. Per prima cosa, disegnare le colonne per gli ingressi e posizionare i negatori in colonne adiacenti per fornire gli ingressi negati, se necessari. Successivamente, disegnare righe di porte AND per ognuno dei mintermini; infine, per ogni uscita della rete, disegnare una porta OR connessa ai mintermini relativi a quell'uscita. Questo stile di disegno è chiamato **matrice logica programmabile** (PLA, *Programmable Logic Array*) perché i negatori, le porte AND e le porte OR sono allineati in maniera sistematica. La PLA è discussa nel paragrafo 5.6.

La **Figura 2.25** mostra la realizzazione dell'espressione semplificata, ricavata grazie all'algebra booleana nell'Esempio 2.6. Si noti che la rete minimizzata richiede molto meno hardware rispetto alla rete rappresentata nella Figura 2.23, e che è quasi certamente più veloce poiché utilizza meno porte con un numero minore di ingressi.

È possibile ridurre ulteriormente il numero di porte (sebbene di un singolo negatore) sfruttando i negatori (anche detti porte invertenti). Si noti che $\overline{B}C$ è una porta AND con gli ingressi negati. La **Figura 2.26** mostra uno schema che utilizza questa ottimizzazione per eliminare il negatore sull'ingresso C. Si deve inoltre ricordare che, per il teorema di De Morgan, una porta AND con gli ingressi negati è equivalente a una porta NOR. A seconda della tecnologia di realizzazione, potrebbe risultare meno costoso l'utilizzo del minor numero di porte o l'uso di alcuni tipi di porte piuttosto che altre. Per esempio, nelle realizzazioni CMOS sono preferibili le porte NAND e NOR alle porte AND e OR.

Molte reti hanno diverse uscite, ognuna delle quali esegue una diversa funzione booleana degli ingressi. È possibile scrivere una tabella delle verità per ognuna delle uscite, ma spesso è più conveniente riportare tutte le uscite su una singola tabella delle verità e disegnare uno schema circuitale che le riporti tutte.

ESEMPIO 2.7

Circuiti a uscite multiple. Il preside di facoltà, il direttore del dipartimento, l'assistente e il responsabile del pensionato studentesco utilizzano di tanto in tanto l'aula magna dell'università. Qualche volta, purtroppo, gli impegni si sovrappongono creando disagi, come la volta in cui l'evento di beneficenza organizzato dal preside di facoltà era in concomitanza con la festa organizzata dal pensionato studentesco. Alyssa Guastacomputer è stata contattata per progettare un sistema di prenotazione dell'aula magna.

Il sistema ha quattro ingressi ($A_3, \dots, A_0$) e quattro uscite ($Y_3, \dots, Y_0$). Questi segnali possono anche essere scritti come $A_{3:0}$ e $Y_{3:0}$. Ogni utente attiva il suo ingresso quando richiede l'aula magna per il giorno successivo e il sistema attiva al massimo un'uscita, prenotando l'aula magna per l'utente con priorità più alta. Il preside di facoltà, che paga i costi del sistema, richiede di avere la priorità più alta (3). Il direttore del dipartimento, l'assistente e il responsabile del pensionato studentesco hanno priorità decrescente dopo il preside.

Scrivere una tabella delle verità e un'espressione booleana per descrivere il sistema. Fare lo schema della rete che esegue questa funzione.

Soluzione Questa funzione viene chiamata rete logica di priorità a quattro ingressi e la sua tabella delle verità e il suo simbolo sono riportati nella **Figura 2.27**.

Si potrebbe scrivere ogni uscita in forma somma di prodotti per poi ridurre l'espressione utilizzando l'algebra booleana. Tuttavia, le espressioni semplificate si derivano chiaramente considerando la descrizione funzionale (e la tabella delle verità): l'uscita Y_3 è VERA ogniqualvolta A_3 vale 1, quindi $Y_3 = A_3$. Y_2 è VERA quando A_2 è attivo e A_3 non lo è, quindi $Y_2 = \overline{A_3}A_2$. Y_1 è VERA se A_1 è attivo ed entrambi gli ingressi con priorità più alta sono spenti, quindi $Y_1 = \overline{A_3}\overline{A_2}A_1$. Infine, Y_0 è VERA quando A_0 , e

solo A_0 , è portato a uno: $Y_0 = \bar{A}_3 \bar{A}_2 \bar{A}_1 A_0$. Lo schema della rete è riportato nella **Figura 2.28**. Un progettista esperto può spesso realizzare una rete logica tramite analisi del suo comportamento. Data una specifica chiara, il passo successivo è semplicemente quello di tradurre le parole in un'espressione e l'espressione in porte logiche.

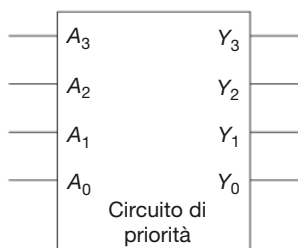
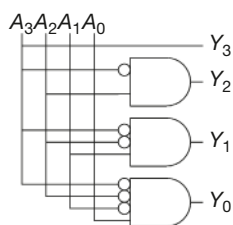


Figura 2.27
Rete logica di priorità.

A_3	A_2	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	0
0	0	1	1	0	0	1	0
0	1	0	0	0	1	0	0
0	1	0	1	0	1	0	0
0	1	1	0	0	1	0	0
0	1	1	1	0	1	0	0
1	0	0	0	1	0	0	0
1	0	0	1	1	0	0	0
1	0	1	0	1	0	0	0
1	0	1	1	1	0	0	0
1	1	0	0	1	0	0	0
1	1	0	1	1	0	0	0
1	1	1	0	1	0	0	0
1	1	1	1	1	0	0	0



A_3	A_2	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	X	0	0	1	0
0	1	X	X	0	1	0	0
1	X	X	X	1	0	0	0

Figura 2.28
Realizzazione della rete logica di priorità.

Figura 2.29
Tabella delle verità per la rete logica di priorità con indifferenze (X).

Si noti che se A_3 è portato a uno nella rete a priorità, le uscite non tengono conto dei valori presenti agli altri ingressi. Tali ingressi, ignorati dalle uscite, vengono contrassegnati con il simbolo X, e sono chiamati **indifferenze**. La **Figura 2.29** mostra che la tabella delle verità della rete a priorità a quattro ingressi si riduce sensibilmente quando si inseriscono le indifferenze. A partire da questa tabella delle verità è più semplice leggere l'espressione booleana in forma somma di prodotti ignorando gli ingressi con una X. Le indifferenze possono anche apparire nelle uscite delle tabelle delle verità, come discusso nel paragrafo 2.7.3.

Il simbolo X significa "indifferenza" nelle tabelle delle verità e "conflitto" nella simulazione logica (vedi par. 2.6.1). Bisogna quindi fare attenzione al contesto per non confondere i due significati. Alcuni autori usano D oppure ? per le indifferenze per evitare ambiguità.

2.5 ■ LOGICA COMBINATORIA SU PIÙ DI DUE LIVELLI

La logica in forma somma di prodotti viene chiamata **logica a due livelli**, perché consiste di letterali connessi a un primo livello di porte AND che, a sua volta, sono connesse a un secondo livello di porte OR. Spesso i progettisti costruiscono reti con più di due livelli di porte logiche, perché può accadere che queste reti combinatorie a più livelli richiedano di utilizzare meno hardware

rispetto alle loro controparti a due livelli. In questo contesto è utile ricordare che lo spostamento delle bolle è particolarmente utile per analizzare e progettare le reti a più livelli.

2.5.1 Riduzione dell’hardware

Alcune funzioni logiche necessitano di una grande quantità di hardware se vengono realizzate sulla base della logica a due livelli. A questo proposito, un esempio rilevante è una funzione XOR a più variabili. Si consideri, per esempio, la costruzione della porta XOR a tre ingressi utilizzando le tecniche a due livelli viste fino ad ora.

Come spiegato in precedenza, la funzione XOR con un numero N di ingressi produce un’uscita VERA quando un numero dispari di ingressi è VERO. La **Figura 2.30** mostra la tabella delle verità per la funzione XOR a tre ingressi, nella quale sono state cerchiiate le righe che producono un’uscita VERA. Da questa tabella delle verità è stata derivata l’espressione booleana in forma somma di prodotti (Espressione 2.6); quest’ultima, sfortunatamente, non può essere semplificata per avere un numero minore di implicant.

$$Y = \overline{A} \overline{B} C + \overline{A} B \overline{C} + A \overline{B} \overline{C} + ABC$$

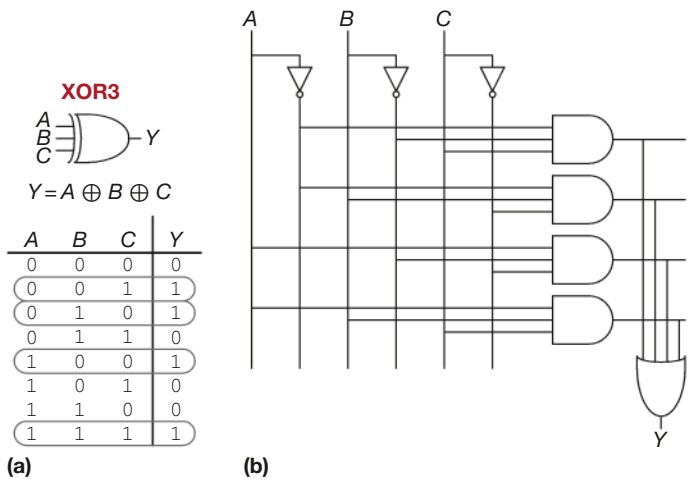
(2.6)

D’altra parte, $A \oplus B \oplus C = (A \oplus B) \oplus C$ (è facile dimostrare quest’uguaglianza usando l’induzione matematica perfetta). Quindi la funzione può essere realizzata con una cascata di XOR a due ingressi, come mostra la **Figura 2.31**.

Analogamente, una porta XOR a otto ingressi richiederebbe 128 porte AND a otto ingressi e una porta OR a 128 ingressi se realizzata come rete a due livelli in forma somma di prodotti. In questo caso è molto meglio utilizzare un albero di porte XOR a due ingressi, come mostra la **Figura 2.32**.

Decidere il miglior numero di livelli per la realizzazione di una funzione è un processo complesso; inoltre, a seconda delle situazioni, la scelta migliore può cambiare: la soluzione migliore può essere quella con il minor numero di porte, oppure la più veloce, o quella che richiede il minor tempo di progettazione, o la più economica, o ancora quella col minor consumo energetico. Nel Capitolo 5 si discute perché la soluzione “migliore” per una tecnologia non è necessariamente la soluzione migliore per un altro tipo di tecnologia. Per esempio, fino ad ora sono state utilizzate le porte AND e OR, tuttavia nella tecnologia CMOS sono più efficienti le porte NAND e NOR. Con un po’ di esperienza si può essere in grado di creare un buon progetto a più livelli tramite la sola indagine del comportamento per la maggior parte delle reti. Parte dell’esperienza necessaria può essere acquisita tramite lo studio delle reti de-

Figura 2.30
Porta XOR a tre ingressi:
(a) specifiche funzionali,
(b) realizzazione logica a due livelli.



scritte come esempi all'interno di questo testo. Man mano che si procede nello studio della materia è opportuno esplorare diverse opzioni di progettazione e confrontarne i risultati. È possibile utilizzare, inoltre, i diversi strumenti di progettazione assistita dal calcolatore (CAD, *Computer Aided Design*) disponibili per trovare, all'interno della vasta gamma di progetti a più livelli possibili, quello che meglio si addice ai limiti dati e ai blocchi costitutivi disponibili.

2.5.2 Spingere le bolle

Nel paragrafo 1.7.6 si è già accennato al fatto che le reti CMOS preferiscono le porte NAND e NOR alle porte AND e OR. Tuttavia, derivare un'espressione a partire da una rete a più livelli con porte NAND e NOR non è un compito semplice, come si deduce dalla **Figura 2.33**, che mostra una rete a più livelli la cui funzione non è immediatamente chiara. Spingere le bolle è molto utile in contesti come questo, per ridurre il numero di bolle e facilitare l'identificazione della funzione. In aggiunta ai principi elencati nel paragrafo 2.3.3, per spingere le bolle si tengano presenti le seguenti linee guida.

- Iniziare all'uscita della rete e lavorare a ritroso verso gli ingressi.
- Spingere qualsiasi bolla sia posizionata sull'uscita finale indietro verso gli ingressi, in modo da poter leggere l'espressione in termini di uscita diritta (per es. Y) piuttosto che in termini di uscita negata ($\bar{Y}$).
- Lavorando a ritroso, disegnare ogni porta in modo tale da cancellare le bolle. Se la porta corrente ha una bolla sull'ingresso, disegnare la porta che la precede con una bolla sull'uscita, mentre se la porta presente non ha una bolla sull'ingresso, disegnare la porta che la precede senza una bolla sull'uscita.

La **Figura 2.34** mostra come ridisegnare la rete della Figura 2.33 seguendo le linee guida per lo spostamento delle bolle.

Partendo dall'uscita Y , la porta NAND ha una bolla all'uscita che sarebbe opportuno eliminare. Per farlo, si spinge la bolla indietro a formare una porta OR con gli ingressi negati, come mostrato nella **Figura 2.34(a)**. Andando verso sinistra, la porta che si trova più a destra ha una bolla in ingresso che si elimina con la bolla in uscita della porta NAND di mezzo, quindi non è neces-

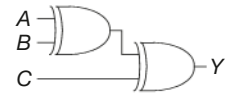


Figura 2.31
Porta XOR a tre ingressi realizzata con porte XOR a due ingressi.

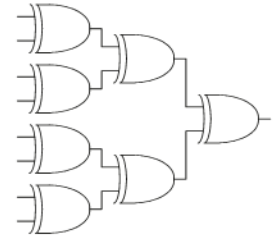


Figura 2.32
Porta XOR a otto ingressi realizzata con sette porte XOR a due ingressi.

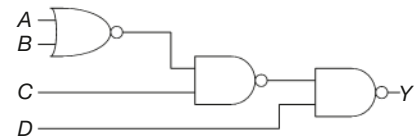


Figura 2.33
Rete logica a più livelli realizzata con porte NAND e NOR.

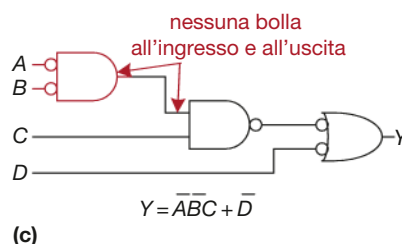
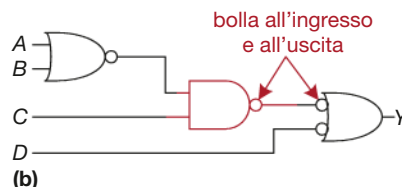
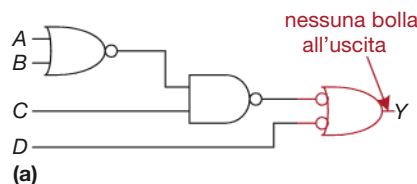


Figura 2.34
Rete logica con spinta di bolle.

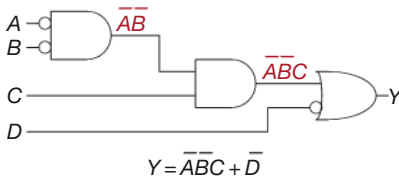


Figura 2.35
Rete logica equivalente con spinta di bolle.

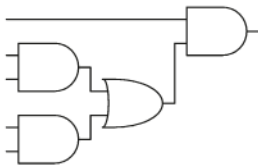


Figura 2.36
Rete logica con porte AND e OR.

Figura 2.37
Rete logica scadente con porte NAND e NOR.

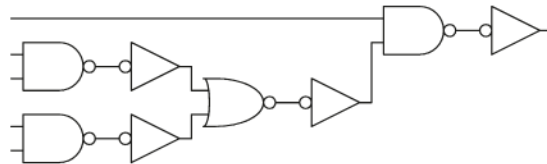
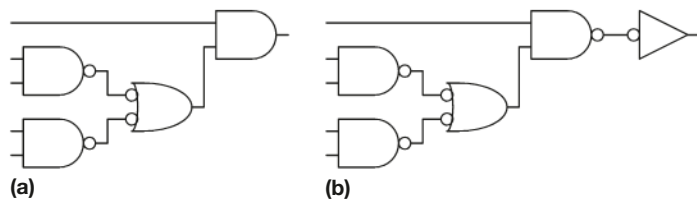


Figura 2.38
Rete logica con un conflitto.



2.6 ■ NON SOLO 0 E 1, ANCHE X E Z

L'algebra booleana si limita ai valori 0 e 1. Tuttavia, le reti reali possono anche dare luogo a valori di tensione illegali o fluttuanti, rappresentati simbolicamente con X e Z.

2.6.1 Il valore illegale: X

Il simbolo X indica che il nodo della rete ha un valore **sconosciuto** o **illegale**. Questo, generalmente, accade se al nodo vengono applicati entrambi i valori 0 e 1 allo stesso tempo, come mostra la **Figura 2.39**, dove il nodo Y è forzato sia a ALTO sia a BASSO. Questa situazione, chiamata **conflitto**, è un errore

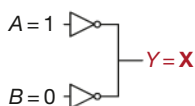


Figura 2.39
Rete logica con un conflitto.

e deve essere evitata. La tensione elettrica effettiva di un nodo che presenta conflitto è un valore indefinito compreso tra 0 e V_{DD} , a seconda della forza relativa delle due porte logiche che tentano di imporre simultaneamente i due valori ALTO e BASSO, e spesso, ma non sempre, ricade nella zona proibita. Questa situazione è anche causa di consumo di una grande quantità di energia da parte delle due porte in conflitto, e ciò risulta in una rete che tende a surriscaldarsi e probabilmente a danneggiarsi.

I valori X vengono a volte utilizzati dai simulatori di reti logiche per indicare un nodo non ancora inizializzato. Per esempio, se il progettista si dimentica di definire il valore di un ingresso, il simulatore associa all'ingresso il valore X per avvertirlo del problema.

Come è stato accennato nel paragrafo 2.4, i progettisti digitali utilizzano il simbolo X anche per indicare le indifferenze nelle tabelle delle verità, quindi è opportuno prestare attenzione per non confondere le due cose. Quando una X appare all'interno di una tabella delle verità, sta a indicare che il valore della variabile nella tabella non è importante (può essere sia uno 0 che un 1). Quando invece una X appare in una rete, indica che il nodo della rete ha un valore sconosciuto o illegale.

2.6.2 Il valore fluttuante: Z

Il simbolo Z indica che un nodo non è portato né al valore ALTO né a quello BASSO. Questo nodo è detto **fluttuante**, o ad **alta impedenza**. Un tipico errore è pensare che un nodo fluttuante o non attivato sia equivalente, a livello logico, a 0; in realtà, un nodo fluttuante può essere sia 0 sia 1, o può avere una qualsiasi tensione compresa tra 0 e 1, a seconda della storia del sistema. Un nodo fluttuante non è sempre sintomo di un errore nel sistema, finché è presente un altro elemento della rete che riporta il nodo a un valore logico valido quando tale valore è rilevante per il funzionamento della rete.

Un modo comune per creare un nodo fluttuante è dimenticarsi di connettere una tensione a un ingresso della rete, o anche assumere che un ingresso non connesso sia equivalente al valore 0. Un errore del genere può causare un comportamento imprevedibile della rete, con l'ingresso fluttuante che passa in maniera casuale da 0 a 1.

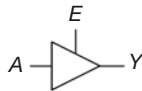
È infatti possibile che sia sufficiente toccare la rete con un dito per innescare il cambiamento di tensione grazie all'elettricità statica proveniente dal corpo umano. Non è la prima volta che si vedono in laboratorio reti che funzionano correttamente finché il dito di uno studente tiene premuto un chip...

Il **buffer tristate**, mostrato nella **Figura 2.40**, possiede tre possibili stati d'uscita: ALTO (1), BASSO (2) e fluttuante (Z). Il buffer tristate ha un ingresso A, un'uscita Y, e un'abilitazione E (da *Enable*). Quando l'abilitazione è VERO, il buffer lavora come un buffer normale e trasferisce il valore dell'ingresso all'uscita. Quando invece l'abilitazione è FALSO, l'uscita può avere il valore fluttuante (Z).

Il buffer tristate rappresentato nella Figura 2.40 ha il segnale di abilitazione E **attivo alto**. Ciò significa che il buffer è abilitato quando l'abilitazione è ALTO (1). Nella **Figura 2.41** viene invece mostrato un buffer tristate con l'abilitazione **attiva bassa**: in questo caso, il buffer è abilitato quando l'abilitazione è BASSO (0). Per mostrare che il segnale è attivo basso, viene posta una bolla sul filo di ingresso, e il segnale viene indicato con un trattino sul nome: $\bar{E}$.

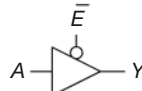
I buffer tristate vengono solitamente utilizzati sui **bus** che connettono più chip tra loro. Per esempio, un microprocessore, un controllore video o un controllore Ethernet possono tutti aver bisogno di comunicare con la memoria di sistema in un personal computer. In questi casi, come mostra la **Figura 2.42**, ogni chip può essere collegato al bus di memoria condiviso tramite un buffer tristate. Solo a un chip alla volta viene data l'abilitazione per consen-

Buffer tristate



E	A	Y
0	0	Z
0	1	Z
1	0	0
1	1	1

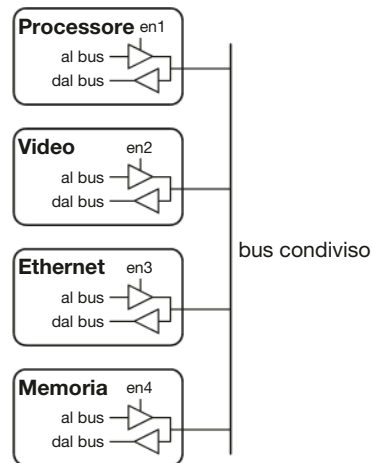
Figura 2.40
Buffer tristate.



$\bar{E}$	A	Y
0	0	0
0	1	1
1	0	Z
1	1	Z

Figura 2.41
Buffer tristate con abilitazione (E, enable) attiva bassa.

Figura 2.42
Bus tristate che collega più circuiti integrati.



tirgli di pilotare con le proprie uscite le linee del bus. Gli altri chip devono avere le proprie uscite fluttuanti per non causare conflitto con il chip che sta comunicando con la memoria. Ogni chip può leggere in qualsiasi momento le informazioni dal bus comune. I buffer tristate erano molto comuni un tempo, nei calcolatori moderni si preferiscono collegamenti punto a punto, in cui i chip sono collegati direttamente tra loro piuttosto che a un bus in comune, per garantire velocità superiori.

2.7 ■ LE MAPPE DI KARNAUGH

Chi ha lavorato alla semplificazione di espressioni booleane tramite l'utilizzo dell'algebra booleana si è senz'altro reso conto del fatto che una disattenzione durante il processo di minimizzazione può portare come risultato a un'espressione diversa da quella di partenza invece che a un'espressione equivalente semplificata. Le **mappe di Karnaugh** (*K-map*) sono un metodo grafico di semplificazione di espressioni booleane, inventato nel 1953 da Maurice Karnaugh, ingegnere delle telecomunicazioni presso i Bell Labs. Le mappe di Karnaugh sono uno strumento molto efficace per risolvere problemi con al massimo quattro variabili e, ancora più importante, permettono di comprendere la manipolazione delle espressioni booleane.

Come detto in precedenza, la minimizzazione logica richiede la combinazione di termini: due termini che contengono entrambi l'implicante P e le forme diritta e negata di una certa variabile A vengono combinati per eliminare A : $PA + P\bar{A} = P$. Le mappe di Karnaugh sono utili perché mettono in evidenza i termini che possono essere combinati affiancandoli all'interno di una griglia.

La **Figura 2.43** mostra la tabella delle verità e la mappa di Karnaugh per una funzione a tre ingressi. La riga in alto nella mappa di Karnaugh mostra le quattro possibili combinazioni di valori degli ingressi A e B , mentre i due possibili valori dell'ingresso C sono riportati nella colonna di sinistra. Ogni quadrato della mappa di Karnaugh corrisponde a una riga della tabella delle verità e contiene il valore, corrispondente a quella riga, dell'uscita Y . Per esempio, il quadrato in alto a sinistra corrisponde alla prima riga della tabella delle verità e indica che l'uscita ha valore $Y = 1$ quando $ABC = 000$. Ogni riquadro delle mappe di Karnaugh rappresenta, proprio come ogni riga delle tabelle delle verità, un singolo mintermine. Per capire meglio quest'ultimo concetto, la Figura 2.43(c) mostra il mintermine corrispondente a ogni riquadro della mappa di Karnaugh.

Ogni riquadro (o mintermine) differisce dal riquadro adiacente per un cambiamento in una sola variabile, il che significa che tutti i riquadri adia-

Maurice Karnaugh, 1924-

Si è laureato in fisica al City College di New York nel 1948 e ha conseguito il dottorato di ricerca in fisica a Yale nel 1952. Ha lavorato ai Bell Labs e all'IBM dal 1952 al 1993, ed è stato professore di informatica al Politecnico della New York University dal 1980 al 1999.

centi condividono tutti gli stessi letterali eccetto uno, che compare in forma diritta in un riquadro e in forma negata in quello adiacente. Per esempio, i riquadri che rappresentano i mintermini $\overline{A}\overline{B}\overline{C}$ e $\overline{A}\overline{B}C$ sono adiacenti e differiscono solo per la variabile C . Un lettore attento si sarà accorto che le combinazioni di A e B della riga in alto compaiono in un ordine particolare: 00, 01, 11, 10. Questo ordine è chiamato **codice Gray** ed è diverso dall'ordine binario ordinario (00, 01, 10, 11) proprio perché gli elementi adiacenti differiscono per una sola variabile. Per esempio, nel passaggio 01 : 11 cambia solo A da 0 a 1, mentre nell'ordine binario ordinario 01 : 10 cambierebbero A da 0 a 1 e B da 1 a 0. Ne consegue che scrivere le combinazioni seguendo l'ordine binario ordinario non avrebbe prodotto come risultato la proprietà utile delle mappe di Karnaugh dei riquadri adiacenti con un'unica variabile diversa.

Le mappe di Karnaugh inoltre “si richiudono su sé stesse”, nel senso che i riquadri all'estrema destra sono effettivamente adiacenti ai riquadri all'estrema sinistra e differiscono unicamente per la variabile A . In altre parole, sarebbe possibile prendere una mappa di Karnaugh e arrotolarla a formare un cilindro, poi unirne le estremità a formare una specie di salvagente, garantendo sempre la proprietà dei riquadri adiacenti diversi per una variabile.

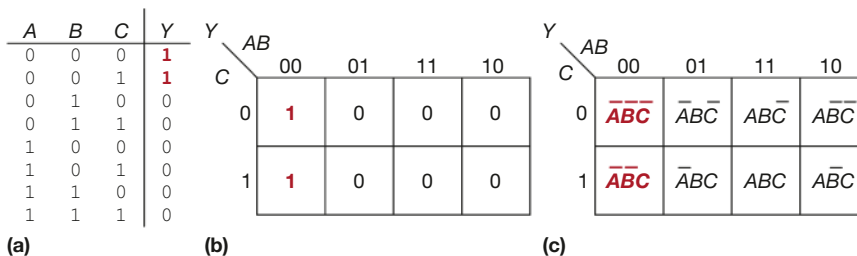
2.7.1 Pensare in cerchi...

Nella mappa di Karnaugh riportata nella Figura 2.43, l'espressione presenta solo due mintermini, $\overline{A}\overline{B}\overline{C}$ e $\overline{A}\overline{B}C$, indicati dagli 1 presenti nella colonna di sinistra. Leggere i mintermini da una mappa di Karnaugh è esattamente equivalente a leggere un'espressione in forma somma di prodotti direttamente da una tabella delle verità.

Come sempre, è possibile utilizzare l'algebra booleana per minimizzare le espressioni in forma somma di prodotti.

$$Y = \overline{A}\overline{B}\overline{C} + \overline{A}\overline{B}C = \overline{A}\overline{B}(\overline{C} + C) = \overline{A}\overline{B} \quad (2.7)$$

Le mappe di Karnaugh ci permettono di eseguire la semplificazione graficamente, **cerchiando** gli 1 nei riquadri adiacenti, come mostrato nella **Figura 2.44**. Per ogni cerchio si deve poi scrivere l'implicante corrispondente. Si tenga a mente, come spiegato nel paragrafo 2.2, che un implicante è il prodotto di uno o più letterali. Le variabili le cui forme diritta e negata sono state entrambe cerciate vengono escluse dall'implicante, come accade alla variabile C nell'esempio, le cui forme diritta (0) e negata (1) sono entrambe cerciate. In altre parole,



I codici Gray sono stati brevettati da Frank Gray, ricercatore ai Bell Labs, nel 1953 (brevetto statunitense n. 2 632 058). Sono particolarmente utili negli encoder meccanici, perché un eventuale piccolo disallineamento può causare al massimo l'errore di un solo bit.

I codici Gray si possono generalizzare a un numero qualsiasi di bit. Per esempio, la sequenza delle configurazioni del codice Gray a tre bit è:

000, 001, 011, 010,
110, 111, 101, 100

Lewis Carroll nel 1879 ha pubblicato su *Vanity Fair* un indovinello relativo al codice Gray.

“Le regole di questo indovinello sono semplici. Si propongono due parole di uguale lunghezza; si devono collegare le due parole con altre parole, ciascuna delle quali deve differire dalla precedente per una sola lettera. Quindi si può cambiare una lettera nella prima parola proposta, poi un'altra lettera nella parola così ottenuta, e così via fino a ottenere la seconda parola proposta.”

Per esempio, per andare da SHIP a DOCK:

SHIP, SLIP, SLOP,
SLOT, SOOT, LOOT,
LOOK, LOCK, DOCK.

Chi è capace di trovare una sequenza più corta?

Figura 2.43
Funzione a tre ingressi: (a) tabella delle verità, (b) mappa di Karnaugh, (c) mappa di Karnaugh con i mintermini.

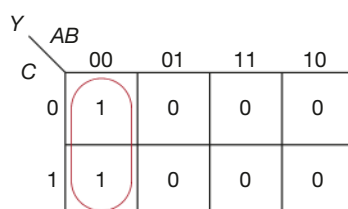


Figura 2.44
Minimizzazione con la mappa di Karnaugh.

Y è VERO quando $A = B = 0$, indipendentemente dal valore assunto da C . Ne consegue che l'implicante è $\overline{A}\overline{B}$ e che la mappa di Karnaugh restituisce lo stesso risultato ottenuto con la semplificazione tramite utilizzo dell'algebra booleana.

2.7.2 Minimizzazione logica con le mappe di Karnaugh

Come detto in precedenza, le mappe di Karnaugh sono un semplice strumento grafico di minimizzazione logica: vengono semplicemente cerchiati i riquadri della mappa che contengono un 1, utilizzando il minor numero possibile di cerchi e includendo in ogni cerchio il maggior numero possibile di riquadri, per poi leggere gli implicanti a partire dai cerchi.

Per esprimere il concetto in maniera più formale, si deve ritornare alla definizione di espressione booleana minima, che è tale quando viene scritta come la somma del minor numero possibile di implicanti primi. Nel caso delle mappe di Karnaugh, ogni cerchio rappresenta un implicante e i cerchi più larghi possibili rappresentano gli implicanti primi.

Per esempio, nella mappa di Karnaugh della Figura 2.44, $\overline{A}\overline{B}\overline{C}$ e $\overline{A}\overline{B}C$ sono entrambi implicanti, tuttavia non sono implicanti primi. In questa particolare mappa di Karnaugh, solo $\overline{A}\overline{B}$ è un implicante primo. Le regole per trovare l'espressione minima a partire da una mappa di Karnaugh sono le seguenti.

- Utilizzare il minor numero possibile di cerchi per includere tutti gli 1.
- Tutti i riquadri racchiusi in ciascun cerchio devono contenere 1.
- Ogni cerchio deve includere un numero di riquadri che sia una potenza di due (cioè 1, 2 o 4 riquadri) in qualsiasi direzione.
- Ogni cerchio deve essere il più largo possibile.
- È possibile disegnare un cerchio che avvolga le estremità della mappa di Karnaugh.
- Un 1 in una mappa di Karnaugh può essere cerchiato più di una volta, se questa operazione permette di utilizzare un numero minore di cerchi.

ESEMPIO 2.9

Minimizzazione di una funzione a tre variabili con l'utilizzo delle mappe di Karnaugh. Si supponga di avere la funzione $Y = F(A, B, C)$ descritta dalla mappa di Karnaugh rappresentata nella Figura 2.45. Minimizzare l'espressione utilizzando la mappa.

Soluzione Cerchiare tutti gli 1 della mappa utilizzando il minor numero possibile di cerchi, come mostrato nella Figura 2.46. Ogni cerchio rappresenta un implicante primo e le dimensioni di ogni cerchio sono pari a una potenza di due (2×1 e 2×2). Formare l'implicante primo corrispondente a ogni cerchio scrivendone le variabili in forma o solo diretta o solo negata.

Per esempio, nel cerchio da 2×1 sono comprese sia la forma diretta sia la forma negata della variabile B , quindi B non viene inclusa nell'implicante primo. Invece, nel cerchio appaiono solo la forma diretta di A (A) e solo la forma negata di C ($\overline{C}$), quindi queste variabili sono da includere nell'implicante primo $A\overline{C}$. Analogamente, il cerchio 2×2 contiene tutti i riquadri in cui $B = 0$, quindi l'implicante primo sarà $\overline{B}$.

Y C	AB			
	00	01	11	10
0	1	0	1	1
1	1	0	0	1

Figura 2.45
Mappa di Karnaugh per l'Esempio 2.9.

Figura 2.46
Soluzione dell'Esempio 2.9.

Y C	AB			
	00	01	11	10
0	1	0	1	1
1	1	0	0	1

$Y = A\overline{C} + \overline{B}$

Si noti come il riquadro (mintermine) in alto a destra sia incluso due volte in un cerchio al fine di disegnare i cerchi degli implicanti primi più grandi possibili. Quest'operazione è analoga alla tecnica di algebra booleana, vista in precedenza, di condivisione di un mintermine al fine di ridurre la dimensione dell'implicante. Infine, si noti anche come il cerchio da quattro riquadri avvolge le estremità laterali della mappa di Karnaugh.

ESEMPIO 2.10

Transcodificatore per display a sette segmenti. Un transcodificatore per display a sette segmenti riceve un dato di ingresso a 4 bit $D_{3:0}$ e produce sette uscite per controllare 7 diodi emettitori di luce (LED, *Light Emitting Diode*) che visualizzano una cifra da 0 a 9. Le sette uscite sono spesso chiamate segmenti da a a g , o S_a-S_g , come indicato nella **Figura 2.47**. Le cifre sono mostrate nella **Figura 2.48**.

Scrivere una tabella delle verità per le uscite e utilizzare una mappa di Karnaugh per trovare le espressioni booleane delle uscite S_a e S_b , facendo in modo che i valori illegali d'ingresso (10-15) non accendano alcun LED.

Soluzione La tabella delle verità è riportata nella **Tabella 2.6** nella quale, per esempio, l'ingresso 0000 deve accendere tutti i segmenti tranne S_g .

Ognuna delle sette uscite rappresenta una funzione indipendente di quattro variabili. Le mappe di Karnaugh corrispondenti alle uscite S_a e S_b sono riportate nella **Figura 2.49**. Per derivare da queste mappe le espressioni booleane si inizia dal concetto principale,

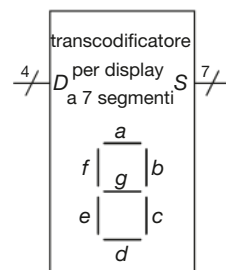


Figura 2.47
Simbolo del transcodificatore per display a sette segmenti.

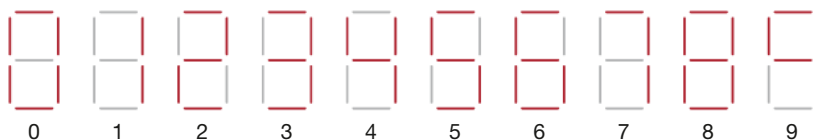


Figura 2.48
Cifre in un display a sette segmenti.

Tabella 2.6 Tabella delle verità del transcodificatore per display a sette segmenti.

$D_{3:0}$	S_a	S_b	S_c	S_d	S_e	S_f	S_g
0000	1	1	1	1	1	1	0
0001	0	1	1	0	0	0	0
0010	1	1	0	1	1	0	1
0011	1	1	1	1	0	0	1
0100	0	1	1	0	0	1	1
0101	1	0	1	1	0	1	1
0110	1	0	1	1	1	1	1
0111	1	1	1	0	0	0	0
1000	1	1	1	1	1	1	1
1001	1	1	1	0	0	1	1
Altri	0	0	0	0	0	0	0

S_a

$D_{3:2}$	00	01	11	10
$D_{1:0}$				
00	1	0	0	1
01	0	1	0	1
11	1	1	0	0
10	1	1	0	0

S_b

$D_{3:2}$	00	01	11	10
$D_{1:0}$				
00	1	1	0	1
01	1	0	0	1
11	1	1	0	0
10	1	0	0	0

Figura 2.49
Mappe di Karnaugh per S_a e S_b .

cioè che i riquadri adiacenti differiscono per una singola variabile, e si etichettano le righe e le colonne seguendo l'ordine del codice Gray: 00, 01, 11, 10. Si deve poi fare attenzione a utilizzare questo stesso ordine anche durante l'inserimento dei valori delle uscite nei riquadri delle mappe.

Successivamente, si cerchiano gli implicanti primi, ricordandosi di utilizzare il minor numero possibile di cerchi per includere tutti gli 1. Un cerchio può includere anche i bordi (verticali e orizzontali) e un 1 può essere cerchiato più di una volta. La **Figura 2.50** riporta gli implicanti primi e le espressioni booleane semplificate.

L'insieme minimo di implicanti primi non è unico: per esempio, il riquadro 0000 nella mappa di Karnaugh di S_a è cerchiato insieme al riquadro 1000 per produrre l'implicante $\bar{D}_2 \bar{D}_1 \bar{D}_0$. Invece, il cerchio avrebbe potuto includere il riquadro 0010, producendo così l'implicante $\bar{D}_3 \bar{D}_1 \bar{D}_0$, come mostrano le linee tratteggiate nella **Figura 2.51**.

La **Figura 2.52** mostra un errore comune, ovvero la scelta di un implicante non primo per includere l'1 nell'angolo in alto a sinistra della mappa. Questo mintermine, $\bar{D}_3 \bar{D}_2 \bar{D}_1 \bar{D}_0$, dà un'espressione in forma somma di prodotti finale che non è minima. La soluzione corretta era combinare il mintermine con uno dei riquadri adiacenti per formare un cerchio più largo, come è stato fatto nelle due figure precedenti.

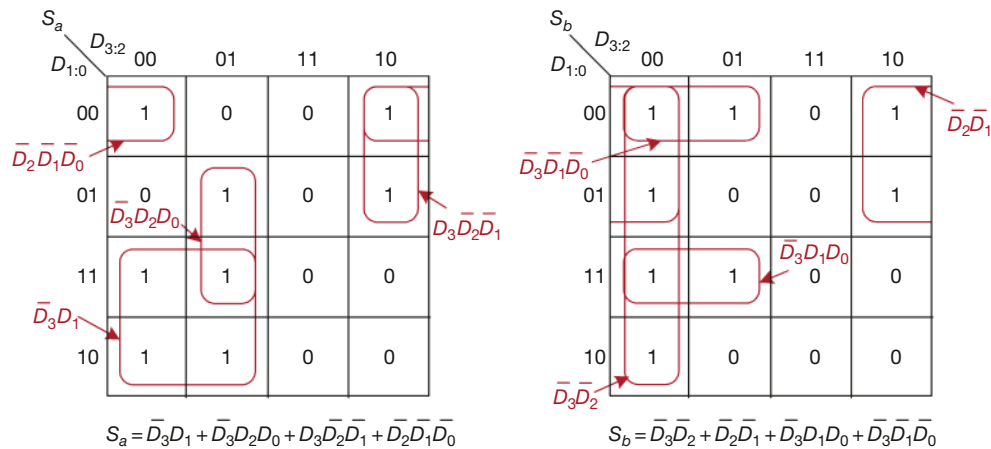
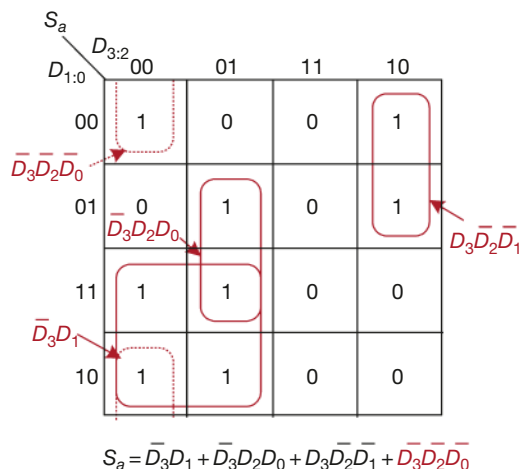


Figura 2.50 Mappe di Karnaugh per la soluzione dell'Esempio 2.10.

Figura 2.51
Mappa di Karnaugh alternativa per S_a con diversi insiemi di implicanti.



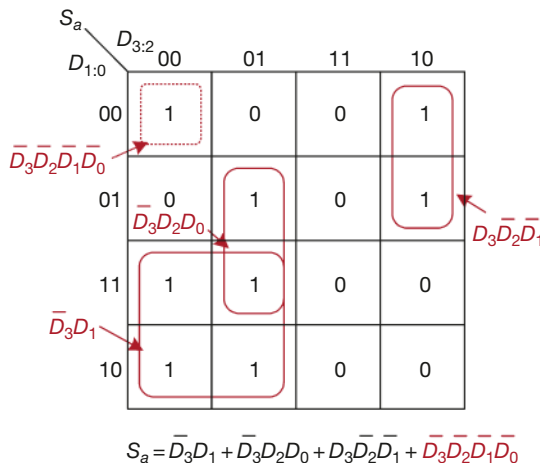


Figura 2.52
Mappa di Karnaugh alternativa per S_a con implicant errati perché non primi.

2.7.3 Indifferenze

Le indifferenze nelle tabelle delle verità degli ingressi sono state introdotte nel paragrafo 2.4 per ridurre il numero di righe della tabella delle verità quando sono presenti valori delle variabili che non hanno effetto su un'uscita. Queste, come detto, vengono indicate col simbolo X, che significa che il valore può essere sia 0 sia 1.

Le indifferenze possono apparire anche nelle uscite delle tabelle delle verità, quando il valore d'uscita non ha importanza o quando la combinazione d'ingresso corrispondente non può presentarsi. Il progettista può considerare a sua discrezione queste uscite come degli 0 o degli 1.

In una mappa di Karnaugh, una X permette di aumentare ulteriormente la minimizzazione logica; infatti, queste possono essere incluse nei cerchi se l'operazione può essere utile per coprire più riquadri a 1 con un numero minore di cerchi, mentre sono da ignorare se non sono d'aiuto.

ESEMPIO 2.11

Transcodificatore per display a sette segmenti con indifferenze. Ripetere l'esercizio dell'Esempio 2.10 con i valori d'ingresso illegali (10-15) come indifferenze.

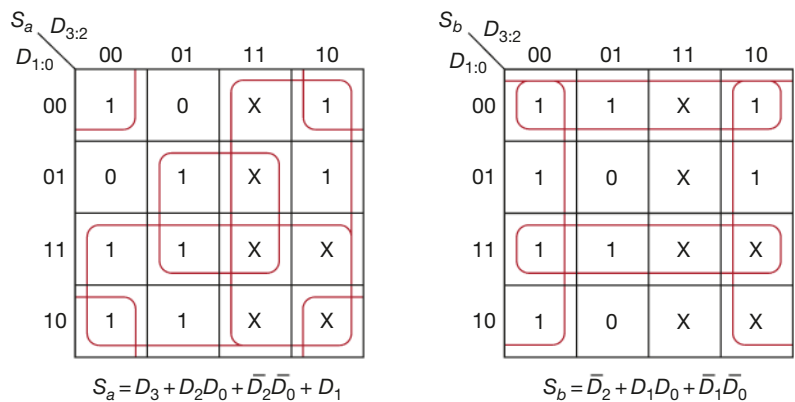
Soluzione La **Figura 2.53** mostra le mappe di Karnaugh adattate, con le X a indicare le indifferenze. Dal momento che un'indifferenza può essere sia uno 0 sia un 1, queste verranno incluse nei cerchi se permettono di diminuirne il numero o aumentarne la dimensione. Le indifferenze incluse nei cerchi verranno quindi considerate come 1, mentre quelle escluse come degli 0. Per il segmento S_a è possibile disegnare un cerchio che include un riquadro 2×2 avvolto intorno ai quattro angoli. L'utilizzo delle indifferenze semplifica considerevolmente la rete logica finale.

2.7.4 Il quadro generale

L'algebra booleana e le mappe di Karnaugh sono due tecniche di semplificazione logica. In definitiva, l'obiettivo è trovare un metodo a basso costo di realizzazione di una particolare funzione logica.

Nella pratica ingegneristica moderna esistono programmi per calcolatori chiamati **sintetizzatori logici** che producono reti semplificate a partire da una descrizione della funzione logica, come verrà spiegato nel Capitolo 4. In caso di grandi problemi, i sintetizzatori logici funzionano meglio della mente umana, mentre per problemi più piccoli una persona con un minimo di esperienza è in grado di trovare una buona soluzione esaminando il compor-

Figura 2.53
Mappe di Karnaugh con
indifferenze per la soluzione
dell'Esempio 2.10.



tamento della funzione. Nessuno dei due autori ha mai utilizzato una mappa di Karnaugh nella vita di tutti i giorni per risolvere un problema pratico, tuttavia la conoscenza acquisita dai principi che sono alla base delle mappe di Karnaugh ha un gran valore. Inoltre, spesso vengono poste domande sulle mappe di Karnaugh durante i colloqui di lavoro!

2.8 ■ BLOCCHI COSTITUTIVI COMBINATORI

La logica combinatoria viene spesso raggruppata in blocchi costitutivi più ampi per costruire sistemi più complessi. Questa è chiaramente un'applicazione del principio dell'astrazione, che nasconde i dettagli non necessari a livello delle porte logiche per enfatizzare la funzione del blocco costitutivo. Sono già stati analizzati in questi primi due capitoli tre blocchi costitutivi: i sommatori (par. 2.1), le reti a priorità (par. 2.4) e i transcodificatori per display a sette segmenti (par. 2.7). In questo paragrafo si introducono altri due blocchi costitutivi utilizzati comunemente: i *multiplexer* e i *decoder*. Il Capitolo 5 tratta altri blocchi costitutivi combinatori.

2.8.1 Multiplexer

I *multiplexer* sono tra le reti combinatorie più comunemente usate. Essi sono in grado di scegliere un'uscita a partire da un certo numero di ingressi possibili basandosi sul valore di un segnale di selezione. I multiplexer sono anche chiamati, in gergo, **mux**.

Multiplexer 2:1

La **Figura 2.54** mostra il simbolo circuitale e la tabella delle verità per un multiplexer 2:1 con due ingressi di dato D_0 e D_1 , un ingresso di selezione S e un'uscita Y . Il multiplexer sceglie tra i due ingressi di dato a seconda del valore assunto da S : se $S = 0$, $Y = D_0$, e se $S = 1$, $Y = D_1$. S viene anche chiamato **segnale di controllo** proprio perché controlla la scelta del multiplexer.

Un multiplexer 2:1 può essere costruito a partire dalla logica a somma di prodotti, come mostra la **Figura 2.55**, e la sua espressione booleana può essere derivata con una mappa di Karnaugh, oppure esaminando il suo comportamento (Y è 1 se $S = 0$ AND $D_0 = 1$ OR se $S = 1$ AND $D_1 = 1$).

In alternativa, i multiplexer possono essere realizzati con buffer tristate, come mostra la **Figura 2.56**: le abilitazioni dei buffer sono organizzate in maniera tale che, in ogni momento, solo un buffer tristate è attivo. Quando $S = 0$, viene attivato il tristate T0, permettendo così a D_0 di passare in Y ; quando invece $S = 1$, viene attivato il tristate T1, che permette a D_1 di passare in Y .

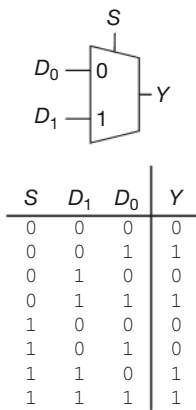


Figura 2.54
Simbolo del multiplexer e relativa
tabella delle verità.

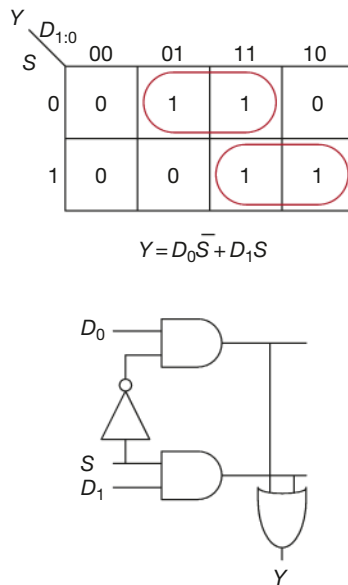


Figura 2.55
Realizzazione del multiplexer
in logica a due livelli.

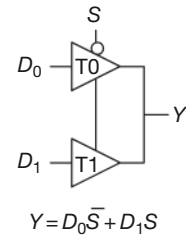


Figura 2.56
Multiplexer realizzato con buffer
tristate.

Multiplexer più grandi

Un multiplexer 4:1 possiede quattro ingressi di dato e un'uscita, come mostra la **Figura 2.57**. In questo caso sono necessari due segnali di selezione per scegliere tra i quattro ingressi di dato. Il multiplexer 4:1 può essere costruito utilizzando la logica a somma di prodotti, i tristate, o alcuni multiplexer 2:1, come mostrato nella **Figura 2.58**.

I termini di prodotto che abilitano i tristate possono essere formati utilizzando porte AND e negatori, ma anche utilizzando un decoder, discusso nel paragrafo 2.8.2.

I multiplexer ancora più grandi, come i multiplexer 8:1 o 16:1, vengono costruiti espandendo i metodi di realizzazione riportati nella Figura 2.58: in linea generale, un multiplexer $N:1$ necessita di $\log_2 N$ ingressi di selezione. Ancora una volta, la migliore scelta di realizzazione dipende dalla tecnologia che si deve usare.

Logica a multiplexer

I multiplexer possono essere utilizzati come tabelle di ricerca (*lookup table*) per eseguire funzioni logiche. Nella **Figura 2.59** è rappresentato un multi-

Collegare tra loro le uscite di porte logiche diverse viola le regole di composizione combinatoria enunciate nel paragrafo 2.1. Ma poiché solo una delle uscite è attiva in ogni istante, questa eccezione è consentita.

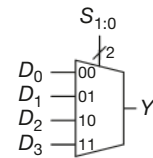


Figura 2.57
Multiplexer 4:1.

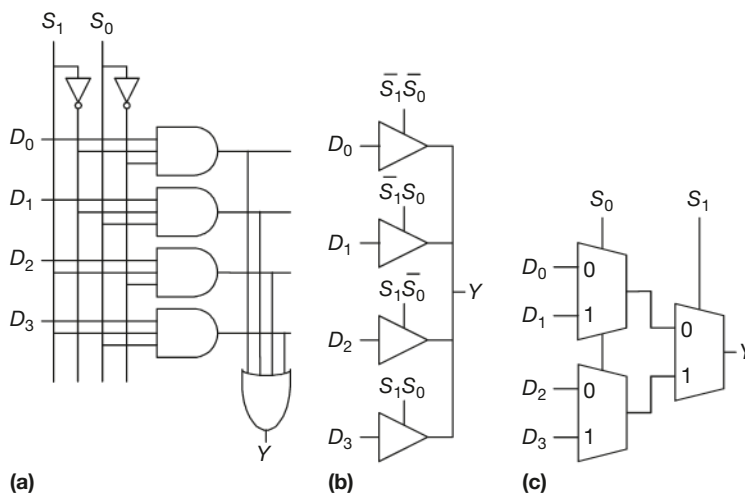


Figura 2.58
Possibili realizzazioni di un
multiplexer: (a) logica a due livelli,
(b) porte tristate, (c) gerarchia di
multiplexer.

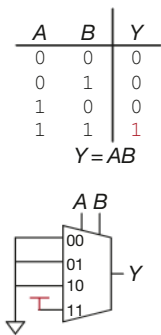


Figura 2.59
Realizzazione mediante multiplexer dell'operatore logico AND a due ingressi.

plexer 4:1 utilizzato per realizzare una porta AND a due ingressi, A e B, che operano come ingressi di selezione.

Gli ingressi di dato del multiplexer sono connessi a 0 o a 1 in base alla corrispondente riga della tabella delle verità. In generale, un multiplexer con 2^N ingressi può essere usato per realizzare una qualsiasi funzione logica a N ingressi applicando gli 0 e gli 1 agli ingressi di dato appropriati. Inoltre, cambiando gli ingressi di dato, è possibile riprogrammare il multiplexer al fine di realizzare una funzione diversa.

Con un po' di astuzia è possibile dimezzare la taglia del multiplexer, utilizzando solo un multiplexer a 2^{N-1} ingressi per eseguire una qualsiasi funzione logica a N ingressi. La strategia di base è fornire uno dei letterali di ingresso della funzione, insieme agli 0 e agli 1, agli ingressi di dato del multiplexer.

Per illustrare meglio questo principio, vengono mostrate nella **Figura 2.60** la funzione AND e la funzione XOR, entrambe a due ingressi, realizzate tramite dei multiplexer 2:1. Si inizia dalla solita tabella delle verità per poi combinare coppie di righe in modo tale da eliminare la variabile d'ingresso all'estrema destra esprimendo l'uscita in termini di questa variabile. Per esempio, nel caso di AND, quando $A = 0$, $Y = 0$ indipendentemente da B. Quando $A = 1$, $Y = 0$ se $B = 0$ e $Y = 1$ se $B = 1$, quindi $Y = B$. Il multiplexer viene quindi utilizzato come lookup table in accordo con la nuova, più piccola, tabella delle verità.

ESEMPIO 2.12

Logica con multiplexer. Alyssa Guastacomputer deve realizzare la funzione $Y = A\bar{B} + \bar{B}C + \bar{A}BC$ per concludere il suo progetto ma, quando va a vedere il suo kit da laboratorio, si accorge di avere solo un multiplexer 8:1. Come può realizzare la funzione?

Soluzione Nella **Figura 2.61** è riportata la realizzazione di Alyssa con l'utilizzo del multiplexer 8:1. Quest'ultimo funge da lookup table, dove ogni riga della tabella delle verità corrisponde a un ingresso del multiplexer.

Figura 2.60
Reti logiche a multiplexer con ingressi variabili.

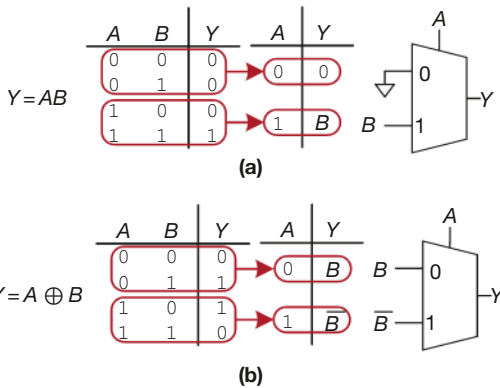
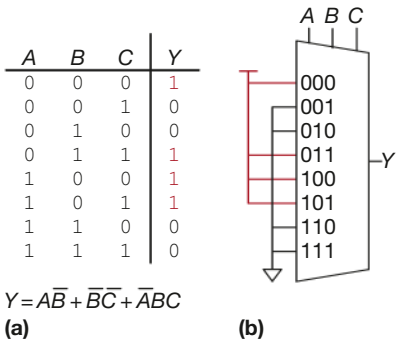


Figura 2.61
La rete logica di Alyssa: (a) tabella delle verità, (b) realizzazione con un multiplexer 8:1.



ESEMPIO 2.13

Ancora logica con multiplexer. Alyssa accende la rete appena costruita un'ultima volta prima della presentazione finale e accidentalmente brucia il multiplexer (dopo non aver dormito tutta la notte, PER SBAGLIO lo alimenta con 20 V al posto dei soliti 5 V). Prega allora i suoi amici affinché le prestino alcune parti avanzate e riesce a recuperare un multiplexer 4:1 e un negatore. È possibile ricostruire la rete con solo queste parti?

Soluzione Alyssa riduce la sua tabella delle verità a quattro righe rendendo l'uscita dipendente da C (ma avrebbe anche potuto scegliere di riorganizzare le colonne della tabella delle verità per fare in modo che l'uscita dipendesse da A o da B). La **Figura 2.62** mostra il nuovo progetto.

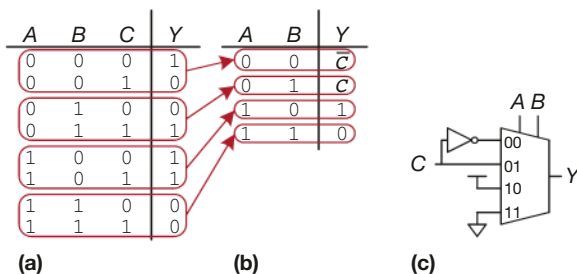
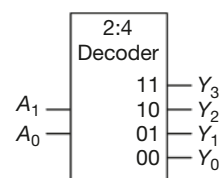


Figura 2.62
Nuova rete logica di Alyssa.



A ₁	A ₀	Y ₃	Y ₂	Y ₁	Y ₀
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

Figura 2.63
Decoder 2:4.

2.8.2 Decoder

Un **decoder** ha N ingressi e 2^N uscite e attiva una delle sue uscite a seconda della combinazione di valori in ingresso. La **Figura 2.63** fornisce un esempio di un decoder 2:4. Quando $A_{1:0} = 00$, Y_0 è 1. Quando $A_{1:0} = 01$, Y_1 è 1 e così via. Le uscite sono dette *one hot* (una calda) proprio perché solo un'uscita è "calda" (ALTA) in ogni momento.

ESEMPIO 2.14

Realizzazione del decoder. Realizzare un decoder 2:4 utilizzando porte AND, OR e NOT.

Soluzione La **Figura 2.64** mostra la realizzazione del decoder 2:4 con l'utilizzo di quattro porte AND. Ogni porta dipende o dalla forma diretta o dalla forma negata di ogni ingresso. In generale, un decoder $N:2^N$ può essere costruito con 2^N porte AND a N ingressi che ricevono le diverse combinazioni possibili degli ingressi diretti o negati. Ogni uscita di un decoder rappresenta un mintermine; per esempio, Y_0 rappresenta il mintermine $\bar{A}_1\bar{A}_0$. Questa caratteristica è da tenere bene a mente quando verranno utilizzati i decoder insieme ad altri blocchi costitutivi digitali.

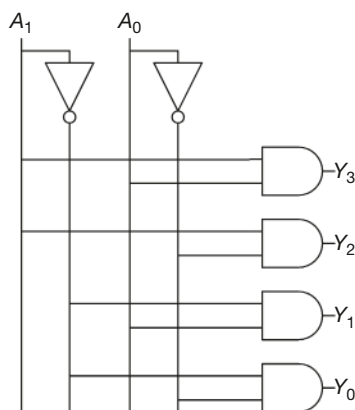


Figura 2.64
Realizzazione del decoder 2:4.

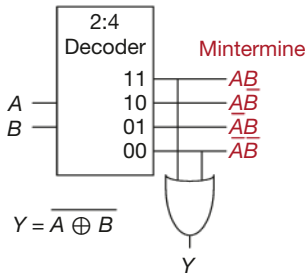


Figura 2.65
Funzione logica realizzata mediante decoder.

Logica a decoder

I decoder possono essere combinati insieme a delle porte OR per costruire funzioni logiche. Nella **Figura 2.65** viene mostrata una funzione XNOR a due ingressi realizzata con un decoder 2:4 e una singola porta OR. Dal momento che ogni uscita di un decoder rappresenta un mintermine, la funzione è stata costruita come OR di tutti i mintermini che la compongono, ovvero, come nella **Figura 2.65**, $Y = \overline{A}\overline{B} + AB = \overline{A} \oplus B$.

Quando si utilizzano i decoder per costruire reti logiche, è più semplice esprimere le funzioni attraverso una tabella delle verità o in forma canonica somma di prodotti. Una funzione a N ingressi con un numero M di 1 in una tabella della verità può essere costruita con un decoder $N:2^N$ e una porta OR a M ingressi connessa a tutti i mintermini associati a un 1 in uscita nella tabella. Questo stesso concetto verrà ripreso e applicato per costruire le memorie a sola lettura (ROM, *Read Only Memory*) nel paragrafo 5.5.6.

2.9 ■ TEMPORIZZAZIONI

Nei paragrafi precedenti si è lavorato principalmente sul far funzionare una rete, possibilmente costruita col minor numero possibile di porte. Tuttavia, come ben sa qualsiasi progettista di reti con una certa esperienza, uno dei problemi più importanti legato alle reti è la **temporizzazione** (*timing*): in altre parole, come fare in modo che la rete funzioni velocemente.

Un'uscita impiega un certo tempo ad adattarsi a un cambiamento avvenuto in un ingresso. Nella **Figura 2.66** viene mostrato il **ritardo** (*delay*) tra un cambiamento in un ingresso e il conseguente adattamento dell'uscita in un *buffer*; questa immagine viene chiamata **diagramma temporale** e raffigura la **risposta transitoria** della rete buffer al cambiare di un ingresso. Il passaggio da BASSO a ALTO viene chiamato **fronte di salita**, mentre il passaggio inverso, da ALTO a BASSO (non mostrato nella figura) viene detto **fronte di discesa**. La freccia rossa presente in figura sta a sottolineare come il fronte di salita di Y dipenda dal fronte di salita di A . Il ritardo viene misurato a partire dal punto al 50% del segnale d'ingresso, A , fino al punto al 50% del segnale d'uscita, Y . Tale punto è il punto al quale il segnale si trova a metà del percorso (50%) tra i valori BASSO e ALTO del suo cambiamento.

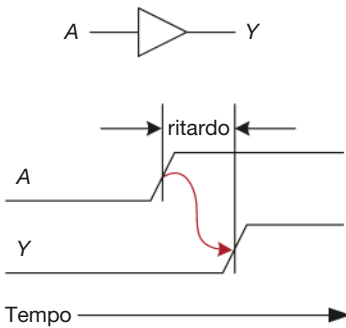


Figura 2.66
Ritardo del circuito.

Quando i progettisti parlano di ritardo (*delay*) di un circuito, si riferiscono generalmente al valore del ritardo di propagazione nel caso peggior, a meno che non sia chiaro dal contesto che si intende altro.

2.9.1 Ritardi di propagazione e di contaminazione

La logica combinatoria è caratterizzata da un **ritardo di propagazione** (*propagation delay*) e da un **ritardo di contaminazione** (*contamination delay*). Il ritardo di propagazione t_{pd} è il tempo massimo che trascorre dal momento in cui avviene un cambiamento nell'ingresso al momento in cui l'uscita (o le uscite) raggiunge il suo valore finale. Invece, il ritardo di contaminazione t_{cd} è il tempo minimo che trascorre dal momento in cui cambia un ingresso al momento in cui una qualsiasi uscita comincia il processo di adattamento del suo valore.

Nella **Figura 2.67** è illustrato il ritardo di propagazione (in rosso) e il ritardo di contaminazione (in grigio) di un buffer. La figura mostra l'ingresso A inizialmente con un valore ALTO o BASSO e il suo cambiamento allo stato opposto in un momento preciso; il fattore importante è il cambiamento di stato dell'ingresso in sé, piuttosto che il valore iniziale. In risposta, l'uscita Y si adatta al nuovo valore dopo un certo tempo. Gli archi presenti nella figura indicano che Y inizia il processo di adattamento dopo il cambio di valore di A al punto t_{cd} per stabilizzarsi al suo nuovo valore al punto t_{pd} .

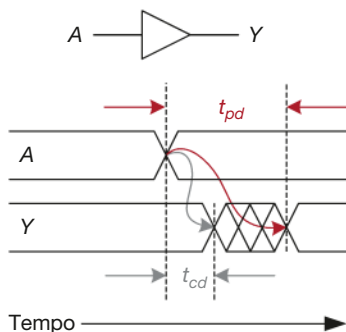


Figura 2.67
Ritardi di propagazione e di contaminazione.

Le cause del ritardo nelle reti includono il tempo richiesto per caricare la capacità elettrica in una rete e la velocità della luce. t_{pd} e t_{cd} possono essere diversi per una serie di motivi, tra cui:

- diversi ritardi tra salita e discesa;
- ingressi e uscite multipli, alcuni più veloci di altri;
- reti che rallentano quando si surriscaldano e che lavorano più velocemente se fredde.

Il calcolo di t_{pd} e t_{cd} richiede di ragionare a livelli più bassi di astrazione di quelli considerati l'obiettivo di questo testo. Ad ogni modo, i produttori solitamente forniscono schede tecniche (*data sheet*) che specificano i ritardi di ogni porta.

I ritardi di propagazione e contaminazione sono determinati, oltre che dai fattori già elencati, anche dal percorso che un segnale segue tra l'ingresso e l'uscita. La **Figura 2.68** mostra una rete logica a quattro ingressi; il **percorso critico** (*critical path*, evidenziato in rosso) è il percorso seguito dal segnale tra l'ingresso A o B fino all'uscita Y. Questo percorso viene definito critico perché limita la velocità alla quale la rete opera. Il **percorso minimo** (*short path*, in grigio) è invece quello seguito dal segnale tra l'ingresso D e l'uscita Y. Quest'ultimo è il percorso più breve possibile (e quindi anche il più veloce) attraverso la rete, perché il segnale attraversa solo una porta tra l'ingresso e l'uscita.

Il ritardo di propagazione di una rete combinatoria è uguale alla somma dei singoli ritardi di propagazione di ogni elemento del percorso critico, mentre il ritardo di contaminazione è la somma dei ritardi di contaminazione attraverso ogni elemento del percorso minimo. Questi due ritardi sono riportati nella **Figura 2.69** e vengono descritti dalle seguenti espressioni:

$$t_{pd} = 2t_{pd_AND} + t_{pd_OR} \quad (2.8)$$

$$t_{cd} = t_{cd_AND} \quad (2.9)$$

I ritardi dei circuiti sono generalmente nell'ordine dai picosecondi ($1 \text{ ps} = 10^{-12}$ secondi) ai nanosecondi ($1 \text{ ns} = 10^{-9}$ secondi). Solo per leggere questo riquadro sono serviti trilioni di picosecondi!

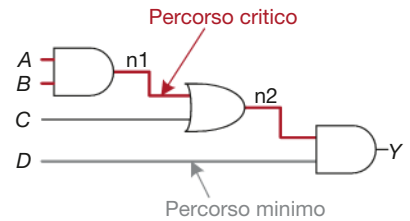


Figura 2.68
Percorso minimo e percorso critico.

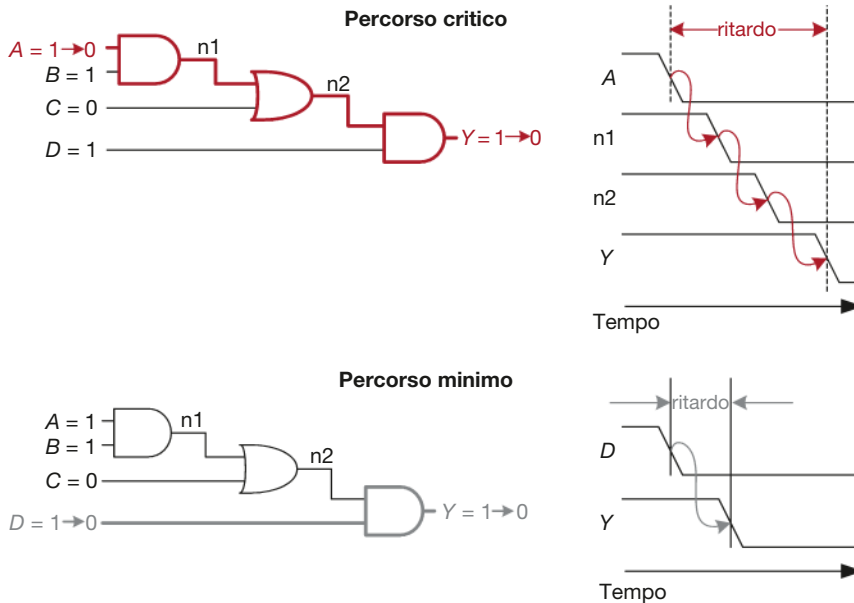


Figura 2.69
Forme d'onda del percorso critico e del percorso minimo.

ESEMPIO 2.15

Calcolo dei ritardi. Ben Imbrogliabit deve trovare i ritardi di propagazione e contaminazione della rete mostrata nella Figura 2.70. Secondo la sua scheda tecnica, ogni porta ha un ritardo di propagazione di 100 picosecondi (ps) e un ritardo di contaminazione di 60 ps.

Anche se in questa analisi è stato ignorato il ritardo di propagazione dei fili, i circuiti digitali sono oggi così veloci che il ritardo introdotto da fili lunghi può avere la stessa importanza del ritardo delle porte logiche. Il tempo di propagazione alla velocità della luce nei fili è discusso in Appendice A.

Soluzione Per prima cosa Ben identifica il percorso critico e il percorso minimo attraverso la rete. Il percorso critico, evidenziato in rosso nella **Figura 2.71**, parte dall'ingresso A o B e attraversa tre porte fino all'uscita Y . Di conseguenza, t_{pd} è tre volte il ritardo di propagazione di una singola porta, cioè 300 ps. Invece il percorso minimo, mostrato nella **Figura 2.72**, parte dagli ingressi C , D o E e attraversa due porte fino all'uscita Y . Ci sono solo due porte sul percorso minimo, quindi t_{cd} è 120 ps.

Figura 2.70
La rete logica di Ben.

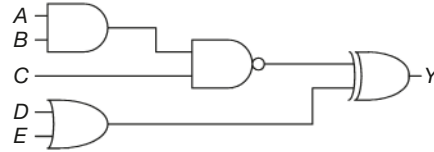


Figura 2.71
Percorso critico nella rete logica di Ben.

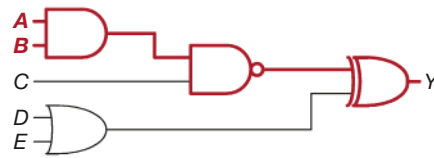
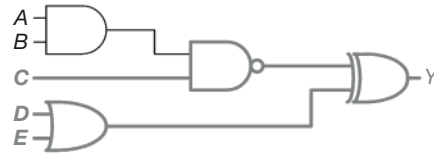


Figura 2.72
Percorso minimo nella rete logica di Ben.



ESEMPIO 2.16

Temporizzazioni dei multiplexer: criticità sul controllo e sui dati. Paragonare le temporizzazioni dei casi peggiori dei tre multiplexer a quattro ingressi riportati nella Figura 2.58 del paragrafo 2.8.1. La **Tabella 2.7** elenca i ritardi di propagazione dei componenti. Qual è il percorso critico di ogni realizzazione? Vista l'analisi delle temporizzazioni, perché sarebbe conveniente scegliere una realizzazione piuttosto di un'altra?

Soluzione Uno dei percorsi critici per ognuna delle tre realizzazioni è evidenziato in rosso nelle **Figure 2.73** e **2.74**. $t_{pd_{sy}}$ indica il ritardo di propagazione dall'ingresso S all'uscita Y , mentre $t_{pd_{dy}}$ indica il ritardo di propagazione dall'ingresso D all'uscita Y . t_{pd} è il caso peggiore: $\max(t_{pd_{sy}}, t_{pd_{dy}})$.

In entrambe le realizzazioni riportate nella Figura 2.73 (realizzazione in logica a due livelli oppure con tristate) il percorso critico è quello che va dal segnale di controllo S all'uscita Y : $t_{pd} = t_{pd_{sy}}$. Queste reti presentano dunque **criticità sul controllo**, perché il percorso critico è quello che parte dal segnale di controllo verso l'uscita. Ogni eventuale aggiunta di ritardo nei segnali di controllo verrà aggiunta direttamente nel ritardo del caso peggiore. Il ritardo da D a Y nella **Figura 2.73(b)** è di soli 50 ps, paragonato al ritardo da S a Y di 125 ps.

La Figura 2.74 mostra la realizzazione gerarchica del multiplexer 4:1 utilizzando due livelli di multiplexer 2:1. Il percorso critico è quello che parte dall'ingresso D per arrivare all'uscita. Questo circuito presenta quindi **criticità sui dati**, dal momento che il percorso critico parte da un ingresso di dato per arrivare all'uscita: $t_{pd} = t_{pd_{dy}}$.

Se gli ingressi di dato sono più veloci rispetto agli ingressi di controllo, è preferibile scegliere la realizzazione con il minor ritardo dall'ingresso di controllo all'uscita (cioè il progetto gerarchico riportato nella Figura 2.74). Analogamente, se gli ingressi di controllo sono più veloci degli ingressi di dato, è meglio scegliere la realizzazione col minor ritardo dagli ingressi di dati all'uscita (cioè il progetto a tristate della Figura 2.73(b)).

Ad ogni modo, la scelta migliore non dipende solo dal percorso critico attraverso la rete e dai tempi di arrivo degli ingressi, ma anche dalla potenza elettrica, dal costo e dai pezzi a disposizione.

Tabella 2.7 Specifiche di temporizzazione per gli elementi circuitali del multiplexer.

Porta	t_{pd} (ps)
NOT	30
AND a 2 ingressi	60
AND a 3 ingressi	80
OR a 4 ingressi	90
tristate (da A a Y)	50
tristate (dall'abilitazione a Y)	35

2.9.2 Alee

Fino ad ora sono stati esaminati casi in cui un singolo cambiamento in ingresso produce un singolo cambiamento in uscita. Tuttavia, è anche possibile che un singolo cambiamento in ingresso causi molteplici cambiamenti in uscita, denominati **alee**. Nonostante le alee spesso non creino alcun problema, è importante rendersi conto della loro esistenza e riconoscerle quando si guarda il diagramma temporale. La **Figura 2.75** mostra una rete con un'alea e la sua mappa di Karnaugh.

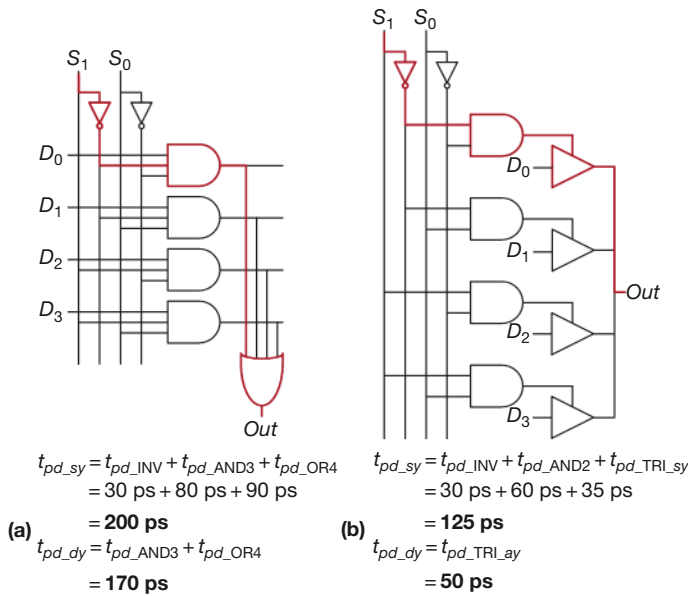


Figura 2.73
Ritardi di propagazione nel multiplexer 4:1: (a) logica a due livelli, (b) tristate.

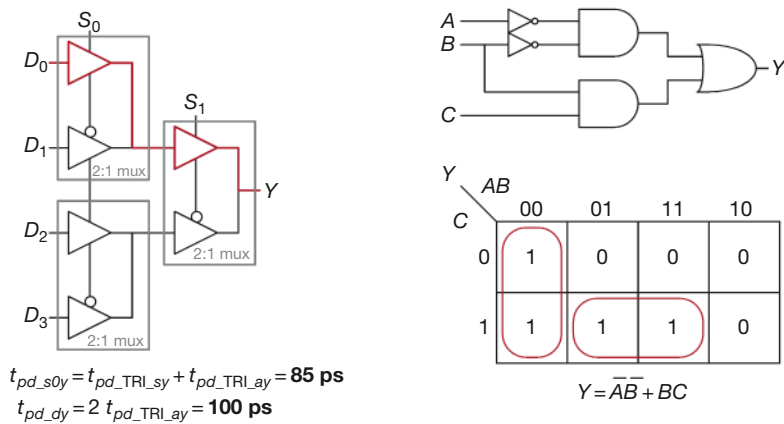


Figura 2.74
Ritardi di propagazione nel multiplexer 4:1 realizzato con gerarchia di multiplexer 2:1.

Figura 2.75
Rete logica con un'alea.

L'espressione booleana corrispondente è stata minimizzata in maniera corretta, ma si faccia attenzione nello specifico a cosa accade quando $A = 0$, $C = 1$, e B passa da 1 a 0. Questo scenario è descritto nella **Figura 2.76**: il percorso minimo (in grigio) attraversa due porte logiche, una porta AND e la porta OR. Invece il percorso critico (in rosso) attraversa un negatore oltre all'altra porta AND e alla porta OR.

Nel momento in cui B passa da 1 a 0, $n2$ (sul percorso minimo) scende a 0 prima che $n1$ (sul percorso critico) possa salire a 1. Finché $n1$ non è salito, i due ingressi della porta OR sono 0, e l'uscita Y si abbassa a 0. Quando finalmente $n1$ passa a uno, Y ritorna a sua volta a 1. Come mostrato nel diagramma temporale della Figura 2.76, il valore iniziale di Y è 1 e così anche quello finale, ma durante il processo passa momentaneamente a 0.

Le alee non rappresentano un problema se si aspetta che il ritardo di propagazione si sia esaurito prima di guardare il valore dell'uscita, perché questa prima o poi torna stabile sul risultato corretto.

Esiste comunque la possibilità di evitare l'alea aggiungendo un'altra porta alla realizzazione della funzione. Quest'ultimo concetto è più facile da comprendere facendo riferimento alla mappa di Karnaugh: la **Figura 2.77** mostra come un cambiamento a livello dell'ingresso B da $ABC = 011$ a $ABC = 001$ si traduce nello spostamento dal cerchio di un implicante primo a quello di un altro. L'attraversamento dei confini dei due implicanti primi nella mappa di Karnaugh indica la presenza di una possibile alea.

Come visto nel diagramma della Figura 2.76, se il circuito che realizza uno degli implicanti primi si spegne prima che il circuito che realizza l'altro implicante primo abbia avuto il tempo di accendersi, si verifica un'alea. Per correggere questo errore è possibile aggiungere un ulteriore cerchio che copra

Figura 2.76
Comportamento temporale di un'alea.

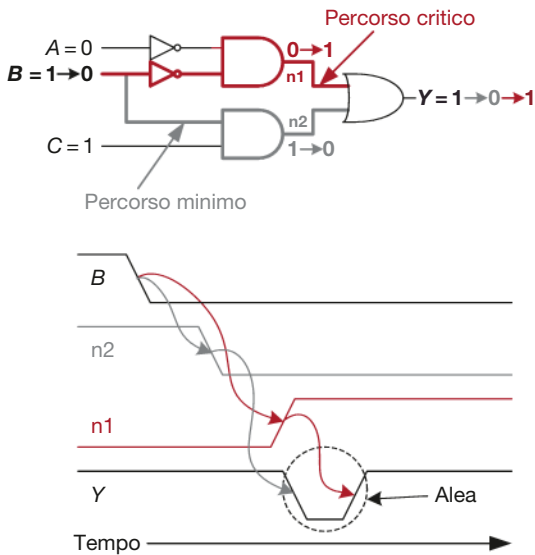
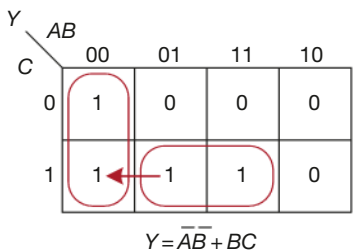


Figura 2.77
La variazione di ingresso attraversa i bordi degli implicanti.



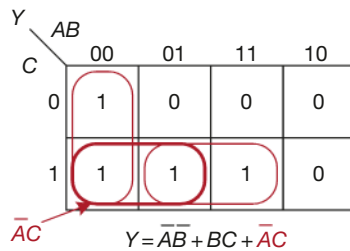


Figura 2.78
Mappa di Karnaugh priva di alee.

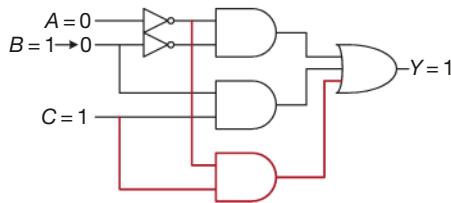


Figura 2.79
Rete logica priva di alee.

il confine dei due implicanti primi, come mostrato nella **Figura 2.78**. Si può riconoscere in quest'operazione il teorema del consenso, nel quale il termine $\overline{A}C$, che viene aggiunto, è il termine di consenso (detto anche termine ridondante).

La **Figura 2.79** mostra la rete priva di alee. La porta AND aggiunta è evidenziata in rosso. In questa rete una transizione su B quando $A = 0$ e $C = 1$ non causa nessuna alea nell'uscita in quanto la porta AND in rosso mantiene un valore 1 fisso alla sua uscita.

In generale, si presenta un'alea quando una variazione in una singola variabile di ingresso attraversa i confini di due implicanti primi in una mappa di Karnaugh. È possibile eliminare l'alea aggiungendo degli implicanti di consenso all'interno della mappa in modo tale da coprire tali confini. Ovviamente c'è un costo aggiuntivo a livello hardware per ottenere questo risultato.

Va però sottolineato che transizioni simultanee su più ingressi possono causare comunque delle alea, che non possono essere corrette con l'aggiunta di hardware. Dal momento che la maggior parte delle reti logiche di un certo interesse utilizza transizioni simultanee (o quasi), le alea sono di fatto presenti. Sebbene sia stato fornito un metodo per eliminare le alea, lo scopo di questo paragrafo non è quello di imparare a eliminarle, ma piuttosto di rendersi conto della loro presenza, che è particolarmente importante quando si studia un diagramma temporale su un simulatore o un oscilloscopio.

2.10 ■ RIASSUNTO

Una rete digitale è un modulo con ingressi e uscite a valori discreti e una specifica che descrive la funzione e la temporizzazione del modulo. In questo capitolo sono state analizzate le reti combinatorie, cioè quelle reti le cui uscite dipendono esclusivamente dai valori presenti in quel momento agli ingressi.

La funzione di una rete combinatoria può essere data da una tabella delle verità o da un'espressione booleana. L'espressione booleana relativa a una qualsiasi tabella delle verità può essere ottenuta in maniera sistematica utilizzando la forma somma di prodotti o la forma prodotto di somme. Nella sua forma somma di prodotti, l'espressione è scritta come la somma (OR) di uno o più implicanti, cioè prodotti (AND) dei letterali. I letterali, a loro volta, sono la forma diritta o negata delle variabili d'ingresso della funzione.

Le espressioni booleane possono essere semplificate utilizzando le regole dell'algebra booleana. In particolare, esse possono essere semplificate nella

loro forma minima somma di prodotti combinando tra loro gli implicant che differiscono solo di un letterale nella forma diritta e negata: $PA + P\bar{A} = P$. Le mappe di Karnaugh sono strumenti grafici utilizzati per minimizzare espressioni che hanno fino a quattro variabili. Con la pratica, un progettista può semplificare espressioni a poche variabili anche solo grazie all'esame del suo comportamento; per le funzioni più complesse si utilizzano gli strumenti di progettazione assistita da calcolatore (*computer-aided design tool*) analizzati nel Capitolo 4.

Le porte logiche sono connesse tra loro per creare reti combinatorie che eseguono la funzione richiesta. Una qualsiasi funzione in forma somma di prodotti può essere costruita utilizzando la logica a due livelli: in particolare, le porte NOT formano i complementi degli ingressi, le porte AND i prodotti e le porte OR formano la somma. A seconda della funzione desiderata e dei blocchi costitutivi disponibili, le realizzazioni in logica a più livelli con diversi tipi di porte possono essere più efficienti. Per esempio, come già detto, le reti CMOS favoriscono le porte NAND e NOR perché queste ultime possono essere costruite direttamente dai transistori CMOS senza necessitare di porte NOT in più. Quando si utilizzano le porte NAND e NOR, il fenomeno dello spostamento delle bolle è particolarmente utile per tenere traccia delle negazioni.

Le porte logiche vengono inoltre combinate per creare reti più grandi, come i multiplexer, i decoder e le reti a priorità. Un multiplexer ha la capacità di scegliere un ingresso di dato basandosi su un ingresso di selezione, un decoder attiva una delle uscite a 1 in base alla configurazione presente agli ingressi, mentre una rete a priorità produce un'uscita che indica l'ingresso a priorità più alta. Queste reti rappresentano esempi di blocchi costitutivi combinatori. Nel Capitolo 5 si introducono altri blocchi costitutivi, compresi alcuni circuiti aritmetici. Questi blocchi costitutivi vengono usati per la costruzione del microprocessore nel Capitolo 7.

La specifica di temporizzazione di una rete combinatoria consiste nei ritardi di propagazione e di contaminazione attraverso la rete, che indicano rispettivamente il tempo più lungo e quello più corto tra un cambiamento di un ingresso della rete e il cambiamento dell'uscita che ne consegue. Calcolare il ritardo di propagazione di una rete implica l'individuazione del percorso critico attraverso il circuito, e successivamente la somma dei vari ritardi di propagazione di ogni elemento lungo il percorso. Ci sono diversi modi per realizzare complesse reti combinatorie che offrono buoni compromessi tra la velocità della rete e il suo costo.

Nel prossimo capitolo si affrontano le reti logiche sequenziali, nelle quali le uscite dipendono sia dai valori presenti in quel momento agli ingressi sia dai loro valori passati: in altre parole, le reti sequenziali possiedono una memoria dei valori passati.

Esercizi

Esercizio 2.1 Scrivere l'espressione booleana della forma canonica di tipo somma di prodotti per ciascuna delle tabelle delle verità riportate nella Figura 2.80.

(a)	(b)	(c)	(d)	(e)
A B Y	A B C Y	A B C Y	A B C D Y	A B C D Y
0 0 1	0 0 0 1	0 0 0 1	0 0 0 0 1	0 0 0 0 1
0 1 0	0 0 1 0	0 0 1 0	0 0 0 1 1	0 0 0 1 0
1 0 1	0 1 0 0	0 1 0 1	0 0 1 0 1	0 0 1 0 0
1 1 1	0 1 1 0	0 1 1 0	0 0 1 1 1	0 0 1 1 1
	1 0 0 0	1 0 0 1	0 1 0 0 0	0 1 0 0 0
	1 0 1 0	1 0 1 1	0 1 0 1 0	0 1 0 1 1
	1 1 0 0	1 1 0 0	0 1 1 0 0	0 1 1 0 1
	1 1 1 1	1 1 1 1	0 1 1 1 0	0 1 1 1 0
			1 0 0 0 1	1 0 0 0 0
			1 0 0 1 0	1 0 0 1 1
			1 0 1 0 1	1 0 1 0 1
			1 0 1 1 0	1 0 1 1 0
			1 1 0 0 0	1 1 0 0 1
			1 1 0 1 0	1 1 0 1 0
			1 1 1 0 1	1 1 1 0 0
			1 1 1 1 0	1 1 1 1 1

Figura 2.80 Tabelle delle verità per gli Esercizi 2.1 e 2.3.

Esercizio 2.2 Scrivere l'espressione booleana della forma canonica di tipo somma di prodotti per ciascuna delle tabelle delle verità riportate nella Figura 2.81.

(a)	(b)	(c)	(d)	(e)
A B Y	A B C Y	A B C Y	A B C D Y	A B C D Y
0 0 0	0 0 0 0	0 0 0 0	0 0 0 0 1	0 0 0 0 0
0 1 1	0 0 1 1	0 0 1 1	0 0 0 1 0	0 0 0 1 0
1 0 1	0 1 0 1	0 1 0 0	0 0 1 0 1	0 0 1 0 0
1 1 1	0 1 1 1	0 1 1 0	0 0 1 1 1	0 0 1 1 1
	1 0 0 1	1 0 0 0	0 1 0 0 0	0 1 0 0 0
	1 0 1 0	1 0 1 0	0 1 0 1 0	0 1 0 1 0
	1 1 0 1	1 1 0 1	0 1 1 0 1	0 1 1 0 1
	1 1 1 0	1 1 1 1	0 1 1 1 1	0 1 1 1 1
			1 0 0 0 1	1 0 0 0 1
			1 0 0 1 0	1 0 0 1 1
			1 0 1 0 1	1 0 1 0 1
			1 0 1 1 0	1 0 1 1 1
			1 1 0 0 0	1 1 0 0 0
			1 1 0 1 0	1 1 0 1 0
			1 1 1 0 0	1 1 1 0 0
			1 1 1 1 0	1 1 1 1 0

Figura 2.81 Tabelle delle verità per gli Esercizi 2.2 e 2.4.

Esercizio 2.3 Scrivere l'espressione booleana della forma canonica di tipo prodotto di somme per ciascuna delle tabelle delle verità riportate nella Figura 2.80.

Esercizio 2.4 Scrivere l'espressione booleana della forma canonica di tipo prodotto di somme per ciascuna delle tabelle delle verità riportate nella Figura 2.81.

Esercizio 2.5 Minimizzare le espressioni booleane ricavate nell'Esercizio 2.1.

Esercizio 2.6 Minimizzare le espressioni booleane ricavate nell'Esercizio 2.2.

Esercizio 2.7 Disegnare una rete combinatoria ragionevolmente semplice per ciascuna delle espressioni booleane ricavate nell'Esercizio 2.5. Ragionevolmente semplice significa che non si devono usare troppe porte logiche, ma neppure passare troppo tempo a valutare tutte le possibili soluzioni.

Esercizio 2.8 Disegnare una rete combinatoria ragionevolmente semplice per ciascuna delle espressioni booleane ricavate nell'Esercizio 2.6.

Esercizio 2.9 Ripetere l'Esercizio 2.7 utilizzando solo porte NOT, AND e OR.

Esercizio 2.27 Ripetere l'Esercizio 2.26 per la rete rappresentata nella Figura 2.84.

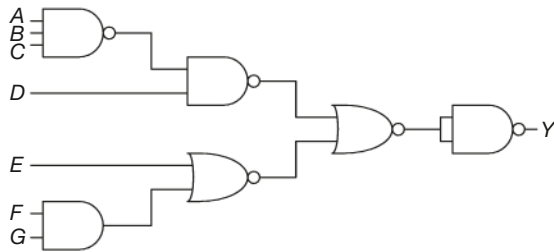


Figura 2.84 Schema della rete logica.

Esercizio 2.28 Trovare l'espressione booleana minima della funzione la cui tabella delle verità è riportata nella Figura 2.85, sfruttando al meglio le indifferenze.

A	B	C	D	Y
0	0	0	0	X
0	0	0	1	X
0	0	1	0	X
0	0	1	1	0
0	1	0	0	0
0	1	0	1	X
0	1	1	0	0
0	1	1	1	X
1	0	0	0	1
1	0	0	1	0
1	0	1	0	X
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	X
1	1	1	1	1

Figura 2.85 Tabella delle verità per l'Esercizio 2.28.

Esercizio 2.29 Disegnare la rete logica relativa all'Esercizio 2.28.

Esercizio 2.30 Ci sono potenziali alee nella rete logica dell'Esercizio 2.29? Se no, motivare il perché. Se sì, indicare come modificare la rete per eliminarle.

Esercizio 2.31 Trovare l'espressione booleana minima della funzione la cui tabella delle verità è riportata nella Figura 2.86, sfruttando al meglio le indifferenze.

A	B	C	D	Y
0	0	0	0	0
0	0	0	1	1
0	0	1	0	X
0	0	1	1	X
0	1	0	0	0
0	1	0	1	X
0	1	1	0	X
0	1	1	1	X
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	X
1	1	1	1	1

Figura 2.86 Tabella delle verità per l'Esercizio 2.31.

Esercizio 2.32 Disegnare la rete logica relativa all'Esercizio 2.31.

Esercizio 2.33 A Ben Imbrogliabit piacciono i picnic nei giorni di sole e senza formiche nei paraggi, ma anche quando riesce a vedere un usignolo, o quando ci sono sì le formiche ma anche le coccinelle. Scrivere l'espressione booleana che dà il Piacere (P) di Ben in funzione del Sole (S), delle Formiche (F), degli Usignoli (U) e delle Coccinelle (C).

Esercizio 2.34 Completare il progetto del transcodificatore a 7 segmenti per i segmenti da S_c a S_g (fare riferimento all'Esempio 2.10).

- Ricavare l'espressione booleana per i segmenti da S_c a S_g nell'ipotesi che configurazioni di ingresso maggiori di 9 siano associate a uscite spente (cioè a zero).
- Ricavare l'espressione booleana per i segmenti da S_c a S_g nell'ipotesi che configurazioni di ingresso maggiori di 9 siano associate a indifferenze.
- Disegnare una rete logica ragionevolmente semplice per le espressioni di cui al punto (b) ricordando che le varie uscite possono condividere le stesse porte logiche quando ciò è conveniente.

Esercizio 2.35 Una rete logica ha quattro ingressi e due uscite. Gli ingressi $A_{3:0}$ rappresentano un numero compreso tra 0 e 15, l'uscita P deve assumere valore VERO se il numero è Primo (si noti che 0 e 1 non sono primi, mentre lo sono 2, 3, 5 e così via) e l'uscita D deve assumere valore VERO se il numero è Divisibile per 3. Minimizzare le espressioni booleane per entrambe le uscite e disegnare la rete logica risultante.

Esercizio 2.36 Un *priority encoder* (codificatore a priorità) ha 2^N ingressi e genera in uscita un numero binario a N bit che indica qual è il bit più significativo degli ingressi che vale 1, oppure 0 se nessuno dei bit di ingresso vale 1. Inoltre genera il bit di uscita NO che vale 1 se nessuno dei bit di ingresso vale 1. Progettare un *priority encoder* a 8 bit, con gli ingressi $A_{7:0}$ e le uscite $Y_{2:0}$ e NO . Per esempio, se gli ingressi sono 00100000, le uscite Y devono dare 101 e l'uscita NO deve dare 0. Minimizzare ogni espressione booleana e disegnare la rete logica che ne deriva.

Esercizio 2.37 Progettare un *priority encoder* modificato (vedi l'Esercizio 2.36) con gli ingressi $A_{7:0}$ e le uscite $Y_{2:0}$ e $Z_{2:0}$. Le uscite Y indicano la posizione dell'1 più significativo nella configurazione di ingresso, e valgono 0 se nessun ingresso vale 1. Le uscite Z indicano la posizione del secondo 1 più significativo nella configurazione di ingresso, e valgono 0 se non c'è più di un ingresso a 1. Minimizzare ogni espressione booleana e disegnare la rete logica che ne deriva.

Esercizio 2.38 Il codice termometrico a M bit del numero k è costituito da k uni nelle posizioni meno significative e da $M-k$ zeri nelle posizioni più significative. Un transcodificatore binario-termometrico ha N ingressi e $2^N - 1$ uscite, e genera la codifica termometrica a $2^N - 1$ bit del numero che riceve in ingresso. Per esempio, se il numero in ingresso è 110, l'uscita generata è 0111111. Progettare un transcodificatore binario-termometrico 3:7, minimizzare ogni espressione booleana e disegnare la rete logica che ne deriva.

Esercizio 2.39 Scrivere l'espressione booleana minima per la funzione associata alla rete logica rappresentata nella Figura 2.87.

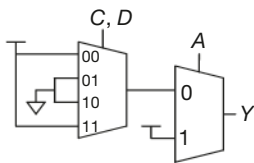


Figura 2.87 Rete a multiplexer.

Esercizio 2.40 Scrivere l'espressione booleana minima per la funzione associata alla rete logica rappresentata nella Figura 2.88.

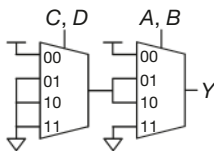


Figura 2.88 Rete a multiplexer.

Esercizio 2.41 Realizzare la funzione della Figura 2.80(b) utilizzando

- (a) un multiplexer 8:1
- (b) un multiplexer 4:1 e un negatore
- (c) un multiplexer 2:1 e due altre porte logiche

Esercizio 2.42 Realizzare la funzione dell'Esercizio 2.17(a) utilizzando

- (a) un multiplexer 8:1
- (b) un multiplexer 4:1 e nessun'altra porta logica
- (c) un multiplexer 2:1, una porta OR e un negatore

Esercizio 2.43 Calcolare i ritardi di propagazione e di contaminazione della rete rappresentata nella Figura 2.83, facendo uso dei valori riportati nella Tabella 2.8.

Tabella 2.8 Ritardi delle porte per gli Esercizi da 2.43 a 2.47.

Porta	t_{pd} (ps)	t_{cd} (ps)
NOT	15	10
NAND a 2 ingressi	20	15
NAND a 3 ingressi	30	25
NOR a 2 ingressi	30	25
NOR a 3 ingressi	45	35
AND a 2 ingressi	30	25
AND a 3 ingressi	40	30
OR a 2 ingressi	40	30
OR a 3 ingressi	55	45
XOR a 2 ingressi	60	40

Esercizio 2.44 Calcolare i ritardi di propagazione e di contaminazione della rete rappresentata nella Figura 2.84, facendo uso dei valori riportati nella Tabella 2.8.

Esercizio 2.45 Disegnare la rete logica di un decoder 3:8 veloce, utilizzando le porte logiche e i tempi riportati nella Tabella 2.8. Minimizzare il percorso critico, indicarlo nello schema e calcolare i ritardi di propagazione e di contaminazione.

Esercizio 2.46 Progettare un multiplexer 8:1 con il minimo ritardo possibile tra ingressi e uscite di dato, utilizzando le porte logiche riportate nella Tabella 2.7. Disegnare la rete logica e calcolare tale ritardo minimo.

Esercizio 2.47 Riprogettare la rete dell'Esercizio 2.35 per renderla il più veloce possibile, utilizzando le porte logiche e i tempi riportati nella Tabella 2.8. Disegnare la nuova rete indicando il percorso critico e calcolare i ritardi di propagazione e di contaminazione.

Esercizio 2.48 Riprogettare il priority encoder dell'Esercizio 2.36 per renderlo il più veloce possibile, utilizzando le porte logiche e i tempi riportati nella Tabella 2.8. Disegnare la nuova rete indicando il percorso critico e calcolare i ritardi di propagazione e di contaminazione.

Domande di valutazione

Queste domande sono state poste a candidati per un posto di lavoro nell'ambito della progettazione di sistemi digitali.

Domanda 2.1 Disegni lo schema di una porta XOR a due ingressi utilizzando solo porte NAND. Qual è il minimo numero di porte che deve utilizzare?

Domanda 2.2 Progetti una rete logica capace di segnalare se un certo mese ha 31 giorni. Il mese viene indicato dai quattro bit di ingresso $A_{3:0}$ (per es., gli ingressi 0001 indicano il mese di gennaio, gli ingressi 1100 il mese di dicembre). L'uscita Y assume valore ALTO se il mese specificato ha 31 giorni. Scriva l'espressione logica ottimizzata della rete e la disegni usando il minimo numero di porte logiche (ricordi che può utilizzare al meglio le indifferenze).

Domanda 2.3 Cos'è un buffer tristate? Quando e perché è utile?

Domanda 2.4 Una porta logica (o un insieme di porte logiche) viene detta "universale" se può essere utilizzata per realizzare qualsiasi espressione booleana. Per esempio, l'insieme di porte logiche {AND, OR, NOT} è universale.

- (a) La porta AND da sola è universale? Perché o perché no?
- (b) L'insieme di porte {OR, NOT} è universale? Perché o perché no?
- (c) La porta NAND da sola è universale? Perché o perché no?

Domanda 2.5 Spieghi perché il ritardo di contaminazione di un circuito digitale può essere minore (e non uguale) al ritardo di propagazione.

Progetto di logica sequenziale

Capitolo

3

3.1 Introduzione

3.2 Latch e flip-flop

3.3 Progetto di reti logiche
sincrone

3.4 Macchine a stati finiti

3.5 Temporizzazione della logica
sequenziale

3.6 Parallelismo

3.7 Riassunto

3.1 ■ INTRODUZIONE

Nel capitolo precedente è stato descritto come analizzare e progettare la logica combinatoria. L'uscita di una logica combinatoria dipende esclusivamente dai valori presenti in quel momento agli ingressi. Date le specifiche in forma di tabella delle verità o di espressione booleana, è possibile creare e ottimizzare una rete che esegua quanto richiesto dalle suddette specifiche.

In questo capitolo viene invece illustrato come analizzare e progettare la **logica sequenziale**, le cui uscite dipendono tanto dai valori presenti in quel momento agli ingressi quanto dai valori precedenti. Di conseguenza, si può dire che la logica sequenziale ha una memoria. La logica sequenziale può ricordare esplicitamente alcuni ingressi precedenti, oppure può avere la capacità di riassumere gli ingressi precedenti in quantità più piccole di informazioni chiamate **stati** del sistema. Lo stato di una rete digitale sequenziale è un insieme di bit, detto **variabili di stato**, che contiene tutte le informazioni sul passato necessarie a spiegare il comportamento futuro della rete.

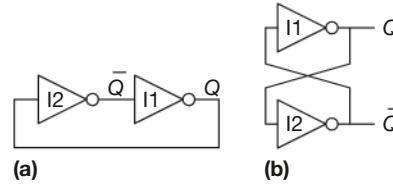
Il capitolo inizia con lo studio dei latch e dei flip-flop: si tratta di semplici reti sequenziali che ricordano un bit di stato. In generale, le reti sequenziali sono complesse da analizzare. Per semplificarne la progettazione è consigliabile costruire solo reti sequenziali sincrone composte da logica combinatoria e banchi di flip-flop contenenti lo stato della rete. Il capitolo descrive macchine a stati finiti, che rappresentano un modo semplice di progettare reti sequenziali. Infine, viene analizzata la velocità delle reti sequenziali e viene introdotto il parallelismo come metodo per aumentare tale velocità.

3.2 ■ LATCH E FLIP-FLOP

Il blocco costitutivo fondamentale di memoria è un elemento **bistabile**, e cioè un elemento con due stati stabili. La **Figura 3.1(a)** mostra un semplice elemento bistabile composto da una coppia di negatori (o *inverter*) connessi ad anello. La stessa rete viene ridisegnata nella **Figura 3.1(b)** per sottolinearne la simmetria. I due negatori sono collegati a croce, nel senso che l'ingresso di I1

Software applicativo	
Sistema operativo	
Architettura	
Micro-architettura	
Blocchi logici	
Reti digitali	
Reti analogiche	
Dispositivi	
Fisica	

Figura 3.1
Coppie di negatori collegati a croce.



Mentre la lettera Y viene solitamente usata per indicare l'uscita di una rete combinatoria, l'uscita di una rete sequenziale viene in genere denominata Q .

è l'uscita di I2 e viceversa. La rete non ha ingressi, ma possiede due uscite, Q e $\bar{Q}$. L'analisi di una rete sequenziale differisce dall'analisi di una rete combinatoria perché la prima è ciclica: Q dipende da $\bar{Q}$, e $\bar{Q}$ dipende a sua volta da Q . Si considerino i due casi in cui Q è 0 oppure in cui Q è 1. Se si analizzano le conseguenze di questi due casi, si arriva alle seguenti conclusioni:

- **Caso I: $Q = 0$**

Come mostrato nella **Figura 3.2(a)**, I2 riceve un ingresso FALSO e, di conseguenza, produce un'uscita VERA su $\bar{Q}$. Invece, I1 riceve un ingresso VERO, $\bar{Q}$, e produce a sua volta un'uscita FALSA su Q . Dal momento che viene confermata la premessa iniziale, cioè $Q = 0$, questo caso viene detto **stabile**.

- **Caso II: $Q = 1$**

Come mostrato nella **Figura 3.2 (b)**, I2 riceve un ingresso VERO e produce un'uscita FALSA su $\bar{Q}$. A sua volta I1 riceve un ingresso FALSO e produce un'uscita VERA su Q . Anche questa volta si può dire che il caso è stabile.

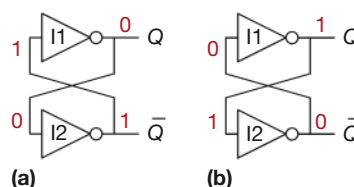
Dal momento che i negatori collegati a croce hanno due stati stabili, $Q = 0$ e $Q = 1$, il circuito viene chiamato bistabile. Tuttavia, esiste la possibilità che la rete abbia un terzo possibile stato, cioè quando entrambe le uscite hanno un valore approssimativamente a metà tra 0 e 1. Quest'ultimo viene detto stato **metastabile** e verrà trattato nel paragrafo 3.5.4.

Un elemento con un numero N di stati stabili è caratterizzato da $\log_2 N$ bit di informazione, quindi un elemento bistabile immagazzina un bit di informazione. Lo stato di una coppia di negatori collegati a croce viene rappresentato da una variabile binaria di stato, Q . Il valore di Q contiene tutte le informazioni sul passato necessarie a spiegare il comportamento futuro della rete. Nello specifico, se $Q = 0$, il valore rimarrà sempre zero, mentre se $Q = 1$, il valore rimarrà sempre 1. La rete ha un secondo nodo, $\bar{Q}$, ma questo non contiene nessuna informazione aggiuntiva, dal momento che conoscere il valore di Q permette di sapere anche il valore di $\bar{Q}$. D'altra parte, anche $\bar{Q}$ può essere una scelta accettabile come variabile di stato.

Quando una rete sequenziale viene accesa, lo stato iniziale è sconosciuto e solitamente imprevedibile, e può essere diverso a ogni nuova accensione della rete.

Anche se due negatori collegati a croce sono in grado di immagazzinare un bit di informazione, non sono molto utili, dal momento che l'utente non ha a disposizione un ingresso che gli permetta di controllare lo stato. Invece, altri elementi bistabili, come ad esempio i latch e i flip-flop, sono provvisti di ingressi per il controllo del valore della variabile di stato. Nella seconda parte di questo paragrafo vengono quindi considerate queste reti.

Figura 3.2
Comportamento bistabile di due negatori collegati a croce.



3.2.1 Latch SR

Il latch SR rappresenta una delle reti sequenziali più semplici ed è composto da due porte NOR collegate a croce, come mostra la **Figura 3.3**. Il latch ha due ingressi, S e R , e due uscite, Q e $\bar{Q}$. Il latch SR è simile ai negatori collegati a croce ma il suo stato può essere controllato mediante gli ingressi, S e R , che attivano (o “settano”, *set*) e disattivano (o “resettano”, *reset*) l’uscita Q .

Un buon metodo per comprendere una rete sconosciuta è lavorare sulla sua tabella delle verità: una porta NOR, come già spiegato, produce un’uscita FALSA quando almeno uno dei suoi ingressi è VERO. A questo punto si considerino le quattro possibili combinazioni di R e S .

- **Caso I:** $R = 1, S = 0$
N1 vede almeno un ingresso VERO, R , quindi produce un’uscita FALSA su Q . N2 vede sia Q sia S come FALSI, quindi produce un’uscita VERA su $\bar{Q}$.
- **Caso II:** $R = 0, S = 1$
N1 riceve come ingressi 0 e $\bar{Q}$. Dal momento che il valore di $\bar{Q}$ a questo punto è ancora sconosciuto, non è possibile determinare che valore assume Q . N2 riceve almeno un ingresso VERO, S , quindi produce un’uscita FALSA su $\bar{Q}$. A questo punto è possibile tornare a N1 e, sapendo che entrambi gli ingressi sono FALSI, calcolare il valore dell’uscita Q , che è VERO.
- **Caso III:** $R = 1, S = 1$
N1 e N2 vedono entrambi almeno un ingresso VERO (R oppure S), quindi entrambi producono un’uscita FALSA. Di conseguenza, sia Q sia $\bar{Q}$ sono FALSE.
- **Caso IV:** $R = 0, S = 0$
N1 riceve gli ingressi 0 e $\bar{Q}$. Dal momento che il valore di $\bar{Q}$ è sconosciuto, non è possibile determinare il valore dell’uscita. N2 riceve a sua volta gli ingressi 0 e Q , ma visto che anche il valore di Q è sconosciuto, anche in questo caso non è possibile determinare il valore dell’uscita. Il problema è lo stesso dei negatori collegati a croce. Si sa che il valore di Q deve essere 0 oppure 1 , quindi è possibile risolvere il problema indagando le conseguenze delle due possibilità.
- **Caso IV(a):** $Q = 0$
Dal momento che S e Q sono entrambe FALSE, N2 produce un’uscita VERA su $\bar{Q}$, come mostra la **Figura 3.4(a)**. A questo punto N1 riceve almeno un ingresso VERO, $\bar{Q}$, quindi la sua uscita Q è FALSA, come era stato presupposto.
- **Caso IV(b):** $Q = 1$
Se Q è VERA, N2 produce un’uscita FALSA su $\bar{Q}$, come mostrato nella **Figura 3.4(b)**. Ora N1 riceve due ingressi FALSI, R e $\bar{Q}$, quindi la sua uscita, Q , è VERA, come da presupposto.

Per riassumere, si supponga che Q assuma un valore precedente noto, denominato Q_{prec} prima di arrivare al caso IV. Q_{prec} può essere 0 o 1 , e rappresenta lo stato del sistema. Quando R e S assumono valore 0 , Q tiene memoria di tale valore precedente Q_{prec} e $\bar{Q}$ è il suo complemento, $\bar{Q}_{prec}$. Questa rete ha dunque una memoria.

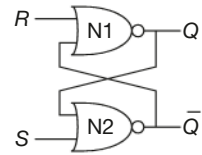


Figura 3.3
Schema del latch SR.

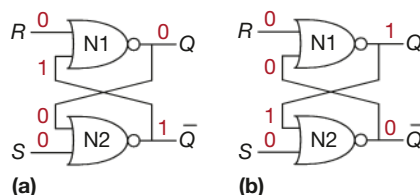


Figura 3.4
I due stati stabili del latch SR (bistabile).

Caso	S	R	Q	$\bar{Q}$
IV	0	0	Q_{prec}	$\bar{Q}_{prec}$
I	0	1	0	1
II	1	0	1	0
III	1	1	0	0

Figura 3.5
Tabella delle verità del latch SR.

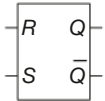


Figura 3.6
Simbolo circuitale del latch SR.

La tabella delle verità nella **Figura 3.5** riassume i casi appena visti. Gli ingressi S e R significano rispettivamente *Set* e *Reset*. Settare un bit (*set*) significa fargli assumere il valore VERO, resettarlo (*reset*) significa fargli assumere il valore FALSO. Le uscite, Q e $\bar{Q}$, sono di norma una il complemento dell'altra: quando viene attivato R , Q viene resettata a 0 e $\bar{Q}$ reagisce in modo opposto. Al contrario, quando viene attivato S , Q è settata a 1 e $\bar{Q}$ reagisce in modo opposto. Quando nessuno dei due ingressi è attivato, Q ricorda il suo valore precedente, che è stato chiamato Q_{prec} . Attivare simultaneamente entrambi gli ingressi, S e R , non ha molto senso, in quanto il latch dovrebbe, in questo caso, settarsi e resettarsi allo stesso tempo, il che non è possibile. In questo caso la rete, non riuscendo a funzionare correttamente, darebbe come risposta 0 in entrambe le uscite.

Il latch SR viene rappresentato con il simbolo riportato nella **Figura 3.6**. L'utilizzo di questo simbolo altro non è che una messa in pratica dei già noti principi di astrazione e modularità. Esistono diversi metodi di costruzione di un latch SR, come ad esempio l'utilizzo di porte logiche differenti oppure di transistori. In ogni caso, qualsiasi elemento circuitale che abbia le relazioni specificate dalla tabella delle verità della Figura 3.5 e il simbolo riportato nella Figura 3.6 viene chiamato latch SR.

Come i negatori collegati a croce, anche il latch SR è un elemento bistabile con un bit di stato immagazzinato in Q . In questo caso però lo stato può essere controllato attraverso gli ingressi S e R . Quando viene attivato R , lo stato viene resettato a 0. Quando viene attivato S , lo stato viene settato a 1. Quando nessuno degli ingressi è attivato, lo stato mantiene il suo valore precedente. Si noti che è possibile tenere traccia della storia degli ingressi tramite la singola variabile di stato Q : indipendentemente da quale sia stata la sequenza di valori di ingresso nel passato, l'unica cosa necessaria per prevedere il comportamento futuro del latch SR è il fatto che l'ultima operazione sia stata un set o un reset.

3.2.2 Latch D

Il latch SR è scomodo in quanto si comporta in maniera imprevedibile quando entrambi gli ingressi, S e R , sono attivati simultaneamente. Inoltre, gli ingressi S e R combinano gli aspetti del "come" e del "quando". Attivare uno degli ingressi, infatti, determina non solo "come" debba diventare lo stato ma anche "quando" questo debba cambiare. La progettazione delle reti diventa più semplice se i due aspetti del "come" e del "quando" sono separati: il latch D rappresentato nella **Figura 3.7(a)** è una soluzione a questo problema. Questo latch ha due ingressi: un ingresso **dati**, D , che controlla il prossimo stato, e un ingresso **clock**, CLK , che controlla invece il momento del cambio di stato.

Di nuovo, per analizzare questo latch si parte dalla sua tabella delle verità, riportata nella **Figura 3.7(b)**. Per comodità, si considerino per primi i nodi interni $\bar{D}$, S e R . Quando $CLK = 0$, sia S sia R sono FALSI, indipendentemente dal valore assunto da D . Se invece $CLK = 1$, allora una porta AND produce un valore VERO e l'altra un valore FALSO, a seconda del valore di D . La Figura 3.5 mostra come, dati i valori di S e di R , sia possibile determinare Q e $\bar{Q}$. Si osservi che, quando $CLK = 0$, Q ricorda il suo valore precedente, Q_{prec} , mentre quando $CLK = 1$, $Q = D$. In ogni caso, $\bar{Q}$ resta, come logico, il complemento di Q . Il latch D è in grado di evitare il caso anomalo in cui gli ingressi S e R vengano attivati simultaneamente.

Per riassumere, si è visto come il clock sia in grado di controllare quando i dati scorrono attraverso il latch. Quando $CLK = 1$, il latch è detto **trasparente**. I dati scorrono da D verso Q , come se il latch fosse un buffer. Quando invece $CLK = 0$, il latch è opaco: viene bloccato il passaggio dei dati verso Q , che mantiene il valore precedente. Per questo motivo, il latch D viene chiamato

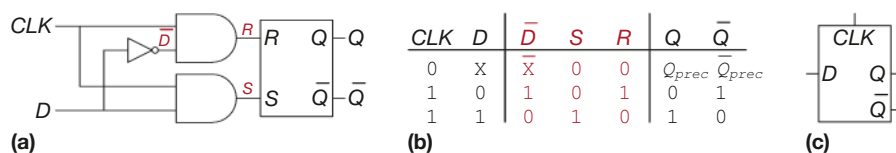


Figura 3.7 Latch D: (a) schema, (b) tabella delle verità, (c) simbolo circuitale.

latch trasparente o **latch sensibile ai livelli** (*level-sensitive*). Il simbolo del latch D è riportato nella **Figura 3.7(c)**.

Il latch D aggiorna continuamente il suo stato mentre $CLK = 1$. Più avanti nel capitolo si vedrà come sia invece utile aggiornare lo stato solo in un dato momento. Nel prossimo paragrafo viene descritto il flip-flop D, che consente proprio questa funzione.

3.2.3 Flip-flop D

Un **flip-flop D** può essere costruito a partire da due latch D in cascata controllati da due segnali di clock complementari, come mostra la **Figura 3.8(a)**. Il primo latch, L1, viene detto **master**, mentre il secondo latch, L2, viene detto **slave**. Il nodo che unisce i due latch prende il nome N1. Il simbolo circuitale del flip-flop D è riportato nella **Figura 3.8(b)**. Quando l'uscita $\bar{Q}$ non è necessaria, il simbolo viene spesso semplificato come nella **Figura 3.8(c)**.

Quando $CLK = 0$, il latch master è trasparente, mentre il latch slave è opaco. Di conseguenza, qualsiasi valore di D viene portato a N1. Quando invece $CLK = 1$, il latch master diventa opaco e quello slave trasparente. In questo caso, il valore di N1 viene trasmesso a Q , ma N1 resta isolato da D . Quindi, qualunque sia il valore di D subito prima del fronte di salita (passaggio da 0 a 1) del clock, questo è il valore che viene trasferito a Q al momento di tale fronte. In tutti gli altri casi, Q mantiene il suo valore precedente, dal momento che c'è sempre un latch opaco che blocca il passaggio di dati tra D e Q .

In altre parole, un flip-flop D copia D su Q al fronte di salita del clock, e ricorda il suo stato in tutti gli altri casi. Si consiglia al lettore di rileggere questa definizione fino a memorizzarla: uno dei problemi più comuni per un progettista digitale inesperto è appunto dimenticarsi quale sia la funzione dei flip-flop. Il fronte di salita del clock viene spesso chiamato con l'abbreviazione "fronte del clock" (*clock edge*). L'ingresso D specifica quale sarà il nuovo stato, mentre il fronte del clock indica il momento di aggiornamento dello stato.

Un flip-flop D viene anche detto **flip-flop master-slave**, o anche **flip-flop attivato sui fronti** (*edge triggered*), o ancora **flip-flop attivato sui fronti di salita**. Il triangolo nei simboli circuitali indica un ingresso di clock attivo sui fronti. L'uscita $\bar{Q}$ viene spesso tralasciata se non necessaria.

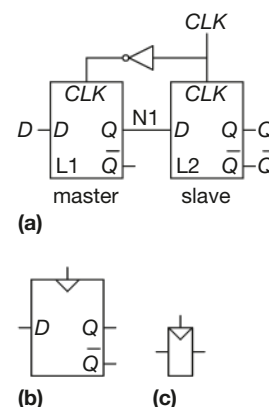


Figura 3.8 Flip-flop D: (a) schema, (b) simbolo circuitale, (c) simbolo circuitale semplificato.

La distinzione fra flip-flop e latch non è così rigorosa, e nel tempo ha anche subito evoluzioni. Nel linguaggio comune dei progettisti, si definisce flip-flop un bistabile edge triggered, cioè un bistabile con un segnale di clock che modifica il proprio stato su un fronte (*edge*) del clock stesso, ovvero durante il passaggio da 0 a 1 (oppure da 1 a 0) del clock. Bistabili non edge triggered vengono definiti latch. E sempre nel linguaggio dei progettisti si fa riferimento implicitamente ai bistabili di tipo D, largamente i più usati nella pratica.

ESEMPIO 3.1

Conteggio dei transistori in un flip-flop. Quanti transistori servono per costruire il flip-flop D descritto nel paragrafo precedente?

Soluzione Una porta NAND o una porta NOR richiedono quattro transistori, mentre una porta NOT ne richiede solo due. Una porta AND viene costruita con una porta NAND e una NOT e utilizza in totale sei transistori. Il latch SR utilizza due porte NOR, quindi otto transistori. Un latch D è formato da un latch SR, due porte AND e una porta NOT e utilizza quindi 22 transistori. Il flip-flop D è formato da due latch D e una porta NOT, quindi utilizza in totale 46 transistori. Nel paragrafo 3.2.7 viene descritta una realizzazione più efficiente in tecnologia CMOS che utilizza le porte di trasmissione.

3.2.4 Registro

Un registro a N bit è un banco di N flip-flop che condividono un ingresso CLK comune, in modo che tutti i bit vengano aggiornati allo stesso tempo. I registri costituiscono i blocchi costitutivi chiave per la maggior parte delle reti sequenziali. Nella **Figura 3.9** vengono mostrati lo schema e il simbolo per un registro a quattro bit che possiede un ingresso $D_{3:0}$ e un'uscita $Q_{3:0}$. Sia $D_{3:0}$ sia $Q_{3:0}$ sono bus a quattro bit.

3.2.5 Flip-flop con abilitazione

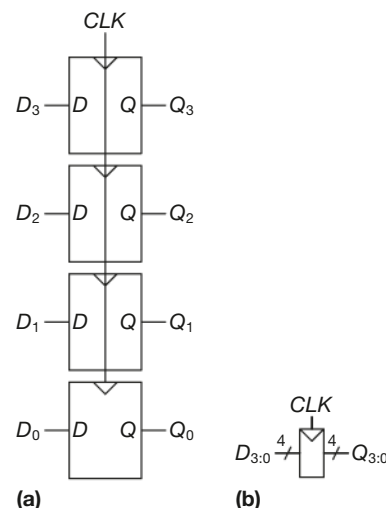
Un flip-flop con abilitazione aggiunge un altro ingresso, chiamato EN o $ENABLE$, per determinare se memorizzare o no il dato sul fronte del clock. Quando EN è VERO, il flip-flop con abilitazione reagisce come un normale flip-flop D; quando invece EN è FALSO, il flip-flop con abilitazione ignora il clock e mantiene il proprio stato. I flip-flop con abilitazione sono utili quando si desidera inserire un nuovo valore in un flip-flop esclusivamente in alcuni precisi momenti, piuttosto che a ogni cambio del clock.

La **Figura 3.10** mostra due possibili metodi di costruzione di un flip-flop con abilitazione a partire da un flip-flop D e una porta aggiuntiva. Nella Figura 3.10(a), un multiplexer in ingresso sceglie di trasmettere il valore dell'ingresso D , quando EN è VERO; al contrario, quando EN è FALSO, sceglie di riciclare il valore precedente preso da Q . Nella Figura 3.10(b), invece, il clock è filtrato da una porta logica (*gated*). Ciò significa che se EN è VERO, l'ingresso del flip-flop CLK si aziona normalmente. Quando invece EN è FALSO, anche l'ingresso CLK assume valore FALSO e il flip-flop mantiene il suo valore precedente. Si noti che EN non deve cambiare mentre $CLK = 1$, affinché il flip-flop non veda un'alea del clock (cioè un clock che cambia al momento sbagliato). In generale, inserire logica combinatoria sul segnale di clock è controproducente: far passare un clock attraverso una porta introduce un ritardo e può causare errori di temporizzazione, come si vede nel paragrafo 3.5.3, quindi è fondamentale applicare questa soluzione solo se ne si conoscono bene gli effetti. Il simbolo circuitale di un flip-flop con abilitazione viene riportato nella Figura 3.10(c).

3.2.6 Flip-flop resettabile

Un flip-flop resettabile aggiunge un altro ingresso, chiamato $RESET$. Quando l'ingresso $RESET$ è FALSO, il flip-flop resettabile si comporta come un normale flip-flop D. Quando invece $RESET$ è VERO, il flip-flop resettabile ignora

Figura 3.9
Registro a 4 bit: (a) schema,
(b) simbolo circuitale.



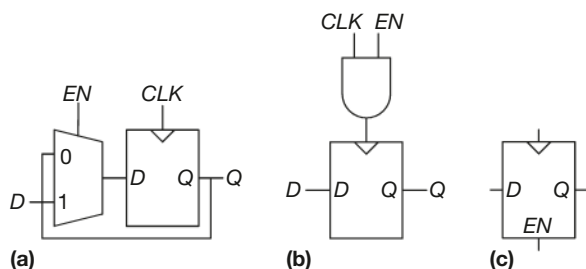


Figura 3.10
Flip-flop con abilitazione:
(a, b) schemi, (c) simbolo circuitale.

D e, appunto, resetta l'uscita a 0. Questa tipologia di flip-flop è utile nel caso in cui si desideri forzare uno stato noto (cioè 0) in tutti i flip-flop della rete quando viene accesa.

Questi flip-flop possono essere resettabili in **modo sincrono** o **asincrono**. I flip-flop resettabili in modo sincrono si resettano solo al fronte di salita di CLK , i flip-flop resettabili in modo asincrono si resettano nel momento in cui $RESET$ diventa VERO, indipendentemente dal valore assunto da CLK .

La **Figura 3.11(a)** mostra come costruire un flip-flop resettabile in modo sincrono a partire da un normale flip-flop D e da una porta AND. Quando $RESET$ è FALSO, la porta AND forza un valore 0 nell'ingresso del flip-flop. Quando invece $RESET$ è VERO, la porta AND trasmette il valore di D al flip-flop. In questo esempio, $RESET$ è un segnale **attivo basso**, il che significa che il segnale di reset esegue la propria funzione quando è 0 e non 1. Con l'aggiunta di un negatore, la rete avrebbe invece un segnale di reset attivo alto. Le **Figure 3.11(b)** e **3.11(c)** riportano i simboli per un flip-flop resettabile con un reset attivo alto.

I flip-flop resettabili asincroni richiedono una modifica della loro struttura interna e viene chiesto al lettore, nell'Esercizio 3.13, di effettuare tale modifica; si trovano comunque a disposizione dei progettisti come componenti standard.

Come facilmente prevedibile, anche flip-flop settabili possono occasionalmente essere utilizzati. Questi flip-flop forzano a 1 l'uscita quando SET viene attivato e hanno a loro volta una versione sincrona e una asincrona. I flip-flop settabili e resettabili possono anche avere un ingresso di abilitazione e possono essere raggruppati in registri da N bit.

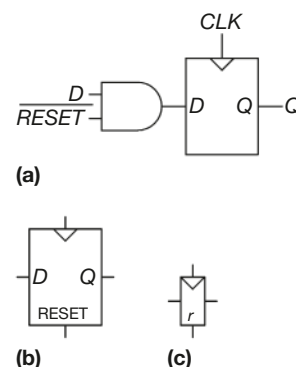


Figura 3.11
Flip-flop resettabile in modo
sincrono: (a) schema, (b, c) simboli
circuitali.

3.2.7 Progetto di latch e flip-flop a livello di transistori

L'Esempio 3.1 ha mostrato che i latch e i flip-flop necessitano di un gran numero di transistori se vengono costruiti a partire dalle porte logiche. Ma il ruolo fondamentale di un latch è semplicemente quello di essere trasparente od opaco, proprio come un interruttore. Come discusso nel paragrafo 1.7.7, una porta di trasmissione costituisce un valido metodo per costruire un interruttore CMOS: si può quindi sfruttare questa caratteristica delle porte di trasmissione per ridurre il numero di transistori necessari.

Un latch D compatto può essere realizzato con una singola porta di trasmissione, come mostrato nella **Figura 3.12(a)**. Quando $CLK = 1$ e $\overline{CLK} = 0$, la porta di trasmissione è accesa, quindi D scorre verso l'uscita Q e il latch è trasparente. Quando invece $CLK = 0$ e $\overline{CLK} = 1$, la porta di trasmissione è spenta, quindi Q viene isolato da D e il latch è opaco. Tuttavia, questo latch ha due grandi limiti:

- **Nodo d'uscita fluttuante.** Quando il latch è opaco, l'uscita Q non viene tenuta al suo valore da nessuna porta logica: Q viene definita un **nodo fluttuante** o **dinamico**. A lungo andare, il rumore e la perdita di carica può alterare il valore di Q .

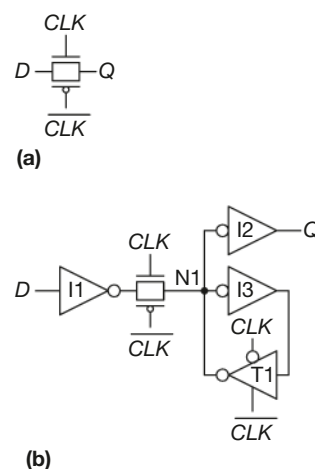


Figura 3.12
Schema del latch D.

- **Assenza di buffer.** L'assenza di buffer ha causato dei malfunzionamenti su diversi chip commerciali. Un picco di rumore che porta D a un voltaggio negativo è in grado di accendere il transistor nMOS, rendendo il latch trasparente anche quando $CLK = 0$. Allo stesso modo, un picco su D che supera V_{DD} può accendere il transistor pMOS anche quando $CLK = 0$. Inoltre la porta di trasmissione è simmetrica, quindi può essere pilotata all'indietro per effetto di rumore su Q , modificando l'ingresso D . La regola generale è che né l'ingresso di una porta di trasmissione né il nodo di stato di una rete sequenziale dovrebbe essere esposto al mondo esterno, dove è probabile che venga affetto dal rumore.

La **Figura 3.12(b)** mostra un latch D più resistente formato da 12 transistori in un moderno chip commerciale. Anche questo viene costruito a partite da una porta di trasmissione con clock, ma, al contrario dell'esempio precedente, questo aggiunge i negatori I1 e I2 per caricare l'ingresso e l'uscita. Lo stato del latch viene mantenuto sul nodo N1, mentre il negatore I3 e il buffer tristate, T1, forniscono la retroazione per trasformare N1 in un **nodo statico**. Se N1 viene alterato da una piccola quantità di rumore mentre CLK ha valore 0, T1 lo riporta a un valore logico valido.

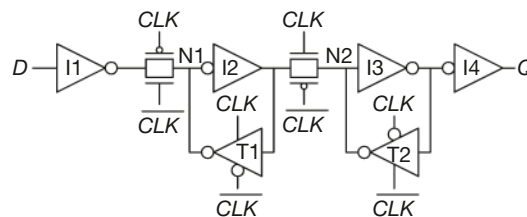
La **Figura 3.13** mostra un flip-flop D costruito a partire da due latch statici controllati da $\overline{CLK}$ e CLK . Alcuni negatori interni in eccesso sono stati rimossi affinché il flip-flop fosse formato solo da 20 transistori.

Questo circuito assume che sia CLK sia $\overline{CLK}$ siano disponibili. In caso contrario, sono necessari due ulteriori transistori per realizzare il negatore del clock.

3.2.8 Per riassumere

I latch e i flip-flop sono blocchi costitutivi fondamentali delle reti sequenziali. Un latch D è sensibile ai livelli, mentre un flip-flop D è attivo sui fronti. Il latch D è trasparente quando $CLK = 1$ e permette quindi a D di scorrere verso l'uscita Q . Invece un flip-flop D copia il valore di D in Q sul fronte di salita di CLK . In qualsiasi altro momento, i latch e i flip-flop mantengono il loro stato precedente. Un registro è un banco formato da diversi flip-flop D che condividono un segnale CLK comune.

Figura 3.13
Schema del flip-flop D.



ESEMPIO 3.2

Confronto tra flip-flop e latch. Ben Imbrogliabit applica gli ingressi D e CLK , mostrati nella **Figura 3.14**, a un latch D e a un flip-flop D. Bisogna aiutarlo a determinare l'uscita Q in entrambi i dispositivi.

Soluzione La **Figura 3.15** mostra le forme d'onda delle uscite, nell'ipotesi di un piccolo ritardo di Q per rispondere ai cambiamenti di livello degli ingressi. La freccia indica la causa di un cambiamento nell'uscita. Il valore iniziale di Q è sconosciuto e può quindi essere 0 o 1, come indica la coppia di linee orizzontali. Per prima cosa si consideri il latch. Sul primo fronte di salita di CLK , $D = 0$ quindi Q diventa 0. Ogni volta che D cambia valore, mentre $CLK = 1$, Q si comporta nello stesso modo. Quando invece D cambia mentre $CLK = 0$, il suo cambiamento viene ignorato. Si consideri ora il flip-flop: per ogni fronte di salita di CLK , D viene copiato in Q . In tutti gli altri casi, Q mantiene il suo stato precedente.

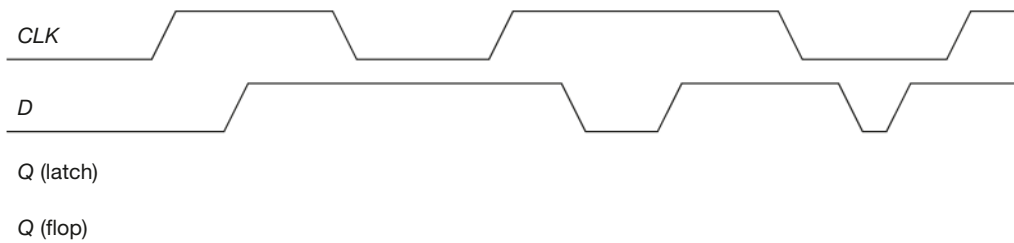


Figura 3.14 Forme d'onda dell'esempio.

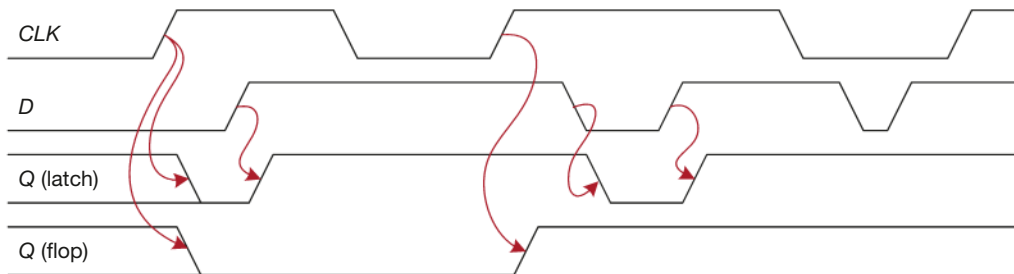


Figura 3.15 Forme d'onda della soluzione.

3.3 ■ PROGETTO DI RETI LOGICHE SINCRONE

In generale, le reti sequenziali includono tutte le reti che non sono combinatorie, cioè quelle la cui uscita non può essere determinata guardando semplicemente i valori presenti in quel momento agli ingressi. Alcune reti sequenziali sono alquanto singolari, come si può vedere nella prima parte di questo paragrafo. Il paragrafo introduce poi la nozione di reti sequenziali sincrone e la disciplina dinamica: limitandosi a considerare reti sequenziali sincrone, è possibile sviluppare metodi sistematici e semplici per analizzare e progettare i sistemi sequenziali.

3.3.1 Alcune reti problematiche

ESEMPIO 3.3

Reti astabili. Alyssa Guastacomputer ha trovato una strana rete logica costituita da tre negatori collegati ad anello, come mostra la **Figura 3.16**. L'uscita del terzo negatore è collegata in retroazione all'ingresso del primo negatore. Ogni negatore ha un ritardo da propagazione di 1 ns. Serve aiutare Alyssa a capire come funziona la rete.

Soluzione Si supponga che il nodo X sia inizialmente 0. Questo significa che $Y = 1$, $Z = 0$, e quindi $X = 1$, il che non corrisponde al nostro presupposto iniziale. La rete non ha stati stabili e viene dunque detta **instabile** o **astabile**. La **Figura 3.17** mostra il comportamento della rete: se al tempo 0 X si alza, Y si abbassa dopo 1 ns, Z si alza dopo 2 ns e X si abbassa dopo 3 ns. Di conseguenza, Y si rialza dopo 4 ns, Z si riabbassa dopo 5 ns e X si rialza di nuovo dopo 6 ns, e così via. Ogni nodo, quindi, oscilla tra 0 e 1 in un **periodo** (cioè in un tempo di ripetizione) pari a 6 ns. Questa rete è chiamata **oscillatore ad anello**.

Il periodo dell'oscillatore ad anello dipende dal ritardo di propagazione di ogni singolo negatore, che a sua volta dipende dal modo in cui il negatore è stato fabbricato, dalla tensione di alimentazione e anche dalla temperatura. Proprio perché dipende da diversi elementi, il periodo di un oscillatore ad anello è difficile da prevedere. In sintesi, l'oscillatore ad anello è una rete sequenziale con zero ingressi e un'uscita che cambia periodicamente.

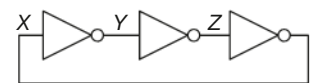


Figura 3.16
Anello di tre negatori.

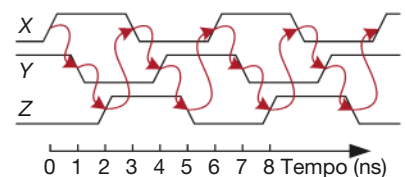


Figura 3.17
Forme d'onda dell'oscillatore ad anello.

ESEMPIO 3.4

Condizioni di corsa. Ben Imbrogliabit ha progettato un nuovo latch D che, a suo dire, è migliore rispetto a quello nella Figura 3.7 perché utilizza un numero minore di porte. Ben ha scritto la tabella delle verità per trovare il valore dell'uscita Q dati i due ingressi D e CLK e lo stato precedente del latch, Q_{prec} . Basandosi su questa tabella delle verità, Ben ha derivato le espressioni booleane della rete, dove Q_{prec} corrisponde all'uscita Q collegata in retroazione. Il suo progetto è riportato nella **Figura 3.18**. Verificare se questo latch lavora in maniera corretta, indipendentemente dal ritardo di ogni porta.

Figura 3.18
Un latch D apparentemente migliore.

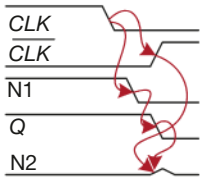
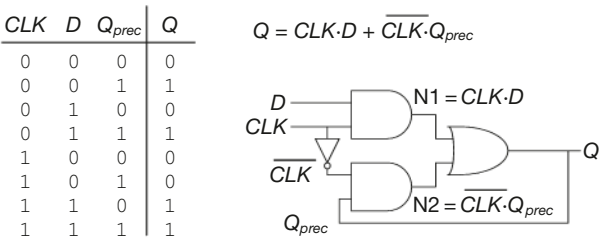


Figura 3.19
Forme d'onda del latch che mostrano il fenomeno della corsa.

Soluzione La **Figura 3.19** mostra che la rete ha una **condizione di corsa** (*race*) che ne causa il malfunzionamento quando alcune porte sono più lente di altre. Si supponga che $CLK = D = 1$: in questo caso il latch è trasparente e trasmette D per portare $Q = 1$. Se a questo punto CLK scende a 0, il latch dovrebbe ricordarsi dell'ultimo valore, mantenendo $Q = 1$. Si supponga però che il ritardo attraverso il negatore che va da CLK a $\overline{CLK}$ sia maggiore dei ritardi delle porte AND e OR: i nodi $N1$ e Q potrebbero entrambi abbassarsi prima che $\overline{CLK}$ abbia il tempo di alzarsi. In questo caso, $N2$ non si alzerebbe mai, e Q rimarrebbe bloccato al valore 0.

Questo è un esempio di progetto di rete **asincrona**, nella quale le uscite sono direttamente collegate in retroazione agli ingressi. Le reti asincrone sono tristemente note proprio perché spesso hanno condizioni di corsa che fanno sì che il comportamento della rete dipenda da quale tra due percorsi dei segnali attraverso le porte logiche è il più veloce. Una rete potrebbe quindi funzionare senza problemi, mentre un'altra rete identica costruita con porte con un ritardo leggermente diverso potrebbe non funzionare affatto. Ancora, la rete potrebbe funzionare solo a una certa temperatura o con una certa tensione di alimentazione che rendono corretti i ritardi. Questi malfunzionamenti sono quindi estremamente difficili da individuare.

3.3.2 Reti sequenziali sincrone

I due esempi precedenti contengono degli anelli chiamati **percorsi ciclici**, nei quali le uscite sono direttamente collegate in retroazione agli ingressi. Queste reti sono sequenziali, dal momento che la logica combinatoria non prevede né percorsi ciclici né condizioni di corsa. Se si applicano valori di ingresso alla logica combinatoria, le uscite si portano sempre al valore corretto dopo un tempo pari al ritardo di propagazione. Al contrario, le reti sequenziali con percorsi ciclici possono avere condizioni di corsa o comportamenti instabili. L'analisi delle reti di questo tipo alla ricerca dei problemi di malfunzionamento è un processo molto lungo e a rischio di errori anche per i progettisti più esperti.

Per evitare questi problemi, i progettisti preferiscono interrompere i percorsi ciclici inserendo in alcuni punti dei registri. Questa operazione trasforma la rete in un insieme di logica combinatoria e registri. I registri contengono lo stato del sistema, che cambia solo in corrispondenza dei fronti di clock, motivo per cui lo stato viene detto **sincronizzato** con il clock. Se il clock è abbastanza lento da far sì che tutti gli ingressi dei registri abbiano il tempo di adeguare il proprio valore prima del fronte di clock successivo, tutte le corse

vengono eliminate. Se si adotta la disciplina di introduzione dei registri nei percorsi di retroazione si giunge alla definizione formale di una rete sequenziale sincrona.

Si ricordi che una rete è definita dai suoi terminali di ingresso e uscita e dalla sua specifica funzionale e temporale. Una rete sequenziale ha una serie finita di **stati** discreti $\{S_0, S_1, \dots, S_{k-1}\}$. Una **rete sequenziale sincrona** ha un ingresso di clock i cui fronti di salita indicano una sequenza di istanti di tempo nei quali hanno luogo le transizioni di stato. Vengono spesso utilizzati i termini **stato presente** e **stato prossimo** per distinguere lo stato in cui il sistema si trova al momento attuale dallo stato in cui si porterà al prossimo fronte di clock. La specifica funzionale descrive in maniera dettagliata lo stato prossimo del sistema e il valore di ogni sua uscita per ogni possibile combinazione di stato presente e valori d'ingresso. La specifica temporale, invece, consiste in un limite superiore (t_{pcq}) e in un limite inferiore (t_{ccq}) del tempo dal fronte di salita del clock fino ai cambiamenti delle uscite, oltre che dei tempi di *setup* (attivazione) e di *hold* (mantenimento), t_{setup} e t_{hold} , che indicano invece quando gli ingressi devono essere stabili rispetto al fronte di salita del clock.

Le regole di composizione delle reti sequenziali sincrone stabiliscono che una rete è una rete sequenziale sincrona se è formata da elementi circuitali interconnessi in modo tale che:

- ogni elemento della rete è o un registro o una rete combinatoria;
- deve essere presente necessariamente almeno un registro;
- tutti i registri ricevono lo stesso segnale di clock;
- ogni percorso ciclico contiene almeno un registro.

Le reti sequenziali che non sono sincrone sono dette **asincrone**.

Un flip-flop rappresenta l'esempio più semplice di rete sequenziale sincrona: possiede un ingresso, D , un clock, CLK , un'uscita, Q , e due stati, $\{0, 1\}$. La specifica funzionale di un flip-flop consiste nell'affermazione che lo stato prossimo è D e che l'uscita, Q , è lo stato presente, come mostrato nella **Figura 3.20**.

Spesso la variabile di stato presente viene indicata con S e la variabile stato prossimo con S' . L'apostrofo che segue la seconda S significa stato prossimo (e non come in altri casi l'operazione di negazione). La temporizzazione delle reti sequenziali viene analizzata nel paragrafo 3.5.

Altre due tipologie comuni di reti sequenziali sincrone sono le macchine a stati finiti e le pipeline, analizzate più avanti nel capitolo.

ESEMPIO 3.5

Reti sequenziali sincrone. Osservare le reti nella **Figura 3.21** e riconoscere quali di esse sono reti sequenziali sincrone.

Soluzione La rete (a) è combinatoria, non sequenziale, perché non possiede nessun registro. La rete (b) è una semplice rete sequenziale senza retroazione. La rete (c) non è né una rete combinatoria né una rete sequenziale sincrona, perché contiene un latch che non è né un registro né una rete combinatoria. Le reti (d) ed (e) sono reti sequenziali sincrone; costituiscono due esempi di macchine a stati finiti che verranno spiegate nel paragrafo 3.4. La rete (f) non è né combinatoria né sequenziale sincrona, perché contiene un percorso ciclico che va dall'uscita della logica combinatoria all'ingresso della stessa logica, senza però un registro al suo interno. La rete (g) è una rete sequenziale sincrona con struttura a pipeline, analizzata nel paragrafo 3.6. La rete (h) non è una rete sequenziale sincrona in senso stretto, dal momento che il secondo registro riceve un segnale di clock diverso rispetto al primo a causa del ritardo dei due negatori.

t_{pcq} indica il tempo di propagazione del segnale dal clock a Q intesa come uscita di una generica rete sequenziale sincrona. t_{ccq} indica invece il tempo di contaminazione dal clock a Q . I due tempi sono analoghi ai corrispondenti t_{pd} e t_{cd} nelle reti combinatorie.

La definizione di rete sequenziale sincrona data nel testo è sufficiente, ma per certi versi più restrittiva del necessario. Per esempio, nei microprocessori ad alte prestazioni, alcuni registri possono ricevere segnali di clock ritardati o propagati da porte logiche per ottimizzare al massimo le prestazioni. Alcuni microprocessori usano anche latch al posto dei registri. Tuttavia la definizione si adatta perfettamente a tutte le reti sequenziali sincrone trattate in questo testo, come pure per la maggior parte dei sistemi digitali in commercio.

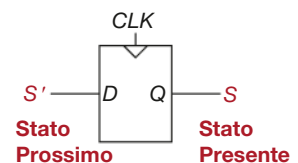
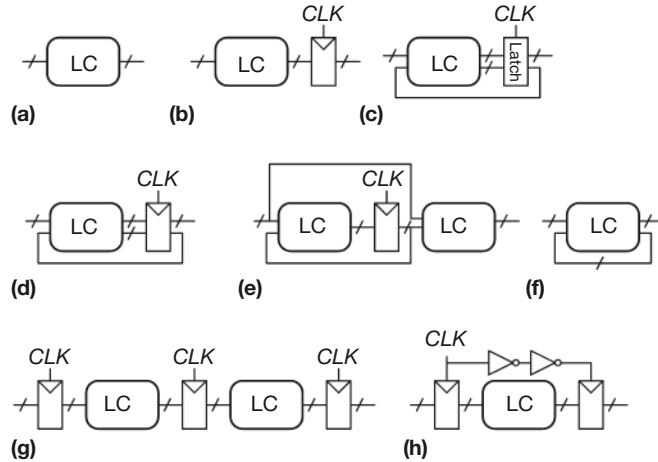


Figura 3.20
Stato presente e stato prossimo del flip-flop.

Figura 3.21
Circuiti dell'esempio.



3.3.3 Reti sincrone e asincrone

In teoria, la progettazione di reti asincrone è più generale rispetto a quella di reti sincrone, perché la temporizzazione del sistema non è limitata dal segnale di clock dei registri. Proprio come le reti analogiche sono più generali di quelle digitali, visto che possono utilizzare qualsiasi valore di tensione, le reti asincrone sono più generali di quelle sincrone perché utilizzano qualsiasi tipo di retroazione. Tuttavia, le reti sincrone sono, come visto, più semplici da progettare e da utilizzare rispetto a quelle asincrone, proprio come le reti digitali sono più semplici di quelle analogiche. Nonostante siano state effettuate per decenni delle ricerche sulle reti asincrone, tutti i sistemi digitali di oggi sono essenzialmente sincroni.

Ovviamente, le reti asincrone sono talvolta necessarie, per esempio quando la comunicazione avviene tra due sistemi con segnali di clock differenti o quando si ricevono gli ingressi in momenti arbitrari, proprio come le reti analogiche sono necessarie quando si deve comunicare col mondo reale caratterizzato da variazioni continue di tensione. Inoltre, la ricerca sulle reti asincrone continua a produrre idee interessanti, alcune delle quali sono in grado di migliorare anche le reti sincrone.

3.4 ■ MACCHINE A STATI FINITI

Le reti sequenziali sincrone possono essere raffigurate nelle forme riportate nella **Figura 3.22**. Queste forme prendono il nome di **macchine a stati finiti** (o **FSM**, *finite state machine*). Il loro nome deriva dal fatto che una rete con k registri può trovarsi in uno di un numero finito (2^k) di stati diversi. Una FSM possiede M ingressi, N uscite e k bit di stato. Inoltre, riceve un segnale di clock e, a volte, anche un segnale di reset. Una FSM è composta da due blocchi di logica combinatoria, la **logica di stato prossimo** e la **logica d'uscita**, e da un registro che immagazzina lo stato. A ogni fronte di salita del clock, la FSM avanza allo stato prossimo, definito in base agli ingressi e allo stato presente. Esistono due classi generali di macchine a stati finiti, ognuna caratterizzata dalla propria specifica funzionale: nelle **macchine alla Moore**, le uscite dipendono esclusivamente dallo stato presente della macchina; nelle **macchine alla Mealy**, invece, le uscite dipendono sia dallo stato presente della macchina sia dagli ingressi attuali. Le macchine a stati finiti costituiscono un metodo sistematico di progettazione delle reti sequenziali sincrone, a partire dalla specifica funzionale. Questo metodo viene spiegato nel resto del paragrafo a partire da un esempio.

Le macchine alla Moore e alla Mealy prendono il nome dai loro "sostenitori", due ricercatori dei *Bell Labs* che hanno contribuito a sviluppare la teoria degli automi, ovvero la matematica che descrive le macchine a stati. Edward F. Moore (1925-2003) – da non confondersi con il fondatore della Intel, Gordon Moore – ha pubblicato il suo articolo fondamentale *Gedanken-experiments on Sequential Machines* nel 1956. Successivamente è divenuto professore di matematica e informatica all'Università del Wisconsin. George H. Mealy (1927-2010) ha pubblicato l'articolo *A Method of Synthesizing Sequential Circuits* nel 1955. Ha scritto ai Bell Labs il primo sistema operativo per il calcolatore IBM 704, e successivamente ha lavorato all'Università di Harvard.

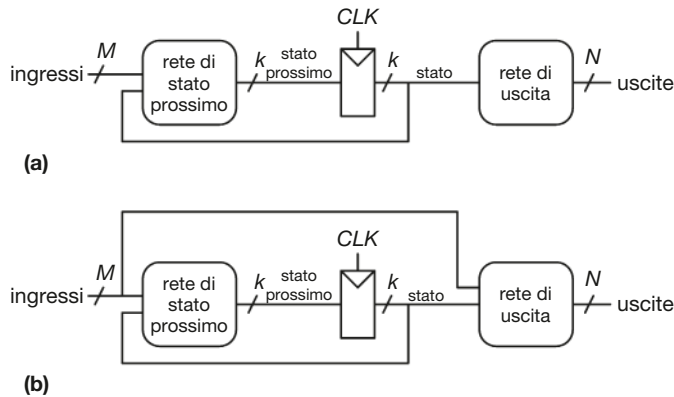


Figura 3.22
Macchine a stati finiti: (a) macchina alla Moore, (b) macchina alla Mealy.

3.4.1 Esempio di progettazione di una FSM

Per capire il progetto di una macchina a stati finiti si prenda in considerazione il problema di costruire un controllore per un semaforo stradale posto a un incrocio di un campus universitario. Gli studenti di ingegneria si trascinano lentamente dal pensionato studentesco verso i laboratori situati in via Accademia, e sono così impegnati a leggere dal loro libro di testo la spiegazione delle macchine a stati finiti da non guardare dove vanno. In quel mentre, alcuni giocatori di football stanno correndo dal campo alla mensa universitaria in viale Ateneo, lanciandosi la palla e non prestando neppure loro attenzione alla strada. All'incrocio di queste due strade si sono verificati spesso degli incidenti; così, il preside chiede a Ben Imbrogliabit di installare un semaforo all'incrocio per prevenire altri incidenti.

Ben decide di risolvere il problema utilizzando una macchina a stati finiti, e installa due sensori per il traffico, T_A e T_B , rispettivamente in via Accademia e in viale Ateneo. Ognuno di questi sensori indica VERO se sono presenti degli studenti sulla strada, e FALSO se non sta passando nessuno. Ben installa anche due semafori stradali, L_A e L_B , per controllare il traffico. Ogni semaforo riceve ingressi digitali che specificano se la luce accesa deve essere verde, gialla o rossa. Quindi la macchina a stati finiti di Ben ha due ingressi, T_A e T_B , e due uscite, L_A e L_B (ciascuna da due bit). L'incrocio con sensori e semafori è schematizzato nella **Figura 3.23**. Ben inserisce un clock con un periodo di 5 secondi: a ogni periodo del clock (fronte di salita) i semafori possono cambiare a seconda di quello che indicano i sensori del traffico. Ben decide di inserire anche un bottone di reset per far sì che i tecnici dell'università possano portare il controllore a uno stato iniziale noto al momento dell'accensione. La **Figura 3.24** mostra lo schema a scatola nera della macchina a stati.

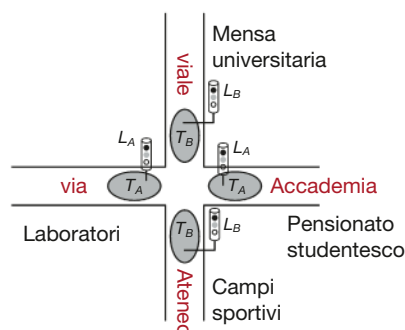
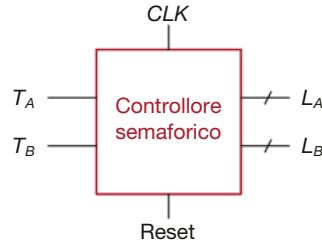


Figura 3.23
Mappa dell'università.

Figura 3.24

Vista a scatola nera della macchina a stati finiti.



Il passo successivo è quello di disegnare il **diagramma degli stati**, riportato nella **Figura 3.25**, che indichi tutti i possibili stati del sistema e le transizioni tra di essi. Quando il sistema viene resettato, il semaforo in via Accademia è verde, mentre quello in viale Ateneo è rosso. Ogni 5 secondi, il controllore esamina la situazione del traffico e decide quale sarà lo stato prossimo. Finché ci sono dei passanti in via Accademia, lo stato dei semafori non cambia. Una volta defluito il traffico in via Accademia, il semaforo in via Accademia diventa giallo per 5 secondi, poi diventa rosso mentre diventa verde quello in viale Ateneo. Allo stesso modo, il semaforo in viale Ateneo rimane verde finché ci sono dei passanti in strada, per poi passare al giallo e infine al rosso.

Nel diagramma degli stati, i cerchi rappresentano gli stati e gli archi rappresentano le transizioni tra di essi. Le transizioni avvengono al fronte di salita del clock, che è stato tralasciato nello schema perché è sempre presente in una rete sequenziale sincrona. Inoltre, il clock ha il solo compito di controllare il momento in cui hanno luogo le transizioni, mentre il diagramma pone l'attenzione sul modo in cui avvengono queste transizioni e sulle loro conseguenze. L'arco denominato Reset che dall'esterno dello schema raggiunge lo stato S0 indica che il sistema, in caso di reset, assume quello stato, indipendentemente dallo stato precedente. Se da uno stato si diramano diversi archi, questi vengono etichettati in modo da mostrare quale ingresso causa quale transizione. Per esempio, se il sistema si trova nello stato S0, rimane in quello stato se T_A è VERO e passa invece allo stato S1 se T_A è FALSO. Se da uno stato parte un solo arco, ciò significa che quella particolare transizione ha luogo indipendentemente dai valori degli ingressi. Per esempio, una volta arrivato allo stato S1, il sistema passa in ogni caso allo stato S2. Il valore assunto dalle uscite in ogni stato è indicato nello stato stesso. Per esempio, quando il sistema si trova in stato S2, L_A è rosso e L_B è verde.

Ben successivamente riscrive il diagramma degli stati nella **tabella degli stati** riportata nella **Tabella 3.1**, che indica, per ogni stato presente e valori di ingresso, lo stato prossimo S' del sistema. Notare che la tabella utilizza il simbolo di indifferenza (X) ogniqualvolta lo stato prossimo non dipende da un

Figura 3.25

Diagramma degli stati.

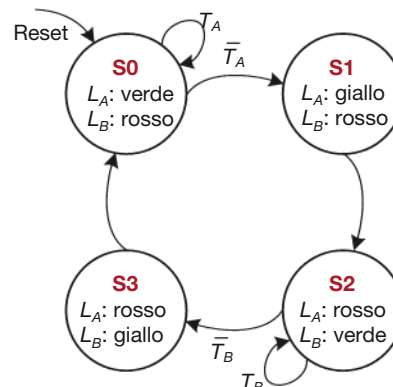


Tabella 3.1 Tabella degli stati.

Stato corrente S	Ingressi		Stato prossimo S'
	T_A	T_B	
S0	0	X	S1
S0	1	X	S0
S1	X	X	S2
S2	X	0	S3
S2	X	1	S2
S3	X	X	S0

Tabella 3.2 Codifica degli stati.

Stato	Codifica $S_{1:0}$
S0	00
S1	01
S2	10
S3	11

Tabella 3.3 Codifica delle uscite.

Uscite	Codifica $L_{1:0}$
Verde	00
Giallo	01
Rosso	10

particolare valore di ingresso. Notare anche che il Reset è stato omissso nella tabella: viene utilizzato un flip-flop resettabile che porta il sistema allo stato S0 a ogni reset, indipendentemente dai valori degli ingressi.

Il diagramma degli stati è astratto nel senso che utilizza nomi simbolici per gli stati {S0, S1, S2, S3} e per le uscite {rosso, giallo, verde}. Per costruire una rete reale, agli stati e alle uscite devono essere assegnate delle **codifiche binarie**. Ben sceglie il semplice metodo di codifica riportato nelle **Tabelle 3.2 e 3.3**: ogni stato e ogni uscita sono codificati con due bit: $S_{1:0}$, $L_{A1:0}$ e $L_{B1:0}$.

Ben converte la tabella degli stati nella **tabella delle transizioni** per utilizzare questi codici binari, come mostrato nella **Tabella 3.4**. Quest'ultima assume quindi l'aspetto di una tabella delle verità che specifica la logica di stato prossimo. In particolare, lo stato prossimo, S' , viene definito in funzione dello stato presente, S , e dei valori di ingresso.

A partire da questa tabella si possono facilmente ricavare le espressioni booleane in forma somma di prodotti corrispondenti allo stato prossimo del sistema.

$$\begin{aligned} S'_1 &= \bar{S}_1 S_0 + S_1 \bar{S}_0 \bar{T}_B + S_1 \bar{S}_0 T_B \\ S'_0 &= \bar{S}_1 \bar{S}_0 \bar{T}_A + S_1 \bar{S}_0 \bar{T}_B \end{aligned} \quad (3.1)$$

Le espressioni possono essere minimizzate usando le mappe di Karnaugh, ma spesso la semplificazione per ispezione è più semplice. Per esempio, i termini T_B e $\bar{T}_B$ nell'espressione S'_1 sono evidentemente ridondanti. È quindi possibile ridurre l'espressione S'_1 a un'operazione XOR. Nell'Espressione 3.2 si riportano le due **espressioni di stato prossimo** nella loro forma ottimizzata.

$$\begin{aligned} S'_1 &= S_1 \oplus S_0 \\ S'_0 &= \bar{S}_1 \bar{S}_0 \bar{T}_A + S_1 \bar{S}_0 \bar{T}_B \end{aligned} \quad (3.2)$$

In maniera simile, Ben scrive anche la **tabella delle uscite** (**Tabella 3.5**) che indica, per ogni stato, quale deve essere il valore assunto dall'uscita. Di nuovo,

Tabella 3.4 Tabella delle transizioni con codifica binaria degli stati.

Stato corrente		Ingressi		Stato prossimo	
S_1	S_0	T_A	T_B	S'_1	S'_0
0	0	0	X	0	1
0	0	1	X	0	0
0	1	X	X	1	0
1	0	X	0	1	1
1	0	X	1	1	0
1	1	X	X	0	0

Si noti che gli stati sono denominati S0, S1, ecc. Si usano invece i pedici come in S_0 , S_1 ecc. quando ci si riferisce ai bit di stato.

Tabella 3.5 Tabella delle uscite.

Stato corrente		Uscite			
S_1	S_0	L_{A1}	L_{A0}	L_{B1}	T_{B0}
0	0	0	0	0	X
0	1	0	1	0	X
1	0	1	0	0	X
1	1	1	0	1	0

è semplice ricavarne le espressioni booleane per le uscite e ottimizzarle. Per esempio, si osservi che L_{A1} è VERO solo nella riga in cui anche S_1 è VERO.

$$\begin{aligned}L_{A1} &= S_1 \\L_{A0} &= \overline{S_1}S_0 \\L_{B1} &= \overline{S_1} \\L_{B0} &= S_1S_0\end{aligned}$$

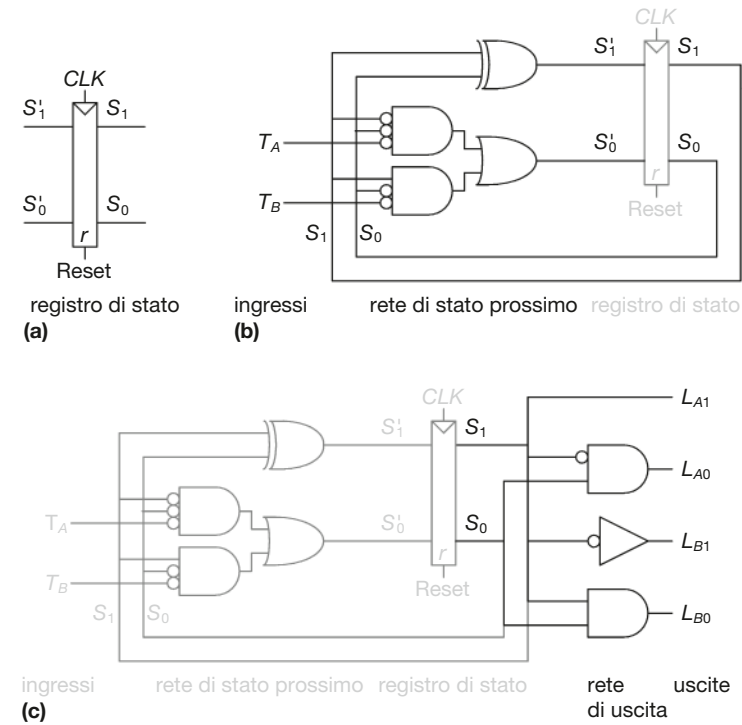
(3.3)

Infine, Ben disegna uno schema della sua macchina alla Moore nella forma riportata nella Figura 3.22(a). Per prima cosa, disegna il registro di stato a 2 bit, come mostra la **Figura 3.26(a)**. A ogni fronte di salita del clock, il registro di stato campiona lo stato prossimo, $S'_{1:0}$, facendolo diventare lo stato presente $S_{1:0}$. Il registro di stato riceve un segnale di reset sincrono o asincrono per inizializzare la FSM all'avvio. Poi Ben disegna la logica di stato prossimo in base all'Espressione 3.2, che calcola lo stato prossimo in base allo stato presente e agli ingressi, come mostrato nella **Figura 3.26(b)**. Infine, disegna la logica di uscita, basata sull'Espressione 3.3, che calcola le uscite in funzione dello stato presente, come mostra la **Figura 3.26(c)**.

La **Figura 3.27** mostra un diagramma temporale che descrive il controllo-re del semaforo mentre attraversa una serie di stati. Il diagramma mostra i se-

Figura 3.26 Struttura della macchina a stati per il controllo semaforico.

Questo schema usa alcune porte AND con le bolle agli ingressi. Tali porte si possono ottenere con porte AND e negatori agli ingressi, con porte NOR e negatori agli ingressi senza bolle, o con altre combinazioni di porte logiche: la scelta dipende naturalmente dalla specifica tecnologia di realizzazione circuitale adottata.



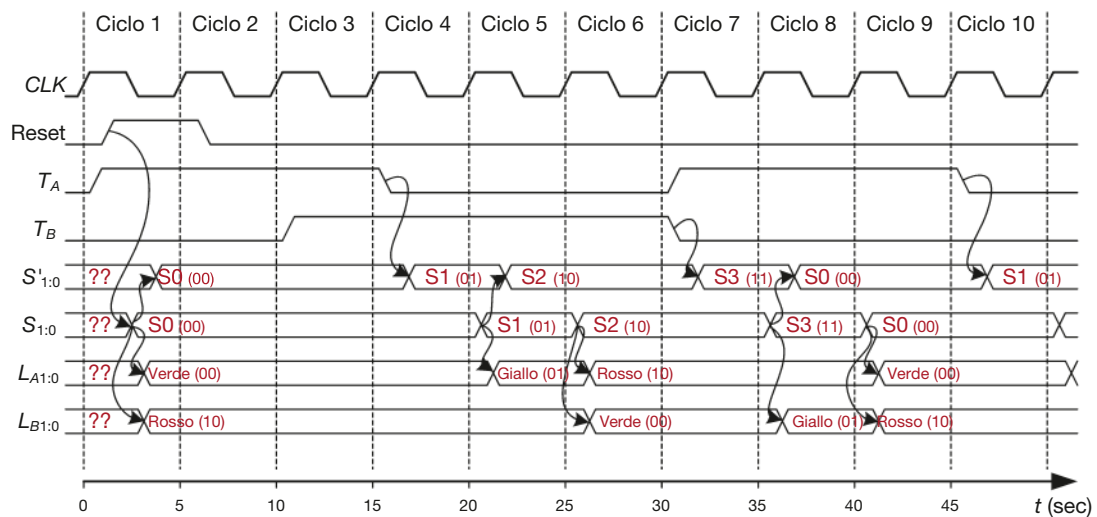


Figura 3.27 Diagramma temporale del controllore semaforico.

gnali CLK , $Reset$, gli ingressi T_A e T_B , lo stato prossimo S' , lo stato presente S e le uscite L_A e L_B . Le frecce indicano la causalità: per esempio, un cambiamento di stato causa un cambiamento delle uscite, e un cambiamento degli ingressi causa un cambiamento dello stato prossimo. Le linee tratteggiate indicano i fronti di salita di CLK in corrispondenza dei cambi di stato.

Il clock ha un periodo di 5 secondi, quindi i semafori cambiano al massimo una volta ogni cinque secondi. Quando la macchina a stati finiti viene accesa per la prima volta, il suo stato è sconosciuto, come indicato dal punto di domanda. Di conseguenza, il sistema deve essere resettato per raggiungere uno stato noto. In questo diagramma temporale, S viene resettato immediatamente a S_0 , e questo indica l'utilizzo di flip-flop resettabili in modo asincrono. Allo stato S_0 , il semaforo L_A è verde, mentre il semaforo L_B è rosso.

In questo esempio, il traffico arriva immediatamente da via Accademia, quindi il controllore rimane in stato S_0 , mantenendo L_A verde anche nel caso in cui arrivi del traffico anche su viale Ateneo. Dopo 15 secondi, il traffico su via Accademia si è esaurito e T_A passa a 0. Al fronte di clock successivo, il controllore si sposta allo stato S_1 , facendo diventare giallo L_A . Passati altri 5 secondi, il controllore passa allo stato S_2 , nel quale L_A diventa rosso e L_B verde. Successivamente il controllore aspetta in stato S_2 fino all'esaurimento del traffico su viale Ateneo, per poi procedere con lo stato S_3 , che fa diventare giallo L_B . 5 secondi più tardi, il controllore torna allo stato S_0 e alle condizioni iniziali.

Nonostante gli sforzi di Ben, gli studenti non fanno attenzione e gli scontri continuano a verificarsi. Il Preside di Facoltà ha chiesto a Ben e Alyssa di progettare una catapulte per lanciare gli studenti direttamente dal pensionato ai laboratori passando per le finestre aperte, per evitare l'incrocio pericoloso, ma il progetto in questione non verrà trattato in questo testo...

3.4.2 Codifica degli stati

Nell'esempio precedente, le codifiche di stati e uscite erano state scelte arbitrariamente, il che significa che una scelta diversa riguardo alla codifica avrebbe portato a una rete differente. Un problema comune è quello di determinare quale codifica produce la rete desiderata col minor numero di porte logiche o col minor ritardo di propagazione. Sfortunatamente, non c'è un modo semplice e diretto per individuare la codifica migliore, ma è necessario provare tutte le possibilità, un'operazione che può diventare molto complessa quando il numero di stati aumenta. Ciononostante, spesso è possibile scegliere una buona codifica tramite ispezione, facendo in modo che gli stati o le uscite connessi condividano dei bit. Anche gli strumenti di progettazione assistita dal calcolatore (CAD) rappresentano una buona alternativa per l'analisi dell'insieme di codifiche possibili e per la scelta di una delle migliori.



Nel caso della codifica degli stati, una delle decisioni più importanti da prendere è la scelta tra una codifica binaria e una codifica a singolo 1. Quando si utilizza la **codifica binaria**, come è stato fatto nell'esempio del controllo del semaforo, ogni stato viene rappresentato da un numero binario. Dal momento che K numeri binari possono essere rappresentati da $\log_2 K$ bit, un sistema con K stati avrà bisogno solo di $\log_2 K$ bit di stato.

Nella **codifica a singolo 1**, invece, viene utilizzato un bit di stato per ognuno degli stati. Viene chiamata in inglese codifica *one hot* perché in ogni momento uno solo dei bit è “caldo”, cioè VERO. Per fare un esempio, una FSM con tre stati con codifica a singolo 1 avrà come codifiche di stato 001, 010 e 100. Ogni bit di stato viene immagazzinato in un flip-flop, quindi una codifica a singolo 1 necessita di più flip-flop rispetto a una codifica binaria. Ciononostante, con la codifica a singolo 1, le logiche di stato prossimo e di uscita risultano spesso più semplici, il che significa che saranno necessarie meno porte. La miglior scelta per la codifica dipende quindi dalla specifica FSM.

ESEMPIO 3.6

Codifica degli stati di una FSM. Un **contatore modulo N** possiede un'uscita e nessun ingresso. L'uscita Y è ALTA per un ciclo di clock ogni N ; in altre parole, l'uscita divide la frequenza del clock per N . La forma d'onda e il diagramma degli stati per un contatore modulo 3 sono raffigurati nella **Figura 3.28**. Disegnare i progetti della rete per questo contatore utilizzando le codifiche di stato binaria e a singolo 1.

Figura 3.28
Contatore modulo 3: (a) forme d'onda e (b) diagramma degli stati.

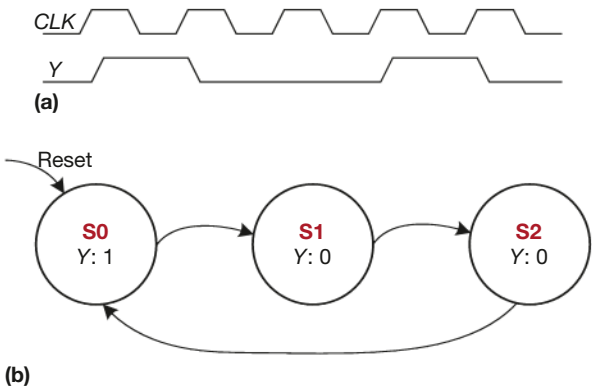


Tabella 3.6 Tabella degli stati del contatore modulo 3.

Stato corrente	Stato prossimo
S0	S1
S1	S2
S2	S0

Tabella 3.7 Tabella delle uscite del contatore modulo 3.

Stato corrente	Uscita
S0	1
S1	0
S2	0

Soluzione Le **Tabelle 3.6 e 3.7** mostrano le tabelle degli stati e delle uscite prima della codifica. La **Tabella 3.8**, invece, confronta la codifica binaria e quella a singolo 1 per i tre stati. La codifica binaria utilizza due bit di stato, e il risultato dell'utilizzo di questa codifica è mostrato nella **Tabella 3.9**, dove viene riportata la tabella delle transizioni. Si noti che non ci sono ingressi; lo stato prossimo dipende quindi esclusivamente dallo stato presente. Viene lasciato al lettore il compito di scrivere la tabella delle uscite. Le espressioni che descrivono lo stato prossimo e l'uscita sono:

$$\begin{aligned} S'_1 &= \overline{S_1} S_0 \\ S'_0 &= \overline{S_1} \overline{S_0} \end{aligned} \tag{3.4}$$

$$Y = \overline{S_1} \overline{S_0} \tag{3.5}$$

La codifica a singolo 1 utilizza invece tre bit di stato. La tabella delle transizioni per questo tipo di codifica è riportata nella **Tabella 3.10** e, ancora una volta, la tabella delle uscite viene lasciata come compito al lettore. Le espressioni per lo stato prossimo e l'uscita sono le seguenti:

$$\begin{aligned} S'_2 &= S_1 \\ S'_1 &= S_0 \end{aligned} \tag{3.6}$$

$$\begin{aligned} S'_0 &= S_1 \\ Y &= S_0 \end{aligned} \tag{3.7}$$

La **Figura 3.29** mostra gli schemi per ognuno dei due progetti. Si noti che l’hardware per il progetto a codifica binaria può essere ottimizzato facendo in modo che Y e S'_0 condividano la stessa porta. Si osservi anche che la codifica a singolo 1 necessita di flip-flop sia settabili (s) che resettabili (r) per inizializzare la macchina allo stato S_0 al reset. La scelta migliore di realizzazione dipende dal costo relativo delle porte e dei flip-flop, ma solitamente è preferibile, per questo specifico esempio, la codifica a singolo 1.

Una codifica simile è la **codifica a singolo 0**, o codifica *one cold*, nella quale K stati sono rappresentati con K bit, uno solo dei quali assume il valore FALSO (“freddo”).

Tabella 3.8 Codifica a singolo 1 e codifica binaria per il contatore modulo 3.

Stato	Codifica a singolo 1			Codifica binaria	
	S_2	S_1	S_0	S_1	S_0
S_0	0	0	1	0	0
S_1	0	1	0	0	1
S_2	1	0	0	1	0

Tabella 3.9 Tabella delle transizioni con codifica binaria degli stati.

Stato corrente		Stato prossimo	
S_1	S_0	S'_1	S'_0
0	0	0	1
0	1	1	0
1	0	0	0

Tabella 3.10 Tabella delle transizioni con codifica a singolo 1 degli stati.

Stato corrente			Stato prossimo		
S_2	S_1	S_0	S'_0	S'_1	S'_2
0	0	1	0	1	0
0	1	0	1	0	0
1	0	0	0	0	1

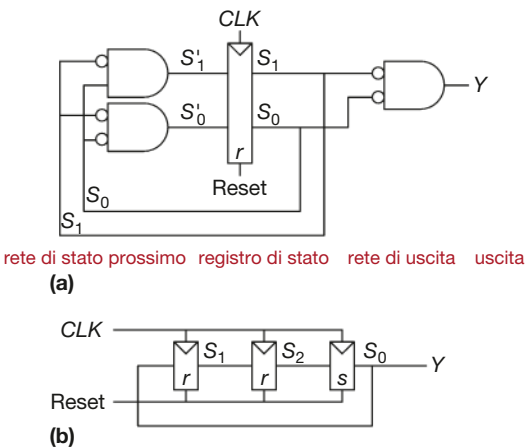
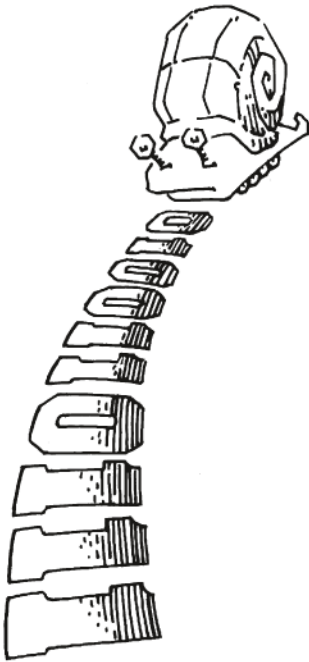


Figura 3.29
Struttura del contatore modulo 3
per (a) codifica binaria
e (b) codifica a singolo 1.

L'aspetto più importante da ricordare circa la differenza fra macchine alla Moore e macchine alla Mealy è che per svolgere la stessa funzione le prime hanno un numero di stati generalmente maggiore, solo in rari casi uguale e mai minore delle seconde.



3.4.3 Macchine alla Moore e macchine alla Mealy

Finora sono stati utilizzati esempi di macchine alla Moore, nelle quali l'uscita dipende solo dallo stato del sistema. Quindi nel diagramma degli stati per le macchine alla Moore i valori delle uscite vengono indicati nei cerchi. Le macchine alla Mealy, come già detto, sono molto simili a quelle alla Moore, ma le uscite possono dipendere sia dallo stato presente sia dagli ingressi. Ne consegue che un diagramma degli stati per una macchina alla Mealy avrà le uscite indicate sugli archi invece che nei cerchi. Il blocco di logica combinatoria che genera le uscite utilizza infatti lo stato presente e gli ingressi, come mostra la Figura 3.22(b).

ESEMPIO 3.7

Confronto tra macchine alla Moore e macchine alla Mealy. Alyssa Guastacomputer ha un robot lumaca con un cervello FSM. La lumaca si sposta da sinistra a destra su un nastro di carta che contiene una sequenza di 1 e 0. A ogni ciclo di clock, la lumaca si sposta fino al bit successivo. La lumaca sorride se gli ultimi due bit attraversati sono 01. Progettare la FSM per calcolare i momenti in cui la lumaca sorride. L'ingresso A è il valore del bit presente in quel momento sotto le antenne della lumaca. L'uscita Y è VERA quando la lumaca sorride. Paragonare i progetti delle due macchine a stati alla Moore e alla Mealy; tracciare un diagramma temporale per ogni macchina che mostri l'ingresso, gli stati e l'uscita quando la lumaca passa sopra la sequenza 0100110111.

Soluzione La macchina alla Moore necessita di tre stati, come mostrato nella **Figura 3.30(a)**. Nell'ipotesi che il diagramma degli stati sia corretto, come si spiega l'arco che va da S_2 a S_1 quando l'ingresso è 0?

In confronto, la macchina alla Mealy richiede solo due stati, come mostra la **Figura 3.30(b)**. Ogni arco è etichettato come A/Y . A è il valore dell'ingresso che causa la transizione, mentre Y è l'uscita corrispondente.

Le **Tabelle 3.11** e **3.12** mostrano la tabella degli stati e delle uscite per la macchina alla Moore, che necessita di almeno due bit di stato. Se si adotta la codifica di stato binaria: $S_0 = 00$, $S_1 = 01$ e $S_2 = 10$, le **Tabelle 3.13** e **3.14** riportano la tabella delle transizioni e la tabella delle uscite con questa codifica.

Da queste tabelle è possibile derivare l'espressione di stato prossimo e l'espressione d'uscita per ispezione. Si noti che queste espressioni si semplificano per il fatto che 11 non esiste, il che significa che lo stato prossimo e l'uscita corrispondenti allo stato non esistente equivalgono a delle indifferenze (non riportate nelle tabelle). Tuttavia, le indifferenze possono essere utilizzate per minimizzare le espressioni:

$$S'_1 = S_0 A \quad (3.8)$$

$$S'_0 = \bar{A}$$

$$Y = S_1 \quad (3.9)$$

La **Tabella 3.15** mostra la tabella unica degli stati e delle uscite per la macchina alla Mealy, che richiede solo un bit di stato. Se si adotta la codifica di stato binaria: $S_0 = 0$ e $S_1 = 1$, si ottiene la tabella delle transizioni e delle uscite riportata nella **Tabella 3.16**. Ancora una volta è possibile derivare le espressioni per ispezione:

$$S'_0 = \bar{A} \quad (3.10)$$

$$Y = S_0 A \quad (3.11)$$

Gli schemi circuitali delle macchine alla Moore e alla Mealy sono riportati nella **Figura 3.31**, mentre la **Figura 3.32** mostra i diagrammi temporali. Le due macchine seguono due diverse sequenze di stati. Inoltre, nella macchina alla Mealy l'uscita passa a 1 in anticipo di un ciclo perché risponde all'ingresso invece di attendere il cambiamento dello stato. Se l'uscita della macchina alla Mealy fosse ritardata da un flip-flop, la sua temporizzazione corrisponderebbe a quella della macchina alla Moore. Nella scelta dello stile di progettazione di una FSM, serve quindi decidere quando si vuole che le uscite rispondano.

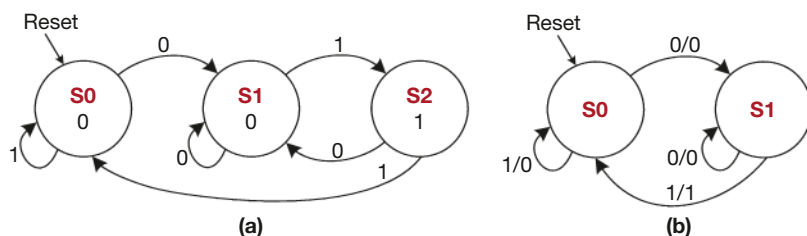


Figura 3.30
Diagrammi degli stati di FSM:
(a) macchina alla Moore,
(b) macchina alla Mealy.

Tabella 3.11 Tabella delle transizioni della macchina alla Moore.

Stato corrente S	Ingresso A	Stato prossimo S'
S0	0	S1
S0	1	S0
S1	0	S1
S1	1	S2
S2	0	S1
S2	1	S0

Tabella 3.12 Tabella delle uscite della macchina alla Moore.

Stato corrente S	Uscita Y
S0	0
S1	0
S2	1

Tabella 3.13 Tabella delle transizioni della macchina alla Moore con codifica degli stati.

Stato corrente S_1 S_0	Ingresso A	Stato prossimo S'_1 S'_0
0 0	0	0 1
0 0	1	0 0
0 1	0	0 1
0 1	1	1 0
1 0	0	0 1
1 0	1	0 0

Tabella 3.14 Tabella delle uscite della macchina alla Moore con codifica degli stati.

Stato corrente S_1 S_0	Ingresso Y
0 0	0
0 1	0
1 0	1

Tabella 3.15 Tabella delle transizioni e tabella delle uscite della macchina alla Mealy.

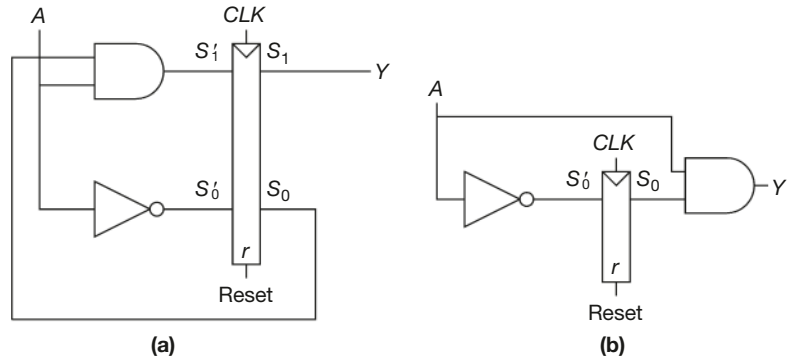
Stato corrente S	Ingresso A	Stato prossimo S'	Uscita Y
S0	0	S1	0
S0	1	S0	0
S1	0	S1	0
S1	1	S0	1

Tabella 3.16 Tabella delle transizioni e tabella delle uscite della macchina alla Mealy con codifica degli stati.

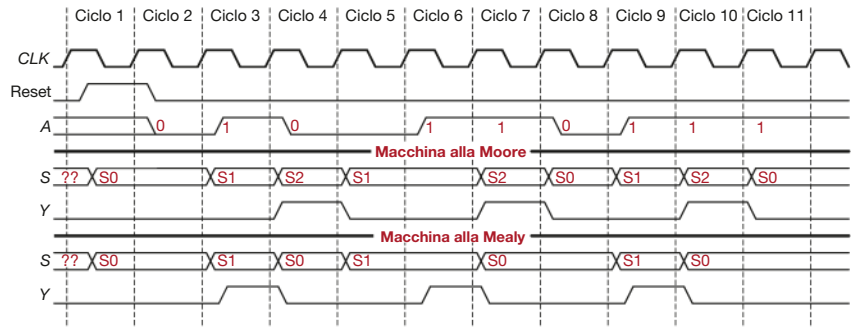
Stato corrente S_0	Ingresso A	Stato prossimo S'_0	Uscita Y
0	0	1	0
0	1	0	0
1	0	1	0
1	1	0	1

Figura 3.31

Schema generale di (a) macchina alla Moore e (b) macchina alla Mealy.

**Figura 3.32**

Diagrammi dei tempi per macchina alla Moore e macchina alla Mealy.



3.4.4 Fattorizzazione delle macchine a stati

Spesso è più semplice progettare FSM complesse se queste possono essere decomposte in diverse macchine a stati più semplici che interagiscono tra loro, facendo sì che le uscite di alcune macchine siano gli ingressi di altre. Questa applicazione dei principi di gerarchia e modularità alle macchine viene chiamata **fattorizzazione** delle macchine a stati.

ESEMPIO 3.8

Macchine a stati senza e con fattorizzazione. Modificare il controllore semaforico del paragrafo 3.4.1 affinché possieda una modalità “parata”, in grado di mantenere il semaforo in viale Ateneo verde mentre gli spettatori e la banda marciano in gruppi verso il campo di calcio. Il controllore riceve due nuovi ingressi: P e R . Se P viene attivato per almeno un ciclo innesca la modalità parata. Attivare R per almeno un ciclo disattiva la modalità parata. Quando in modalità parata, il controllore segue la sua normale sequenza di lavoro fino a che L_B diventa verde, dopodiché rimane nello stato con L_B verde fino a che la modalità parata non viene disattivata.

Per prima cosa, disegnare il diagramma degli stati per una FMS singola, come mostra la **Figura 3.33(a)**. Successivamente, disegnare i diagrammi degli stati per due FMS che interagiscono tra di loro, come mostrato nella **Figura 3.33(b)**. La FSM chiamata Modalità attiva l'uscita M quando, appunto, in modalità parata. La FSM Semafori controlla i semafori basandosi su M e sui sensori di traffico, T_A e T_B .

Soluzione La **Figura 3.34(a)** mostra il progetto di una FSM singola. Gli stati da S_0 a S_3 sono gli stati della modalità normale, mentre la modalità parata è descritta dagli stati da S_4 a S_7 . Le due metà del diagramma sono praticamente identiche, ma nella modalità parata la FSM rimane fissa allo stato S_6 con il semaforo verde su viale Ateneo. Gli ingressi P e R controllano i movimenti tra le due metà del diagramma. La FSM è complessa da progettare. La **Figura 3.34(b)** mostra invece il progetto con fattorizzazione. La FSM Modalità ha due stati per identificare quando è in modalità normale e quando è in modalità parata, mentre la FSM Semafori è stata modificata per rimanere nello stato S_2 quando M è VERO.

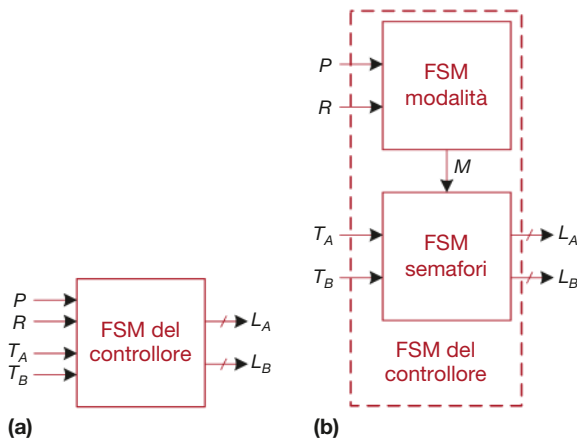


Figura 3.33
Struttura della FSM per il
controllore semaforico modificato
(a) non fattorizzata e
(b) fattorizzata.

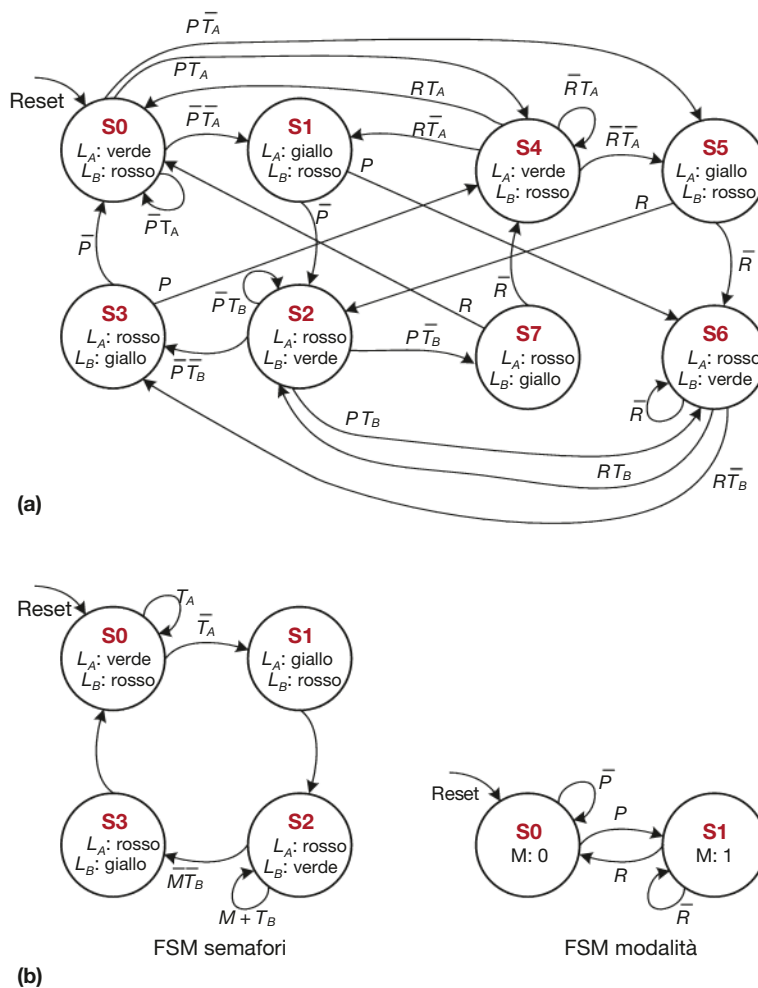


Figura 3.34
Diagrammi degli stati: (a) non
fattorizzato e (b) fattorizzato.

3.4.5 Derivare una FSM da uno schema circuitale

Derivare il diagramma degli stati da uno schema circuitale è praticamente il processo inverso della progettazione di una FSM. Questo processo può essere necessario, ad esempio, quando si studia un progetto la cui documentazione non è completa o quando si vuole capire il funzionamento di un sistema creato da qualcun altro. Serve:

- Esaminare la rete, gli ingressi e le uscite, i bit di stato.
- Scrivere le espressioni di stato prossimo e di uscita.
- Creare le tabelle delle transizioni e delle uscite.
- Ridurre le tabelle eliminando gli stati che non possono essere raggiunti.
- Assegnare un nome simbolico a ogni valida combinazione di bit di stato.
- Riscrivere le tabelle delle transizioni e delle uscite con i nuovi nomi.
- Disegnare il diagramma degli stati.
- Esprimere a parole il funzionamento della FSM.

Nel passo finale si faccia attenzione a descrivere in maniera breve e concisa la funzione e lo scopo generale della FSM, e non semplicemente a raccontare a parole ogni transizione del diagramma degli stati.

ESEMPIO 3.9

Derivare una FSM dal suo schema circuitale. Alyssa Guastacomputer arriva a casa e si accorge che la sua serratura digitale è stata aggiornata e il suo vecchio codice non funziona più. Alla serratura è attaccato un foglio che mostra il diagramma della rete, riportato nella **Figura 3.35**. Alyssa è convinta che la rete sia una macchina a stati finiti e decide quindi di derivare il diagramma degli stati per vedere se può esserle utile per aprire la porta.

Soluzione Per prima cosa, Alyssa esamina la rete: l'ingresso è $A_{1:0}$ e l'uscita è *Sblocca*. I bit di stato sono già stati etichettati nella Figura 3.35. Si tratta, in questo caso, di una macchina alla Moore, perché l'uscita dipende esclusivamente dai bit di stato. A partire dalla rete, Alyssa scrive direttamente le espressioni di stato prossimo e di uscita:

$$\begin{aligned} S'_1 &= S_0 \overline{A_1} A_0 \\ S'_0 &= \overline{S_1} S_0 A_1 A_0 \\ \text{Sblocca} &= S_1 \end{aligned} \quad (3.12)$$

Dopodiché, scrive le tabelle delle transizioni e delle uscite corrispondenti alle espressioni, che sono riportate nelle **Tabelle 3.17** e **3.18**, iniziando a posizionare gli 1 come indicato dall'Espressione 3.12, per poi inserire degli 0 negli spazi rimanenti.

Alyssa riduce la tabella eliminando gli stati che non vengono utilizzati e unendo tra loro le righe grazie alle indifferenze. Lo stato $S_{1:0} = 11$ non è mai indicato come possibile stato prossimo nella Tabella 3.17, quindi le righe con questo come stato presente vengono eliminate. Per lo stato presente $S_{1:0} = 10$, lo stato prossimo è sempre $S_{1:0} = 00$, indipendentemente dagli ingressi, che vengono quindi sostituiti con delle indifferenze. Le tabelle ridotte sono riportate nelle **Tabelle 3.19** e **3.20**.

Successivamente Alyssa assegna un nome simbolico a ogni combinazione dei bit di stato: S_0 è $S_{1:0} = 00$, S_1 è $S_{1:0} = 01$, e S_2 è $S_{1:0} = 10$. Le **Tabelle 3.21** e **3.22** mostrano le tabelle degli stati e delle uscite coi nomi di stato.

Figura 3.35
Struttura della FSM
dell'Esempio 3.9.

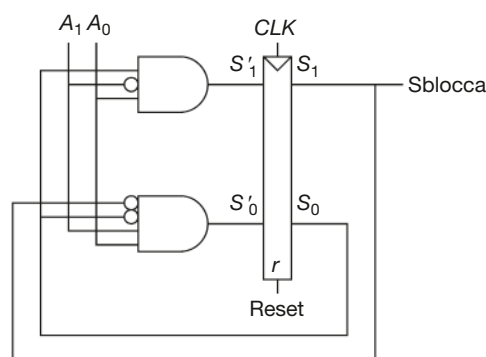


Tabella 3.17 Tabella degli stati prossimi ricavata dalla rete della Figura 3.35.

Stato corrente		Ingresso		Stato prossimo	
S_1	S_0	A_1	A_0	S'_1	S'_0
0	0	0	0	0	0
0	0	0	1	0	0
0	0	1	0	0	0
0	0	1	1	0	1
0	1	0	0	0	0
0	1	0	1	1	0
0	1	1	0	0	0
0	1	1	1	0	0
1	0	0	0	0	0
1	0	0	1	0	0
1	0	1	0	0	0
1	0	1	1	0	0
1	1	0	0	1	0
1	1	0	1	0	0
1	1	1	0	0	0
1	1	1	1	0	0

Tabella 3.18 Tabella delle uscite ricavata dalla rete della Figura 3.35.

Stato corrente		Uscita
S_1	S_0	$Sblocca$
0	0	0
0	1	0
1	0	1
1	1	1

Tabella 3.19 Tabella degli stati prossimi ridotta.

Stato corrente		Ingresso		Stato prossimo	
S_1	S_0	A_1	A_0	S'_1	S'_0
0	0	0	0	0	0
0	0	0	1	0	0
0	0	1	0	0	0
0	0	1	1	0	1
0	1	0	0	0	0
0	1	0	1	1	0
0	1	1	0	0	0
0	1	1	1	0	0
1	0	X	X	0	0

Tabella 3.20 Tabella delle uscite ridotta.

Stato corrente		Uscita
S_1	S_0	$Sblocca$
0	0	0
0	1	0
1	0	1

Tabella 3.21 Tabella degli stati prossimi con codifica simbolica degli stati.

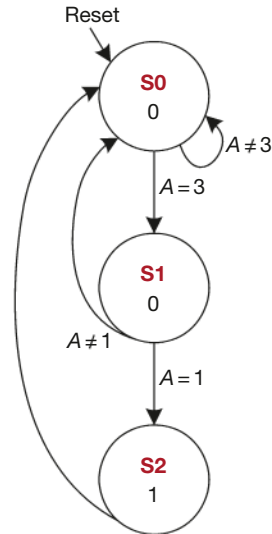
Stato corrente S	Ingresso A	Stato prossimo S'
S0	0	S0
S0	1	S0
S0	2	S0
S0	3	S1
S1	0	S0
S1	1	S2
S1	2	S0
S1	3	S0
S2	X	S0

Tabella 3.22 Tabella delle uscite con codifica simbolica degli stati.

Stato corrente S	Uscita $Sblocca$
S0	0
S1	0
S2	1

Utilizzando queste ultime due tabelle, Alyssa traccia il diagramma degli stati riportato nella **Figura 3.36**. Per ispezione, si accorge del fatto che la macchina a stati finiti sblocca la porta dopo aver riconosciuto all'ingresso il valore tre seguito dal valore uno. Dopodiché la porta si blocca nuovamente. Alyssa prova quindi questo codice sulla serratura digitale e apre finalmente la porta.

Figura 3.36
Diagramma degli stati per la FSM
dell'Esempio 3.9.



3.4.6 Riassunto sulle FSM

Le macchine a stati finiti costituiscono un metodo molto valido di progettazione sistematica di reti sequenziali a partire da una specifica scritta. Per progettare una FSM si utilizza la procedura seguente:

- Identificare gli ingressi e le uscite.
- Disegnare un diagramma degli stati.
- Per una macchina alla Moore:
 - Scrivere la tabella degli stati.
 - Scrivere la tabella delle uscite.
- Per una macchina alla Mealy:
 - Scrivere una tabella unica di stati e uscite.
- Decidere la codifica degli stati, tenendo presente che la scelta influenza il progetto hardware.
- Scrivere le espressioni booleane per la logica di stato prossimo e la logica di uscita.
- Disegnare lo schema della rete.

Le FSM verranno utilizzate spesso nel resto del libro per progettare sistemi digitali anche complessi.

3.5 ■ TEMPORIZZAZIONE DELLA LOGICA SEQUENZIALE

Come già visto, un flip-flop copia il valore presente all'ingresso D sull'uscita Q a ogni fronte di salita del clock. Questo processo viene detto **campionamento** di D al fronte del clock. Se D è **stabile** a 0 o a 1 quando il clock ha il fronte di salita, il comportamento del flip-flop è definito in modo chiaro, ma cosa accade se D sta cambiando nello stesso momento in cui il clock passa da 0 a 1?

Questo problema è paragonabile a quello di una macchina fotografica quando si scatta una fotografia, per esempio di una rana che salta da una foglia di ninfea dentro all'acqua di uno stagno: se la fotografia viene scattata prima che la rana inizi il salto, si vedrà solo la rana ferma sulla ninfea; se viene scattata quando la rana ha già saltato, si vedranno solo le increspature sull'acqua. Ma se la fotografia viene scattata mentre la rana sta saltando, l'immagine della rana risulterà mossa. Una macchina fotografica è caratterizzata dal suo **tempo di apertura**, durante il quale l'oggetto che si sta fotografando deve essere fermo perché l'immagine risulti nitida. Allo stesso modo, un elemento sequenziale ha un tempo di apertura intorno al fronte del clock durante il quale l'ingresso deve essere stabile affinché il flip-flop produca un'uscita ben definita.



Il tempo di apertura di un elemento sequenziale viene definito da un tempo di *setup* e da un tempo di *hold*, rispettivamente prima e dopo il fronte del clock. Proprio come la disciplina statica costringe a utilizzare livelli logici fuori dalla zona proibita, così la **disciplina dinamica** costringe a lavorare con segnali che cambiano al di fuori del tempo di apertura. Facendo riferimento alla disciplina dinamica, è possibile interpretare il tempo come costituito da unità discrete chiamate **cicli di clock**, proprio come si può pensare ai livelli di segnale in termini di valori discreti 1 e 0. Un segnale può variare e oscillare per un certo periodo di tempo; tuttavia, nella disciplina dinamica, l'unico aspetto che interessa è il valore finale assunto al termine del ciclo di clock, quando il segnale si è stabilizzato su un valore definito. Si può dunque scrivere semplicemente $A[n]$ per indicare il valore del segnale A alla fine dell' n -esimo ciclo di clock, dove n è un numero intero, piuttosto che scrivere $A(t)$, che corrisponde al valore del segnale A all'istante t , dove t è un qualsiasi numero reale.

Il periodo del clock deve essere abbastanza lungo da permettere a tutti i segnali di stabilizzarsi, il che costituisce un limite per la velocità del sistema. Nei sistemi reali, il clock solitamente non raggiunge tutti i flip-flop allo stesso tempo e questa differenza di tempo, detta **sfasamento del clock**, aumenta ulteriormente il periodo di clock necessario.

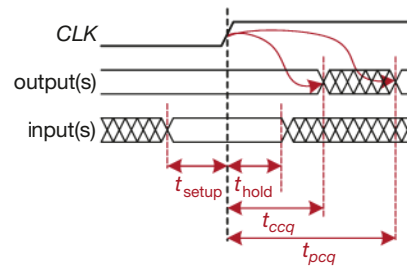
A volte risulta impossibile soddisfare la disciplina dinamica, specialmente quando bisogna interfacciarsi con il mondo esterno. Per esempio, si consideri una rete con un ingresso dato da un pulsante: l'operatore potrebbe premere il pulsante proprio al momento del fronte di clock. Questo potrebbe dare luogo al fenomeno chiamato **metastabilità**, che si verifica quando un flip-flop campiona un valore compreso tra 0 e 1; la metastabilità può richiedere un tempo a priori illimitato perché la rete sia in grado di ritornare a un valore logico accettabile. Una soluzione nel caso di ingressi asincroni di questo tipo è l'utilizzo di un sincronizzatore, che tuttavia ha una probabilità piccola ma non nulla di produrre un valore logicamente incorretto.

La parte restante di questo paragrafo approfondisce questi concetti.

3.5.1 La disciplina dinamica

È stata finora studiata la specifica funzionale delle reti sequenziali. Ma una rete sequenziale sincrona, come un flip-flop o una FSM, ha anche una specifica temporale, come mostrato nella **Figura 3.37**. Quando il clock presenta il fronte di salita, l'uscita o le uscite iniziano a cambiare dopo il **ritardo di contaminazione** da clock a Q (chiamato t_{cq}) e devono stabilizzarsi sul valore definitivo entro il **ritardo di propagazione** da clock a Q (chiamato t_{pcq}). Questi ritardi rappresentano rispettivamente il ritardo più rapido e il più lento di attraversamento della rete. Perché la rete interpreti in maniera corretta l'ingresso o gli ingressi, questi devono essersi stabilizzati almeno entro il **tempo**

Figura 3.37
Specifica temporale di una rete
sequenziale sincrona.



di setup (o tempo di attivazione) t_{setup} prima del fronte di salita del clock e devono rimanere stabili per la durata almeno del **tempo di hold** (tempo di mantenimento) t_{hold} dopo il fronte di salita del clock. La somma del tempo di setup e del tempo di hold è detta **tempo di apertura** perché rappresenta il tempo totale durante il quale l'ingresso deve rimanere stabile.

La disciplina dinamica afferma che gli ingressi di una rete sequenziale sincrona devono rimanere stabili durante il tempo di apertura (setup più hold) prima e dopo il fronte del clock. Imponendo questo requisito, si garantisce il campionamento dei segnali da parte del flip-flop quando questi sono stabili. Dal momento che l'aspetto importante è il valore finale degli ingressi al momento in cui questi vengono campionati, è possibile in questo modo trattare i segnali come discreti sia nel tempo sia nei livelli logici.

3.5.2 Temporizzazione del sistema

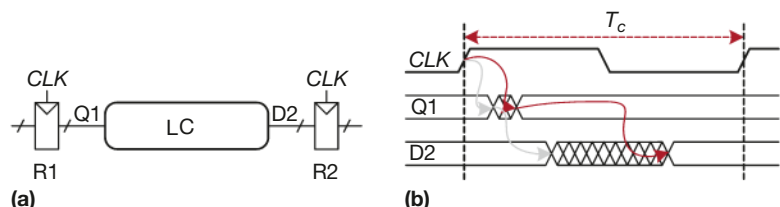
Il **periodo di clock** o **tempo di ciclo**, T_c , è il tempo tra i fronti di salita del segnale periodico di clock. Il suo reciproco, $f_c = 1/T_c$, è la **frequenza di clock**. Se tutti gli altri parametri restano identici, aumentare la frequenza di clock aumenta la quantità di lavoro per unità di tempo che un sistema digitale può svolgere. La frequenza viene misurata in Hertz (Hz), ovvero in cicli per secondo: 1 megahertz (MHz) = 10^6 Hz, e 1 gigahertz (GHz) = 10^9 Hz.

La **Figura 3.38(a)** mostra un percorso generico in una rete sequenziale sincrona di cui si vuole calcolare il periodo di clock. Al fronte di salita del clock, il registro R1 produce un'uscita (o delle uscite) Q1. Questi segnali entrano in un blocco di logica combinatoria e producono D2, che rappresenta l'ingresso (o gli ingressi) per il registro R2. Il diagramma temporale nella **Figura 3.38(b)** mostra che ogni segnale d'uscita inizia a cambiare dopo un ritardo di contaminazione da quando il suo ingresso è cambiato, e si stabilizza sul valore finale entro un ritardo di propagazione da quando il suo ingresso si è stabilizzato. Le frecce grigie rappresentano il ritardo di contaminazione attraverso R1 e la logica combinatoria, mentre le frecce rosse rappresentano il ritardo di propagazione attraverso R1 e la logica combinatoria. Si possono ora analizzare i vincoli temporali riguardo ai tempi di setup e hold del secondo registro R2.

Vincolo sul tempo di setup

La **Figura 3.39** rappresenta il diagramma temporale che mostra unicamente il ritardo massimo attraverso il sistema, indicato dalle frecce rosse. Per soddisfare il tempo di setup di R2, D2 deve stabilizzarsi non più tardi di un tempo

Figura 3.38
Percorso tra i registri e diagramma
temporale.



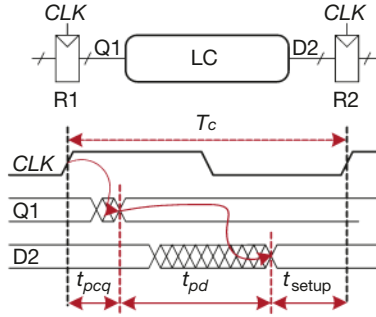


Figura 3.39
Ritardo massimo per rispettare i vincoli sul tempo di setup.

di setup prima del fronte di clock successivo. È quindi possibile derivare l'espressione che esprime il periodo minimo del clock:

$$T_c \geq t_{pcq} + t_{pd} + t_{setup} \quad (3.13)$$

Nei progetti commerciali, il periodo del clock viene spesso dettato dal capo progetto o dal reparto vendite (in modo da assicurarsi che il prodotto sia competitivo sul mercato). Inoltre, vengono specificati dal produttore il ritardo di propagazione da clock a Q e il tempo di setup (t_{pcq} e t_{setup}) del flip-flop. È dunque possibile modificare l'Espressione 3.13 per calcolare il massimo ritardo di propagazione attraverso la logica combinatoria, che è solitamente l'unica variabile soggetta al controllo diretto del progettista.

$$t_{pd} \leq T_c - (t_{pcq} + t_{setup}) \quad (3.14)$$

Il termine racchiuso in parentesi, $t_{pcq} + t_{setup}$, è chiamato **sovraccarico di sequenziamento**. Idealmente, l'intero tempo di ciclo T_c sarebbe disponibile per le elaborazioni da parte della logica combinatoria, t_{pd} . Invece il sovraccarico di sequenziamento del flip-flop riduce questo tempo. L'Espressione 3.14 viene chiamata **vincolo sul tempo di setup** o **vincolo di ritardo massimo**, perché dipende dal tempo di setup e vincola il ritardo massimo attraverso la logica combinatoria.

Se il ritardo di propagazione attraverso la logica combinatoria è troppo grande, D2 potrebbe non stabilizzarsi sul suo valore finale in tempo per permettere a R2 di elaborarlo, dato che R2 necessita di un valore stabile. Di conseguenza R2 potrebbe produrre un risultato incorretto o addirittura un livello logico illegale, ovvero un livello che rientra nella regione proibita. In questo caso, la rete darebbe luogo a un malfunzionamento. Il problema può essere risolto aumentando il periodo di clock o ridefinendo la logica combinatoria affinché abbia un ritardo di propagazione inferiore.

Vincolo sul tempo di hold

Il registro R2 nella Figura 3.38(a) possiede anche un **vincolo sul tempo di hold**. Infatti il suo ingresso, D2, non deve cambiare per un dato periodo di tempo, t_{hold} , dopo il fronte di salita del clock. Secondo la **Figura 3.40**, D2 può cambiare appena trascorso un tempo $t_{ccq} + t_{cd}$ dopo il fronte di salita del clock. Di conseguenza:

$$t_{ccq} + t_{cd} \geq t_{hold} \quad (3.15)$$

Ancora una volta, t_{ccq} e t_{hold} sono caratteristiche del flip-flop che non sono generalmente sotto il controllo del progettista. Si può modificare l'espressione per ottenere il minimo ritardo di contaminazione attraverso la logica combinatoria come segue:

$$t_{cd} \geq t_{hold} - t_{ccq} \quad (3.16)$$

L'Espressione 3.16 viene detta **vincolo sul tempo di hold** o **vincolo di ritardo minimo** perché vincola il ritardo minimo attraverso la logica combinatoria.

Figura 3.40
Ritardo minimo per rispettare
i vincoli sul tempo di hold.

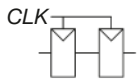
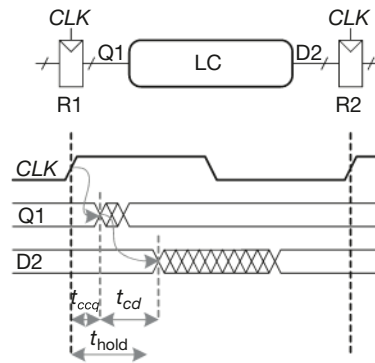


Figura 3.41
Due flip-flop collegati direttamente.

Come già detto, si parte dal presupposto che ogni elemento logico possa essere connesso ad altri elementi senza introdurre problemi temporali. In particolare, ci si aspetta che due flip-flop possano essere connessi tra loro come mostra la **Figura 3.41** senza creare problemi di tempo di hold.

In questo caso, $t_{cd} = 0$ perché non c'è nessuna logica combinatoria tra i due flip-flop. Inserendo questo termine nell'Espressione 3.16 si arriva a

$$t_{\text{hold}} \geq t_{\text{ccq}} \quad (3.17)$$

In altre parole, un flip-flop affidabile dovrebbe avere un tempo di hold minore del suo ritardo di contaminazione. Spesso, i flip-flop sono progettati in modo da avere $t_{\text{hold}} = 0$, quindi l'Espressione 3.17 è sempre soddisfatta. A meno che non sia specificato altrimenti, nel libro si parte sempre da questo presupposto, in modo da ignorare il vincolo sul tempo di hold.

Tuttavia, i vincoli sul tempo di hold sono estremamente importanti: se non vengono rispettati, l'unica soluzione è l'aumento del ritardo di contaminazione attraverso la logica, il che significa riprogettare completamente la rete. Diversamente dai vincoli sul tempo di setup, i vincoli sul tempo di hold non possono essere risolti modificando il periodo del clock. Riprogettare una rete già integrata e produrre il progetto rivisto è un'operazione che richiede mesi e milioni di dollari con le tecnologie avanzate disponibili al giorno d'oggi, quindi le **violazioni del tempo di hold** devono essere tenute nella massima considerazione.

Riassumendo

Le reti sequenziali hanno vincoli sul tempo di setup e sul tempo di hold che impongono i ritardi massimi e minimi della logica combinatoria tra i diversi flip-flop. I flip-flop moderni vengono generalmente progettati in modo che il ritardo minimo attraverso la logica combinatoria sia 0 (cioè in modo che i flip-flop possano essere collegati direttamente l'uno all'altro). Il vincolo sul ritardo massimo limita il numero di porte consecutive che possono essere poste sul percorso critico di una rete ad alta velocità, visto che un'alta frequenza di clock corrisponde a un periodo di clock breve.

ESEMPIO 3.10

Analisi temporale. Ben Imbrogliabit ha progettato la rete mostrata nella **Figura 3.42**. Secondo le specifiche tecniche dei componenti usati, i flip-flop hanno un ritardo di contaminazione da clock a Q di 30 ps e un ritardo di propagazione di 80 ps. Possiedono inoltre un tempo di setup di 50 ps e un tempo di hold di 60 ps. Ogni porta logica ha un ritardo di propagazione di 40 ps e un ritardo di contaminazione di 25 ps. Serve aiutare Ben a determinare la massima frequenza di clock e a capire se può verificarsi una violazione del tempo di hold. Questo processo prende il nome di **analisi temporale**.

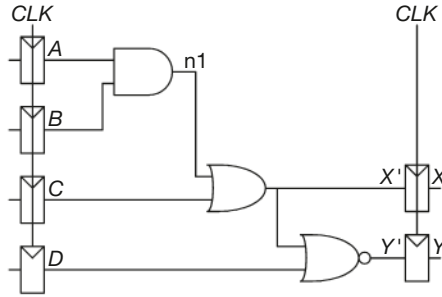


Figura 3.42
Esempio di rete per l'analisi temporale.

Soluzione La **Figura 3.43(a)** mostra le forme d'onda che illustrano i momenti in cui i segnali potrebbero cambiare. Gli ingressi, A e D, sono uscite di registri, quindi i loro valori cambiano solo sul fronte di salita di CLK.

Il percorso critico si verifica quando $B = 1$, $C = 0$, $D = 0$, e A passa da 0 a 1, facendo sì che n1 si alzi, X' si alzi, e Y' si abbassi, come mostrato nella **Figura 3.43(b)**. Questo percorso include tre ritardi di porte. Per valutare il percorso critico, si parte dal presupposto che ogni porta necessiti di tutto il suo ritardo di propagazione. Y' deve stabilizzarsi prima del prossimo fronte in salita di CLK. Di conseguenza, il tempo di ciclo minimo è:

$$T_c \geq t_{pcq} + 3t_{pd} + t_{setup} = 80 + 3 \times 40 + 50 = 250 \text{ ps} \quad (3.18)$$

La frequenza massima di clock è dunque $f_c = 1/T_c = 4 \text{ GHz}$.

Un percorso breve ha luogo quando $A = 0$ e C si alza, causando l'innalzamento di X' , come mostra la **Figura 3.43(c)**. Per valutare il percorso breve si parte dal presupposto che ogni porta cambi subito dopo il ritardo di contaminazione. Questo percorso contiene solo un ritardo di porta, quindi un cambiamento può verificarsi dopo $t_{ccq} + t_{cd} = 30 + 25 = 55 \text{ ps}$. Tuttavia, il flip-flop ha un tempo di hold di 60 ps, il che significa che X' deve rimanere stabile per 60 ps dopo il fronte di salita di CLK affinché il suo valore venga campionato correttamente. In questo caso, $X' = 0$ al primo fronte di salita di CLK, e si vuole che il flip-flop campioni il valore $X = 0$. Dal momento che X' non rimane stabile per un tempo sufficiente, il valore finale di X non è prevedibile. La rete presenta quindi una violazione del tempo di hold e potrebbe avere un comportamento incorretto a qualsiasi frequenza del clock.

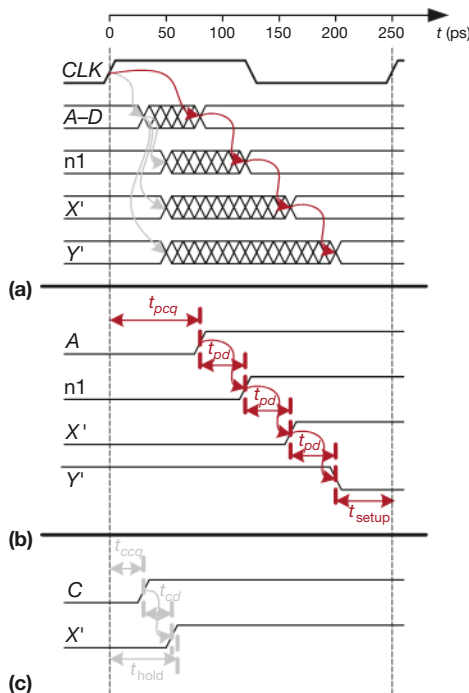


Figura 3.43
Diagrammi dei tempi: (a) caso generale, (b) percorso critico, (c) percorso breve.

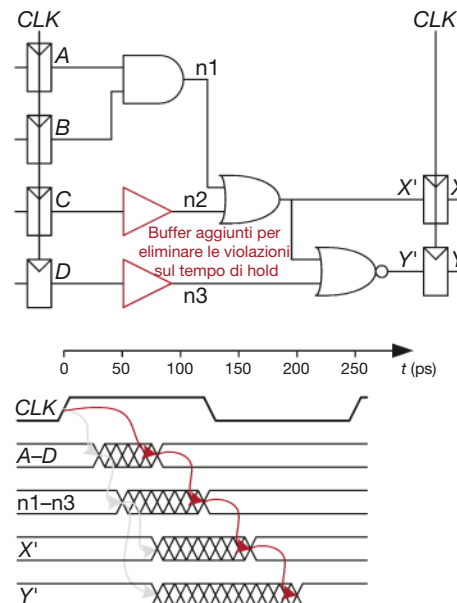
ESEMPIO 3.11

Eliminare le violazioni del vincolo sul tempo di hold. Alyssa Guastacomputer propone di correggere la rete di Ben aggiungendo dei buffer per rallentare i percorsi brevi, come mostrato nella **Figura 3.44**. I buffer hanno gli stessi ritardi delle altre porte. Serve aiutare Alyssa a determinare la massima frequenza di clock e a capire se possono verificarsi dei problemi sul tempo di hold.

Soluzione La **Figura 3.45** mostra le forme d'onda che illustrano i momenti in cui i segnali potrebbero cambiare. Il percorso critico da A a Y non subisce variazioni, dal momento che non attraversa nessun buffer e quindi la sua frequenza di clock massima resta 4 GHz. Tuttavia, i percorsi brevi vengono rallentati dal ritardo di contaminazione del buffer: ora X' non cambia fino a che non sono trascorsi $t_{ccq} + 2t_{cd} = 30 + 2 \times 25 = 80$ ps. Questo succede quindi dopo che sono trascorsi i 60 ps del tempo di hold, dunque la rete opera in maniera corretta.

Figura 3.44
Rete modificata per risolvere il problema sul tempo di hold.

Figura 3.45
Diagrammi dei tempi con i buffer inseriti per risolvere il problema sul tempo di hold.



Questo esempio aveva un tempo di hold particolarmente lungo per evidenziare i problemi sul tempo di hold; la maggior parte dei flip-flop è progettata con $t_{hold} < t_{ccq}$ per evitare questi problemi. Alcuni microprocessori ad alte prestazioni, incluso il Pentium 4, utilizzano però un elemento chiamato **latch a impulsi** al posto di un flip-flop. Il latch a impulsi si comporta come un flip-flop ma ha un ritardo da clock a Q breve e un tempo di hold lungo. In generale, l'aggiunta dei buffer è spesso, ma non sempre, in grado di risolvere i problemi di tempo senza rallentare il percorso critico della rete.

3.5.3 Sfasamento del clock*

Nell'analisi precedente si è partiti dal presupposto che il clock raggiungesse tutti i registri esattamente nello stesso istante. Nella realtà, questo tempo mostra una certa variabilità, che fa sì che i fronti di clock non si presentino tutti nello stesso istante: tale fenomeno è chiamato **sfasamento del clock** (*clock skew*). Per esempio, i fili che collegano la sorgente del clock ai diversi registri possono essere di lunghezze differenti, il che dà come risultato dei ritardi leggermente diversi, come mostrato nella **Figura 3.46**. Anche il rumore può portare a ritardi differenti. La scelta di far passare il segnale di clock attraverso porte logiche (il cosiddetto *clock gated*) descritta nel paragrafo 3.2.5, rallenta ulteriormente il clock. Se alcuni clock sono gated mentre altri non lo sono, tra i primi e i secondi ci sarà uno sfasamento importante. Nella **Figura 3.46** il CLK2 è in anticipo rispetto al CLK1 perché il filo di clock tra i due registri

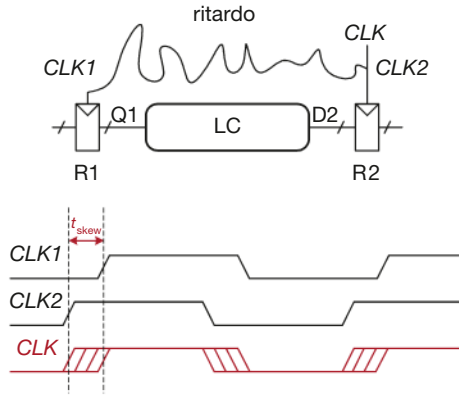


Figura 3.46
Sfasamento del clock dovuto ai ritardi nei fili.

segue uno strano percorso tortuoso. Se il percorso del clock fosse stato differente, si sarebbe potuto avere $CLK1$ in anticipo rispetto a $CLK2$. Quando si esegue l'analisi temporale bisogna considerare lo scenario peggiore, in modo da garantire che la rete funzioni in qualsiasi circostanza.

La **Figura 3.47** aggiunge lo sfasamento al diagramma temporale della **Figura 3.38**. Il tratto spesso indica il momento più ritardato nel quale il segnale del clock può raggiungere qualsiasi registro; i tratti ripetuti a rastrello servono a indicare che il clock può arrivare con un anticipo massimo pari a t_{skew} .

Per prima cosa si consideri il vincolo sul tempo di setup mostrato nella **Figura 3.48**. Nel caso peggiore, R1 riceve il clock con il massimo sfasamento e R2 lo riceve con il minimo sfasamento, lasciando il minor tempo possibile ai dati per propagarsi tra i registri.

I dati attraversano i registri e la logica combinatoria e devono stabilizzarsi prima che R2 li campioni. Si può quindi concludere che

$$T_c \geq t_{pcq} + t_{pd} + t_{setup} + t_{skew} \quad (3.19)$$

$$T_{pd} \leq T_c - (t_{pcq} + t_{setup} + t_{skew}) \quad (3.20)$$

Si consideri ora il vincolo sul tempo di hold nella **Figura 3.49**. Nello scenario peggiore, R1 riceve il clock con il minimo sfasamento, $CLK1$, e R2 lo riceve

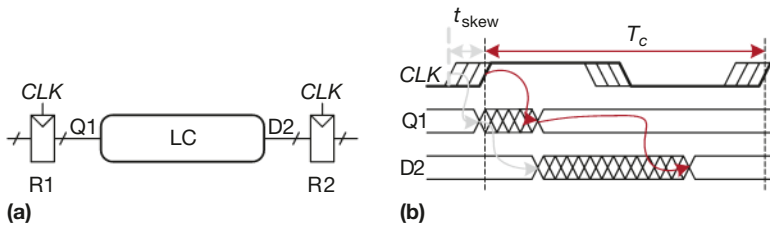


Figura 3.47
Diagramma dei tempi con sfasamento del clock.

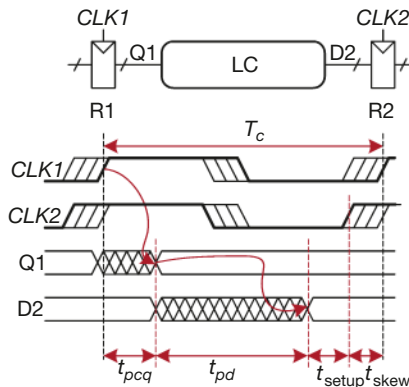
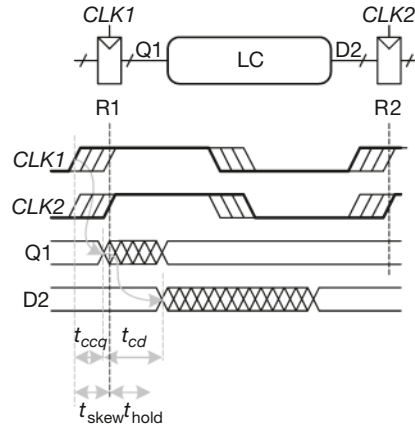


Figura 3.48
Vincoli sul tempo di setup con sfasamento del clock.

Figura 3.49
Vincoli sul tempo di hold
con sfasamento del clock.



con il massimo sfasamento, $CLK2$. I dati attraversano il registro e la logica combinatoria ma non devono arrivare prima di un tempo di hold dopo l'ultimo clock. Quindi, si deduce che

$$t_{ccq} + t_{cd} \geq t_{hold} + t_{skew} \quad (3.21)$$

$$t_{cd} \geq t_{hold} + t_{skew} - t_{ccq} \quad (3.22)$$

Riassumendo, lo sfasamento del clock aumenta sia il tempo di setup sia il tempo di hold. Si aggiunge quindi al sovraccarico di sequenziamento, riducendo il tempo disponibile per il lavoro utile a livello di logica combinatoria. Aumenta inoltre anche il ritardo minimo richiesto attraverso la logica combinatoria. Anche se $t_{hold} = 0$, una coppia di flip-flop connessi direttamente violerebbe l'Espressione 3.22 se $t_{skew} > t_{ccq}$. Per prevenire i gravi errori legati al tempo di hold, i progettisti non devono permettere che ci sia un grande sfasamento del clock. A volte i flip-flop vengono appositamente progettati per essere lenti (cioè per avere un grande t_{ccq}) per prevenire problemi a livello di tempo di hold anche quando lo sfasamento del clock è significativo.

ESEMPIO 3.12

Analisi temporale con lo sfasamento del clock. Rivedere l'Esempio 3.10 assumendo che il sistema abbia 50 ps di sfasamento del clock.

Soluzione Il percorso critico rimane invariato, ma il tempo di setup viene aumentato dallo sfasamento. Di conseguenza, il minimo tempo di ciclo diventa

$$T_c \geq t_{pcq} + 3t_{pd} + t_{setup} + t_{skew} = 80 + 3 \times 40 + 50 + 50 = 300 \text{ ps} \quad (3.23)$$

La frequenza massima di clock è $f_c = 1/T_c = 3.33 \text{ GHz}$.

Anche il percorso breve resta invariato a 55 ps. Tuttavia, il tempo di hold viene aumentato dallo sfasamento a $60 + 50 = 110 \text{ ps}$, che è molto più di 55 ps. Di conseguenza, la rete viola il vincolo sul tempo di hold e non funziona correttamente indipendentemente dalla frequenza. Questa rete violava il vincolo sul tempo di hold anche senza sfasamento, quindi lo sfasamento non fa altro che peggiorare la situazione.

ESEMPIO 3.13

Eliminare le violazioni del vincolo sul tempo di hold. Rivedere l'Esempio 3.11 assumendo che il sistema abbia 50 ps di sfasamento del clock.

Soluzione Il percorso critico rimane invariato, quindi la frequenza massima del clock rimane 3.3 GHz.

Il percorso breve viene invece aumentato a 80 ps, che tuttavia è comunque inferiore a $t_{hold} + t_{skew} = 110 \text{ ps}$, quindi la rete viola ancora il vincolo sul tempo di hold.

Per risolvere il problema si potrebbero aggiungere altri buffer. Questi ultimi andrebbero però aggiunti anche al percorso critico, cosa che ridurrebbe la frequenza di clock. Una soluzione alternativa è l'utilizzo di un flip-flop migliore, con un tempo di hold più breve.

3.5.4 Metastabilità

Come già accennato, non è sempre possibile garantire che l'ingresso di una rete sequenziale sia stabile durante il tempo di apertura, specialmente quando questo ingresso arriva dal mondo esterno. Si consideri un pulsante collegato all'ingresso di un flip-flop, come mostrato nella **Figura 3.50**. Quando il pulsante non viene premuto, $D = 0$. Quando invece si preme il pulsante, $D = 1$. Un operatore preme il pulsante in un momento casuale rispetto ai fronti di salita di CLK . Si vuole conoscere il valore dell'uscita Q dopo il fronte di salita di CLK . Nel Caso I, quando il pulsante viene premuto molto in anticipo rispetto a CLK , $Q = 1$. Nel Caso II, quando il pulsante non viene premuto se non parecchio dopo CLK , $Q = 0$. Ma nel Caso III, quando il pulsante viene premuto in un momento indefinito compreso tra t_{setup} prima di CLK e t_{hold} dopo CLK , l'ingresso viola la disciplina dinamica e l'uscita non è definita.

Stato metastabile

Quando un flip-flop campiona un ingresso che sta cambiando durante il tempo di apertura, l'uscita Q potrebbe momentaneamente assumere una tensione compresa tra 0 e V_{DD} , cioè nella zona proibita. Questo viene chiamato **stato metastabile**. Col tempo il flip-flop risolve il problema facendo sì che l'uscita raggiunga uno **stato stabile** pari a 0 o a 1, ma non c'è un limite superiore al tempo di risoluzione della metastabilità (*resolution time*) necessario per portarsi in uno stato stabile.

Lo stato metastabile di un flip-flop può essere paragonato a una palla posta sulla sommità di una collina, con una valle da ogni lato, come mostra la **Figura 3.51**. Le due vallate rappresentano gli stati stabili, visto che una palla nella valle rimane ferma fino a che non viene disturbata. La sommità della collina rappresenta invece lo stato metastabile, perché la palla può rimanere sulla sommità solo se perfettamente bilanciata. Dal momento che un bilanciamento perfetto non è possibile, la palla finirà col rotolare da un lato o dall'altro. Il tempo necessario affinché la palla ricada in una delle due vallate dipende da quanto stabile e bilanciata era all'inizio. Ogni dispositivo bistabile ha uno stato metastabile tra due stati stabili.

Tempo di stabilizzazione

Se l'ingresso di un flip-flop cambia in un istante casuale durante il ciclo di clock, il tempo di risoluzione della metastabilità, t_{res} , richiesto perché l'uscita venga riportata a uno stato stabile, costituisce una variabile altrettanto casuale. Se l'ingresso cambia al di fuori del tempo di apertura, allora $t_{res} = t_{pcq}$. Se invece l'ingresso cambia all'interno del tempo di apertura, t_{res} può essere sensibilmente più lungo. Analisi sia teoriche sia sperimentali (vedi par. 3.5.6) hanno dimostrato che la probabilità che il tempo di stabilizzazione, t_{res} , ecceda un tempo arbitrario, t , diminuisce esponenzialmente con t :

$$P(t_{res} > t) = \frac{T_0}{T_c} e^{-\frac{t}{\tau}} \quad (3.24)$$

dove T_c è il periodo di clock, e T_0 e τ sono caratteristici del flip-flop. L'espressione è valida solo se t è significativamente più lungo di t_{pcq} .

Intuitivamente, T_0/T_c rappresenta la probabilità che l'ingresso cambi al momento sbagliato (cioè durante il tempo di apertura); questa probabilità

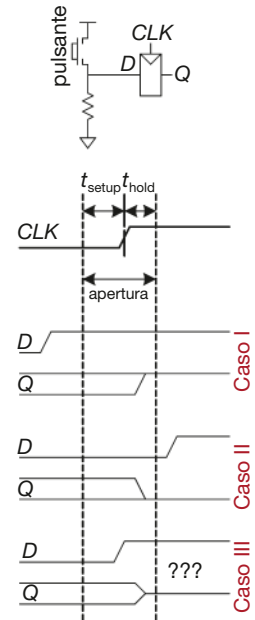


Figura 3.50
Ingresso che cambia prima, dopo o durante il tempo di apertura.

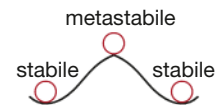


Figura 3.51
Stati stabili e metastabili.

diminuisce col tempo di ciclo, T_c . τ è una costante di tempo che indica quanto velocemente il flip-flop si allontana dallo stato metastabile; in particolare, questo tempo è legato al ritardo che si accumula tra le due porte collegate a croce del flip-flop.

Per riassumere, se l'ingresso di un dispositivo bistabile come un flip-flop cambia durante il tempo di apertura, l'uscita potrebbe assumere un valore metastabile per un certo periodo di tempo prima di stabilizzarsi su uno 0 o un 1. La quantità di tempo richiesta perché il problema venga risolto non ha limiti, poiché per qualsiasi tempo finito, t , la probabilità che il flip-flop sia ancora metastabile non è 0. Tuttavia questa probabilità diminuisce esponenzialmente all'aumentare di t . Di conseguenza, se si attende abbastanza, e cioè un tempo ben maggiore di t_{pcp} , c'è un'elevatissima probabilità che il flip-flop raggiunga un livello logico valido.

3.5.5 Sincronizzatori

La presenza di ingressi asincroni provenienti dal mondo reale in un sistema digitale è inevitabile. Per esempio, un ingresso umano è un ingresso asincrono. Se non vengono trattati con la dovuta cura e attenzione, questi ingressi possono portare a tensioni metastabili all'interno del sistema, che causano malfunzionamenti particolarmente difficili da identificare e correggere. L'obiettivo di un progettista di sistemi digitali dovrebbe quindi essere quello di assicurarsi che, data la presenza di ingressi asincroni, la probabilità di incontrare tensioni metastabili sia sufficientemente bassa, tenendo presente che quel "sufficientemente" viene definito dal contesto. Per un telefono cellulare, un guasto ogni 10 anni è più che accettabile, perché l'utente può sempre spegnere il dispositivo e riaccenderlo quando questo si blocca. Per un dispositivo medico, invece, il punto di riferimento dovrebbe essere piuttosto un guasto nel tempo di vita dell'universo (10^{10} anni). Per garantire dei livelli logici corretti, tutti gli ingressi asincroni dovrebbero essere fatti passare attraverso dei **sincronizzatori**.

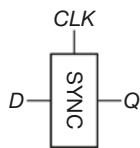


Figura 3.52
Simbolo del sincronizzatore.

Il sincronizzatore, mostrato nella **Figura 3.52**, è un dispositivo che riceve un ingresso asincrono D e un clock CLK , per produrre un'uscita Q entro un certo periodo di tempo; la probabilità che l'uscita abbia un livello logico valido è estremamente alta. Se D è stabile durante il tempo di apertura, Q assume lo stesso valore di D . Se invece D cambia durante il tempo di apertura, Q può assumere un valore ALTO o BASSO, ma non può assumere un valore metastabile.

La **Figura 3.53** mostra un modo semplice di costruire un sincronizzatore a partire da due flip-flop. F1 campiona D al fronte di salita di CLK : se D cambia in quel momento, l'uscita D2 potrebbe momentaneamente assumere un valore metastabile. Se il periodo del clock è abbastanza lungo, D2 con elevata probabilità si stabilizza su un livello logico valido prima della fine del periodo. Successivamente F2 campiona D2, che a questo punto è stabile, producendo un'uscita accettabile Q .

Si dice che un sincronizzatore fallisce se Q , l'uscita del sincronizzatore, diventa metastabile. Questo può succedere se D2 non si stabilizza su un livello valido prima di essere campionato da F2 (e cioè se $t_{res} > T_c - t_{setup}$). Secondo l'Espressione 3.24, la probabilità di un errore per il cambiamento di un singolo ingresso a un istante qualsiasi è

$$P(\text{errore}) = \frac{T_0}{T_c} e^{-\frac{T_c - T_{\text{setup}}}{\tau}} \quad (3.25)$$

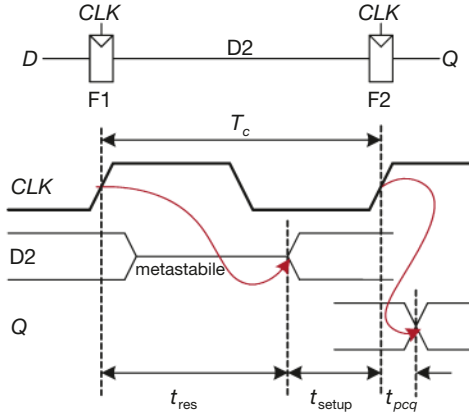


Figura 3.53
Un semplice sincronizzatore.

La probabilità di errore, $P(\text{errore})$, è la probabilità che l'uscita Q sia metastabile in seguito a un singolo cambiamento in D . Se D cambia ogni secondo, la probabilità di errore al secondo è pari a $P(\text{errore})$; tuttavia, se D cambia N volte al secondo, la probabilità di errore al secondo è N volte più grande:

$$P(\text{errore}) / \text{sec} = N \frac{T_0}{T_c} e^{-\frac{T_c - T_{\text{setup}}}{\tau}} \quad (3.26)$$

L'affidabilità di un sistema viene spesso misurata come **tempo medio tra errori** (MTBF, *Mean Time Between Failures*). Come suggerito dal nome, MTBF è la quantità media di tempo tra due errori del sistema. Si tratta del reciproco della probabilità che il sistema generi un errore a un certo istante di tempo:

$$MTBF = \frac{1}{P(\text{errore}) / \text{sec}} = \frac{T_c e^{\frac{T_c - T_{\text{setup}}}{\tau}}}{N T_0} \quad (3.27)$$

L'Espressione 3.27 mostra che MTBF aumenta esponenzialmente con il crescere del tempo di attesa T_c del sincronizzatore. Per la maggior parte dei sistemi, un sincronizzatore che attende per un ciclo di clock garantisce un MTBF sicuro. Nei sistemi particolarmente veloci, invece, potrebbe essere necessario attendere per più cicli.

ESEMPIO 3.14

Sincronizzatore per un ingresso di FSM. La FSM del controllore semaforico del paragrafo 3.4.1 riceve ingressi asincroni dai sensori di traffico. Si supponga che venga utilizzato un sincronizzatore per garantire ingressi stabili al controllore. Il traffico arriva in media 0.2 volte al secondo. I flip-flop nel sincronizzatore hanno le seguenti caratteristiche: $\tau = 200$ ps, $T_0 = 150$ ps, $t_{\text{setup}} = 500$ ps. Quanto lungo deve essere il periodo di clock del sincronizzatore affinché MTBF sia maggiore di 1 anno?

Soluzione 1 anno $\approx \pi \times 10^7$ secondi. Si deve risolvere l'Equazione 3.27 nella variabile T_c .

$$\pi \times 10^7 = \frac{T_c e^{\frac{T_c - 500 \times 10^{-12}}{200 \times 10^{-12}}}}{(0.2)(150 \times 10^{-12})} \quad (3.28)$$

Questa equazione non ha una soluzione in forma chiusa. Tuttavia, è abbastanza facile da risolvere andando per tentativi. Si può usare un foglio elettronico per provare diversi valori di T_c e calcolarne MTBF fino a trovare il valore di T_c che restituisce MTBF di 1 anno. $T_c = 3.036$ ns.

3.5.6 Formulazione del tempo di risoluzione*

L'Espressione 3.24 può essere ricavata se si ha una conoscenza di base di teoria delle reti, delle equazioni differenziali e del calcolo delle probabilità. Se non si è interessati alla formulazione o se non si ha dimestichezza con la matematica è possibile saltare questo paragrafo.

L'uscita di un flip-flop è metastabile dopo un certo tempo, t , se il flip-flop campiona un ingresso che cambia (e che causa, quindi, la condizione di metastabilità) e l'uscita non si stabilizza su un livello valido entro quel tempo a partire dal fronte del clock. Simbolicamente, questo può essere espresso come

$$P(t_{res} > t) = P(\text{campionamento di ingresso che cambia}) \times P(\text{mancata risoluzione}) \quad (3.29)$$

Si consideri ogni termine di probabilità separatamente. Il segnale d'ingresso asincrono commuta (*switch*) da 0 a 1 in un certo tempo, t_{switch} , come mostra la **Figura 3.54**. La probabilità che l'ingresso cambi durante il periodo di apertura intorno al fronte del clock è

$$P(\text{campionamento di ingresso che cambia}) = \frac{t_{switch} + t_{setup} + t_{hold}}{T_c} \quad (3.30)$$

Se il flip-flop entra in uno stato metastabile – cioè con probabilità $P(\text{campionamento di ingresso che cambia})$ – il tempo di risoluzione della metastabilità dipende da ciò che succede all'interno della rete. Questo tempo di risoluzione determina a sua volta $P(\text{mancata risoluzione})$, cioè la probabilità che il flip-flop non si sia ancora stabilizzato su un livello valido dopo un tempo t . Il resto di questo paragrafo analizza un semplice modello di dispositivo bistabile per stimare questa probabilità.

Un dispositivo bistabile utilizza immagazzinamento di carica con retroazione positiva. La **Figura 3.55(a)** mostra questa retroazione realizzata con due negatori; il comportamento di questa rete è rappresentativo della maggior parte degli elementi bistabili. Una coppia di negatori si comporta come un buffer; si supponga che questo buffer abbia le caratteristiche di trasferimento in corrente continua (DC) simmetriche mostrate nella **Figura 3.55(b)**, con una pendenza G . Il buffer può emettere una quantità finita di corrente di uscita, il che può essere rappresentato come una resistenza d'uscita, R . Tutte le reti reali possiedono anche una certa capacità C che deve essere caricata. Caricare il condensatore equivalente a tale capacità tramite la resistenza causa un ritardo RC, che fa sì che il buffer non commuti istantaneamente. Il modello completo della rete è dunque quello mostrato nella **Figura 3.55(c)**, dove $v_{out}(t)$ è la tensione di interesse che codifica lo stato del dispositivo bistabile.

Figura 3.54
Temporizzazione degli ingressi.

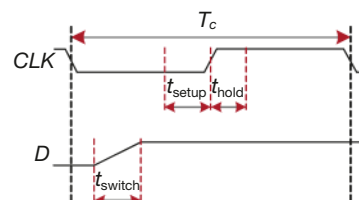
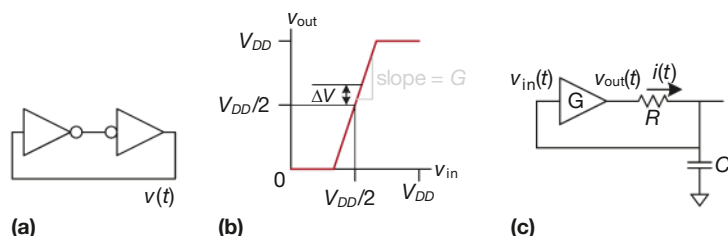


Figura 3.55
Modello circuitale di un dispositivo bistabile.



Il punto metastabile di questa rete è $v_{out}(t) = v_{in}(t) = V_{DD}/2$; se la rete inizialmente si trova esattamente in quel punto, in assenza di rumore vi rimane per un tempo infinito. Dal momento che le tensioni sono variabili continue, la probabilità che la rete inizi esattamente al punto metastabile è estremamente bassa. Tuttavia, al tempo 0 la rete può trovarsi alla tensione $v_{out}(0) = V_{DD}/2 + \Delta V$, diversa dalla metastabilità solo per un piccolo scostamento ΔV . In questo caso, il feedback positivo porta $v_{out}(t)$ a V_{DD} se $\Delta V > 0$ e a 0 se $\Delta V < 0$. Il tempo richiesto per raggiungere V_{DD} o 0 è il tempo di risoluzione della metastabilità per il dispositivo bistabile.

La caratteristica di trasferimento DC è non-lineare, ma è quasi lineare nelle vicinanze del punto metastabile, che è la regione di interesse. In particolare, se $v_{in}(t) = V_{DD}/2 + \Delta V/G$, allora $v_{out}(t) = V_{DD}/2 + \Delta V$ per piccoli valori di ΔV . La corrente che attraversa la resistenza è $i(t) = (v_{out}(t) - v_{in}(t))/R$. Il condensatore si carica alla velocità $dv_{in}(t)/dt = i(t)/C$. Mettendo insieme queste relazioni si trova l'equazione che fornisce la tensione di uscita.

$$\frac{dv_{out}(t)}{dt} = \frac{(G-1)}{RC} \left[v_{out}(t) - \frac{V_{DD}}{2} \right] \quad (3.31)$$

Si tratta di un'equazione differenziale lineare del primo ordine. Se la si risolve con la condizione iniziale $v_{out}(0) = V_{DD}/2 + \Delta V$ si ottiene

$$v_{out}(t) = \frac{V_{DD}}{2} + \Delta V e^{\frac{(G-1)t}{RC}} \quad (3.32)$$

La **Figura 3.56** traccia le curve di $v_{out}(t)$ per diversi punti di partenza. $v_{out}(t)$ si allontana esponenzialmente dal punto metastabile $V_{DD}/2$ fino a che si stabilizza su V_{DD} o 0. Quindi prima o poi l'uscita si porta a 0 o a 1. La quantità di tempo necessaria per questo processo dipende dallo scostamento iniziale (ΔV) dal punto di metastabilità ($V_{DD}/2$).

Esprimendo l'Equazione 3.32 per il tempo di risoluzione della metastabilità, t_{res} , tale per cui $v_{out}(t_{res}) = V_{DD}$ o 0, si ottiene

$$|\Delta V| e^{\frac{(G-1)t_{res}}{RC}} = \frac{V_{DD}}{2} \quad (3.33)$$

$$t_{res} = \frac{RC}{G-1} \ln \frac{V_{DD}}{2|\Delta V|} \quad (3.34)$$

In conclusione, il tempo di risoluzione aumenta se il dispositivo bistabile ha un'alta resistenza o capacità che fanno sì che l'uscita cambi lentamente. Al contrario, diminuisce se il dispositivo bistabile ha un forte guadagno, G . Il tempo di risoluzione inoltre aumenta in modo logaritmico con l'avvicinarsi iniziale della rete al punto di metastabilità ($\Delta V \rightarrow 0$).

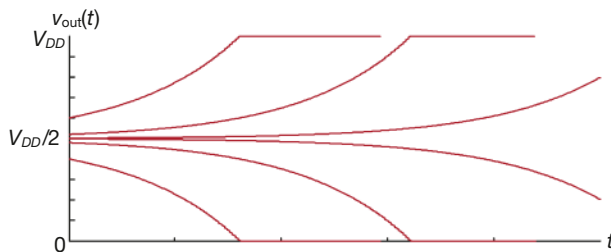


Figura 3.56
Diverse curve di stabilizzazione.

Detto $\tau = \frac{RC}{G-1}$ si può esprimere l'Espressione 3.34 per ΔV per trovare lo scostamento iniziale, ΔV_{res} , che dà un certo tempo di risoluzione, t_{res} :

$$\Delta V_{res} = \frac{V_{DD}}{2} e^{-t_{res}/\tau} \quad (3.35)$$

Se il dispositivo bistabile campiona l'ingresso durante il suo cambiamento, rileva una tensione, $v_{in}(0)$, che si può assumere sia uniformemente distribuita tra 0 e V_{DD} . La probabilità che l'uscita non si sia stabilizzata su un valore valido dopo un tempo t_{res} dipende dalla probabilità che lo scostamento iniziale sia sufficientemente piccolo. Nello specifico, lo scostamento iniziale rispetto a v_{out} deve essere minore di ΔV_{res} , in modo che lo scostamento iniziale v_{in} sia minore di $\Delta V_{res}/G$. A questo punto la probabilità che il dispositivo bistabile campioni l'ingresso a un istante di tempo che permette di avere uno scostamento iniziale abbastanza piccolo è

$$P(\text{mancata risoluzione}) = P\left(\left|v_{in}(0) - \frac{V_{DD}}{2}\right| < \frac{\Delta V_{res}}{G}\right) = \frac{2\Delta V_{res}}{GV_{DD}} \quad (3.36)$$

In conclusione, la probabilità che il tempo di risoluzione della metastabilità sia maggiore di un dato tempo t è data dall'equazione seguente:

$$P(t_{res} > t) = \frac{t_{switch} + t_{setup} + t_{hold}}{GT_c} e^{-\frac{t}{\tau}} \quad (3.37)$$

Si osservi che l'Espressione 3.37 ha la stessa forma dell'Espressione 3.24, dove $T_0 = (t_{switch} + t_{setup} + t_{hold})/G$ e $\tau = RC/(G-1)$. Per riassumere, è stata ricavata l'Espressione 3.24 e si è mostrato come T_0 e τ dipendano da proprietà fisiche del dispositivo bistabile.

3.6 ■ PARALLELISMO

La velocità di un sistema è caratterizzata dalla latenza e dalla capacità produttiva (*throughput*) intesa come quantità di informazioni per unità di tempo che lo attraversano. Definiamo **token** un gruppo di ingressi che vengono elaborati per produrre un gruppo di uscite. Il nome deriva dall'idea di posizionare le pedine sullo schema circuitale di una rete e muoverle per visualizzare i dati che si muovono attraverso il circuito. La **latenza** di un sistema è il tempo richiesto a un token per attraversare il sistema dall'inizio alla fine. La **capacità produttiva** è il numero di token che possono essere elaborati per unità di tempo.

ESEMPIO 3.15

Capacità produttiva e latenza dei biscotti. Ben Imbrogliabit ha organizzato una festa in cui vuole offrire latte e biscotti per festeggiare l'installazione del suo controllore di semaforo stradale. Ben impiega 5 minuti a impastare i biscotti e posizzarli sulla teglia. Dopodiché, sono necessari 15 minuti perché i biscotti si cuociano nel forno. Una volta che i biscotti sono cotti, Ben prepara un'altra teglia. Quali sono capacità produttiva e latenza per una teglia di biscotti?

Soluzione In questo esempio, una teglia di biscotti è un token. La latenza è pari a 1/3 di ora per teglia. La capacità produttiva è di 3 teglie all'ora.

Come si può facilmente immaginare, la capacità produttiva può essere aumentata se si elaborano più token allo stesso tempo. Questo modo di lavorare viene chiamato **parallelismo** e può essere di due tipi: spaziale o temporale. Con il **parallelismo spaziale** vengono usate più copie dell'hardware



in modo che più lavori possano essere svolti contemporaneamente. Con il **parallelismo temporale**, invece, ogni compito viene diviso in fasi, come in una catena di montaggio. Più compiti possono essere distribuiti tra le varie fasi. Nonostante ogni compito debba passare attraverso tutte le fasi, compiti diversi possono trovarsi in ogni fase in un qualsiasi momento, cosicché i diversi compiti si sovrappongono. Il parallelismo temporale viene spesso anche chiamato *pipelining*,¹ mentre il parallelismo spaziale viene a volte chiamato semplicemente parallelismo, ma in questo libro le convenzioni riguardo ai nomi verranno ignorate perché ambigue.

ESEMPIO 3.16

Parallelismo dei biscotti. Ben Imbrogliabit ha centinaia di amici che verranno alla sua festa e ha bisogno di aumentare la sua produzione di biscotti. Per questo, pensa di utilizzare un parallelismo spaziale e/o temporale.

Parallelismo spaziale Ben chiede ad Alyssa Guastacomputer di aiutarlo, visto che anche lei possiede una teglia da biscotti e un forno.

Parallelismo temporale Ben compra una seconda teglia per biscotti. Così, una volta inserita una teglia nel forno, Ben può cominciare a impastare i biscotti sulla nuova teglia mentre aspetta che gli altri siano cotti.

Quali sono capacità produttiva e latenza in caso di parallelismo spaziale? E col parallelismo temporale? E se invece Ben li usasse entrambi?

Soluzione La latenza è il tempo necessario per completare un'azione dall'inizio alla fine. In tutti i casi, la latenza è pari a $1/3$ di ora. Se Ben comincia da zero, la latenza è il tempo che gli occorre a produrre la prima teglia di biscotti.

La capacità produttiva è il numero di teglie di biscotti prodotte all'ora. Con l'utilizzo del parallelismo spaziale, Ben e Alyssa producono ognuno una teglia ogni 20 minuti, il che significa che in due duplicano la capacità produttiva, portandola a 6 teglie all'ora. Con il parallelismo temporale, Ben mette una nuova teglia nel forno ogni 15 minuti, con una capacità produttiva totale di 4 teglie all'ora. I due tipi di parallelismo sono mostrati nella **Figura 3.57**.

Se sia Ben che Alyssa utilizzano entrambe le tecniche riescono a produrre 8 teglie di biscotti all'ora.

Si consideri un compito con latenza L . In un sistema senza parallelismi, la capacità produttiva è pari a $1/L$. In un sistema con parallelismo spaziale con N copie dell'hardware, la capacità produttiva diventa N/L . Invece, in un sistema con parallelismo temporale, il compito viene idealmente diviso in N fasi di eguale durata. In questo caso, la capacità produttiva è comunque N/L , e viene richiesta solo una copia dell'hardware. Tuttavia, come mostrato dall'esempio dei biscotti di Ben, trovare N fasi della stessa durata è spesso poco pratico: se il passaggio più lungo ha una latenza L_1 , la capacità produttiva con parallelismo temporale è $1/L_1$.

Il parallelismo temporale è particolarmente interessante perché è in grado di rendere più rapida una rete senza duplicare l'hardware: vengono infatti posizionati dei registri tra i blocchi di logica combinatoria per dividere la logica in stadi più semplici che possono funzionare con un clock più rapido. La **Figura 3.58** mostra un esempio di una rete senza parallelismo temporale, che contiene quattro blocchi di logica tra i registri. Il percorso critico passa attraverso i blocchi 2, 3 e 4. Assumendo che il registro abbia un ritardo di

¹ Letteralmente “conduttura”, ma si usa sempre il termine inglese pipeline per riferirsi a una struttura di elaborazione dove il singolo risultato viene ottenuto da una sequenza di operazioni più semplici eseguite una dopo l'altra da elementi circuitali diversi sullo stesso semilavorato, come in una catena di montaggio.

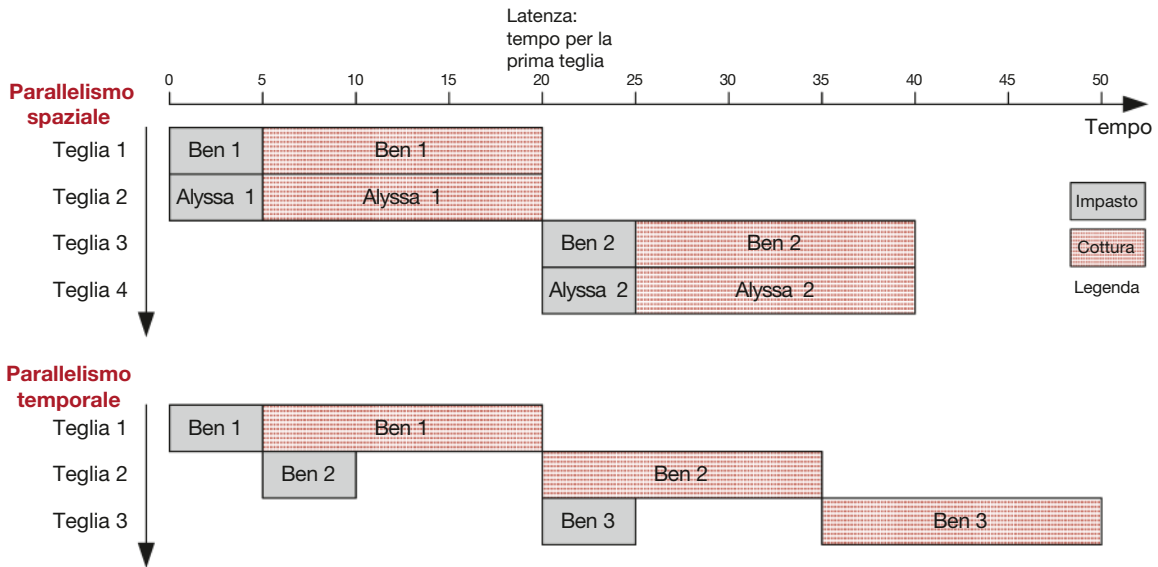
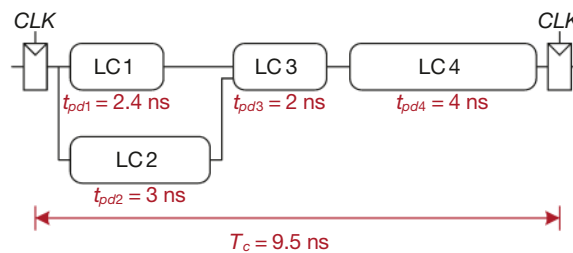


Figura 3.57 Parallelismo spaziale e parallelismo temporale per la cottura dei biscotti.

Figura 3.58
Rete senza uso di pipeline.



propagazione da clock a Q di 0.3 ns e un tempo di setup di 0.2 ns, il tempo di ciclo è pari a $T_c = 0.3 + 3 + 2 + 4 + 0.2 = 9.5 \text{ ns}$. La rete ha una latenza di 9.5 ns e una capacità produttiva di $1/9.5 \text{ ns} = 105 \text{ MHz}$.

La **Figura 3.59** mostra la stessa rete divisa, grazie al parallelismo temporale, in due stadi con l'introduzione di un registro tra i blocchi 3 e 4. Il primo stadio ha un periodo di clock minimo di $0.3 + 3 + 2 + 0.2 = 5.5 \text{ ns}$. Il secondo stadio ha, invece, un periodo di clock minimo pari a $0.3 + 4 + 0.2 = 4.5 \text{ ns}$. Dal momento che il clock deve essere abbastanza lento da permettere a tutti gli stadi di funzionare, $T_c = 5.5 \text{ ns}$. La latenza è pari a due cicli di clock, ovvero 11 ns. Infine, la capacità produttiva è $1/5.5 \text{ ns} = 182 \text{ MHz}$. Questo esempio mostra che, nei circuiti reali, il parallelismo temporale a due stadi quasi raddoppia la capacità produttiva e alza leggermente la latenza. Se paragonato, il parallelismo temporale ideale raddoppierebbe esattamente la capacità produttiva e non penalizzerebbe affatto la latenza. Questa discrepanza si crea perché le reti non possono essere divise in due metà esattamente uguali e perché i registri introducono un sovraccarico di sequenziamento aggiuntivo.

La **Figura 3.60** mostra la stessa rete divisa in tre stadi. Si noti che sono necessari due registri in più per immagazzinare i risultati dei blocchi 1 e 2 alla fine del primo stadio. In questo caso il minimo tempo di ciclo è limitato dal terzo stadio a 4.5 ns. La latenza è pari a tre cicli, quindi 13.5 ns. La capacità produttiva è di $1/4.5 \text{ ns} = 222 \text{ MHz}$. Ancora una volta, aggiungere stadi di parallelismo temporale migliora la capacità produttiva a scapito della latenza.

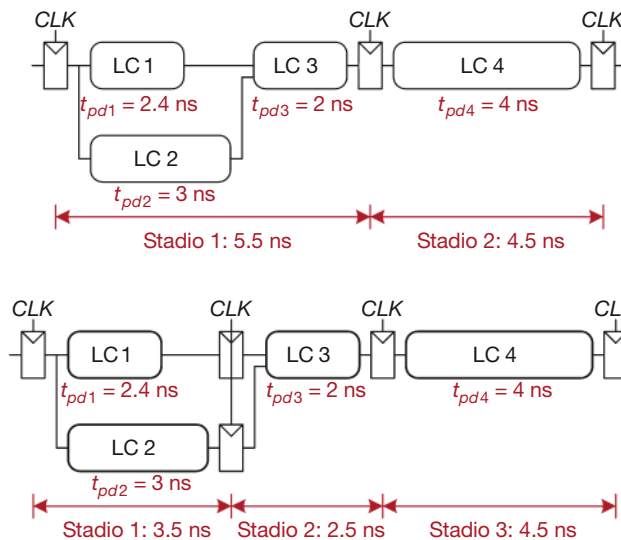


Figura 3.59
Rete con una pipeline a due stadi.

Figura 3.60
Rete con una pipeline a tre stadi.

Anche se queste tecniche sono utili, esse non si applicano universalmente a tutte le situazioni. Il problema principale del parallelismo è dato dalle **dipendenze**. Se un'azione presente dipende da un'azione precedente, invece di dipendere solo dai passi precedenti compiuti sulla stessa azione, tale azione non può iniziare finché quella prima non si è conclusa. Per esempio, se Ben volesse assaggiare la prima teglia di biscotti per assicurarsi che siano buoni prima di preparare la seconda, si troverebbe di fronte a una dipendenza che impedirebbe qualsiasi forma di parallelismo. Il parallelismo è una delle tecniche più importanti per progettare sistemi digitali ad alte prestazioni. Nel Capitolo 7 il *pipelining* verrà approfondito e verranno analizzati degli esempi per comprendere come gestire le dipendenze.

3.7 ■ RIASSUNTO

In questo capitolo sono stati descritti l'analisi e il progetto della logica sequenziale. Al contrario della logica combinatoria, le cui uscite dipendono solo dagli ingressi presenti in quel momento, le uscite della logica sequenziale dipendono sia dagli ingressi presenti in quel momento sia dagli ingressi precedenti. In altre parole, la logica sequenziale ricorda le informazioni relative agli ingressi precedenti. Questa memoria viene chiamata stato della logica.

Le reti sequenziali possono essere difficili da analizzare ed è facile commettere errori nel progettarle, quindi ci si limita a un numero ridotto di blocchi costitutivi progettati opportunamente. Per gli obiettivi di questo libro, l'elemento più importante è il flip-flop, che riceve un clock e un ingresso D e produce un'uscita Q . Il flip-flop copia D su Q al fronte di salita del clock e in tutte le altre situazioni ricorda lo stato precedente di Q . Un gruppo di flip-flop che condividono lo stesso clock viene detto registro. I flip-flop possono anche avere ingressi di reset e abilitazione.

Nonostante esistano molte forme di logica sequenziale, si è scelto di utilizzare le reti sequenziali sincrone perché più semplici da progettare. Le reti sequenziali sincrone consistono di blocchi di logica combinatoria divisi da registri con clock. Lo stato della rete viene immagazzinato nei registri e aggiornato solo ai fronti del clock.

Le macchine a stati finiti (FSM) costituiscono una tecnica molto efficace per progettare le reti sequenziali. Per progettare una FSM si devono innanzi tutto identificare gli ingressi e le uscite della macchina e disegnarne il diagramma degli stati, indicando gli stati e le transizioni tra essi. Dopodiché si

Chi riuscisse a inventare una rete logica le cui uscite dipendessero dagli ingressi futuri diventerebbe ricchissimo...

adotta una codifica per gli stati, e si riscrive il diagramma come una tabella delle transizioni e una tabella delle uscite, che riportano lo stato prossimo e le uscite in funzione dello stato precedente e degli ingressi. A partire da queste tabelle si progetta la logica combinatoria per calcolare lo stato prossimo e le uscite, e si disegna la rete.

Le reti sequenziali sincrone hanno una specifica temporale che include i ritardi di propagazione e di contaminazione da clock a Q , rispettivamente t_{pcq} e t_{ccq} , e i tempi di setup e hold, t_{setup} e t_{hold} . Perché la rete operi correttamente, gli ingressi devono essere stabili per un tempo di apertura che inizia un tempo di setup prima del fronte di salita del clock e finisce un tempo di hold dopo il fronte di salita del clock. Il tempo di ciclo minimo T_c del sistema è uguale al ritardo di propagazione t_{pd} attraverso la logica combinatoria più $t_{pcq} + t_{setup}$ del registro. Inoltre, il ritardo di contaminazione attraverso il registro e la logica combinatoria deve essere maggiore di t_{hold} . Nonostante l'errata convinzione comune, il tempo di hold non ha effetti sul tempo di ciclo.

Le prestazioni globali del sistema vengono misurate in termini di latenza e di capacità produttiva. La latenza è il tempo necessario a un token per attraversare la rete dall'inizio alla fine. La capacità produttiva è il numero di token che il sistema può elaborare per unità di tempo. Il parallelismo migliora la capacità produttiva del sistema.

Esercizi

Esercizio 3.1 Date le forme d'onda di ingresso a un latch SR della Figura 3.61, tracciare la forma d'onda dell'uscita Q .



Figura 3.61 Forme d'onda di ingresso del latch SR dell'Esercizio 3.1.

Esercizio 3.2 Date le forme d'onda di ingresso a un latch SR della Figura 3.62, tracciare la forma d'onda dell'uscita Q .



Figura 3.62 Forme d'onda di ingresso del latch SR dell'Esercizio 3.2.

Esercizio 3.3 Date le forme d'onda di ingresso a un latch D della Figura 3.63, tracciare la forma d'onda dell'uscita Q .



Figura 3.63 Forme d'onda di ingresso del latch o flip-flop D degli Esercizi 3.3 e 3.5.

Esercizio 3.4 Date le forme d'onda di ingresso a un latch D della Figura 3.64, tracciare la forma d'onda dell'uscita Q .



Figura 3.64 Forme d'onda di ingresso del latch o flip-flop D degli Esercizi 3.4 e 3.6.

Esercizio 3.5 Date le forme d'onda di ingresso a un flip-flop D della Figura 3.63, tracciare la forma d'onda dell'uscita Q .

Esercizio 3.6 Date le forme d'onda di ingresso a un flip-flop D della Figura 3.64, tracciare la forma d'onda dell'uscita Q .

Esercizio 3.7 La rete mostrata nella Figura 3.65 è combinatoria o sequenziale? Spiegare la relazione fra ingressi e uscite della rete, e darle un nome.

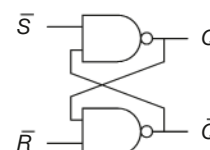


Figura 3.65 Rete misteriosa.

Esercizio 3.8 La rete mostrata nella Figura 3.66 è combinatoria o sequenziale? Spiegare la relazione fra ingressi e uscite della rete, e darle un nome.

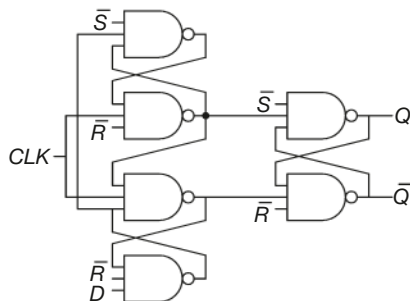


Figura 3.66 Rete misteriosa.

Esercizio 3.9 Il flip-flop T (a commutazione, da *toggle*) ha un ingresso, CLK, e un'uscita, Q. A ogni fronte di salita di CLK, Q commuta il proprio valore al valore opposto. Disegnare la rete del flip-flop T utilizzando un flip-flop D e un negatore.

Esercizio 3.10 Il flip-flop JK riceve il clock e due ingressi, J e K. Sul fronte di salita del clock, l'uscita viene modificata nel modo seguente: se J e K sono entrambi 0, Q mantiene il valore precedente; se solo J è 1, Q diventa 1; se solo K è 1, Q diventa 0; se J e K sono entrambi 1, Q diventa l'opposto del valore precedente.

- Realizzare un flip-flop JK utilizzando un flip-flop D e una rete combinatoria.
- Realizzare un flip-flop D utilizzando un flip-flop JK e una rete combinatoria.
- Realizzare un flip-flop T (vedi Esercizio 3.9) utilizzando un flip-flop JK.

Esercizio 3.11 La rete della Figura 3.67 è definita elemento C di Muller. Spiegare la relazione fra ingressi e uscite della rete.

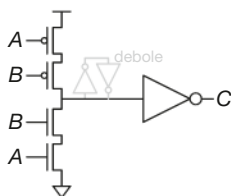


Figura 3.67 Elemento C di Muller.

Esercizio 3.12 Realizzare un latch D resettabile in modo asincrono utilizzando porte logiche.

Esercizio 3.13 Realizzare un flip-flop D resettabile in modo asincrono utilizzando porte logiche.

Esercizio 3.14 Realizzare un flip-flop D settabile in modo sincrono utilizzando porte logiche.

Esercizio 3.15 Realizzare un flip-flop D settabile in modo asincrono utilizzando porte logiche.

Esercizio 3.16 Si consideri un oscillatore costituito da N negatori collegati ad anello. Ogni negatore introduce un ritardo minimo t_{cd} e un ritardo massimo t_{pd} . Con N dispari, calcolare l'intervallo di frequenze alle quali l'oscillatore può funzionare.

Esercizio 3.17 Perché il numero N di negatori dell'Esercizio 3.16 deve essere dispari?

Esercizio 3.18 Quali tra le reti della Figura 3.68 sono sequenziali sincrone? Perché?

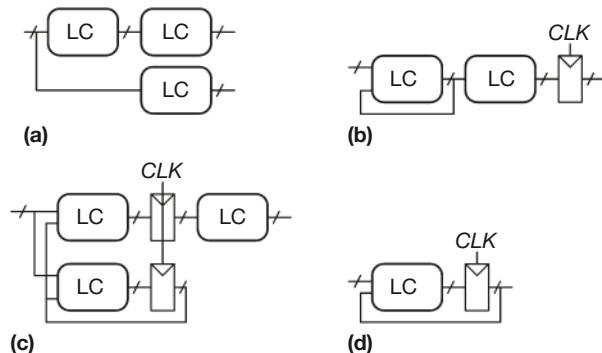


Figura 3.68 Reti varie.

Esercizio 3.19 Si deve realizzare il controllore di un ascensore per un edificio con 25 piani. Il controllore ha due ingressi, SU e GIÙ, e genera in uscita l'indicazione del piano al quale si trova l'ascensore. Per scaramanzia, il piano 13 non esiste... Qual è il minimo numero di bit di stato che deve avere il controllore?

Esercizio 3.20 Si deve realizzare una FSM per tenere traccia dell'umore di quattro studenti che lavorano nel laboratorio di progettazione digitale. L'umore di ogni studente può essere FELICE (la sua rete funziona), TRISTE (la sua rete si è bruciata), IMPEGNATO (sta progettando la sua rete), SMARRITO (non capisce cosa diavolo fa la sua rete) oppure ADDORMENTATO (è caduto di faccia sulla sua rete). Quanti stati almeno deve avere la FSM? E qual è il minimo numero di bit per rappresentarli?

Esercizio 3.21 Come si potrebbe fattorizzare la FSM dell'Esercizio 3.20 in più macchine più semplici? Quanti stati ha ciascuna di queste macchine più semplici? Qual è in totale il minimo numero di bit necessari nella macchina fattorizzata?

Esercizio 3.22 Descrivere a parole il comportamento della macchina il cui diagramma degli stati è riportato nella Figura 3.69. Utilizzando codifica binaria per gli stati, costruire la tabella delle transizioni e quella delle uscite. Scrivere le espressioni booleane di stato prossimo e di uscita e disegnare lo schema completo della FSM.

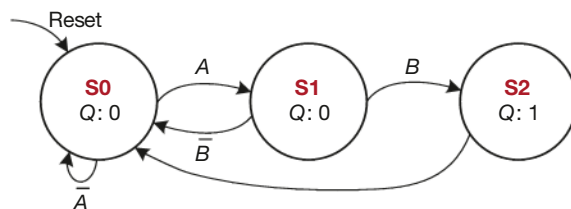


Figura 3.69 Diagramma degli stati.

Esercizio 3.23 Descrivere a parole il comportamento della macchina il cui diagramma degli stati è riportato nella Figura 3.69.

ra 3.70. Utilizzando codifica binaria per gli stati, costruire la tabella delle transizioni e quella delle uscite. Scrivere le espressioni booleane di stato prossimo e di uscita e disegnare lo schema completo della FSM.

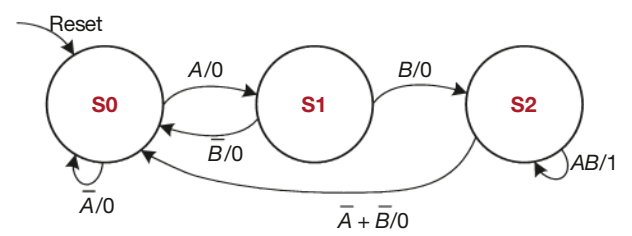


Figura 3.70 Diagramma degli stati.

Esercizio 3.24 All’incrocio tra via Accademia e viale Ateneo si verificano ancora incidenti. La squadra di calcio arriva di corsa all’incrocio nel momento in cui il semaforo *B* diventa verde, e si scontra con gli assonnati specializzandi in informatica che avanzano barcollando nell’incrocio quando il semaforo *A* sta per diventare rosso. Modificare il controllore semaforico del paragrafo 3.4.1 in modo tale che entrambi i semafori siano rossi per 5 secondi prima che uno dei due diventi verde. Tracciare il diagramma degli stati del controllore così modificato, scegliere la codifica degli stati, tracciare la tabella delle transizioni e quella delle uscite, le espressioni di stato prossimo e di uscita, lo schema completo della FSM.

Esercizio 3.25 Il robot lumaca di Alyssa Guastacomputer descritto nel paragrafo 3.4.3 ha una sorella, con un cervello costituito da una FSM alla Mealy, che sorride ogni volta che passa sopra la sequenza 1101 oppure la sequenza 1110. Tracciare il diagramma degli stati di questa FSM utilizzando il minimo numero di stati possibile, scegliere la codifica degli stati, tracciare la tabella delle transizioni e quella delle uscite, le espressioni di stato prossimo e di uscita, lo schema completo della FSM.

Esercizio 3.26 Si deve realizzare un distributore automatico di bottigliette di acqua minerale per la sala studio dell’università di Harvard. Ogni bottiglietta costa 25 centesimi di dollaro (grazie al contributo dell’ateneo). Il distributore accetta monete da 5, 10 e 25 centesimi: quando il totale inserito è sufficiente, eroga la bottiglietta e restituisce l’eventuale resto. Progettare la FSM del distributore, dotata di tre ingressi *Nickel*, *Dime* e *Quarter* (corrispondenti alle tre diverse monete) e di quattro uscite *Eroga*, *RendiNickel*, *RendiDime* e *RendiDueDime*. Nell’ipotesi che a ogni ciclo di clock venga inserita una moneta, quando l’importo inserito raggiunge o supera 25 centesimi la FSM deve attivare *Eroga* e le eventuali uscite *Rendi...* per rendere il resto quando dovuto. Quindi la FSM si prepara ad accettare nuove monete.

Esercizio 3.27 Il codice Gray ha l’utilissima proprietà di rappresentare numeri consecutivi con configurazioni che differiscono per un solo bit. Nella Tabella 3.23 è riportato il codice Gray a 3 bit per la rappresentazione dei numeri da 0 a 7. Progettare un contatore modulo 8 a codice Gray senza ingressi e con tre uscite. (Si ricordi che un contatore modulo *N* conta da 0 a *N* – 1, quindi riparte da 0. Per esempio, un contatore modulo 60 per i minuti primi o secondi conta da 0 a 59.) Quando viene resettato, il contatore in questione deve partire da 000 e generare in uscita il prossimo codice Gray a ogni fronte di salita del clock; una volta raggiunta la configurazione 100, deve ricominciare da 000.

Tabella 3.23 Codice Gray a tre bit.

Numero	Codice Gray		
0	0	0	0
1	0	0	1
2	0	1	1
3	0	1	0
4	1	1	0
5	1	1	1
6	1	0	1
7	1	0	0

Esercizio 3.28 Modificare il contatore dell’Esercizio 3.27 in modo che diventi un contatore UP/DOWN aggiungendo un ingresso *UP*. Se *UP* vale 1, il contatore avanza alla prossima configurazione; se *UP* vale 0, il contatore retrocede alla precedente.

Esercizio 3.29 Si deve progettare una FSM con due ingressi, *A* e *B*, e un’uscita *Z*. All’*n*-esimo ciclo di clock, l’uscita *Z_n* deve essere l’AND oppure l’OR del valore di ingresso *A_n* allo stesso ciclo e del valore di ingresso *A_{n-1}* al ciclo precedente, a seconda del valore dell’ingresso *B_n*:

$$Z_n = A_n A_{n-1} \quad \text{se } B_n = 0$$

$$Z_n = A_n + A_{n-1} \quad \text{se } B_n = 1$$

- (a) Tracciare la forma d’onda di *Z* se gli ingressi si comportano come indicato nella Figura 3.71.
- (b) La FSM è una macchina alla Moore o alla Mealy?
- (c) Progettare la FSM: tracciare il diagramma degli stati, la tabella delle transizioni e quella delle uscite, le espressioni booleane di stato prossimo e di uscita e lo schema completo della FSM.

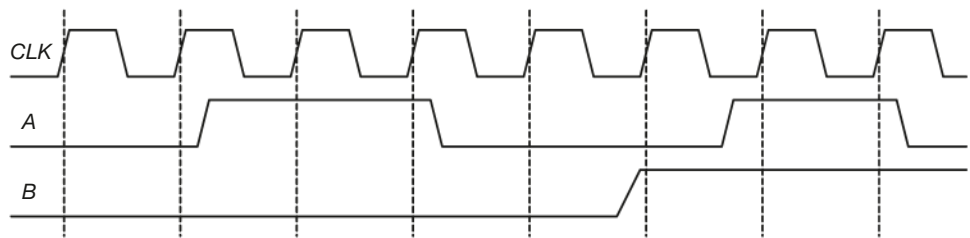


Figura 3.71 Forme d’onda di ingresso della FSM.

Esercizio 3.30 Progettare una FSM con un ingresso, A , e due uscite, X e Y . X deve valere 1 se A ha assunto il valore 1 per almeno tre cicli di clock (non necessariamente consecutivi) mentre Y deve valere 1 se A ha assunto il valore 1 in due cicli consecutivi di clock. Tracciare il diagramma degli stati, la tabella delle transizioni e quella delle uscite, le espressioni booleane di stato prossimo e di uscita e lo schema completo della FSM.

Esercizio 3.31 Analizzare la FSM con la struttura riportata nella Figura 3.72. Tracciare le tabelle delle transizioni e delle uscite e il diagramma degli stati, e descrivere a parole il funzionamento della FSM.

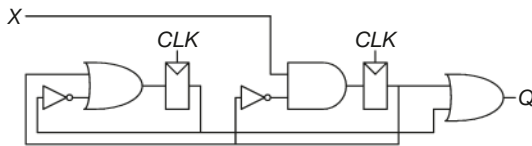


Figura 3.72 Struttura della FSM.

Esercizio 3.32 Ripetere l'Esercizio 3.31 per la FSM con la struttura riportata nella Figura 3.73, ricordando che gli ingressi s e r dei registri sono rispettivamente Set e Reset.

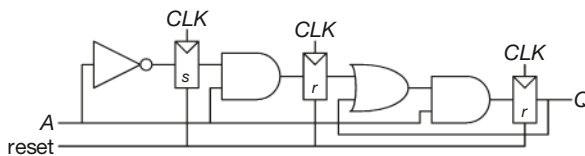


Figura 3.73 Struttura della FSM.

Esercizio 3.33 Ben Imbrogliabit ha disegnato la rete di Figura 3.74 per ottenere la funzione XOR a quattro ingressi con uscita a registri. Ogni XOR a due ingressi ha un ritardo di propagazione di 100 ps e un ritardo di contaminazione di 55 ps. Ciascun flip-flop ha un tempo di setup di 60 ps, un tempo di hold di 20 ps, un ritardo massimo clock-Q di 70 ps e un ritardo minimo clock-Q di 50 ps.

- (a) Se non c'è deriva del clock, qual è la massima frequenza alla quale la rete può lavorare?
- (b) Quale deriva massima del clock è accettabile se la rete deve operare a 2 GHz?
- (c) Quale deriva massima del clock è accettabile prima che nella rete si verifichi una violazione dei vincoli sul tempo di hold?
- (d) Alyssa Guastacomputer sostiene di essere in grado di riprogettare le reti combinatorie presenti tra i registri in modo che la FSM sia più veloce e capace anche di tollerare una deriva maggiore del clock. La sua proposta usa ancora tre porte XOR a due ingressi, ma collegate in modo diverso. Qual è la rete di Alyssa? Se non c'è deriva del clock, qual è la massima frequenza alla quale la rete può lavorare? Quale deriva massima del clock è accettabile prima che nella rete si verifichi una violazione dei vincoli sul tempo di hold?

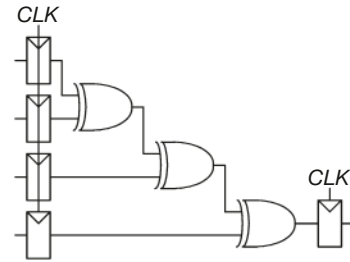


Figura 3.74 Rete XOR a quattro ingressi con uscita a registri.

Esercizio 3.34 Si deve realizzare un sommatore per il processore superveloce a due bit Pentium, utilizzando due circuiti sommatore completi (*full adder*) con il riporto (*carry*) di uscita del primo che diventa il riporto di ingresso del secondo, come mostrato nella Figura 3.75. Il sommatore ha registri di ingresso e di uscita, e deve effettuare la somma in un ciclo di clock. Ogni sommatore ha i seguenti ritardi di propagazione: 20 ps da R_{in} a R_{out} o a S (bit di *Somma*), 25 ps da A o da B a R_{out} e 30 ps da A o da B a S ; e i seguenti ritardi di contaminazione: 15 ps da R_{in} a tutte le uscite e 22 ps da A o B a tutte le uscite. Ciascun flip-flop ha un tempo di setup di 30 ps, un tempo di hold di 10 ps, un ritardo di propagazione clock-Q di 35 ps e un ritardo di contaminazione clock-Q di 21 ps.

- Se non c'è deriva del clock, qual è la massima frequenza alla quale la rete può lavorare?
- Quale deriva massima del clock è accettabile se la rete deve operare a 8 GHz?
- Quale deriva massima del clock è accettabile prima che nella rete si verifichi una violazione dei vincoli sul tempo di hold?

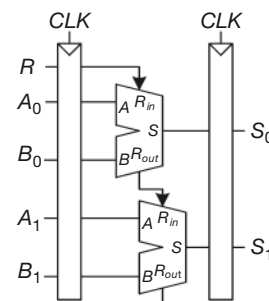


Figura 3.75 Struttura di un sommatore a due bit.

Esercizio 3.35 Un circuito integrato FPGA (*Field Programmable Gate Array*) usa blocchi logici configurabili (CLB, *Configurable Logic Block*) al posto delle porte logiche per realizzare reti combinatorie. L’FPGA Spartan 3 della ditta Xilinx ha ritardi di propagazione e di contaminazione per ogni CLB rispettivamente di 0.61 e 0.30 ns, e contiene anche alcuni flip-flop con ritardi di propagazione e contaminazione rispettivamente di 0.72 e 0.50 ns, e tempi di setup e di hold rispettivamente di 0.53 ns e 0 ns.

- (a) Se si deve costruire una rete che deve operare a 40 MHz, quanti CLB si possono collegare in serie tra due flip-flop, nell'ipotesi che non ci sia deriva di clock né ritardi sui fili tra i CLB?

(b) Se tutti i percorsi dei segnali tra i due flip-flop passano per almeno un CLB, quale deriva massima del clock può avere l'FPGA prima che si verifichi una violazione dei vincoli sul tempo di hold?

Esercizio 3.36 Un sincronizzatore è realizzato con una coppia di flip-flop con $t_{\text{setup}} = 50$ ps, $T_0 = 20$ ps e $\tau = 30$ ps. Deve campionare un segnale di ingresso asincrono che varia 10^8 volte al secondo. Qual è il minimo periodo di clock del sincronizzatore per garantire un tempo medio tra guasti (MTBF, *Mean Time Between Failures*) di 100 anni?

Esercizio 3.37 Si deve realizzare un sincronizzatore con ingresso asincrono in grado di garantire un tempo medio tra guasti di 50 anni. La rete lavora a 1 GHz, e i flip-flop di campionamento sono caratterizzati da $\tau = 100$ ps, $T_0 = 110$ ps e $t_{\text{setup}} = 70$ ps. Il sincronizzatore riceve un nuovo valore di ingresso in media 0.5 volte al secondo (cioè una volta ogni 2 secondi). Qual è la probabilità di guasto necessaria per garantire l'MTBF desiderato? Quanti cicli di clock si devono attendere prima di leggere il segnale campionato per ottenere tale probabilità di guasto?

Esercizio 3.38 Stai camminando lungo un corridoio, e ti trovi davanti il tuo compagno di laboratorio che avanza in direzione opposta. Entrambi vi spostate da una parte per passare, e siete ancora uno di fronte all'altro. Allora entrambi vi spostate dalla parte opposta, e di nuovo non potete passare. A questo punto ciascuno di voi aspetta un po' sperando che l'altro si sposti. Si può modellizzare questa situazione come metastabile, applicando la medesima teoria usata per flip-flop e sincronizzatori. Da un punto di vista matematico, se ti trovi davanti il tuo compagno e sei in stato metastabile, la probabilità che tu rimanga in tale stato dopo t secondi è data da $e^{-\frac{t}{\tau}}$ dove τ è il tuo tempo di risposta. Siccome oggi hai molto sonno, hai un $\tau = 20$ secondi.

(a) Quanto tempo passa prima che tu abbia il 99% di probabilità di essere uscito dallo stato metastabile (cioè siate riusciti a evitarvi e a procedere nel corridoio)?

(b) Non sei solo assonnato, ma anche affamatissimo, e se non arrivi al bar entro 3 minuti rischi il collasso... Qual è la probabilità che il tuo compagno di laboratorio debba chiamare l'ambulanza perché sei svenuto?

Esercizio 3.39 Si è realizzato un sincronizzatore usando flip-flop con $T_0 = 20$ ps e $\tau = 30$ ps, ma il capo dice che bisogna aumentare l'MTBF di un fattore 10. Di quanto si deve aumentare il ciclo di clock per farlo contento?

Esercizio 3.40 Ben Imbrogliabit ha inventato il nuovo e migliorato sincronizzatore della Figura 3.76, che lui sostiene sia capace di eliminare la metastabilità in un solo ciclo di clock. Spiega che il circuito nella scatola M è un "rilevatore di metastabilità" analogico che genera in uscita un valore ALTO se la tensione di ingresso si trova nella zona proibita tra V_{IL} e V_{IH} . Il rilevatore di metastabilità controlla se il primo flip-flop ha generato un'uscita metastabile in $D2$. Se questo accade, resetta il flip-flop in modo da avere uno 0 corretto in $D2$. Quindi il secondo flip-flop che campiona $D2$ produce sempre un valore corretto all'uscita Q . Alyssa Guastacomputer dice a Ben che ci dev'essere un errore nel progetto, perché eliminare la metastabilità è impossibile esattamente come costruire la macchina del moto perpetuo. Chi ha ragione? Trovare l'errore di Ben oppure spiegare perché Alyssa ha torto.

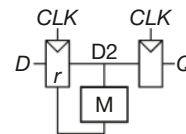


Figura 3.76 Sincronizzatore "nuovo e migliorato".

Domande di valutazione

Queste domande sono state poste a candidati per un posto di lavoro nell'ambito della progettazione di sistemi digitali.

Domanda 3.1 Progetti una macchina a stati finiti capace di rilevare la sequenza di ingresso 01010.

Domanda 3.2 Progetti una FSM che sia capace di calcolare il complemento a due di un numero seriale (cioè comunicato un bit dopo l'altro). La FSM ha due ingressi, *Start* e *A*, e un'uscita, *Q*. Un numero binario di lunghezza arbitraria viene comunicato all'ingresso *A* a cominciare dal bit meno significativo, e il corrispondente bit del numero complementato deve comparire all'uscita *Q* nel medesimo ciclo di clock. L'ingresso *Start* viene attivato a 1 per inizializzare la FSM un ciclo di clock prima dell'arrivo del bit meno significativo del numero.

Domanda 3.3 Conosce la differenza tra latch e flip-flop? Quando è preferibile usare l'uno piuttosto dell'altro?

Domanda 3.4 Progetti un contatore a 5 bit come macchina a stati finiti.

Domanda 3.5 Progetti un rilevatore di fronti. L'uscita deve assumere il valore ALTO per un ciclo di clock quando si sia verificata in ingresso la transizione $0 \rightarrow 1$.

Domanda 3.6 Descriva il concetto di *pipelining* e i motivi del suo utilizzo.

Domanda 3.7 Cosa significa per un flip-flop avere un tempo di hold negativo?

Domanda 3.8 Dato il segnale *A* mostrato in Figura 3.77, progetta una rete in grado di generare il segnale *B*.

Domanda 3.9 Data una rete combinatoria tra due registri, discuta i vincoli temporali. Se aggiunge un buffer all'ingresso di clock del ricevitore (cioè del secondo flip-flop) i vincoli sul tempo di setup migliorano o peggiorano?



Figura 3.77 Forme d'onda dei segnali.

Linguaggi di descrizione dell'hardware

Capitolo

4

- 4.1 Introduzione
- 4.2 Logica combinatoria
- 4.3 Modellazione strutturale
- 4.4 Logica sequenziale
- 4.5 Ancora logica combinatoria
- 4.6 Macchine a stati finiti
- 4.7 Tipi di dati*
- 4.8 Moduli parametrici*
- 4.9 Testbench
- 4.10 Riassunto

4.1 ■ INTRODUZIONE

Sinora ci si è occupati della progettazione di reti combinatorie e sequenziali a livello di schemi logici, ma trovare un insieme efficiente di porte logiche in grado di svolgere una certa funzione è un'attività onerosa e a rischio di errori, perché si devono semplificare "a mano" le tabelle delle verità o le espressioni booleane per tradurre una macchina a stati finiti (FSM) in una rete. Dal 1990 circa i progettisti si sono resi conto che è molto più produttivo lavorare a un livello di astrazione più alto, fornendo le specifiche delle funzioni logiche e lasciando a uno strumento di CAD (*Computer Aided Design*) il compito di produrre una rete di porte logiche ottimizzata. Tali specifiche sono fornite in genere tramite linguaggi di descrizione dell'hardware (HDL, *Hardware Description Language*) tra i quali i più diffusi e importanti sono **SystemVerilog** e **VHDL**.

SystemVerilog e VHDL sono stati costruiti sulla base degli stessi principi, ma adottano sintassi differenti: per questo motivo il capitolo li presenta su due colonne affiancate per consentire un confronto puntuale. Alla prima lettura si consiglia di scegliere uno dei due linguaggi e concentrarsi su quello: una volta imparato quello, è facile impadronirsi anche del secondo, se necessario.

Nei capitoli successivi l'hardware è descritto sia a livello di schemi logici sia a livello di HDL, quindi non è necessaria la lettura di questo capitolo per comprendere i principi fondamentali dell'organizzazione dei calcolatori, che possono essere dedotti appunto dagli schemi logici. È però opportuno tenere presente che la stragrande maggioranza dei sistemi commerciali è oggi realizzata usando HDL e non schemi, quindi se si pensa di dover lavorare prima o poi nella progettazione dei sistemi digitali la conoscenza di un HDL diventa quanto mai necessaria.

4.1.1 Moduli

Un blocco circuitale con ingressi e uscite viene definito **modulo**. Una porta AND, un multiplexer, un circuito a priorità sono tutti esempi di moduli circuitali. I due stili con i quali si possono descrivere le funzionalità di un modulo sono lo stile **comportamentale** (*behavioral*) e lo stile **strutturale** (*structural*): il primo descrive cosa fa il modulo, mentre il secondo dà una visione gerar-

Software applicativo	
Sistema operativo	
Architettura	
Micro-architettura	
Blocchi logici	
Reti digitali	
Reti analogiche	
Dispositivi	
Fisica	

chica di come il modulo è fatto scomponendolo in pezzi via via più semplici. Il codice SystemVerilog e il codice VHDL dell'**Esempio HDL 4.1** mostrano la descrizione comportamentale di un modulo che calcola l'espressione booleana dell'Esempio 2.6, ovvero $y = \bar{a}\bar{b}\bar{c} + a\bar{b}\bar{c} + a\bar{b}c$. In entrambi i linguaggi, il modulo è denominato *funzionequalunque* e ha tre ingressi *a*, *b* e *c*, e un'uscita *y*.

ESEMPIO HDL 4.1 LOGICA COMBINATORIA	
SystemVerilog <pre>module funzionequalunque (input logic a, b, c, output logic y); assign y = ~a & ~b & ~c a & ~b & ~c a & ~b & c; endmodule</pre> Un modulo di SystemVerilog inizia con il nome del modulo seguito da un elenco di ingressi e uscite. L'istruzione <code>assign</code> descrive la parte di logica combinatoria, il carattere <code>~</code> indica l'operatore NOT, <code>&</code> l'AND e <code> </code> l'OR. I segnali definiti <code>logic</code> come gli ingressi e le uscite sono variabili booleane, che sono normalmente 0 o 1 ma possono assumere valori fluttuanti o indefiniti, come discusso nel paragrafo 4.2.8. Il tipo <code>logic</code> è stato introdotto in SystemVerilog, sostituendo il precedente tipo <code>reg</code>, fonte continua di confusione in Verilog. Il tipo <code>logic</code> va usato sempre tranne nel caso di segnali con molte sorgenti, definiti <code>nets</code> e discussi nel paragrafo 4.7.	VHDL <pre>library IEEE; use IEEE.STD_LOGIC_1164.all; entity funzionequalunque is port(a, b, c: in STD_LOGIC; y: out STD_LOGIC); end; architecture sintesi of funzionequalunque is begin y <= (not a and not b and not c) or (a and not b and not c) or (a and not b and c); end;</pre> Il codice VHDL si compone di tre parti: la clausola <code>library use</code>, la dichiarazione di <code>entity</code> e il corpo della <code>architecture</code>. La clausola <code>library use</code> è discussa nel paragrafo 4.7.2, la dichiarazione <code>entity</code> elenca il nome del modulo e i suoi ingressi e uscite, il corpo della <code>architecture</code> definisce il funzionamento del modulo. I segnali VHDL, come gli ingressi e le uscite, devono avere una dichiarazione di tipo, che è <code>STD_LOGIC</code> per le variabili booleane, che sono normalmente 0 o 1 ma possono assumere valori fluttuanti o indefiniti, come discusso nel paragrafo 4.2.8. Il tipo <code>STD_LOGIC</code> è definito nella libreria <code>IEEE.STD_LOGIC_1164</code>, ecco perché tale libreria deve essere citata nel codice VHDL. In VHDL gli operatori logici AND e OR non seguono la normale regola di precedenza di valutazione (che prevede di eseguire prima gli AND e poi gli OR) per cui è necessario usare le parentesi.

È evidente che il concetto di modulo è un ottimo esempio di applicazione della modularità: infatti il modulo ha un'interfaccia ben definita con altri moduli, data dai suoi ingressi e dalle sue uscite, e svolge una funzione specifica. Come sia codificato importa poco a chi vuole utilizzarlo, l'importante è che esegua correttamente la funzione per la quale è stato progettato.

4.1.2 Origini dei linguaggi

Le università dove si insegna progettazione digitale si dividono quasi equamente riguardo a quale dei due linguaggi insegnare per primo: l'industria sembra orientarsi verso SystemVerilog ma molte aziende usano ancora VHDL e molti progettisti devono sapersi esprimere correntemente in entrambi i linguaggi. VHDL è più verboso e pesante di SystemVerilog, come spesso accade per i linguaggi sviluppati da comitati di esperti invece che da una particolare azienda.

Entrambi i linguaggi sono naturalmente in grado di descrivere un circuito digitale, così come entrambi hanno le loro idiosincrasie: la scelta va dunque fatta sulla base del linguaggio usato nella propria sede di studio o lavoro o nelle richieste dei propri clienti. Comunque, molti strumenti CAD moderni consentono di utilizzare entrambi i linguaggi, cioè di avere moduli diversi scritti in linguaggi diversi.

SystemVerilog

Verilog è stato sviluppato nel 1984 dalla ditta Gateway Design Automation come linguaggio proprietario per la simulazione logica. Gateway è stata acquistata da Cadence nel 1989, e nel 1990 Verilog è divenuto un *Open Standard* sotto il controllo di Open Verilog. International, ed è diventato uno standard IEEE¹ nel 1995. Il linguaggio è stato esteso nel 2005 per snellire alcune idiosincrasie e migliorare il supporto alla modellizzazione e alla verifica dei sistemi. Queste modifiche sono state riunite in un linguaggio standard chiamato oggi SystemVerilog (IEEE STD 1800-2009). I file SystemVerilog hanno normalmente estensione .sv.

VHDL

VHDL è l'acronimo di *VHSIC Hardware Description Language*, dove VHSIC è a sua volta l'acronimo di *Very High Speed Integrated Circuits*, un programma di sviluppo del Dipartimento della Difesa statunitense.

VHDL è stato sviluppato nel 1981 dal Dipartimento della Difesa per descrivere struttura e funzionamento dell'hardware. Le sue radici derivano dal linguaggio di programmazione Ada. Inizialmente pensato come linguaggio di documentazione, è divenuto presto uno strumento di simulazione e sintesi circuitale. Lo IEEE ha standardizzato per la prima volta il linguaggio nel 1987, con numerosi aggiornamenti successivi. In questo capitolo del testo si fa riferimento alla versione 2008 di VHDL (IEEE STD 1076-2008), snellita in vario modo rispetto alle precedenti. Quando è stato scritto questo testo, non tutte le caratteristiche di VHDL 2008 erano supportate dagli strumenti CAD, quindi ci si limita a utilizzare quelle accettate da Synplicity, Altera Quartus e ModelSim. I file VHDL hanno normalmente estensione .vhd.

Per usare VHDL 2008 con ModelSim può essere necessario attivare l'opzione VHDL93=2008 nel file di configurazione modelsim.ini.

4.1.3 Simulazione e sintesi

I due obiettivi principali degli HDL sono la **simulazione** e la **sintesi**. Nella simulazione si forniscono valori di ingresso al modulo e si controlla alle uscite se il modulo funziona correttamente. Nella sintesi, la descrizione testuale del modulo viene tradotta in rete di porte logiche.

Simulazione

L'essere umano per natura può commettere errori. Nella progettazione circuitale, e in generale in informatica, il termine spesso usato in italiano per indicare uno di questi errori è **baco**, ricavato per assonanza dalla parola inglese *bug* (pulce): tale parola è molto più appropriata, infatti il verbo "spulciare" dà proprio l'idea di cercare (in un programma) gli errori nascosti, cioè le pulci nel pelo. E siccome non esiste un verbo simile per il baco, in italiano si parla spesso con un orribile barbarismo di "debuggare" intendendo l'attività di ricerca e correzione degli errori. Tale attività è ovviamente importantissima, visto che i clienti pagano per avere un sistema digitale e che in certi casi la loro vita dipende dal corretto funzionamento di questo sistema. D'altro canto, collaudare un sistema digitale in laboratorio richiede molto tempo, e può essere estremamente difficile scoprire la causa di un errore dal momento che si possono osservare solo i segnali elettrici ai piedini dei circuiti integrati, e non c'è possibilità di osservare ciò che avviene all'interno dei chip. Inoltre, correggere un errore quando il sistema digitale è già stato prodotto può essere enormemente costoso: si arriva facilmente a milioni di dollari e mesi di lavoro nel caso di circuiti integrati all'avanguardia. Basti pensare al tristemente famoso baco FDIV (*Floating-point DIVision*) del processore Pentium, che costrinse l'Intel a ritirare dal mercato i chip già distribuiti, con un costo totale di 475 milioni di dollari. La simulazione logica è dunque fondamentale per consentire di collaudare un sistema prima di costruirlo fisicamente.

Il termine *bug* (pulce) con l'accezione di errore nasce ben prima dell'informatica. Thomas Edison nel 1878 già chiamava bug "i piccoli errori e le difficoltà" nelle sue invenzioni.

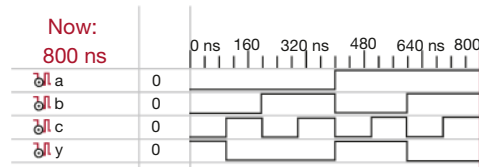
Il primo vero bug informatico è stato una falena, rimasta imprigionata nel 1947 tra i relè del calcolatore elettromeccanico Mark II ad Harvard. Fu trovata da Grace Hopper, che ha registrato l'incidente e conservato la falena con il commento "il primo vero caso di bug trovato."



Fonte: annotazione su quaderno. Fotografia N° NII 96566-KN, gentilmente concessa da Naval Historical Center, US Navy.

¹ L'Istituto degli ingegneri elettrici ed elettronici (IEEE, Institute of Electrical and Electronics Engineers) è una società di professionisti responsabile di molti standard relativi alle tecnologie dell'informazione e della comunicazione, inclusi il Wi-Fi (802.11), Ethernet (802.3) e la codifica in virgola mobile dei numeri (754).

Figura 4.1
Forme d'onda di simulazione.



La **Figura 4.1** mostra le forme d'onda della simulazione² del modulo *funzionequalunque* visto prima, che ne dimostrano il corretto funzionamento: *y* assume valore VERO quando *a*, *b* e *c* assumono le configurazioni 000, 100 oppure 101 come specificato dall'espressione booleana.

Sintesi

La sintesi logica trasforma il codice HDL in uno schema di porte logiche e fili (denominato *netlist*) che descrivono la realizzazione circuitale del modulo. Il sintetizzatore logico può naturalmente fare ottimizzazioni per ridurre la quantità di hardware effettivamente necessaria. La *netlist* può essere un semplice file di testo oppure uno schema grafico per facilitare la visualizzazione del circuito risultante. La **Figura 4.2** mostra l'esito della sintesi³ del modulo *funzionequalunque*: si noti come le porte AND a tre ingressi siano state sostituite dalle più semplici porte AND a due ingressi, usando i metodi già visti nell'Esempio 2.6 di utilizzo dell'algebra booleana.

La descrizione dei circuiti in HDL è simile al codice nei linguaggi di programmazione, anche se bisogna ricordare che in questo caso il codice serve a descrivere componenti circuitali. SystemVerilog e VHDL sono linguaggi molto ricchi, con numerose istruzioni, e non tutte queste istruzioni possono essere sintetizzate in hardware: per esempio, un'istruzione di visualizzazione su schermo durante la simulazione non viene tradotta in dispositivi hardware. Dato lo scopo di questo testo, ci si concentra nel seguito sul sottoinsieme di istruzioni sintetizzabili. In particolare, il codice HDL viene diviso tra moduli sintetizzabili e "banco di prova" (*testbench*), dove i moduli sintetizzabili sono la parte circuitale e il banco di prova contiene le istruzioni per applicare valori agli ingressi, verificare la correttezza dei valori di uscita ed evidenziare le discrepanze tra valori attesi e valori effettivi. Quindi il codice del banco di prova serve solo alla simulazione e non può essere sintetizzato a livello hardware.

Uno degli errori tipici di un principiante è ritenere HDL un programma per calcolatore invece di un metodo per descrivere circuiti digitali. Se non si ha un'idea approssimativa di ciò che dovrebbe risultare dalla sintesi HDL, il risultato che si ottiene può non piacere: si può ottenere un circuito molto più complesso del necessario, o scrivere del codice che consente una simulazione corretta ma che non può essere sintetizzato in hardware. Meglio quindi pensare al sistema da realizzare in termini di blocchi di logica combinatoria, registri e macchine a stati finiti, poi fare uno schizzo su carta di come i vari elementi sono collegati e, solo in seguito, cominciare a codificare in HDL.

L'esperienza degli Autori insegna che si impara meglio un HDL ricorrendo a esempi. I vari HDL hanno modi specifici di descrivere le varie classi di componenti logiche, definiti **idiomi**: nel capitolo si vedrà come scrivere gli idiomi HDL adatti per ogni tipo di blocco logico e, successivamente, come mettere insieme

Lo strumento di sintesi associa un identificatore (label) a ogni porta logica sintetizzata. In Figura 4.2, sono un5_y, un8_y e y.

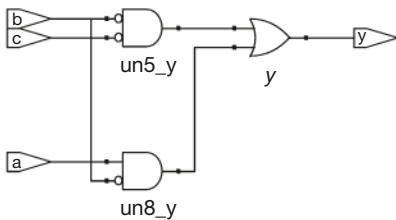


Figura 4.2
Sintesi del circuito.

² La simulazione è stata eseguita utilizzando lo strumento ModelSim PE Student Edition versione 10.3c. ModelSim è stato scelto perché pur essendo uno strumento commerciale rende disponibile in forma gratuita una versione per studenti capace di gestire programmi fino a 10 000 linee di codice.

³ La sintesi è stata eseguita utilizzando lo strumento Synplify Premier della ditta Synplify. Esso è stato scelto perché è lo strumento più diffuso di sintesi da HDL a FPGA (*Field Programmable Gate Array*, descritti nel paragrafo 5.6.2) e perché è disponibile in forma gratuita per le università.

i vari blocchi per ottenere il sistema finale. Quando si deve descrivere un particolare circuito, è quindi opportuno guardare un esempio simile e adattarlo ai propri scopi. Non si vuole certo definire in modo rigoroso tutta la sintassi degli HDL, perché sarebbe estremamente noioso ma soprattutto perché favorirebbe l'idea che l'HDL è un linguaggio di programmazione e non una descrizione circuitale. Le specifiche di SystemVerilog e di VHDL, così come numerosi libri di testo sull'argomento, contengono tutte le informazioni qualora servisse approfondire un aspetto particolare (*vedi* al riguardo la Bibliografia alla fine del testo).

4.2 ■ LOGICA COMBINATORIA

Come già sottolineato, ci si limita in questo testo alla progettazione di reti sequenziali sincrone, costituite da reti combinatorie e registri, dove le reti combinatorie sono reti logiche le cui uscite dipendono solo dai valori in ingresso correnti. In questa sezione si discute come scrivere i modelli comportamentali di reti combinatorie in HDL.

4.2.1 Operatori a singolo bit

Gli operatori a singolo bit (*bitwise*) agiscono su segnali costituiti da bit singoli o su bus multibit. Per esempio, il modulo `neg` nell'**Esempio HDL 4.2** descrive quattro negatori collegati a bus a 4 bit.

L'ordinamento dei bit di un bus (*little-endian* oppure *big-endian*: *vedi* il riquadro nel paragrafo 6.2.2 per la spiegazione dell'origine di questi termini) è assolutamente arbitrario e ininfluenza nell'esempio in esame, perché il funzionamento di una batteria di negatori non dipende dall'ordine dei bit. L'ordinamento diventa importante solo per operatori come la somma, dove il riporto tra i bit di una colonna deve essere sommato ai bit della colonna successiva. Si può scegliere quale ordinamento adottare purché si rimanga sempre consi-

ESEMPIO HDL 4.2 NEGATORI

SystemVerilog

```
module neg (input  logic [3:0] a,
           output logic [3:0] y);
    assign y = ~a;
endmodule
```

`a[3:0]` rappresenta un bus a 4 bit, denominati dal più significativo al meno significativo `a[3]`, `a[2]`, `a[1]` e `a[0]`. Questo ordine viene definito in inglese *little-endian* perché il bit meno significativo ha come numero indice il valore minore. Si sarebbe potuto denominare il bus `a[4:1]`, nel qual caso `a[4]` sarebbe stato il bit più significativo, come pure `a[0:3]` e allora i nomi dei bit dal più significativo al meno significativo sarebbero stati `a[0]`, `a[1]`, `a[2]` e `a[3]` seguendo l'ordine noto come *big-endian*.

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;
entity neg is
    port(a: in  STD_LOGIC_VECTOR(3 downto 0);
         y: out STD_LOGIC_VECTOR(3 downto 0));
end;
architecture sintesi of neg is
begin
    y <= not a;
end;
```

VHDL usa la notazione `STD_LOGIC_VECTOR` per indicare dei bus di tipo `STD_LOGIC`. Quindi `STD_LOGIC_VECTOR(3 downto 0)` indica un bus a 4 bit. I bit, dal più al meno significativo, sono denominati `a(3)`, `a(2)`, `a(1)` e `a(0)`. Questo ordine viene definito in inglese *little-endian* perché il bit meno significativo ha come numero indice il valore minore. Si sarebbe potuto definire il bus `STD_LOGIC_VECTOR(4 downto 1)`, nel qual caso `a(4)` sarebbe stato il bit più significativo, come pure `STD_LOGIC_VECTOR(0 to 3)` e allora i nomi dei bit dal più significativo al meno significativo sarebbero stati `a(0)`, `a(1)`, `a(2)` e `a(3)` seguendo l'ordine noto come *big-endian*.

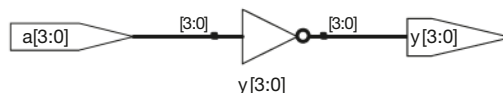


Figura 4.3 Sintesi del circuito `neg`.

stenti con la scelta fatta: in questo testo si è deciso di adottare l'ordinamento little-endian, che per un bus a N bit è definito $[N-1:0]$ in SystemVerilog e $(N-1 \text{ downto } 0)$ in VHDL.

Dopo ogni esempio di codice, il capitolo riporta lo schema prodotto a partire dal codice SystemVerilog dallo strumento di sintesi Synplify Premier. La **Figura 4.3** mostra che il modulo `neg` viene sintetizzato da una batteria di quattro negatori, indicati dalla sigla $y[3:0]$, collegati a un bus di ingresso e uno di uscita a 4 bit. Un circuito simile viene prodotto a partire dal codice VHDL.

Il modulo `porte` dell'**Esempio HDL 4.3** mostra operazioni a bit singolo su bus a 4 bit per altri tipi di funzioni logiche elementari.

ESEMPIO HDL 4.3 PORTE LOGICHE

SystemVerilog

```
module porte(input  logic [3:0] a, b,
            output logic [3:0] y1, y2, y3, y4, y5);

    /* cinque diverse porte logiche a 2 ingressi
       collegate a bus a 4 bit */
    assign y1 = a & b;    // AND
    assign y2 = a | b;    // OR
    assign y3 = a ^ b;    // XOR
    assign y4 = ~(a & b); // NAND
    assign y5 = ~(a | b); // NOR
endmodule
```

`~`, `&` e `|` sono esempi di **operatori** di SystemVerilog, mentre `a`, `b` e `y1` sono operandi. Una combinazione di operatori e operandi, come `a & b` oppure `~(a | b)`, è un'espressione. Una scritta completa come `assign y4 = ~(a & b);` è un'istruzione (*statement*).

`assign out = in1 op in2;` (terminata dal punto e virgola) è definita **istruzione di assegnamento continuo**. Ogni volta che uno degli ingressi nella parte a destra del simbolo `=` di un'istruzione di assegnamento continuo varia, l'uscita nella parte sinistra viene ricalcolata. Quindi le istruzioni di assegnamento continuo descrivono la logica combinatoria.

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;
entity porte is
    port(a, b: in STD_LOGIC_VECTOR(3 downto 0);
         y1, y2, y3, y4,
         y5: out STD_LOGIC_VECTOR(3 downto 0));
end;
architecture sintesi of porte is
begin
    -- cinque diverse porte logiche a 2 ingressi
    -- collegate a bus a 4 bit
    y1 <= a and b;
    y2 <= a or b;
    y3 <= a xor b;
    y4 <= a nand b;
    y5 <= a nor b;
end;
```

`not`, `xor` e `or` sono esempi di **operatori** di VHDL, mentre `a`, `b` e `y1` sono operandi. Una combinazione di operatori e operandi, come `a and b` oppure `a nor b`, è un'espressione. Una scritta completa come `y4 <= a nand b;` è un'istruzione (*statement*).

`out <= in1 op in2;` (terminata dal punto e virgola) è definita **istruzione di assegnamento di segnale concorrente**. Ogni volta che uno degli ingressi nella parte a destra del simbolo `<=` di un'istruzione di assegnamento di segnale concorrente varia, l'uscita nella parte sinistra viene ricalcolata. Quindi le istruzioni di assegnamento di segnale concorrente descrivono la logica combinatoria.

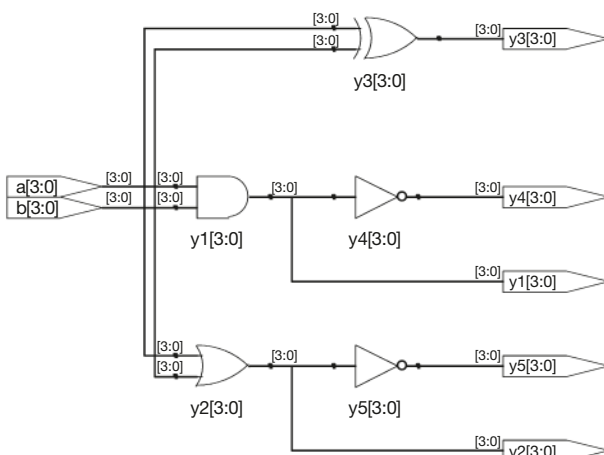


Figura 4.4
Sintesi del circuito porte.

4.2.2 Commenti e spazio vuoto

L'esempio `porte` ha mostrato anche che formato dare ai commenti. SystemVerilog e VHDL non si curano degli spazi vuoti (cioè dei caratteri spazio, tabulazione e ritorno a capo), tuttavia è buona norma indentare opportunamente le istruzioni e inserire righe vuote per aiutare la leggibilità dei progetti non banali. Anche un uso consistente delle maiuscole e del carattere di sottolineatura nei nomi di moduli e segnali è raccomandato (in questo testo si usano sempre lettere minuscole). Si noti che i nomi di moduli e segnali non possono cominciare con una cifra.

SystemVerilog

I commenti in SystemVerilog sono simili a quelli in C o in Java: iniziano con la coppia di caratteri `/*` e continuano, anche su più righe del file, fino alla coppia di caratteri `*/`. Commenti che iniziano con la coppia di caratteri `//` terminano invece alla fine della riga.

Si noti che SystemVerilog distingue fra maiuscole e minuscole, quindi `y1` e `Y1` sono due segnali diversi. Naturalmente, usare solo maiuscole e minuscole per distinguere segnali diversi rischia solo di fare confusione.

VHDL

I commenti in VHDL sono simili a quelli in C o in Java: iniziano con la coppia di caratteri `/*` e continuano, anche su più righe del file, fino alla coppia di caratteri `*/`. Commenti che iniziano con la coppia di caratteri `--` terminano invece alla fine della riga.

Si noti che VHDL non distingue fra maiuscole e minuscole, quindi `y1` e `Y1` sono lo stesso segnale. Però altri strumenti che usano i file scritti in VHDL possono distinguere fra maiuscole e minuscole, generando strani errori se si mescolano maiuscole e minuscole senza fare attenzione.

4.2.3 Operatori di riduzione

Gli operatori di riduzione sono costituiti da porte logiche a tanti ingressi che producono un'unica uscita. L'**Esempio HDL 4.4** descrive una porta AND con 8 ingressi $a_7, a_6, \dots, a_0$. Simili operatori di riduzione esistono con le porte OR, XOR, NAND, NOR, XNOR. Si ricordi che una porta XOR a multi ingressi calcola la parità, restituendo il valore VERO se un numero dispari di ingressi vale VERO.

ESEMPIO HDL 4.4 PORTA AND A 8 INGRESSI

SystemVerilog

```
module and8(input logic [7:0] a,
            output logic y);

    assign y = &a;

    // assign y = &a è molto più facile da scrivere di
    // assign y = a[7] & a[6] & a[5] & a[4] &
    //          a[3] & a[2] & a[1] & a[0];
endmodule
```

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity and8 is
    port(a: in  STD_LOGIC_VECTOR(7 downto 0);
          y: out STD_LOGIC);
end;

architecture sintesi of and8 is
begin
    y <= and a;
    -- y <= and a è molto più facile da scrivere di
    -- y <= a(7) and a(6) and a(5) and a(4) and
    --       a(3) and a(2) and a(1) and a(0);
end;
```

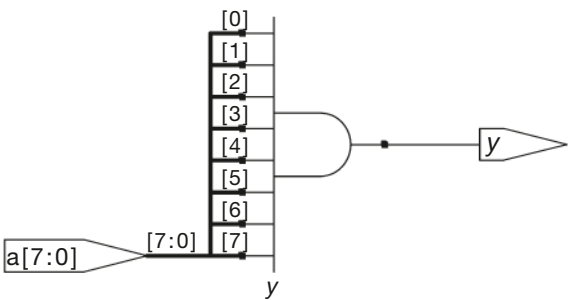


Figura 4.5 Sintesi del circuito and8.

4.2.4 Assegnamento condizionale

Un **assegnamento condizionale** seleziona l'uscita da generare tra varie alternative sulla base di un ingresso chiamato **condizione**. L'**Esempio HDL 4.5** mostra un multiplexer 2:1 che fa uso di assegnamento condizionale.

ESEMPIO HDL 4.5 MULTIPLEXER 2:1

SystemVerilog

L'**operatore condizionale** ?: seleziona, sulla base della prima espressione, la seconda o la terza espressione. La prima espressione è denominata **condizione**: se la condizione vale 1 l'operatore sceglie la seconda espressione, se vale 0 la terza. ?: è particolarmente utile per descrivere un multiplexer, che sulla base del valore del primo ingresso ne seleziona uno degli altri due. Il codice sottostante mostra l'idioma di un multiplexer 2:1 a 4 bit di ingresso e uscita, che utilizza l'operatore condizionale.

```
module mux2(input  logic [3:0] d0, d1,
            input  logic      s,
            output logic [3:0] y);
    assign y = s ? d1 : d0;
endmodule
```

Se s vale 1, allora y = d1, se s vale 0 allora y = d0. ?: è anche definito un **operatore ternario** perché ha tre ingressi (quindi tre operandi). È usato con lo stesso scopo anche nei linguaggi di programmazione C e Java.

VHDL

Gli **assegnamenti condizionali di segnale** eseguono operazioni diverse a seconda di opportune condizioni. Sono particolarmente utili per descrivere un multiplexer. Per esempio, un multiplexer 2:1 può usare un assegnamento condizionale di segnale per scegliere uno di due ingressi a 4 bit.

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity mux2 is
    port(d0, d1: in  STD_LOGIC_VECTOR(3 downto 0);
          s:      in  STD_LOGIC;
          y:      out STD_LOGIC_VECTOR(3 downto 0));
end;

architecture sintesi of mux2 is
begin
    y <= d1 when s else d0;
end;
```

L'assegnamento condizionale assegna a y i valori di d1 se s vale 1, altrimenti quelli di d0. Si noti che nelle versioni di VHDL precedenti il 2008 si doveva scrivere when s = '1' invece di when s.

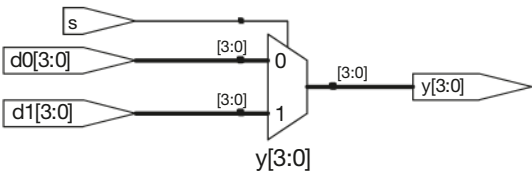


Figura 4.6 Sintesi del circuito mux2.

L'**Esempio HDL 4.6** mostra un multiplexer 4:1 realizzato con lo stesso principio del multiplexer 2:1 dell'**Esempio HDL 4.5**. La **Figura 4.7** mostra lo schema del multiplexer 4:1 prodotto da Synplify Premier: questo strumento usa per il multiplexer un simbolo diverso da quello usato sin qui nel testo. Il multiplexer ha ingressi multipli di dato (*d*) e ingressi di abilitazione (*e* da *enable*). Quando uno dei segnali di abilitazione è attivato, i dati corrispondenti vengono passati in uscita. Per esempio, quando $s[1]=s[0]=0$ la porta AND in basso, un `un1_s_5`, produce in uscita il valore 1 che abilita l'ingresso in basso del multiplexer che a sua volta seleziona `d0[3:0]`.

ESEMPIO HDL 4.6 MULTIPLEXER 4:1

SystemVerilog

Un multiplexer 4:1 seleziona uno di quattro ingressi utilizzando operatori condizionali annidati.

```
module mux4(input  logic [3:0] d0, d1, d2, d3
            input  logic [1:0] s,
            output logic [3:0] y);
    assign y = s[1] ? (s[0] ? d3 : d2)
              : (s[0] ? d1 : d0);
endmodule
```

Se $s[1]$ vale 1, allora il multiplexer sceglie la prima espressione, $(s[0] ? d3 : d2)$, che a sua volta sceglie $d3$ oppure $d2$ in base a $s[0]$ ($y = d3$ se $s[0]$ vale 1, $y = d2$ se $s[0]$ vale 0). Analogamente se $s[1]$ vale 0 il multiplexer sceglie la seconda espressione, che dà $d1$ o $d0$ in base al valore di $s[0]$.

VHDL

Un multiplexer 4:1 seleziona uno di quattro ingressi utilizzando più clausole `else` nell'assegnamento condizionale di segnale.

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity mux4 is
    port(d0, d1, d2, d3: in  STD_LOGIC_VECTOR(3 downto 0);
          s:               in  STD_LOGIC_VECTOR(1 downto 0);
          y:               out STD_LOGIC_VECTOR(3 downto 0));
end;
```

```
architecture sintesi1 of mux4 is
begin
    y <= d0 when s = "00" else
         d1 when s = "01" else
         d2 when s = "10" else
         d3;
end;
```

VHDL supporta anche istruzioni di **assegnamento con selezione di segnale** che sono abbreviazioni quando si deve scegliere fra diverse possibilità. Il discorso è analogo all'uso dell'istruzione `switch/case` invece di una serie di istruzioni `if/else` in alcuni linguaggi di programmazione. Utilizzando l'assegnamento con selezione di segnale il multiplexer 4:1 può essere riscritto in questo modo:

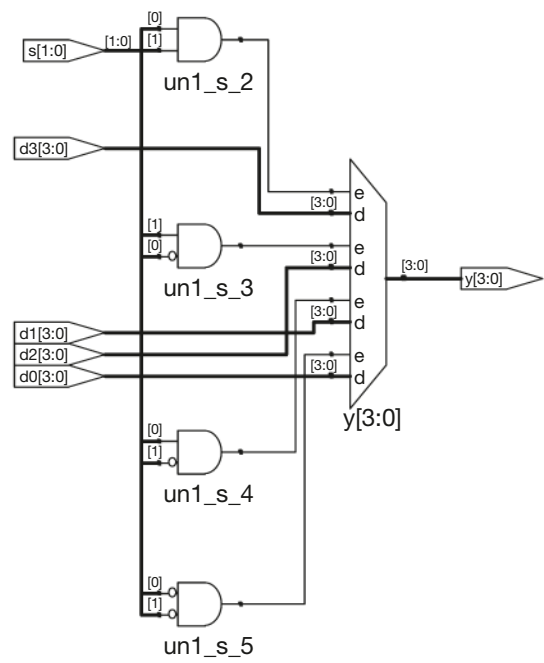
```
architecture sintesi2 of mux4 is
begin
    with s select y <=
        d0 when "00",
        d1 when "01",
        d2 when "10",
        d3 when others;
end;
```

4.2.5 Variabili interne

Può essere utile dividere una funzione complessa in passi intermedi. Per esempio, il sommatore completo, discusso nel paragrafo 5.2.1, è un circuito con tre ingressi e due uscite, descritto dalle seguenti espressioni:

$$\begin{aligned} S &= A \oplus B \oplus R_{in} \\ R_{out} &= AB + AR_{in} + BR_{in} \end{aligned} \quad (4.1)$$

Figura 4.7
Sintesi del circuito mux4.



Se si definiscono i due segnali intermedi, P e G :

$$\begin{aligned} P &= A \oplus B \\ G &= AB \end{aligned} \tag{4.2}$$

il sommatore può essere riscritto come segue:

$$\begin{aligned} S &= P \oplus R_{in} \\ R_{out} &= G + PR_{in} \end{aligned} \tag{4.3}$$

Si può convincersi della correttezza di queste espressioni compilando la corrispondente tabella delle verità.

P e G sono dette **variabili interne**, perché non sono né ingressi né uscite ma sono utilizzate solo all'interno del modulo. Sono simili alle variabili locali nei linguaggi di programmazione. L'**Esempio HDL 4.7** mostra come si usano negli HDL.

ESEMPIO HDL 4.7 SOMMATORE COMPLETO	
SystemVerilog In SystemVerilog i segnali interni sono solitamente dichiarati di tipo logic. <pre>module sommatore (input logic a, b, rin, output logic s, rout); logic p, g; assign p = a ^ b; assign g = a & b; assign s = p ^ rin; assign rout = g (p & rin); endmodule</pre>	VHDL In VHDL si usano segnali per rappresentare variabili interne i cui valori sono definiti da istruzioni di assegnamento concorrente di segnale come <code>p <= a xor b</code> . <pre>library IEEE; use IEEE.STD_LOGIC_1164.all; entity sommatore is port(a, b, rin: in STD_LOGIC; s, rout: out STD_LOGIC); end; architecture sintesi of sommatore is signal p, g: STD_LOGIC; begin p <= a xor b; g <= a and b; s <= p xor rin; rout <= g or (p and rin); end;</pre>

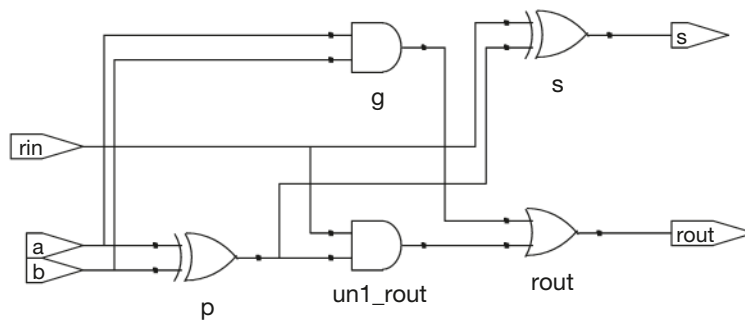


Figura 4.8
Sintesi del circuito sommatore.

Le istruzioni di assegnamento negli HDL (`assign` in SystemVerilog e `<=` in VHDL) vengono eseguite in modo concorrente, al contrario di quanto accade nei linguaggi di programmazione come C e Java, nei quali vengono eseguite nell'ordine in cui sono scritte. In un linguaggio di programmazione è importante che $S = P \oplus R_{in}$ venga dopo $P = A \oplus B$ perché le istruzioni vengono eseguite in ordine. Invece negli HDL l'ordine non conta: come accade a livello hardware, le istruzioni di assegnamento vengono valutate non appena gli ingressi, ovvero i segnali nella parte destra delle istruzioni, modificano il loro valore, indipendentemente da dove le istruzioni compaiono nel modulo.

4.2.6 Precedenza

Si noti l'uso delle parentesi nel calcolo di `rout` nell'**Esempio HDL 4.7** per eseguire le operazioni nell'ordine $R_{out} = G + (P \cdot R_{in})$ invece di $R_{out} = (G + P) \cdot R_{in}$. Se non si fa uso di parentesi, l'ordine di esecuzione dipende dal linguaggio: l'**Esempio HDL 4.8** indica la precedenza degli operatori dalla più alta alla più bassa per entrambi gli HDL utilizzati. Le tabelle includono gli operatori aritmetici, le traslazioni e gli operatori di confronto che saranno definiti nel Capitolo 5.

ESEMPIO HDL 4.8 PRECEDENZA DEGLI OPERATORI

SystemVerilog

Tabella 4.1 Precedenza degli operatori in SystemVerilog.

	Op	Significato
A	~	NOT
I	*, /, %	MUL, DIV, MOD
t	+, -	PIÙ, MENO
a	<<, >>	Traslazione logica a sinistra/destra
	<<<, >>>	Traslazione aritmetica a sinistra/destra
	<, <=, >, >=	Confronto relativo
	==, !=	Confronto di uguaglianza
B	&, ~&	AND, NAND
a	^, ~^	XOR, XNOR
s	, ~	OR, NOR
s	?:	Condizionale

La precedenza degli operatori in SystemVerilog è simile a quella dei normali linguaggi di programmazione. In particolare, AND ha precedenza su OR, quindi si possono eliminare le parentesi nell'istruzione:

```
assign rout g | p & rin;
```

VHDL

Tabella 4.2 Precedenza degli operatori in VHDL.

	Op	Significato
A	not	NOT
I	*, /, mod, rem	MUL, DIV, MOD, REM
t	+, -	PIÙ, MENO
a	rol, ror,	Rotazione
	srl, sll	Traslazione logica
	<, <=, >, >=	Confronto relativo
	=, /=	Confronto di uguaglianza
B	and, or, nand,	Operatori logici
a	nor, xor,	
s	xnor	

La moltiplicazione in VHDL ha precedenza sulla somma, come d'abitudine. Invece, al contrario di SystemVerilog, tutti gli operatori logici (and, or ecc.) hanno la medesima precedenza, a differenza di quanto avviene nell'algebra booleana, quindi le parentesi diventano indispensabili, perché l'istruzione `rout <= g or p and rin` senza parentesi verrebbe interpretata da sinistra a destra come `rout <= (g or p) and rin`.

4.2.7 Numeri

I numeri possono essere rappresentati in binario, ottale, decimale o esadecimale (rispettivamente base 2, 8, 10 e 16). La dimensione, ovvero il numero di bit, può opzionalmente essere specificata, e nelle posizioni più significative vengono inseriti zeri fino a riempire tale dimensione. Il carattere di sottolineatura (il trattino basso, in inglese *underscore*) inserito nei numeri viene ignorato, e può essere utile per rendere più leggibili numeri lunghi spezzandoli in blocchi. L'Esempio HDL 4.9 mostra come i numeri sono scritti in ciascun linguaggio.

4.2.8 Z e X

Gli HDL usano *z* per indicare un valore fluttuante (*floating*), particolarmente utile per descrivere un buffer tristate, il cui valore di uscita è appunto fluttuante se il segnale di abilitazione è 0 (cioè il buffer è disabilitato). Si ricordi che nel paragrafo 2.6.2 si è visto che un bus può essere pilotato da vari buffer tristate, solo uno dei quali deve essere abilitato. L'Esempio HDL 4.10 mostra l'idioma per un buffer tristate. Se il buffer è abilitato l'uscita riproduce il valore di ingresso. Se il buffer è disabilitato all'uscita viene assegnato il valore fluttuante *z*. Analogamente, gli HDL usano *x* per indicare un valore logico non valido. Se un bus viene pilotato simultaneamente a 0 e a 1 da due buffer tristate entrambi abilitati (o da altre porte logiche) il risultato è *x* che indica un conflitto. Se invece tutti i buffer tristate sono disabilitati, il bus è fluttuante e viene indicato con *z*.

ESEMPIO HDL 4.9 NUMERI									
SystemVerilog					VHDL				
Il formato per dichiarare costanti è <code>N'Bvalore</code>, dove <i>N</i> è la dimensione in bit, <i>B</i> è una lettera che indica la base, e <i>valore</i> specifica il valore. Per esempio, <code>9'h25</code> indica un numero esadecimale a 9 bit, il cui valore è $25_{16} = 37_{10} = 000100101_2$. SystemVerilog riconosce <code>'b</code> per binario, <code>'o</code> per ottale, <code>'d</code> per decimale e <code>'h</code> per esadecimale. Se si omette la base, il numero viene considerato decimale. Se non si specifica la dimensione, si assume che il numero abbia la stessa quantità di bit dell'espressione nella quale viene utilizzato. Sono automaticamente aggiunti zeri nelle posizioni più significative per riempire tutti i bit. Per esempio, se <code>w</code> è un bus a 6 bit, <code>assign w='b11</code> assegna a <i>w</i> il valore <code>000011</code>. Tuttavia è buona norma specificare sempre la dimensione. Un'eccezione sono gli idiomi <code>'0</code> e <code>'1</code>, che per SystemVerilog servono a riempire un bus rispettivamente con tutti 0 e tutti 1.					In VHDL, i numeri <code>STD_LOGIC</code> sono scritti in binario racchiusi fra apici singoli: <code>'0</code> e <code>'1</code> indicano i valori logici 0 e 1. Il formato per dichiarare costanti di tipo <code>STD_LOGIC_VECTOR</code> è <code>NB"valore"</code>, dove <i>N</i> è la dimensione in bit, <i>B</i> è una lettera che indica la base, e <i>valore</i> specifica il valore. Per esempio, <code>9X"25"</code> indica un numero esadecimale a 9 bit, il cui valore è $25_{16} = 37_{10} = 000100101_2$. VHDL 2008 riconosce <code>B</code> per binario, <code>O</code> per ottale, <code>D</code> per decimale e <code>X</code> per esadecimale. Se si omette la base, il numero viene considerato binario. Se non si specifica la dimensione, si assume che il numero abbia la quantità di bit necessaria a rappresentarlo. Si tenga presente che fino a ottobre 2011 lo strumento Synplify Premier di Synopsys non supporta ancora la specifica della dimensione. <code>others=>'0</code> e <code>others=>'1</code> sono gli idiomi che per VHDL servono a riempire un bus rispettivamente con tutti 0 o tutti 1.				
Tabella 4.3 Numeri in SystemVerilog.					Tabella 4.4 Numeri in VHDL.				
Numeri	Bit	Base	Val	Risultato	Numeri	Bit	Base	Val	Risultato
3'b101	3	2	5	101	3B"101"	3	2	5	101
'b11	?	2	3	000 ... 0011	B"11"	2	2	3	11
8'b11	8	3	3	00000011	8B"11"	8	2	3	00000011
8'b1010_1011	8	2	171	10101011	8B"1010_1011"	8	2	171	10101011
3'd6	3	10	6	110	3D"6"	3	10	6	110
6'o42	6	8	34	100010	6O"42"	6	8	34	100010
8'hAB	8	16	171	10101011	8X"AB"	8	16	171	10101011
42	?	10	2	00 ... 0101010	"101"	3	2	5	101
					B"101"	3	2	5	101
					X"AB"	8	16	171	10101011

ESEMPIO HDL 4.10 BUFFER TRISTATE**SystemVerilog**

```
module tristate(input  logic[3:0] a,
               input  logic    en,
               output tri  [3:0] y);
    assign y = en ? a : 4'bz;
endmodule
```

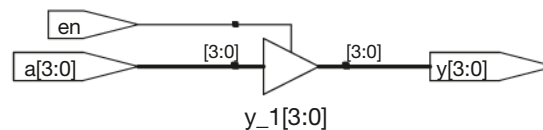
Si noti che *y* è stato dichiarato *tri* invece di *logic*. Infatti i segnali *logic* possono avere un solo circuito che li pilota. I bus tristate possono avere più circuiti pilota, quindi devono essere dichiarati come *net*. Due tipi di *net* in SystemVerilog sono chiamate *tri* e *triereg*. Normalmente solo un circuito pilota è attivo in ogni momento, e la *net* assume il valore di tale circuito. Se nessun circuito pilota è attivo, una *net tri* è fluttuante (*z*) mentre una *net triereg* mantiene l'ultimo valore significativo. Se non si specifica il tipo di *net* per un ingresso o un'uscita, la *net* viene considerata *tri*. Si noti anche che l'uscita *tri* di un modulo può essere usata come ingresso *logic* di un altro modulo. Nel paragrafo 4.7 si discutono ulteriormente le *net* con più circuiti pilota.

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity tristate is
    port(a: in  STD_LOGIC_VECTOR(3 downto 0);
         en: in  STD_LOGIC;
         y: out STD_LOGIC_VECTOR(3 downto 0));
end;

architecture sintesi of tristate is
begin
    y <= a when en else "ZZZZ";
end;
```

**Figura 4.9** Sintesi del circuito *tristate*.

All'inizio della simulazione, i nodi di stato come le uscite dei flip-flop sono inizializzati a un valore sconosciuto (*x* per SystemVerilog e *u* per VHDL). Questo è molto utile per scoprire gli errori causati dalla possibilità di dimenticarsi di resettare un flip-flop prima di utilizzarne i valori di uscita.

Se una porta logica riceve un ingresso fluttuante, può produrre un'uscita *x* se non è in grado di determinare il valore di uscita corretto. Analogamente, se riceve un ingresso illegale o non inizializzato, può produrre un'uscita *x*. L'**Esempio HDL 4.11** mostra come SystemVerilog e VHDL combinano questi valori dei segnali nelle porte logiche.

Se si trovano valori *x* o *u* nella simulazione, quasi sempre si tratta di errori o di inadeguata scrittura del codice HDL. Nella sintesi, questo corrisponde a ingressi di porte logiche fluttuanti, stati non inizializzati o conflitti. I valori *x* e

ESEMPIO HDL 4.11 TABELLE DELLE VERITÀ CON INGRESSI NON DEFINITI O FLUTTUANTI**SystemVerilog**

I valori dei segnali in SystemVerilog sono 0, 1, *z* e *x*. Costanti SystemVerilog che iniziano con *z* o *x* sono riempite di *z* e *x* invece di zeri nelle posizioni più significative fino a riempire lo spazio allocato.

La **Tabella 4.5** mostra la tabella delle verità di una porta AND con tutti i quattro possibili valori dei segnali. Si noti che in qualche caso la porta è in grado di determinare il corretto valore di uscita anche in presenza di valori di ingresso non noti. Per esempio 0 & *z* restituisce 0 perché l'uscita di una porta AND è sempre 0 se uno dei suoi ingressi è 0. In altri casi, ingressi fluttuanti o non validi producono uscite non valide, visualizzate come *x* in SystemVerilog.

VHDL

I valori dei segnali in VHDL STD_LOGIC sono '0', '1', 'z', 'x' e 'u'. La **Tabella 4.6** mostra la tabella delle verità di una porta AND con tutti i cinque possibili valori dei segnali. Si noti che in qualche caso la porta è in grado di determinare il corretto valore di uscita anche in presenza di valori di ingresso non noti. Per esempio '0' and 'z' restituisce '0' perché l'uscita di una porta AND è sempre 0 se uno dei suoi ingressi è '0'. In altri casi, ingressi fluttuanti o non validi producono uscite non valide, visualizzate come 'x' in VHDL. Ingressi non inizializzati producono uscite non inizializzate, visualizzate come 'u' in VHDL.

ESEMPIO HDL 4.13 PORTE LOGICHE CON RITARDI**SystemVerilog**

```
'timescale 1ns/1ps
module esempio(input logic a, b, c,
               output logic y);

  logic ab, bb, cb, n1, n2, n3;

  assign #1 {ab, bb, cb} = ~{a, b, c};
  assign #2 n1 = ab & bb & cb;
  assign #2 n2 = a & bb & cb;
  assign #2 n3 = a & bb & c;
  assign #4 y = n1 | n2 | n3;
endmodule
```

I file SystemVerilog possono includere una direttiva di scala temporale che indica il valore di ogni unità di tempo. La direttiva è nella forma `'timescale unit/precision`. In questo esempio ogni unità è 1 ns, e la simulazione ha 1 ps di precisione. Se non si specifica la scala temporale, viene adottato, in genere, il valore di 1 ns sia per l'unità sia per la precisione. In SystemVerilog il simbolo # è usato per indicare il numero di unità di ritardo. Può essere indicato nelle istruzioni `assign`, sia non bloccanti (`<=`) sia bloccanti (`=`), discusse nel paragrafo 4.5.4.

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity esempio is
  port(a, b, c: in  STD_LOGIC;
        y:          out STD_LOGIC);
end;

architecture sintesi of esempio is
  signal ab, bb, cb, n1, n2, n3: STD_LOGIC;
begin
  ab <= not a after 1 ns;
  bb <= not b after 1 ns;
  cb <= not c after 1 ns;
  n1 <= ab and bb and cb after 2 ns;
  n2 <= a and bb and cb after 2 ns;
  n3 <= a and bb and c after 2 ns;
  y  <= n1 or n2 or n3 after 4 ns;
end;
```

In VHDL la clausola `after` è usata per indicare i ritardi. Le unità in questo esempio sono specificate essere nanosecondi.

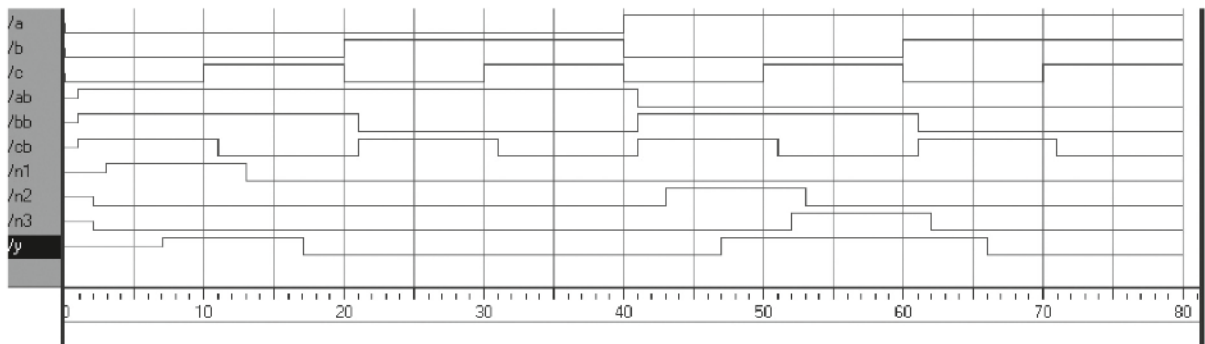


Figura 4.10 Esempi di forme d'onda di simulazione con ritardi (con il simulatore ModelSim).

d'onda di simulazione, con `y` che ritarda di 7 ns dopo l'arrivo dei valori di ingresso. Si noti che all'inizio della simulazione `y` è sconosciuto.

4.3 ■ MODELLAZIONE STRUTTURALE

Il paragrafo precedente ha affrontato la modellazione comportamentale (*behavioral*), che descrive un modulo in termini di uscite come funzioni degli ingressi. In questo paragrafo si esamina invece la modellazione **strutturale** (*structural*), che descrive come un modulo è composto in termini di moduli più semplici.

L'**Esempio HDL 4.14** mostra come costruire un multiplexer 4:1 con tre multiplexer 2:1. Ogni copia del multiplexer 2:1 è chiamata **istanza**. Più istanze di uno stesso modulo si distinguono grazie a nomi diversi, in questo caso `muxbasso`, `muxalto` e `muxuscita`. Ecco un esempio di regolarità, nel quale il multiplexer 2:1 è utilizzato più volte.

L'**Esempio HDL 4.15** usa la modellazione strutturale per costruire un multiplexer 2:1 partendo da una coppia di buffer tristate. Tuttavia non è consigliabile costruire circuiti logici facendo uso di buffer tristate.

ESEMPIO HDL 4.14 MODELLO STRUTTURALE DI UN MULTIPLEXER 4:1**SystemVerilog**

```

module mux4(input  logic [3:0] d0, d1, d2, d3,
            input  logic [1:0] s,
            output logic [3:0] y);

    logic [3:0] basso, alto;

    mux2 muxbasso(d0, d1, s[0], basso);
    mux2 muxalto(d2, d3, s[0], alto);
    mux2 muxuscita(basso, alto, s[1], y);
endmodule

```

Le tre istanze di mux2 sono denominate muxbasso, muxalto e muxuscita. Il modulo mux2 deve essere definito da qualche altra parte nel codice SystemVerilog – vedi gli Esempi HDL 4.5, 4.15 o 4.34.

VHDL

```

library IEEE; use IEEE.STD_LOGIC_1164.all;

entity mux4 is
    port(d0, d1,
          d2, d3: in  STD_LOGIC_VECTOR(3 downto 0);
          s:      in  STD_LOGIC_VECTOR(1 downto 0);
          y:      out STD_LOGIC_VECTOR(3 downto 0));
end;

architecture struttura of mux4 is
    component mux2
        port(d0,
              d1: in  STD_LOGIC_VECTOR(3 downto 0);
              s: in  STD_LOGIC;
              y: out STD_LOGIC_VECTOR(3 downto 0));
    end component;
    signal basso, alto: STD_LOGIC_VECTOR(3 downto 0);
begin
    muxbasso: mux2 port map(d0, d1, s(0), basso);
    muxalto:  mux2 port map(d2, d3, s(0), alto);
    muxuscita: mux2 port map(basso, alto, s(1), y);
end;

```

Nell'architettura si devono innanzitutto dichiarare ingressi e uscite di mux2 usando l'istruzione di dichiarazione component. Questo consente agli strumenti VHDL di verificare che il componente che si vuole usare abbia gli stessi ingressi e le stesse uscite dichiarate da qualche altra parte in un'altra istruzione di entità, per evitare errori dovuti a modifiche dell'entità e non delle sue istanze. La cosa però rende il codice VHDL più pesante. Si noti che in questo caso l'architettura di mux4 è stata dichiarata di tipo struct mentre nelle descrizioni comportamentali dei moduli nel paragrafo 4.2 si è usato il tipo synth. VHDL consente di avere più architetture (cioè implementazioni) della stessa entità, distinte per nome. I nomi non hanno importanza per gli strumenti CAD, ma i due tipi struct e synth sono abituali. Il codice VHDL destinato alla sintesi normalmente contiene una sola descrizione architetturale per ogni entità, quindi nel resto del testo non ci si occupa della sintassi da usare in VHDL per specificare quale architettura usare in presenza di più descrizioni architetture per la stessa entità.

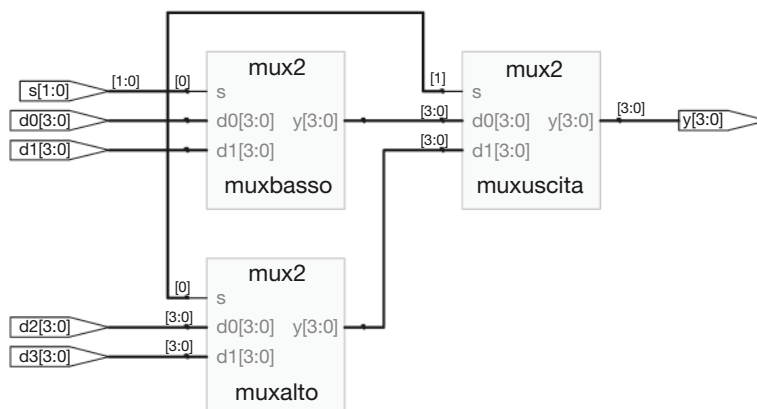


Figura 4.11 Sintesi del circuito mux4.

ESEMPIO HDL 4.15 MODELLO STRUTTURALE DI UN MULTIPLEXER 2:1**SystemVerilog**

```

module mux2(input  logic [3:0] d0, d1,
            input  logic      s,
            output tri  [3:0] y);

    tristate t0(d0, ~s, y);
    tristate t1(d1, s, y);
endmodule

```

In SystemVerilog, espressioni come `~s` sono permesse nell'elenco di ingressi e uscite di un'istanza. Espressioni arbitrariamente complesse sono comunque sconsigliate perché rendono il codice poco leggibile.

VHDL

```

library IEEE; use IEEE.STD_LOGIC_1164.all;

entity mux2 is
    port(d0, d1: in  STD_LOGIC_VECTOR(3 downto 0);
          s:      in  STD_LOGIC;
          y:      out STD_LOGIC_VECTOR(3 downto 0));
end;

architecture struttura of mux2 is
    component tristate
        port(a: in  STD_LOGIC_VECTOR(3 downto 0);
              en: in  STD_LOGIC;
              y: out STD_LOGIC_VECTOR(3 downto 0));
    end component;
    signal sneg: STD_LOGIC;
begin
    sneg <= not s;
    t0: tristate port map(d0, sneg, y);
    t1: tristate port map(d1, s, y);
end;

```

In VHDL, espressioni come `not s` non sono consentite nell'elenco di ingressi e uscite di un'istanza. Quindi, `sneg` deve essere definito come un segnale separato.

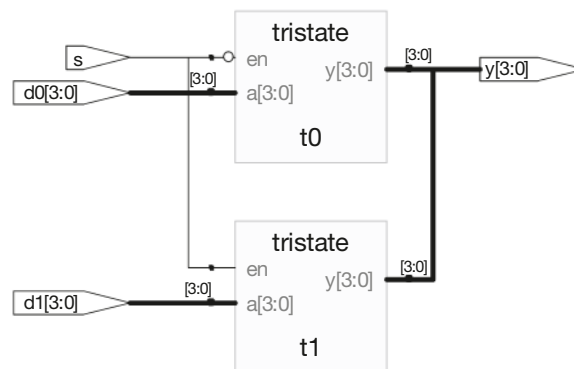


Figura 4.12 Sintesi del circuito `mux2`.

L'**Esempio HDL 4.16** mostra come i moduli possano accedere a parti di un bus. Un multiplexer 2:1 a 8 bit è realizzato con due dei multiplexer 2:1 a 4 bit già definiti, che lavorano rispettivamente sul *nibble* alto e su quello basso degli 8 bit.

In generale, i sistemi complessi vengono realizzati in modo gerarchico. Il sistema completo viene descritto strutturalmente istanziando le sue componenti principali, quindi ogni componente viene descritta strutturalmente in termini di blocchi che la costituiscono, e il processo viene ripetuto ricorsivamente finché i blocchi sono abbastanza semplici da poter essere descritti in modo comportamentale. È buona norma evitare (o comunque contenere al massimo) di mischiare descrizioni strutturali e comportamentali all'interno di un singolo modulo.

ESEMPIO HDL 4.16 ACCESSO A PARTI DI BUS

SystemVerilog	VHDL
<pre>module mux2_8(input logic [7:0] d0, d1, input logic s, output logic [7:0] y); mux2 muxlsb(d0[3:0], d1[3:0], s, y[3:0]); mux2 muxmsb(d0[7:4], d1[7:4], s, y[7:4]); endmodule</pre>	<pre>library IEEE; use IEEE.STD_LOGIC_1164.all; entity mux2_8 is port(d0, d1: in STD_LOGIC_VECTOR(7 downto 0); s: in STD_LOGIC; y: out STD_LOGIC_VECTOR(7 downto 0)); end; architecture struttura of mux2_8 is component mux2 port(d0, d1: in STD_LOGIC_VECTOR(3 downto 0); s: in STD_LOGIC; y: out STD_LOGIC_VECTOR(3 downto 0)); end component; begin muxlsb: mux2 port map(d0(3 downto 0), d1(3 downto 0), s, y(3 downto 0)); muxmsb: mux2 port map(d0(7 downto 4), d1(7 downto 4), s, y(7 downto 4)); end;</pre>

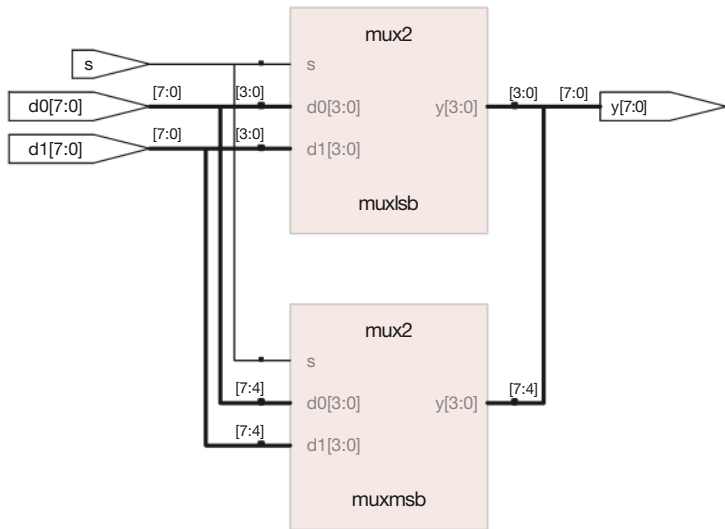


Figura 4.13 Sintesi del circuito mux2_8.

4.4 ■ LOGICA SEQUENZIALE

Gli strumenti di sintesi HDL riconoscono alcuni idiomi e li convertono in reti logiche sequenziali. Altri stili di scrittura HDL possono essere simulati correttamente ma in fase di sintesi dare luogo a reti con errori, a volte clamorosi a volte sottili. In questo paragrafo si presentano gli idiomi adatti a descrivere registri e latch.

4.4.1 Registri

La maggior parte dei moderni sistemi commerciali è realizzata con registri che usano flip-flop di tipo D sensibili ai fronti di salita (*positive edge-triggered*). L'**Esempio HDL 4.17** mostra l'idioma per questo flip-flop.

ESEMPIO HDL 4.17 REGISTRO**SystemVerilog**

```

module flop(input logic      clk,
            input logic [3:0] d,
            output logic [3:0] q);

    always_ff @(posedge clk)
        q <= d;
endmodule

```

In generale, un'istruzione `always` in SystemVerilog ha la forma:

```

always @(sensitivity list)
    istruzione

```

L'istruzione viene eseguita solo quando l'evento specificato nella sensitivity list si è verificato. In questo esempio, l'istruzione è `q <= d`, quindi il flip-flop copia `d` in `q` sul fronte positivo (di salita) del clock, in tutti gli altri casi tiene memoria del vecchio valore di `q`. La sensitivity list viene anche chiamata lista degli stimoli.

`<=` viene chiamato **assegnamento non bloccante**. Per il momento può essere considerato analogo al normale segno `=`: si tornerà sull'argomento con considerazioni più sottili nel paragrafo 4.5.4. Si noti l'uso di `<=` invece di `assign` all'interno di un'istruzione `always`.

Come si vedrà nei paragrafi successivi, un'istruzione `always` può essere usata per indicare flip-flop, latch o anche logica combinatoria, a seconda della sensitivity list e dell'istruzione stessa. A causa di questa flessibilità, è facile produrre inavvertitamente dell'hardware sbagliato. SystemVerilog introduce le istruzioni `always_ff`, `always_latch` e `always_comb` per ridurre il rischio di questo genere di errori comuni. `always_ff` funziona come `always` ma può essere usata solo per indicare flip-flop, e gli strumenti HDL generano un messaggio di allerta se si indica altro.

VHDL

```

library IEEE; use IEEE.STD_LOGIC_1164.all;

entity flop is
    port(clk: in  STD_LOGIC;
         d:  in  STD_LOGIC_VECTOR(3 downto 0);
         q:  out STD_LOGIC_VECTOR(3 downto 0));
end;

```

architecture sintesi of flop is
begin

```

    process(clk) begin
        if rising_edge(clk) then
            q <= d;
        end if;
    end process;
end;

```

Un process VHDL ha la forma:

```

process(sensitivity list) begin
    istruzione;
end process;

```

L'istruzione viene eseguita quando una qualsiasi delle variabili della sensitivity list cambia. Nell'esempio l'istruzione `if` controlla se il cambiamento è stato un fronte di salita di `clk`. Se sì, allora `q <= d` ovvero il flip-flop copia `d` in `q` sul fronte positivo (di salita) del clock, in tutti gli altri casi tiene memoria del vecchio valore di `q`.

Un idioma alternativo per il flip-flop in VHDL è il seguente:

```

process(clk) begin
    if clk'event and clk = '1' then
        q <= d;
    end if;
end process;

```

`rising_edge(clk)` è sinonimo di `clk'event and clk = '1'`.

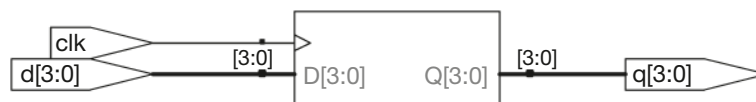


Figura 4.14 Sintesi del circuito flop.

Nelle istruzioni `always` in SystemVerilog e `process` in VHDL, ogni segnale mantiene il valore assunto in precedenza finché un evento nella *sensitivity list* (l'elenco di segnali da cui tale segnale dipende) non ne causa la modifica. Quindi queste istruzioni, assieme a un'opportuna sensitivity list, possono essere usate per descrivere le reti logiche sequenziali con memoria. Per esempio, un flip-flop include il solo segnale `clk` nella sua sensitivity list: quindi mantiene il vecchio valore di `q` fino al prossimo fronte di salita di `clk`, anche se nel frattempo `d` cambia.

Al contrario, le istruzioni di assegnamento continuo in SystemVerilog (`assign`) e le istruzioni di assegnamento concorrente in VHDL (`<=`) sono nuovamente valutate ogni volta che uno degli ingressi citati nella parte destra dell'istruzione cambia. Quindi, queste istruzioni necessariamente descrivono reti combinatorie.

4.4.2 Registri resettabili

Quando inizia la simulazione o quando un circuito viene acceso, l'uscita dei flip-flop e dei registri ha un valore sconosciuto, indicato con *x* in SystemVerilog e *u* in VHDL. È buona norma quindi usare registri resettabili, in modo

ESEMPIO HDL 4.18 REGISTRO RESETTABILE

SystemVerilog

```
module flopr(input logic clk,
            input logic reset,
            input logic [3:0] d,
            output logic [3:0] q);

    // reset asincrono
    always_ff @(posedge clk, posedge reset)
        if (reset) q <= 4'b0;
        else q <= d;
endmodule

module flopr(input logic clk,
            input logic reset,
            input logic [3:0] d,
            output logic [3:0] q);

    // reset sincrono
    always_ff @(posedge clk)
        if (reset) q <= 4'b0;
        else q <= d;
endmodule
```

Più segnali elencati nella sensitivity list di un'istruzione `always` sono separati da una virgola o dalla parola `or`. Si noti che l'indicazione `posedge reset` è presente nella sensitivity list del flip-flop resettabile in modo asincrono ma non in quella del flip-flop resettabile in modo sincrono. Quindi il flip-flop resettabile in modo asincrono reagisce immediatamente al verificarsi di un fronte di salita su `reset`, mentre il flip-flop resettabile in modo sincrono reagisce al `reset` solo al prossimo fronte di salita del clock.

Siccome i due moduli hanno lo stesso nome, `flopr`, si può includere uno solo dei due nel proprio progetto.

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity flopr is
    port(clk, reset: in STD_LOGIC;
         d: in STD_LOGIC_VECTOR(3 downto 0);
         q: out STD_LOGIC_VECTOR(3 downto 0));
end;
```

```
architecture asincrono of flopr is
begin
    process(clk, reset) begin
        if reset then
            q <= "0000";
        elsif rising_edge(clk) then
            q <= d;
        end if;
    end process;
end;
```

```
library IEEE; use IEEE.STD_LOGIC_1164.all;
```

```
entity flopr is
    port(clk, reset: in STD_LOGIC;
         d: in STD_LOGIC_VECTOR(3 downto 0);
         q: out STD_LOGIC_VECTOR(3 downto 0));
end;
```

```
architecture sincrono of flopr is
begin
    process(clk) begin
        if rising_edge(clk) then
            if reset then q <= "0000";
            else q <= d;
            end if;
        end if;
    end process;
end;
```

Più segnali elencati nella sensitivity list di un `process` sono separati da una virgola. Si noti che l'indicazione `posedge reset` è presente nella sensitivity list del flip-flop resettabile in modo asincrono ma non in quella del flip-flop resettabile in modo sincrono. Quindi il flip-flop resettabile in modo asincrono reagisce immediatamente al verificarsi di un fronte di salita su `reset`, mentre il flip-flop resettabile in modo sincrono reagisce al `reset` solo al prossimo fronte di salita del clock. Si ricordi inoltre che lo stato di un flip-flop è inizializzato a '*u*' all'inizio della simulazione VHDL.

Come detto in precedenza, il nome dell'architettura (`asincrono` o `sincrono`, in questo esempio) è ignorato dagli strumenti VHDL ma può essere di aiuto all'uomo nella lettura del codice.

Siccome entrambe le architetture descrivono la stessa entità `flopr`, si può includere una sola delle due nel proprio progetto.

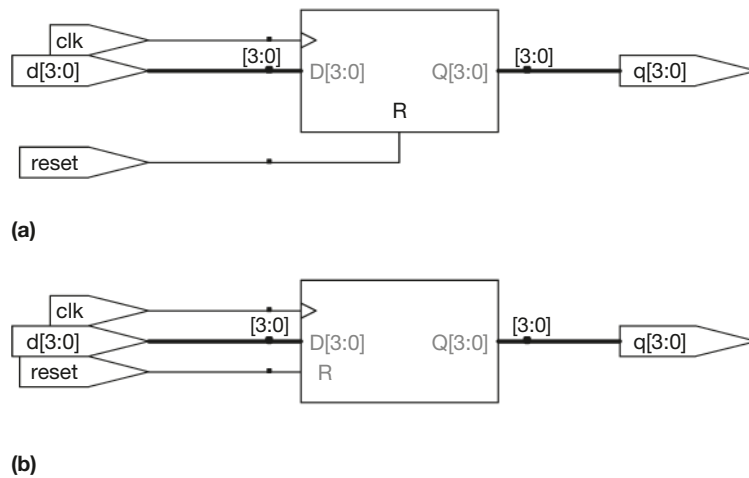


Figura 4.15 Sintesi del circuito *flopr* (a) con reset asincrono, (b) con reset sincrono.

tale che all'accensione sia possibile forzare il sistema in uno stato noto. Il segnale di reset può essere sia asincrono sia sincrono: il primo agisce immediatamente, il secondo azzerava le uscite solo al prossimo fronte di salita del clock. L'**Esempio HDL 4.18** mostra gli idiomi per flip-flop con reset asincrono e sincrono. Negli schemi circuitali non è facile distinguere fra i due tipi di reset, però negli schemi prodotti da Synplify Premier il segnale di reset asincrono è sempre posizionato nel flip-flop in basso, quello sincrono sul lato sinistro.

4.4.3 Registri con abilitazione

I registri con abilitazione (*enable*) reagiscono al clock solo se il segnale di abilitazione è attivo. L'**Esempio HDL 4.19** mostra un registro con abilitazione resettabile in modo asincrono, che mantiene il valore precedente se entrambi i segnali *reset* ed *en* sono FALSO.

ESEMPIO HDL 4.19 REGISTRO RESETTABILE CON ABILITAZIONE

SystemVerilog

```
module flopenr(input logic clk,
               input logic reset,
               input logic en,
               input logic [3:0] d,
               output logic [3:0] q);

    // reset asincrono
    always_ff @(posedge clk, posedge reset)
        if (reset) q <= 4'b0;
        else if (en) q <= d;
endmodule
```

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;
entity flopenr is
    port(clk,
          reset,
          en: in STD_LOGIC;
          d: in STD_LOGIC_VECTOR(3 downto 0);
          q: out STD_LOGIC_VECTOR(3 downto 0));
end;

architecture asincrono of flopenr is
    -- reset asincrono
    begin
        process(clk, reset) begin
            if reset then
                q <= "0000";
            elsif rising_edge(clk) then
                if en then
                    q <= d;
                end if;
            end if;
        end process;
    end;
```

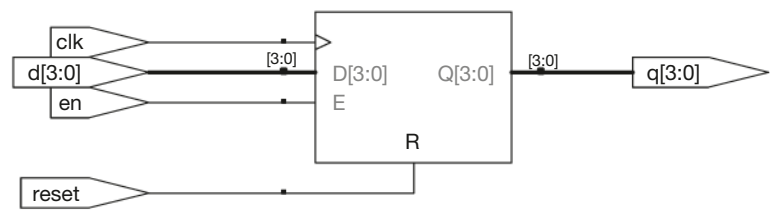


Figura 4.16 Sintesi del circuito flopenr.

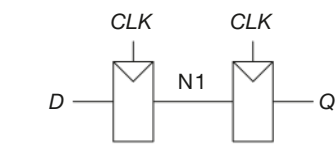


Figura 4.17 Circuito sincronizzatore.

4.4.4 Registri multipli

Una singola istruzione `always/process` può essere usata per descrivere più parti di hardware. Per esempio, si consideri il sincronizzatore del paragrafo 3.5.5, costituito da due flip-flop collegati in cascata, come mostrato nella **Figura 4.17**. L'**Esempio HDL 4.20** descrive il sincronizzatore. Sul fronte di salita di `clk`, `d` viene copiato in `n1`, e allo stesso tempo `n1` viene copiato in `q`.

ESEMPIO HDL 4.20 SINCRONIZZATORE

SystemVerilog

```
module sinc(input  logic clk,
             input  logic d,
             output logic q);
    logic n1;
    always_ff @(posedge clk)
    begin
        n1 <= d; // non bloccante
        q  <= n1; // non bloccante
    end
endmodule
```

Il costrutto `begin/end` è necessario perché ci sono più istruzioni all'interno dell'istruzione `always`. La cosa è analoga a quanto avviene in C o Java con le parentesi graffe `{}`. `begin/end` non era necessario nell'esempio `flopr` perché il costrutto `if/else` corrisponde a una singola istruzione.

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity sinc is
    port(clk: in  STD_LOGIC;
          d:  in  STD_LOGIC;
          q:  out STD_LOGIC);
end;

architecture buona of sinc is
    signal n1: STD_LOGIC;
begin
    process(clk) begin
        if rising_edge(clk) then
            n1 <= d;
            q  <= n1;
        end if;
    end process;
end;
```

`n1` deve essere dichiarato `signal` perché è un segnale interno usato nel modulo.

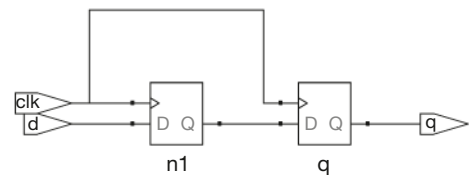


Figura 4.18 Sintesi del circuito sinc.

4.4.5 Latch

Nel paragrafo 3.2.2 si è visto come un latch D sia trasparente quando il clock è ALTO, permettendo ai dati di passare dall'ingresso all'uscita. Il latch diventa "opaco" quando il clock è BASSO, e mantiene lo stato raggiunto in precedenza. L'**Esempio HDL 4.21** mostra l'idioma per un latch D.

Non tutti gli strumenti di sintesi supportano bene i latch, quindi se non è davvero necessario e non si è sicuri circa tale supporto, meglio usare flip-flop attivi sul fronte del clock (*edge triggered*). Si deve fare inoltre attenzione a che il proprio progetto HDL non implichi un latch non voluto, cosa tutt'altro che rara se non si sta più che attenti. Molti strumenti di sintesi avvertono quando viene creato un latch: se non si voleva inserirlo, cercare l'errore nel proprio codice. Se poi si ha il dubbio se si voleva o meno inserire un latch, è molto probabile che si stia usando l'HDL come un linguaggio di programmazione, e i problemi sono ben più gravi.

4.5 ■ ANCORA LOGICA COMBINATORIA

Nel paragrafo 4.2 sono state usate istruzioni di assegnamento per descrivere la logica combinatoria da un punto di vista comportamentale. L'istruzione `always` in SystemVerilog e l'istruzione `process` in VHDL sono invece usate per descrivere reti sequenziali perché tengono memoria dello stato precedente quando non si deve assumere un nuovo stato. Tuttavia, le istruzioni `always/process` possono essere usate anche per descrivere logica combinatoria se la *sensitivity list* è scritta in modo tale da rispondere a cambiamenti in tutti gli ingressi e se il corpo del programma specifica il valore di uscita per ogni possi-

ESEMPIO HDL 4.21 LATCH D

SystemVerilog

```
module latch(input  logic      clk,
             input  logic [3:0] d,
             output logic [3:0] q);
    always_latch
        if (clk) q <= d;
endmodule
```

`always_latch` equivale ad `always @(clk,d)` ed è l'idioma da usare di preferenza in SystemVerilog per descrivere un latch. Viene valutato ogni volta che `clk` o `d` cambiano. Se `clk` è ALTO, `d` attraversa i latch e arriva a `q`, quindi questo codice descrive un latch sensibile al livello positivo. Altrimenti `q` mantiene il valore precedente. SystemVerilog può generare un avvertimento se un blocco `always_latch` non implica un latch.

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;
entity latch is
    port(clk: in  STD_LOGIC;
         d:   in  STD_LOGIC_VECTOR(3 downto 0);
         q:   out STD_LOGIC_VECTOR(3 downto 0));
end;
```

```
architecture sintesi of latch is
begin
    process(clk, d) begin
        if clk = '1' then
            q <= d;
        end if;
    end process;
end;
```

La *sensitivity list* contiene sia `clk` sia `d`, quindi `process` viene valutato ogni volta che `clk` o `d` cambiano. Se `clk` è ALTO, `d` attraversa i latch e arriva a `q`.

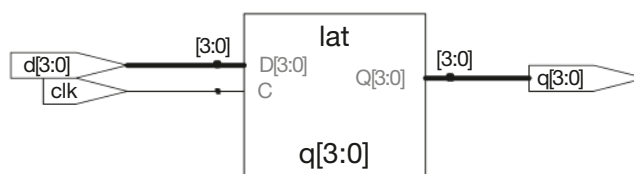


Figura 4.19 Sintesi del circuito latch.

ESEMPIO HDL 4.22 NEGATORE FACENDO USO DI `always/process`**SystemVerilog**

```
module neg(input  logic [3:0] a,
           output logic [3:0] y);
    always_comb
        y = ~a;
endmodule
```

`always_comb` rivaluta le istruzioni all'interno dell'istruzione `always` ogni volta che i segnali a destra di `<=` oppure `=` nell'istruzione `always` cambiano. In questo caso, equivale a `always @(a)` ma è migliore perché evita errori se i segnali nell'istruzione `always` vengono rinominati o aggiunti. Se il codice nel blocco `always` non è combinatorio, SystemVerilog genera una segnalazione. `always_comb` è anche equivalente a `always@(*)` ma la prima forma è preferita in SystemVerilog. Il simbolo `=` nell'istruzione `always` è chiamato assegnamento bloccante, rispetto all'assegnamento non bloccante `<=`. In SystemVerilog è buona norma usare assegnamenti bloccanti per la logica combinatoria e non bloccanti per quella sequenziale. L'argomento è ulteriormente discusso nel paragrafo 4.5.4.

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity neg is
    port(a: in  STD_LOGIC_VECTOR(3 downto 0);
          y: out STD_LOGIC_VECTOR(3 downto 0));
end;
```

```
architecture sintesi of neg is
begin
    process(all) begin
        y <= not a;
    end process;
end;
```

`process(all)` rivaluta le istruzioni all'interno dell'istruzione `process` ogni volta che i segnali nell'istruzione stessa cambiano. Equivale a `process(a)` ma è migliore perché evita errori se i segnali nell'istruzione `process` vengono rinominati o aggiunti. Le istruzioni `begin` ed `end process` sono necessarie in VHDL, anche se `process` contiene solo un assegnamento.

bile combinazione di ingresso. L'**Esempio HDL 4.22** usa le istruzioni `always/process` per descrivere una batteria di quattro negatori (la rete sintetizzata è riportata nella **Figura 4.3**).

Gli HDL supportano sia assegnamenti bloccanti sia non bloccanti all'interno di un'istruzione `always/process`. Un gruppo di assegnamenti bloccanti viene valutato nell'ordine in cui essi sono presenti nel codice, come ci si aspetterebbe in un qualsiasi linguaggio di programmazione. Un gruppo di assegnamenti non bloccanti viene valutato in modo concorrente: tutte le istruzioni sono valutate prima di modificare i segnali nella parte sinistra delle istruzioni stesse.

L'**Esempio HDL 4.23** definisce un sommatore completo usando i segnali intermedi `p` e `g` per calcolare `s` e `rou`. Produce lo stesso circuito della **Figura 4.8**, ma utilizza le istruzioni `always/process` invece delle istruzioni di assegnamento.

Questi due sono brutti esempi di uso delle istruzioni `always/process` per descrivere logica combinatoria perché richiedono più istruzioni di quelle necessarie utilizzando le istruzioni di assegnamento degli **Esempi HDL 4.2** e **4.7**. Le istruzioni `case` e `if` diventano convenienti per modellizzare logiche combinatorie più complesse. Queste istruzioni devono comparire all'interno di istruzioni `always/process` e sono esaminate in paragrafi successivi.

SystemVerilog

In un'istruzione `always` di SystemVerilog, `=` indica un assegnamento bloccante e `<=` un assegnamento non bloccante (detto anche assegnamento concorrente). Questi due tipi di assegnamento non vanno confusi con l'assegnamento continuo dell'istruzione `assign`. Le istruzioni `assign` devono essere utilizzate al di fuori delle istruzioni `always` e sono anch'esse valutate in modo concorrente.

VHDL

In un'istruzione `process` di VHDL, `:=` indica un assegnamento bloccante e `<=` un assegnamento non bloccante (detto anche assegnamento concorrente). Questo è il primo punto in cui viene introdotta la notazione `:=`. Gli assegnamenti non bloccanti si fanno a uscite e segnali, gli assegnamenti bloccanti a variabili dichiarate nelle istruzioni `process` (vedi l'Esempio HDL 4.23). `<=` può comparire anche al di fuori delle istruzioni `process`, e viene valutato in modo concorrente.

ESEMPIO HDL 4.23 SOMMATORE COMPLETO FACENDO USO DI `always/process`**SystemVerilog**

```

module sommatore(input  logic a, b, rin,
                  output logic s, rout);

  logic p, g;

  always_comb
  begin
    p = a ^ b;           // bloccante
    g = a & b;           // bloccante
    s = p ^ rin;         // bloccante
    rout = g | (p & rin); // bloccante
  end
endmodule

```

In questo caso, `always@(a, b, rin)` sarebbe stato equivalente ad `always_comb`, ma la seconda forma è migliore perché evita gli errori frequenti di dimenticanza di un segnale nella sensitivity list.

Per motivi che saranno discussi nel paragrafo 4.5.4, è meglio usare assegnamenti bloccanti per la logica combinatoria, come in questo esempio dove si calcola prima `p`, poi `g`, poi `s` e da ultimo `rout`.

VHDL

```

library IEEE; use IEEE.STD_LOGIC_1164.all;

entity neg is
  port(a: in  STD_LOGIC_VECTOR(3 downto 0);
        y: out STD_LOGIC_VECTOR(3 downto 0));
end;

architecture sintesi of neg is
begin
  process(all) begin
    y <= not a;
  end process;
end;

```

`process(all)` rivaluta le istruzioni all'interno dell'istruzione `process` ogni volta che i segnali nell'istruzione stessa cambiano. Equivale a `process(a)` ma è migliore perché evita errori se i segnali nell'istruzione `process` vengono rinominati o aggiunti.

Le istruzioni `begin` ed `end process` sono necessarie in VHDL, anche se `process` contiene solo un assegnamento.

4.5.1 Istruzione case

Un'applicazione migliore dell'istruzione `always/process` per logica combinatoria è il trascodificatore per display a sette segmenti che sfrutta l'istruzione `case` che deve apparire all'interno dell'istruzione `always/process`.

Come si può ricordare dall'**Esempio 2.10**, il progetto di grossi blocchi di logica combinatoria è noioso e a rischio di errori. Gli HDL offrono un notevole miglioramento, consentendo di specificare la funzione a un livello di astrazione più alto, e sintetizzando in modo automatico la funzione in termini di porte logiche. L'**Esempio HDL 4.24** usa l'istruzione `case` per descrivere un trascodificatore per display a sette segmenti sulla base della sua tabella delle verità. L'istruzione `case` svolge operazioni diverse a seconda del valore del suo ingresso: se tutte le possibili combinazioni d'ingresso sono definite, implica logica combinatoria, altrimenti implica logica sequenziale perché l'uscita mantiene il valore precedente nei casi non definiti.

Synplify Premier sintetizza il trascodificatore per display a sette segmenti in una memoria ROM (*Read Only Memory*) contenente le sette uscite per ciascuna delle sedici possibili configurazioni di ingresso. Le ROM sono discusse nel paragrafo 5.5.6.

Se non si fossero usate le clausole `default` od `others` nell'istruzione `case`, il trascodificatore avrebbe ricordato il valore precedente ogni volta che avesse ricevuto valori di ingresso compresi fra 10 e 15, un comportamento evidentemente strano per un circuito.

Anche i decodificatori o *decoder* sono normalmente descritti da istruzioni `case`, come l'**Esempio HDL 4.25** di un decoder 3:8.

4.5.2 Istruzione if

Le istruzioni `always/process` possono contenere anche istruzioni `if`. L'istruzione `if` può essere seguita da un'istruzione `else`. Se tutte le possibili combinazioni di ingresso sono gestite, l'istruzione implica logica combinatoria, altrimenti produce logica sequenziale (come il latch nel paragrafo 4.4.5).

ESEMPIO HDL 4.24 TRANSCODIFICATORE PER DISPLAY A SETTE SEGMENTI**SystemVerilog**

```

module sette_segmenti(input  logic [3:0] dati,
                     output logic [6:0] segmenti);

  always_comb
  case(dati)
    //          abc_defg
    0:      segmenti = 7'b111_1110;
    1:      segmenti = 7'b011_0000;
    2:      segmenti = 7'b110_1101;
    3:      segmenti = 7'b111_1001;
    4:      segmenti = 7'b011_0011;
    5:      segmenti = 7'b101_1011;
    6:      segmenti = 7'b101_1111;
    7:      segmenti = 7'b111_0000;
    8:      segmenti = 7'b111_1111;
    9:      segmenti = 7'b111_0011;
    default: segmenti = 7'b000_0000;
  endcase
endmodule

```

L'istruzione `case` verifica il valore di `dati`. Se `dati` vale 0, l'istruzione esegue l'azione specificata dopo i due punti, portando `segmenti` a 1111110. Analogamente, verifica gli altri valori dei dati fino a 9 (si noti l'uso della base 10 come base per i numeri in assenza di ulteriori specifiche).

La clausola `default` è un modo conveniente per definire le uscite in tutti i casi non elencati esplicitamente, garantendo in questo modo una logica combinatoria.

In SystemVerilog, l'istruzione `case` deve comparire all'interno di un'istruzione `always`.

VHDL

```

library IEEE; use IEEE.STD_LOGIC_1164.all;

entity sette_segmenti is
  port(dati:      in  STD_LOGIC_VECTOR(3 downto 0);
       segmenti: out STD_LOGIC_VECTOR(6 downto 0));
end;

architecture sintesi of sette_segmenti is
begin
  process(all) begin
    case dati is
      --          abcdefg
      when X"0" => segmenti <= "1111110";
      when X"1" => segmenti <= "0110000";
      when X"2" => segmenti <= "1101101";
      when X"3" => segmenti <= "1111001";
      when X"4" => segmenti <= "0110011";
      when X"5" => segmenti <= "1011011";
      when X"6" => segmenti <= "1011111";
      when X"7" => segmenti <= "1110000";
      when X"8" => segmenti <= "1111111";
      when X"9" => segmenti <= "1110011";
      when others => segmenti <= "0000000";
    end case;
  end process;
end;

```

L'istruzione `case` verifica il valore di `dati`. Se `dati` vale 0, l'istruzione esegue l'azione specificata dopo il `=>`, portando `segmenti` a 1111110. Analogamente, verifica gli altri valori dei dati fino a 9 (si noti l'uso di `X` per indicare numeri esadecimali). La clausola `others` è un modo conveniente per definire le uscite in tutti i casi non elencati esplicitamente, garantendo in questo modo una logica combinatoria.

A differenza di SystemVerilog, VHDL supporta le istruzioni di assegnamento condizionale di segnali (vedi l'Esempio HDL 4.6) che sono molto simili all'istruzione `case` ma che possono apparire al di fuori dei processi. Ci sono quindi meno ragioni di usare i processi per descrivere la logica combinatoria.



Figura 4.20 Sintesi del circuito `sette_segmenti`.

L'**Esempio HDL 4.26** usa l'istruzione `if` per descrivere un circuito a priorità, definito nel paragrafo 2.4. Si ricordi che un circuito a priorità forza a VERO l'uscita che corrisponde al bit di ingresso più significativo avente valore VERO.

4.5.3 Tabelle delle verità con indifferenze

Come esaminato nel paragrafo 2.7.3, le tabelle delle verità possono contenere indifferenze per consentire migliori ottimizzazioni logiche. L'**Esempio HDL 4.27** mostra come descrivere il circuito a priorità con indifferenze.

Synplify Premier sintetizza con un circuito leggermente diverso questo modulo, mostrato nella **Figura 4.23**, rispetto a quanto fatto con il precedente nella **Figura 4.22**. I due circuiti sono comunque equivalenti da un punto di vista logico.

4.5.4 Assegnamenti bloccanti e non bloccanti

Le linee guida spiegano quando e come usare ciascun tipo di istruzione di assegnamento. Se non si seguono queste linee guida, si può scrivere del codice che sembra funzionare in simulazione ma che viene sintetizzato in strutture hardware non corrette. La parte rimanente di questo paragrafo, opzionale, spiega i principi che stanno alla base delle suddette linee guida.

ESEMPIO HDL 4.25 DECODER 3:8

SystemVerilog

```
module decoder3_8(input  logic [2:0] a,
                 output logic [7:0] y);

  always_comb
  case(a)
    3'b000: y = 8'b00000001;
    3'b001: y = 8'b00000010;
    3'b010: y = 8'b00000100;
    3'b011: y = 8'b00001000;
    3'b100: y = 8'b00010000;
    3'b101: y = 8'b00100000;
    3'b110: y = 8'b01000000;
    3'b111: y = 8'b10000000;
    default: y = 8'bxxxxxxxx;
  endcase
endmodule
```

La clausola `default` non è strettamente necessaria per la sintesi logica in questo caso, perché tutte le possibili combinazioni di ingresso sono definite, ma è meglio usarla per la simulazione in caso uno degli ingressi assuma valore `x` o `z`.

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity decoder3_8 is
  port(a: in  STD_LOGIC_VECTOR(2 downto 0);
       y: out STD_LOGIC_VECTOR(7 downto 0));
end;

architecture sintesi of decoder3_8 is
begin
  process(all) begin
    case a is
      when "000" => y<= "00000001";
      when "001" => y<= "00000010";
      when "010" => y<= "00000100";
      when "011" => y<= "00001000";
      when "100" => y<= "00010000";
      when "101" => y<= "00100000";
      when "110" => y<= "01000000";
      when "111" => y<= "10000000";
      when others => y<= "XXXXXXXX";
    end case;
  end process;
end;
```

La clausola `others` non è strettamente necessaria per la sintesi logica in questo caso, perché tutte le possibili combinazioni di ingresso sono definite, ma è meglio usarla per la simulazione in caso uno degli ingressi assuma valore `x`, `z` o `u`.

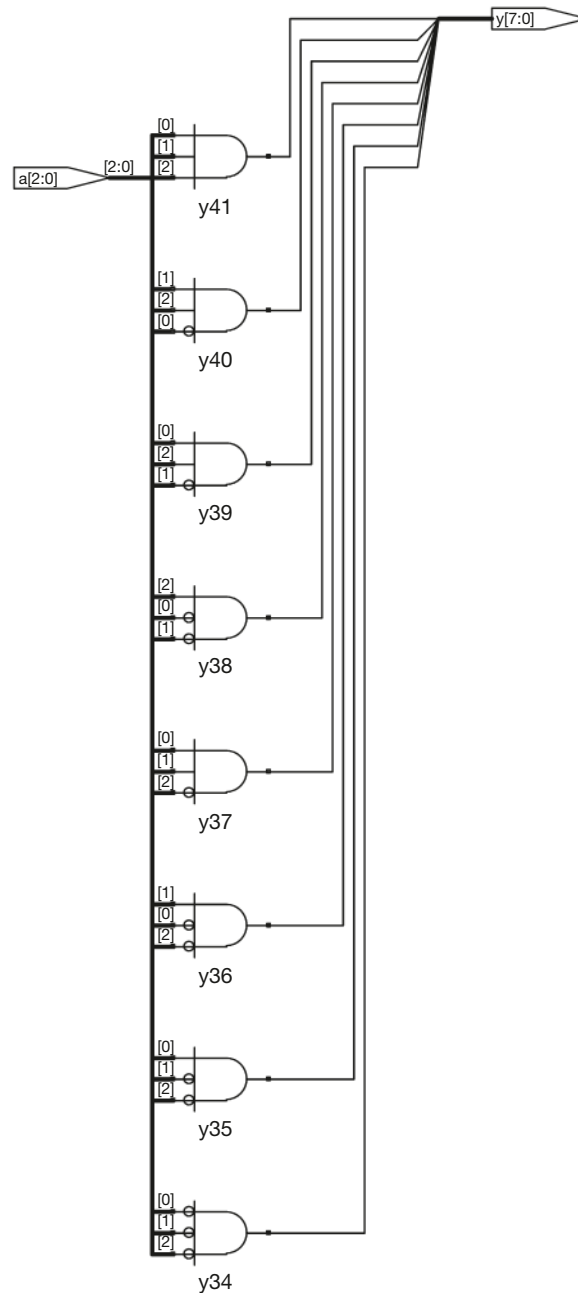


Figura 4.21 Sintesi del circuito `decoder3_8`.

Logica combinatoria*

Il sommatore completo dell'**Esempio HDL 4.23** è modellizzato correttamente usando assegnamenti bloccanti. Questo paragrafo analizza come opera il sommatore e come opererebbe se gli assegnamenti fossero non bloccanti.

Si assuma che `a`, `b` e `rin` siano inizialmente a 0. `p`, `g`, `s` e `rout` sono quindi pure a 0. A un certo istante, `a` diventa 1, attivando l'istruzione `always/process`. I quattro assegnamenti bloccanti vengono valutati nell'ordine qui mostrato. (Nel codice VHDL, `s` e `rout` sono assegnati in modo concorrente.) Si

ESEMPIO HDL 4.26 CIRCUITO A PRIORITÀ**SystemVerilog**

```

module circprio(input  logic [3:0] a,
                output logic [3:0] y);

  always_comb
    if (a[3])    y = 4'b1000;
    else if (a[2]) y = 4'b0100;
    else if (a[1]) y = 4'b0010;
    else if (a[0]) y = 4'b0001;
    else          y = 4'b0000;
endmodule

```

In SystemVerilog, l'istruzione `if` deve comparire all'interno di un'istruzione `always`.

VHDL

```

library IEEE; use IEEE.STD_LOGIC_1164.all;

entity circprio is
  port(a: in  STD_LOGIC_VECTOR(3 downto 0);
        y: out STD_LOGIC_VECTOR(3 downto 0));
end;

architecture sintesi of circprio is
begin
  process(all) begin
    if a(3) then y <= "1000";
    elsif a(2) then y <= "0100";
    elsif a(1) then y <= "0010";
    elsif a(0) then y <= "0001";
    else          y <= "0000";
    end if;
  end process;
end;

```

A differenza di SystemVerilog, VHDL supporta le istruzioni di assegnamento condizionale di segnali (*vedi* l'Esempio HDL 4.6) che sono molto simili all'istruzione `if` ma che possono apparire al di fuori dei processi. Ci sono quindi meno ragioni di usare i processi per descrivere logica combinatoria.

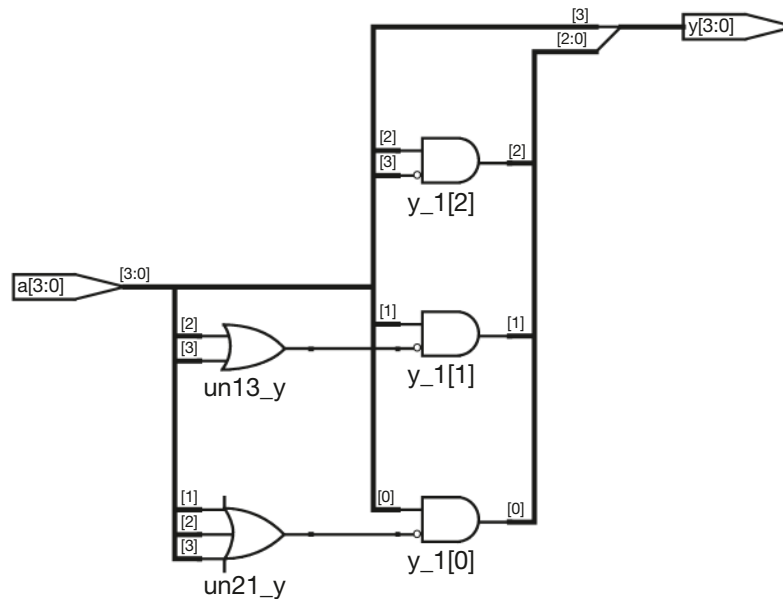


Figura 4.22 Sintesi del circuito `circprio`.

ESEMPIO HDL 4.27 CIRCUITO A PRIORITÀ CON INDIFFERENZE**SystemVerilog**

```

module circprio_indiff(input  logic [3:0] a,
                      output logic [3:0] y);

  always_comb
  casez(a)
    4'b1??? : y = 4'b1000;
    4'b01?? : y = 4'b0100;
    4'b001? : y = 4'b0010;
    4'b0001 : y = 4'b0001;
    default : y = 4'b0000;
  endcase
endmodule

```

L'istruzione `casez` funziona come l'istruzione `case` ma riconosce anche il simbolo `?` come indifferenza.

VHDL

```

library IEEE; use IEEE.STD_LOGIC_1164.all;

entity circprio_indiff is
  port(a: in  STD_LOGIC_VECTOR(3 downto 0);
       y: out STD_LOGIC_VECTOR(3 downto 0));
end;

architecture sintesi of circprio_indiff is
begin
  process(all) begin
    case? a is
      when "1---" => y <= "1000";
      when "01--" => y <= "0100";
      when "001-" => y <= "0010";
      when "0001" => y <= "0001";
      when others => y <= "0000";
    end case?;
  end process;
end;

```

L'istruzione `case?` funziona come l'istruzione `case` ma riconosce anche il simbolo `-` come indifferenza.

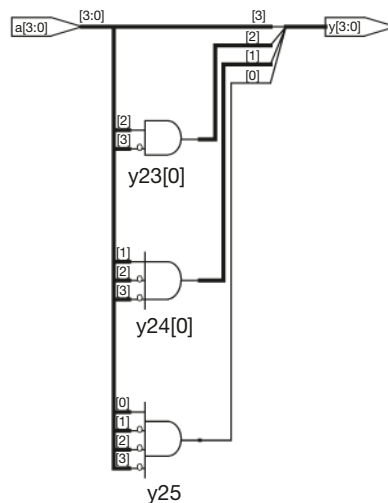


Figura 4.23 Sintesi del circuito `circprio_indiff`.

noti che `p` e `g` assumono il nuovo valore prima che `s` e `rou` siano calcolati, per via degli assegnamenti bloccanti. Questo è importante perché `s` e `rou` devono essere calcolati in base ai nuovi valori di `p` e `g`.

1. $p \leftarrow 1 \oplus 0 = 1$
2. $g \leftarrow 1 \cdot 0 = 0$
3. $s \leftarrow 1 \oplus 0 = 1$
4. $rou \leftarrow 0 + 1 \cdot 0 = 0$

Al contrario, l'**Esempio HDL 4.28** mostra l'uso di assegnamenti non bloccanti.

LINEE GUIDA PER GLI ASSEGNAMENTI BLOCCANTI E NON BLOCCANTI**SystemVerilog**

1. Usare `always_ff @(posedge clk)` e assegnamenti non bloccanti per modellizzare logica sequenziale sincrona.

```
always_ff @(posedge clk)
begin
    n1 <= d; // non bloccante
    q <= n1; // non bloccante
end
```

2. Usare assegnamenti continui per modellizzare logica combinatoria semplice.

```
assign y = s ? d1 : d0;
```

3. Usare `always_comb` e assegnamenti bloccanti per modellizzare logica combinatoria più complessa, dove l'istruzione `always` è molto utile.

```
always_comb
begin
    p = a ^ b; // bloccante
    g = a & b; // bloccante
    s = p ^ rin;
    rout = g | (p & rin);
end
```

4. Non fare più assegnamenti allo stesso segnale in più di un'istruzione `always` o più di un'istruzione di assegnamento continuo.

VHDL

1. Usare `process(clk)` e assegnamenti non bloccanti per modellizzare logica sequenziale sincrona.

```
process(clk) begin
    if rising_edge(clk) then
        n1 <= d; -- non bloccante
        q <= n1; -- non bloccante
    end if;
end process;
```

2. Usare assegnamenti concorrenti al di fuori di istruzioni `process` per modellizzare la logica combinatoria semplice.

```
y <= d0 when s = '0' else d1;
```

3. Usare `process(all)` per modellizzare logica combinatoria più complessa, dove l'istruzione `always` è molto utile. Usare assegnamenti bloccanti per le variabili interne.

```
process(all)
    variable p, g: STD_LOGIC;
begin
    p := a xor b; -- bloccante
    g := a and b; -- bloccante
    s <= p xor rin;
    rout <= g or (p and rin);
end process;
```

4. Non fare più assegnamenti alla stessa variabile in più di un'istruzione `process` o più di un'istruzione di assegnamento concorrente.

ESEMPIO HDL 4.28 SOMMATORE COMPLETO CON USO DI ASSEGNAMENTI NON BLOCCANTI**SystemVerilog**

```
// assegnamenti non bloccanti (sconsigliati)
module sommatore(input logic a, b, rin,
                 output logic s, rout);

    logic p, g;

    always_comb
    begin
        p <= a ^ b; // non bloccante
        g <= a & b; // non bloccante
        s <= p ^ rin;
        rout <= g | (p & rin);
    end
endmodule
```

VHDL

```
-- assegnamenti non bloccanti (sconsigliati)
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity sommatore is
    port(a, b, rin: in STD_LOGIC;
         s, rout: out STD_LOGIC);
end;

architecture non_bloccante of sommatore is
    signal p, g: STD_LOGIC;
begin
    process(all) begin
        p <= a xor b; -- non bloccante
        g <= a and b; -- non bloccante
        s <= p xor rin;
        rout <= g or (p and rin);
    end process;
end;
```

Dal momento che `p` e `g` compaiono a sinistra di assegnamenti non bloccanti in un'istruzione `process`, devono essere dichiarate di tipo `signal` invece di `variable`. La dichiarazione di `signal` compare prima del `begin` in `architecture`, non in `process`.

Si consideri di nuovo la situazione in cui *a* passa da 0 a 1 mentre *b* e *rin* sono a 0. I quattro assegnamenti non bloccanti vengono calcolati in modo concorrente, quindi simultaneamente:

$$p \leftarrow 1 \oplus 0 = 1 \quad g \leftarrow 1 \cdot 0 = 0 \quad s \leftarrow 0 \oplus 0 = 0 \quad rout \leftarrow 0 + 0 \cdot 0 = 0$$

Si noti che *s* viene calcolato simultaneamente a *p* e quindi usa il vecchio valore di *p* invece del nuovo. Quindi *s* rimane a 0 invece di diventare 1. Però *p* da 0 passa a 1, e questo attiva una seconda volta l'istruzione *always/process* con il seguente risultato:

$$p \leftarrow 1 \oplus 0 = 1 \quad g \leftarrow 1 \cdot 0 = 0 \quad s \leftarrow 1 \oplus 0 = 1 \quad rout \leftarrow 0 + 1 \cdot 0 = 0$$

Questa volta *p* vale già 1, quindi *s* passa correttamente a 1. Gli assegnamenti non bloccanti raggiungono infine il risultato corretto, ma l'istruzione *always/process* viene valutata due volte, rallentando la simulazione, anche se la sintesi hardware è la stessa.

SystemVerilog	VHDL
Se la sensitivity list dell'istruzione <i>always</i> nell'Esempio HDL 4.28 fosse stata scritta come <i>always @(a, b, rin)</i> invece di <i>always_comb</i> , allora l'istruzione non sarebbe stata ricalcolata in caso di cambiamento di <i>p</i> o <i>g</i> , e <i>s</i> sarebbe rimasta al valore errato 0 invece di 1.	Se la sensitivity list dell'istruzione <i>process</i> nell'Esempio HDL 4.28 fosse stata scritta come <i>process(a, b, rin)</i> invece di <i>process(all)</i> , allora l'istruzione non sarebbe stata ricalcolata in caso di cambiamento di <i>p</i> o <i>g</i> , e <i>s</i> sarebbe rimasta al valore errato 0 invece di 1.

Un altro aspetto negativo degli assegnamenti non bloccanti nella modellizzazione di logica combinatoria è il fatto che gli HDL producono risultati sbagliati se ci si dimentica di includere le variabili intermedie nella sensitivity list.

Anche peggio, alcuni strumenti di sintesi producono circuiti corretti anche quando una sensitivity list sbagliata causa errori di simulazione. Questo porta a una mancata corrispondenza tra risultati della simulazione ed effettivo comportamento del circuito sintetizzato.

Logica sequenziale*

Il sincronizzatore dell'**Esempio HDL 4.20** è modellizzato correttamente usando assegnamenti non bloccanti. Al fronte di salita del clock, *d* viene copiato in *n1* nello stesso tempo in cui *n1* viene copiato in *q*, quindi il codice descrive correttamente il comportamento di due registri. Si supponga per esempio che inizialmente *d* = 0, *n1* = 1 e *q* = 0. Al fronte di salita del clock, i seguenti due assegnamenti avvengono in modo concorrente, quindi dopo il fronte del clock *n1* = 0 e *q* = 1.

$$n1 \leftarrow d = 0 \quad q \leftarrow n1 = 1$$

L'**Esempio HDL 4.29** cerca di descrivere lo stesso modulo usando assegnamenti bloccanti. Al fronte di salita di *clk*, *d* viene copiato in *n1*. Poi il nuovo valore di *n1* viene copiato in *q*, producendo il risultato scorretto di *d* che compare sia in *n1* sia in *q*. Gli assegnamenti vengono eseguiti uno dopo l'altro, quindi dopo il fronte del clock si ottiene *q* = *n1* = 0.

- 1. *n1* ← *d* = 0
- 2. *q* ← *n1* = 0

Dal momento che *n1* non è visibile all'esterno del modulo e non influenza il comportamento di *q*, lo strumento di sintesi ottimizza il circuito eliminandolo completamente come mostrato nella **Figura 4.24**.

ESEMPIO HDL 4.29 SINCRONIZZATORE CON USO DI ASSEGNAMENTI BLOCCANTI**SystemVerilog**

```
// implementazione sbagliata di un sincronizzatore
// utilizzando assegnamenti bloccanti

module bruttosinc(input  logic clk,
                  input  logic d,
                  output logic q);

    logic n1;

    always_ff @(posedge clk)
    begin
        n1 = d; // bloccante
        q = n1; // bloccante
    end
endmodule
```

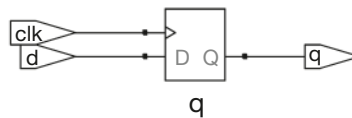
VHDL

```
-- implementazione sbagliata di un sincronizzatore
-- utilizzando assegnamenti bloccanti

library IEEE; use IEEE.STD_LOGIC_1164.all;

entity bruttosinc is
    port(clk: in  STD_LOGIC;
          d:  in  STD_LOGIC;
          q:  out STD_LOGIC);
end;

architecture brutta of bruttosinc is
begin
    process(clk)
        variable n1: STD_LOGIC;
    begin
        if rising_edge(clk) then
            n1 := d; -- bloccante
            q <= n1;
        end if;
    end process;
end;
```

**Figura 4.24** Sintesi del circuito `bruttosinc`.

La morale di questa discussione è usare sempre assegnamenti non bloccanti nelle istruzioni `always/process` per modellizzare la logica sequenziale. Con un po' di furbizia, per esempio invertendo l'ordine degli assegnamenti, si sarebbe potuto ottenere il funzionamento corretto anche con gli assegnamenti bloccanti, che però non introducono alcun vantaggio mentre creano pericolo di comportamenti inattesi. E, in certi casi, i circuiti sequenziali non funzionerebbero comunque, indipendentemente dall'ordine degli assegnamenti.

4.6 ■ MACCHINE A STATI FINITI

Una macchina a stati finiti (FSM, *Finite State Machine*) consiste di un registro di stato e di due blocchi di logica combinatoria per calcolare lo stato prossimo e le uscite a partire dallo stato presente e dagli ingressi, come già visto nella **Figura 3.22**. Le descrizioni HDL delle macchine a stati finiti sono quindi divise in tre parti per modellizzare il registro di stato, la logica di stato prossimo e la logica di uscita.

L'**Esempio HDL 4.30** descrive la FSM divisore-per-3 del paragrafo 3.4.2. È previsto un segnale di reset asincrono per inizializzare la FSM. Il registro di stato usa normali flip-flop. Le logiche di stato prossimo e di uscita sono combinatorie.

Lo strumento di sintesi Synplify Premier produce solo uno schema del blocco e un diagramma degli stati per le macchine a stati finiti: non mostra la struttura interna in termini di porte logiche né i valori di ingressi e uscite

ESEMPIO HDL 4.30 MACCHINA A STATI FINITI DIVISORE-PER-3**SystemVerilog**

```

module FSM_dividiper3(input  logic clk,
                     input  logic reset,
                     output logic y);
    typedef enum logic [1:0] {S0, S1, S2} tipostato;
    tipostato stato, statopross;

    // registro di stato
    always_ff @(posedge clk, posedge reset)
        if (reset) stato <= S0;
        else      stato <= statopross;

    // logica di stato prossimo
    always_comb
        case (stato)
            S0: statopross = S1;
            S1: statopross = S2;
            S2: statopross = S0;
            default: statopross = S0;
        endcase

    // logica di uscita
    assign y = (stato == S0);
endmodule

```

L'istruzione `typedef` definisce `tipostato` come valore di tipo `logic` a due bit con tre possibilità: S0, S1 o S2. `stato` e `statopross` sono segnali di tipo `tipostato`.

Le codifiche enumerative per default seguono l'ordine numerico, quindi S0 = 00, S1 = 01 e S2 = 10. L'utente può specificare valori diversi, ma lo strumento di sintesi prende comunque tali valori come suggerimento, non come requisito imprescindibile. Questo esempio codifica gli stati a singolo 1, cioè con tre bit di cui uno solo a 1:

```

typedef enum logic [2:0] {S0=3'b001, S1=3'b010, S2=3'b100} tipostato;

```

Si noti l'uso dell'istruzione `case` per definire la tabella delle transizioni di stato. Dal momento che la logica di stato prossimo deve essere combinatoria, è necessaria la clausola `default` anche se lo stato 2'b11 non sarà mai raggiunto.

L'uscita, `y`, vale 1 quando lo stato è S0. Il confronto di uguaglianza `a == b` vale 1 se `a` è uguale a `b` e 0 negli altri casi. Il confronto di disuguaglianza `a != b` fa l'opposto, cioè vale 1 se `a` non è uguale a `b`.

VHDL

```

library IEEE; use IEEE.STD_LOGIC_1164.all;

entity FSM_dividiper3 is
    port(clk, reset: in  STD_LOGIC;
         y:      out STD_LOGIC);
end;

architecture sintesi of FSM_dividiper3 is
    type tipostato is (S0, S1, S2);
    signal stato, statopross: tipostato;
begin
    -- registro di stato
    process(clk, reset) begin
        if reset then stato <= S0;
        elsif rising_edge(clk) then
            stato <= statopross;
        end if;
    end process;

    -- logica di stato prossimo
    statopross <= S1 when stato = S0 else
                 S2 when stato = S1 else
                 S0;

    -- logica di uscita
    y <= '1' when stato = S0 else '0';
end;

```

Questo esempio definisce un nuovo tipo di dato **enumerativo**, `tipostato`, con tre possibilità: S0, S1 o S2. `stato` e `statopross` sono segnali di tipo `tipostato`. Se si usa l'enumerazione invece di scegliere esplicitamente la codifica degli stati, VHDL evita che il sintetizzatore analizzi le varie possibilità per scegliere la migliore.

L'uscita, `y`, vale 1 quando lo stato è S0. Il confronto di disuguaglianza usa la notazione `/=`, quindi per avere 1 quando lo stato è qualsiasi tranne S0 il confronto va scritto `stato /= S0`.

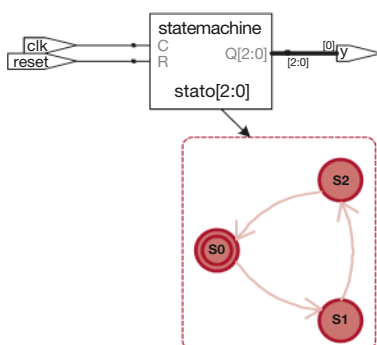


Figura 4.25
Sintesi del circuito `FSM_dividiper3`.

associati agli archi e agli stati del diagramma. Bisogna quindi fare attenzione a specificare correttamente la FSM nel codice HDL. Il diagramma degli stati nella **Figura 4.25** per la FSM divisore-per-3 è analogo al diagramma della **Figura 3.28(b)**. Il doppio cerchio indica che S0 è lo stato di reset. Le realizzazioni a livello di porte logiche della FSM divisore-per-3 sono state presentate nel paragrafo 3.4.2.

Si noti che gli stati sono denominati usando il tipo di dati enumerativo invece dei valori binari. Questo rende il codice più leggibile e facile da modificare.

Volendo avere l'uscita al valore ALTO negli stati S0 e S1, la logica di uscita andrebbe modificata in questo modo.

SystemVerilog

```
// logica di uscita
assign y = (stato==S0 | stato==S1);
```

VHDL

```
-- logica di uscita
y <= '1' when (stato = S0 or stato = S1) else '0';
```

I prossimi due esempi descrivono la lumaca riconosctrice di sequenza del paragrafo 3.4.3. Il codice mostra come usare le istruzioni `case` e `if` per gestire lo stato prossimo e l'uscita che dipendono dagli ingressi e dallo stato presente. Sono mostrati sia il modulo alla Moore sia quello alla Mealy. Nella macchina alla Moore (**Esempio HDL 4.31**) l'uscita dipende solo dal-

Si noti che lo strumento di sintesi usa una codifica a 3 bit (`Q[2:0]`) invece della codifica a 2 bit suggerita nel codice di SystemVerilog.

ESEMPIO HDL 4.31 RICONOSCITORE DI SEQUENZE COME MACCHINA ALLA MOORE**SystemVerilog**

```
module sequenzeMoore(input  logic clk,
                     input  logic reset,
                     input  logic a,
                     output logic y);
    typedef enum logic [1:0] {S0, S1, S2} tipostato;
    tipostato stato, statopross;

    // registro di stato
    always_ff @(posedge clk, posedge reset)
        if (reset) stato <= S0;
        else stato <= statopross;

    // logica di stato prossimo
    always_comb
        case (state)
            S0: if (a) statopross = S0;
                else statopross = S1;
            S1: if (a) statopross = S2;
                else statopross = S1;
            S2: if (a) statopross = S0;
                else statopross = S1;
            default: statopross = S0;
        endcase

    // logica di uscita
    assign y = (stato==S2);
endmodule
```

Si noti l'uso degli assegnamenti non bloccanti (`<=`) nel registro di stato per descrivere logica sequenziale, e degli assegnamenti bloccanti (`=`) nelle logiche di stato prossimo e di uscita per descrivere logica combinatoria.

VHDL

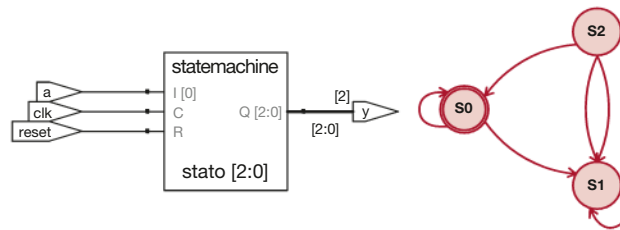
```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity sequenzeMoore is
    port(clk, reset: in  STD_LOGIC;
          a:          in  STD_LOGIC;
          y:          out STD_LOGIC);
end;

architecture sintesi of sequenzeMoore is
    type tipostato is (S0, S1, S2);
    signal stato, statopross: tipostato;
begin
    -- registro di stato
    process(clk, reset) begin
        if reset then stato <= S0;
        elsif rising_edge(clk) then stato <= statopross;
        end if;
    end process;

    -- logica di stato prossimo
    process(all) begin
        case stato is
            when S0 =>
                if a then statopross <= S0;
                else      statopross <= S1;
                end if;
            when S1 =>
                if a then statopross <= S2;
                else      statopross <= S1;
                end if;
            when S2 =>
                if a then statopross <= S0;
                else      statopross <= S1;
                end if;
            when others =>
                statopross <= S0;
        end case;
    end process;

    -- logica di uscita
    y <= '1' when stato = S2 else '0';
end;
```

**Figura 4.26** Sintesi del circuito `sequenzeMoore`.

lo stato presente, mentre nella macchina alla Mealy (**Esempio HDL 4.32**) l'uscita dipende sia dallo stato presente sia dagli ingressi.

ESEMPIO HDL 4.32 RICONOSCITORE DI SEQUENZE COME MACCHINA ALLA MEALY
SystemVerilog

```
module sequenzeMealy(input  logic clk,
                    input  logic reset,
                    input  logic a,
                    output logic y);

    typedef enum logic {S0, S1} tipostato;
    tipostato stato, statopross;

    // registro di stato
    always_ff @(posedge clk, posedge reset)
        if (reset) stato <= S0;
        else      stato <= statopross;

    // logica di stato prossimo
    always_comb
        case (stato)
            S0: if (a) statopross = S0;
                else statopross = S1;
            S1: if (a) statopross = S0;
                else statopross = S1;
            default: statopross = S0;
        endcase

    // logica di uscita
    assign y = (a & stato==S1);
endmodule
```

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity sequenzeMealy is
    port(clk, reset: in  STD_LOGIC;
          a:         in  STD_LOGIC;
          y:         out STD_LOGIC);
end;

architecture sintesi of sequenzeMealy is
    type tipostato is (S0, S1);
    signal stato, statopross: tipostato;
begin
    -- registro di stato
    process(clk, reset) begin
        if reset then          stato <= S0;
        elsif rising_edge(clk) then stato <= statopross;
        end if;
    end process;

    -- logica di stato prossimo
    process(all) begin
        case stato is
            when S0 =>
                if a then      statopross <= S0;
                else           statopross <= S1;
                end if;
            when S1 =>
                if a then      statopross <= S0;
                else           statopross <= S1;
                end if;
            when others => statopross <= S0;
        end case;
    end process;

    -- logica di uscita
    y <= '1' when (a = '1' and stato = S1) else '0';
end;
```

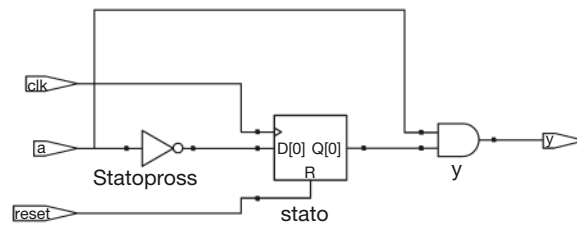



Figura 4.27 Sintesi del circuito *sequenzeMealy*.

4.7 ■ TIPI DI DATI*

Questo paragrafo approfondisce alcuni aspetti di dettaglio relativi a SystemVerilog e VHDL.

4.7.1 SystemVerilog

Prima di SystemVerilog, Verilog usava principalmente due tipi di dati: *reg* e *wire*. A dispetto del nome, un segnale *reg* può anche non essere associato a un registro. Questo era fonte di confusione per chi doveva imparare il linguaggio. SystemVerilog ha introdotto il tipo *logic* per eliminare questa confusione, quindi in questo testo si dà enfasi a tale tipo di dati. In questo paragrafo si parla invece di *reg* e *wire* per coloro che devono lavorare con il vecchio codice Verilog.

In Verilog, se un segnale compare a sinistra di un assegnamento `<=` o = in un blocco `always`, deve essere dichiarato come *reg*. Altrimenti deve essere dichiarato come *wire*. Quindi un segnale di tipo *reg* può essere l'uscita di un flip-flop, di un latch o di una rete combinatoria a seconda della sensitivity list e delle istruzioni del blocco `always`.

Gli ingressi e le uscite delle porte logiche sono per default di tipo *wire* a meno che siano dichiarati esplicitamente di tipo *reg*. L'esempio seguente mostra come un flip-flop viene descritto convenzionalmente in Verilog. Si noti che *clk* e *d* sono per default di tipo *wire*, mentre *q* è esplicitamente dichiarato di tipo *reg* perché compare a sinistra di `<=` nel blocco `always`.

```
module flop(input          clk,
            input    [3:0] d,
            output reg [3:0] q);
    always @(posedge clk)
        q <= d;
endmodule
```

SystemVerilog introduce il tipo *logic* come sinonimo di *reg* per evitare di confondere gli utenti circa il fatto che si parli effettivamente di un flip-flop. Inoltre, SystemVerilog rilassa le regole sulle istruzioni `assign` e sulle istanziazioni gerarchiche delle porte in modo che *logic* possa essere usato anche al di fuori di blocchi `always`, dove si sarebbe dovuto usare *wire*. Quindi praticamente tutti i segnali SystemVerilog possono essere di tipo *logic*. L'eccezione è che segnali con molteplici circuiti pilota (per es. un bus tristate) devono essere dichiarati come reti (*net*), come descritto nell'**Esempio HDL 4.10**. Questa regola consente a SystemVerilog di generare un messaggio di errore invece di un valore *x* quando un segnale *logic* viene erroneamente connesso a molteplici circuiti pilota.

Il tipo di rete più comune è chiamata `wire` o `tri`. I due tipi sono sinonimi, ma convenzionalmente `wire` è usato se è presente un solo circuito pilota e `tri` se ne sono presenti diversi. Quindi il tipo `wire` è obsoleto in SystemVerilog perché per segnali con singolo pilota si preferisce `logic`.

Quando una rete `tri` viene forzata a un singolo valore da uno o più circuiti pilota, assume tale valore. Se nessun pilota è attivo rimane fluttuante (`z`). Se viene forzata a valori diversi (`0`, `1` o `x`) da diversi piloti è in situazione di conflitto (`x`).

Ci sono altri tipi di rete che gestiscono diversamente le situazioni di mancanza di piloti attivi o di presenza di più piloti simultaneamente attivi. Sono usate raramente, ma si possono inserire dove normalmente compare una rete di tipo `tri` (cioè tipicamente per segnali con molti piloti). Tutte queste reti sono descritte nella **Tabella 4.7**.

4.7.2 VHDL

A differenza di SystemVerilog, VHDL impone un rigoroso sistema di tipizzazione dei dati, che aiuta l'utente a evitare alcuni errori ma che può risultare in alcuni casi un po' bizzarro.

Nonostante sia fondamentale, il tipo `STD_LOGIC` non è costruito in VHDL, ma fa parte della libreria `IEEE.STD_LOGIC_1164`. Quindi ogni file deve contenere l'istruzione di riferimento alla libreria presente negli esempi precedenti.

Inoltre, `IEEE.STD_LOGIC_1164` manca di alcune operazioni di base come somma, confronto, traslazione e conversione a interi per dati di tipo `STD_LOGIC_VECTOR`. Tali operazioni sono state finalmente aggiunte allo standard VHDL 2008 nella libreria `IEEE.NUMERIC_STD_UNSIGNED`.

VHDL ha anche un tipo `BOOLEAN` con due possibili valori: `true` e `false`. Valori di tipo `BOOLEAN` sono il risultato di confronti (come il confronto di uguaglianza `s = '0'`) e sono usati in istruzioni condizionali come `when` e `if`. Nonostante la tentazione di considerare un `BOOLEAN true` equivalente a un `STD_LOGIC '1'` e un `BOOLEAN false` equivalente a un `STD_LOGIC '0'`, questi due tipi di dati non erano intercambiabili fino alla versione VHDL 2008. Per esempio, in vecchie versioni di VHDL, si sarebbe dovuto scrivere

```
y <= d1 when (s = '1') else d0;
```

mentre in VHDL 2008 l'istruzione `when` converte automaticamente `s` da `STD_LOGIC` a `BOOLEAN` consentendo di scrivere semplicemente

```
y <= d1 when s else d0;
```

Anche in VHDL 2008 è ancora necessario scrivere

```
q <= '1' when (stato = S2) else '0';
```

invece di

```
q <= (stato = S2);
```

Tabella 4.7 Decisione della rete.

Tipo di rete	Nessun pilota	Piloti in conflitto
<code>tri</code>	<code>z</code>	<code>x</code>
<code>triereg</code>	valore precedente	<code>x</code>
<code>triand</code>	<code>z</code>	0 se ci sono altri 0
<code>trior</code>	<code>z</code>	1 se ci sono altri 1
<code>tri0</code>	0	<code>x</code>
<code>tri1</code>	1	<code>x</code>

perché (stato = S2) restituisce un risultato `BOOLEAN` che non può essere direttamente assegnato al segnale `y` di tipo `STD_LOGIC`.

Anche se non si definiscono tutti i segnali come `BOOLEAN`, questi vengono automaticamente implicati dai confronti e usati nelle istruzioni condizionali. Analogamente, VHDL ha un tipo `INTEGER` che rappresenta numeri interi positivi e negativi: i segnali `INTEGER` possono assumere almeno tutti i valori compresi tra $-(2^{31} - 1)$ e $(2^{31} - 1)$. I valori interi sono usati come indici nei bus. Per esempio, nell'istruzione

```
y <= a(3) and a(2) and a(1) and a(0);
```

0, 1, 2 e 3 sono valori interi usati come indici per selezionare i bit di un segnale. Non si può indicizzare direttamente un bus con un segnale `STD_LOGIC` o `STD_LOGIC_VECTOR`, ma si deve prima convertire il segnale a tipo `INTEGER`. La conversione è mostrata nell'esempio che segue per un multiplexer 8:1 che seleziona un bit di un vettore utilizzando un indice a 3 bit. La funzione `TO_INTEGER` è definita nella libreria `IEEE.NUMERIC_STD_UNSIGNED` ed esegue la conversione da `STD_LOGIC_VECTOR` a `INTEGER` dei valori positivi (senza segno).

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.NUMERIC_STD_UNSIGNED.all;

entity mux8 is
  port(d: in  STD_LOGIC_VECTOR(7 downto 0);
        s: in  STD_LOGIC_VECTOR(2 downto 0);
        y: out STD_LOGIC);
end;

architecture sintesi of mux8 is
begin
  y <= d(TO_INTEGER(s));
end;
```

VHDL restringe anche l'uso del tipo `out` di `port` alle sole uscite. Per esempio, il codice seguente per porte `AND` a due e tre ingressi è illegale in VHDL perché `v` è un'uscita che viene usata anche per valutare `w`.

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity and23 is
  port(a, b, c: in STD_LOGIC;
        v, w: out  STD_LOGIC);
end;

architecture sintesi of and23 is
begin
  v <= a and b;
  w <= v and c;
end;
```

VHDL definisce un tipo speciale di `port`, il `buffer`, per risolvere questo problema. Un segnale collegato a un `buffer` si comporta come un'uscita ma può essere usato anche all'interno del modulo. La definizione corretta dell'esempio precedente è riportata di seguito. Verilog e SystemVerilog non hanno questa limitazione e non richiedono i `buffer`. VHDL 2008 elimina questa restrizione consentendo di leggere i valori di `port out`, ma questa modifica non è ancora supportata da Synplify Premier al momento della scrittura di questo testo.

```
entity and23 is
  port(a, b, c: in STD_LOGIC;
```

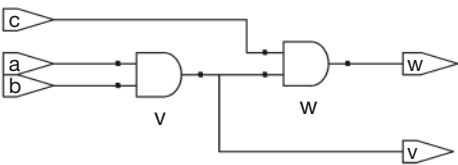


Figura 4.28
Sintesi del circuito and23.

```
v: buffer    STD_LOGIC;  
w: out      STD_LOGIC);  
  
end;
```

Molte operazioni come somma, sottrazione e operazioni della logica booleana sono identiche sia per i numeri senza segno sia per quelli con segno. Tuttavia, confronti di grandezza, moltiplicazioni e traslazioni aritmetiche a destra sono eseguiti diversamente per numeri in complemento a due e per numeri binari senza segno. Queste operazioni sono esaminate nel Capitolo 5. L'**Esempio HDL 4.33** descrive come indicare che un segnale rappresenta un numero con segno.

4.8 ■ MODULI PARAMETRICI*

I moduli visti sinora hanno un numero fisso di ingressi e uscite. Per esempio, si sono definiti due moduli diversi per i multiplexer 2:1 a 4 e 8 bit. Gli HDL consentono però di avere numeri di bit variabili grazie ai moduli parametrici.

L'**Esempio HDL 4.34** dichiara un multiplexer 2:1 parametrico di default a 8 bit, e lo usa poi per costruire multiplexer 4:1 a 8 e 12 bit.

L'**Esempio HDL 4.35** mostra un decoder, che è un'applicazione ancora migliore dei moduli parametrici. Un grosso decoder $N:2^N$ è lungo da specificare con istruzioni case, mentre è molto più semplice con codice parametrico che semplicemente forza a 1 il bit di uscita appropriato. Specificatamente, il decoder usa assegnamenti bloccanti per forzare a 0 tutte le uscite, poi modifica il bit opportuno a 1.

ESEMPIO HDL 4.33 (a) MOLTIPLICATORE SENZA SEGNO, (b) MOLTIPLICATORE CON SEGNO	
SystemVerilog <pre>// 4.33(a): moltiplicatore senza segno module moltiplicatore(input logic [3:0] a, b, output logic [7:0] y); assign y =a * b; endmodule // 4.33(b): moltiplicatore con segno module moltiplicatore(input logic signed [3:0] a, b, output logic signed [7:0] y); assign y =a * b; endmodule</pre> In SystemVerilog i segnali per default sono considerati senza segno. L'aggiunta del modificatore signed (come in logic signed [3:0] a) porta il segnale a a essere considerato con segno.	VHDL <pre>-- 4.33(a): moltiplicatore senza segno library IEEE; use IEEE.STD_LOGIC_1164.all; use IEEE.NUMERIC_STD_UNSIGNED.all; entity moltiplicatore is port(a, b: in STD_LOGIC_VECTOR(3 downto 0); y: out STD_LOGIC_VECTOR(7 downto 0)); end; architecture sintesi of moltiplicatore is begin y <= a * b; end;</pre> VHDL usa la libreria NUMERIC_STD_UNSIGNED per eseguire operazioni aritmetiche e confronti su vettori STD_LOGIC_VECTOR. I vettori sono considerati senza segno. use IEEE.NUMERIC_STD_UNSIGNED.all; VHDL definisce inoltre i tipi UNSIGNED e SIGNED nella libreria IEEE.NUMERIC_STD, ma questi richiedono conversioni di tipo che vanno oltre gli obiettivi di questo capitolo.

ESEMPIO HDL 4.34 MULTIPLEXER 2:1 A N BIT PARAMETRICI**SystemVerilog**

```

module mux2
  #(parameter numbit = 8)
  (input logic [numbit-1:0] d0, d1,
   input logic s,
   output logic [numbit-1:0] y);

  assign y = s ? d1 : d0;
endmodule

```

SystemVerilog consente l'istruzione `#(parameter...)` prima della dichiarazione di ingressi e uscite per definire parametri. L'istruzione `parameter` include un valore di default (8) del parametro che in questo esempio è denominato `numbit` . Il numero di bit di ingresso e uscita può essere reso dipendente da questo parametro.

```

module mux4_8(input logic [7:0] d0, d1, d2, d3,
              input logic [1:0] s,
              output logic [7:0] y);

  logic [7:0] basso, alto;

  mux2 muxbasso(d0, d1, s[0], basso);
  mux2 muxalto(d2, d3, s[0], alto);
  mux2 muxuscita(basso, alto, s[1], y);
endmodule

```

Il multiplexer 4:1 a 8 bit, `mux4_8` , istanzia tre multiplexer 2:1 utilizzando i numeri di bit di default.

Al contrario, il multiplexer 4:1 a 12 bit, `mux4_12` , deve modificare il valore di default utilizzando `#( )` prima del nome di istanza, come mostrato sotto.

```

module mux4_12(input logic [11:0] d0, d1, d2, d3,
               input logic [1:0] s,
               output logic [11:0] y);

  logic [11:0] basso, alto;

  mux2 #(12) muxbasso(d0, d1, s[0], basso);
  mux2 #(12) muxalto(d2, d3, s[0], alto);
  mux2 #(12) muxuscita(basso, alto, s[1], y);
endmodule

```

Si badi a non confondere il carattere `#` per indicare ritardi con la notazione `#(...)` per definire e modificare parametri.

VHDL

```

library IEEE; use IEEE.STD_LOGIC_1164.all;

entity mux2 is
  generic(numbit: integer := 8);
  port(d0,
        d1: in  STD_LOGIC_VECTOR(numbit-1 downto 0);
        s: in  STD_LOGIC;
        y: out STD_LOGIC_VECTOR(numbit-1 downto 0));
end;

```

```

architecture sintesi of mux2 is
begin
  y <= d1 when s else d0;
end;

```

L'istruzione `generic` include un valore di default (8) per `numbit` . Il valore è un numero intero.

```

library IEEE; use IEEE.STD_LOGIC_1164.all;

entity mux4_8 is
  port(d0, d1, d2,
        d3: in  STD_LOGIC_VECTOR(7 downto 0);
        s: in  STD_LOGIC_VECTOR(1 downto 0);
        y: out STD_LOGIC_VECTOR(7 downto 0));
end;

```

```

architecture sintesi of mux4_8 is
  component mux2
    generic(numbit: integer := 8);
    port(d0,
          d1: in  STD_LOGIC_VECTOR(numbit-1 downto 0);
          s: in  STD_LOGIC;
          y: out STD_LOGIC_VECTOR(numbit-1 downto 0));
  end component;
  signal basso, alto: STD_LOGIC_VECTOR(7 downto 0);
begin
  muxbasso: mux2 port map(d0, d1, s(0), basso);
  muxalto: mux2 port map(d2, d3, s(0), alto);
  muxuscita: mux2 port map(basso, alto, s(1), y);
end;

```

Il multiplexer 4:1 a 8 bit, `mux4_8` , istanzia tre multiplexer 2:1 utilizzando i numeri di bit di default.

Al contrario, il multiplexer 4:1 a 12 bit, `mux4_12` , deve modificare il valore di default utilizzando `generic map` prima del nome di istanza, come mostrato sotto.

```

muxbasso: mux2 generic map(12)
  port map(d0, d1, s(0), basso);
muxalto: mux2 generic map(12)
  port map(d2, d3, s(0), alto);
muxuscita: mux2 generic map(12)
  port map(basso, alto, s(1), y);

```

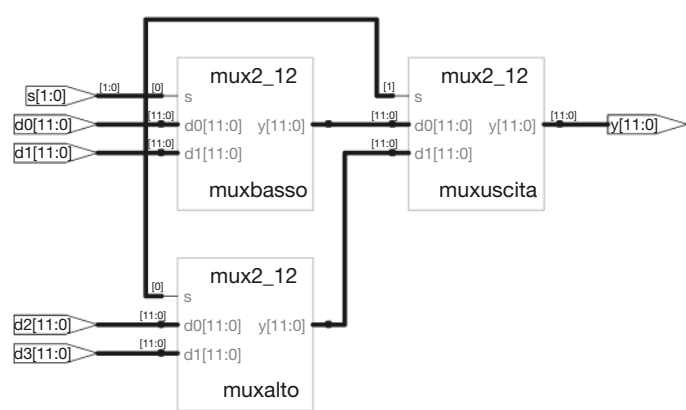


Figura 4.29 Sintesi del circuito mux4_12.

Gli HDL forniscono anche un'istruzione `generate` per produrre una quantità variabile di circuiti in funzione del valore di un parametro. `generate` supporta cicli `for` e istruzioni `if` per determinare quanti circuiti produrre e di che tipo. L'**Esempio HDL 4.36** mostra come usare l'istruzione `generate` per produrre un AND a N ingressi utilizzando una cascata di porte AND a 2 ingressi. Un operatore di riduzione sarebbe stato più chiaro e semplice per questo esempio, ma in questo modo si vede il principio generale di generazione dei circuiti.

Bisogna comunque fare attenzione all'uso dell'istruzione `generate`, perché si possono facilmente produrre grosse quantità di hardware senza volerlo.

ESEMPIO HDL 4.35 DECODER $N:2^N$ PARAMETRICO	
SystemVerilog <pre>module decoder #(parameter N = 3) (input logic [N-1:0] a, output logic [2**N-1:0] y); always_comb begin y = 0; y[a] = 1; end endmodule</pre> $2**N$ indica 2^N.	VHDL <pre>library IEEE; use IEEE.STD_LOGIC_1164.all; use IEEE.NUMERIC_STD_UNSIGNED.all; entity decoder is generic(N: integer := 3); port(a: in STD_LOGIC_VECTOR(N-1 downto 0); y: out STD_LOGIC_VECTOR(2**N-1 downto 0)); end; architecture sintesi of decoder is begin process(all) begin y <= (OTHERS => '0'); y(TO_INTEGER(a)) <= '1'; end process; end;</pre> $2**N$ indica 2^N.

ESEMPIO HDL 4.36 PORTA AND A N INGRESSI PARAMETRICA**SystemVerilog**

```

module andN
#(parameter numbit = 8)
  (input logic [numbit-1:0] a,
   output logic y);

  genvar i;
  logic [numbit-1:0] x;

  generate
    assign x[0] = a[0];
    for(i=1; i<numbit; i=i+1) begin: ciclofor
      assign x[i] = a[i] & x[i-1];
    end
  endgenerate

  assign y = x[numbit-1];
endmodule

```

L'istruzione `for` cicla per $i=1, 2, \dots, \text{numbit}-1$ per produrre una serie di porte AND consecutive. Il `begin` in un ciclo `generate for` deve essere seguito dal carattere `:` e da un'etichetta arbitraria (`ciclofor` in questo caso).

VHDL

```

library IEEE; use IEEE.STD_LOGIC_1164.all;

entity andN is
  generic(width: integer := 8);
  port(a: in STD_LOGIC_VECTOR(width-1 downto 0);
       y: out STD_LOGIC);
end;

architecture sintesi of andN is
  signal x: STD_LOGIC_VECTOR(width-1 downto 0);
begin
  x(0) <= a(0);
  ciclofor: for i in 1 to numbit-1 generate
    x(i) <= a(i) and x(i-1);
  end generate;
  y <= x(width-1);
end;

```

La variabile i del ciclo `generate` non richiede di essere dichiarata.

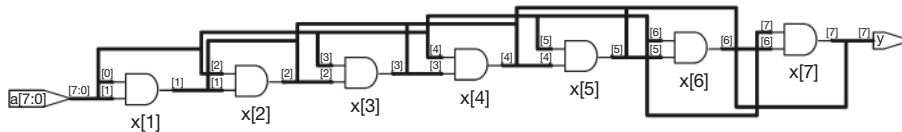


Figura 4.30 Sintesi del circuito `andN`.

4.9 ■ TESTBENCH

Un **testbench** (letteralmente “banco di prova”) è un modulo HDL usato per collaudare un altro modulo, chiamato DUT (*Device Under Test*). Il testbench contiene istruzioni per applicare ingressi al DUT e per verificare che le uscite assumano i valori corretti. Gli ingressi e le uscite corrispondenti sono chiamati **vettori di test**.

Si vuole collaudare il modulo *funzione* qualunque del paragrafo 4.1.1 che calcola $y = \bar{a}\bar{b}\bar{c} + a\bar{b}\bar{c} + a\bar{b}c$. È un modulo semplice per cui è possibile un collaudo esaustivo nel quale applicare al modulo tutte le otto possibili configurazioni di ingresso.

L'**Esempio HDL 4.37** mostra un semplice testbench: si istanzia DUT e quindi si applicano gli ingressi. Assegnamenti bloccanti e ritardi sono utilizzati per applicare gli ingressi nell'ordine opportuno. L'utente può vedere i risultati della simulazione e verificare esaminandoli che sono prodotti i valori di uscita corretti. I testbench sono simulabili come tutti gli altri moduli HDL, ma non sono sintetizzabili.

Verificare se le uscite sono corrette è noioso e a rischio di errori. Inoltre determinare la correttezza delle uscite è molto più facile se si ha il progetto ben presente in testa: se si fa qualche piccola modifica e si verifica il progetto qualche settimana dopo, verificare le uscite diventa un problema. Molto meglio progettare un testbench capace di autoverifica, come mostrato nell'**Esempio HDL 4.38**.

Anche scrivere il codice per ogni vettore di test è noioso, specie per moduli che richiedono grandi numeri di vettori. Un approccio ancora migliore

Alcuni strumenti definiscono il modulo da verificare *UUT (Unit Under Test)*.

ESEMPIO HDL 4.37 TESTBENCH**SystemVerilog**

```

module testbench1();
  logic a, b, c, y;

  // istanzia il Device Under Test
  funzionequalunque dut(a, b, c, y);

  // applica le configurazioni di ingresso una
  // alla volta
  initial begin
    a = 0; b = 0; c = 0; #10;
    c = 1;                #10;
    b = 1; c = 0;         #10;
    c = 1;                #10;
    a = 1; b = 0; c = 0; #10;
    c = 1;                #10;
    b = 1; c = 0;         #10;
    c = 1;                #10;
  end
endmodule

```

L'istruzione `initial` esegue le istruzioni al proprio interno all'inizio della simulazione. In questo caso, applica per prima cosa la configurazione 000 e attende 10 unità di tempo. Poi applica 001 e attende altre 10 unità di tempo, e così via finché tutte le otto possibili configurazioni di ingresso sono state applicate. L'istruzione `initial` va usata solo nei testbench per la simulazione, non nei moduli destinati a essere sintetizzati in circuiti effettivi. Non c'è infatti modo per l'hardware di eseguire una sequenza di passi speciali quando viene acceso.

VHDL

```

library IEEE; use IEEE.STD_LOGIC_1164.all;

entity testbench1 is -- né ingressi né uscite
end;

architecture simulazione of testbench1 is
  component funzionequalunque
    port(a, b, c: in STD_LOGIC;
          y:      out STD_LOGIC);
  end component;
  signal a, b, c, y: STD_LOGIC;
begin
  -- istanzia il Device Under Test
  dut: funzionequalunque port map(a, b, c, y);

  -- applica le configurazioni di ingresso una alla volta
  process begin
    a <= '0'; b <= '0'; c <= '0'; wait for 10 ns;
    c <= '1';                        wait for 10 ns;
    b <= '1'; c <= '0';              wait for 10 ns;
    c <= '1';                        wait for 10 ns;
    a <= '1'; b <= '0'; c <= '0'; wait for 10 ns;
    c <= '1';                        wait for 10 ns;
    b <= '1'; c <= '0';              wait for 10 ns;
    c <= '1';                        wait for 10 ns;
    wait; -- attende per sempre
  end process;
end;

```

L'istruzione `process` applica per prima cosa la configurazione 000 e attende 10 ns. Poi applica 001 e attende altri 10 ns, e così via finché tutte le otto possibili configurazioni di ingresso sono state applicate. Alla fine, il processo attende per sempre, altrimenti avrebbe ricominciato dall'inizio, continuando ad applicare ripetutamente il vettore di test.

è quello di mettere i vettori di test in un file separato. Il testbench si limita a leggere i vettori di test dal file, applica i vettori di ingresso al DUT, verifica se le uscite del DUT corrispondono ai vettori di uscita, e ripete queste operazioni fino alla fine del file dei vettori di test.

L'**Esempio HDL 4.39** mostra questo tipo di testbench, che genera un clock usando l'istruzione `always/process` senza sensitivity list cosicché viene continuamente rivalutato. All'inizio della simulazione, legge i vettori di test da un file di testo e genera impulsi su `reset` per due cicli. Sebbene `clock` e `reset` non siano necessari per simulare logica combinatoria, sono stati inseriti perché importanti nel caso di collaudo di DUT sequenziali. `esempio.tv` è un file di testo che contiene gli ingressi e le uscite scritti come numeri binari:

```

000_1
001_0
010_0
011_0
100_1
101_1
110_0
111_0

```

ESEMPIO HDL 4.38 TESTBENCH CON AUTOVERIFICA**SystemVerilog**

```

module testbench2();
  logic a, b, c, y;

  // istanzia il Device Under Test
  funzionequalunque dut(a, b, c, y);

  // applica le configurazioni di ingresso una alla
  // volta
  // verificando i risultati
  initial begin
    a = 0; b = 0; c = 0; #10;
    assert (y === 1) else $error("000 sbagliato.");
    c = 1; #10;
    assert (y === 0) else $error("001 sbagliato.");
    b = 1; c = 0; #10;
    assert (y === 0) else $error("010 sbagliato.");
    c = 1; #10;
    assert (y === 0) else $error("011 sbagliato.");
    a = 1; b = 0; c = 0; #10;
    assert (y === 1) else $error("100 sbagliato.");
    c = 1; #10;
    assert (y === 1) else $error("101 sbagliato.");
    b = 1; c = 0; #10;
    assert (y === 0) else $error("110 sbagliato.");
    c = 1; #10;
    assert (y === 0) else $error("111 sbagliato.");
  end
endmodule

```

L'istruzione SystemVerilog `assert` verifica se una determinata condizione è vera. In caso contrario, esegue l'istruzione `else`. La funzione di sistema `$error` nelle istruzioni `else` visualizza un messaggio di errore che descrive il tipo di comportamento sbagliato. `assert` è ignorata durante la sintesi.

In SystemVerilog, confronti che usano `==` o `!=` funzionano con segnali che non assumono i valori `x` e `z`. I testbench usano invece gli operatori `===` e `!==` per i confronti di uguaglianza e disuguaglianza, perché tali operatori funzionano correttamente con operandi che possono essere anche `x` o `z`.

VHDL

```

library IEEE; use IEEE.STD_LOGIC_1164.all;

entity testbench2 is -- né ingressi né uscite
end;

architecture simulazione of testbench2 is
  component funzionequalunque
    port(a, b, c: in STD_LOGIC;
         y:      out STD_LOGIC);
  end component;
  signal a, b, c, y: STD_LOGIC;
begin
  -- istanzia il Device Under Test
  dut: funzionequalunque port map(a, b, c, y);

  -- applica le configurazioni di ingresso una alla volta
  -- verificando i risultati
  process begin
    a <= '0'; b <= '0'; c <= '0'; wait for 10 ns;
    assert y = '1' report "000 sbagliato.";
    c <= '1'; wait for 10 ns;
    assert y = '0' report "001 sbagliato.";
    b <= '1'; c <= '0'; wait for 10 ns;
    assert y = '0' report "010 sbagliato.";
    c <= '1'; wait for 10 ns;
    assert y = '0' report "011 sbagliato.";
    a <= '1'; b <= '0'; c <= '0'; wait for 10 ns;
    assert y = '1' report "100 sbagliato.";
    c <= '1'; wait for 10 ns;
    assert y = '1' report "101 sbagliato.";
    b <= '1'; c <= '0'; wait for 10 ns;
    assert y = '0' report "110 sbagliato.";
    c <= '1'; wait for 10 ns;
    assert y = '0' report "111 sbagliato.";
    wait; -- attende per sempre
  end process;
end;

```

L'istruzione `assert` verifica una condizione e visualizza il messaggio dato dalla clausola `report` se la condizione non è soddisfatta. `assert` ha significato solo in simulazione, non in sintesi.

I nuovi ingressi vengono applicati sul fronte di salita del clock, e l'uscita viene verificata sul fronte di discesa del clock. Gli errori vengono segnalati quando si verificano. Alla fine della simulazione il testbench visualizza il numero totale di vettori di test applicati e il numero di errori rilevati.

Il testbench dell'**Esempio HDL 4.39** è eccessivo per un circuito così semplice, tuttavia può essere facilmente modificato per collaudare circuiti più complessi modificando il file `esempio.tv`, istanziando il nuovo DUT e modificando qualche linea di codice per applicare gli ingressi e verificare le uscite.

ESEMPIO HDL 4.39 TESTBENCH CON FILE DEI VETTORI DI TEST**SystemVerilog**

```

module testbench3();
    logic      clk, reset;
    logic      a, b, c, y, yattesa;
    logic [31:0] numvett, errori;
    logic [3:0] testvett[10000:0];

    // istanzia il Device Under Test
    funzionequalunque dut(a, b, c, y);

    // genera il clock
    always
    begin
        clk = 1; #5; clk = 0; #5;
    end

    // all'inizio del test, carica i vettori
    // e genera impulsi di reset
    initial
    begin
        $readmemb("esempio.tv", testvett);
        numvett = 0; errori = 0;
        reset = 1; #27; reset = 0;
    end

    // applica i vettori di test sul fronte di salita di clk
    always @(posedge clk)
    begin
        #1; {a, b, c, yattesa} = testvett[numvett];
    end

    // verifica i risultati sul fronte di discesa di clk
    always @(negedge clk)
    if (~reset) begin // salta durante il reset
        if (y != yattesa) begin // verifica il risultato
            $display("Errore: ingressi = %b", {a, b, c});
            $display("uscite = %b (%b attesa)", y, yattesa);
            errors = errors + 1;
        end
        numvett = numvett + 1;
        if (testvett[numvett] == 4'bx) begin
            $display("%d test finite con %d errori",
                numvett, errori);
            $finish;
        end
    end
endmodule

```

VHDL

```

library IEEE; use IEEE.STD_LOGIC_1164.all;
use IEEE.STD_LOGIC_TEXTIO.ALL; use STD.TEXTIO.all;

entity testbench3 is -- né ingressi né uscite
end;

architecture simulazione of testbench3 is
    component funzionequalunque
        port(a, b, c: in  STD_LOGIC;
              y:          out STD_LOGIC);
    end component;
    signal a, b, c, y: STD_LOGIC;
    signal y_attesa:  STD_LOGIC;
    signal clk, reset: STD_LOGIC;
begin
    -- istanzia il Device Under Test
    dut: funzionequalunque port map(a, b, c, y);

    -- genera il clock
    process begin
        clk <= '1'; wait for 5 ns;
        clk <= '0'; wait for 5 ns;
    end process;

    -- all'inizio del test, genera impulsi di reset
    process begin
        reset <= '1'; wait for 27 ns; reset <= '0';
        wait;
    end process;

    -- esegue i test
    process is
        file tv: text;
        variable L: line;
        variable vettore_in: std_logic_vector(2 downto 0);
        variable vuota: character;
        variable vettore_out: std_logic;
        variable numvett: integer := 0;
        variable errori: integer := 0;
    begin
        FILE_OPEN(tv, "esempio.tv", READ_MODE);
        while not endfile(tv) loop

            -- cambia i vettori sul fronte di salita
            wait until rising_edge(clk);

            -- legge la prossima riga di testvett e la divide
            readline(tv, L);
            read(L, vettore_in);

```

`$readmemb` legge un file di numeri binari nell'array `testvett`. `$readmemh` è simile ma legge numeri esadecimali.

Il blocco di codice successivo attende un'unità di tempo dopo il fronte di salita del clock (per evitare confusioni se clock e dati cambiano simultaneamente), quindi forza i valori dei tre ingressi (`a`, `b` e `c`) e l'uscita attesa (`yattesa`) usando i quattro bit del vettore di test corrente.

Il *testbench* confronta l'uscita, `y`, con l'uscita attesa, `yattesa`, e visualizza un errore se non coincidono. `%b` e `%d` indicano di visualizzare i valori in binario e in decimale rispettivamente. `$display` è una funzione di sistema per visualizzare nella finestra di simulazione: per esempio, `$display("%b %b", y, yattesa)`; visualizza i due valori, `y` e `yattesa`, in binario. `%h` visualizza un valore in esadecimale.

Questo processo si ripete finché non ci sono più vettori di test validi nell'array `testvett`. `$finish` termina la simulazione.

Si noti che anche se il modulo SystemVerilog supporta fino a 10001 vettori di test, esso terminerà la simulazione dopo aver eseguito gli otto vettori presenti nel file.

```
read(L, vuota); -- salta il carattere underscore
read(L, vettore_out);
(a, b, c) <= vettore_in(2 downto 0) after 1 ns;
y_attesa <= vettore_out after 1 ns;

-- verifica i risultati sul fronte di discesa
wait until falling_edge(clk);

if y /= y_attesa then
    report "Errore: y = " & std_logic'image(y);
    errori := errori + 1;
end if;

numvett := numvett + 1;
end loop;

-- riassume i risultati alla fine della simulazione
if (errors = 0) then
    report "NO ERRORI -- " &
        integer'image(numvett) &
        " test completati con successo."
        severity failure;
else
    report integer'image(numvett) &
        " test completati, errori =" &
        integer'image(errori)
        severity failure;
end if;
end process;
end;
```

Il codice VHDL usa comandi di lettura file che vanno oltre gli obiettivi di questo capitolo, ma serve a dare un'idea di come un testbench con autoverifica appare in VHDL.

4.10 ■ RIASSUNTO

I linguaggi di descrizione dell'hardware (HDL) sono strumenti estremamente importanti per i moderni progettisti di circuiti digitali. Una volta imparato SystemVerilog o VHDL, si diventa capaci di fornire le specifiche dei sistemi digitali molto più rapidamente di quanto avverrebbe dovendo fornire gli schemi circuitali. Anche il ciclo di correzione degli errori è molto più rapido, perché le modifiche richiedono cambiamenti del codice invece di lunghe correzioni degli schemi. Tuttavia tale ciclo può diventare molto lungo se non si ha una chiara idea del circuito che il codice HDL sottintende.

Gli HDL sono usati sia per simulazione sia per sintesi. La simulazione logica è un modo molto potente di collaudare un sistema mediante il calcolatore prima di realizzarlo in hardware. I simulatori consentono di verificare i valori di segnali nel proprio sistema che sarebbe impossibile misurare in un circuito fisico. La sintesi logica converte il codice HDL in circuiti logici.

La cosa più importante da ricordare quando si scrive il codice HDL è il fatto che si sta descrivendo un circuito fisico, non si sta programmando un calcolatore. L'errore più comune dei principianti è scrivere il codice HDL senza pensare all'hardware che si intende produrre. Se non si ha idea di quale

hardware è sottinteso dal codice HDL, non si otterrà certo quanto desiderato. Quindi meglio iniziare con uno schema a blocchi del sistema che si ha in mente, identificando le parti combinatorie e quelle sequenziali o costituite da macchine a stati finiti, e così via. Successivamente si può scrivere il codice HDL per ogni blocco, usando gli idiomi corretti per generare le strutture circuitali necessarie.

Esercizi

I seguenti esercizi possono essere svolti usando il linguaggio HDL che si preferisce. Se si ha la disponibilità di un simulatore si può verificare il proprio progetto, visualizzando le forme d'onda e spiegando in che modo ne dimostrano il corretto funzionamento. Se si ha la disponibilità di uno strumento di sintesi, si può effettuare la sintesi del proprio codice, esaminare lo schema circuitale generato e spiegare perché soddisfa le proprie aspettative.

Esercizio 4.1 Tracciare uno schema del circuito descritto dal seguente codice HDL, e semplificare tale schema minimizzando il numero di porte logiche. (**LISTATO 1**)

LISTATO 1

SystemVerilog

```
module esercizio_4_1(input  logic a, b, c,
                    output logic y, z);

    assign y = a & b & c | a & b & ~c | a & ~b & c;
    assign z = a & b | ~a & ~b;
endmodule
```

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity esercizio_4_1 is
    port(a, b, c: in  STD_LOGIC;
          y, z:  out STD_LOGIC);
end;

architecture sintesi of esercizio_4_1 is
begin
    y <= (a and b and c) or (a and b and not c) or
        (a and not b and c);
    z <= (a and b) or (not a and not b);
end;
```

Esercizio 4.2 Tracciare uno schema del circuito descritto dal seguente codice HDL, e semplificare tale schema minimizzando il numero di porte logiche. (**LISTATO 2**)

LISTATO 2

SystemVerilog

```
module esercizio_4_2(input  logic [3:0] a,
                    output logic [1:0] y);

    always_comb
        if      (a[0]) y = 2'b11;
        else if (a[1]) y = 2'b10;
        else if (a[2]) y = 2'b01;
        else if (a[3]) y = 2'b00;
        else          y = a[1:0];
endmodule
```

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity esercizio_4_2 is
    port(a: in  STD_LOGIC_VECTOR(3 downto 0);
          y: out STD_LOGIC_VECTOR(1 downto 0));
end;

architecture sintesi of esercizio_4_2 is
begin
    process(all) begin
        if      a(0) then y <= "11";
        elsif a(1) then y <= "10";
        elsif a(2) then y <= "01";
        elsif a(3) then y <= "00";
        else          y <= a(1 downto 0);
        end if;
    end process;
end;
```


Esercizio 4.3 Scrivere in HDL un modulo che calcola la funzione XOR a 4 ingressi. L'ingresso è $a_{3:0}$ e l'uscita è y .

Esercizio 4.4 Scrivere in HDL un testbench con autoverifica per l'Esercizio 4.3. Creare un file di vettori di test con tutti i 16 casi. Simulare il circuito e mostrare che funziona. Introdurre un errore nel file dei vettori di test e mostrare che il *testbench* segnala un errore.

Esercizio 4.5 Scrivere in HDL un modulo denominato *minoranza*, che riceve i tre ingressi a , b e c e produce l'uscita y con valore VERO se almeno due degli ingressi hanno valore FALSO.

Esercizio 4.6 Scrivere in HDL un modulo per un trascodificatore da esadecimale a sette segmenti, capace di gestire anche le cifre esadecimali A, B, C, D, E, F oltre alle normali cifre decimali 0-9.

Esercizio 4.7 Scrivere in HDL un testbench con autoverifica per l'Esercizio 4.6. Creare un file di vettori di test con tutti i 16 casi. Simulare il circuito e mostrare che funziona. Introdurre un errore nel file dei vettori di test e mostrare che il *testbench* segnala un errore.

Esercizio 4.8 Scrivere in HDL un modulo multiplexer 8:1 di nome *mux8*, con ingressi $s_{2:0}$, d_0 , d_1 , d_2 , d_3 , d_4 , d_5 , d_6 , d_7 , e uscita y .

Esercizio 4.9 Scrivere in HDL un modulo strutturale per calcolare la funzione logica $y = ab + \bar{b}c + \bar{a}bc$ usando logica a multiplexer. Fare uso del multiplexer 8:1 dell'Esercizio 4.8.

Esercizio 4.10 Ripetere l'Esercizio 4.9 con multiplexer 4:1 e tante porte NOT quante sono necessarie.

Esercizio 4.11 Nel paragrafo 4.5.4 si è visto come sia possibile descrivere correttamente un sincronizzatore con assegnamenti di tipo bloccante purché dati nell'ordine opportuno. Immaginare un semplice circuito che non può essere in ogni caso descritto con assegnamenti bloccanti, indipendentemente dal loro ordine.

Esercizio 4.12 Scrivere in HDL un modulo per un circuito a priorità a 8 ingressi.

Esercizio 4.13 Scrivere in HDL un modulo per un decoder 2:4.

Esercizio 4.14 Scrivere in HDL un modulo per un decoder 6:64 usando tre istanze del decoder 2:4 dell'Esercizio 4.13 e un po' di porte AND a 3 ingressi.

Esercizio 4.15 Scrivere in HDL i moduli che realizzano le funzioni booleane dell'Esercizio 2.13.

Esercizio 4.16 Scrivere in HDL un modulo che realizza il circuito dell'Esercizio 2.26.

Esercizio 4.17 Scrivere in HDL un modulo che realizza il circuito dell'Esercizio 2.27.

Esercizio 4.18 Scrivere in HDL un modulo che realizza la funzione logica dell'Esercizio 2.28. Fare attenzione alla gestione delle indifferenze.

Esercizio 4.19 Scrivere in HDL un modulo che realizza le funzioni dell'Esercizio 2.35.

Esercizio 4.20 Scrivere in HDL un modulo che realizza l'encoder a priorità dell'Esercizio 2.36.

Esercizio 4.21 Scrivere in HDL un modulo che realizza l'encoder a priorità modificato dell'Esercizio 2.37.

Esercizio 4.22 Scrivere in HDL un modulo che realizza il trascodificatore binario-termometrico dell'Esercizio 2.38.

Esercizio 4.23 Scrivere in HDL un modulo che realizza la funzione giorni-in-un-mese della Domanda 2.2.

Esercizio 4.24 Tracciare il diagramma degli stati della FSM descritta dal seguente codice HDL. (**LISTATO 3**)

Esercizio 4.25 Tracciare il diagramma degli stati della FSM descritta dal seguente codice HDL. Una FSM di questo tipo viene usata per la predizione dei salti in alcuni microprocessori. (**LISTATO 4**)

Esercizio 4.26 Scrivere in HDL un modulo che realizza un latch SR.

Esercizio 4.27 Scrivere in HDL un modulo che realizza un flip-flop JK, con ingressi clk , J e K e uscita Q . Sul fronte di salita del clock, Q mantiene il proprio valore se $J = K = 0$, passa a 1 se $J = 1$, passa a 0 se $K = 1$ e commuta se $J = K = 1$.

Esercizio 4.28 Scrivere in HDL un modulo che realizza il latch della Figura 3.18. Usare un'istruzione di assegnamento per ciascuna porta logica. Specificare un ritardo di una unità di tempo o di 1 ns per ciascuna porta logica, e simulare il latch per dimostrarne il corretto funzionamento. Poi aumentare il ritardo del negatore: quanto deve essere aumentato perché si verifichi una situazione di corsa che porta al malfunzionamento del latch?

Esercizio 4.29 Scrivere in HDL un modulo che realizza il controllore semaforico del paragrafo 3.4.1.

Esercizio 4.30 Scrivere in HDL tre moduli che realizzino il controllore semaforico fattorizzato con modalità parata dell'Esempio 3.8. I tre moduli devono chiamarsi *controllore*, *modalità* e *semafori* e avere gli ingressi e le uscite mostrati in Figura 3.33(b).

Esercizio 4.31 Scrivere in HDL un modulo che realizza il circuito della Figura 3.42.

Esercizio 4.32 Scrivere in HDL un modulo a partire dalla FSM con diagramma degli stati mostrato nella Figura 3.69 relativa all'Esercizio 3.22.

Esercizio 4.33 Scrivere in HDL un modulo a partire dalla FSM con diagramma degli stati mostrato nella Figura 3.70 relativa all'Esercizio 3.23.

LISTATO 3**SystemVerilog**

```

module fsm_4_24(input  logic clk, reset,
               input  logic a, b,
               output logic y);
    logic [1:0] stato, statopross;

    parameter S0 = 2'b00;
    parameter S1 = 2'b01;
    parameter S2 = 2'b10;
    parameter S3 = 2'b11;

    always_ff @(posedge clk, posedge reset)
        if (reset) stato <= S0;
        else      stato <= statopross;

    always_comb
        case (stato)
            S0: if (a ^ b) statopross = S1;
                else      statopross = S0;
            S1: if (a & b) statopross = S2;
                else      statopross = S0;
            S2: if (a | b) statopross = S3;
                else      statopross = S0;
            S3: if (a | b) statopross = S3;
                else      statopross = S0;
        endcase
    assign y = (stato== S1) | (stato== S2);
endmodule

```

VHDL

```

library IEEE; use IEEE.STD_LOGIC_1164.all;

entity fsm_4_24 is
    port(clk, reset: in  STD_LOGIC;
         a, b:      in  STD_LOGIC;
         y:         out STD_LOGIC);
end;

architecture sintesi of fsm_4_24 is
    type tipostato is (S0, S1, S2, S3);
    signal stato, statopross: tipostato;
begin
    process(clk, reset) begin
        if reset then stato <= S0;
        elsif rising_edge(clk) then
            stato <= statopross;
        end if;
    end process;

    process(all) begin
        case stato is
            when S0 => if (a xor b) then
                           statopross <= S1;
                       else statopross <= S0;
                       end if;
            when S1 => if (a and b) then
                           statopross <= S2;
                       else statopross <= S0;
                       end if;
            when S2 => if (a or b) then
                           statopross <= S3;
                       else statopross <= S0;
                       end if;
            when S3 => if (a or b) then
                           statopross <= S3;
                       else statopross <= S0;
                       end if;
        end case;
    end process;

    y <= '1' when ((stato = S1) or (stato = S2))
        else '0';
end;

```

Esercizio 4.34 Scrivere in HDL un modulo che realizza il controllore semaforico migliorato dell'Esercizio 3.24.

Esercizio 4.35 Scrivere in HDL un modulo che realizza la sorella lumaca dell'Esercizio 3.25.

Esercizio 4.36 Scrivere in HDL un modulo che realizza il distributore automatico di bottigliette di acqua minerale dell'Esercizio 3.26.

Esercizio 4.37 Scrivere in HDL un modulo che realizza il contatore codice Gray dell'Esercizio 3.27.

Esercizio 4.38 Scrivere in HDL un modulo che realizza il contatore UP/DOWN codice Gray dell'Esercizio 3.28.

Esercizio 4.39 Scrivere in HDL un modulo che realizza la FSM dell'Esercizio 3.29.

Esercizio 4.40 Scrivere in HDL un modulo che realizza la FSM dell'Esercizio 3.30.

Esercizio 4.41 Scrivere in HDL un modulo che realizza il circuito per il calcolo seriale del complemento a due della Domanda 3.2.

LISTATO 4**SystemVerilog**

```

module fsm_4_25(input  logic clk, reset,
               input  logic fatto, indietro,
               output logic predicefatto);

    logic [4:0] stato, statopross;

    parameter S0 = 5'b00001;
    parameter S1 = 5'b00010;
    parameter S2 = 5'b00100;
    parameter S3 = 5'b01000;

    always_ff @(posedge clk, posedge reset)
        if (reset) stato <= S2;
        else      stato <= statopross;

    always_comb
        case (stato)
            S0: if (fatto) statopross = S1;
                else      statopross = S0;
            S1: if (fatto) statopross = S2;
                else      statopross = S0;
            S2: if (fatto) statopross = S3;
                else      statopross = S1;
            S3: if (fatto) statopross = S4;
                else      statopross = S2;
            S4: if (fatto) statopross = S4;
                else      statopross = S3;
            default:      statopross = S2;
        endcase

    assign predicefatto = (stato == S4) |
                          (stato == S3) |
                          (stato == S2 && indietro);
endmodule

```

VHDL

```

library IEEE; use IEEE.STD_LOGIC_1164.all;

entity fsm_4_25 is
    port(clk, reset:      in  STD_LOGIC;
         fatto, indietro: in  STD_LOGIC;
         predicefatto:    out STD_LOGIC);
end;

architecture sintesi of fsm_4_25 is
    type tipostato is (S0, S1, S2, S3, S4);
    signal stato, statopross: tipostato;
begin
    process(clk, reset) begin
        if reset then stato <= S2;
        elsif rising_edge(clk) then
            stato <= statopross;
        end if;
    end process;

    process(all) begin
        case stato is
            when S0 => if fatto then
                           statopross <= S1;
                       else statopross <= S0;
                       end if;
            when S1 => if fatto then
                           statopross => S2;
                       else statopross <= S0;
                       end if;
            when S2 => if fatto then
                           statopross <= S3;
                       else statopross <= S1;
                       end if;
            when S3 => if fatto then
                           statopross <= S4;
                       else statopross <= S2;
                       end if;
            when S4 => if fatto then
                           statopross <= S4;
                       else statopross <= S3;
                       end if;
            when others => statopross <= S2;
        end case;
    end process;

    — logica di uscita
    predicefatto <= '1' when
        ((stato = S4) or (stato = S3) or
         (stato = S2 and back = '1'))
    else '0';
end;

```

Esercizio 4.42 Scrivere in HDL un modulo che realizza il circuito dell'Esercizio 3.31.

Esercizio 4.43 Scrivere in HDL un modulo che realizza il circuito dell'Esercizio 3.32.

Esercizio 4.44 Scrivere in HDL un modulo che realizza il circuito dell'Esercizio 3.33.

Esercizio 4.45 Scrivere in HDL un modulo che realizza il circuito dell'Esercizio 3.34. Può essere utile fare uso del sommatore completo del paragrafo 4.2.5.

Esercizi SystemVerilog

Gli esercizi che seguono sono specifici per SystemVerilog.

Esercizio 4.46 Cosa significa per un segnale essere dichiarato di tipo `tri` in SystemVerilog?

Esercizio 4.47 Riscrivere il modulo `bruttosinc` dell'Esempio HDL 4.29. Fare uso di assegnamenti non bloccanti, ma modificare il codice per produrre un sincronizzatore corretto con due flip-flop.

Esercizio 4.48 Considerare i seguenti due moduli SystemVerilog. Fanno la stessa funzione? Schematizzare la struttura circuitale di entrambi.

```
module codice1(input  logic clk, a, b, c,
               output logic y);
    logic x;
    always_ff @(posedge clk) begin
        x <= a & b;
        y <= x | c;
    end
endmodule

module codice2 (input  logic a, b, c, clk,
                 output logic y);
    logic x;
    always_ff @(posedge clk) begin
        y <= x | c;
        x <= a & b;
    end
endmodule
```

Esercizio 4.49 Ripetere l'Esercizio 4.48 sostituendo `<=` con `=` in ogni assegnamento.

Esercizio 4.50 I seguenti moduli SystemVerilog mostrano errori che gli Autori del libro hanno visto fare agli studenti in laboratorio. Spiegare l'errore presente in ciascun modulo e mostrare come eliminarlo.

(a)

```
module latch(input  logic      clk,
             input  logic [3:0] d,
             output reg [3:0] q);
    always @(clk)
        if (clk) q <= d;
endmodule
```

(b)

```
module porte(input  logic [3:0] a, b,
             output logic [3:0] y1, y2, y3, y4, y5);
    always @(a)
    begin
        y1 = a & b;
        y2 = a | b;
        y3 = a ^ b;
        y4 = ~(a & b);
        y5 = ~(a | b);
    end
endmodule
```

(c)

```
module mux2(input  logic [3:0] d0, d1,
            input  logic      s,
            output logic [3:0] y);
    always @(posedge s)
        if (s) y <= d1;
        else y <= d0;
endmodule
```

(d)

```
module duefflop(input  logic clk,
                input  logic d0, d1,
                output logic q0, q1);
    always @(posedge clk)
        q1 = d1;
        q0 = d0;
endmodule
```

(e)

```
module FSM(input  logic clk,
            input  logic a,
            output logic out1, out2);
    logic stato;
    // logica di stato prossimo e registro (sequenziale)
    always_ff @(posedge clk)
        if (stato == 0) begin
            if (a) stato <= 1;
        end else begin
            if (~a) stato <= 0;
        end
    always_comb // logica di uscita (combinatoria)
        if (stato == 0) out1 = 1;
        else out2 = 1;
endmodule
```

(f)

```
module priorit (input  logic [3:0] a,
                output logic [3:0] y);
    always_comb
        if      (a[3]) y = 4'b1000;
        else if (a[2]) y = 4'b0100;
        else if (a[1]) y = 4'b0010;
        else if (a[0]) y = 4'b0001;
endmodule
```

(g)

```
module FSM_dividiper3(input  logic clk,
                     input  logic reset,
                     output logic out);
    logic [1:0] stato, statopross;
    parameter S0 = 2'b00;
    parameter S1 = 2'b01;
    parameter S2 = 2'b10;
    // registro di stato
    always_ff @(posedge clk, posedge reset)
        if (reset) stato <= S0;
        else stato <= statopross;
    // logica di stato prossimo
    always @(stato)
        case (stato)
            S0: statopross = S1;
            S1: statopross = S2;
```

```

        S2: statopross = S0;
    endcase
    // logica di uscita
    assign out = (stato == S2);
endmodule

```

(h)

```

module mux2tri(input  logic [3:0] d0, d1,
               input  logic      s,
               output tri  [3:0] y);
    tristate t0(d0, s, y);
    tristate t1(d1, s, y);
endmodule

```

(i)

```

module flopsren(input  logic      clk,
                input  logic      reset,
                input  logic      set,
                input  logic [3:0] d,
                output logic [3:0] q);
    always_ff @(posedge clk, posedge reset)
        if (reset) q <= 0;
    else      q <= d;
    always @(set)
        if (set) q <= 1;
endmodule

```

(j)

```

module and3(input  logic a, b, c,
            output logic y);
    logic temp;
    always @(a, b, c)
    begin
        temp <= a & b;
        y <= temp & c;
    end
endmodule

```

Esercizi VHDL

Gli esercizi che seguono sono specifici per VHDL.

Esercizio 4.51 Perché in VHDL è necessario scrivere

```
q <= '1' when stato = S0 else '0';
```

invece di scrivere semplicemente

```
q <= (stato = S0);
```

Esercizio 4.52 Ciascuno dei seguenti moduli VHDL contiene un errore. Per brevità, viene mostrata solo l'architettura, assumendo che la clausola di uso della `library` e la dichiarazione di `entity` siano corrette. Spiegare l'errore presente in ciascun modulo e mostrare come eliminarlo.

(a)

```

architecture sintesi of latch is
begin
    process(clk) begin
        if clk = '1' then q <= d;
        end if;
    end process;
end;

```

(b)

```

architecture proc of porte is
begin
    process(a) begin
        Y1 <= a and b;
        y2 <= a or b;
        y3 <= a xor b;
        y4 <= a nand b;
        y5 <= a nor b;
    end process;
end;

```

(c)

```

architecture sintesi of flop is
begin
    process(clk)
        if rising_edge(clk) then
            q <= d;
        end if;
    end process;
end;

```

(d)

```

architecture sintesi of prioritata is
begin
    process(all) begin
        if a(3) then y <= "1000";
        elsif a(2) then y <= "0100";
        elsif a(1) then y <= "0010";
        elsif a(0) then y <= "0001";
        end if;
    end process;
end;

```

(e)

```

architecture sintesi of FSM_dividiper3 is
    type tipostato is (S0, S1, S2);
    signal stato, statopross: tipostato;
begin
    process(clk, reset) begin
        if reset then stato <= S0;
        elsif rising_edge(clk) then
            stato <= statopross;
        end if;
    end process;
    process(stato) begin
        case stato is
            when S0 => statopross <= S1;
            when S1 => statopross <= S2;
            when S2 => statopross <= S0;
        end case;
    end process;
    q <= '1' when stato = S0 else '0';
end;

```

(f)

```

architecture struttura of mux2 is
    component tristate
        port(a: in STD_LOGIC_VECTOR(3 downto 0);
             en: in STD_LOGIC;
             y: out STD_LOGIC_VECTOR(3 downto 0));
    end component;
begin
    t0: tristate port map(d0, s, y);
end;

```

```
    t1: tristate port map(d1, s, y);
end;

(g)
architecture asincrono of flopsr is
begin
    process(clk, reset) begin
        if reset then
            q <= '0';
        elsif rising_edge(clk) then
```

```
            q <= d;
        end if;
    end process;
    process(set) begin
        if set then
            q <= '1';
        end if;
    end process;
end;
```

Domande di valutazione

Queste domande sono state poste a candidati per un posto di lavoro nell'ambito della progettazione di sistemi digitali.

Domanda 4.1 Scriva una riga di codice HDL che campiona un bus a 32 bit di nome `dati` con un segnale di nome `sel` e produce il risultato a 32 bit. Se `sel` è VERO `risultato = dati`, altrimenti tutti i bit di `risultato` devono essere azzerati.

Domanda 4.2 Spieghi la differenza tra assegnamenti bloccanti e non bloccanti, facendo qualche esempio.

Domanda 4.3 Cosa fa la seguente istruzione SystemVerilog?

```
risultato = | (dati[15:0] & 16'hC820);
```


Blocchi costruttivi digitali

Capitolo

5

5.1 Introduzione

5.2 Circuiti aritmetici

5.3 Sistemi di numerazione

5.4 Blocchi costruttivi sequenziali

5.5 Componenti di memoria

5.6 Matrici logiche

5.7 Riassunto

5.1 ■ INTRODUZIONE

Nei capitoli precedenti è stata esaminata la progettazione delle reti combinatorie e sequenziali usando le espressioni booleane, gli schemi elettrici e gli HDL. In questo capitolo vengono introdotti blocchi costruttivi sia combinatori sia sequenziali più complessi, utilizzati nei sistemi digitali. Questi blocchi comprendono i circuiti aritmetici, i contatori, i registri a scorrimento, le memorie e gli array logici. Questi blocchi costruttivi non solo sono utili di per sé, ma dimostrano anche i principi di gerarchia, modularità e regolarità. I blocchi costruttivi vengono infatti assemblati gerarchicamente a partire dai componenti più semplici come le porte logiche, i multiplexer e i decoder. Ogni blocco costruttivo possiede un'interfaccia ben definita e può inoltre essere considerato come una scatola nera quando la realizzazione interna non è importante. La struttura regolare di ogni blocco costruttivo consente inoltre una facile estensione a dimensioni diverse delle informazioni da elaborare. Il Capitolo 7 mostra come utilizzare questi blocchi costruttivi per realizzare un microprocessore.

5.2 ■ CIRCUITI ARITMETICI

I circuiti aritmetici sono i blocchi costruttivi centrali dei calcolatori. I calcolatori e la logica digitale eseguono molte funzioni aritmetiche: addizioni, sottrazioni, confronti, traslazioni, moltiplicazioni e divisioni. In questo paragrafo viene descritta la realizzazione hardware per ognuna di queste operazioni.

5.2.1 Addizione

L'addizione è una delle operazioni più comuni nei sistemi digitali. Si inizia esaminando come sia possibile sommare due numeri binari a 1 bit. Dopodiché, l'operazione viene estesa ai numeri binari a N bit. I circuiti sommatore sono anche un ottimo esempio di criteri di scelta tra velocità e complessità.

Software applicativo	
Sistema operativo	
Architettura	
Micro-architettura	
Blocchi logici	
Reti digitali	
Reti analogiche	
Dispositivi	
Fisica	

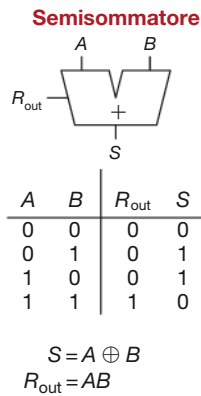


Figura 5.1
Semisommatore a 1 bit.

```

      1
    0001
  + 0101
  -----
    0110
  
```

Figura 5.2
Bit di riporto (carry).

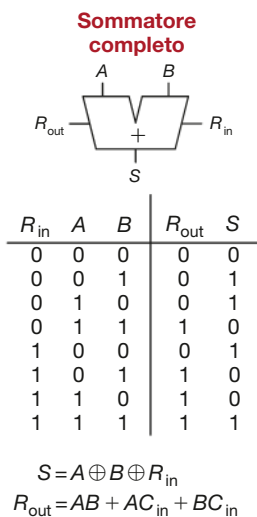


Figura 5.3
Sommatore completo a 1 bit.

Semisommatore

Si comincia dalla costruzione di un **semisommatore** (*half adder*) a 1 bit. Come mostrato nella **Figura 5.1**, il semisommatore ha due ingressi, A e B, e due uscite, S e R_{out} . S rappresenta la somma di A e B. Se sia A sia B hanno valore 1, S è uguale a 2, un valore che non può essere rappresentato con una sola cifra binaria. Di conseguenza, il valore 2 viene rappresentato con un **riporto** (carry) R_{out} nella colonna successiva. Il semisommatore può essere costruito con una porta XOR e una porta AND.

In un sommatore a più bit, R_{out} viene sommato al bit più significativo successivo. Per esempio, nella **Figura 5.2** il bit di riporto mostrato in rosso è l'uscita R_{out} della prima colonna della somma a 1 bit e l'ingresso R_{in} della seconda colonna della somma. Però al semisommatore manca un ingresso R_{in} per accettare R_{out} della colonna precedente. Per risolvere questo problema si utilizza il sommatore completo descritto nel prossimo paragrafo.

Sommatore completo

Un **sommatore completo** (*full adder*), già introdotto nel paragrafo 2.1, accetta l'ingresso R_{in} come mostrato nella **Figura 5.3**. La figura mostra anche le espressioni logiche per le uscite S e R_{out} .

Sommatore a propagazione di riporto

Un sommatore a N bit somma due ingressi a N bit, A e B, e aggiunge R_{in} per produrre un risultato a N bit S e un riporto R_{out} . Questo sommatore viene comunemente chiamato **sommatore a propagazione di riporto** (o CPA, *Carry Propagate Adder*) perché il riporto di un bit si propaga nel bit successivo. Il simbolo circuitale di un sommatore a propagazione di riporto è riportato nella **Figura 5.4**; è identico al simbolo del full adder, con l'unica differenza che A, B e S non rappresentano bit singoli ma bus. Le tre realizzazioni più comuni di CPA sono i sommatore a propagazione di riporto a onda, i sommatore ad anticipazione di riporto (*carry lookahead*) e i sommatore a prefissi (*prefix*).

Sommatore a propagazione di riporto a onda

Il metodo più semplice per costruire un sommatore a propagazione di riporto a onda a N bit è collegare in cascata N full adder completi. In questo modo, R_{out} di uno stadio costituisce R_{in} per lo stadio successivo, come mostrato nella **Figura 5.5** per una somma a 32 bit. Questo particolare sommatore viene chiamato **sommatore a propagazione di riporto a onda** (*ripple carry*) e rappresenta una buona applicazione dei principi di modularità e regolarità. Infatti, il modulo del full adder viene riutilizzato più volte per formare un sistema di maggiori dimensioni. Il principale svantaggio legato a questo sommatore è il progressivo rallentamento all'aumentare di N. Infatti, S_{31} dipende da R_{30} , che dipende da R_{29} , che dipende a sua volta da R_{28} e così via fino a risalire a R_{in} , come mostrato in rosso nella Figura 5.5. Si dice quindi che il riporto si propaga a onda attraverso la catena. Il ritardo di propagazione nel somma-

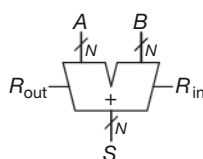


Figura 5.4
Sommatore a propagazione di riporto.

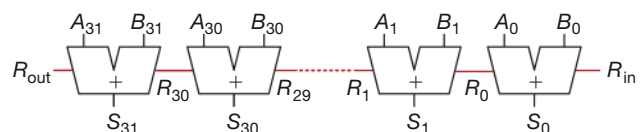


Figura 5.5
Sommatore a 32 bit a propagazione di riporto a onda.

tore, t_{propag} , aumenta all'aumentare del numero di bit coinvolti, come riporta l'Espressione 5.1, dove t_{FA} rappresenta il ritardo di un full adder.

$$t_{\text{propag}} = N t_{\text{FA}} \quad (5.1)$$

Sommatore ad anticipazione di riporto

La ragione principale per cui i sommatore a propagazione di riporto a onda di grandi dimensioni sono lenti è il fatto che i segnali devono propagarsi attraverso ogni bit del sommatore. Un **sommatore ad anticipazione di riporto** (CLA, *Carry-Lookahead Adder*) è un altro tipo di sommatore a propagazione di riporto che risolve il problema della velocità dividendo il sommatore stesso in **blocchi** e aggiungendo un circuito per determinare velocemente il riporto di uscita da ciascun blocco appena è noto il riporto di ingresso. Per questo si dice che il sommatore è in grado di “anticipare” o “guardare avanti” (*look ahead*) attraverso i blocchi invece di attendere che il riporto si propaghi attraverso tutti i full adder del blocco. Per esempio, un sommatore a 32 bit può essere diviso in otto blocchi da 4 bit ciascuno.

I sommatore ad anticipazione di riporto utilizzano segnali di **generazione** (G) e di **propagazione** (P) che descrivono come una colonna o un blocco determinano il proprio riporto. Si dice che la colonna i di un sommatore **genera** un riporto se questa produce un riporto di uscita indipendentemente dal valore del riporto di ingresso. La colonna i di un sommatore genera sicuramente R_i se A_i e B_i sono entrambi uguali a 1. Di conseguenza G_i , cioè il riporto generato dalla colonna i , viene calcolato come $G_i = A_i B_i$. Si dice che la colonna i **propaga** un riporto se produce un riporto di uscita ogniqualvolta ci sia un riporto di ingresso. La colonna i propaga il proprio riporto di ingresso, R_{i-1} , se o A_i o B_i sono uguali a 1. Di conseguenza, $P_i = A_i + B_i$. Utilizzando queste definizioni è possibile riscrivere la logica di riporto di una specifica colonna del sommatore: la colonna i del sommatore produce un riporto di uscita R_i se genera un riporto, G_i , o se propaga il riporto di ingresso, $P_i R_{i-1}$. L'espressione corrispondente è:

$$R_i = A_i B_i + (A_i + B_i) R_{i-1} = G_i + P_i R_{i-1} \quad (5.2)$$

Le definizioni di generazione e propagazione si estendono ai blocchi formati da più bit. Si dice che un blocco genera un riporto se questo produce un riporto di uscita indipendentemente dal valore del riporto di ingresso al blocco. Si dice invece che il blocco propaga un riporto se produce un riporto ogniqualvolta sia presente un riporto di ingresso al blocco. $G_{i:j}$ e $P_{i:j}$ vengono definiti, rispettivamente, come segnali di generazione e di propagazione per i blocchi che vanno dalla colonna i alla colonna j .

Un blocco genera un riporto se la colonna più significativa genera un riporto, oppure se la colonna più significativa propaga un riporto e quella precedente ne genera uno, e così via. Per esempio, la logica di generazione di un blocco che copre dalla colonna 3 alla colonna 0 è

$$G_{3:0} = G_3 + P_3(G_2 + P_2(G_1 + P_1 G_0)) \quad (5.3)$$

Un blocco propaga un riporto se tutte le colonne del blocco propagano un riporto. Per esempio, la logica di propagazione per un blocco che copre le colonne dalla 3 alla 0 è

$$P_{3:0} = P_3 P_2 P_1 P_0 \quad (5.4)$$

Usando i segnali di generazione e propagazione è possibile calcolare velocemente il riporto di uscita del blocco, R_i , usando il riporto di ingresso al blocco, R_{j-1} .

$$R_i = G_{i:j} + P_{i:j} R_{j-1} \quad (5.5)$$

Negli schemi elettrici normalmente i segnali vanno da sinistra a destra. I circuiti aritmetici fanno eccezione perché i riporti si propagano da destra a sinistra (dalla colonna meno significativa verso la colonna più significativa).



Nella storia dell'umanità si sono usati vari strumenti per eseguire calcoli aritmetici. I bambini piccoli contano sulle dita (e anche qualche adulto, magari di nascosto...). Cinesi e Babilonesi hanno inventato l'abaco all'incirca nel 2400 a.C. Il regolo calcolatore, inventato nel 1630, è stato usato fino agli anni '70 quando si sono diffuse le calcolatrici scientifiche tascabili. PC e calcolatrici digitali sono oggi presenti ovunque. Cosa riserva il futuro?

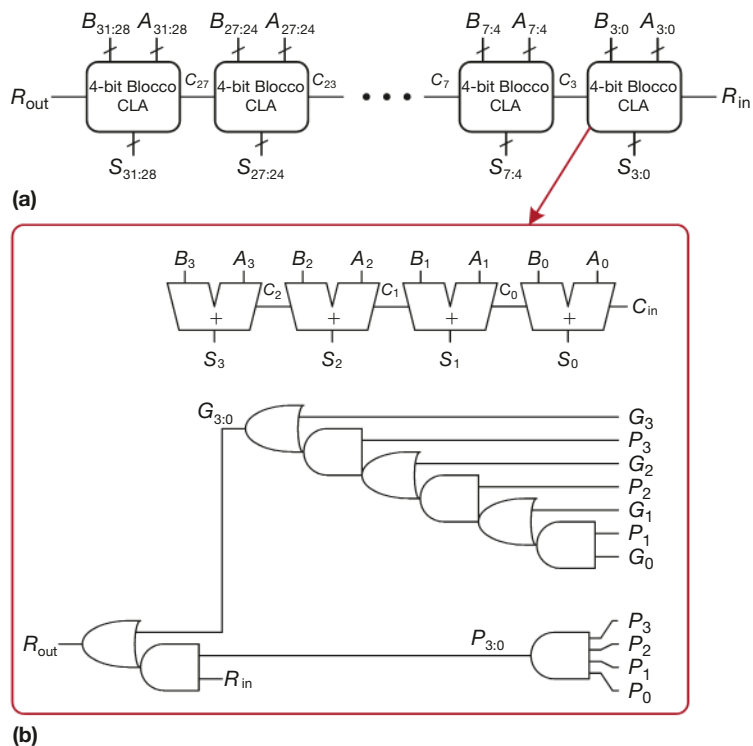
La **Figura 5.6(a)** mostra un sommatore ad anticipazione di riporto a 32 bit composto da otto blocchi ognuno da 4 bit. Ogni blocco contiene un sommatore a propagazione di riporto a onda a 4 bit e la logica di anticipazione (*look-ahead*) per calcolare i riporti di uscita dei blocchi dati i riporti di ingresso, come mostrato nella **Figura 5.6(b)**. Le porte AND e OR necessarie a calcolare i segnali di generazione e di propagazione delle singole colonne, G_i e P_i , a partire da A_i e B_i non sono state inserite nella figura per brevità. Ancora una volta è possibile riconoscere i principi di modularità e regolarità nella realizzazione di un sommatore ad anticipazione di riporto.

Tutti i blocchi del CLA calcolano i segnali di generazione e di propagazione sia di colonna sia di blocco simultaneamente. Il percorso critico della rete inizia con il calcolo di G_0 e $G_{3:0}$ nel primo blocco del sommatore ad anticipazione di riporto. Dopodiché, R_{in} avanza direttamente fino a R_{out} attraverso la porta AND/OR in ogni blocco fino all'ultimo. Quando si ha a che fare con un sommatore di grandi dimensioni, questo metodo risulta decisamente più rapido perché non bisogna attendere che i segnali si propaghino attraverso ognuno dei bit consecutivi del sommatore. Infine, il percorso critico attraverso l'ultimo blocco include un corto sommatore a propagazione di riporto a onda. Quindi, un sommatore a N bit diviso in blocchi da k bit ha un ritardo pari a

$$t_{CLA} = t_{pg} + t_{pg_blocco} + \left(\frac{N}{k} - 1 \right) t_{AND_OR} + k t_{FA} \quad (5.6)$$

dove t_{pg} rappresenta il ritardo delle porte di generazione e di propagazione per calcolare P_i e G_i di ogni colonna (una singola porta AND o OR), t_{pg_blocco} è il ritardo per calcolare i segnali di generazione e di propagazione P_{ij} e G_{ij} per ogni blocco a k bit, e t_{AND_OR} è il ritardo da R_{in} a R_{out} attraverso la porta AND/OR finale del blocco ad anticipazione di riporto a k bit. Per $N > 16$ il sommatore ad anticipazione di riporto è generalmente molto più veloce rispetto

Figura 5.6
(a) Sommatore ad anticipazione di riporto (CLA) a 32 bit, (b) blocco CLA a 4 bit.



al sommatore a propagazione di riporto a onda. Tuttavia, il ritardo aumenta comunque all'aumentare di N .

ESEMPIO 5.1

Il ritardo nei sommatore a propagazione di riporto a onda e ad anticipazione di riporto. Confrontare il ritardo di un sommatore a propagazione di riporto a onda a 32 bit con quello di un sommatore ad anticipazione di riporto sempre a 32 bit, diviso in blocchi da 4 bit ciascuno. Assumere che il ritardo di ogni porta a due ingressi sia uguale a 100 ps e che il ritardo di un full adder sia uguale a 300 ps.

Soluzione Secondo l'Espressione 5.1, il ritardo di propagazione del sommatore a propagazione di riporto a onda a 32 bit è $32 \times 300 \text{ ps} = 9.6 \text{ ns}$.

Il sommatore ad anticipazione di riporto ha $t_{pg} = 100 \text{ ps}$, $t_{pg_blocco} = 6 \times 100 \text{ ps} = 600 \text{ ps}$, e $t_{AND_OR} = 2 \times 100 \text{ ps} = 200 \text{ ps}$. Secondo l'Espressione 5.6, il ritardo di propagazione del sommatore ad anticipazione di riporto a 32 bit è quindi $100 \text{ ps} + 600 \text{ ps} + (32/4 - 1) \times 200 \text{ ps} + (4 \times 300 \text{ ps}) = 3.3 \text{ ns}$, il che significa che questo sommatore è quasi tre volte più veloce del sommatore a propagazione di riporto a onda.

Sommatore a prefissi*

Il **sommatore a prefissi** estende la logica di generazione e di propagazione del sommatore ad anticipazione di riporto per eseguire l'addizione ancora più rapidamente. Per prima cosa, questi sommatore calcolano G e P per coppie di colonne, poi per gruppi di 4 colonne, poi di 8, 16 ecc., fino a che il segnale di generazione di ogni colonna non è noto. Le somme sono calcolate a partire da questi segnali di generazione.

In altre parole, la strategia di un sommatore a prefissi è di calcolare il più rapidamente possibile il segnale di ingresso R_{i-1} per ogni colonna i per poi eseguire la somma, utilizzando l'espressione

$$S_i = (A_i \oplus B_i) \oplus R_{i-1} \quad (5.7)$$

Si definisca la colonna $i = -1$ per contenere R_{in} , quindi $G_{-1} = R_{in}$ e $P_{-1} = 0$. Di conseguenza $R_{i-1} = G_{i-1:-1}$ perché si ha un riporto di uscita dalla colonna $i - 1$ se il blocco che copre le colonne da $i - 1$ a -1 genera un riporto: tale riporto è stato generato dalla colonna $i - 1$ oppure da una colonna precedente e poi propagato. È quindi possibile riscrivere l'espressione 5.7 come

$$S_i = (A_i \oplus B_i) \oplus G_{i-1:-1} \quad (5.8)$$

Lo scopo principale di questo sommatore è calcolare il più velocemente possibile tutti i segnali di generazione dei blocchi: $G_{-1:-1}$, $G_{0:-1}$, $G_{1:-1}$, $G_{2:-1}$, ..., $G_{N-2:-1}$. Questi segnali, insieme ai segnali $P_{-1:-1}$, $P_{0:-1}$, $P_{1:-1}$, $P_{2:-1}$, ..., $P_{N-2:-1}$, vengono chiamati **prefissi**.

La **Figura 5.7** mostra un sommatore a prefissi a $N = 16$ bit. Il sommatore comincia con un pre-calcolo per ottenere P_i e G_i per ogni colonna da A_i e B_i con l'uso delle porte AND o OR. Dopodiché, vengono utilizzati $\log_2 N = 4$ livelli di celle nere per calcolare i prefissi di $G_{i:j}$ e di $P_{i:j}$. Una cella nera riceve gli ingressi dalla parte superiore di un blocco che comprende i bit $i:k$ e dalla parte inferiore che comprende i bit $k - 1:j$. Queste parti vengono combinate per ottenere i segnali di generazione e di propagazione per l'intero blocco che copre i bit $i:j$ usando le seguenti espressioni:

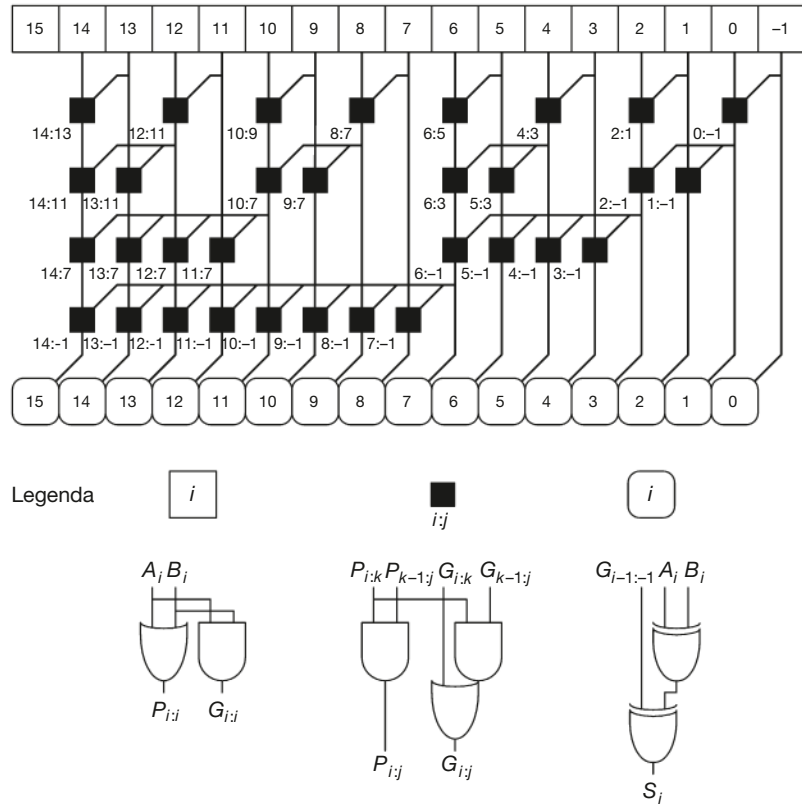
$$G_{i:j} = G_{i:k} + P_{i:k}G_{k-1:j} \quad (5.9)$$

$$P_{i:j} = P_{i:k} + P_{i:k}P_{k-1:j} \quad (5.10)$$

In altre parole, un blocco che comprende i bit $i:j$ genera un riporto se la parte superiore genera un riporto o se la parte superiore propaga un riporto generato dalla parte inferiore. Inoltre, il blocco propaga un riporto se sia la parte

I primi calcolatori usavano sommatore a propagazione di riporto a onda, perché gli elementi circuitali erano costosi e questi sommatore usavano la minore quantità possibile di hardware. Praticamente tutti i PC moderni usano sommatore a prefissi nei percorsi critici, perché i transistori sono oggi molto economici e la velocità di calcolo ha sempre più importanza.

Figura 5.7
Sommatore a prefissi a 16 bit.



superiore sia la parte inferiore propagano un riporto. Infine, il sommatore a prefissi esegue le somme utilizzando l'Espressione 5.8.

Per riassumere, il sommatore a prefissi ha un ritardo che cresce con il numero di colonne nel sommatore in maniera logaritmica piuttosto che lineare. Questa accelerazione è significativa, specialmente per i sommatore a più di 32 bit, ma implica un costo a livello di hardware superiore rispetto a quello di un sommatore ad anticipazione di riporto. La rete di celle nere viene detta **albero dei prefissi**.

Il principio generale di utilizzare alberi di prefissi per eseguire le operazioni in un tempo che cresce in modo logaritmico col numero di ingressi presenti è una tecnica potente. Con un po' di ingegno può essere applicata a molti altri tipi di reti (*vedi* come esempio l'Esercizio 5.7).

Il percorso critico per un sommatore a prefissi di N bit richiede il pre-calcolo di P_i e di G_i , seguito da $\log_2 N$ stadi di celle nere per ottenere tutti i prefissi. Infine, $G_{i-1:-1}$ passa attraverso la porta XOR finale in basso per calcolare S_i . A livello matematico, il ritardo di un sommatore a prefissi a N bit è

$$t_{PA} = t_{pg} + \log_2 N (t_{pg_prefisso}) + t_{XOR} \quad (5.11)$$

dove $t_{pg_prefisso}$ è il ritardo di una cella nera di prefisso.

ESEMPIO 5.2

Ritardo di un sommatore a prefissi. Calcolare il ritardo di un sommatore a prefissi a 32 bit. Assumere che ogni porta a due ingressi abbia un ritardo pari a 100 ps.

Soluzione Il ritardo di propagazione di ogni cella nera di prefisso, $t_{pg_prefisso}$, è di 200 ps (cioè il ritardo di due porte). Quindi il ritardo di propagazione di un sommatore a prefissi a 32 bit è uguale, secondo l'Espressione 5.11, a $100 \text{ ps} + \log_2(32) \times 200 \text{ ps} + 100 \text{ ps} = 1.2 \text{ ns}$, quindi questo sommatore è circa tre volte più veloce del sommatore

ad anticipazione di riporto e otto volte più veloce del sommatore con propagazione a onda di riporto dell'Esempio 5.1. In pratica, i benefici derivanti dall'uso di questo tipo di sommatore non sono poi così grandi, ma ciò non toglie che i sommatore a prefissi siano comunque sostanzialmente più rapidi delle loro alternative.

In conclusione

In questo paragrafo sono stati introdotti il semisommatore (half adder), il sommatore completo (full adder) e tre tipi diversi di sommatore a propagazione di riporto: il sommatore a propagazione di riporto a onda, il sommatore ad anticipazione di riporto e il sommatore a prefissi. I sommatore più veloci richiedono una quantità maggiore di hardware e quindi sono più costosi e consumano più energia. Questi aspetti vanno sempre tenuti presenti quando si sceglie il sommatore adatto al proprio progetto.

Il linguaggio di descrizione hardware prevede l'operazione + per specificare un sommatore a propagazione di riporto. Gli strumenti moderni di sintesi scelgono tra molte differenti possibilità di realizzazione, adottando la soluzione meno cara (e quindi più piccola) in grado di soddisfare i requisiti di velocità. Questo semplifica enormemente il lavoro del progettista. L'**Esempio HDL 5.1** descrive un sommatore a propagazione di riporto con riporto di ingresso e di uscita.

5.2.2 Sottrazione

Come discusso nel paragrafo 1.4.6, i sommatore sono in grado di sommare numeri sia positivi sia negativi utilizzando la rappresentazione dei numeri in complemento a due. La sottrazione è quindi facile quanto l'addizione: si

ESEMPIO HDL 5.1 SOMMATORE

SystemVerilog

```
module sommatore #(parameter N = 8)
    (input  logic [N-1:0] a, b,
     input  logic      rin,
     output logic [N-1:0] s,
     output logic      rout);

    assign {rout, s} = a + b + rin;

endmodule
```

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;
use IEEE.NUMERIC_STD_UNSIGNED.ALL;

entity sommatore is
    generic(N: integer := 8);
    port(a, b: in  STD_LOGIC_VECTOR(N-1 downto 0);
         rin: in  STD_LOGIC;
         s: out STD_LOGIC_VECTOR(N-1 downto 0);
         rout: out STD_LOGIC);
end;

architecture sintesi of sommatore is
    signal risultato: STD_LOGIC_VECTOR(N downto 0);
begin
    risultato <= ("0" & a) + ("0" & b) + rin;
    s <= risultato(N-1 downto 0);
    rout <= risultato(N);
end;
```

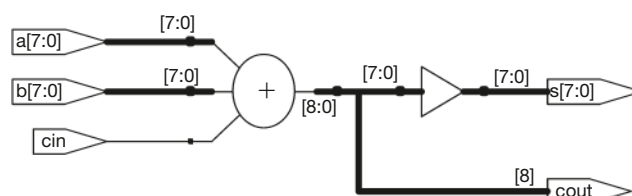


Figura 5.8 Sintesi del sommatore.

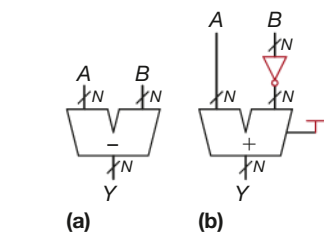


Figura 5.9
Sottrattore: (a) simbolo,
(b) realizzazione.

inverte il segno del secondo numero e poi si esegue la somma. Il cambio di segno di un numero in complemento a due si esegue negando tutti i bit e aggiungendo un 1.

Per calcolare $Y = A - B$, per prima cosa si crea il numero in complemento a due di B : si negano tutti i bit di B per ottenere $\overline{B}$ e si aggiunge 1 per ottenere $-B = \overline{B} + 1$. Questo valore viene aggiunto ad A per ottenere $Y = A + \overline{B} + 1 = A - B$. Questo risultato può essere ottenuto con l'utilizzo di un sommatore a propagazione di riporto facendo la somma $A + \overline{B}$ con $R_{in} = 1$. La **Figura 5.9** mostra il simbolo circuitale di un sottrattore e la relativa struttura hardware che esegue $Y = A - B$. L'**Esempio HDL 5.2** descrive un sottrattore.

ESEMPIO HDL 5.2 SOTTRATTORE

SystemVerilog	VHDL
<pre>module sottrattore #(parameter N = 8) (input logic [N-1:0] a, b, output logic [N-1:0] y); assign y = a - b; endmodule</pre>	<pre>library IEEE; use IEEE.STD_LOGIC_1164.all; use IEEE.NUMERIC_STD_UNSIGNED.ALL; entity sottrattore is generic(N: integer := 8); port(a, b: in STD_LOGIC_VECTOR(N-1 downto 0); y: out STD_LOGIC_VECTOR(N-1 downto 0)); end; architecture sintesi of sottrattore is begin y <= a - b; end;</pre>

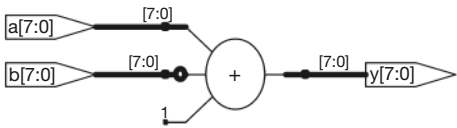


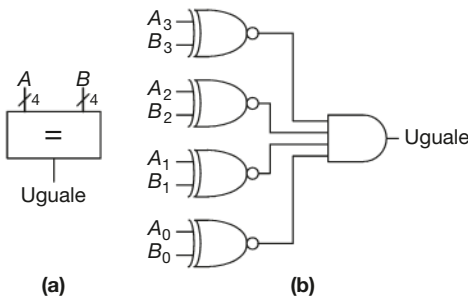
Figura 5.10 Sintesi del sottrattore.

5.2.3 Comparatori

Un **comparatore** determina se due numeri binari sono uguali o se uno dei due è maggiore o minore dell'altro. Un comparatore riceve due numeri binari a N bit, A e B . Esistono due tipi comuni di comparatori.

Un **comparatore di uguaglianza** produce una singola uscita che indica se A è uguale a B ($A == B$) oppure no. Un **comparatore di valore** produce invece una o più uscite che indicano i valori relativi di A e di B . Il comparatore di uguaglianza è naturalmente più semplice a livello hardware. La **Figura 5.11** mostra il simbolo e la realizzazione di un comparatore di uguaglianza a 4 bit. Per prima cosa, il comparatore determina se i bit corrispondenti a ogni colon-

Figura 5.11
Comparatore di uguaglianza a 4 bit:
(a) simbolo, (b) realizzazione.



na A e B sono uguali utilizzando delle porte XNOR. I due numeri sono uguali se tutte le colonne sono uguali.

La comparazione di valore dei numeri con segno viene solitamente effettuata calcolando $A - B$ e guardando il segno (cioè il bit più significativo) del risultato dell'operazione, come mostra la **Figura 5.12**. Se il risultato è negativo (cioè se il bit del segno è uguale a 1) allora A è minore di B . Al contrario, se il risultato è positivo, A è maggiore o uguale a B . Tuttavia, questo comparatore non lavora correttamente in caso di traboccamento (*overflow*). Gli Esercizi 5.9 e 5.10 esaminano in dettaglio questo problema e ne propongono una soluzione.

L'**Esempio HDL 5.3** mostra come usare varie operazioni di comparazione per numeri senza segno.

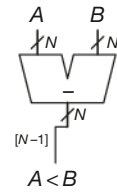


Figura 5.12
Comparatore per numeri con segno a N bit.

5.2.4 ALU

Un'**unità logica/aritmetica** (ALU, *Arithmetic/Logical Unit*) unisce all'interno di una singola unità una serie di operazioni logiche e matematiche. Per

ESEMPIO HDL 5.3 COMPARATORI

SystemVerilog

```
module comparatore #(parameter N = 8)
    (input logic [N-1:0] a, b,
     output logic ugu, div, min, miu,
     mag, mau);
    assign ugu = (a == b);
    assign div = (a != b);
    assign min = (a < b);
    assign miu = (a <= b);
    assign mag = (a > b);
    assign mau = (a >= b);
endmodule
```

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.ALL;
entity comparatore is
    generic(N: integer := 8);
    port(a, b: in STD_LOGIC_VECTOR(N-1 downto 0);
         ugu, div, min, miu, mag, mau: out STD_LOGIC);
end;
architecture sintesi of comparatore is
begin
    ugu <= '1' when (a = b) else '0';
    div <= '1' when (a /= b) else '0';
    min <= '1' when (a < b) else '0';
    miu <= '1' when (a <= b) else '0';
    mag <= '1' when (a > b) else '0';
    mau <= '1' when (a >= b) else '0';
end;
```

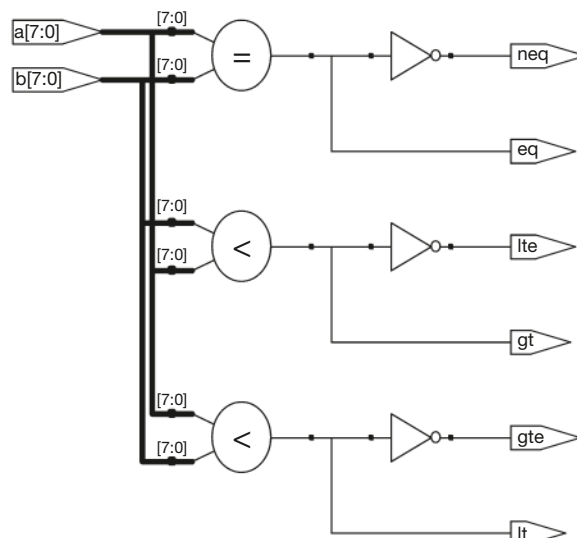


Figura 5.13 Sintesi dei comparatori.

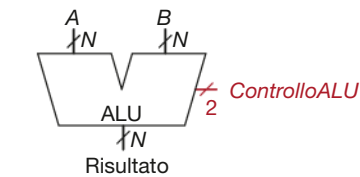


Figura 5.14
Simbolo dell'ALU.

esempio, una ALU tipica è in grado di eseguire le operazioni di addizione, sottrazione, AND e OR logici bit a bit. L'ALU è il cuore della maggior parte dei calcolatori.

La **Figura 5.14** mostra il simbolo di una ALU a N bit con ingressi e uscite di N bit. La ALU riceve un segnale di controllo a 2 bit, chiamato *ControlloALU*, che specifica quale funzione debba eseguire. I segnali di controllo vengono indicati in rosso per distinguerli dai dati. La **Tabella 5.1** elenca le funzioni tipiche che una ALU può eseguire.

La **Figura 5.15** mostra la realizzazione dell'ALU. La ALU contiene un sommatore a N bit e un numero N di porte AND o OR a due ingressi. Con-

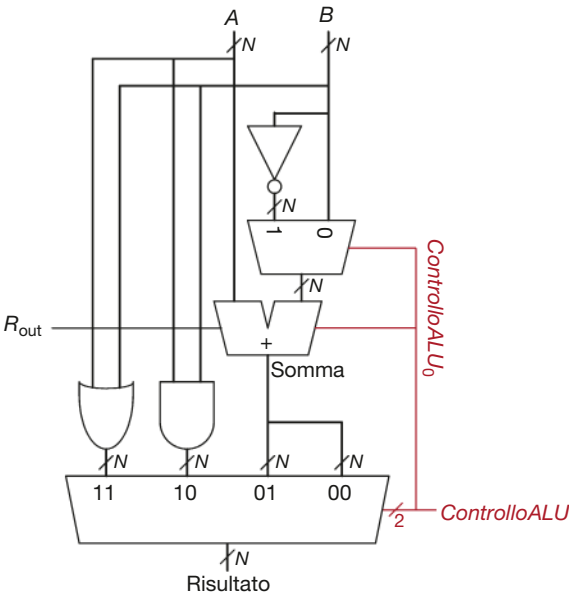
Tabella 5.1 Operazioni dell'ALU.

ControlloALU _{1:0}	Funzione
00	Addizione
01	Sottrazione
10	AND
11	OR

tiene inoltre dei negatori e un multiplexer per invertire l'ingresso B quando il segnale di controllo $ControlloALU_0$ è attivato. Un multiplexer 4:1 sceglie l'operazione desiderata sulla base di $ControlloALU$. Più specificatamente, se $ControlloALU = 00$, il multiplexer di uscita sceglie $A + B$. Se invece $ControlloALU = 01$, la ALU calcola $A - B$ (come discusso nel paragrafo 5.2.2, $\bar{B} + 1 = -B$ nell'aritmetica in complemento a due; dal momento che $ControlloALU_0$ è uguale a 1, il sommatore riceve gli ingressi A e $\bar{B}$ e un riporto di ingresso a 1, il che fa sì che il sommatore esegua la sottrazione: $A + \bar{B} + 1 = A - B$). Se $ControlloALU = 10$, l'ALU esegue $A \text{ AND } B$. Se invece $ControlloALU = 11$, l'ALU esegue $A \text{ OR } B$.

Alcune ALU producono uscite ulteriori, chiamate *flag* (bandiere), che danno informazioni aggiuntive sul risultato dell'ALU. La **Figura 5.16** mo-

Figura 5.15
ALU a N bit.



stra il simbolo di una ALU con un'uscita $Flag_{ALU}$ a 4 bit. Come mostrato nello schema dell'ALU nella **Figura 5.17**, l'uscita $Flag_{ALU}$ è composta dalle flag N , Z , C e V che indicano, rispettivamente, che il risultato dell'ALU è negativo (*Negative*) o uguale a zero (*Zero*) o che il sommatore ha generato un riporto (*Carry*) o un traboccamento (*oVerflow*). Il lettore ricordi che il bit più significativo di un numero in complemento a due è 1 se il numero è negativo e 0 in caso contrario. Quindi, la flag N è connessa al bit più significativo del risultato dell'ALU, cioè $Risultato_{31}$. La flag Z si attiva quando tutti i bit del risultato ($Risultato$) sono uguali a 0, come rileva la porta NOR a N bit in Figura 5.17. La flag C si attiva quando il sommatore produce un riporto e l'ALU sta eseguendo una somma o una sottrazione ($Controllo_{ALU_1} = 0$).

L'identificazione di un traboccamento, come mostrato nel lato sinistro della Figura 5.17, è un po' più complessa. Nel paragrafo 1.4.6 è stato spiegato che un traboccamento si verifica quando la somma di due numeri di egual segno produce come risultato un numero di segno opposto. Di conseguenza, V è attiva nel caso in cui si verificano le tre seguenti condizioni: (1) la ALU sta eseguendo una somma o una sottrazione ($Controllo_{ALU_1} = 0$), (2) A e $Somma$ hanno segni opposti, come identificato dalla porta XOR e (3) come identificato dalla porta XNOR, o A e B hanno lo stesso segno e il sommatore sta seguendo un'addizione ($Controllo_{ALU_0} = 0$), oppure A e B hanno segno opposto e il sommatore sta eseguendo una sottrazione ($Controllo_{ALU_0} = 1$). La porta AND a tre ingressi riconosce quando tutte e tre le condizioni si avverano e quindi attiva V .

La descrizione HDL per una ALU a N bit con delle flag di uscita è oggetto degli Esercizi 5.11 e 5.12. Esistono diverse varianti di questa versione base di ALU in grado di eseguire altre operazioni, come XOR bit a bit o comparazione di uguaglianza.

5.2.5 Traslatori e rotatori

I **traslatori** (*shifter*) e i **rotatori** (*rotator*) traslano i bit ed eseguono la moltiplicazione o la divisione per potenze di 2. Come suggerisce il nome, i traslatori traslano un numero binario a destra o a sinistra di uno specifico numero di posizioni. Esistono diversi tipi di traslatori usati comunemente:

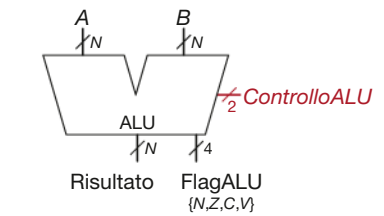
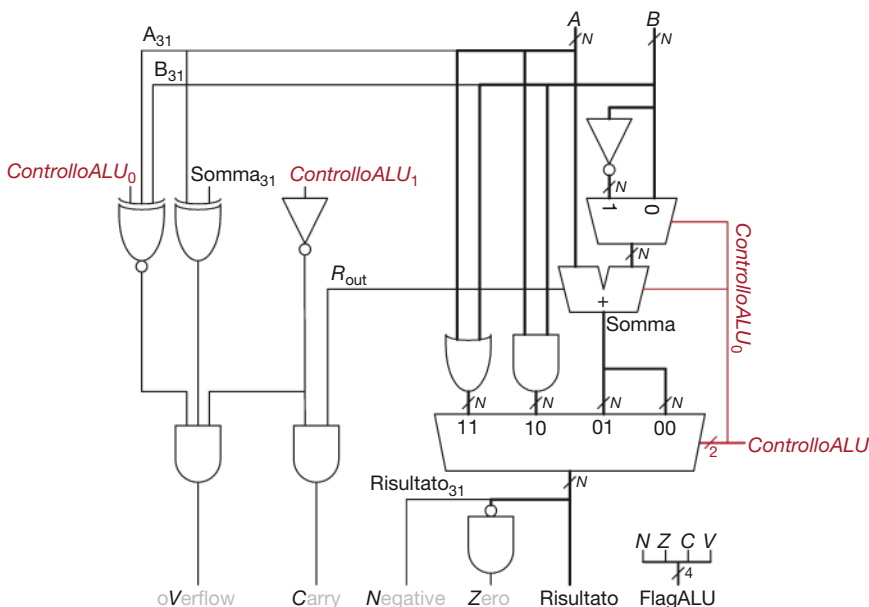


Figura 5.16
Simbolo dell'ALU con le flag di uscita.

Figura 5.17
ALU a N bit con i flag di uscita.

- **Traslatore logico:** trasla un numero verso sinistra (LSL, *Logical Shift Left*) o verso destra (LSR, *Logical Shift Right*) e riempie gli spazi lasciati vuoti con 0.

Esempi: 11001 LSR 2 = 00110; 11001 LSL 2 = 00100

- **Traslatore aritmetico:** esegue la stessa funzione di un traslatore logico, ma quando trasla un numero verso destra (ARS, *Arithmetic Shift Right*), riempie i bit più significativi con una copia del precedente bit più significativo (msb, *most significant bit*). Questa funzione è utile per quando è necessario moltiplicare o dividere numeri con segno (vedi parr. 5.2.6 e 5.2.7). La traslazione aritmetica verso sinistra (ASL, *Arithmetic Shift Left*) si comporta come la traslazione logica verso sinistra.

Esempi: 11001 ASR 2 = 11110; 11001 ASL 2 = 00100

- **Rotatore:** trasla un numero verso sinistra (ROL, *Rotate Left*) o verso destra (ROR, *Rotate Right*) circolarmente, in modo che gli spazi lasciati vuoti vengano riempiti dai bit all'estremità opposta del numero.

Esempi: 11001 ROR 2 = 01110; 11001 ROL 2 = 00111

Un traslatore a N bit può essere costruito con un numero N di multiplexer $N:1$. L'ingresso viene traslato da 0 a $N - 1$ posizioni, a seconda dei valori presenti sulle $\log_2 N$ linee di selezione. La **Figura 5.18** mostra il simbolo e la struttura hardware dei traslatori a 4 bit. Gli operatori $\ll$, $\gg$ e $\ggg$ stanno a significare rispettivamente una traslazione verso sinistra, una traslazione logica verso destra, e una traslazione aritmetica verso destra. A seconda del valore di traslazione a 2 bit $trasl_{1:0}$, l'uscita Y corrisponde all'ingresso A traslato da 0 fino a 3 bit. In tutti i traslatori, quando $trasl_{1:0} = 00$, $Y = A$. L'Esercizio 5.18 illustra il progetto dei rotatori.

Una traslazione verso sinistra rappresenta un caso particolare di moltiplicazione. Una traslazione verso sinistra di N bit, infatti, moltiplica il numero per 2^N . Per esempio, $000011_2 \ll 4 = 110000_2$ è uguale a $3_{10} \times 2^4 = 48_{10}$.

Una traslazione aritmetica verso destra, invece, rappresenta un caso particolare di divisione. Una traslazione aritmetica verso destra di N bit divide il numero per 2^N . Per esempio, $11100_2 \ggg 2 = 11111_2$ è uguale a $-4_{10}/2^2 = -1_{10}$.

5.2.6 Moltiplicazione*

La moltiplicazione tra numeri binari senza segno è simile alla moltiplicazione decimale ma con solo uni e zeri. La **Figura 5.19** confronta la moltiplicazione di numeri decimali e binari. In entrambi i casi vengono formati dei **prodotti parziali** moltiplicando una singola cifra del moltiplicatore per tutte le cifre del moltiplicando. I prodotti parziali traslati vengono poi sommati per ottenere il risultato finale.

In generale un moltiplicatore $N \times N$ moltiplica due numeri a N bit e produce un risultato a $2N$ bit. I prodotti parziali nella moltiplicazione binaria corrispondono o al moltiplicando o a tutti 0. La moltiplicazione tra numeri binari a 1 bit è equivalente a un'operazione AND, quindi vengono utilizzate porte AND per formare i prodotti parziali.

Le moltiplicazioni con segno e senza segno sono diverse. Per esempio, si consideri $0xFE \times 0xFD$. Se questi numeri a 8 bit vengono interpretati come numeri interi con segno, rappresentano i numeri -2 e -3 , quindi il prodotto a 16 bit è $0x0006$. Se invece questi numeri vengono interpretati come numeri interi senza segno, il prodotto a 16 bit diventa $0xFB06$. Si noti che in entrambi i casi, il byte meno significativo è $0x06$.

La **Figura 5.20** mostra il simbolo, la funzione e la realizzazione di un mol-

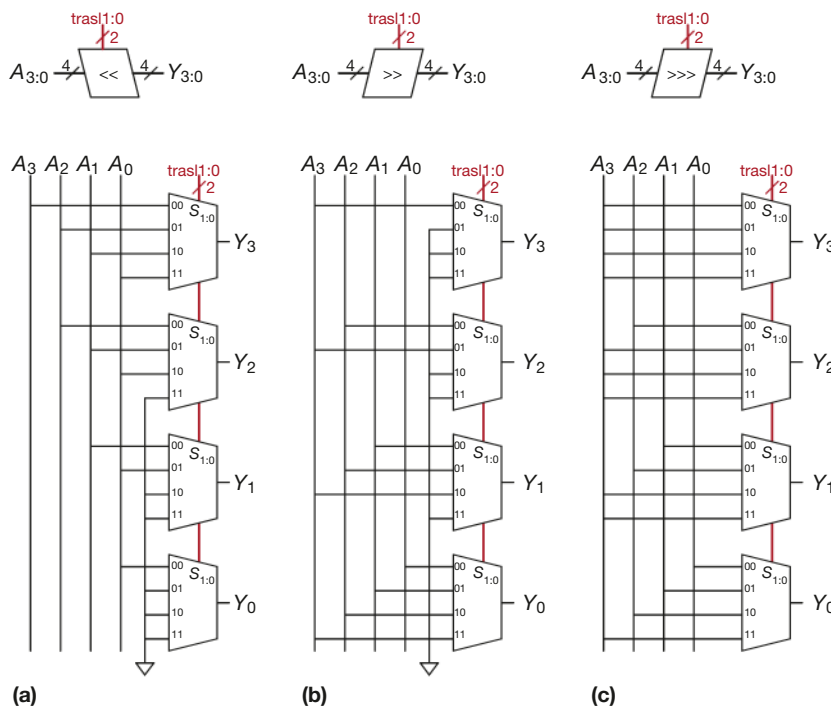


Figura 5.18
Traslatori a 4 bit: (a) a sinistra, (b) a destra logico, (c) a destra aritmetico.

$\begin{array}{r} 230 \\ \times 42 \\ \hline 460 \\ +920 \\ \hline 9660 \end{array}$	moltiplicando moltiplicatore prodotti parziali risultato	$\begin{array}{r} 0101 \\ \times 0111 \\ \hline 0101 \\ 0101 \\ 0101 \\ +0000 \\ \hline 0100011 \end{array}$
$230 \times 42 = 9660$		$5 \times 7 = 35$
(a)		(b)

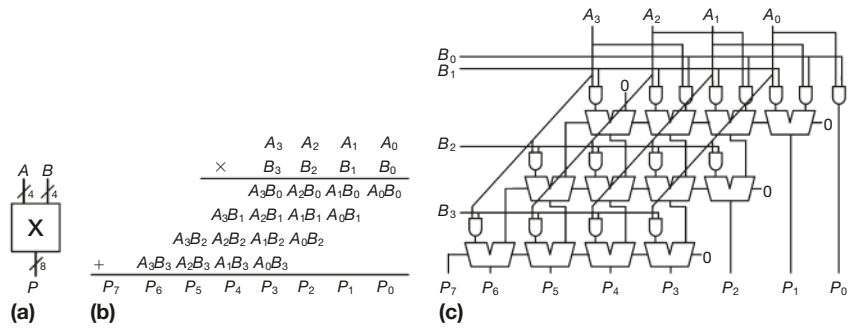
Figura 5.19
Moltiplicazione: (a) decimale, (b) binaria.

tiplicatore 4×4 senza segno. Il moltiplicatore senza segno riceve il moltiplicando e il moltiplicatore, A e B , e produce il prodotto P . La **Figura 5.20(b)** mostra come vengono formati i prodotti parziali. Ogni prodotto parziale è un singolo bit del moltiplicatore (B_3, B_2, B_1 , o B_0) AND i bit del moltiplicando (A_3, A_2, A_1, A_0). Con degli operandi a N bit ci sono N prodotti parziali e $N - 1$ stadi di sommatore a 1 bit. Per esempio, per un moltiplicatore 4×4 , il prodotto parziale della prima riga è B_0 AND (A_3, A_2, A_1, A_0). Questo prodotto parziale viene sommato al secondo prodotto parziale traslato, B_1 AND (A_3, A_2, A_1, A_0). Le righe seguenti di porte AND e di sommatore formano e sommano i prodotti parziali restanti.

La descrizione HDL per i moltiplicatori con segno e senza segno è riportata nell'Esempio HDL 4.33. Come per i sommatore, esistono molte possibili strutture di moltiplicatori, con diversi compromessi tra velocità e costo. Gli strumenti di sintesi sono in grado di selezionare la soluzione più appropriata dati i limiti di tempo.

Un'operazione **moltiplica e accumula** (MAC, *Multiply ACCumulate*) è in grado di moltiplicare due numeri e poi sommarli a un terzo numero, tipicamente il valore accumulato. Viene spesso utilizzata in algoritmi di **elaborazione di segnali digitali** (DSP, *Digital Signal Processing*) come la trasformata di Fourier, che necessita di una somma di prodotti.

Figura 5.20
Moltiplicatore 4×4 : (a) simbolo,
(b) funzione, (c) realizzazione.



5.2.7 Divisione*

La divisione binaria può essere effettuata utilizzando l'algoritmo seguente per i numeri senza segno a N bit nell'intervallo $[0, 2^{N-1}]$:

```

R' = 0
for i = N-1 to 0
  R = {R' << 1, Ai}
  D = R - B
  if D < 0 then   Qi = 0, R' = R   // R < B
  else           Qi = 1, R' = D   // R ≥ B
R = R'

```

Il **resto parziale** R viene inizializzato a 0 ($R' = 0$) e il bit più significativo del dividendo A diventa il bit meno significativo di R ($R = \{R' << 1, A_i\}$). Il divisore B viene sottratto da questo resto parziale per determinare se viene soddisfatto ($D = R - B$). Se la differenza D è negativa (e cioè, se il segno del bit D è 1), allora il bit quoziente Q_i è 0 e la differenza viene scartata. Altrimenti, Q_i è uguale a 1 e il resto parziale viene aggiornato per diventare la differenza. In ogni caso, il resto parziale viene successivamente raddoppiato (e quindi spostato a sinistra di una colonna), il bit più significativo di A diventa il bit meno significativo di R e il processo si ripete. Il risultato finale soddisfa la relazione $\frac{A}{B} = Q + \frac{R}{B}$.

La **Figura 5.21** mostra lo schema di un divisore a matrice a 4 bit. Il divisore calcola A/B e produce un quoziente Q e un resto R . La legenda mostra il simbolo e lo schema per ognuno dei blocchi presenti nel divisore. Ogni riga esegue un'iterazione dell'algoritmo di divisione. Nello specifico, ogni riga calcola la differenza $D = R - B$ (si ricordi che $R + \overline{B} + 1 = R - B$). Il segnale N indica se D è negativo o meno. Quindi, le linee di selezione di un multiplexer di riga ricevono il bit più significativo di D , che è uguale a 1 quando la differenza è negativa. Il quoziente (Q_i) è uguale a 0 quando D è negativo, altrimenti 1. Il multiplexer trasmette R alla riga successiva se la differenza è negativa, altrimenti trasmette D . La riga successiva trasla il nuovo resto parziale verso sinistra di un bit, appende il successivo bit più significativo di A e ripete il processo.

Il ritardo di un divisore a N bit cresce in maniera proporzionale a N^2 perché il riporto deve propagarsi attraverso tutti gli N stadi di una riga prima che venga determinato il segno e che il multiplexer selezioni R o D . Questo processo viene ripetuto per tutte le N righe. La divisione è un'operazione lenta e dispendiosa a livello hardware e deve quindi essere utilizzata solo se strettamente necessario.

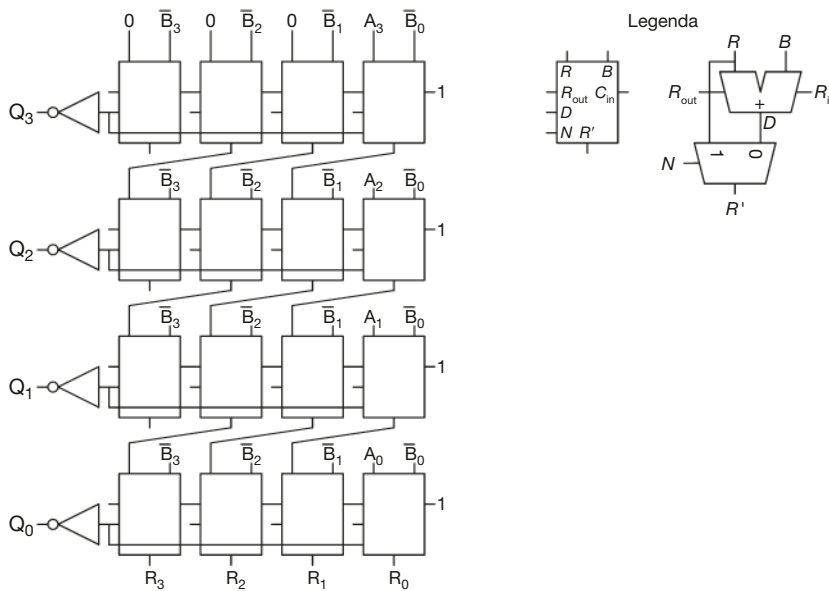


Figura 5.21
Divisore a matrice.

5.2.8 Letture aggiuntive

L'aritmetica dei calcolatori può costituire l'argomento di un intero libro. *Digital Arithmetic* di Ercegovac e Lang tratta in maniera esaustiva tale argomento. *CMOS VLSI Design* di Weste e Harris affronta invece il progetto di reti ad alte prestazioni per le operazioni aritmetiche.

5.3 ■ SISTEMI DI NUMERAZIONE

I calcolatori operano sia con numeri interi sia con numeri frazionari. Sinora sono stati considerati solo numeri interi con segno e senza segno, come introdotto nel paragrafo 1.4. Questo paragrafo introduce invece i sistemi di numerazione in virgola fissa e in virgola mobile, che sono in grado di rappresentare i numeri razionali. I numeri in virgola fissa sono uguali ai numeri decimali: alcuni bit rappresentano la parte intera e i bit rimanenti rappresentano la parte frazionaria. I numeri in virgola mobile sono invece equivalenti ai numeri in notazione scientifica, con una mantissa e un esponente.

5.3.1 Numeri in virgola fissa

La **notazione in virgola fissa** si basa su una **virgola fissa** implicita tra i bit della parte intera e quelli della parte frazionaria, analoga al punto decimale tra le cifre intere e le cifre frazionarie di un normale numero decimale. Per esempio, la **Figura 5.22(a)** mostra un numero in virgola fissa con quattro bit interi e quattro bit frazionari. La **Figura 5.22(b)** mostra la virgola binaria implicita, evidenziata in rosso, mentre la **Figura 5.22(c)** mostra il valore decimale equivalente.

I numeri in virgola fissa con segno possono essere rappresentati sia in complemento a due sia in modulo e segno. La **Figura 5.23** mostra la rappresentazione in virgola fissa di -2.375 usando entrambe le rappresentazioni con quattro bit interi e quattro bit di frazione. La virgola binaria implicita è stata nuovamente evidenziata in rosso. Nella rappresentazione in modulo e segno, il bit più significativo viene utilizzato per indicare il segno. Invece, la rappresentazione in complemento a due si ottiene negando i bit del valore assoluto e aggiungendo un 1 al bit meno significativo (cioè al bit più a destra). In questo caso, la posizione del bit meno significativo è la colonna 2^{-4} .

- (a) 01101100
- (b) 0110.1100
- (c) $2^2 + 2^1 + 2^{-1} + 2^{-2} = 6.75$

Figura 5.22
Rappresentazione in virgola fissa di 6.75 con quattro bit interi e quattro bit frazionari.

- (a) 0010.0110
- (b) 1010.0110
- (c) 1101.1010

Figura 5.23
Rappresentazione in virgola fissa di -2.375 : (a) valore assoluto, (b) modulo e segno, (c) complemento a due.

Come tutte le rappresentazioni di numeri binari, i numeri in virgola fissa sono solo una serie di bit. Non c'è modo di individuare l'esistenza della virgola binaria se non attraverso un accordo preventivo delle persone che devono interpretare il numero.

ESEMPIO 5.3

Aritmetica coi numeri in virgola fissa. Calcolare $0.75 + -0.625$ usando i numeri in virgola fissa.

Soluzione Per prima cosa serve convertire 0.625 , il valore assoluto del secondo numero, in notazione binaria in virgola fissa. $0.625 \geq 2^{-1}$, quindi si deve mettere un 1 nella colonna 2^{-1} ; rimane $0.625 - 0.5 = 0.125$. Dal momento che $0.125 < 2^{-2}$, nella colonna 2^{-2} si deve mettere uno 0. Dal momento che $0.125 \geq 2^{-3}$, nella colonna 2^{-3} si deve mettere un 1; rimane $0.125 - 0.125 = 0$. Quindi, deve per forza esserci uno 0 nella colonna 2^{-4} . Unendo il tutto, $0.625_{10} = 0000.1010_2$.

Per poter effettuare correttamente la somma, si deve adottare la rappresentazione in complemento a due dei numeri con segno. La Figura 5.24 mostra la conversione di -0.625 in complemento a due in virgola fissa. La Figura 5.25 mostra l'addizione binaria in virgola fissa confrontata con e l'equivalente decimale. Si noti che l'1 più significativo nell'addizione binaria in virgola fissa della Figura 5.25(a) viene eliminato dal risultato a 8 bit.

I numeri in virgola fissa sono frequentemente usati in applicazioni bancarie e finanziarie, dove è necessaria precisione con un intervallo di variabilità non molto grande. Anche le applicazioni di elaborazione di segnali digitali (DSP, *Digital Signal Processing*) usano spesso numeri in virgola fissa perché i calcoli sono più rapidi e consumano meno potenza di quanto servirebbe con i numeri in virgola mobile.

Figura 5.24
 Conversione da virgola fissa a complemento a due.

0000.1010	Valore Assoluto Binario
1111.0101	Complemento a uno
+ 1	Sommare 1
1111.0110	Complemento a due

Figura 5.25
 Somma: (a) binaria in virgola fissa, (b) equivalente in decimale.

0000.1100	0.75
+ 1111.0110	+ (-0.625)
10000.0010	0.125
(a)	(b)

5.3.2 Numeri in virgola mobile*

I numeri in virgola mobile sono equivalenti alla notazione scientifica. Superano la limitazione di avere un numero costante di bit interi e frazionari, permettendo quindi la rappresentazione di numeri molto piccoli ma anche molto grandi. Come la notazione scientifica, anche i numeri in virgola mobile hanno un **segno**, una **mantissa** (M), una **base** (B) e un **esponente** (E), come mostra la Figura 5.26. Per esempio, il numero 4.1×10^3 è la notazione scientifica del numero decimale 4100: ha una mantissa 4.1, una base 10 e un esponente 3. Il punto decimale è mobile perché viene posizionato immediatamente a destra della cifra più significativa. I numeri in virgola mobile sono in base 2 con una mantissa binaria: vengono usati 32 bit per rappresentare 1 bit di segno, 8 bit di esponente e 23 bit di mantissa.

$$\pm M \times B^E$$

Figura 5.26
 Numeri in virgola mobile.

ESEMPIO 5.4

Numeri in virgola mobile a 32 bit. Scrivere la rappresentazione in virgola mobile del numero decimale 228.

Soluzione Per prima cosa serve convertire il numero decimale in numero binario: $228_{10} = 11100100_2 = 1.11001_2 \times 2^7$. La Figura 5.27 mostra la codifica a 32 bit, modificata nel seguito per ragioni di efficienza. Il bit di segno è positivo (0), gli 8 bit dell'esponente danno un valore 7 e i restanti 23 bit sono i bit della mantissa.

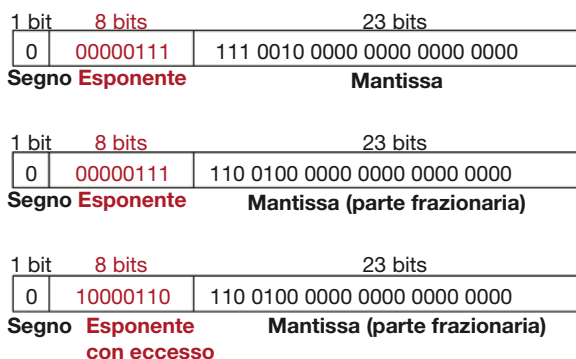


Figura 5.27
Numeri in virgola mobile a 32 bit, versione 1.

Figura 5.28
Numeri in virgola mobile a 32 bit, versione 2.

Figura 5.29
Notazione in virgola mobile IEEE 754.

Nei numeri binari in virgola mobile, il primo bit della mantissa (posizionato alla sinistra della virgola) è sempre uguale a 1 e quindi non ha bisogno di essere memorizzato. Questo bit viene chiamato **uno più significativo implicito**. La **Figura 5.28** mostra la rappresentazione in virgola mobile modificata di $228_{10} = 11100100_2 \times 2^0 = 1.11001_2 \times 2^7$. L'uno più significativo implicito non viene riportato nei 23 bit della mantissa per questioni di efficienza. Solo i bit frazionari vengono riportati, così da liberare il posto per un ulteriore bit di dato utile.

Si esegue infine un'ultima modifica a livello dell'esponente. L'esponente deve essere in grado di rappresentare valori sia positivi sia negativi. Per fare ciò, la virgola mobile utilizza un esponente con **codifica a eccesso**, costituito dall'esponente originale più un eccesso costante. La rappresentazione in virgola mobile a 32 bit utilizza l'eccesso 127. Per esempio, per un valore pari a 7, l'esponente a eccesso è $7 + 127 = 134 = 10000110_2$. Per un valore pari a -4 , l'esponente a eccesso è $-4 + 127 = 123 = 01111011_2$. La **Figura 5.29** mostra $1.11001_2 \times 2^7$ rappresentato in notazione in virgola mobile con l'uno più significativo implicito e l'esponente a eccesso 134 ($7 + 127$). Questa notazione è conforme allo standard IEEE 754 di rappresentazione dei numeri in virgola mobile.

Casi speciali: 0, $\pm\infty$ e NaN

Lo standard IEEE 754 di rappresentazione dei numeri in virgola mobile prevede codici speciali per rappresentare numeri come lo 0, l'infinito e i valori impossibili. Per esempio, risulta problematica la rappresentazione dello 0 in virgola mobile a causa dell'uno più significativo implicito. Vengono usati per questi casi particolari dei codici speciali con esponenti di tutti 0 o di tutti 1. La **Tabella 5.2** mostra le rappresentazioni in virgola mobile di 0, $\pm\infty$ e NaN (*Not a Number*). Come per i numeri in modulo e segno, la rappresentazione in virgola mobile ha sia lo 0 positivo sia lo 0 negativo. NaN viene usato per i numeri che non esistono, come ad esempio $\sqrt{-1}$ o $\log_2(-5)$.

Formati a precisione singola e doppia

Sono stati finora esaminati i numeri in virgola mobile a 32 bit. Questo formato viene anche chiamato a **singola precisione**, o *float*. Lo standard IEEE

Come abbastanza evidente, ci sono varie ragioni per rappresentare i numeri in virgola mobile. Per molti anni, i costruttori di calcolatori hanno usato formati in virgola mobile incompatibili tra loro. I risultati prodotti da un calcolatore non potevano essere interpretati direttamente da un altro calcolatore.

L'*Institute of Electrical and Electronics Engineers* ha risolto il problema definendo nel 1975 lo *standard IEEE 754 per numeri in virgola mobile*. Questo formato è oggi usato universalmente, ed è l'oggetto di questo paragrafo del testo.

In virgola mobile non si possono rappresentare esattamente alcuni numeri, per esempio 1.7. Tuttavia, se si inserisce 1.7 nella calcolatrice, questo viene visualizzato 1.7 e non 1.69999... Per consentire questo comportamento, alcune applicazioni come le calcolatrici o i programmi finanziari utilizzano numeri decimali codificati in binario (BCD, *Binary Coded Decimal*) con un esponente in base 10 invece di 2. I numeri BCD codificano ogni cifra decimale con quattro bit nell'intervallo da 0 a 9. Per esempio, la notazione BCD in virgola fissa di 1.7 con quattro bit per la parte intera e quattro per quella decimale sarebbe 0001.0111. Naturalmente questa scelta non è priva di costi, in termini di complessità dei circuiti aritmetici e di configurazioni binarie inutilizzate (le codifiche delle cifre esadecimali A-F non vengono infatti usate), quindi di prestazioni ridotte. Per applicazioni che richiedano molti calcoli, la notazione in virgola mobile è molto più veloce.

Tabella 5.2 Rappresentazioni in virgola mobile IEEE 754 di 0, $\pm\infty$ e NaN.

Numero	Segno	Esponente	Mantissa
0	X	00000000	000000000000000000000000
∞	0	11111111	000000000000000000000000
$-\infty$	1	11111111	000000000000000000000000
NaN	X	11111111	Non nulla

Tabella 5.3 Formati in virgola mobile in singola e doppia precisione.

Formato	Bit totali	Bit di segno	Bit di esponente	Bit di mantissa
Singola precisione	32	1	8	23
Doppia precisione	64	1	11	52

754 definisce anche i numeri a 64 bit come numeri a **doppia precisione** (o *double*) cioè numeri che hanno una maggiore precisione e un intervallo maggiore. La **Tabella 5.3** mostra la quantità di bit usati nei campi di ogni formato.

Escludendo i casi speciali appena visti, i numeri normali a singola precisione coprono un intervallo da $\pm 1.175494 \times 10^{-38}$ a $\pm 3.402824 \times 10^{38}$. Hanno una precisione di circa sette cifre decimali significative (perché $2^{-24} \approx 10^{-7}$). I numeri normali a doppia precisione coprono un intervallo da $\pm 2.22507385850720 \times 10^{-308}$ a $\pm 1.79769313486232 \times 10^{308}$ e hanno una precisione di circa 15 cifre decimali significative.

Arrotondamento

I risultati aritmetici che superano la precisione disponibile devono essere arrotondati a un numero vicino. I metodi di arrotondamento sono: arrotondamento per difetto, arrotondamento per eccesso, arrotondamento verso lo zero e arrotondamento al numero più vicino. Il metodo di arrotondamento utilizzato per default è l'arrotondamento al numero più vicino. In questa modalità, se due numeri sono egualmente vicini, viene scelto quello dei due con uno 0 nella posizione meno significativa della mantissa.

Si genera un **traboccamento per eccesso** (*overflow*) quando un numero è troppo grande per essere rappresentato. Analogamente, si genera un **traboccamento per difetto** (*underflow*) quando un numero è troppo piccolo per essere rappresentato. Nella modalità di arrotondamento al numero più vicino, nel primo caso il numero viene arrotondato a $\pm\infty$, nel secondo viene arrotondato a 0.

Addizione in virgola mobile

L'addizione di numeri in virgola mobile non è semplice come l'addizione tra numeri in complemento a due. I passaggi per sommare due numeri in virgola mobile con lo stesso segno sono:

1. Estrarre i bit dell'esponente e quelli della mantissa.
2. Aggiungere l'1 più significativo per ottenere la mantissa effettiva.
3. Confrontare gli esponenti.
4. Traslare se necessario la mantissa con esponente minore.
5. Sommare le due mantisse.
6. Normalizzare la mantissa risultante sistemando, se necessario, l'esponente.
7. Arrotondare il risultato.
8. Assemblare l'esponente e la mantissa nella notazione in virgola mobile.

La **Figura 5.30** mostra la somma in virgola mobile di $7.875 (1.11111 \times 2^2)$ e $0.1875 (1.1 \times 2^{-3})$. Il risultato è $8.0625 (1.0000001 \times 2^3)$. Una volta che sono stati estratti i bit dell'esponente e i bit della mantissa, e una volta che è stato inserito l'1 più significativo nei passi 1 e 2, gli esponenti vengono confrontati sottraendo l'esponente minore dall'esponente maggiore. Il risultato è il numero di bit per il quale il numero minore deve essere traslato a destra perché sia allineato alla virgola binaria implicita (cioè, affinché gli

I calcoli aritmetici in virgola mobile sono generalmente eseguiti in hardware per farli velocemente. Tale hardware prende il nome di *Floating-Point Unit* (FPU) ed è generalmente separato dalla CPU (*Central Processing Unit*). Il tristemente noto "baco" della divisione in virgola mobile (FDIV) nella FPU del Pentium è costato alla Intel 475 milioni di dollari per ritirare dal mercato i chip difettosi e sostituirli. E tutto è stato originato da una tabella di ricerca (nota in informatica come *lookup table*) non correttamente caricata!

Numeri in virgola mobile			
	0	10000001	111 1100 0000 0000 0000 0000
	0	01111100	100 0000 0000 0000 0000 0000
		Esponente	Mantissa
		10000001	111 1100 0000 0000 0000 0000
Passo 1		01111100	100 0000 0000 0000 0000 0000
		10000001	1.111 1100 0000 0000 0000 0000
Passo 2		01111100	1.100 0000 0000 0000 0000 0000
		10000001	1.111 1100 0000 0000 0000 0000
Passo 3	-	01111100	1.100 0000 0000 0000 0000 0000
	101 (traslazione)		
		10000001	1.111 1100 0000 0000 0000 0000
Passo 4		10000001	0.000 0110 0000 0000 0000 0000 00000
		10000001	1.111 1100 0000 0000 0000 0000
Passo 5		10000001	0.000 0110 0000 0000 0000 0000
			10.000 0010 0000 0000 0000 0000
Passo 6		10000001	10.000 0010 0000 0000 0000 0000 >> 1
	+	1	
		10000010	1.000 0001 0000 0000 0000 0000
Passo 7	(non serve arrotondamento)		
Passo 8	0	10000010	000 0001 0000 0000 0000 0000

Figura 5.30
Somma in virgola mobile.

esponenti siano eguali) nel passaggio 4. I numeri allineati vengono successivamente sommati tra loro. Dal momento che la somma ha una mantissa che è maggiore o uguale a 2,0, il risultato viene normalizzato traslandolo verso destra di un bit e aumentando l'esponente. In questo esempio, il risultato è esatto, quindi non è necessaria nessuna forma di arrotondamento. Il risultato viene quindi immagazzinato in notazione in virgola mobile rimuovendo l'uno più significativo implicito della mantissa e impostando il bit di segno.

5.4 ■ BLOCCHI COSTRUTTIVI SEQUENZIALI

Questo paragrafo esamina i blocchi costruttivi sequenziali, inclusi i contatori e i registri a scorrimento.

5.4.1 Contatori

Un **contatore binario** a N bit, mostrato nella **Figura 5.31**, è una rete sequenziale aritmetica con ingressi di clock e di reset e un'uscita Q a N bit. Il reset inizializza l'uscita a 0. Il contatore successivamente genera tutti i 2^N possibili valori di uscita in ordine binario crescente, aumentando di uno a ogni fronte di salita del clock. La **Figura 5.32** mostra un contatore a N bit composto da un sommatore e da un registro resettabile. A ogni ciclo di clock, il contatore aggiunge 1 al valore immagazzinato nel registro. L'**Esempio HDL 5.4** descrive un contatore binario con reset asincrono.

Altri tipi di contatori, come i contatori *Up/Down*, sono discussi negli Esercizi dal 5.47 al 5.50.

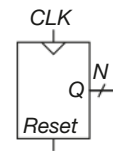


Figura 5.31
Simbolo del contatore.

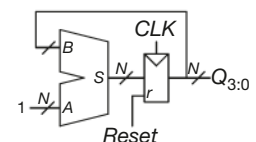


Figura 5.32
Contatore a N bit.

ESEMPIO HDL 5.4 CONTATORE

SystemVerilog

```
module contatore #(parameter N = 8)
    (input logic      clk,
     input logic      reset,
     output logic [N-1:0] q);

    always_ff @(posedge clk, posedge reset)
        if (reset) q <= 0;
        else      q <= q + 1;
endmodule
```

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD_UNSIGNED.ALL;

entity contatore is
    generic(N: integer := 8);
    port(clk, reset: in  STD_LOGIC;
         q:      out STD_LOGIC_VECTOR(N-1 downto 0));
end;

architecture sintesi of contatore is
begin
    process(clk, reset) begin
        if reset then          q <= (OTHERS => '0');
        elsif rising_edge(clk) then q <= q + '1';
        end if;
    end process;
end;
```

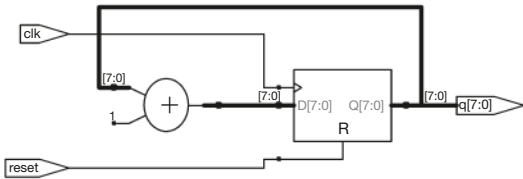


Figura 5.33 Sintesi del contatore.

5.4.2 Registri a scorrimento

Un **registro a scorrimento** ha un clock, un ingresso seriale S_{in} , un'uscita seriale S_{out} , e N uscite parallele $Q_{N-1:0}$, come mostrato nella **Figura 5.34**. A ogni fronte di salita del clock, un nuovo bit viene inserito dall'ingresso S_{in} e tutti i bit seguenti vengono traslati in avanti. L'ultimo bit nel registro diventa quindi disponibile all'uscita S_{out} . I registri a scorrimento possono essere visti come **convertitori serie-parallelo**: l'ingresso viene infatti ricevuto in serie (un bit alla volta) da S_{in} . Dopo N cicli, gli N ingressi ricevuti sono disponibili in parallelo su Q .

Un registro a scorrimento può essere costruito con N flip-flop connessi in serie, come mostra la **Figura 5.35**. Alcuni registri a scorrimento hanno anche un segnale di reset per inizializzare tutti i flip-flop.

Un circuito imparentato al registro a scorrimento è il **convertitore parallelo-serie**, che carica N bit in parallelo per poi restituirli all'esterno uno alla volta. Un registro a scorrimento può essere realizzato in modo che esegua sia la conversione serie-parallelo sia quella parallelo-serie, aggiungendo un ingresso parallelo $D_{N-1:0}$ e un segnale di controllo *Carica*, come mostrato in **Figura 5.36**. Quando il segnale *Carica* viene attivato, i flip-flop vengono caricati in parallelo a partire dall'ingresso D . Altrimenti, il registro effettua la normale traslazione. L'**Esempio HDL 5.5** descrive questo tipo di registro.

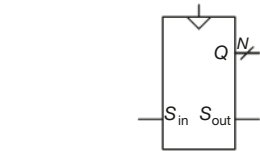
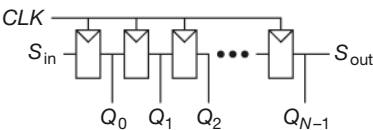


Figura 5.34
Simbolo del registro a scorrimento.

Figura 5.35
Schema del registro a scorrimento.



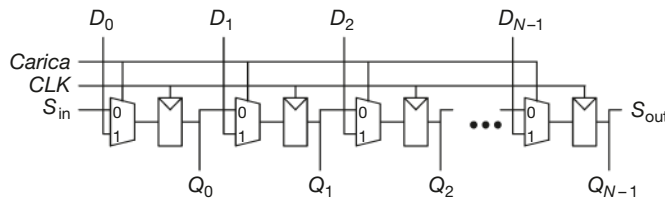


Figura 5.36
Registro a scorrimento
a caricamento parallelo.

ESEMPIO HDL 5.5 REGISTRO A SCORRIMENTO A CARICAMENTO PARALLELO

SystemVerilog

```
module regscorr #(parameter N = 8)
    (input logic clk,
     input logic reset, carica,
     input logic sin,
     input logic [N-1:0] d,
     output logic [N-1:0] q,
     output logic sout);

    always_ff @(posedge clk, posedge reset)
        if (reset) q <= 0;
        else if (carica) q <= d;
        else q <= {q[N-2:0], sin};

    assign sout = q[N-1];
endmodule
```

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.ALL;

entity regscorr is
    generic(N: integer := 8);
    port(clk, reset: in STD_LOGIC;
         carica, sin: in STD_LOGIC;
         d: in STD_LOGIC_VECTOR(N-1 downto 0);
         q: out STD_LOGIC_VECTOR(N-1 downto 0);
         sout: out STD_LOGIC);
end;

architecture sintesi of regscorr is
begin
    process(clk, reset) begin
        if reset = '1' then q <= (OTHERS => '0');
        elsif rising_edge(clk) then
            if carica then q <= d;
            else q <= q(N-2 downto 0) & sin;
            end if;
        end if;
    end process;

    sout <= q(N-1);
end;
```

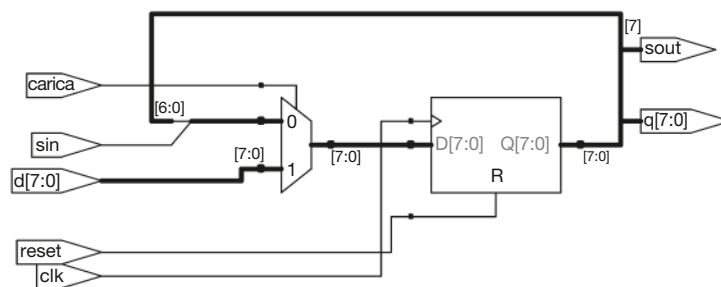


Figura 5.37 Sintesi del registro a scorrimento.

Catene di scansione*

I registri a scorrimento vengono spesso utilizzati per collaudare le reti sequenziali usando la tecnica basata sulle **catene di scansione** (*scan chain*). Collaudare le reti combinatorie è un processo relativamente semplice: vengono applicate agli ingressi delle configurazioni note chiamate **vettori di test** e le uscite vengono confrontate con il risultato previsto. Collaudare le reti sequenziali è invece un processo più complesso, dal momento che queste reti possiedono degli stati. A partire da una condizione iniziale nota, può essere necessario un gran numero di cicli di vettori di test per portare la rete nello stato desiderato. Per esempio, verificare il passaggio da 0 a 1 del bit più significativo di un con-

Non si confondano i registri a scorrimento (*shift register*) con i traslatori (*shifter*) del paragrafo 5.2.5. I registri a scorrimento sono blocchi logici sequenziali che fanno entrare un nuovo bit a ogni colpo di clock. I traslatori sono blocchi logici combinatori privi di clock che traslano un ingresso per una determinata quantità di bit.

tatore a 32 bit richiede il reset del contatore e, successivamente, l'applicazione di 2^{31} impulsi di clock (circa due miliardi!).

Per risolvere questo problema, i progettisti preferiscono essere in grado di osservare e controllare direttamente tutti gli elementi di stato della macchina. Per fare ciò viene aggiunta una modalità test (collaudo) nella quale i contenuti di tutti i flip-flop possono essere letti o caricati coi valori desiderati. La maggior parte dei sistemi possiede un numero di flip-flop troppo elevato per dedicare piedini per leggere e modificare ogni flip-flop. Si preferisce quindi connettere tra loro tutti i flip-flop presenti nel sistema in un registro a scorrimento, chiamato catena di scansione. Nelle operazioni normali, i flip-flop caricano i dati a partire dal loro ingresso D e ignorano la catena di scansione. In modalità test, invece, i flip-flop traslano in uscita i propri contenuti e ne inseriscono di nuovi usando S_{in} e S_{out} . Il multiplexer necessario per scegliere quale dato caricare viene di solito integrato all'interno di ciascun flip-flop per produrre un **flip-flop scansionabile**. La **Figura 5.38** mostra lo schema e il simbolo circuitale di un flip-flop scansionabile e illustra come i flip-flop vengono collegati a cascata per creare un registro scansionabile a N bit.

Per esempio, il contatore a 32 bit potrebbe essere facilmente collaudato inserendo nel registro a scorrimento il valore 011111...111 in modalità test, contando per un ciclo in modalità normale per poi produrre in uscita il risultato, che dovrebbe essere 100000...000. Questa operazione richiede solo $32 + 1 + 32 = 65$ cicli.

5.5 ■ COMPONENTI DI MEMORIA

Il paragrafo precedente ha introdotto i circuiti aritmetici e i circuiti sequenziali per l'elaborazione dei dati. I sistemi digitali richiedono anche delle **memorie** per immagazzinare i dati utilizzati e generati da questi tipi di circuiti. I registri costruiti con i flip-flop sono un esempio di memoria in grado di immagazzinare una piccola quantità di dati. In questo paragrafo vengono descritti i **componenti di memoria**, che sono in grado di immagazzinare grandi quantità di dati.

Questo paragrafo comincia con una panoramica descrittiva delle caratteristiche comuni a tutti i componenti di memoria. Successivamente, verranno introdotti tre tipi di memorie differenti: la **memoria ad accesso casuale dinamica** (DRAM, *Dynamic Random Access Memory*), la **memoria ad accesso casuale statica** (SRAM, *Static Random Access Memory*) e la **memoria a sola lettura** (ROM, *Read Only Memory*). Ognuna di queste memorie è differente nella modalità di immagazzinamento dei dati. Vengono qui anche discussi brevemente i compromessi relativi ad area di silicio e ritardo di risposta, e viene mostrato come i componenti di memoria vengano utilizzati non solo per l'immagazzinamento dei dati, ma anche per l'esecuzione di funzioni logiche. Il paragrafo si conclude con la descrizione HDL di un componente di memoria.

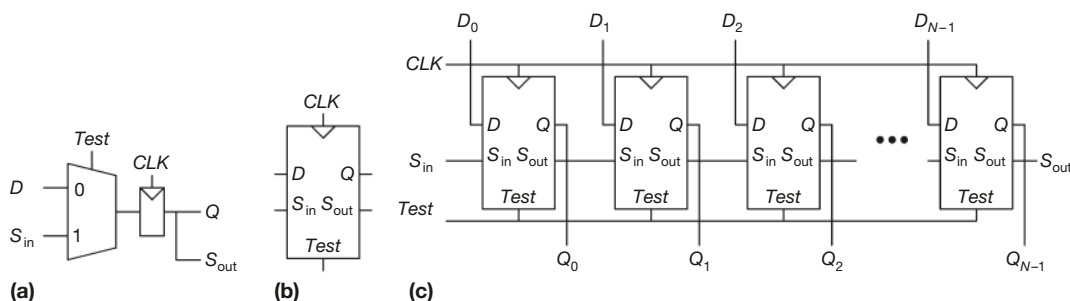


Figura 5.38 Flip-flop scansionabile: (a) schema, (b) simbolo, (c) registro scansionabile a N bit.

5.5.1 Panoramica

La **Figura 5.39** mostra il generico simbolo circuitale di un componente di memoria. La memoria è organizzata come una matrice bidimensionale di celle di memoria. A ogni accesso la memoria può leggere o scrivere il contenuto di una riga della matrice. Questa riga viene specificata da un **indirizzo** (*address*). Il valore letto o scritto nella memoria viene chiamato **dato** (*data*). Un componente con un numero N di bit di indirizzo e un numero M di bit di dato possiede 2^N righe e M colonne. Ogni riga di dati viene chiamata **parola**. Quindi, tale componente contiene 2^N parole da M bit.

La **Figura 5.40** mostra un componente di memoria con due bit di indirizzo e tre bit di dato. I due bit di indirizzo specificano una delle quattro righe (o parole) all'interno della matrice. Ogni parola è composta da tre bit. Nella **Figura 5.40(b)** vengono mostrati alcuni possibili contenuti del componente di memoria.

La **lunghezza** di un componente di memoria indica il numero di righe che possiede, mentre la sua **larghezza** fa riferimento al numero di colonne e quindi alla dimensione di parola. La dimensione totale del componente è uguale al prodotto lunghezza $\times$ larghezza. La **Figura 5.40** mostra un componente di memoria di 4 parole $\times$ 3 bit, o, più semplicemente, una memoria 4 $\times$ 3. Il simbolo circuitale per una memoria di 1024 parole $\times$ 32 bit viene mostrato nella **Figura 5.41**. La dimensione totale di questa matrice è 32 kilobit (Kb).

Celle di bit

I componenti di memoria vengono realizzati come matrici di **celle di bit**, ognuna delle quali può contenere un bit di dato. La **Figura 5.42** mostra che ogni cella di dato è connessa a una **linea di parola** e a una **linea di bit**. Per ogni configurazione dei bit di indirizzo, la memoria attiva una sola linea di parola che, a sua volta, attiva le celle di bit presenti nella riga corrispondente. Quando la linea di parola è ALTA, il bit memorizzato viene inviato alla linea di bit o prelevato dalla stessa. Altrimenti, la linea di bit è disconnessa dalla cella di bit. La circuiteria per l'immagazzinamento del bit varia a seconda del tipo di memoria.

Per leggere una cella di bit, la linea di bit viene inizialmente lasciata elettricamente fluttuante (Z). Viene quindi attivata la linea di parola, che permette al valore immagazzinato di forzare la linea di bit a 0 o a 1. Per scrivere la cella di bit, invece, la linea di bit viene forzata in modo deciso al valore desiderato. Viene poi attivata la linea di parola, che collega quindi la linea di bit alla cella di bit in cui memorizzare il dato. La linea di bit forzata in modo deciso sovrasta il contenuto della cella di bit, scrivendo il valore voluto nel bit in questione.

Organizzazione

La **Figura 5.43** mostra l'organizzazione interna di un componente di memoria 4 $\times$ 3. Ovviamente, le memorie realmente utilizzate sono molto più grandi, ma il loro comportamento può essere dedotto dall'osservazione di questo componente più piccolo. In questo esempio, il componente memorizza i dati citati nella **Figura 5.40(b)**.

Durante la lettura da memoria, la linea di parola è attiva e la riga corrispondente di celle di bit porta le linee di bit a un valore ALTO o BASSO. Durante la scrittura in memoria, invece, per prima cosa vengono portate le linee di bit a un valore ALTO o BASSO, e solo successivamente viene attivata la linea di parola, permettendo così ai valori delle linee di bit di essere immagazzinati in quella riga di celle di bit. Per esempio, per leggere da *Indirizzo* 10, le linee di bit vengono lasciate fluttuanti, il decoder attiva *linea_di_parola₂* e

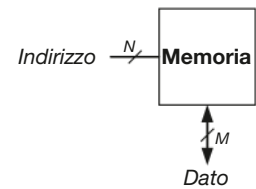


Figura 5.39
Simbolo di memoria generica.

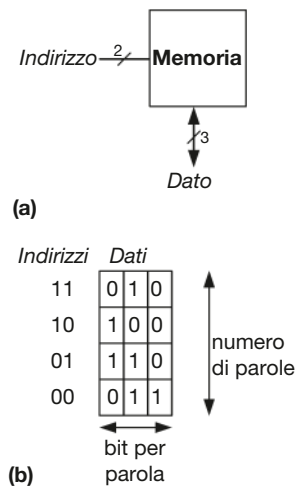


Figura 5.40
Memoria 4 $\times$ 3: (a) simbolo, (b) organizzazione.

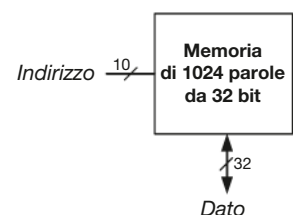


Figura 5.41
Memoria da 32 Kb: $2^{10} = 1024$ parole da 32 bit.

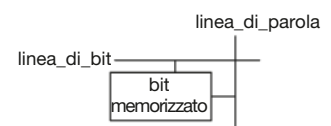
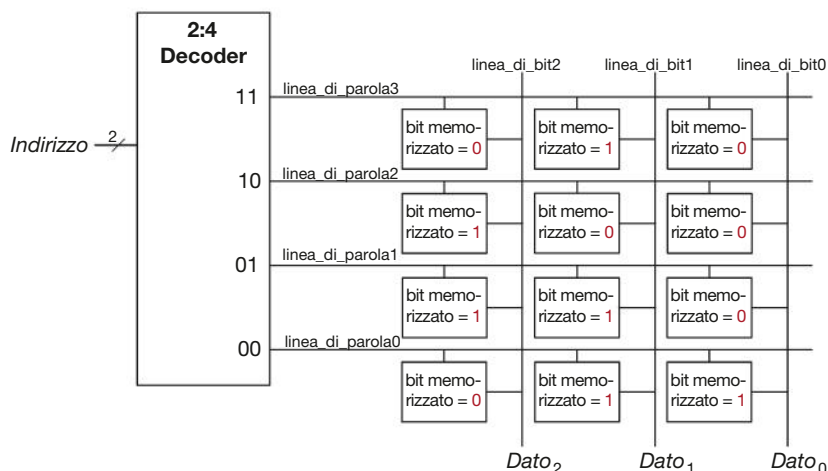


Figura 5.42
Cella di bit.

Figura 5.43
Memoria 4 × 3.



il dato immagazzinato in quella riga di celle di bit (100) viene reso disponibile sulle linee di bit *Dato*. Invece, per scrivere il valore 001 a *Indirizzo* 11, le linee di bit vengono portate al valore 001, poi viene attivata *linea_di_parola*₃ e il nuovo valore (001) viene immagazzinato nelle celle di bit.

Porte di memoria

Tutte le memorie possiedono una o più **porte**. Ognuna di queste fornisce un accesso in lettura e/o in scrittura a un indirizzo di memoria. Gli esempi precedenti rappresentavano tutti memorie a una sola porta.

Le memorie **multi-porta** possono accedere a più indirizzi nello stesso momento. La **Figura 5.44** mostra una memoria a tre porte con due porte di lettura e una porta di scrittura. La Porta 1 legge il dato dall'indirizzo A1 sull'uscita di lettura dati RD1. La Porta 2 legge il dato dall'indirizzo A2 su RD2. Infine, la Porta 3 scrive il dato dall'ingresso di scrittura dati WD3 all'indirizzo A3 sul fronte di salita del clock se l'abilitazione in scrittura WE3 è attivata.

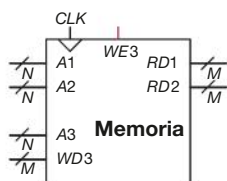


Figura 5.44
Memoria a tre porte.

Tipi di memorie

I componenti di memoria sono specificati dalla loro dimensione (lunghezza × larghezza) e dal numero e tipo delle loro porte. Tutti i componenti di memoria immagazzinano i dati in una matrice di celle di bit, ma differiscono nel procedimento di immagazzinamento dei bit.

Le memorie vengono classificate in base a come immagazzinano i bit nella cella di bit. La classificazione più generale distingue le **memorie ad accesso casuale (RAM)** dalle **memorie a sola lettura (ROM)**. La RAM è una memoria **volatile**, cioè una memoria che perde traccia dei suoi dati una volta spenta. Invece, la ROM è una memoria **non volatile**, cioè una memoria che trattiene i suoi dati per un tempo indefinito, anche in assenza di alimentazione.

L'origine dei nomi RAM e ROM è legata a ragioni storiche oggi poco significative. La RAM è chiamata memoria ad accesso casuale perché si può accedere a qualsiasi parola con lo stesso ritardo di qualsiasi altra. Al contrario, una memoria ad accesso sequenziale, come per esempio un registratore a nastro, accede ai dati più vicini più velocemente rispetto ai dati più lontani (ovvero ai dati che si trovano all'estremo opposto del nastro). La ROM viene invece chiamata memoria **a sola lettura** perché, storicamente, poteva essere esclusivamente letta, e non scritta. Questi nomi possono risultare ambigui dal momento che anche le memorie ROM sono ad accesso casuale. Ancora peggio, la maggior parte delle ROM moderne possono essere sia lette sia scritte! Quindi, la distinzione importante da tenere a mente è che le memorie RAM sono volatili mentre le ROM non lo sono.



Robert Dennard, 1932-. Ha inventato le DRAM in IBM nel 1966. Nonostante lo scetticismo iniziale di molti colleghi, a metà degli anni '70 le DRAM erano presenti praticamente in tutti i calcolatori. Diceva di aver fatto pochi lavori creativi finché, arrivando in IBM, qualcuno gli aveva portato un quaderno per brevetti dicendogli "scrivi qui tutte le idee che ti vengono". Dal 1965 ha ottenuto 35 brevetti nel campo dei semiconduttori e della microelettronica. (Fotografia gentilmente concessa da IBM.)

I due tipi principali di memorie RAM sono la RAM **dinamica** (DRAM) e la RAM **statica** (SRAM). Una RAM dinamica immagazzina i dati come una carica su un condensatore, mentre una RAM statica immagazzina i dati utilizzando una coppia di negatori collegati a croce. Esistono diversi tipi di memorie ROM che si differenziano per il modo in cui vengono scritte e cancellate. I vari tipi di memoria sono presentati nei paragrafi seguenti.

5.5.2 Memoria ad accesso casuale dinamica

La **RAM dinamica** (DRAM, *Dynamic RAM*) memorizza un bit come presenza o assenza di carica in un condensatore. La **Figura 5.45** mostra la cella di bit di una DRAM. Il valore del bit viene memorizzato in un condensatore. Il transistor nMOS si comporta come un interruttore che connette o disconnette il condensatore dalla linea di bit. Quando la linea di parola è attiva, il transistor nMOS si accende e il valore del bit immagazzinato viene trasferito alla o dalla linea di bit.

Come mostrato nella **Figura 5.46(a)**, quando il condensatore viene caricato a V_{DD} , il bit immagazzinato è 1; quando invece viene scaricato fino a GND (**Figura 5.46(b)**), il bit immagazzinato è 0. Il terminale del condensatore è **dinamico** perché non viene portato a un valore ALTO o BASSO da un transistor tenuto a V_{DD} o GND.

In caso di lettura, il valore del dato viene trasferito dal condensatore alla linea di bit. Al contrario, in caso di scrittura, il valore del dato viene trasferito dalla linea di bit al condensatore. La lettura distrugge il valore del bit immagazzinato nel condensatore, quindi la parola di dato deve essere **rinfrescata** (riscritta) dopo ogni lettura. Anche quando la DRAM non viene letta è necessario ricaricarne i contenuti (cioè leggerli e riscriverli) ogni pochi millisecondi poiché la carica sul condensatore si perde gradualmente.

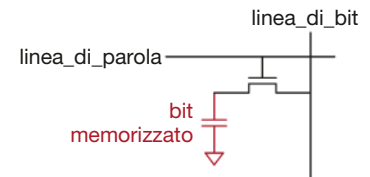
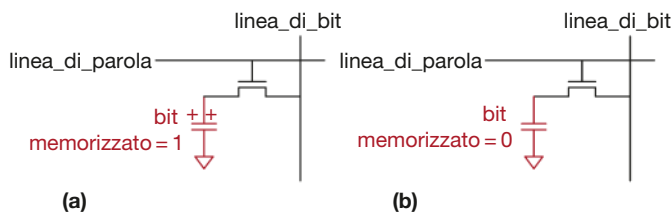


Figura 5.45
Cella di bit DRAM.

Figura 5.46
Valori memorizzati in DRAM.

5.5.3 Memoria ad accesso casuale statica

Una **RAM statica** (SRAM, *Static RAM*) viene chiamata in questo modo perché i bit immagazzinati nella memoria non hanno bisogno di essere ricaricati. La **Figura 5.47** mostra una cella di bit di una SRAM. Il bit di dato viene immagazzinato grazie a dei negatori collegati a croce come quelli descritti nel paragrafo 3.2. Ogni cella possiede due uscite, *linea_di_bit* e *linea_di_bit*. Quando la linea di parola viene attivata, entrambi i transistori nMOS si accendono e i bit di dato vengono trasferiti da o verso le linee di bit. A differenza della DRAM, se il rumore deteriora il valore del bit immagazzinato, i negatori collegati a croce lo riportano al valore di partenza.

5.5.4 Area e ritardo

I flip-flop, le SRAM e le DRAM sono tutti tipi di memorie volatili, ma ognuna di esse ha caratteristiche di area e di ritardo differenti. La **Tabella 5.4** confronta queste tre memorie volatili. Il bit di dato immagazzinato in un flip-flop è immediatamente disponibile alla sua uscita. Tuttavia, per costruire un flip-flop sono necessari almeno 20 transistori. In generale, maggiore

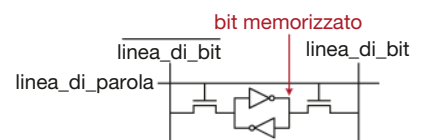


Figura 5.47
Cella di bit SRAM.

Tabella 5.4 Confronto di memorie.

Tipo di memoria	Transistor per cella di bit	Latenza
Flip-flop	~20	Breve
SRAM	6	Media
DRAM	1	Lunga

è il numero di transistori necessari per costruire un dispositivo, maggiori sono i requisiti di area, energia e costo richiesti. La latenza di una memoria DRAM è più lunga rispetto a quella di una SRAM perché la sua linea di bit non è pilotata in maniera attiva da un transistor. Una DRAM deve attendere che la carica si sposti in modo (relativamente) lento dal condensatore alla linea di bit. Inoltre una DRAM ha un **throughput** (quantità di bit scambiabili per unità di tempo) nettamente inferiore a quella di una SRAM, perché deve rinfrescare periodicamente i dati, oltre che dopo ogni lettura. Sono state sviluppate varie tecnologie DRAM, come le **DRAM sincrone (SDRAM, Synchronous DRAM)** e le **SDRAM a doppia velocità (DDR SDRAM, Double Data Rate SDRAM)** proprio per sopperire a questo problema. Una SDRAM utilizza un clock per organizzare a pipeline gli accessi alla memoria. Le DDR SDRAM, dette anche semplicemente DDR, utilizzano sia i fronti di salita sia quelli di discesa del clock per accedere ai dati e in questo modo duplicano il throughput a pari frequenza di clock. La DDR fu standardizzata per la prima volta nel 2000 e lavorava da 100 a 200 MHz. Gli standard successivi, DDR2, DDR3 e DDR4, hanno aumentato la frequenza di clock fino ad arrivare, nel 2015, a superare 1 GHz.

La latenza di una memoria e il suo throughput dipendono anche dalla dimensione della memoria stessa; memorie più grandi tendono a essere più lente di quelle più piccole, a parità di tutti gli altri aspetti. Come sempre, il miglior tipo di memoria da utilizzare per un particolare progetto dipende dai limiti di velocità, potenza e costo del progetto stesso.

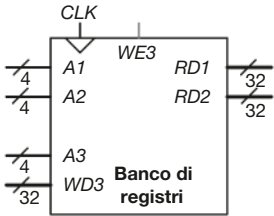


Figura 5.48
Banco di registri 16 × 32 con due porte di lettura e una di scrittura.

5.5.5 Banchi di registri

I sistemi digitali utilizzano spesso un certo numero di registri per immagazzinare variabili temporanee. Questo gruppo di registri, chiamato **banco di registri (register file)**, viene solitamente costruito come una piccola componente SRAM multi-porta, perché questa è più compatta rispetto a una matrice di flip-flop.

La **Figura 5.48** mostra un banco di 16 registri × 32 bit a tre porte, costruito a partire da una memoria a tre porte simile a quella descritta nella Figura 5.44. Il banco di registri ha due porte di lettura (A1/RD1 e A2/RD2) e una porta di scrittura (A3/WD3). Gli indirizzi a 4 bit, A1, A2 e A3, sono tutti in grado di accedere ai 2⁴ = 16 registri. In questo modo due registri possono essere consultati e un registro può essere scritto simultaneamente.

5.5.6 Memorie a sola lettura

Una **memoria a sola lettura (ROM, Read Only Memory)** memorizza un bit come presenza o assenza di un transistor. La **Figura 5.49** mostra una semplice cella di bit di una ROM. Per leggere la cella, la linea di bit viene portata debolmente ALTA. Dopodiché, viene accesa la linea di parola. Se il transistor è presente, questo spinge la linea di bit a un valore BASSO; se invece è assente, la linea di bit resta ALTA. Si noti che una cella di bit di una ROM è una rete combinatoria e non ha uno stato che le consenta di “dimenticare” il proprio contenuto quando viene spenta.

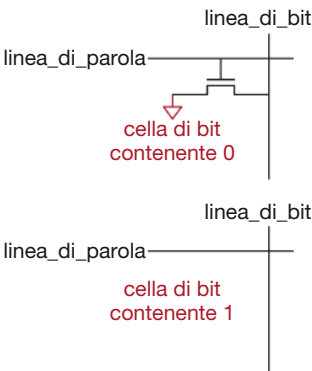


Figura 5.49
Celle di bit ROM contenenti 0 e 1.

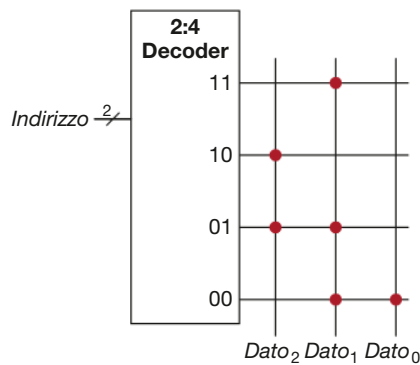


Figura 5.50
ROM 4 × 3: notazione a punti.

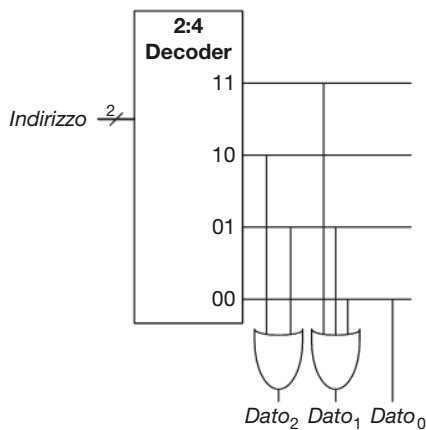


Figura 5.51
ROM 4 × 3: realizzazione mediante porte logiche.

Il contenuto di una ROM può essere indicato con la **notazione a punti**. La **Figura 5.50** mostra la notazione a punti per una ROM da 4 parole × 3 bit contenente i dati della Figura 5.40. Un punto posto all'intersezione tra una riga (linea di parola) e una colonna (linea di bit) indica che il bit di dato è uguale a 1. Per esempio, la prima linea di parola ha un unico punto su *Dato₁*, quindi la parola memorizzata all'*Indirizzo* 11 è 010.

Concettualmente, le ROM possono essere costruite utilizzando la logica a due livelli, con una serie di porte AND seguite da una serie di porte OR. Le porte AND producono tutti i mintermini possibili, quindi vanno a formare un decoder. La **Figura 5.51** mostra la ROM della Figura 5.50 costruita usando un decoder e delle porte OR. Ogni punto nella Figura 5.50 corrisponde a una linea di ingresso a una porta OR della Figura 5.51. Per linee di bit con un solo punto, in questo caso *Dato₀*, non è necessaria una porta OR. Questa rappresentazione di una ROM è particolarmente interessante perché mostra come la ROM sia in grado di realizzare una qualsiasi funzione logica a due livelli. In pratica, le ROM vengono costruite a partire da dei transistori invece che a partire da delle porte logiche per ridurne la grandezza e il costo. Nel paragrafo 5.6.3 viene approfondita la realizzazione a livello di transistori.

I contenuti delle celle della ROM della Figura 5.49 vengono specificati durante la sua costruzione dalla presenza o assenza di un transistor in ogni cella di bit. Una **ROM programmabile (PROM, Programmable ROM)** ha un transistor in ogni cella di bit ma consente di connettere o disconnettere il transistor da massa.

La **Figura 5.52** mostra la cella di bit per una **ROM programmabile a fusibili**. In questo caso l'utente può programmare la ROM applicando un'alta tensione per bruciare selettivamente alcuni fusibili. Se il fusibile è presente, allora il transistor è connesso a massa e la cella contiene il valore 0. Se in-

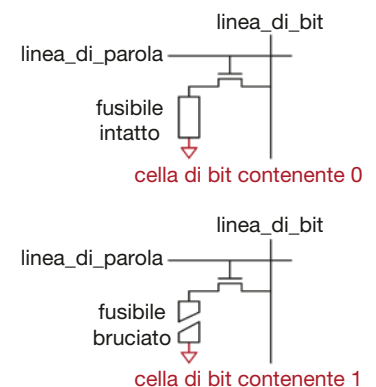


Figura 5.52
Celle di bit ROM programmabili a fusibile.



Fujio Masuoka, 1944-. Ha conseguito il dottorato di ricerca alla Tohoku University, in Giappone. Dal 1971 al 1994 ha sviluppato memorie e circuiti ad alta velocità alla Toshiba. Ha inventato le memorie flash nell'ambito di un progetto non autorizzato, lavorando di notte e nei fine settimana verso la fine del 1970. Il nome di queste memorie deriva dal fatto che il processo di cancellazione assomiglia al flash di una macchina fotografica. La Toshiba è stata lenta nel commercializzare l'idea, e l'Intel è stata la prima sul mercato nel 1988. La crescita di mercato delle memorie flash è arrivata a 25 miliardi di dollari all'anno. Successivamente il dott. Masuoka è entrato a far parte del corpo docente della Tohoku University, e sta lavorando allo sviluppo di un transistor tridimensionale.

vece il fusibile viene bruciato, il transistor è disconnesso da massa e la cella contiene il valore 1. Questo tipo di memoria viene anche chiamata ROM programmabile una sola volta, perché quando i fusibili sono stati bruciati non è più possibile ripristinarli.

Le ROM riprogrammabili possiedono invece un meccanismo reversibile per connettere o disconnettere il transistor da massa. Le **PROM cancellabili** (**EPROM**, *Erasable PROM*) sostituiscono il transistor e il fusibile con un **transistor a gate sommerso**. Il gate sommerso non viene collegato fisicamente ad alcun contatto. Quando viene applicata una tensione opportuna, gli elettroni attraversano lo strato isolante e raggiungono il gate, accendendo così il transistor e connettendo la linea di bit alla linea di parola (ovvero l'uscita del decoder). Quando una memoria EPROM viene esposta per circa mezz'ora a una potente luce ultravioletta (UV), gli elettroni vengono espulsi dal gate sommerso, spegnendo nuovamente il transistor. Queste operazioni vengono chiamate, rispettivamente, **programmazione** e **cancellazione**. Le **PROM cancellabili elettricamente** (**EEPROM**, *Electrically Erasable PROM*) e le **memorie flash** utilizzano principi simili a quello appena visto, ma includono un circuito posto sul chip per cancellare e programmare, in modo che non sia necessaria la luce ultravioletta. Le celle di bit delle EEPROM sono cancellabili individualmente; le memorie flash, invece, cancellano blocchi più grandi di bit e sono più economiche perché necessitano di circuiti più semplici per la cancellazione. Nel 2015, una memoria flash costava all'incirca 0.35 dollari per GB e il prezzo ha continuato a scendere dal 30 al 40% ogni anno. Le memorie flash costituiscono ormai un metodo molto utilizzato per immagazzinare grandi quantità di dati in sistemi portatili alimentati a batteria, come le macchine fotografiche e i riproduttori musicali.

Per riassumere, le ROM moderne non sono più memorie di sola lettura, ma possono anche essere programmate (scritte). La differenza tra una ROM e una RAM è che le ROM hanno bisogno di più tempo per essere scritte ma sono memorie non volatili.

5.5.7 Reti logiche realizzate con componenti di memoria

Nonostante il loro utilizzo primario rimanga la memorizzazione di dati, i componenti di memoria sono anche in grado di svolgere funzioni logiche combinatorie. Per esempio, l'uscita $Dato_2$ della ROM della Figura 5.50 costituisce lo XOR dei due ingressi *Indirizzo*. Analogamente, $Dato_0$ è il NAND di tali ingressi. Una memoria a 2^N parole $\times M$ bit può svolgere una qualsiasi funzione combinatoria di N ingressi e M uscite. Per esempio, la ROM della Figura 5.50 svolge tre funzioni di due ingressi.

Componenti di memoria usati per svolgere funzioni logiche vengono chiamati **lookup table** (letteralmente "tabelle di consultazione"). La **Figura 5.53** mostra un componente di memoria a 4 parole $\times$ 1 bit usato come lookup table per eseguire la funzione $Y = AB$. Utilizzando una memoria per eseguire funzioni logiche, l'utente può consultare il valore di uscita corrispondente a una data combinazione di ingressi (indirizzo). Ogni indirizzo corrisponde a una riga nella tabella delle verità e ogni bit di dato corrisponde a un valore di uscita.

5.5.8 Descrizione HDL delle memorie

L'**Esempio HDL 5.6** descrive una RAM da 2^N parole $\times M$ bit. La RAM ha un'abilitazione in scrittura sincrona. In altre parole, le scritture avvengono sui fronti di salita del clock se l'abilitazione in scrittura, *we*, è attivata. Le letture avvengono immediatamente. Quando viene applicata tensione per la prima volta, i contenuti della RAM sono imprevedibili.



Le unità di memoria flash con connettore USB (*Universal Serial Bus*) hanno sostituito i floppy e i CD per la condivisione dei file, perché il loro costo è diminuito drasticamente.

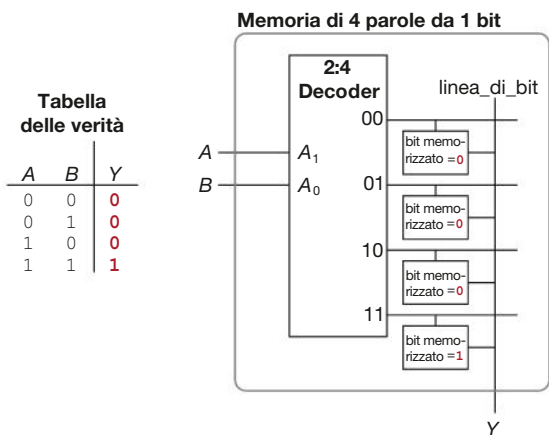
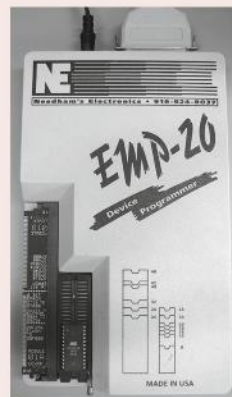


Figura 5.53 Memoria da 4 parole di 1 bit usata come *lookup table*.

L'**Esempio HDL 5.7** descrive invece una ROM da 4 parole $\times$ 3 bit. I contenuti della ROM vengono specificati nell'istruzione HDL `case`. Può succedere che una ROM piccola come quella dell'esempio venga sintetizzata mediante porte logiche piuttosto che come matrice. Si noti che il transcodificatore per display a sette segmenti dell'Esempio HDL 4.24 era stato sintetizzato come ROM nella Figura 4.20.

Le memorie ROM programmabili possono essere configurate con un dispositivo programmatore come quello mostrato sotto. Il programmatore è collegato a un calcolatore, che specifica il tipo di ROM e i valori dei dati da memorizzare. Il programmatore brucia i fusibili oppure inserisce cariche elettriche nei gate fluttuanti della ROM. Il processo di programmazione viene per questo denominato a volte *bruciatura* della ROM.



ESEMPIO HDL 5.6 RAM

SystemVerilog

```
module memoria_ram #(parameter N = 6, M = 32)
    (input logic clk,
     input logic we,
     input logic [N-1:0] ind,
     input logic [M-1:0] ding,
     output logic [M-1:0] dusc);
    logic [M-1:0] mem [2*N-1:0];
    always_ff @(posedge clk)
        if (we) mem[ind] <= ding;
    assign dusc = mem[ind];
endmodule
```

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.ALL;
entity memoria_ram is
    generic(N: integer := 6; M: integer := 32);
    port(clk,
         we: in STD_LOGIC;
         ind: in STD_LOGIC_VECTOR(N-1 downto 0);
         ding: in STD_LOGIC_VECTOR(M-1 downto 0);
         dusc: out STD_LOGIC_VECTOR(M-1 downto 0));
end;
architecture sintesi of memoria_ram is
    type mem_array is array ((2*N-1) downto 0)
        of STD_LOGIC_VECTOR (M-1 downto 0);
    signal mem: mem_array;
begin
    process(clk) begin
        if rising_edge(clk) then
            if we then mem(TO_INTEGER(ind)) <= ding;
            end if;
        end if;
    end process;
    dusc <= mem(TO_INTEGER(ind));
end;
```

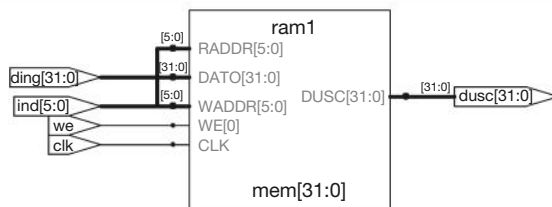


Figura 5.54 Sintesi della RAM.

5.6 ■ MATRICI LOGICHE

Come le memorie, anche le porte logiche possono essere organizzate in matrici regolari. Se le connessioni sono fatte in modo da essere programmabili, queste **matrici logiche** possono essere configurate in modo da eseguire qualsiasi funzione senza che l'utente debba connettere i fili in una maniera particolare. La struttura regolare semplifica il progetto. Le matrici logiche vengono prodotte in grandi quantità e sono quindi poco costose. Gli strumenti software permettono agli utenti di mappare i propri progetti logici su queste matrici. Inoltre, la maggior parte delle matrici logiche sono anche riconfigurabili, il che permette di modificare i progetti senza dover sostituire l'hardware. La riconfigurabilità di una matrice è un aspetto di gran valore durante la fase di sviluppo ed è anche utile sul campo, perché un sistema può essere migliorato semplicemente scaricando una nuova configurazione.

Questo paragrafo introduce due tipi di matrici logiche: le matrici logiche programmabili (PLA, *Programmable Logic Array*) e le matrici di porte logiche programmabili sul campo (FPGA, *Field Programmable Gate Array*). Le PLA, che costituiscono la tecnologia più vecchia, possono eseguire solo funzioni logiche combinatorie. Le FPGA, invece, possono eseguire funzioni sia di logica combinatoria sia di logica sequenziale.

5.6.1 Matrici logiche programmabili

Le **matrici logiche programmabili (PLA)** realizzano funzioni combinatorie a due livelli nella forma somma di prodotti. Le PLA sono costituite da una matrice AND seguita da una matrice OR, come mostrato nella **Figura 5.55**. Gli ingressi (in forma diritta e negata) pilotano la matrice AND, che produce degli implicant, a loro volta combinati nella matrice OR per generare le uscite. Una PLA da $M \times N \times P$ bit possiede M ingressi, N implicant e P uscite.

La **Figura 5.56** mostra la notazione a punti per una PLA da $3 \times 3 \times 2$ bit che esegue le funzioni $X = \overline{A}BC + AB\overline{C}$ e $Y = A\overline{B}$. Ogni riga della matrice AND genera un implicants. I punti presenti in ogni riga della matrice AND indicano quali letterali sono inclusi nell'implicants. La matrice AND nella **Figura 5.56** forma tre implicants: $\overline{A}BC$, $AB\overline{C}$ e $A\overline{B}$. I punti nella matrice OR indicano invece quali implicants fanno parte di ciascuna funzione di uscita.

ESEMPIO HDL 5.7 ROM	
SystemVerilog	VHDL
<pre>module rom(input logic [1:0] ind, output logic [2:0] dusc): always_comb case(ind) 2'b00: dusc = 3'b011; 2'b01: dusc = 3'b110; 2'b10: dusc = 3'b100; 2'b11: dusc = 3'b010; endcase endmodule</pre>	<pre>library IEEE; use IEEE.STD_LOGIC_1164.ALL; entity rom is port(ind: in STD_LOGIC_VECTOR(1 downto 0); dusc: out STD_LOGIC_VECTOR(2 downto 0)); end; architecture sintesi of rom is begin process(all) begin case ind is when "00" => dusc <= "011"; when "01" => dusc <= "110"; when "10" => dusc <= "100"; when "11" => dusc <= "010"; end case; end process; end;</pre>

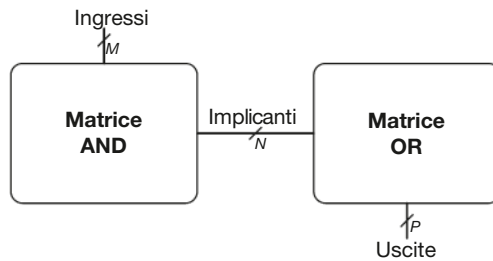


Figura 5.55
PLA da $M \times N \times P$ bit.

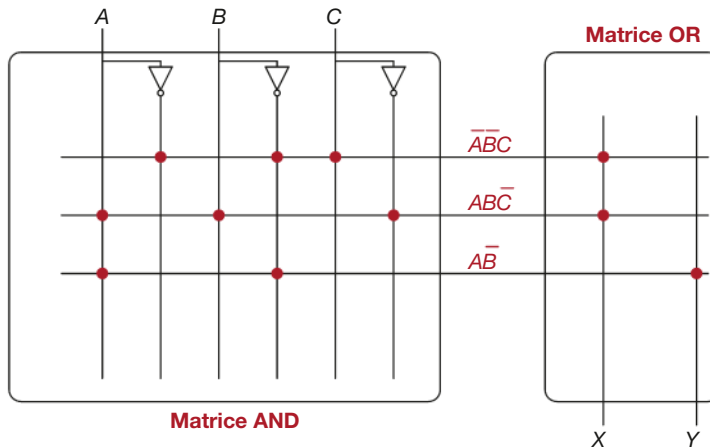


Figura 5.56
PLA da $3 \times 3 \times 2$ bit: notazione a punti.

La **Figura 5.57** mostra come le PLA possono essere costruite utilizzando logica a due livelli. Una realizzazione alternativa viene proposta nel paragrafo 5.6.3.

Le ROM possono essere viste come casi particolari di PLA. Una ROM da 2^M parole $\times N$ bit è semplicemente una PLA a $M \times 2^M \times N$ bit. Il decoder si comporta come una matrice AND che produce tutti i 2^M mintermini. La matrice ROM si comporta invece come una matrice OR che produce le uscite. Se la funzione non dipende da tutti i 2^M mintermini, una PLA risulta molto probabilmente più piccola di una ROM. Per esempio, è necessaria una ROM da 8 parole $\times 2$ bit per eseguire le stesse funzioni che esegue la PLA da $3 \times 3 \times 2$ bit mostrata nelle Figure 5.56 e 5.57.

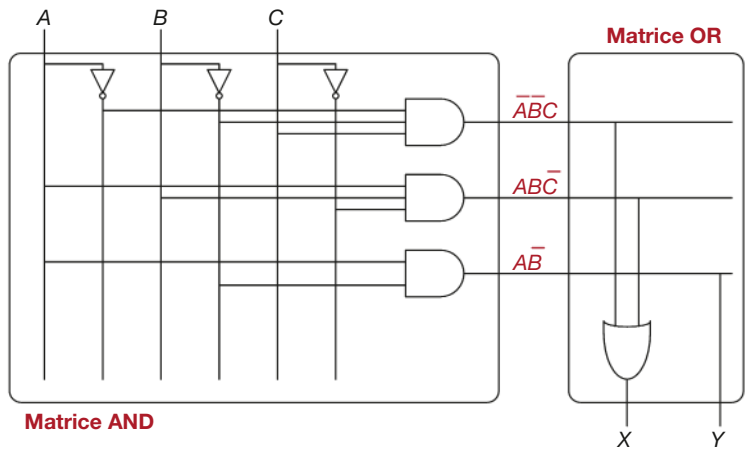
I **dispositivi logici programmabili semplici (SPLD, Simple Programmable Logic Devices)** sono delle PLA modificate che aggiungono dei registri e diverse altre caratteristiche alle matrici AND/OR. Tuttavia, SPLD e PLA sono stati rimpiazzati in larga misura dalle FPGA, che sono più flessibili ed efficienti per costruire grandi sistemi.

5.6.2 Matrici di porte logiche programmabili sul campo

Una **FPGA** è una matrice di porte logiche riconfigurabile. Con l'utilizzo di strumenti di programmazione software, l'utente può realizzare dei progetti sulla FPGA utilizzando sia un HDL sia uno schema circuitale. Le FPGA sono più potenti e più flessibili rispetto alle PLA per diverse ragioni. Innanzi tutto consentono di realizzare sia funzioni combinatorie sia funzioni sequenziali. Inoltre, possono realizzare funzioni logiche a più livelli, mentre le PLA si fermano alla logica a due livelli. Le FPGA moderne integrano altre caratteristiche utili preconfigurate, come moltiplicatori, ingressi e uscite ad alta velocità, convertitori di dati inclusi i convertitori analogico-digitali, memorie RAM di grandi dimensioni e processori.



Figura 5.57
PLA da 3 × 3 × 2 bit usando logica a due livelli.



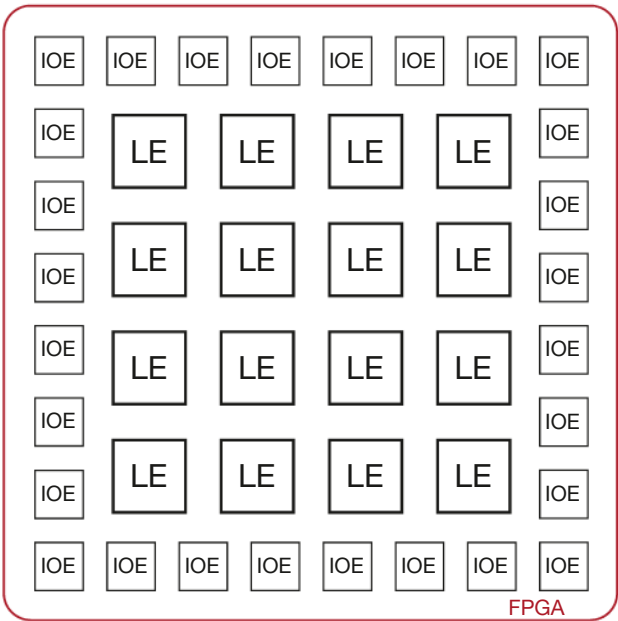
Le FPGA sono la parte intelligente di molti prodotti di consumo, incluse automobili, strumentazione medica, dispositivi multimediali come i riproduttori MP3. Le Mercedes Benz della Classe S, per esempio, hanno a bordo più di una dozzina di FPGA o PLD Xilinx per scopi che vanno dall'intrattenimento alla navigazione al controllo della velocità. Le FPGA consentono di ridurre il tempo di realizzazione di un prodotto per il mercato (*time to market*) e facilitano la ricerca degli errori e l'aggiunta di nuove caratteristiche al progetto iniziale.

Le FPGA vengono costruite come una matrice di **elementi logici** (LE, *Logic Element*) configurabili, chiamati anche **blocchi logici configurabili** (CLB, *Configurable Logic Block*). Ogni LE può essere configurato in modo da eseguire funzioni combinatorie o sequenziali. La **Figura 5.58** mostra un generico schema a blocchi di una FPGA. Gli LE sono circondati da **elementi di ingresso/uscita** (IOE, *Input/Output Element*) per interfacciarsi al mondo esterno. Gli IOE connettono gli ingressi e le uscite degli LE ai piedini esterni del chip. Gli LE possono essere connessi ad altri LE o a degli IOE attraverso canali di instradamento programmabili.

Due delle più importanti aziende produttrici di FPGA sono Altera Corp. e Xilinx, Inc. La **Figura 5.59** mostra un singolo LE della FPGA Cyclone IV della Altera, introdotta nel 2009. Gli elementi chiave degli LE sono una lookup table a 4 ingressi e un registro a 1 bit. L'LE contiene anche dei multiplexer configurabili per instradare i segnali all'interno dell'LE stesso. La FPGA viene configurata specificando il contenuto delle lookup table e i segnali di selezione dei vari multiplexer.

L'LE di Cyclone IV possiede una lookup table a 4 ingressi e un flip-flop. Inserendo i valori appropriati nella lookup table, questa può essere confi-

Figura 5.58
Schema generale di una FPGA.



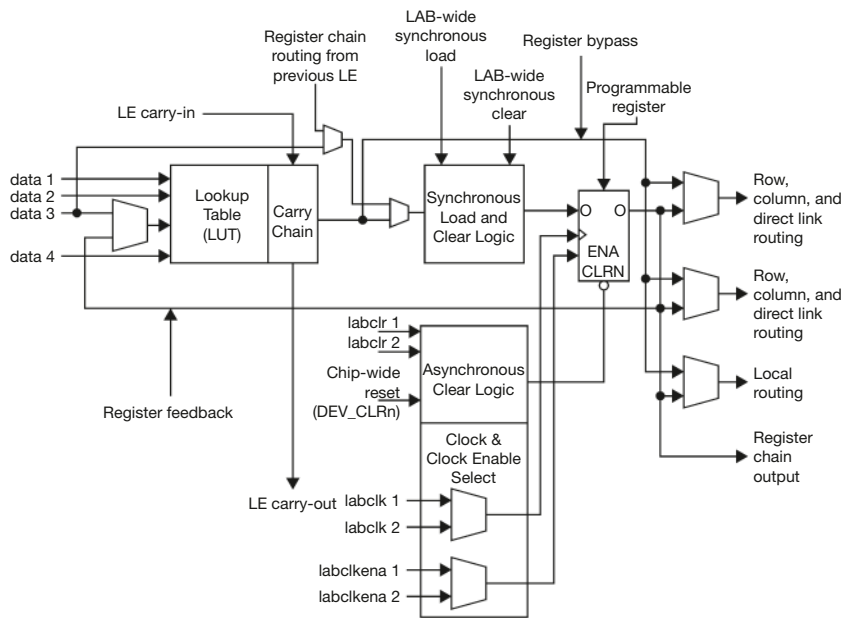


Figura 5.59
Elemento logico (LE, Logic Element)
di Cyclone IV. Riprodotto per gentile
 concessione da Altera Cyclone™
 IV Handbook (© 2010 Altera
 Corporation.)

gurata per eseguire qualsiasi funzione fino a 4 variabili. La configurazione della FPGA richiede anche la definizione dei segnali che determinano come i multiplexer guidano i dati all'interno dell'LE e verso i vicini LE e IOE. Per esempio, a seconda della configurazione del multiplexer, la *lookup table* può ricevere uno dei suoi ingressi da *dato 3*, oppure dall'uscita del registro dello stesso LE. Gli altri tre ingressi provengono sempre da *dato 1*, *dato 2* e *dato 4*. Gli ingressi *dato 1-4* provengono dagli IOE oppure dalle uscite di altri LE a seconda dell'instradamento esterno all'LE. L'uscita della *lookup table* può passare direttamente all'uscita dell'LE per funzioni combinatorie, mentre per funzioni sequenziali viene fatta arrivare al flip-flop. L'ingresso del flip-flop può provenire dall'uscita della *lookup table* dello stesso LE, oppure dall'ingresso *dato 3*, oppure dall'uscita memorizzata nell'LE a monte. L'hardware aggiuntivo fornisce supporti per l'addizione tramite la catena dei riporti, altri multiplexer per instradamento e segnali di abilitazione e reset del flip-flop. Altera raggruppa 16 LE per creare un **blocco di matrice logica (LAB, Logic Array Block)** e fornisce connessioni locali tra gli LE all'interno del LAB.

Per riassumere, l'LE di Cyclone IV può effettuare una funzione combinatoria e/o sequenziale fino a quattro variabili. Altre marche di FPGA sono organizzate diversamente ma applicano gli stessi principi generali. Per esempio, la FPGA Xilinx serie 7 utilizza una *lookup table* a 6 ingressi invece che a 4.

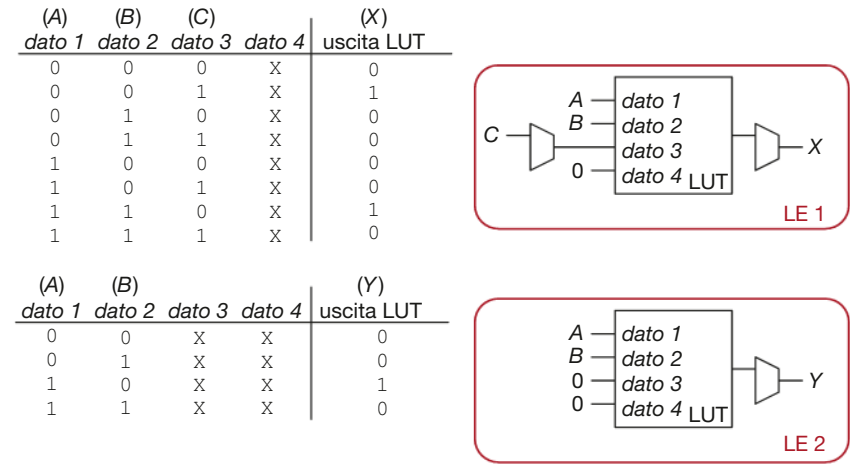
Il progettista configura una FPGA a partire dalla costruzione dello schema circuitale o da una descrizione HDL del progetto. Il progetto viene successivamente sintetizzato per la FPGA. Lo strumento di sintesi prescelto determina come le *lookup table*, i multiplexer e i canali di instradamento debbano essere configurati per eseguire le funzioni specifiche. Queste informazioni di configurazione vengono successivamente scaricate nella FPGA. Le FPGA Cyclone IV possono essere facilmente riprogrammate, perché immagazzinano le informazioni di configurazione in una SRAM. Una FPGA può scaricare il contenuto della propria SRAM da un calcolatore in laboratorio o da un componente EEPROM quando viene acceso il sistema. Alcuni produttori integrano questa EEPROM direttamente nella FPGA, oppure utilizzano dei fusibili programmabili una sola volta per configurare l'FPGA.

ESEMPIO 5.5

Funzioni costruite utilizzando LE. Spiegare come sia possibile configurare una o più FPGA Cyclone IV per realizzare le seguenti funzioni: (a) $X = \overline{ABC} + ABC$ e $Y = \overline{AB}$; (b) $Y = JKLPQR$; (c) un contatore modulo 3 con codifica binaria degli stati binari (vedi la Figura 3.29(a)). Evidenziare le interconnessioni tra gli LE quando necessario.

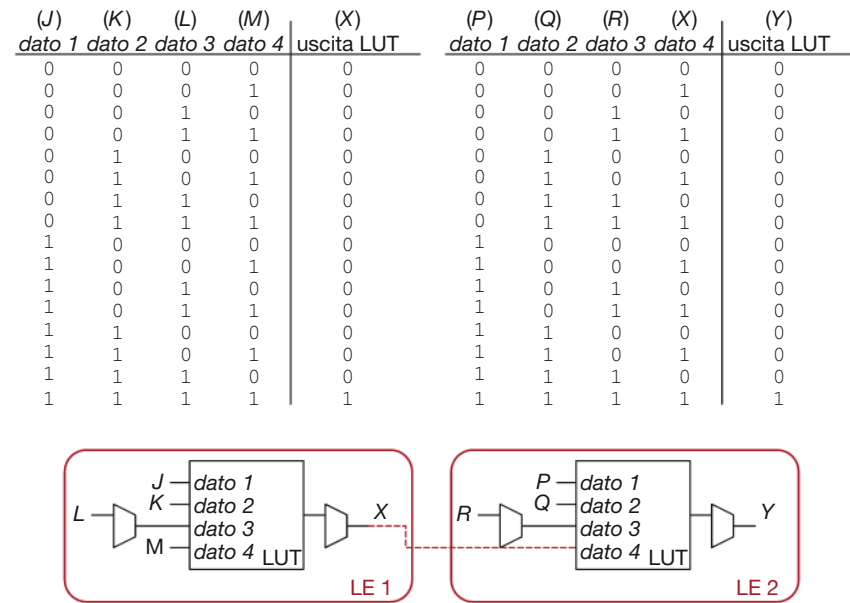
Soluzione (a) Configurare due LE. Una lookup table realizza X e la seconda realizza Y , come mostrato nella **Figura 5.60**. Per il primo LE, gli ingressi *dato 1*, *dato 2* e *dato 3* sono rispettivamente A , B e C (queste connessioni sono realizzate dai canali di instradamento). *dato 4* è un'indifferenza, ma deve essere comunque impostato a un valore, quindi viene forzato a 0. Per il secondo LE, gli ingressi *dato 1* e *dato 2* sono A e B ; gli altri ingressi della lookup table sono indifferenze e vengono, ancora una volta, forzati a 0. Configurare il multiplexer finale in modo che selezioni le uscite combinatorie delle due lookup table per generare X e Y . In generale, un singolo LE può eseguire una qualsiasi funzione che abbia fino a quattro variabili di ingresso.

Figura 5.60
Configurazione degli LE per due funzioni di al massimo quattro ingressi ciascuna.



(b) Configurare la lookup table del primo LE per eseguire $X = JKLM$ e la lookup table del secondo LE per eseguire $Y = XPQR$. Configurare il multiplexer finale per selezionare le uscite combinatorie X e Y da ogni LE. Questa configurazione è mostrata nella **Figura 5.61**. I canali di instradamento tra i due LE, indicati dalle linee tratteggiate rosse, collegano l'uscita di LE 1 all'ingresso di LE 2. In generale, in questo modo un gruppo di LE può realizzare funzioni di N variabili di ingresso.

Figura 5.61
Configurazione degli LE per una funzione di più di quattro ingressi.



		(S_0)	(S_1)	(S_1')
dato 1	dato 2	dato 3	dato 4	uscita LUT
X	X	0	0	1
X	X	0	1	0
X	X	1	0	0
X	X	1	1	0

		(S_1)	(S_0)	(S_1')
dato 1	dato 2	dato 3	dato 4	uscita LUT
X	X	0	0	0
X	X	0	1	1
X	X	1	0	0
X	X	1	1	0

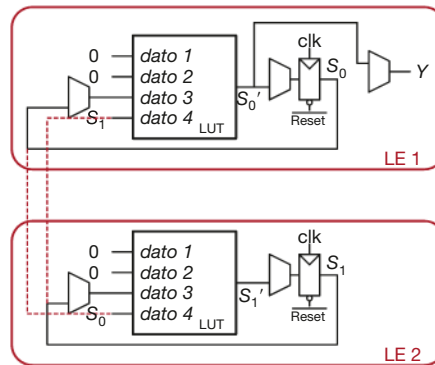


Figura 5.62
Configurazione degli LE per una
FSM con due bit di stato.

(c) La FSM ha due bit di stato ($S_{1:0}$) e un'uscita (Y). Lo stato prossimo dipende dai due bit dello stato presente. Utilizzare due LE per calcolare lo stato successivo a partire dallo stato presente, come mostrato nella **Figura 5.62**. Utilizzare i due flip-flop, uno per ogni LE, per memorizzare questo stato. I flip-flop possiedono un ingresso di reset che può essere collegato a un segnale di *Reset* esterno. Le uscite dei flip-flop vengono collegate agli ingressi delle lookup table utilizzando i multiplexer su *dato 3* e i canali di instradamento tra gli LE, come indicato nella figura dalle linee tratteggiate rosse. In generale, sarebbe necessario un altro LE per generare l'uscita Y . Tuttavia, in questo caso $Y = S_0'$, quindi Y può essere derivata da LE 1: è quindi possibile realizzare l'intera FSM con solo due LE. In generale, una FSM necessita di almeno un LE per ogni bit di stato, e potrebbe aver bisogno di più LE per la logica di uscita o per la logica di stato prossimo se queste sono troppo complesse per essere inserite in un'unica lookup table.

ESEMPIO 5.6

Ritardo di un LE. Alyssa Guastacomputer sta costruendo una macchina a stati finiti che deve funzionare a 200 MHz. Decide di utilizzare una FPGA Cyclone IV con le seguenti specifiche: $t_{LE} = 381$ ps per LE, $t_{setup} = 76$ ps, e $t_{pcq} = 199$ ps per tutti i flip-flop. Il ritardo del circuito elettrico tra gli LE è pari a 246 ps. Assumere che il tempo di hold per i flip-flop sia uguale a 0. Qual è il numero massimo di LE che Alyssa può utilizzare per il suo progetto?

Soluzione Alyssa utilizza l'Espressione 3.13 per risolvere il ritardo di propagazione massimo della logica: $t_{pd} \leq T_c - (t_{pcq} + t_{setup})$.

$t_{pd} = 5$ ns $- (0.199$ ns $+ 0.076$ ns), quindi $t_{pd} \leq 4.725$ ns. Il ritardo di ogni LE più il ritardo dei canali tra gli LE, $t_{LE+wire}$ è pari a 381 ps $+ 246$ ps $= 627$ ps. Il numero massimo di LE che Alyssa può utilizzare, N , è $N t_{LE+wire} \leq 4.725$ ns. Quindi, $N = 7$.

5.6.3 Realizzazione delle matrici di memoria*

Per minimizzare la loro dimensione e il loro costo, le RAM e le PLA solitamente utilizzano circuiti pseudo-nMOS o dinamici (vedi par. 1.7.8) invece di porte logiche convenzionali.

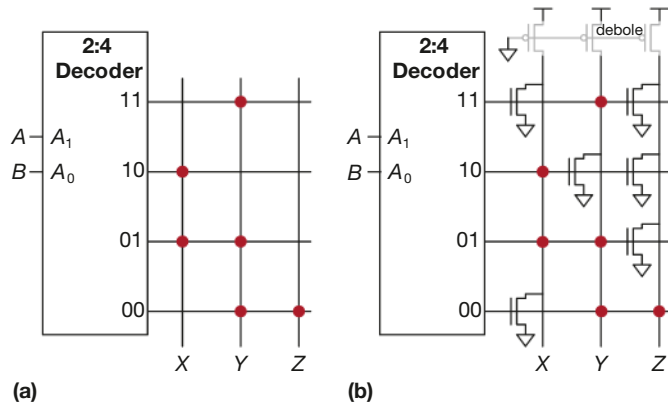
La **Figura 5.63(a)** mostra la notazione a punti per una ROM da 4×3 bit che esegue le seguenti funzioni: $X = A \oplus B$, $Y = \overline{A} + B$ e $Z = \overline{AB}$. Queste sono le stesse funzioni della Figura 5.50, nella quale gli ingressi di indirizzo sono stati rinominati A e B e le uscite di dato sono state rinominate X , Y e Z . La realizzazione mediante pseudo-nMOS è riportata nella **Figura 5.63(b)**. Ogni uscita del decoder è collegata ai gate dei transistori nMOS della sua riga. Si deve ricordare che nelle reti pseudo-nMOS il transistor debole pMOS spinge l'uscita ad ALTO solo se non è presente un percorso verso massa attraverso la rete nMOS di pull-down.

I transistori pull-down vengono inseriti a ogni giunzione senza un punto. I punti del diagramma della notazione a punti della Figura 5.63(a) sono lasciati

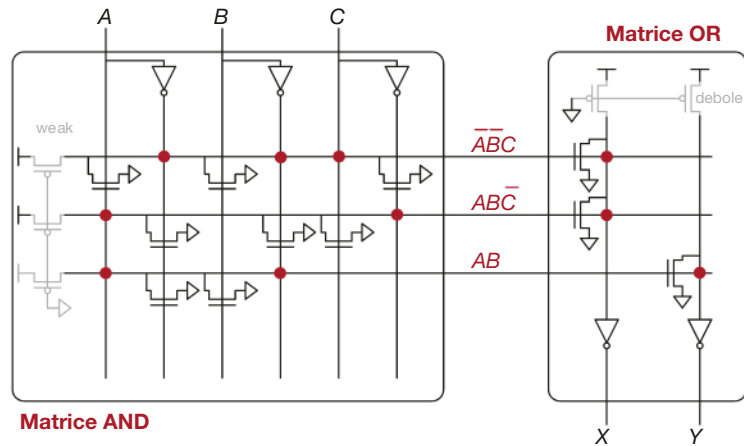
Molte ROM e PLA usano circuiti dinamici al posto degli pseudo-nMOS. Le porte dinamiche attivano il transistor pMOS solo per una frazione del tempo, risparmiando potenza quando il pMOS è spento e non serve conoscere il risultato. A parte questo aspetto, le matrici di memoria dinamiche e le pseudo-nMOS sono simili sia in progettazione sia in comportamento.

Figura 5.63

Realizzazione della ROM: (a) notazione a punti, (b) circuito pseudo nMOS.

**Figura 5.64**

PLA da $3 \times 3 \times 2$ bit usando circuiti pseudo nMOS.



visibili anche nella Figura 5.63(b) per facilitare il confronto. I transistori pull-up deboli portano il valore dell'uscita ad ALTO per ogni linea di parola nella quale non è presente un transistore pull-down. Per esempio, quando $AB = 11$, la linea di parola 11 è ALTA e i transistori su X e Z si accendono e portano i valori di quelle uscite a BASSO. L'uscita Y non ha transistori collegati alla linea di parola 11, quindi il valore di Y viene tenuto ALTO dal pull-up debole.

Anche le PLA possono essere costruite utilizzando le reti pseudo-nMOS, come mostra la **Figura 5.64**, che riprende la PLA della Figura 5.56. I transistori pull-down (nMOS) vengono posizionati sui negati dei letterali con punto nella matrice AND e sulle righe con punto nella matrice OR. Le colonne della matrice OR vengono fatte passare attraverso un negatore prima di essere collegate ai bit di uscita. Ancora una volta, i punti rossi del diagramma della notazione a punti della Figura 5.56 sono stati evidenziati anche nella Figura 5.64, per rendere più semplice il confronto tra le due.

5.7 ■ RIASSUNTO

In questo capitolo sono stati introdotti i blocchi costruttivi utilizzati in molti sistemi digitali. Questi blocchi includono circuiti aritmetici, come ad esempio i sommatore, i comparatori, i traslatori, i moltiplicatori e i divisori; includono inoltre le reti sequenziali come i contatori e i registri a scorrimento, nonché le matrici di memoria e le matrici logiche. In questo capitolo sono anche state analizzate le rappresentazioni in virgola fissa e mobile per i numeri frazionari. Nel Capitolo 7 questi blocchi costruttivi sono impiegati per la costruzione di un microprocessore.

I sommatore sono la base della maggior parte dei circuiti aritmetici. Un semisommatore (half adder) somma due ingressi a 1 bit, A e B , e produce una somma e un riporto. Un sommatore intero (full adder) estende il semisommatore in modo che questo possa accettare anche un riporto di ingresso. N full adder possono essere collegati tra loro a cascata per formare un sommatore con propagazione di riporto che somma due numeri a N bit. Questo tipo di sommatore viene chiamato sommatore a propagazione di riporto a onda perché il riporto si propaga attraverso ognuno dei sommatore. È possibile costruire sommatore più veloci utilizzando le tecniche lookahead e a prefissi.

Un sottrattore trasforma il secondo ingresso in negativo per poi sommarlo al primo. Un comparatore di valori sottrae un numero da un altro e ne determina il valore relativo basandosi sul segno del risultato. Un moltiplicatore calcola prodotti parziali utilizzando delle porte AND, e successivamente somma questi bit utilizzando dei full adder. Un divisore sottrae ripetutamente il numero divisore dal resto parziale e controlla il segno della differenza per determinare i bit del quoziente. Infine, un contatore utilizza un sommatore e un registro per effettuare un conteggio.

I numeri frazionari vengono rappresentati utilizzando le notazioni in virgola fissa o mobile. I numeri in virgola fissa sono analoghi ai numeri decimali, mentre i numeri in virgola mobile sono analoghi ai numeri rappresentati in notazione scientifica. I numeri in virgola fissa utilizzano circuiti aritmetici ordinari, mentre i numeri in virgola mobile necessitano di circuiti più complessi per estrarre ed elaborare il segno, l'esponente e la mantissa.

Le grandi memorie sono organizzate in matrici di parole. Le memorie hanno una o più porte in grado di leggere e/o scrivere le parole. Le memorie volatili, come le SRAM e le DRAM, perdono il loro stato nel momento in cui non sono più alimentate. Una SRAM è più rapida di una memoria DRAM, ma richiede un numero maggiore di transistori. Un banco di registri è un piccolo componente SRAM a più porte. Le memorie non volatili, chiamate ROM, mantengono il loro stato anche in assenza di alimentazione. Nonostante il loro nome, la maggior parte delle ROM moderne può anche essere scritta.

Le matrici di memoria costituiscono anche un metodo regolare per la realizzazione di reti logiche. I componenti di memoria possono essere utilizzati come lookup table per eseguire funzioni combinatorie. Le PLA sono costituite da connessioni dedicate tra una matrice AND e una matrice OR configurabili, e svolgono solo funzioni combinatorie. Le FPGA sono invece composte da tante piccole lookup table e da registri, e sono in grado di svolgere funzioni sia combinatorie sia sequenziali. I contenuti delle lookup table e le loro interconnessioni possono essere configurati in modo da eseguire qualsiasi funzione logica. Le FPGA moderne sono semplici da riprogrammare; sono inoltre abbastanza grandi e abbastanza economiche da poter essere usate per la realizzazione di sistemi digitali altamente sofisticati. Per questo sono molto comunemente usate nei prodotti commerciali a basso e medio volume, come pure a scopo didattico.

Esercizi

Esercizio 5.1 Qual è il ritardo per i seguenti tipi di sommatore a 64 bit? Si assuma che ogni porta a due ingressi abbia un ritardo di 150 ps e che un full adder a un bit abbia un ritardo di 450 ps.

- (a) sommatore a propagazione di riporto a onda
- (b) sommatore ad anticipazione di riporto con blocchi a 4 bit
- (c) sommatore a prefissi

Esercizio 5.2 Progettare due sommatore: un sommatore a propagazione di riporto a onda a 64 bit e un sommatore ad anticipazione di riporto a 64 bit con blocchi a 4 bit. Usare solo porte logiche a due ingressi. Ogni porta logica occupa $1.5 \mu\text{m}^2$, ha un ritardo di 50 ps e una capacità totale di 20 fF. Si assuma che la potenza statica sia trascurabile.

- (a) Confrontare area, ritardo e potenza consumata dai due sommatore (se funzionanti a 100 MHz e 1.2 V).
- (b) Discutere i compromessi tra potenza, area e ritardo.

Esercizio 5.3 Spiegare perché un progettista potrebbe preferire un sommatore a propagazione di riporto a onda a un sommatore ad anticipazione di riporto.

Esercizio 5.4 Progettare in HDL il sommatore a 16 bit a prefissi della Figura 5.7. Simulare e collaudare il modulo per dimostrarne il corretto funzionamento.

Esercizio 5.5 La rete a prefissi mostrata nella Figura 5.7 usa scatole nere per calcolare tutti i prefissi. Alcuni dei blocchi di propagazione dei segnali non sono effettivamente necessari. Progettare una “scatola grigia” che riceve i segnali G e P per i bit $i:k$ e $k-1:j$ ma produce solo G_{ij} e non P_{ij} . Ridisegnare la rete a prefissi sostituendo le scatole nere con scatole grigie ovunque possibile.

Esercizio 5.6 La rete a prefissi mostrata nella Figura 5.7 non è l'unico modo di calcolare tutti i prefissi in un tempo logaritmico. La rete Kogge-Stone è un'altra rete a prefissi che esegue le stesse operazioni usando collegamenti diversi tra le scatole nere. Cercare i sommatore Kogge-Stone e disegnarne lo schema simile alla Figura 5.7 mostrando i collegamenti tra le scatole nere.

Esercizio 5.7 Un *priority encoder* (codificatore a priorità) a N ingressi ha $\log_2 N$ uscite che codificano quale tra gli N ingressi attivi ha la priorità (vedi l'Esercizio 2.36).

- (a) Progettare un *priority encoder* a N ingressi che abbia un ritardo crescente in modo logaritmico con N . Tracciare lo schema circuitale e calcolare il ritardo del circuito in funzione del ritardo dei suoi elementi.
- (b) Codificare in HDL il progetto. Simulare e collaudare il modulo per dimostrarne il corretto funzionamento.

Esercizio 5.8 Progettare i seguenti comparatore per numeri senza segno a 32 bit, tracciando lo schema circuitale

- (a) diverso
- (b) maggiore o uguale
- (c) minore

Esercizio 5.9 Si consideri il comparatore per numeri con segno della Figura 5.12.

- (a) Mostrare un esempio di due numeri a 4 bit A e B per i quali un comparatore a 4 bit fornisce il corretto risultato per $A < B$.
- (b) Mostrare un esempio di due numeri a 4 bit A e B per i quali un comparatore a 4 bit fornisce un risultato errato per $A < B$.
- (c) In generale, quando un comparatore per numeri con segno a N bit non funziona correttamente?

Esercizio 5.10 Modificare il comparatore per numeri con segno della Figura 5.12 in modo che calcoli correttamente $A < B$ per qualsiasi valore dei due numeri a N bit A e B .

Esercizio 5.11 Progettare in HDL l'ALU a 32 bit mostrata nella Figura 5.15. Il modulo di più alto livello può essere realizzato sia in modo combinatorio sia in modo sequenziale.

Esercizio 5.12 Progettare in HDL l'ALU a 32 bit mostrata nella Figura 5.17. Il modulo di più alto livello può essere realizzato sia in modo combinatorio sia in modo sequenziale.

Esercizio 5.13 Scrivere un testbench per collaudare l'ALU dell'Esercizio 5.11, poi utilizzarlo per l'effettivo collaudo. Includere tutti i vettori di test necessari, e utilizzare sufficienti casi particolari per convincere anche gli scettici del corretto funzionamento dell'ALU.

Esercizio 5.14 Ripetere l'Esercizio 5.13 per l'ALU dell'Esercizio 5.12.

Esercizio 5.15 Costruire un'unità di confronto per numeri senza segno, che confronta i due numeri A e B . Gli ingressi dell'unità sono i segnali flag ALU (N , Z , C e V) dell'ALU della Figura 5.16, quando l'ALU esegue la sottrazione $A - B$. Le uscite dell'unità sono MaU , MiU , Mag e Min che indicano che A è maggiore o uguale (MaU), minore o uguale (MiU), maggiore (Mag) o minore (Min) di B .

- (a) Scrivere le espressioni booleane minimizzate per MaU , MiU , Mag e Min in funzione di N , Z , C e V .
- (b) Tracciare gli schemi circuitali per MaU , MiU , Mag e Min .

Esercizio 5.16 Costruire un'unità di confronto per numeri con segno, che confronta i due numeri A e B . Gli ingressi dell'unità sono i segnali *flag* ALU (N , Z , C e V) dell'ALU di Figura 5.16, quando l'ALU esegue la sottrazione $A - B$. Le uscite dell'unità sono MaU , MiU , Mag e Min che indicano che A è maggiore o uguale (MaU), minore o uguale (MiU), maggiore (Mag), o minore (Min) di B .

- (a) Scrivere le espressioni booleane minimizzate per MaU , MiU , Mag e Min in funzione di N , Z , C e V .
- (b) Tracciare gli schemi circuitali per MaU , MiU , Mag e Min .

Esercizio 5.17 Progettare un traslatore che trasla a sinistra di 2 bit l'ingresso a 32 bit. Ingresso e uscita sono entrambi a 32 bit. Descrivere a parole il progetto, tracciarne lo schema circuitale e codificarlo in HDL.

Esercizio 5.18 Progettare dei rotatori a sinistra e a destra a 4 bit. Tracciare lo schema circuitale del progetto e codificarlo in HDL.

Esercizio 5.19 Progettare un traslatore a sinistra a 8 bit utilizzando solo 24 multiplexer 2:1. Il traslatore riceve l'ingresso A a 8 bit e la quantità di traslazione, $qtrasl_{2:0}$, a 3 bit e produce l'uscita Y a 8 bit. Tracciarne lo schema circuitale.

Esercizio 5.20 Spiegare come si possa realizzare un traslatore o un rotatore a N bit utilizzando solo $N \log_2 N$ multiplexer 2:1.

Esercizio 5.21 Il "traslatore a imbuto" di Figura 5.65 può eseguire un'operazione di traslazione o di rotazione a N bit, traslando a destra di k bit l'ingresso di $2N$ bit. L'uscita Y è costituita dagli N bit meno significativi del risultato. Gli N bit più significativi dell'ingresso sono chiamati B e gli N bit meno significativi C . Mediante appropriati valori di B , C e k è possibile eseguire ogni tipo di traslazione o rotazione. Spiegare come devono essere tali valori in funzione di A , $qtrasl$ e N per

- traslare il valore logico di A a destra di $qtrasl$
- traslare il valore aritmetico di A a destra di $qtrasl$
- traslare il valore di A a sinistra di $qtrasl$
- ruotare il valore di A a destra di $qtrasl$
- ruotare il valore di A a sinistra di $qtrasl$

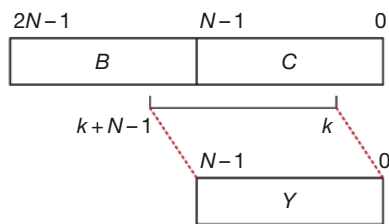


Figura 5.65 Traslatore a imbuto.

Esercizio 5.22 Trovare il percorso critico per il moltiplicatore 4×4 di Figura 5.20 in termini di ritardo di una porta AND (t_{AND}) e di ritardo di un sommatore completo a un bit (t_{FA}). Qual è il ritardo di un moltiplicatore $N \times N$ costruito nello stesso modo?

Esercizio 5.23 Trovare il percorso critico per il divisore 4×4 di Figura 5.21 in termini di ritardo di un multiplexer 2:1 (t_{MUX}) di ritardo di un sommatore completo a un bit (t_{FA}) e di ritardo di un negatore (t_{NOT}). Qual è il ritardo di un moltiplicatore $N \times N$ costruito nello stesso modo?

Esercizio 5.24 Progettare un moltiplicatore per numeri in complemento a due.

Esercizio 5.25 Un'unità di estensione del segno estende un numero in complemento a due da M a N bit (con $N > M$) ricopiando il bit più significativo dell'ingresso nei bit più significativi dell'uscita (vedi par. 1.4.6). Riceve il numero A a M bit e produce l'uscita Y a N bit. Tracciare lo schema circuitale di un'unità di estensione del segno da 4 a 8 bit e codificarla in HDL.

Esercizio 5.26 Un'unità di estensione dello zero estende un numero senza segno da M a N bit (con $N > M$) inserendo degli zeri nei bit più significativi dell'uscita. Tracciare lo schema circuitale di un'unità di estensione dello zero da 4 a 8 bit e codificarla in HDL.

Esercizio 5.27 Calcolare $111001.000_2 / 001100.000_2$ in binario usando la regola di divisione imparata alle elementari, mostrando i passaggi.

Esercizio 5.28 Qual è l'intervallo di numeri rappresentabili dai seguenti sistemi numerici?

- Numeri in virgola fissa a 24 bit, con 12 bit di parte intera e 12 di parte frazionaria.
- Numeri in modulo e segno a 24 bit, con 12 bit di parte intera e 12 di parte frazionaria.
- Numeri in complemento a due a 24 bit, con 12 bit di parte intera e 12 di parte frazionaria.

Esercizio 5.29 Convertire i seguenti numeri decimali in numeri in modulo e segno in virgola fissa a 16 bit, con 8 bit di parte intera e 8 di parte frazionaria, scrivendo i risultati in esadecimale.

- 13.5625
- 42.3125
- 17.15625

Esercizio 5.30 Convertire i seguenti numeri decimali in numeri in modulo e segno in virgola fissa a 12 bit, con 6 bit di parte intera e 6 di parte frazionaria, scrivendo i risultati in esadecimale.

- 30.5
- 16.25
- 8.078125

Esercizio 5.31 Convertire i numeri decimali dell'Esercizio 5.29 in numeri in complemento a due in virgola fissa a 16 bit, con 8 bit di parte intera e 8 di parte frazionaria, scrivendo i risultati in esadecimale.

Esercizio 5.32 Convertire i numeri decimali dell'Esercizio 5.30 in numeri in complemento a due in virgola fissa a 12 bit, con 6 bit di parte intera e 6 di parte frazionaria, scrivendo i risultati in esadecimale.

Esercizio 5.33 Convertire i numeri decimali dell'Esercizio 5.29 in numeri in virgola mobile in formato IEEE 754 a singola precisione, scrivendo i risultati in esadecimale.

Esercizio 5.34 Convertire i numeri decimali dell'Esercizio 5.30 in numeri in virgola mobile in formato IEEE 754 a singola precisione, scrivendo i risultati in esadecimale.

Esercizio 5.35 Convertire in decimale i seguenti numeri binari in complemento a due in virgola fissa. La virgola implicita è mostrata esplicitamente per aiutare l'interpretazione.

- 0101.1000
- 1111.1111
- 1000.0000

Esercizio 5.36 Ripetere l'Esercizio 5.35 per i seguenti numeri binari in complemento a due in virgola fissa.

- (a) 011101.10101
- (b) 100110.11010
- (c) 101000.00100

Esercizio 5.37 Quando si sommano due numeri in virgola mobile, il numero con l'esponente minore viene traslato. Perché? Spiegare a parole e fornire un esempio per giustificare le proprie affermazioni.

Esercizio 5.38 Sommare i seguenti numeri in virgola mobile in formato IEEE 754 a singola precisione.

- (a) C0123456 + 81C564B7
- (b) D0B10301 + D1B43203
- (c) 5EF10324 + 5E039020

Esercizio 5.39 Sommare i seguenti numeri in virgola mobile in formato IEEE 754 a singola precisione.

- (a) C0D20004 + 72407020
- (b) C0D20004 + 40DC0004
- (c) (5FBE4000 + 3FF80000) + DFDE4000 (Perché il risultato va contro quanto ci si sarebbe aspettato? Spiegare)

Esercizio 5.40 Estendere i passi del paragrafo 5.3.2 relativi alla somma in virgola mobile per operare sia con numeri negativi sia con numeri positivi.

Esercizio 5.41 Si considerino i numeri in virgola mobile in formato IEEE 754 a singola precisione.

- (a) Quanti numeri possono essere rappresentati in virgola mobile in formato IEEE 754 a singola precisione non contando $\pm\infty$ e NaN?
- (b) Quanti altri numeri sarebbero rappresentati se si eliminassero $\pm\infty$ e NaN?
- (c) Spiegare perché $\pm\infty$ e NaN sono riservate rappresentazioni speciali.

Esercizio 5.42 Si considerino i due numeri decimali 245 e 0.0625.

- (a) Codificare i due numeri in virgola mobile a singola precisione, scrivendo i risultati in esadecimale.
- (b) Eseguire un confronto di valore dei due numeri a 32 bit del punto (a). In altre parole, interpretare i due numeri a 32 bit come numeri interi in complemento a due e confrontarli. Il confronto dà il risultato corretto?
- (c) Si vuole introdurre un nuovo formato per numeri in virgola mobile a singola precisione. Il formato è identico allo standard IEEE 754 tranne per il fatto che l'esponente viene codificato in complemento a due invece di utilizzare l'eccesso. Codificare i due numeri in questo formato, scrivendo i risultati in esadecimale.
- (d) Il confronto fra interi funziona nel nuovo formato introdotto al punto (c)?
- (e) Perché è conveniente per il confronto tra interi lavorare con numeri in virgola mobile?

Esercizio 5.43 Progettare in HDL un sommatore in virgola mobile a singola precisione. Prima di codificare il progetto in HDL, tracciarne lo schema circuitale. Simulare e collaudare il sommatore per convincere gli scettici del suo corretto funzionamento. Limitarsi ai numeri positivi, usare troncamento invece di arrotondamento e trascurare i casi speciali riportati nella Tabella 5.2.

Esercizio 5.44 In questo problema si esamina il progetto di un moltiplicatore in virgola mobile a 32 bit, con due ingressi e un'uscita tutti in virgola mobile a 32 bit. Limitarsi ai numeri positivi, usare troncamento invece di arrotondamento e trascurare i casi speciali riportati in Tabella 5.2.

- (a) Scrivere i passi necessari per eseguire la moltiplicazione in virgola mobile a 32 bit.
- (b) Tracciare lo schema circuitale di un moltiplicatore in virgola mobile a 32 bit.
- (c) Progettare in HDL il moltiplicatore in virgola mobile a 32 bit. Simulare e collaudare il moltiplicatore per convincere gli scettici del suo corretto funzionamento.

Esercizio 5.45 In questo problema si esamina il progetto di un sommatore a prefissi a 32 bit.

- (a) Tracciare lo schema circuitale del progetto.
- (b) Codificare in HDL il sommatore a prefissi a 32 bit. Simulare e collaudare il progetto per dimostrarne il corretto funzionamento.
- (c) Qual è il ritardo del sommatore a prefissi a 32 bit del punto (a)? Si assuma che ogni porta logica a due ingressi abbia un ritardo di 100 ps.
- (d) Progettare una versione a pipeline del sommatore a prefissi a 32 bit. Tracciare lo schema circuitale del progetto. Quanto velocemente può lavorare questa versione pipeline, assumendo un sovraccarico per il sequenziamento ($t_{pcq} + t_{setup}$) di 80 ps? Rendere il progetto il più veloce possibile.
- (e) Codificare in HDL il sommatore a prefissi a 32 bit pipeline.

Esercizio 5.46 Un incrementatore somma 1 a un numero a N bit. Costruire un incrementatore a 8 bit usando semisommatori.

Esercizio 5.47 Costruire un contatore sincrono $Up/Down$, con ingressi *Reset* e *Up*. Quando *Reset* vale 1 tutte le uscite vanno a 0. Altrimenti, se $Up = 1$ il contatore si incrementa, se $Up = 0$ il contatore si decrementa.

Esercizio 5.48 Costruire un contatore a 32 bit che somma 4 a ogni colpo di clock, con ingressi di reset e di clock. In caso di reset, tutte le uscite vanno a 0.

Esercizio 5.49 Modificare il contatore dell'Esercizio 5.48 in modo che possa sia incrementare di 4 il proprio valore sia caricare un nuovo valore D a 32 bit a ogni colpo di clock, in base a un segnale di controllo *Carica*. Quando *Carica* = 1 il contatore carica il nuovo valore D .

Esercizio 5.50 Un contatore *Johnson* consiste di un registro a scorrimento a N bit con un segnale di reset. L'uscita del registro a scorrimento (S_{out}) viene negata e reinserita dell'ingresso (S_{in}). In caso di reset, tutti i bit di uscita vanno a 0.

- (a) Mostrare la sequenza di uscita, $Q_{3:0}$, di un contatore *Johnson* a 4 bit subito dopo il reset.
- (b) Quanti cicli passano prima che il contatore *Johnson* ripeta la sequenza? Spiegare il perché.
- (c) Progettare un contatore decimale usando un contatore *Johnson* a 5 bit, dieci porte AND, e negatori. Il contatore decimale ha un clock, un reset e dieci uscite $Y_{9:0}$ di cui sempre solo una a 1 (one hot). Al reset viene forzata a 1 Y_0 . A ogni successivo colpo di clock viene forzata a 1 l'uscita successiva. Dopo dieci cicli la sequenza si ripete. Tracciare uno schema circuitale del contatore.
- (d) Quali sono i possibili vantaggi di un contatore *Johnson* rispetto a un contatore tradizionale?

Esercizio 5.51 Codificare in HDL un flip-flop a 4 bit scansionabile come quello della Figura 5.38. Simulare e collaudare il progetto per dimostrarne il corretto funzionamento.

Esercizio 5.52 La lingua italiana ha un buon livello di ridondanza che consente di ricostruire trasmissioni alterate. Anche i dati binari possono essere trasmessi utilizzando codifiche ridondanti per consentire la correzione di errori. Per esempio, il numero 0 potrebbe essere codificato 00000 e il numero 1 diventare 11111. Tali codifiche potrebbero poi essere trasmesse in un canale disturbato che potrebbe alterare il valore di uno o due bit. In queste ipotesi, il ricevitore sarebbe in grado di ricostruire il dato originale perché lo 0 avrebbe ancora almeno tre bit a 0, e l'1 almeno tre bit a 1.

- (a) Proporre una codifica per spedire 00, 01, 10 o 11 usando cinque bit in modo tale che errori che modifichino un bit siano correggibili. Attenzione: le codifiche 00000 per 0 e 11111 per 1 non funzionano.
- (b) Progettare un circuito che riceve un codice di cinque bit e restituisce uno dei quattro valori 00, 01, 10 o 11 anche in presenza di un bit errato.
- (c) Supponendo di voler usare una diversa codifica a cinque bit, come si può impostare il progetto in modo che sia possibile farlo senza variare la struttura circuitale?

Esercizio 5.53 La memoria EEPROM flash, più nota semplicemente come memoria flash, è un'invenzione relativamente recente che ha rivoluzionato l'elettronica di consumo. Cercare informazioni per spiegare come funziona la memoria flash. Utilizzare un diagramma per illustrare il funzionamento del gate fluttuante. Descrivere come viene programmato un bit di memoria. Citare le fonti usate per rispondere alle domande.

Esercizio 5.54 La squadra di studio della vita extraterrestre ha appena scoperto la presenza di alieni in fondo al Lago d'Iseo. Deve ora progettare un circuito per classificare gli alieni in base al pianeta di possibile provenienza, usando le caratteristiche misurate dalla sonda dell'Azienda Spaziale Italiana: i livelli di verde, di marrone, di viscidità e di bruttezza. Approfonditi consulti con xenobiologi portano alle seguenti conclusioni:

- Se l'alieno è verde e viscido, oppure brutto, marrone e viscido, potrebbe venire da Marte.

- Se è brutto, marrone e viscido, oppure brutto e non viscido, oppure verde e viscido, potrebbe venire da Venere.
- Se è marrone e non brutto né viscido, oppure verde e viscido, potrebbe venire da Giove.

Si noti che questa è una suddivisione scientificamente inesatta: per esempio, una forma di vita screziata di verde e di marrone, viscida ma non brutta potrebbe venire sia da Marte sia da Giove.

- (a) Programmare una PLA $4 \times 4 \times 3$ per identificare l'alieno. Si può usare la notazione a punti.
- (b) Programmare una ROM 16×3 per identificare l'alieno. Si può usare la notazione a punti.
- (c) Codificare il progetto in HDL.

Esercizio 5.55 Realizzare le seguenti funzioni utilizzando una sola ROM 16×3 . Usare la notazione a punti per indicare il contenuto della ROM.

- (a) $X = AB + \overline{BCD} + \overline{A}\overline{B}$
- (b) $Y = AB + BD$
- (c) $Z = A + B + C + D$

Esercizio 5.56 Realizzare le funzioni dell'Esercizio 5.55 utilizzando una PLA $4 \times 8 \times 3$. Si può usare la notazione a punti.

Esercizio 5.57 Specificare le dimensioni di una ROM che si potrebbe usare per realizzare ciascuna delle seguenti reti combinatorie. L'utilizzo di una ROM è una buona scelta di progetto? Spiegare perché sì o perché no.

- (a) un sommatore/sottrattore con R_{in} e R_{out}
- (b) un moltiplicatore 8×8
- (c) un priority encoder a 16 bit (vedi l'Esercizio 2.36)

Esercizio 5.58 Si considerino le reti a ROM di Figura 5.66. Per ogni riga si può sostituire il circuito nella colonna I con quello nella colonna II a patto di programmare opportunamente la ROM di tale colonna?

Esercizio 5.59 Quanti LE di FPGA Cyclone IV sono necessari per realizzare ciascuna delle seguenti funzioni? Mostrare come configurare uno o più LE per realizzare tali funzioni. Si può rispondere senza necessariamente effettuare la sintesi logica.

- (a) la funzione combinatoria dell'Esercizio 2.13(c)
- (b) la funzione combinatoria dell'Esercizio 2.17(c)
- (c) la funzione a due uscite dell'Esercizio 2.24
- (d) la funzione dell'Esercizio 2.35
- (e) un priority encoder a 4 ingressi (vedi l'Esercizio 2.36)

Esercizio 5.60 Ripetere l'Esercizio 5.59 per le seguenti funzioni:

- (a) un priority encoder a 8 ingressi (vedi l'Esercizio 2.36)
- (b) un decoder 3:8
- (c) un sommatore a propagazione di riporto a 4 bit (senza riporto in ingresso né in uscita)
- (d) la FSM dell'Esercizio 3.22
- (e) il contatore a codice Gray dell'Esercizio 3.27

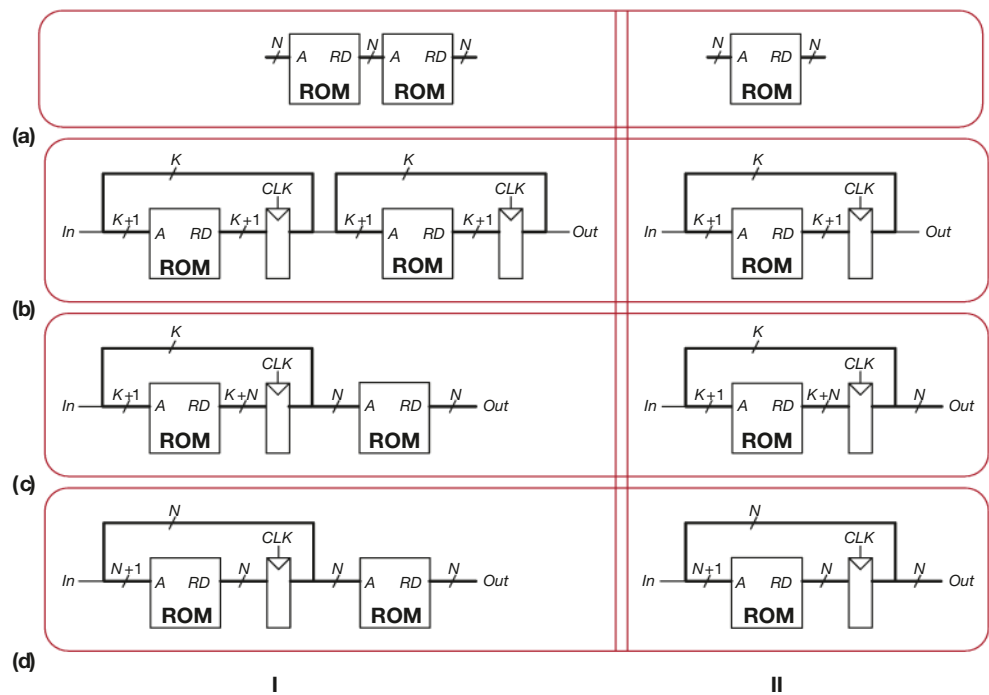


Figura 5.66 Circuiti a ROM.

Esercizio 5.61 Si consideri l’LE di FPGA Cyclone IV mostrata nella Figura 5.59. Secondo il manuale, le sue specifiche temporali sono quelle riportate nella Tabella 5.5.

(a) Qual è il minimo numero di LE di FPGA Cyclone IV necessari per realizzare la FSM della Figura 3.26?

Tabella 5.5 Temporizzazioni di Cyclone IV.

Nome	Valore (ps)
t_{pcq}, t_{ccq}	199
t_{setup}	76
t_{hold}	0
t_{pd} (per LE)	381
t_{wire} (tra due LE)	246
t_{skew}	0

- (b) Senza ritardi nella propagazione del clock, qual è la massima frequenza alla quale la FSM opera correttamente?
- (c) Con un ritardo nella propagazione del clock di 3 ns, qual è la massima frequenza alla quale la FSM opera correttamente?

Esercizio 5.62 Ripetere l’Esercizio 5.61 per la FSM della Figura 3.31(b).

Esercizio 5.63 Si vuole usare una FPGA per realizzare un separatore di caramelle con un sensore di colore e motori in grado di mandare le caramelle rosse in un barattolo e quelle verdi in un altro. Il progetto va realizzato con una FSM basata su una FPGA Cyclone IV. Secondo il manuale, le specifiche temporali della FPGA sono quelle riportate nella Tabella 5.5. Il separatore di caramelle deve operare a 100 MHz. Qual è il massimo numero di LE che possono trovarsi sul percorso critico? Qual è la massima velocità alla quale la FSM lavora?

Domande di valutazione

Queste domande sono state poste a candidati per un posto di lavoro nell’ambito della progettazione di sistemi digitali.

Domanda 5.1 Ci sa dire qual è il massimo risultato possibile di una moltiplicazione tra due numeri interi senza segno a N bit?

Domanda 5.2 La codifica BCD (*Binary Coded Decimal*) usa 4 bit per codificare ogni cifra decimale. Per esempio, 42_{10} viene rappresentato come 01000010_{BCD} . Ci sa spiegare

perché i calcolatori possono dover usare la rappresentazione BCD?

Domanda 5.3 Progetti un circuito che somma due numeri BCD senza segno a 8 bit (tenga presente la domanda 5.2). Tracci lo schema circuitale del suo progetto e codifichi in HDL un modulo per il sommatore BCD. Gli ingressi sono A, B e R_{in} , le uscite sono S e R_{out} . R_{in} e R_{out} sono i riporti a un bit, mentre A, B e S sono numeri BCD a 8 bit.

Architettura

Capitolo

6

6.1 Introduzione

6.2 Il linguaggio *assembly*

6.3 Programmare

6.4 Linguaggio macchina

6.5 Compilare, assemblare e caricare*

6.6 Qualche dettaglio

6.7 Evoluzione dell'architettura ARM

6.8 Un'altra prospettiva: l'architettura x86

6.9 Riassunto

6.1 ■ INTRODUZIONE

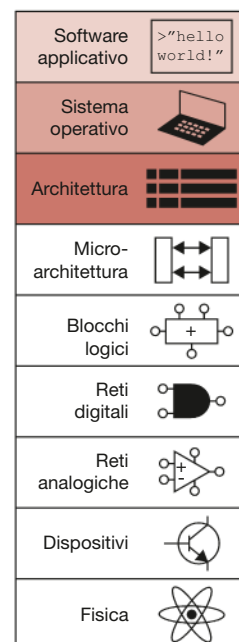
Nei capitoli precedenti si sono introdotti i principi del progetto di sistemi digitali e i relativi blocchi costruttivi. In questo capitolo si sale di qualche livello di astrazione per definire l'**architettura** di un calcolatore, ovvero come il calcolatore viene visto dal programmatore. Tale architettura è definita in termini di set di istruzioni (il linguaggio) e di locazione degli operandi (registri e memoria). Esistono molte diverse architetture, come ARM, x86, SPARC e PowerPC.

Il primo passo per comprendere l'architettura di un calcolatore è conoscere il linguaggio. Le parole che lo costituiscono sono chiamate **istruzioni**, e il vocabolario delle possibili parole è il **set** (insieme) **di istruzioni** del calcolatore. Tutti i programmi in esecuzione su un calcolatore usano il medesimo set di istruzioni: anche applicativi software complessi come elaboratori di testi e fogli elettronici devono essere tradotti in sequenze di semplici istruzioni come addiziona, sottrai, salta. Le istruzioni indicano sia l'operazione da eseguire sia gli operandi da usare, che si possono trovare in memoria, nei registri del processore o all'interno delle istruzioni stesse.

L'hardware del calcolatore capisce solo zeri e uni, quindi le istruzioni devono essere codificate come stringhe binarie in quello che si definisce **linguaggio macchina**: esattamente come l'uomo usa lettere per rappresentare i linguaggi naturali, il calcolatore usa simboli binari per rappresentare il linguaggio macchina. L'architettura ARM rappresenta ogni istruzione con una parola di 32 bit (*word*).

I processori sono dunque sistemi digitali che leggono ed eseguono istruzioni scritte in linguaggio macchina; tuttavia è molto poco agevole per l'uomo leggere e scrivere il linguaggio macchina, quindi si preferisce rappresentare le istruzioni in una forma simbolica chiamata **linguaggio assembly** (letteralmente "montaggio", ma non si usa mai).

I set di istruzioni di architetture diverse sono dialetti piuttosto che linguaggi differenti: tutte le architetture definiscono infatti alcune istruzioni di base, come addiziona, sottrai, salta, che operano sulla memoria o sui registri



del processore. Quindi una volta imparato un set di istruzioni è molto facile comprenderne altri.

L'architettura del calcolatore non definisce la sottostante struttura circuitale: esistono spesso differenti realizzazioni circuitali della medesima architettura. Per esempio, le ditte Intel e AMD (*Advanced Micro Devices*) producono entrambe vari microprocessori tutti appartenenti alla medesima architettura x86. Possono quindi eseguire gli stessi programmi, ma utilizzando strutture circuitali diverse, quindi con diversi criteri relativamente a prestazioni, costo, consumo di potenza: alcuni processori sono ottimizzati per macchine server ad alte prestazioni, altri per minimizzare il consumo di potenza nei dispositivi portatili. L'organizzazione di dettaglio dei registri del processore, della memoria, dell'ALU e degli altri blocchi costruttivi viene chiamata **microarchitettura** ed è oggetto del Capitolo 7; possono esistere anche diverse microarchitetture per la stessa architettura.

L'“architettura ARM” descritta in questo testo è la versione 4 (ARMv4) che costituisce il nucleo del set di istruzioni. Il paragrafo 6.7 riassume le nuove caratteristiche delle versioni 5-8. Il manuale ARM (*Architecture Reference Manual*) disponibile online è il riferimento ufficiale dell'architettura.

In questo testo si introduce l'architettura ARM, sviluppata per la prima volta nel 1980 da Acom Computer Group, da cui si è scorporata Advanced RISC Machines Ltd., oggi nota appunto come ARM. Ogni anno vengono venduti più di 10 miliardi di processori ARM: quasi tutti i telefoni cellulari e i tablet ne contengono più di uno. L'architettura è usata praticamente ovunque, dai flipper, alle macchine fotografiche, ai robot, alle automobili e ai server. ARM è un'azienda insolita nel senso che non vende direttamente i processori, ma concede la licenza ad altri produttori di produrre i suoi processori come parti di sistemi su singolo chip. Per esempio, Samsung, Altera, Apple e Qualcomm costruiscono tutte processori ARM, sia utilizzando microarchitetture comperate da ARM sia sviluppando internamente microarchitetture con licenza di ARM. Si è deciso di usare ARM nel testo perché è un leader di mercato e perché è un'architettura “pulita”, con poche idiosincrasie. Si inizia introducendo le istruzioni del linguaggio assembly, le locazioni degli operandi e i costrutti di programmazione più comuni come i salti, i cicli, la gestione di vettori e matrici, le chiamate di sottoprogrammi. Quindi si descrive come il linguaggio assembly viene tradotto in linguaggio macchina e come un programma viene caricato in memoria ed eseguito.

Nel capitolo si motiva il progetto dell'architettura ARM facendo riferimento ai quattro principi definiti da David Patterson e John Hennessy nel loro libro *Computer Organization and Design*: (1) la regolarità favorisce la semplicità; (2) rendere veloci le cose frequenti; (3) più piccolo è più veloce; e (4) un buon progetto richiede buoni compromessi.

6.2 ■ IL LINGUAGGIO ASSEMBLY

Il **linguaggio assembly** è la forma leggibile dall'uomo del linguaggio nativo del calcolatore. Ogni istruzione in linguaggio assembly specifica sia l'operazione da svolgere sia gli operandi da elaborare. Si introducono quindi alcune semplici operazioni aritmetiche e si vede come scriverle in linguaggio assembly. Poi si definiscono gli operandi delle istruzioni ARM: registri, memoria e costanti.

Nel capitolo si assume che il lettore abbia una certa familiarità con linguaggi di alto livello come C, C++ o Java (i tre linguaggi sono praticamente identici per molti degli esempi che seguono: quando differiscono, si usa il C). L'Appendice C fornisce un'introduzione al linguaggio C per chi non ha alcuna conoscenza pregressa di programmazione.

6.2.1 Istruzioni

L'operazione più comune dei calcolatori è l'addizione. L'**Esempio di Codice 6.1** mostra il codice da scrivere per sommare le variabili *b* e *c* scrivendo il

Si usa l'ambiente di sviluppo di Keil (MDK-ARM, *Microcontroller Development Kit*) per compilare, assemblare e simulare gli esempi di codice di questo capitolo. MDK-ARM è uno strumento di sviluppo gratuito, dotato di un compilatore ARM completo. La guida alle attività di laboratorio disponibili sul sito Web relativo a questo testo (vedi la Prefazione) mostra come installare e utilizzare lo strumento per codificare, compilare, simulare e collaudare programmi sia in linguaggio C sia in linguaggio assembly.

ESEMPIO DI CODICE 6.1 ADDIZIONE

Codice di alto livello	Codice <i>assembly</i> ARM
<code>a = b + c;</code>	<code>ADD a, b, c</code>

risultato in `a`. Il codice è scritto a sinistra in un linguaggio di alto livello (usando la sintassi di C, C++ e Java) e riscritto a destra nel linguaggio assembly di ARM. Si noti che le istruzioni C terminano con il punto e virgola.

La prima parte dell'istruzione assembly, `ADD`, è chiamata **mnemonico** e indica l'operazione da eseguire. Tale operazione deve essere eseguita su `b` e `c`, gli **operandi sorgente** (*source*), e il risultato deve essere scritto in `a`, l'**operando destinazione** (*destination*).

ESEMPIO DI CODICE 6.2 SOTTRAZIONE

Codice di alto livello	Codice <i>assembly</i> ARM
<code>a = b - c;</code>	<code>SUB a, b, c</code>

L'**Esempio di Codice 6.2** mostra che la sottrazione è simile all'addizione: il formato è lo stesso dell'istruzione `ADD`, tranne per l'operazione da svolgere che diventa `SUB`. Questo formato costante delle istruzioni è un esempio di applicazione del primo principio:

Principio progettuale n. 1: la regolarità favorisce la semplicità.

Istruzioni con un numero costante di operandi – in questo caso, due sorgenti e una destinazione – sono più facili da gestire in hardware. Istruzioni di alto livello più complesse vengono tradotte in sequenze di più istruzioni ARM, come mostrato nell'**Esempio di Codice 6.3**.

ESEMPIO DI CODICE 6.3 CODICE PIÙ COMPLESSO

Codice di alto livello	Codice <i>assembly</i> ARM
<code>a = b + c - d; // commento su una sola riga</code>	<code>ADD t, b, c ; t = b + c</code>
<code> /* commento su</code>	<code>SUB a, t, d ; a = t - d</code>
<code> più righe */</code>	

Nei linguaggi di alto livello, commenti su riga singola cominciano con `//` e continuano fino a fine riga, mentre commenti su più righe cominciano con `/*` e terminano con `*/`. Nel linguaggio assembly di ARM si usano solo commenti su riga singola: cominciano con il punto e virgola (`;`) e continuano fino a fine riga. L'Esempio di Codice 6.3 richiede una variabile temporanea `t` per memorizzare il risultato intermedio. L'uso di più istruzioni assembly per eseguire attività complesse è un esempio di applicazione del secondo principio:

Principio progettuale n. 2: rendere veloci le cose frequenti.

Il set di istruzioni di ARM rende veloci le cose frequenti inserendo solo istruzioni semplici usate spesso. Il numero di diverse istruzioni è tenuto basso in modo tale che il circuito richiesto per decodificare istruzioni e operandi sia semplice, piccolo e veloce. Elaborazioni più complesse eseguite più raramente sono realizzate con sequenze di molteplici istruzioni semplici. ARM è quindi un calcolatore con architettura RISC (*Reduced Instruction Set Computer*, cal-

colatore con un insieme limitato di istruzioni); al contrario, architetture con molte istruzioni complesse, come la famiglia Intel x86, sono definite CISC (*Complex Instruction Set Computer*). Per esempio, x86 definisce un’istruzione “muovi stringa” che copia una sequenza (stringa) di caratteri da una parte all’altra della memoria. Tale operazione richiede molte istruzioni RISC (magari anche centinaia) per essere eseguita, ma il costo di realizzare istruzioni complesse in un’architettura CISC comporta hardware aggiuntivo e sovraccarichi che rallentano anche le istruzioni semplici.

L’architettura RISC minimizza la complessità hardware e la codifica necessaria per le istruzioni mantenendo piccolo l’insieme di diverse istruzioni. Per esempio, un set di istruzioni con 64 diverse istruzioni semplici richiede $\log_2 64 = 6$ bit per codificare le diverse operazioni, mentre un set di istruzioni con 256 istruzioni complesse richiede $\log_2 256 = 8$ bit per codificare le diverse operazioni. In un’architettura CISC, anche se le istruzioni più complesse vengono usate solo raramente, aggiungono sovraccarico a tutte le istruzioni, anche le più semplici e frequenti.

6.2.2 Operandi: registri, memoria e costanti

Un’istruzione lavora su **operandi**. Nell’Esempio di Codice 6.1, le variabili *a*, *b* e *c* sono tutte operandi. Ma i calcolatori lavorano su zeri e uni, non su nomi di variabili: le istruzioni richiedono quindi locazioni fisiche dalle quali prelevare i dati binari. Gli operandi possono essere memorizzati nei registri del processore, o in memoria, oppure essere costanti memorizzate nelle istruzioni stesse. I calcolatori usano locazioni diverse per gli operandi al fine di ottimizzare velocità e capienza: gli operandi memorizzati come costanti o nei registri sono veloci da raggiungere ma possono contenere solo piccole quantità di dati. Dati aggiuntivi devono essere prelevati dalla memoria, capiente ma lenta. ARM (prima della versione ARMv8) è chiamata architettura a 32 bit perché opera su dati a 32 bit.

La versione 8 dell’architettura ARM è stata estesa a 64 bit, ma su questo libro ci si focalizza sulla versione a 32 bit.

Registri

Le istruzioni hanno bisogno di raggiungere rapidamente gli operandi per poter essere eseguite velocemente, ma gli operandi salvati in memoria richiedono tempi lunghi per essere recuperati. Per questo motivo, quasi tutte le architetture definiscono un limitato numero di registri per memorizzare gli operandi più usati. L’architettura ARM usa 16 registri, globalmente indicati come register file. Meno sono i registri, più rapidamente sono accessibili. Ecco un esempio di applicazione del terzo principio:

Principio progettuale n. 3: più piccolo è più veloce.

Cercare un’informazione dai pochi libri presenti sulla scrivania è molto più rapido che cercarla negli scaffali di una biblioteca. Allo stesso modo, leggere un dato da pochi registri è molto più rapido che leggerlo da una memoria grande. Il register file è generalmente costituito da un piccolo *array* (vettore) di memoria SRAM (*vedi* par. 5.5.3).

L’**Esempio di Codice 6.4** mostra l’istruzione **ADD** con operandi registro: i nomi dei registri di ARM sono preceduti dalla lettera “R”. Le variabili *a*, *b* e *c* sono arbitrariamente memorizzate in R0, R1 e R2. L’istruzione somma i

ESEMPIO DI CODICE 6.4 OPERANDI REGISTRO	
Codice di alto livello	Codice <i>assembly</i> ARM
<code>a = b + c;</code>	<code>; R0 = a, R1 = b, R2 = c</code> <code>ADD R0, R1, R2 ; a = b + c</code>

ESEMPIO DI CODICE 6.5 REGISTRI PER DATI**Codice di alto livello**

```
a = b + c - d;
```

Codice assembly ARM

```
; R0 = a, R1 = b, R2 = c, R3 = d; R4 = t
ADD R4, R1, R2    ; t = b + c
SUB R0, R4, R3    ; a = t - d
```

valori a 32 bit contenuti in R1 (a) e R2 (b) e scrive il risultato a 32 bit in R0 (c). L'**Esempio di Codice 6.5** mostra il codice assembly di ARM che usa R4 per memorizzare il risultato intermedio dell'operazione $b + c$.

ESEMPIO 6.1

Tradurre codice di alto livello in linguaggio assembly. Tradurre il seguente codice di alto livello in linguaggio assembly di ARM. Assumere che le variabili a-c siano memorizzate in R0-R2 e le variabili f-j in R3-R7.

```
a = b - c;
f = (g + h) - (i + j);
```

Soluzione Il programma usa quattro istruzioni in linguaggio assembly.

```
; codice assembly di ARM
; R0 = a, R1 = b, R2 = c, R3 = f, R4 = g, R5 = h, R6 = i, R7 = j
SUB R0, R1, R2    ; a = b - c
ADD R8, R4, R5    ; R8 = g + h
ADD R9, R6, R7    ; R9 = i + j
SUB R3, R8, R9    ; f = (g + h) - (i + j)
```

Registri di ARM

La **Tabella 6.1** elenca il nome e l'utilizzo dei 16 registri di ARM. R0-R12 sono usati per memorizzare variabili; R0-R3 hanno anche un utilizzo particolare nelle chiamate a sottoprogramma; R13-R15 sono anche chiamati SP, LR e PC, e sono descritti più avanti nel capitolo.

Costanti/Immediati

Oltre ai registri, le istruzioni ARM possono usare operandi costanti o **immediati**, così chiamati perché i loro valori sono immediatamente disponibili nell'istruzione stessa e non richiedono accessi a registri o memoria. L'**Esempio di Codice 6.6** mostra l'istruzione `ADD` che somma un immediato a un registro. In linguaggio assembly, l'immediato è preceduto dal carattere `#` e può essere scritto in decimale o esadecimale. In linguaggio assembly di ARM, le costanti esadecimali cominciano con `0x` come in linguaggio C. Gli immediati sono numeri senza segno (*unsigned*, contrapposti ai numeri con segno o *signed*) a 8 o 12 bit codificati nel modo particolare descritto nel paragrafo 6.4.

Tabella 6.1 Registri di ARM.

Nome	Utilizzo
R0	Parametro/valore da restituire/variabile temporanea
R1-R3	Parametri/variabili temporanee
R4-R11	Variabili salvate
R12	Variabile temporanea
R13 (SP)	<i>Stack Pointer</i>
R14 (LR)	<i>Link Register</i>
R15 (PC)	<i>Program Counter</i>

ESEMPIO DI CODICE 6.6 OPERANDI IMMEDIATI

Codice di alto livello	Codice <i>assembly</i> ARM
<code>a = a + 4;</code> <code>b = a - 12;</code>	<code>; R7 = a, R8 = b</code> <code>ADD R7, R7, #4 ; a = a + 4</code> <code>SUB R8, R7, #0xC ; b = a - 12</code>

ESEMPIO DI CODICE 6.7 INIZIALIZZAZIONE DI VALORI USANDO IMMEDIATI

Codice di alto livello	Codice <i>assembly</i> ARM
<code>i = 0;</code> <code>x = 4080;</code>	<code>; R4 = i, R5 = x</code> <code>MOV R4, #0 ; i = 0</code> <code>MOV R5, #0xFF0 ; x = 4080</code>

L’istruzione `MOV` è molto comoda per inizializzare i valori dei registri: l’**Esem-
pio di Codice 6.7** inizializza le variabili `i` e `x` rispettivamente a 0 e a 4080. `MOV`
può anche usare registri come operandi sorgente: per esempio, `MOV R1, R7`
copia il contenuto di `R7` in `R1`.

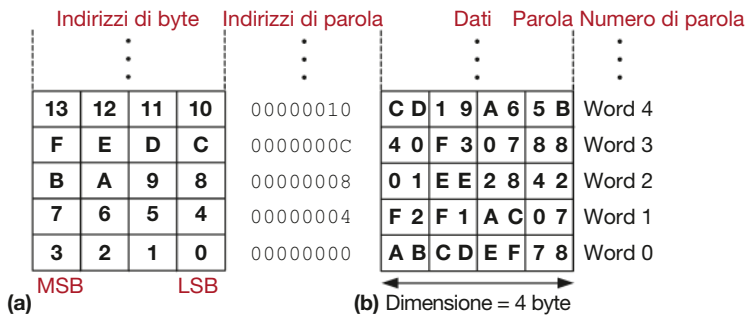
Memoria

Se i registri fossero l’unico spazio in cui memorizzare operandi, si potrebbero
scrivere solo programmi con non più di 16 variabili. Ma i dati possono anche
essere memorizzati nella memoria di lavoro, che al contrario dei registri, po-
chi e veloci, è grande e più lenta. Per questo motivo, le variabili usate spesso
vengono tenute nei registri. In ARM le istruzioni operano solo sui registri,
quindi i dati presenti in memoria devono essere copiati nei registri prima di
essere elaborati. Combinando registri e memoria, un programma può dunque
utilizzare abbastanza rapidamente grosse quantità di dati. Come discusso nel
paragrafo 5.5, la memoria è organizzata come un vettore di parole: l’architet-
tura ARM usa 32 bit per gli indirizzi di memoria e 32 bit per le parole di dato.

ARM fa uso di memoria indirizzabile a byte (*byte addressable*), cioè ogni
byte di memoria ha un indirizzo univoco, come mostrato nella **Figura 6.1(a)**.
Una parola di 32 bit (*word*) è costituita da quattro byte da 8 bit ciascuno,
quindi ogni indirizzo di parola è un multiplo di 4. Il byte più significativo
(MSB, *Most Significant Byte*) è a sinistra, il meno significativo (LSB, *Least
Significant Byte*) è a destra. Nella **Figura 6.1(b)** sia gli indirizzi a 32 bit sia i
dati a 32 bit sono espressi in esadecimale. Per esempio, la parola di dato `0xF-
2F1AC07` è memorizzata all’indirizzo 4. Per convenzione, la memoria viene
disegnata con gli indirizzi bassi in fondo e quelli alti in cima.

ARM fornisce l’istruzione `LDR` (*LoaD Register*) per leggere una parola di
dato dalla memoria in un registro. L’**Esempio di Codice 6.8** carica la parola
di memoria 2 nella variabile `a` (`R7`). In C, il numero tra parentesi è l’**indice** di

Figura 6.1
La memoria di ARM indirizzabile
a byte, che mostra: (a) gli indirizzi
di byte e (b) i dati.



ESEMPIO DI CODICE 6.8 LETTURA DA MEMORIA**Codice di alto livello**

```
a = mem[2];
```

Codice assembly ARM

```
; R7 = a
MOV R5, #0      ; indirizzo base = 0
LDR R7, [R5, #8] ; R7 <= dato memorizzato nella cella
                  ; di indirizzo (R5+8)
```

parola, discusso nel paragrafo 6.3.6. L'istruzione `LDR` specifica l'indirizzo di memoria usando un **registro base** (R5) e uno **spiazzamento** od *offset* (8). Si ricordi che ogni parola di dato è di 4 byte, quindi la parola di indice 1 è all'indirizzo 4, quella di indice 2 all'indirizzo 8, e così via: l'indirizzo di parola è 4 volte l'indice di parola. L'indirizzo di memoria viene formato sommando il contenuto del registro base (R5) e lo spiazzamento. ARM offre vari modi per accedere alla memoria, come discusso nel paragrafo 6.3.6.

Dopo che l'istruzione di caricamento registro (`LDR`) dell'Esempio di Codice 6.8 è stata eseguita, R7 contiene il valore 0x01EE2842, cioè il dato contenuto all'indirizzo 8 della memoria.

ARM usa l'istruzione `STR` (*STore Register*) per scrivere una parola di dato da un registro in memoria. L'Esempio di Codice 6.9 scrive il valore 42 dal registro R9 alla parola di memoria di indice 5.

Le memorie indirizzabili a byte sono organizzate in modalità *big-endian* oppure *little-endian*, come mostrato nella **Figura 6.2**. In entrambe, il byte più significativo (MSB) è a sinistra, il meno significativo (LSB) a destra. Gli indirizzi di parola sono gli stessi in entrambe le modalità e fanno riferimento agli stessi quattro byte; solo gli indirizzi dei singoli byte all'interno della parola sono diversi: in macchine *big-endian* i byte sono numerati partendo con 0 dal lato *big* (quindi il byte più significativo), mentre in macchine *little-endian* i byte sono numerati partendo con 0 dal lato *little* (quindi il byte meno significativo).

Il processore IBM Power PC (una volta presente nei calcolatori Macintosh) usa indirizzamento *big-endian*. L'architettura Intel x86 dei PC usa indirizzamento *little-endian*. ARM preferisce *little-endian* ma in alcune versioni fornisce supporto per indirizzamento *big-endian*, cioè consente letture

Una lettura dall'indirizzo base (cioè con indice 0) è un caso particolare che non richiede offset nel codice assembly. Quindi, per esempio, una lettura da memoria dall'indirizzo base contenuto in R5 si scrive `LDR R3, [R5]`.

ARMv4 richiede indirizzi *word-aligned* (allineati a parola) per `LDR` e `STR`, ovvero indirizzi divisibili per quattro. Dalla versione ARMv6 in poi questa limitazione può essere rimossa settando un bit nel registro di controllo di sistema di ARM, ma le prestazioni negli accessi a memoria non allineati sono in generale peggiori. Alcune architetture, come la famiglia x86, consentono letture e scritture di dati non *word-aligned*; altre, come il MIPS, richiedono rigorosamente l'allineamento per semplicità. Naturalmente, gli indirizzi di byte per le letture e le scritture di byte, `LDRB` e `STRB` (discusse nel paragrafo 6.3.6) non richiedono allineamento a parola.

ESEMPIO DI CODICE 6.9 SCRITTURA IN MEMORIA**Codice di alto livello**

```
mem[5] = 42;
```

Codice assembly ARM

```
MOV R1, #0      ; indirizzo base = 0
MOV R9, #42
STR R9, [R1, #0x14] ; dato memorizzato nella cella
                    ; di indirizzo (R1+20) = 42
```

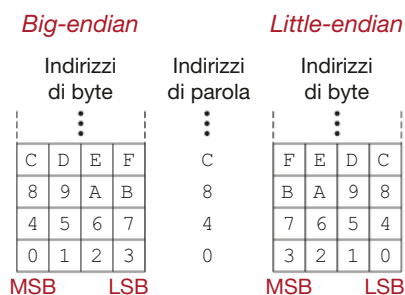
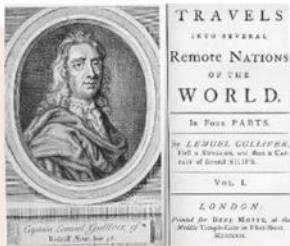


Figura 6.2
Indirizzamenti a memoria big-endian e little-endian.



I termini *big-endian* e *little-endian* derivano da “I Viaggi di Gulliver” di Jonathan Swift, pubblicato per la prima volta nel 1726 sotto lo pseudonimo di Isaac Bickerstaff. Nel racconto, il re dei Lillipuziani obbligava i suoi sudditi (i *Little-Endian*) a rompere le uova dalla parte più stretta. I *Big-Endian* erano ribelli che rompevano le uova dalla parte più larga. Questi termini sono stati applicati ai calcolatori per la prima volta da Danny Cohen nell’articolo “On Holy Wars and a Plea for Peace” (Guerre sante e un appello alla pace) pubblicato su *April Fools Day* nel 1980 (USC/ISI IEN 137). (Fotografia gentilmente concessa da The Brothers Collection, Leeds University Library.)

e scritte in entrambe le modalità. La scelta di quale usare è arbitraria ma naturalmente comporta problemi se si vogliono condividere dati fra calcolatori big-endian e calcolatori little-endian. Negli esempi di questo testo si usa sempre little-endian quando l’ordinamento dei byte in memoria è importante.

6.3 ■ PROGRAMMARE

Linguaggi di programmazione come C e Java sono chiamati di **alto livello** perché si collocano a un livello di astrazione dalla struttura del calcolatore molto maggiore rispetto al linguaggio assembly. Molti linguaggi di alto livello usano costrutti di programmazione come operazioni aritmetiche e logiche, esecuzioni condizionate, strutture di selezione if/else, cicli for e while, indicizzazione degli *array*, chiamate a funzioni. Si faccia riferimento all’Appendice C per esempi di questi costrutti in linguaggio C. In questo paragrafo si esamina come tradurre questi costrutti di alto livello in codice assembly di ARM.

6.3.1 Istruzioni di elaborazione dati

L’architettura ARM definisce una serie di **istruzioni di elaborazione dati** (definite spesso, in altre architetture, istruzioni aritmetiche e logiche). Si introducono qui brevemente queste istruzioni in quanto necessarie per realizzare costrutti di più alto livello. L’Appendice B contiene l’elenco delle istruzioni ARM.

Istruzioni logiche

Le **istruzioni logiche** di ARM includono AND, ORR (OR), EOR (XOR) e BIC (*Bit Clear*). Tutte operano bit a bit e scrivono il risultato in un registro destinazione. La prima sorgente è sempre un registro e la seconda può essere un immediato o un altro registro. Un’altra istruzione logica, MVN (*MoVe and Not*) esegue la negazione bit a bit della seconda sorgente (la prima sorgente non esiste) e scrive il risultato nel registro destinazione. La **Figura 6.3** riporta alcuni esempi di queste istruzioni sui due valori sorgente 0x46A1F1B7 e 0xFFFF0000, mostrando i valori memorizzati nel registro destinazione dopo l’esecuzione di ciascuna istruzione.

L’istruzione BIC è molto utile per “mascherare” bit (cioè per forzare a 0 i bit che non interessano). BIC R6, R1, R2 calcola R1 AND NOT R2, cioè azzeri i bit che sono a 1 in R2. In questo caso, i due byte più significativi di R1 sono azzerati o “mascherati”, e i due byte meno significativi non mascherati, ovvero 0xF1B7, sono scritti in R6. Si può naturalmente mascherare qualsiasi sottoinsieme dei bit di un registro.

L’istruzione ORR è molto utile per combinare i campi di bit di due registri: per esempio, 0x347A0000 ORR 0x000072FC = 0x347A72FC.

Figura 6.3
Istruzioni logiche.

		Registri sorgente			
R1	0100 0110	1010 0001	1111 0001	1011 0111	
	1111 1111	1111 1111	0000 0000	0000 0000	
Codice assembly		Risultati			
AND R3, R1, R2	R3	0100 0110	1010 0001	0000 0000	0000 0000
ORR R4, R1, R2	R4	1111 1111	1111 1111	1111 0001	1011 0111
EOR R5, R1, R2	R5	1011 1001	0101 1110	1111 0001	1011 0111
BIC R6, R1, R2	R6	0000 0000	0000 0000	1111 0001	1011 0111
MVN R7, R2	R7	0000 0000	0000 0000	1111 1111	1111 1111

Istruzioni di traslazione (*shift*)

Le **istruzioni di traslazione** (o di *shift*) traslano a sinistra o a destra il valore in un registro eliminando i bit a una delle due estremità. L'istruzione di rotazione fa invece ruotare il valore di un registro fino a un massimo di 31 posizioni. Si considerano genericamente istruzioni di traslazione anche quelle di rotazione. Le istruzioni di traslazione in ARM sono `LSL` (*Logical Shift Left*), `LSR` (*Logical Shift Right*), `ASR` (*Arithmetic Shift Right*) e `ROR` (*ROtate Right*). Non esiste l'istruzione `ROL` perché una rotazione a sinistra può essere realizzata con una rotazione a destra di un numero di passi complementare.

Come discusso nel paragrafo 5.2.5, le traslazioni a sinistra inseriscono sempre di zeri nei bit meno significativi. Invece le traslazioni a destra possono essere di tipo logico (zeri inseriti nei bit più significativi) o di tipo aritmetico (i bit inseriti corrispondono al valore del bit di segno, ovvero il più significativo prima della traslazione). L'ampiezza della traslazione (cioè di quante posizioni traslare) può essere un immediato o un registro.

La **Figura 6.4** mostra il codice assembly e i valori risultanti nei registri per `LSL`, `LSR`, `ASR` e `ROR` quando si trasla di un immediato. R5 viene traslato dell'ampiezza fornita come immediato, e il risultato viene scritto nel registro destinazione. Traslare a sinistra di N posizioni un valore equivale a moltiplicarlo per 2^N . Analogamente, traslare aritmeticamente a destra di N posizioni un valore equivale a dividerlo per 2^N . Le traslazioni logiche sono anche usate per estrarre o mettere insieme campi di bit.

La **Figura 6.5** mostra il codice assembly e i valori risultanti nei registri per le istruzioni di traslazione quando l'ampiezza di traslazione è contenuta in un registro. Queste istruzioni usano il modo di indirizzamento **a registro con traslazione a registro**, nel quale un registro (R8) viene traslato di un'ampiezza (20) contenuta in un altro registro (R9).

Istruzioni di moltiplicazione*

La moltiplicazione è in qualche modo diversa dalle altre operazioni aritmetiche: se si moltiplicano due numeri a 32 bit si ottiene un risultato a 64 bit. L'architettura ARM fornisce due **istruzioni di moltiplicazione**, che producono un risultato a 32 o 64 bit. L'istruzione `MUL` (*MULTiPLY*) moltiplica due valori a 32 bit e produce un risultato anche a 32 bit. `MUL R1, R2, R3` moltiplica i valori contenuti in R2 e R3 e memorizza i 32 bit meno significativi del risultato in R1; i 32 bit più significativi vengono trascurati. L'istruzione è utile per molti-

		Registro sorgente			
R5		1111 1111	0001 1100	0001 0000	1110 0111
Codice assembly		Risultati			
<code>LSL R0, R5, #7</code>	R0	1000 1110	0000 1000	0111 0011	1000 0000
<code>LSR R1, R5, #17</code>	R1	0000 0000	0000 0000	0111 1111	1000 1110
<code>ASR R2, R5, #3</code>	R2	1111 1111	1110 0011	1000 0010	0001 1100
<code>ROR R3, R5, #21</code>	R3	1110 0000	1000 0111	0011 1111	1111 1000

Figura 6.4
Istruzioni di traslazione con ampiezza di traslazione immediata.

		Registri sorgente			
R8		0000 1000	0001 1100	0001 0110	1110 0111
R6		0000 0000	0000 0000	0000 0000	0001 0100
Codice assembly		Risultati			
<code>LSL R4, R8, R6</code>	R4	0110 1110	0111 0000	0000 0000	0000 0000
<code>ROR R5, R8, R6</code>	R5	1100 0001	0110 1110	0111 0000	1000 0001

Figura 6.5
Istruzioni di traslazione con ampiezza di traslazione a registro.



Figura 6.6
CPSR: Registro di Stato Corrente del Programma.

I cinque bit meno significativi di CPSR sono **bit di modo** e sono descritti nel paragrafo 6.6.3.

Altre utili istruzioni per confrontare due valori sono CMN, TST e TEQ. Ciascuna di loro esegue un'operazione, aggiorna le flag di condizione e scarta il risultato. CMN (CoMpare Negative) confronta il primo operando con il negato del secondo sommandoli. Come discusso nel paragrafo 6.4, le istruzioni ARM codificano valori immediati solo positivi: quindi si deve usare CMN R2, #20 invece di CMP R2, #-20. TST (TeSt) fa l'AND logico dei due operandi. È utile per verificare se una parte del contenuto di un registro è nulla o non nulla. Per esempio, TST R2, #0xFF porta a 1 la flag Z se il byte meno significativo di R2 è nullo. TEQ (Test if Equal) verifica l'uguaglianza facendo lo XOR dei due operandi. Quindi la flag Z viene portata a 1 se i due operandi sono uguali, mentre viene portata a 1 la flag N se i segni dei due operandi sono diversi.

plicare numeri piccoli, il cui prodotto sta in 32 bit. Le istruzioni UMULL (*Unsigned MuLtiply Long*) e SMULL (*Signed MuLtiply Long*) moltiplicano due valori a 32 bit e producono un risultato a 64 bit. Per esempio, UMULL R1, R2, R3, R4 moltiplica i valori contenuti in R3 e R4 e memorizza i 32 bit meno significativi del risultato in R1 e i 32 bit più significativi in R2.

Tutte queste istruzioni hanno una variante di moltiplicazione con accumulazione, rispettivamente MLA (*MuLtiply Accumulate*), UMLAL (*Unsigned MuLtiply Accumulate Large*) e SMLAL (*Signed MuLtiply Accumulate Large*), che somma il prodotto a un totalizzatore a 32 o 64 bit. Queste istruzioni aumentano le prestazioni di calcolo matematico in applicazioni come prodotti di matrici o elaborazioni di segnali che richiedono ripetuti prodotti e somme.

6.3.2 Flag di condizione

L'esecuzione sempre della stessa sequenza di istruzioni non ha alcuna utilità. Le istruzioni ARM possono opzionalmente impostare a 0 o a 1 delle **flag di condizione** (letteralmente “bandiere”, cioè segnalazioni che si possono alzare o abbassare, ma in informatica si usa il termine inglese) in base al fatto che il risultato sia nullo, negativo ecc. Le istruzioni seguenti possono essere eseguite sotto condizione, a seconda dello stato di tali flag. In ARM, le flag di condizione, dette anche *flag di stato*, sono N (*Negative*), Z (*Zero*), C (*Carry*, ovvero riporto) e V (*oVerflow*, ovvero traboccamento), come riportato nella **Tabella 6.2**. Le flag vengono impostate dall'ALU (vedi par. 5.2.4) e sono memorizzate nei 4 bit più significativi del registro CPSR (*Current Program Status Register*, Registro di Stato Corrente del Programma) come mostrato nella **Figura 6.6**.

Il modo tipico di impostare le flag di condizione è tramite l'istruzione di confronto CMP (*CoMpare*), che sottrae il secondo operando sorgente dal primo e imposta le flag sulla base del risultato. Per esempio, se i due numeri sono uguali, il risultato è zero e viene messa a 1 la flag Z. Se il primo numero è un valore *unsigned* maggiore o uguale al secondo, la sottrazione produce un riporto in uscita e viene messa a 1 la flag C.

Le istruzioni successive possono essere eseguite sotto condizione a seconda dello stato delle flag. Lo mnemonico dell'istruzione è seguito da uno **mnemonico di condizione** che indica quando eseguirla. La **Tabella 6.3** riporta il campo di condizione a 4 bit, lo mnemonico di condizione, il nome e lo stato delle flag di condizione che dà luogo all'esecuzione dell'istruzione (CondEse). Per esempio, si consideri un programma che contiene le due istruzioni CMP R4, R5 e ADDEQ R1, R2, R3. L'istruzione di confronto mette a 1 la flag Z se R4 e R5 sono uguali, e l'istruzione ADDEQ viene eseguita solo se la flag Z è a 1. Il campo di condizione viene usato nella traduzione in linguaggio macchina, come discusso nel paragrafo 6.4.

Altre istruzioni di elaborazione dati impostano le flag di condizione quando lo mnemonico dell'istruzione è seguito da "S". Per esempio, SUBS R2, R3, R7 sottrae R7 da R3, mette il risultato in R2 e imposta le flag di condizione. La Tabella B.5 nell'Appendice B riassume quali flag sono modificate da ciascuna istruzione. Tutte le istruzioni di elaborazione dati modificano le flag Z e N in base al fatto che il risultato sia nullo oppure che abbia il bit più significativo a 1. ADDS e SUBS modificano anche V e C, e le traslazioni modificano C.

Tabella 6.2 Flag di condizione.

Flag	Nome	Descrizione
N	Negative	Il risultato dell'istruzione è negativo, cioè il bit 31 vale 1
Z	Zero	Il risultato dell'istruzione è zero
C	Carry	L'istruzione ha generato riporto (<i>carry</i>) di uscita
V	oVerflow	L'istruzione ha causato traboccamento (<i>overflow</i>)

Tabella 6.3 Mnemonici di condizione.

cond	Mnemonico	Nome	CondEse
0000	EQ	Uguale (<i>Equal</i>)	Z
0001	NE	Diverso (<i>Not Equal</i>)	$\bar{Z}$
0010	CS/HS	Attiva riporto/maggiore o uguale senza segno (<i>Carry Set/unsigned Higher or Same</i>)	C
0011	CC/LO	Disattiva riporto/minore senza segno (<i>Carry Clear/unsigned Lower</i>)	$\bar{C}$
0100	MI	Meno/negativo (<i>Minus/negative</i>)	N
0101	PL	Più/positivo o nullo (<i>Plus/positive or zero</i>)	$\bar{N}$
0110	VS	Traboccamento/attiva traboccamento (<i>overflow/oVerflow Set</i>)	V
0111	VC	No traboccamento/disattiva traboccamento (<i>overflow/oVerflow Clear</i>)	$\bar{V}$
1000	HI	Maggiore senza segno (<i>unsigned Higher</i>)	$\bar{Z}C$
1001	LS	Minore o uguale senza segno (<i>unsigned Lower or Same</i>)	$Z \text{ OR } \bar{C}$
1010	GE	Maggiore o uguale con segno (<i>signed Greater than or Equal</i>)	$\overline{N \oplus V}$
1011	LT	Minore con segno (<i>signed Less Than</i>)	$N \oplus V$
1100	GT	Maggiore con segno (<i>signed Greater Than</i>)	$\bar{Z} (\overline{N \oplus V})$
1101	LE	Minore o uguale con segno (<i>signed Less than or Equal</i>)	$Z \text{ OR } (N \oplus V)$
1110	AL (o niente)	Sempre/incondizionato (<i>ALways/unconditional</i>)	Ignorato

L'**Esempio di Codice 6.10** mostra istruzioni che vengono eseguite sotto condizione. La prima istruzione, `CMP R2, R3`, viene eseguita comunque e imposta le flag di condizione. Le restanti istruzioni vengono eseguite sotto condizione, in base ai valori delle flag. Se, per esempio, R2 e R3 contengono i valori 0x80000000 e 0x00000001, l'istruzione di confronto calcola $R2 - R3 = 0x80000000 - 0x00000001 = 0x80000000 + 0xFFFFFFFF = 0x7FFFFFFF$ con riporto di uscita (quindi $C = 1$). Inoltre i due valori sorgenti hanno segno opposto, e il segno del risultato è diverso da quello del primo valore, quindi si è generato traboccamento ($V = 1$). Le altre due flag (N e Z) hanno valore 0. Quindi l'istruzione `ANDHS` viene seguita perché $C = 1$, e anche l'istruzione `EORLT` viene seguita perché $N = 0$ e $V = 1$ (Tabella 6.3). Come intuibile, `ANDHS` e `EORLT` vengono eseguite perché $R2 \geq R3$ (senza segno, *unsigned*) e $R2 < R3$ (con segno, *signed*). Invece `ADDEQ` e `ORRMI` non vengono eseguite perché il risultato di $R2 - R3$ non è zero (cioè $R2 \neq R3$) né negativo.

6.3.3 Salti

Il vantaggio di un calcolatore programmabile rispetto a una macchina calcolatrice è la capacità di prendere decisioni: il calcolatore esegue attività diverse a

Gli mnemonici di condizione differiscono per confronti tra numeri con e senza segno. Per esempio, ARM offre due tipi di confronto maggiore o uguale: HS/CS è usato per numeri senza segno e GE per numeri con segno. Per numeri senza segno, $A - B$ genera un riporto di uscita (C) se $A \geq B$. Per numeri con segno, $A - B$ porta N e V entrambi a 0 o a 1 se $A \geq B$. La Figura 6.7 sottolinea le differenze tra i confronti HS e GE con due esempi di numeri a 4 bit per facilitare la comprensione.

	Senza segno	Con segno
A = 1001₂	A = 9	A = -7
B = 0010₂	B = 2	B = 2
A - B:	1001	NZCV = 0011 ₂
	+ 1110	HS: VERO
(a)	10111	GE: FALSO
	Senza segno	Con segno
A = 0101₂	A = 5	A = 5
B = 1101₂	B = 13	B = -3
A - B:	0101	NZCV = 1001 ₂
	+ 0011	HS: FALSO
(b)	1000	GE: VERO

Figura 6.7
Confronto con e senza segno: GE rispetto a HS.

ESEMPIO DI CODICE 6.10 ESECUZIONE CONDIZIONATA

Codice assembly ARM

```

CMP    R2, R3
ADDEQ  R4, R5, #78
ANDHS  R7, R8, R9
ORRMI  R10, R11, R12
EORLT  R12, R7, R10

```


seconda dei valori che riceve in ingresso. Per esempio, i costrutti di selezione *if/else* e *switch/case*, come pure i cicli *while* e *for*, eseguono o meno parti di codice in base a qualche verifica.

Un modo di prendere decisioni è quello di usare l'esecuzione condizionata per saltare alcune istruzioni. Questo va bene per costrutti *if* semplici, dove piccoli numeri di istruzioni devono essere saltate, ma fa perdere parecchio tempo in costrutti con molte istruzioni nel corpo, e non è in grado di gestire i cicli. Quindi ARM, come la maggior parte delle altre architetture, usa le **istruzioni di salto** (*branch instructions*, o “istruzioni di ramificazione”, perché dividono una strada in due come un ramo che si diparte da un altro) per saltare parti di codice o per ripeterle.

Un programma viene normalmente eseguito in sequenza, con il registro PC (*Program Counter*) che si incrementa di 4 dopo ogni istruzione per puntare alla successiva (si ricordi che in ARM le istruzioni sono lunghe 4 byte e che l'architettura usa indirizzamento a byte). Le istruzioni di salto modificano il PC. ARM presenta due tipi di istruzioni di salto: un salto semplice (B) e un “salto con collegamento” (BL). BL viene usato per le chiamate a sottoprogramma, come discusso nel paragrafo 6.3.7. Come altre istruzioni ARM, i salti possono essere incondizionati o condizionati.

L'**Esempio di Codice 6.11** mostra un salto incondizionato mediante l'istruzione B. Quando si raggiunge l'istruzione B DEST, il salto viene effettuato (*taken*), ovvero la prossima istruzione che viene eseguita è l'istruzione SUB che segue la *label* (in italiano “etichetta”, ma si usa comunemente il termine inglese) denominata DEST.

Il codice assembly usa le label per indicare le posizioni di alcune istruzioni all'interno del programma: quando tale codice viene tradotto in linguaggio macchina, le *label* vengono tradotte negli indirizzi di memoria delle istruzioni (par. 6.4.3). Le label di ARM non possono essere parole riservate, come gli mnemonici delle istruzioni. Molti programmatori indentano le istruzioni nel codice ma non le label per tenerle in evidenza. In ARM questa diventa una regola: le label non devono essere indentate, e le istruzioni devono essere precedute da spazi vuoti. Alcuni compilatori, come GCC, impongono inoltre che le label siano terminate dal carattere due punti.

Le istruzioni di salto possono essere condizionate in base agli mnemonici di condizione elencati nella Tabella 6.3. L'**Esempio di Codice 6.12** illustra l'uso di BEQ, l'istruzione di salto condizionata all'uguaglianza ($Z = 1$).

ESEMPIO DI CODICE 6.11

SALTO INCONDIZIONATO

Codice *assembly* ARM

```
ADD R1, R2, #17      ; R1 = R2 + 17
B   DEST             ; salta a DEST
ORR R1, R1, R3        ; non eseguita
AND R3, R1, #0xFF     ; non eseguita
DEST SUB R1, R1, #78   ; R1 = R1 - 78
```

ESEMPIO DI CODICE 6.12

SALTO CONDIZIONATO

Codice *assembly* ARM

```
MOV R0, #4           ; R0 = 4
ADD R1, R0, R0        ; R1 = R0 + R0 = 8
CMP R0, R1            ; sistema flag per R0-R1 = -4. NZCV = 1000
BEQ DEST              ; non salta (Z != 1)
ORR R1, R1, #1        ; R1 = R1 OR 1 = 9
DEST ADD R1, R1, #78   ; R1 = R1 + 78 = 87
```


Quando si arriva all'istruzione `BEQ DEST`, la flag `Z` è 0 (infatti `R0 ≠ R1`) quindi il salto non viene effettuato (*not taken*) e l'istruzione successiva a essere eseguita è `ORR`.

6.3.4 Costrutti di selezione

I costrutti *if*, *if/else* e *switch/case* sono costrutti di selezione comunemente usati nei linguaggi di alto livello. Tutti eseguono sotto condizione un blocco di codice costituito da una o più istruzioni. In questo paragrafo si mostra come tradurre questi costrutti in linguaggio assembly di ARM.

Costrutto *if*

Il costrutto *if* esegue un blocco di codice, il cosiddetto **blocco *if***, solo se una certa condizione è verificata. L'**Esempio di Codice 6.13** mostra come tradurre il costrutto *if* in linguaggio assembly di ARM.

Il codice assembly del costrutto *if* valuta la condizione opposta rispetto a quella presente nel codice di alto livello: nell'Esempio di Codice 6.13 la condizione di alto livello è `mele == arance`, mentre quella assembly è `mele != arance`, che utilizza l'istruzione `BNE` per saltare il blocco *if* se tale condizione è verificata, quindi se **non** è verificata la condizione di alto livello. Se invece la condizione di alto livello è verificata (cioè `mele == arance`) il salto non viene effettuato e si esegue il blocco *if*.

Dal momento che ogni istruzione ARM può essere condizionata, il codice assembly ARM dell'Esempio di Codice 6.13 potrebbe essere sostituito dalla seguente forma più compatta:

```
CMP    R0, R1        ; mele == arance ?
ADDEQ  R2, R3, #1     ; f = i + 1 in caso di uguaglianza (cioè Z = 1)
SUB     R2, R2, R3     ; f = f - i
```

Questa versione con istruzioni condizionate è più corta e anche più veloce perché richiede di eseguire meno istruzioni. Inoltre, come si vedrà nel paragrafo 7.5.3, i salti a volte introducono ritardi mentre l'esecuzione condizionata di istruzioni è sempre veloce. Questo esempio mostra proprio la potenza dell'esecuzione condizionata dell'architettura ARM.

In generale, quando un blocco di codice è costituito da una sola istruzione, conviene usare l'esecuzione condizionata invece dei salti, che diventano utili quando il blocco di codice è più lungo perché evitano le fasi di fetch di istruzioni che poi non saranno eseguite.

Costrutto *if/else*

Il costrutto *if/else* esegue uno di due blocchi di istruzioni in base a una certa condizione. Se la condizione è verificata, si esegue il blocco *if*, altrimenti si esegue il blocco *else*. L'**Esempio di Codice 6.14** mostra un esempio di costrutto *if/else*.

Come per il costrutto *if*, il codice assembly del costrutto *if/else* valuta la condizione opposta rispetto a quella presente nel codice di alto livello: nell'Esempio di Codice 6.14 la condizione di alto livello è `mele == arance`, mentre

Si ricorda che nel codice di alto livello il confronto di disuguaglianza si indica con `!=` e quello di uguaglianza con `==`.

ESEMPIO DI CODICE 6.13 COSTRUTTO *IF*

Codice di alto livello	Codice assembly ARM
<code>if (mele == arance)</code>	<code>; R0 = mele, R1 = arance, R2 = f, R3 = i</code>
<code> f = i + 1;</code>	<code> CMP R0, R1 ; mele == arance ?</code>
<code> f = f - i;</code>	<code> BNE NOIF ; se diverse, salta blocco if</code>
	<code> ADD R2, R3, #1 ; blocco if: f = i + 1</code>
	<code>NOIF SUB R2, R2, R3 ; f = f - i</code>

ESEMPIO DI CODICE 6.14 COSTRUTTO IF/ELSE	
Codice di alto livello	Codice assembly ARM
<pre>if (mele == arance) f = i + 1; else f = f - i;</pre>	<pre>; R0 = mele, R1 = arance, R2 = f, R3 = i CMP R0, R1 ; mele == arance ? BNE ELSE ; se diverse, va a blocco ELSE ADD R2, R3, #1 ; blocco IF: f = i + 1 B ENDIF ; salta blocco ELSE ELSE SUB R2, R2, R3 ; blocco ELSE: f = f - i ENDIF</pre>

quella *assembly* è mele != arance. Se la condizione opposta è VERA, l’istruzione BNE salta il blocco *if* e va a eseguire il blocco *else*. Altrimenti si esegue il blocco *if* che termina con un salto incondizionato (B) per saltare il blocco *else*.

Di nuovo, ogni istruzione può essere condizionata, e dato che le istruzioni del blocco *if* non modificano le flag il codice assembly ARM dell’Esempio di Codice 6.14 potrebbe essere sostituito dalla seguente forma molto più compatta:

```
CMP    R0, R1          ; mele == arance ?
ADDEQ  R2, R3, #1       ; f = i + 1 in caso di uguaglianza (cioè Z = 1)
SUBNE  R2, R2, R3       ; f = f - i in caso di disuguaglianza (cioè Z = 0)
```

Costrutto switch/case*

Il costrutto *switch/case* esegue uno di vari blocchi di istruzioni in base a diverse condizioni. Se nessuna condizione è soddisfatta, si esegue il blocco *default*. Tale costrutto è quindi equivalente a una serie di costrutti *if/else* annidati. L’**Esempio di Codice 6.15** mostra due frammenti di codice di alto livello che fanno la stessa cosa: entrambi calcolano se erogare una banconota da 20, 50 o 100 euro da un bancomat a seconda del pulsante premuto. La traduzione in *assembly* è la stessa per entrambi i frammenti di alto livello.

6.3.5 Cicli

I cicli eseguono ripetutamente un blocco di codice in base a una certa condizione. I cicli *while* e *for* sono costrutti ciclici molto comuni usati nei linguaggi di alto livello. Questa sezione mostra come tradurli in linguaggio assembly di ARM sfruttando i salti condizionati.

ESEMPIO DI CODICE 6.15 COSTRUTTO SWITCH/CASE	
Codice di alto livello	Codice assembly ARM
<pre>switch (pulsante) { case 1: bancomat = 20; break; case 2: bancomat = 50; break; case 3: bancomat = 100; break; default: bancomat = 0; } // codice equivalente con l’istruzione if/else if (pulsante == 1) bancomat = 20; else if (pulsante == 2) bancomat = 50; else if (pulsante == 3) bancomat = 100; else bancomat = 0;</pre>	<pre>; R0 = pulsante, R1 = bancomat CMP R0, #1 ; è pulsante 1 ? MOVEQ R1, #20 ; bancomat = 20 se pulsante==1 BEQ FINE ; break CMP R0, #2 ; è pulsante 2 ? MOVEQ R1, #50 ; bancomat = 50 se pulsante==2 BEQ FINE ; break CMP R0, #3 ; è pulsante 3 ? MOVEQ R1, #100 ; bancomat = 100 se pulsante==3 BEQ FINE ; break MOV R1, #0 ; default bancomat = 0 FINE</pre>

Ciclo *while*

Il ciclo *while* esegue ripetutamente un blocco di codice finché una condizione non è più verificata. Il ciclo *while* dell'**Esempio di Codice 6.16** calcola il valore di x tale che $2^x = 128$. Viene eseguito 7 volte, finché *potenza* diventa uguale a 128.

Come per il costrutto *if/else*, il codice assembly del ciclo *while* valuta la condizione opposta rispetto a quella presente nel codice di alto livello: se la condizione opposta è vera (in questo caso, $R0 == 128$) si esce dal ciclo, altrimenti ($R0 \neq 128$) il salto condizionato non viene effettuato e si esegue il corpo del ciclo. Nell'Esempio di Codice 6.16 per tradurre il ciclo *while* si confronta *potenza* con 128 e si esce se sono uguali, altrimenti si moltiplica *potenza* per due (con la traslazione a sinistra), si incrementa x e si salta di nuovo all'inizio del ciclo.

Il tipo di dato `int` in C si riferisce a una parola di memoria che codifica un intero in complemento a due. ARM usa parole da 32 bit, quindi un `int` può rappresentare un numero compreso nell'intervallo $[-2^{31}, 2^{31} - 1]$.

ESEMPIO DI CODICE 6.16 CICLO WHILE

Codice di alto livello

```
int potenza = 1;
int x = 0;
while (potenza != 128) {
    potenza = potenza * 2;
    x = x + 1;
}
```

Codice assembly ARM

```
; R0 = potenza, R1 = x
MOV R0, #1      ; potenza = 1
MOV R1, #0      ; x = 0
WHILE CMP R0, #128 ; potenza != 128 ?
      BEQ FINE    ; se potenza == 128, esce dal ciclo
      LSL R0, R0, #1 ; potenza = potenza * 2
      ADD R1, R1, #1 ; x = x + 1
      B WHILE     ; ripete il ciclo WHILE
FINE
```

Ciclo *for*

È molto frequente inizializzare una variabile prima di un ciclo *while*, utilizzare il valore di tale variabile nella condizione del ciclo e modificarlo ad ogni iterazione del ciclo stesso. Il ciclo *for* è una scorciatoia molto comoda che mette insieme inizializzazione, verifica e modifica della variabile in un posto solo. Il formato del ciclo *for* è il seguente:

```
for (inizializzazione; condizione; operazione nel ciclo)
    istruzione
```

L'inizializzazione viene eseguita prima dell'inizio del ciclo *for*. La condizione viene valutata all'inizio di ogni iterazione, e se non è verificata si esce dal ciclo. L'operazione viene eseguita alla fine di ogni iterazione del ciclo.

L'**Esempio di Codice 6.17** somma i numeri da 0 a 9. La variabile di ciclo, i , in questo caso è inizializzata a 0 e incrementata di uno al termine di ogni iterazione. Il ciclo viene ripetuto fintantoché i è minore di 10. Si noti che

ESEMPIO DI CODICE 6.17 CICLO FOR

Codice di alto livello

```
int i;
int totale = 0;

for (i = 0; i < 10; i = i + 1) {
    totale = totale + i;
}
```

Codice assembly ARM

```
; R0 = i, R1 = totale
MOV R1, #0      ; totale = 0
MOV R0, #0      ; i = 0 inizializzazione del ciclo
FOR CMP R0, #10 ; i < 10 ? verifica condizione
      BGE FINE    ; se (i >= 10) esce dal ciclo
      ADD R1, R1, R0 ; totale = totale + i corpo del ciclo
      ADD R0, R0, #1 ; i = i + 1 incremento del contatore
      B FOR      ; ripete il ciclo FOR
FINE
```

questo esempio mostra anche i confronti relativi. Il ciclo di alto livello valuta la condizione < per continuare, quindi il codice assembly valuta la condizione opposta >= per uscire dal ciclo.

I cicli sono molto utili soprattutto per accedere a grandi quantità di dati simili presenti in memoria, come discusso nella prossima sezione.

6.3.6 La memoria

Per facilitare memorizzazione e accessi, dati simili possono essere raggruppati in una struttura chiamata array (traducibile in italiano con “vettore” o “matrice” a seconda che si tratti di una struttura monodimensionale o pluridimensionale, ma si preferisce usare il termine inglese). Un array monodimensionale memorizza i suoi dati (detti **elementi**) a indirizzi sequenziali di memoria. Ogni elemento dell’array è identificato da un numero detto **indice** dell’elemento, e il numero di elementi costituisce la **lunghezza** dell’array.

La **Figura 6.8** mostra un array di 200 punteggi memorizzato in memoria. L’**Esempio di Codice 6.18** è un algoritmo di aumento dei punteggi che aggiunge 10 punti a ogni punteggio memorizzato. Si noti che il codice necessario per inizializzare i punteggi nell’array non è mostrato. L’indice dell’array è una variabile (i) e non un valore costante, quindi va moltiplicata per 4 prima di essere sommata all’indirizzo base.

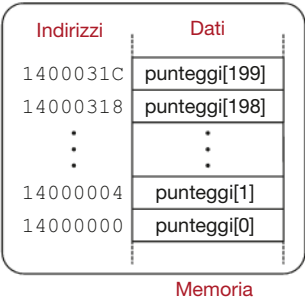


Figura 6.8
Memoria contenente punteggi[200]
a partire dall’indirizzo base
0x14000000.

ESEMPIO DI CODICE 6.18 ACCESSO A UN ARRAY UTILIZZANDO UN CICLO FOR	
Codice di alto livello int i; int punteggi[200]; ... for (i = 0; i < 200; i = i + 1) punteggi[i] = punteggi[i] + 10;	Codice assembly ARM ; R0 = indirizzo base dell’array, R1 = i ; inizializzazioni ... MOV R0, #0x14000000 ; R0 = indirizzo base MOV R1, #0 ; i = 0 FOR CMP R1, #200 ; i < 200? BGE FINE ; se i ≥ 200, esce dal ciclo LSL R2, R1, #2 ; R2 = i * 4 LDR R3, [R0, R2] ; R3 = punteggi[i] ADD R3, R3, #10 ; R3 = punteggi[i] + 10 STR R3, [R0, R2] ; punteggi[i] = punteggi[i] + 10 ADD R1, R1, #1 ; i = i + 1 B FOR ; ripete il ciclo FOR FINE

ARM può “scalare” (cioè moltiplicare) l’indice, sommarlo all’indirizzo base e leggere il dato da memoria in un’unica istruzione macchina: invece della sequenza di istruzioni LSL e LDR dell’Esempio di Codice 6.18, si può usare l’unica istruzione:

```
LDR R3, [R0, R1, LSL #2]
```

nella quale R1 viene scalato (traslato a sinistra di due posizioni), quindi sommato al registro base (R0) per costruire l’indirizzo di memoria $R0 + (R1 \times 4)$.

Oltre alla possibilità di scalare il registro indice, ARM fornisce indirizziamenti a *offset* (spiazzamento) pre-indicizzati e post-indicizzati per consentire di scrivere codice compatto ed efficiente nella gestione degli *array* e nelle chiamate a sottoprogramma. La **Tabella 6.4** fornisce un esempio di ciascuno di questi modi di indirizzamento. In ogni caso, il registro base è R1 e l’*offset* è R2. Si può sottrarre l’*offset* scrivendo -R2. L’*offset* può anche essere un valore immediato nell’intervallo 0-4095 che può essere sommato (per es. #20) o sottratto (per es. #-20).

Tabella 6.4 Modi di indirizzamento di ARM.

Modo	Assembly ARM	Indirizzo	Registro base
Offset	LDR R0, [R1, R2]	R1 + R2	non modificato
Pre-indicizzato	LDR R0, [R1, R2]!	R1 + R2	R1 = R1 + R2
Post-indicizzato	LDR R0, [R1], R2	R1	R1 = R1 + R2

L'indirizzamento a *offset* calcola l'indirizzo come registro base $\pm$ *offset* e il registro base rimane inalterato. L'indirizzamento pre-indicizzato calcola l'indirizzo come registro base $\pm$ *offset* e modifica il registro base con questo nuovo valore appena calcolato. L'indirizzamento post-indicizzato calcola l'indirizzo come registro base solamente, e dopo aver effettuato l'accesso a memoria modifica il registro base con $\pm$ *offset*. Si sono già visti diversi esempi di post-indicizzazione. L'**Esempio di Codice 6.19** mostra il ciclo *for* dell'Esempio di Codice 6.18 riscritto utilizzando post-indicizzazione, che elimina l'istruzione ADD usata per incrementare *i*.

ESEMPIO DI CODICE 6.19 CICLO FOR CON POST-INDICIZZAZIONE**Codice di alto livello**

```
int i;
int punteggi[200];
...
for (i = 0; i < 200; i = i + 1)
    punteggi[i] = punteggi[i] + 10;
```

Codice assembly ARM

```
; R0 = indirizzo base dell'array
; inizializzazioni ...
    MOV R0, #0x14000000 ; R0 = indirizzo base
    ADD R1, R0, #800    ; R1 = indirizzo base + (200*4)

FOR    CMP R0, R1        ; arrivato a fine array?
        BGE FINE        ; se sì, esce dal ciclo
        LDR R2, [R0]     ; R2 = punteggi[i]
        ADD R2, R2, #10  ; R2 = punteggi[i] + 10
        STR R2, [R0], #4 ; punteggi[i] = punteggi[i] + 10
                          ; poi R0 = R0 + 4
        B FOR           ; ripete il ciclo

FINE
```

Byte e caratteri

I numeri compresi nell'intervallo $[-128, 127]$ possono essere memorizzati in un byte invece di occupare un'intera parola. Dal momento che i caratteri necessari per scrivere in lingua inglese sono molto meno di 256, spesso vengono codificati in un solo byte. Il linguaggio C usa il tipo *char* per rappresentare un carattere, cioè un byte.

Agli albori dell'informatica non esisteva uno standard di codifica dei caratteri inglesi in un byte, con ovvi problemi di passaggio di testo tra calcolatori diversi. Nel 1963, l'associazione americana di standardizzazione (*American Standards Association*) ha pubblicato il codice ASCII (*American Standard Code for Information Interchange*) che associa a ogni carattere utilizzabile nel testo una codifica di un byte. La **Tabella 6.5** mostra la codifica ASCII per tutti i caratteri cosiddetti stampabili (cioè escludendo i caratteri di controllo come ritorni a capo, tabulazioni ecc.) utilizzando la notazione esadecimale. Si noti che le lettere minuscole differiscono dalle corrispondenti maiuscole del valore 0x20 (in decimale 32).

ARM fornisce le istruzioni LDRB (*LoaD Register with Byte*), LDRSB (*LoaD Register with Signed Byte*) e STRB (*STore Register Byte*) per accedere a singoli byte in memoria. LDRB riempie di zeri i tre byte più significativi del registro destinazione, mentre LDRSB li riempie con il bit di segno del byte letto. STRB memorizza il byte meno significativo del registro sorgente nella locazione di memoria individuata dall'indirizzo di byte specificato. Le tre istruzioni sono illustrate nella **Figura 6.9**, nella quale si assume che l'indirizzo base contenuto

Altri linguaggi di programmazione, come Java, usano diverse codifiche per i caratteri, in particolare Unicode. Tale codifica usa 16 bit per rappresentare ciascun carattere, quindi supporta le lettere accentate, le diresis (in tedesco *umlaut*) e gli alfabeti asiatici. Per maggiori informazioni vedi www.unicode.org.

LDRH, LDRSH e STRH sono simili, ma accedono a valori a 16 bit (*halfword*: mezze parole).

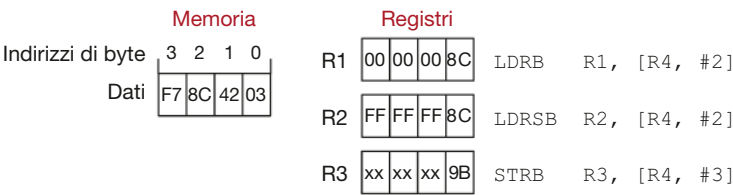
Il codice ASCII si è sviluppato a partire da precedenti codifiche dei caratteri. A partire dal 1838, i telegrafi hanno adottato il codice Morse: una serie di punti (.) e linee (-): per esempio, le lettere A, B, C e D sono codificate -, ..., --., e --.. rispettivamente. Il numero di linee e punti varia da lettera a lettera: per maggiore efficienza, le lettere più comuni usano codici più corti.

Nel 1874 Jean Maurice Emile Baudot ha inventato un codice a 5 bit chiamato appunto codice Baudot. Per esempio, le lettere A, B, C e D sono codificate 00011, 11001, 01110 e 01001 rispettivamente. Però le 32 combinazioni di 5 bit non sono sufficienti a codificare tutti i caratteri usati nella lingua inglese, mentre con 8 bit la cosa è possibile. Quando la comunicazione elettronica è diventata predominante, il codice ASCII a 8 bit è diventato lo standard.

Tabella 6.5 Codifica ASCII.

#	Carat- tere	#	Carat- tere	#	Carat- tere	#	Carat- tere	#	Carat- tere	#	Carat- tere
20	spazio	30	0	40	@	50	P	60	`	70	p
21	!	31	1	41	A	51	Q	61	a	71	q
22	"	32	2	42	B	52	R	62	b	72	r
23	#	33	3	43	C	53	S	63	c	73	s
24	\$	34	4	44	D	54	T	64	T	74	t
25	%	35	5	45	E	55	U	65	e	75	u
26	&	36	6	46	F	56	V	66	f	76	v
27	'	37	7	47	G	57	W	67	g	77	w
28	(	38	8	48	H	58	X	68	h	78	x
29	)	39	9	49	I	59	Y	69	i	79	y
2A	*	3A	:	4A	J	5A	Z	6A	j	7A	z
2B	+	3B	;	4B	K	5B	[	6B	k	7B	{
2C	,	3C	<	4C	L	5C	\	6C	l	7C	
2D	-	3D	=	4D	M	5D	]	6D	m	7D	}
2E	.	3E	>	4E	N	5E	^	6E	n	7E	~
2F	/	3F	?	4F	O	5F	-	6F	o		

Figura 6.9
Istruzioni per leggere e scrivere byte.



in R4 sia 0. **LDRB** carica il byte che si trova in memoria a indirizzo 2 nel byte meno significativo di R1 e riempie di zeri i rimanenti bit del registro, **LDRSB** carica lo stesso byte nel byte meno significativo di R2 e ne estende il segno nei rimanenti 24 bit del registro, **STRB** memorizza il byte meno significativo di R3 (0x9B) nel byte di memoria di indirizzo 3 (quindi sostituisce 0xF7 con 0x9B) e trascura i tre byte più significativi di R3.

ESEMPIO 6.2

Uso di **ldrb e **strb** per accedere ai caratteri di un array.** Il seguente codice di alto livello converte i caratteri di un array di 10 elementi di nome `arraycar` da minuscoli a maiuscoli, sottraendo 32 dalla codifica di ogni elemento. Tradurre il codice in assembly di ARM, ricordando che in questo caso la distanza tra due elementi consecutivi dell'array è di 1 byte e non di 4 byte. Assumere che l'indirizzo base di `arraycar` sia già contenuto in R0.

```
// codice di alto livello
// arraycar[10] già dichiarato e inizializzato
int i;

for (i = 0; i < 10; i = i + 1)
    arraycar[i] = arraycar[i] - 32;
```

Soluzione

```
; codice assembly ARM
; R0 = indirizzo base di arraycar (già inizializzato), R1 = i
MOV R1, #0 ; i = 0
```



```

FOR    CMP  R1, #10      ; i < 10 ?
      BGE  FINE          ; se (i >=10), esce dal ciclo
      LDRB R2, [R0, R1]  ; R2 = mem[R0+R1] = arraycar[i]
      SUB  R2, R2, #32    ; R2 = arraycar[i] - 32
      STRB R2, [R0, R1]  ; arraycar[i] = R2
      ADD  R1, R1, #1     ; i = i + 1
      B    FOR           ; ripete il ciclo FOR

```

FINE

Una sequenza di caratteri viene definita **stringa** in informatica. Le stringhe possono naturalmente avere lunghezza diversa, quindi i linguaggi di programmazione devono fornire un modo per determinare tale lunghezza o per indicare la fine di ciascuna stringa. In C si usa come terminatore di stringa il carattere *null* (0x00). Per esempio, la Figura 6.10 mostra la stringa “Hello!” (0x48 65 6C 6C 6F 21 00) memorizzata in memoria. La stringa è lunga 7 caratteri incluso il terminatore, e occupa le locazioni da 0x1522FFF0 a 0x1522FFF6. Il primo carattere della stringa (H = 0x48) è memorizzato all’indirizzo più basso.

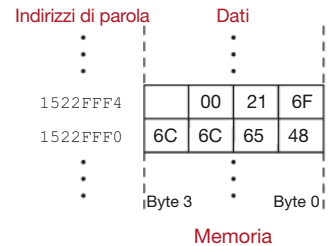


Figura 6.10
La stringa “Hello!” scritta nella memoria.

6.3.7 Chiamate a sottoprogrammi

I linguaggi di alto livello supportano i **sottoprogrammi** (che a seconda delle caratteristiche si chiamano anche procedure, subroutine, o funzioni) per consentire di riutilizzare parti comuni di codice e rendere i programmi più leggibili. In particolare, le **funzioni** ricevono valori di ingresso, denominati **parametri**, e forniscono un solo valore di uscita, denominato **valore di ritorno**. Le funzioni dovrebbero calcolare tale valore senza provocare altri effetti collaterali indesiderati.

Quando una funzione ne chiama un’altra, la prima funzione, detta **il chiamante**, e la seconda funzione, detta **il chiamato**, devono accordarsi su dove mettere i parametri e il valore di ritorno. In ARM, convenzionalmente il chiamante mette fino a quattro parametri nei registri R0-R3 prima di eseguire la chiamata a sottoprogramma, e il chiamato mette il valore di ritorno in R0 prima di terminare. Con questa convenzione entrambe le funzioni sanno dove trovare parametri e valore di ritorno, anche se sono state scritte da programmatori diversi.

Il chiamato non deve interferire con le attività del chiamante. Questo significa che deve sapere dove mettere il valore di ritorno ma anche che non deve modificare nessuno dei registri o delle celle di memoria necessari al chiamante. Il chiamante memorizza l’indirizzo di ritorno nel registro LR (*Link Register*) nel momento in cui salta a eseguire il chiamato con l’istruzione BL (*Branch and Link*). Il chiamato non deve modificare nessun valore dell’architettura né della memoria dai quali dipende il successivo comportamento del chiamante: in particolare deve lasciare inalterati i **registri preservati** (R4-R11 oltre a LR) e lo *stack*, ovvero la porzione di memoria usata dal chiamante per le proprie variabili temporanee.

In questo paragrafo si vede come chiamare una funzione e come ritornare al chiamante, quindi come accedere ai parametri e al valore di ritorno e come usare lo stack per memorizzare le variabili temporanee.

Chiamata a e ritorno da funzione

ARM usa l’istruzione BL (*Branch and Link*) per chiamare una funzione e copia il contenuto di tale registro nel PC (MOV PC, LR) per ritornare al chiamante. L’**Esempio di Codice 6.20** mostra la funzione `main` (il chiamante) che chiama la funzione `niente` (il chiamato). Il chiamato non riceve parametri di ingresso e non restituisce alcun valore di ritorno: si limita a tornare al chiamante. Nell’Esempio di Codice 6.20, gli indirizzi di memoria delle istruzioni ARM sono indicati in esadecimale a sinistra di ogni istruzione assembly.

ESEMPIO DI CODICE 6.20 CHIAMATA DELLA FUNZIONE niente

Codice di alto livello	Codice assembly ARM
<pre>int main() { niente(); ... } // void = nessun valore di ritorno dalla funzione void niente() { return; }</pre>	<pre>0x00008000 MAIN 0x00008020 BL NIENTE ; chiama la funzione "niente" ... 0x0000902C NIENTE MOV PC, LR ; ritorna</pre>

Si ricordi che PC e LR sono nomi alternativi rispettivamente per R15 e R14. ARM è insolito nel senso che il PC fa parte dei registri di lavoro, quindi il ritorno da un sottoprogramma può essere fatto con un'istruzione MOV. Molti altri set di istruzioni tengono il PC in un registro riservato e usano opportune istruzioni di ritorno o di salto per terminare un sottoprogramma. Oggi i compilatori ARM effettuano un ritorno da sottoprogramma con l'istruzione BX LR. L'istruzione BX (*Branch and eXchange*) è come una normale istruzione di salto, ma costituisce anche il passaggio tra il set di istruzioni ARM standard e il set di istruzioni Thumb descritto nel paragrafo 6.7.1. In questo capitolo non si usano istruzioni Thumb né BX, ma si rimane alla versione ARMv4 con il metodo MOV PC, LR. Si vedrà nel Capitolo 7 che trattare il PC come un registro di lavoro complica la realizzazione circuitale del processore.

L'Esempio di Codice 6.21 contiene qualche errore insidioso. Gli Esempi di Codice 6.22-6.25 sono versioni migliorate del programma.

BL e MOV PC, LR sono le due istruzioni chiave per chiamata a e ritorno da funzione. BL fa due cose: memorizza l'indirizzo di ritorno (cioè l'indirizzo dell'istruzione successiva a BL) nel registro LR (*Link Register*) e salta alla destinazione.

Nell'Esempio di Codice 6.20, la funzione main chiama la funzione niente eseguendo l'istruzione BL, che salta alla label NIENTE dopo aver salvato 0x00008024 in LR. La funzione niente ritorna immediatamente al chiamante eseguendo l'istruzione MOV PC, LR che copia nel PC l'indirizzo salvato in LR. A questo punto la funzione main prosegue la propria esecuzione dall'istruzione situata a tale indirizzo (0x00008024).

Parametri di ingresso e valore di ritorno

La funzione niente dell'Esempio di Codice 6.20 non riceve alcun parametro di ingresso e non restituisce alcun valore di ritorno. Per convenzione ARM, le funzioni usano i registri R0-R3 per i parametri di ingresso e il registro R0 per il valore di ritorno. Nell'Esempio di Codice 6.21, la funzione diffdisomme viene chiamata con quattro parametri e restituisce un risultato. risultato è una variabile locale che si è deciso di tenere in R4.

In base alla convenzione ARM, il chiamante, main, mette i parametri di ingresso da sinistra a destra nel codice di alto livello nei registri R0-R3, e il chiamato, diffdisomme, memorizza in R0 il valore di ritorno. Se è necessario chiamare una funzione con più di quattro parametri, i parametri aggiuntivi vanno messi nello stack come discusso subito oltre.

ESEMPIO DI CODICE 6.21 CHIAMATA DI FUNZIONE CON PARAMETRI E VALORE DI RITORNO

Codice di alto livello	Codice assembly ARM
<pre>int main() { int y; ... y = diffdisomme(2, 3, 4, 5); ... } int diffdisomme(int f, int g, int h, int i) { int risultato; risultato = (f + g) - (h + i); return risultato; }</pre>	<pre>; R4 = y MAIN ... MOV R0, #2 ; parametro 0 = 2 MOV R1, #3 ; parametro 1 = 3 MOV R2, #4 ; parametro 2 = 4 MOV R3, #5 ; parametro 3 = 5 BL DIFFDISOMME ; chiamata della funzione MOV R4, R0 ; y = valore restituito ... ; R4 = risultato DIFFDISOMME ADD R8, R0, R1 ; R8 = f + g ADD R9, R2, R3 ; R9 = h + i SUB R4, R8, R9 ; risultato = (f + g) - (h + i) MOV R0, R4 ; mette il valore di ritorno in R0 MOV PC, LR ; ritorna al chiamante</pre>

Lo stack

Lo stack è la parte di memoria usata per salvare informazioni all'interno di una funzione. Si espande (cioè usa più memoria) quando il processore nell'esecuzione del programma ha bisogno di più spazio e si contrae (cioè libera memoria) quando il processore non ha più bisogno delle variabili temporanee memorizzate in precedenza. Prima di vedere come le funzioni usano lo stack è quindi necessario vedere come funziona lui.

Lo stack (letteralmente “pila”) è una coda di tipo LIFO (*Last In First Out*). Come in una pila di piatti, l'ultimo elemento messo (*pushed*) sullo stack, cioè il piatto in cima alla pila, è il primo che potrà essere estratto (*popped*). Una funzione può allocare spazio sullo stack per memorizzare le variabili locali, ma deve deallocare tale spazio prima di ritornare al chiamante. La cima dello stack (*top of stack*) è lo spazio allocato più di recente. Al contrario però della pila di piatti che cresce verso l'alto, lo stack di ARM cresce in memoria verso il basso, cioè verso indirizzi minori quando il programma ha bisogno di spazio ulteriore.

La **Figura 6.11** mostra un disegno dello stack: lo *stack pointer*, SP (R13), è un normale registro di ARM, ma per convenzione “punta” alla cima dello stack (si dice “punta a” per indicare che SP “contiene l'indirizzo di”). Per esempio, nella **Figura 6.11(a)**, SP contiene il valore 0xBEFFFAE8 e punta alla parola di memoria contenente 0xAB000001.

Lo stack pointer SP inizia a un indirizzo di memoria alto e si decrementa se serve spazio aggiuntivo. La **Figura 6.11(b)** mostra lo stack che cresce per contenere due ulteriori parole di memoria temporanea. Per fare ciò, SP si decrementa di otto e diventa 0xBEFFFAE0. Due ulteriori parole di dato, 0x12345678 e 0xFFEEDDCC, vengono temporaneamente memorizzate sullo stack.

Uno degli usi più importanti dello stack è il salvataggio e il ripristino dei registri usati da una funzione, che deve restituire il valore di ritorno senza altri effetti collaterali, in particolare senza modificare alcun registro tranne R0, che contiene appunto il valore di ritorno. La funzione `diffdisomme` dell'Esempio di Codice 6.21 non rispetta questa regola perché modifica R4, R8 e R9. Se `main` avesse usato questi registri prima di chiamare `diffdisomme`, i loro contenuti sarebbero stati alterati dalla chiamata di funzione.

Per evitare questo problema, la funzione deve salvare i registri sullo stack prima di modificarli e ripristinare i valori originari prima di terminare. In particolare, deve svolgere i seguenti passi.

- Allocare spazio sullo stack per memorizzare i valori contenuti in uno o più registri.
- Salvare i valori dei registri nello stack.
- Eseguire le proprie attività utilizzando i registri salvati.
- Ripristinare i valori originali dei registri prelevandoli dallo stack.
- Deallocare lo spazio nello stack.

L'**Esempio di Codice 6.22** mostra una versione migliorata di `diffdisomme`, che salva e ripristina i registri R4, R8 e R9. La **Figura 6.12** mostra lo stack prima, durante e dopo la chiamata di `diffdisomme`. Lo stack parte da 0xBEF0F0FC; `diffdisomme` fa spazio nello stack per tre parole, decrementando di 12 lo stack pointer, quindi salva il contenuto di R4, R8 e R9 nello spazio appena allocato; poi esegue le proprie attività modificando il contenuto di questi tre registri; alla fine, `diffdisomme` recupera i valori di questi registri dallo stack, dealloca lo spazio prima allocato e ritorna al chiamante. Quando ritorna, R0 contiene il valore di ritorno, ma non ci sono effetti collaterali perché R4, R8, R9 e SP contengono i valori che avevano prima della chiamata.

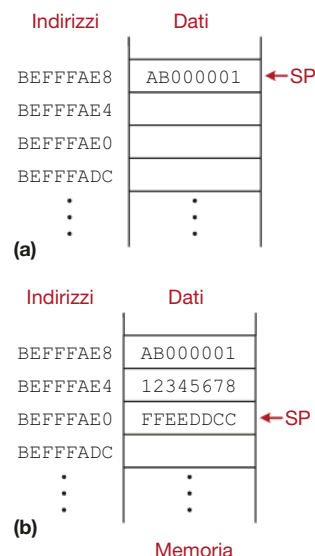


Figura 6.11
Lo stack (a) prima dell'inserimento e (b) dopo l'inserimento di due parole.

Lo stack è generalmente memorizzato in modo tale che l'indirizzo di memoria minore corrisponda alla cima dello stack, che cresce quindi verso indirizzi di memoria progressivamente inferiori. Viene pertanto definito stack discendente. ARM consente anche la realizzazione di stack ascendenti, che crescono verso indirizzi di memoria progressivamente superiori. Lo stack pointer (SP) generalmente punta alla cima dello stack, cioè all'ultimo elemento inserito: in questo caso si parla di stack pieno. ARM consente anche la realizzazione di stack vuoti, dove SP punta alla prima locazione libera dopo la cima dello stack. La ABI (*Application Binary Interface*) di ARM definisce un modo standard per il passaggio di parametri e l'uso dello stack da parte delle funzioni, in modo che librerie sviluppate da compilatori differenti possano interoperare. La specifica è per uno stack pieno discendente, che sarà quindi utilizzato in questo capitolo.



ESEMPIO DI CODICE 6.22 SALVATAGGIO E RIPRISTINO DI REGISTRI

Codice assembly ARM

```
; R4 = risultato
DIFFDISOMME
    SUB SP, SP, #12 ; alloca spazio nello stack per 3 registri
    STR R9, [SP, #8] ; salva R9 nello stack
    STR R8, [SP, #4] ; salva R8 nello stack
    STR R4, [SP]      ; salva R4 nello stack
    ADD R8, R0, R1    ; R8 = f + g
    ADD R9, R2, R3    ; R9 = h + i
    SUB R4, R8, R9    ; risultato = (f + g) - (h + i)
    MOV R0, R4        ; mette il valore di ritorno in R0
    LDR R4, [SP]      ; recupera R4 dallo stack
    LDR R8, [SP, #4]  ; recupera R8 dallo stack
    LDR R9, [SP, #8]  ; recupera R9 dallo stack
    ADD SP, SP, #12   ; dealloca lo spazio nello stack
    MOV PC, LR        ; ritorna al chiamante
```

Lo spazio di stack che una funzione alloca per i propri scopi viene chiamato **stack frame**. Lo stack frame di `diffdisomme` è di tre parole di memoria. Il principio di modularità impone che ogni funzione debba accedere solo al proprio stack frame senza usare quello di altre funzioni.

Salvataggio e ripristino di registri multipli

Il salvataggio e il ripristino dei registri sullo stack è un’operazione così frequente che ARM fornisce due istruzioni di accesso multiplo a memoria: **LDM** (*Load Multiple*) e **STM** (*Store Multiple*), ottimizzate a tale scopo. L’**Esempio di Codice 6.23** riscrive `diffdisomme` usando queste istruzioni. Lo stack mantiene le stesse informazioni della versione precedente, ma il codice è molto più corto.

LDM e STM si presentano in quattro versioni per stack pieni o vuoti, ascendenti o discendenti (FD, *Full Descending*; ED, *Empty Descending*; FA, *Full Ascending*; EA, *Empty Ascending*). La notazione `SP!` nell’istruzione indica di salvare i dati relativamente a SP e di modificare opportunamente SP dopo il salvataggio o il ripristino. Le istruzioni **PUSH** e **POP** sono sinonimi rispettivamente di **STMFD** `SP!, {registri}` e di **LDMFD** `SP!, {registri}`, e sono il modo preferenziale di salvare e ripristinare registri nello stack pieno discendente convenzionale.

Registri preservati

Gli Esempi di Codice 6.22 e 6.23 presuppongono che tutti i registri utilizzati (R4, R8 e R9) vengano salvati e ripristinati. Tuttavia, se la funzione chiamante non usa questi registri, lo sforzo di salvarli e ripristinarli è inutile. Per evitare

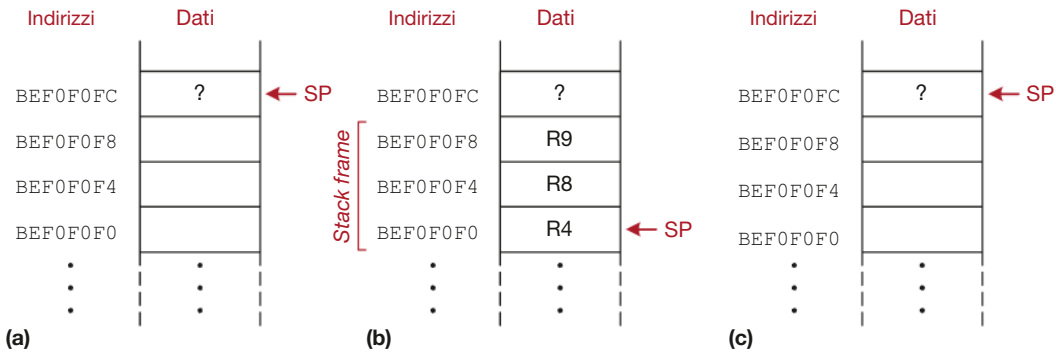


Figura 6.12 Lo stack: (a) prima, (b) durante e (c) dopo la chiamata della funzione `diffdisomme`.

ESEMPIO DI CODICE 6.23 SALVATAGGIO E RIPRISTINO DI REGISTRI MULTIPLI**Codice assembly ARM**

```

; R4 = risultato
DIFFDISOMME
    STMFD SP!, {R4, R8, R9} ; salva R4/8/9 in uno stack pieno discendente

    ADD R8, R0, R1          ; R8 = f + g
    ADD R9, R2, R3          ; R9 = h + i
    SUB R4, R8, R9          ; risultato = (f + g) - (h + i)
    MOV R0, R4              ; mette il valore di ritorno in R0

    LDMFD SP!, {R4, R8, R9} ; recupera R4/8/9 dallo stack pieno discendente
    MOV PC, LR              ; ritorna al chiamante

```

questa perdita di tempo, ARM divide i registri in **preservati** e **non preservati**. I registri preservati sono R4–R11, quelli non preservati sono R0–R3 e R12. SP e LR (cioè R13 e R14) devono pure essere preservati. Ogni funzione deve quindi salvare e ripristinare tutti i registri preservati che intende usare, ma può modificare liberamente i registri non preservati.

L'**Esempio di Codice 6.24** mostra una versione ulteriormente migliorata di `diffdisomme`, che salva solo R4 sullo stack, facendo inoltre uso delle istruzioni preferenziali `PUSH` e `POP`. Il codice riutilizza i registri di parametro non preservati R1 e R3 per memorizzare le somme intermedie quando i parametri di ingresso di tali registri non servono più.

Dal momento che per convenzione il chiamato può modificare qualsiasi registro non preservato, è responsabilità del chiamante salvare il contenuto di questo tipo di registri prima di effettuare la chiamata, e ripristinarlo dopo il ritorno del chiamato. Per questo motivo i registri preservati sono definiti “a salvataggio di chiamato” (*callee-save*), quelli non preservati “a salvataggio di chiamante” (*caller-save*).

La **Tabella 6.6** riassume quali registri sono preservati e quali no. R4–R11 sono generalmente usati per memorizzare variabili di uso interno della funzione e devono quindi essere salvati. LR deve essere pure salvato, perché contiene l'indirizzo di ritorno dalla funzione chiamata. R0–R3 e R12 sono usati per memorizzare risultati intermedi. Questi calcoli nel chiamante terminano di solito prima di chiamare una funzione, quindi sono registri non preservati e raramente il chiamante ha bisogno di salvarli prima della chiamata.

R0–R3 sono generalmente sovrascritti durante la chiamata, quindi devono essere salvati dal chiamante solo se ne ha ancora bisogno dopo l'esecuzione del chiamato. Sicuramente non è preservato R0 perché conterrà il valore di ritorno. Anche il registro di stato corrente del programma (CPSR) che contiene le *flag* di condizione non è preservato nelle chiamate a funzione.

La convenzione in base alla quale i registri sono preservati o meno fa parte dello Standard di Chiamata a Sottoprogramma dell'architettura ARM e non dell'architettura stessa. Esistono anche altri standard di chiamata dei sottoprogrammi.

ESEMPIO DI CODICE 6.24 RIDUZIONE DEL NUMERO DI REGISTRI PRESERVATI**Codice assembly ARM**

```

; R4 = risultato
DIFFDISOMME
    PUSH {R4}              ; salva R4 nello stack
    ADD R1, R0, R1          ; R1 = f + g
    ADD R3, R2, R3          ; R3 = h + i
    SUB R4, R1, R3          ; risultato = (f + g) - (h + i)
    MOV R0, R4              ; mette il valore di ritorno in R0
    POP {R4}               ; recupera R4 dallo stack
    MOV PC, LR              ; ritorna al chiamante

```

Le istruzioni `PUSH` (e `POP`) salvano (e recuperano) i contenuti dei registri nello stack in ordine crescente di numero di registro, con il registro di numero minore nella cella di indirizzo più basso, indipendentemente dall'ordine indicato nell'istruzione assembly. Quindi, per esempio, l'istruzione `PUSH {R8, R1, R3}` mette R1 nell'indirizzo di memoria più basso, poi R3 e infine R8 nei successivi indirizzi più alti sullo stack.

Tabella 6.6 Registri preservati e non preservati.

Preservato	Non preservato
Registri da salvare: R4–R11	Registro temporaneo: R12
Stack pointer: SP (R13)	Registri dei parametri: R0–R3
Indirizzo di ritorno: LR (R14)	Registro di stato corrente del programma
Stack sopra lo stack pointer	Stack sotto lo stack pointer

Lo *stack* più in alto dello stack pointer è automaticamente preservato se il chiamato evita qualsiasi modifica di parole di memoria a indirizzi maggiori di SP: in questo modo evita di accedere agli stack frame di altre funzioni. Lo stack pointer stesso è preservato, perché il chiamato dealloca lo spazio allocato all’inizio prima di terminare, sommando a SP lo stesso valore che aveva sottratto.

Un programmatore attento o un compilatore ottimizzante potrebbero accorgersi che la variabile locale `risultato` viene restituita immediatamente senza essere usata per nessun altro scopo. Si può quindi eliminare la variabile e memorizzare il risultato direttamente nel registro di ritorno R0, evitando di dover salvare e ripristinare R4 e di copiare `risultato` da R4 a R0. L’**Esempio di Codice 6.25** mostra questa versione ulteriormente ottimizzata di `diffdisomme`.

ESEMPIO DI CODICE 6.25 **VERSIONE OTTIMIZZATA DI `diffdisomme`**

Codice assembly ARM

```
; R4 = risultato
DIFFDISOMME
    ADD R1, R0, R1    ; R1 = f + g
    ADD R3, R2, R3    ; R3 = h + i
    SUB R0, R1, R3    ; risultato = (f + g) - (h + i)
    MOV PC, LR        ; ritorna al chiamante
```

Chiamate di funzioni non-foglia

Una funzione che non chiama un’altra funzione viene definita **funzione foglia** (*leaf function*): `diffdisomme` ne è un esempio. Una funzione che al suo interno chiama un’altra funzione viene quindi definita **funzione non-foglia**. Questa tipologia di funzione è un po’ più complicata perché può dover salvare sullo stack alcuni registri non preservati prima di chiamare a sua volta una funzione, e ripristinarli successivamente. In particolare:

- regola di salvataggio del chiamante:** prima di una chiamata a funzione, il chiamante deve salvare ogni registro non preservato (R0–R3 e R12) che intende usare dopo la chiamata, e successivamente ripristinarlo prima di utilizzarlo;
- regola di salvataggio del chiamato:** prima di modificare un registro preservato (R4–R11 e LR), il chiamato deve salvarne il contenuto e ripristinarlo prima di ritornare al chiamante.

L’**Esempio di Codice 6.26** mostra una funzione non-foglia `f1` e una funzione foglia `f2` con tutti i salvataggi e i ripristini del caso. Si suppone che `f1` mantenga `i` in R4 e `x` in R5, e che `f2` mantenga `r` in R4. `f1` usa i registri preservati R4, R5 e LR, quindi li salva inizialmente nello stack, secondo la regola di salvataggio del chiamato. Dato che fa uso di R12 per salvare il ri-

Una funzione “non-foglia”, che voglia cioè a sua volta chiamare con BL un’altra funzione, sovrascrive LR. Una funzione di questo genere deve quindi sempre salvare LR sul suo stack e ripristinarlo prima di ritornare al chiamante.

ESEMPIO DI CODICE 6.26 CHIAMATA DI FUNZIONE NON-FOGLIA**Codice di alto livello**

```
int f1(int a, int b) {
    int i, x;
    x = (a + b)*(a - b);
    for (i=0; i<a; i++)
        x = x + f2(b+i);
    return x;
}
```

```
int f2(int p) {
    int r;
    r = p + 5;
    return r + p;
}
```

Codice assembly ARM

```
; R0 = a, R1 = b, R4 = i, R5 = x
F1
    PUSH {R4, R5, LR} ; salva i registri preservati usati da f1
    ADD R5, R0, R1    ; x = (a + b)
    SUB R12, R0, R1   ; temp = (a - b)
    MUL R5, R5, R12   ; x = x * temp = (a + b) * (a - b)
    MOV R4, #0        ; i = 0
CICLO
    CMP R4, R0        ; i < a?
    BGE RITORNO      ; no: esce dal ciclo
    PUSH {R0, R1}     ; salva i registri non preservati
    ADD R0, R1, R4     ; il parametro è b + i
    BL F2             ; chiama f2(b+i)
    ADD R5, R5, R0     ; x = x + f2(b+i)
    POP {R0, R1}      ; ripristina i registri non preservati
    ADD R4, R4, #1     ; i++
    B CICLO           ; ripete il ciclo for
RITORNO
    MOV R0, R5 ; il valore di ritorno è x
    POP {R4, R5, LR} ; ripristina i registri preservati
    MOV PC, LR      ; ritorno da f1

; R0 = p, R4 = r
F2
    PUSH {R4}        ; salva i registri preservati usati da f2
    ADD R4, R0, #5    ; r = p + 5
    ADD R0, R4, R0    ; il valore di ritorno è r + p
    POP {R4}          ; ripristina i registri preservati
    MOV PC, LR        ; ritorno da f2
```

sultato intermedio $(a - b)$, non ha bisogno di preservare un altro registro a questo scopo. Prima di chiamare `f2`, `f1` salva sullo stack `R0` e `R1`, secondo la regola di salvataggio del chiamante, perché si tratta di registri non preservati che `f2` potrebbe modificare e di cui `f1` ha ancora bisogno dopo la chiamata. Sebbene anche `R12` sia un registro non preservato che `f2` potrebbe modificare, `f1` non ne ha più bisogno per cui non deve salvarlo. Quindi `f1` mette in `R0` il parametro per `f2`, fa la chiamata e usa il valore di ritorno in `R0`. Poi ripristina `R0` e `R1` perché ne ha ancora bisogno. Quando ha finito, `f1` mette il suo valore di ritorno in `R0`, ripristina i registri preservati `R4`, `R5` e `LR` e ritorna. `f2` si limita a salvare e ripristinare `R4`, secondo la regola di salvataggio del chiamante.

La **Figura 6.13** mostra lo stack durante l'esecuzione di `f1`. Lo stack pointer inizialmente contiene `0xBEF7FF0C`.

Chiamate di funzioni ricorsive

Una **funzione ricorsiva** è una funzione non-foglia che chiama se stessa. Si comporta quindi sia da chiamante sia da chiamato, e deve pertanto salvare sia i registri preservati sia quelli non preservati. Per esempio, il calcolo del fattoriale può essere effettuato con una funzione ricorsiva. Come noto, $fattoriale(n) = n \times (n - 1) \times (n - 2) \times \dots \times 2 \times 1$. Quindi l'espressione del fattoriale può essere scritta nella forma ricorsiva $fattoriale(n) = n \times fattoriale(n - 1)$ come è stato fatto nell'**Esempio di Codice 6.27**. Il fattoriale di 1 è semplicemente 1. Per facilitare il riferimento agli indirizzi del programma, si ipotizza di partire dalla cella `0x8500`.

Se si osserva attentamente, si può notare che `f2` non modifica `R1`, quindi `f1` non aveva bisogno di salvarlo e recuperarlo. Tuttavia un compilatore non può sempre accertare quali tra i registri non preservati saranno alterati da una chiamata a funzione. Quindi un compilatore non sofisticato fa sempre in modo che il chiamante salvi e recuperi tutti i registri non preservati che deve utilizzare dopo la chiamata.

Un compilatore ottimizzante potrebbe accorgersi che `f2` è una funzione che non chiama altre funzioni, cioè è una funzione "foglia", e potrebbe allocare `r` a un registro non preservato evitando di salvare e recuperare `R4`.

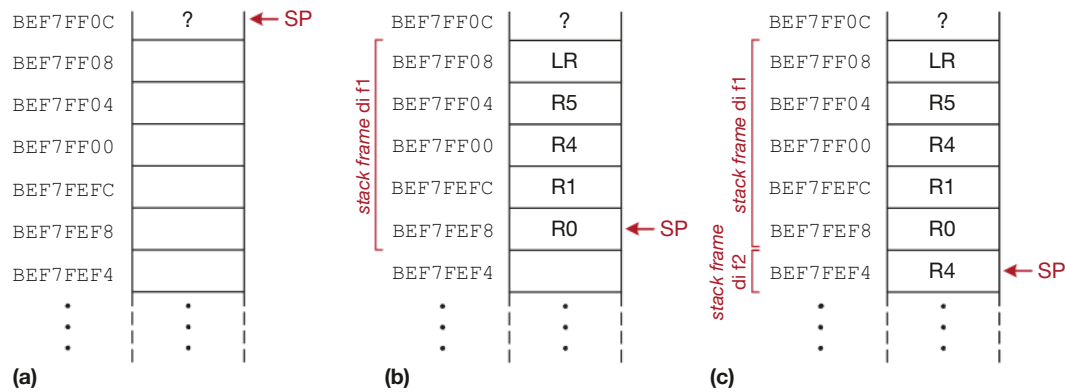


Figura 6.13 Lo stack: (a) prima delle chiamate di funzione, (b) durante f1, e (c) durante f2.

ESEMPIO DI CODICE 6.27 LA FUNZIONE RICORSIVA fattoriale

Codice di alto livello	Codice assembly ARM
<pre>int fattoriale(int n) { if (n <= 1) return 1; else return (n * fattoriale(n - 1)); }</pre>	<pre>0x8500 FATTORIALE PUSH {R0, LR} ; salva n e LR sullo stack 0x8504 CMP R0, #1 ; R0 <= 1? 0x8508 BGT ELSE ; no: salta al ramo else 0x850C MOV R0, #1 ; altrimenti restituisce 1 0x8510 ADD SP, SP, #8 ; ripristina SP 0x8514 MOV PC, LR ; ritorna 0x8518 ELSE SUB R0, R0, #1 ; n = n - 1 0x851C BL FATTORIALE ; chiamata ricorsiva 0x8520 POP {R1, LR} ; ripristina n (in R1) e LR 0x8524 MUL R0, R1, R0 ; R0 = n * fattoriale(n - 1) 0x8528 MOV PC, LR ; ritorna</pre>

Secondo la regola di salvataggio del chiamato, fattoriale è una funzione non-foglia e deve salvare LR. Secondo la regola di salvataggio del chiamante, fattoriale avrà bisogno di n dopo che si sarà chiamata, quindi deve salvare R0. Quindi salva entrambi i registri nello stack all’inizio. Poi controlla se $n \leq 1$. Se sì, mette il valore di ritorno 1 in R0, ripristina lo stack pointer e ritorna al chiamante. Non deve in questo caso ripristinare R0 e LR perché non li ha modificati. Se $n > 1$, la funzione chiama ricorsivamente fattoriale(n - 1). Quindi recupera il valore di n e il registro LR dallo stack, effettua la moltiplicazione e restituisce il risultato. Si noti che la funzione recupera il valore di n in R1 per non sovrascrivere il valore di ritorno. L’istruzione di moltiplicazione (MUL R0, R1, R0) moltiplica n (R1) e il valore di ritorno (R0) e mette il risultato in R0.

La Figura 6.14 mostra lo stack durante l’esecuzione di fattoriale(3). Per chiarezza, si mostra nella Figura 6.14(a) SP che inizialmente punta a 0xBEFF0FF0. La funzione crea uno stack frame di due parole per salvare n (R0) e LR. Alla prima chiamata, fattoriale salva R0 (che contiene n = 3) in 0xBEFF0FE8 e LR in 0xBEFF0FEC, come mostrato nella Figura 6.14(b). Quindi la funzione porta n a 2 e chiama ricorsivamente fattoriale(2), facendo sì che LR punti a 0x8520. Alla seconda chiamata, salva R0 (che contiene n = 2) in 0xBEFF0FE0 e LR in 0xBEFF0FE4. Quindi la funzione porta n a 1 e chiama ricorsivamente fattoriale(1). Alla terza chiamata, salva R0 (che contiene n = 1) in 0xBEFF0FD8 e LR in 0xBEFF0FDC. A questo punto, LR

Per chiarezza, negli esempi si salvano sempre i registri all’inizio di una chiamata a sottoprogramma. Un compilatore ottimizzante potrebbe rilevare che non c’è bisogno di salvare R0 e LR quando $n \leq 1$, quindi potrebbe salvare tali registri sullo stack solo nel ramo ELSE della funzione.

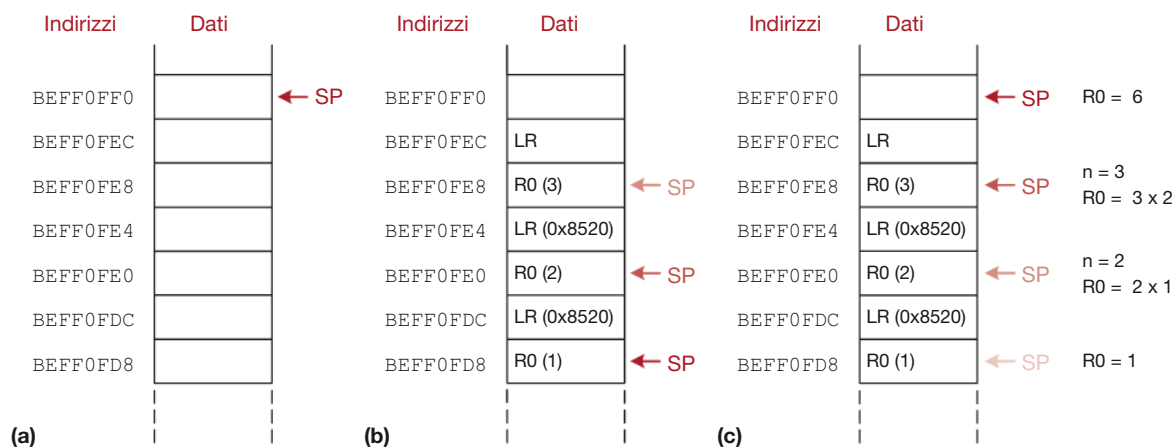


Figura 6.14 Lo stack: (a) prima, (b) durante e (c) dopo la chiamata alla funzione fattoriale con $n = 3$.

contiene di nuovo 0x8520. La terza chiamata di `fattoriale` restituisce il valore 1 in R0 e dealloca lo stack frame prima di ritornare alla seconda chiamata. La seconda chiamata recupera n (in R1) al valore 2, ripristina LR a 0x8520 (che peraltro già conteneva tale valore) dealloca lo stack frame e restituisce $R0 = 2 \times 1 = 2$ alla prima chiamata. La prima chiamata recupera n (in R1) al valore 3, ripristina LR all'indirizzo di ritorno al chiamante, dealloca lo stack frame e restituisce $R0 = 3 \times 2 = 6$. La **Figura 6.14(c)** mostra lo stack quando la funzione `fattoriale` che si è chiamata ricorsivamente torna al chiamante: lo stack pointer è alla sua posizione iniziale 0xBEFF0FF0, nessuna delle celle sopra SP è stata modificata, e tutti i registri preservati contengono i valori originali. R0 contiene il valore di ritorno, cioè 6.

Parametri aggiuntivi e variabili locali*

Le funzioni possono avere più di quattro parametri e troppe variabili locali per poterle memorizzare tutte nei registri preservati. Per memorizzare queste informazioni si usa lo stack. Per convenzione ARM, se una funzione ha più di quattro parametri, i primi quattro sono passati nei registri di parametro come al solito; i parametri aggiuntivi sono memorizzati sullo stack, appena sopra SP. Quindi il chiamante deve espandere il proprio stack per fare posto a tali parametri. La **Figura 6.15(a)** mostra lo stack del chiamante per chiamare una funzione con più di quattro parametri.

Una funzione può anche dichiarare variabili singole o array locali: queste variabili sono dichiarate all'interno della funzione e possono essere usate solo dall'interno di tale funzione. Le variabili locali sono memorizzate in R4-R11;

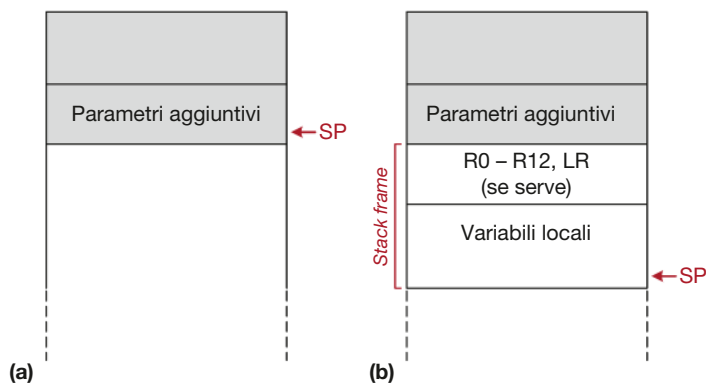


Figura 6.15 Uso dello stack: (a) prima e (b) dopo la chiamata.

se sono troppe, possono essere memorizzate nello stack frame della funzione. Questo avviene in particolare per gli array locali.

La **Figura 6.15(b)** mostra l'organizzazione dello stack del chiamato: lo stack frame contiene i registri temporanei e LR (se serve salvarli per successive chiamate di funzione) e ogni registro preservato che la funzione intende modificare; contiene inoltre gli array locali e ogni ulteriore variabile locale. Se il chiamato ha più di quattro parametri, trova quelli aggiuntivi nello stack del chiamante. L'accesso ai parametri aggiuntivi è un'eccezione nella quale una funzione può accedere a dati sullo stack che non appartengono al proprio stack frame.

6.4 ■ LINGUAGGIO MACCHINA

Il linguaggio assembly è ragionevolmente comodo da leggere per un essere umano, ma naturalmente per essere compreso dai circuiti digitali del calcolatore deve essere tradotto in sequenze di uni e zeri, che costituiscono il **linguaggio macchina**. In questa sezione si descrive il linguaggio macchina di ARM e il processo tutt'altro che agevole di traduzione da linguaggio assembly a linguaggio macchina.

ARM usa istruzioni da 32 bit. Di nuovo, regolarità garantisce semplicità, e la scelta che dà la massima regolarità è appunto quella di destinare una parola di memoria a ciascuna istruzione. Anche se non tutte le istruzioni hanno bisogno di 32 bit per essere codificate, usare istruzioni di lunghezza variabile vorrebbe dire aumentare inutilmente la complessità. La semplicità suggerirebbe anche di avere un solo formato per le istruzioni, ma sarebbe troppo restrittivo: questo consente comunque di introdurre l'ultimo principio di progetto:

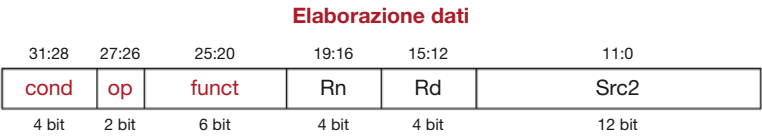
Principio progettuale n. 4: un buon progetto richiede buoni compromessi.

ARM sceglie il compromesso di avere tre formati principali di istruzioni: elaborazione dati, accesso a memoria e salti. Questo numero limitato di formati consente di avere una certa regolarità tra le varie istruzioni, quindi di semplificare i circuiti di decodifica pur soddisfacendo le esigenze di diverse istruzioni. Le istruzioni di elaborazione dati hanno un primo operando sorgente a registro, un secondo operando sorgente che può essere un immediato o un registro, eventualmente traslato, e un registro destinazione. Ci sono diverse varianti relative al secondo operando sorgente. Le istruzioni di accesso a memoria hanno tre operandi: un registro base, uno spiazzamento (*offset*) che può essere un immediato o un registro eventualmente traslato, e un ulteriore registro che è la destinazione per l'istruzione LDR oppure un'altra sorgente per l'istruzione STR. Le istruzioni di salto usano uno spiazzamento di salto immediato da 24 bit. In questa sezione si discutono i formati delle istruzioni ARM e il modo in cui vengono codificate in binario. L'Appendice B contiene una guida rapida a tutte le istruzioni ARMv4.

6.4.1 Istruzioni di elaborazione dati

Il formato delle istruzioni di elaborazione dati è il più comune. Il primo operando sorgente è un registro, il secondo può essere un immediato oppure un registro, eventualmente traslato. Un terzo registro è la destinazione. La **Figura 6.16** mostra questo formato. L'istruzione a 32 bit ha sei campi: *cond*, *op*, *funct*, *Rn*, *Rd* e *Src2*.

Figura 6.16
Formato delle istruzioni di elaborazione dati.



L'operazione che l'istruzione deve svolgere è codificata nei campi evidenziati in rosso *op* (da *opcode*, contrazione di *Operation CODE*) e *funct* (da *function*, funzione); il campo *cond* (da *condition*, condizioni) codifica l'eventuale esecuzione condizionata sulla base delle flag descritte nel paragrafo 6.3.2. Si noti che si ha $cond = 1110_2$ per istruzioni non condizionate, da eseguire comunque. Per le istruzioni di elaborazione dati, $op = 00_2$.

Gli operandi sono codificati in tre campi: *Rn* è il registro del primo operando sorgente, *Src2* è il secondo operando sorgente, *Rd* è il registro destinazione.

La **Figura 6.17** mostra il formato del campo *funct* e le tre varianti del campo *Src2* per le istruzioni di elaborazione dati. *funct* ha tre sottocampi: *I*, *cmd* e *S*. Il bit *I* vale 1 quando *Src2* è un immediato. Il bit *S* vale 1 quando l'istruzione imposta le flag di condizione. Per esempio, `SUBS R1, R9, #11` ha $S = 1$. *cmd* indica la specifica operazione da svolgere, come elencato nella Tabella B.1 dell'Appendice B. Per esempio, *cmd* vale 4 (0100_2) per `ADD` e 2 (0010_2) per `SUB`.

Le tre varianti di *Src2* consentono di avere come secondo operando: (1) un immediato; (2) un registro (*Rm*) eventualmente traslato di una costante (*shamt5*, da *shift amount* su 5 bit); oppure (3) un registro (*Rm*) traslato del contenuto di un altro registro (*Rs*). Per le ultime due possibilità, il sottocampo *sh* codifica il tipo di traslazione, come mostrato successivamente nella Tabella 6.8.

Le istruzioni di elaborazione dati hanno anche una rappresentazione inconsueta basata su un immediato a 8 bit, *imm8*, e su una rotazione a 4 bit, *rot*. *imm8* viene ruotato a destra di $2 \times rot$ per costruire una costante a 32 bit. La **Tabella 6.7** mostra alcuni esempi di rotazione e i valori a 32 bit che ne derivano partendo dall'immediato a 8 bit `0xFF`. Questa rappresentazione è comoda perché consente di codificare molte utili costanti, compresi piccoli multipli di potenze di due, in pochi bit. Il paragrafo 6.6.1 mostra come codificare costanti a 32 bit arbitrarie.

La **Figura 6.18** mostra il codice macchina di `ADD` e `SUB` quando *Src2* è un registro. Il modo più semplice per tradurre codice assembly in codice macchina è quello di scrivere il valore da inserire in ogni campo dell'istruzione e poi convertirlo in binario; si possono raggruppare a 4 a 4 i bit e convertire ogni gruppo in una cifra esadecimale per avere una rappresentazione più compatta. Attenzione che il registro destinazione è il primo nell'istruzione assembly, mentre si trova nel secondo campo (*Rd*) nell'istruzione macchina. *Rn* e *Rm* sono rispettivamente il primo e il secondo operando sorgente. Per esempio, l'istruzione `ADD R5, R6, R7` ha $Rn = 6$, $Rd = 5$ e $Rm = 7$.

Rd è l'abbreviazione di "registro destinazione". *Rn* e *Rm* indicano il primo e il secondo registro sorgente.

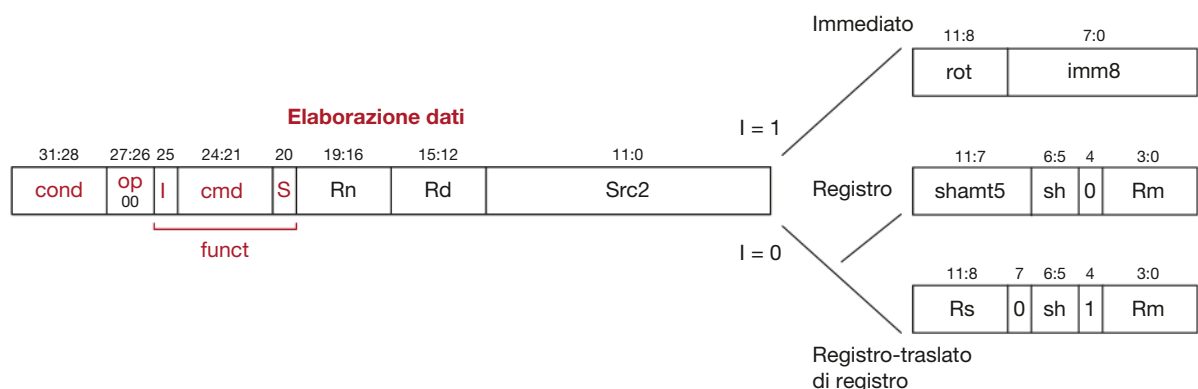


Figura 6.17 Formato delle istruzioni di elaborazione dati con dettagli del campo *funct* e delle alternative per *Src2*.

Se una costante immediata ha diverse possibili codifiche, la rappresentazione scelta è quella con il valore di rotazione *rot* minimo. Per esempio, #12 viene rappresentata come (*rot*, *imm8*) = (0000, 00001100) e non come (0001, 00110000).

Tabella 6.7 Rotazioni immediate e costanti a 32 bit risultanti per *imm8* = 0xFF.

rot		costante a 32 bit							
0000	0000	0000	0000	0000	0000	0000	0000	1111	1111
0001	1100	0000	0000	0000	0000	0000	0000	0011	1111
0010	1111	0000	0000	0000	0000	0000	0000	0000	1111
...		...							
1111	0000	0000	0000	0000	0000	0000	0011	1111	1100

La **Figura 6.19** mostra il codice macchina per le istruzioni `ADD` e `SUB` con un operando immediato e due operandi a registro. Di nuovo, il registro destinazione è il primo nell'istruzione assembly, mentre si trova nel secondo campo (*Rd*) nell'istruzione macchina. L'immediato dell'istruzione `ADD` (42) può essere codificato su 8 bit quindi non richiede rotazione (*imm8* = 42, *rot* = 0), mentre l'immediato dell'istruzione `SUB R8, R3, 0xFF0` non può essere codificato direttamente negli 8 bit di *imm8*. Si usa quindi *imm8* = 255 (0xFF) ruotato a destra di 28 bit (*rot* = 14). Si noti che ruotare a destra di 28 bit equivale a ruotare a sinistra di 32 – 28 = 4 bit.

Anche le traslazioni (*shift*) sono istruzioni di elaborazione dati. Come detto nel paragrafo 6.3.1, il valore di traslazione (cioè il numero di posizioni di bit di cui traslare) può essere codificato con un immediato a 5 bit o con un registro.

La **Figura 6.20** mostra il codice macchina per la traslazione logica a sinistra (`LSL`) e per la rotazione a destra (`ROT`) con valori di traslazione immediati. Il campo *cmd* vale 13 (1101₂) per tutte le istruzioni di traslazione, mentre il campo *sh* (da *shift*) codifica il tipo di traslazione da effettuare, come riportato nella **Tabella 6.8**. *Rm* (cioè *R5*) contiene il valore a 32 bit da traslare, e *shamt5* specifica il valore di traslazione. Il valore traslato viene memorizzato in *Rd*. *Rn* non è usato, quindi viene lasciato a 0.

La **Figura 6.21** mostra il codice macchina per `LSR` e `ASR` con il valore di traslazione codificato negli 8 bit meno significativi di *Rs* (rispettivamente *R6* e *R12*). Come prima, *cmd* è 13 (1101₂), *sh* codifica il tipo di traslazione, *Rm* contiene il valore da traslare, e il risultato della traslazione viene memorizzato in *Rd*. Questa istruzione usa il modo di indirizzamento a registro con traslazione a registro, nel quale un registro (*Rm*) viene traslato del valore contenuto in un altro registro (*Rs*). Dal momento che si usano gli 8 bit meno significativi di *Rs*, *Rm* può essere traslato al massimo di 255 posizioni. Per esempio, se *Rs* contie-

Codice assembly	Valori dei campi												Codice macchina																									
	31:28		27:26		25	24:21		20	19:16		15:12		11:7		6:5	4	3:0			31:28		27:26		25	24:21		20	19:16		15:12		11:7		6:5	4	3:0		
ADD R5, R6, R7 (0xE0865007)	1110 ₂		00 ₂		0	0100 ₂		0	6		5		0		0	0	0	7			1110		00		0	0100		0	0110		0101		00000		00	0	0111	
SUB R8, R9, R10 (0xE049800A)	1110 ₂		00 ₂		0	0010 ₂		0	9		8		0		0	0	0	10			1110		00		0	0010		0	1001		1000		00000		00	0	1010	
	cond		op		I	cmd		S	Rn		Rd		shamt5		sh		Rm			cond		op		I	cmd		S	Rn		Rd		shamt5		sh		Rm		

Figura 6.18 Istruzioni di elaborazione dati con tre operandi registro.

Codice assembly		Valori dei campi										Codice macchina									
		31:28	27:26	25	24:21	20	19:16	15:12	11:8		7:0	31:28	27:26	25	24:21	20	19:16	15:12	11:8		7:0
ADD R0, R1, #42 (0xE281002A)		1110 ₂	00 ₂	1	0100 ₂	0	1	0	0		42	1110	00	1	0100	0	0001	0000	0000		00101010
SUB R2, R3, #0xFF0 (0xE2432EFF)		1110 ₂	00 ₂	1	0010 ₂	0	3	2	14		255	1110	00	1	0010	0	0011	0010	1110		11111111
		cond	op	I	cmd	S	Rn	Rd	rot		imm8	cond	op	I	cmd	S	Rn	Rd	rot		imm8

Figura 6.19 Istruzioni di elaborazione dati con un operando immediato e due operandi registro.

Codice assembly		Valori dei campi											Codice macchina										
		31:28	27:26	25	24:21	20	19:16	15:12	11:7	6:5	4	3:0	31:28	27:26	25	24:21	20	19:16	15:12	11:7	6:5	4	3:0
LSL	R0, R9, #7 (0xE1A00389)	1110 ₂	00 ₂	0	1101 ₂	0	0	0	7	00 ₂	0	9	1110	00	0	1101	0	0000	0000	00111	00	0	1001
ROR	R3, R5, #21 (0xE1A03AE5)	1110 ₂	00 ₂	0	1101 ₂	0	0	3	21	11 ₂	0	5	1110	00	0	1101	0	0000	0011	10101	11	0	0101
		cond	op	I	cmd	S	Rn	Rd	shamt5	sh		Rm	cond	op	I	cmd	S	Rn	Rd	shamt5	sh		Rm

Figura 6.20 Istruzioni di traslazione con valore di traslazione immediato.

Codice assembly	Valori dei campi													Codice macchina												
	31:28	27:26	25	24:21	20	19:16	15:12	11:8	7	6:5	4	3:0		31:28	27:26	25	24:21	20	19:16	15:12	11:8	7	6:5	4	3:0	
LSR R4, R8, R6 (0xE1A04638)	1110 ₂	00 ₂	0	1101 ₂	0	0	4	6	0	01 ₂	1	8		1110	00	0	1101	0	0000	0100	0110	0	01	1	1000	
ASR R5, R1, R12 (0xE1A05C51)	1110 ₂	00 ₂	0	1101 ₂	0	0	5	12	0	10 ₂	1	1		1110	00	0	1101	0	0000	0101	1100	0	10	1	0001	
	cond	op	I	cmd	S	Rn	Rd	Rs		sh		Rm		cond	op	I	cmd	S	Rn	Rd	Rs		sh		Rm	

Figura 6.21 Istruzioni di traslazione con valore di traslazione a registro.

Tabella 6.8 Codifiche del campo *sh*.

Istruzione	sh	Operazione
LSL	00 ₂	Logical shift left
LSR	01 ₂	Logical shift right
ASR	10 ₂	Arithmetic shift right
ROR	11 ₂	Rotate right

ne 0xF001001C, il valore di traslazione è 0x1C (28). Si noti che una traslazione di più di 31 bit butta tutti i bit fuori da una parte e produce una sequenza di tutti zeri; invece la rotazione è ciclica, quindi ruotare di 50 posizioni è come ruotare di $50 - 32 = 18$ posizioni.

6.4.2 Istruzioni di accesso a memoria

Le istruzioni di accesso a memoria usano un formato simile a quello delle istruzioni di elaborazione dati, con gli stessi sei campi *cond*, *op*, *funct*, *Rn*, *Rd* e *Src2*, come mostrato nella Figura 6.22. Hanno però una diversa codifica del campo *funct*, due varianti del campo *Src2* e il campo *op* = 01₂. *Rn* è il registro base, *Src2* costituisce lo spiazzamento (*offset*) e *Rd* è il registro destinazione nelle istruzioni di lettura da memoria e il registro sorgente in quelle di scrittura in memoria. L'*offset* può essere un immediato *unsigned* a 12 bit (*imm12*) oppure un registro (*Rm*) eventualmente traslato di una costante (*shamt5*). *funct* è costituito da sei bit di controllo: *I*, *P*, *U*, *B*, *W* e *L*. I due bit *I* (*immediate*) e *U* (*add*) determinano se lo spiazzamento è un immediato o un registro, e se deve essere sommato o sottratto, secondo quanto specificato nella Tabella 6.9. I due bit *P* (*pre-index*) e *W* (*writeback*) specificano il modo di gestione indice secondo quanto specificato nella Tabella 6.10. I due bit *L* (*load*) e *B* (*byte*) specificano il tipo di accesso a memoria secondo quanto specificato nella Tabella 6.11.

Tabella 6.9 Bit di controllo del tipo di spiazzamento per istruzioni di accesso a memoria.

Significato		
Bit	I	U
0	Spiazzamento immediato in Src2	Sottrae lo spiazzamento dalla base
1	Spiazzamento a registro in Src2	Somma lo spiazzamento alla base

Tabella 6.10 Bit di controllo del modo di gestione indice per istruzioni di accesso a memoria.

P	W	Modo di gestione indice
0	0	Post-indice
0	1	Non supportato
1	0	Spiazzamento
1	1	Pre-indice

Tabella 6.11 Bit di controllo del tipo di operazione per istruzioni di accesso a memoria.

L	B	Istruzione
0	0	STR
0	1	STRB
1	0	LDR
1	1	LDRB

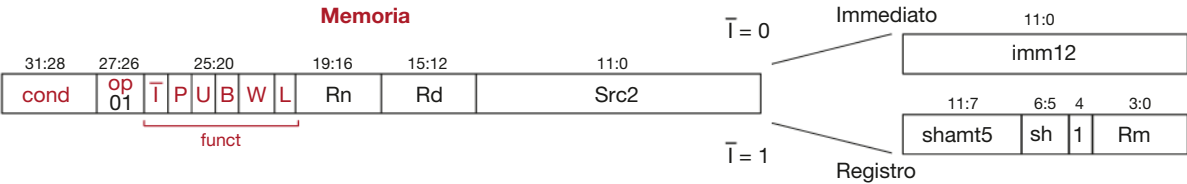


Figura 6.22 Formato delle istruzioni di accesso a memoria LDR, STR, LDRB e STRB.

ESEMPIO 6.3

Tradurre istruzioni di accesso a memoria in linguaggio macchina. Tradurre in codice macchina la seguente istruzione *assembly*.

```
STR R11, [R5], #-26
```

Soluzione STR è un'istruzione di accesso a memoria, quindi ha il campo *op* che vale 01₂. In base alla Tabella 6.11, L = 0 e B = 0 per STR. L'istruzione usa post-indicizzazione e quindi, in base alla Tabella 6.10, P = 0 e W = 0. Lo spiazzamento immediato (26) è sottratto alla base, quindi $\bar{T} = 0$ e U = 0. La **Figura 6.23** mostra ogni campo dell'istruzione macchina, che risulta essere 0xE405B01A.

Si noti la codifica contro-intuitiva del modo post-indice.

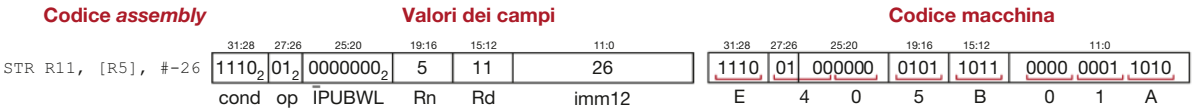


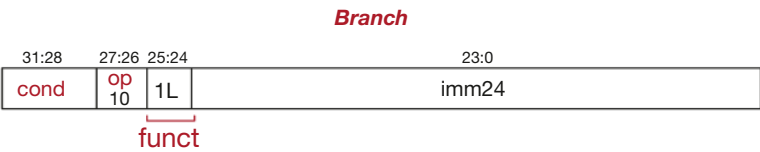
Figura 6.23 Codice macchina dell'istruzione di accesso a memoria dell'Esempio 6.3.

6.4.3 Istruzioni di salto

Le istruzioni di salto usano un solo operando immediato *signed* a 24 bit, *imm24*, come mostrato nella **Figura 6.24**. Come le istruzioni di elaborazione dati e di accesso a memoria, anche i salti cominciano con un campo a 4 bit per l'esecuzione condizionata e un campo *op* = 10₂. Il campo *funct* è di soli 2 bit: il più significativo è sempre 1, il meno significativo, L, indica il tipo di salto: 1 per BL, 0 per B. Il rimanente campo *imm24* codificato come numero in complemento a due di 24 bit serve a specificare l'indirizzo dell'istruzione alla quale saltare, relativamente a PC + 8.

L'**Esempio di Codice 6.28** mostra l'uso dell'istruzione di salto se minore (BLT) mentre la **Figura 6.25** mostra il codice macchina di tale istruzione. L'**indirizzo di destinazione del salto** (BTA, *Branch Target Address*) è l'indirizzo della prossima istruzione da eseguire se il salto viene effettuato. L'istruzione BLT della Figura 6.25 ha BTA = 0x80B4, cioè l'indirizzo della label LAGGIU.

Figura 6.24
Formato delle istruzioni di salto.

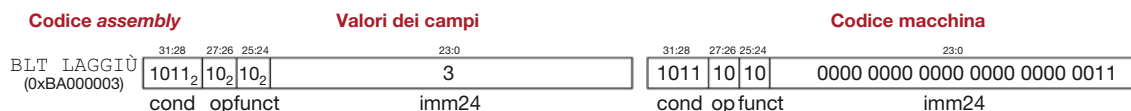


ESEMPIO DI CODICE 6.28 CALCOLO DELL'INDIRIZZO DI DESTINAZIONE DEL SALTO
Codice assembly ARM

```

0x80A0      BLT LAGGIU
0x80A4      ADD R0, R1, R2
0x80A8      SUB R0, R0, R9
0x80AC      ADD SP, SP, #8
0x80B0      MOV PC, LR
0x80B4 LAGGIU SUB R0, R0, #1
0x80B8      ADD R3, R3, #0x5

```


Figura 6.25 Codice macchina dell'istruzione di salto se minore di (BLT).

Il campo immediato a 24 bit (*imm24*) contiene il numero di istruzioni comprese tra BTA e PC + 8 (ovvero due istruzioni dopo quella di salto). In questo caso, il valore di *imm24* nell'istruzione BLT vale 3 perché BTA (0x80B4) è appunto tre istruzioni dopo PC + 8 (0x80A8).

Il processore calcola BTA a partire dall'istruzione estendendo a 32 bit il segno dell'immediato a 24 bit, traslandolo a sinistra di 2 posizioni (per convertire indirizzi di parola in indirizzi di byte) e sommandolo a PC + 8.

ESEMPIO 6.4

Calcolare il campo immediato per indirizzamento relativo al PC. Calcolare il campo immediato e mostrare il codice macchina dell'istruzione di salto presente nel seguente programma in linguaggio assembly.

```

0x8040 TEST   LDRB R5, [R0, R3]
0x8044      STRB R5, [R1, R3]
0x8048      ADD R3, R3, #1
0x8044      MOV PC, LR
0x8050      BL TEST
0x8054      LDR R3, [R1], #4
0x8058      SUB R4, R3, #9

```

Soluzione La **Figura 6.26** mostra il codice macchina per l'istruzione di salto con collegamento (BL, *Branch and Link*). Il suo indirizzo di destinazione del salto (0x8040) è sei istruzioni prima di PC + 8, quindi il campo immediato deve valere -6.


Figura 6.26 Codice macchina dell'istruzione BL.

ARM è inconsueta tra le architetture RISC poiché consente al secondo operando sorgente di essere traslato nei modi di indirizzamento base e a registro. Questo richiede un circuito traslatore in serie all'ALU ma riduce notevolmente la lunghezza del codice macchina in programmi comuni, come per esempio gli accessi agli array. Per esempio, in un array di elementi da 32 bit, l'indice dell'array deve essere traslato a sinistra di due posizioni per calcolare lo spiazzamento del byte nell'array. Sono consentiti tutti i tipi di traslazione, anche se le più frequenti sono quelle a sinistra per moltiplicare.

6.4.4 Modi di indirizzamento

Questo paragrafo riassume i modi utilizzati per indirizzare gli operandi delle istruzioni. ARM utilizza quattro modi principali: a registro, immediato, base e relativo al PC. Molte altre architetture forniscono modi simili, quindi comprenderli consente di imparare facilmente anche altri linguaggi assembly. Gli indirizzamenti a registro e base hanno molti sotto-modi descritti oltre. I primi tre modi (a registro, immediato e base) definiscono dove leggere e scrivere gli operandi, l'ultimo modo relativo al PC definisce come modificare appunto il PC (*Program Counter*). La **Tabella 6.12** riassume i modi di indirizzamento e ne fornisce qualche esempio.

Le istruzioni di elaborazione dati usano indirizzamento a registro o immediato, dove il primo operando sorgente è un registro e il secondo un registro o un immediato. ARM consente che il secondo registro possa essere traslato di un numero di posizioni specificato in un immediato oppure in un terzo registro. Le istruzioni di accesso a memoria usano indirizzamento base, nel quale l'indirizzo base viene da un registro e lo spiazzamento (*offset*) da un immediato, da un registro o da un registro traslato di un immediato. I salti usano indirizzamento relativo al PC, con l'indirizzo di destinazione del salto calcolato aggiungendo uno spiazzamento a PC + 8.

6.4.5 Interpretare il linguaggio macchina

Per interpretare il linguaggio macchina si devono decodificare i vari campi di ciascuna parola a 32 bit contenente un'istruzione. Le diverse istruzioni usano formati diversi, ma tutti iniziano con il campo *cond* a 4 bit per l'esecuzione condizionata, seguito dai due bit del campo *op*. Proprio da quest'ultimo conviene partire: se vale 00₂, allora si tratta di un'istruzione di elaborazione dati; se vale 01₂, è un'istruzione di accesso a memoria; se vale 10₂, è un'istruzione di salto. Sulla base di questa prima divisione, si possono interpretare i campi successivi.

Tabella 6.12 Modi di indirizzamento degli operandi in ARM.

Modo di indirizzamento dell'operando	Esempio	Descrizione
A registro		
A registro semplice	ADD R3, R2, R1	$R3 \leftarrow R2 + R1$
A registro con traslazione immediata	SUB R4, R5, R9, LSR #2	$R4 \leftarrow R5 - (R9 \gg 2)$
A registro con traslazione a registro	ORR R0, R10, R2, ROR R7	$R0 \leftarrow R10 \mid (R2 \text{ ROR } R7)$
Immediato	SUB R3, R2, #25	$R3 \leftarrow R2 - 25$
Base		
Spiazzamento immediato	STR R6, [R11, #77]	$\text{mem}[R11 + 77] \leftarrow R6$
Spiazzamento a registro	LDR R12, [R1, -R5]	$R12 \leftarrow \text{mem}[R1 - R5]$
Spiazzamento a registro con traslazione immediata	LDR R8, [R9, R2, LSL #2]	$R8 \leftarrow \text{mem}[R9 + (R2 \ll 2)]$
Relativo al PC	B DEST	Salta a DEST

ESEMPIO 6.5

Tradurre dal linguaggio macchina al linguaggio *assembly*. Tradurre le seguenti istruzioni macchina in istruzioni assembly.

0xE0475001
0xE5949010

Soluzione Per prima cosa si scrivono le due istruzioni in binario e si guardano i bit 27:26 per individuare il valore del campo *op* di ciascuna istruzione, come mostrato

nella **Figura 6.27**. I valori del campo *op* sono in questo caso 00_2 e 01_2 , che indicano rispettivamente un'istruzione di elaborazione dati e una di accesso a memoria. Poi si può esaminare il campo *funct* delle istruzioni.

Il campo *cmd* dell'istruzione di elaborazione dati è 2 (0010_2) e il bit *I* (il bit di posto 25) vale 0 , quindi si tratta di un'istruzione *SUB* con un registro come secondo operando sorgente *Src2*. *Rd* è 5 , *Rn* è 7 e *Rm* è 1 .

Il campo *funct* dell'istruzione di accesso a memoria è 011001_2 . *B* = 0 e *L* = 1 , quindi si tratta di un'istruzione *LDR*. *P* = 1 e *W* = 0 , che indicano un indirizzamento con spiazzamento. $\bar{I}$ = 0 , dunque lo spiazzamento è un immediato. *U* = 1 , quindi lo spiazzamento va sommato. Si tratta di un'istruzione di caricamento di un registro, con uno spiazzamento immediato da sommare al registro base. *Rd* è 9 , *Rn* è 4 e *imm12* è 16 . La Figura 6.27 mostra il codice *assembly* corrispondente alle due istruzioni.

Codice macchina										Valori dei campi										Codice assembly												
cond	op	I	cmd	S	Rn	Rd	shamt5	sh	Rm	31:28	27:26	25	24:21	20	19:16	15:12	11:7	6:5	4	3:0												
1110	00	0	0010	0	0111	0101	00000	00	0	1110	00	0	2	0	7	5	0	0	0	1	SUB R5, R7, R1											
E	0			4	7	5	0	0	1																							
cond	op	I	PUBWL	Rn	Rd	imm12											31:28	27:26	25:20	19:16	15:12	11:0										
1110	01		011001	0100	1001	0000 0001 0000											1110	01	25	4	9	16	LDR R9, [R4, #16]									
E	5			9	4	9	0	1	0																							
										cond	op	I	PUBWL	Rn	Rd	imm12																

Figura 6.27 Traduzione da codice macchina a codice assembly.

6.4.6 La potenza di un programma scritto in memoria

Un programma scritto in linguaggio macchina è una serie di numeri a 32 bit che rappresentano le istruzioni. Come tutti i numeri, anche le istruzioni possono essere scritte in memoria. Questo concetto di “programma scritto in memoria” è il motivo chiave della potenza dei calcolatori: eseguire un programma diverso non richiede alcuno sforzo di riprogettazione o ricostruzione a livello circuitale, ma semplicemente la scrittura in memoria del nuovo programma. Invece di richiedere un hardware dedicato, questo consente di lavorare con hardware di tipo generale (*general purpose*) ed eseguire applicazioni diverse (calcoli numerici, elaborazione di testi, visualizzazione di filmati) semplicemente cambiando il programma in memoria.

Le istruzioni del programma vengono recuperate (*fetched*) dalla memoria ed eseguite dal processore: quindi programmi anche complessi sono in ultima analisi una serie di letture da memoria ed esecuzioni di istruzioni.

La **Figura 6.28** mostra come sono memorizzate in memoria le istruzioni. In ARM si parte normalmente da indirizzi bassi (in questo caso $0x00008000$): si deve ricordare che la memoria di ARM è indirizzata a byte, quindi gli indirizzi delle istruzioni (di 32 bit, quindi di 4 byte) avanzano di 4 in 4.

Per eseguire il programma memorizzato (*run*) il processore esegue il *fetch* delle istruzioni in sequenza. Dopo il *fetch* l'istruzione viene decodificata ed eseguita dall'hardware del processore. L'indirizzo dell'istruzione corrente è mantenuto nel registro denominato *program counter* (PC), che in ARM è R15. Per ragioni storiche, un'operazione di lettura del registro PC restituisce l'indirizzo dell'istruzione corrente più 8.

Per eseguire il codice della Figura 6.28, il PC viene inizializzato a $0x00008000$. Il processore esegue il *fetch* dell'istruzione memorizzata a quell'indirizzo ed esegue l'istruzione, in questo caso $0xE3A01064$ (MOV R1, #100). Poi incrementa di 4 il PC che diventa $0x00008004$, fa il *fetch* ed esegue la nuova istruzione e così via.



Ada Lovelace, 1815-1852.

Matematica inglese che ha scritto il primo programma per calcolatore, per calcolare i numeri di Bernoulli usando la macchina analitica di Charles Babbage. Era la figlia del poeta Lord Byron.

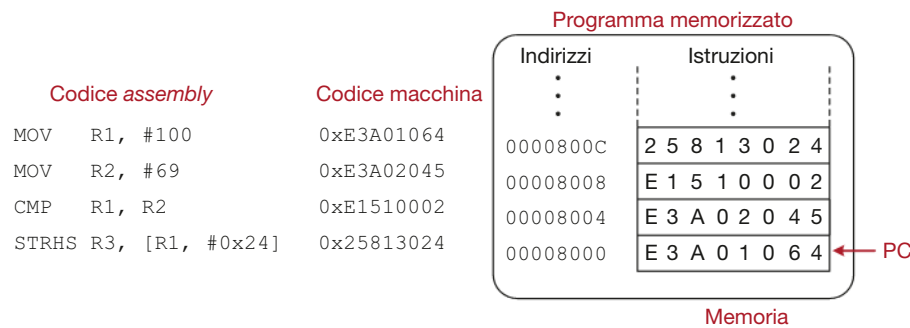


Figura 6.28 Programma memorizzato.

Lo stato di esecuzione di un microprocessore corrisponde allo stato del programma. In ARM tale stato comprende il contenuto dei registri di lavoro e dei registri di stato. Se il sistema operativo salva lo stato di esecuzione di un programma a un certo punto, può interrompere il programma, fare altro, poi ripristinare lo stato del programma interrotto e riprendere l'esecuzione senza che il programma stesso abbia risentito dell'interruzione. Lo stato di esecuzione è dunque un aspetto molto importante nel progetto di un microprocessore, come discusso nel Capitolo 7.

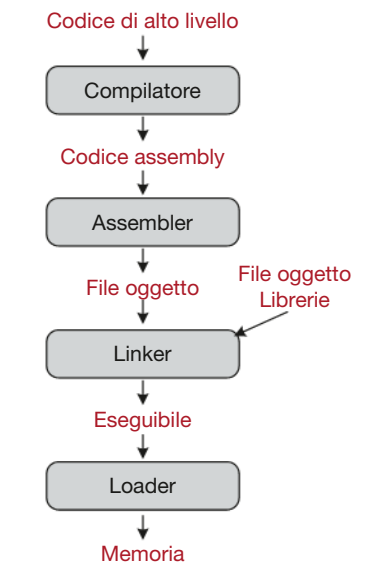


Figura 6.29
Passi per tradurre e lanciare un programma.

6.5 ■ **COMPILARE, ASSEMBLARE E CARICARE***

Si è visto sinora come tradurre brevi frammenti di codice di alto livello in linguaggio assembly e macchina. In questo paragrafo si descrive come compilare e assemblare un programma di alto livello completo e come caricarlo in memoria per farlo eseguire. Si inizia con un esempio di **mappa di memoria** di ARM che mostra dove sono posizionati il codice, i dati e lo stack.

La **Figura 6.29** mostra i passi necessari per tradurre un programma da alto livello a codice macchina e iniziarne l'esecuzione. Per prima cosa il **compilatore** traduce il codice di alto livello in codice assembly. Quindi l'**assembler** (letteralmente “assemblatore”) traduce il codice assembly in codice macchina e lo salva nel file oggetto (*object*). Successivamente il **linker** (letteralmente “collegatore”) unisce il codice macchina con quello delle librerie e di altri file, determina i corretti indirizzi di salto e le locazioni delle variabili, e produce l'intero programma eseguibile. In pratica, la maggior parte dei compilatori esegue tutte le tre fasi di compilazione, assemblaggio e collegamento. Da ultimo, il **loader** (letteralmente “caricatore”) carica il programma in memoria e ne inizia l'esecuzione. Nel resto di questo paragrafo si esaminano i tre passaggi appena citati usando un semplice programma come esempio.

6.5.1 **La mappa di memoria**

Con indirizzi a 32 bit, ARM ha uno spazio di indirizzamento di 2³² byte (4 GB). Gli indirizzi di parola sono multipli di 4, e vanno quindi da 0 a 0xFFFFF000. La **Figura 6.30** mostra un esempio di mappa di memoria. L'architettura ARM divide lo spazio di indirizzamento in cinque parti o **segmenti**: il segmento testo, il segmento dati globali, il segmento dati dinamici, il segmento per i gestori delle eccezioni, il segmento per il sistema operativo (OS) e per l'ingresso/uscita (I/O). Di seguito è descritto ciascuno di questi segmenti.

Il segmento testo

Il **segmento testo** contiene il programma in linguaggio macchina. ARM chiama questo segmento anche **segmento a sola lettura** (RO, *Read Only*). Oltre al codice può contenere costanti (dette anche *literal*) e dati a sola lettura.

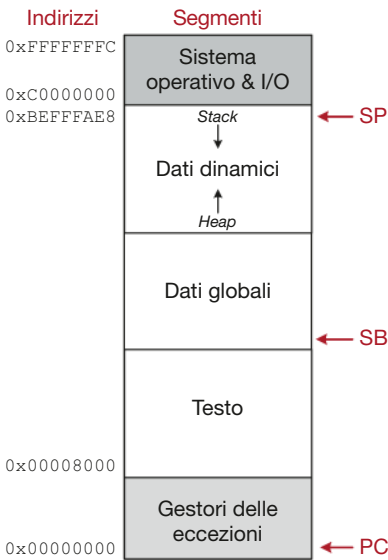


Figura 6.30
Esempio di mappa di memoria di ARM.

Il segmento dati globali

Il **segmento dati globali** memorizza le variabili globali che, al contrario di quelle locali, sono accessibili a tutte le funzioni del programma. Le variabili globali vengono allocate in memoria prima di iniziare l'esecuzione del programma. ARM chiama questo segmento anche **segmento a lettura/scrittura** (RW, *Read Write*). Si accede alle variabili globali tipicamente con un registro base statico che punta all'inizio del segmento dati globali. ARM usa per convenzione R9 a questo scopo (SB, *Static Base register*).

Il segmento dati dinamici

Il **segmento dati dinamici** contiene lo stack e l'heap. I dati di questo segmento non sono noti a priori, ma vengono allocati e deallocati dinamicamente durante l'esecuzione del programma.

All'inizio, il sistema operativo carica nello stack pointer (SP) l'indirizzo della cima del segmento, perché di solito SP cresce verso il basso, come mostrato in questo testo. Lo stack contiene i dati temporanei e le variabili locali, come gli array, che sono troppo grandi per stare nei registri. Come discusso nel paragrafo 6.3.7, le funzioni usano lo stack anche per salvare e ripristinare i registri. Si accede a ogni stack frame in modalità LIFO (*Last In First Out*, l'ultimo elemento entrato sarà il primo a uscire).

L'heap memorizza dati allocati dal programma durante l'esecuzione. In C, le allocazioni di memoria si fanno con la funzione `malloc`, in Java e C++ con la funzione `new`. Al contrario di quanto avviene per lo stack, i dati nell'heap possono essere utilizzati ed eliminati in qualsiasi ordine. L'heap normalmente cresce verso l'alto, dall'inizio del segmento dati dinamici.

Se stack e heap crescono troppo e si sovrappongono, i dati risultano evidentemente danneggiati. L'allocatore della memoria cerca di assicurarsi che questo non succeda restituendo un errore di tipo *out-of-memory* se non c'è sufficiente spazio per allocare nuovi dati dinamici.

I segmenti dei gestori delle eccezioni, di OS e di I/O

La parte più bassa della memoria di ARM è riservata alla tabella del vettore delle eccezioni e ai **gestori delle eccezioni**, a partire dall'indirizzo 0x0 (par. 6.6.3). La parte più alta della memoria è riservata al **sistema operativo** e all'**ingresso/uscita mappato in memoria** (vedi par. 9.2).

6.5.2 Compilazione

Un compilatore traduce codice di alto livello in codice assembly. Gli esempi in questo paragrafo sono basati su GCC, un compilatore gratuito molto diffuso, eseguito sul calcolatore a singola scheda Raspberry Pi (vedi par. 9.3). L'**Esempio di Codice 6.29** mostra un semplice esempio di programma di alto livello, con tre variabili e due funzioni, assieme alla traduzione in codice *assembly* prodotta da GCC.

Per compilare, assemblare e collegare un programma in C di nome `prog.c` si deve usare il comando

```
gcc -O1 -g prog.c -o prog
```

Questo comando produce un codice eseguibile denominato `prog`. L'opzione `-O1` dice al compilatore di effettuare le ottimizzazioni essenziali invece di produrre codice grossolanamente inefficiente. L'opzione `-g` dice al compilatore di includere informazioni utili al collaudo (*debugging*) del programma.

Per vedere i passi intermedi si può usare l'opzione `-S` che dice a GCC di compilare senza assemblare né collegare.

```
gcc -O1 -S prog.c -o prog.s
```

Viene qui presentato un esempio di mappa di memoria di ARM, ma la mappa di memoria ha qualche grado di libertà. La tabella del vettore delle eccezioni deve essere posizionata all'indirizzo 0x0 e l'I/O mappato in memoria di solito è posizionato a indirizzi alti, ma l'utente può decidere dove sono posizionati il testo (codice macchina e dati costanti), lo stack e i dati globali. Si tenga presente che soprattutto qualche tempo fa la maggior parte dei sistemi ARM aveva meno di 4 GB di memoria.



Grace Hopper, 1906-1992.

Si è laureata a Yale dove ha conseguito il dottorato in matematica. Ha scritto il primo compilatore quando lavorava per la Remington Rand Corporation ed è stata determinante nello sviluppo del linguaggio di programmazione COBOL. Nel suo ruolo di ufficiale di marina ha ricevuto molti premi, inclusa la medaglia per la vittoria nella Seconda Guerra Mondiale e la medaglia del Servizio Nazionale della Difesa.

Nell'Esempio di Codice 6.29 si accede alle variabili globali usando due istruzioni di accesso a memoria: una per caricare l'indirizzo della variabile, l'altra per leggere o scrivere la variabile. Gli indirizzi delle variabili globali sono posizionati dopo il codice, a partire dall'etichetta .L3. LDR R3, .L3 carica l'indirizzo di f in R3, e STR R0, [R3, #0] scrive in f, LDR R3, .L3+4 carica l'indirizzo di g in R3, e STR R1, [R3, #0] scrive in g, e così via. Questo costrutto di codice è ulteriormente discusso nel paragrafo 6.6.1.

ESEMPIO DI CODICE 6.29 COMPILAZIONE DI UN PROGRAMMA DI ALTO LIVELLO	
Codice di alto livello	Codice assembly ARM
<pre>int f, g, y; // variabili globali int somma(int a, int b) { return (a + b); } int main(void) { f = 2; g = 3; y = somma(f, g); return y; }</pre>	<pre>.text .global somma .type somma, %function somma: add r0, r0, r1 bx lr .global main .type main, %function main: push {r3, lr} mov r0, #2 ldr r3, .L3 str r0, [r3, #0] mov r1, #3 ldr r3, .L3+4 str r1, [r3, #0] bl somma ldr r3, .L3+8 str r0, [r3, #0] pop {r3, pc} .L3: .word f .word g</pre>

Il prodotto, `prog.s`, è un file alquanto verboso: le parti interessanti sono riportate nell'Esempio di Codice 6.29. GCC vuole che le label siano seguite dal carattere due punti. Il prodotto di GCC è scritto in minuscolo e contiene altre direttive per l'assembler non discusse in questo testo. Si noti che la funzione `somma` ritorna usando l'istruzione `BX` invece di `MOV PC, LR`. Si noti anche che GCC ha deciso di salvare e ripristinare `R3` anche se non fa parte dei registri preservati. Gli indirizzi delle variabili globali vengono memorizzati in una tabella che inizia alla label `.L3`.

6.5.3 Assemblaggio

L'assembler converte il linguaggio assembly in un file oggetto (*object*) contenente codice macchina. GCC può creare il file oggetto sia da `prog.s` sia direttamente da `prog.c` usando

```
gcc -c prog.s -o prog.o
```

oppure

```
gcc -O1 -g -c prog.c -o prog.o
```

L'assembler fa due passate sul codice assembly. Nella prima passata assegna gli indirizzi alle istruzioni e individua tutti i simboli, cioè le label e i nomi delle variabili globali; nomi e indirizzi dei simboli sono salvati nella tabella dei simboli (*symbol table*). Nella seconda passata produce il codice macchina prelevando gli indirizzi delle *label* e delle variabili globali dalla tabella dei simboli. Il codice macchina e la tabella dei simboli sono salvati nel file oggetto.

Si può “disassemblare” il file oggetto con il comando `objdump` per vedere il codice assembly corrispondente al codice macchina generato. Se il codice di alto livello era stato compilato con l'opzione `-g`, il *disassembler* mostra anche le corrispondenti righe di codice C:

```
objdump -S prog.o
```

Ecco il disassemblato della sezione `.text`:

```
00000000 <somma>:
int somma(int a, int b) {
    return (a + b);
}
0: e0800001 add r0, r0, r1
4: e12ffffe bx lr
00000008 <main>:
int f, g, y; // variabili globali
int somma(int a, int b);
int main(void) {
    8: e92d4008 push {r3, lr}
    f = 2;
    c: e3a00002 mov r0, #2
    10: e59f301c ldr r3, [pc, #28] ; 34 <main+0x2c>
    14: e5830000 str r0, [r3]
    g = 3;
    18: e3a01003 mov r1, #3
    1c: e59f3014 ldr r3, [pc, #20] ; 38 <main+0x30>
    20: e5831000 str r1, [r3]
    y = somma(f,g);
    24: ebfffffe bl 0 <somma>
    28: e59f300c ldr r3, [pc, #12] ; 3c <main+0x34>
    2c: e5830000 str r0, [r3]
    return y;
}
30: e8bd8008 pop {r3, pc}
...
```

Si ricordi dal paragrafo 6.4.6 che una lettura del PC riporta l'indirizzo dell'istruzione corrente più 8. Quindi `LDR R3, [PC, #28]` carica l'indirizzo di `f`, che si trova proprio all'indirizzo $(PC + 8) + 28 = (0x10 + 0x8) + 0x1C = 0x34$.

Si può anche vedere la tabella dei simboli del file oggetto con l'opzione `-t` del comando `objdump`. Le parti interessanti sono mostrate sotto. Si noti che la funzione `somma` parte dall'indirizzo 0 e occupa 8 byte, il `main` parte dall'indirizzo 8 e occupa 0x38 byte. I nomi simbolici delle tre variabili globali `f`, `g` e `h` sono elencati e occupano 4 byte ciascuno, ma non hanno ancora indirizzi di memoria assegnati.

```
objdump -t prog.o
```

```
SYMBOL TABLE:
00000000 l d .text 00000000 .text
00000000 l d .data 00000000 .data
00000000 g F .text 00000008 somma
00000008 g F .text 00000038 main
00000004 O *COM* 00000004 f
00000004 O *COM* 00000004 g
00000004 O *COM* 00000004 y
```

6.5.4 Collegamento

La maggior parte dei programmi di grandi dimensioni è costituita da più di un file di codice di alto livello. Se il programmatore fa una modifica in un file, è una perdita di tempo ricompilare e riassemblare anche tutti gli altri. Inoltre, i programmi spesso chiamano funzioni presenti nelle librerie, che praticamente non vengono mai modificate. Infine un programma solitamente richiede del codice di inizializzazione di stack, heap ecc., che deve essere eseguito prima di lanciare la funzione `main`.

Il compito del linker è quello di unire tutti i file oggetto e il codice di inizializzazione in un unico file in linguaggio macchina definito **eseguibile** (*executable*) e assegnare gli indirizzi alle variabili globali. Il linker riloca dati e istruzioni dei file oggetto in modo che non si sovrappongano e usa le tabelle

dei simboli per risistemare il codice macchina sulla base dei nuovi indirizzi delle label e delle variabili globali. Per collegare il file oggetto si attiva il linker con il comando:

```
gcc prog.o -o prog
```

Si può disassemblare nuovamente l'eseguibile con il comando:

```
objdump -S -t prog
```

Il codice di inizializzazione è troppo lungo da mostrare; il programma in esame inizia all'indirizzo 0x8390 nel segmento testo e alle variabili globali sono assegnati indirizzi a partire da 0x10570 nel segmento dati globali. Si notino le direttive `.word` all'assembler per definire le variabili globali `f`, `g` e `h`.

```
00008390 <sum>:
int somma(int a, int b) {
    return (a + b);
}
    8390: e0800001 add r0, r0, r1
    8394: e12ffff1e bx lr

00008398 <main>:
int f, g, y; // variabili globali
int somma(int a, int b);

int main(void) {
    8398: e92d4008 push {r3, lr}
    f = 2;
    839c: e3a00002 mov r0, #2
    83a0: e59f301c ldr r3, [pc, #28] ; 83c4 <main+0x2c>
    83a4: e5830000 str r0, [r3]
    g = 3;
    83a8: e3a01003 mov r1, #3
    83ac: e59f3014 ldr r3, [pc, #20] ; 83c8 <main+0x30>
    83b0: e5831000 str r1, [r3]
    y = somma(f,g);
    83b4: ebf5ffff5 bl 8390 <somma>
    83b8: e59f300c ldr r3, [pc, #12] ; 83cc <main+0x34>
    83bc: e5830000 str r0, [r3]
    return y;
}
    83c0: e8bd8008 pop {r3, pc}
    83c4: 00010570 .word 0x00010570
    83c8: 00010574 .word 0x00010574
    83cc: 00010578 .word 0x00010578
```

L'istruzione `LDR R3, [PC, #28]` nell'eseguibile carica dalla cella di indirizzo $(PC + 8) + 28 = (0x83A0 + 0x8) + 0x1C = 0x83C4$. Questa cella di memoria contiene il valore `0x10570`, che è l'indirizzo della variabile globale `f`.

L'eseguibile contiene anche una tabella dei simboli aggiornata con gli indirizzi rilocati delle funzioni e delle variabili globali.

```
SYMBOL TABLE:
000082e4 l d .text 00000000 .text
00010564 l d .data 00000000 .data
00008390 g F .text 00000008 somma
00008398 g F .text 00000038 main
00010570 g O .bss 00000004 f
00010574 g O .bss 00000004 g
00010578 g O .bss 00000004 y
```

6.5.5 Caricamento

Il sistema operativo carica in memoria un programma copiando il segmento testo del file eseguibile da un dispositivo di memoria di massa (solitamente il disco rigido della macchina) nel segmento testo della memoria di lavoro. Poi esegue un salto alla prima istruzione del programma per iniziarne l'esecuzione.

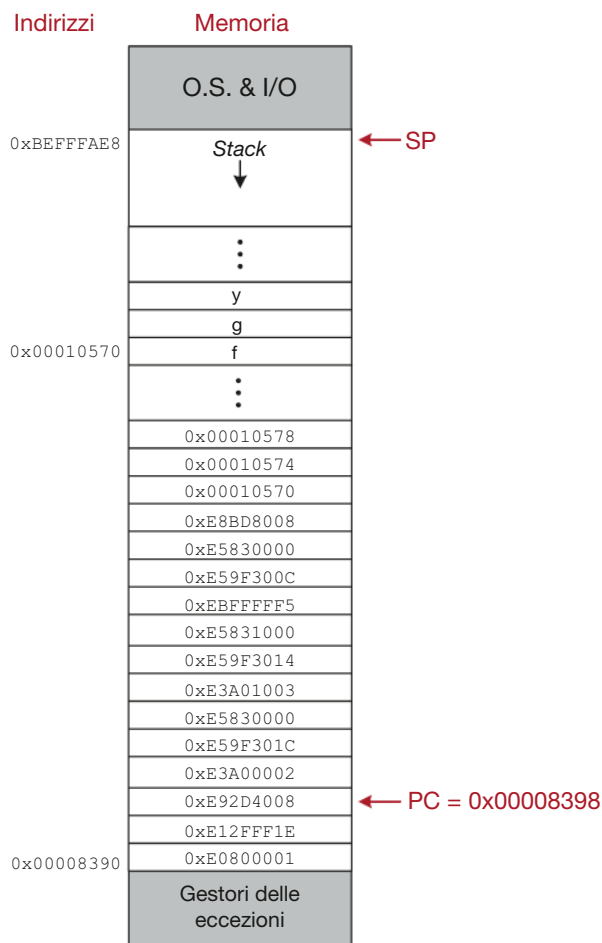


Figura 6.31
Eseguibile caricato in memoria.

ne. La **Figura 6.31** mostra la mappa di memoria all'inizio dell'esecuzione del programma.

6.6 ■ QUALCHE DETTAGLIO

Questo paragrafo affronta alcuni aspetti opzionali difficili da collocare nel resto del capitolo, come il caricamento di costanti a 32 bit (i cosiddetti *literal*), l'istruzione NOP (*NO Operation*) e le eccezioni.

6.6.1 Caricamento di *literal*

Molti programmi hanno bisogno di caricare *literal* a 32 bit, come costanti o indirizzi di memoria. L'istruzione MOV prevede una sorgente a 12 bit, quindi serve usare l'istruzione LDR per caricare questi valori prelevandoli da un serbatoio di *literal* nel segmento testo. ARM consente questi caricamenti nelle forme

```
LDR Rd, =literal
LDR Rd, =label
```

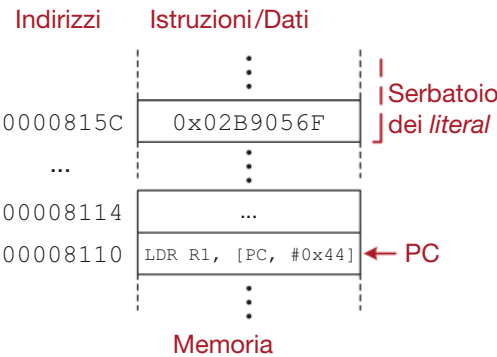
La prima forma carica nel registro *Rd* la costante a 32 bit specificata da *literal*, mentre la seconda carica l'indirizzo di memoria associato a *label*. In entrambi i casi il valore viene mantenuto nel **serbatoio dei literal**, una parte del segmento testo. Tale serbatoio deve trovarsi a meno di 4096 byte di distanza dall'istruzione LDR in modo che il caricamento si possa fare con LDR Rd, [PC, #offset_dal_literal]. Bisogna naturalmente fare in modo che il

programma durante la sua esecuzione “giri attorno” al serbatoio dei literal, cioè eviti di considerarli istruzioni (cosa evidentemente priva di senso).

L’**Esempio di Codice 6.30** mostra il caricamento di un literal. Come mostrato nella **Figura 6.32**, si supponga che l’istruzione `LDR` sia all’indirizzo `0x8110` e il literal all’indirizzo `0x815C`. Come già detto, una lettura del PC restituisce l’indirizzo 8 byte dopo quello dell’istruzione in esecuzione; quindi leggere il PC durante l’istruzione `LDR` restituisce `0x8118`. Dunque `LDR` deve usare un *offset* di `0x44` per raggiungere il serbatoio dei literal: `LDR R1, [PC, #0x44]`.

ESEMPIO DI CODICE 6.30 CAMPO IMMEDIATO DI GRANDI DIMENSIONI USANDO UN SERBATOIO DEI LITERAL	
Codice di alto livello	Codice <i>assembly</i> ARM
<code>int a = 0x2B9056F;</code>	<code>; R1 = a</code> <code> LDR R1, =0x2B9056F</code> <code> ...</code>

Figura 6.32
Esempio di serbatoio dei literal.



Le pseudo-istruzioni non fanno parte del set di istruzioni del processore, ma sono abbreviazioni di istruzioni o sequenze di istruzioni usate spesso da programmatori e compilatori. L’assembler traduce le pseudo-istruzioni in una o più istruzioni normali.

6.6.2 NOP

NOP è lo mnemonico di *NO Operation*. Si tratta di una pseudo-istruzione che non fa nulla: l’assembler la traduce come `MOV R0, R0` (in codice macchina `0xE1A00000`). Tra i suoi utilizzi vi sono introdurre ritardi e allineare istruzioni.

6.6.3 Eccezioni

Un’**eccezione** è una chiamata a sottoprogramma inaspettata, che può essere causata a livello sia hardware sia software. Per esempio, il processore può ricevere la segnalazione che l’utente ha schiacciato un tasto sulla tastiera: può a questo punto interrompere ciò che stava facendo, determinare quale tasto è stato premuto, salvarlo per uso futuro e infine riprendere l’esecuzione del programma che era in esecuzione. Questa eccezione hardware generata da un dispositivo di ingresso/uscita come la tastiera prende il nome di *interrupt*. Oppure, il programma può trovarsi in una condizione di errore, come per esempio il fetch di un’istruzione non definita nel linguaggio macchina del processore: salta quindi a una parte di codice del sistema operativo che può emulare a livello software l’istruzione inesistente oppure brutalmente abortire il programma che ha incontrato l’errore.

Le eccezioni software sono a volte chiamate *trap*: una *trap* particolarmente importante è la cosiddetta *system call*, ovvero la chiamata da parte del programma di una funzione presente nel sistema operativo che viene eseguita a livello di privilegio maggiore. Altre cause di eccezione sono il segnale di *reset* e il tentativo di accedere a celle di memoria inesistenti.

Come ogni altra chiamata di funzione, un'eccezione deve salvare l'indirizzo di ritorno, saltare a qualche indirizzo di memoria, svolgere le proprie attività, ripristinare il contesto e ritornare al programma dal quale era stata attivata. Le eccezioni usano una tabella denominata **vettore delle eccezioni** per determinare a quale locazione di memoria contenente il gestore dell'eccezione (*exception handler*) saltare, e usano altri banchi di registri rispetto a quello usato dai normali programmi per evitare di alterare il contesto del programma che ha generato l'eccezione. Le eccezioni modificano anche il livello di **privilegio** del programma, permettendo al gestore di eccezione di accedere a zone protette della memoria.

Modi di esecuzione e livelli di privilegio

Un processore ARM può lavorare in diversi **modi di esecuzione** caratterizzati da differenti livelli di privilegio. I diversi modi consentono a un'eccezione di verificarsi all'interno del gestore di un'altra eccezione senza corromperne lo stato: per esempio un *interrupt* può verificarsi mentre il processore sta eseguendo codice di sistema operativo in modo *Supervisor*, e successivamente si può verificare un'eccezione di *Abort* se l'*interrupt* ha tentato di accedere a un indirizzo di memoria non valido. Il gestore dell'eccezione può a sua volta tornare a eseguire il codice in modo *Supervisor*. Il modo di esecuzione corrente è specificato dai cinque bit meno significativi del registro CPSR, come mostrato nella Figura 6.6. La **Tabella 6.13** elenca i modi di esecuzione e le loro codifiche. Il modo *User* (utente) opera a livello di privilegio PL0, che non è abilitato ad accedere a zone di memoria protette come il codice del sistema operativo. Gli altri modi operano a livello di privilegio PL1, che consente di accedere a tutte le risorse del sistema. I livelli di privilegio sono molto importanti per evitare che programmi con errori o maligni possano corrompere altri programmi o bloccare o infettare il calcolatore.

Tabella del vettore delle eccezioni

Quando si verifica un'eccezione il processore salta a un determinato spiazamento nella **tabella del vettore delle eccezioni**, in base alla causa dell'eccezione. La **Tabella 6.14** descrive la tabella del vettore, normalmente situata a partire dall'indirizzo 0x00000000 di memoria. Per esempio, se si verifica un *interrupt*, il processore salta all'indirizzo 0x00000018. Analogamente, all'accensione (quindi nella situazione di *reset*), il processore salta all'indirizzo 0x00000000. Ogni locazione del vettore delle eccezioni contiene tipicamente un'istruzione di salto al gestore della relativa eccezione, che al termine potrà saltare al sistema operativo o riprendere l'esecuzione del programma utente interrotto.

Banchi di registri

Prima che un'eccezione modifichi il PC è necessario salvare l'indirizzo di ritorno in LR, in modo che il gestore dell'eccezione sappia dove ritornare. Ma ciò non deve alterare il valore già presente in LR che può essere l'indirizzo di ritorno di una funzione chiamata dal programma in esecuzione. Per questo motivo il processore contiene un banco di differenti registri LR da usare nei diversi modi di esecuzione. Analogamente, il gestore dell'eccezione non deve alterare i bit del registro di stato, quindi serve un banco di registri di stato salvati (SPSR, *Saved Program Status Registers*) per salvare una copia del registro CPSR durante le eccezioni.

Se l'eccezione avviene mentre il programma sta modificando il proprio stack frame, ci si potrebbe trovare in una situazione incongruente (per es., i dati sono già stati scritti nello stack ma lo stack pointer non è ancora stato aggiornato). Quindi ogni modo di esecuzione usa il proprio stack e un ban-

Tabella 6.13 Modi di esecuzione di ARM.

Modo	CPSR _{4:0}
<i>User</i>	10000
<i>Supervisor</i>	10011
<i>Abort</i>	10111
Indefinito	11011
<i>Interrupt</i> (IRQ)	10010
<i>Fast Interrupt</i> (FIQ)	10001

Tabella 6.14 Tabella del vettore delle eccezioni.

Eccezione	Indirizzo	Modo
Reset	0x00	<i>Supervisor</i>
Istruzione indefinita	0x04	Indefinito
<i>Supervisor call</i>	0x08	<i>Supervisor</i>
<i>Prefetch Abort</i> (errore nel <i>fetch</i> di un'istruzione)	0x0C	<i>Abort</i>
<i>Data Abort</i> (errore di lettura o scrittura di un dato)	0x10	<i>Abort</i>
Riservata	0x14	Non Disponibile
<i>Interrupt</i>	0x18	IRQ
<i>Fast interrupt</i>	0x1C	FIQ

co di registri SP per puntare alla cima dello *stack* corrente. Questo implica che all'accensione si debbano riservare aree di memoria per gli *stack* di ogni modo di esecuzione e che il banco di SP debba essere adeguatamente inizializzato.

La prima cosa che un gestore di eccezione deve fare è salvare tutti i registri che intende utilizzare nello *stack*. Questo richiede tempo, ecco perché ARM dispone anche di un modo *fast interrupt* (*interrupt* veloce) FIQ che fa uso di un banco di registri R8-R12, in modo che il gestore possa iniziare subito le proprie attività senza salvare questi registri.

Gestione di un'eccezione

Ora che si sono definiti i modi di esecuzione, il vettore delle eccezioni e i banchi di registri, è possibile vedere cosa succede durante un'eccezione. Quando il processore rileva un'eccezione si comporta nel modo seguente:

1. Salva il registro CPSR nel banco SPSR.
2. Imposta il modo di esecuzione in base al tipo di eccezione.
3. Imposta i bit di mascheratura degli *interrupt* nel registro CPSR in modo che il gestore di eccezione non possa essere interrotto.
4. Salva l'indirizzo di ritorno nel banco LR.
5. Salta alla tabella del vettore delle eccezioni in base al tipo di eccezione.

Quindi il processore esegue l'istruzione presente nel vettore, tipicamente un salto al gestore dell'eccezione, che a sua volta salva altri registri nello *stack*, gestisce l'eccezione e ripristina i registri salvati prima di tornare con l'istruzione `MOVS PC, LR`, una versione particolare dell'istruzione `MOV` che effettua le seguenti operazioni aggiuntive:

1. Ripristina il valore originario del registro CPSR dal banco SPSR.

2. Copia nel PC il valore salvato nel banco LR (eventualmente modificato per particolari eccezioni).
3. Ripristina il modo di esecuzione e il livello di privilegio.

Istruzioni relative alle eccezioni

I programmi lavorano al livello di privilegio basso (PL0), mentre il sistema operativo lavora al livello di privilegio superiore (PL1). Per passare da un livello all'altro in modo controllato, il programma mette nei registri i parametri e poi effettua una *supervisor call* con l'istruzione `SVC` che genera un'eccezione e alza il livello di privilegio. Il sistema operativo esamina i parametri e svolge la funzione richiesta, quindi ritorna al programma chiamante.

Il sistema operativo e in generale il codice macchina eseguito a PL1 può accedere ai banchi di registri per i vari modi di esecuzione con le istruzioni `MRS` (*Move to Register from Special register*) e `MSR` (*Move to Special register from Register*). Per esempio, all'accensione, il sistema operativo usa queste istruzioni per inizializzare gli stack per i gestori delle eccezioni.

Accensione

All'accensione, il processore salta alla posizione del vettore associata al *reset* e inizia a eseguire in modo *supervisor* il cosiddetto *boot loader*: questa parte di codice tipicamente configura la mappa di memoria, inizializza lo stack pointer e carica da disco il sistema operativo vero e proprio, che a sua volta inizia un ben più lungo processo di preparazione della macchina (denominato *bootstrap*). Al termine, il sistema operativo può caricare in memoria un programma, passare al livello di privilegio inferiore dell'utente e saltare all'inizio del programma caricato.

6.7 ■ EVOLUZIONE DELL'ARCHITETTURA ARM

Il processore ARM1 è stato sviluppato in Inghilterra dalla Acorn Computer per i calcolatori BBC Micro nel 1985, come evoluzione del microprocessore 6502 utilizzato in molti personal computer dell'epoca. Nell'anno successivo è stato seguito da ARM2 entrato nella produzione del calcolatore Acorn Archimedes. ARM è l'acronimo di *Acorn RISC Machine*. Questo processore era basato sulla seconda versione del set di istruzioni di ARM (ARMv2); aveva un bus indirizzi di soli 26 bit, e i 6 bit più significativi del PC a 32 bit erano usati per memorizzare bit di stato. L'architettura presentava già praticamente tutte le istruzioni usate in questo capitolo: tutte quelle di elaborazione dati, la maggior parte di quelle di accesso alla memoria, i salti e le moltiplicazioni.

ARM ha rapidamente esteso il proprio bus a 32 bit, spostando i bit di stato nel registro dedicato CPSR (*Current Program Status Register*). ARMv4, introdotto nel 1993, ha aggiunto le istruzioni di lettura e scrittura di mezze parole (*halfword*) e le istruzioni di caricamento di valori *signed* e *unsigned* per *halfword* e *byte*. Questa versione costituisce quindi il nucleo della moderna architettura ARM, e a essa si è fatto riferimento nel capitolo.

Il set di istruzioni di ARM ha avuto poi molti miglioramenti, descritti nei paragrafi che seguono. Il processore di grande successo ARM7TDMI del 1995 ha introdotto il set di istruzioni a 16 bit denominato *Thumb* in ARMv4T, per aumentare la densità del codice macchina. ARMv5TE ha aggiunto capacità di elaborazione dei segnali (DSP, *Digital Signal Processing*) e istruzioni opzionali in virgola mobile. ARMv6 ha aggiunto istruzioni multimediali ed esteso il set di istruzioni *Thumb*. ARMv7 ha migliorato le istruzioni in virgola mobile e quelle multimediali, denominandole *Advanced SIMD*. ARMv8 ha introdotto

A partire da ARMv7, il registro CPSR prende il nome di APSR (*Application Program Status Register*).

un'architettura completamente nuova a 64 bit. Numerose altre istruzioni per la programmazione di sistema sono state introdotte man mano che l'architettura si è evoluta.

6.7.1 Set di istruzione *Thumb*

Le istruzioni *Thumb* sono lunghe 16 bit per aumentare la densità del codice macchina; sono identiche alle normali istruzioni ARM ma con alcune limitazioni in quanto:

- possono accedere solo ai primi otto registri;
- utilizzano un registro sia come sorgente sia come destinazione;
- gestiscono solo immediati più corti;
- non dispongono dell'esecuzione condizionata;
- modificano sempre le flag di stato.

Praticamente tutte le istruzioni ARM hanno un equivalente *Thumb*. Dal momento che queste ultime sono meno potenti, ne servono di più per codificare il medesimo programma, tuttavia dal momento che sono lunghe la metà consentono al codice finale *Thumb* di occupare in media il 65% dell'equivalente codice con istruzioni ARM. Le istruzioni *Thumb* sono utili non solo perché riducono le dimensioni e il costo della memoria, ma anche perché permettono di usare un più economico bus a 16 bit per accedere alla memoria e riducono la potenza consumata per il fetch delle istruzioni.

I processori ARM hanno un registro di stato del set di istruzioni, ISETSTATE, che contiene un bit T usato per indicare se il processore opera in modo normale (T = 0) oppure in modo *Thumb* (T = 1): questo modo determina come le istruzioni devono essere acquisite nella fase di fetch e interpretate. Le istruzioni di salto *BX* e *BLX* commutano il bit T per passare da un modo all'altro.

La codifica delle istruzioni *Thumb* è più complessa e irregolare di quella delle istruzioni normali, per comprimere la maggior quantità possibile di informazione nei 16 bit di una mezza parola (halfword) di memoria. La **Figura 6.33** mostra la codifica di istruzioni *Thumb* di uso frequente. I bit più significativi specificano il tipo di istruzione. Le istruzioni di elaborazione dati tipicamente indicano due registri, uno dei quali è sia sorgente sia destinazione, e modificano sempre le flag di stato. Le istruzioni di somma, sottrazione e traslazione possono specificare un immediato corto. I salti condizionati indicano un codice di condizione a 4 bit e uno spiazzamento corto, mentre i salti incondizionati permettono di specificare uno spiazzamento maggiore. Si noti che *BX* usa un identificatore di registro a 4 bit, quindi può accedere al registro LR. Sono inoltre state definite forme particolari di *LDR*, *STR*, *ADD* e *SUB* per operare relativamente a SP (quindi per accedere allo stack frame nelle chiamate di funzione). Un'altra forma speciale di *LDR* fa caricamenti relativi al PC (per accedere a serbatoi di literal). Ci sono forme di *ADD* e *MOV* che consentono di accedere a tutti i 16 registri, e *BL* richiede sempre due halfword per specificare una destinazione a 22 bit.

Le codifiche irregolari usate nel set di istruzioni *Thumb* e le istruzioni di lunghezza variabile (1 o 2 halfword) sono caratteristiche delle architetture a 16 bit, che devono comprimere tante informazioni in istruzioni corte. Questo naturalmente complica la fase di decodifica.

Successivamente ARM ha ridefinito il set di istruzioni *Thumb* aggiungendo un certo numero di istruzioni a 32 bit denominate *Thumb-2*, per aumentare le prestazioni nelle attività più frequenti e per consentire di scrivere in modo *Thumb* qualsiasi programma. Le istruzioni *Thumb-2* sono identificate dai 5 bit più significativi che possono valere 11101, 11110 oppure 11111. In questi casi, il processore fa il fetch di una seconda halfword contenente la parte restante dell'istruzione. La serie di processori Cortex-M lavora esclusivamente in modo *Thumb*.

15															0																												
0	1	0	0	0	0	funct					Rm					Rdn					<funct>S Rdn, Rdn, Rm (elaborazione dati)																						
0	0	0	ASR LSR			imm5					Rm					Rd					LSLS/LSRS/ASRS Rd, Rm, #imm5																						
0	0	0	1	1	1	SUB		imm3					Rm					Rd					ADDS/SUBS Rd, Rm, #imm3																				
0	0	1	1	SUB		Rdn			imm8												ADDS/SUBS Rdn, Rdn, #imm8																						
0	1	0	0	0	1	0	0	Rdn [3]		Rm					Rdn[2:0]					ADD Rdn, Rdn, Rm																							
1	0	1	1	0	0	0	0	SUB		imm7														ADD/SUB SP, SP, #imm7																			
0	0	1	0	1	Rn			imm8																	CMP Rn, #imm8																		
0	0	1	0	0	Rd			imm8																	MOV Rd, #imm8																		
0	1	0	0	0	1	1	0	Rdn [3]		Rm					Rdn[2:0]					MOV Rdn, Rm																							
0	1	0	0	0	1	1	1	L		Rm					0 0 0					BX/BLX Rm																							
1	1	0	1	cond			imm8																	B<cond> imm8																			
1	1	1	0	0	imm8																			B imm11																			
0	1	0	1	L B H		Rm					Rn					Rd					STR(B/H)/LDR(B/H) Rd, [Rn, Rm]																						
0	1	1	0	L		imm5					Rn					Rd					STR/LDR Rd, [Rn, #imm5]																						
1	0	0	1	L		Rd			imm8																	STR/LDR Rd, [SP, #imm8]																	
0	1	0	0	1	Rd			imm8																	LDR Rd, [PC, #imm8]																		
1	1	1	1	0	imm22[21:11]																			1 1 1 1 1					imm22[10:0]										BL imm22				

Figura 6.33 Esempi di codifica di istruzioni Thumb.

6.7.2 Istruzioni DSP

I processori di segnale (o processori DSP, *Digital Signal Processing*) sono progettati per eseguire in modo efficace algoritmi di elaborazione di segnali come la trasformata veloce di Fourier (FFT, *Fast Fourier Transform*) e i filtri a risposta finita/infinita all'impulso (FIR/IIR, *Finite/Infinite Impulse Response*). Tipiche applicazioni sono codifica e decodifica di audio e video, controllo di motori, riconoscimento del parlato. ARM fornisce una serie di istruzioni a questo scopo, che includono la moltiplicazione, la somma e la funzione di moltiplicazione con accumulo, detta MAC (*Multiply and ACcumulate*), che effettua una moltiplicazione e somma il risultato a un totalizzatore: $\text{totale} = \text{totale} + \text{src1} \times \text{src2}$. La funzione MAC è una tipica caratteristica che differenzia i set di istruzioni orientati al DSP da quelli normali; tale funzione è infatti molto comune negli algoritmi DSP e la presenza di istruzioni MAC raddoppia le prestazioni rispetto alla situazione in cui si devono eseguire separatamente moltiplicazioni e somme, ma richiede la presenza nel processore di registri aggiuntivi dedicati per mantenere i totalizzatori.

Le istruzioni DSP lavorano spesso su dati corti (a 16 bit, denominati *short*) che rappresentano i campioni letti da un sensore da parte di un convertitore analogico-digitale. Tuttavia i risultati intermedi vengono memorizzati a precisione maggiore (a 32 bit, denominati *long*, o a 64 bit, denominati *long long*) o saturati per evitare traboccamenti. In **aritmetica a saturazione** i risultati maggiori del massimo valore positivo rappresentabile sono forzati ad assumere tale valore massimo, così come i risultati minori del minimo valore rappresentabile sono forzati ad assumere tale valore minimo. Per esempio, in aritmetica a 32 bit, risultati maggiori di $2^{31} - 1$ vengono fatti saturare a $2^{31} - 1$, e risultati minori di -2^{31} a -2^{31} . I tipi di dati DSP più comuni sono riportati nella **Tabella 6.15**. Per i numeri in complemento a due si indica la presenza di un bit di segno anche se tale bit fa parte della codifica del numero. I tipi a 16, 32 e 64 bit sono anche conosciuti come numeri a mezza, singola e doppia precisione (*half*, *single* e *double precision*), da non confondere con gli stessi termini usati per i numeri in virgola mobile. Per motivi di efficienza, due numeri a mezza precisione vengono compattati (*packed*) in una sola parola a 32 bit.

La trasformata veloce di Fourier (FFT, *Fast Fourier Transform*), l'algoritmo DSP più frequente, è complessa e critica dal punto di vista delle prestazioni. Le istruzioni DSP dei processori mirano a eseguire efficacemente tale trasformata, soprattutto su dati frazionari a 16 bit.

Le istruzioni base di moltiplicazione elencate in Appendice B fanno parte dell'architettura ARMv4. La versione ARMv5TE ha aggiunto istruzioni di aritmetica a saturazione e moltiplicazioni su dati corti o frazionari per supportare gli algoritmi DSP.

Tabella 6.15 Tipi di dati DSP.

Tipo	Bit di segno	Bit interi	Bit frazionari
<i>short</i>	1	15	0
<i>unsigned short</i>	0	16	0
<i>long</i>	1	31	0
<i>unsigned long</i>	0	32	0
<i>long long</i>	1	63	0
<i>unsigned long long</i>	0	64	0
Q15	1	0	15
Q31	1	0	31

L'aritmetica a saturazione è un metodo importante per ridurre in modo controllato l'accuratezza negli algoritmi DSP. Di solito, l'aritmetica a singola precisione è sufficiente per gestire la maggior parte degli ingressi, ma in casi particolari si possono avere traboccamenti dagli intervalli rappresentabili in tale precisione. Un traboccamento causa un improvviso cambio di segno nel numero, che porta a un risultato completamente sbagliato, che a sua volta può provocare un rumore di scatto in un file audio o un pixel di colore strano in un'immagine. Lavorare in doppia precisione evita il traboccamento ma degrada le prestazioni e aumenta il consumo di potenza anche nei casi normali. L'aritmetica a saturazione riduce il traboccamento al massimo o minimo valore, solitamente molto vicino al risultato vero e proprio, introducendo quindi un'inaccuratezza molto limitata.

I tipi interi (*integer*) sono sia senza segno (*unsigned*) che con segno (*signed*), con il segno nel bit più significativo. I tipi frazionari (*fractional*) Q15 e Q31 sono numeri *signed* con tutte le cifre a destra della virgola: per esempio, Q31 varia nell'intervallo $[-1, 1-2^{-31}]$ con un passo di 2^{-31} tra coppie di numeri consecutivi. Questi tipi non sono presenti nel linguaggio C standard ma sono supportati da alcune librerie. Q31 può essere convertito in Q15 mediante troncamento o arrotondamento. Con il troncamento, il risultato Q15 è semplicemente la metà superiore; con l'arrotondamento, il valore 0x00008000 viene sommato a Q31, quindi si procede a troncamento. Quando un calcolo implica molti passi successivi, l'arrotondamento è molto utile perché evita di accumulare molteplici piccoli errori di troncamento che alla fine danno un errore significativo.

ARM ha aggiunto la flag *Q* ai registri di stato per indicare che nelle istruzioni DSP si è verificato un traboccamento o una saturazione. Nelle applicazioni in cui l'accuratezza è un aspetto critico, il programma può azzerare la flag *Q* prima di effettuare i calcoli, eseguire i calcoli in singola precisione e al termine verificare la flag *Q*. Se la flag è a 1, si è verificato traboccamento, quindi se necessario si può ripetere il calcolo usando precisione doppia.

Somme e sottrazioni sono eseguite in modo identico indipendentemente dal formato usato. La moltiplicazione invece dipende dal tipo: per esempio, con numeri a 16 bit, il valore 0xFFFF rappresenta 65 535 per numeri *short unsigned*, -1 per numeri corti e -2^{-15} per numeri Q15. Quindi 0xFFFF $\times$ 0xFFFF ha risultati completamente diversi nei vari casi (rispettivamente: 4 294 836 225, 1 e 2^{-30}). Servono quindi istruzioni diverse per la moltiplicazione con e senza segno.

Il numero *A* in formato Q15 può essere scritto come $a \times 2^{-15}$, dove *a* è l'interpretazione di *A* come numero *signed* nell'intervallo $[-2^{15}, 2^{15} - 1]$. Quindi il prodotto di due numeri Q15 è:

$$A \times B = a \times b \times 2^{-30} = 2 \times a \times b \times 2^{-31}$$

Questo significa che per moltiplicare due numeri Q15 e ottenere il risultato Q31 basta fare il normale prodotto con segno di numeri interi e raddoppiare il risultato finale. Tale risultato può poi essere troncato o arrotondato per essere di nuovo ricondotto al formato Q15.

Il ricco assortimento di istruzioni di moltiplicazione e moltiplicazione con accumulo è riportato nella **Tabella 6.16**. La funzione MAC richiede fino a quattro registri: *RdHi*, *RdLo*, *Rn* e *Rm*. Nelle operazioni in doppia precisione, *RdHi* e *RdLo* contengono rispettivamente i 32 bit più e meno significativi. Per esempio, UMLAL *RdHi*, *RdLo*, *Rn*, *Rm* calcola $\{RdHi, RdLo\} = \{RdHi, RdLo\} + Rn \times Rm$. Le moltiplicazioni in mezza precisione hanno diverse versioni indi-

Tabella 6.16 Istruzioni di moltiplicazione e di moltiplicazione con accumulo.

Istruzione	Funzione	Descrizione
La normale moltiplicazione a 32 bit opera sia con sia senza segno		
MUL	$32 = 32 \times 32$	Moltiplicazione
MLA	$32 = 32 + 32 \times 32$	Moltiplicazione con accumulo
MLS	$32 = 32 - 32 \times 32$	Moltiplicazione e sottrazione
unsigned long long = unsigned long × unsigned long		
UMULL	$64 = 32 \times 32$	Moltiplicazione di <i>unsigned long</i>
UMLAL	$64 = 64 + 32 \times 32$	Moltiplicazione con accumulo di <i>unsigned long</i>
UMAAL	$64 = 32 + 32 \times 32 + 32$	Moltiplicazione e somma con accumulo di <i>unsigned long</i>
long long = long × long		
SMULL	$64 = 32 \times 32$	Moltiplicazione di <i>signed long</i>
SMLAL	$64 = 64 + 32 \times 32$	Moltiplicazione con accumulo di <i>signed long</i>
Aritmetica su valori compattati (packed) = short × short		
SMUL (BB/BT/TB/TT)	$32 = 16 \times 16$	Moltiplicazione con segno (basso/alto)
SMLA (BB/BT/TB/TT)	$32 = 32 + 16 \times 16$	Moltiplicazione con segno (basso/alto) con accumulo
SMLAL (BB/BT/TB/TT)	$64 = 64 + 16 \times 16$	Moltiplicazione con accumulo di <i>signed long</i> (basso/alto)
Moltiplicazione di frazionari (Q31/Q15)		
SMULW (B/T)	$32 = (32 \times 16) \gg 16$	Moltiplicazione di parola/mezza parola <i>signed</i> (basso/alto)
SMLAW (B/T)	$32 = 32 + (32 \times 16) \gg 16$	Moltiplicazione e somma di parola/mezza parola <i>signed</i> (basso/alto)
SMMUL (R)	$32 = (32 \times 32) \gg 32$	Moltiplicazione di MSW <i>signed</i> (arrotondata)
SMMLA (R)	$32 = 32 + (32 \times 32) \gg 32$	Moltiplicazione di MSW <i>signed</i> (arrotondata) con accumulo
SMMLS (R)	$32 = 32 - (32 \times 32) \gg 32$	Moltiplicazione e sottrazione di <i>signed</i> (arrotondata)
long oppure long long = short × short + short × short		
SMUAD	$32 = 16 \times 16 + 16 \times 16$	Duplica moltiplicazione e somma di <i>signed</i>
SMUSD	$32 = 16 \times 16 - 16 \times 16$	Duplica moltiplicazione e sottrazione di <i>signed</i>
SMLAD	$32 = 32 + 16 \times 16 + 16 \times 16$	Duplica moltiplicazione con accumulo di <i>signed</i>
SMLS	$32 = 32 + 16 \times 16 - 16 \times 16$	Duplica moltiplicazione e sottrazione con accumulo di <i>signed</i>
SMLALD	$64 = 64 + 16 \times 16 + 16 \times 16$	Duplica moltiplicazione con accumulo di <i>signed long</i>
SMLS	$32 = 64 + 16 \times 16 - 16 \times 16$	Duplica moltiplicazione e sottrazione con accumulo di <i>signed long</i>

cate tra parentesi per scegliere gli operandi dalla metà superiore o inferiore di una parola, o in forma duale dove entrambe le metà vengono usate nella moltiplicazione. Le operazioni MAC su ingressi a mezza precisione con accumulatore a singola precisione (SMLA*, SMLAW*, SMUAD, SMUSD, SMLAD, SMLS) forzano a 1 la flag Q se si ha traboccamento dell'accumulatore. Anche le istruzioni di moltiplicazione della parola più significativa (MSW, *Most Significant Word*) hanno la versione con suffisso R che arrotonda invece di troncato il risultato.

Le istruzioni DSA includono anche una somma e una sottrazione con saturazione (QADD e QSUB) di parole a 32 bit (*word*) che saturano i risultati invece di generare traboccamenti. Esistono anche le versioni QDADD e QDSUB che moltiplicano per 2 il secondo operando prima di sommarlo/sottrarlo dal primo: la loro utilità sarà chiarita tra poco in relazione alle operazioni MAC su numeri frazionari. Queste istruzioni forzano a 1 la flag Q se si verifica saturazione.

Infine, le istruzioni DSP includono LDRD e STRD per caricare o memorizzare una coppia pari/dispari di registri in una doppia parola di memoria (*double word*) da 64 bit. Servono ad aumentare l'efficienza dei trasferimenti di dati in doppia precisione tra memoria e registri.

La **Tabella 6.17** riassume come usare le istruzioni DSP per moltiplicare o per applicare l'operazione MAC a vari tipi di dati. Gli esempi assumono che i dati da mezza parola (halfword) siano nella metà inferiore di un registro e che la metà superiore sia azzerata; se il dato è nella metà superiore si usa invece la variante T di SMUL. Il risultato viene memorizzato in R2, o in [R3, R2] per dati in doppia precisione. Operazioni sui numeri frazionari (Q15/Q31) raddoppiano il risultato utilizzando somme a saturazione per evitare traboccamenti quando si effettua la moltiplicazione -1×-1 .

6.7.3 Istruzioni in virgola mobile

La codifica in virgola mobile (*floating point*) è più flessibile di quella in virgola fissa utilizzata nelle applicazioni DSP e rende la programmazione più semplice. È usata soprattutto nella grafica, nelle applicazioni scientifiche e negli algoritmi di controllo. L'aritmetica in virgola mobile può essere eseguita con sequenze di normali istruzioni DSP, ma è più veloce e consuma meno potenza se si utilizzano istruzioni specifiche in virgola mobile e hardware dedicato.

Il set di istruzioni di ARMv5 include istruzioni opzionali in virgola mobile, che fanno uso di almeno 16 registri a 64 bit, quindi in doppia precisione, distinti dai registri ordinari. Tali registri possono anche essere visti come coppie di registri a 32 bit, quindi in singola precisione. I nomi di questi registri

Tabella 6.17 Istruzioni di moltiplicazione e di MAC (*Multiply And Accumulate*) per vari tipi di dati.

Primo Operando (R0)	Secondo Operando (R1)	Prodotto (R3/R2)	Moltiplicazione	MAC
short	short	short	SMULBB R2, R0, R1 LDR R3, =0x0000FFFF AND R2, R3, R2	SMLABB R2, R0, R1 LDR R3, =0x0000FFFF AND R2, R3, R2
short	short	long	SMULBB R2, R0, R1	SMLABB R2, R0, R1, R2
short	short	long long	MOV R2, #0 MOV R3, #0 SMLALBB R2, R3, R0, R1	SMLALBB R2, R3, R0, R1
long	short	long	SMULWB R2, R0, R1	SMLAWB R2, R0, R1, R2
long	long	long	MUL R2, R0, R1	MLA R2, R0, R1, R2
long	long	long long	SMULL R2, R3, R0, R1	SMLAL R2, R3, R0, R1
unsigned short	unsigned short	unsigned short	MUL R2, R0, R1 LDR R3, =0x0000FFFF AND R2, R3, R2	MLA R2, R0, R1, R2 LDR R3, =0x0000FFFF AND R2, R3, R2
unsigned short	unsigned short	unsigned long	MUL R2, R0, R1	MLA R2, R0, R1, R2
unsigned long	unsigned short	unsigned long	MUL R2, R0, R1	MLA R2, R0, R1, R2
unsigned long	unsigned long	unsigned long	MUL R2, R0, R1	MLA R2, R0, R1,
unsigned long	unsigned long	unsigned long long	UMULL R2, R3, R0, R1	UMLAL R2, R3, R0, R1
Q15	Q15	Q15	SMULBB R2, R0, R1 QADD R2, R2, R2 LSR R2, R2, #16	SMLABB R2, R0, R1, R2 SSAT R2, 16, R2
Q15	Q15	Q31	SMULBB R2, R0, R1 QADD R2, R2, R2	SMULBB R3, R0, R1 QDADD R2, R2, R3
Q31	Q15	Q31	SMULWB R2, R0, R1 QADD R2, R2, R2	SMULWB R3, R0, R1 QDADD R2, R2, R3
Q31	Q31	Q31	SMMUL R2, R0, R1 QADD R2, R2, R2	SMMUL R3, R0, R1 QDADD R2, R2, R3

sono D0-D15 se usati in doppia precisione, S0-S31 se usati in singola precisione. Per esempio, le istruzioni `VADD.F32 S2, S0, S1` e `VADD.F64 D2, D0, D1` effettuano la somma di due numeri rispettivamente in singola e in doppia precisione. Si noti che queste istruzioni – elencate nella **Tabella 6.18** – usano i suffissi `.F32` oppure `.F64` per indicare rispettivamente operazioni in virgola mobile in singola o in doppia precisione.

Le istruzioni `MRC` e `MCR` sono usate per trasferire dati tra i registri ordinari e i registri in virgola mobile del coprocessore.

ARM definisce anche un registro di stato e controllo della virgola mobile (FPSCR, *Floating-Point Status and Control Register*). Come il registro ordinario, questo registro contiene le flag `N`, `Z`, `C` e `V` relative alle operazioni in virgola mobile, e specifica le modalità di arrotondamento, le eccezioni, le situazioni anomale come traboccamenti e divisioni per zero. Le istruzioni `VMRS` e `VMSR` trasferiscono informazioni tra il registro ordinario e il registro FPSCR.

6.7.4 Istruzioni per il risparmio di potenza e per la sicurezza

I dispositivi alimentati a batteria risparmiano potenza passando la maggior parte del tempo in modalità *sleep* (dormiente). ARMv6K ha introdotto istruzioni per supportare questo risparmio di potenza: l'istruzione di attesa interrupt (`WFI`, *Wait For Interrupt*) consente al processore di portarsi in uno stato di basso consumo finché non si verifica un interrupt dall'esterno. Tale interrupt può essere dovuto a eventi dovuti all'utente (per es. un tocco sullo schermo) oppure generato periodicamente da un temporizzatore. L'istruzione `WFE` (*Wait For Event*) è simile ma molto utile in sistemi multiprocessore (vedi par. 7.7.8) perché consente a un processore di portarsi in modo *sleep*, fino a quando un altro processore lo risveglierà. Dopo aver eseguito tale istruzione, infatti, il processore può essere risvegliato sia da un interrupt sia da un altro processore con l'istruzione `SEV` (*Send an Event*).

ARMv7 migliora la gestione delle eccezioni per supportare la virtualizzazione e la sicurezza. Con la **virtualizzazione**, vari sistemi operativi possono essere eseguiti concorrentemente sullo stesso processore senza interazioni reciproche. Un supervisore (*hypervisor*) commuta l'esecuzione fra i vari sistemi operativi, operando a livello di privilegio PL2. Viene invocato da un'eccezione

Tabella 6.18 Istruzioni in virgola mobile di ARM.

Istruzione	Funzione
VABS Rd, Rm	$Rd = Rm $
VADD Rd, Rn, Rm	$Rd = Rn + Rm$
VCMP Rd, Rm	Confronta e imposta le flag di stato della virgola mobile
VCVT Rd, Rm	Converte da intero a virgola mobile
VDIV Rd, Rn, Rm	$Rd = Rn/Rm$
VMLA Rd, Rn, Rm	$Rd = Rd + Rn * Rm$
VMLS Rd, Rn, Rm	$Rd = Rd - Rn * Rm$
VMOV Rd, Rm or #const	$Rd = Rm$ o costante
VMUL Rd, Rn, Rm	$Rd = Rn * Rm$
VNEG Rd, Rm	$Rd = -Rm$
VNMLA Rd, Rn, Rm	$Rd = -(Rd + Rn * Rm)$
VNMLS Rd, Rn, Rm	$Rd = -(Rd - Rn * Rm)$
VNMUL Rd, Rn, Rm	$Rd = -Rn * Rm$
VSQRT Rd, Rm	$Rd = \sqrt{Rm}$
VSUB Rd, Rn, Rm	$Rd = Rn - Rm$

di tipo *hypervisor trap*. Con le estensioni relative alla **sicurezza**, il processore può definire uno stato sicuro con limitate modalità di attivazione e accesso protetto a porzioni sicure di memoria. Anche se un attaccante compromette il sistema operativo, il nucleo (*kernel*) sicuro può resistere alle manomissioni. Per esempio, un *kernel* sicuro può essere usato per disabilitare un telefono rubato o per rafforzare i diritti di gestione in modo che un utente non possa duplicare contenuti protetti da diritti d'autore.

6.7.5 Istruzioni SIMD

L'acronimo SIMD deriva da *Single Instruction Multiple Data*, e si riferisce a singole istruzioni che possono operare in parallelo su dati molteplici. Una tipica applicazione della modalità SIMD è l'esecuzione di semplici operazioni aritmetiche simultaneamente su molti dati, utile specialmente in elaborazioni grafiche. Tale aritmetica viene anche detta *packed arithmetic*.

I dati di tipo *short* sono spesso presenti in elaborazioni grafiche. Per esempio, un *pixel* in una fotografia digitale usa tipicamente 8 bit per memorizzare ciascuna delle tre componenti cromatiche rosso, verde e blu. Usare un'intera parola (*word*) da 32 bit per memorizzare ciascuna di queste componenti spreca i 24 bit più significativi. Inoltre, se le componenti di 16 *pixel* adiacenti dell'immagine sono riunite in una parola quadrupla (*quadword*) da 128 bit, l'elaborazione può essere 16 volte più rapida. Analogamente, le coordinate nello spazio grafico tridimensionale sono generalmente rappresentate da numeri in virgola mobile a 32 bit (in singola precisione). Quattro di tali coordinate possono essere riunite in una *quadword* da 128 bit.

Molte architetture moderne offrono operazioni aritmetiche di tipo SIMD che fanno uso di grossi registri SIMD nei quali vengono riuniti molteplici operandi di dimensioni inferiori. Per esempio, le istruzioni SIMD di ARMv7 condividono i registri con l'unità in virgola mobile. Tali registri possono anche essere accoppiati per agire come otto registri *quadword* da 128 bit, denominati Q0-Q7, che possono riunire valori interi o in virgola mobile a 8, 16, 32 o 64 bit. Queste istruzioni hanno i suffissi .I8, .I16, .I32, .I64, .F32 e .F64 per indicare come trattare i registri.

La **Figura 6.34** mostra l'istruzione di somma vettoriale VADD.I8 D2, D1, D0 che agisce su otto coppie di interi a 8 bit riuniti in doppie parole (*doubleword*) da 64 bit. Analogamente, VADD.I32 Q2, Q1, Q0 somma quattro coppie di interi a 32 bit riuniti in *quadword* da 128 bit, e VADD.F32 D2, D1, D0 somma due coppie di numeri in virgola mobile a singola precisione riuniti in doppie parole da 64 bit. Per svolgere operazioni in *packed arithmetic* è necessario modificare l'ALU per eliminare i riporti tra dati di dimensioni inferiori a quelle dei registri. Per esempio, un eventuale riporto finale di $a_0 + b_0$ non deve influire su $a_1 + b_1$.

Le istruzioni SIMD iniziano con V e comprendono le seguenti categorie:

- istruzioni aritmetiche di base, anche su numeri in virgola mobile;
- caricamenti e memorizzazioni di elementi multipli, anche interallacciati;
- operazioni logiche bit a bit;
- confronti;
- molte varianti di traslazioni, somme e sottrazioni con e senza saturazione;
- molte varianti di moltiplicazioni e di operazioni MAC;
- istruzioni varie.

ARMv6 definisce un insieme più limitato di istruzioni SIMD che operano sui registri ordinari a 32 bit. Tali istruzioni comprendono somme e sottrazioni

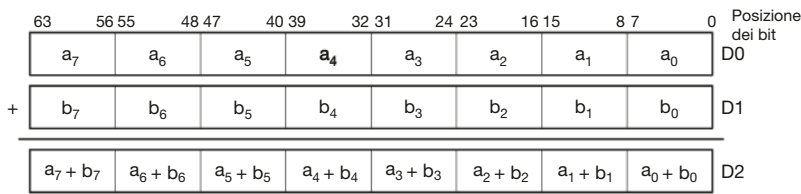


Figura 6.34
Packed arithmetic: otto somme simultanee a 8 bit.

a 8 e 16 bit, e istruzioni per riunire e ridistribuire in modo efficiente byte e mezze parole (*halfword*) in parole a 32 bit. Queste istruzioni sono molto utili per manipolare dati a 16 bit in programmi DSP.

6.7.6 Architettura a 64 bit

Le architetture a 32 bit consentono a un programma di accedere direttamente a $2^{32} = 4$ GB di memoria. I grossi calcolatori usati come server hanno spinto verso la transizione ad architetture a 64 bit, in grado di accedere a spazi di memoria molto più ampi. A seguire, anche personal computer e dispositivi mobili hanno adottato architetture a 64 bit, che possono essere anche più veloci perché elaborano più informazioni con una singola istruzione.

Molte architetture si limitano a estendere i registri di uso generale da 32 a 64 bit, ma ARMv8 ha anche introdotto un nuovo set di istruzioni per eliminare alcuni problemi. Il set di istruzioni classico di ARM difetta di un numero adeguato di registri di uso generale per programmi complessi, cosa che richiede onerosi trasferimenti di informazione tra registri e memoria. Inoltre tenere il registro PC in R15 e il registro SP in R13 complica la realizzazione del processore. Infine i programmi hanno spesso bisogno di un registro contenente il valore 0.

Le istruzioni di ARMv8 sono ancora a 32 bit e il set di istruzioni è molto simile a quello di ARMv7, ma con alcuni problemi eliminati. I registri sono estesi a 31 da 64 bit, denominati X0-X30, e PC e SP non fanno più parte di tali registri. X30 funge da registro di collegamento (*Link*) per i sottoprogrammi. Si noti che manca il registro X31: tale registro viene chiamato “registro zero” (ZR, *Zero Register*) ed è forzato a livello circuitale a contenere sempre il valore zero. Le istruzioni di elaborazione dati possono lavorare su dati a 32 o 64 bit, mentre le istruzioni di trasferimento operano sempre a 64 bit. Per fare spazio per bit aggiuntivi per specificare gli indirizzi degli operandi sorgente e destinazione, il campo *condition* è stato eliminato dalla maggior parte delle istruzioni, anche se naturalmente i salti possono essere ancora condizionati. ARMv8 semplifica anche la gestione delle eccezioni, raddoppia il numero di registri SIMD, aggiunge istruzioni per le crittografie AES e SHA. La codifica delle istruzioni risulta complessa e non facilmente classificabile in poche categorie.

Al reset, ARMv8 parte in modalità a 64 bit, e può passare in modalità a 32 bit attivando un bit in un registro di sistema e invocando un'eccezione. Ritorna in modalità a 64 bit quando la gestione dell'eccezione ritorna al programma interrotto.

6.8 ■ UN'ALTRA PROSPETTIVA: L'ARCHITETTURA x86

La maggior parte dei personal computer (PC) di oggi usa microprocessori con architettura x86, anche nota come IA-32: un'architettura a 32 bit originariamente sviluppata dalla ditta Intel, ma anche la ditta AMD vende microprocessori compatibili x86.

L'architettura x86 ha una lunga e travagliata storia, che ha origine nel 1978 quando Intel ha annunciato l'uscita del microprocessore a 16 bit 8086. La IBM ha scelto l'8086, e il suo “cugino” 8088, per i suoi primi personal computer. Nel

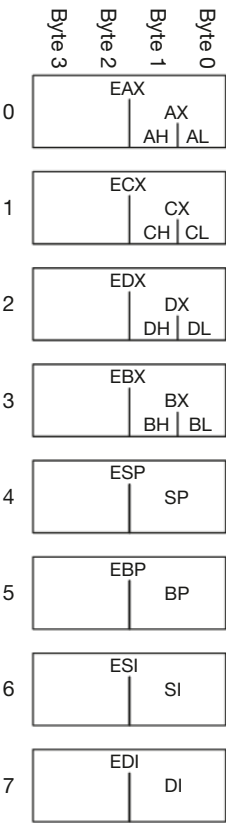


Figura 6.35
Registri x86.

1985, Intel ha introdotto il microprocessore a 32 bit 80386, retrocompatibile (*backward compatible*) con l’8086, cioè in grado di eseguire programmi sviluppati per i PC più vecchi. Le architetture compatibili con l’80386 sono chiamate processori x86: Pentium, Core e Athlon sono processori x86 ben noti.

Vari gruppi di progettisti alla Intel e alla AMD hanno inserito istruzioni e potenzialità aggiuntive nell’architettura originaria. Il risultato è molto meno elegante di quanto accaduto per ARM, ma la compatibilità software è molto più importante dell’eleganza tecnica, e x86 è rimasta per almeno due decenni lo standard *de facto* per i PC. Ogni anno vengono venduti più di 100 milioni di processori x86: un mercato che giustifica l’investimento annuo di più di 5 miliardi di dollari per il miglioramento continuo di tali processori.

x86 è un esempio di architettura CISC (*Complex Instruction Set Computer*, calcolatore con un insieme complesso di istruzioni): a differenza delle architetture RISC come ARM, ogni istruzione CISC può svolgere più operazioni. I programmi tradotti in linguaggio macchina per architetture CISC richiedono normalmente meno istruzioni degli stessi tradotti per architetture RISC. Le codifiche CISC sono state definite in modo da essere più compatte, per risparmiare memoria quando la RAM era molto più costosa di oggi: tali codifiche hanno lunghezza variabile e occupano spesso meno di 32 bit. La controindicazione è che le istruzioni complesse sono più difficili da decodificare e tendono a essere più lente da eseguire.

In questo paragrafo si introduce l’architettura x86. Non si vuole fare del lettore un programmatore nel linguaggio assembly x86, piuttosto illustrare alcune similitudini e differenze tra x86 e ARM. Sebbene gli autori ritengano interessante vedere come lavora x86, nessuno degli argomenti trattati in questo paragrafo è necessario per comprendere il resto del libro. Le principali differenze tra x86 e ARM sono elencate nella **Tabella 6.19**.

6.8.1 Registri x86

Il microprocessore 8086 aveva 8 registri a 16 bit, e poteva accedere separatamente alla metà inferiore e superiore di alcuni di questi registri. Quando è stato introdotto l’80386 a 32 bit, i registri sono stati estesi a 32 bit, e sono denominati EAX, ECX, EDX, EBX, ESP, EBP, ESI e EDI. Per retrocompatibilità, le metà a 16 bit meno significative e alcuni dei quarti a 8 bit meno significativi sono indirizzabili come registri autonomi, come illustrato dalla **Figura 6.35**.

Gli otto registri sono di uso quasi generale, nel senso che alcune istruzioni non possono fare uso di alcuni registri, mentre altre mettono i loro risultati sempre negli stessi registri. Come SP in ARM, ESP è normalmente utilizzato come stack pointer.

Il *program counter* di x86 è denominato EIP (*Extended Instruction Pointer*): come per ARM, avanza da un’istruzione alla successiva e può essere modificato dalle istruzioni di salto e di chiamata a sottoprogramma.

Tabella 6.19 Principali differenze tra ARM e x86.

Caratteristica	ARM	x86
n° di registri	15 di uso generale	8, con alcune restrizioni sull’uso
n° di operandi	3-4 (2-3 sorgenti, 1 destinazione)	2 (1 sorgente, 1 destinazione)
posizione degli operandi	registri o immediati	registri, immediati o celle di memoria
dimensione degli operandi	32 bit	8, 16 o 32 bit
flag di condizione	sì	sì
tipi di istruzioni	semplici	semplici e complesse
codifica delle istruzioni	fissa, 4 byte	variabile, 1-15 byte

6.8.2 Operandi x86

Le istruzioni ARM operano sempre su registri o immediati. Si devono usare esplicite istruzioni di caricamento e memorizzazione (*load* e *store*) per spostare dati tra i registri e la memoria. Al contrario, le istruzioni x86 possono operare su registri, su immediati o direttamente sulla memoria: questo compensa in parte lo scarso numero di registri.

Le istruzioni ARM specificano in genere tre operandi: due sorgenti e una destinazione. Le istruzioni x86 specificano solo due operandi: il primo è una sorgente, il secondo è sia una sorgente sia una destinazione. Quindi le istruzioni x86 sovrascrivono sempre una delle due sorgenti con il risultato. La **Tabella 6.20** elenca le combinazioni di posizioni degli operandi per x86: sono possibili tutte le combinazioni tranne memoria/memoria.

Come ARM, anche x86 ha uno spazio di memoria a 32 bit indirizzabile a byte. Tuttavia x86 supporta una maggior varietà di modi di indirizzamento della memoria: le locazioni di memoria sono specificate da una qualsiasi combinazione di **registro base**, **spiazzamento**, e **registro indice scalato**. La **Tabella 6.21** mostra queste combinazioni. Lo spiazzamento può essere un valore a 8, 16 o 32 bit. Il modo base+spiazzamento è analogo all'indirizzamento base di ARM per trasferimenti da e verso la memoria. Come ARM, anche x86 fornisce un indice scalato, che costituisce un metodo di facile accesso a vettori o strutture con elementi di 2, 4 o 8 byte senza richiedere sequenze di istruzioni per costruire gli indirizzi di memoria.

Mentre ARM opera sempre su parole (*word*) a 32 bit, x86 può operare su dati a 8, 16 o 32 bit: la **Tabella 6.22** mostra queste varianti.

Tabella 6.20 Posizione degli operandi.

Sorgente/Destinazione	Sorgente	Esempio	Significato
registro	registro	add EAX, EBX	EAX ← EAX + EBX
registro	immediato	add EAX, 42	add EAX, 42
registro	memoria	add EAX, [20]	EAX ← EAX + Mem[20]
memoria	registro	add [20], EAX	Mem[20] ← Mem[20] + EAX
memoria	immediato	add [20], 42	Mem[20] ← Mem[20] + 42

Tabella 6.21 Modi di indirizzamento a memoria.

Esempio	Significato	Commento
add EAX, [20]	EAX ← EAX + Mem[20]	spiazzamento
add EAX, [ESP]	EAX ← EAX + Mem[ESP]	indirizzamento base
add EAX, [EDX+40]	EAX ← EAX + Mem[EDX+40]	base + spiazzamento
add EAX, [60+EDI*4]	EAX ← EAX + Mem[60+EDI*4]	spiazzamento + indice scalato
add EAX, [EDX+80+EDI*2]	EAX ← EAX + Mem[EDX+80+EDI*2]	base + spiazzamento + indice scalato

Tabella 6.22 Istruzioni che operano su dati a 8, 16 o 32 bit.

Esempio	Significato	Dimensione dei dati
add AH, BL	AH ← AH + BL	8 bit
add AX, -1	AX ← AX + 0xFFFF	16 bit
add EAX, EDX	EAX ← EAX + EDX	32 bit

L'uso delle flag di condizione da parte di ARM lo differenzia da altre architetture RISC.

6.8.3 Flag di stato

x86, come molte altre architetture CISC, usa le flag di condizione (dette anche flag di stato) per decidere cosa fare nei salti condizionati e per tenere traccia di riporti e traboccamenti nelle operazioni aritmetiche. x86 usa un registro a 32 bit, denominato EFLAGS, per memorizzare le flag di stato. Alcuni bit del registro sono elencati nella **Tabella 6.23**, altri sono usati dal sistema operativo. Lo stato dell'architettura x86 comprende EFLAGS oltre agli otto registri e al registro EIP.

Tabella 6.23 Alcuni bit di EFLAGS.

Nome	Significato
CF (flag riporto, da <i>carry</i>)	Riporto in uscita generato dall'ultima operazione aritmetica. Indica traboccamento in caso di aritmetica senza segno (<i>unsigned</i>). Viene anche usata per propagare il riporto tra parole in aritmetica a multipla precisione
ZF (flag zero)	Il risultato dell'ultima operazione è zero
SF (flag segno)	Il risultato dell'ultima operazione è negativo (bit più significativo a 1)
OF (flag traboccamento, da <i>overflow</i>)	Si è generato traboccamento nell'aritmetica in complemento a due

6.8.4 Istruzioni x86

x86 ha un set di istruzioni più ampio di ARM. La **Tabella 6.24** descrive alcune delle istruzioni di uso generale. x86 ha anche istruzioni per aritmetica in virgola mobile e per aritmetica su molteplici dati corti riuniti in parole più lunghe. **D** indica la destinazione (registro o memoria) e **S** indica la sorgente (registro, memoria o immediato). Si noti che alcune istruzioni agiscono su registri specifici. Per esempio, la moltiplicazione a 32×32 bit prende sempre una delle sorgenti da EAX e mette sempre il risultato a 64 bit in EDX e EAX. L'istruzione **LOOP** mette sempre il contatore di ciclo in ECX, le istruzioni **PUSH**, **POP**, **CALL** e **RET** usano tutte lo stack pointer ESP. I salti condizionati verificano le flag ed eseguono il salto se la condizione indicata è verificata. Esistono diverse varianti: per esempio, **JZ** salta se la flag Zero (**ZF**) vale 1, mentre **JNZ** salta se la flag Zero (**ZF**) vale 0. Come per ARM, i salti normalmente seguono un'istruzione, come il confronto (**CMP**, **CoMPare**) che modifica i valori delle flag. La **Tabella 6.25** elenca alcune delle istruzioni di salto condizionato e come dipendono dalle flag impostate dalla precedente istruzione di confronto.

6.8.5 Codifica delle istruzioni x86

Le codifiche delle istruzioni x86 sono davvero disordinate, eredità di decenni di modifiche fatte un po' alla volta. A differenza di quanto succede per ARM, le cui istruzioni sono tutte da 32 bit, quelle di x86 variano da 1 a 15 byte, come mostrato nella **Figura 6.36**.¹ *Opcode* (il codice operativo) può essere di 1, 2 o 3 byte, ed è seguito da quattro campi opzionali: *ModR/M*, *SIB*, *Displacement* e *Immediate*. *ModR/M* specifica un modo di indirizzamento. *SIB* specifica i registri scala, indice e base in alcuni modi di indirizzamento.

¹ Sarebbe possibile costruire istruzioni da 17 byte se si usassero tutti i campi opzionali, ma x86 mette un limite a 15 byte per le istruzioni ammissibili.

Tabella 6.24 Alcune istruzioni x86.

Istruzione	Significato	Funzione
ADD/SUB	somma/sottrazione	$D = D + S$ / $D = D - S$
ADDC	somma con riporto	$D = D + S + CF$
INC/DEC	incremento/decremento	$D = D + 1$ / $D = D - 1$
CMP	confronto	Set flags based on $D - S$
NEG	negazione	$D = -D$
AND/OR/XOR	AND/OR/XOR logico	$D = D \text{ op } S$
NOT	NOT logico	$D = \bar{D}$
IMUL/MUL	moltiplicazione con segno/senza segno	$EDX:EAX = EAX \times D$
IDIV/DIV	divisione con segno/senza segno	$EDX:EAX/D$ $EAX = \text{Quoziente}; EDX = \text{Resto}$
SAR/SHR	traslazione a destra aritmetica/logica	$D = D \gg S$ / $D = D \gg S$
SAL/SHL	traslazione a sinistra	$D = D \ll S$
ROR/ROL	rotazione a destra/sinistra	Ruota D di S bit
RCR/RCL	rotazione a destra/sinistra con riporto	Ruota CF e D di S bit
BT	verifica di bit	$CF = D[S]$ (il bit di posto S di D)
BTR/BTS	verifica di bit e cancellazione/attivazione del bit	$CF = D[S]; D[S] = 0/1$
TEST	impostazione delle flag in base a maschera di bit	Imposta le flag sulla base di $D \text{ AND } S$
MOV	movimento	$D = S$
PUSH	inserimento (push) nello stack	$ESP = ESP - 4; \text{Memoria}[ESP] = S$
POP	estrazione (pop) dallo stack	$D = \text{Memoria}[ESP]; ESP = ESP + 4$
CLC, STC	cancella/attiva la flag di riporto	$CF = 0/1$
JMP	salto incondizionato	salto relativo: $EIP = EIP + S$ salto assoluto: $EIP = S$
Jcc	salto condizionato	se (flag) $EIP = EIP + S$
LOOP	ciclo	$ECX = ECX - 1$ se ($ECX \neq 0$) $EIP = EIP + \text{immediato}$
CALL	chiamata di sottoprogramma	$ESP = ESP - 4;$ $\text{Memoria}[ESP] = EIP; EIP = S$
RET	ritorno da sottoprogramma	$EIP = \text{Memoria}[ESP]; ESP = ESP + 4$

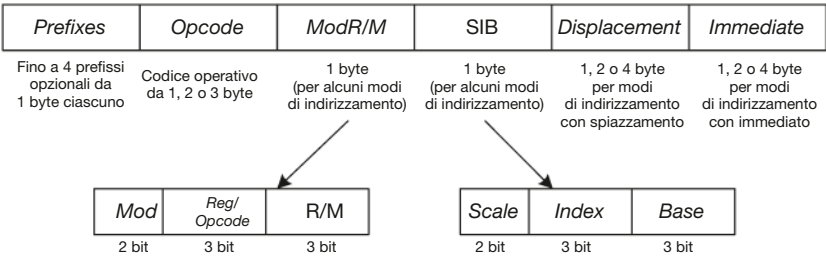
Displacement indica uno spiazzamento di 1, 2 o 4 byte in alcuni modi di indirizzamento. *Immediate* è una costante di 1, 2 o 4 byte per le istruzioni che usano un immediato come operando sorgente. Inoltre un'istruzione può essere preceduta da un massimo di quattro prefissi opzionali che ne modificano il comportamento.

Il byte *ModR/M* usa il campo di 2 bit *Mod* e il campo di 3 bit *R/M* per specificare il modo di indirizzamento per uno degli operandi. Tale operando può essere in uno degli otto registri o provenire da uno dei 24 modi di indirizzamento a memoria. A causa delle variazioni alle codifiche negli anni, i registri ESP e EBP non possono essere usati come registri base o indice in alcuni modi di indirizzamento. Il campo *Reg* specifica il registro usato come secondo operando. Per alcune istruzioni che non richiedono un secondo operando, il campo viene usato per specificare tre ulteriori bit di *Opcode*.

Tabella 6.25 Alcune condizioni di salto.

Istruzione	Significato	Funzione dopo istruzione <i>CMP D, S</i>
JZ/JE	salta se ZF = 1	salta se D = S
JNZ/JNE	salta se ZF = 0	salta se D ≠ S
JGE	salta se SF = OF	salta se D ≥ S
JG	salta se SF = OF e ZF = 0	salta se D > S
JLE	salta se SF ≠ OF o ZF = 1	salta se D ≤ S
JL	salta se SF ≠ OF	salta se D < S
JC/JB	salta se CF = 1	
JNC	salta se CF = 0	
JO	salta se OF = 1	
JNO	salta se OF = 0	
JS	salta se SF = 1	
JNS	salta se SF = 0	

Figura 6.36
Codifiche delle istruzioni x86.



Nei modi di indirizzamento che usano un registro indice scalato, il byte *SIB* specifica il registro indice e la scala (1, 2, 4 o 8). Se si usano sia un registro base sia un registro indice, il byte *SIB* specifica anche il registro indice.

ARM specifica completamente l'istruzione nei campi *cond*, *op* e *funct*. x86 usa invece un numero variabile di bit per specificare le diverse istruzioni, con numeri minori di bit per le istruzioni più frequenti, in modo da diminuire la lunghezza media delle istruzioni. Alcune istruzioni hanno addirittura più codici operativi. Per esempio, *add AL, imm8* esegue la somma a 8 bit di AL con un immediato. È codificata con un byte di codice operativo, 0x04, seguito da un byte di immediato. Il registro A (AL, AX o EAX) è detto “accumulatore”. D'altro canto, *add D, imm8* esegue la somma a 8 bit di un immediato con una destinazione arbitraria, *D* (registro o memoria). È codificata con un byte di codice operativo, 0x80, seguito da uno o più byte che specificano *D*, quindi da un byte che contiene l'immediato. Molte istruzioni hanno codifiche abbreviate se la destinazione è l'accumulatore.

Nell'8086 originario, il codice operativo specificava se l'istruzione agiva su operandi a 8 o 16 bit. Quando l'80386 ha introdotto operandi a 32 bit, non c'erano più codici operativi disponibili per indicare le operazioni a 32 bit, quindi alcuni codici operativi sono stati usati per operazioni sia a 16 sia a 32 bit. Un bit nel descrittore del segmento codice usato dal sistema operativo, specifica quale tipo di operazioni eseguire: tale bit viene forzato a 0 per retrocompatibilità con i programmi 8086, utilizzando operandi a 16 bit, mentre viene portato a 1 se si vogliono usare operandi a 32 bit. Inoltre il programmatore può specificare dei prefissi per modificare il tipo di operandi per una particolare istruzione: se il prefisso 0x66 compare prima del codice operativo, viene usato il formato di operandi alternativo rispetto a quello in uso (cioè 16 bit se si è in modalità 32 bit, e viceversa).

6.8.6 Altre particolarità di x86

L'80286 ha introdotto la **segmentazione** per dividere la memoria in segmenti lunghi fino a 64 KB. Quando il sistema operativo abilita la segmentazione, gli indirizzi di memoria vengono calcolati relativamente all'inizio del segmento. Il processore controlla se gli indirizzi superano la fine del segmento, e in caso affermativo genera errore per evitare che i programmi possano accedere a dati al di fuori del proprio segmento. La segmentazione si è dimostrata problematica da gestire da parte dei programmatori, e non è più utilizzata nelle versioni moderne del sistema operativo *Windows*.

x86 ha istruzioni stringa che operano su intere stringhe (sequenze) di byte o parole. Le operazioni includono trasferimenti, confronti, ricerca di uno specifico valore. Nei processori moderni, queste istruzioni sono generalmente più lente rispetto a eseguire la stessa operazione con una sequenza di istruzioni più semplici, quindi è meglio evitarle.

Come detto prima, il prefisso 0x66 è usato per scegliere fra operandi a 16 o 32 bit. Tra gli altri prefissi ci sono quelli per bloccare il bus (per controllare l'accesso a variabili condivise in sistemi multiprocessore), per predire se un salto verrà intrapreso oppure no, per ripetere l'istruzione durante il trasferimento di stringhe.

La rovina di qualsiasi architettura è oltrepassare la capacità di memoria. Con 32 bit di indirizzo, x86 può accedere a 4 GB di memoria, molto di più di quanto disponibile anche nei calcolatori più grandi nel 1985, ma uno spazio limitante a partire dai primi anni 2000. Nel 2003, AMD ha esteso lo spazio di indirizzamento e le dimensioni dei registri a 64 bit, dando il nome AMD64 a questa architettura potenziata. AMD64 può lavorare in modalità di compatibilità per eseguire i programmi a 32 bit senza modificarli, mentre il sistema operativo può utilizzare lo spazio di indirizzamento aumentato. Nel 2004 la Intel si è adeguata adottando le estensioni a 64 bit, rinominate EM64T (*Extended Memory 64 Technology*). Con 64 bit, i processori possono accedere a 16 exabyte (16 miliardi di GB) di memoria.

Chi avesse ulteriori curiosità relativamente all'architettura x86 può scaricare gratuitamente dal sito Web della Intel il manuale del programmatore: *x86 Intel Architecture Software Developer's Manual*.

6.8.7 Il quadro generale

Questo paragrafo ha fornito qualche dettaglio riguardo alle differenze tra l'architettura RISC di ARM e l'architettura CISC di x86. x86 tende ad avere programmi più corti perché un'istruzione complessa è equivalente a una sequenza delle più semplici istruzioni ARM e perché le istruzioni x86 sono state codificate per minimizzare l'uso di memoria. Però l'architettura x86 è un minestrone di caratteristiche accumulate negli anni, alcune oggi inutili ma da mantenere per retrocompatibilità con i vecchi programmi. Ha troppo pochi registri, e le istruzioni sono difficili da decodificare, e anche solo spiegare il set di istruzioni è difficile. Nonostante tutto ciò, x86 rimane saldamente l'architettura dominante per i PC, perché la compatibilità a livello software è fondamentale e perché il vastissimo mercato giustifica gli sforzi necessari per realizzare microprocessori x86 sempre più veloci.

Intel e Hewlett-Packard nel 1990 hanno sviluppato insieme una nuova architettura a 64 bit, denominata IA-64, progettata ex novo senza il retaggio della tormentata storia di x86, facendo tesoro di 20 anni di ricerca nel campo delle architetture di calcolatore, e pensata per offrire uno spazio di indirizzamento a 64 bit. Tuttavia IA-64 non è ancora riuscita a diventare popolare sul mercato: praticamente tutti i calcolatori che richiedono un ampio spazio di indirizzamento usano oggi le estensioni a 64 bit di x86.

ARM ha trovato un ottimo compromesso tra istruzioni semplici e densità del codice grazie a caratteristiche come le flag di condizione e gli operandi a registro traslati, che rendono il codice ARM più compatto di quello delle altre architetture RISC.

6.9 ■ RIASSUNTO

Per far lavorare un calcolatore, bisogna parlare il suo linguaggio. L'architettura del calcolatore definisce come far lavorare il processore. Oggi ci sono molte diverse architetture di calcolatori in uso in prodotti commerciali, ma una vol-

ta che se ne sia imparata una, diventa molto più semplice imparare anche le altre. Le domande chiave da porsi quando si affronta una nuova architettura sono:

- Qual è la lunghezza dei dati?
- Quali sono i registri?
- Come è organizzata la memoria?
- Quali sono le istruzioni?

ARM è un'architettura a 32 bit perché opera su dati a 32 bit. Ha 16 registri: 15 di uso generale più il *Program Counter*. In linea di principio, ogni registro di uso generale può essere usato in ogni programma, ma per convenzione alcuni registri sono riservati a usi particolari per facilitare la programmazione e la comunicazione fra funzioni scritte da programmatori differenti. Per esempio, R14 (cioè LR, *Link Register*) contiene l'indirizzo di ritorno dopo un'istruzione BL, e R0-R3 contengono i parametri di una funzione. ARM ha una memoria indirizzabile a byte con 32 bit di indirizzo. Le istruzioni sono lunghe 32 bit e allineate alle parole di memoria per avere accessi efficienti. Nel capitolo si sono discusse le istruzioni ARM usate più di frequente.

L'importanza di definire un'architettura di calcolatore è il fatto che un programma scritto per tale architettura può essere eseguito su molte diverse realizzazioni dell'architettura stessa. Per esempio, un programma scritto nel 1993 per il Pentium Intel può essere eseguito, e molto più velocemente, dai processori Intel Xeon o AMD Phenom nel 2015.

Nella prima parte del libro si è parlato di circuiti e livelli logici di astrazione. In questo capitolo si è saltati al livello dell'architettura. Nel prossimo capitolo si affronta la microarchitettura, cioè l'organizzazione di blocchi costruttivi digitali che realizzano l'architettura del processore. La microarchitettura è il collegamento tra l'ingegneria dell'hardware e quella del software, e secondo gli autori è anche uno degli argomenti più emozionanti perché insegna a costruire il proprio microprocessore.

Esercizi

Esercizio 6.1 Fare tre esempi con l'architettura ARM di ciascuno dei principi della progettazione architetturale: (1) la regolarità favorisce la semplicità; (2) rendere veloci le cose frequenti; (3) più piccolo è più veloce; (4) un buon progetto richiede buoni compromessi. Spiegare come ciascun esempio dimostra il principio.

Esercizio 6.2 L'architettura ARM ha 16 registri a 32 bit. È possibile progettare un'architettura senza registri? Se sì, descrivere brevemente l'architettura, compreso il set di istruzioni. Quali sono i vantaggi e gli svantaggi di questa architettura rispetto ad ARM?

Esercizio 6.3 Si consideri la scrittura in memoria di una parola (word) da 32 bit nella parola di memoria 42 di una memoria indirizzabile a byte.

- Qual è l'indirizzo di byte della parola 42?
- Su quali indirizzi di byte si estende la parola 42?
- Scrivere il numero 0xFF223344 memorizzato nella parola 42 per macchine big-endian e little-endian, mostrando

chiaramente l'indirizzo di memoria corrispondente a ciascuno dei byte della parola.

Esercizio 6.4 Ripetere l'Esercizio 6.3 per la scrittura in memoria di una parola da 32 bit nella parola di memoria 15 di una memoria indirizzabile a byte.

Esercizio 6.5 Spiegare come può essere usato il seguente programma ARM per determinare se il calcolatore opera in modalità big-endian e little-endian:

```
MOV    R0, #100
LDR    R1, =0xABCD876 ; R1 = 0xABCD876
STR    R1, [R0]
LDRB   R2, [R0, #1]
```

Esercizio 6.6 Tradurre in codice ASCII le seguenti stringhe di caratteri, usando la notazione esadecimale.

- SOS
- Bello
- mah!

Esercizio 6.7 Ripetere l'Esercizio 6.6 per le seguenti stringhe di caratteri.

- (a) salve
- (b) leoni
- (c) Al salvataggio!

Esercizio 6.8 Mostrare come le stringhe dell'Esercizio 6.6 sono memorizzate in una memoria indirizzabile a byte a partire dall'indirizzo di memoria 0x00001050 su una macchina little-endian, evidenziando l'indirizzo di memoria corrispondente a ciascuno dei byte della stringa.

Esercizio 6.9 Ripetere l'Esercizio 6.8 per le stringhe dell'Esercizio 6.7.

Esercizio 6.10 Tradurre il seguente codice *assembly* di ARM in codice macchina usando la notazione esadecimale.

```
MOV    R10, #63488
LSL    R9, R6, #7
STR    R4, [R11, R8]
ASR    R6, R7, R3
```

Esercizio 6.11 Ripetere l'Esercizio 6.10 per il seguente codice *assembly* di ARM.

```
ADD    R8, R0, R1
LDR    R11, [R3, #4]
SUB    R5, R7, #0x58
LSL    R3, R2, #14
```

Esercizio 6.12 Si considerino le istruzioni di elaborazione dati con un operando immediato Src2.

- (a) Quali delle istruzioni dell'Esercizio 6.10 hanno questo formato?
- (b) Scrivere i valori del campo immediato a 12 bit (*imm12*) delle istruzioni del punto (a), quindi convertire tali valori in immediati a 32 bit.

Esercizio 6.13 Ripetere l'Esercizio 6.12 per le istruzioni dell'Esercizio 6.11.

Esercizio 6.14 Convertire il seguente programma da linguaggio macchina in linguaggio *assembly* di ARM. I numeri a sinistra sono gli indirizzi di memoria, e i numeri a destra le istruzioni presenti a tali indirizzi. Quindi scrivere il programma in linguaggio di alto livello che sarebbe compilato in questo codice *assembly*, spiegando a parole cosa fa tale programma. R0 e R1 sono i valori di ingresso, e contengono all'inizio i numeri positivi a e b; alla fine del programma il risultato è in R0.

```
0x00008008    0xE3A02000
0x0000800C    0xE1A03001
0x00008010    0xE1510000
0x00008014    0x8A000002
0x00008018    0xE2822001
0x0000801C    0xE0811003
0x00008020    0xEAFFFA
0x00008024    0xE1A00002
```

Esercizio 6.15 Ripetere l'Esercizio 6.14 per il seguente codice macchina. R0 e R1 sono i valori di ingresso: R0 contiene un

numero a 32 bit e R1 contiene l'indirizzo di un array di 32 caratteri (char).

```
0x00008104    0xE3A0201F
0x00008108    0xE1A03230
0x0000810C    0xE2033001
0x00008110    0xE4C13001
0x00008114    0xE2522001
0x00008118    0x5AFFFA
0x0000811C    0xE1A0F00E
```

Esercizio 6.16 L'istruzione NOR non fa parte del set di istruzioni di ARM perché si può ottenere lo stesso risultato usando altre istruzioni. Scrivere un pezzetto di codice *assembly* che svolge l'operazione: R0 = R1 NOR R2, usando il minimo numero possibile di istruzioni.

Esercizio 6.17 L'istruzione NAND non fa parte del set di istruzioni di ARM perché si può ottenere lo stesso risultato usando altre istruzioni. Scrivere un pezzetto di codice *assembly* che svolge l'operazione: R0 = R1 NAND R2, usando il minimo numero possibile di istruzioni.

Esercizio 6.18 Si considerino i seguenti frammenti di codice di alto livello, assumendo che le variabili (con segno) g e h siano rispettivamente nei registri R0 e R1.

```
(i)
if (g >= h)
    g = g + h;
else
    g = g - h;
```

```
(ii)
if (g < h)
    h = h + 1;
else
    h = h * 2;
```

- (a) Tradurre in codice *assembly* di ARM i due frammenti assumendo che l'esecuzione condizionata sia possibile solo per le istruzioni di salto e usando il minimo numero possibile di istruzioni.
- (b) Tradurre in codice *assembly* di ARM i due frammenti assumendo che l'esecuzione condizionata sia possibile per tutte le istruzioni e usando il minimo numero possibile di istruzioni.
- (c) Confrontare la densità del codice (cioè il numero di istruzioni) tra (a) e (b) e discutere vantaggi e svantaggi delle due soluzioni.

Esercizio 6.19 Ripetere l'Esercizio 6.18 per i seguenti frammenti di codice.

```
(i)
if (g > h)
    g = g + 1;
else
    h = h - 1;
```

```
(ii)
if (g <= h)
    g = 0;
else
    h = 0;
```

Esercizio 6.20 Si consideri il seguente frammento di codice di alto livello, assumendo che gli indirizzi base di `array1` e `array2` siano contenuti rispettivamente in R1 e R2 e che `array2` sia stato inizializzato prima di essere usato.

```
int i;
int array1[100];
int array2[100];
...
for (i=0; i<100; i=i+1)
    array1[i] = array2[i];
```

- Tradurre in codice assembly di ARM il frammento senza fare uso di pre- o post-indicizzazione né di un registro scalato e usando il minimo numero possibile di istruzioni.
- Tradurre in codice assembly di ARM il frammento facendo uso se necessario di pre- o post-indicizzazione e di un registro scalato e usando il minimo numero possibile di istruzioni.
- Confrontare la densità del codice (cioè il numero di istruzioni) tra (a) e (b) e discutere vantaggi e svantaggi delle due soluzioni.

Esercizio 6.21 Ripetere l'Esercizio 6.20 per il seguente frammento di codice, assumendo che `temp` sia stato inizializzato prima di essere usato e che R3 contenga l'indirizzo base di `temp`.

```
int i;
int temp[100];
...
for (i=0; i<100; i=i+1)
    temp[i] = temp[i] * 128;
```

Esercizio 6.22 Si considerino i seguenti frammenti di codice di alto livello, dove R1 contiene `i` e R0 l'indirizzo base dell'array `valori`.

- ```
int i;
int valori[200];
for (i=0; i < 200; i=i+1)
 valori[i] = i;
```
- ```
int i;
int valori[200];
for (i=199; i >= 0; i = i-1)
    valori[i] = i;
```

- I due frammenti sono funzionalmente equivalenti?
- Tradurre in linguaggio assembly entrambi i frammenti, usando il minimo numero possibile di istruzioni.
- Discutere vantaggi e svantaggi di un costrutto rispetto all'altro.

Esercizio 6.23 Ripetere l'Esercizio 6.22 per i seguenti frammenti, assumendo che R1 contenga `i`, R0 l'indirizzo base dell'array `numeri`, e che `numeri` sia stato inizializzato prima di essere usato.

- ```
int i;
int numeri[10];
```

```
...
for (i=0; i < 10; i=i+1)
 numeri[i] = numeri[i]/2;
(ii)
int i;
int numeri[10];
...
for (i=9; i >= 0; i = i-1)
 numeri[i] = numeri[i]/2;
```

**Esercizio 6.24** Scrivere in un linguaggio di alto livello la funzione `int trova42(int array[], int dims)`. `dims` indica il numero di elementi di `array`, e `array` è l'indirizzo base di `array`. La funzione deve restituire l'indice del primo elemento di `array` che contiene il valore 42. Se nessun elemento contiene tale valore, deve restituire -1.

**Esercizio 6.25** La funzione di alto livello `strcpy` copia la stringa di caratteri `sorg` nella stringa di caratteri `dest`.

```
// codice C
void strcpy(char dest[], char sorg[]) {
 int i = 0;
 do {
 dest[i] = sorg[i];
 } while (sorg[i++]);
}
```

- Tradurre in linguaggio assembly di ARM la funzione `strcpy`, usando R4 per `i`.
- Tracciare la situazione dello stack prima, durante e dopo la chiamata della funzione `strcpy`.

Si assuma che sia `SP = 0xBEFFF000` subito prima della chiamata a `strcpy`.

Questa semplice funzione di ricopiatura stringhe ha un grosso limite: non ha alcun modo di capire se `dest` ha sufficiente spazio per contenere tutta `sorg`. Un utilizzo maligno della funzione con una stringa `sorg` molto lunga consente praticamente di scrivere in tutta la memoria, modificando anche codice presente in celle successive a `dest` con istruzioni capaci di prendere possesso del calcolatore. Quest'operazione viene definita "attacco per traboccamento di buffer" e viene utilizzata da numerosi virus, compreso il tristemente famoso verme (*worm*) Blaster, che ha causato nel 2003 danni stimati in 525 milioni di dollari.

**Esercizio 6.26** Tradurre la funzione di alto livello dell'Esercizio 6.24 in linguaggio assembly di ARM.

**Esercizio 6.27** Si consideri il seguente codice assembly di ARM, dove `funz1`, `funz2` e `funz3` sono funzioni non foglia (cioè sono funzioni che chiamano altre funzioni) mentre `funz4` lo è. Il codice di ogni funzione non è mostrato, ma i commenti indicano quali registri sono usati da ciascuna funzione.

```
0x00091000 funz1 ... ; funz1 usa R4-R10
0x00091020 BL funz2
...
0x00091100 funz2 ... ; funz2 usa R0-R5
0x0009117C BL funz3
...
0x00091400 funz3 ... ; funz3 usa R3, R7-R9
0x00091704 BL funz4
...
```

**Tabella 6.26** Sequenza di Fibonacci.

| $n$      | 1 | 2 | 3 | 4 | 5 | 6 | 7  | 8  | 9  | 10 | 11 | ... |
|----------|---|---|---|---|---|---|----|----|----|----|----|-----|
| $fib(n)$ | 1 | 1 | 2 | 3 | 5 | 8 | 13 | 21 | 34 | 55 | 89 | ... |

```
0x00093008 funz4 ... ; funz4 usa R11-R12
0x00093118 MOV PC, LR
```

- Quante parole di memoria devono essere disponibili negli stack frame di ciascuna funzione?
- Tracciare la situazione dello stack dopo la chiamata di `funz4`, indicando quali registri sono memorizzati dove, evidenziando ciascuno degli stack frame e dove possibile mostrando i valori in essi contenuti.

**Esercizio 6.28** Ogni numero della sequenza di Fibonacci è la somma dei due numeri precedenti. La Tabella 6.26 elenca i primi numeri della sequenza,  $fib(n)$ .

- Quanto vale  $fib(n)$  per  $n = 0$  e per  $n = -1$ ?
- Scrivere in un linguaggio di alto livello la funzione `fib` che restituisce il numero di Fibonacci per ogni valore non negativo di  $n$ . Si suggerisce di utilizzare un ciclo, e di commentare adeguatamente il proprio codice.
- Convertire la funzione scritta in (b) in linguaggio assembly di ARM, aggiungendo commenti dopo ogni linea di codice che ne spieghino chiaramente le operazioni svolte. Usare il simulatore MDK-ARM di Keil per collaudare il proprio codice sulla chiamata  $fib(9)$ . (Vedi la Prefazione per le istruzioni su come installare il simulatore MDK-ARM di Keil).

**Esercizio 6.29** Si consideri l'Esempio di Codice 6.27. Per questo esercizio, si faccia l'ipotesi che  $fattoriale(n)$  sia chiamato con parametro di ingresso  $n = 5$ .

- Che valore c'è in R0 quando `fattoriale` ritorna al chiamante?
- Si supponga di sostituire le istruzioni agli indirizzi 0x8500 e 0x8520 rispettivamente con `PUSH [R0, R1]` e `POP [R1, R2]`. Il programma:
  - entra in un ciclo infinito ma non si blocca;
  - si blocca perché provoca una crescita dello stack oltre le dimensioni del segmento dati dinamici oppure perché il PC salta a una locazione fuori dal programma;
  - restituisce un valore scorretto in R0 quando ritorna al chiamante (in questo caso, quale valore?), oppure
  - funziona correttamente nonostante le righe di codice cancellate?
- Ripetere il punto (b) con le seguenti modifiche alle istruzioni:
  - sostituire le istruzioni agli indirizzi 0x8500 e 0x8520 rispettivamente con `PUSH [R3, LR]` e `POP [R3, LR]`.
  - sostituire le istruzioni agli indirizzi 0x8500 e 0x8520 rispettivamente con `PUSH [LR]` e `POP [LR]`.
  - cancellare l'istruzione all'indirizzo 0x8510.

**Esercizio 6.30** Ben Imbrogliabit sta cercando di calcolare la funzione  $f(a, b) = 2a + 3b$  per valori non negativi di  $b$ . Esagera con le chiamate a funzione e la ricorsione e produce il seguente codice di alto livello per le funzioni `f` e `g`.

```
// codice di alto livello per le funzioni f e g
int f(int a, int b) {
 int j;
 j = a;
 return j + a + g(b);
}

int g(int x) {
 int k;
 k = 3;
 if (x == 0) return 0;
 else return k + g(x - 1);
}
```

Poi Ben traduce le due funzioni in linguaggio assembly come segue, e scrive anche la funzione `test` che chiama la funzione `f(5, 3)`.

```
; codice assembly di ARM
; f: R0 = a, R1 = b, R4 = j;
; g: R0 = x, R4 = k
0x00008000 test MOV R0, #5 ; a = 5
0x00008004 MOV R1, #3 ; b = 3
0x00008008 BL f ; chiama f(5, 3)
0x0000800C ciclo B ciclo ; e cicla per sempre
0x00008010 f PUSH {R1,R0,LR,R4} ; save registers on stack
0x00008014 MOV R4, R0 ; j = a
0x00008018 MOV R0, R1 ; mette b come parametro per g
```

```

0x0000801C BL g ; chiama g(b)
0x00008020 MOV R2, R0 ; mette valore da restituire in R2
0x00008024 POP {R1,R0} ; ripristina a e b dopo la chiamata
0x00008028 ADD R0, R2, R0 ; R0 = g(b) + a
0x0000802C ADD R0, R0, R4 ; R0 = (g(b) + a) + j
0x00008030 POP {R4,LR} ; ripristina R4, LR
0x00008034 MOV PC, LR ; ritorna
0x00008038 g PUSH {R4,LR} ; salva i registri nello stack
0x0000803C MOV R4, #3 ; k = 3
0x00008040 CMP R0, #0 ; x == 0?
0x00008044 BNE else ; salta se diversi
0x00008048 MOV R0, #0 ; se uguali restituisce valore 0
0x0000804C B fine ; e risistema i registri
0x00008050 else SUB R0, R0, #1 ; x = x - 1
0x00008054 BL g ; chiama g(x - 1)
0x00008058 ADD R0, R0, R4 ; R0 = g(x - 1) + k
0x0000805C fine POP {R4,LR} ; ripristina R0,R4,LR dallo stack
0x00008060 MOV PC, LR ; ritorna

```

Può essere utile fare una rappresentazione grafica dello stack simile a quella della Figura 6.14 per rispondere alle seguenti domande.

- Se il programma viene eseguito a partire da `test`, che valore contiene `R0` quando il programma va in ciclo? È stato correttamente calcolato il valore  $2a + 3b$ ?
- Se Ben sostituisce le istruzioni agli indirizzi `0x00008010` e `0x00008030` rispettivamente con `PUSH [R1, R0, R4]` e `POP [R4]`, il programma:
  - entra in un ciclo infinito ma non si blocca;
  - si blocca perché provoca una crescita dello stack oltre le dimensioni del segmento dati dinamici oppure perché il PC salta a una locazione fuori dal programma;
  - restituisce un valore scorretto in `R0` quando rimane nel ciclo (in questo caso, quale valore?), oppure
  - funziona correttamente nonostante le righe di codice cancellate?
- Ripetere il punto (b) con le seguenti modifiche alle istruzioni (si noti che le etichette non sono cambiate, cambiano solo le istruzioni):
  - sostituire le istruzioni agli indirizzi `0x00008010` e `0x00008024` rispettivamente con `PUSH [R1, LR, R4]` e `POP [R1]`.
  - sostituire le istruzioni agli indirizzi `0x00008010` e `0x00008024` rispettivamente con `PUSH [R0, LR, R4]` e `POP [R1]`.
  - sostituire le istruzioni agli indirizzi `0x00008010` e `0x00008024` rispettivamente con `PUSH [R0, LR, R4]` e `POP [R0]`.
  - cancellare le istruzioni agli indirizzi `0x00008010`, `0x00008024` e `0x00008030`.
  - sostituire le istruzioni agli indirizzi `0x00008038` e `0x0000805C` rispettivamente con `PUSH [R4]` e `POP [R4]`.
  - sostituire le istruzioni agli indirizzi `0x00008038` e `0x0000805C` rispettivamente con `PUSH [LR]` e `POP [LR]`.
  - cancellare le istruzioni agli indirizzi `0x00008038` e `0x0000805C`.

**Esercizio 6.31** Tradurre in codice macchina le seguenti istruzioni di salto. Gli indirizzi delle istruzioni sono indicati a sinistra di ciascuna.

```

(a)
0x0000A000 BEQ CICLO
0x0000A004 ...
0x0000A008 ...
0x0000A00C CICLO ...

(b)
0x00801000 BGE FINE
...
0x00802040 FINE ...

(c)
0x0000B10C DIETRO ...
...
0x0000D000 BHI DIETRO

(d)
0x00103000 BL FUNZ
...
0x0011147C FUNZ ...

(e)
0x00008004 L1 ...
...
0x0000F00C B L1

```

**Esercizio 6.32** Si consideri il seguente frammento di programma in linguaggio assembly ARM. I numeri a sinistra di ogni istruzione indicano i rispettivi indirizzi.

```

0x000A0028 FUNZ1 MOV R4, R1
0x000A002C ADD R5, R3, R5, LSR #2
0x000A0030 SUB R4, R0, R3, ROR R4
0x000A0034 BL FUNZ2
...
0x000A0038 FUNZ2 LDR R2, [R0, #4]
0x000A003C STR R2, [R1, -R2]
0x000A0040 CMP R3, #0
0x000A0044 BNE ELSE
0x000A0048 MOV PC, LR
0x000A004C ELSE SUB R3, R3, #1
0x000A0050 B FUNZ2

```

- Tradurre in codice macchina la sequenza di istruzioni, utilizzando la notazione esadecimale.

- (b) Indicare i modi di indirizzamento usati in ciascuna linea di codice.

**Esercizio 6.33** Si consideri il seguente frammento di programma in linguaggio C.

```
// codice C
void iniArray(int num) {
 int i;
 int array[10];

 for (i = 0; i < 10; i = i + 1)
 array[i] = confronta(num, i);
}

int confronta(int a, int b) {
 if (sottr(a, b) >= 0)
 return 1;
 else
 return 0;
}

int sottr(int a, int b) {
 return a - b;
}
```

- (a) Tradurre il codice C in linguaggio assembly di ARM. Usare R4 per la variabile *i*, e assicurarsi di utilizzare correttamente lo stack pointer. L'array è memorizzato nello stack della funzione *iniArray* (vedi la fine del par. 6.3.7).
- (b) Se la funzione *iniArray* è la prima a essere chiamata, tracciare lo stato dello *stack* prima della chiamata di *iniArray* e durante ogni chiamata di funzione. Indicare i nomi dei registri e delle variabili memorizzate nello stack, segnare la locazione di SP ed evidenziare ogni stack frame.
- (c) Come si comporterebbe il programma se ci si dimenticasse di salvare LR nello stack?

**Esercizio 6.34** Si consideri la seguente funzione di alto livello.

```
// codice C
int f(int n, int k) {
 int b;

 b = k + 2;
 if (n == 0) b = 10;
 else b = b + (n * n) + f(n - 1, k + 1);
 return b * k;
}
```

- (a) Tradurre il codice C in linguaggio assembly di ARM, facendo particolare attenzione a salvare e ripristinare i registri nelle chiamate a funzione e usando le convenzioni ARM sui registri preservati. Commentare adeguatamente il programma prodotto. Si può usare l'istruzione *MUL* di ARM. La funzione inizia con la prima istruzione all'indirizzo 0x00008100. Tenere in R4 la variabile locale *b*.
- (b) Eseguire passo passo su carta la funzione del punto (a) per il caso  $f(2, 4)$ . Tracciare uno schema dello stack simile a quello della Figura 6.14, assumendo che SP valga 0xBFF00100 quando viene chiamata *f*. Scrivere i nomi dei registri e i dati memorizzati in ogni posizione dello stack, tenendo traccia del valore di SP nei vari passi. Può essere utile anche tenere traccia dei valori presenti in R0, R1 e R4 durante l'esecuzione.

ne. Si faccia l'ipotesi che quando *f* viene chiamata sia  $R4 = 0xABCD$  e  $LR = 0x00008010$ . Qual è il valore finale di R0?

**Esercizio 6.35** Fare un esempio di caso pessimo per un salto in avanti, cioè a un'istruzione con indirizzo maggiore. Il caso pessimo si verifica quando il salto non può essere fatto perché la destinazione è troppo lontana. Mostrare le istruzioni e i loro indirizzi.

**Esercizio 6.36** Le seguenti domande esaminano i limiti dell'istruzione di salto *B*. Rispondere in termini di numero di istruzioni relative all'istruzione di salto.

- (a) Nel caso pessimo, quanto lontano può saltare in avanti (cioè a indirizzo maggiore) l'istruzione *B*? (Il caso pessimo si verifica quando il salto non può essere fatto perché la destinazione è troppo lontana.) Spiegare a parole e con esempi.
- (b) Nel caso ottimo, quanto lontano può saltare in avanti l'istruzione *B*? (Il caso ottimo si verifica quando la destinazione è la più lontana raggiungibile dal salto.) Spiegare.
- (c) Nel caso pessimo, quanto lontano può saltare all'indietro (cioè a un'istruzione con indirizzo minore) l'istruzione *B*? Spiegare.
- (d) Nel caso ottimo, quanto lontano può saltare all'indietro l'istruzione *B*? Spiegare.

**Esercizio 6.37** Spigare perché sia vantaggioso avere un campo immediato *imm24* di grandi dimensioni nella codifica in linguaggio macchina delle istruzioni *B* e *BL*.

**Esercizio 6.38** Scrivere il codice assembly che salta a un'istruzione lontana 32 "Mistruzioni" dall'istruzione di salto, dove 1 Mistruzione =  $2^{20}$  istruzioni = 1 048 576 istruzioni, nell'ipotesi che il codice inizi all'indirizzo 0x00008000. Usare il minimo numero possibile di istruzioni.

**Esercizio 6.39** Scrivere in un linguaggio di alto livello una funzione che prende un array di 10 elementi costituiti da numeri interi a 32 bit memorizzati in modalità *little-endian* e lo converte in modalità *big-endian*. Dopo aver scritto la funzione, tradurla in linguaggio assembly di ARM, commentando ogni istruzione e usando il minimo numero possibile di istruzioni.

**Esercizio 6.40** Si hanno due stringhe: *stringa1* e *stringa2*.

- (a) Scrivere in un linguaggio di alto livello una funzione denominata *concat* che concatena (cioè unisce) le due stringhe: `void concat(char stringa1[], char stringa2[], char stringaconcat[])`. La funzione non restituisce alcun valore: si limita a concatenare *stringa1* e *stringa2* mettendo il risultato in *stringaconcat*. Si assuma che *stringaconcat* sia abbastanza grande da contenere le due stringhe concatenate.
- (b) Tradurre la funzione del punto (a) in linguaggio assembly di ARM.

**Esercizio 6.41** Scrivere un programma in linguaggio assembly di ARM che somma due numeri positivi in virgola mobile a singola precisione contenuti nei registri R0 e R1 senza usare le istruzioni ARM per le operazioni in virgola mobile. Non è necessario preoccuparsi delle codifiche in vir-

gola mobile per situazioni particolari (come 0 o NaN) né di possibili traboccamenti. Usare il simulatore MDK-ARM di Keil per collaudare il proprio codice. (Vedi la Prefazione per le istruzioni su come installare il simulatore MDK-ARM di Keil.) Sarà necessario inizializzare a mano i due registri R0 e R1 per fare il suddetto collaudo. Dimostrare l'affidabilità del proprio programma.

**Esercizio 6.42** Si consideri il seguente programma ARM, assumendo che le istruzioni siano memorizzate a partire dall'indirizzo 0x8400 e che L1 corrisponda all'indirizzo 0x9024.

```
; codice assembly di ARM
MAIN PUSH {LR}
 LDR R2, =L1 ; questa è un'istruzione di
 ; lettura relativa al PC

 LDR R0, [R2]
 LDR R1, [R2, #4]
 BL DIFF
 POP {LR}
 MOV PC, LR
DIFF SUB R0, R0, R1
 MOV PC, LR
 ...

L1
```

- Per prima cosa indicare l'indirizzo di ciascuna istruzione.
- Descrivere la tabella dei simboli, cioè l'elenco degli indirizzi associati alle varie label.
- Tradurre in linguaggio macchina tutte le istruzioni.
- Quanto sono grandi il segmento dati e il segmento testo (in numero di byte)?
- Tracciare uno schema della memoria che mostri dove sono

memorizzati dati e istruzioni, simile a quello della Figura 6.31.

**Esercizio 6.43** Ripetere l'Esercizio 6.42 Per il seguente programma ARM, assumendo che le istruzioni siano memorizzate a partire dall'indirizzo 0x8534 e che L2 corrisponda all'indirizzo 0x9305.

```
; codice assembly di ARM
MAIN PUSH {R4, LR}
 MOV R4, #15
 LDR R3, =L2 ; questa è un'istruzione di
 ; lettura relativa al PC

 STR R4, [R3]
 MOV R1, #27
 STR R1, [R3, #4]
 LDR R0, [R3]
 BL MAGG
 POP {R4, LR}
 MOV PC, LR
MAGG CMP R0, R1
 MOV R0, #0
 MOVGT R0, #1
 MOV PC, LR
 ...

L2
```

**Esercizio 6.44** Citare due istruzioni ARM che possono aumentare la densità del codice (cioè diminuire il numero di istruzioni di un programma). Fare esempi di entrambe, mostrando il codice ARM equivalente senza e con l'uso di tali istruzioni.

**Esercizio 6.45** Illustrare vantaggi e svantaggi dell'esecuzione condizionata.

## Domande di valutazione

Queste domande sono state poste a candidati per un posto di lavoro nell'ambito della progettazione di sistemi digitali (possono essere utilizzate per qualsiasi linguaggio assembly).

**Domanda 6.1** Ci scriva il codice assembly di ARM che scambi tra loro i contenuti dei due registri R0 e R1 senza utilizzare altri registri.

**Domanda 6.2** Supponga di avere un array di numeri sia positivi sia negativi, e ci scriva il codice assembly di ARM che trova il sottoinsieme dell'array che ha il valore di somma più elevato. R0 contiene l'indirizzo base dell'array e R1 il numero di elementi. Il suo codice deve memorizzare il sottoinsieme trovato a partire dall'indirizzo base contenuto in R2. Scriva il programma che abbia un tempo di esecuzione il più breve possibile.

**Domanda 6.3** Supponga di avere un array che contenga una stringa di caratteri C che formano una frase. Ci progetti un algoritmo che inverta l'ordine delle parole della frase e me-

morizzi la frase rovesciata di nuovo nell'array. Poi ci codifichi l'algoritmo in linguaggio assembly di ARM.

**Domanda 6.4** Ci progetti un algoritmo che conti il numero di uni presenti in un numero di 32 bit. Poi ci codifichi l'algoritmo in linguaggio assembly di ARM.

**Domanda 6.5** Ci scriva il codice assembly di ARM che rovesci i bit contenuti nel registro R3 (cioè fa diventare bit più significativo il meno significativo, e così via). Usi il minimo numero possibile di istruzioni.

**Domanda 6.6** Ci scriva il codice assembly di ARM che controlli se si è verificato traboccamento nella somma di R2 e R3. Usi il minimo numero possibile di istruzioni.

**Domanda 6.7** Ci progetti un algoritmo che verifichi se una stringa è palindroma (cioè contiene una parola che può essere letta indifferentemente da sinistra verso destra o da destra verso sinistra, come per esempio "oro" o "anilina"). Poi ci codifichi l'algoritmo in linguaggio assembly di ARM.



# Microarchitettura

- 7.1 Introduzione
- 7.2 Analisi delle prestazioni
- 7.3 Processore a ciclo singolo
- 7.4 Processore multi ciclo
- 7.5 Processore pipeline
- 7.6 Rappresentazione HDL\*
- 7.7 Microarchitetture avanzate\*
- 7.8 Uno sguardo al mondo reale: evoluzione dell'architettura ARM\*
- 7.9 Riassunto

## 7.1 ■ INTRODUZIONE

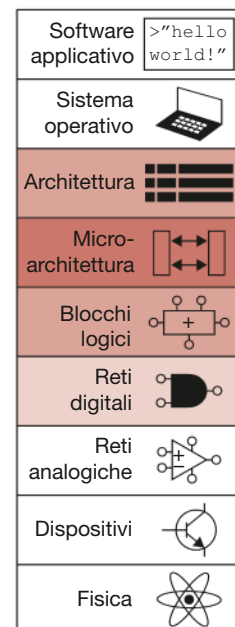
In questo capitolo si vuole illustrare come mettere insieme i pezzi necessari alla costruzione di un microprocessore. Si prenderanno in considerazione tre diverse versioni di microprocessori, ciascuna con diversi rapporti prestazioni/costo/complessità.

Per un inesperto, il progetto di un microprocessore può sembrare un'impresa sovrumana, ma in realtà la cosa è abbastanza semplice, e nei capitoli precedenti si è visto tutto ciò che serve per procedere: si è visto come progettare circuiti logici combinatori e sequenziali a partire dalle specifiche funzionali e di temporizzazione; si è presa familiarità con circuiti aritmetici e memorie; si è vista l'architettura ARM che definisce come il microprocessore appare al programmatore in termini di registri, istruzioni e memoria.

Questo capitolo affronta la **microarchitettura**, che rappresenta l'anello di congiunzione tra i circuiti logici e l'architettura: consiste infatti nella specifica combinazione di registri, ALU, macchine a stati finiti (FSM, *Finite State Machine*), memorie e altri blocchi logici necessari per la realizzazione dell'architettura. Una particolare architettura come ARM può avere molte diverse microarchitetture, caratterizzate da diversi rapporti prestazioni/costo/complessità. Tutte devono essere in grado di eseguire gli stessi programmi, ma la loro struttura interna può essere anche molto diversa. Nel capitolo si discutono tre diverse microarchitetture per illustrare tali rapporti prestazioni/costo/complessità.

### 7.1.1 Stato architetturale e set di istruzioni

L'architettura del calcolatore è definita da un set di istruzioni e da uno stato architetturale. Lo **stato architetturale** del processore ARM è definito dal contenuto di 16 registri a 32 bit e di un registro di stato, quindi ogni microarchitettura ARM deve poter memorizzare per intero questo stato. A partire dallo stato architetturale corrente, il processore esegue una particolare istruzione su un particolare insieme di dati per produrre un nuovo stato architetturale. Alcune microarchitetture possono contenere anche uno stato non architetturale



aggiuntivo, utile per semplificare le reti logiche o migliorare le prestazioni: si farà cenno a questo aspetto quando opportuno.

Per facilitare la comprensione della microarchitettura, si considera nel capitolo un sottoinsieme del set di istruzioni di ARM. Nello specifico, ci si limita alle seguenti istruzioni:

- le istruzioni di elaborazione dati ADD, SUB, AND e ORR (con modi di indirizzamento a registro e immediato ma senza traslazioni);
- le istruzioni di accesso alla memoria LDR e STR (con spiazzamento immediato positivo);
- l'istruzione di salto B.

La scelta di queste istruzioni è dovuta al fatto che sono sufficienti per scrivere molti programmi significativi. Una volta capito come realizzare tali istruzioni, si può estendere la struttura hardware per eseguire anche le altre.

### 7.1.2 Progettazione

È opportuno dividere le microarchitetture in due parti tra loro interagenti: il **percorso dati** (*datapath*) e l'**unità di controllo** (a volte denominata *control path*). Il percorso dati opera su parole di dati: è costituito da strutture come le memorie, i registri, l'ALU (*Arithmetic Logic Unit*) e i *multiplexer*. Si vuole realizzare l'architettura ARM a 32 bit, quindi si usa un percorso dati a 32 bit. L'unità di controllo riceve l'istruzione corrente dal percorso dati e comunica al percorso dati come eseguirla, attivando opportunamente gli ingressi di selezione dei multiplexer, le abilitazioni dei registri e i segnali di lettura e scrittura in memoria per controllare le operazioni del percorso dati.

Un buon modo per iniziare il progetto è quello di partire dall'hardware necessario per gli elementi di stato, che includono le memorie e lo stato architetturale (il program counter, i registri di lavoro, il registro di stato). Si aggiungono poi i blocchi di logica combinatoria tra gli elementi di stato per generare il nuovo stato a partire dallo stato corrente. L'istruzione da eseguire viene letta da una zona di memoria, poi le istruzioni di caricamento e scrittura leggono e scrivono dati in un'altra zona di memoria: è quindi comodo dividere la memoria in due parti, una contenente istruzioni e l'altra dati. La **Figura 7.1** mostra uno schema a blocchi con i cinque elementi di stato appena citati: il program counter, il banco di registri di lavoro (*register file*), il registro di stato, le memorie istruzioni e dati.

Anche se il program counter (PC) fa concettualmente parte del banco di registri, viene letto e scritto in ogni ciclo indipendentemente dalle normali operazioni sul banco di registri, e conviene realizzarlo come registro autonomo a 32 bit. La sua uscita, *PC*, punta all'istruzione corrente, mentre il suo ingresso, *PC'*, è l'indirizzo della prossima istruzione da eseguire.

La **memoria istruzioni** ha una sola porta di lettura.<sup>1</sup> Riceve in ingresso un indirizzo di istruzione a 32 bit, *A*, ed emette sull'uscita di lettura dato *RD* (*Read Data*) il dato a 32 bit (cioè l'istruzione) contenuto nella parola di indirizzo *A*.

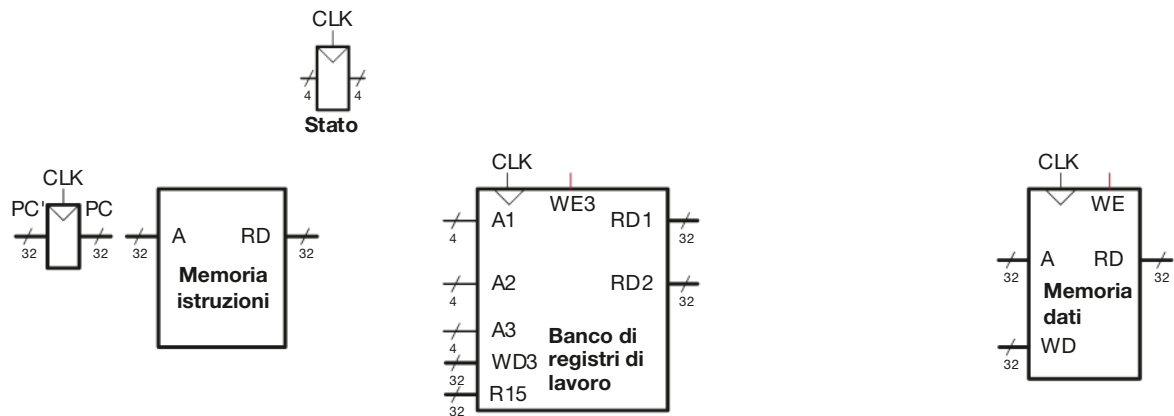
Il banco di registri di 15 elementi da 32 bit contiene i registri R0-R14, e ha un ingresso aggiuntivo per ricevere R15 dal PC. Il banco di registri ha due porte di lettura e una di scrittura. Le porte di lettura ricevono in ingresso due indirizzi a 4 bit, *A1* e *A2*, ciascuno dei quali specifica uno dei  $2^4 = 16$  registri

#### Reset del PC

Come minimo, il program counter (PC) deve avere un segnale di reset per inizializzare il proprio valore all'accensione del calcolatore. Il processore ARM normalmente inizializza il PC al valore  $0x00000000$  al reset, e da tale locazione cominciano i programmi da eseguire.

Trattare il PC come parte del banco dei registri di lavoro (il cosiddetto *register file*) complica il progetto del sistema, che si traduce in un numero maggiore di porte logiche e in un maggior consumo di potenza. La maggior parte delle altre architetture tratta il PC come un registro speciale, modificabile solo dalle istruzioni di salto e non dalle normali istruzioni di elaborazione dati. Come discusso nel paragrafo 6.7.6, anche la versione di ARM a 64 bit ARMv8 realizza il PC come registro speciale, separato dal banco di registri.

<sup>1</sup> Questa è una visione molto semplificata, che vede la memoria istruzioni come se fosse una ROM; nella maggior parte dei processori reali la memoria istruzioni deve essere scrivibile per consentire al sistema operativo di caricare un nuovo programma da eseguire. La microarchitettura multi ciclo descritta nel paragrafo 7.4 è più realistica, nel senso che usa una memoria unica per istruzioni e dati, che può essere sia letta sia scritta.



**Figura 7.1**  
Elementi di stato del processore ARM.

come operando sorgente, ed emettono sulle uscite di lettura dati *RD1* e *RD2* i valori a 32 bit dei registri indirizzati. La porta di scrittura riceve in ingresso un indirizzo a 4 bit, *A3*, un dato a 32 bit, *WD3*, un segnale di abilitazione alla scrittura, *WE3*, e il clock; se il segnale di abilitazione è attivo, il banco di registri memorizza il dato nel registro specificato in corrispondenza del fronte di salita del clock. Una lettura di *R15* restituisce il valore del *PC* + 8, e le scritture in *R15* devono essere gestite in modo particolare per aggiornare anche il *PC* che non fa parte del banco di registri.

La **memoria dati** ha una sola porta di lettura/scrittura. Se viene attivato il suo segnale di abilitazione alla scrittura, *WE*, il dato di ingresso *WD* viene scritto nella parola di indirizzo *A* in corrispondenza del fronte di salita del clock. Se il segnale di abilitazione alla scrittura vale 0, il contenuto della parola di indirizzo *A* viene emesso sull'uscita *RD*.

La memoria istruzioni, il banco di registri e la memoria dati sono tutti letti in modo combinatorio: in altre parole, se si modificano gli indirizzi, i nuovi dati vengono emessi alle uscite *RD* dopo il ritardo di propagazione, senza alcun segnale di clock. Al contrario, le scritture avvengono solo in corrispondenza dei fronti di salita del clock. In questo modo, lo stato del sistema si modifica solo in corrispondenza di tali fronti del clock. Quindi gli indirizzi, i dati e i segnali di abilitazione alla scrittura devono essere attivati prima del fronte del clock e rimanere stabili per un tempo minimo di mantenimento (*hold*) dopo il fronte.

Dal momento che gli elementi di stato modificano il loro stato solo in corrispondenza dei fronti di salita del clock, sono circuiti sequenziali sincroni. Il microprocessore è costituito da elementi di stato con clock e da reti combinatorie, quindi è a sua volta un circuito sequenziale sincrono. Si può quindi considerarlo come un'unica grande FSM o come un insieme di FSM più semplici che interagiscono tra loro.

### 7.1.3 Microarchitetture

In questo capitolo si sviluppano tre microarchitetture per l'architettura ARM: una a ciclo singolo, una multi ciclo e una *pipeline*. Esse differiscono per il modo in cui i vari elementi di stato sono connessi tra loro e per la quantità di stato non architetturale inserito.

La **microarchitettura a ciclo singolo** esegue un'intera istruzione in un ciclo. È la più facile da comprendere e ha un'unità di controllo piuttosto semplice. Dal momento che completa le operazioni in un ciclo non ha bisogno di alcuno stato non architetturale. Tuttavia, il tempo di ciclo è imposto dall'istruzione più lenta; inoltre il processore richiede memoria istruzioni e memoria dati separate, una situazione generalmente non realistica.

Esempi di classici processori multi ciclo sono il Whirlwind (letteralmente "mulinello") sviluppato al MIT nel 1947, il System/360 della IBM, il VAX della Digital Equipment Corporation, il 6502 usato nei calcolatori Apple II, e l'8088 dei primi PC IBM. Microarchitetture multi ciclo sono ancora utilizzate in microcontrollori a basso costo, come l'8051, il 68HC11 e la famiglia PIC16, utilizzati negli elettrodomestici, nei giocattoli e nei gadget elettronici.

I processori Intel hanno struttura pipeline sin dall'introduzione dell'80486 nel 1989. Praticamente tutti i processori RISC sono pure strutturati pipeline. I processori ARM hanno struttura pipeline sin dal capostipite ARM1 del 1985. Un processore ARM Cortex-M0 pipeline richiede soltanto circa 12 000 porte logiche per essere realizzato: in un circuito integrato moderno è così piccolo da richiedere l'uso del microscopio per essere visto, e la sua produzione costa molto meno di un centesimo di dollaro. Mettendo insieme anche memoria e periferiche, un chip Cortex-M0 commerciale come Freescale Kinetis costa meno di 50 centesimi. Quindi i processori pipeline stanno progressivamente sostituendo i loro lenti fratelli multi ciclo anche nelle applicazioni per le quali il costo è un fattore determinante.

La **microarchitettura multi ciclo** esegue le istruzioni in sequenze di cicli più brevi. Le istruzioni più semplici vengono eseguite in meno cicli di quelle più complesse; inoltre la microarchitettura multi ciclo riduce il costo hardware riutilizzando blocchi circuitali costosi come i sommatore e le memorie. Per esempio, il sommatore può essere usato in cicli differenti per scopi diversi mentre si porta avanti una singola istruzione. Il microprocessore multi ciclo ottiene questo risultato aggiungendo vari registri non architetturali per memorizzare risultati intermedi. Questo microprocessore esegue una sola istruzione alla volta, ma ogni istruzione richiede più cicli di clock per essere completata. Serve una sola memoria, cui si accede in un ciclo per fare il fetch dell'istruzione e in un altro ciclo per leggere o scrivere dati. Per questi motivi, i processori multi ciclo sono stati la scelta tipica nei sistemi a basso costo.

La **microarchitettura pipeline** applica il concetto di pipeline alla microarchitettura a ciclo singolo. Può quindi eseguire più istruzioni contemporaneamente, migliorando sensibilmente le prestazioni; serve però aggiungere logica per gestire le dipendenze tra le istruzioni simultaneamente in esecuzione, e sono necessari registri di pipeline non architetturali. I processori pipeline devono poter accedere a istruzioni e dati nello stesso ciclo: generalmente usano a questo scopo memorie *cache* separate per istruzioni e dati, come discusso nel Capitolo 8. L'aggiunta di logica e di registri risulta però molto utile, tanto che oggi tutti i processori commerciali ad alte prestazioni hanno struttura pipeline.

Nei paragrafi seguenti si discutono i diversi rapporti prestazioni/costo/complessità di queste tre microarchitetture. Alla fine del capitolo si menzionano brevemente ulteriori tecniche usate per ottenere prestazioni ancora superiori nei moderni microprocessori.

## 7.2 ■ ANALISI DELLE PRESTAZIONI

Come già detto, una particolare architettura di processore può avere diverse microarchitetture con diversi rapporti costo/prestazioni. Il costo dipende dalla quantità di hardware necessaria e dalla tecnologia di realizzazione. Una valutazione precisa richiede la conoscenza approfondita della tecnologia di realizzazione, ma in generale più porte logiche e più memoria significano costi maggiori.

In questo paragrafo si affrontano le basi dell'analisi delle prestazioni. Ci sono molti modi per misurare le prestazioni di un sistema di elaborazione, e gli uffici marketing sono tristemente famosi per scegliere il metodo di misura che fa apparire i loro calcolatori i più veloci, indipendentemente dal fatto che la misura abbia una qualche correlazione con le prestazioni nei casi reali. Per esempio, i costruttori di microprocessori spesso pubblicizzano i propri prodotti in termini di frequenza di clock e di numero di *core*, ma spesso sorvolano sul fatto che alcuni processori svolgono più lavoro di altri in un ciclo di clock, e che questo varia da programma a programma. Quindi come dovrebbe comportarsi l'acquirente?

L'unico modo privo di trucchi per misurare le prestazioni è quello di misurare il tempo di esecuzione di un programma di interesse per l'acquirente. Il calcolatore che esegue quel programma più rapidamente di tutti è quello con le prestazioni migliori. La scelta successiva è quella di misurare il tempo totale di esecuzione di una collezione di programmi simili a quelli che si pensa di dover eseguire; questa scelta è imposta se l'acquirente non ha ancora scritto il programma che intende usare, o se le misure sono fatte da qualcun altro che non ha il programma in questione. Queste collezioni di programmi sono chiamate *benchmark*, e i loro tempi di esecuzione sono resi disponibili per fornire qualche indicazione su come si comporta un processore.

Dhrystone, CoreMark e SPEC sono nomi di *benchmark* molto diffusi. I primi due sono benchmark sintetici, costituiti da importanti parti di codice comuni a molti programmi. Dhrystone è stato sviluppato nel 1984 ed è ancora molto utilizzato per processori *embedded* (cioè inseriti in dispositivi da controllare) anche se il suo codice non è molto rappresentativo dei programmi reali eseguiti da tali processori. CoreMark è un miglioramento di Dhrystone e comprende moltiplicazioni di matrici per sollecitare i circuiti moltiplicatori e sommatore, gestione di liste per sollecitare la memoria, macchine a stati per sollecitare la logica di gestione dei salti e controlli ciclici di ridondanza (CRC) che coinvolgono molte parti del processore. Entrambi i benchmark occupano meno di 16 KB e non sollecitano la memoria cache delle istruzioni. Il benchmark SPEC CINT2006 definito dalla Standard Performance Evaluation Corporation è invece composto da programmi reali, compresi h264ref (per la compressione video), sjeng (un programma di intelligenza artificiale per il gioco degli scacchi), hammer (un programma per l'analisi delle sequenze proteiche) e gcc (un compilatore del linguaggio C). Il benchmark è molto usato con processori ad alte prestazioni, perché sollecita in modo significativo l'intera CPU.

L'espressione 7.1 fornisce il tempo di esecuzione di un programma, misurato in secondi.

$$\text{tempo di esecuzione} = (\text{n° di istruzioni}) \left( \frac{\text{cicli}}{\text{istruzione}} \right) \left( \frac{\text{secondi}}{\text{ciclo}} \right) \quad (7.1)$$

Il numero di istruzioni in un programma dipende dall'architettura del processore. Alcune architetture hanno istruzioni complesse che svolgono più lavoro per istruzione, riducendo il numero di istruzioni in un programma. Tuttavia queste istruzioni complesse sono spesso più lente da eseguire in hardware. Il numero di istruzioni dipende anche moltissimo dall'abilità del programmatore. In questo capitolo si assume che si eseguano programmi noti su un processore ARM, quindi il numero di istruzioni di ciascun programma è costante, indipendentemente dalla microarchitettura. Il numero di cicli per istruzione (**CPI**, *Cycles Per Instruction*) è il numero di cicli di clock richiesti in media per eseguire un'istruzione. È il reciproco della potenza di elaborazione (**IPC**, *Instructions Per Cycle*). Diverse microarchitetture hanno diversi CPI. Nel capitolo si ipotizza di avere una memoria ideale che non influisce sul CPI. Nel Capitolo 8 si esamina il perché in alcuni casi il processore deve attendere la memoria, con conseguente incremento del CPI.

Il numero di secondi per ciclo è il periodo del clock,  $T_c$ . Tale periodo è determinato dal percorso critico attraverso i circuiti del processore: diverse microarchitetture hanno quindi diversi periodi di clock. Anche il progetto delle reti logiche e dei circuiti influenza significativamente il periodo di clock: per esempio, un sommatore ad anticipazione di riporto (*carry lookahead*) è più veloce di un sommatore a propagazione a onda di riporto. I miglioramenti nella tecnologia di produzione hanno storicamente raddoppiato la velocità dei transistori ogni 4-6 anni, quindi un microprocessore di oggi è più veloce di uno di dieci anni fa, anche se le reti logiche e la microarchitettura non sono cambiate.

La sfida del microarchitetto è saper scegliere il progetto che minimizza il tempo di esecuzione pur soddisfacendo i vincoli relativi ai costi e/o ai consumi di potenza. Dal momento che le decisioni a livello microarchitetturale influenzano sia il CPI sia  $T_c$ , e sono a loro volta influenzate dal progetto delle reti logiche e dei circuiti, la scelta migliore richiede un grande sforzo di analisi.

Molti altri fattori influenzano le prestazioni generali di un calcolatore. Per esempio, il disco rigido, la memoria, il sistema grafico e le connessioni di rete possono divenire i fattori limitanti al punto da rendere irrilevanti le prestazioni del processore: il più veloce processore del mondo non aiuta a navigare in Internet se si sta usando una connessione telefonica. Ma questi altri fattori limitanti vanno oltre gli scopi di questo testo.

## 7.3 ■ PROCESSORE A CICLO SINGOLO

Si parte con il progetto di una microarchitettura che esegue le istruzioni in un singolo ciclo. Il primo passo è costruire il percorso dati interconnettendo gli elementi di stato della **Figura 7.1** con logica combinatoria in grado di eseguire le varie istruzioni. I segnali di controllo determinano quale specifica istruzione viene eseguita dal percorso dati a ogni istante. L'unità di controllo contiene logica combinatoria che genera i segnali di controllo appropriati in base all'istruzione corrente. Da ultimo si analizzano le prestazioni del processore a ciclo singolo.

### 7.3.1 Percorso dati a ciclo singolo

In questo paragrafo si costruisce gradualmente il percorso dati a ciclo singolo, aggiungendo un pezzo alla volta agli elementi di stato della Figura 7.1. Le

nuove connessioni sono evidenziate in nero (o in rosso se si tratta di segnali di controllo) mentre le componenti hardware già discusse vengono rappresentate in grigio. Il registro di stato fa parte dell'unità di controllo e viene omissso quando ci si concentra sul percorso dati.

Il program counter contiene l'indirizzo dell'istruzione da eseguire. Per prima cosa si deve dunque leggere l'istruzione dalla memoria istruzioni. La **Figura 7.2** mostra che il PC è semplicemente collegato all'ingresso di indirizzo della memoria istruzioni, che emette l'istruzione a 32 bit, denominata *Instr*, in modo che possa essere prelevata dal processore (questa fase è denominata **fase di fetch**).

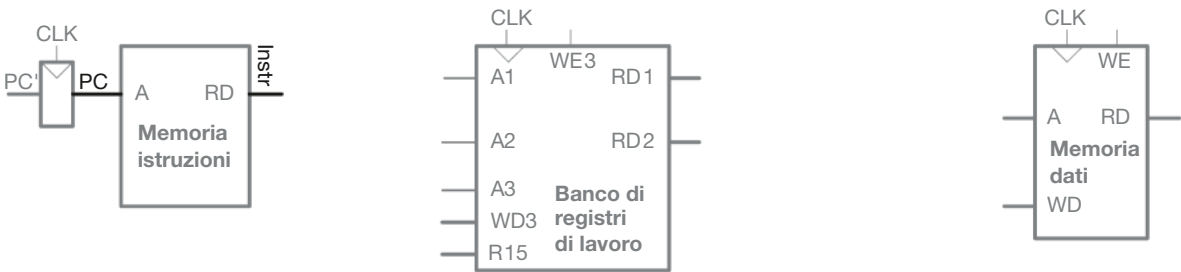
Le successive attività del processore dipendono dall'istruzione prelevata durante la fase di fetch. Si parte considerando le connessioni del percorso dati necessarie per eseguire l'istruzione `LDR` con spiazzamento immediato positivo, poi si vede come generalizzare il percorso dati per gestire anche le altre istruzioni.

**LDR**

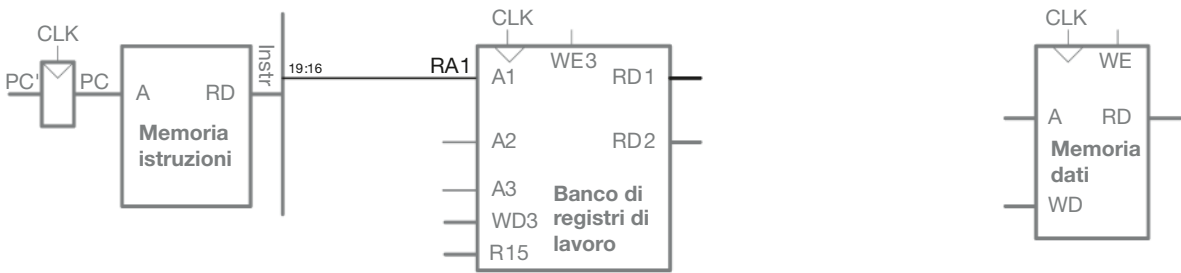
Per eseguire l'istruzione `LDR`, il prossimo passo è leggere il registro sorgente contenente l'indirizzo base. Tale registro è specificato dal campo *Rn* dell'istruzione, cioè i bit *Instr*<sub>19:16</sub>. Questi bit dell'istruzione vengono collegati agli ingressi di indirizzo di una delle porte del banco di registri di lavoro (*register file*), la porta *A1*, come mostrato nella **Figura 7.3**. Il banco di registri emette il valore del registro all'uscita *RD1*.

L'istruzione `LDR` richiede anche uno spiazzamento, memorizzato nel campo immediato dell'istruzione, *Instr*<sub>11:0</sub>. È un valore senza segno, quindi deve essere esteso con zeri fino a 32 bit, come mostrato nella **Figura 7.4**. Il valore esteso a 32 bit è denominato *ExtImm*. L'estensione con zeri richiede semplicemente di riempire di zeri i bit più significativi: quindi *ImmExt*<sub>31:12</sub> = 0 e *ImmExt*<sub>11:0</sub> = *Instr*<sub>11:0</sub>.

Il processore deve sommare lo spiazzamento al registro base per ottenere l'indirizzo di memoria da cui leggere il dato. La **Figura 7.5** introduce



**Figura 7.2** Fetch dell'istruzione da memoria.



**Figura 7.3** Lettura dell'operando sorgente dal banco di registri.



un'ALU per eseguire tale somma. L'ALU riceve due operandi: *SrcA* e *SrcB*. *SrcA* proviene dal banco di registri, *SrcB* dall'immediato esteso a 32 bit. Come descritto nel paragrafo 5.2.4, l'ALU può compiere varie operazioni: il segnale di controllo *ALUControl* a 2 bit serve a specificare l'operazione da eseguire. L'ALU genera il risultato a 32 bit *ALUResult*. Per l'istruzione *LDR*, *ALUControl* deve valere 00 per indicare la somma, e *ALUResult* viene inviato alla memoria dati come indirizzo della parola da leggere, come mostrato nella Figura 7.5.

Il dato viene emesso dalla memoria dati sul bus *ReadData* e scritto nel registro destinazione alla fine del ciclo, come mostrato nella Figura 7.6. La porta 3 del banco di registri è la porta di scrittura. Il registro destinazione per l'istruzione *LDR* è specificato del campo *Rd*, cioè i bit *Instr*<sub>15:12</sub>, collegati agli ingressi di indirizzo *A3* del banco di registri. Il bus *ReadData* è collegato agli ingressi di dato della porta di scrittura, *WD3*, del banco di registri. Un segnale di controllo denominato *RegWrite* è collegato all'abilitazione alla scrittura della porta 3, *WE3*, e viene attivato durante l'istruzione *LDR* per scrivere il dato letto da memoria del banco di registri. La scrittura avviene in corrispondenza del fronte di salita del clock alla fine del ciclo.

Mentre l'istruzione viene eseguita, il processore deve calcolare l'indirizzo dell'istruzione successiva, *PC'*. Dal momento che le istruzioni sono lunghe 32 bit (4 byte), l'istruzione successiva si trova a *PC + 4*. La Figura 7.7 usa un sommatore per incrementare di 4 il PC. Il nuovo indirizzo viene scritto nel PC in corrispondenza del prossimo fronte di salita del clock. Questo completa il percorso dati per l'istruzione *LDR*, tranne nel caso particolare in cui il registro base o il registro destinazione sia R15.

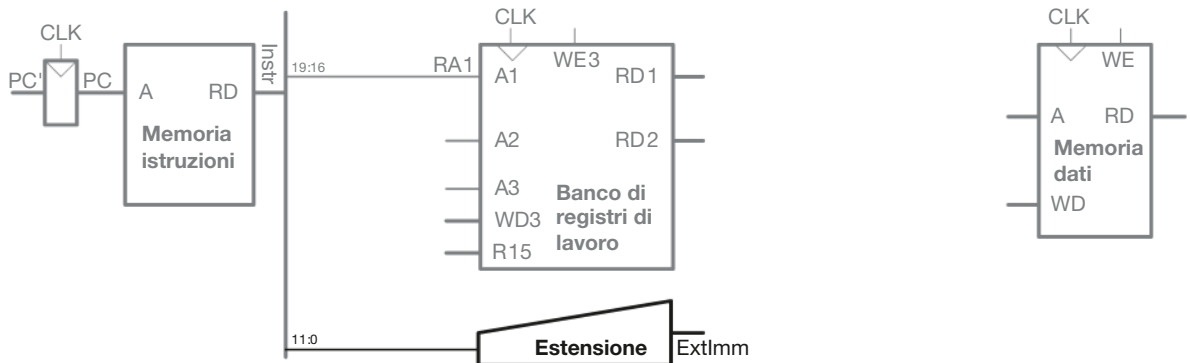


Figura 7.4 Estensione dell'immediato con zeri.

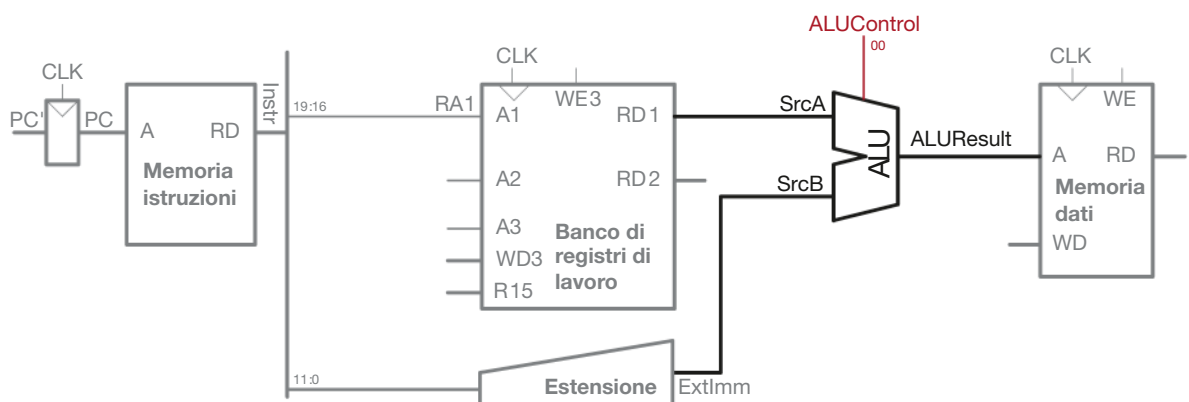
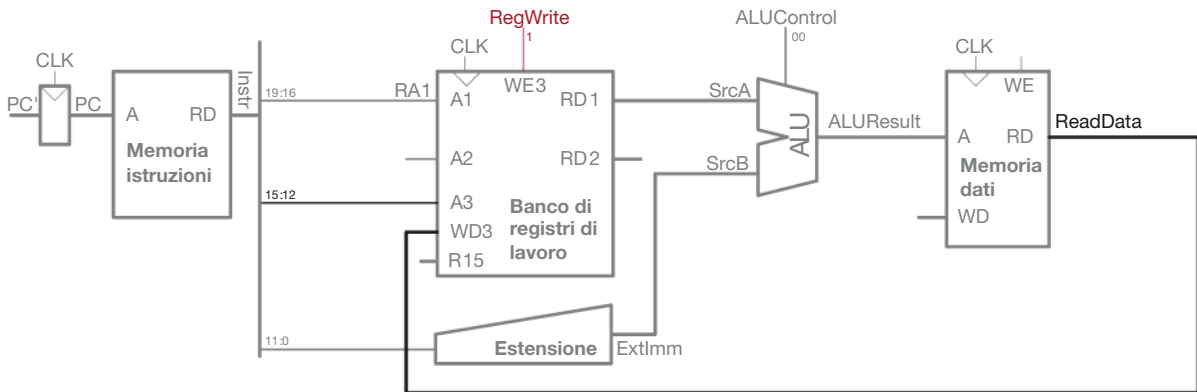
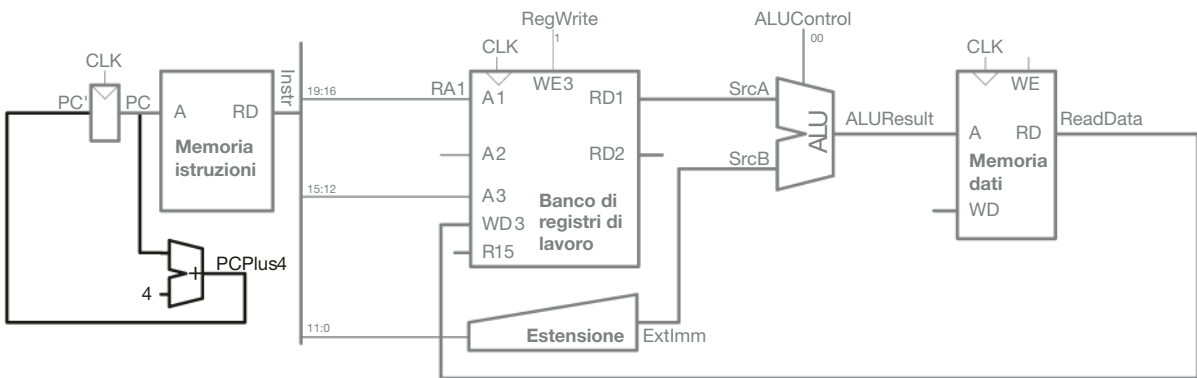


Figura 7.5 Calcolo dell'indirizzo di memoria.



**Figura 7.6** Scrittura del risultato (*writeback*) nel banco di registri.



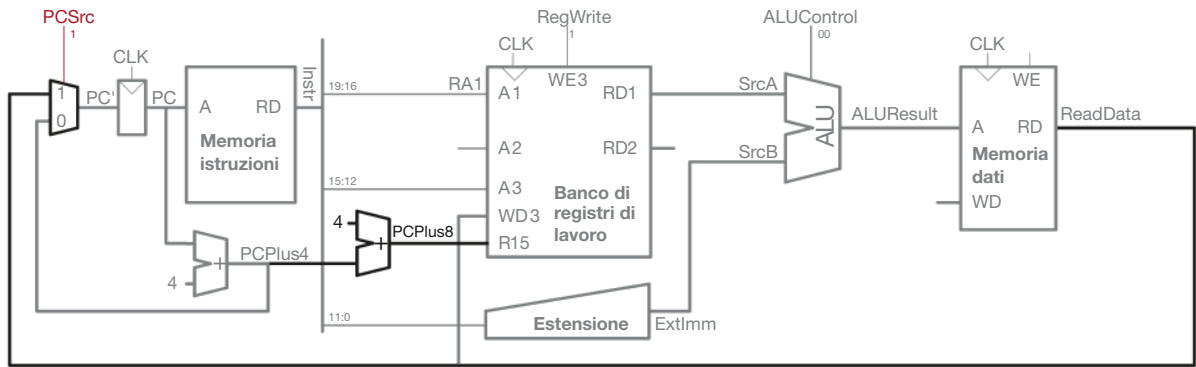
**Figura 7.7** Incremento del program counter.

Come detto nel paragrafo 6.4.6, nell'architettura ARM leggere dal registro R15 restituisce il valore  $PC + 8$ . Serve quindi un altro sommatore per incrementare ulteriormente il PC e passare il risultato alla porta  $R15$  del banco di registri. Analogamente, scrivere nel registro R15 modifica anche il PC. Quindi il valore  $PC$  può provenire dal risultato dell'istruzione (*ReadData*) invece che da *PCPlus4*. Serve quindi un multiplexer per scegliere tra le due possibilità. Il segnale di controllo *PCSrc* è messo a 0 per selezionare *PCPlus4* e a 1 per selezionare *ReadData*. Questi aspetti legati al PC sono evidenziati nella **Figura 7.8**.

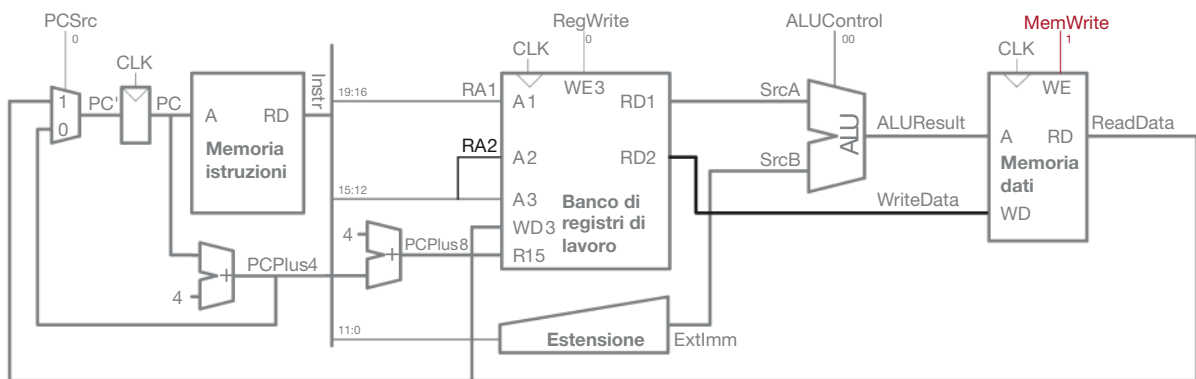
### STR

Si può ora estendere il percorso dati per gestire anche l'istruzione *STR*. Come *LDR*, anche *STR* legge un indirizzo base dalla porta 1 del banco di registri ed estende con zeri l'immediato. L'ALU somma l'immediato esteso all'indirizzo base per trovare l'indirizzo di memoria. Tutti questi passi sono già supportati dal percorso dati.

L'istruzione *STR* legge anche un secondo registro dal banco e lo scrive nella memoria dati. La **Figura 7.9** mostra le nuove connessioni necessarie per questa operazione. Il registro è specificato nel campo *Rd*, cioè i bit  $Instr_{15:12}$ , collegati alla porta  $A2$  del banco di registri. Il valore del registro viene emesso sulla porta  $RD2$ , collegata alla porta di scrittura  $WD$  della memoria dati. L'abilitazione alla scrittura della memoria dati,  $WE$ , è controllata dal segnale *MemWrite*: per l'istruzione *STR*, deve essere *MemWrite* = 1 per scrivere il dato; *ALUControl* = 00 per sommare indirizzo base e spiazzamento; *RegWrite* = 0 perché non si deve scrivere nulla nel banco di registri. Si noti che il dato viene comunque letto dalla parola di memoria dati indirizzata, ma tale dato in *ReadData* viene ignorato perché *RegWrite* = 0.



**Figura 7.8** Lettura o scrittura del program counter come registro R15.



**Figura 7.9** Scrittura di dati in memoria per l'istruzione STR.

### Istruzioni di elaborazione dati con indirizzamento immediato

Si passa ora all'estensione del percorso dati per gestire le istruzioni di elaborazione dati: ADD, SUB, AND e ORR, utilizzando il modo di indirizzamento immediato. Tutte le istruzioni leggono un registro sorgente dal banco e un immediato dai bit meno significativi dell'istruzione, eseguono un'operazione dell'ALU sui valori così ricavati e scrivono il risultato in un registro. Possono quindi essere eseguite con lo stesso hardware semplicemente utilizzando differenti segnali *ALUControl*. Come descritto nel paragrafo 5.2.4, *ALUControl* è 00 per ADD, 01 per SUB, 10 per AND e 11 per ORR. L'ALU genera quattro flag, *ALUFlags*<sub>3:0</sub> (*Zero*, *Negative*, *Carry*, o *Verflow*), che vengono inviate all'unità di controllo.

La **Figura 7.10** mostra il percorso dati esteso per gestire le istruzioni di elaborazione dati con indirizzamento immediato. Come per LDR, il percorso dati legge il primo operando per l'ALU dalla porta 1 del banco di registri ed estende con zeri l'immediato contenuto nei bit meno significativi di *Instr*. Queste istruzioni però usano un immediato a 8 bit invece che a 12, quindi serve il nuovo segnale *ImmSrc* al blocco circuitale dell'estensione: quando vale 0, *ExtImm* viene esteso con zeri da *Instr*<sub>7:0</sub> in avanti per le istruzioni di elaborazione dati; quando vale 1, *ExtImm* viene esteso con zeri da *Instr*<sub>11:0</sub> in avanti per le istruzioni LDR e STR.

Nell'istruzione `LDR`, il banco di registri riceve il dato da scrivere dalla memoria dati. Invece le istruzioni di elaborazione dati devono scrivere nel banco di registri il risultato dell'ALU: `ALUResult`. Serve quindi un ulteriore multiplexer per selezionare tra `ReadData` e `ALUResult`, la cui uscita è denominata `Result`. Il multiplexer è pilotato da un nuovo segnale di controllo, `MemtoReg`: questo segnale vale 0 per le istruzioni di elaborazione dati per selezionare

come *Result* il risultato dell'ALU *ALUResult*; vale invece 1 per LDR per selezionare *ReadData*. Per STR il valore di *MemtoReg* non interessa perché STR non scrive nel banco di registri.

### Istruzioni di elaborazione dati con indirizzamento a registro

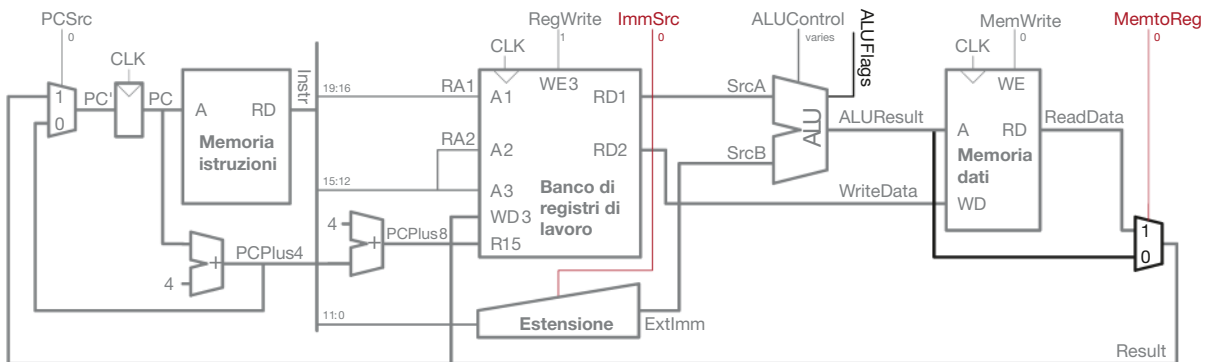
Le istruzioni di elaborazione dati con indirizzamento a registro ricevono il secondo operando sorgente da *Rm*, indicato dai bit *Instr<sub>3:0</sub>*, invece che dall'immediato. Servono quindi ulteriori multiplexer agli ingressi del banco di registri e dell'ALU per selezionare questo secondo registro sorgente, come mostrato nella **Figura 7.11**.

*RA2* viene prelevato dal campo *Rd* (i bit *Instr<sub>15:12</sub>*) per l'istruzione STR e dal campo *Rm* (i bit *Instr<sub>3:0</sub>*) per le istruzioni di elaborazione dati con indirizzamento a registro in base al valore del segnale di controllo *RegSrc*. Analogamente, il segnale di controllo *ALUSrc* seleziona come secondo operando dell'ALU *ExtImm* per le istruzioni che usano indirizzamento immediato e il banco di registri per le istruzioni che usano indirizzamento a registro.

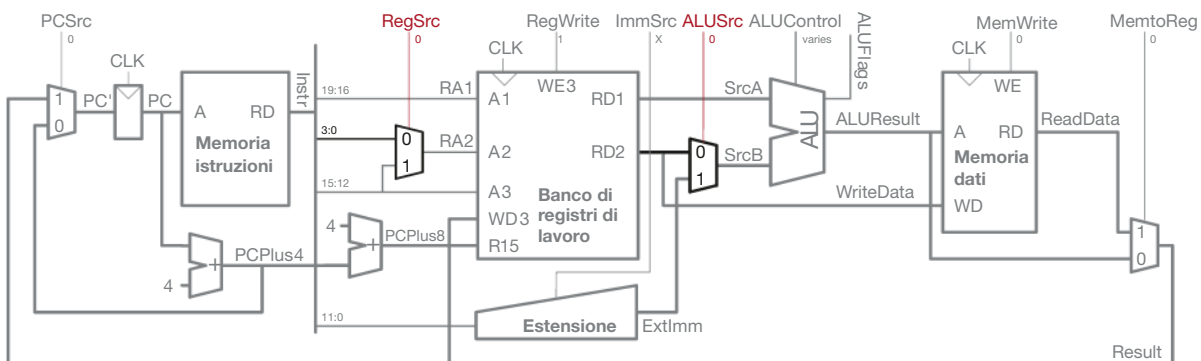
### B

Infine si estende il percorso dati per gestire l'istruzione di salto B, come mostrato nella **Figura 7.12**. L'istruzione somma un immediato a 24 bit a  $PC + 8$  e scrive il risultato nel PC. L'immediato deve essere moltiplicato per 4 ed esteso con segno. Quindi la logica di estensione richiede un'ulteriore modalità di funzionamento: *ImmSrc* deve essere aumentato a 2 bit, con le codifiche riportate nella **Tabella 7.1**.

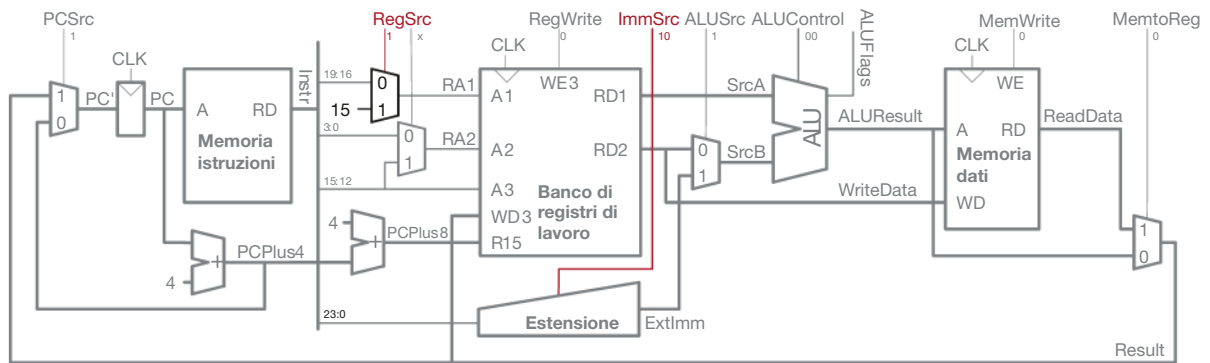
$PC + 8$  è letto dalla prima porta del banco di registri, quindi serve un multiplexer per selezionare R15 come ingresso *RA1*: questo multiplexer è pilotato



**Figura 7.10** Aggiunte al percorso dati per istruzioni di elaborazione dati con indirizzamento immediato.



**Figura 7.11** Aggiunte al percorso dati per istruzioni di elaborazione dati con indirizzamento a registro.



**Figura 7.12** Aggiunte al percorso dati per l'istruzione B.

**Tabella 7.1** Codifica di ImmSrc.

| ImmSrc | ExtImm                                                          | Descrizione                                                        |
|--------|-----------------------------------------------------------------|--------------------------------------------------------------------|
| 00     | {24 0s} <i>Instr</i> <sub>7:0</sub>                             | Immediato senza segno a 8 bit per elaborazione dati                |
| 01     | {20 0s} <i>Instr</i> <sub>11:0</sub>                            | Immediato senza segno a 12 bit per le istruzioni LDR/STR           |
| 10     | {6 <i>Instr</i> <sub>23</sub> } <i>Instr</i> <sub>23:0</sub> 00 | Immediato con segno a 24 bit moltiplicato per 4 per l'istruzione B |

da un altro bit di *RegSrc*, che seleziona *Instr*<sub>19:16</sub> per le altre istruzioni e 15 per l'istruzione B.

*MemtoReg* è portato a 0 e *PCSrc* a 1 per selezionare il nuovo valore di PC da *ALUResult*.

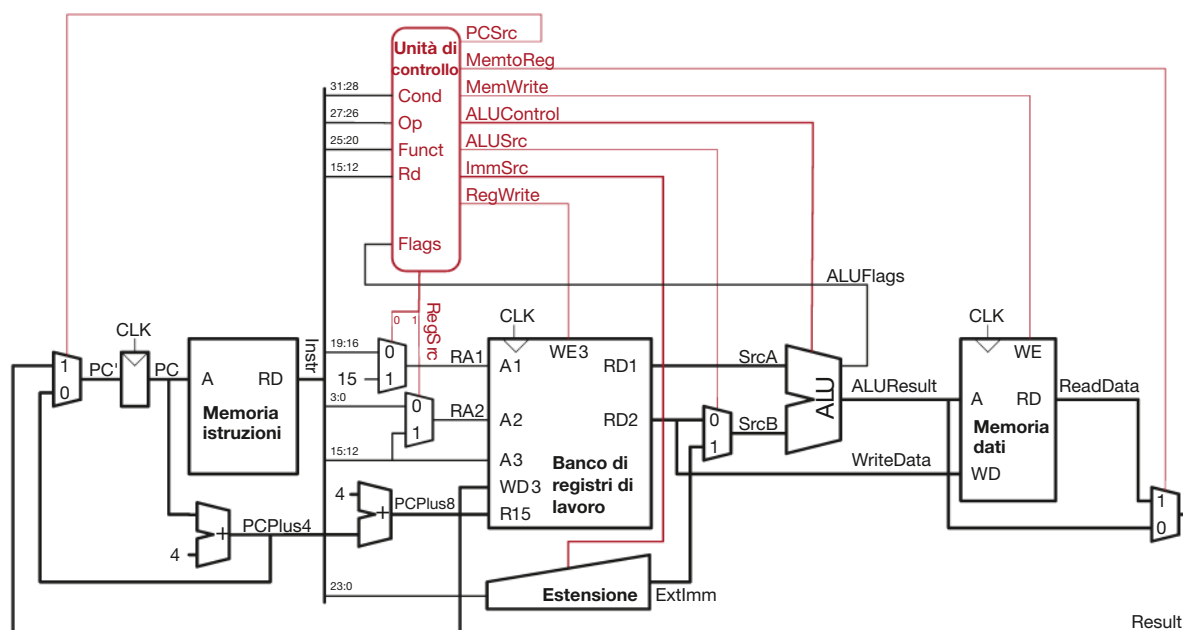
In questo modo il progetto del percorso dati per il processore a ciclo singolo è completo. Si è arrivati a questo punto illustrando il processo di progettazione, nel quale si identificano i vari elementi di stato e si aggiungono progressivamente le parti di logica combinatoria necessarie. Nel prossimo paragrafo si mostra come generare i segnali di controllo che pilotano le operazioni di questo percorso dati.

### 7.3.2 Unità di controllo a ciclo singolo

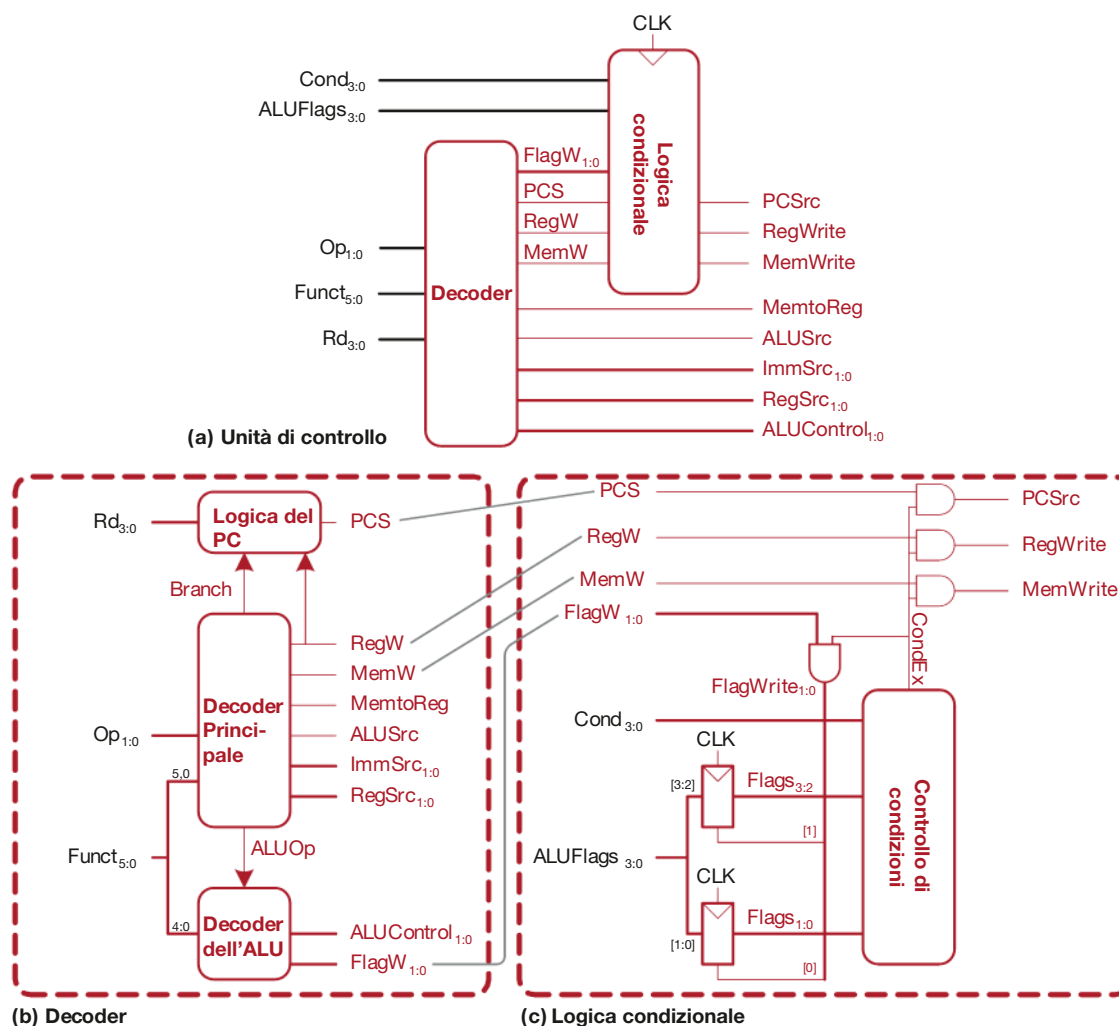
L'unità di controllo genera i segnali di controllo sulla base dei campi *cond*, *op* e *funct* dell'istruzione (i bit *Instr*<sub>31:28</sub>, *Instr*<sub>27:26</sub> e *Instr*<sub>25:20</sub>), come pure delle flag in uscita dall'ALU e del fatto che il registro destinazione sia il PC. L'unità di controllo deve anche memorizzare e aggiornare opportunamente le flag di stato. La **Figura 7.13** mostra l'intera struttura del processore a ciclo singolo con l'unità di controllo collegata al percorso dati.

La **Figura 7.14** mostra in dettaglio lo schema dell'unità di controllo, suddivisa in due parti principali: il *Decoder*, che genera i segnali di controllo in base a *Instr*, e la Logica Condizionale, che mantiene le flag di stato e abilita gli aggiornamenti dello stato architetturale quando l'istruzione deve essere eseguita in modo condizionato. Il Decoder, mostrato nella **Figura 7.14(b)**, è costituito da un Decoder Principale che genera la maggior parte dei segnali di controllo, da un Decoder dell'ALU che usa il campo *funct* per determinare il tipo di operazione aritmetica, e da una Logica del PC per decidere se il PC deve essere modificato per un'istruzione di salto o una scrittura in R15.

Il comportamento del Decoder Principale è descritto dalla tabella delle verità riportata nella **Tabella 7.2**. Il Decoder Principale determina il tipo di istruzione: elaborazione dati a registro, elaborazione dati a immediato, STR, LDR o B, e genera opportunamente i segnali di controllo per il percorso dati. Invia *MemtoReg*, *ALUSrc*, *ImmSrc*<sub>1:0</sub> e *RegSrc*<sub>1:0</sub> direttamente al percorso dati. Le abilitazioni alla scrittura *MemW* e *RegW* devono invece passare nella



**Figura 7.13** Processore a ciclo singolo completo.



**Figura 7.14** Unità di controllo a ciclo singolo.



**Tabella 7.2** Tabella delle verità del Decoder Principale.

| Op | Funct <sub>5</sub> | Funct <sub>0</sub> | Tipo   | Branch | MemtoReg | MemW | ALUSrc | ImmSrc | RegW | RegSrc | ALUOp |
|----|--------------------|--------------------|--------|--------|----------|------|--------|--------|------|--------|-------|
| 00 | 0                  | X                  | DP Reg | 0      | 0        | 0    | 0      | XX     | 1    | 00     | 1     |
| 00 | 1                  | X                  | DP Imm | 0      | 0        | 0    | 1      | 00     | 1    | X0     | 1     |
| 01 | X                  | 0                  | STR    | 0      | X        | 1    | 1      | 01     | 0    | 10     | 0     |
| 01 | X                  | 1                  | LDR    | 0      | 1        | 0    | 1      | 01     | 1    | X0     | 0     |
| 10 | X                  | X                  | B      | 1      | 0        | 0    | 1      | 10     | 0    | X1     | 0     |

**Tabella 7.3** Tabella delle verità del Decoder dell'ALU.

| ALUOp | Funct <sub>4:1</sub><br>(cmd) | Funct <sub>0</sub><br>(S) | Tipo   | ALUControl <sub>1:0</sub> | FlagW <sub>1:0</sub> |
|-------|-------------------------------|---------------------------|--------|---------------------------|----------------------|
| 0     | X                             | X                         | Not DP | 00 (Add)                  | 00                   |
| 1     | 0100                          | 0                         | ADD    | 00 (Add)                  | 00                   |
|       |                               | 1                         |        |                           | 11                   |
|       | 0010                          | 0                         | SUB    | 01 (Sub)                  | 00                   |
|       |                               | 1                         |        |                           | 11                   |
|       | 0000                          | 0                         | AND    | 10 (And)                  | 00                   |
|       |                               | 1                         |        |                           | 10                   |
|       | 1100                          | 0                         | ORR    | 11 (Or)                   | 00                   |
|       |                               | 1                         |        |                           | 10                   |

Logica Condizionale prima di diventare i segnali di controllo *MemWrite* e *RegWrite* del percorso dati. Tali segnali di controllo possono essere forzati a 0 dalla Logica Condizionale se la condizione di scrittura non si è verificata. Il Decoder Principale genera anche i segnali *Branch* e *ALUOp* usati all'interno dell'unità di controllo per indicare rispettivamente che l'istruzione è B oppure un'istruzione di elaborazione dati. La rete logica del Decoder Principale può essere ricavata a partire dalla tabella delle verità usando un qualsiasi metodo di sintesi di reti combinatorie.

Il comportamento del Decoder dell'ALU è descritto dalla tabella delle verità riportata nella **Tabella 7.3**. Per le istruzioni di elaborazione dati, il Decoder dell'ALU genera *ALUControl* in base al tipo di istruzione (ADD, SUB, AND od ORR). Inoltre attiva *FlagW* per aggiornare le flag di stato quando il bit S vale 1. Si noti che ADD e SUB aggiornano tutte le flag, mentre AND e ORR aggiornano solo le flag N e Z: servono quindi due bit per *FlagW*: *FlagW*<sub>1</sub> per aggiornare N e Z (cioè i bit *Flags*<sub>3:2</sub>) e *FlagW*<sub>0</sub> per aggiornare C e V (cioè i bit *Flags*<sub>1:0</sub>). *FlagW*<sub>1:0</sub> è forzato a 0 dalla Logica Condizionale quando la condizione non si è verificata (*CondEx* = 0).

La Logica del PC controlla se l'istruzione è una scrittura di R15 o un salto, che implicano aggiornamento del PC. L'espressione logica corrispondente è:

$$PCS = ((Rd = 15) \& RegW) \mid Branch$$

*PCS* può quindi essere forzato a 0 della Logica Condizionale prima di essere inviato al percorso dati come *PCSrc*.

La Logica Condizionale, mostrata nella Figura 7.14(c), determina se l'istruzione deve essere eseguita (*CondEx*) in base al campo *cond* dell'istruzione e ai valori attuali delle flag N, Z, C e V (i bit *Flags*<sub>3:0</sub>), come era stato descritto nella Tabella 6.3. Se l'istruzione non deve essere eseguita, le abilitazioni alla scrittura e il segnale *PCSrc* sono forzati a 0 in modo tale che lo

stato architetturale non venga modificato. La Logica Condizionale aggiorna anche alcune o tutte le *flag* ai valori *ALUFlags* quando *FlagW* è attivato dal *Decoder* dell'ALU e la condizione di esecuzione dell'istruzione si è verificata (*CondEx* = 1).

### ESEMPIO 7.1

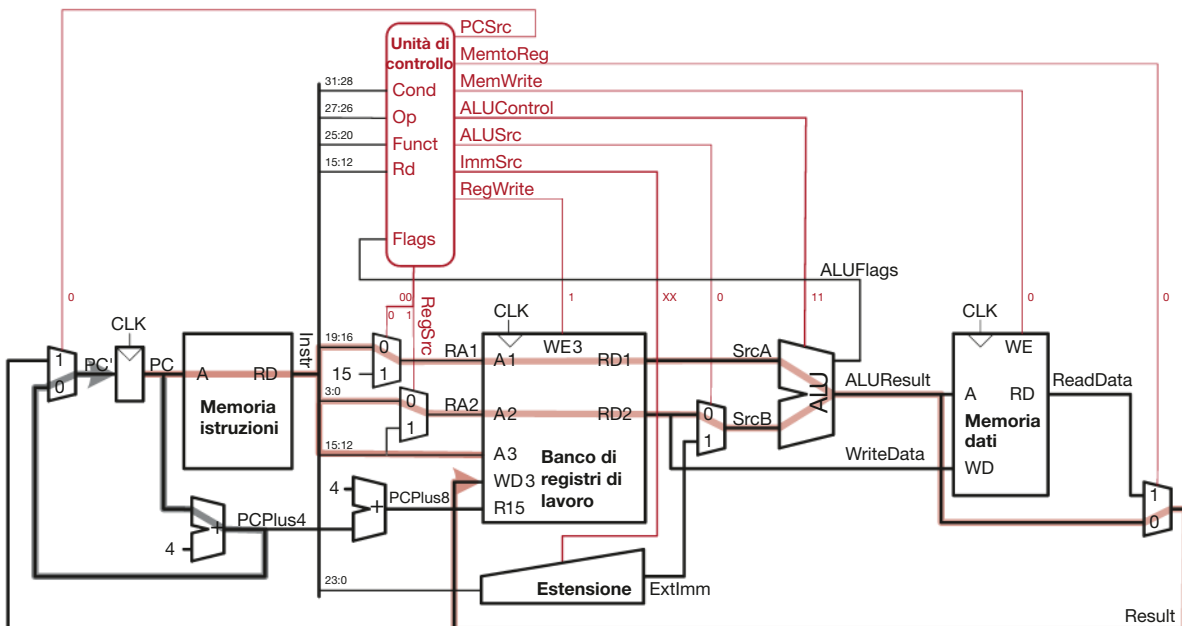
**Funzionamento del processore a ciclo singolo.** Determinare i valori dei segnali di controllo e le parti del percorso dati usate quando si esegue l'istruzione *ORR* con modo di indirizzamento a registro.

**Soluzione** La **Figura 7.15** mostra i segnali di controllo e il flusso dei dati durante l'esecuzione dell'istruzione *ORR*. Il PC punta alla locazione di memoria che contiene l'istruzione, e la memoria istruzioni restituisce tale istruzione.

Il flusso di dati attraverso il banco di registri e l'ALU è mostrato dalle linee rosse più spesse: il banco di registri emette i due operandi sorgente indicati da *Instr*<sub>19:16</sub> e da *Instr*<sub>3:0</sub>, quindi *RegSrc* deve valere 00. *SrcB* proviene dalla seconda porta del banco di registri (non da *ExtImm*), quindi *ALUSrc* deve valere 0. L'ALU deve eseguire l'operazione logica OR bit a bit, quindi *ALUControl* deve valere 11. Il risultato proviene dall'ALU, quindi *MemtoReg* deve valere 0. Tale risultato va scritto nel banco di registri, quindi *RegWrite* deve valere 1. Non si deve scrivere niente in memoria, quindi *MemWrite* deve valere 0.

L'aggiornamento di PC con *PCPlus4* è mostrato dalla linea grigia spessa. *PCSrc* è forzato a 0 per selezionare il valore incrementato PC.

Si noti che c'è comunque flusso di dati anche nei percorsi non evidenziati, ma i valori di tali dati non sono significativi per l'istruzione considerata. Per esempio, l'immediato viene esteso con zeri e si legge dalla memoria dati, ma questi valori non influenzano lo stato prossimo del sistema.



**Figura 7.15** Segnali di controllo e flusso dei dati durante l'esecuzione dell'istruzione *ORR*.

### 7.3.3 Istruzioni aggiuntive

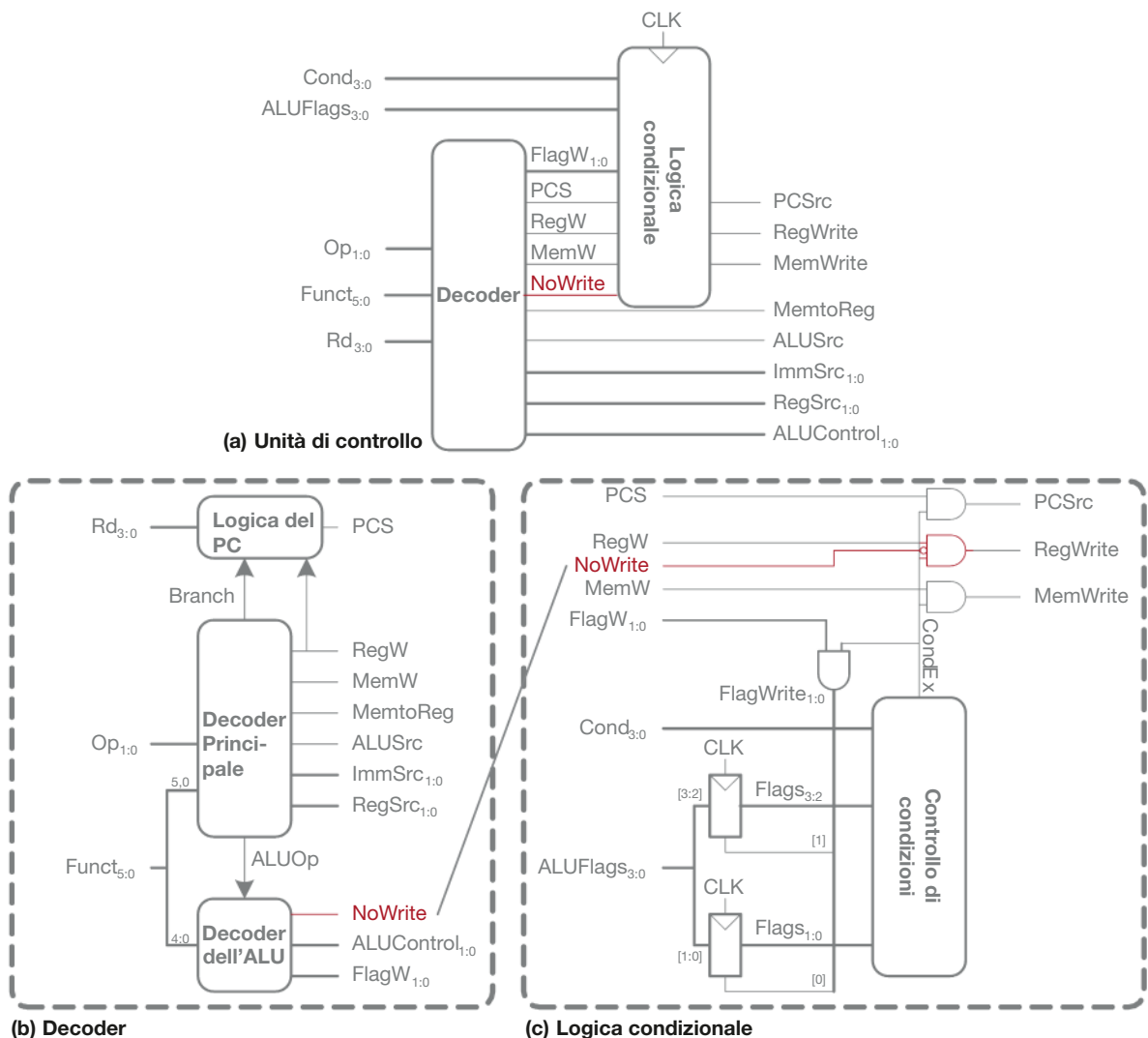
Si è finora considerato un limitato sottoinsieme del set di istruzioni ARM. In questo paragrafo si aggiungono i supporti per l'istruzione di confronto (*CMP*)

e per i modi di indirizzamento nei quali il secondo operando sorgente è un registro traslato. Questi esempi mostrano come trattare istruzioni aggiuntive: con il dovuto sforzo si può estendere il processore a ciclo singolo in modo che possa gestire l'intero set di istruzioni ARM. Come si vede in questa sezione, per gestire alcune istruzioni aggiuntive basta estendere i decoder, mentre in altri casi serve aggiungere hardware al percorso dati.

### ESEMPIO 7.2

**Istruzione CMP.** L'istruzione di confronto **CMP** sottrae *SrcB* da *SrcA* e aggiorna le *flag* ma non scrive il risultato della sottrazione in alcun registro. Il percorso dati è già in grado di svolgere queste operazioni. Determinare le modifiche all'unità di controllo necessarie per supportare l'istruzione **CMP**.

**Soluzione** Si introduce un nuovo segnale di controllo, *NoWrite*, per evitare la scrittura di *Rd* durante l'istruzione **CMP** (questo segnale servirebbe anche per altre istruzioni come **TST** che non scrivono nei registri). Si estende il Decoder dell'ALU per generare il suddetto segnale e la logica di *RegWrite* perché ne faccia uso, come mostrato in rosso nella **Figura 7.16**. La tabella delle verità per il Decoder dell'ALU così ampliato è riportata nella **Tabella 7.4**, dove sono evidenziati il nuovo segnale di controllo e la nuova istruzione.



**Figura 7.16** Modifiche al controllore per l'istruzione **CMP**.

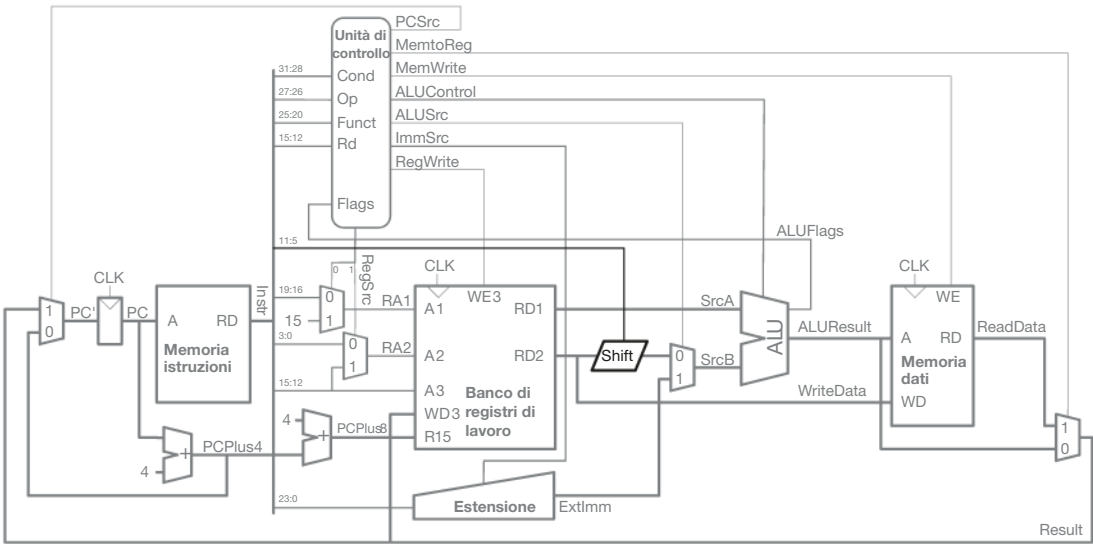
**Tabella 7.4** Tabella delle verità del Decoder dell'ALU con le aggiunte per l'istruzione *CMP*.

| ALUOp | Funct <sub>4:1</sub><br>(cmd) | Funct <sub>0</sub><br>(S) | Note   | ALUControl <sub>1:0</sub> | FlagW <sub>1:0</sub> | NoWrite |
|-------|-------------------------------|---------------------------|--------|---------------------------|----------------------|---------|
| 0     | X                             | X                         | Not DP | 00                        | 00                   | 0       |
| 1     | 0100                          | 0                         | ADD    | 00                        | 00                   | 0       |
|       |                               | 1                         |        |                           | 11                   | 0       |
|       | 0010                          | 0                         | SUB    | 01                        | 00                   | 0       |
|       |                               | 1                         |        |                           | 11                   | 0       |
|       | 0000                          | 0                         | AND    | 10                        | 00                   | 0       |
|       |                               | 1                         |        |                           | 10                   | 0       |
|       | 1100                          | 0                         | ORR    | 11                        | 00                   | 0       |
|       |                               | 1                         |        |                           | 10                   | 1       |
|       | 1010                          | 1                         | CMP    | 01                        | 11                   | 1       |

**ESEMPIO 7.3**

**Modo di indirizzamento aggiuntivo: a registro con traslazione a costante.** Sin qui si è ipotizzato che le istruzioni di elaborazione dati con indirizzamento a registro non facessero alcuna traslazione del secondo operando. Modificare il processore a ciclo singolo perché supporti anche la traslazione di un immediato.

**Soluzione** Si inserisce un traslatore (*shift*) prima dell'ALU. La **Figura 7.17** mostra il percorso dati così ampliato. Il traslatore usa i bit *Instr*<sub>11:7</sub> per specificare la quantità di traslazione e i bit *Instr*<sub>6:5</sub> per indicare il tipo di traslazione.



**Figura 7.17** Aggiunte al percorso dati per indirizzamento a registro con traslazione a costante.

**7.3.4 Analisi delle prestazioni**

Ogni istruzione occupa un ciclo di clock nel processore a ciclo singolo, quindi il CPI vale 1. Il percorso critico per l'istruzione *LDR* è indicato nella **Figura 7.18** da una linea spessa rossa. La linea parte con il PC che genera il nuovo indirizzo in corrispondenza del fronte di salita del clock; la memoria istruzioni emette la nuova istruzione; il *Decoder Principale* calcola *RegSrc*<sub>0</sub>, che forza la *multiplexer* a selezionare *Instr*<sub>19:16</sub> come *RA1*, e il banco di registri emette il contenuto di questo registro come *SrcA*. Mentre il banco di registri emette il suddetto valore, il

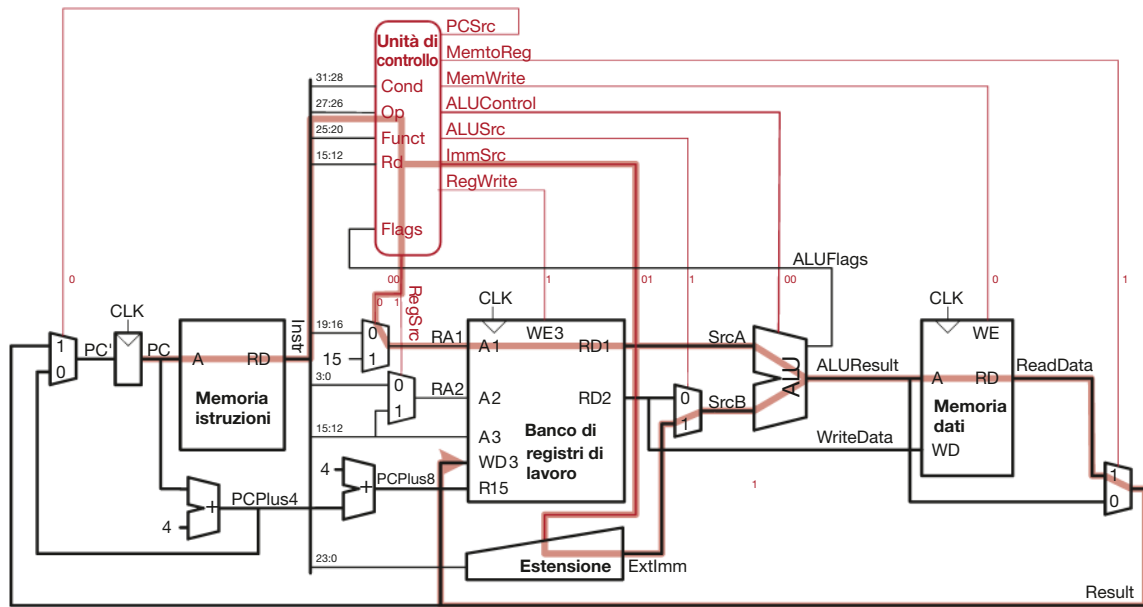


Figura 7.18 Percorso critico per l'istruzione LDR.

campo immediato viene esteso con zeri e selezionato dal multiplexer controllato dal segnale *ALUSrc* per determinare *SrcB*. L'ALU somma *SrcA* e *SrcB* per trovare l'indirizzo effettivo, e si va a leggere dalla memoria dati con questo indirizzo: il multiplexer controllato da *MemtoReg* seleziona *ReadData*. Infine, il controllo *Result* del banco di registri deve essere attivato prima del prossimo fronte di salita del clock per effettuare correttamente la scrittura. Il tempo di ciclo è dunque:

$$T_{c1} = t_{pcq\_PC} + t_{mem} + t_{dec} + \max[t_{mux} + t_{RFread} + t_{ext} + t_{mux}] + t_{ALU} + t_{mem} + t_{mux} + t_{RFsetup} \quad (7.2)$$

Si usa il pedice 1 nella sigla  $T_{c1}$  di questo tempo di ciclo per distinguerlo dai tempi di ciclo dei progetti successivi di microarchitettura. In molte tecnologie di realizzazione, l'ALU, la memoria e il banco di registri sono decisamente più lenti delle altre parti combinatorie, quindi l'espressione del tempo di ciclo si semplifica nel modo seguente:

$$T_{c1} = t_{pcq\_PC} + 2t_{mem} + t_{dec} + t_{RFread} + t_{ALU} + 2t_{mux} + t_{RFsetup} \quad (7.3)$$

I valori effettivi di questi tempi dipendono naturalmente dalla tecnologia di realizzazione.

Altre istruzioni hanno percorsi critici più brevi: per esempio, le istruzioni di elaborazione dati non hanno bisogno di accedere alla memoria dati. Ma se si considera un progetto di rete sequenziale sincrona, il periodo di clock deve essere costante e dimensionato sull'istruzione più lenta.

#### ESEMPIO 7.4

**Prestazioni del processore a ciclo singolo.** Ben Imbrogliabit sta pensando di realizzare un processore a ciclo singolo in tecnologia CMOS a 16 nanometri. Ha calcolato che i ritardi degli elementi logici hanno i valori riportati nella **Tabella 7.5**. Bisogna aiutarlo a calcolare quanto tempo impiegherà sul processore l'esecuzione di un programma con 100 miliardi di istruzioni.

**Soluzione** In base all'Espressione 7.3, il tempo di ciclo del processore a ciclo singolo risulta essere  $T_{c1} = 40 + 2(200) + 70 + 100 + 120 + 2(25) + 60 = 840$  ps. In base all'Espressione 7.1, il tempo totale di esecuzione è  $T_1 = (100 \times 10^9 \text{ istruzioni}) (1 \text{ ciclo/istruzione}) (840 \times 10^{-12} \text{ s/ciclo}) = 84$  secondi.

**Tabella 7.5** Ritardo degli elementi circuitali.

| Elemento                      | Parametro     | Ritardo (ps) |
|-------------------------------|---------------|--------------|
| Registro: clock-uscita        | $t_{pcq}$     | 40           |
| Registro: <i>setup</i>        | $t_{setup}$   | 50           |
| Multiplexer                   | $t_{mux}$     | 25           |
| ALU                           | $t_{ALU}$     | 120          |
| Decoder                       | $t_{dec}$     | 70           |
| Lettura da memoria            | $t_{mem}$     | 200          |
| Lettura dal banco di registri | $t_{RFread}$  | 100          |
| Setup del banco di registri   | $t_{RFsetup}$ | 60           |

**7.4 ■ PROCESSORE MULTI CICLO**

Il processore a ciclo singolo ha tre elementi di debolezza. In primo luogo, richiede memorie separate per istruzioni e dati, mentre la maggior parte dei processori ha una sola memoria esterna che contiene sia istruzioni sia dati. In secondo luogo, richiede un ciclo di clock abbastanza lungo da consentire l'esecuzione dell'istruzione più lenta (*LDR*) anche se molte istruzioni potrebbero essere più veloci. Infine, richiede tre circuiti sommatore (uno nell'ALU, e due per la Logica del PC): circuiti relativamente costosi soprattutto se devono essere veloci.

Il processore multi ciclo si propone di eliminare queste tre debolezze dividendo l'istruzione in una sequenza di passi più brevi: in ciascun passo, il processore legge o scrive in memoria o nel banco di registri, oppure usa l'ALU. L'istruzione viene letta da memoria in un passo, e i dati possono essere letti o scritti in passi successivi, quindi è possibile usare una sola memoria per contenere entrambi. Istruzioni diverse usano numeri diversi di passi, quindi le istruzioni più semplici vengono portate a termine in meno tempo rispetto a quelle più complesse. E il processore richiede un solo circuito sommatore, usato a scopi differenti nei diversi passi.

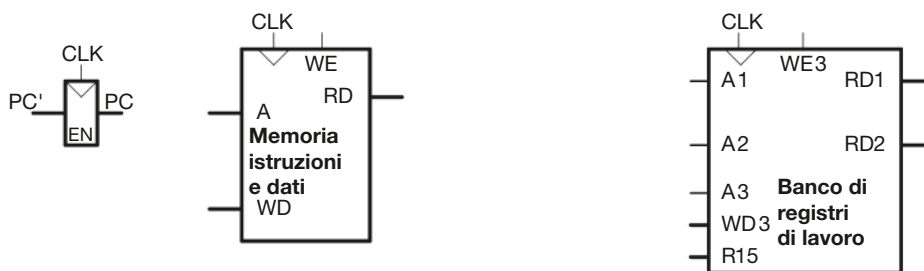
Il progetto del processore multi ciclo segue lo stesso procedimento già usato per il processore a ciclo singolo. Prima si costruisce il percorso dati interconnettendo gli elementi di stato architetturale e le memorie con logica combinatoria; questa volta però si aggiungono elementi di stato non architetturale per memorizzare i risultati intermedi tra un passo e l'altro. Poi si progetta l'unità di controllo, che deve generare segnali di controllo diversi nei diversi passi di esecuzione di una singola istruzione, quindi serve una FSM (macchina a stato finiti) invece della logica combinatoria usata per il processore a ciclo singolo. Infine si analizzano le prestazioni del processore multi ciclo e le si confrontano con quelle del processore a ciclo singolo.

**7.4.1 Percorso dati multi ciclo**

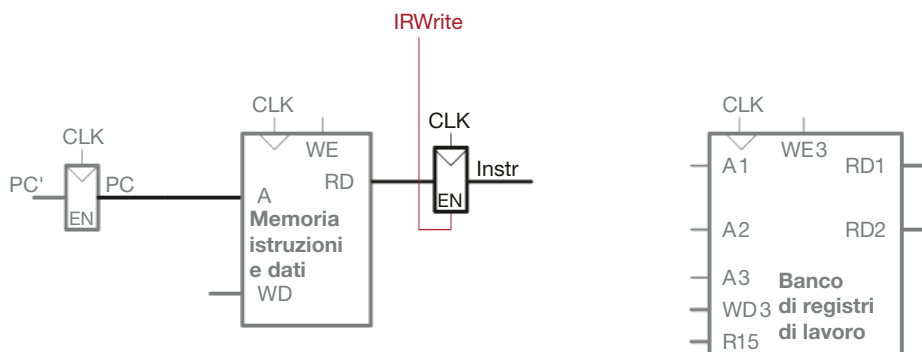
Si inizia ancora il progetto dalla memoria e dallo stato architetturale, come mostrato nella **Figura 7.19**. Nel progetto a ciclo singolo si erano usate memorie separate per istruzioni e dati perché era necessario leggere l'istruzione dalla memoria e leggere o scrivere un dato in memoria nello stesso ciclo. Qui si possono invece riunire istruzioni e dati in un'unica memoria. La soluzione è molto più realistica, ed è fattibile perché si può leggere l'istruzione in un ciclo, quindi leggere o scrivere un dato in un ciclo diverso. PC e banco di registri rimangono gli stessi. Come per il processore a ciclo singolo, si costruisce gradualmente il percorso dati aggiungendo elementi per gestire ciascun passo di ciascuna istruzione.

Il PC contiene l'indirizzo dell'istruzione da eseguire. Il primo passo è dunque la lettura (*fetch*) di tale istruzione dalla memoria. La **Figura 7.20** mostra





**Figura 7.19** Elementi di stato con memoria istruzioni e dati unificata.



**Figura 7.20** Fetch dell'istruzione da memoria.

che il PC viene semplicemente connesso all'ingresso di indirizzo della memoria. L'istruzione viene letta e memorizzata in un registro non architetturale, il registro istruzioni (IR, *Instruction Register*), in modo da renderla disponibile nei passi successivi. IR riceve un segnale di abilitazione, denominato *IRWrite*, che viene attivato quando IR deve essere caricato con la nuova istruzione.

### LDR

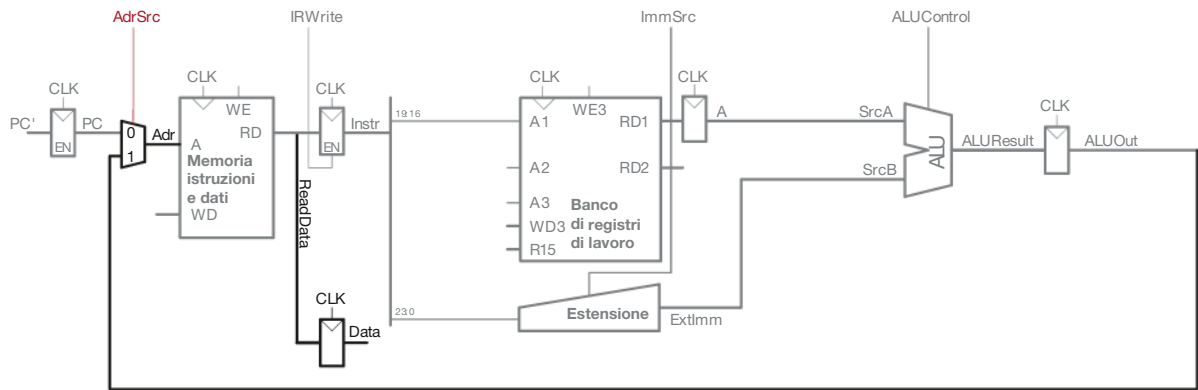
Come già fatto per il processore a ciclo singolo, si inizia a definire le interconnessioni del percorso dati per eseguire l'istruzione *LDR*.

Dopo la fase di fetch di *LDR*, il passo successivo è la lettura del registro sorgente contenente l'indirizzo base. Tale registro è specificato nel campo *Rn* dell'istruzione, cioè i bit  $Instr_{19:16}$ ; tali bit vengono collegati all'ingresso di indirizzo *A1* del banco di registri, come mostrato nella **Figura 7.21**. Il banco emette il contenuto del registro indirizzato sull'uscita di dato *RD1*, e il valore viene memorizzato in un altro registro non architetturale: *A*.

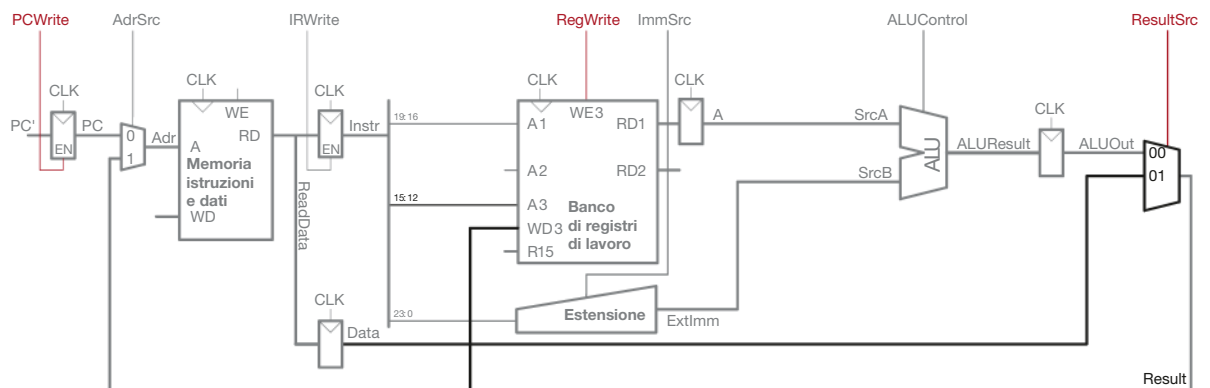
L'istruzione *LDR* richiede anche uno spiazzamento a 12 bit, che si trova nel campo immediato dell'istruzione,  $Instr_{11:0}$ , e che deve essere esteso con zeri a 32 bit, come mostrato nella **Figura 7.21**. Come nel caso del processore a ciclo singolo, il blocco di estensione riceve un segnale di controllo *ImmSrc* che specifica se si deve estendere un immediato a 8, 12 o 24 bit per i vari tipi di istruzioni. L'immediato esteso a 32 bit è denominato *ExtImm*, e per consistenza con il resto del percorso dati dovrebbe essere memorizzato in un altro registro non architetturale, ma *ExtImm* è una funzione combinatoria di *Instr* e non varia durante tutta l'esecuzione dell'istruzione corrente, quindi non serve dedicare un registro per memorizzare un valore che rimane costante.

L'indirizzo del dato da caricare è la somma dell'indirizzo base e dello spiazzamento. Si può usare l'ALU per fare questa somma, come mostrato nella **Figura 7.22**: *ALUControl* deve valere 00 per eseguire la somma, e il risultato *ALUResult* viene memorizzato in un altro registro non architetturale, denominato *ALUOut*.





**Figura 7.23** Lettura del dato da memoria.



**Figura 7.24** Scrittura del risultato (*writeback*) nel banco di registri.

usare l'ALU che non è utilizzata durante la fase di fetch. Per fare questo, si devono inserire dei multiplexer per selezionare *PC* e il valore costante 4 come ingressi all'ALU, come mostrato nella **Figura 7.25**. Un multiplexer controllato da *ALUSrcA* seleziona *PC* oppure il registro *A* come ingresso *SrcA*, e un altro multiplexer seleziona 4 oppure *ExtImm* come ingresso *SrcB*. Per aggiornare il *PC*, l'ALU somma *SrcA* (cioè *PC*) e *SrcB* (cioè 4) e il risultato deve essere scritto nel program counter. Il multiplexer controllato da *ResultSrc* deve poter selezionare questa somma da *ALUResult* invece che da *ALUOut*: serve quindi un terzo ingresso al multiplexer. Il segnale di controllo *PCWrite* abilita la scrittura nel *PC* solo nei cicli opportuni.

Di nuovo ci si trova di fronte all'idiosincrasia dell'architettura ARM, nella quale la lettura di R15 deve restituire  $PC + 8$  e la scrittura di R15 deve modificare il PC. Per la lettura di R15 si è già calcolato il valore  $PC + 4$  durante la fase di fetch, e tale valore è presente nel registro PC. Si può quindi ottenere  $PC + 8$  nel secondo passo sommando con l'ALU ancora il valore 4 al PC già aggiornato. *ALUResult* è selezionato come *Result* e inviato alla porta di ingresso R15 del banco di registri. La **Figura 7.26** mostra il percorso dati completo per l'istruzione `LDR` con questa nuova connessione: una lettura di R15, che pure avviene nel secondo passo dell'istruzione, produce il valore  $PC + 8$  all'uscita di lettura dati del banco di registri. Le scritture in R15 richiedono di scrivere nel PC invece che nel banco di registri. Quindi, nel passo finale dell'istruzione, *Result* deve essere inviato al registro PC (invece che al banco di registri di lavoro) e si deve attivare *PCWrite* invece di *RegWrite*. Il percorso dati è già in grado di fare ciò, per cui non servono ulteriori modifiche.

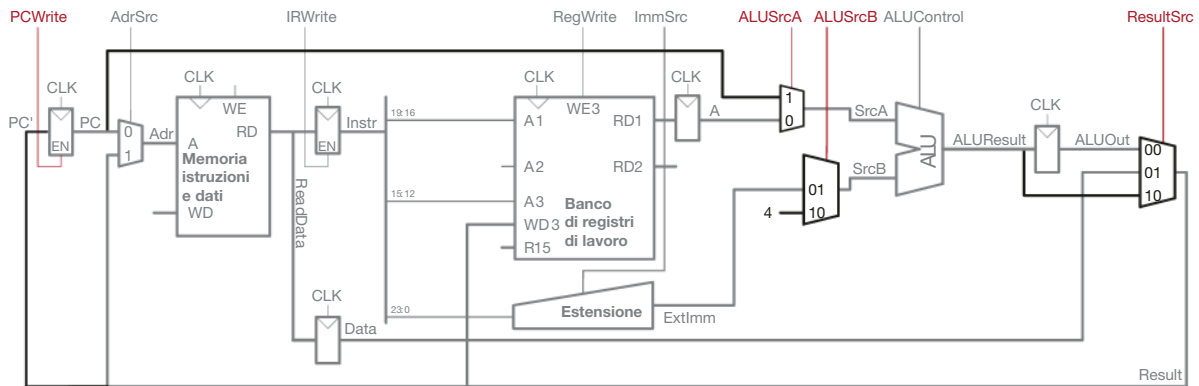


Figura 7.25 Incremento di 4 del PC.

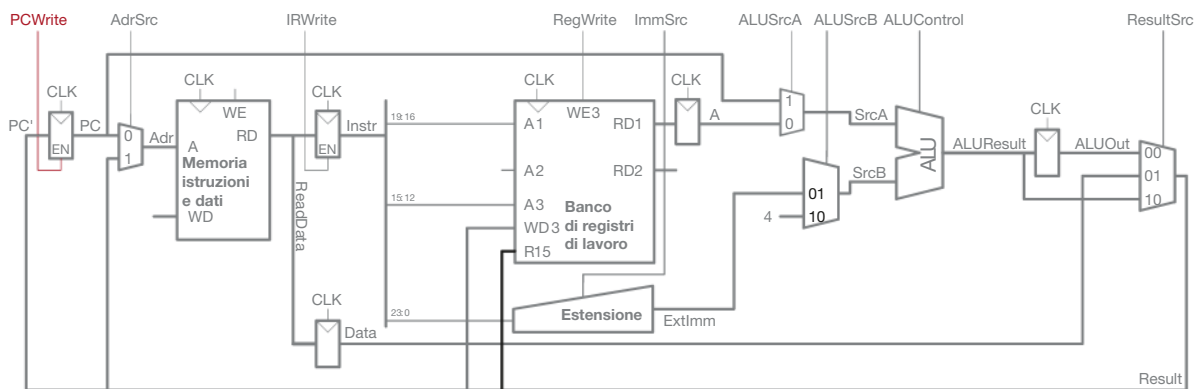


Figura 7.26 Gestione delle letture e scritture del registro R15.

### STR

Si estende ora il percorso dati per gestire l'istruzione *STR*. Come *LDR*, *STR* legge un indirizzo base dalla porta 1 del banco di registri ed estende un immediato. L'ALU somma l'indirizzo base all'immediato per trovare l'indirizzo di memoria. Tutte queste operazioni sono già supportate dall'hardware presente nel percorso dati.

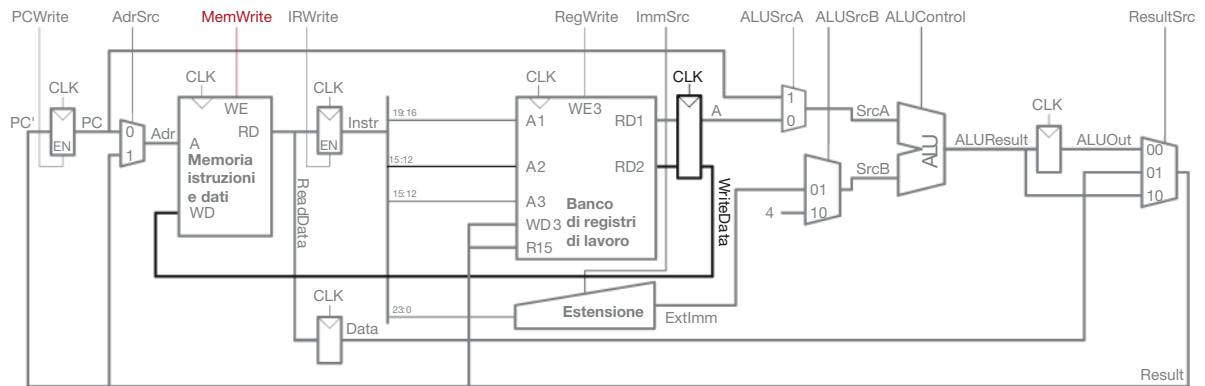
L'unica nuova funzione di *STR* è la necessità di leggere un secondo registro dal banco di registri e scrivere in memoria il valore letto, come mostrato nella **Figura 7.27**. Il registro è indicato dal campo *Rd* dell'istruzione,  $Instr_{15:12}$ , che viene collegato alla seconda porta del banco di registri. Quando il registro viene letto, il valore viene memorizzato in un altro registro non architetturale: *WriteData*. Al passo successivo, tale valore viene inviato alla porta di scrittura *WD* della memoria, la quale riceve il segnale di controllo *MemWrite* che attiva appunto l'operazione di scrittura in memoria.

### Istruzioni di elaborazione dati con indirizzamento immediato

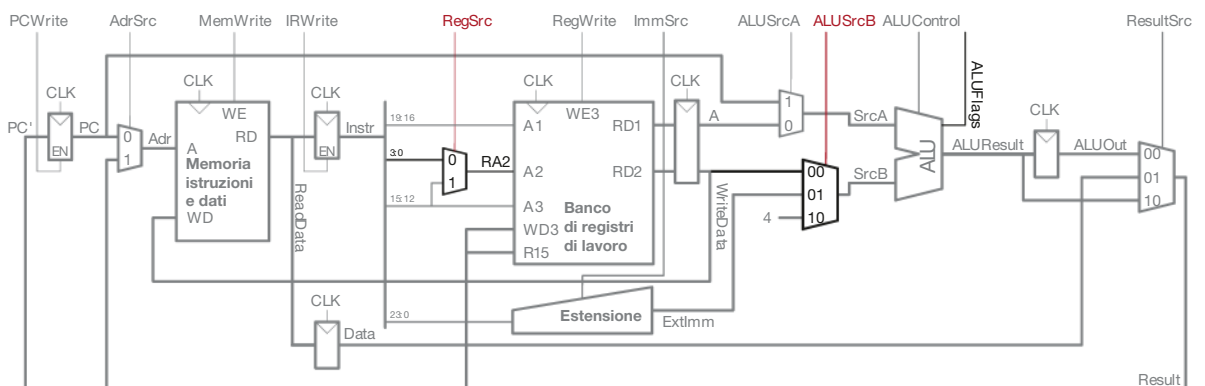
Le istruzioni di elaborazione dati con indirizzamento immediato leggono il primo operando sorgente da *Rn* ed estendono il secondo operando sorgente da un immediato a 8 bit. Operano su tali operandi e scrivono il risultato nel banco di registri. Il percorso dati contiene già tutte le connessioni necessarie per effettuare queste operazioni. L'ALU usa il segnale di controllo *ALUControl* per determinare il tipo di operazione richiesta; le *ALUFlags* sono inviate all'unità di controllo per aggiornare il registro di stato.

### Istruzioni di elaborazione dati con indirizzamento a registro

Le istruzioni di elaborazione dati con indirizzamento a registro selezionano il secondo operando dal banco di registri. Il registro è specificato nel campo



**Figura 7.27** Aggiunte al percorso dati per l'istruzione STR.



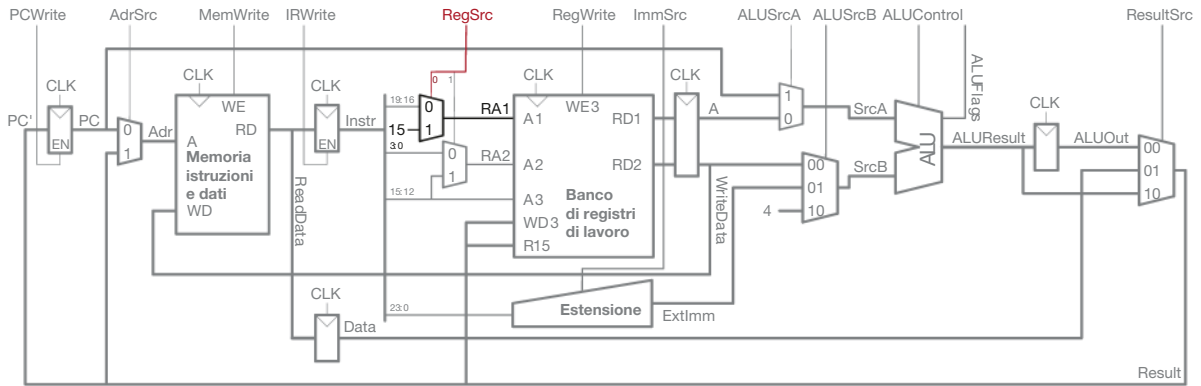
**Figura 7.28** Aggiunte al percorso dati per le istruzioni di elaborazione dati con indirizzamento a registro.

$Rm$ , ovvero  $Instr_{3:0}$ , quindi serve un multiplexer per selezionare questo campo come  $RA2$  del banco di registri. Serve inoltre estendere il multiplexer  $SrcB$  per ricevere il valore letto dal banco di registri, come mostrato nella **Figura 7.28**. Per il resto, il comportamento è identico a quello delle istruzioni di elaborazione dati con indirizzamento immediato.

## B

L'istruzione di salto B legge  $PC + 8$  e un immediato a 24 bit, li somma e somma il risultato al contenuto del PC. Si è visto nel paragrafo 6.4.6 che una lettura di  $R15$  restituisce  $PC + 8$ , quindi serve un ulteriore multiplexer per selezionare  $R15$  come  $RA1$  del banco di registri, come mostrato nella **Figura 7.29**. Il resto dell'hardware per effettuare la somma e scrivere nel PC è già presente nel percorso dati.

A questo punto il progetto del percorso dati multi ciclo è finito. È molto simile a quello del processore a ciclo singolo: si sono progressivamente aggiunte parti hardware tra gli elementi di stato per gestire ogni istruzione. La differenza principale è il fatto che l'istruzione viene eseguita in passi successivi, quindi sono stati aggiunti registri non architetturali per memorizzare i risultati intermedi di ogni passo. In questo modo la memoria può essere condivisa da istruzioni e dati, e l'ALU può essere riutilizzata diverse volte riducendo i costi hardware. Nel prossimo paragrafo si sviluppa come FSM l'unità di controllo per generare la sequenza opportuna di segnali di controllo del percorso dati a ogni passo di ogni istruzione.

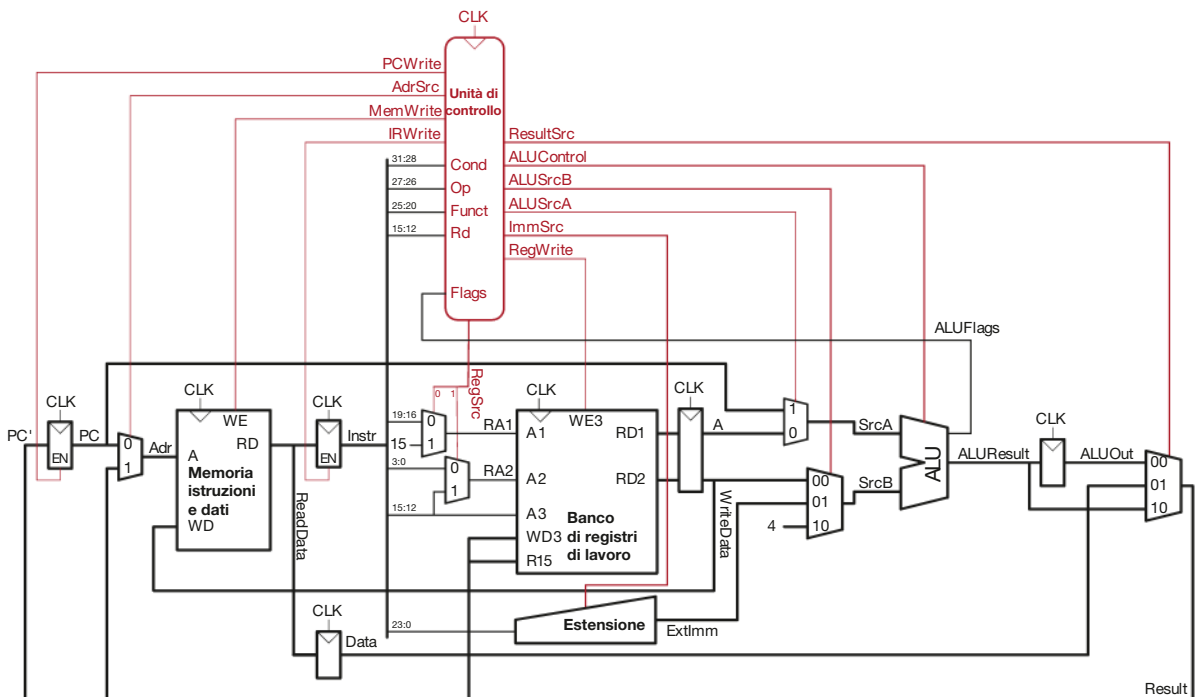


**Figura 7.29** Aggiunte al percorso dati per l'istruzione B.

### 7.4.2 Unità di controllo multi ciclo

Come per il processore a ciclo singolo, l'unità di controllo genera i segnali di controllo sulla base dei campi *cond*, *op* e *funct* dell'istruzione (i bit *Instr*<sub>31:28</sub>, *Instr*<sub>27:26</sub> e *Instr*<sub>25:20</sub>), come pure delle flag in uscita dall'ALU e del fatto che il registro destinazione sia il PC. L'unità di controllo deve anche memorizzare e aggiornare opportunamente le flag di stato. La **Figura 7.30** mostra l'intera struttura del processore multi ciclo con l'unità di controllo collegata al percorso dati. Il percorso dati è disegnato in nero e l'unità di controllo in rosso.

Come nel caso del processore a ciclo singolo, l'unità di controllo è divisa nei due blocchi Decoder e Logica Condizionale, come mostrato nella **Figura 7.31(a)**. Il Decoder è a sua volta suddiviso in parti, come mostrato nella **Figura 7.31(b)**. Il Decoder Principale di tipo combinatorio del processore a ciclo singolo viene sostituito nel processore multi ciclo da una FSM Principa-



**Figura 7.30** Processore multi ciclo completo.



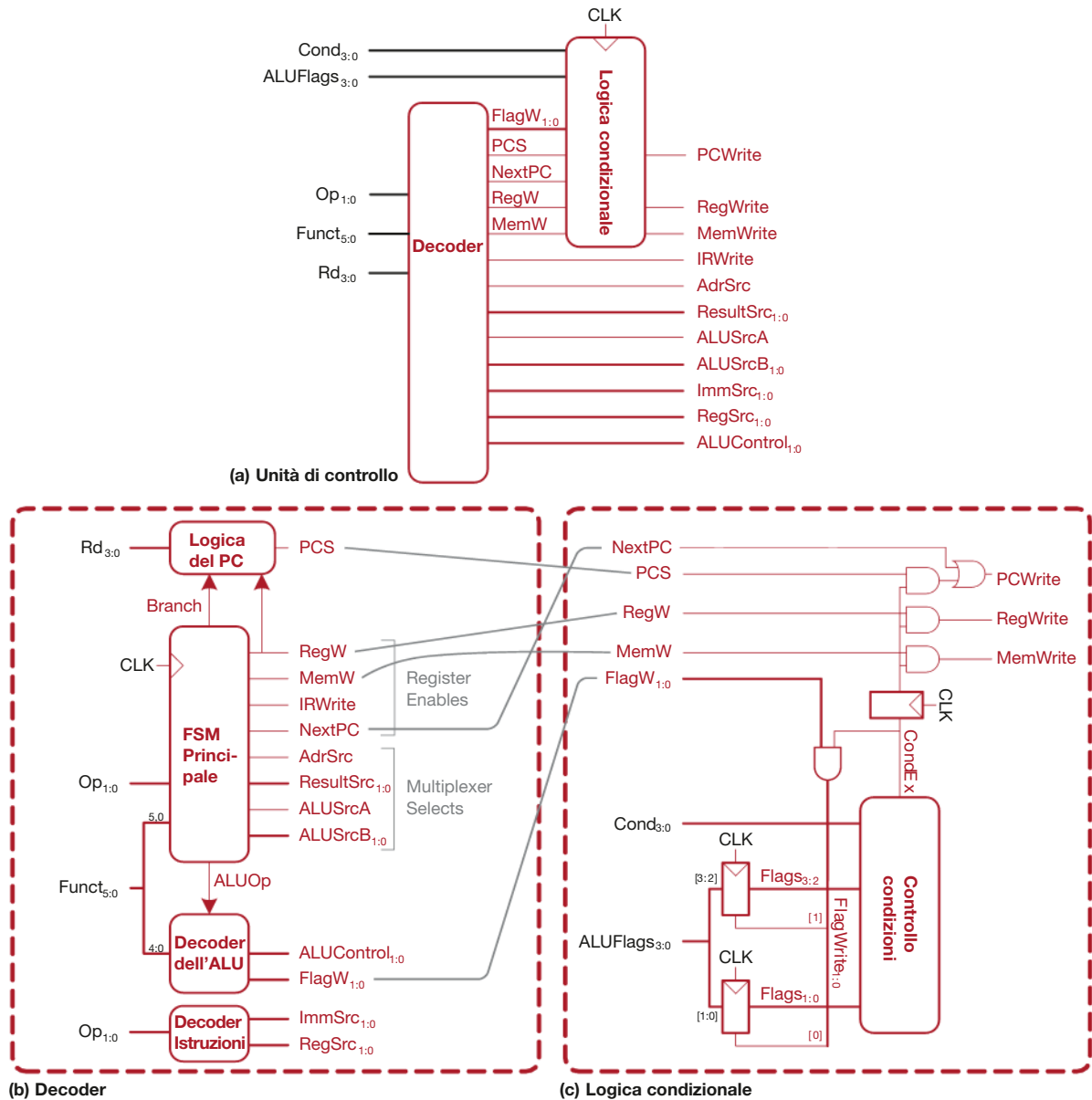
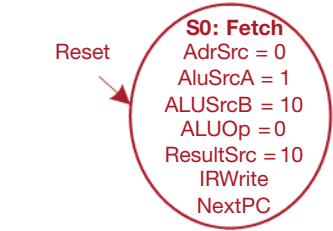


Figura 7.31 Unità di controllo multi ciclo.

le, che deve produrre la sequenza di segnali di controllo nei passi opportuni. Si progetta questa FSM come macchina di Moore, in modo che le sue uscite siano funzione del solo stato presente. Si vedrà però durante il progetto della FSM che *ImmSrc* e *RegSrc* sono funzione di *op* invece che dello stato presente, quindi si userà un piccolo Decoder Istruzioni per generare tali segnali, come descritto dalla **Tabella 7.6**. Il Decoder dell'ALU e la Logica del PC sono identici a quelli del processore a ciclo singolo. La Logica Condizionale è quasi uguale a quella del processore a ciclo singolo: serve solo il segnale aggiuntivo *NextPC* per forzare una scrittura nel PC quando si calcola  $PC + 4$ . Serve anche ritardare di un ciclo *Conde x* prima di inviare tale segnale a *PCWrite*, *RegWrite* e *MemWrite* in modo che le flag di condizione aggiornate non siano visibili fino al termine dell'istruzione corrente. Nel resto di questo paragrafo si sviluppa il diagramma degli stati della FSM Principale.



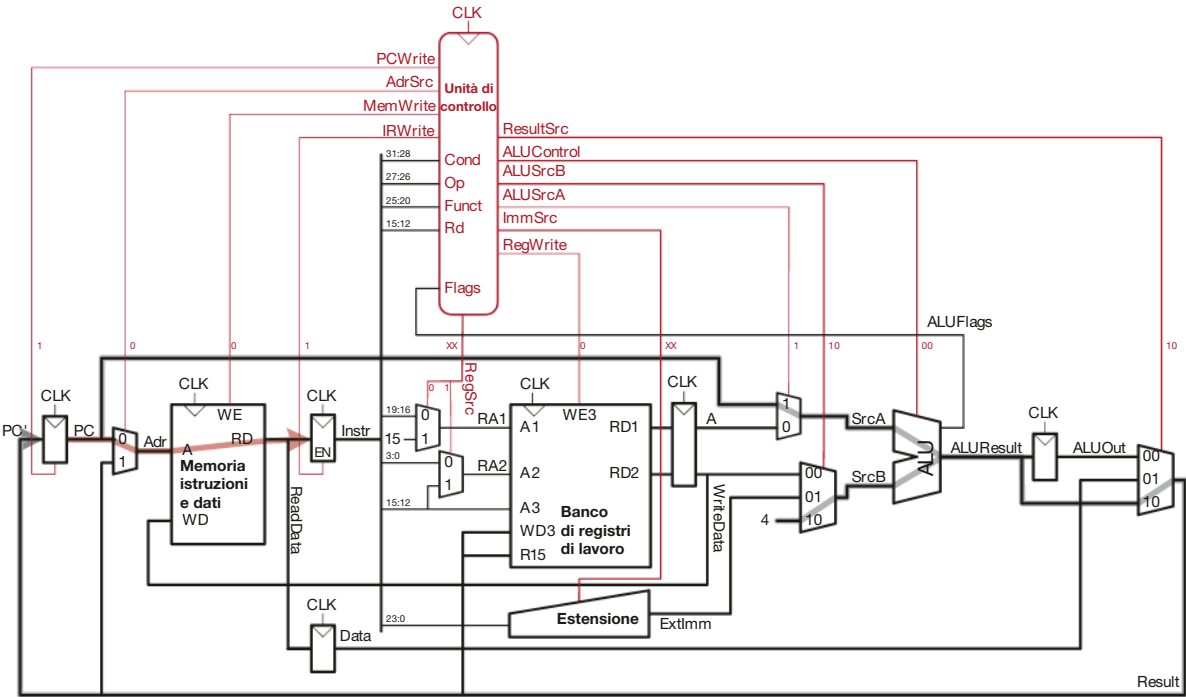
**Figura 7.32**  
Fetch.

**Tabella 7.6** Logica del Decoder istruzioni per *RegSrc* e *ImmSrc*.

| Istruzione    | Op | Funct <sub>5</sub> | Funct <sub>0</sub> | RegSrc <sub>1</sub> | RegSrc <sub>0</sub> | ImmSrc <sub>1:0</sub> |
|---------------|----|--------------------|--------------------|---------------------|---------------------|-----------------------|
| LDR           | 01 | X                  | 1                  | X                   | 0                   | 01                    |
| STR           | 01 | X                  | 0                  | 1                   | 0                   | 01                    |
| ED immediato  | 00 | 1                  | X                  | X                   | 0                   | 00                    |
| ED a registro | 00 | 0                  | X                  | 0                   | 0                   | 00                    |
| B             | 10 | X                  | X                  | X                   | 1                   | 10                    |

La FSM Principale deve generare i segnali di selezione dei multiplexer, le abilitazioni dei registri e i segnali di scrittura in memoria del percorso dati. Per avere un diagramma degli stati leggibile, si elencano solo i segnali di controllo rilevanti, cioè quelli i cui valori sono necessari per il funzionamento: tutti gli altri sono da considerare indifferenze. I segnali di abilitazione (*RegW*, *MemW*, *IRWrite* e *NextPC*) sono elencati solo quando devono essere attivati, cioè portati a 1: se non sono elencati si assume che valgano 0.

Il primo passo di ogni istruzione è il fetch dalla memoria all'indirizzo presente nel PC, e l'incremento del PC per puntare all'istruzione successiva. La FSM si porta in questo stato denominato Fetch *al reset*, e i segnali di controllo sono elencati nella **Figura 7.32**. Il flusso di dati in questo passo è mostrato nella **Figura 7.33**, con l'istruzione prelevata durante il fetch evidenziata in rosso e l'incremento del PC evidenziato in grigio. Per leggere dalla memoria, *AddrSrc* = 0, in modo che l'indirizzo sia preso dal PC. *IRWrite* è attivato per salvare l'istruzione del registro istruzioni IR. Nel mentre, il PC deve essere incrementato di 4 per puntare all'istruzione successiva. Dal momento che l'ALU non è utilizzata per altri scopi, il processore può adoperarla per calcolare *PC + 4* in parallelo alla fase di fetch: *ALUSrcA* = 1, in modo che *SrcA* provenga dal PC; *ALUSrcB* = 10, quindi *SrcB* è la costante 4. *ALUOp* = 0, quindi l'unità



**Figura 7.33** Flusso dei dati durante la fase di fetch.

di controllo genera  $ALUControl = 00$  per far effettuare all'ALU la somma. Per aggiornare il PC con il valore  $PC + 4$ ,  $ResultSrc = 10$  per selezionare  $ALUResult$  e  $NextPC = 1$  per abilitare  $PCWrite$ .

Il secondo passo è la lettura del banco di registri e/o dell'immediato e la decodifica delle istruzioni. I registri e l'immediato sono selezionati da  $RegSrc$  e  $ImmSrc$ , generati dal Decoder Istruzioni sulla base dei bit  $Instr$  dell'istruzione.  $RegSrc_0$  deve valere 1 per i salti per leggere  $PC + 8$  come  $SrcA$ .  $RegSrc_1$  deve essere 1 per le istruzioni di scrittura in memoria, per leggere come  $SrcB$  il valore da memorizzare.  $ImmSrc$  deve valere 00 per le istruzioni di elaborazione dati per selezionare un immediato a 8 bit, 01 per letture e scritture in memoria per selezionare un immediato a 12 bit, e 10 per i salti per selezionare un immediato a 24 bit. Dal momento che la FSM multi ciclo è una macchina di Moore, le cui uscite dipendono solo dallo stato presente, non è in grado di generare direttamente queste selezioni che dipendono dai bit  $Instr$ . Si potrebbe optare per una macchina alla Mealy, le cui uscite dipendono sia dallo stato presente sia dagli ingressi  $Instr$ , ma la cosa creerebbe confusione. Meglio adottare la soluzione più semplice, che consiste nel generare i segnali di selezione come funzioni combinatorie dei bit  $Instr$ , come elencato nella Tabella 7.6. Sfruttando le indifferenze, la logica del Decoder Istruzioni può essere semplificata nel modo seguente:

$$RegSrc_1 = (Op = 01)$$

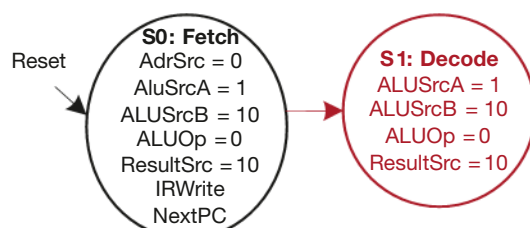
$$RegSrc_0 = (Op = 10)$$

$$ImmSrc_{1:0} = Op$$

Nel mentre, l'ALU viene usata per calcolare  $PC + 8$  sommando ancora 4 al PC già incrementato nello stato Fetch. I segnali di controllo sono generati in modo tale da selezionare il valore PC come primo ingresso dell'ALU ( $ALUSrcA = 1$ ) e 4 come secondo ingresso dell'ALU ( $ALUSrcB = 10$ ) e da attivare l'operazione di somma ( $ALUOp = 0$ ). La somma viene selezionata come  $Result$  ( $ResultSrc = 10$ ) e inviata all'ingresso R15 del banco di registri in modo tale che una lettura di R15 restituisca  $PC + 8$ . Il passo Decode della FSM è mostrato nella **Figura 7.34**, e il relativo flusso di dati nella **Figura 7.35**, che evidenzia il calcolo di R15 e la lettura dal banco dei registri.

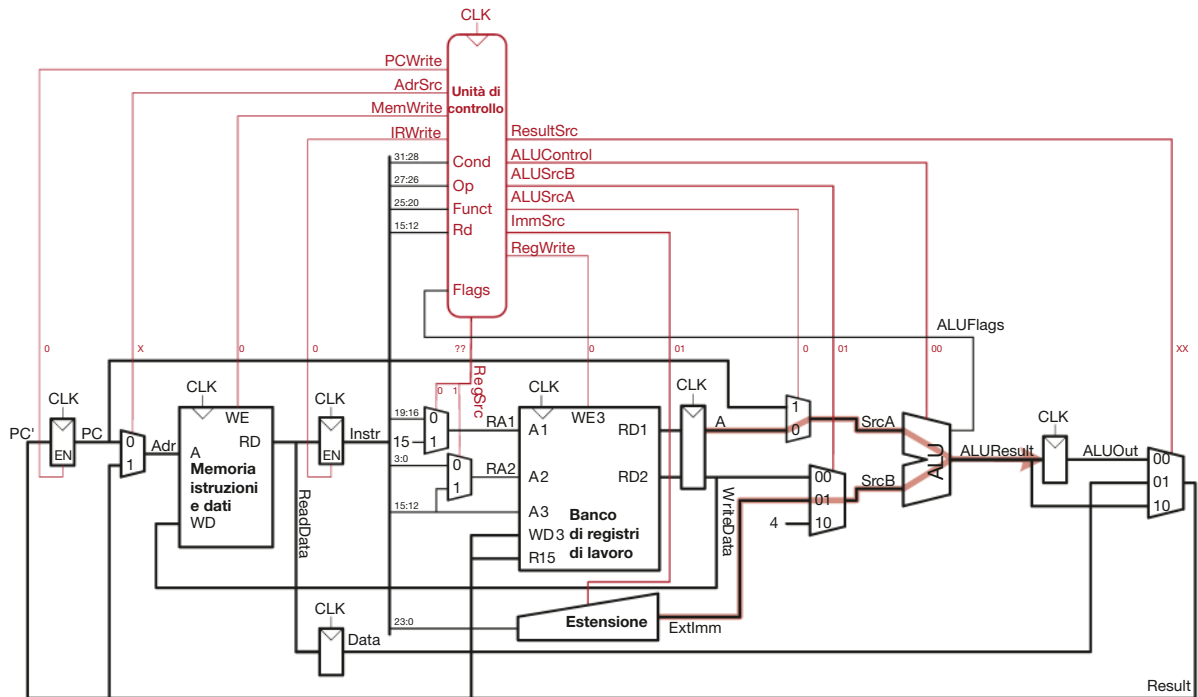
Ora la FSM procede a uno di diversi possibili stati, in base ai campi  $op$  e  $funct$  dell'istruzione esaminati durante il passo Decode. Se l'istruzione è un accesso a memoria (LDR oppure STR, quindi  $op = 01$ ) il processore multi ciclo deve calcolare l'indirizzo sommando all'indirizzo base lo spiazzamento esteso con zeri. Questo richiede  $ALUSrcA = 0$  per selezionare l'indirizzo base dal banco di registri,  $ALUSrcB = 01$  per selezionare  $ExtImm$  e  $ALUOp = 0$  per effettuare la somma. L'indirizzo calcolato viene memorizzato nel registro  $ALUOut$  per i passi successivi. Lo stato MemAdr della FSM è mostrato nella **Figura 7.36**, e il relativo flusso di dati è evidenziato nella **Figura 7.37**.

Se l'istruzione è LDR ( $funct_0 = 1$ ) il processore multi ciclo deve leggere un dato dalla memoria e scriverlo nel banco di registri: questi due passi sono mo-

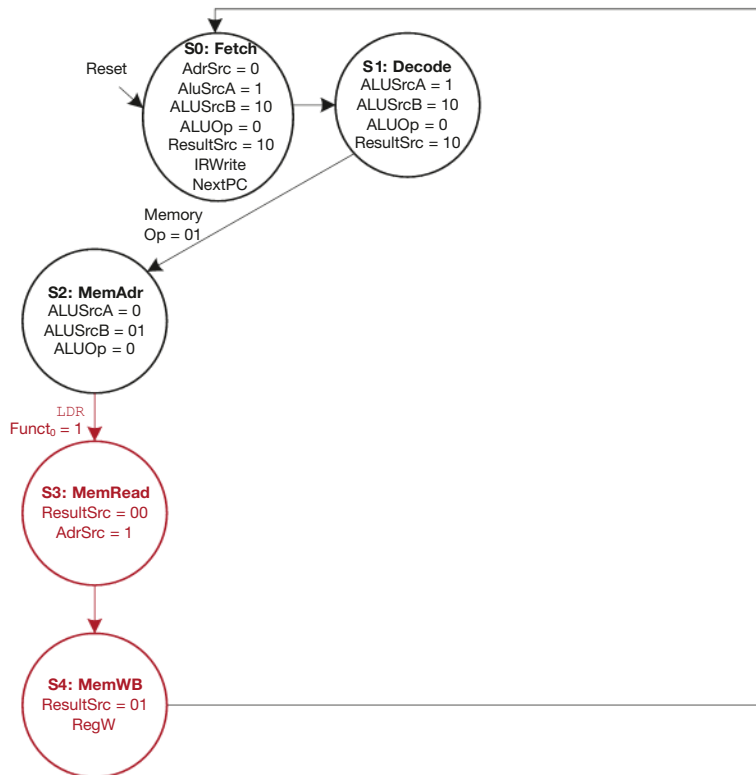


**Figura 7.34**  
Decodifica.





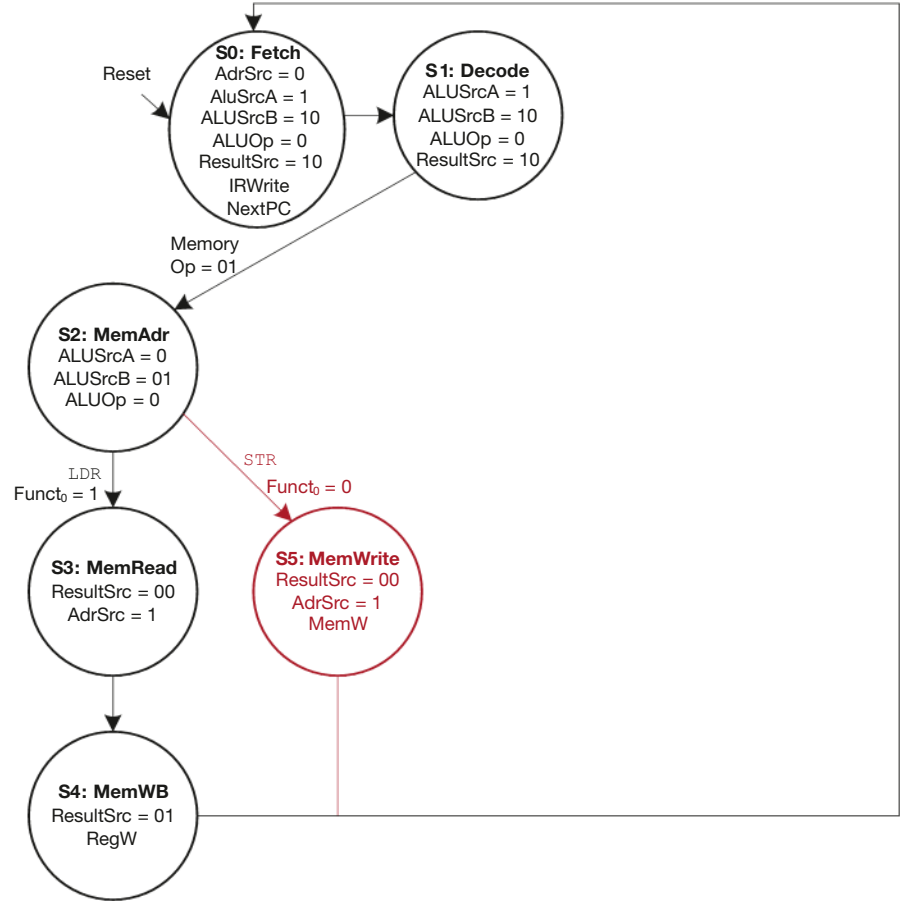
**Figura 7.37** Flusso dei dati durante il calcolo dell'indirizzo di memoria.



**Figura 7.38**  
**Lettura da memoria.**

banco di registri. *ResultSrc* = 01 per selezionare *Result* da *Data* e viene attivato *RegW* per scrivere nel banco di registri, completando l'esecuzione dell'istruzione *LDR*. Infine la FSM torna allo stato *Fetch* per iniziare l'istruzione successiva. Si lascia al lettore il compito di visualizzare il flusso di dati per questi passi e per quelli introdotti nel seguito.

**Figura 7.39**  
Scrittura in memoria.



Dallo stato MemAdr, se l'istruzione è STR ( $funct_0 = 0$ ) il dato letto dalla seconda porta del banco di registri deve essere semplicemente scritto in memoria. Si va quindi nello stato MemWrite, con  $ResultSrc = 00$  e  $AdrSrc = 1$  per selezionare l'indirizzo calcolato nello stato MemAdr e salvato in  $ALUOut$ .  $MemW$  viene attivato per scrivere in memoria, e la FSM torna di nuovo nello stato Fetch. Lo stato MemWrite è mostrato nella **Figura 7.39**.

Per le istruzioni di elaborazione dati ( $Op = 00$ ) il processore multi ciclo deve calcolare il risultato usando l'ALU e scriverlo nel banco di registri. Il primo operando sorgente proviene sempre da un registro ( $ALUSrcA = 0$ ).  $ALUOp = 1$ , in modo che il Decoder dell'ALU possa selezionare il valore appropriato di  $ALUControl$  per la specifica istruzione usando il campo  $cmd$  (bit  $funct_{4:1}$ ). Il secondo operando sorgente proviene dal banco di registri per istruzioni con modo di indirizzamento a registro ( $ALUSrcB = 00$ ) oppure da  $ExtImm$  per istruzioni con modo di indirizzamento immediato ( $ALUSrcB = 01$ ). Quindi la FSM ha bisogno dei due stati ExecuteR ed ExecuteI per gestire le due diverse situazioni. In entrambi i casi, l'istruzione di elaborazione dati avanza poi allo stato ALUWB di scrittura del risultato dell'ALU, nel quale il risultato del calcolo viene selezionato da  $ALUOut$  ( $ResultSrc = 00$ ) e scritto nel banco di registri ( $RegW = 1$ ). Tutti questi stati sono mostrati nella **Figura 7.40**.

Per l'istruzione di salto (*branch*), il processore deve calcolare l'indirizzo di destinazione ( $PC + 8 +$  spiazzamento) e scriverlo nel PC. Durante il passo associato allo stato Decode,  $PC + 8$  era già stato calcolato e portato al banco di registri come  $RD1$ . Quindi nello stato Branch l'unità di controllo usa  $ALUSrcA = 0$  per selezionare  $R15$  ( $PC + 8$ ),  $ALUSrcB = 01$  per selezionare  $ExtImm$  e  $ALUOp = 0$  per effettuare la somma. Il multiplexer che produce  $Result$  se-

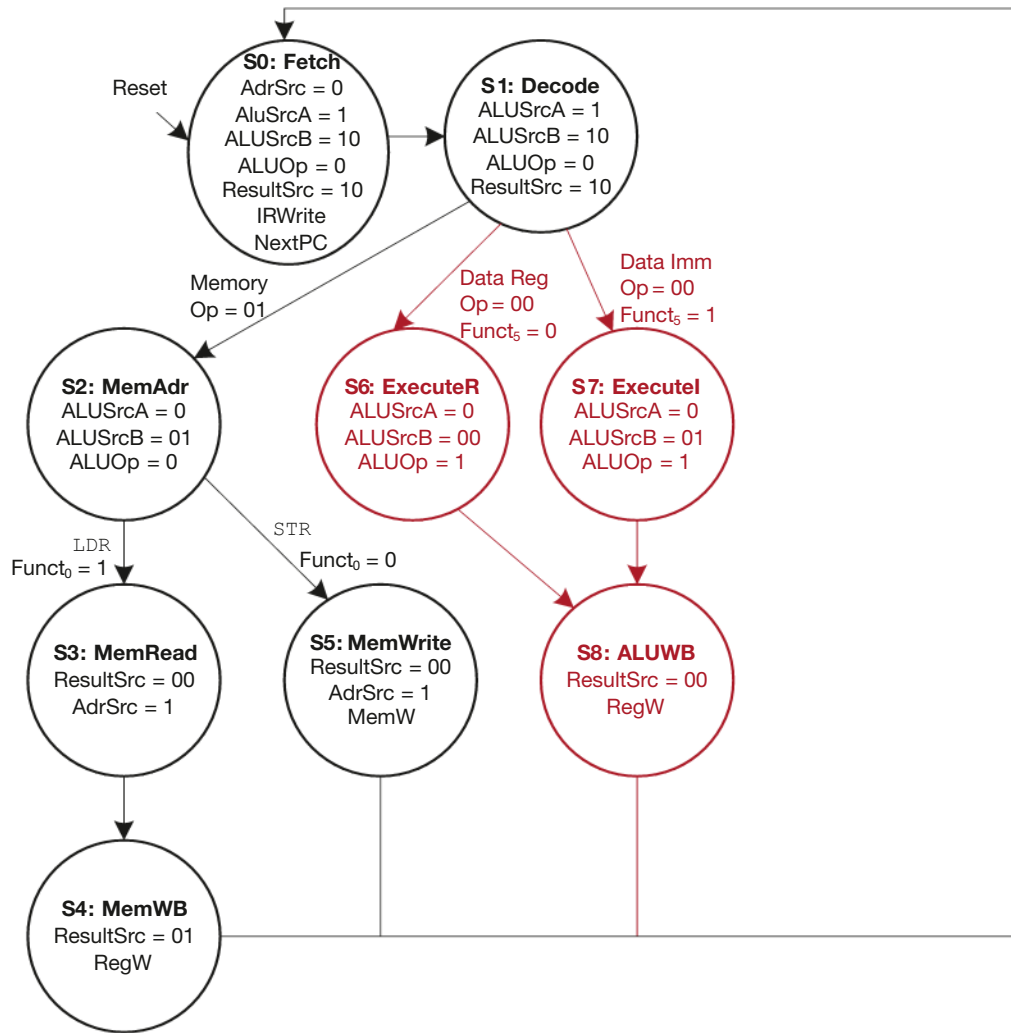


Figura 7.40 Elaborazione dati.

leziona *ALUResult* (*ResultSrc* = 10). *Branch* è attivato per scrivere il risultato nel PC.

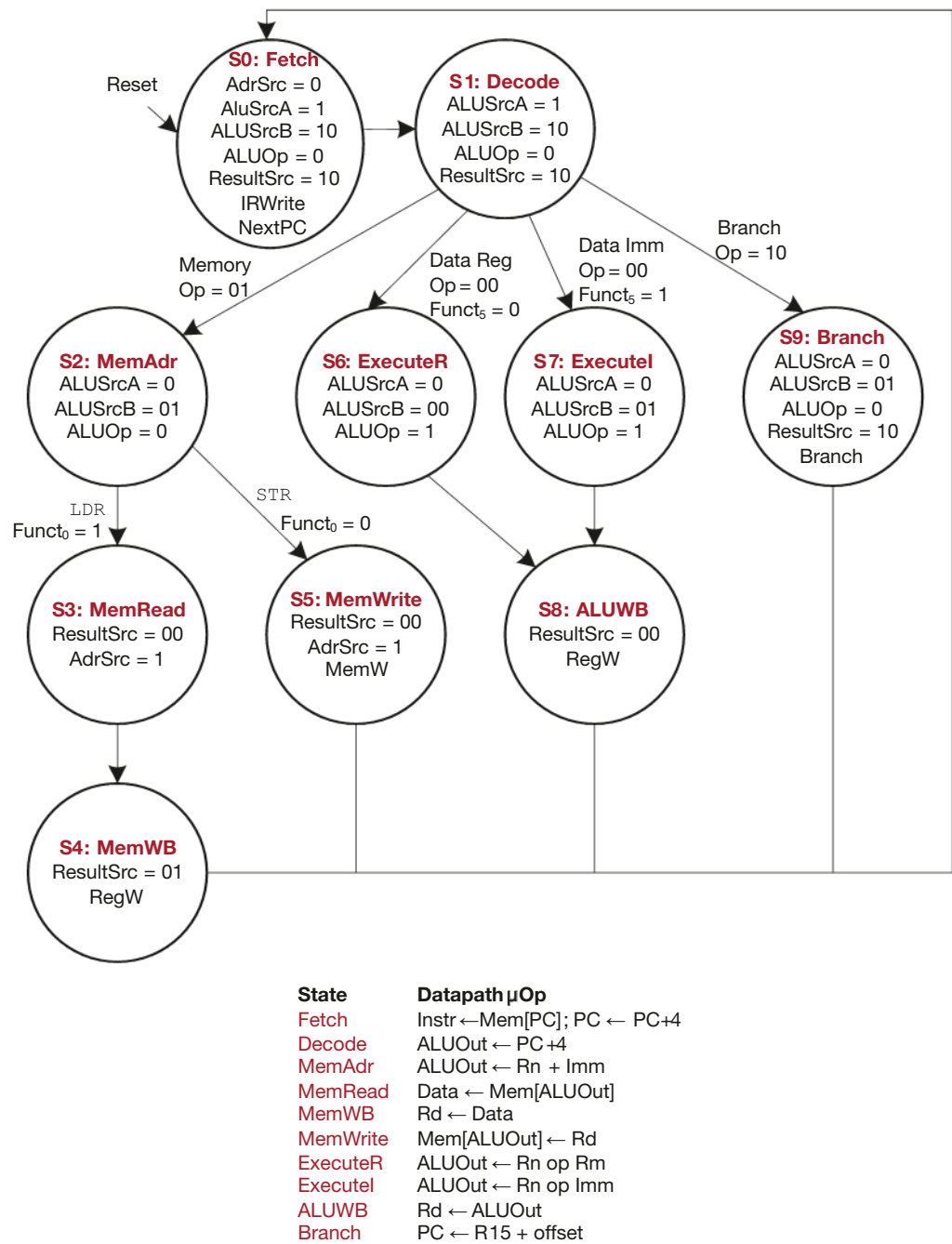
Mettendo tutto insieme si ottiene il diagramma degli stati completo per la FSM Principale per il processore multi ciclo riportato nella **Figura 7.41**. La funzione di ogni stato è riassunta sotto la figura. Trasformare tale diagramma in hardware usando i metodi visti nel Capitolo 3 è banale ma molto lungo: meglio codificare la FSM in un HDL e sintetizzarla usando le tecniche viste nel Capitolo 4.

### 7.4.3 Analisi delle prestazioni

Il tempo di esecuzione di un'istruzione dipende dal tempo di ciclo e dal numero di cicli necessari all'istruzione stessa. Mentre il processore a ciclo singolo eseguiva tutte le istruzioni in un solo ciclo, il processore multi ciclo usa numeri variabili di cicli per le diverse istruzioni, però questo processore svolge meno attività in ogni ciclo, quindi ha senz'altro un tempo di ciclo inferiore.

Il processore multi ciclo richiede tre cicli per i salti, quattro per le istruzioni di elaborazione dati e di scrittura in memoria e cinque per le letture da memoria. Il CPI dipende quindi dalla probabilità relativa di utilizzo di ciascuna tipologia di istruzioni nel programma.





**Figura 7.41** FSM completa dell'unità di controllo multi ciclo.

**ESEMPIO 7.5**

**CPI del processore multi ciclo.** Il benchmark SPECINT2000 è costituito all'incirca da 25% di istruzioni di lettura da memoria, 10% di scritture in memoria, 13% di salti e 52% di istruzioni di elaborazione dati.<sup>2</sup> Determinare il CPI medio per questo benchmark.

**Soluzione** Il CPI medio è la somma per tutte le istruzioni del CPI di ogni istruzione moltiplicato per la frazione di tempo per la quale l'istruzione è usata. Per questo benchmark si ottiene quindi:  $CPI_{medio} = (0.13) \times 3 + (0.52 + 0.10) \times 4 + (0.25) \times 5 = 4.12$ . Tale valore è migliore del caso pessimo di  $CPI = 5$ , che si avrebbe se tutte le istruzioni richiedessero il tempo più lungo per essere eseguite.

Si deve ricordare che si è progettato un processore multi ciclo nel quale ogni ciclo implica un'operazione dell'ALU, un accesso a memoria o un accesso al banco di registri. Si assuma che il banco di registri sia più veloce della memoria, e che scrivere in memoria sia più veloce di leggere. L'esame del percorso dati mostra due possibili percorsi critici che possono determinare il limite inferiore del tempo di ciclo:

1. dal PC attraverso il multiplexer *SrcA*, l'ALU e il multiplexer del risultato fino alla porta R15 del banco di registri;
2. da *ALUOut* attraverso i multiplexer *Result* e *Adr* fino alla lettura da memoria nel registro *Data*

$$T_{c2} = t_{pcq} + 2t_{mux} + \max[t_{ALU} + t_{mux}, t_{mem}] + t_{setup} \quad (7.4)$$

I valori numerici di questi tempi dipendono naturalmente dalla specifica tecnologia di realizzazione.

**ESEMPIO 7.6**

**Confronto delle prestazioni dei processori.** Ben Imbrogliabit non è sicuro che il processore multi ciclo sia davvero più veloce di quello a ciclo singolo. Per entrambi fa riferimento alla tecnologia CMOS a 16 nanometri, i cui ritardi sono elencati nella Tabella 7.5. Bisogna aiutarlo a confrontare le prestazioni di entrambi i processori nell'esecuzione di 100 miliardi di istruzioni del benchmark SPECINT2000 citato nell'Esempio 7.5.

**Soluzione** In base all'Espressione 7.4, il tempo di ciclo del processore multi ciclo è  $T_{c2} = 40 + 2(25) + 200 + 50 = 340$  ps. Usando il CPI di 4.12 dell'Esempio 7.5, il tempo totale di esecuzione risulta essere  $T_2 = (100 \times 10^9 \text{ istruzioni}) (4.12 \text{ cicli/istruzione}) (340 \times 10^{-12} \text{ s/ciclo}) = 140$  secondi. Dall'Esempio 7.4 risulta che il processore a ciclo singolo aveva un tempo totale di esecuzione  $T_1 = 84$  secondi.

Una delle motivazioni principali per passare a un processore multi ciclo era quella di evitare che tutte le istruzioni durassero il tempo della più lenta. Ma questo esempio mostra che, con le ipotesi di CPI e di ritardi considerate, il processore multi ciclo è più lento di quello a ciclo singolo. Il motivo principale è legato al fatto che anche se l'istruzione più lenta (cioè *LDR*) è stata divisa in cinque passi, il tempo di ciclo del processore multi ciclo non è migliorato di cinque volte. Questo è dovuto in parte al fatto che non tutti i passi hanno la stessa durata, in parte al fatto che il sovraccarico di sequenziamento di 90 ps dovuto al ritardo dal clock all'uscita e al ritardo di setup dei registri è presente in questo caso in tutti i passi, e non solo una volta per l'intera istruzione come per il processore a ciclo singolo. In generale, si vede che è difficile

<sup>2</sup> Dati tratti dal testo Patterson e Hennessy, *Computer Organization and Design*, 4<sup>a</sup> edizione, Morgan Kaufmann, 2011.

sfruttare il fatto che alcune elaborazioni sono più veloci di altre, a meno che le differenze siano molto grandi.

In confronto al processore a ciclo singolo, il multi ciclo è però molto probabilmente meno costoso, perché unifica la memoria istruzioni e dati e perché elimina due circuiti sommatori. Introduce tuttavia cinque registri non architetturali e alcuni multiplexer aggiuntivi.

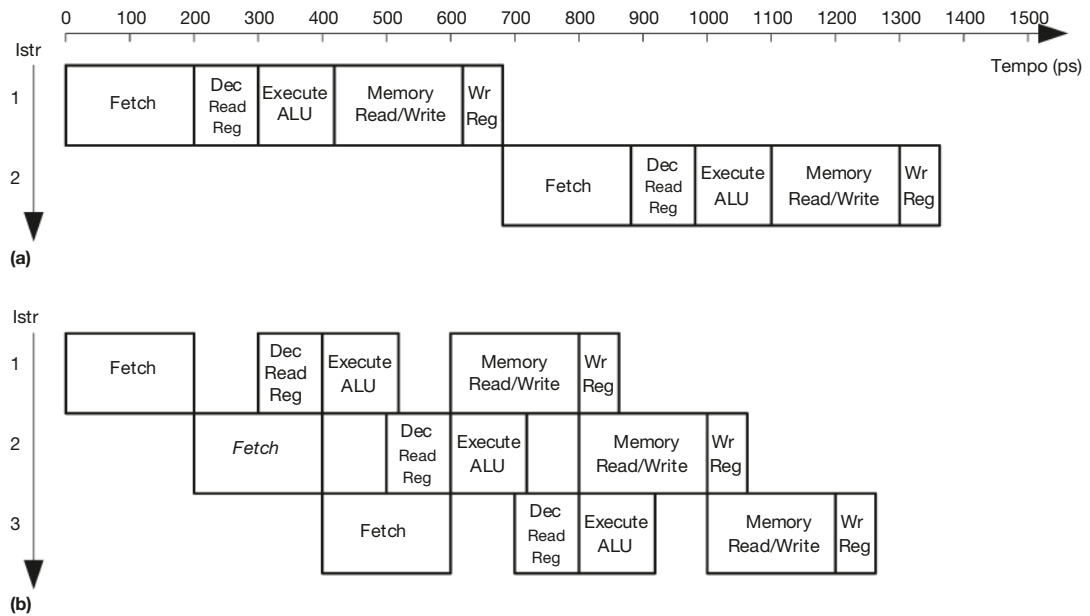
## 7.5 ■ PROCESSORE PIPELINE

L'adozione di strutture pipeline, introdotte nel paragrafo 3.6, è una tecnica molto efficace per aumentare le prestazioni di un sistema digitale. Si progetta qui un processore pipeline suddividendo il processore a ciclo singolo in cinque stadi di pipeline. In questo modo, cinque istruzioni alla volta possono essere in esecuzione, una per ogni stadio. Dal momento che ogni stadio ha circa un quinto della logica totale, la frequenza di clock dovrebbe risultare circa cinque volte superiore. Quindi la latenza di ogni istruzione (cioè il tempo che passa dall'inizio del fetch dell'istruzione al termine della sua esecuzione) rimane quasi inalterata, ma la capacità di lavoro (*throughput*) svolto dal processore è idealmente cinque volte superiore. I microprocessori eseguono milioni o miliardi di istruzioni al secondo, quindi la capacità di lavoro è un parametro molto più importante della latenza. In realtà l'adozione di una struttura pipeline richiede attività aggiuntive, per cui la capacità di lavoro non cresce come ci si potrebbe idealmente attendere, ma si hanno comunque vantaggi tali a costi così bassi che tutti i microprocessori moderni adottano una struttura pipeline.

Lettura e scrittura in memoria e nel banco di registri sono le attività che introducono i maggiori ritardi nel processore. Si è deciso qui di adottare una pipeline a cinque stadi proprio perché in questo modo ogni stadio coinvolge una sola di queste attività lente. I cinque stadi sono denominati Fetch, Decode, Execute, Memory e Writeback. Sono simili ai cinque passi che il processore multi ciclo svolge per eseguire l'istruzione LDR. Nello stadio Fetch il processore legge l'istruzione dalla memoria; nello stadio Decode legge gli operandi dal banco di registri e decodifica l'istruzione per generare i segnali di controllo appropriati; nello stadio Execute esegue i calcoli con l'ALU; nello stadio Memory legge o scrive dati in memoria; infine nello stadio Writeback scrive il risultato nel banco di registri, quando previsto dall'istruzione.

La **Figura 7.42** mostra un diagramma temporale che confronta il processore a ciclo singolo con il processore pipeline. Il tempo è sull'asse orizzontale, e le istruzioni su quello verticale. Il diagramma fa riferimento ai ritardi riportati nella Tabella 7.5 ma trascura i ritardi introdotti dai multiplexer e dai registri. Nel processore a ciclo singolo (Figura 7.42(a)) la prima istruzione viene letta (fetch) dalla memoria istruzioni al tempo 0; poi si leggono gli operandi dal banco di registri; quindi l'ALU esegue l'operazione richiesta; infine può essere necessario accedere alla memoria dati per leggere o scrivere un dato, e il risultato viene salvato nel banco registri in un tempo totale di 680 ps. La seconda istruzione comincia quando la prima è finita, quindi sulla base di questo diagramma il processore a ciclo singolo ha una latenza di  $200 + 100 + 120 + 200 + 60 = 680$  ps e una capacità di lavoro pari a 1 istruzione ogni 680 ps (cioè 1.47 miliardi di istruzioni al secondo).

Nel processore pipeline, la durata di ogni stadio di pipeline è forzata a 200 ps dall'attività più lenta, cioè l'accesso a memoria (negli stadi Fetch e Memory). All'istante 0 lo stadio Fetch inizia la lettura della prima istruzione. Dopo 200 ps la prima istruzione entra nello stadio Decode e può iniziare il fetch della seconda istruzione. Dopo 400 ps la prima istruzione entra nello stadio Execute, la seconda entra nello stadio Decode e si può iniziare il fetch della

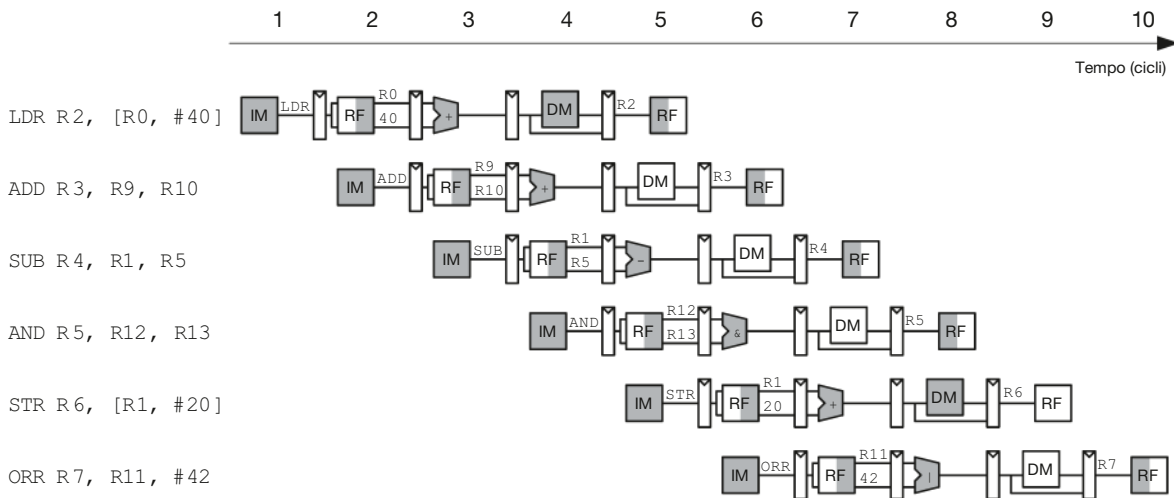


**Figura 7.42** Diagrammi dei tempi: (a) processore a ciclo singolo e (b) processore pipeline.

terza. E così via fino all'ultima istruzione. La latenza di ogni istruzione è  $5 \times 200 = 1000$  ps. La capacità di lavoro è 1 istruzione ogni 200 ps (5 miliardi di istruzioni al secondo). Dal momento che gli stadi non sono perfettamente bilanciati, la latenza nel processore pipeline è maggiore rispetto a quella nel processore a ciclo singolo, e analogamente la capacità di lavoro del processore con cinque stadi di pipeline non è cinque volte quella del processore a ciclo singolo, ma i vantaggi sono comunque notevoli.

La **Figura 7.43** mostra una rappresentazione schematica del funzionamento della pipeline con ogni stadio disegnato con un simbolo. In ogni stadio viene indicato il componente principale relativo a tale stadio – ovvero memoria istruzioni (IM, *Instruction Memory*), lettura dal banco di registri (RF, *Register File*), ALU, memoria dati (DM, *Data Memory*), scrittura finale nel banco di registri (di nuovo RF) – per illustrare il flusso di istruzioni attraverso la pipeline. Ogni riga fa vedere i cicli di clock nei quali l'istruzione corrispondente si trova nei vari stadi: per esempio, è stato fatto il fetch dell'istruzione SUB nel ciclo 3, ed è stata eseguita nel ciclo 5. Ogni colonna fa vedere cosa stanno facendo i vari stadi della pipeline in ogni ciclo di clock: per esempio, nel ciclo 6, si sta facendo il fetch dell'istruzione ORR, si sta leggendo R1 dal banco di registri, l'ALU sta calcolando  $R12 \text{ AND } R13$ , la memoria dati è inattiva e si sta scrivendo il risultato di una somma nel registro R3 del banco di registri. Gli stadi sono colorati di grigio quando vengono effettivamente utilizzati: per esempio, la memoria dati è usata da LDR nel ciclo 4 e da STR nel ciclo 8; la memoria istruzioni e l'ALU sono utilizzate in ogni ciclo; il banco di registri viene modificato da tutte le istruzioni tranne STR. Nel processore pipeline si scrive nel banco di registri nella prima parte di un ciclo, e si legge nella seconda parte del ciclo, come evidenziato dalla colorazione in grigio. È quindi possibile scrivere un valore nel banco di registri e rileggerlo nello stesso ciclo.

Un grosso problema presente nei sistemi pipeline è la gestione delle **dipendenze**, che si verificano se i risultati di un'istruzione servono a un'istruzione successiva quando ancora l'istruzione che li deve produrre non è terminata. Per esempio, se l'istruzione ADD della Figura 7.43 avesse usato R2 invece di R10, si sarebbe verificata una dipendenza perché il registro R2 non



**Figura 7.43** Rappresentazione schematica del funzionamento della pipeline.

è stato ancora caricato dall'istruzione `LDR` nel momento in cui l'istruzione `ADD` lo va a leggere. Dopo il progetto del percorso dati e dell'unità di controllo per processore pipeline, questa sezione analizza le tecniche di inoltro, stallo e svuotamento usate per risolvere il problema delle dipendenze. Da ultimo si rivede l'analisi delle prestazioni per tenere conto del sovraccarico dovuto al sequenziamento e dell'impatto delle dipendenze.

### 7.5.1 Percorso dati pipeline

Il percorso dati del processore pipeline è ottenuto dividendo il percorso dati del processore a ciclo singolo in cinque stadi, separati da registri di pipeline.

La **Figura 7.44(a)** mostra il percorso dati del processore a ciclo singolo “stirato” in orizzontale per lasciare spazio ai registri di pipeline, mentre la **Figura 7.44(b)** mostra il percorso dati del processore pipeline ottenuto inserendo quattro registri di pipeline per separare il percorso dati in cinque stadi. Gli stadi e i loro confini sono evidenziati in rosso. Ai segnali è stato aggiunto un suffisso (F, D, E, M o W) per indicare in quale stadio tali segnali sono contenuti.

Il banco di registri è un caso particolare, perché viene letto nello stadio Decode e scritto nello stadio Writeback. È stato disegnato nello stadio Decode, ma l'indirizzo di scrittura e il dato da scrivere provengono dallo stadio Writeback. Questo collegamento all'indietro (o retroazione) è la causa delle dipendenze, discusse nel paragrafo 7.5.3. Il banco di registri nel processore pipeline viene modificato in corrispondenza del fronte di discesa del clock in modo che si possa scrivere un risultato nella prima metà del ciclo e rileggere lo stesso risultato nella seconda metà dello stesso ciclo per poterlo utilizzare in istruzioni successive.

Uno degli aspetti critici nelle pipeline è il fatto che tutti i segnali associati a una particolare istruzione devono procedere all'unisono attraverso la *pipeline*. In questo senso, la Figura 7.44(b) contiene un errore nella logica di scrittura del banco di registri, da eseguire nello stadio Writeback: il dato da scrivere proviene da *ResultW*, che è un segnale dello stadio Writeback, ma l'indirizzo del registro in cui scrivere proviene da *InstrD<sub>15:12</sub>* (cioè da *WA3D*), che fa parte dello stadio Decode; quindi, facendo riferimento alla sequenza di istruzioni della Figura 7.43, durante il ciclo 5 il risultato dell'istruzione `LDR` verrebbe scritto erroneamente in R5 invece che in R2.

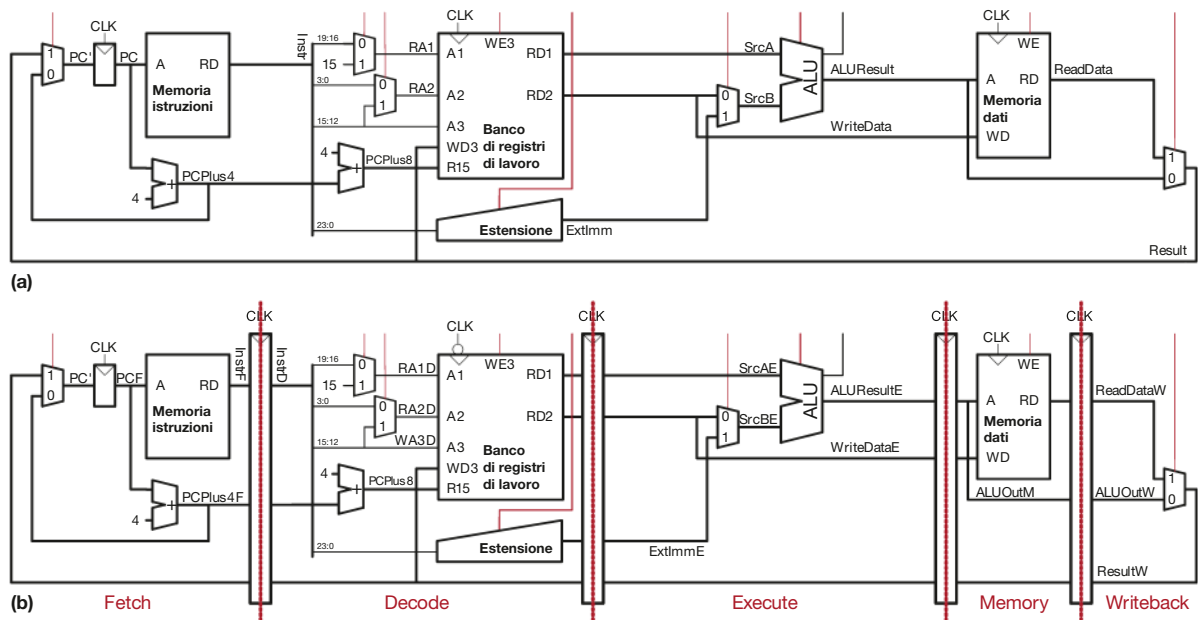


Figura 7.44 Diversi percorsi dati: (a) a ciclo singolo e (b) pipeline.

La **Figura 7.45** mostra il percorso dati corretto, con la modifica evidenziata in nero: il segnale *WA3* attraversa ora la pipeline passando per gli stadi *Execute*, *Memory* e *Writeback*, in modo tale da rimanere sincronizzato con il resto dell'istruzione. Così facendo *WA3W* e *ResultW* sono correttamente inviati insieme al banco di registri nello stadio *Writeback*.

Il lettore attento si sarà accorto che anche la logica di generazione di *PC'* crea problemi, perché il PC potrebbe essere modificato da segnali sia dello stadio *Fetch* sia dello stadio *Writeback* (*PCPlusF* e *ResultW*). Questo problema (dipendenza di controllo) è discusso nel paragrafo 7.5.3.

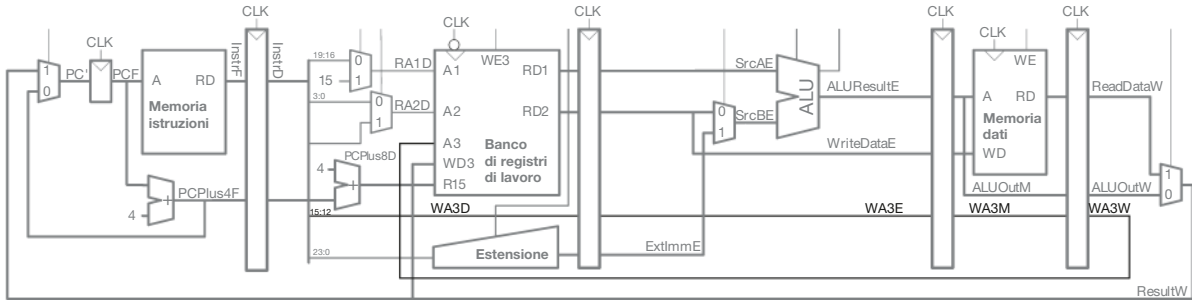
La **Figura 7.46** mostra un'altra ottimizzazione per risparmiare un sommatore a 32 bit e un registro nella Logica del PC. Si può notare nella Figura 7.45 che, ogni volta che il program counter viene incrementato, *PCPlus4F* viene scritto sia nel registro PC sia nel registro di pipeline tra gli stadi *Fetch* e *Decode*; inoltre, nel ciclo successivo, il valore presente in entrambi i registri viene nuovamente incrementato di 4. Quindi *PCPlus4F* per l'istruzione nello stadio *Fetch* è logicamente equivalente a *PCPlus4D* nello stadio *Decode*: se si propaga in avanti questo segnale si risparmia un registro di pipeline e il secondo sommatore.<sup>3</sup>

## 7.5.2 Unità di controllo della pipeline

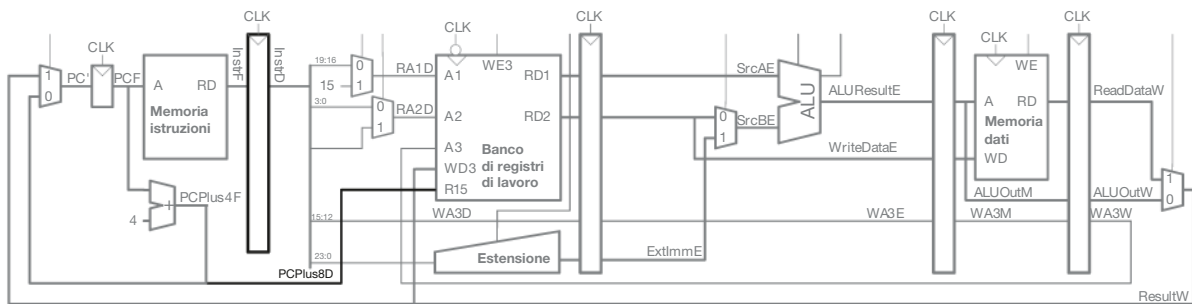
Il processore pipeline usa gli stessi segnali di controllo del processore a ciclo singolo, quindi ha la stessa unità di controllo, che prende in considerazione i campi *op* e *funct* dell'istruzione nello stadio *Decode* per generare i segnali di controllo, come discusso nel paragrafo 7.3.2. Tali segnali devono essere propagati nella pipeline insieme ai dati per rimanere sincronizzati con l'istruzione cui si riferiscono. L'unità di controllo esamina anche il campo *Rd* per gestire le scritture nel registro R15 (PC).

<sup>3</sup> C'è un possibile problema in questa semplificazione, che si verifica quando il PC viene aggiornato da *ResultW* invece che da *PCPlus4F*. Si tratta però del caso della dipendenza di controllo discusso nel paragrafo 7.5.3, che si risolve svuotando la pipeline: a quel punto *PCPlus8D* diventa una indifferenza e la pipeline funziona correttamente.

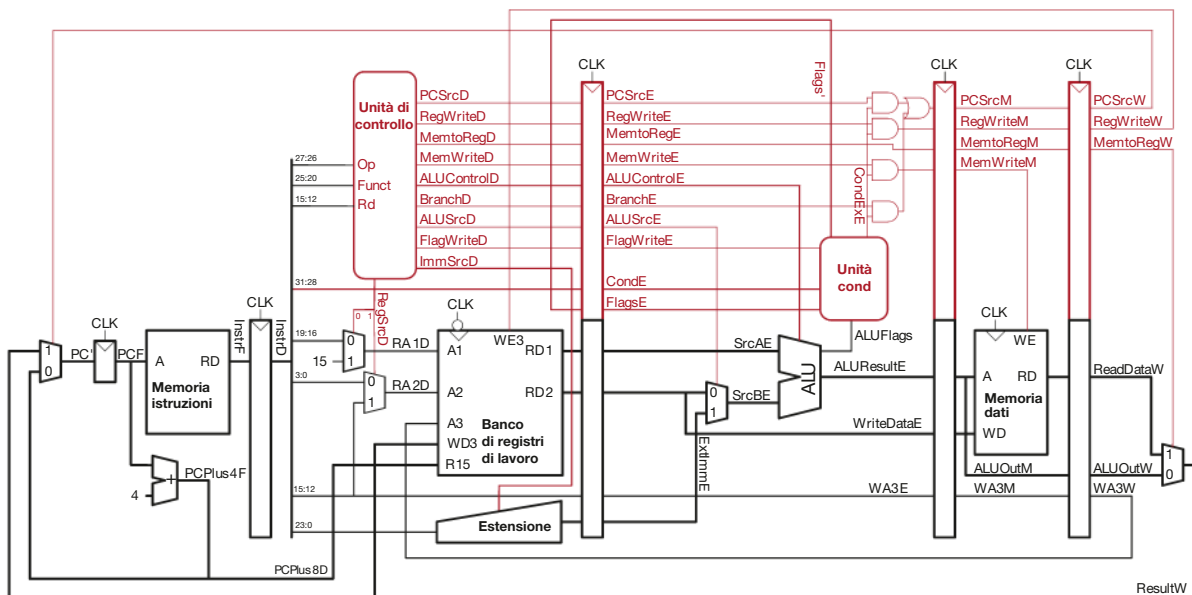
L'intero processore pipeline con l'unità di controllo è mostrato nella **Figura 7.47**. *RegWrite* deve essere propagato nella pipeline fino allo stadio Writeback prima di essere inviato al banco di registri, allo stesso modo in cui era stato propagato *WA3* nella Figura 7.45.



**Figura 7.45** Percorso dati pipeline corretto.



**Figura 7.46** Logica del PC ottimizzata con eliminazione di un registro e del sommatore.



**Figura 7.47** Processore pipeline con controllo.



### 7.5.3 Dipendenze

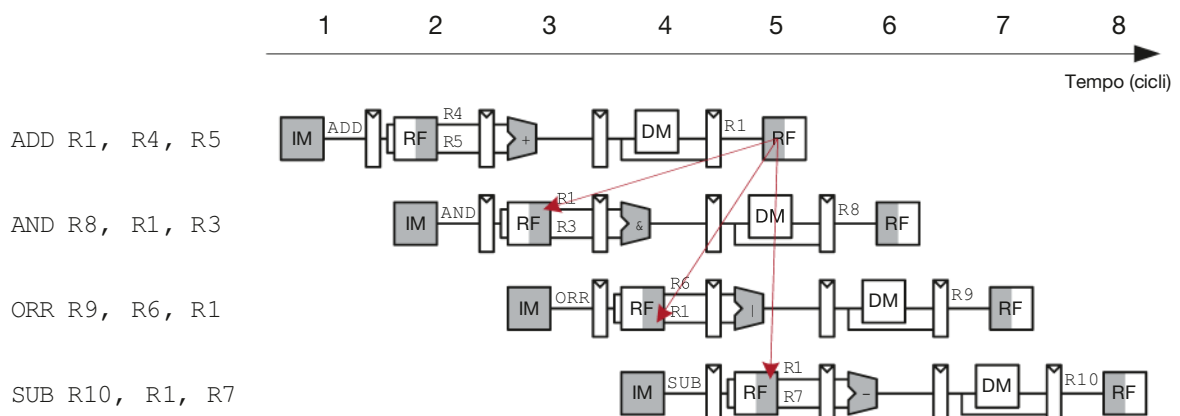
In una struttura pipeline, più istruzioni sono eseguite in modo concorrente. Si verifica **dipendenza** (*hazard*) quando un'istruzione dipende dai risultati di un'istruzione che la precede e che non è ancora conclusa.

Il banco di registri può essere modificato e letto nel medesimo ciclo. La scrittura che lo modifica ha luogo nella prima metà del ciclo, mentre la lettura nella seconda metà, quindi un registro può essere modificato e riletto nello stesso ciclo senza introdurre dipendenze.

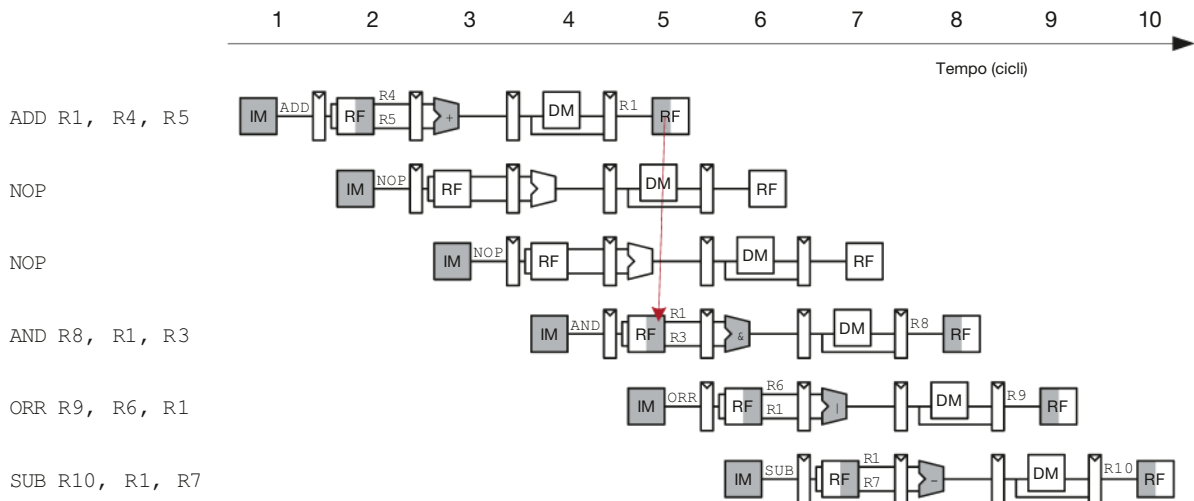
La **Figura 7.48** mostra le dipendenze che si verificano quando un'istruzione scrive in un registro (R1) che deve essere letto da istruzioni successive. Questo tipo di dipendenza viene denominato **RAW** (*Read After Write*, lettura dopo la scrittura). L'istruzione **ADD** scrive il proprio risultato in R1 nella prima metà del ciclo 5, ma l'istruzione **AND** legge R1 nel ciclo 3, ottenendo quindi un valore scorretto; l'istruzione **ORR** legge R1 nel ciclo 4, ottenendo anche lei un valore scorretto; invece l'istruzione **SUB** legge R1 nella seconda metà del ciclo 5, ottenendo il valore corretto che era stato scritto in R1 nella prima metà dello stesso ciclo, e come lei tutte le istruzioni successive. La rappresentazione schematica mostra quindi che la dipendenza si verifica quando un'istruzione scrive in un registro che deve essere letto da una delle due istruzioni successive. Senza opportuni provvedimenti, la pipeline produrrebbe quindi risultati sbagliati.

Una soluzione a livello software sarebbe quella di inserire (a cura del programmatore o del compilatore) due istruzioni **NOP** tra **ADD** e **AND** in modo che l'istruzione che ha dipendenza non vada a leggere R1 finché non sia stato reso disponibile nel banco di registri, come mostrato nella **Figura 7.49**. Ma questo approccio complica non poco la programmazione e degrada le prestazioni, quindi non è certo la soluzione ideale.

Se però si esamina più attentamente la **Figura 7.48**, si può notare che la somma dell'istruzione **ADD** viene eseguita dall'ALU nel ciclo 3, e non serve all'istruzione **AND** finché l'ALU non userà il risultato di tale somma nel ciclo 4. In linea di principio si è quindi in grado di inoltrare il risultato di un'istruzione alla successiva per risolvere le dipendenze di tipo RAW senza aspettare che tale risultato venga scritto nel banco di registri e senza quindi degradare le prestazioni. In altri casi esaminati più avanti in questo paragrafo può essere necessario mettere in stallo la pipeline per lasciare il tempo a un'istruzione di produrre un risultato prima che istruzioni successive lo usino. In tutte queste situazioni è comunque necessario agire per risolvere le dipendenze, affinché il programma sia eseguito correttamente nonostante il processore sia strutturato pipeline.



**Figura 7.48** Rappresentazione schematica della pipeline con illustrazione delle dipendenze.



**Figura 7.49** Risoluzione della dipendenza di dato con l'istruzione NOP.

Le dipendenze sono classificate in dipendenze di dato e dipendenze di controllo. Una **dipendenza di dato** si verifica quando un'istruzione vuole leggere un registro che non è ancora stato aggiornato da un'istruzione precedente. Una **dipendenza di controllo** si verifica quando la decisione di quale istruzione debba essere prelevata da memoria nella fase di fetch non è ancora stata presa al momento del fetch. Nel resto del paragrafo si estende la struttura del processore pipeline con un'unità di gestione delle dipendenze in grado di identificarle e di gestirle opportunamente, in modo che il processore esegua correttamente il programma.

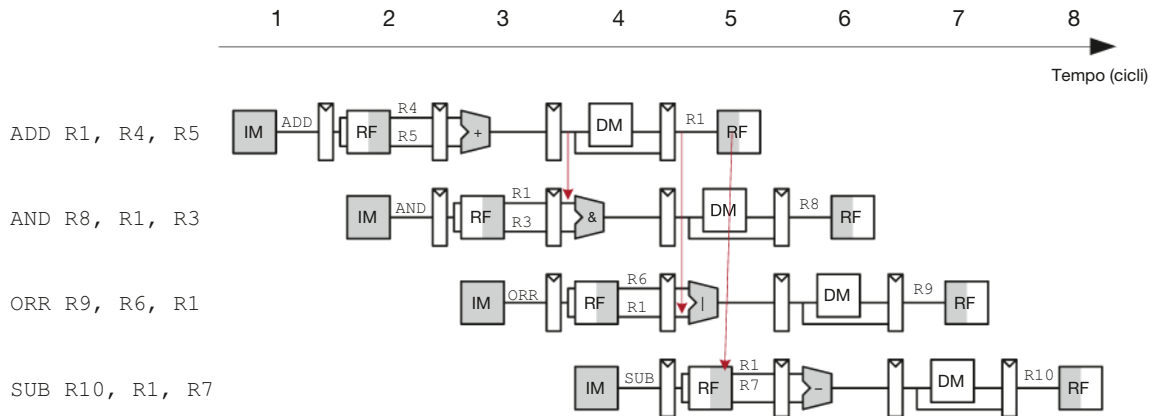
#### Gestione delle dipendenze di dato tramite inoltro

Alcune dipendenze di dato possono essere risolte mediante la tecnica dell'**inoltro** (*forwarding*) di un risultato disponibile negli stadi Memory o Writeback all'istruzione che ha dipendenza e che si trova nello stadio Execute. Questo richiede l'aggiunta di multiplexer davanti all'ALU per selezionare l'operando dal banco di registri oppure dagli stadi Memory o Writeback. La **Figura 7.50** mostra questa tecnica: nel ciclo 4, R1 è inoltrato dallo stadio Memory dell'istruzione ADD allo stadio Execute dell'istruzione dipendente AND; nel ciclo 5, R1 è inoltrato dallo stadio Writeback dell'istruzione ADD allo stadio Execute dell'istruzione dipendente ORR.

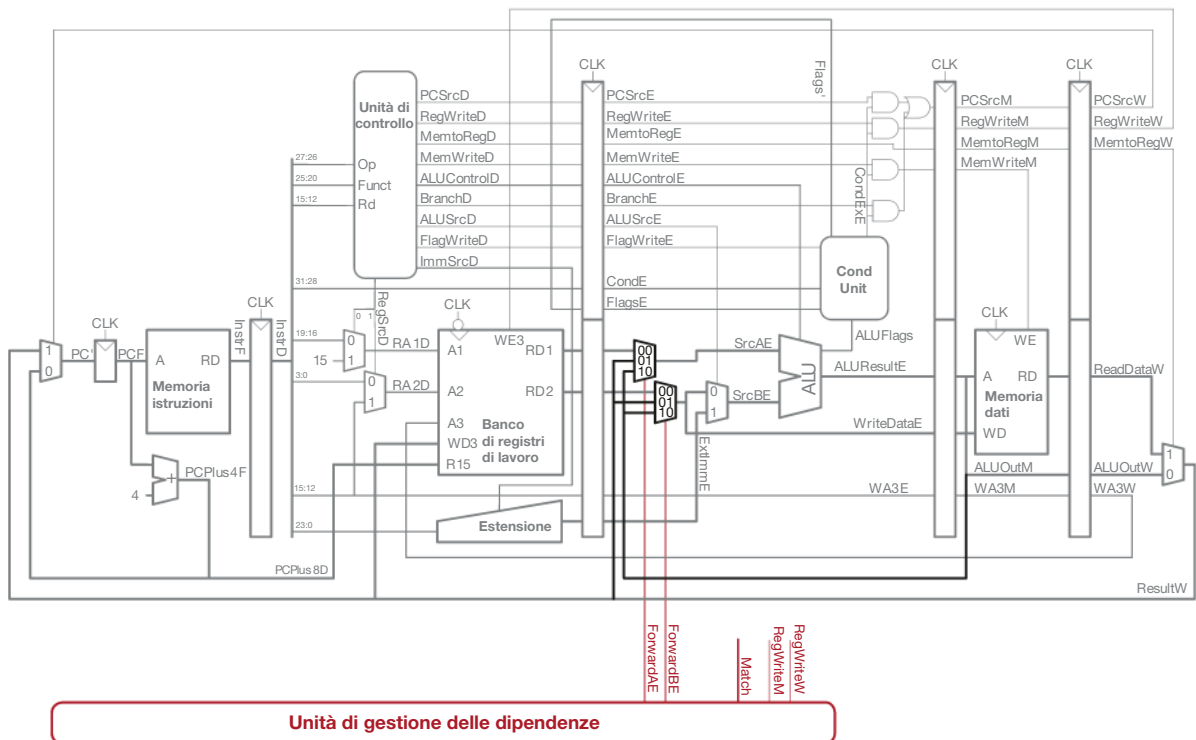
L'inoltro è necessario quando un'istruzione nello stadio Execute ha un registro sorgente coincidente con il registro destinazione delle istruzioni nello stadio Memory oppure Writeback. Nella **Figura 7.51** è mostrata la modifica del processore pipeline per gestire l'inoltro. Si aggiungono un'unità di gestione delle dipendenze e due multiplexer di inoltro. L'unità di gestione delle dipendenze riceve quattro segnali di uguaglianza dal percorso dati (abbreviati in *Match* nella Figura 7.51) che indicano se un registro sorgente nello stadio Execute è uguale al registro destinazione negli stadi Memory o Writeback:

```
Match_1E_M = (RA1E == WA3M)
Match_1E_W = (RA1E == WA3W)
Match_2E_M = (RA2E == WA3M)
Match_2E_W = (RA2E == WA3W)
```

L'unità di gestione delle dipendenze riceve anche i segnali *RegWrite* dagli stadi Memory e Writeback per sapere se il registro destinazione verrà davvero modificato (per es., le istruzioni STR e B non modificano il banco di registri, quindi non hanno bisogno che i loro risultati siano inoltrati). Si noti che nella Figura



**Figura 7.50** Rappresentazione schematica della pipeline con illustrazione dell'inoltro (*forwarding*).



**Figura 7.51** Processore pipeline con inoltro per risolvere le dipendenze.

7.51 questi segnali sono “collegati per nome”: in altre parole, invece di rendere confuso lo schema con lunghi fili che vanno dalla parte superiore dell'unità di controllo fino in basso all'unità di gestione delle dipendenze, i collegamenti sono indicati da un pezzetto di filo etichettato con il nome del segnale al quale è connesso. Anche la logica di gestione dei segnali Match e i registri di pipeline per RA1E e RA2E non sono riportati per limitare la complessità dello schema.

L'unità di gestione delle dipendenze genera i segnali per i multiplexer di inoltro per selezionare gli operandi dal banco dei registri oppure dai risultati negli stadi Memory o Writeback (*ALUOutM* o *ResultW*). L'inoltro deve essere attivato da uno dei due stadi se quello stadio scrive in un registro che è sorgente nello stadio Execute. Se entrambi gli stadi Memory e Writeback contengono registri destinazione uguali allo stesso registro sorgente di Execute, si deve dare la precedenza allo stadio Memory che contiene il valore più recente.

Quindi, la funzione di inoltro per *SrcAE* è quella riportata sotto. La funzione di inoltro per *SrcBE* (*ForwardBE*) è identica, ad eccezione dell'utilizzo del segnale di controllo *Match\_2E*.

```
if (Match_1E_M • RegWriteM) ForwardAE = 10; // SrcAE = ALUOutM
else if (Match_1E_W • RegWriteW) ForwardAE = 01; // SrcAE = ALUOutW
else ForwardAE = 00; // SrcAE dal banco di registri
```

### Gestione delle dipendenze di dato tramite stalli

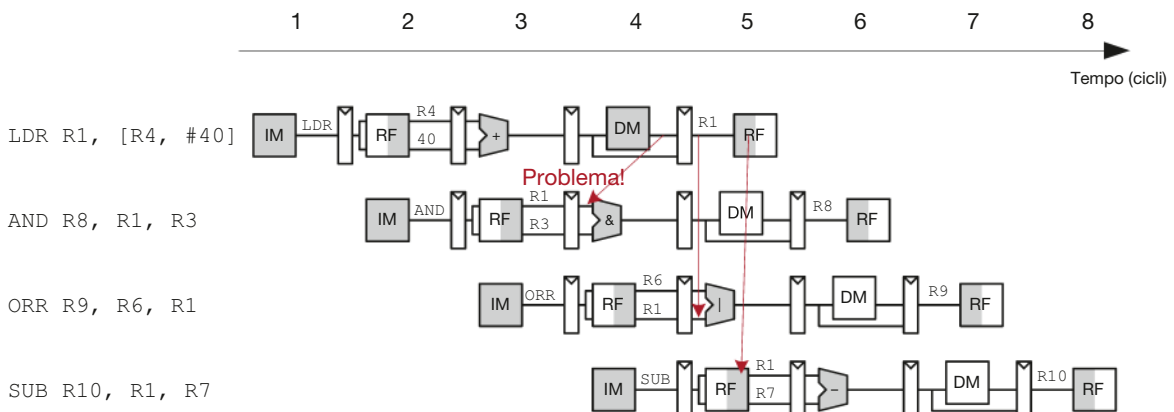
L'inoltro è sufficiente a risolvere le dipendenze di dato di tipo RAW quando il risultato viene calcolato nello stadio *Execute* di un'istruzione, perché può essere inoltrato allo stadio *Execute* dell'istruzione successiva. Sfortunatamente l'istruzione *LDR* termina di leggere il dato alla fine dello stadio *Memory*, per cui il risultato non può essere inoltrato allo stadio *Execute* dell'istruzione successiva. Si può dire che l'istruzione *LDR* ha una latenza di due cicli, perché un'istruzione che dipenda da lei può avere i suoi risultati solo due cicli dopo. La **Figura 7.52** mostra questo problema: l'istruzione *LDR* riceve il dato da memoria alla fine del ciclo 4, ma l'istruzione *AND* ha bisogno di avere tale dato come operando all'inizio del ciclo 4, quindi non c'è modo di risolvere questa dipendenza tramite inoltro.

La soluzione alternativa è quella di forzare in **stallo** la pipeline, sospendendo le operazioni fino all'arrivo del dato. La **Figura 7.53** mostra lo stallo dell'istruzione dipendente (*AND*) nello stadio *Decode*: *AND* entra nello stadio *Decode* nel ciclo 3 e viene mantenuta in stallo nel ciclo 4. L'istruzione successiva (*ORR*) deve a sua volta rimanere per due cicli nello stadio *Fetch* perché lo stadio *Decode* è occupato.

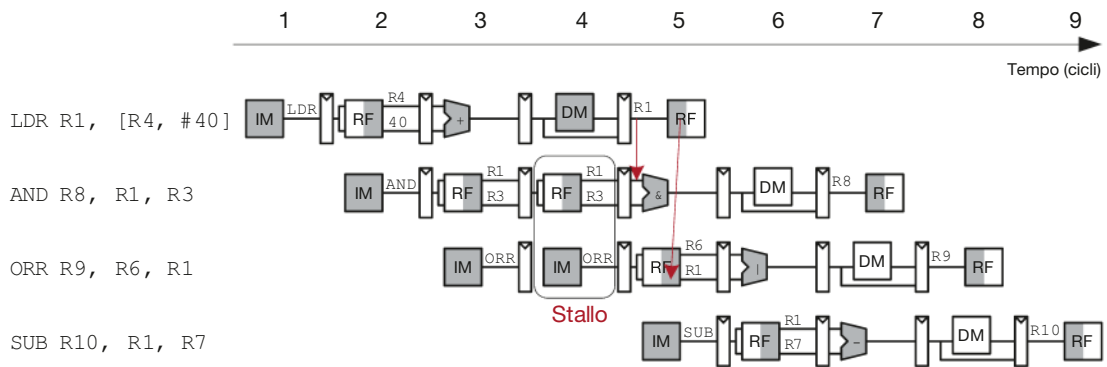
Nel ciclo 5 il risultato può essere inoltrato dallo stadio *Writeback* di *LDR* allo stadio *Execute* di *AND*. Sempre nel ciclo 5, il registro sorgente *R1* dell'istruzione *ORR* può essere letto direttamente dal banco di registri senza bisogno di inoltro.

Si noti che lo stadio *Execute* non è usato nel ciclo 4, così come non lo sono lo stadio *Memory* nel ciclo 5 e lo stadio *Writeback* nel ciclo 6. Questo mancato utilizzo di uno stadio che si propaga lungo la pipeline è denominato **bolla** (*bubble*) e si comporta come un'istruzione *NOP*. La bolla viene generata azzerando i segnali di controllo dello stadio *Execute* durante lo stallo dello stadio *Decode*, in modo tale che non vengano eseguite dalla bolla azioni che modifichino lo stato architetturale del processore.

In ultima analisi, lo stallo di uno stadio viene effettuato disabilitando i registri di pipeline in modo che lo stato corrente non sia modificato. Quando uno stadio viene portato in stallo, devono subire la stessa sorte anche tutti gli stadi precedenti, in modo che nessuna istruzione venga persa. Il registro di pipeline subito dopo lo stato portato in stallo deve essere svuotato (cancella-



**Figura 7.52** Rappresentazione schematica della pipeline con illustrazione del problema di inoltro nell'istruzione *LDR*.



**Figura 7.53** Rappresentazione schematica della pipeline con illustrazione dello stallo per risolvere le dipendenze.

to) per evitare che informazioni fasulle si propaghino nella pipeline. Lo stallo degrada le prestazioni, quindi va usato solo quando strettamente necessario.

La **Figura 7.54** modifica il processore pipeline per aggiungere gli stalli per le dipendenze di dato dall'istruzione *LDR*. L'unità di gestione delle dipendenze esamina l'istruzione presente nello stadio *Execute*: se è un'istruzione *LDR* e il suo registro destinazione (*WA3E*) coincide con uno dei due registri sorgente nello stadio *Decode* (*RA1D* oppure *RA2D*), allora si deve mettere in stallo l'istruzione nello stadio *Decode* finché il dato sorgente non sia stato reso disponibile.

Gli stalli sono realizzati aggiungendo ingressi di abilitazione ai registri di pipeline degli stadi *Fetch* e *Decode* e un ingresso di reset sincrono (*CLR*) al registro di pipeline dello stadio *Execute*. Quando serve generare uno stallo per *LDR*, *StallD* e *StallF* sono attivati per mettere in stallo gli stadi *Decode* e *Fetch*, e viene attivato anche *FlushE* per svuotare il contenuto del registro di pipeline dello stadio *Execute*, introducendo una bolla. Il segnale *MemtoReg* viene attivato per l'istruzione *LDR*, quindi la logica per generare stalli e svuotamenti risulta essere:

```
Match_12D_E = (RA1D == WA3E) + (RA2D == WA3E)
LDRstall = Match_12D_E • MemtoRegE
StallF = StallD = FlushE = LDRstall
```

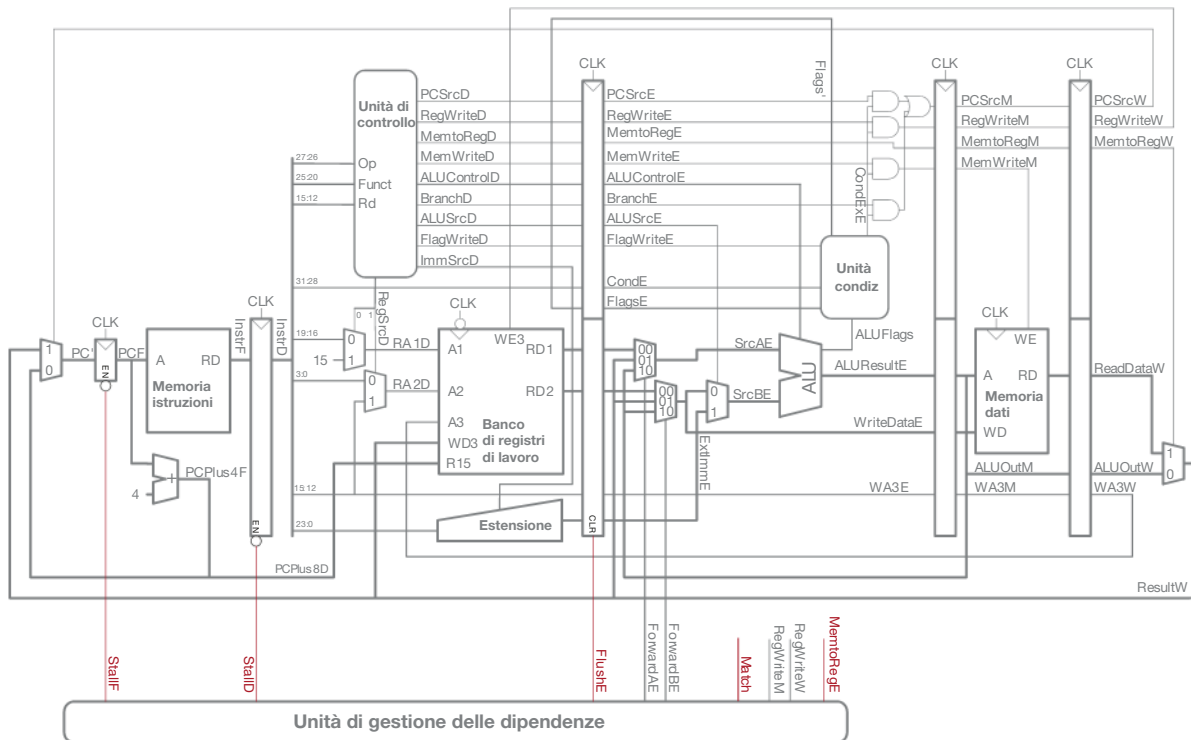
### Gestione delle dipendenze di controllo

L'istruzione *B* presenta una dipendenza di controllo: il processore pipeline non sa di quale istruzione fare il fetch come istruzione successiva, perché la decisione circa il fatto di saltare o meno non è ancora stata presa al momento di tale fetch. Le scritture nel registro *R15* (cioè nel *PC*) presentano lo stesso tipo di dipendenza.

Un modo per gestire le dipendenze di controllo è mandare in stallo la pipeline fino a quando la decisione circa il salto sarà stata presa (cioè si sarà calcolato *PCSrcW*). Dal momento che tale decisione viene presa nello stadio *Writeback*, la pipeline deve essere messa in stallo per quattro cicli a ogni istruzione di salto, con conseguente notevole degrado di prestazioni.

L'alternativa è prevedere se il salto dovrà essere fatto oppure no, e cominciare l'esecuzione delle istruzioni sulla base di tale previsione: una volta presa la decisione circa il salto, il processore dovrà eliminare tali istruzioni se la previsione si fosse rivelata sbagliata. Nella pipeline vista sino a qui (**Figura 7.54**) il processore prevede che i salti non vengano effettuati, e semplicemente procede a eseguire le istruzioni in sequenza finché non viene eventualmente generato *PCSrcW* per selezionare il prossimo valore del *PC* da *ResultW*. Se ciò succede, cioè il salto andava fatto, le quattro istruzioni successive al salto devono essere scartate, cioè la pipeline deve essere svuotata cancellando i registri di pipeline per tali istruzioni. Queste istruzioni da eliminare sono la cosiddetta penalizzazione per salto mal previsto.

La **Figura 7.55** mostra questo schema, in cui deve essere eseguito un salto



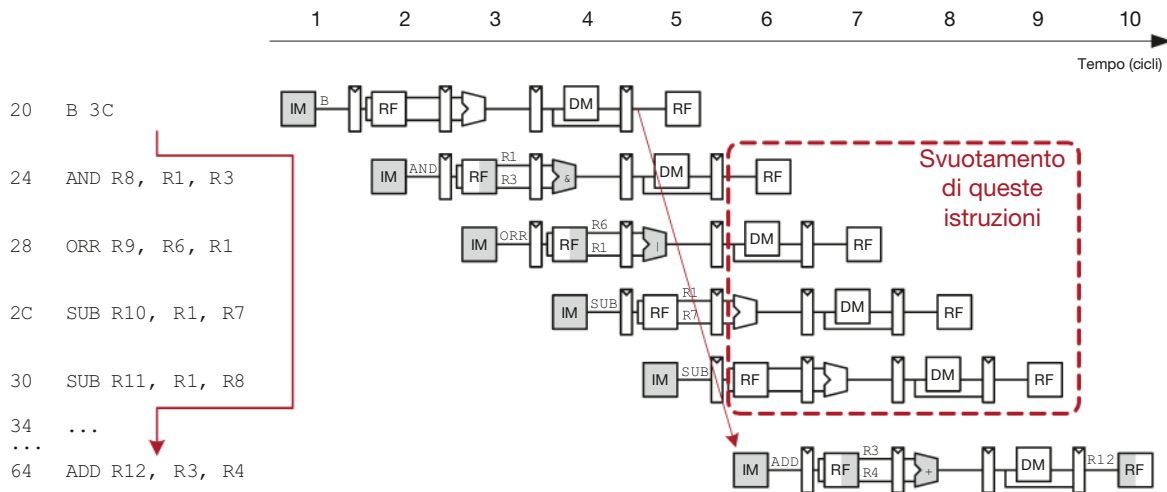
**Figura 7.54** Processore pipeline con stallo per risolvere le dipendenze di dato nell'istruzione `LDR`.

da 0x20 a 0x64. Il PC non viene modificato fino al ciclo 5, quando si è già fatto il fetch delle istruzioni `AND`, `ORR` e delle due `SUB` agli indirizzi 0x24, 0x28, 0x2C e 0x30. Si deve svuotare la pipeline da queste istruzioni ed eseguire il fetch dell'istruzione `ADD` all'indirizzo 0x64. La situazione è un po' migliorata, ma svuotare la pipeline di così tante istruzioni ogni volta che si esegue un salto è comunque un degrado di prestazioni considerevole.

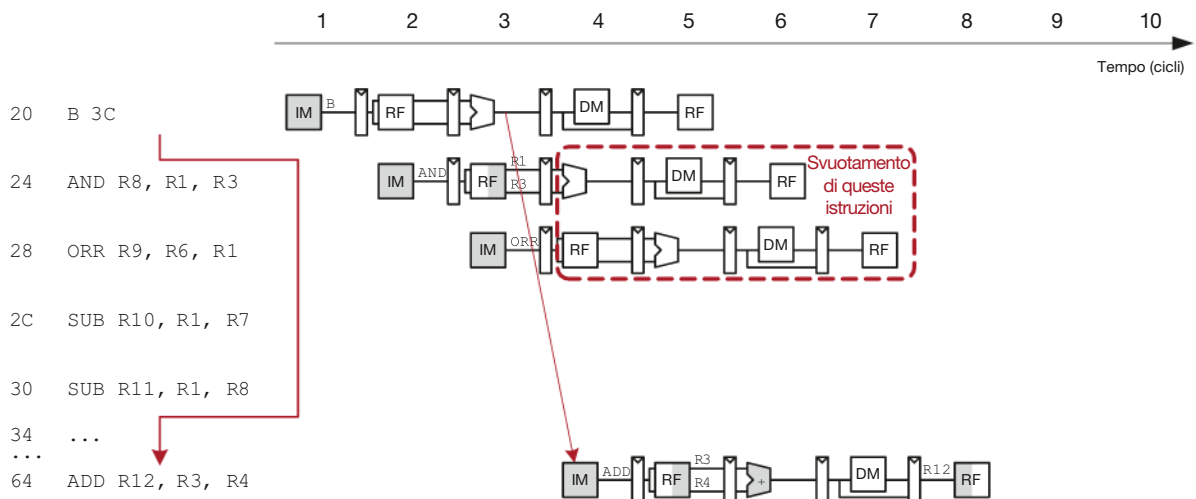
Si può ridurre la penalizzazione per salto mal previsto se si riesce ad anticipare la decisione se saltare o meno. In effetti tale decisione viene presa nello stadio `Execute` quando la destinazione di salto è già stata calcolata e si conosce il valore di `CondEx`. La **Figura 7.56** mostra le operazioni della pipeline con la decisione di salto anticipata presa nel ciclo 3: nel ciclo 4 si svuota la pipeline delle istruzioni `AND` e `ORR`, e si procede con il fetch della `ADD`, riducendo la penalizzazione per salto mal previsto da quattro a due sole istruzioni.

La **Figura 7.57** modifica il processore pipeline per anticipare la decisione di salto e gestire le dipendenze di controllo: si aggiunge un multiplexer di salto prima del registro PC per selezionare la destinazione di salto da `ALUResultE`, e il segnale `BranchTakenE` che controlla tale multiplexer è attivato per le istruzioni di salto le cui condizioni sono soddisfatte. `PCSrcW` viene attivato solo per scritture nel PC, che ancora avvengono nello stadio `Writeback`.

Infine, serve generare i segnali di controllo per stallo e svuotamento per gestire salti e scritture nel PC, che rischiano di generare errori perché le condizioni da verificare sono piuttosto complicate. Quando si fa un salto si devono svuotare i registri di pipeline delle due istruzioni successive negli stadi `Fetch` e `Decode`. Quando nella pipeline è presente una scrittura nel PC bisogna mettere in stallo la pipeline fino al termine della scrittura: questo è ottenuto mettendo in stallo lo stadio di `Fetch` (ricordando che mettere in stallo uno stadio implica svuotare lo stadio seguente per evitare che l'istruzione venga eseguita ripetutamente). La logica per gestire queste situazioni è riportata di seguito. `PCWrPending` viene attivato quando è in corso una scrittura nel PC (negli



**Figura 7.55** Rappresentazione schematica della *pipeline* con illustrazione dello svuotamento quando viene eseguito un salto.



**Figura 7.56** Rappresentazione schematica della *pipeline* con illustrazione dell'anticipazione del salto.

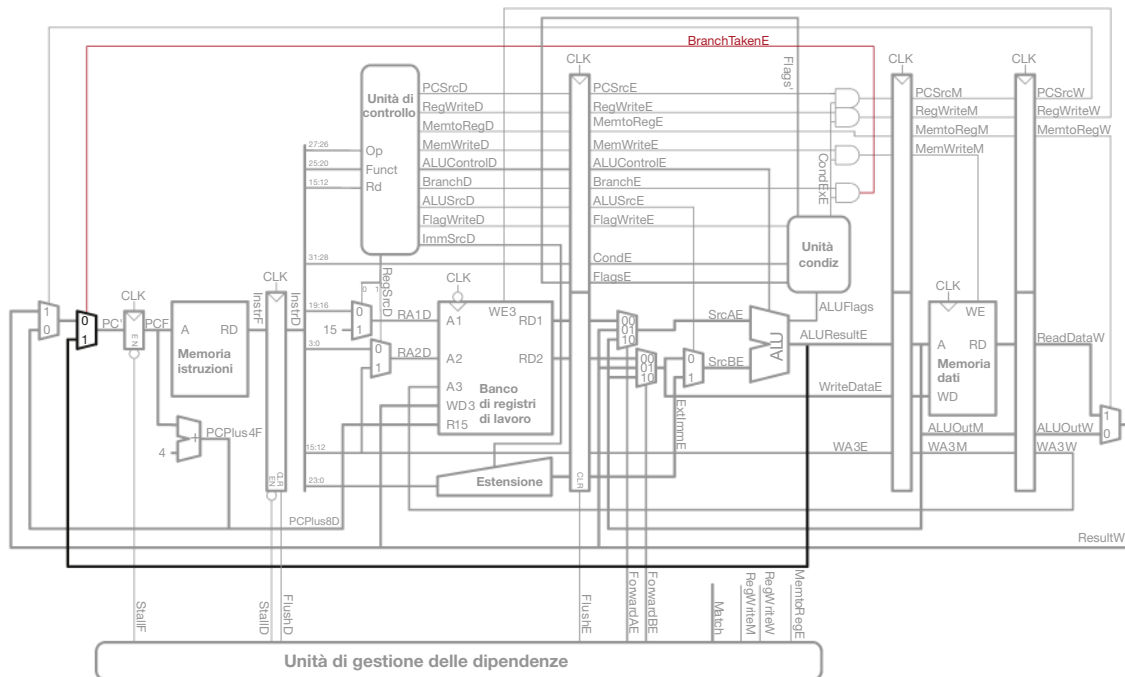
stadi Decode, Execute o Memory): in tal caso lo stadio Fetch viene messo in stallo e lo stadio Decode svuotato. Quando la scrittura nel PC raggiunge lo stadio Writeback (*PCSrcW* attivato) viene disattivato *StallF* per lasciare che la scrittura venga eseguita, ma *FlushD* rimane attivo per evitare che l'istruzione indesiderata presente nello stadio di Fetch possa avanzare.

```
PCWrPendingF = PCSrcD + PCSrcE + PCSrcM;
StallD = LDRstall;
StallF = LDRstall + PCWrPendingF;
FlushE = LDRstall + BranchTakenE;
FlushD = PCWrPendingF + PCSrcW + BranchTakenE;
```

Per evitare confusione, i collegamenti dall'unità di gestione delle dipendenze *PCSrcD*, *PCSrcE*, *PCSrcM* e *BranchTakenE* al percorso dati non sono mostrati nelle Figure 7.57 e 7.58.

Le istruzioni di salto sono molto frequenti, e anche una penalizzazione per salto mal previsto di due cicli ha un impatto non trascurabile sulle prestazioni. Con un po' di fatica in più si può ridurre a un ciclo la penalizzazione per molte istruzioni di salto, calcolando l'indirizzo di destinazione nello stadio Decode come *PCBranchD* = *PCPlus8D* + *ExtImmD*; anche *BranchTakenD* deve essere generato dello stadio Decode sulla base di *ALUFlagsE* prodotti dall'istruzione precedente, anche se ciò può portare a un aumento del tempo di ciclo se tali flag arrivano in ritardo. Si lascia al lettore la realizzazione delle modifiche necessarie (vedi l'Esercizio 7.36).





**Figura 7.57** Processore pipeline con gestione delle dipendenze di controllo.

### Conclusioni sulle dipendenze

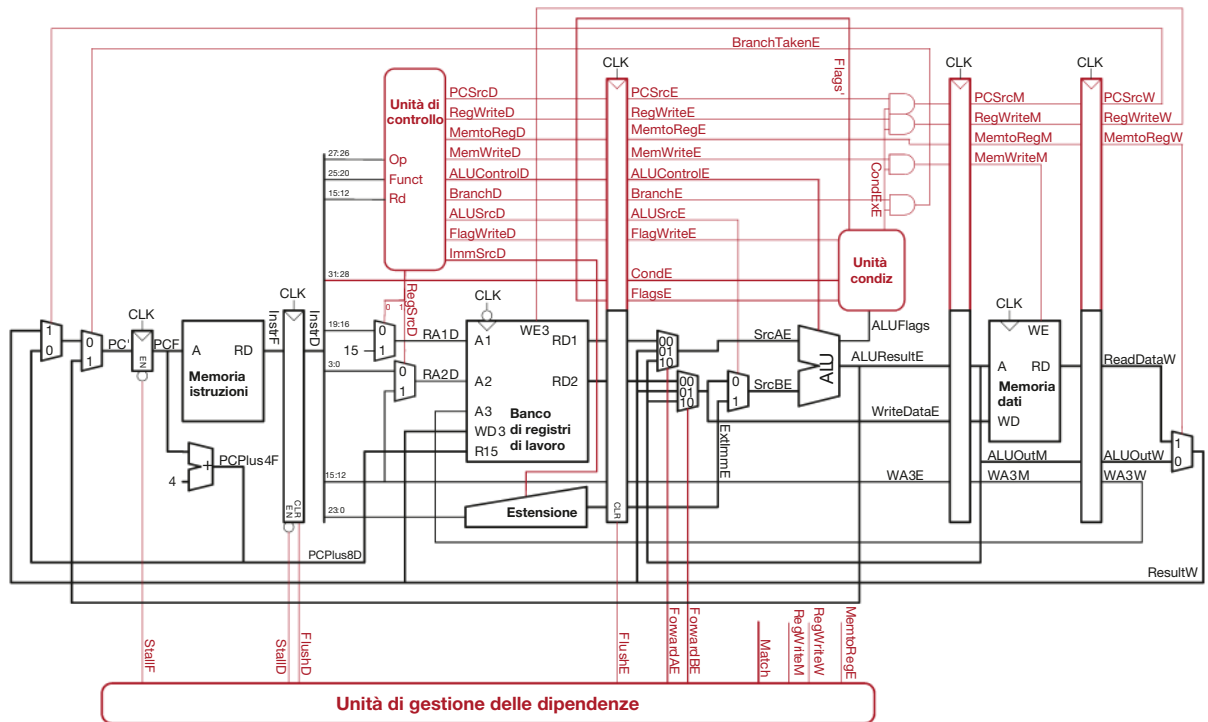
Per riassumere, le dipendenze di dato di tipo RAW si verificano quando un'istruzione dipende dal risultato di un'altra istruzione che non è ancora stato scritto nel banco di registri; possono essere gestite inoltrando tale risultato se è stato calcolato in tempo, oppure mettendo in stallo la pipeline finché non viene reso disponibile. Le dipendenze di controllo si verificano quando la decisione di quale sia la prossima istruzione da prelevare non sia ancora stata presa al momento del fetch; possono essere gestite prevedendo quale sia l'istruzione da leggere e svuotando la pipeline se la previsione si rivela errata, o mettendo in stallo la pipeline finché la decisione se saltare o meno non sia stata presa. Anticipare il più possibile la decisione minimizza il numero di istruzioni da eliminare con lo svuotamento in caso di previsione errata. Come si è senz'altro intuito, una delle sfide principali nel progetto di un processore pipeline è studiare tutte le possibili interazioni tra le varie istruzioni per individuare tutte le dipendenze che possono verificarsi. La **Figura 7.58** mostra il processore pipeline completo, capace di gestire tutte le dipendenze.

### 7.5.4 Analisi delle prestazioni

Idealmente il processore pipeline dovrebbe avere un CPI pari a 1, perché a ogni ciclo viene attivata una nuova istruzione. Tuttavia, gli stalli e gli svuotamenti sprecano cicli, quindi il CPI è un po' maggiore di 1 e dipende dallo specifico programma in esecuzione.

#### ESEMPIO 7.7

**CPI del processore pipeline.** Il benchmark SPECINT2000 considerato nell'Esempio 7.5 è costituito all'incirca da 25% di istruzioni di lettura da memoria, 10% di scritture in memoria, 13% di salti e 52% di istruzioni di elaborazione dati. Si faccia l'ipotesi che il 40% delle istruzioni di lettura da memoria sia immediatamente seguito da un'istruzione che usa il risultato della lettura, con la necessità quindi di introdurre uno stallo, e che il 50% dei salti sia da fare (quindi risultino mal previsti) richiedendo svuotamento. Calcolare il CPI medio del processore pipeline.



**Figura 7.58** Processore pipeline con gestione completa delle dipendenze.

**Soluzione** Il CPI medio è la somma per tutte le istruzioni del CPI di ogni istruzione moltiplicato per la frazione di tempo per la quale l'istruzione è usata. Le letture da memoria richiedono un ciclo di clock se non c'è dipendenza e due cicli di clock se il processore deve essere messo in stallo perché c'è dipendenza, quindi hanno un CPI pari a  $(0.6) \times 1 + (0.4) \times 2 = 1.4$ . I salti richiedono un ciclo di clock se sono previsti correttamente e tre cicli di clock se sono mal previsti, per cui hanno un CPI pari a  $(0.5) \times 1 + (0.5) \times 3 = 2.0$ . Tutte le altre istruzioni hanno un CPI pari a 1. Per questo *benchmark*, si ottiene quindi:  $CPI_{\text{medio}} = (0.25) \times 1.4 + (0.1) \times 1 + (0.13) \times 2.0 + (0.52) \times 1 = 1.23$ .

Si può calcolare il tempo di ciclo considerando il percorso critico in ciascuno dei cinque stadi della pipeline come mostrato nella Figura 7.58. Si deve ricordare che il banco di registri viene modificato nella prima metà del ciclo nello stadio Writeback e letto nella seconda metà del ciclo nello stadio Decode, quindi il tempo di ciclo degli stadi Decode e Writeback deve essere il doppio del tempo necessario a svolgere queste operazioni da mezzo ciclo ciascuna.

$$T_{c3} = \max \begin{bmatrix} t_{pcq} + t_{mem} + t_{setup} & \text{Fetch} \\ 2(t_{Rfread} + t_{setup}) & \text{Decode} \\ t_{pcq} + 2t_{mux} + t_{ALU} + t_{setup} & \text{Execute} \\ t_{pcq} + t_{mem} + t_{setup} & \text{Memory} \\ 2(t_{pcq} + t_{mux} + t_{Rfsetup}) & \text{Writeback} \end{bmatrix} \quad (7.5)$$

### ESEMPIO 7.8

**Confronto delle prestazioni dei processori.** Ben Imbrogliabit deve confrontare le prestazioni del processore pipeline con quelle dei processori a ciclo singolo e multi ciclo considerati nell'Esempio 7.6, usando logiche con i ritardi elencati nella Tabella 7.5. Bisogna aiutarlo a confrontare le prestazioni dei vari processori nell'esecuzione di 100 miliardi di istruzioni del benchmark SPECINT2000.

**Soluzione** In base all'Espressione 7.5, il tempo di ciclo del processore *pipeline* è  $T_{c3} = \max[40 + 200 + 50, 2(100 + 50), 40 + 2(25) + 120 + 50, 40 + 200 + 50, 2(40 + 2 + 60)] = 300$  ps. In base all'Espressione 7.1, il tempo totale di esecuzione è  $T_3 = (100 \times 10^9 \text{ istruzioni}) (1.23 \text{ cicli/istruzione}) (300 \times 10^{-12} \text{ s/ciclo}) = 36.9$  secondi, in confronto agli 84 secondi del processore a ciclo singolo e ai 140 secondi del processore multi ciclo.

Il processore pipeline è significativamente più veloce degli altri due. Tuttavia il suo vantaggio sul processore a ciclo singolo è ben lontano dal fattore cinque che ci si potrebbe aspettare con una pipeline a cinque stadi. Le dipendenze introducono senz'altro una piccola penalizzazione in termini di CPI. Più significativo è il sovraccarico di sequenziamento (ritardo dal clock all'uscita e ritardo di setup) dei registri che deve essere applicato a tutti gli stadi di pipeline e non una volta sola all'intero percorso dati, un'esigenza che limita i potenziali benefici della struttura pipeline. In termini di requisiti hardware, il processore pipeline è simile al processore a ciclo singolo, ma richiede 8 registri aggiuntivi di pipeline a 32 bit, oltre a qualche multiplexer, qualche registro di pipeline più piccolo e un po' di logica di controllo per la gestione delle dipendenze.

## 7.6 ■ RAPPRESENTAZIONE HDL \*

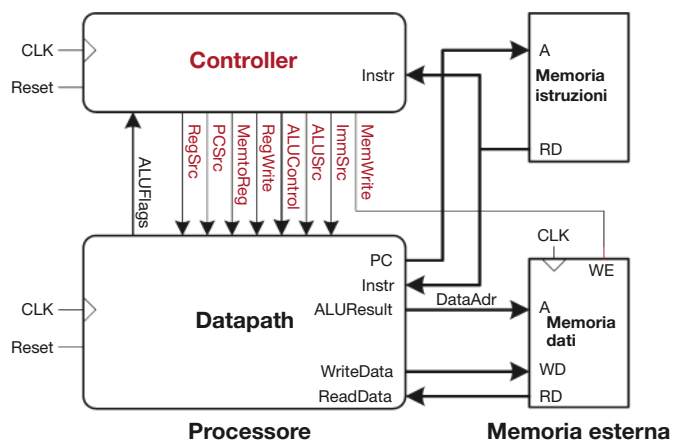
Questo paragrafo presenta il codice HDL del processore a ciclo singolo capace di eseguire le istruzioni discusse nel capitolo. Il codice costituisce un esempio di buona scrittura HDL per un sistema di una certa complessità. I codici HDL per i processori multi ciclo e pipeline sono lasciati come Esercizi 7.25 e 7.40.

In questo paragrafo, le memorie istruzioni e dati sono separate dal percorso dati e collegate mediante opportuni bus indirizzi e dati. In pratica, molti processori prelevano istruzioni e dati da memorie *cache* separate, ma un processore completo deve anche poter leggere dati dalla memoria istruzioni per gestire particolari lookup table. Nel Capitolo 8 si affronta il sistema di memoria, comprese le interazioni fra memorie *cache* e memoria principale.

Il processore è composto dal *datapath* (cioè il percorso dati) e dal *controller* (cioè l'unità di controllo), a sua volta costituito dal Decoder e dalla Logica Condizionale. La **Figura 7.59** mostra lo schema a blocchi del processore a ciclo singolo interfacciato alle memorie esterne.

Il codice HDL è suddiviso in vari paragrafi: il paragrafo 7.6.1 contiene il codice per il percorso dati e il *controller*; il paragrafo 7.6.2 presenta vari blocchi costitutivi, come registri e multiplexer, usati nella microarchitettura. Il paragrafo 7.6.3 introduce le memorie e il testbench. Il codice HDL è disponibile in formato elettronico sul sito web del libro (*vedi* la Prefazione): per questo motivo è stato riportato nella sua forma originale senza interventi di traduzione dei nomi dei componenti né dei commenti.

**Figura 7.59**  
Processore a ciclo singolo  
interfacciato con una memoria  
esterna.



### 7.6.1 Processore a ciclo singolo

I moduli principali del modulo processore a ciclo singolo sono riportati nei seguenti Esempi HDL.

#### ESEMPIO HDL 7.1 PROCESSORE A CICLO SINGOLO

##### SystemVerilog

```
module arm(input logic clk, reset,
 output logic [31:0] PC,
 input logic [31:0] Instr,
 output logic MemWrite,
 output logic [31:0] ALUResult, WriteData,
 input logic [31:0] ReadData);

 logic [3:0] ALUFlags;
 logic RegWrite,
 ALUSrc, MemtoReg, PCSrc;
 logic [1:0] RegSrc, ImmSrc, ALUControl;

 controller c(clk, reset, Instr[31:12], ALUFlags,
 RegSrc, RegWrite, ImmSrc,
 ALUSrc, ALUControl,
 MemWrite, MemtoReg, PCSrc);

 datapath dp(clk, reset,
 RegSrc, RegWrite, ImmSrc,
 ALUSrc, ALUControl,
 MemtoReg, PCSrc,
 ALUFlags, PC, Instr,
 ALUResult, WriteData, ReadData);

endmodule
```

##### VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;
entity arm is -- single cycle processor
 port(clk, reset: in STD_LOGIC;
 PC: out STD_LOGIC_VECTOR(31 downto 0);
 Instr: in STD_LOGIC_VECTOR(31 downto 0);
 MemWrite: out STD_LOGIC;
 ALUResult, WriteData: out STD_LOGIC_VECTOR(31 downto 0);
 ReadData: in STD_LOGIC_VECTOR(31 downto 0));
end;

architecture struct of arm is
 component controller
 port(clk, reset: in STD_LOGIC;
 Instr: in STD_LOGIC_VECTOR(31 downto 12);
 ALUFlags: in STD_LOGIC_VECTOR(3 downto 0);
 RegSrc: out STD_LOGIC_VECTOR(1 downto 0);
 RegWrite: out STD_LOGIC;
 ImmSrc: out STD_LOGIC_VECTOR(1 downto 0);
 ALUSrc: out STD_LOGIC;
 ALUControl: out STD_LOGIC_VECTOR(1 downto 0);
 MemWrite: out STD_LOGIC;
 MemtoReg: out STD_LOGIC;
 PCSrc: out STD_LOGIC);
 end component;
 component datapath
 port(clk, reset: in STD_LOGIC;
 RegSrc: in STD_LOGIC_VECTOR(1 downto 0);
 RegWrite: in STD_LOGIC;
 ImmSrc: in STD_LOGIC_VECTOR(1 downto 0);
 ALUSrc: in STD_LOGIC;
 ALUControl: in STD_LOGIC_VECTOR(1 downto 0);
 MemtoReg: in STD_LOGIC;
 PCSrc: in STD_LOGIC;
 ALUFlags: out STD_LOGIC_VECTOR(3 downto 0);
 PC: buffer STD_LOGIC_VECTOR(31 downto 0);
 Instr: in STD_LOGIC_VECTOR(31 downto 0);
 ALUResult, WriteData: buffer STD_LOGIC_VECTOR(31 downto 0);
 ReadData: in STD_LOGIC_VECTOR(31 downto 0));
 end component;
 signal RegWrite, ALUSrc, MemtoReg, PCSrc: STD_LOGIC;
 signal RegSrc, ImmSrc, ALUControl: STD_LOGIC_VECTOR
 (1 downto 0);
 signal ALUFlags: STD_LOGIC_VECTOR(3 downto 0);
begin
 cont: controller port map(clk, reset, Instr(31 downto 12),
 ALUFlags, RegSrc, RegWrite,
 ImmSrc, ALUSrc, ALUControl,
 MemWrite, MemtoReg, PCSrc);
 dp: datapath port map(clk, reset, RegSrc, RegWrite, ImmSrc,
 ALUSrc, ALUControl, MemtoReg, PCSrc,
 ALUFlags, PC, Instr, ALUResult,
 WriteData, ReadData);
end;
```

**ESEMPIO HDL 7.2 CONTROLLER****SystemVerilog**

```

module controller(input logic clk,
 reset,
 input logic [31:12] Instr,
 input logic [3:0] ALUFlags,
 output logic [1:0] RegSrc,
 output logic RegWrite,
 output logic [1:0] ImmSrc,
 output logic ALUSrc,
 output logic [1:0] ALUControl,
 output logic MemWrite,
 output logic MemtoReg,
 output logic PCSrc);

logic [1:0] FlagW;
logic PCS, RegW, MemW;

decoder dec(
Instr[27:26], Instr[25:20], Instr[15:12], FlagW,
PCS, RegW, MemW, MemtoReg, ALUSrc, ImmSrc, RegSrc,
ALUControl);
condlogic cl(
clk, reset, Instr[31:28], ALUFlags, FlagW, PCS,
RegW, MemW, PCSrc, RegWrite, MemWrite);
endmodule

```

**VHDL**

```

library IEEE; use IEEE.STD_LOGIC_1164.all;
entity controller is -- single cycle control
port(clk, reset: in STD_LOGIC;
 Instr: in STD_LOGIC_VECTOR(31 downto 12);
 ALUFlags: in STD_LOGIC_VECTOR(3 downto 0);
 RegSrc: out STD_LOGIC_VECTOR(1 downto 0);
 RegWrite: out STD_LOGIC;
 ImmSrc: out STD_LOGIC_VECTOR(1 downto 0);
 ALUSrc: out STD_LOGIC;
 ALUControl: out STD_LOGIC_VECTOR(1 downto 0);
 MemWrite: out STD_LOGIC;
 MemtoReg: out STD_LOGIC;
 PCSrc: out STD_LOGIC);
end;

architecture struct of controller is
component decoder
port(Op: in STD_LOGIC_VECTOR(1 downto 0);
 Funct: in STD_LOGIC_VECTOR(5 downto 0);
 Rd: in STD_LOGIC_VECTOR(3 downto 0);
 FlagW: out STD_LOGIC_VECTOR(1 downto 0);
 PCS, RegW, MemW: out STD_LOGIC;
 MemtoReg, ALUSrc: out STD_LOGIC;
 ImmSrc, RegSrc: out STD_LOGIC_VECTOR(1 downto 0);
 ALUControl: out STD_LOGIC_VECTOR(1 downto 0));
end component;
component condlogic
port(clk, reset: in STD_LOGIC;
 Cond: in STD_LOGIC_VECTOR(3 downto 0);
 ALUFlags: in STD_LOGIC_VECTOR(3 downto 0);
 FlagW: in STD_LOGIC_VECTOR(1 downto 0);
 PCS, RegW, MemW: in STD_LOGIC;
 PCSrc, RegWrite: out STD_LOGIC;
 MemWrite: out STD_LOGIC);
end component;
signal FlagW: STD_LOGIC_VECTOR(1 downto 0);
signal PCS, RegW, MemW: STD_LOGIC;
begin
dec: decoder port map(Instr(27 downto 26), Instr(25
downto 20), Instr(15 downto
12), FlagW, PCS, RegW, MemW,
MemtoReg, ALUSrc, ImmSrc, RegSrc,
ALUControl);
cl: condlogic port map(clk, reset, Instr(31 downto 28),
ALUFlags, FlagW, PCS, RegW, MemW,
PCSrc, RegWrite, MemWrite);
end;

```

**ESEMPIO HDL 7.3 DECODER****SystemVerilog**

```

module decoder(input logic [1:0] Op,
 input logic [5:0] Funct,
 input logic [3:0] Rd,
 output logic [1:0] FlagW,
 output logic PCS, RegW, MemW,
 output logic MemtoReg, ALUSrc,
 output logic [1:0] ImmSrc, RegSrc,
 ALUControl);

logic [9:0] controls;
logic Branch, ALUOp;

// Main Decoder
always_comb
 casex(Op)
 // Data-processing immediate
 2'b00: if (Funct[5]) controls = 10'b0000101001;
 // Data-processing register
 else controls = 10'b0000001001;
 // LDR
 2'b01: if (Funct[0]) controls = 10'b0001111000;
 // STR
 else controls = 10'b1001110100;
 // B
 2'b10: controls = 10'b0110100010;
 // Unimplemented
 default: controls = 10'bx;
 endcase

assign {RegSrc, ImmSrc, ALUSrc, MemtoReg, RegW, MemW,
 Branch, ALUOp} = controls;

// ALU Decoder
always_comb
 if (ALUOp) begin // which DP Instr?
 case(Funct[4:1])
 4'b0100: ALUControl = 2'b00; // ADD
 4'b0010: ALUControl = 2'b01; // SUB
 4'b0000: ALUControl = 2'b10; // AND
 4'b1100: ALUControl = 2'b11; // ORR
 default: ALUControl = 2'bx; // unimplemented
 endcase
 end

// update flags if S bit is set (C & V only for arith)
FlagW[1] = Funct[0];
FlagW[0] = Funct[0] &
 (ALUControl == 2'b00 | ALUControl == 2'b01);
end else begin
 ALUControl = 2'b00; // add for non-DP instructions
 FlagW = 2'b00; // don't update Flags
end

// PC Logic
assign PCS = ((Rd == 4'b1111) & RegW) | Branch;
endmodule

```

**VHDL**

```

library IEEE; use IEEE.STD_LOGIC_1164.all;
entity decoder is -- main control decoder
 port(Op: in STD_LOGIC_VECTOR(1 downto 0);
 Funct: in STD_LOGIC_VECTOR(5 downto 0);
 Rd: in STD_LOGIC_VECTOR(3 downto 0);
 FlagW: out STD_LOGIC_VECTOR(1 downto 0);
 PCS, RegW, MemW: out STD_LOGIC;
 MemtoReg, ALUSrc: out STD_LOGIC;
 ImmSrc, RegSrc: out STD_LOGIC_VECTOR(1 downto 0);
 ALUControl: out STD_LOGIC_VECTOR(1 downto 0));
end;

architecture behave of decoder is
 signal controls: STD_LOGIC_VECTOR(9 downto 0);
 signal ALUOp, Branch: STD_LOGIC;
 signal op2: STD_LOGIC_VECTOR(3 downto 0);
begin
 op2 <= (Op, Funct(5), Funct(0));
 process(all) begin -- Main Decoder
 case? (op2) is
 when "000-" => controls <= "0000001001";
 when "001-" => controls <= "0000101001";
 when "01-0" => controls <= "1001110100";
 when "01-1" => controls <= "0001111000";
 when "10- -" => controls <= "0110100010";
 when others => controls <= "- - - - -";
 end case?;
 end process;

 (RegSrc, ImmSrc, ALUSrc, MemtoReg, RegW, MemW, Branch,
 ALUOp) <= controls;

 process(all) begin -- ALU Decoder
 if (ALUOp) then
 case Funct(4 downto 1) is
 when "0100" => ALUControl <= "00"; -- ADD
 when "0010" => ALUControl <= "01"; -- SUB
 when "0000" => ALUControl <= "10"; -- AND
 when "1100" => ALUControl <= "11"; -- ORR
 when others => ALUControl <= "- -"; -- unimplemented
 end case;
 FlagW(1) <= Funct(0);
 FlagW(0) <= Funct(0) and (not ALUControl(1));
 else
 ALUControl <= "00";
 FlagW <= "00";
 end if;
 end process;

 PCS <= ((and Rd) and RegW) or Branch;
end;

```

**ESEMPIO HDL 7.4 LOGICA CONDIZIONALE****SystemVerilog**

```

module condlogic(input logic clk, reset,
 input logic [3:0] Cond,
 input logic [3:0] ALUFlags,
 input logic [1:0] FlagW,
 input logic PCS, RegW, MemW,
 output logic PCSrc, RegWrite,
 MemWrite);

 logic [1:0] FlagWrite;
 logic [3:0] Flags;
 logic CondEx;

 flopenr #(2) flagreg1(clk, reset, FlagWrite[1],
 ALUFlags[3:2], Flags[3:2]);
 flopenr #(2) flagreg0(clk, reset, FlagWrite[0],
 ALUFlags[1:0], Flags[1:0]);

 // write controls are conditional
 condcheck cc(Cond, Flags, CondEx);
 assign FlagWrite = FlagW & {2{CondEx}};
 assign RegWrite = RegW & CondEx;
 assign MemWrite = MemW & CondEx;
 assign PCSrc = PCS & CondEx;
endmodule

module condcheck(input logic [3:0] Cond,
 input logic [3:0] Flags,
 output logic CondEx);

 logic neg, zero, carry, overflow, ge;

 assign {neg, zero, carry, overflow} = Flags;
 assign ge = (neg == overflow);

 always_comb
 case(Cond)
 4'b0000: CondEx = zero; // EQ
 4'b0001: CondEx = ~zero; // NE
 4'b0010: CondEx = carry; // CS
 4'b0011: CondEx = ~carry; // CC
 4'b0100: CondEx = neg; // MI
 4'b0101: CondEx = ~neg; // PL
 4'b0110: CondEx = overflow; // VS
 4'b0111: CondEx = ~overflow; // VC
 4'b1000: CondEx = carry & ~zero; // HI
 4'b1001: CondEx = ~(carry & ~zero); // LS
 4'b1010: CondEx = ge; // GE
 4'b1011: CondEx = ~ge; // LT
 4'b1100: CondEx = ~zero & ge; // GT
 4'b1101: CondEx = ~(~zero & ge); // LE
 4'b1110: CondEx = 1'b1; // Always
 default: CondEx = 1'bx; // undefined
 endcase
endmodule

```

**VHDL**

```

library IEEE; use IEEE.STD_LOGIC_1164.all;
entity condlogic is -- Conditional logic
 port(clk, reset: in STD_LOGIC;
 Cond: in STD_LOGIC_VECTOR(3 downto 0);
 ALUFlags: in STD_LOGIC_VECTOR(3 downto 0);
 FlagW: in STD_LOGIC_VECTOR(1 downto 0);
 PCS, RegW, MemW: in STD_LOGIC;
 PCSrc, RegWrite: out STD_LOGIC;
 MemWrite: out STD_LOGIC);
end;

architecture behave of condlogic is
 component condcheck
 port(Cond: in STD_LOGIC_VECTOR(3 downto 0);
 Flags: in STD_LOGIC_VECTOR(3 downto 0);
 CondEx: out STD_LOGIC);
 end component;
 component flopenr generic(width: integer);
 port(clk, reset, en: in STD_LOGIC;
 d: in STD_LOGIC_VECTOR (width-1 downto 0);
 q: out STD_LOGIC_VECTOR (width-1 downto 0));
 end component;
 signal FlagWrite: STD_LOGIC_VECTOR(1 downto 0);
 signal Flags: STD_LOGIC_VECTOR(3 downto 0);
 signal CondEx: STD_LOGIC;
begin
 flagreg1: flopenr generic map(2)
 port map(clk, reset, FlagWrite(1), ALUFlags(3 downto 2),
 Flags(3 downto 2));
 flagreg0: flopenr generic map(2)
 port map(clk, reset, FlagWrite(0), ALUFlags(1 downto 0),
 Flags(1 downto 0));
 cc: condcheck port map(Cond, Flags, CondEx);

 FlagWrite <= FlagW and (CondEx, CondEx);
 RegWrite <= RegW and CondEx;
 MemWrite <= MemW and CondEx;
 PCSrc <= PCS and CondEx;
end;

library IEEE; use IEEE.STD_LOGIC_1164.all;
entity condcheck is
 port(Cond: in STD_LOGIC_VECTOR(3 downto 0);
 Flags: in STD_LOGIC_VECTOR(3 downto 0);
 CondEx: out STD_LOGIC);
end;

architecture behave of condcheck is
 signal neg, zero, carry, overflow, ge: STD_LOGIC;
begin
 (neg, zero, carry, overflow) <= Flags;
 ge <= (neg xnor overflow);

 process(all) begin -- Condition checking
 case Cond is

```



```

when "0000" => CondEx <= zero;
when "0001" => CondEx <= not zero;
when "0010" => CondEx <= carry;
when "0011" => CondEx <= not carry;
when "0100" => CondEx <= neg;
when "0101" => CondEx <= not neg;
when "0110" => CondEx <= overflow;
when "0111" => CondEx <= not overflow;
when "1000" => CondEx <= carry and (not zero);
when "1001" => CondEx <= not(carry and (not zero));
when "1010" => CondEx <= ge;
when "1011" => CondEx <= not ge;
when "1100" => CondEx <= (not zero) and ge;
when "1101" => CondEx <= not ((not zero) and ge);
when "1110" => CondEx <= '1';
when others => CondEx <= '-';
end case;
end process;
end;

```

## ESEMPIO HDL 7.5 DATAPATH

### SystemVerilog

```

module datapath(input logic clk, reset,
 input logic [1:0] RegSrc,
 input logic RegWrite,
 input logic [1:0] ImmSrc,
 input logic ALUSrc,
 input logic [1:0] ALUControl,
 input logic MemtoReg,
 input logic PCSrc,
 output logic [3:0] ALUFlags,
 output logic [31:0] PC,
 input logic [31:0] Instr,
 output logic [31:0] ALUResult, WriteData,
 input logic [31:0] ReadData);

logic [31:0] PCNext, PCPlus4, PCPlus8;
logic [31:0] ExtImm, SrcA, SrcB, Result;
logic [3:0] RA1, RA2;

// next PC logic
mux2 #(32) pcmux(PCPlus4, Result, PCSrc, PCNext);
flop #(32) pcreg(clk, reset, PCNext, PC);
adder #(32) pcadd1(PC, 32'b100, PCPlus4);
adder #(32) pcadd2(PCPlus4, 32'b100, PCPlus8);

// register file logic
mux2 #(4) ralmux(Instr[19:16], 4'b1111, RegSrc[0], RA1);
mux2 #(4) ra2mux(Instr[3:0], Instr[15:12], RegSrc[1],
RA2);
regfile rf(clk, RegWrite, RA1, RA2,
Instr[15:12], Result, PCPlus8, SrcA, WriteData);
mux2 #(32) resmux(ALUResult, ReadData, MemtoReg, Result);
extend ext(Instr[23:0], ImmSrc, ExtImm);

// ALU logic
mux2 #(32) srcbmux(WriteData, ExtImm, ALUSrc, SrcB);
alu alu(SrcA, SrcB, ALUControl, ALUResult, ALUFlags);
endmodule

```

### VHDL

```

library IEEE; use IEEE.STD_LOGIC_1164.all;
entity datapath is
port(clk, reset: in STD_LOGIC;
 RegSrc: in STD_LOGIC_VECTOR(1 downto 0);
 RegWrite: in STD_LOGIC;
 ImmSrc: in STD_LOGIC_VECTOR(1 downto 0);
 ALUSrc: in STD_LOGIC;
 ALUControl: in STD_LOGIC_VECTOR(1 downto 0);
 MemtoReg: in STD_LOGIC;
 PCSrc: in STD_LOGIC;
 ALUFlags: out STD_LOGIC_VECTOR(3 downto 0);
 PC: buffer STD_LOGIC_VECTOR(31 downto 0);
 Instr: in STD_LOGIC_VECTOR(31 downto 0);
 ALUResult, WriteData: buffer STD_LOGIC_VECTOR(31 downto 0);
 ReadData: in STD_LOGIC_VECTOR(31 downto 0));
end;

architecture struct of datapath is
component alu
port(a, b: in STD_LOGIC_VECTOR(31 downto 0);
 ALUControl: in STD_LOGIC_VECTOR(1 downto 0);
 Result: buffer STD_LOGIC_VECTOR(31 downto 0);
 ALUFlags: out STD_LOGIC_VECTOR(3 downto 0));
end component;
component regfile
port(clk: in STD_LOGIC;
 we3: in STD_LOGIC;
 ral, ra2, wa3: in STD_LOGIC_VECTOR(3 downto 0);
 wd3, r15: in STD_LOGIC_VECTOR(31 downto 0);
 rd1, rd2: out STD_LOGIC_VECTOR(31 downto 0));

```

```

end component;
component adder
port(a, b: in STD_LOGIC_VECTOR(31 downto 0);
 y: out STD_LOGIC_VECTOR(31 downto 0));
end component;
component extend
port(Instr: in STD_LOGIC_VECTOR(23 downto 0);
 ImmSrc: in STD_LOGIC_VECTOR(1 downto 0);
 ExtImm: out STD_LOGIC_VECTOR(31 downto 0));
end component;
component flopr generic(width: integer);
port(clk, reset: in STD_LOGIC;
 d: in STD_LOGIC_VECTOR(width-1 downto 0);
 q: out STD_LOGIC_VECTOR(width-1 downto 0));
end component;
component mux2 generic(width: integer);
port(d0, d1: in STD_LOGIC_VECTOR(width-1 downto 0);
 s: in STD_LOGIC;
 y: out STD_LOGIC_VECTOR(width-1 downto 0));
end component;
signal PCNext, PCPlus4,
 PCPlus8: STD_LOGIC_VECTOR(31 downto 0);
signal ExtImm, Result: STD_LOGIC_VECTOR(31 downto 0);
signal SrcA, SrcB: STD_LOGIC_VECTOR(31 downto 0);
signal RA1, RA2: STD_LOGIC_VECTOR(3 downto 0);
begin
 -- next PC logic
 pcmux: mux2 generic map(32)
 port map(PCPlus4, Result, PCSrc, PCNext);
 pcreg: flopr generic map(32) port map(
clk, reset, PCNext, PC);
 pcadd1: adder port map(PC, X"00000004", PCPlus4);
 pcadd2: adder port map(
PCPlus4, X"00000004", PCPlus8);
 -- register file logic
 ralmux: mux2 generic map (4)
 port map(Instr(19 downto 16), "1111", RegSrc(0), RA1);
 ra2mux: mux2 generic map
(4) port map(Instr(3 downto 0), Instr(15 downto 12),
RegSrc(1), RA2);
 rf: regfile port map(
clk, RegWrite, RA1, RA2, Instr(15 downto 12), Result,
PCPlus8, SrcA, WriteData);
 resmux: mux2 generic map(32)
 port map(ALUResult, ReadData, MemtoReg, Result);
 ext: extend port map(Instr(23 downto 0), ImmSrc, ExtImm);

 -- ALU logic
 srcbmux: mux2 generic map(32)
 port map(WriteData, ExtImm, ALUSrc, SrcB);
 i_alu: alu port map(
SrcA, SrcB, ALUControl, ALUResult, ALUFlags);
end;

```

### 7.6.2 Altri blocchi costruttivi

Questo paragrafo contiene alcuni blocchi costruttivi generici utili in qualsiasi sistema digitale, come un banco di registri, un sommatore, dei *flip-flop* e un multiplexer 2:1. Il codice HDL per l'ALU è lasciato come Esercizi 5.11 e 5.12.

#### ESEMPIO HDL 7.6 BLOCCO DI REGISTRI

##### SystemVerilog

```
module regfile(input logic clk,
 input logic we3,
 input logic [3:0] ra1, ra2,
 wa3,
 input logic [31:0] wd3, r15,
 output logic [31:0] rd1, rd2);
 logic [31:0] rf[14:0];
 // three ported register file
 // read two ports combinationaly
 // write third port on rising edge of clock
 // register 15 reads PC+8 instead
 always_ff @(posedge clk)
 if (we3) rf[wa3] <= wd3;
 assign rd1 = (ra1 == 4'b1111) ? r15 : rf[ra1];
 assign rd2 = (ra2 == 4'b1111) ? r15 : rf[ra2];
endmodule
```

##### VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;
use IEEE.NUMERIC_STD_UNSIGNED.all;
entity regfile is -- three-port register file
 port(clk: in STD_LOGIC;
 we3: in STD_LOGIC;
 ra1, ra2, wa3: in STD_LOGIC_VECTOR(3 downto 0);
 wd3, r15: in STD_LOGIC_VECTOR(31 downto 0);
 rd1, rd2: out STD_LOGIC_VECTOR(31 downto 0));
end;

architecture behave of regfile is
 type ramtype is array (31 downto 0) of
 STD_LOGIC_VECTOR(31 downto 0);
 signal mem: ramtype;
begin
 process(clk) begin
 if rising_edge(clk) then
 if we3 = '1' then mem(to_integer(wa3)) <= wd3;
 end if;
 end if;
 end process;
 process(all) begin
 if (to_integer(ra1) = 15) then rd1 <= r15;
 else rd1 <= mem(to_integer(ra1));
 end if;
 if (to_integer(ra2) = 15) then rd2 <= r15;
 else rd2 <= mem(to_integer(ra2));
 end if;
 end process;
end;
```

#### ESEMPIO HDL 7.7 SOMMATORE

##### SystemVerilog

```
module adder #(parameter WIDTH=8)
 (input logic [WIDTH-1:0] a, b,
 output logic [WIDTH-1:0] y);
 assign y = a + b;
endmodule
```

##### VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;
use IEEE.NUMERIC_STD_UNSIGNED.all;
entity adder is -- adder
 port(a, b: in STD_LOGIC_VECTOR(31 downto 0);
 y: out STD_LOGIC_VECTOR(31 downto 0));
end;

architecture behave of adder is
begin
 y <= a + b;
end;
```

**ESEMPIO HDL 7.8    ESTENSIONE DI UN IMMEDIATO****SystemVerilog**

```

module extend(input logic [23:0] Instr,
 input logic [1:0] ImmSrc,
 output logic [31:0] ExtImm);

always_comb
 case(ImmSrc)
 // 8-bit unsigned immediate
 2'b00: ExtImm = {24'b0, Instr[7:0]};
 // 12-bit unsigned immediate
 2'b01: ExtImm = {20'b0, Instr[11:0]};
 // 24-bit two's complement shifted branch
 2'b10: ExtImm = {{6{Instr[23]}}, Instr[23:0],
 2'b00};
 default: ExtImm = 32'bx; // undefined
 endcase
endmodule

```

**VHDL**

```

library IEEE; use IEEE.STD_LOGIC_1164.all;
entity extend is
 port(Instr: in STD_LOGIC_VECTOR(23 downto 0);
 ImmSrc: in STD_LOGIC_VECTOR(1 downto 0);
 ExtImm: out STD_LOGIC_VECTOR(31 downto 0));
end;

architecture behave of extend is
begin
 process(all) begin
 case ImmSrc is
 when "00" => ExtImm <= (X"000000", Instr(7 downto 0));
 when "01" => ExtImm <= (X"000000", Instr(11 downto 0));
 when "10" => ExtImm <= (
Instr(23), Instr(23), Instr(23), Instr(23), Instr(23),
Instr(23), Instr(23 downto 0), "00");
 when others => ExtImm <= X"- - - - -";
 end case;
 end process;
end;

```

**ESEMPIO HDL 7.9    FLIP-FLOP RESETTABILE****SystemVerilog**

```

module flopr #(parameter WIDTH = 8)
 (input logic clk, reset,
 input logic [WIDTH-1:0] d,
 output logic [WIDTH-1:0] q);
 always_ff @(posedge clk, posedge reset)
 if (reset) q <= 0;
 else q <= d;
endmodule

```

**VHDL**

```

library IEEE; use IEEE.STD_LOGIC_1164.all;
entity flopr is -- flip-flop with synchronous reset
 generic(width: integer);
 port(clk, reset: in STD_LOGIC;
 d: in STD_LOGIC_VECTOR(width-1 downto 0);
 q: out STD_LOGIC_VECTOR(width-1 downto 0));
end;

architecture asynchronous of flopr is
begin
 process(clk, reset) begin
 if reset then q <= (others => '0');
 elsif rising_edge(clk) then
 q <= d;
 end if;
 end process;
end;

```

**ESEMPIO HDL 7.10 FLIP-FLOP RESETTABILE CON SEGNALE DI ABILITAZIONE****SystemVerilog**

```

module flopenr #(parameter WIDTH = 8)
 (input logic clk, reset, en,
 input logic [WIDTH-1:0] d,
 output logic [WIDTH-1:0] q);

 always_ff @(posedge clk, posedge reset)
 if (reset) q <= 0;
 else if (en) q <= d;
endmodule

```

**VHDL**

```

library IEEE; use IEEE.STD_LOGIC_1164.all;
entity flopenr is -- flip-flop with enable and
 synchronous reset
 generic(width: integer);
 port(clk, reset, en: in STD_LOGIC;
 d: in STD_LOGIC_VECTOR(width-1 downto 0);
 q: out STD_LOGIC_VECTOR(width-1 downto 0));
end;

architecture asynchronous of flopenr is
begin
 process(clk, reset) begin
 if reset then q <= (others => '0');
 elsif rising_edge(clk) then
 if en then
 q <= d;
 end if;
 end if;
 end process;
end;

```

**ESEMPIO HDL 7.11 MULTIPLEXER 2:1****SystemVerilog**

```

module mux2 #(parameter WIDTH = 8)
 (input logic [WIDTH-1:0] d0, d1,
 input logic s,
 output logic [WIDTH-1:0] y);

 assign y = s ? d1 : d0;
endmodule

```

**VHDL**

```

library IEEE; use IEEE.STD_LOGIC_1164.all;
entity mux2 is -- two-input multiplexer
 generic(width: integer);
 port(d0, d1: in STD_LOGIC_VECTOR(width-1 downto 0);
 s: in STD_LOGIC;
 y: out STD_LOGIC_VECTOR(width-1 downto 0));
end;

architecture behave of mux2 is
begin
 y <= d1 when s else d0;
end;

```

**7.6.3 Testbench**

Il codice di testbench carica un programma in memoria. Tale programma, riportato nella **Figura 7.60**, collauda tutte le istruzioni facendo un calcolo che produce il risultato atteso solo se tutte le istruzioni sono state eseguite in modo corretto dal processore. Nello specifico, il risultato atteso è la scrittura del valore 7 nella cella di memoria di indirizzo 100, prodotto se la struttura circuitale funziona correttamente e molto probabilmente non prodotto se qualcosa non funziona. È un classico esempio di collaudo *ad hoc*.

Il codice macchina del programma è memorizzato in esadecimale in un file denominato memfile.dat, caricato dal testbench durante la simulazione. Il file contiene il codice macchina di ciascuna istruzione, una per riga. I codici HDL del modulo testbench, del modulo di primo livello ARM e delle memorie esterne sono riportati negli Esempi HDL seguenti. Ogni modulo di memoria di questi esempi contiene 64 parole a 32 bit.

| ADDR      | PROGRAM            | ; COMMENTS                   | BINARY MACHINE CODE                      | HEX CODE |
|-----------|--------------------|------------------------------|------------------------------------------|----------|
| 00 MAIN   | SUB R0, R15, R15   | ; R0 = 0                     | 1110 000 0010 0 1111 0000 0000 0000 1111 | E04F000F |
| 04        | ADD R2, R0, #5     | ; R2 = 5                     | 1110 001 0100 0 0000 0010 0000 0000 0101 | E2802005 |
| 08        | ADD R3, R0, #12    | ; R3 = 12                    | 1110 001 0100 0 0000 0011 0000 0000 1100 | E280300C |
| 0C        | SUB R7, R3, #9     | ; R7 = 3                     | 1110 001 0010 0 0011 0111 0000 0000 1001 | E2437009 |
| 10        | ORR R4, R7, R2     | ; R4 = 3 OR 5 = 7            | 1110 000 1100 0 0111 0100 0000 0000 0010 | E1874002 |
| 14        | AND R5, R3, R4     | ; R5 = 12 AND 7 = 4          | 1110 000 0000 0 0011 0101 0000 0000 0100 | E0035004 |
| 18        | ADD R5, R5, R4     | ; R5 = 4 + 7 = 11            | 1110 000 0100 0 0101 0101 0000 0000 0100 | E0855004 |
| 1C        | SUBS R8, R5, R7    | ; R8 = 11 - 3 = 8, set Flags | 1110 000 0010 1 0101 1000 0000 0000 0111 | E0558007 |
| 20        | BEQ END            | ; shouldn't be taken         | 0000 1010 0000 0000 0000 0000 0000 1100  | 0A00000C |
| 24        | SUBS R8, R3, R4    | ; R8 = 12 - 7 = 5            | 1110 000 0010 1 0011 1000 0000 0000 0100 | E0538004 |
| 28        | BGE AROUND         | ; should be taken            | 1010 1010 0000 0000 0000 0000 0000 0000  | AA000000 |
| 2C        | ADD R5, R0, #0     | ; should be skipped          | 1110 001 0100 0 0000 0101 0000 0000 0000 | E2805000 |
| 30 AROUND | SUBS R8, R7, R2    | ; R8 = 3 - 5 = -2, set Flags | 1110 000 0010 1 0111 1000 0000 0000 0010 | E0578002 |
| 34        | ADDLT R7, R5, #1   | ; R7 = 11 + 1 = 12           | 1011 001 0100 0 0101 0111 0000 0000 0001 | B2857001 |
| 38        | SUB R7, R7, R2     | ; R7 = 12 - 5 = 7            | 1110 000 0010 0 0111 0111 0000 0000 0010 | E0477002 |
| 3C        | STR R7, [R3, #84]  | ; mem[12+84] = 7             | 1110 010 1100 0 0011 0111 0000 0101 0100 | E5837054 |
| 40        | LDR R2, [R0, #96]  | ; R2 = mem[96] = 7           | 1110 010 1100 1 0000 0010 0000 0110 0000 | E5902060 |
| 44        | ADD R15, R15, R0   | ; PC = PC+8 (skips next)     | 1110 000 0100 0 1111 1111 0000 0000 0000 | E08FF000 |
| 48        | ADD R2, R0, #14    | ; shouldn't happen           | 1110 001 0100 0 0000 0010 0000 0000 0001 | E280200E |
| 4C        | B END              | ; always taken               | 1110 1010 0000 0000 0000 0000 0000 0001  | EA000001 |
| 50        | ADD R2, R0, #13    | ; shouldn't happen           | 1110 001 0100 0 0000 0010 0000 0000 0001 | E280200D |
| 54        | ADD R2, R0, #10    | ; shouldn't happen           | 1110 001 0100 0 0000 0010 0000 0000 0001 | E280200A |
| 58 END    | STR R2, [R0, #100] | ; mem[100] = 7               | 1110 010 1100 0 0000 0010 0000 0101 0100 | E5802064 |

Figura 7.60 Codice assembly e codice macchina del programma di collaudo.

**ESEMPIO HDL 7.12 TESTBENCH****SystemVerilog**

```

module testbench();
 logic clk;
 logic reset;
 logic [31:0] WriteData, DataAdr;
 logic MemWrite;

 // instantiate device to be tested
 top dut(clk, reset, WriteData, DataAdr, MemWrite);

 // initialize test
 initial
 begin
 reset <= 1; # 22; reset <= 0;
 end

 // generate clock to sequence tests
 always
 begin
 clk <= 1; # 5; clk <= 0; # 5;
 end

 // check that 7 gets written to address 0x64
 // at end of program
 always @(negedge clk)
 begin
 if (MemWrite) begin
 if (DataAdr == 100 & WriteData == 7) begin
 $display("Simulation succeeded");
 $stop;
 end else if (DataAdr != 96) begin
 $display("Simulation failed");
 $stop;
 end
 end
 end
endmodule

```

**VHDL**

```

library IEEE;
use IEEE.STD_LOGIC_1164.all; use IEEE.NUMERIC_STD_UNSIGNED.all;
entity testbench is
end;

architecture test of testbench is
 component top
 port (clk, reset: in STD_LOGIC;
 WriteData, DataAdr: out STD_LOGIC_VECTOR(31 downto 0);
 MemWrite: out STD_LOGIC);
 end component;
 signal WriteData, DataAdr: STD_LOGIC_VECTOR(31 downto 0);
 signal clk, reset, MemWrite: STD_LOGIC;
begin
 - - instantiate device to be tested
 dut: top port map (clk, reset, WriteData, DataAdr, MemWrite);

 - - generate clock with 10 ns period
 process begin
 clk <= '1';
 wait for 5 ns;
 clk <= '0';
 wait for 5 ns;
 end process;

 - - generate reset for first two clock cycles
 process begin
 reset <= '1';
 wait for 22 ns;
 reset <= '0';
 wait;
 end process;

 - - check that 7 gets written to address 0x64
 - - at end of program

```

```

process (clk) begin
 if (clk'event and clk = '0' and MemWrite = '1') then
 if (to_integer(DataAdr) = 100 and
 to_integer(WriteData) = 7) then
 report "NO ERRORS: Simulation succeeded" severity
 failure;
 elsif (DataAdr /= 96) then
 report "Simulation failed" severity failure;
 end if;
 end if;
end process;
end;

```

### ESEMPIO HDL 7.13 MODULO DI PRIMO LIVELLO

#### SystemVerilog

```

module top(input logic clk, reset,
 output logic [31:0] WriteData, DataAdr,
 output logic MemWrite);

 logic [31:0] PC, Instr, ReadData;
 // instantiate processor and memories
 arm arm(clk, reset, PC, Instr, MemWrite,
 DataAdr, WriteData, ReadData);
 imem imem(PC, Instr);
 dmem dmem(clk, MemWrite, DataAdr, WriteData,
 ReadData);
endmodule

```

#### VHDL

```

library IEEE;
use IEEE.STD_LOGIC_1164.all; use IEEE.NUMERIC_STD_UNSIGNED.all;
entity top is -- top-level design for testing
 port(clk, reset: in STD_LOGIC;
 WriteData, DataAdr: buffer STD_LOGIC_VECTOR(31 downto 0);
 MemWrite: buffer STD_LOGIC);
end;

architecture test of top is
 component arm
 port(clk, reset: in STD_LOGIC;
 PC: out STD_LOGIC_VECTOR(31 downto 0);
 Instr: in STD_LOGIC_VECTOR(31 downto 0);
 MemWrite: out STD_LOGIC;
 ALUResult, WriteData: out STD_LOGIC_VECTOR(31 downto 0);
 ReadData: in STD_LOGIC_VECTOR(31 downto 0));
 end component;
 component imem
 port(a: in STD_LOGIC_VECTOR(31 downto 0);
 rd: out STD_LOGIC_VECTOR(31 downto 0));
 end component;
 component dmem
 port(clk, we: in STD_LOGIC;
 a, wd: in STD_LOGIC_VECTOR(31 downto 0);
 rd: out STD_LOGIC_VECTOR(31 downto 0));
 end component;
 signal PC, Instr,
 ReadData: STD_LOGIC_VECTOR(31 downto 0);
begin
 -- instantiate processor and memories
 i_arm: arm port map(
 clk, reset, PC, Instr, MemWrite, DataAdr, WriteData, ReadData);
 i_imem: imem port map(PC, Instr);
 i_dmem: dmem port map(
 clk, MemWrite, DataAdr, WriteData, ReadData);
end;

```



**ESEMPIO HDL 7.14 MEMORIA DATI****SystemVerilog**

```

module dmem(input logic clk, we,
 input logic [31:0] a, wd,
 output logic [31:0] rd);

 logic [31:0] RAM[63:0];

 assign rd = RAM[a[31:2]]; // word aligned

 always_ff @(posedge clk)
 if (we) RAM[a[31:2]] <= wd;
endmodule

```

**VHDL**

```

library IEEE;
use IEEE.STD_LOGIC_1164.all; use STD.TEXTIO.all;
use IEEE.NUMERIC_STD_UNSIGNED.all;
entity dmem is -- data memory
 port(clk, we: in STD_LOGIC;
 a, wd: in STD_LOGIC_VECTOR(31 downto 0);
 rd: out STD_LOGIC_VECTOR(31 downto 0));
end;

architecture behave of dmem is
begin
 process is
 type ramtype is array (63 downto 0) of
 STD_LOGIC_VECTOR(31 downto 0);
 variable mem: ramtype;
 begin -- read or write memory
 loop
 if clk'event and clk = '1' then
 if (we = '1') then
 mem(to_integer(a(7 downto 2))) := wd;
 end if;
 end if;
 rd <= mem(to_integer(a(7 downto 2)));
 wait on clk, a;
 end loop;
 end process;
end;

```

**ESEMPIO HDL 7.15 MEMORIA ISTRUZIONI****SystemVerilog**

```

module imem(input logic [31:0] a,
 output logic [31:0] rd);

 logic [31:0] RAM[63:0];

 initial
 $readmemh("memfile.dat", RAM);
 assign rd = RAM[a[31:2]]; // word aligned
endmodule

```

**VHDL**

```

library IEEE;
use IEEE.STD_LOGIC_1164.all; use STD.TEXTIO.all;
use IEEE.NUMERIC_STD_UNSIGNED.all;
entity imem is -- instruction memory
 port(a: in STD_LOGIC_VECTOR(31 downto 0);
 rd: out STD_LOGIC_VECTOR(31 downto 0));
end;

architecture behave of imem is -- instruction memory
begin
 process is
 file mem_file: TEXT;
 variable L: line;
 variable ch: character;
 variable i, index, result: integer;
 type ramtype is array (63 downto 0) of
 STD_LOGIC_VECTOR(31 downto 0);
 variable mem: ramtype;
 begin
 -- initialize memory from file
 for i in 0 to 63 loop -- set all contents low
 mem(i) := (others => '0');
 end loop;
 index := 0;
 FILE_OPEN(mem_file, "memfile.dat", READ_MODE);
 while not endfile(mem_file) loop

```

```

readline(mem_file, L);
result := 0;
for i in 1 to 8 loop
 read(L, ch);
 if '0' <= ch and ch <= '9' then
 result := character'pos(ch) - character'pos('0');
 elsif 'a' <= ch and ch <= 'f' then
 result := character'pos(ch) - character'pos('a')+10;
 elsif 'A' <= ch and ch <= 'F' then
 result := character'pos(ch) - character'pos('A')+10;
 else report "Format error on line " &
 integer'image(index) severity error;
 end if;
 mem(index)(35-i*4 downto 32-i*4) :=
 to_std_logic_vector(result,4);
end loop;
index := index + 1;
end loop;

-- read memory
loop
 rd <= mem(to_integer(a(7 downto 2)));
 wait on a;
end loop;
end process;
end;
```

## 7.7 ■ MICROARCHITETTURE AVANZATE\*

I microprocessori ad alte prestazioni usano una miriade di tecniche per eseguire più velocemente i programmi. Come visto prima, il tempo richiesto per eseguire un programma è proporzionale al periodo del clock e al numero di cicli di clock per istruzione (CPI): per migliorare le prestazioni serve quindi velocizzare il clock e/o diminuire il CPI. In questo paragrafo si introducono alcune tecniche utilizzate per aumentare le prestazioni. I dettagli implementativi sono piuttosto complessi, per cui ci si limita ai concetti. Il testo di Hennessy e Patterson *Computer Architecture* è senz'altro il riferimento per chi volesse approfondire questi dettagli.

I miglioramenti dei processi produttivi dei circuiti integrati hanno ridotto in modo continuo le dimensioni dei transistori. Transistor più piccoli sono più veloci e generalmente consumano meno potenza, quindi anche se la microarchitettura non viene modificata la frequenza di clock può essere aumentata perché tutte le porte logiche del circuito sono più veloci. Ma la riduzione delle dimensioni consente anche di mettere più transistor sullo stesso *chip*: i microarchitetti possono usare questi transistor per costruire processori più complessi, o per mettere addirittura più processori su un unico *chip*. Purtroppo il consumo di potenza aumenta con il numero di transistor e con la velocità alla quale lavorano (*vedi* par. 1.8): al giorno d'oggi la potenza è una preoccupazione fondamentale, e i progettisti di microprocessori devono affrontare la sfida sempre più difficile di trovare il miglior compromesso fra velocità, potenza e costo con i miliardi di transistor presenti in sistemi tra i più complessi mai costruiti dall'uomo.

### 7.7.1 Pipeline lunghe

Oltre a contare sui miglioramenti produttivi, il modo più semplice per velocizzare il clock è quello di dividere la pipeline in più stadi: ogni stadio contiene meno logica, quindi può essere più veloce. Nel capitolo si è considerata una pipeline a cinque stadi, ma 10-20 stadi sono molto frequenti nei processori moderni.

Il massimo numero di stadi di pipeline è limitato dalle dipendenze, dal sovraccarico di sequenziamento e dal costo. Pipeline più lunghe introducono più dipendenze: alcune possono essere risolte con inoltri ma altre richiedono stalli che aumentano il CPI. I registri di pipeline tra uno stadio e l'altro comportano un sovraccarico di sequenziamento per il ritardo dal clock all'uscita e il ritardo di setup, oltre a una deriva del segnale di clock (clock skew): questo sovraccarico fa sì che aggiungere stadi di pipeline porti vantaggi sempre minori. Infine, aggiungere stadi comporta costi maggiori per i registri di pipeline aggiuntivi e per le componenti hardware necessarie per gestire le dipendenze.

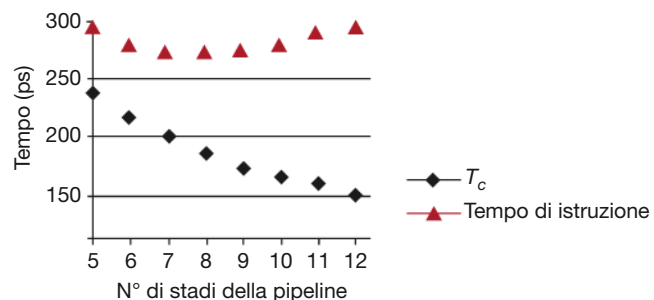
#### ESEMPIO 7.9

Verso la fine degli anni '90 e nei primi anni 2000 i processori erano pubblicizzati sul mercato soprattutto in termini di frequenza di clock ( $1/T_c$ ). Questo ha spinto verso processori con pipeline molto lunghe (20-31 stadi nel Pentium 4) per massimizzare la frequenza di clock, anche se gli incrementi effettivi di prestazioni erano discutibili. La potenza consumata è infatti proporzionale alla frequenza di clock e cresce con il numero di registri di pipeline, ecco perché ora che il consumo di potenza è un aspetto così importante le pipeline tendono ad accorciarsi.

Si vuole costruire un processore pipeline suddividendo il processore a ciclo singolo in  $N$  stadi. Il processore a ciclo singolo ha un ritardo di propagazione di 740 ps nella sua logica combinatoria. Il sovraccarico di sequenziamento di un registro è di 90 ps. Si supponga che il ritardo della logica combinatoria possa essere arbitrariamente diviso in un numero qualsiasi di stadi e che la gestione delle dipendenze non aumenti il ritardo. La pipeline a cinque stadi dell'Esempio 7.7 ha un CPI di 1.23. Si supponga anche che ogni stadio aggiuntivo aumenti il CPI di 0.1 a causa dei salti mal previsti e di altre dipendenze. Quanti stadi di pipeline devono essere usati per eseguire i programmi alla massima velocità possibile?

**Soluzione** Il tempo di ciclo per una pipeline a  $N$  stadi è  $T_c = (740/N + 90)$  ps e il CPI vale  $1.23 + 0.1(N - 5)$ . Il tempo di istruzione è il prodotto del tempo di ciclo e del CPI. Nella **Figura 7.61** sono tracciati il tempo di ciclo  $T_c$  e il tempo di istruzione in funzione del numero di stadi. Il tempo di istruzione ha un minimo di 279 ps per  $N = 8$  stadi, solo di poco migliore del tempo di istruzione di 293 ps ottenuto con la pipeline a cinque stadi.

**Figura 7.61**  
Tempo di ciclo,  $T_c$ , e tempo di istruzione in funzione del numero di stadi di pipeline.



### 7.7.2 Micro operazioni

Tra gli altri, si sono citati a suo tempo i due principi per una buona progettazione, “la regolarità favorisce la semplicità” e “rendere veloci le cose frequenti”. Le architetture a set di istruzioni ridotto (RISC) “ortodosse” come MIPS presentano solo istruzioni semplici, che possono essere eseguite in un solo ciclo su un percorso dati semplice e veloce, con un banco di registri a tre porte, una sola ALU, e un solo accesso alla memoria dati, come quelle usate in questo capitolo. Le architetture a set di istruzioni complesso (CISC), invece,

includono generalmente istruzioni che necessitano di più registri, più addizioni, più di un accesso a memoria per istruzione. Per esempio, l'istruzione x86 `ADD [ESP], [EDX + 80 + EDI*2]` deve leggere tre registri, sommare la base, lo spiazzamento e l'indice scalato, leggere due locazioni di memoria, sommare i valori ottenuti e scrivere di nuovo in memoria il risultato. Un microprocessore capace di fare tutte queste cose nello stesso ciclo risulterebbe inutilmente più lento nell'eseguire le istruzioni più semplici e più frequenti nei programmi.

Gli architetti di calcolatori rendono veloci i casi frequenti definendo un insieme di semplici **micro operazioni** (spesso citate in inglese come *micro-ops* o *μops*) che possono essere eseguite su percorsi dati semplici. Le istruzioni vere e proprie vengono scomposte in una o più micro operazioni. Per esempio, se si definiscono delle micro operazioni che assomigliano alle istruzioni base di ARM e alcuni registri temporanei T1 e T2 per memorizzare risultati intermedi, l'istruzione x86 appena citata può tradursi nelle seguenti sette micro operazioni:

```
ADD T1, [EDX + 80] ; T1 <- EDX + 80
LSL T2, EDI, 2 ; T2 <- EDI*2
ADD T1, T2, T2 ; T1 <- EDX + 80 + EDI*2
LDR T1, [T1] ; T1 <- MEM[EDX + 80 + EDI*2]
LDR T2, [ESP] ; T2 <- MEM[ESP]
ADD T1, T2, T1 ; T1 <- MEM[ESP] + MEM[EDX + 80 + EDI*2]
STR T1, [ESP] ; MEM[ESP] <- MEM[ESP] + MEM[EDX + 80 + EDI*2]
```

Anche se la maggior parte delle istruzioni ARM è costituita da istruzioni semplici, alcune possono comunque essere scomposte in micro operazioni. Per esempio, i caricamenti da memoria con indirizzamento a post indicizzazione (come `LDR R1, [R2], #4`) richiedono una seconda porta di scrittura nel banco di registri, e istruzioni di elaborazione dati con indirizzamento a registro e registro traslato (come `ORR R3, R4, R5, LSL R6`) richiedono una terza porta di lettura dal banco di registri. Invece di costruire un banco di registri più grande con cinque porte, il percorso dati di ARM può scomporre queste istruzioni complesse in coppie di istruzioni più semplici:

| Operazione complessa                | Sequenza di micro operazioni                               |
|-------------------------------------|------------------------------------------------------------|
| <code>LDR R1, [R2], #4</code>       | <code>LDR R1, [R2]</code><br><code>ADD R2, R2, #4</code>   |
| <code>ORR R3, R4, R5, LSL R6</code> | <code>LSL T1, R5, R6</code><br><code>ORR R3, R4, T1</code> |

Anche se il programmatore avrebbe potuto scrivere direttamente le sequenze di istruzioni più semplici nel programma, l'istruzione complessa occupa meno spazio in memoria, e dal momento che leggere istruzioni dalla memoria richiede potenza, l'istruzione complessa fa risparmiare potenza. Il successo del set di istruzioni di ARM è dovuto, in parte, alle scelte oculate degli architetti che hanno definito istruzioni che assicurano una densità maggiore del codice rispetto alle architetture RISC ortodosse come MIPS ma che garantiscono una maggiore efficienza di decodifica rispetto alle architetture CISC come x86.

### 7.7.3 Previsione dei salti

Come detto, un processore pipeline dovrebbe avere idealmente un CPI pari a 1. La causa principale di aumento del CPI è la penalizzazione per salti mal previsti. Con l'allungarsi delle pipeline, la decisione se saltare o meno viene presa più avanti nella pipeline, con il risultato di aumentare la penalizzazione

I progettisti delle microarchitetture devono decidere se fornire hardware per realizzare direttamente un'operazione complessa oppure se suddividerla in una sequenza di micro operazioni. Simili decisioni devono essere prese anche relativamente ad altri aspetti discussi più avanti in questa sezione. Naturalmente queste decisioni portano a punti differenti nello spazio progettuale prestazioni/potenza/costo.

per salti mal previsti perché la pipeline deve essere svuotata di tutte le istruzioni iniziate dopo il salto. Per affrontare questo problema, la maggior parte dei processori *pipeline* adotta un **predittore di salto** (*branch predictor*) per cercare di prevedere se il salto andrà eseguito o meno. Nel caso visto nel paragrafo 7.5.3, la previsione della *pipeline* era semplicemente che il salto non dovesse essere fatto.

Alcuni salti sono posizionati alla fine di un ciclo, e saltano indietro all'inizio del ciclo per ripeterlo (per es. nei cicli *for* e *while*). I cicli sono di solito eseguiti molte volte, quindi i salti all'indietro molto spesso sono da fare. Una forma molto semplice di previsione dei salti è quindi quella che verifica la direzione del salto e assume che i salti all'indietro debbano essere fatti. Viene chiamata **previsione statica dei salti** perché non dipende da ciò che è successo nell'esecuzione del programma, cioè nella sua storia.

I salti in avanti sono più difficili da prevedere senza conoscere meglio la struttura del programma in esecuzione, quindi molti processori usano una **previsione dinamica dei salti** che si basa sulla storia del programma per cercare di indovinare se il salto vada o meno eseguito. I predittori dinamici memorizzano una tabella che contiene le ultime centinaia (se non migliaia) di istruzioni di salto eseguite dal processore. La tabella, denominata **buffer delle destinazioni di salto** (*branch target buffer*) o anche **tabella di previsione dei salti** (*branch prediction table*), include la destinazione di ciascun salto e la storia del salto, ovvero se sia stato o meno eseguito in passato.

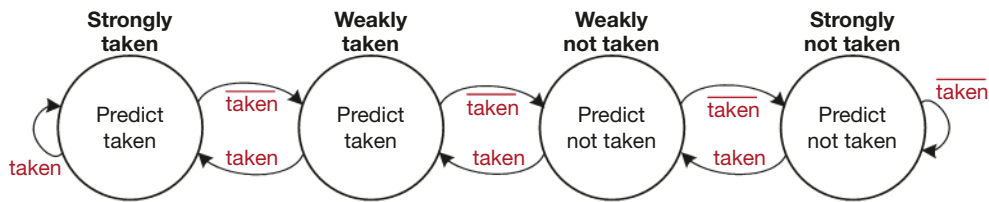
Per comprendere le operazioni di un predittore dinamico dei salti, si consideri il seguente ciclo preso dall'Esempio di Codice 6.17. Il ciclo viene ripetuto 10 volte, e il salto *BGE* che esce dal ciclo viene eseguito solo una volta all'ultima iterazione del ciclo stesso.

```

 MOV R1, #0
 MOV R0, #0
FOR CMP R0, #10
 BGE FINE
 ADD R1, R1, R0
 ADD R0, R0, #1
 B FOR
FINE
```

Un predittore dinamico a un bit ricorda se il salto è stato eseguito oppure no l'ultima volta che è stato incontrato, e quando lo incontra di nuovo ripete la scelta precedente: finché il ciclo viene ripetuto, si ricorda che l'istruzione *BGE* non aveva saltato all'iterazione precedente, e prevede di non dover saltare neanche all'iterazione corrente. Tale previsione si rivela corretta fino all'ultima iterazione, quando il salto va fatto. Sfortunatamente, se successivamente si ritorna a eseguire lo stesso ciclo, il predittore ricorda che l'ultima volta il salto era stato fatto, e prevede erroneamente di doverlo fare nella prima iterazione del ciclo. In totale quindi il predittore dinamico dei salti a un bit sbaglia la previsione nella prima e nell'ultima iterazione di un ciclo.

Un predittore dinamico dei salti a due bit risolve parzialmente il problema memorizzando quattro stati relativi a ogni salto, denominati *strongly taken*, *weakly taken*, *weakly not taken*, *strongly not taken* (traducibili come salto “decisamente da fare”, “forse da fare”, “forse da non fare”, “decisamente da non fare”), come mostrato nella **Figura 7.62**. Quando il ciclo viene ripetuto, il predittore entra nello stato *strongly not taken* e prevede di non dover saltare la prossima volta che incontrerà il salto. La previsione è corretta fino all'ultima iterazione del ciclo, quando il salto va eseguito, cosa che sposta il predittore nello stato *weakly not taken*. Se si rientra una seconda volta nel ciclo, il pre-



**Figura 7.62** Diagramma degli stati per un predittore di salto a due bit.

Il predittore prevede correttamente di non dover saltare, e si sposta di nuovo nello stato *strongly not taken*, quindi in conclusione sbaglia soltanto la previsione relativa all'ultima iterazione del ciclo.

Il predittore opera nello stadio Fetch della pipeline per decidere di quale istruzione fare il fetch nel prossimo ciclo: se la previsione dice che il salto dovrebbe essere fatto, il processore preleva l'indirizzo dell'istruzione di cui fare il fetch dal buffer delle destinazioni di salto.

Come si può intuire, i predittori possono tenere una traccia anche più dettagliata della storia del programma per migliorare l'accuratezza delle previsioni: i migliori predittori riescono a raggiungere un'accuratezza superiore al 90% nell'esecuzione dei programmi più tipici.

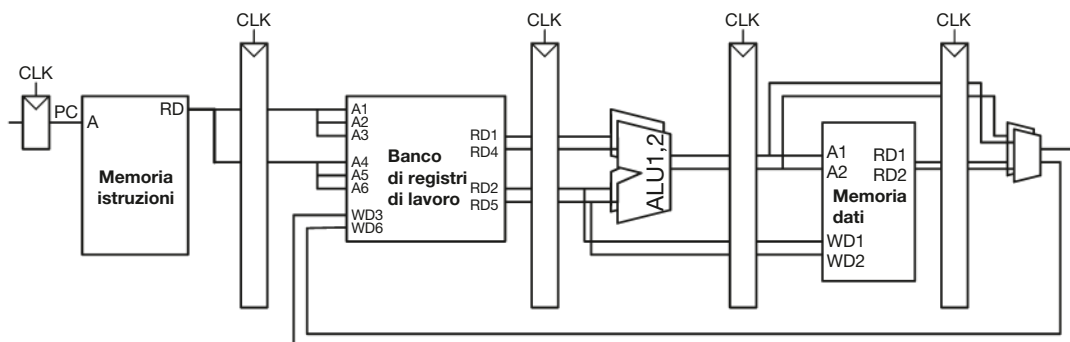
#### 7.7.4 Processori superscalari

Un **processore superscalare** contiene più copie dell'hardware del percorso dati, per poter eseguire più istruzioni contemporaneamente. La **Figura 7.63** mostra lo schema a blocchi di un processore superscalare a due vie, che attiva due istruzioni in ogni ciclo. Il percorso dati esegue il fetch di due istruzioni alla volta dalla memoria istruzioni; ha un banco di registri a sei porte per consentire di leggere quattro operandi sorgente e scrivere due risultati in ogni ciclo; contiene inoltre due ALU e una memoria dati a due porte per eseguire contemporaneamente le due istruzioni.

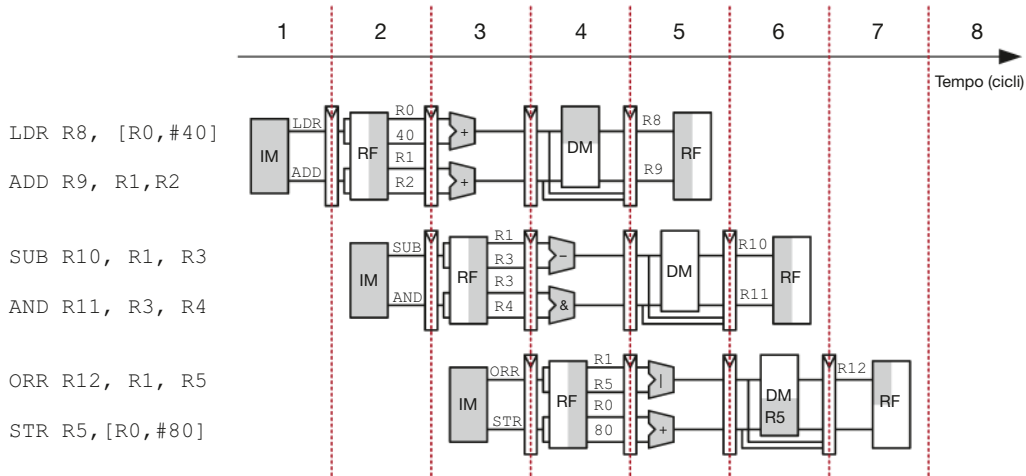
La **Figura 7.64** mostra la rappresentazione schematica del funzionamento della pipeline di un processore superscalare che esegue due istruzioni in ogni ciclo: per il programma mostrato, il processore ha quindi un CPI pari a 0.5. A volte i progettisti fanno riferimento al reciproco del CPI, denominato IPC (*Instructions Per Cycle*): in questo caso, il processore in questione ha un IPC pari a 2.

L'esecuzione simultanea di più istruzioni crea problemi per le dipendenze. La **Figura 7.65** mostra per esempio la rappresentazione schematica del funzionamento della pipeline che sta eseguendo un programma con dipendenze di dato, mostrate in rosso nel codice: l'istruzione **ADD** è dipendente per R8, che è prodotto dall'istruzione **LDR**, quindi non può essere attivata insieme alla

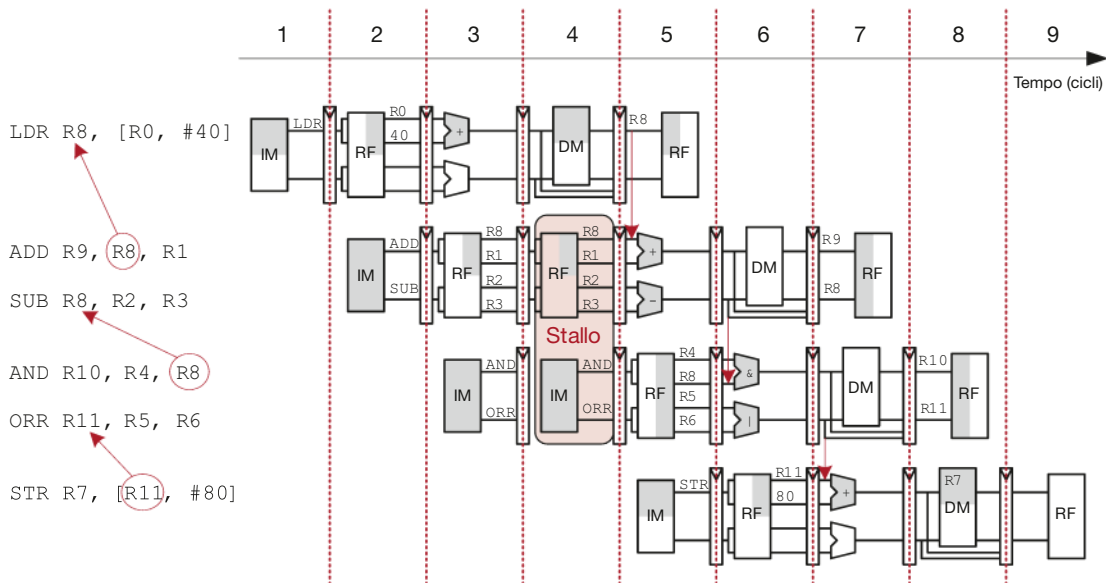
Un processore **scalare** opera su un solo "pezzo" di dati alla volta (per es. i due addendi di una somma). Un processore **vettoriale** opera su più pezzi di dati svolgendo la stessa istruzione. Un processore **superscalare** esegue più istruzioni alla volta, ciascuna su un singolo pezzo di dati. Il processore ARM pipeline è un processore scalare. I processori vettoriali erano molto usati nei supercalcolatori negli anni '80 e '90 perché erano in grado di gestire in modo efficiente i lunghi vettori di dati presenti nei calcoli scientifici, e sono oggi molto utilizzati nelle unità di elaborazione grafica (GPU, *Graphics Processing Unit*). I moderni microprocessori ad alte prestazioni sono superscalari, perché eseguire più istruzioni fra loro indipendenti è molto più flessibile che elaborare vettori. I moderni processori hanno però anche unità hardware per la gestione di corti vettori di dati, molto usati in applicazioni multimediali e grafiche. Tali unità sono denominate SIMD (*Single Instruction Multiple Data*) e sono state discusse nel paragrafo 6.7.5.



**Figura 7.63** Percorso dati di tipo superscalare.



**Figura 7.64** Rappresentazione schematica del funzionamento di una pipeline superscalare.



**Figura 7.65** Programma con dipendenze di dato.

LDR ma deve entrare nella pipeline un ciclo dopo; inoltre l'istruzione ADD deve essere messa in stallo per un ulteriore ciclo per consentire alla LDR di inoltrarle R8 nel ciclo 5. Le altre dipendenze (fra SUB e AND per R8 e fra ORR e STR per R11) sono gestite inoltrando i risultati prodotti in un ciclo per utilizzarli immediatamente nel ciclo successivo. Servono quindi sei cicli per attivare le cinque istruzioni, con un IPC pari a 1.2.

Quando si parla di parallelismo di esecuzione si fa riferimento a due forme di parallelismo: temporale e spaziale. La pipeline è un caso di parallelismo temporale, mentre unità di esecuzione multiple sono un caso di parallelismo spaziale. I processori superscalari sfruttano entrambe le forme di parallelismo per "spremere" dalla microarchitettura tutte le prestazioni possibili ben oltre i limiti dei processori a ciclo singolo o multi ciclo.

I processori commerciali hanno strutture superscalari a tre, quattro e anche sei vie, e devono gestire le dipendenze non solo di dati ma anche di controllo come i salti. Purtroppo i programmi reali contengono molte dipendenze, quindi i processori superscalari a molte vie difficilmente sfruttano appieno



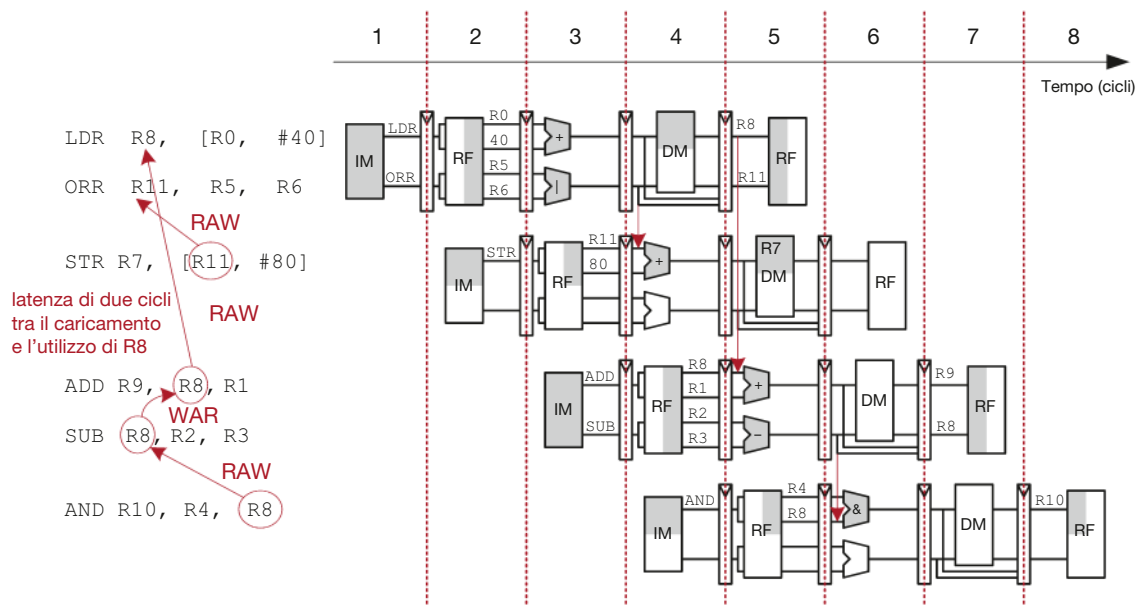
tutte le unità di esecuzione, senza contare che grossi numeri di unità di esecuzione e reti di inoltro complesse consumano grosse quantità di circuiteria e di potenza.

### 7.7.5 Processore *out-of-order*

Per far fronte al problema delle dipendenze, un processore *out-of-order* (letteralmente, fuori ordine) esamina un certo numero di prossime istruzioni per iniziare a eseguire istruzioni indipendenti il più rapidamente possibile: le istruzioni possono infatti venire attivate in ordine diverso dal quello scritto dal programmatore, a patto naturalmente che le dipendenze vengano rispettate e che il programma produca comunque i risultati previsti.

Si consideri ancora il programma della Figura 7.65 da eseguire su un processore *out-of-order* superscalare a due vie: il processore può attivare due istruzioni qualsiasi del programma in ogni ciclo, a patto di rispettare le dipendenze. La **Figura 7.66** mostra le dipendenze di dato presenti e le operazioni svolte dal processore (la classificazione delle dipendenze come RAW o WAR viene discussa sotto). I vincoli da rispettare per attivare le istruzioni sono i seguenti:

- **Ciclo 1**
  - L’istruzione LDR viene attivata.
  - Le istruzioni ADD, SUB e AND dipendono da LDR per R8 quindi non possono ancora essere attivate, ma l’istruzione ORR è indipendente, quindi viene attivata anche lei.
- **Ciclo 2**
  - Ricordando che c’è una latenza di due cicli tra un’istruzione LDR e un’istruzione che dipende da lei, ADD non può essere attivata perché dipendente per R8; SUB scrive R8 quindi non può essere attivata prima di ADD altrimenti questa riceverebbe un valore di R8 errato; AND dipende da SUB.
  - Solo l’istruzione STR viene attivata.



**Figura 7.66** Esecuzione fuori ordine di un programma con dipendenze.

- **Ciclo 3**
  - In questo ciclo R8 è disponibile, quindi **ADD** viene attivata. Anche **SUB** viene attivata, perché modifica R8 solo dopo che **ADD** ha già letto il valore corretto.
- **Ciclo 4**
  - L'istruzione **AND** viene attivata. R8 viene inoltrato da **SUB** a **AND**.

Il processore out-of-order attiva dunque sei istruzioni in quattro cicli, con un IPC di 1.5.

La dipendenza dell'istruzione **ADD** dall'istruzione **LDR** a causa di R8 è una **dipendenza di tipo RAW (Read After Write)**: **ADD** non deve leggere R8 finché **LDR** non ha scritto in R8 il proprio risultato: è il tipo di dipendenza che si incontra nei processori pipeline. Inevitabilmente riduce la velocità di esecuzione dei programmi, anche se sono disponibili numerose unità di esecuzione. Analogamente sono di tipo RAW le dipendenze di **STR** da **ORR** per R11 e di **AND** da **SUB** per R8.

La dipendenza di **SUB** da **ADD** a causa di R8 è una **dipendenza di tipo WAR (Write After Read)**, nota anche come **antidipendenza**. **SUB** non deve scrivere R8 prima che **ADD** lo abbia letto, in modo che **ADD** usi il corretto valore secondo l'ordine originario delle istruzioni del programma. Le dipendenze di tipo WAR non possono verificarsi in una pipeline normale, mentre possono verificarsi in processori out-of-order se l'istruzione dipendente (**SUB** in questo caso) viene anticipata troppo.

La dipendenza di tipo WAR non è intrinsecamente presente nel programma: è semplicemente una scelta del programmatore quella di usare lo stesso registro per due istruzioni che in realtà sono indipendenti; se l'istruzione **SUB** avesse usato R12 invece di R8, la dipendenza non ci sarebbe stata e **SUB** avrebbe potuto essere attivata prima di **ADD**. Il problema è che l'architettura ARM ha solo 16 registri, quindi il programmatore può essere costretto a riutilizzare un registro (introducendo una dipendenza) solo perché tutti gli altri registri sono occupati.

Un terzo tipo di dipendenza, non presente nel programma considerato, è la **dipendenza di tipo WAW (Write After Write)**, nota anche come **dipendenza di uscita**. Questa dipendenza è presente quando un'istruzione deve scrivere in un registro nel quale deve scrivere anche un'istruzione successiva del programma. Se la prima istruzione scrive nel registro quando l'istruzione successiva lo ha già fatto, il registro contiene alla fine un valore sbagliato. Per esempio, nel codice seguente, sia **LDR** sia **ADD** devono scrivere in R8. In base all'ordine delle istruzioni nel programma il valore finale di R8 deve essere quello calcolato da **ADD**, ma se un processore out-of-order esegue **ADD** per prima la dipendenza di tipo WAW produce un risultato scorretto.

```
LDR R8, [R3]
ADD R8, R1, R2
```

Anche le dipendenze di tipo WAW non sono intrinsecamente presenti: di nuovo è una scelta del programmatore quella di usare lo stesso registro per due istruzioni indipendenti; se l'istruzione **ADD** viene attivata per prima, il programma può eliminare la dipendenza semplicemente scartando il risultato di **LDR** invece di scriverlo in R8. Questa operazione è denominata *squashing* (soffocamento) di **LDR**.<sup>4</sup>

<sup>4</sup> Ci si potrebbe chiedere se non sia il caso di non attivare del tutto l'istruzione **LDR**: il motivo per farlo è che il processore out-of-order deve garantire anche il verificarsi degli stessi errori che si sarebbero verificati nell'esecuzione in ordine del programma. L'istruzione **LDR** può potenzialmente produrre un'eccezione di tipo *Data Abort*, quindi deve essere comunque attivata per controllare se l'eccezione si verifica o meno, anche se poi il suo risultato viene scartato.

I processori out-of-order usano una tabella per tenere traccia delle istruzioni che attendono di essere attivate; la tabella contiene informazioni riguardanti le dipendenze, e la sua dimensione determina come ovvio il numero di istruzioni che possono essere considerate per decidere quali attivare. A ogni ciclo il processore esamina la tabella e attiva il maggior numero di istruzioni possibile, in base alle dipendenze e al numero di unità di esecuzione (cioè ALU e porte di memoria) disponibili.

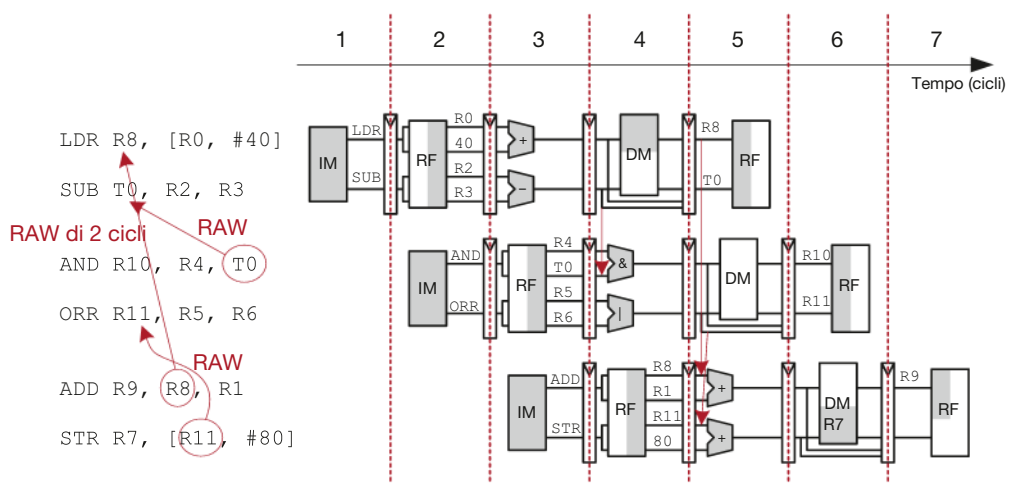
Il **parallelismo a livello di istruzioni** (ILP, *Instruction Level Parallelism*) è il numero di istruzioni che possono essere eseguite simultaneamente per un certo programma e una certa microarchitettura. Studi teorici hanno dimostrato che ILP può essere abbastanza elevato per microarchitetture con predittori di salto perfetti e con un gran numero di unità di esecuzione: in pratica, però, raramente si ottengono ILP superiori a 2 o 3 anche con processori out-of-order superscalari a sei vie.

### 7.7.6 Ridenominazione dei registri

I processori out-of-order usano una tecnica chiamata **ridenominazione dei registri** (*register renaming*) per gestire le dipendenze di tipo WAR e WAW. Tale tecnica aggiunge alcuni registri non architetturali da usare per la ridenominazione. Per esempio, un processore potrebbe aggiungere 20 registri di ridenominazione: T0-T19. Il programmatore non può usare direttamente questi registri, perché non fanno parte dell'architettura, mentre può usarli il processore per gestire le dipendenze.

Nel paragrafo precedente, era presente una dipendenza di tipo WAR tra le istruzioni SUB e ADD a causa del riutilizzo di R8. Il processore out-of-order può ridenominare R8 come T0 per l'istruzione SUB, che può in questo modo essere eseguita prima, perché T0 non causa dipendenza con l'istruzione ADD. Il processore memorizza in una tabella quali registri sono stati ridenominati per effettuare altre ridenominazioni in modo consistente nelle istruzioni successive che hanno dipendenze: in questo esempio, R8 deve pure essere ridenominato T0 per l'istruzione AND, che fa riferimento al risultato di SUB.

La **Figura 7.67** mostra lo stesso programma della Figura 7.65 eseguito da un processore out-of-order con ridenominazione dei registri: R8 viene ridenominato T0 per SUB e AND per eliminare la dipendenza di tipo WAR. I vincoli da rispettare per attivare le istruzioni sono i seguenti:



**Figura 7.67** Esecuzione fuori ordine di un programma con ridenominazione dei registri.

- Ciclo 1
  - L'istruzione `LDR` viene attivata.
  - L'istruzione `ADD` dipende da `LDR` per `R8`, quindi non può ancora essere attivata, ma l'istruzione `SUB` è ora indipendente poiché il suo registro destinazione è stato ridenominato `T0`, quindi viene attivata anche lei.
- Ciclo 2
  - Ricordando che c'è una latenza di due cicli tra un'istruzione `LDR` e un'istruzione che dipende da lei, `ADD` non può essere attivata perché dipende per `R8`.
  - L'istruzione `AND` dipende da `SUB`, quindi può essere attivata; `T0` viene inoltrato da `SUB` a `AND`.
  - L'istruzione `ORR` è indipendente, quindi viene attivata.
- Ciclo 3
  - In questo ciclo `R8` è disponibile, quindi `ADD` viene attivata.
  - Anche `R11` è disponibile, quindi viene attivata anche `STR`.

Il processore out-of-order con ridenominazione dei registri attiva sei istruzioni in tre cicli, con un IPC pari a 2.

### 7.7.7 Multithreading

Dal momento che l'ILP di programmi reali tende a essere piuttosto basso, introdurre più unità di esecuzione in un processore superscalare dà un ritorno in termini di prestazioni progressivamente minore. Un altro problema, discusso nel Capitolo 8, è il fatto che la memoria è molto più lenta del processore: la maggior parte delle letture e delle scritture accede a una piccola memoria veloce, la memoria *cache*, ma quando un dato non è presente in *cache* il processore può dover aspettare anche più di 100 cicli prima che tale dato venga recuperato dalla memoria principale. La tecnica chiamata *multithreading* aiuta a mantenere occupato il processore con molte unità di esecuzione anche se l'ILP di un programma è basso o se il programma è bloccato in attesa di dati dalla memoria.

Per spiegare il multithreading serve introdurre un po' di terminologia. Un programma in esecuzione su un calcolatore è denominato **processo**. I calcolatori possono eseguire molti processi in parallelo: per esempio si può ascoltare una canzone su PC mentre si naviga in Internet e mentre il programma antivirus fa uno dei suoi controlli. Ogni processo a sua volta è composto da uno o più *thread* ("fili") che possono essere anche loro eseguiti in parallelo: per esempio, un programma di elaborazione testi (*word processor*) può avere un thread che gestisce l'utente che sta inserendo il testo, un secondo thread che controlla l'ortografia mentre l'utente lavora e un terzo thread che stampa un documento. In questo modo, l'utente non deve aspettare ad esempio la fine della stampa di un documento per iniziare a lavorare su un altro documento. Il numero di thread in cui un processo può essere suddiviso prende il nome di *Thread Level Parallelism* (TLP).

In un processore convenzionale, i thread danno solo l'illusione di essere eseguiti in parallelo: in realtà vengono eseguiti a turno dal processore, sotto il controllo del sistema operativo. Quando il turno di un thread finisce, il sistema operativo salva il suo stato architetturale, carica lo stato architetturale del prossimo thread e inizia a eseguirlo. Questa procedura è definita **cambio di contesto** (*context switching*): se il processore passa da un contesto all'altro abbastanza frequentemente, l'utente ha l'impressione che tutti i thread siano eseguiti effettivamente in parallelo.

Un processore multithread contiene più di una copia del proprio stato architetturale, in modo che più di un thread possa essere attivo in ogni istante. Per esempio, in un processore con quattro program counter e 64 registri possono essere disponibili fino a quattro thread simultaneamente. Se un thread si blocca in attesa di dati dalla memoria, il processore può commutare il contesto a un altro thread senza alcun ritardo perché il program counter e i registri del secondo thread sono già disponibili. Inoltre, se un thread non ha abbastanza parallelismo nelle istruzioni da occupare tutte le unità di esecuzione di un processore superscalare, un altro thread può attivare le proprie istruzioni sulle unità di esecuzione inutilizzate.

Il multithreading non migliora le prestazioni del singolo thread, perché non aumenta l'ILP. Tuttavia migliora la capacità globale di lavoro del processore, perché più thread possono usare le risorse del processore che sarebbero state lasciate inutilizzate dall'esecuzione di un singolo thread. È una tecnica relativamente economica da realizzare perché richiede di replicare solo PC e banco di registri, non unità di esecuzione né memorie.

### 7.7.8 Multiprocessori

*con il contributo di Matthew Watkins*

I processori moderni hanno un numero enorme di transistori disponibili: usarli per aumentare la lunghezza della pipeline o per aggiungere unità di esecuzione a un processore superscalare porta vantaggi limitati e spreca potenza elettrica. Intorno al 2005, gli architetti di calcolatori hanno introdotto un importante cambiamento di strategia inserendo più copie del processore nello stesso chip: ciascuna di queste copie prende il nome di **core**.

Un sistema **multiprocessore** consiste dunque di più processori e di una struttura di comunicazione tra i processori stessi. Le tre tipologie più comuni di multiprocessori sono i multiprocessori simmetrici (detti anche omogenei), i multiprocessori eterogenei e i **cluster**.

#### Multiprocessori simmetrici

I multiprocessori simmetrici consistono di due o più processori identici che condividono la stessa memoria principale. Questi processori possono essere realizzati in chip diversi oppure essere diversi core nello stesso chip.

I multiprocessori possono essere usati sia per eseguire più thread simultaneamente sia per eseguire più rapidamente un singolo thread. Eseguire più *thread* simultaneamente è facile: basta distribuire i thread fra i processori. Sfortunatamente i tipici utenti di PC hanno bisogno di eseguire in parallelo solo pochi thread alla volta. Eseguire un singolo thread più velocemente è molto più complicato: il programmatore deve suddividere il thread esistente in molti thread per farli eseguire dai diversi processori. La cosa diventa problematica quando i processori devono comunicare tra loro. In effetti, una delle sfide più ardue per i progettisti di calcolatori e per i programmatori è usare in modo efficiente grandi numeri di processori core.

I processori simmetrici hanno un certo numero di vantaggi. Sono relativamente facili da progettare, perché una volta disegnato il processore basta replicarlo un certo numero di volte. Anche programmare un multiprocessore simmetrico e fargli eseguire il codice prodotto è relativamente facile, perché ogni programma può essere eseguito da ogni processore circa con le stesse prestazioni.

#### Multiprocessori eterogenei

Purtroppo continuare ad aggiungere core simmetrici non garantisce un aumento continuo di prestazioni. Facendo riferimento alla situazione del 2015, le applicazioni software tipiche usano pochi thread alla volta, e un utente tipi-

co ha di solito un paio di applicazioni simultaneamente in esecuzione. Questo è sufficiente a tenere occupati sistemi *dual-core* o *quad-core* (con due o quattro core) ma se i programmi non cominciano a incorporare al loro interno più parallelismo, continuare ad aggiungere core non porta a benefici significativi. C'è poi un altro aspetto: dal momento che i processori per uso generale sono progettati per offrire buone prestazioni medie, non sono la soluzione più efficiente dal punto di vista energetico per svolgere un'operazione specifica. E questa inefficienza energetica è particolarmente critica in sistemi con vincoli di potenza stringenti, come i telefoni portatili.

I multiprocessori eterogenei affrontano questi aspetti incorporando tipi diversi di core e/o hardware specializzato in un singolo sistema. Ogni applicazione può quindi usare le risorse del sistema che forniscono le migliori prestazioni, o il miglior rapporto potenza/prestazioni, per quella particolare applicazione. Vista l'abbondanza di transistori al giorno d'oggi, il fatto che non tutte le applicazioni usino ogni pezzo di hardware disponibile non è così preoccupante. I sistemi eterogenei possono avere tante forme: si possono infatti incorporare *core* con microarchitetture differenti, con diversi rapporti potenza/prestazioni/area di chip.

Una strategia eterogenea resa popolare da ARM è *big.LITTLE*, che si basa su sistemi contenenti sia *core* a basso consumo sia core ad alte prestazioni. I core "LITTLE" come il Cortex-A53 sono processori in grado di attivare una o due istruzioni alla volta, in ordine, con consumi limitati di potenza, adatti ai compiti routinari. I core "big" come il Cortex-A57 sono processori out-of-order superscalari più complessi, in grado di offrire elevate prestazioni per i picchi di lavoro.

Un'altra strategia eterogenea è quella che fa uso di acceleratori: il sistema contiene hardware specializzato ottimizzato in termini di prestazioni o di potenza per compiti specifici. Per esempio, un sistema a singolo chip per applicazioni mobili contiene oggi acceleratori per elaborazioni grafiche e video, comunicazione senza fili, attività in tempo reale, crittografia. Nello svolgimento di queste attività, gli acceleratori possono essere 10-100 volte più efficienti dei processori di uso generale. I processori di segnali digitali (DSP, *Digital Signal Processor*) sono un altro esempio di acceleratori, con un set di istruzioni ottimizzato per l'esecuzione di programmi con molti calcoli matematici.

I sistemi eterogenei hanno però anche i loro difetti: aggiungono complessità sia nel progetto dei diversi elementi eterogenei sia nello sforzo di programmazione per decidere quando e come usare i diversi tipi di risorse. I sistemi simmetrici e quelli eterogenei trovano entrambi il loro campo di applicazione nei moderni sistemi: i sistemi simmetrici sono ideali in situazioni come i grossi centri di elaborazione dati, con tanti thread simultaneamente in esecuzione; i sistemi eterogenei sono adatti dove si hanno carichi di lavoro variabili o specialistici.

## Cluster

Nei multiprocessori a cluster ogni processore ha la sua struttura di memoria locale. Un esempio di cluster è un gruppo di personal computer collegati in rete che eseguono insieme del software per risolvere un problema di grandi dimensioni. Un altro esempio di cluster divenuto sempre più importante è un centro di elaborazione dati (*data center*) con molti armadi di calcolatori e di memorie a disco collegati in rete e che condividono alimentazione elettrica e sistema di raffreddamento. Le più importanti aziende dell'era Internet come Google, Amazon e Facebook hanno spinto il rapido sviluppo dei centri di elaborazione dati per supportare milioni di utenti sparsi nel mondo.

Gli scienziati che cercano tracce dell'esistenza di forme di vita extraterrestre usano il più grande cluster multiprocessore per analizzare i dati provenienti dai radio telescopi alla ricerca di sequenze che potrebbero costituire segni di tale esistenza negli altri sistemi solari. Il *cluster*, in attività dal 1999, è costituito dai personal computer di più di 6 milioni di volontari nel mondo. Quando un personal computer del cluster è inattivo, preleva una parte di dati da un server centrale, li analizza e invia i risultati al server. Gli interessati a diventare volontari del cluster possono fare riferimento al sito [setiathome.berkeley.edu](http://setiathome.berkeley.edu).



## 7.8 ■ UNO SGUARDO AL MONDO REALE: EVOLUZIONE DELL'ARCHITETTURA ARM\*

Questo paragrafo illustra l'evoluzione dell'architettura e della microarchitettura ARM dalla sua nascita nel 1985. La **Tabella 7.7** ne riassume gli aspetti essenziali, mostrando un miglioramento di un fattore 10 nell'IPC e un aumento di un fattore 250 della frequenza di clock nell'arco di tre decenni e otto successive versioni dell'architettura. La frequenza, l'area sul chip e la potenza variano con la tecnologia di produzione e con gli obiettivi, il piano di lavoro e l'abilità dei progettisti. Le frequenze rappresentative sono riferite al processo produttivo disponibile al momento dell'introduzione di ogni prodotto sul mercato, quindi il miglioramento in termini di frequenza è quasi tutto dovuto alle dimensioni dei transistori e non alla microarchitettura. La dimensione relativa è normalizzata alla dimensione dei transistori e può variare anche di molto in base alle dimensioni delle memorie cache e ad altri fattori.

La **Figura 7.68** mostra la fotografia del chip del processore ARM1, che conteneva 25 000 transistori ed era costituito da una pipeline a tre stadi. Contando con attenzione si possono vedere i 32 bit del percorso dati in basso. Il banco di registri è sulla sinistra e l'ALU sulla destra. In fondo a sinistra c'è il program counter: i due bit meno significativi in basso sono vuoti (bloccati a 0) e i sei in alto sono diversi perché sono usati per i bit di stato. L'unità di controllo sta sopra il percorso dati. Alcuni dei blocchi rettangolari sono circuiti PLA che realizzano la logica di controllo. I rettangoli sulla cornice sono i contatti di ingresso/uscita, con sottili fili d'oro che escono dall'immagine.

Nel 1990 Acorn ha generato uno spin-off con i componenti del gruppo di progetto creando l'azienda Advanced RISC Machines (successivamente denominata ARM Holdings), che ha iniziato concedendo i diritti di produzione dell'architettura ARMv3. L'architettura aveva spostato i bit di stato dal PC al Registro di Stato Corrente del Programma ed esteso il PC a 32 bit. Apple aveva acquisito una quota importante di ARM e usato ARM610 nel suo computer Newton, il primo PDA (*Personal Digital Assistant*) del mondo, e una delle prime applicazioni commerciali del riconoscimento della scrittura a mano. Newton si era rivelato troppo avanzato per l'epoca, ma ha gettato le basi per PDA di maggiore successo e successivamente per gli smartphone e i tablet.

DMIPS (*Dhrystone Millions of Instructions Per Second*) è un'unità di misura delle prestazioni.

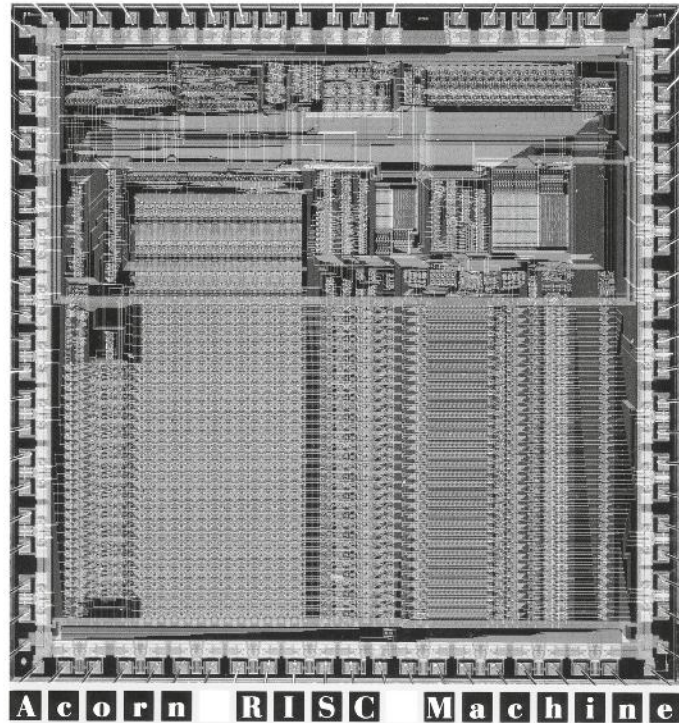
**Tabella 7.7** Evoluzione dei processori ARM.

| Microarchitettura | Anno | Architettura | Stadi di pipeline | DMIPS/MHz | Frequenza di clock (MHz) | Cache L1          | Dimensione relativa |
|-------------------|------|--------------|-------------------|-----------|--------------------------|-------------------|---------------------|
| ARM1              | 1985 | v1           | 3                 | 0.33      | 8                        | Nessuna           | 0.1                 |
| ARM6              | 1992 | v3           | 3                 | 0.65      | 30                       | 4 KB unica        | 0.6                 |
| ARM7              | 1994 | v4T          | 3                 | 0.9       | 100                      | 0-8 KB unica      | 1                   |
| ARM9E             | 1999 | v5TE         | 5                 | 1.1       | 300                      | 0-16 KB I + D     | 3                   |
| ARM11             | 2002 | v6           | 8                 | 1.25      | 700                      | 4-64 KB I + D     | 30                  |
| Cortex-A9         | 2009 | v7           | 8                 | 2.5       | 1000                     | 16-64 KB I + D    | 100                 |
| Cortex-A7         | 2011 | v7           | 8                 | 1.9       | 1500                     | 8-64 KB I + D     | 40                  |
| Cortex-A15        | 2011 | v7           | 15                | 3.5       | 2000                     | 32 KB I + D       | 240                 |
| Cortex-M0*        | 2012 | v7M          | 2                 | 0.93      | 60-250                   | Nessuna           | 0.3                 |
| Cortex-A53        | 2012 | v8           | 8                 | 2.3       | 1500                     | 8-64 KB I + D     | 50                  |
| Cortex-A57        | 2012 | v8           | 15                | 4.1       | 2000                     | 48 KB I + 32 KB D | 300                 |



**Figura 7.68****Fotografia del chip ARM1.**

(Con permesso di riproduzione da ARM. © 1985 ARM Ltd.)



Sophie Wilson e Steve Furber hanno progettato insieme ARM1.

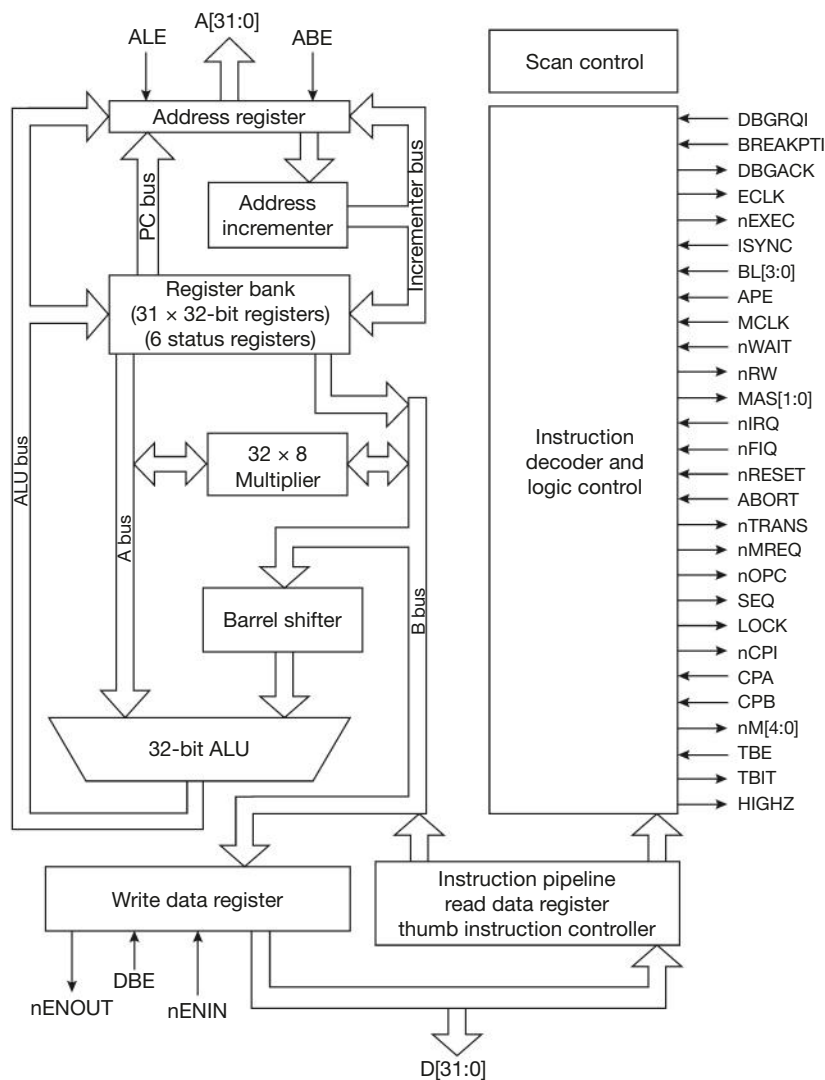
**Sophie Wilson (1957-)** è nata nello Yorkshire, in Inghilterra, e ha studiato Informatica all'Università di Cambridge. Ha progettato il sistema operativo e scritto l'interprete BBC del linguaggio Basic per il computer Acorn, ha poi collaborato alla progettazione di ARM1 e dei successori fino ad ARM7. Nel 1999 ha progettato il processore SIMD per elaborazione di segnali Firepath creando un'azienda spin off acquistata da Broadband nel 2001. Attualmente è Senior Director della Broadband Corporation e Fellow della Royal Society, della Royal Academy of Engineering, della British Computer Society e della Women's Engineering Society.



(Fotografia © Sophie Wilson. Con permesso di riproduzione.)

ARM ha ottenuto un grosso successo con la linea ARM7 nel 1994, soprattutto con ARM7TDMI, divenuto uno dei processori RISC più usati nei sistemi *embedded* per i successivi 15 anni. ARM7TDMI usava il set di istruzioni ARMv4T che aveva introdotto le istruzioni *Thumb* per ottenere una maggiore densità del codice e definito le *halfword* e le istruzioni di caricamento e scrittura di *byte* con segno. La sigla TDMI sta per *Thumb, JTAG Debug, fast Multiply and In-circuit debug*. Le varie caratteristiche legate al *debug* aiutavano il programmatore a scrivere il codice direttamente sull'hardware e a collaudarlo collegandolo semplicemente con un cavo al PC, con grossi risparmi di tempo. ARM7 usava una semplice pipeline a tre stadi: Fetch, Decode ed Execute. Il processore aveva una *cache* unica per istruzioni e dati. Dal momento che la *cache* in un processore pipeline è normalmente occupata in ogni ciclo, ARM7 metteva in stallo le istruzioni di accesso a memoria nello stadio Execute per lasciare il tempo alla *cache* di accedere i dati. La **Figura 7.69** mostra uno schema a blocchi del processore. Invece di produrlo direttamente, ARM dava i diritti di produzione ad altre aziende, che lo hanno inserito come parte di sistemi a singolo chip. Questi clienti potevano comperare il processore come *hard macro* (cioè come matrice completa ma non flessibile di circuito integrato, da inserire direttamente in un chip) oppure come *soft macro* (cioè codice Verilog da sintetizzare a cura del cliente). ARM7 è stato usato in una gran quantità di prodotti, inclusi telefoni portatili, iPod Apple, Mindstorms NXT della Lego, videogame Nintendo, automobili. Da allora, quasi tutti i telefoni portatili sono stati realizzati con processori ARM.

La versione ARM9E ha migliorato ARM7 con una pipeline a cinque stadi simile a quella descritta in questo capitolo, con *cache* separate per istruzioni e dati e nuove istruzioni *Thumb* e di elaborazione di segnali digitali nell'architettura ARMv5TE. La **Figura 7.70** mostra uno schema a blocchi di ARM9 con molti degli elementi descritti in questo capitolo, oltre a un moltiplicatore e un traslatore; i segnali IA/ID/DA/DD sono i bus indirizzi e dati per la memoria istruzioni e dati, e IAREg è il PC. La generazione successiva ARM11 ha



**Figura 7.69**  
**Schema a blocchi di ARM7.**  
(Con permesso di riproduzione di  
ARM. © 1998 ARM Ltd.)

**Steve Furber (1953-)** è nato a Manchester, Inghilterra, e ha conseguito il dottorato di ricerca in aerodinamica all'Università di Cambridge. È entrato alla Acorn Computer, dove ha collaborato al progetto del calcolatore BBC Micro e del microprocessore ARM1. Nel 1990 è entrato all'Università di Manchester: le sue ricerche si sono focalizzate sull'elaborazione asincrona e sui sistemi neurali.



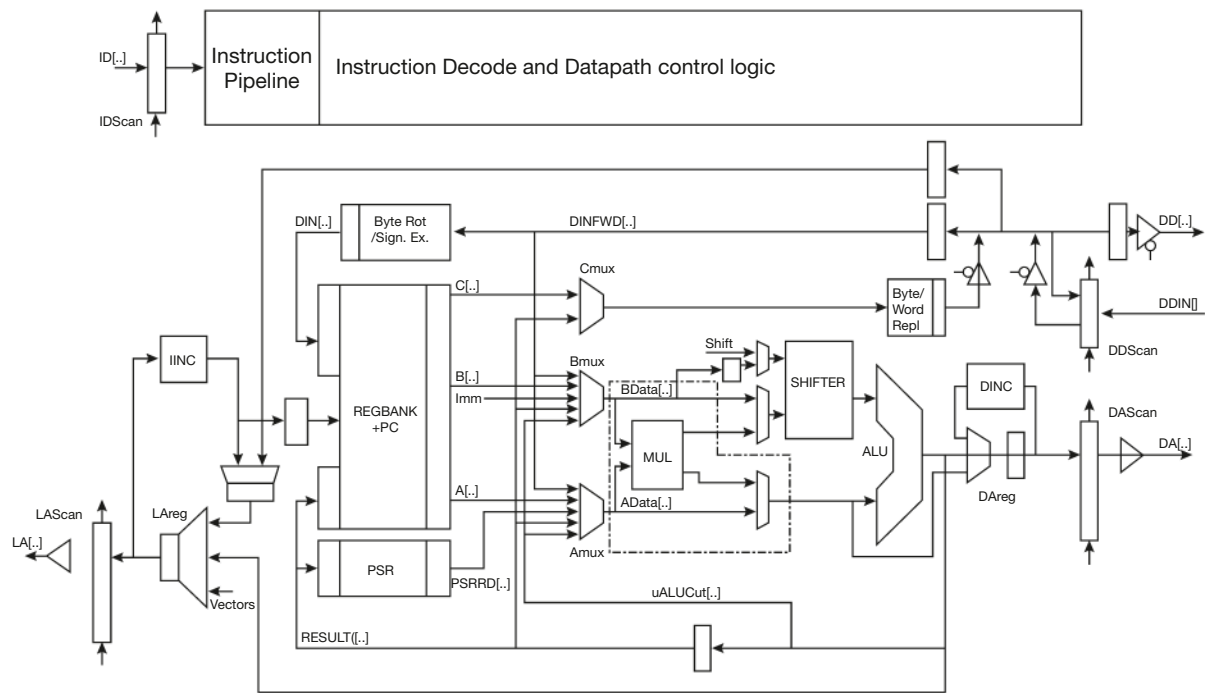
(Fotografia © 2012 The University of Manchester. Con permesso di riproduzione.)

ulteriormente esteso la pipeline a otto stadi per aumentare le prestazioni e definito le istruzioni *Thumb2* e SIMD.

Il set di istruzioni ARMv7 ha aggiunto istruzioni SIMD avanzate operanti su registri di due e quattro parole, e definito la variante v7-M per supportare le sole istruzioni *Thumb*. ARM ha introdotto le famiglie di processori Cortex-A e Cortex-M. La famiglia Cortex-A ad alte prestazioni è oggi usata praticamente in tutti gli smartphone e i tablet. La famiglia Cortex-M, basata sul set di istruzioni *Thumb*, comprende piccoli ed economici microcontrollori usati nei sistemi *embedded*. Per esempio, il Cortex-M0+ ha una pipeline a due stadi e richiede solo 12 000 porte logiche, rispetto alle centinaia di migliaia di un processore della serie A. Costa molto meno di un dollaro come chip autonomo, e meno di un centesimo se integrato in un sistema a singolo chip più complesso. Il consumo di potenza è circa di 3  $\mu\text{W}/\text{MHz}$ , quindi il processore alimentato da una batteria da orologio può funzionare ininterrottamente per circa un anno a 10 MHz.

I processori ARMv7 di fascia alta hanno invaso il mercato degli smartphone e dei tablet. Cortex-A9 è stato molto usato in telefoni portatili, spesso come parte di un sistema a singolo chip a due core contenente due processori Cortex-A9, un acceleratore grafico, un modem cellulare e altre periferiche. La **Figura 7.71** mostra uno schema a blocchi di Cortex-A9. Il processore decodi-

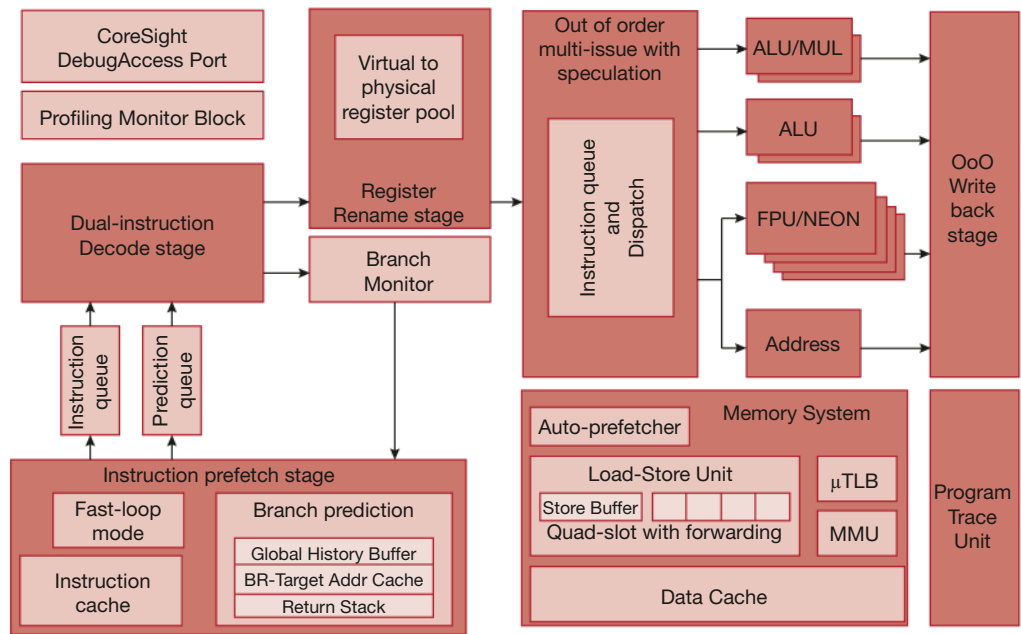
**Figura 7.71** mostra uno schema a blocchi di Cortex-A9. Il processore decodi-



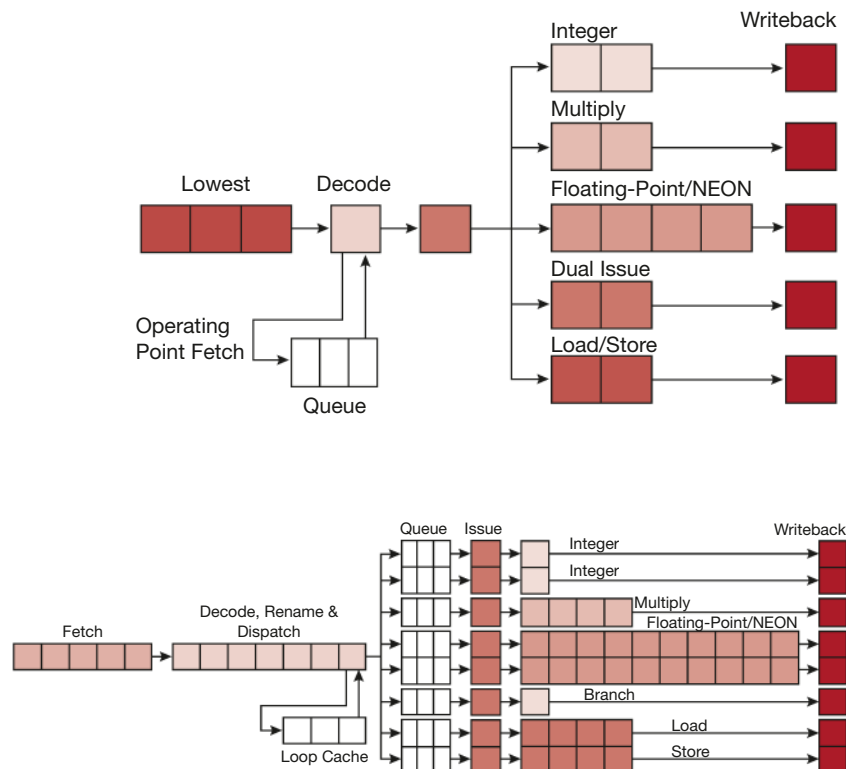
**Figura 7.70** Schema a blocchi di ARM9. (Con permesso di riproduzione da ARM9TDMI Technical Reference Manual. © 1999 ARM Ltd.)

fica due istruzioni a ogni ciclo, esegue ridenominazione dei registri e invia le istruzioni out-of-order alle unità di esecuzione.

Efficienza energetica e prestazioni sono entrambe aspetti critici per i dispositivi mobili, quindi ARM ha introdotto l'architettura *big.LITTLE* che unisce vari core “big” ad alte prestazioni per i picchi di carico con core “LITTLE” ad alta efficienza energetica per gestire la maggior parte dei pro-



**Figura 7.71** Schema a blocchi di Cortex-A9. (Questa figura proviene dagli autori e non implica approvazione da parte di ARM.)



**Figura 7.72** Schemi a blocchi di Cortex-A7 e -A15. (Questa figura proviene dagli autori e non implica approvazione da parte di ARM.)

cessi routinari. Per esempio, il Samsung Exynos 5 Octa presente nel telefono Galaxy S5 contiene quattro “big” core Cortex-A15 a 2.1 GHz e quattro “LITTLE” core Cortex-A7 a 1.5 GHz. La **Figura 7.72** mostra gli schemi delle pipeline dei due tipi di core. Il Cortex-A7 è un processore *in-order* che può decodificare e attivare fino a una istruzione di accesso a memoria e un’altra istruzione a ogni ciclo. Il Cortex-A15 è un processore *out-of-order* molto più complesso che può decodificare fino a tre istruzioni per ciclo. La lunghezza della pipeline è circa raddoppiata, per gestire la complessità e aumentare la velocità del clock, quindi un predittore di salto più accurato è necessario per compensare la maggiore penalizzazione per salti mal previsti. Il Cortex-A15 fornisce prestazioni che sono circa 2.5 volte quelle di Cortex-A7, ma consuma 6 volte la potenza di quest’ultimo. Gli smartphone possono usare i *core* “big” solo per poco tempo prima che il chip si surriscaldi e limiti automaticamente le proprie prestazioni.

L’architettura ARMv8 è un’architettura a 64 bit. Cortex-A53 e Cortex-A57 hanno pipeline simili rispettivamente a quelle di Cortex-A7 e Cortex-A15, ma allargano i registri e i percorsi dati a 64 bit per gestire ARMv8. Apple ha diffuso l’architettura a 64 bit nel 2013 quando ha introdotto le nuove versioni di iPhone e iPad.

## 7.9 ■ RIASSUNTO

In questo capitolo si sono descritti tre modi di costruire un processore, ciascuno con differenti rapporti costo/prestazioni. Dopo questo capitolo, quella che poteva sembrare una macchina complessa e misteriosa per i non addetti ai lavori dovrebbe invece risultare ragionevolmente comprensibile, con una struttura interna che non occupa più di mezza pagina.

Le microarchitetture presentate hanno usato praticamente tutti gli argomenti dei capitoli precedenti: mettere insieme i componenti della microarchitettura richiede il progetto di reti combinatorie e sequenziali (descritte nei Capitoli 2 e 3) e l'uso di molti dei blocchi funzionali visti nel Capitolo 5, oltre naturalmente alla conoscenza dell'architettura ARM di cui si è parlato nel Capitolo 6. Le microarchitetture possono essere descritte in qualche pagina di codice HDL usando le tecniche viste nel Capitolo 4.

Il progetto delle microarchitetture ha anche fatto pesante uso delle tecniche di gestione della complessità: il livello di astrazione della microarchitettura è il ponte tra quello delle reti logiche e quello dell'architettura, e costituisce quindi il punto cruciale di questo testo sul progetto di sistemi digitali e sull'architettura dei calcolatori. Si è usata anche l'astrazione degli schemi a blocchi e dei linguaggi HDL per dare una descrizione sintetica dei collegamenti tra i vari componenti. Le microarchitetture sfruttano anche la regolarità e la modularità, riutilizzando una libreria di blocchi costruttivi di uso generale come le ALU, le memorie, i multiplexer, i registri. La gerarchia è usata in vari modi: la microarchitettura è suddivisa in percorso dati e unità di controllo, e ciascuna di queste parti è realizzata con blocchi logici costruttivi, costituiti da porte logiche a loro volta ottenute mediante transistori, come visto nei primi cinque capitoli del testo.

In questo capitolo si sono confrontate le microarchitetture a ciclo singolo, multi ciclo e pipeline per il processore ARM: tutte le tre microarchitetture realizzano lo stesso sottoinsieme del set di istruzioni ARM e hanno il medesimo stato architetturale. La più semplice è quella del processore a ciclo singolo con un CPI pari a 1.

Il processore multi ciclo usa un numero variabile di passi più brevi per eseguire le istruzioni, quindi può riutilizzare l'ALU invece di richiedere la presenza di vari sommatore. Ha però bisogno di diversi registri non architetturali per memorizzare i risultati intermedi tra un passo e l'altro. In linea di principio potrebbe essere più veloce del precedente, perché non tutte le istruzioni hanno la stessa durata, ma in pratica si rivela spesso più lento perché il tempo di ciclo è vincolato dal passo più lungo e per il sovraccarico di sequenziamento in ogni passo.

Il processore pipeline divide il processore a ciclo singolo in cinque stadi di pipeline relativamente veloci, e aggiunge registri di pipeline tra gli stadi per separare le cinque istruzioni simultaneamente in esecuzione. Teoricamente ha un CPI pari a 1, ma le dipendenze costringono a stalli e svuotamenti che aumentano un po' il CPI. La gestione delle dipendenze costa anche in termini di hardware aggiuntivo e di complessità progettuale. Il periodo di clock potrebbe in teoria essere un quinto di quello del processore a ciclo singolo, ma in pratica è vincolato dallo stadio più lento e dal sovraccarico di sequenziamento di ogni stadio. In ogni caso la struttura pipeline assicura un sostanziale incremento di prestazioni, tanto che tutti i moderni microprocessori ad alte prestazioni la adottano.

Anche se nel capitolo si sono prese in esame solo alcune istruzioni dell'architettura ARM, si è visto che estendere il numero di istruzioni gestite dalla microarchitettura richiede semplicemente di estendere il percorso dati e l'unità di controllo.

Il limite principale di questo capitolo è l'ipotesi di avere una memoria veloce e grande abbastanza per contenere l'intero programma e i suoi dati: in realtà, memorie veloci e grandi hanno costi proibitivi. Quindi nel prossimo capitolo si illustra come poter disporre di quasi tutte le caratteristiche di una memoria veloce e grande usando una piccola memoria veloce che contiene le informazioni usate più di frequente e memorie molto più grandi ma più lente che contengono il resto delle informazioni.



## Esercizi

**Esercizio 7.1** Si supponga che uno dei seguenti segnali del processore ARM a ciclo singolo abbia un guasto del tipo “bloccato a 0” (*stuck-at-0*) cioè valga sempre 0, anche quando dovrebbe valere 1.

- (a) *RegW*
- (b) *ALUOp*
- (c) *MemW*

Quali istruzioni non saranno in grado di funzionare? Perché?

**Esercizio 7.2** Ripetere l'Esercizio 7.1 nell'ipotesi che il guasto sia del tipo “bloccato a 1” (*stuck-at-1*).

**Esercizio 7.3** Modificare il processore ARM a ciclo singolo per realizzare una delle seguenti istruzioni (fare riferimento all'Appendice B per la definizione delle istruzioni). Tracciare su una copia della Figura 7.13 le modifiche al percorso dati e dare un nome a tutti i nuovi segnali. Riportare su una copia delle Tabelle 7.2 e 7.3 le modifiche al decoder principale e al decoder dell'ALU, e descrivere tutte le altre modifiche resesi necessarie.

- (a) TST
- (b) LSL con traslazione di un immediato
- (c) CMN
- (d) ADC

**Esercizio 7.4** Ripetere l'Esercizio 7.3 per una delle seguenti istruzioni.

- (a) EOR
- (b) LSR con traslazione di un immediato
- (c) TEQ
- (d) RSB

**Esercizio 7.5** ARM possiede l'istruzione LDR con post-indicizzazione, che modifica il registro base dopo aver effettuato il caricamento da memoria. LDR *Rd*, [*Rn*], *Rm* è equivalente alle seguenti due istruzioni:

```
LDR Rd, [Rn]
ADD Rn, Rn, Rm
```

Ripetere l'Esercizio 7.3 per l'istruzione LDR con post-indicizzazione. È possibile aggiungere tale istruzione senza modificare il banco di registri?

**Esercizio 7.6** ARM possiede l'istruzione LDR con pre-indicizzazione, che modifica il registro base dopo aver effettuato il caricamento da memoria. LDR *Rd*, [*Rn*, *Rm*]! è equivalente alle seguenti due istruzioni:

```
LDR Rd, [Rn, Rm]
ADD Rn, Rn, Rm
```

Ripetere l'Esercizio 7.3 per l'istruzione LDR con pre-indicizzazione. È possibile aggiungere tale istruzione senza modificare il banco di registri?

**Esercizio 7.7** Il classico secchione di turno sostiene di poter riprogettare una qualsiasi delle unità del processore ARM a ciclo singolo e dimezzarne il ritardo. Con riferimento ai ritardi riportati nella Tabella 7.5, quale unità va riprogettata per ottenere il massimo incremento di prestazioni globali del processore? E quale sarà il tempo di ciclo del processore così migliorato?

**Esercizio 7.8** Si considerino i ritardi riportati nella Tabella 7.5. Ben Imbrogliabit realizza un sommatore a prefissi che riduce di 20 ps il ritardo dell'ALU. Se i ritardi degli altri elementi rimangono invariati, calcolare il nuovo tempo di ciclo del processore ARM a ciclo singolo e determinare quanto tempo è richiesto per eseguire un benchmark di 100 miliardi di istruzioni.

**Esercizio 7.9** Modificare il codice HDL del processore ARM a ciclo singolo, riportato nel paragrafo 7.6.1, per eseguire una delle nuove istruzioni indicate nell'Esercizio 7.3. Estendere il benchmark riportato nel paragrafo 7.6.3 per collaudare anche la nuova istruzione.

**Esercizio 7.10** Ripetere l'Esercizio 7.9 per una delle nuove istruzioni dell'Esercizio 7.4.

**Esercizio 7.11** Si supponga che uno dei seguenti segnali del processore ARM multi ciclo abbia un guasto del tipo “bloccato a 0” (*stuck-at-0*).

- (a) *RegSrc<sub>1</sub>*
- (b) *AdrSrc*
- (c) *NextPC*

Quali istruzioni non saranno in grado di funzionare? Perché?

**Esercizio 7.12** Ripetere l'Esercizio 7.11 nell'ipotesi che il guasto sia del tipo “bloccato a 1” (*stuck-at-1*).

**Esercizio 7.13** Modificare il processore ARM multi ciclo per realizzare una delle seguenti istruzioni (fare riferimento all'Appendice B per la definizione delle istruzioni). Tracciare su una copia della Figura 7.30 le modifiche al percorso dati e dare un nome a tutti i nuovi segnali. Tracciare su una copia della Figura 7.41 le modifiche alla FSM di controllo, e descrivere tutte le altre modifiche resesi necessarie.

- (a) ASR con traslazione di un immediato
- (b) TST
- (c) SBC
- (d) ROR con traslazione di un immediato

**Esercizio 7.14** Ripetere l'Esercizio 7.13 per le seguenti istruzioni.

- (a) BL
- (b) LDR (con spiazzamento immediato positivo o negativo)
- (c) LDRB (con spiazzamento immediato solo positivo)
- (d) BIC

**Esercizio 7.15** Ripetere l'Esercizio 7.5 per il processore ARM multi ciclo. Mostrare le modifiche al percorso dati multi ciclo e alla FSM di controllo. È possibile aggiungere tale istruzione senza modificare il banco di registri?

**Esercizio 7.16** Ripetere l'Esercizio 7.6 per il processore ARM multi ciclo. Mostrare le modifiche al percorso dati multi ciclo e alla FSM di controllo. È possibile aggiungere tale istruzione senza modificare il banco di registri?

**Esercizio 7.17** Ripetere l'Esercizio 7.7 per il processore ARM multi ciclo, considerando le istruzioni dell'Esempio 7.5.

**Esercizio 7.18** Ripetere l'Esercizio 7.8 per il processore ARM multi ciclo, considerando le istruzioni dell'Esempio 7.5.

**Esercizio 7.19** Il secchione di prima sostiene di poter riprogettare una qualsiasi delle unità del processore ARM multi ciclo e renderla molto più veloce. Con riferimento ai ritardi riportati nella Tabella 7.5, quale unità va riprogettata per ottenere il massimo incremento di prestazioni globali del processore? Quanto deve essere veloce? (Farla più veloce del necessario è un inutile sforzo di progetto, da evitare.) Quale sarà il tempo di ciclo del processore così migliorato?

**Esercizio 7.20** La Esagerata srl sostiene di avere il brevetto per la costruzione del banco di registri a tre porte. Invece di denunciare la Esagerata srl, Ben Imbrogliabit progetta un nuovo banco di registri con una sola porta di lettura/scrittura (come la memoria unica per istruzioni e dati). Riprogettare il percorso dati e l'unità di controllo del processore ARM multi ciclo per usare il suo nuovo banco di registri.

**Esercizio 7.21** Si supponga che le componenti del processore ARM multi ciclo abbiano i ritardi riportati nella Tabella 7.5. Alyssa Guastacomputer progetta un nuovo banco di registri che consuma il 40% in meno di potenza ma che ha un ritardo doppio del precedente. Le conviene utilizzare questo nuovo banco di registri più lento ma con minori consumi per il progetto del suo processore multi ciclo?

**Esercizio 7.22** Quanto vale il CPI (*Cycles Per Instruction*) della nuova versione del processore ARM multi ciclo dell'Esercizio 7.20? Considerare le istruzioni dell'Esempio 7.5.

**Esercizio 7.23** Quanti cicli sono necessari per eseguire il seguente programma sul processore ARM multi ciclo? Quanto vale il CPI di questo programma?

```
MOV R0, #5 ; risultato = 5
MOV R1, #0 ; R1 = 0
```

```
L1
 CMP R0, R1
 BEQ FINE ; se risultato > 0, ciclo
 SUB R0, R0, #1 ; risultato = risultato-1
 B L1
FINE
```

**Esercizio 7.24** Ripetere l'Esercizio 7.23 per il seguente programma

```
MOV R0, #0 ; i = 0
MOV R1, #0 ; somma = 0
MOV R2, #10 ; R2 = 10

CICLO
 CMP R2, R0 ; R2 == R0?
 BEQ L2
 ADD R1, R1, R0 ; somma = somma + i
 ADD R0, R0, #1 ; incrementa i
 B CICLO
L2
```

**Esercizio 7.25** Scrivere il codice HDL del processore ARM multi ciclo. Il processore deve essere compatibile con il seguente modulo di livello top. Il modulo mem è usato per contenere sia istruzioni sia dati. Collaudare il processore con il testbench del paragrafo 7.6.3.

```
module top(input logic clk, reset,
 output logic [31:0] WriteData, Addr,
 output logic MemWrite);
 logic [31:0] ReadData;

 // istanzia il processore e la memoria condivisa
 arm arm(clk, reset, MemWrite, Addr,
 WriteData, ReadData);
 mem mem(clk, MemWrite, Addr, WriteData, ReadData);
endmodule

module mem(input logic clk, we,
 input logic [31:0] a, wd,
 output logic [31:0] rd);
 logic [31:0] RAM[63:0];
 initial
 $readmemh("memfile.dat", RAM);
 assign rd = RAM[a[31:2]]; // allineato a word
 always_ff @(posedge clk)
 if (we) RAM[a[31:2]] <= wd;
endmodule
```

**Esercizio 7.26** Estendere il codice HDL del processore ARM multi ciclo dell'Esercizio 7.25 per gestire una delle nuove istruzioni dell'Esercizio 7.14. Estendere il testbench per collaudare anche la nuova istruzione.

**Esercizio 7.27** Ripetere l'Esercizio 7.26 per una delle nuove istruzioni dell'Esercizio 7.13.

**Esercizio 7.28** Il processore ARM pipeline esegue il seguente frammento di codice. Quali registri vengono scritti e quali vengono letti nel quinto ciclo? Si ricordi che il processore ARM pipeline ha un'unità di gestione delle dipendenze.

```
MOV R1, #42
```



```

SUB R0, R1, #5
LDR R3, [R0, #18]
STR R4, [R1, #63]
ORR R2, R0, R3

```

**Esercizio 7.29** Ripetere l'Esercizio 7.28 per il seguente frammento di codice ARM.

```

ADD R0, R4, R5
SUB R1, R6, R7
AND R2, R0, R1
ORR R3, R2, R5
LSL R4, R2, R3

```

**Esercizio 7.30** Usando un diagramma simile a quello della Figura 7.53, mostrare le operazioni di inoltramento (*forwarding*) e gli stalli necessari per eseguire le istruzioni seguenti con il processore ARM pipeline.

```

ADD R0, R4, R9
SUB R0, R0, R2
LDR R1, [R0, #60]
AND R2, R1, R0

```

**Esercizio 7.31** Ripetere l'Esercizio 7.30 per le istruzioni seguenti.

```

ADD R0, R11, R5
LDR R2, [R1, #45]
SUB R5, R0, R2
AND R5, R2, R5

```

**Esercizio 7.32** Quanti cicli sono necessari al processore ARM pipeline per eseguire tutte le istruzioni del programma dell'Esercizio 7.24? Quanto vale il CPI del processore per quel programma?

**Esercizio 7.33** Ripetere l'Esercizio 7.32 per il programma dell'Esercizio 7.23.

**Esercizio 7.34** Spiegare come si deve estendere il processore ARM pipeline per eseguire anche l'istruzione EOR.

**Esercizio 7.35** Spiegare come si deve estendere il processore ARM pipeline per eseguire anche l'istruzione CMN.

**Esercizio 7.36** Nel paragrafo 7.5.3 si è sottolineato come le prestazioni del processore pipeline sarebbero migliori se i salti fossero effettuati nella fase di decodifica invece che in quella di

esecuzione. Mostrare come modificare il processore pipeline della Figura 7.58 per fare i salti nella fase di decodifica. Come cambiano i segnali di stallo, di svuotamento e di *forwarding*? Ripetere gli Esempi 7.7 e 7.8 per calcolare il nuovo CPI, il nuovo tempo di ciclo e il nuovo tempo totale necessario per eseguire il programma.

**Esercizio 7.37** Sempre il solito seccellone sostiene di poter riprogettare una qualsiasi delle unità del processore ARM pipeline e renderla molto più veloce. Con riferimento ai ritardi riportati nella Tabella 7.5, quale unità va riprogettata per ottenere il massimo incremento di prestazioni globali del processore? Quanto deve essere veloce? (Farla più veloce del necessario è un inutile sforzo di progetto, da evitare.) Quale sarà il tempo di ciclo del processore così migliorato?

**Esercizio 7.38** Si considerino i ritardi riportati nella Tabella 7.5. Si supponga ora che l'ALU sia del 20% più veloce: il tempo di ciclo del processore ARM pipeline cambia? E se l'ALU fosse del 20% più lenta?

**Esercizio 7.39** Si supponga che il processore ARM pipeline sia diviso in 10 stadi da 400 ps ciascuno, incluso il sovraccarico per il sequenziamento, e si faccia riferimento alle istruzioni dell'Esempio 7.7. Si supponga anche che il 50% delle istruzioni di caricamento da memoria sia immediatamente seguito da un'istruzione che usa il loro risultato, necessitando quindi di sei stalli, e che si sbagli la predizione per il 30% dei salti. L'indirizzo di destinazione di un'istruzione di salto è calcolato alla fine del secondo stadio. Calcolare il CPI medio e il tempo di esecuzione di 100 miliardi di istruzioni del benchmark SPECINT2000 per questo processore a 10 stadi di pipeline.

**Esercizio 7.40** Scrivere il codice HDL del processore ARM pipeline. Il processore deve essere compatibile con il modulo di livello top dell'Esempio HDL 7.13, e deve supportare le sette istruzioni descritte in questo capitolo: ADD, SUB, AND, ORR (con modi di indirizzamento a registro e immediato ma senza traslazioni), LDR, STR (con spiazzamento immediato positivo) e B. Collaudare il codice con il testbench dell'Esempio HDL 7.12.

**Esercizio 7.41** Progettare l'unità di gestione delle dipendenze mostrata nella Figura 7.58 per il processore ARM pipeline. Usare un HDL per realizzare il progetto. Schematizzare quali parti hardware sarebbero generate da uno strumento di sintesi per il progetto in questione.

## Domande di valutazione

---

Queste domande sono state poste a candidati per un posto di lavoro nell'ambito della progettazione di sistemi digitali.

**Domanda 7.1** Ci spieghi quali sono i vantaggi dei microprocessori pipeline.

**Domanda 7.2** Saprebbe dirci come mai, se stadi di pipeline aggiuntivi consentono al processore di essere più veloce, non esistono processori con 100 stadi?

**Domanda 7.3** Ci descriva il concetto di dipendenza in un microprocessore e ci spieghi come può essere gestita. Quali sono i pro e i contro di ogni soluzione?

**Domanda 7.4** Ci descriva il concetto di processore superscalare discutendone pro e contro.

# Sistemi di memoria

## Capitolo

# 8

**8.1** Introduzione

**8.2** Analisi delle prestazioni  
del sistema di memoria

**8.3** Memoria cache

**8.4** Memoria virtuale

**8.5** Riassunto

## 8.1 ■ INTRODUZIONE

Le prestazioni dei calcolatori dipendono dal sistema di memoria tanto quanto dalla microarchitettura del processore. Nel Capitolo 7 si è ipotizzato un sistema di memoria ideale in grado di consentire ogni accesso in un singolo ciclo di clock, ma questa ipotesi è realistica solo per una memoria molto piccola o per un processore molto lento. I processori di un tempo erano relativamente lenti, e la memoria era in grado di reggere il passo, ma la velocità dei processori è cresciuta a un ritmo maggiore di quella della memoria, e le memorie dinamiche (DRAM) di oggi sono da 10 a 100 volte più lente dei processori, e questo crescente divario di prestazioni fra processore e memoria DRAM richiede sistemi di memoria sempre più sofisticati per cercare di approssimare una memoria veloce quanto il processore. Questo capitolo discute i sistemi di memoria e ne valuta le caratteristiche in termini di rapporti tra velocità, capacità e costo.

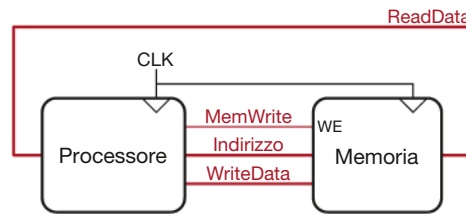
Il processore comunica con la memoria tramite un'**interfaccia a memoria**. La **Figura 8.1** mostra la semplice interfaccia a memoria utilizzata dal processore ARM multi ciclo. Il processore invia un indirizzo al sistema di memoria tramite il **bus indirizzi**. In caso di lettura, il segnale *MemWrite* (scrittura in memoria) vale 0 e la memoria restituisce il dato sul **bus di lettura dati**. In caso di scrittura, il segnale *MemWrite* vale 1 e il processore invia il dato alla memoria sul **bus di scrittura dati**.

Gli aspetti salienti nel progetto di un sistema di memoria possono essere compresi tramite la metafora dei libri in una biblioteca. Una biblioteca contiene moltissimi libri sui vari scaffali. Se si deve fare una ricerca sul significato dei sogni, ci si può recare in biblioteca,<sup>1</sup> prelevare dallo scaffale il libro

<sup>1</sup> In realtà l'uso della biblioteca da parte degli studenti è sempre meno frequente a causa di Internet. Ma gli autori sono convinti che le biblioteche contengano molti tesori di sapere umano conquistato a fatica non disponibili in forma elettronica, e sperano che la ricerca di informazioni su Web non riesca a eliminare completamente l'arte della ricerca bibliografica.

|                      |                                                                                       |
|----------------------|---------------------------------------------------------------------------------------|
| Software applicativo |  |
| Sistema operativo    |  |
| Architettura         |  |
| Micro-architettura   |  |
| Blocchi logici       |  |
| Reti digitali        |  |
| Reti analogiche      |  |
| Dispositivi          |  |
| Fisica               |  |

**Figura 8.1**  
Interfaccia a memoria.



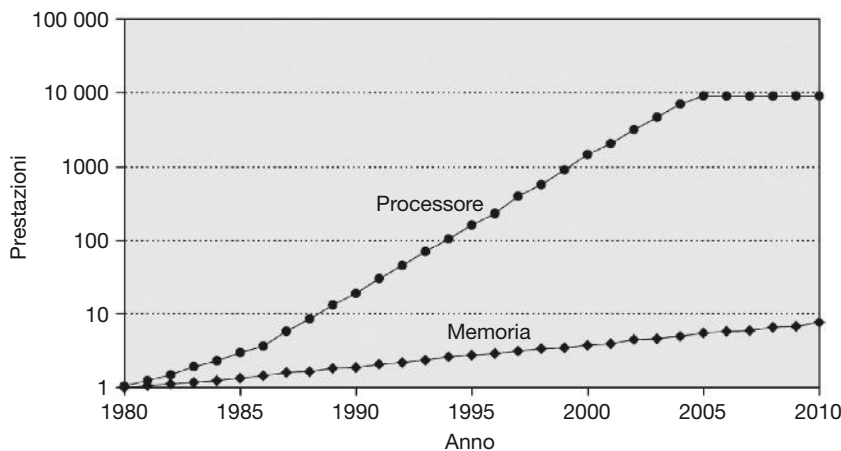
di Freud “L’interpretazione dei sogni” e portarlo nella propria postazione di lettura. Dopo averlo sfogliato per un po’, si può riportarlo sullo scaffale e prelevare il libro di Jung “Psicologia dell’inconscio”. Poi si può dover tornare per un’altra citazione a “L’interpretazione dei sogni”, e magari fare ancora un altro giro per prelevare il libro di Freud “L’Io e l’Es”. Molto presto ci si stanca di questi viaggi tra scaffali e postazione di lettura, e se si è furbi ci si accorge che si risparmia tempo tenendo nella postazione di lettura i libri prelevati dagli scaffali invece di fare la fatica di trasportarli avanti e indietro. Inoltre, quando si preleva un libro di Freud, si può prenderne anche altri dello stesso autore dal medesimo scaffale.

Questa metafora sottolinea il principio, introdotto nel paragrafo 6.2.1, di rendere veloci i casi più frequenti: tenendo nella postazione di lettura i libri usati più di recente o che è probabile dover usare a breve si riduce il tempo richiesto dai viaggi tra postazione di lettura e scaffali. In particolare, si sono usati in questa metafora i principi della **località temporale** e della **località spaziale**. Per località temporale si intende il fatto che se si è usato recentemente un libro è molto probabile doverlo usare ancora a breve. Per località spaziale si intende il fatto che se si usa un libro di un particolare scaffale è molto probabile avere bisogno a breve di altri libri dello stesso scaffale.

La biblioteca stessa rende veloce il caso comune usando questi principi di località. Nessuna biblioteca ha lo spazio né i soldi per ospitare tutti i libri del mondo: quindi conserva i libri meno usati in un qualche magazzino in cantina, e stipula accordi di prestito con altre biblioteche per offrire ai propri frequentatori più libri di quelli effettivamente di proprietà.

In conclusione, si ottengono i benefici di un’ampia collezione di libri e di un rapido accesso a quelli più frequentemente richiesti tramite una gerarchia di immagazzinamento: i libri usati dal lettore sono nella sua postazione di lettura; un’ampia collezione di libri di frequente utilizzo si trova sugli scaffali; una collezione molto più ampia è disponibile previa prenotazione nei magazzini della biblioteca o a prestito da altre biblioteche. Analogamente, i sistemi di memoria usano una gerarchia di memoria per accedere rapidamente ai dati più usati pur offrendo la capacità di immagazzinare grosse quantità di dati.

I sottosistemi di memoria usati per costruire questa gerarchia sono stati introdotti nel paragrafo 5.5: le memorie dei calcolatori sono costituite principalmente da RAM dinamiche (DRAM) e RAM statiche (SRAM). Idealmente, il sistema di memoria deve essere veloce, grande ed economico. In pratica, ogni tipo di memoria ha solo due di questi requisiti, e risulta lenta, oppure piccola, oppure costosa, ma si può approssimare la memoria ideale combinando una memoria veloce, piccola ed economica con una memoria lenta, grande ed economica. La memoria veloce memorizza le istruzioni e i dati usati più di frequente, quindi in media il sistema di memoria risulta veloce. La memoria grande memorizza il resto delle istruzioni e dei dati, quindi la capacità totale del sistema di memoria risulta elevata. La combinazione delle due memorie economiche è molto meno costosa di un’unica memoria grande e veloce. Si possono estendere questi principi costruendo una gerarchia di memorie di capacità crescente e velocità decrescente.



**Figura 8.2**  
**Prestazioni divergenti di processore e memoria.** (Con permesso di adattamento da Hennessy e Patterson, *Computer Architecture: A Quantitative Approach*, 5a ed., Morgan Kaufmann, 2011.)

La memoria dei calcolatori è generalmente costituita da chip DRAM. Nel 2015, un tipico PC aveva una **memoria principale** costituita da 8 a 16 GB di DRAM, con un costo di circa 7 dollari per GB. I costi della DRAM sono diminuiti del 25% circa all'anno nell'ultimo decennio, ma la capacità di memoria ha avuto un tasso di crescita praticamente uguale, quindi il costo della memoria del PC è rimasto pressoché costante. Sfortunatamente la velocità della DRAM è cresciuta solo del 7% ogni anno, mentre quella del processore è cresciuta dal 25% al 50% all'anno, come mostra la **Figura 8.2** che traccia le velocità di DRAM e processore usando come riferimento i valori del 1980. Nel 1980 le velocità erano praticamente uguali, ma da quel momento in poi si sono differenziate, con la velocità della memoria largamente inferiore.<sup>2</sup>

La DRAM poteva reggere il ritmo del processore negli anni '70 e nei primi anni '80, ma è ora ampiamente troppo lenta: il tempo di accesso della DRAM è di uno o due ordini di grandezza più lungo del tempo di ciclo del processore (decine di nanosecondi rispetto a meno di un nanosecondo).

Per contrastare questa tendenza, i calcolatori memorizzano istruzioni e dati usati più di frequente in una memoria più veloce ma più piccola, denominata **cache** e costituita generalmente da SRAM situata a bordo dello stesso chip del processore. La velocità della cache è comparabile a quella del processore, perché la SRAM è intrinsecamente più veloce della DRAM e perché la posizione a bordo del processore elimina i ritardi dovuti alla propagazione dei segnali elettrici tra chip diversi. Nel 2015, i costi della SRAM a bordo erano dell'ordine di 5000 dollari al GB, ma la cache è relativamente piccola (da qualche kilobyte a qualche megabyte) quindi il costo totale risulta modesto. Le cache possono memorizzare sia istruzioni sia dati, anche se spesso si parla genericamente di "dati".

Se il processore richiede un dato che è presente nella cache, tale dato viene reso disponibile rapidamente: questo evento di "dato trovato" viene denominato in inglese **hit** ("colpito"). In caso contrario, il processore recupera il dato dalla memoria principale (DRAM); l'evento di "dato non trovato" viene denominato in inglese **miss** ("mancato"). Se la cache dà luogo a hit nella maggior parte dei casi, il processore deve attendere solo raramente le risposte della lenta memoria principale, e il tempo medio di accesso risulta breve.

Il terzo livello della gerarchia di memoria è costituito dal disco rigido. Allo stesso modo in cui la biblioteca usa il magazzino in cantina per conservare i

<sup>2</sup> Anche se recentemente le prestazioni del singolo processore sono rimaste praticamente costanti, come mostrato nella Figura 8.2 per gli anni 2005-2010, la diffusione di sistemi multi-core (non rappresentata nel grafico) non ha fatto altro che peggiorare ulteriormente il divario di prestazioni tra processore e memoria.

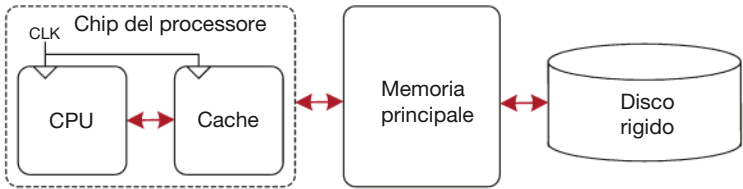
libri che non ci stanno sugli scaffali, i calcolatori usano il disco rigido per contenere i dati che non trovano spazio nella memoria principale. Nel 2015, un disco rigido (HDD, *Hard Disk Drive*) di tipo magnetico costava meno di 0.05 dollari per GB, con un tempo di accesso di circa 5 ms. I costi dei dischi rigidi magnetici sono diminuiti del 60% ogni anno, ma i tempi di accesso sono migliorati solo di poco. I dischi a stato solido (SSD, *Solid State Drives*) realizzati con la tecnologia delle memorie *flash* sono un’alternativa sempre più diffusa rispetto agli HDD. Gli SSD sono stati usati in mercati di nicchia per più di due decenni, e solo nel 2007 sono entrati nel mercato principale dei calcolatori. Gli SSD superano alcuni dei guasti meccanici degli HDD, ma costano circa dieci volte di più: 0,40 dollari per GB.

Il disco rigido dà l’illusione di una capacità di memoria più ampia delle dimensioni della memoria principale: si parla quindi di **memoria virtuale**. Come per i libri in cantina, un dato in memoria virtuale richiede un tempo lungo per essere utilizzato. La memoria principale, detta anche memoria fisica, contiene un sottoinsieme della memoria virtuale, quindi può essere considerata una specie di cache dei dati del disco rigido usati più di frequente.

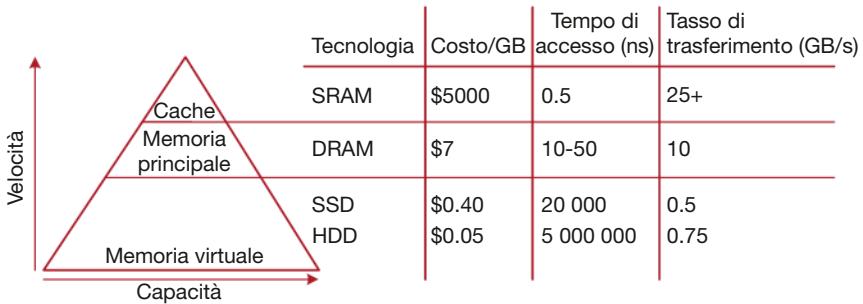
La **Figura 8.3** riassume il concetto di gerarchia di memoria del calcolatore oggetto di questo capitolo. Il processore per prima cosa cerca il dato in una memoria piccola ma veloce generalmente presente a bordo del suo stesso chip. Se non trova il dato, il processore guarda nella memoria principale. Se non lo trova neanche lì, lo preleva dalla memoria virtuale sul disco rigido, capiente ma lento. La **Figura 8.4** illustra questo rapporto capacità/velocità nella gerarchia di memoria ed elenca i valori tipici di costo, tempo di accesso e tasso di trasferimento relativamente alle tecnologie del 2015. Come ovvio, al diminuire del tempo di accesso aumenta la velocità.

Il paragrafo 8.2 introduce l’analisi delle prestazioni dei sistemi di memoria. Il paragrafo 8.3 presenta varie possibili organizzazioni per la memoria cache, mentre il paragrafo 8.4 si occupa dei sistemi di memoria virtuale.

**Figura 8.3**  
Tipica gerarchia di memoria.



**Figura 8.4**  
Componenti della gerarchia di memoria, con le caratteristiche riferite al 2015.



**8.2 ■ ANALISI DELLE PRESTAZIONI DEL SISTEMA DI MEMORIA**

Progettisti e acquirenti di calcolatori hanno bisogno di metodi quantitativi per misurare le prestazioni dei sistemi di memoria e poter quindi valutare il rapporto costo/prestazioni delle varie alternative. Le metriche per i sistemi

di memoria sono i **tassi** (o percentuali) di **hit/miss** (*hit rate* e *miss rate*) e il tempo medio di accesso a memoria. I tassi di miss e hit sono calcolati come:

$$\begin{aligned}\text{Tasso di miss} &= \frac{\text{Numero di miss}}{\text{Numero totale di accessi a memoria}} = 1 - \text{tasso di hit} \\ \text{Tasso di hit} &= \frac{\text{Numero di hit}}{\text{Numero totale di accessi a memoria}} = 1 - \text{tasso di miss}\end{aligned}\quad (8.1)$$

### ESEMPIO 8.1

**Calcolo delle prestazioni della cache.** Si supponga che un programma esegua 2000 istruzioni di accesso a memoria (letture o scritture) e che 1250 di tali istruzioni trovino il dato richiesto in cache. Le altre 750 istruzioni trovano il dato in memoria principale o su disco. Quanto valgono il tasso di miss e il tasso di hit per la cache?

**Soluzione** Il tasso di miss vale  $750/2000 = 0.375 = 37.5\%$ . Il tasso di hit vale  $1250/2000 = 0.625 = 1 - 0.375 = 62.5\%$ .

**Il tempo medio di accesso a memoria** (*AMAT*, *Average Memory Access Time*) è il tempo medio di attesa da parte del processore per completare un'istruzione di lettura da o scrittura in memoria. Nel tipico calcolatore della Figura 8.3, il processore guarda prima nella cache. Se si verifica un miss, guarda nella memoria principale. Se si verifica un secondo miss, il processore accede alla memoria virtuale su disco rigido. *AMAT* è dunque calcolato come:

$$AMAT = t_{\text{cache}} + TM_{\text{cache}}(t_{MP} + TM_{MP}t_{MV}) \quad (8.2)$$

dove  $t_{\text{cache}}$ ,  $t_{MP}$  e  $t_{MV}$  sono i tempi di accesso rispettivamente della cache, della Memoria Principale e della Memoria Virtuale, e  $TM_{\text{cache}}$  e  $TM_{MP}$  sono i Tassi di Miss rispettivamente della cache e della Memoria Principale.

### ESEMPIO 8.2

**Calcolo del tempo medio di accesso a memoria.** Si supponga che un calcolatore abbia una gerarchia di memoria a due soli livelli: cache e memoria principale. Qual è il tempo medio di accesso se i tempi di accesso alle due memorie e i relativi tassi di miss sono quelli riportati nella **Tabella 8.1**?

**Soluzione** Il tempo medio di accesso a memoria è  $1 + 0.1(100) = 11$  cicli.

**Tabella 8.1** Tempi di accesso e tassi di miss.

| Livello di memoria | Tempo di accesso (cicli) | Tasso di miss |
|--------------------|--------------------------|---------------|
| Cache              | 1                        | 10%           |
| Memoria principale | 100                      | 0%            |

### ESEMPIO 8.3

**Miglioramento del tempo di accesso.** Un tempo medio di accesso a memoria di 11 cicli significa che il processore passa dieci cicli di clock in attesa del dato per ogni ciclo di effettivo utilizzo di tale dato. Che tasso di miss della cache è necessario per ridurre il tempo medio di accesso a 1.5 cicli, usando i tempi riportati nella Tabella 8.1?

**Soluzione** Detto  $m$  il tasso di miss, il tempo medio di accesso è  $1 + 100m$  cicli. Imponendo che tale tempo sia 1.5 cicli e risolvendo l'equazione in  $m$  si ottiene un tasso di miss dello 0.5%.





**Gene Amdahl, 1922-2015.** È famoso principalmente per la Legge di Amdahl, una considerazione da lui fatta nel 1965. Ancora alle scuole superiori, aveva iniziato a progettare calcolatori nel tempo libero, attività che gli ha fruttato il titolo di Dottore di Ricerca in fisica teorica nel 1952. È entrato in IBM subito dopo la laurea, e successivamente ha fondato tre aziende, inclusa la Amdahl Corporation nel 1970.

Cache (dal francese *caché*, nascosto): un nascondiglio per occultare e conservare provviste o attrezzi. Merriam Webster Online Dictionary, 2015. [www.merriam-webster.com](http://www.merriam-webster.com)

Bisogna però fare attenzione che i miglioramenti di prestazioni non sempre sono quelli che sembrano. Per esempio, rendere il sistema di memoria di un calcolatore dieci volte più veloce non significa necessariamente che un programma viene eseguito dieci volte più rapidamente. Se il 50% delle prestazioni del programma è dovuto a letture e scritture in memoria, un miglioramento di un fattore 10 del sistema di memoria porta a un miglioramento solo di un fattore 1.82 nelle prestazioni del programma. Questo principio generale è noto come **Legge di Amdahl**, che evidenzia come lo sforzo per migliorare le prestazioni di un sottosistema è utile solo se tale sottosistema influisce su una larga percentuale delle prestazioni globali.

### 8.3 ■ MEMORIA CACHE

La memoria cache contiene i dati usati più di frequente. Il numero di parole che la cache può contenere è definito **capacità  $C$**  della cache. Dal momento che la capacità della cache è largamente inferiore a quella della memoria principale, il progettista di calcolatori deve decidere quale sottoinsieme della memoria principale mantenere in cache.

Quando il processore deve accedere a un dato, per prima cosa verifica se tale dato è presente in cache. In caso di hit, il dato è immediatamente disponibile. In caso di miss, il processore preleva il dato da memoria principale e lo copia in cache per futuro utilizzo. Naturalmente, per fare spazio al nuovo dato, la cache deve **sostituire** (*replace*) un dato vecchio. In questo paragrafo si discutono gli aspetti principali del progetto della memoria cache rispondendo alle domande seguenti: (1) Quali dati devono essere memorizzati nella cache? (2) Come si verifica se un dato è in cache? (3) Quale dato viene sostituito dal nuovo dato quando la cache è piena?

Proseguendo nella lettura, si tenga presente che il motivo trainante nel rispondere a queste domande è la località spaziale e temporale intrinseca degli accessi a memoria nella maggior parte delle applicazioni. Le cache usano la località spaziale e quella temporale per prevedere quali dati saranno necessari nel prossimo futuro, quindi se un programma accede a dati in ordine casuale non trae alcun beneficio dalla presenza della cache.

Come spiegato nei paragrafi seguenti, le cache sono dimensionate in termini di capacità ( $C$ ), numero di set ( $S$ ), dimensione del blocco ( $b$ ), numero di blocchi ( $B$ ) e grado di associatività ( $N$ ).

Si fa riferimento a letture di dati dalle cache, ma gli stessi principi si applicano alle fasi di fetch dalle cache che contengono istruzioni. Le operazioni di scrittura in cache sono simili, e vengono ulteriormente discusse nel paragrafo 8.3.4.

#### 8.3.1 Quali dati devono essere memorizzati nella cache?

Una cache ideale dovrebbe prevedere in anticipo tutti i dati necessari al processore e prelevarli dalla memoria principale con anticipo sufficiente ad avere un tasso di miss pari a zero. Dal momento che è ovviamente impossibile predire il futuro con completa accuratezza, la cache deve indovinare quali dati saranno necessari in base alle sequenze di accessi a memoria verificatesi nel passato. In particolare, la cache sfrutta la località temporale e spaziale per ottenere un basso tasso di miss.

Come detto, località temporale significa che il processore ha un'elevata probabilità di accedere nuovamente nel prossimo futuro a un dato se lo ha utilizzato da poco. Quindi, quando il processore legge o scrive un dato non presente in cache, tale dato viene copiato in cache in modo che successivi accessi allo stesso dato diano luogo a hit.

Come già detto, località spaziale significa che, quando il processore accede a un certo dato, ha un'elevata probabilità di accedere nel prossimo futu-

ro ad altri dati in locazioni di memoria vicine al dato in questione. Quindi quando la cache preleva una parola da memoria principale, preleva anche alcune altre parole adiacenti: questo gruppo di parole è denominato **blocco di cache** o linea di cache. Il numero  $b$  di parole in un blocco di cache è definito **dimensione di blocco**. Una cache di capacità  $C$  contiene quindi  $B = C/b$  blocchi.

I principi di località temporale e spaziale sono stati verificati sperimentalmente nei programmi reali. Se in un programma si usa una variabile, è molto probabile che tale variabile venga usata nuovamente a breve, dando luogo a località temporale. Se si accede a un elemento di un array, altri elementi dello stesso array saranno molto probabilmente usati a breve, dando luogo a località spaziale.

### 8.3.2 Come si verifica se un dato è in cache?

Ogni cache è organizzata in  $S$  insiemi o **set**, ciascuno dei quali contiene uno o più blocchi di dati. La relazione tra l'indirizzo di un dato in memoria principale e la locazione di tale dato in cache è definita **mappatura** (*mapping*). Ogni indirizzo di memoria viene mappato in un set della cache; alcuni dei bit dell'indirizzo sono utilizzati per determinare in quale set della cache è contenuto il dato. Se il set contiene più di un blocco, il dato può essere memorizzato in uno qualsiasi dei blocchi del set.

Le cache sono categorizzate in base al numero di blocchi presenti in un set. In una **cache a mappatura diretta** (*direct mapped*) ogni set contiene un solo blocco, quindi la cache ha  $S = B$  set, e dunque un qualsiasi indirizzo di memoria principale è mappato in un solo blocco della cache. In una **cache parzialmente associativa** (*set associative*) a  $N$  vie ogni set contiene  $N$  blocchi. L'indirizzo di memoria principale è mappato in un solo set, con  $S = B/N$  set, ma il dato corrispondente a tale indirizzo può finire in uno qualsiasi degli  $N$  blocchi di quel set. Una **cache completamente associativa** (*fully associative*) ha solo  $S = 1$  set. Un dato può andare in uno qualsiasi dei  $B$  blocchi del set, quindi una cache completamente associativa può essere definita come una cache parzialmente associativa a  $B$  vie.

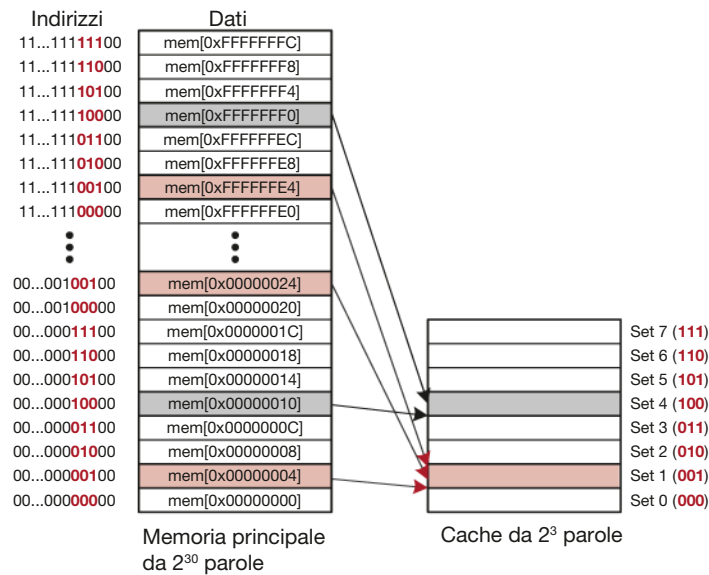
Per illustrare queste diverse organizzazioni di cache si fa riferimento a un sistema di memoria per il processore ARM, con indirizzi a 32 bit e parole di memoria da 32 bit. La memoria è indirizzabile a byte, e ogni parola è costituita da quattro byte, quindi la memoria è costituita da  $2^{30}$  parole allineate rispetto ai bordi di parola. Per semplicità, si considerano cache con una capacità  $C$  di otto parole, partendo da blocchi con dimensione di blocco  $b$  di una sola parola per generalizzare più avanti a blocchi di dimensioni maggiori.

#### Cache a mappatura diretta

Una cache a mappatura diretta ha un solo blocco in ogni set, quindi è organizzata in  $S = B$  set. Per comprendere la mappatura degli indirizzi di memoria nei blocchi di cache si deve immaginare la memoria principale suddivisa in blocchi di  $b$  parole ciascuno, esattamente come la cache. Un indirizzo nel blocco 0 della memoria principale viene mappato nel set 0 della cache, un indirizzo nel blocco 1 della memoria principale viene mappato nel set 1 della cache, e così via fino a un indirizzo nel blocco  $B - 1$  della memoria principale che viene mappato nel set  $B - 1$  della cache. Non ci sono altri blocchi nella cache, quindi la mappatura si ripete circolarmente con il blocco  $B$  della memoria principale che viene mappato nel set 0 della cache, e così via.

Questa mappatura è illustrata nella **Figura 8.5**, per una cache a mappatura diretta con capacità di otto parole e dimensione di blocco di una parola. La cache ha otto set, ciascuno contenente un blocco da una parola. I due bit meno significativi dell'indirizzo sono sempre 00, perché le parole

**Figura 8.5**  
Mappatura della memoria  
principale in una cache a mappatura  
diretta.



sono allineate ai bordi di parola. I successivi  $\log_2 8 = 3$  bit indicano il set nel quale l'indirizzo di memoria viene mappato. Quindi i dati presenti agli indirizzi 0x00000004, 0x00000024, ..., 0xFFFFFEE4 sono tutti mappati nel set 1, come evidenziato in rosso nella figura. Analogamente, i dati presenti agli indirizzi 0x00000010, ..., 0xFFFFFFF0 sono tutti mappati nel set 4, e così via. Ogni indirizzo di memoria principale viene mappato in un solo set della cache.

**ESEMPIO 8.4**

**Campi di cache.** In quale set della cache della Figura 8.5 viene mappata la parola di indirizzo 0x00000014? Indicare un altro indirizzo che viene mappato nel medesimo set.

**Soluzione** I due bit meno significativi dell'indirizzo sono sempre 00, perché l'indirizzo è allineato ai bordi di parola. I successivi tre bit sono 101, quindi la parola viene mappata nel set 5. Le parole agli indirizzi 0x34, 0x54, 0x74, ..., 0xFFFFF04 sono tutte mappate nel medesimo set.

Dal momento che molti indirizzi sono mappati nel medesimo set, la cache deve tenere traccia dell'indirizzo del dato effettivamente presente in ogni set. I bit meno significativi dell'indirizzo indicano in quale set è mappato il dato, i restanti bit più significativi sono denominati **tag** (etichetta) e indicano quale dei tanti possibili indirizzi è effettivamente presente in quel particolare set.

Negli esempi precedenti, i due bit meno significativi sono denominati **spiazzamento di byte** perché indicano un byte all'interno della parola. I successivi tre bit sono denominati **bit di set** perché indicano in quale set viene mappato l'indirizzo (in generale, il numero di bit di set è  $\log_2 S$ ). I rimanenti 27 **bit di tag** indicano l'indirizzo del dato effettivamente contenuto in un certo set della cache. La **Figura 8.6** mostra i campi di cache per l'indirizzo 0xFFFFFEE4: tale indirizzo viene mappato nel set 1 e il suo tag è costituito da tutti 1.

**Figura 8.6**  
Campi di cache dell'indirizzo  
0xFFFFFEE4 mappato nella cache  
della Figura 8.5.



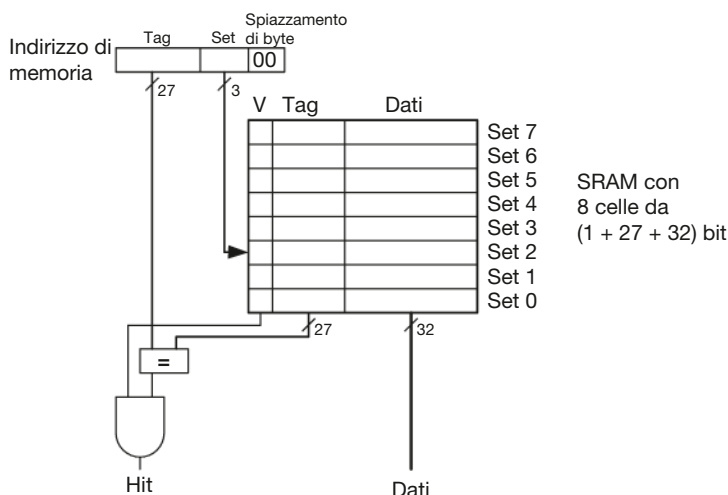
**ESEMPIO 8.5**

**Campi di cache.** Trovare il numero di bit di set e di tag per una cache a mappatura diretta con 1024 ( $2^{10}$ ) set e dimensione di blocco di una parola. L'indirizzo è a 32 bit.

**Soluzione** Una cache con  $2^{10}$  set richiede  $\log_2(2^{10}) = 10$  bit di set. I due bit meno significativi dell'indirizzo sono lo spiazzamento di byte, e i rimanenti  $32 - 10 - 2 = 20$  bit formano il tag.

Occasionalmente, per esempio all'accensione del calcolatore, i set della cache non contengono dati. La cache usa un **bit di validità** per ogni set che indica se il set contiene dati significativi. Se il bit di validità è 0, il contenuto del set non è significativo.

La **Figura 8.7** mostra la realizzazione circuitale della memoria a mappatura diretta della Figura 8.5. La cache è realizzata con una SRAM a otto elementi. Ogni elemento, cioè ogni set, è una riga contenente 32 bit di dato, 27 bit di tag e 1 bit di validità. Si accede alla cache con un indirizzo da 32 bit. I due bit meno significativi sono ignorati negli accessi a parole. I successivi tre bit, ovvero i bit di set, specificano l'elemento (il set) nella cache. Un'istruzione di lettura legge l'elemento così specificato della cache e controlla i bit di tag e il bit di validità. Se il tag corrisponde ai 27 bit più significativi dell'indirizzo e il bit di validità è 1, si è verificato un hit, e il dato della cache viene restituito al processore. In caso contrario, si è verificato un miss e il sistema di memoria deve prelevare il dato dalla memoria principale.



**Figura 8.7**  
Cache a mappatura diretta  
con 8 gruppi.

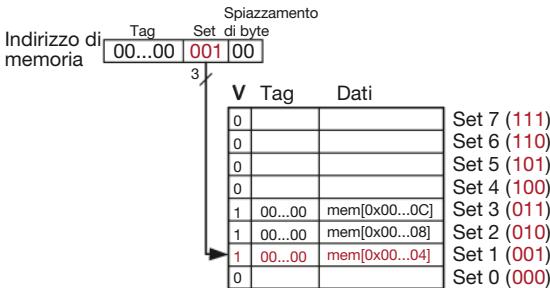
**ESEMPIO 8.6**

**Località temporale in una cache a mappatura diretta.** I cicli sono una causa comune di località temporale e spaziale nelle applicazioni. Facendo riferimento alla cache a otto elementi della **Figura 8.7**, mostrare il contenuto della cache dopo l'esecuzione del seguente ciclo scritto in codice assembly ARM, nell'ipotesi che la cache sia inizialmente vuota. Quanto vale il tasso di miss?

```

MOV R0, #5
MOV R1, #0
CICLO CMP R0, #0
 BEQ FINE
 LDR R2, [R1, #4]
 LDR R3, [R1, #12]
 LDR R4, [R1, #8]
 SUB R0, R0, #1
 B CICLO
FINE
```

**Figura 8.8**  
Contenuto della cache a mappatura diretta.



Quando due indirizzi generati di recente dal processore si mappano nel medesimo blocco di cache, si verifica un **conflitto**, e il dato cui si accede per ultimo **espelle** il precedente dal blocco. Le cache a mappatura diretta hanno un solo blocco in ogni set, quindi due indirizzi che si mappano nel medesimo set causano sempre un conflitto. L'Esempio 8.7 illustra i conflitti.

**ESEMPIO 8.7**

**Conflitti sui blocchi di cache.** Qual è il tasso di miss quando si esegue il seguente ciclo sulla cache a mappatura diretta della Figura 8.7, nell'ipotesi che la cache sia inizialmente vuota?

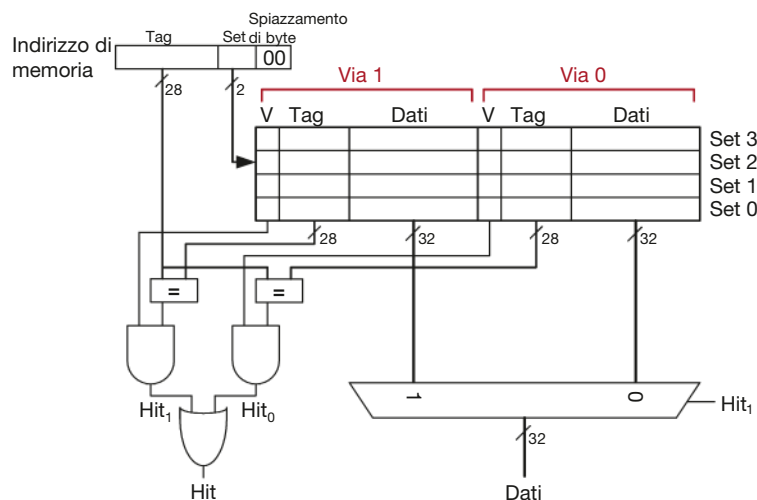
```
CICLO MOV R0, #5
 MOV R1, #0
 CMP R0, #0
 BEQ FINE
 LDR R2, [R1, #0x4]
 LDR R3, [R1, #0x24]
 SUB R0, R0, #1
 B CICLO
FINE
```

**Soluzione** Entrambi gli indirizzi 0x4 e 0x24 si mappano nel medesimo set 1. Alla prima iterazione del ciclo, il dato all'indirizzo 0x4 viene caricato nel set 1 della cache, poi il dato all'indirizzo 0x24 viene a sua volta caricato nel set 1, espellendo il dato di indirizzo 0x4. Alla seconda iterazione, la sequenza si ripete, e la cache deve ripristinare il dato di indirizzo 0x4 espellendo quello di indirizzo 0x24, e così via. I due indirizzi entrano continuamente in conflitto, e il tasso di miss è pari al 100%.

**Cache parzialmente associativa a molte vie**

Una *cache parzialmente associativa a N vie* riduce i conflitti prevedendo N blocchi in ciascun set dove il dato che viene mappato in tale set può trovarsi: ogni indirizzo di memoria viene dunque mappato in uno specifico set, ma può essere copiato in uno qualsiasi degli N blocchi di tale set. Si può dunque dire che la cache a mappatura diretta equivale a una cache parzialmente associativa a una via. N viene definito come il **grado di associatività** della cache.

La **Figura 8.9** mostra la struttura circuitale di una cache parzialmente associativa a N = 2 vie di capacità C = 8 parole. La cache ha in questo caso solo



**Figura 8.9**  
Cache parzialmente associativa  
a due vie.

4 set invece di 8, quindi solo  $\log_2 4 = 2$  bit di set invece di 3 vengono usati per selezionare il set, mentre il tag aumenta da 27 a 28 bit. Ogni set contiene due vie ovvero gradi di associatività: ogni via è costituita da un blocco di dati e dai bit di tag e di validità. La cache legge i blocchi da entrambe le vie del set selezionato e controlla i tag e i bit di validità: se si verifica un hit in una delle due vie, un multiplexer seleziona il dato da tale via.

Le cache parzialmente associative hanno solitamente dei tassi di miss inferiori alle cache a mappatura diretta di uguale capacità, perché presentano meno conflitti. Sono tuttavia generalmente più lente e più costose da realizzare a causa del multiplexer di uscita e dei comparatori aggiuntivi. Sorge inoltre il problema di quale via espellere quando entrambe sono piene, argomento ripreso nel paragrafo 8.3.3. La maggior parte dei sistemi commerciali usa cache parzialmente associative.

### ESEMPIO 8.8

**Tasso di miss di una cache parzialmente associativa.** Ripetere l'Esempio 8.7 usando la cache parzialmente associativa a due vie di otto parole della Figura 8.9.

**Soluzione** Entrambi gli accessi a memoria agli indirizzi 0x4 e 0x24 si mappano nel medesimo set 1, ma la cache ha due vie, quindi può accogliere i dati di entrambi gli indirizzi. Durante la prima iterazione del ciclo, la cache vuota dà luogo a due miss, e carica i due dati nelle due vie del set 1, come mostrato nella **Figura 8.10**. Nelle successive quattro iterazioni si verificano sempre e solo hit. Il tasso di miss risulta quindi pari a  $2/10 = 20\%$ , da confrontare con il tasso di miss del 100% della cache a mappatura diretta di uguale capacità dell'Esempio 8.7.

| Via 1 |         |                | Via 0 |         |                |       |
|-------|---------|----------------|-------|---------|----------------|-------|
| V     | Tag     | Dati           | V     | Tag     | Dati           |       |
| 0     |         |                | 0     |         |                | Set 3 |
| 0     |         |                | 0     |         |                | Set 2 |
| 1     | 00...00 | mem[0x00...24] | 1     | 00...10 | mem[0x00...04] | Set 1 |
| 0     |         |                | 0     |         |                | Set 0 |

**Figura 8.10**  
Contenuto della cache parzialmente  
associativa a due vie.

### Cache completamente associativa

Una *cache completamente associativa* è costituita da un unico set a  $B$  vie, dove  $B$  è il numero di blocchi, quindi un indirizzo di memoria può essere mappato in una qualsiasi di queste vie. La cache completamente associativa è dunque una cache parzialmente associativa a  $B$  vie.

La **Figura 8.11** mostra la SRAM di una cache completamente associativa con otto blocchi. Alla richiesta di un dato, otto confronti di tag (non mostrati)



**Figura 8.11** Cache completamente associativa a otto blocchi.

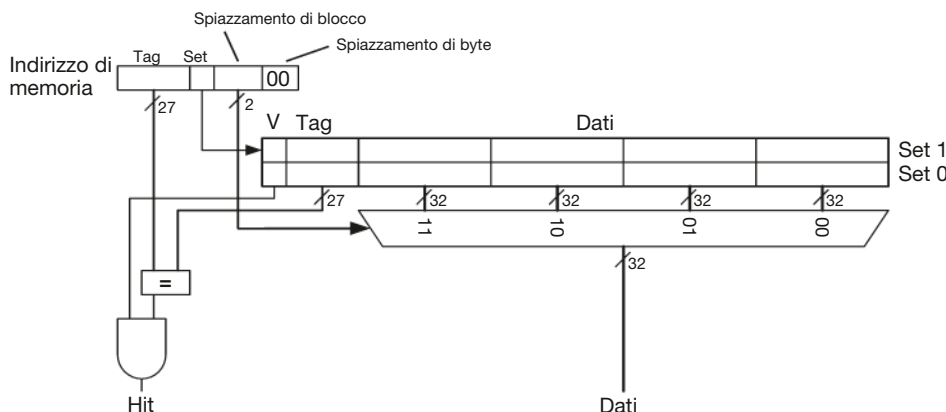
devono essere fatti, dal momento che il dato potrebbe trovarsi in un blocco qualsiasi. Un multiplexer 8:1 seleziona il dato corretto in caso di hit. Le cache completamente associative tendono ad avere il minor numero di miss causati da conflitti per una data capacità di cache, ma richiedono hardware aggiuntivo per i confronti dei tag. Sono quindi adatte per cache relativamente piccole a causa dell'elevato numero di comparatori.

### Dimensione del blocco

Negli esempi fatti finora si è sfruttata solo la località temporale, perché la dimensione del blocco era di una sola parola. Per sfruttare anche la località spaziale, una cache utilizza blocchi più grandi per memorizzare più parole di memoria consecutive.

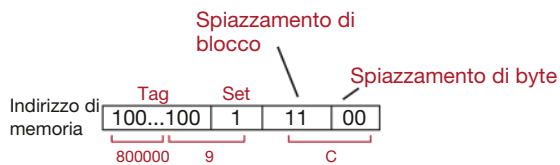
Il vantaggio di una dimensione di blocco maggiore di uno è il fatto che in caso di miss vengono copiate in cache la parola non trovata e anche le parole adiacenti nel blocco di memoria principale. Gli accessi successivi hanno dunque maggiore probabilità di dare luogo a hit grazie alla località spaziale. Tuttavia, una dimensione di blocco grande significa, a parità di capacità della cache, che questa ha meno blocchi, quindi può dare luogo a un maggior numero di conflitti aumentando il tasso di miss. Inoltre, serve più tempo in caso di miss per prelevare da memoria principale tutte le parole del blocco e trasferirle in cache: il tempo necessario per caricare in cache il blocco mancante viene detto **penalizzazione di miss**. Se le parole adiacenti del blocco non vengono utilizzate nel prossimo futuro, lo sforzo per portarle in cache è sprecato, ma la maggior parte dei programmi trae vantaggio dalla presenza di dimensioni di blocco più grandi.

La **Figura 8.12** mostra la struttura circuitale di una cache a mappatura diretta con una dimensione di blocco  $b = 4$  parole. La cache ha in questo caso solo  $B = C/b = 2$  blocchi. Una cache a mappatura diretta ha solo un blocco in ciascun set, quindi questa cache è organizzata in due set: serve dunque solo  $\log_2 2 = 1$  bit per selezionare il set. Serve poi un multiplexer per selezionare la parola all'interno del blocco: tale multiplexer è controllato da  $\log_2 4 = 2$  bit di **spiazzamento di blocco** nell'indirizzo. I 27 bit più significativi dell'indirizzo costituiscono il tag; serve un solo tag per l'intero blocco, perché le parole del blocco si trovano a indirizzi consecutivi.



**Figura 8.12** Cache a mappatura diretta con due blocchi e una dimensione di blocco di quattro parole.





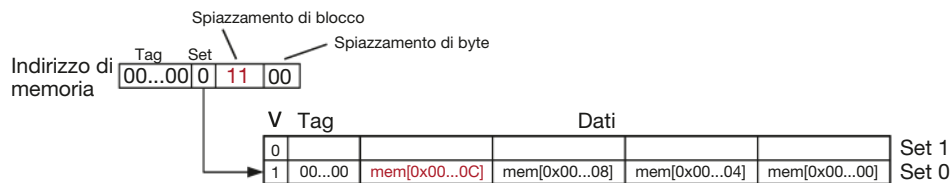
**Figura 8.13**  
Campi dell'indirizzo 0x8000009C  
mappato nella cache della Figura  
8.12.

La **Figura 8.13** mostra i campi di cache per l'indirizzo 0x8000009C quando viene mappato nella cache a mappatura diretta della Figura 8.12. I bit di spiazzamento di byte sono sempre 0 per accessi a intere parole. I successivi  $\log_2 b = 2$  bit di spiazzamento di blocco indicano la parola all'interno del blocco, e il bit seguente indica il set. I restanti 27 bit sono il tag. Dunque la parola 0x8000009C viene mappata nel set 1 parola 3 della cache. Naturalmente il principio di usare dimensioni di blocco maggiori di uno per sfruttare la località spaziale si applica anche alle cache associative.

### ESEMPIO 8.9

**Località spaziale con una cache a mappatura diretta.** Ripetere l'Esempio 8.6 per una cache a mappatura diretta di otto parole con dimensione di blocco di quattro parole.

**Soluzione** La **Figura 8.14** mostra il contenuto della cache dopo il primo accesso a memoria. Alla prima iterazione del ciclo si verifica un miss nell'accesso all'indirizzo 0x4, e vengono caricati in cache tutti i dati dall'indirizzo 0x0 all'indirizzo 0xC. Tutti gli accessi successivi (come mostrato per l'indirizzo 0xC) danno luogo a hit, quindi il tasso di miss risulta essere  $1/15 = 6.67\%$ .



**Figura 8.14** Contenuto della cache con una dimensione del blocco  $b$  pari a quattro parole.

### Riassumendo

Le cache sono organizzate come matrici bidimensionali. Le righe sono denominate set, e le colonne vie. Ogni elemento della matrice è costituito da un blocco di dati con i relativi bit di validità e di tag. Le cache sono caratterizzate da:

- capacità  $C$ ;
- dimensione di blocco  $b$  (e numero di blocchi  $B = C/b$ );
- numero  $N$  di blocchi in ogni set.

La **Tabella 8.2** riassume le possibili organizzazioni di cache. Ogni indirizzo di memoria viene mappato in un solo set, ma può essere memorizzato in una qualsiasi delle vie del set.

La capacità della cache, la sua associatività e le dimensioni di set e di blocco sono tipicamente potenze di 2, quindi i campi di cache (bit di tag, di set e di spiazzamento di blocco) sono sottoinsiemi dei bit di indirizzo.

Incrementare l'associatività  $N$  riduce generalmente il tasso di miss causato dai conflitti, ma implica l'uso di più comparatori di tag. Incrementare la dimensione di blocco  $b$  sfrutta la località spaziale per ridurre il tasso di miss, ma diminuisce il numero di set a parità di capacità della cache e può quindi causare più conflitti, oltre ad accrescere la penalizzazione di miss.

**Tabella 8.2** Possibili organizzazioni della cache.

| Organizzazione            | Numero di vie (N) | Numero di set in cache (S) |
|---------------------------|-------------------|----------------------------|
| A mappatura diretta       | 1                 | B                          |
| Parzialmente associativa  | $1 < N < B$       | B/N                        |
| Completamente associativa | B                 | 1                          |

**8.3.3 Quale dato viene sostituito?**

In una cache a mappatura diretta, ogni indirizzo viene mappato in un unico blocco e set, quindi se un set è pieno quando serve caricare in cache un nuovo dato il blocco in quel set viene sostituito dal nuovo dato. Nelle cache parzialmente o completamente associative, invece, si deve scegliere quale blocco espellere quando il set è pieno. Il principio di località temporale suggerisce che la scelta migliore sia quella di espellere il blocco utilizzato meno recentemente, perché è quello con la minore probabilità di essere riutilizzato a breve. La politica di sostituzione dei blocchi adottata dalla maggior parte delle cache associative è dunque la politica **LRU** (*Least Recently Used*, utilizzato meno recentemente).

In una cache parzialmente associativa a due vie, un **bit di utilizzo**, *U*, indica quale via nel set è stata utilizzata meno recentemente: ogni volta che si usa una delle due vie, il bit viene aggiornato per indicare l'altra via. Per cache parzialmente associative a più di due vie, tenere traccia della via utilizzata meno recentemente diventa complicato: per semplificare la cosa, le vie sono spesso divise in due gruppi e *U* indica quale gruppo di vie è quello usato meno recentemente. Al momento della sostituzione, il nuovo blocco sostituisce un blocco a caso del gruppo usato meno recentemente. Questa politica prende il nome di **pseudo-LRU** e in pratica si dimostra sufficientemente valida.

**ESEMPIO 8.10**

**Sostituzione LRU.** Mostrare il contenuto di una cache parzialmente associativa a due vie di otto parole dopo l'esecuzione del seguente codice, assumendo una politica di sostituzione LRU, una dimensione di blocco di una parola e la cache inizialmente vuota.

```
MOV R0, #0
LDR R1, [R0, #4]
LDR R2, [R0, #0x24]
LDR R3, [R0, #0x54]
```

**Soluzione** Le prime due istruzioni caricano i dati dagli indirizzi di memoria 0x4 e 0x24 nel set 1 della cache, come mostrato nella **Figura 8.15(a)**. *U* = 0 indica che il dato nella via 0 è quello usato meno recentemente. Il prossimo accesso a memoria, a indirizzo 0x54, viene pure mappato nel set 1, e sostituisce il dato della via 0 in quanto usato meno recentemente, come mostrato nella **Figura 8.15(b)**. Il bit di uso *U* viene portato a 1 per indicare che ora il dato nella via 1 è quello usato meno recentemente.

**8.3.4 Progetto di cache avanzate\***

I moderni calcolatori usano livelli multipli di cache per ridurre i tempi di accesso a memoria. In questo paragrafo si considerano le prestazioni di un sistema di cache a due livelli, analizzando come la dimensione di blocco, l'associatività e la capacità delle cache influenzano il tasso di miss. Si vede anche come le cache gestiscono le scritture in memoria, adottando una delle due politiche *write-through* oppure *write-back*.

| Via 1 |   |          |                | Via 0 |          |                |            |
|-------|---|----------|----------------|-------|----------|----------------|------------|
| V     | U | Tag      | Dati           | V     | Tag      | Dati           |            |
| 0     | 0 |          |                | 0     |          |                | Set 3 (11) |
| 0     | 0 |          |                | 0     |          |                | Set 2 (10) |
| 1     | 0 | 00...010 | mem[0x00...24] | 1     | 00...000 | mem[0x00...04] | Set 1 (01) |
| 0     | 0 |          |                | 0     |          |                | Set 0 (00) |

(a)

| Via 1 |   |          |                | Via 0 |          |                |            |
|-------|---|----------|----------------|-------|----------|----------------|------------|
| V     | U | Tag      | Dati           | V     | Tag      | Dati           |            |
| 0     | 0 |          |                | 0     |          |                | Set 3 (11) |
| 0     | 0 |          |                | 0     |          |                | Set 2 (10) |
| 1     | 1 | 00...010 | mem[0x00...24] | 1     | 00...101 | mem[0x00...54] | Set 1 (01) |
| 0     | 0 |          |                | 0     |          |                | Set 0 (00) |

(b)

**Figura 8.15**  
Cache parzialmente associativa  
a due vie con politica  
di sostituzione LRU.

### Cache multi livello

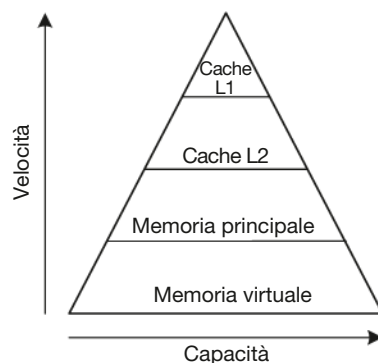
Cache di grandi dimensioni sono utili perché hanno maggiore probabilità di contenere i dati necessari e quindi di ridurre il tasso di miss, ma tendono a essere più lente delle cache piccole. I moderni calcolatori adottano spesso almeno due livelli di cache, come mostrato nella **Figura 8.16**: la cache di primo livello (L1) è abbastanza piccola da garantire un tempo di accesso di uno o due cicli di clock; la cache di secondo livello (L2) è pure realizzata con SRAM ma è più grande, quindi più lenta, della cache L1. Il processore guarda prima se il dato richiesto è presente nella cache L1; in caso di miss, il processore guarda nella cache L2; in caso di ulteriore miss, preleva il dato dalla memoria principale. Molti calcolatori moderni aggiungono ulteriori livelli di cache nella gerarchia di memoria, perché l'accesso alla memoria principale è decisamente lento.

#### ESEMPIO 8.11

**Calcolatore con cache L2.** Utilizzare il calcolatore della Figura 8.16, con tempi di accesso di 1, 10 e 100 cicli di clock, rispettivamente per la cache L1, la cache L2 e la memoria principale, assumendo che le cache L1 e L2 abbiano tassi di miss rispettivamente del 5% e del 20% (quindi, del 5% di accessi che provoca miss in L1, il 20% provoca miss anche in L2). Qual è il tempo medio di accesso a memoria (AMAT)?

**Soluzione** Ogni accesso a memoria verifica la cache L1. In caso di miss (5% delle volte) il processore guarda in L2. In caso di ulteriore miss (20% delle volte) il processore preleva il dato dalla memoria principale. Utilizzando l'Espressione 8.2 si può calcolare il tempo medio di accesso a memoria:  $AMAT = 1 \text{ ciclo} + 0.05 [10 \text{ cicli} + 0.2 (100 \text{ cicli})] = 2.5 \text{ cicli}$ .

Il tasso di miss della cache L2 è alto perché tale cache riceve solo gli accessi a memoria più critici, ovvero quelli che hanno generato miss in L1. Se tutti gli accessi fossero fatti direttamente in L2, il suo tasso di miss sarebbe circa dell'1%.



**Figura 8.16**  
Gerarchia di memoria con due livelli  
di cache.

### Come ridurre il tasso di miss

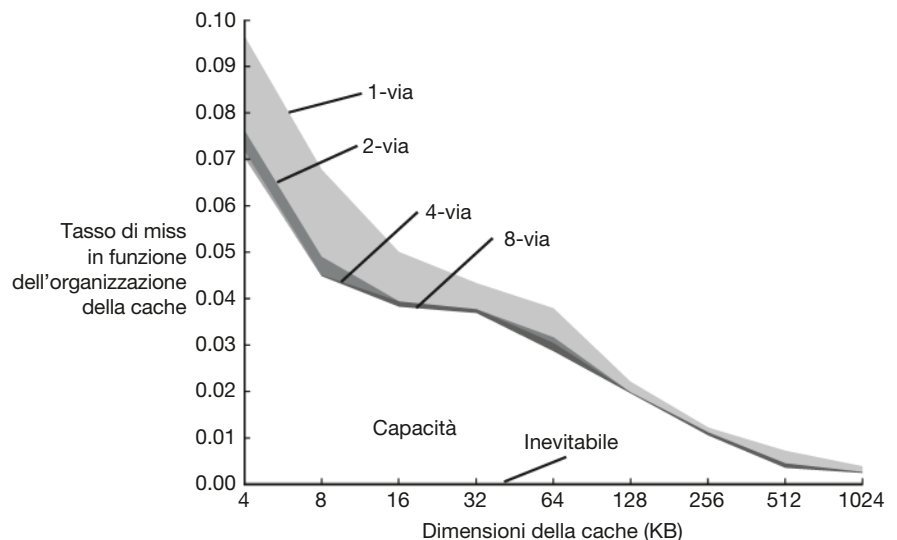
Le situazioni di miss possono essere ridotte modificando la capacità, la dimensione di blocco e/o l'associatività della cache. Il primo passo per ridurre tali situazioni è quello di capirne le cause. Le situazioni di miss si possono classificare come inevitabili, dovute alla capacità e dovute ai conflitti. La prima richiesta di un blocco di cache è definita **miss inevitabile** perché il blocco deve essere per forza letto dalla memoria principale indipendentemente dall'organizzazione della cache. I **miss di capacità** si verificano quando la cache è troppo piccola per contenere tutti i dati utilizzati in modo concorrente. I **miss di conflitto** si verificano, infine, quando più indirizzi vengono mappati nello stesso set ed espellono blocchi ancora necessari.

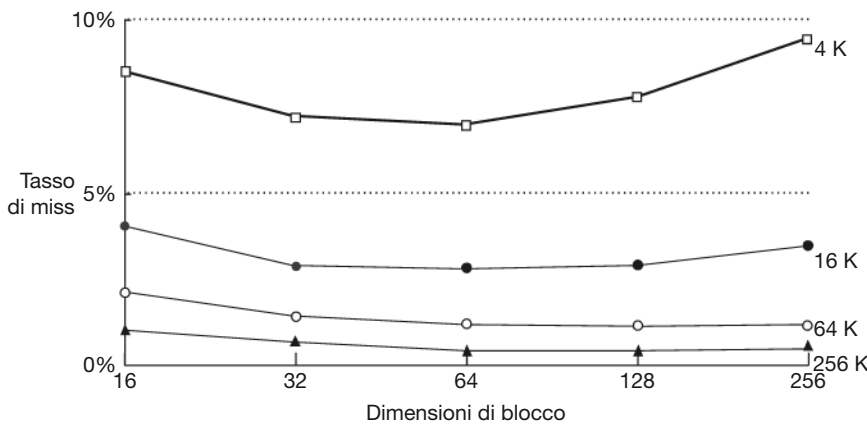
La modifica dei parametri della cache può influenzare uno o più tipi di miss: per esempio, incrementare la capacità della cache può ridurre i miss di conflitto e di capacità, ma non ha effetto sui miss inevitabili. D'altra parte, aumentare la dimensione di blocco può ridurre i miss inevitabili (grazie alla località spaziale) ma può anche aumentare i miss di conflitto (perché più indirizzi si mappano nel medesimo set e possono entrare in conflitto).

I sistemi di memoria sono così complessi che il modo migliore per valutarne le prestazioni è quello di eseguire programmi di benchmark modificando i parametri delle cache. Il grafico della **Figura 8.17** riporta il tasso di miss in funzione della dimensione di cache e del grado di associatività per il benchmark SPEC2000. Questo benchmark ha un numero ridotto di miss inevitabili, indicati dall'area scura vicino all'asse x. Come ci si poteva attendere, al crescere della capacità della cache i miss di capacità diminuiscono. Aumentare il grado di associatività, soprattutto per cache piccole, diminuisce il numero di miss di conflitto, come indicato nella parte alta della curva. Adottare gradi di associatività maggiori di quattro od otto dà solo minime riduzioni del tasso di miss.

Come detto, il tasso di miss può essere ridotto anche aumentando la dimensione di blocco per sfruttare la località spaziale. Ma a parità di capacità totale della cache, al crescere di tale dimensione diminuisce il numero di set e aumenta quindi la probabilità di conflitti. Il grafico della **Figura 8.18** riporta il tasso di miss in funzione della dimensione di blocco (espressa in numero di byte) per cache di diversa capacità. Per cache piccole, come una cache da 4 KB, aumentare la dimensione di blocco oltre 64 byte aumenta il tasso di miss a causa dei conflitti. Invece, per cache grandi, aumentare la dimensione

**Figura 8.17**  
Tasso di miss in funzione delle dimensioni e dell'organizzazione della cache usando il benchmark SPEC2000. (Con permesso di adattamento da Hennessy e Patterson, *Computer Architecture: A Quantitative Approach*, 5a ed., Morgan Kaufmann, 2011.)





**Figura 8.18**  
Tasso di miss in funzione delle dimensioni di blocco e di cache usando il benchmark SPEC92. (Con permesso di adattamento da Hennessy e Patterson, *Computer Architecture: A Quantitative Approach*, 5a ed., Morgan Kaufmann, 2011.)

di blocco oltre 64 byte non modifica il tasso di miss. Tuttavia blocchi grandi possono aumentare il tempo di esecuzione a causa di una maggiore penalizzazione di miss, ovvero il tempo necessario a prelevare dalla memoria principale il blocco mancante.

### Politiche di scrittura

I paragrafi precedenti si sono concentrati sulle operazioni di lettura da memoria. Le scritture in memoria seguono una procedura analoga. Il processore guarda per prima cosa nella cache: in caso di miss, il blocco mancante viene prelevato dalla memoria principale e copiato nella cache, quindi si esegue la scrittura del dato; in caso di hit, si esegue semplicemente la suddetta scrittura.

Le cache sono classificate nelle due categorie *write-through* e *write-back*. In una cache **write-through**, il dato viene scritto simultaneamente sia nella memoria cache sia nella memoria principale. In una cache **write-back**, un **bit di modifica M** (*dirty bit*) è associato a ogni blocco di cache: tale bit vale 1 se il blocco è stato modificato da almeno una scrittura, altrimenti vale 0. I blocchi di cache modificati vengono riscritti nella memoria principale solo al momento di essere espulsi dalla cache. Una cache *write-through* non ha bisogno del bit di modifica, ma richiede generalmente più accessi alla memoria principale di una cache *write-back*. Le moderne cache sono quasi sempre *write-back* per l'eccessiva lentezza della memoria principale.

### ESEMPIO 8.12

**Confronto tra *write-through* e *write-back*.** Si consideri una cache con dimensione di blocco di quattro parole. Quanti accessi a memoria principale sono necessari nell'esecuzione del seguente codice usando la politica *write-through* oppure la politica *write-back*?

```
MOV R5, #0
STR R1, [R5]
STR R2, [R5, #12]
STR R3, [R5, #8]
STR R4, [R5, #4]
```

**Soluzione** Tutte le quattro istruzioni di scrittura accedono al medesimo blocco di cache. Con la politica *write-through*, ogni istruzione deve accedere alla memoria principale, quindi servono quattro accessi. Con la politica *write-back* è necessario un solo accesso alla memoria principale, quando il blocco di cache modificato deve essere espulso.

8.3.5 Evoluzione delle cache di ARM

La **Tabella 8.3** riassume l’evoluzione delle organizzazioni di cache usate dal processore ARM dal 1985 al 2012. Le principali tendenze sono l’introduzione di più livelli di cache, l’aumento della capacità, la separazione Istruzioni e Dati (I/D) nelle cache L1. Tali tendenze sono la conseguenza della crescente disparità tra la frequenza della CPU e la velocità della memoria principale, e dei sempre minori costi dei transistori: la crescente disparità tra la frequenza della CPU e la velocità della memoria principale richiede tassi di miss più bassi per evitare il collo di bottiglia della memoria principale, mentre i minori costi dei transistori consentono di realizzare cache sempre più grandi.

8.4 ■ MEMORIA VIRTUALE

La maggior parte dei moderni calcolatori usa un **disco rigido** in tecnologia magnetica o a stato solido al gradino più basso della gerarchia di memoria, come mostrato nella Figura 8.4. Confrontato con la memoria ideale grande, veloce ed economica, il disco rigido è grande ed economico ma terribilmente lento: può fornire infatti una capacità molto maggiore di una qualsiasi DRAM economicamente ragionevole, ma se una frazione significativa degli accessi a memoria coinvolge il disco rigido, le prestazioni diventano inaccettabili, come può capitare se sul proprio PC sono in esecuzione simultaneamente troppi programmi.

La **Figura 8.19** mostra un’unità a dischi rigidi magnetici, cui è stato tolto il coperchio. Come suggerisce il nome, l’unità contiene uno o più dischi rigidi (piatti), ciascuno dotato di una **testina di lettura/scrittura** posizionata sulla punta di un braccio triangolare. La testina si sposta sulla corretta locazione e sfrutta l’elettromagnetismo per leggere o scrivere dati sul disco in rotazione sotto di lei. La testina impiega vari millisecondi per raggiungere la corretta locazione sul disco: un tempo breve su scala umana ma milioni di volte più lento di quello del processore. I dischi rigidi magnetici sono progressivamente sostituiti da dischi a stato solido, perché in questi ultimi la velocità di lettura ha ordini di grandezza maggiori (Figura 8.4) e perché non sono soggetti a guasti meccanici.

L’obiettivo di aggiungere un disco rigido alla gerarchia di memoria è quello di dare l’illusione di uno spazio di memoria molto grande pur mantenendo la velocità delle memorie più rapide per la maggior parte degli accessi: per esempio, un calcolatore con solo 128 MB di DRAM potrebbe fornire una memoria da 2 GB usando un disco rigido. Questa memoria da 2 GB prende il

Un calcolatore con 32 bit di indirizzo può accedere al massimo a  $2^{32}$  byte = 4 GB di memoria. Questo è uno dei motivi per passare a calcolatori a 64 bit, che possono accedere a una memoria di gran lunga più grande.

Tabella 8.3    Evoluzione della cache di ARM.

| Anno | CPU        | MHz    | Cache L1        | Cache L2                  |
|------|------------|--------|-----------------|---------------------------|
| 1985 | ARM1       | 8      | Nessuna         | Nessuna                   |
| 1992 | ARM6       | 30     | 4 KB, singola   | Nessuna                   |
| 1994 | ARM7       | 100    | 8 KB, singola   | Nessuna                   |
| 1999 | ARM9E      | 300    | 0-128 KB, I/D   | Nessuna                   |
| 2002 | ARM11      | 700    | 4-64 KB, I/D    | 0-128 KB, esterna al chip |
| 2009 | Cortex-A9  | 1000   | 16-64 KB, I/D   | 0-8 MB                    |
| 2011 | Cortex-A7  | 1500   | 32 KB, I/D      | 0-4 MB                    |
| 2011 | Cortex-A15 | 2000   | 32 KB, I/D      | 0-4 MB                    |
| 2012 | Cortex-M0+ | 60-250 | Nessuna         | Nessuna                   |
| 2012 | Cortex-A53 | 1500   | 8-64 KB, I/D    | 128 KB-2 MB               |
| 2012 | Cortex-A57 | 2000   | 48 KB I/32 KB D | 512 KB-2 MB               |



Figura 8.19  
Disco rigido.

nome di **memoria virtuale**, mentre la memoria principale (da 128 MB in questo esempio) è denominata **memoria fisica** quando si fa riferimento – come in questo paragrafo – all’uso di una gerarchia di memoria con disco rigido.

I programmi possono accedere a un qualsiasi dato della memoria virtuale, quindi devono utilizzare **indirizzi virtuali** che specificano le locazioni dei dati nella memoria virtuale. La memoria fisica contiene un sottoinsieme della parte di memoria virtuale più recentemente utilizzata. In questo modo, la memoria fisica agisce da cache della memoria virtuale: la maggior parte degli accessi dà luogo a hit, quindi accede a memoria con la velocità della DRAM, ma il programma può fare riferimento alla memoria virtuale ben più grande.

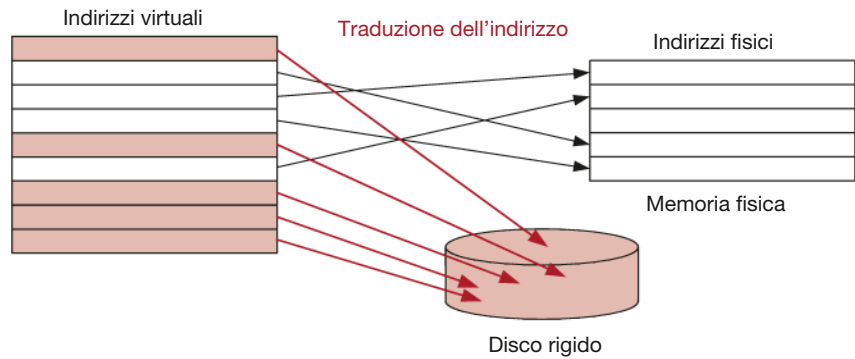
I sistemi di memoria virtuale usano una terminologia diversa per gli stessi principi usati nelle cache discusse nel paragrafo 8.3: la **Tabella 8.4** riporta i termini concettualmente analoghi. La memoria virtuale è suddivisa in **pagine virtuali**, tipicamente di 4 KB. La memoria fisica è anch’essa suddivisa in **pagine fisiche** della stessa dimensione. Una pagina virtuale può trovarsi in memoria fisica (DRAM) oppure su disco. Per esempio, la **Figura 8.20** mostra una memoria virtuale più grande della memoria fisica. I rettangoli indicano le pagine: alcune sono presenti in memoria fisica, altre si trovano su disco. Il processo di determinare l’indirizzo fisico a partire da quello virtuale prende

Tabella 8.4 Termini corrispondenti tra cache e memoria virtuale.

| Cache                   | Memoria virtuale          |
|-------------------------|---------------------------|
| Blocco                  | Pagina                    |
| Dimensione di blocco    | Dimensione di pagina      |
| Spiazzamento di blocco  | Spiazzamento di pagina    |
| Miss (dato non trovato) | Mancanza di pagina        |
| Tag                     | Numero di pagina virtuale |



**Figura 8.20**  
Pagine virtuali e fisiche.



il nome di **traduzione dell'indirizzo**. Se il processore tenta di accedere a un indirizzo virtuale non presente in memoria fisica, si verifica un'eccezione di **mancanza di pagina** (*page fault*) e il sistema operativo si occupa di caricare la pagina mancante da disco in memoria fisica.

Per evitare mancanze di pagina dovute ai conflitti, ogni pagina virtuale può essere mappata in qualsiasi pagina fisica: in altre parole, la memoria fisica si comporta da cache completamente associativa per la memoria virtuale. Nella cache completamente associativa vera e propria, ogni blocco di cache ha un comparatore che confronta i bit più significativi dell'indirizzo con il tag per determinare se la richiesta ha portato a una situazione di hit, quindi per analogia ogni pagina fisica dovrebbe avere un comparatore per confrontare i bit più significativi dell'indirizzo virtuale e un opportuno tag per determinare se la pagina virtuale desiderata si trova in memoria fisica.

Un sistema di memoria virtuale realistico ha però troppe pagine fisiche per poter prevedere a costi ragionevoli un comparatore per ogni pagina. Si usa invece una tabella delle pagine per effettuare la traduzione dell'indirizzo. La tabella delle pagine ha un elemento per ogni pagina virtuale, che indica in quale pagina fisica tale pagina virtuale si trova, oppure che è presente solo su disco. Ogni istruzione di lettura o scrittura in memoria richiede quindi un accesso alla tabella delle pagine seguito da un accesso alla memoria fisica: l'accesso alla tabella delle pagine traduce l'indirizzo virtuale usato dal programma in indirizzo fisico, che viene poi utilizzato per l'effettivo accesso di lettura o scrittura del dato.

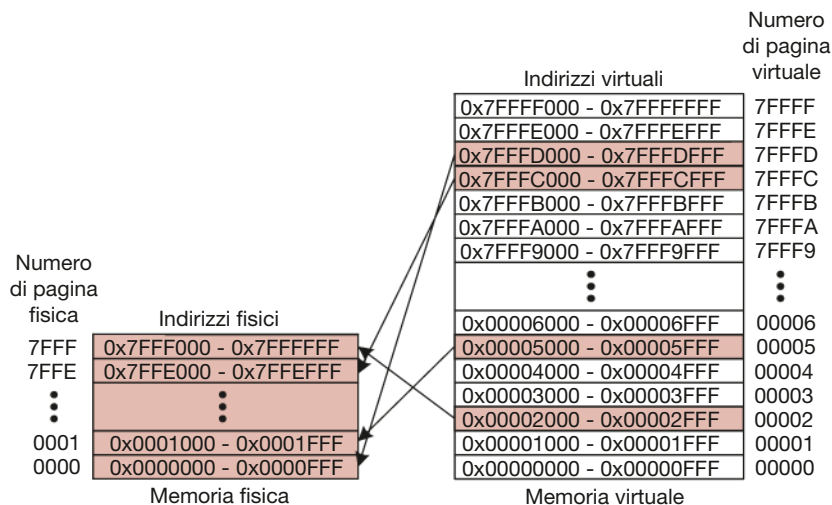
La tabella delle pagine è normalmente così grande da essere memorizzata in memoria fisica, quindi ogni lettura o scrittura richiede due accessi alla memoria principale, uno alla tabella delle pagine, l'altro al dato. Per velocizzare l'operazione si usa spesso il cosiddetto **Translation Lookaside Buffer (TLB)**, che tiene in cache gli elementi della tabella delle pagine di utilizzo più frequente.

Il resto del paragrafo approfondisce i concetti di traduzione dell'indirizzo, tabella delle pagine e TLB.

### 8.4.1 Traduzione dell'indirizzo

Nei calcolatori con memoria virtuale, i programmi usano indirizzi virtuali per poter accedere a un ampio spazio di memoria. Il processore deve quindi tradurre tali indirizzi virtuali per fare riferimento al dato in memoria fisica oppure per generare un'eccezione di mancanza di pagina e prelevare il dato da disco rigido.

Dal momento che sia la memoria virtuale sia la memoria fisica sono divise in pagine, i bit più significativi dell'indirizzo virtuale e dell'indirizzo fisico specificano il **numero di pagina** rispettivamente virtuale e fisica, mentre i bit meno significativi indicano la singola parola all'interno della pagina e sono denominati **spiazzamento di pagina**.



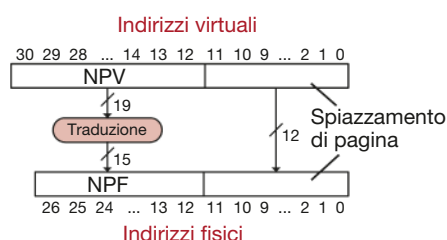
**Figura 8.21**  
Pagine fisiche e virtuali.

La **Figura 8.21** mostra l'organizzazione delle pagine di un sistema di memoria virtuale con 2 GB di memoria virtuale e 128 MB di memoria fisica, suddivise in pagine da 4 KB ciascuna. Se il processore ha indirizzi a 32 bit, con una memoria virtuale da 2 GB =  $2^{31}$  byte solo i 31 bit meno significativi dell'indirizzo virtuale vengono utilizzati: il 32-mo bit vale sempre 0. Analogamente, con una memoria fisica da 128 MB =  $2^{27}$  byte, solo i 27 bit meno significativi dell'indirizzo virtuale vengono utilizzati: i 5 bit più significativi valgono sempre 0.

Dal momento che la dimensione di pagina è 4 KB =  $2^{12}$  byte, ci sono  $2^{31}/2^{12} = 2^{19}$  pagine virtuali e  $2^{27}/2^{12} = 2^{15}$  pagine fisiche. I numeri di pagina virtuale e fisica occupano dunque rispettivamente 19 e 15 bit. In ogni istante, la memoria fisica può contenere al massimo 1/16 delle pagine virtuali: le restanti vengono mantenute su disco.

La **Figura 8.21** mostra la mappatura della pagina virtuale 5 nella pagina fisica 1, della pagina virtuale 0x7FFFC nella pagina fisica 0x7FFE, e così via. Per esempio, l'indirizzo virtuale 0x53F8 (che ha uno spaziamento di 0x3F8 nella pagina virtuale 5) viene mappato nell'indirizzo fisico 0x13F8 (che ha uno spaziamento di 0x3F8 nella pagina fisica 1). I 12 bit meno significativi sia dell'indirizzo virtuale sia di quello fisico sono gli stessi (0x3F8) e indicano lo spaziamento di pagina sia nella pagina virtuale sia in quella fisica. Solo il numero di pagina deve essere tradotto per ottenere l'indirizzo fisico a partire da quello virtuale.

La **Figura 8.22** illustra la traduzione da indirizzo virtuale a fisico. I 12 bit meno significativi indicano lo spaziamento di pagina e non richiedono traduzione. I successivi 19 bit più significativi dell'indirizzo virtuale indicano il **numero di pagina virtuale (NPV)** e vengono tradotti nei 15 bit del **numero di pagina fisica (NPF)**. I prossimi due paragrafi spiegano come usare la tabella delle pagine e il TLB per effettuare questa traduzione.



**Figura 8.22**  
Traduzione da indirizzo virtuale a indirizzo fisico.

ESEMPIO 8.13

**Traduzione da indirizzo virtuale a indirizzo fisico.** Trovare l'indirizzo fisico corrispondente all'indirizzo virtuale 0x247C nel sistema di memoria virtuale della Figura 8.21.

**Soluzione** Lo spiazzamento di pagina di 12 bit (0x47C) non richiede traduzione. I restanti 19 bit dell'indirizzo virtuale sono il numero di pagina virtuale, quindi l'indirizzo virtuale 0x247C si trova nella pagina virtuale 2. Nella Figura 8.21, la pagina virtuale 2 è mappata nella pagina fisica 0x7FFF, quindi l'indirizzo virtuale 0x247C è mappato nell'indirizzo fisico 0x7FFF47C.

| V | Numero di pagina fisica | Numero di pagina virtuale |
|---|-------------------------|---------------------------|
| 0 |                         | 7FFFF                     |
| 0 |                         | 7FFFE                     |
| 1 | 0x0000                  | 7FFFD                     |
| 1 | 0x7FFE                  | 7FFFC                     |
| 0 |                         | 7FFFB                     |
| 0 |                         | 7FFFA                     |
|   | ⋮                       | ⋮                         |
| 0 |                         | 00007                     |
| 0 |                         | 00006                     |
| 1 | 0x0001                  | 00005                     |
| 0 |                         | 00004                     |
| 0 |                         | 00003                     |
| 1 | 0x7FFF                  | 00002                     |
| 0 |                         | 00001                     |
| 0 |                         | 00000                     |

Tabella delle pagine

Figura 8.23  
La tabella delle pagine per la Figura 8.21.

8.4.2 La tabella delle pagine

Il processore usa la **tabella delle pagine** per tradurre indirizzi virtuali in indirizzi fisici. La tabella delle pagine è dotata di un elemento per ogni pagina virtuale; ogni elemento contiene un numero di pagina fisica e un bit di validità V: se tale bit vale 1, la pagina virtuale è mappata nella pagina fisica specificata nell'elemento, altrimenti la pagina virtuale va caricata da disco.

Dal momento che la tabella delle pagine è molto grande, viene memorizzata in memoria fisica. Nell'ipotesi che sia memorizzata come un array di elementi contigui, la tabella delle pagine relativa alla mappatura del sistema di memoria della Figura 8.21 risulta essere quella riportata nella **Figura 8.23**. L'indice usato per individuare ogni elemento della tabella è il numero di pagina virtuale (NPV): per esempio, l'elemento 5 indica che la pagina virtuale numero 5 è mappata nella pagina fisica numero 1. L'elemento 6 è non valido (V = 0), quindi la pagina virtuale numero 6 si trova su disco.

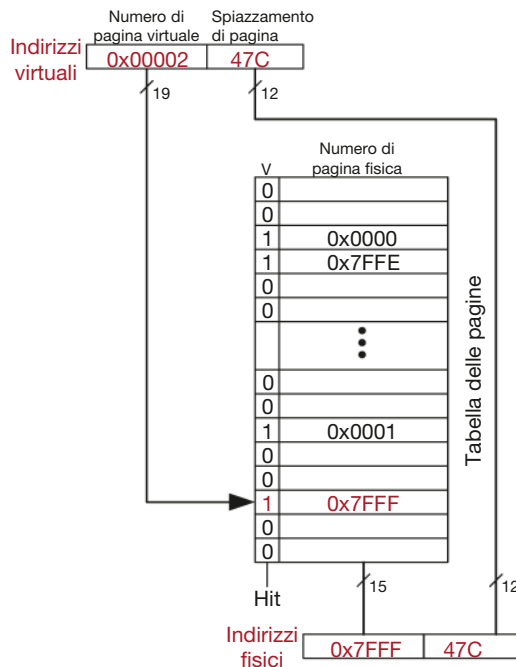
ESEMPIO 8.14

**Uso della tabella delle pagine per effettuare la traduzione dell'indirizzo.** Trovare l'indirizzo fisico corrispondente all'indirizzo virtuale 0x247C utilizzando la tabella delle pagine mostrata nella Figura 8.23.

**Soluzione** La **Figura 8.24** mostra la traduzione da virtuale a fisico per l'indirizzo virtuale 0x247C. Lo spiazzamento di pagina di 12 bit (0x47C) non richiede traduzione. I restanti 19 bit dell'indirizzo virtuale sono il numero di pagina virtuale, 0x2, e costituiscono l'indice nella tabella delle pagine. La tabella mappa la pagina virtuale 0x2 nella pagina fisica 0x7FFF, quindi l'indirizzo virtuale 0x247C è mappato nell'indirizzo fisico 0x7FFF47C. I 12 bit meno significativi sono gli stessi in entrambi gli indirizzi, virtuale e fisico.

La tabella delle pagine può essere memorizzata ovunque nella memoria fisica, a discrezione del sistema operativo. Il processore usa tipicamente un registro dedicato, il **registro di tabella delle pagine**, per memorizzare l'indirizzo base della tabella delle pagine in memoria fisica.

Per effettuare una lettura o una scrittura, il processore deve prima tradurre l'indirizzo virtuale in indirizzo fisico, poi accedere al dato in memoria fisica. Per far questo, estrae il numero di pagina virtuale dall'indirizzo virtuale e lo somma al registro di tabella delle pagine per trovare l'indirizzo fisico dell'elemento della tabella delle pagine necessario; poi legge tale elemento per ottenere il numero di pagina fisica. Se l'elemento è valido, il processore unisce il numero di pagina fisica e lo spiazzamento di pagina per ottenere l'indirizzo fisico cui accedere; infine effettua la lettura o la scrittura. Dal momento che la tabella delle pagine è mantenuta in memoria fisica, questo comporta due accessi a memoria fisica.



**Figura 8.24**  
Traduzione dell'indirizzo mediante tabella delle pagine.

### 8.4.3 Il Translation Lookaside Buffer (TLB)

La memoria virtuale avrebbe un impatto molto grave sulle prestazioni se richiedesse davvero un accesso alla tabella delle pagine per ogni lettura o scrittura di un dato, raddoppiando il ritardo di tali operazioni. Fortunatamente, gli accessi alla tabella delle pagine mostrano un'elevata località temporale: le località temporale e spaziale degli accessi ai dati e l'ampia dimensione delle pagine fanno sì che sia molto probabile che molte letture e scritture consecutive facciano riferimento alla stessa pagina. Se dunque il processore tiene traccia dell'ultimo elemento letto dalla tabella delle pagine, avrà ottime probabilità di riutilizzare tale elemento senza doverlo rileggere dalla tabella. In generale, il processore può tenere traccia di un certo numero di elementi della tabella delle pagine nel cosiddetto **Translation Lookaside Buffer (TLB)**: il processore "sbircia in disparte" nel TLB per trovare la traduzione dell'indirizzo prima di dover accedere alla memoria principale per leggerla dalla tabella delle pagine. Nei programmi reali, la stragrande maggioranza degli accessi dà luogo a hit nel TLB, evitando i lenti accessi a memoria fisica per leggere la tabella delle pagine.

Il TLB è organizzato come una cache completamente associativa, in grado di memorizzare tipicamente da 16 a 512 elementi: ogni elemento contiene un numero di pagina virtuale e il corrispondente numero di pagina fisica. Si accede al TLB mediante il numero di pagina virtuale. Se si verifica hit, il TLB restituisce il corrispondente numero di pagina fisica. Altrimenti il processore deve leggere la tabella delle pagine nella memoria fisica. Il TLB è abbastanza piccolo da poter essere interrogato in meno di un ciclo di clock: nonostante ciò, i TLB hanno tassi di hit normalmente superiori al 99%, riducendo quindi da due a uno il numero di accessi a memoria necessari per eseguire le istruzioni di lettura e scrittura.

#### ESEMPIO 8.15

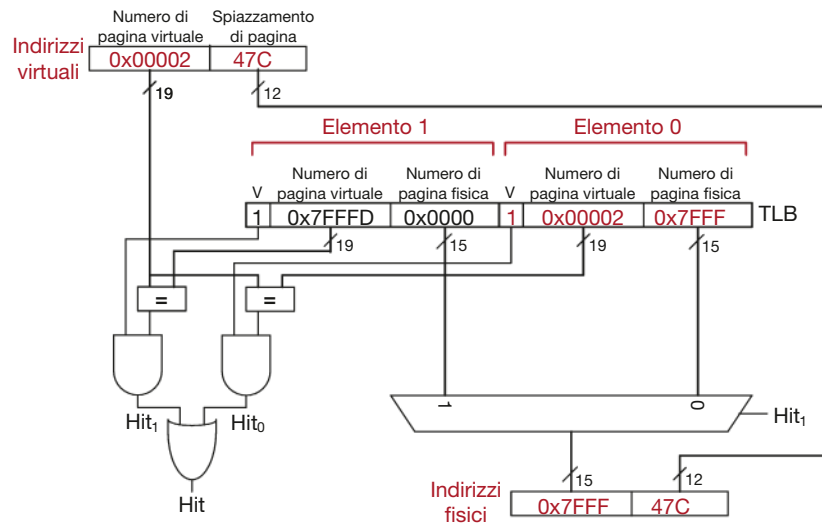
**Uso del TLB per effettuare la traduzione dell'indirizzo.** Riferendosi al sistema di memoria virtuale della Figura 8.21, mostrare l'uso di un TLB a due elementi oppure spiegare perché è necessario un accesso alla tabella delle pagine per tradurre gli indi-

rizzi virtuali 0x247C e 0x5FB0 in indirizzi fisici. Si supponga che il TLB contenga in questo momento le traduzioni valide per le pagine virtuali 0x2 e 0x7FFFD.

**Soluzione** La **Figura 8.25** mostra il TLB a due elementi al momento della richiesta dell'indirizzo virtuale 0x247C. Il TLB riceve il numero di pagina virtuale, 0x2, dall'indirizzo richiesto e lo confronta con ogni suo elemento. L'elemento 0 contiene lo stesso valore ed è valido, quindi si ha situazione di hit: l'indirizzo fisico tradotto è il numero di pagina fisica dell'elemento che ha dato confronto positivo, 0x7FFF, concatenato con lo spiazzamento di pagina dell'indirizzo virtuale (come sempre, lo spiazzamento di pagina non richiede traduzione).

La richiesta dell'indirizzo virtuale 0x5FB0 dà luogo a una situazione di miss nel TLB, quindi la richiesta viene inoltrata alla tabella delle pagine per effettuare la traduzione dell'indirizzo.

**Figura 8.25**  
Traduzione dell'indirizzo mediante  
TLB a due elementi.



#### 8.4.4 Protezione della memoria

Si è finora parlato di memoria virtuale come un mezzo per fornire una memoria veloce, economica e grande. Un'altra ragione ugualmente importante per usare la memoria virtuale è quella di fornire protezione tra i programmi in esecuzione concorrente sul calcolatore.

Come noto, i moderni calcolatori eseguono più programmi o **processi** allo stesso tempo, e tutti questi programmi sono simultaneamente presenti nella memoria fisica. In un calcolatore ben progettato, i programmi devono essere protetti l'uno dall'altro, in modo che nessun programma possa far abortire un altro programma o spiare le attività. Più in dettaglio, nessun programma deve poter accedere alla memoria di un altro programma senza permesso esplicito. Questa garanzia si chiama **protezione della memoria**.

I sistemi di memoria virtuale garantiscono protezione della memoria assegnando a ogni programma un suo proprio **spazio di indirizzamento virtuale**. Ogni programma può usare quanta memoria vuole del proprio spazio virtuale, ma solo una parte di tale memoria è presente in ogni istante in memoria fisica. Ogni programma può usare l'intera sua memoria virtuale senza preoccuparsi di dove gli altri programmi sono fisicamente allocati. Ma un programma può accedere solo alle pagine fisiche mappate nella propria tabella delle pagine, quindi non può accedere alle pagine fisiche degli altri programmi – né accidentalmente né con intenti dolosi – perché tali pagine non sono mappate nella sua tabella. In alcuni casi, più programmi hanno bi-

sogno di accedere a istruzioni o dati comuni: in tal caso, il sistema operativo può aggiungere dei bit di controllo a ogni elemento delle tabelle delle pagine per determinare se e quali programmi hanno diritto di scrivere in pagine fisiche condivise.

#### 8.4.5 Politiche di sostituzione\*

I sistemi di memoria virtuale usano una politica di scrittura *write-back* e una politica di sostituzione approssimativamente di tipo LRU (pagina usata meno recentemente). Adottare una politica di scrittura *write-through*, che richiede una scrittura su disco a ogni scrittura in memoria fisica, è chiaramente improponibile: le istruzioni di scrittura lavorerebbero alla velocità del disco rigido invece che alla velocità del processore (millisecondi invece di nanosecondi) mentre con la politica *write-back* una pagina viene riscritta su disco solo quando deve essere espulsa dalla memoria fisica. L'operazione di scrivere su disco una pagina fisica e caricare nella stessa pagina fisica un'altra pagina virtuale viene detta **paging** e il disco rigido usato nella memoria virtuale prende il nome di **swap area**. Il processore svuota una delle pagine fisiche usate meno di recente quando si verifica un'eccezione di mancanza di pagina, e vi carica la pagina virtuale mancante. Per supportare queste politiche di sostituzione, ogni elemento della tabella delle pagine contiene due ulteriori bit di stato: il bit di Modifica (*M*) e il bit di Utilizzo (*U*).

Il bit di Modifica vale 1 se una qualsiasi istruzione di scrittura in memoria ha modificato la pagina fisica dopo la sua lettura da disco. Quando una pagina fisica viene svuotata, deve essere prima riscritta su disco solo se il suo bit di modifica vale 1, altrimenti su disco è già presente una copia identica della stessa pagina.

Il bit di Utilizzo vale 1 se la pagina fisica è stata recentemente utilizzata. Come in un sistema di cache, una politica LRU esatta sarebbe impraticabile per la sua complessità. Il sistema operativo realizza quindi una politica LRU approssimata resettando periodicamente tutti i bit di Utilizzo nella tabella delle pagine; quando si accede a una pagina, il suo bit di Utilizzo viene forzato a 1; in caso di mancanza di pagina, il sistema operativo cerca una pagina con  $U = 0$  e la sostituisce. Questa procedura naturalmente non sostituisce la pagina usata meno recentemente, ma solo una di quelle non usate recentemente.

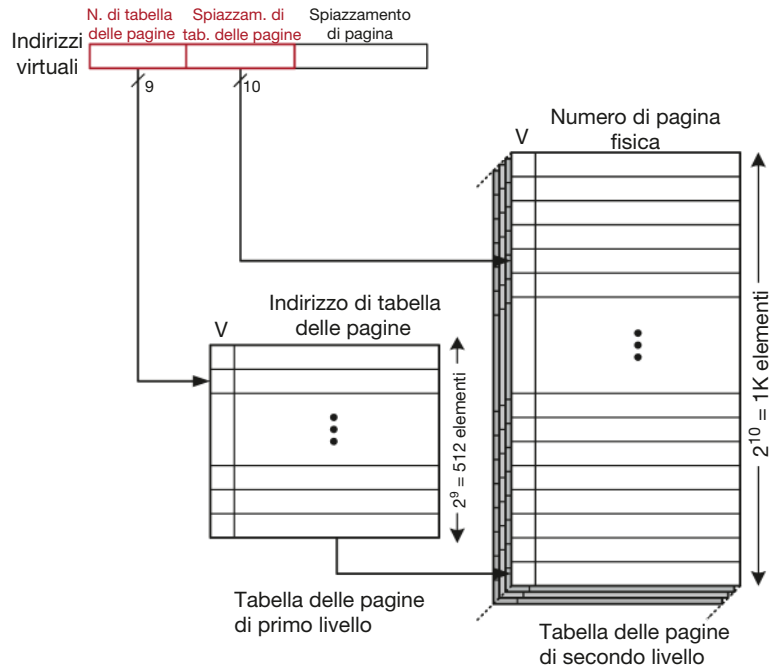
#### 8.4.6 Tabelle delle pagine multi livello\*

Le tabelle delle pagine possono occupare molta memoria fisica. Per esempio, la tabella delle pagine per la memoria virtuale da 2 GB con pagine da 4 KB usata nei paragrafi precedenti è costituita da  $2^{19}$  elementi. Se ogni elemento è di 4 byte, la tabella delle pagine occupa  $2^{19} \times 2^2 \text{ byte} = 2^{21} \text{ byte} = 2 \text{ MB}$ .

Per risparmiare memoria, le tabelle delle pagine possono essere divise in più livelli (generalmente due): la tabella di primo livello viene sempre mantenuta in memoria fisica, e indica dove si trovano in memoria fisica piccole tabelle di secondo livello. Ogni tabella di secondo livello contiene l'effettiva traduzione di un intervallo di pagine virtuali. Se un particolare intervallo di pagine virtuali non è attivamente utilizzato, la corrispondente tabella di secondo livello può essere lasciata su disco senza sprecare memoria fisica.

In una tabella delle pagine a due livelli, il numero di pagina virtuale è diviso in due parti: il **numero di tabella delle pagine** e lo **spiazzamento di tabella delle pagine**, come mostrato nella **Figura 8.26**. Il numero di tabella delle pagine è l'indice nella tabella di primo livello, che deve essere presente in memoria fisica: l'elemento corrispondente nella tabella di primo livello fornisce l'indirizzo base della tabella di secondo livello oppure segnala che deve essere prelevata da disco se  $V = 0$ . Lo spiazzamento di tabella delle pagine è l'indice

**Figura 8.26**  
Tabelle delle pagine gerarchiche.



nella tabella di secondo livello. I restanti 12 bit dell'indirizzo virtuale sono come al solito lo spiazzamento di pagina per pagine di  $2^{12}$  byte = 4 KB.

Nella Figura 8.26 il numero di pagina virtuale da 19 bit è suddiviso in 9 e 10 bit, che indicano rispettivamente il numero di tabella delle pagine e lo spiazzamento di tabella delle pagine. La tabella di primo livello ha dunque  $2^9 = 512$  elementi; ognuna delle 512 tabelle di secondo livello ha  $2^{10} = 1$  K elementi. Se ogni elemento delle tabelle di primo e secondo livello è di 32 bit (4 byte) e solo due tabelle di secondo livello alla volta sono presenti in memoria fisica, questo sistema gerarchico di tabelle delle pagine usa solo  $(512 \times 4 \text{ byte}) + 2 \times (1 \text{ K} \times 4 \text{ byte}) = 10 \text{ KB}$  di memoria fisica: una piccola frazione dello spazio necessario per memorizzare l'intera tabella delle pagine (2 MB). Lo svantaggio della tabella a due livelli è la necessità di un ulteriore accesso a memoria in caso di miss nel TLB.

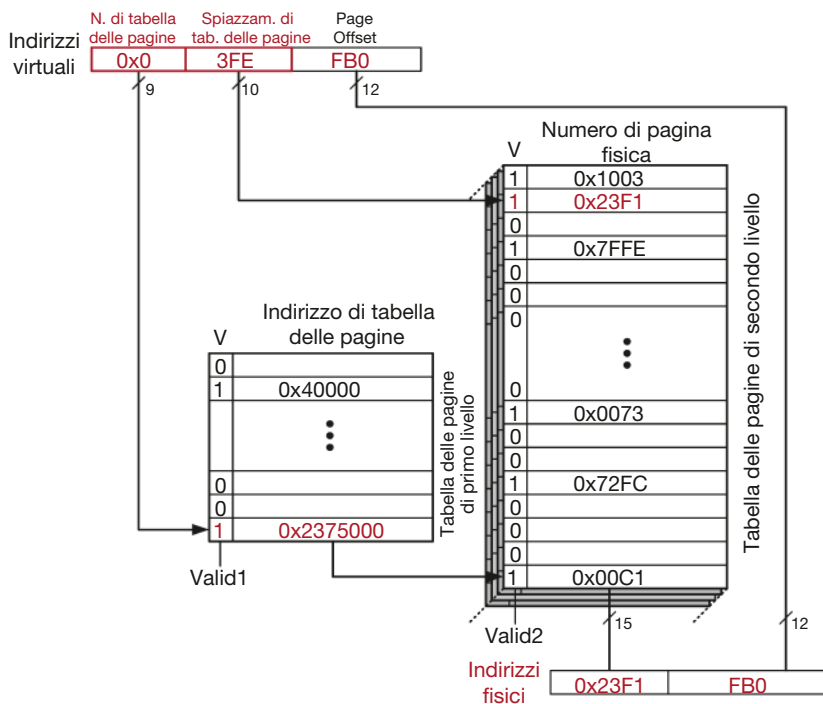
#### ESEMPIO 8.16

**Uso di una tabella delle pagine multi livello per effettuare la traduzione dell'indirizzo.** La Figura 8.27 mostra un possibile contenuto della tabella delle pagine a due livelli della Figura 8.26. Viene mostrato il contenuto di una sola tabella di secondo livello. Utilizzando questa struttura, descrivere cosa succede nell'accesso all'indirizzo virtuale 0x003FEFB0.

**Soluzione** Come sempre, solo il numero di pagina virtuale richiede traduzione. I nove bit più significativi dell'indirizzo virtuale, 0x0, sono il numero di tabella delle pagine, ovvero l'indice nella tabella di primo livello. L'elemento di indice 0x0 nella tabella di primo livello indica che la tabella di secondo livello si trova in memoria ( $V = 1$ ) all'indirizzo fisico 0x2375000.

I successivi dieci bit dell'indirizzo virtuale, 0x3FE, sono lo spiazzamento di tabella delle pagine, ovvero l'indice nella tabella di secondo livello. L'elemento di indice 0 nella tabella di secondo livello è quello più in basso, l'elemento di indice 0x3FF quello più in alto. L'elemento di indice 0x3FE nella tabella di secondo livello indica che la pagina virtuale è presente in memoria fisica ( $V = 1$ ) e che il numero di pagina fisica è 0x23F1. Il numero di pagina fisica concatenato con lo spiazzamento di pagina dà l'indirizzo fisico: 0x23F1FB0.





**Figura 8.27**  
Traduzione dell'indirizzo mediante tabella delle pagine a due livelli.

## 8.5 ■ RIASSUNTO

L'organizzazione del sistema di memoria è uno dei principali fattori che determinano le prestazioni di un calcolatore. Le diverse tecnologie di memorizzazione, come DRAM, SRAM e dischi rigidi, offrono diverse caratteristiche in termini di capacità, velocità e costo. Nel capitolo si è parlato di cache e memoria virtuale, che utilizzano una gerarchia di dispositivi di memoria per approssimare una memoria ideale grande, veloce ed economica. La memoria principale è tipicamente realizzata con componenti DRAM, significativamente più lenti del processore. La cache riduce il tempo di accesso mantenendo i dati usati più di frequente in componenti SRAM, decisamente più veloci. La memoria virtuale aumenta la capacità della memoria usando il disco rigido per memorizzare i dati che non trovano spazio nella memoria principale. Le cache e la memoria virtuale aggiungono complessità ed elementi circuitali al calcolatore, ma i benefici che ne derivano sono in genere più che adeguati a giustificarne i costi: praticamente tutti i moderni calcolatori anche personali usano cache e memoria virtuale.

## EPILOGO

Questo capitolo conclude il viaggio nel reame dei sistemi digitali. Gli Autori si augurano che questo testo sia stato capace di trasmettere la bellezza e il brivido dell'arte oltre alla conoscenza ingegneristica. Si è visto come progettare logica combinatoria e sequenziale utilizzando sia gli schemi circuitali sia i linguaggi di descrizione dell'hardware, tanto da avere familiarità con blocchi costruttivi come i multiplexer, le ALU e le memorie. I calcolatori sono una delle applicazioni più affascinanti dei sistemi digitali: si è visto come programmare il processore ARM nel suo linguaggio assembly nativo, e come costruire il processore e il sistema di memoria utilizzando i blocchi costruttivi digitali. In tutto il percorso si sono applicate le tecniche di astrazione, disciplina, gerarchia, modularità, regolarità: grazie a queste tecniche si è visto come mettere insieme tutti i pezzi che consentono il funzionamento intimo del micropro-

cessore. Il mondo di oggi è sempre più digitale: dai telefoni cellulari, alla televisione digitale, alla navigazione su Marte, ai sistemi di analisi delle immagini mediche.

Charles Babbage avrebbe fatto un patto col diavolo per fare questo stesso viaggio un secolo e mezzo fa: la sua aspirazione all'epoca era solo quella di calcolare tabelle matematiche con precisione. I sistemi digitali di oggi sono la fantascienza di ieri: Dick Tracy non si sarebbe mai immaginato di poter ascoltare iTunes sul suo telefono cellulare, né Jules Verne di poter lanciare nello spazio una costellazione di satelliti per il sistema di posizionamento globale, né Ippocrate di poter curare le malattie con immagini digitali ad alta risoluzione del cervello! Ma, al tempo stesso, l'incubo della sorveglianza totale da parte degli organi di governo immaginata da George Orwell è ogni giorno più vicino ad avverarsi: gli hacker, ma anche i governi, intraprendono cyberguerre senza dichiararlo, attaccando l'infrastruttura industriale e le reti finanziarie, e i Paesi a rischio sviluppano armi nucleari usando calcolatori portatili più potenti dei supercalcolatori grandi come una stanza usati per simulare l'impatto delle bombe all'epoca della Guerra Fredda. La rivoluzione dei microprocessori continua ad accelerare, e i cambiamenti nel prossimo decennio supereranno quelli del passato. Chi ha acquisito la capacità di progettare e costruire questi nuovi sistemi che cambieranno il futuro di tutti ha un grande potere ma anche una grande responsabilità: è speranza degli Autori che tale potere venga usato non solo per divertimento e per denaro, ma soprattutto per il bene dell'umanità.

## Esercizi

**Esercizio 8.1** Descrivere in meno di una pagina quattro attività quotidiane che mostrano località temporale o spaziale. Elencare in modo specifico due attività di ciascun tipo.

**Esercizio 8.2** Descrivere in un paragrafo due applicazioni dei calcolatori che mostrano località temporale e/o spaziale, spiegando in modo specifico come manifestano tali caratteristiche.

**Esercizio 8.3** Definire una sequenza di indirizzi per i quali una memoria cache a mappatura diretta con una capacità (dimensione totale) di 16 parole e una dimensione di blocco di 4 parole ha prestazioni superiori a quelle di una cache completamente associativa con politica di sostituzione LRU (*Least Recently Used*, blocco usato meno di recente) con la stessa capacità e dimensione di blocco.

**Esercizio 8.4** Ripetere l'Esercizio 8.3 per un caso in cui è la cache completamente associativa ad avere prestazioni superiori rispetto a quella a mappatura diretta.

**Esercizio 8.5** Descrivere pregi e difetti di accrescere ciascuno dei seguenti parametri di una cache mantenendo costanti gli altri due:

- (a) dimensione di blocco
- (b) associatività
- (c) dimensione della cache.

**Esercizio 8.6** Il tasso di miss di una memoria cache di tipo parzialmente associativa a due vie è sempre, di solito, a volte o mai migliore di quello di una cache a mappatura diretta con la stessa capacità e dimensione di blocco? Giustificare la risposta.

**Esercizio 8.7** Ciascuna delle seguenti affermazioni riguarda il tasso di miss delle memorie cache. Indicare se tali affermazioni sono vere o false, giustificando la propria risposta. Nel caso siano false, presentare un contro esempio.

- (a) Una cache parzialmente associativa a due vie ha un tasso di miss sempre minore di una cache a mappatura diretta con la stessa capacità e dimensione di blocco.
- (b) Una cache a mappatura diretta da 16 KB ha un tasso di miss sempre minore di quello di una cache a mappatura diretta da 8 KB con la stessa dimensione di blocco.
- (c) Una cache istruzioni con dimensione di blocco di 32 byte ha un tasso di miss generalmente minore di quello di una cache istruzioni con dimensione di blocco di 8 byte, nell'ipotesi di uguale grado di associatività e uguale capacità totale.

**Esercizio 8.8** Una cache è caratterizzata dai seguenti parametri:  $b$  (dimensione di blocco in numero di parole);  $S$  (numero di gruppi);  $N$  (numero di vie);  $A$  (numero di bit di indirizzo).

- (a) Qual è la capacità  $C$  della cache in funzione dei suddetti parametri?

- (b) Qual è il numero totale di bit necessari per memorizzare i tag in funzione dei suddetti parametri?
- (c) Quanto valgono  $S$  e  $N$  per una cache completamente associativa di capacità  $C$  parole con dimensione di blocco  $b$ ?
- (d) Quanto vale  $S$  per una cache a mappatura diretta di capacità  $C$  parole con dimensione di blocco  $b$ ?

**Esercizio 8.9** Una cache da 16 parole è caratterizzata dai parametri elencati nell'Esercizio 8.8. Si consideri la seguente sequenza ripetitiva di indirizzi esadecimali associati a istruzioni LDR:

40 44 48 4C 70 74 78 7C 80 84 88 8C 90 94 98 9C 0 4 8 C 10  
14 18 1C 20

Nell'ipotesi di politica LRU per le cache di tipo associativo, determinare i tassi di dato non trovato se tale sequenza viene applicata alle cache seguenti, trascurando gli effetti di avviamento (cioè i mancati ritrovamenti di dati dovuto alla cache ancora vuota):

- (a) cache a mappatura diretta,  $b = 1$  parola
- (b) cache completamente associativa,  $b = 1$  parola
- (c) cache parzialmente associativa a due vie,  $b = 1$  parola
- (d) cache a mappatura diretta,  $b = 2$  parole.

**Esercizio 8.10** Ripetere l'Esercizio 8.9 per la seguente sequenza ripetitiva, con le cache da 16 parole sotto indicate:

74 A0 78 38C AC 84 88 8C 7C 34 38 13C 388 18C

- (a) cache a mappatura diretta,  $b = 1$  parola
- (b) cache completamente associativa,  $b = 2$  parole
- (c) cache parzialmente associativa a due vie,  $b = 2$  parole
- (d) cache a mappatura diretta,  $b = 4$  parole.

**Esercizio 8.11** Si supponga di far eseguire un programma che presenta la seguente sequenza di accessi a memoria, eseguita una sola volta:

0x0 0x8 0x10 0x18 0x20 0x28

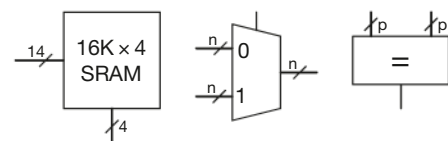
- (a) Se si usa una cache a mappatura diretta di 1 KB con dimensione di blocco di 8 byte (2 parole) quanti gruppi ci sono nella cache?
- (b) Con la cache di cui al punto (a), qual è il tasso di miss per la sequenza di accessi sopra riportata?
- (c) Per la sequenza di accessi sopra riportata e mantenendo costante la dimensione della cache, quale tra le seguenti modifiche diminuisce maggiormente il tasso di miss?
- (i) Aumentare a 2 il grado di associatività.
- (ii) Aumentare la dimensione di blocco a 16 byte.
- (iii) Sia (i) sia (ii).
- (iv) Né (i) né (ii).

**Esercizio 8.12** Si deve realizzare una cache istruzioni per il processore ARM. La cache ha una capacità totale di  $4C = 2^{c+2}$  byte, un'associatività di  $N = 2^n$  vie (con  $N \geq 8$ ) e una dimensione di blocco di  $b = 2^b$  byte (con  $b \geq 8$ ). Rispondere alle seguenti domande in funzione dei suddetti parametri.

- (a) Quali bit dell'indirizzo sono usati per selezionare una parola all'interno del blocco?
- (b) Quali bit dell'indirizzo sono usati per selezionare un set della cache?
- (c) Quanti bit ci sono in ogni tag?
- (d) Quanti bit di tag in totale ci sono nella cache?

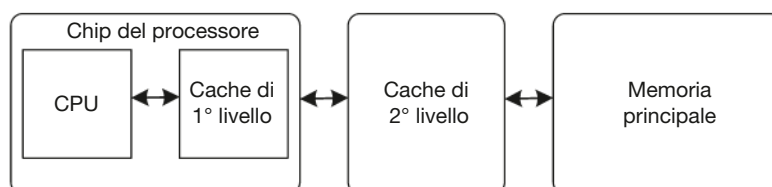
**Esercizio 8.13** Si consideri una cache con i seguenti parametri:  $N$  (associatività) = 2,  $b$  (dimensione di blocco) = 2 parole,  $W$  (dimensione di parola) = 32 bit,  $C$  (dimensione della cache) = 32 K parole,  $A$  (dimensione dell'indirizzo) = 32 bit. Limitandosi a considerare solo gli indirizzi di parola:

- (a) Indicare quali e quanti sono i bit di tag, di set, di spiazzamento di blocco e di spiazzamento di byte nell'indirizzo.
- (b) Qual è la dimensione in bit di tutti i tag della cache?
- (c) Si supponga che ogni blocco di cache abbia anche un bit di validità  $V$  e un bit di modifica  $M$ . Qual è la dimensione di ogni set della cache, inclusi dati, tag e bit di stato?
- (d) Progettare la cache utilizzando i blocchi costruttivi della Figura 8.28 e un numero limitato di porte logiche a due ingressi. Il progetto deve includere la memoria di tag e dati, la logica di confronto degli indirizzi, la selezione dei dati di uscita e ogni altra parte ritenuta rilevante. I blocchi multiplexer e comparatore possono avere dimensione qualsiasi (cioè con un qualsiasi numero di bit rispettivamente  $n$  e  $p$ ) ma i blocchi di SRAM devono essere tutti da  $16\text{ K} \times 4$  bit. Limitarsi a considerare le operazioni di lettura da memoria.



**Figura 8.28** Blocchi costruttivi.

**Esercizio 8.14** Un'intraprendente piccola azienda informatica vuole lanciare un nuovo orologio da polso dotato di cerapersona e di Web browser. L'orologio contiene un processore dotato dello schema di cache multilivello rappresentato nella Figura 8.29, con una piccola cache sul chip e una cache di secondo livello molto più grande all'esterno del chip (è vero, l'orologio risulta decisamente pesante, ma è una meraviglia come naviga su Web!). Il processore ha indirizzi di memoria a 32



**Figura 8.29** Struttura del calcolatore.

**Tabella 8.5** Caratteristiche delle memorie.

| Caratteristica       | Cache a bordo del chip                 | Cache esterna       |
|----------------------|----------------------------------------|---------------------|
| Organizzazione       | Parzialmente associativa a quattro vie | A mappatura diretta |
| Tasso di miss        | $A$                                    | $B$                 |
| Tempo di accesso     | $t_a$                                  | $t_b$               |
| Dimensioni di blocco | 16 byte                                | 16 byte             |
| Numero di blocchi    | 512                                    | 256K                |

bit, ma accede ai dati usando solo indirizzi di parola. Le cache hanno le caratteristiche elencate nella Tabella 8.5. La DRAM usata per la memoria principale ha un tempo di accesso  $t_m$  e una dimensione di 512 MB.

- Data una certa parola di memoria, in quante diverse posizioni può trovarsi nella cache a bordo del processore e nella cache di secondo livello?
- Qual è la dimensione in bit di ogni tag della cache a bordo del processore e della cache di secondo livello?
- Scrivere l'espressione del tempo medio di accesso a memoria in lettura, tenendo presente che si accede alle cache in sequenza.
- Le misure fatte in una certa condizione di funzionamento mostrano che la cache a bordo del processore ha un tasso di hit dell'85% e la cache di secondo livello un tasso di hit del 90%. Se però viene disabilitata la cache a bordo del processore, il tasso di hit della cache di secondo livello balza al 98.5%. Spiegarne il motivo.

**Esercizio 8.15** Nel capitolo si è parlato della politica di sostituzione LRU (*Least Recently Used*, utilizzato meno di recente) per le cache associative a più vie. Altre politiche, meno usate, sono la politica FIFO (*First-In-First-Out*, primo entrato primo a uscire) e la politica casuale. La politica FIFO elimina il blocco rimasto più a lungo nella cache, indipendentemente dal fatto che sia stato utilizzato di recente o meno. La politica casuale sceglie a caso il blocco da eliminare.

- Discutere vantaggi e svantaggi di ciascuna di queste politiche di sostituzione.
- Descrivere una sequenza di accesso ai dati per la quale la politica FIFO funziona meglio della politica LRU.

**Esercizio 8.16** Si deve costruire un calcolatore con gerarchia di memoria che prevede due cache separate per istruzioni e dati seguite dalla memoria principale, utilizzando il processore ARM multi ciclo della Figura 7.30 alla frequenza di 1 GHz.

- Si supponga che la cache istruzioni sia perfetta (cioè che trovi sempre la parola cercata) ma che la cache dati abbia un tasso di miss del 5%. Quando non trova il dato, il processore si arresta per 60 ns per accedere alla memoria principale, quindi riprende a lavorare. Qual è il tempo medio di accesso a memoria considerando le situazioni di dato non trovato?
- Quanti cicli di clock per istruzione (CPI) sono necessari in media per le istruzioni di lettura e scrittura di parole

di memoria considerando il suddetto sistema di memoria non ideale?

- Si faccia riferimento al benchmark dell'Esempio 7.5 che ha 25% di letture da memoria, 10% di scritture, 13% di salti e 52% di istruzioni di elaborazione dati. Considerando la struttura di memoria non ideale, qual è il CPI per questo benchmark?
- Si supponga ora che anche la cache istruzioni sia non ideale, e abbia un tasso di miss del 7%. Cosa diventa il CPI del punto (c) considerando le situazioni di dato non trovato in entrambe le cache?

**Esercizio 8.17** Ripetere l'Esercizio 8.16 con i parametri seguenti.

- La cache istruzioni è perfetta (cioè trova sempre la parola cercata) ma la cache dati ha un tasso di miss del 15%. Quando non trova il dato, il processore si arresta per 200 ns per accedere alla memoria principale, quindi riprende a lavorare. Qual è il tempo medio di accesso a memoria considerando le situazioni di dato non trovato?
- Quanti cicli di clock per istruzione (CPI) sono necessari in media per le istruzioni di lettura e scrittura di parole di memoria considerando il suddetto sistema di memoria non ideale?
- Si faccia riferimento al benchmark dell'Esempio 7.5 che ha 25% di letture da memoria, 10% di scritture, 13% di salti e 52% di istruzioni di elaborazione dati. Considerando la struttura di memoria non ideale, qual è il CPI per questo benchmark?
- Si supponga ora che anche la cache istruzioni sia non ideale, e abbia un tasso di miss del 10%. Cosa diventa il CPI del punto (c) considerando le situazioni di dato non trovato in entrambe le cache?

**Esercizio 8.18** Se un calcolatore usa indirizzi virtuali a 64 bit, a quanta memoria virtuale può accedere? Si ricordi che  $2^{40}$  byte sono 1 **terabyte**,  $2^{50}$  byte sono 1 **petabyte** e  $2^{60}$  byte sono 1 **exabyte**.

**Esercizio 8.19** Il progettista di un supercalcolatore decide di investire un milione di dollari per la DRAM e altrettanti per i dischi rigidi per realizzare la memoria virtuale. Usando i costi riportati nella Figura 8.4, quanta memoria fisica e quanta memoria virtuale avrà il supercalcolatore? Quanti bit di indirizzo fisico e quanti di indirizzo virtuale saranno necessari per accedere a memoria?

**Esercizio 8.20** Si consideri un sistema di memoria virtuale che può indirizzare in totale  $2^{32}$  byte. Lo spazio su disco è illimitato, ma la memoria a semiconduttori (quindi la memoria fisica) è limitata a 8 MB. Le pagine di memoria virtuale e fisica sono da 4 KB ciascuna.

- Quanti bit sono necessari per l'indirizzo fisico?
- Qual è il massimo numero di pagine virtuali nel sistema?
- Quante pagine fisiche sono presenti nel sistema?
- Quanti bit sono necessari per i numeri di pagina virtuale e fisica?
- Si supponga di usare uno schema di mappatura diretta per mappare le pagine virtuali in quelle fisiche: lo schema usa i bit meno significativi del numero di pagina virtuale per determinare il numero di pagina fisica. Quante pagine virtuali sono mappate in ogni pagina fisica? Perché la scelta di questa mappatura diretta è una scelta sbagliata?
- Serve ovviamente uno schema di mappatura più flessibile e dinamico di quello descritto al punto (e). Se si adotta una tabella delle pagine per memorizzare la mappatura (cioè la traduzione da numero di pagina virtuale a numero di pagina fisica) quanti elementi deve avere tale tabella?
- Si supponga che ogni elemento della tabella contenga, oltre al numero di pagina fisica, anche informazioni di stato che consistono di un bit di validità  $V$  e di un bit di modifica  $M$ . Quanti byte servono per ogni elemento della tabella?
- Tracciare lo schema della tabella delle pagine. Qual è in byte la sua dimensione totale?

**Esercizio 8.21** Si consideri un sistema di memoria virtuale che può indirizzare  $2^{50}$  byte. Lo spazio su disco è illimitato, ma la memoria a semiconduttori (quindi la memoria fisica) è limitata a 2 GB. Le pagine di memoria virtuale e fisica sono da 4 KB ciascuna.

- Quanti bit sono necessari per l'indirizzo fisico?
- Qual è il massimo numero di pagine virtuali nel sistema?
- Quante pagine fisiche sono presenti nel sistema?
- Quanti bit sono necessari per i numeri di pagina virtuale e fisica?
- Quanti sono gli elementi presenti nella tabella delle pagine?
- Si supponga che ogni elemento della tabella contenga, oltre al numero di pagina fisica, anche informazioni di stato che consistono di un bit di validità  $V$  e di un bit di modifica  $M$ . Quanti byte servono per ogni elemento della tabella?
- Tracciare lo schema della tabella delle pagine. Qual è in byte la sua dimensione totale?

**Esercizio 8.22** Si vuole velocizzare il sistema di memoria virtuale dell'Esercizio 8.20 usando un TLB (*Translation Lookaside Buffer*). Il sistema di memoria ha le caratteristiche riportate nella Tabella 8.6. I tassi di miss del TLB e della cache indicano quanto spesso l'elemento cercato non viene trovato. Il tasso di miss della memoria principale indica quanto spesso si verifica un errore di pagina.

- Qual è il tempo medio di accesso a memoria del sistema di memoria virtuale prima e dopo l'introduzione del TLB? Si supponga che la tabella delle pagine sia sempre residente in memoria fisica e mai nella cache dati.
- Se il TLB ha 64 elementi, quanto è grande in bit? Quanti bit servono per il dato (numero di pagina fisica) il tag (numero di pagina virtuale) e i bit di validità di ogni elemento del TLB?
- Tracciare lo schema del TLB, mostrando chiaramente i vari campi e le relative dimensioni.
- Quali sono le dimensioni (numero di bit per parola e numero di parole) della SRAM necessaria per realizzare il TLB descritto al punto (c)?

**Esercizio 8.23** Si vuole velocizzare il sistema di memoria virtuale dell'Esercizio 8.21 usando un TLB (*Translation Lookaside Buffer*) di 128 elementi.

- Quanto è grande in bit il TLB? Quanti bit servono per il dato (numero di pagina fisica), il tag (numero di pagina virtuale) e i bit di validità di ogni elemento del TLB?
- Tracciare lo schema del TLB, mostrando chiaramente i vari campi e le relative dimensioni.
- Quali sono le dimensioni (numero di bit per parola e numero di parole) della SRAM necessaria per realizzare il TLB descritto al punto (b)?

**Esercizio 8.24** Si supponga che il processore ARM multi ciclo descritto nel paragrafo 7.4 usi un sistema di memoria virtuale.

- Aggiungere il TLB nello schema del processore multi ciclo.
- Descrivere come variano le prestazioni del processore a seguito dell'introduzione del TLB.

**Esercizio 8.25** Si deve progettare un sistema di memoria virtuale che usa una tabella delle pagine a livello singolo realizzata con hardware dedicato (SRAM e logica relativa). La tabella gestisce indirizzi virtuali a 25 bit, indirizzi fisici a 22 bit e pagine da  $2^{16}$  byte (ovvero da 64 KB). Ogni elemento della tabella contiene un numero di pagina virtuale, un bit di validità  $V$  e un bit di modifica  $M$ .

**Tabella 8.6** Caratteristiche delle memorie.

| Unità di memoria   | Tempo di accesso (cicli) | Tasso di miss |
|--------------------|--------------------------|---------------|
| TLB                | 1                        | 0.05%         |
| Cache              | 1                        | 2%            |
| Memoria principale | 100                      | 0.0003%       |
| Disco rigido       | 1 000 000                | 0%            |

- (a) Qual è la dimensione totale della tabella delle pagine, in bit?
- (b) Gli sviluppatori del sistema operativo propongono di ridurre la dimensione di pagina da 64 KB a 16 KB, ma i progettisti hardware obiettano citando il costo aggiuntivo a livello circuitale. Spiegare il motivo di questa obiezione.
- (c) La tabella delle pagine deve essere integrata nel chip del processore, insieme alla cache a bordo del processore. Tale cache lavora solo su indirizzi fisici (non virtuali). È possibile accedere in modo concorrente al set opportuno della cache a bordo del processore e alla tabella delle pagine per un determinato indirizzo di memoria? Spiegare la relazione necessaria per accessi concorrenti al set della cache e all'elemento della tabella delle pagine.

**Esercizio 8.26** Descrivere uno scenario nel quale il sistema di memoria virtuale potrebbe influenzare come un programma applicativo viene scritto, precisando come la dimensione di pagina e la dimensione di memoria fisica possono influire sulle prestazioni dell'applicazione.

**Esercizio 8.27** Si supponga di avere un PC che usa indirizzi virtuali a 32 bit.

- (a) Qual è la massima quantità di memoria virtuale che ciascun programma può usare?
- (b) Come influisce sulle prestazioni la dimensione del disco rigido del PC?
- (c) Come influisce sulle prestazioni la quantità di memoria fisica presente nel PC?

## Domande di valutazione

Queste domande sono state poste a candidati per un posto di lavoro.

**Domanda 8.1** Ci sa spiegare qual è la differenza tra cache a mappatura diretta, cache parzialmente associativa e cache completamente associativa? Per ogni tipo di cache, ci sa indicare un'applicazione per la quale tale tipo di cache si comporta meglio delle altre?

**Domanda 8.2** Ci sa spiegare come funziona un sistema di memoria virtuale?

**Domanda 8.3** Ci sa indicare vantaggi e svantaggi nell'uso di un sistema di memoria virtuale?

**Domanda 8.4** Ci sa spiegare in che modo le prestazioni della cache possono essere influenzate dalla dimensione delle pagine virtuali?



# Realizzazione dei sistemi digitali

# A

**A.1** Introduzione

**A.2** La logica 74xx

**A.3** Logica programmabile

**A.4** Circuiti integrati specifici per un'applicazione

**A.5** Data sheet

**A.6** Famiglie logiche

**A.7** Packaging e assemblaggio

**A.8** Linee di trasmissione

**A.9** Aspetti economici

## A.1 ■ INTRODUZIONE

Questa appendice introduce alcuni aspetti pratici relativi al progetto dei sistemi digitali. Il materiale non è necessario per comprendere il resto del libro: si pone piuttosto l'obiettivo di demistificare il processo di costruzione di un sistema digitale. Comunque, il modo migliore per comprendere i sistemi digitali è quello di costruirne e collaudarne uno in laboratorio.

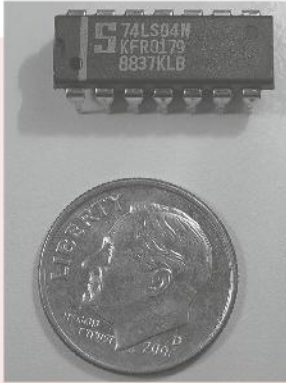
I sistemi digitali sono costituiti con uno o più circuiti integrati (chip). Un metodo è quello di interconnettere chip contenenti singole porte logiche con elementi di dimensioni maggiori come ALU e memorie. Un altro metodo è quello di ricorrere alla logica programmabile, che contiene matrici generiche di elementi circuitali che possono essere programmati per eseguire una specifica funzione logica. Un terzo metodo è quello di progettare un circuito integrato *custom* (su misura) contenente la logica necessaria per il sistema che si vuole costruire. I tre metodi presentano diversi rapporti tra costo, velocità, consumo di potenza e tempo di sviluppo, esaminati nei paragrafi che seguono. Questa appendice analizza anche i contenitori fisici (*package*) e le modalità di assemblaggio dei circuiti, le linee di trasmissione che collegano i chip e gli aspetti economici legati ai sistemi digitali.

## A.2 ■ LA LOGICA 74xx

Negli anni '70 e '80, molti sistemi digitali erano realizzati utilizzando semplici chip, ciascuno contenente una manciata di porte logiche. Per esempio, il chip 7404 contiene sei porte NOT, il 7408 quattro porte AND e il 7474 due flip-flop. Questi chip erano genericamente chiamati **serie logica 74xx**. Erano venduti da varie case produttrici, con prezzi dai 10 ai 25 centesimi di dollaro per chip. Oggi questi chip sono largamente superati, ma sono ancora utili per realizzare semplici sistemi digitali, o esercitazioni scolastiche, per il loro bassissimo costo e per la facilità d'uso. Vengono solitamente venduti in package DIP a 14 piedini (*Dual Inline Package*, con due file di piedini o *pin*).

|                      |  |
|----------------------|--|
| Software applicativo |  |
| Sistema operativo    |  |
| Architettura         |  |
| Micro-architettura   |  |
| Blocchi logici       |  |
| Reti digitali        |  |
| Reti analogiche      |  |
| Dispositivi          |  |
| Fisica               |  |





Il chip di negatori 74LS04 in un package dual inline a 14 piedini. Il numero del componente (*part number*) è indicato sulla prima riga. La sigla LS indica la famiglia logica (vedi il par. A.6). Il suffisso N indica un package DIP. La grossa S è il logo del produttore: Signetics. Le altre due righe di cifre incomprensibili sono i codici di identificazione della partita nella quale il chip è stato fabbricato.

### A.2.1 Porte logiche

La **Figura A.1** mostra i diagrammi della disposizione dei piedini (*pinout*) per vari chip della serie 74xx contenenti le porte logiche di base. Si parla in questo caso di chip SSI (*Small Scale Integration*, piccola scala di integrazione) perché ciascuno contiene solo pochi transistori. I package a 14 piedini hanno un incavo in alto o un buchino in alto a sinistra per indicare l'orientamento: i piedini sono numerati da 1 in alto a sinistra e in senso antiorario. Devono essere collegati all'alimentazione ( $V_{DD} = 5\text{ V}$ ) e alla massa con i piedini rispettivamente 14 e 7. Il numero di porte logiche nel chip è determinato dal numero di piedini. Si noti che i piedini 3 e 11 del chip 7421 non sono collegati a nulla (NC, *Not Connected*). Il flip-flop 7474 ha i soliti contatti *D*, *CLK* e *Q*, oltre all'uscita negata  $\overline{Q}$ . Inoltre ha i due ingressi asincroni per forzare a 1 (*PRE*, *preset*) e a 0 (*CLR*, *clear* o *reset*) l'uscita. Questi due segnali sono attivi bassi: in altre parole, l'uscita del flip-flop viene forzata a 1 quando  $\overline{PRE} = 0$  e forzata a 0 quando  $\overline{CLR} = 0$ ; il flip-flop lavora normalmente quando  $\overline{PRE} = \overline{CLR} = 1$ .

### A.2.2 Altre funzioni

La serie 74xx include anche alcune funzioni logiche più complesse come quelle mostrate nelle **Figure A.2** ed **A.3**. Si parla in questo caso di chip MSI (*Medium Scale Integration*, media scala di integrazione). Molti di questi chip usano package più grandi per avere più piedini di ingresso e uscita. Alimentazione e massa sono ancora fornite rispettivamente al piedino in alto a destra e a quello in basso a sinistra di ogni chip. Per ogni chip viene qui fornita una descrizione generale della funzione svolta. Fare riferimento alla documentazione tecnica (*data sheet*) fornita dal costruttore per una descrizione completa.

## A.3 ■ LOGICA PROGRAMMABILE

La **logica programmabile** è costituita da matrici di elementi circuitali che possono essere configurate per svolgere particolari funzioni logiche. Si sono già introdotte in questo libro tre tipologie di logica programmabile: le PROM (*Programmable Read Only Memory*, memoria a sola lettura programmabile), le PLA (*Programmable Logic Array*, matrice logica programmabile) e le FPGA (*Field Programmable Gate Array*, matrice di porte logiche programmabile sul campo). In questo paragrafo si vede come sono realizzati i chip di queste tre tipologie. La configurazione di questi chip può essere effettuata bruciando o meno i fusibili presenti nei chip per disconnettere o lasciare connessi gli elementi circuitali: si parla in questo caso di logica OTP (*One-Time Programmable*) perché un fusibile una volta bruciato non può più essere ripristinato. L'alternativa è memorizzare la configurazione in una qualche forma di memoria, che possa essere modificata all'esigenza. La logica riprogrammabile è molto comoda in laboratorio perché lo stesso chip può essere riutilizzato durante lo sviluppo del sistema.

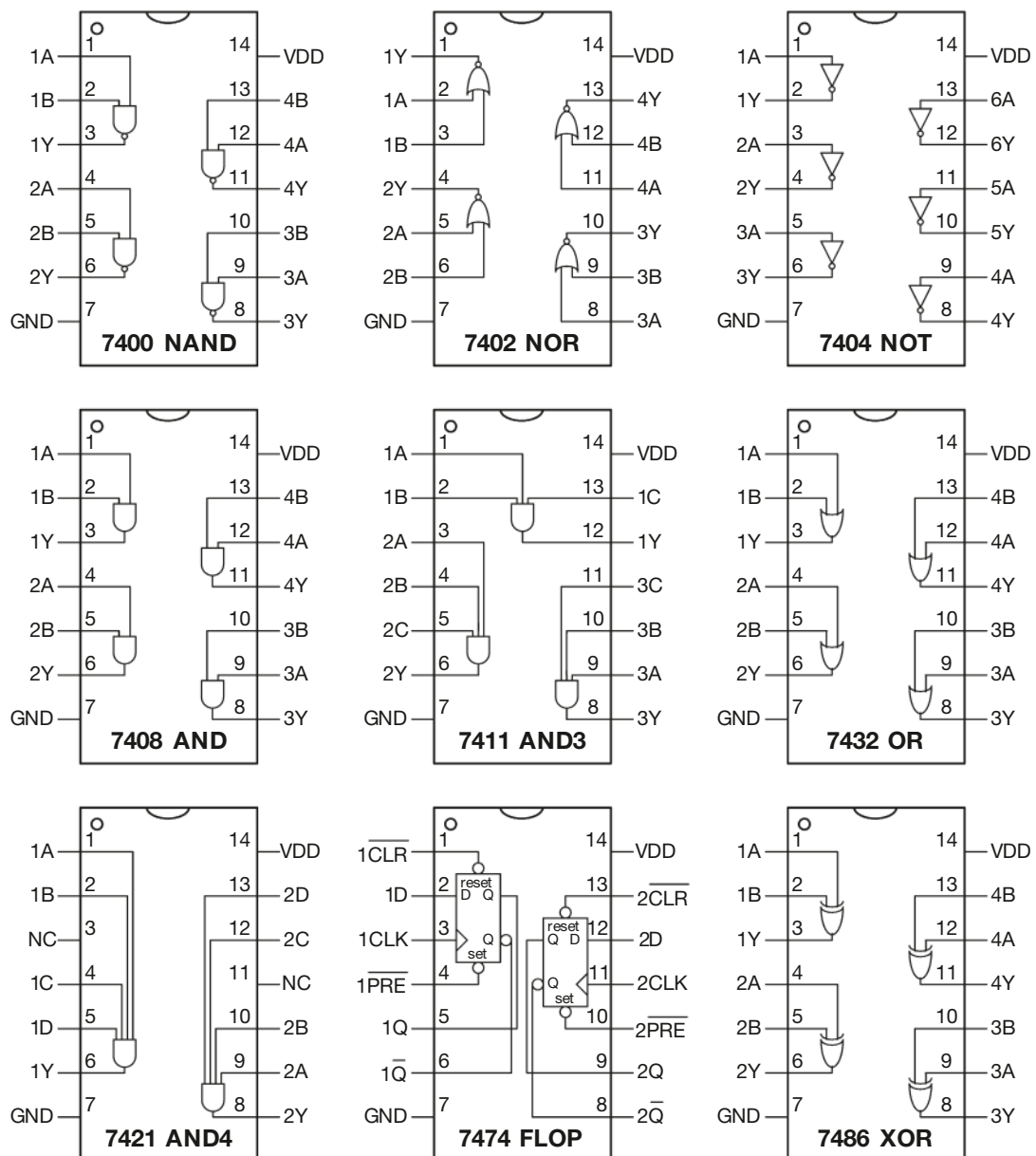
### A.3.1 PROM

Come discusso nel paragrafo 5.5.7, i componenti PROM possono essere utilizzati come *lookup table*. Una PROM da  $2^N$  parole di  $M$  bit può essere programmata per realizzare una qualsiasi funzione combinatoria con  $N$  ingressi e  $M$  uscite.

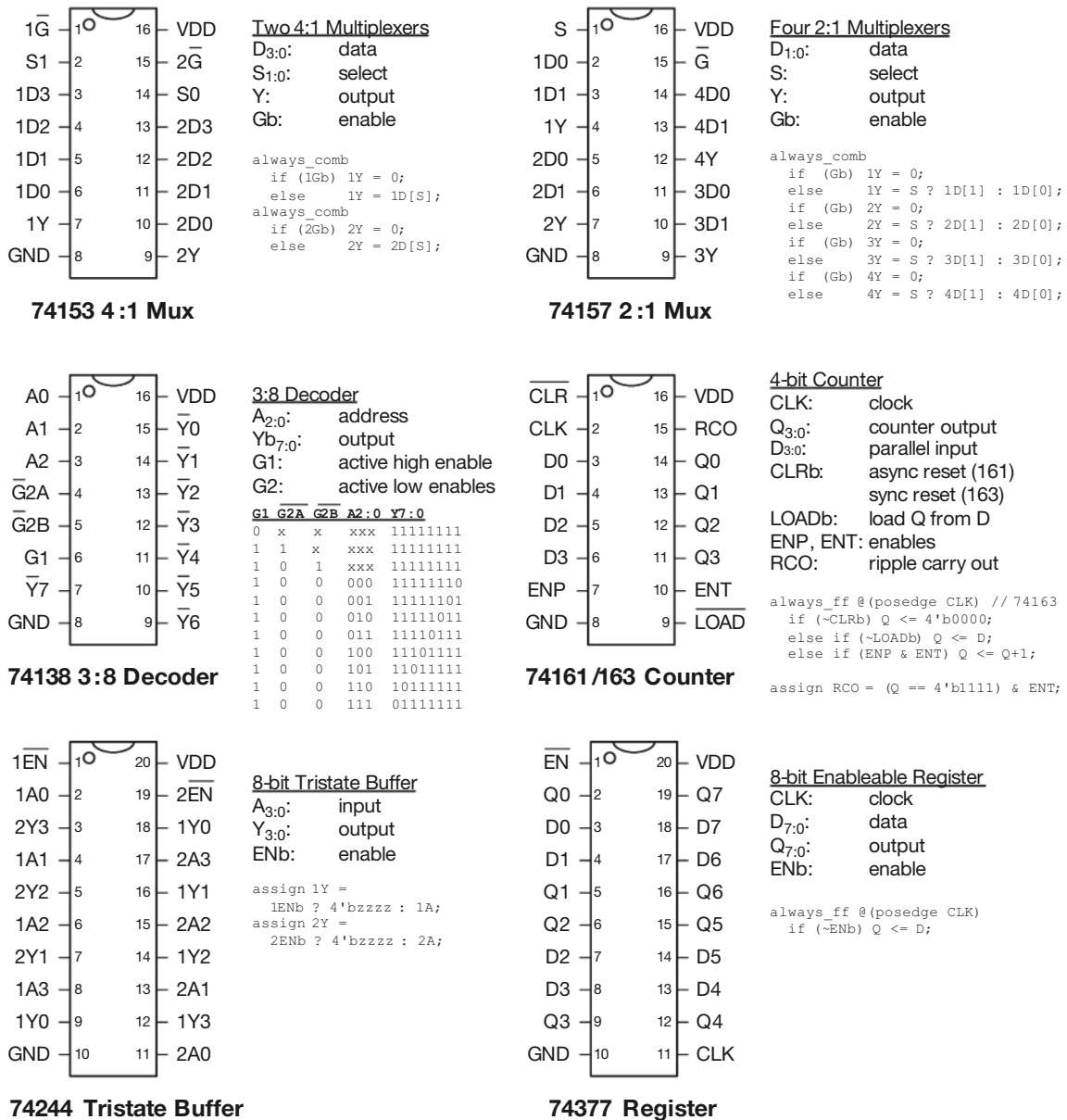
Modifiche al progetto implicano semplicemente di modificare il contenuto della PROM invece di dover rifare i collegamenti a filo tra i vari chip. Le *lookup table* sono molto comode per piccole funzioni logiche ma diventano rapidamente proibitive al crescere del numero degli ingressi.

Per esempio, la **Figura A.4** mostra la classica EPROM (*Erasable PROM*) 2764 da 8 KB (64 Kb). La EPROM ha 13 linee di indirizzo per selezionare una delle 8 K parole e 8 linee di dato per leggere il byte contenuto nella parola indirizzata. I segnali di abilitazione del chip (*chip enable*) e di abilitazione dell'uscita (*output enable*) devono essere attivati entrambi per poter leggere un dato. Il massimo ritardo di propagazione è di 200 ps. Nel normale funzionamento,  $\overline{PGM} = 1$  e  $V_{PP}$  non è usato. La EPROM è programmata in genere mediante un particolare programmatore che porta  $\overline{PGM} = 0$ , applica 13 V a  $V_{PP}$  e usa una particolare sequenza di valori di ingresso per configurare la memoria.

Le PROM moderne sono simili da un punto di vista concettuale ma han-

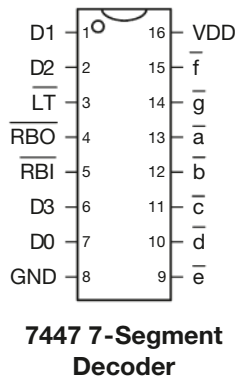


**Figura A.1** Comuni porte logiche della serie 74xx.



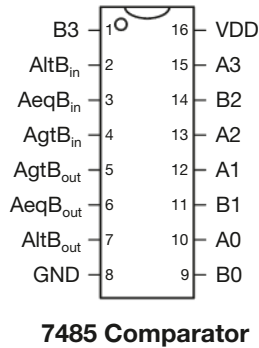
**Figura A.2** Chip a media scala di integrazione.

no capacità molto maggiori e di conseguenza più piedini. La memoria flash è il tipo più economico di PROM, poiché costa circa 0.30 dollari al Gigabyte (prezzi del 2015). La tendenza dei prezzi è quella di diminuire del 30-40% ogni anno.



**7-segment Display Decoder**

|    | RBO | LT | RBI | D3:0 | a | b | c | d | e | f | g |
|----|-----|----|-----|------|---|---|---|---|---|---|---|
| 0  | x   | x  | x   | 1    | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 1  | 0   | x  | x   | 0    | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 2  | 1   | 1  | 1   | 0000 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 3  | 1   | 1  | 1   | 0001 | 1 | 0 | 0 | 1 | 1 | 1 | 1 |
| 4  | 1   | 1  | 1   | 0010 | 0 | 0 | 1 | 0 | 0 | 1 | 0 |
| 5  | 1   | 1  | 1   | 0011 | 0 | 0 | 0 | 0 | 1 | 1 | 0 |
| 6  | 1   | 1  | 1   | 0100 | 1 | 0 | 0 | 1 | 1 | 0 | 0 |
| 7  | 1   | 1  | 1   | 0101 | 0 | 1 | 0 | 0 | 1 | 0 | 0 |
| 8  | 1   | 1  | 1   | 0110 | 1 | 1 | 0 | 0 | 0 | 0 | 0 |
| 9  | 1   | 1  | 1   | 0111 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 10 | 1   | 1  | 1   | 1000 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 11 | 1   | 1  | 1   | 1001 | 0 | 0 | 0 | 1 | 1 | 0 | 0 |
| 12 | 1   | 1  | 1   | 1010 | 1 | 1 | 0 | 0 | 1 | 0 | 0 |
| 13 | 1   | 1  | 1   | 1011 | 1 | 1 | 0 | 0 | 1 | 1 | 0 |
| 14 | 1   | 1  | 1   | 1100 | 1 | 0 | 1 | 1 | 1 | 0 | 0 |
| 15 | 1   | 1  | 1   | 1101 | 0 | 1 | 1 | 0 | 1 | 0 | 0 |
| 16 | 1   | 1  | 1   | 1110 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 17 | 1   | 1  | 1   | 1111 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

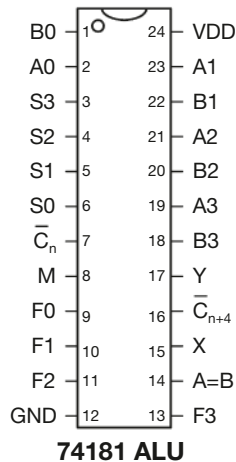


**4-bit Comparator**  
 A<sub>3:0</sub>, B<sub>3:0</sub>: data  
 rel<sub>in</sub>: input relation  
 rel<sub>out</sub>: output relation

```

always_comb
 if (A > B | (A == B & AgtBin)) begin
 AgtBout = 1; AeqBout = 0; AltBout = 0;
 end
 else if (A < B | (A == B & AltBin)) begin
 AgtBout = 0; AeqBout = 0; AltBout = 1;
 end else begin
 AgtBout = 0; AeqBout = 1; AltBout = 0;
 end
 end

```



**4-bit ALU**  
 A<sub>3:0</sub>, B<sub>3:0</sub>: inputs  
 Y<sub>3:0</sub>: output  
 F<sub>3:0</sub>: function select  
 M: mode select  
 Cb<sub>n</sub>: carry in  
 Cb<sub>nplus4</sub>: carry out  
 AeqB: equality  
 (in some modes)  
 X, Y: carry lookahead  
 adder outputs

```

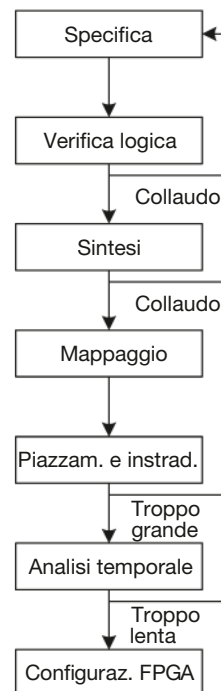
always_comb
 case (F)
 0000: Y = M ? ~A : A + ~Cb_n;
 0001: Y = M ? ~(A | B) : A + B + ~Cb_n;
 0010: Y = M ? (~A) & B : A + ~B + ~Cb_n;
 0011: Y = M ? 4'b0000 : 4'b1111 + ~Cb_n;
 0100: Y = M ? ~(A & B) : A + (A & ~B) + ~Cb_n;
 0101: Y = M ? ~B : (A | B) + (A & ~B) + ~Cb_n;
 0110: Y = M ? A ^ B : A - B - Cb_n;
 0111: Y = M ? A & ~B : (A & ~B) - Cb_n;
 1000: Y = M ? ~A + B : A + (A & B) + ~Cb_n;
 1001: Y = M ? ~(A ^ B) : A + B + ~Cb_n;
 1010: Y = M ? B : (A | ~B) + (A & B) + ~Cb_n;
 1011: Y = M ? A & B : (A & B) + ~Cb_n;
 1100: Y = M ? 1 : A + A + ~Cb_n;
 1101: Y = M ? A | ~B : (A | B) + A + ~Cb_n;
 1110: Y = M ? A | B : (A | ~B) + A + ~Cb_n;
 1111: Y = M ? A : A - Cb_n;
 endcase

```

**Figura A.3** Altri chip a media scala di integrazione.



FPGA consentono l'uso di schemi circuitali. Il progetto viene poi **simulato** su calcolatore: si applicano valori di ingresso e si confrontano le uscite con i valori attesi per verificare la correttezza della logica. Successivamente si passa alla **sintesi**, che traduce la descrizione HDL in funzioni booleane; gli strumenti di sintesi migliori producono anche uno schema circuitale, molto utile al progettista per un'ulteriore verifica di correttezza della logica prodotta, tenendo in considerazione tutte le segnalazioni (*warning*) comunicate dal sintetizzatore e ricordando che a volte un codice poco accurato produce circuiti molto più grandi del dovuto o addirittura circuiti asincroni. Quando il prodotto della sintesi è buono, lo strumento di lavoro con le FPGA effettua il **mappaggio** delle funzioni logiche sugli LE del chip utilizzato. Questo strumento, detto di piazzamento e instradamento (*place and route*), definisce quali funzioni devono essere realizzate da quali lookup table e come vengono interconnesse. I ritardi introdotti dalle connessioni aumentano con la lunghezza, quindi i circuiti critici devono essere posizionati uno vicino all'altro. Se il progetto è troppo grande per essere realizzato con un solo chip, si deve procedere a una reingegnerizzazione. L'**analisi temporale** verifica i vincoli temporali (per es., la scelta di una frequenza di clock di 100 MHz) rispetto agli effettivi ritardi nel circuito, e segnala eventuali errori: se la logica è troppo lenta, si deve procedere a riprogettazione o modifiche della struttura pipeline. Quando tutto è corretto, viene generato un file che specifica il contenuto di tutti gli LE e la programmazione di tutte le connessioni della FPGA. Molte FPGA memorizzano queste informazioni di **configurazione** in memoria RAM statica al loro interno, memoria che va ricaricata a ogni accensione della FPGA. La configurazione può essere scaricata da PC in laboratorio, oppure da una memoria ROM non volatile.



**Figura A.5**  
Processo di progettazione di una FPGA.

### ESEMPIO A.1

**Analisi della temporizzazione di una FPGA.** Alyssa Guastacomputer sta usando una FPGA per realizzare una macchina smistatrice di confetti M&M, con un sensore di colore e dei motori per mandare i confetti rossi in un vaso e quelli verdi in un altro. Progetta la macchina come FSM e vuole utilizzare una FPGA Cyclone IV. In base al data sheet, la FPGA ha le caratteristiche temporali riportate nella **Tabella A.1**.

Alyssa vuole far andare la sua FSM a 100 MHz. Qual è il massimo numero di LE sul percorso critico? Qual è la massima velocità alla quale la FPGA può lavorare?

**Soluzione** A 100 MHz, il tempo di ciclo del clock,  $T_c$ , è di 10 ns. Alyssa usa l'espressione 3.14 per calcolare il massimo ritardo di propagazione attraverso la logica combinatoria,  $t_{pd}$ , con questo tempo di ciclo:

$$t_{pd} \leq 10 \text{ ns} - (0.199 \text{ ns} + 0.076 \text{ ns}) = 9.725 \text{ ns} \quad (\text{A.1})$$

Con un ritardo combinato di LE e collegamento pari a  $381 \text{ ps} + 246 \text{ ps} = 627 \text{ ps}$ , la FSM di Alyssa può usare al massimo 15 LE consecutivi ( $9.725/0.627$ ) per realizzare la logica di stato prossimo.

La massima velocità alla quale una FSM può lavorare su questa FPGA Cyclone IV si ha quando si usa un solo LE per la logica di stato prossimo. Il minimo tempo di ciclo risulta essere:

$$T_c \geq 381 \text{ ps} + 199 \text{ ps} + 76 \text{ ps} = 656 \text{ ps} \quad (\text{A.2})$$

Quindi la massima frequenza è pari a 1.5 GHz.

**Tabella A.1**    Temporizzazioni di Cyclone IV.

| Nome                       | Valore (ps) |
|----------------------------|-------------|
| $t_{pcq}$                  | 199         |
| $t_{setup}$                | 76          |
| $t_{hold}$                 | 0           |
| $t_{pd}$ (per LE)          | 381         |
| $t_{connessione}$ (tra LE) | 246         |
| $t_{skew}$                 | 0           |

Altera pubblicizza Cyclone IV con 14000 LE a 25 dollari (prezzi del 2015). Acquistate in grosse quantità, le FPGA di media dimensione costano alcuni dollari. Le FPGA più grandi possono costare centinaia o anche migliaia di dollari. I prezzi però diminuiscono circa del 30% ogni anno, quindi le FPGA si stanno diffondendo sempre più.

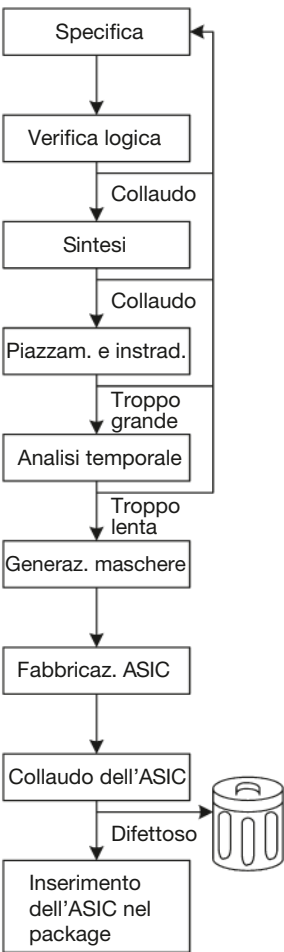
**A.4 ■ CIRCUITI INTEGRATI SPECIFICI PER UN’APPLICAZIONE**

I circuiti integrati specifici per un’applicazione (ASIC, *Application-Specific Integrated Circuit*) sono chip progettati apposta per un particolare scopo. Acceleratori grafici, interfacce di rete, chip per telefoni cellulari sono tipici esempi di ASIC. Il progettista di ASIC disegna transistori per realizzare le porte logiche e connette le porte tra di loro. Dal momento che è costruito a livello hardware per svolgere una particolare funzione, un ASIC è diverse volte più veloce di una FPGA e occupa un’area di silicio (quindi ha un costo) un ordine di grandezza inferiore rispetto alla FPGA. Però le **maschere** necessarie per posizionare transistori e interconnessioni sul chip costano da centinaia a migliaia di dollari, e il processo di produzione richiede dalle 6 alle 12 settimane per realizzare i chip ASIC, inserirli nei package e collaudarli. Se si scoprono errori nei chip finiti, si deve correggere il problema, generare nuove maschere e attendere l’arrivo di un’altra infornata di chip. Quindi gli ASIC sono adatti solo a prodotti da realizzare in grandi quantità e le cui funzioni sono ben definite a priori.

La **Figura A.6** mostra il processo di progettazione di un ASIC, simile a quello di una FPGA della Figura A.5. La verifica logica è particolarmente importante perché la correzione di errori dopo che le maschere sono state prodotte è molto costosa. La sintesi produce la cosiddetta *netlist*, ovvero un elenco di porte logiche e di interconnessioni: le porte logiche vengono posizionate nel chip e le interconnessioni instradate. Quando il progetto è soddisfacente, si generano le maschere e le si usano per la fabbricazione degli ASIC. Anche una singola particella di polvere può rovinare un ASIC, quindi i chip devono essere collaudati dopo la fabbricazione. La frazione di chip costruiti che funziona viene chiamata **resa** (*yield*) e varia generalmente tra il 50% e il 90% a seconda della complessità del chip e della maturità del processo produttivo. Al termine i chip funzionanti vengono inscatolati nei package, come discusso nel paragrafo A.7.

**A.5 ■ DATA SHEET**

I produttori di circuiti integrati pubblicano documentazione tecnica sotto forma di **data sheet**, letteralmente “fogli di dati”, che descrivono funzioni e prestazioni dei chip. È molto importante leggere e comprendere i data sheet, perché una delle principali cause di errore nella progettazione dei sistemi di-



**Figura A.6**  
Processo di progettazione di un ASIC.



gitali deriva proprio dalla mancata comprensione delle operazioni di un chip.

I data sheet sono normalmente disponibili sul sito Web del produttore: se non si trova il data sheet di un certo chip o non si ha a disposizione una fonte alternativa di informazioni, meglio non usarlo. Alcune delle voci nel data sheet possono risultare alquanto criptiche. Spesso i produttori pubblicano volumi di dati contenenti i data sheet di molti chip: in tal caso il volume contiene all'inizio delle spiegazioni ulteriori, pure reperibili su Web anche se magari con una ricerca un po' faticosa.

In questo paragrafo si analizza in dettaglio il data sheet della Texas Instruments (TI) per il chip di negatori 74HC04. È un data sheet abbastanza semplice ma che contiene molti degli elementi principali. TI produce ancora un'ampia gamma di chip della serie 74xx. Nel passato anche molte altre aziende producevano questi chip, ma quando le vendite calano il mercato tende a concentrare la produzione.

La **Figura A.7** mostra la prima pagina del data sheet, con le parti chiave evidenziate in rosso. Il titolo è SN54HC04, SN74HC04 HEX INVERTERS perché il chip contiene sei porte NOT o invertitori (*inverter*). SN indica che il produttore è TI (altri produttori hanno sigle diverse, come MC per Motorola e DM per National Semiconductors): in genere si possono ignorare queste sigle perché tutti i produttori forniscono chip della serie 74xx compatibili tra loro. HC è la sigla della famiglia logica (*High speed CMOS*). La famiglia logica determina la velocità e il consumo di potenza del chip, non la sua funzione. Per esempio, i chip 7404, 74HC04 e 74LS04 contengono tutti sei porte NOT ma con diverse prestazioni e diversi costi. Altre famiglie logiche sono discusse nel paragrafo A.6. I chip 74xx sono in grado di funzionare negli intervalli di temperatura commerciale (da 0 a 70 °C) o industriale (da -40 a 85 °C), mentre i chip 54xx funzionano nell'intervallo di temperatura militare (da -55 a 125 °C), costano di più ma per il resto sono completamente compatibili con i 74xx.

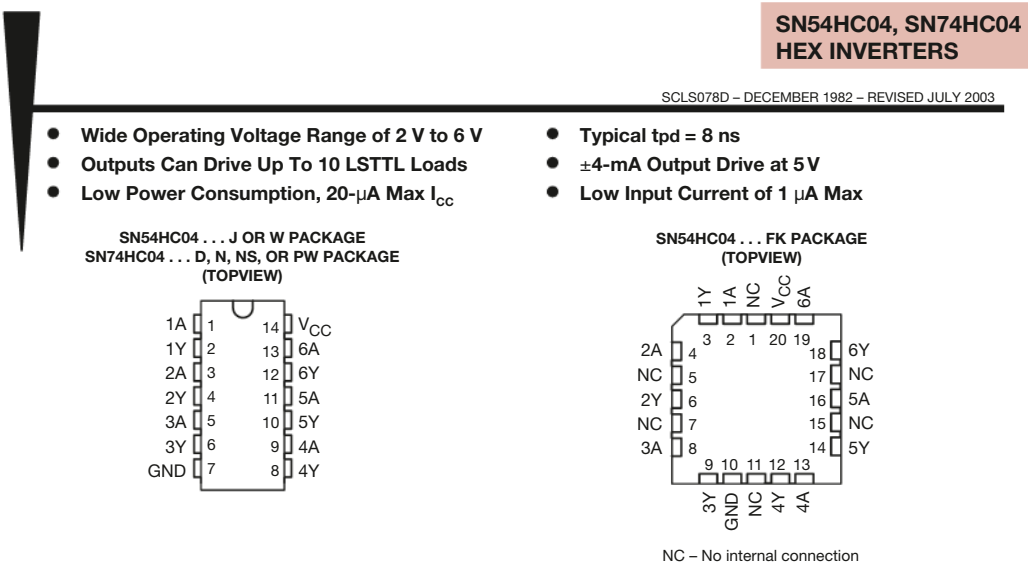
Il chip 7404 è disponibile in vari package, e naturalmente bisogna fare attenzione a ordinare il package adatto ai propri scopi. I package sono indicati da un suffisso nel *part number*: il suffisso N indica un *plastic dual inline package* (PDIP), che va bene per le schede prototipali oppure può essere saldato ai buchi di una scheda di circuito stampato. Altri tipi di package sono discussi nel paragrafo A.7.

La *function table* (tabella delle funzioni) mostra che ogni porta logica inverte il proprio ingresso: se A è H (*High*, alto) Y è L (*Low*, basso) e viceversa. La tabella è ovvia in questo caso, ma molto più interessante per chip più complessi.

La **Figura A.8** mostra la seconda pagina del data sheet. Il *logic diagram* (schema logico) indica che il chip contiene sei negatori. La sezione *absolute maximum* (massimi assoluti) indica le condizioni oltre le quali il chip può essere distrutto. In particolare, la tensione di alimentazione ( $V_{CC}$ , detta anche  $V_{DD}$  in questo libro) non può superare i 7 V. La corrente di uscita continua non può superare i 25 mA. La *thermal resistance* o *impedance* (resistenza o impedenza termica)  $\theta_{JA}$  è usata per calcolare l'incremento di temperatura dovuto alla dissipazione di potenza da parte del chip. Se la temperatura ambiente dove il chip sta funzionando è  $T_A$  e il chip dissipa una potenza  $P_{chip}$ , la temperatura del chip stesso al punto di *junction* (giunzione) con il package è data da:

$$T_J = T_A + P_{chip} \theta_{JA} \quad (\text{A.3})$$

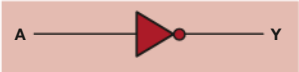
Per esempio, se il chip 7404 in un package DIP plastico sta funzionando in una scatola calda a 50 °C e consuma 20 mW, la temperatura di giunzione sale a 50 °C + 0.02 W × 80 °C/W = 51.6 °C. La dissipazione di potenza per i chip



SN54HC04, SN74HC04  
HEX INVERTERS

SCLS078D – DECEMBER 1982 – REVISED JULY 2003

logic diagram (positive logic)



absolute maximum ratings over operating free-air temperature range (unless otherwise noted)†

|                                                                          |                  |
|--------------------------------------------------------------------------|------------------|
| Supply voltage range, $V_{CC}$                                           | −0.5 V to 7 V    |
| Input clamp current, $I_K$ ( $V_I < 0$ or $V_I > V_{CC}$ ) (see Note 1)  | ±20 mA           |
| Output clamp current, $I_K$ ( $V_O < 0$ or $V_O > V_{CC}$ ) (see Note 1) | ±20 mA           |
| Continuous output current, $I_O$ ( $V_O = 0$ to $V_{CC}$ )               | ±25 mA           |
| Continuous current through $V_{CC}$ or GND                               | ±50 mA           |
| Package thermal impedance, $\theta_{JA}$ (see Note 2): D package         | 86° C/W          |
| N package                                                                | 80° C/W          |
| NS package                                                               | 76° C/W          |
| PW package                                                               | 131° C/W         |
| Storage temperature range, $T_{stg}$                                     | −65° C to 150° C |

† Stresses beyond those listed under “absolute maximum ratings” may cause permanent damage to the device. These are stress ratings only, and functional operation of the device at these or any other conditions beyond those indicated under “recommended operating conditions” is not implied. Exposure to absolute-maximum-rated conditions for extended periods may affect device reliability.

NOTES: 1. The input and output voltage ratings may be exceeded if the input and output current ratings are observed.  
2. The package thermal impedance is calculated in accordance with JEDEC 51-7.

recommended operating conditions (see Note 3)

|                 |                                 |                         | SN54HC04 |                 |     | SN74HC04 |                 |     | UNIT |
|-----------------|---------------------------------|-------------------------|----------|-----------------|-----|----------|-----------------|-----|------|
|                 |                                 |                         | MIN      | NOM             | MAX | MIN      | NOM             | MAX |      |
| V <sub>CC</sub> | Supply voltage                  |                         | 2        | 5               | 6   | 2        | 5               | 6   | V    |
| V <sub>IH</sub> | High-level input voltage        | V <sub>CC</sub> = 2 V   | 1.5      |                 |     | 1.5      |                 |     | V    |
|                 |                                 | V <sub>CC</sub> = 4.5 V | 3.15     |                 |     | 3.15     |                 |     |      |
|                 |                                 | V <sub>CC</sub> = 6 V   | 4.2      |                 |     | 4.2      |                 |     |      |
| V <sub>IL</sub> | Low-level input voltage         | V <sub>CC</sub> = 2 V   | 0.5      |                 |     | 0.5      |                 |     | V    |
|                 |                                 | V <sub>CC</sub> = 4.5 V | 1.35     |                 |     | 1.35     |                 |     |      |
|                 |                                 | V <sub>CC</sub> = 6 V   | 1.8      |                 |     | 1.8      |                 |     |      |
| V <sub>I</sub>  | Input voltage                   |                         | 0        | V <sub>CC</sub> |     | 0        | V <sub>CC</sub> |     | V    |
| V <sub>O</sub>  | Output voltage                  |                         | 0        | V <sub>CC</sub> |     | 0        | V <sub>CC</sub> |     | V    |
| Δt/Δv           | Input transition rise/fall time | V <sub>CC</sub> = 2 V   | 1000     |                 |     | 1000     |                 |     | ns   |
|                 |                                 | V <sub>CC</sub> = 4.5 V | 500      |                 |     | 500      |                 |     |      |
|                 |                                 | V <sub>CC</sub> = 6 V   | 400      |                 |     | 400      |                 |     |      |
| T <sub>A</sub>  | Operating free-air temperature  |                         | −55      | 125             |     | −40      | 85              |     | °C   |

NOTE 3: All unused inputs of the device must be held at  $V_{CC}$  or GND to ensure proper device operation. Refer to the TI application report, *Implications of Slow or Floating CMOS Inputs*, literature number SCBA004.



POST OFFICE BOX 655303 • DALLAS, TEXAS 75265

Figura A.8 Pagina 2 del data sheet del componente 7404.

della serie 74xx è raramente importante, ma lo diventa per i chip moderni che dissipano le decine di watt o anche più.

Le *recommended operating conditions* (condizioni di funzionamento raccomandate) definiscono l'ambiente in cui si dovrebbe utilizzare il chip. In queste condizioni, il chip garantisce il rispetto delle sue specifiche. Le condizioni sono più stringenti dei massimi assoluti. Per esempio, la tensione di alimentazione deve essere compresa tra 2 e 6 V. I livelli logici di ingresso per la famiglia HC dipendono da  $V_{DD}$ . Si deve fare riferimento ai valori del data sheet relativi a  $V_{DD} = 4.5$  V per tollerare un abbassamento del 10% nell'alimentazione dovuto a disturbi nel sistema.

La **Figura A.9** mostra la terza pagina del data sheet. Le *electrical characteristics* (caratteristiche elettriche) definiscono come si comporta il dispositivo se usato rispettando le condizioni di funzionamento raccomandate e con valori di ingresso tenuti costanti. Per esempio, se  $V_{CC} = 5$  V (e si abbassa a 4.5 V) e la corrente di uscita  $I_{OH}/I_{OL}$  non supera i 20  $\mu$ A, nel caso peggiore si ha  $V_{OH} = 4.4$  V e  $V_{OL} = 0.1$  V. Se la corrente di uscita aumenta, le tensioni di uscita tendono a diventare meno ideali, perché i transistori nel chip si sforzano per fornire la corrente richiesta. La famiglia logica HC usa transistori CMOS che assorbono correnti molto basse. La corrente in ogni ingresso è garantita essere inferiore a 1000 nA ma a temperatura ambiente è normalmente di soli 0.1 nA. La corrente di alimentazione *quiescent*  $I_{DD}$  (consumata quando il chip è inattivo) è meno di 20  $\mu$ A, e ogni ingresso ha una capacità inferiore a 10 pF.

Le *switching characteristics* (caratteristiche di commutazione) definiscono come si comporta il dispositivo se usato rispettando le condizioni di funzionamento raccomandate quando gli ingressi cambiano. Il *propagation delay* (ritardo di propagazione),  $t_{pd}$  è misurato a partire dall'istante in cui l'ingresso attraversa il valore  $0.5 V_{CC}$  fino all'istante in cui l'uscita attraversa il medesimo valore  $0.5 V_{CC}$ . Se  $V_{CC}$  ha il valore nominale di 5 V e il chip pilota in uscita una capacità inferiore a 50 pF, il ritardo di propagazione non è superiore a 24 ns (e normalmente è molto inferiore a tale valore). Si ricordi che ogni ingresso ha una capacità di 10 pF, quindi il chip non può pilotare più di cinque chip di identiche caratteristiche alla massima velocità. Le capacità parassite dei fili di interconnessione tra i chip riducono ulteriormente il carico utile. Il *transition time* (tempo di transizione), noto anche come tempo di salita/discesa, è misurato sulla transizione di uscita da 0.1 a 0.9  $V_{CC}$ .

Come visto nel paragrafo 1.8, il chip consuma sia potenza statica sia potenza dinamica. La potenza statica è bassa per chip HC: a 85 °C la massima corrente di alimentazione quiescente è di 20  $\mu$ A. A 5 V, questo corrisponde a un consumo di potenza di 0.1 mW. La potenza dinamica dipende dalle capacità pilotate dalle uscite e dalla frequenza di commutazione. Il 7404 ha una capacità interna di dissipazione di potenza di 20 pF per negatore. Se tutti i sei negatori commutano a 10 MHz e pilotano carichi esterni da 25 pF ciascuno, la potenza dinamica dissipata data dall'Espressione 1.4 risulta pari a  $\frac{1}{2}(6)(20 \text{ pF} + 25 \text{ pF})(5^2)(10 \text{ MHz}) = 33.75 \text{ mW}$  e la massima potenza totale dissipata è quindi 33.85 mW.

## A.6 ■ FAMIGLIE LOGICHE

I chip logici della serie 74xx sono stati realizzati utilizzando diverse tecnologie, definite **famiglie logiche**, che offrono diversi rapporti tra velocità, potenza consumata e valori dei livelli logici. Altri componenti sono generalmente progettati per essere compatibili con alcune di queste famiglie. I chip originari, come il 7404, sono stati realizzati usando transistori bipolari nella tecnologia nota come **TTL** (*Transistor-Transistor Logic*, logica a transistor-transistore). Tecnologie più recenti introducono una o più lettere dopo il 74 per indicare la

# SN54HC04, SN74HC04 HEX INVERTERS

SCLS078D – DECEMBER 1982 – REVISED JULY 2003

electrical characteristics over recommended operating free-air temperature range (unless otherwise noted)

| PARAMETER       | TEST CONDITIONS                                           | V <sub>CC</sub>           | T <sub>A</sub> = 25 °C |      |       | SN54HC04 |       | SN74HC04 |       | UNIT |
|-----------------|-----------------------------------------------------------|---------------------------|------------------------|------|-------|----------|-------|----------|-------|------|
|                 |                                                           |                           | MIN                    | TYP  | MAX   | MIN      | MAX   | MIN      | MAX   |      |
| V <sub>OH</sub> | V <sub>I</sub> = V <sub>IH</sub> or V <sub>IL</sub>       | I <sub>OH</sub> = -20 µA  | 2 V                    | 1.9  | 1.998 | 1.9      |       | 1.9      |       | V    |
|                 |                                                           |                           | 4.5 V                  | 4.4  | 4.499 | 4.4      |       | 4.4      |       |      |
|                 |                                                           |                           | 6 V                    | 5.9  | 5.999 | 5.9      |       | 5.9      |       |      |
|                 |                                                           | I <sub>OH</sub> = -4 mA   | 4.5 V                  | 3.98 | 4.3   | 3.7      |       | 3.84     |       |      |
|                 |                                                           | I <sub>OH</sub> = -5.2 mA | 6 V                    | 5.48 | 5.8   | 5.2      |       | 5.34     |       |      |
| V <sub>OL</sub> | V <sub>I</sub> = V <sub>IH</sub> or V <sub>IL</sub>       | I <sub>OL</sub> = 20 µA   | 2 V                    |      | 0.002 | 0.1      |       | 0.1      |       | V    |
|                 |                                                           |                           | 4.5 V                  |      | 0.001 | 0.1      |       | 0.1      |       |      |
|                 |                                                           |                           | 6 V                    |      | 0.001 | 0.1      |       | 0.1      |       |      |
|                 |                                                           | I <sub>OL</sub> = 4 mA    | 4.5 V                  |      | 0.17  | 0.26     |       | 0.4      | 0.33  |      |
|                 |                                                           | I <sub>OL</sub> = 5.2 mA  | 6 V                    |      | 0.15  | 0.26     |       | 0.4      | 0.33  |      |
| I <sub>I</sub>  | V <sub>I</sub> = V <sub>CC</sub> or 0                     | 6 V                       |                        | ±0.1 | ±100  |          | ±1000 |          | ±1000 | nA   |
| I <sub>CC</sub> | V <sub>I</sub> = V <sub>CC</sub> or 0, I <sub>O</sub> = 0 | 6 V                       |                        |      | 2     |          | 40    |          | 20    | µA   |
| C <sub>I</sub>  |                                                           | 2 V to 6 V                |                        | 3    | 10    |          | 10    |          | 10    | pF   |

switching characteristics over recommended operating free-air temperature range, CL = 50 pF (unless otherwise noted) (see Figure 1)

| PARAMETER       | FROM (INPUT) | TO (OUTPUT) | V <sub>CC</sub> | T <sub>A</sub> = 25 °C |     |     | SN54HC04 |     | SN74HC04 |     | UNIT |
|-----------------|--------------|-------------|-----------------|------------------------|-----|-----|----------|-----|----------|-----|------|
|                 |              |             |                 | MIN                    | TYP | MAX | MIN      | MAX | MIN      | MAX |      |
| t <sub>pd</sub> | A            | Y           | 2 V             |                        | 45  | 95  |          | 145 |          | 120 | ns   |
|                 |              |             | 4.5 V           |                        | 9   | 19  |          | 29  |          | 24  |      |
|                 |              |             | 6 V             |                        | 8   | 16  |          | 25  |          | 20  |      |
| t <sub>t</sub>  |              | Y           | 2 V             |                        | 38  | 75  |          | 110 |          | 95  | ns   |
|                 |              |             | 4.5 V           |                        | 8   | 15  |          | 22  |          | 19  |      |
|                 |              |             | 6 V             |                        | 6   | 13  |          | 19  |          | 16  |      |

operating characteristics, T<sub>A</sub> = 25 °C

| PARAMETER                                                  | TEST CONDITIONS | TYP | UNIT |
|------------------------------------------------------------|-----------------|-----|------|
| C <sub>pd</sub> Power dissipation capacitance per inverter | No load         | 20  | pF   |



TEXAS  
INSTRUMENTS

POST OFFICE BOX 655303 • DALLAS, TEXAS 75265

Figura A.9 Pagina 3 del data sheet del componente 7404.

famiglia logica, come 74LS04, 74HC04 o 74HCT04. La **Tabella A.2** riassume le più diffuse famiglie logiche a 5 V.

I miglioramenti nella realizzazione dei circuiti bipolari e nel processo di fabbricazione hanno portato alle famiglie logiche *Schottky* (S) e *Low-Power Schottky* (LS, Schottky a bassa potenza), entrambe più veloci della famiglia TTL. La famiglia Schottky consuma più potenza, la famiglia Schottky a bassa potenza ne consuma meno. Le due famiglie avanzate *Advanced Schottky* (AS, Schottky avanzata) e *Advanced Low-Power Schottky* (ALS, Schottky avanzata a bassa potenza) hanno ulteriormente migliorato velocità e consumo di potenza rispetto alle famiglie S e LS. La famiglia *Fast* (F, veloce) è più veloce e consuma meno potenza della famiglia AS. Tutte queste famiglie forniscono più corrente quando le uscite sono basse rispetto a quando sono alte, quindi hanno livelli logici asimmetrici. Sono conformi ai livelli logici “TTL” ovvero:  $V_{IH} = 2\text{ V}$ ,  $V_{IL} = 0.8\text{ V}$ ,  $V_{OH} > 2.4\text{ V}$  e  $V_{OL} < 0.5\text{ V}$ .

Con la loro maturazione negli anni '80 e '90, i circuiti CMOS sono diventati molto diffusi per il loro consumo estremamente ridotto di potenza di alimentazione e di corrente di ingresso. Le famiglie *High-Speed CMOS* (HC, CMOS ad alta velocità) e *Advanced High-Speed CMOS* (AHC, CMOS avanzata ad alta velocità) hanno consumo statico di potenza praticamente nullo, e forniscono la stessa quantità di corrente per valori di uscita alti e bassi. Sono conformi ai livelli logici “CMOS” ovvero:  $V_{IH} = 3.15\text{ V}$ ,  $V_{IL} = 1.35\text{ V}$ ,  $V_{OH} > 3.8\text{ V}$  e  $V_{OL} < 0.44\text{ V}$ . Purtroppo questi valori non sono compatibili con i circuiti TTL, perché un'uscita TTL alta di 2.4 V potrebbe non essere riconosciuta come corretto valore di ingresso CMOS alto. Questo ha motivato l'uso delle famiglie *High-Speed TTL-compatible CMOS* (HCT, CMOS ad alta velocità compatibile TTL) e *Advanced High-Speed TTL-compatible CMOS* (AHCT, CMOS avanzata ad alta velocità compatibile TTL), che accettano in ingresso valori logici validi per TTL e generano in uscita valori logici validi per CMOS. Queste famiglie compatibili TTL sono un po' più lente delle corrispondenti famiglie non compatibili. Tutti i chip CMOS sono sensibili alle scariche elettrostatiche (ESD, *ElectroStatic Discharge*) quindi è necessario scaricare il proprio corpo (per es. toccando un grosso oggetto metallico) prima di prendere in mano un chip CMOS per evitare di danneggiarlo.

**Tabella A.2** Parametri tipici delle famiglie logiche a 5 V.

| Caratteristica   | Bipolare/TTL    |      |      |      |      |      | CMOS  |       | Compatibile CMOS/TTL |       |
|------------------|-----------------|------|------|------|------|------|-------|-------|----------------------|-------|
|                  | TTL             | S    | LS   | AS   | ALS  | F    | HC    | AHC   | HCT                  | AHCT  |
| $t_{pd}$ (ns)    | 22              | 9    | 12   | 7.5  | 10   | 6    | 21    | 7.5   | 30                   | 7.7   |
| $V_{IH}$ (V)     | 2               | 2    | 2    | 2    | 2    | 2    | 3.15  | 3.15  | 2                    | 2     |
| $V_{IL}$ (V)     | 0.8             | 0.8  | 0.8  | 0.8  | 0.8  | 0.8  | 1.35  | 1.35  | 0.8                  | 0.8   |
| $V_{OH}$ (V)     | 2.4             | 2.7  | 2.7  | 2.5  | 2.5  | 2.5  | 3.84  | 3.8   | 3.84                 | 3.8   |
| $V_{OL}$ (V)     | 0.4             | 0.5  | 0.5  | 0.5  | 0.5  | 0.5  | 0.33  | 0.44  | 0.33                 | 0.44  |
| $I_{OH}$ (mA)    | 0.4             | 1    | 0.4  | 2    | 0.4  | 1    | 4     | 8     | 4                    | 8     |
| $I_{OL}$ (mA)    | 16              | 20   | 8    | 20   | 8    | 20   | 4     | 8     | 4                    | 8     |
| $I_{IL}$ (mA)    | 1.6             | 2    | 0.4  | 0.5  | 0.1  | 0.6  | 0.001 | 0.001 | 0.001                | 0.001 |
| $I_{IH}$ (mA)    | 0.04            | 0.05 | 0.02 | 0.02 | 0.02 | 0.02 | 0.001 | 0.001 | 0.001                | 0.001 |
| $I_{DD}$ (mA)    | 33              | 54   | 6.6  | 26   | 4.2  | 15   | 0.02  | 0.02  | 0.02                 | 0.02  |
| $C_{pd}$ (pF)    | non applicabile |      |      |      |      |      | 20    | 12    | 20                   | 14    |
| costo* (dollari) | obsoleta        | 0.63 | 0.25 | 0.53 | 0.32 | 0.22 | 0.12  | 0.12  | 0.12                 | 0.12  |

\* Costo al pezzo per ordini da 1000 pezzi del componente Texas Instruments 7408 nel 2012.

La serie 74xx è molto economica, e le famiglie logiche più recenti costano spesso meno di quelle meno recenti, ormai obsolete. La famiglia LS è una scelta molto comune nei laboratori o per hobbisti, dove non sono richieste prestazioni particolari.

Lo standard a 5 V è entrato in crisi a metà degli anni '90, quando i transistori sono diventati troppo piccoli per sopportare tale tensione. Inoltre, livelli di tensioni più bassi comportano minori consumi di potenza elettrica. Oggi sono comunemente usate tensioni a 3.3, 2.5, 1.8, 1.2 V e anche meno. Questa esplosione del numero di diversi livelli di tensione crea problemi non banali di comunicazione tra chip con alimentazioni diverse. La **Tabella A.3** elenca alcune delle famiglie a bassa tensione: non tutti i componenti 74xx sono disponibili in tutte queste famiglie.

Tutte le famiglie logiche a bassa tensione usano transistori CMOS, che sono diventati i cavalli da tiro dei circuiti integrati moderni. Sono in grado di funzionare in un ampio intervallo di valori di  $V_{DD}$ , ma la loro velocità degrada alle tensioni più basse. Le famiglie *Low-Voltage CMOS* (LVC, CMOS a bassa tensione) e *Advanced Low-Voltage CMOS* (ALVC, CMOS avanzata a bassa tensione) sono usate a 3.3, 2.5 o 1.8 V. LVC resiste a tensioni di ingresso fino a 5.5 V, quindi può ricevere ingressi da circuiti sia CMOS sia TTL a 5 V. La famiglia *Advanced Ultra-Low-Voltage CMOS* (AUC, CMOS avanzata a tensione ultra bassa) è usata a 2.5, 1.8 e 1.2 V ed è particolarmente veloce. Sia ALVC sia AUC resistono a tensioni di ingresso fino a 3.6 V, quindi possono ricevere ingressi da circuiti a 3.3 V.

I componenti FPGA spesso prevedono differenti tensioni di alimentazione per la logica interna (il cosiddetto *core*) e per i piedini di I/O. Con l'avanzamento tecnologico, la tensione di alimentazione del core è scesa da 5 a 3.3, 2.5, 1.8 e 1.2 V per risparmiare potenza ed evitare di danneggiare i transistori sempre più piccoli. Le FPGA hanno piedini di I/O configurabili che possono operare in un'ampia gamma di tensioni, per poter essere compatibili con il resto del sistema.

## A.7 ■ PACKAGING E ASSEMBLAGGIO

I circuiti integrati sono normalmente inseriti in *package* (contenitori) di plastica o di ceramica. I package svolgono numerose funzioni, tra le quali il collegamento dei piccoli punti di I/O presenti sul circuito integrato ai piedini metallici esterni al package per facilitare le connessioni, la protezione del chip

**Tabella A.3** Parametri tipici delle famiglie logiche a bassa tensione.

| $V_{dd}$ (V)    | LVC  |      |      | ALVC |       |      | AUC             |       |      |
|-----------------|------|------|------|------|-------|------|-----------------|-------|------|
|                 | 3.3  | 2.5  | 1.8  | 3.3  | 2.5   | 1.8  | 2.5             | 1.8   | 1.2  |
| $t_{pd}$ (ns)   | 4.1  | 6.9  | 9.8  | 2.8  | 3     | ?*   | 1.8             | 2.3   | 3.4  |
| $V_{IH}$ (V)    | 2    | 1.7  | 1.17 | 2    | 1.7   | 1.17 | 1.7             | 1.17  | 0.78 |
| $V_{IL}$ (V)    | 0.8  | 0.7  | 0.63 | 0.8  | 0.7   | 0.63 | 0.7             | 0.63  | 0.42 |
| $V_{OH}$ (V)    | 2.2  | 1.7  | 1.2  | 2    | 1.7   | 1.2  | 1.8             | 1.2   | 0.8  |
| $V_{OL}$ (V)    | 0.55 | 0.7  | 0.45 | 0.55 | 0.7   | 0.45 | 0.6             | 0.45  | 0.3  |
| $I_O$ (mA)      | 24   | 8    | 4    | 24   | 12    | 12   | 9               | 8     | 3    |
| $I_I$ (mA)      |      | 0.02 |      |      | 0.005 |      |                 | 0.005 |      |
| $I_{DD}$ (mA)   |      | 0.01 |      |      | 0.01  |      |                 | 0.01  |      |
| $C_{pd}$ (pF)   | 10   | 9.8  | 7    | 27.5 | 23    | ?*   | 17              | 14    | 14   |
| costo (dollari) |      | 0.17 |      | 0.20 |       |      | non disponibile |       |      |

\* Ritardo e capacità non disponibili al momento della stesura del libro.



da danni fisici, la distribuzione del calore generato dal chip su una superficie più grande per facilitarne il raffreddamento. I package sono poi posizionati su una scheda prototipale o su una scheda di circuito stampato e connessi mediante fili o piste metalliche per assemblare l'intero sistema.

### Package

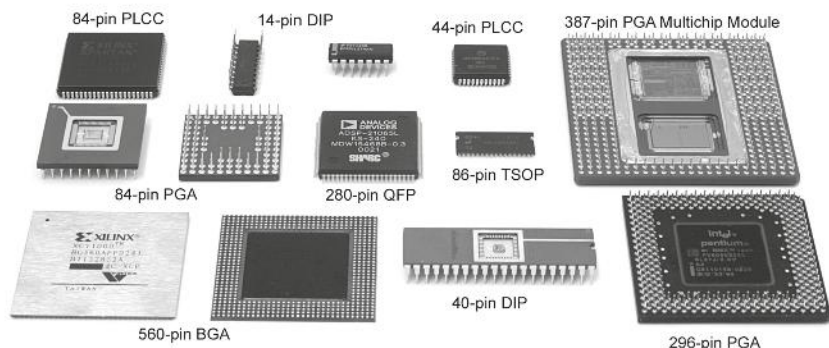
La **Figura A.10** mostra una varietà di package per circuiti integrati. I package si possono generalmente categorizzare in package per **buchi passanti** (PTH, *Pin Through-Hole*) e per **montaggio superficiale** (SMT, *Surface Mounting*). I package PTH hanno piedini metallici che possono essere inseriti nei buchi di una scheda di circuito stampato o di uno zoccolino (*socket*). I package DIP (*Dual Inline Package*) hanno due file di piedini spaziate di 0.254 cm (0.1 pollici) uno dall'altro. I package PGA (*Pin Grid Array*, a matrice di piedini) consentono di avere più piedini in meno spazio sistemandoli sotto il package. I package SMT vengono saldati direttamente sulla superficie di una scheda di circuito stampato senza bisogno di buchi. I piedini dei package SMT vengono definiti contatti (*leads*). Il package TSOP (*Thin Small Outline Package*, con contorno sottile e piccolo) ha due file di piedini molto ravvicinati, generalmente spaziate di 0.0508 cm (0.02 pollici). Il package PLCC (*Plastic Leaded Chip Carrier*, contenitore di chip plastico a contatti) ha contatti a forma di J su tutti i quattro lati, spaziate di 0.127 cm (0.05 pollici): può essere saldato direttamente sulla scheda o sistemato in un opportuno zoccolino. Il package QFP (*Quad Flat Pack*, package piatto a quattro file) permette un elevato numero di piedini sistemati su tutti e quattro i lati. Il package BGA (*Ball Grid Array*, a matrice di palline) elimina completamente i piedini: ha invece centinaia di piccole palline a saldare, poste sul lato inferiore; viene posizionato accuratamente sui corrispondenti contatti nella scheda di circuito stampato, poi riscaldato in modo che le palline si fondano e saldino il package alla scheda sottostante.

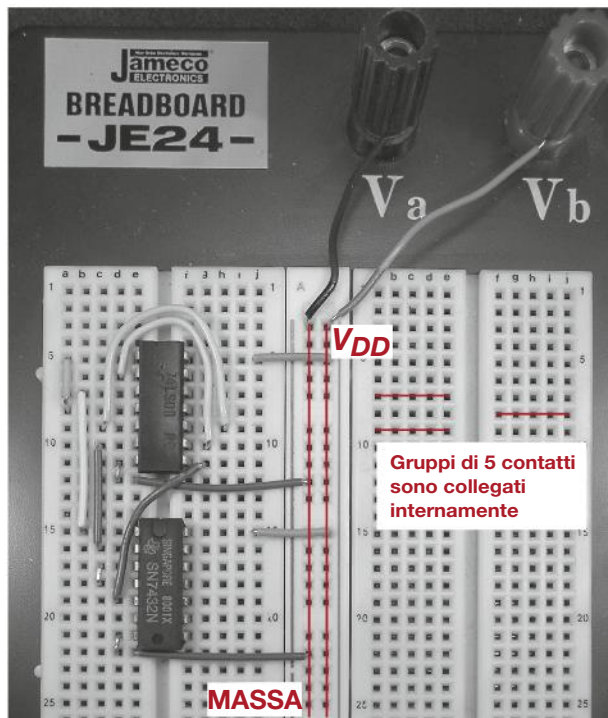
### Schede prototipali

I package DIP sono facili da usare per i prototipi perché possono essere inseriti in una scheda prototipale: la cosiddetta **breadboard** (letteralmente “tagliere per il pane”). Una breadboard è una scheda plastica contenente righe di buchi di contatto come quelli degli zoccolini, come mostrato nella **Figura A.11**. Tutti i gruppi di cinque buchi sulla stessa riga sono collegati tra loro, e ogni piedino del package viene inserito in una riga diversa. I fili possono essere inseriti in buchi adiacenti sulla stessa riga per effettuare i collegamenti. Sono spesso disponibili colonne separate di buchi di collegamento lungo tutta l'altezza della breadboard per distribuire alimentazione e massa.

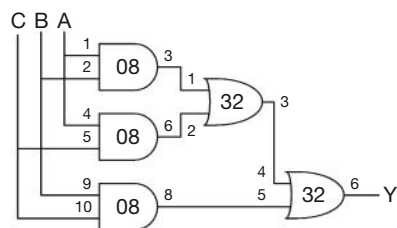
La Figura A.11 mostra una breadboard con una porta a maggioranza realizzata con i chip 74LS08 (porte AND) e 74LS32 (porte OR). Lo schema del circuito è mostrato nella **Figura A.12**. Ogni porta logica nello schema è iden-

**Figura A.10**  
Alcuni package per circuiti integrati.





**Figura A.11**  
La porta logica a maggioranza realizzata su breadboard.



**Figura A.12**  
Schema circuitale della porta logica a maggioranza con indicazione dei chip e dei piedini.

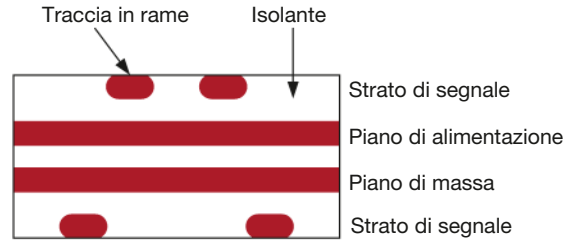
tificata dalla sigla del chip (08 o 32) e dai numeri dei piedini degli ingressi e delle uscite (come indicato dalla Figura eA.1). Come si può notare, i collegamenti nello schema sono stati realizzati sulla breadboard: gli ingressi sono collegati ai piedini 1, 2 e 5 del chip 08 e l'uscita è generata al piedino 6 del chip 32. Alimentazione e massa sono collegate rispettivamente ai piedini 14 e 7 di entrambi i chip prelevandole dalle colonne verticali di alimentazione e massa collegate agli spinotti a banana Vb e Va. Contrassegnare in questo modo lo schema e controllare come sono state fatte le connessioni sulla breadboard è un ottimo metodo per ridurre il numero di errori durante la prototipazione.

Bisogna fare attenzione perché è facile inserire un filo in un buco sbagliato della breadboard, o trovarsi con un filo che salta fuori dal buco, quindi la prototipazione richiede molta attenzione (e in genere un buon collaudo in laboratorio). Naturalmente le breadboard vanno bene per la prototipazione e non certo per la produzione.

### Schede di circuito stampato

Invece di essere inseriti in breadboard, i chip possono essere saldati a una scheda di circuito stampato (PCB, *Printed Circuit Board*) costituita da strati alternati di rame conduttivo e di plastica epossidica isolante. Il rame viene intagliato a formare piste elettriche di connessione dette **tracce**. Vengono poi creati dei buchi nella piastra, successivamente foderati di metallo per collegare elettricamente i diversi strati. Le schede vengono generalmente progettate utilizzando supporti CAD (*Computer Aided Design*). Si può tracciare e

**Figura A.13**  
Sezione trasversale di una scheda  
di circuito stampato.



perforare una scheda in laboratorio, oppure inviare il progetto a una ditta specializzata per produzioni di massa a basso costo. Queste ditte hanno tempi di consegna di qualche giorno (o qualche settimana per produzioni di massa a basso costo) con un costo di qualche centinaio di dollari per l'avvio della produzione e di qualche dollaro per ogni scheda prodotta, facendo riferimento a schede di media complessità da produrre in molti esemplari.

Le tracce sulla scheda di circuito stampato sono generalmente realizzate in rame per la sua bassa resistenza, e vengono racchiuse in materiale isolante costituito in genere da una plastica verde resistente al fuoco denominata FR4. La scheda di solito ha anche strati di rame per l'alimentazione e la massa, denominati **piani**, tra gli strati che trasportano i segnali. La **Figura A.13** mostra una sezione trasversale di una scheda di circuito stampato: gli strati di segnale sono in alto e in basso, e i piani di alimentazione e massa sono annegati all'interno della scheda. Questi piani hanno resistenza molto bassa, quindi distribuiscono alimentazione stabile ai componenti montati sulla scheda. Consentono inoltre di garantire valori di capacità e induttanza delle tracce uniformi e predicibili.

La **Figura A.14** mostra la scheda di circuito stampato per il vecchio computer Apple II+ del 1970. In alto c'è il processore 6502. Subito sotto sei chip di ROM da 16 kB che costituiscono un banco di ROM da 12 KB contenente il sistema operativo. Ancora più sotto, tre file ciascuna con otto chip di DRAM da 16 Kb, che costituiscono il banco di RAM da 48 KB. A destra diverse file di componenti logici della serie 74xx per la decodifica degli indirizzi di memoria e altre funzioni. Le righe tra i chip sono le tracce che li collegano tra loro. I punti alla fine di alcune tracce sono i buchi riempiti di metallo.

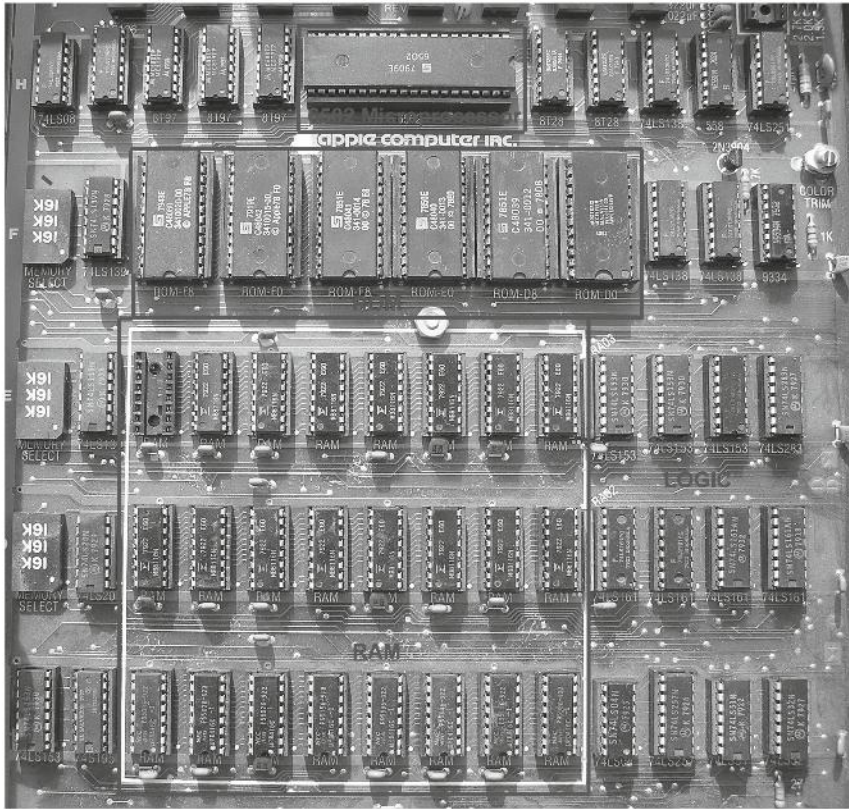
### Riassumendo

Molti chip moderni con grandi numeri di piedini di ingresso e uscita usano package di tipo SMT, specialmente delle categorie QFP e BGA. Tali package possono essere usati solo su schede di circuito stampato, non su breadboard. Soprattutto i package BGA sono complessi da utilizzare perché richiedono particolari attrezzature di assemblaggio, e le palline di collegamento non consentono di fare verifiche in laboratorio con la sonda di un voltmetro o di un oscilloscopio durante il collaudo perché sono nascoste sotto il package.

I progettisti devono quindi considerare quasi da subito il problema dei package, per determinare se sia possibile l'uso di una scheda prototipale nella fase di sviluppo o se si deve ricorrere a BGA. I professionisti esperti ricorrono invece molto di rado alle schede prototipali, fiduciosi nella correttezza del proprio progetto senza passare per la fase di sperimentazione.

## A.8 ■ LINEE DI TRASMISSIONE

Sin qui si è fatta l'ipotesi che i fili di collegamento fossero connessioni **equipotenziali** con lo stesso valore di tensione lungo tutto il filo. In realtà i segnali si propagano lungo i fili alla velocità della luce sotto forma di onde elettroma-



**Figura A.14**  
La scheda di circuito stampato dell'Apple II+.

gnetiche. Se i fili sono abbastanza corti o se i segnali cambiano lentamente nel tempo, l'ipotesi di equipotenzialità è accettabile. Se però un filo è molto lungo oppure il segnale cambia molto velocemente, il **tempo di trasmissione** (o di propagazione) del segnale lungo il filo diventa importante per determinare accuratamente il ritardo del circuito. Si deve quindi modellizzare il filo come **linea di trasmissione** nella quale un'onda di tensione e corrente si propaga alla velocità della luce. Quando l'onda arriva alla fine della linea, può riflettersi all'indietro lungo la linea stessa: tale riflessione può produrre rumore o causare comportamenti anomali se non si prendono le dovute precauzioni. I progettisti di sistemi digitali devono quindi considerare il comportamento delle linee di trasmissione per tener conto degli effetti in termini di ritardo e rumore causati dalle linee lunghe.

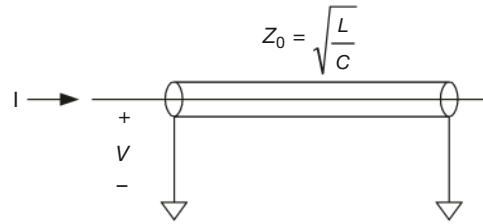
Le onde elettromagnetiche si propagano alla velocità della luce specifica del mezzo di trasmissione, quindi in un tempo breve ma non nullo. La velocità della luce  $v$  dipende dalla **permettività**  $\epsilon$  e dalla **permeabilità**  $\mu$  del mezzo considerato<sup>1</sup>:  $v = \frac{1}{\sqrt{\mu\epsilon}} = \frac{1}{\sqrt{LC}}$ .

La velocità della luce nello spazio vuoto è  $v = c = 3 \times 10^8$  m/s. I segnali in una scheda di circuito stampato viaggiano circa a metà di tale velocità, perché l'isolante FR4 ha una permettività quattro volte quella dell'aria, quindi circa a  $1.5 \times 10^8$  m/s, ovvero 15 cm/ns. Il ritardo  $t_d$  (*time delay*) di un segnale trasmesso da una linea di lunghezza  $l$  è dunque pari a:

$$t_d = \frac{l}{v} \quad (\text{A.4})$$

<sup>1</sup> La capacità  $C$  e l'induttanza  $L$  di un filo dipendono dalla permettività e dalla permeabilità del mezzo fisico nel quale il filo è collocato.



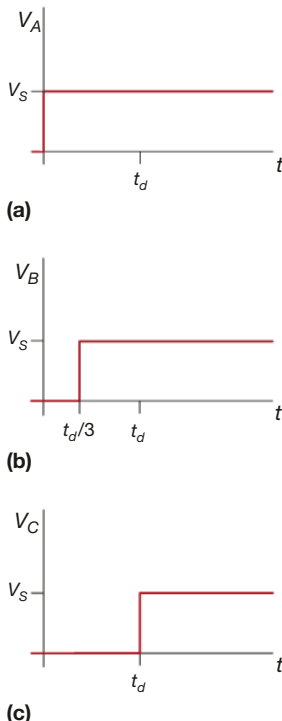
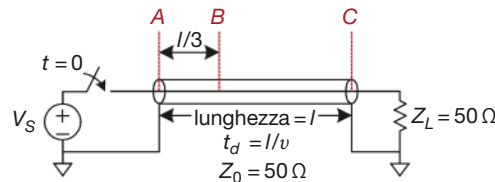
**Figura A.15** Simbolo della linea di trasmissione.

L'**impedenza caratteristica**  $Z_0$  di una linea di trasmissione è il rapporto tra tensione e corrente di un'onda che viaggia nella linea:  $Z_0 = V/I$ , che **non** è la resistenza del filo (una buona linea di trasmissione di un sistema digitale ha resistenza trascurabile).  $Z_0$  dipende da induttanza e capacità della linea (vedi la derivazione matematica nel paragrafo A.8.7) e ha tipicamente valori compresi tra 50 e 75  $\Omega$ .

$$Z_0 = \sqrt{\frac{L}{C}} \quad (\text{A.5})$$

La **Figura A.15** mostra il simbolo di una linea di trasmissione. Il simbolo ricorda un **cavo coassiale** (*coaxial cable*) con il conduttore interno di segnale e il conduttore esterno collegato a massa, come nei cavi usati per l'antenna del televisore.

Per comprendere il comportamento delle linee di trasmissione si deve immaginare l'onda di tensione che si propaga lungo la linea; quando l'onda arriva alla fine della linea, essa può venire assorbita o riflessa, a seconda della terminazione (ovvero del carico) presente. Le riflessioni si propagano all'indietro lungo la linea, sommandosi alla tensione in arrivo. Le terminazioni possono essere classificate in adattate, aperte, cortocircuitate e non adattate. I paragrafi che seguono descrivono come un'onda si propaga lungo una linea e cosa accade quando raggiunge la terminazione.

**Figura A.17**  
Forme d'onda della tensione  
ai punti A, B e C della Figura A.16.**Figura A.16** Linea di trasmissione con terminazione adattata.

### A.8.1 Terminazione adattata

La **Figura A.16** mostra una linea di trasmissione di lunghezza  $l$  con una **terminazione adattata**, nella quale cioè l'impedenza del carico (*Load*)  $Z_L$  è uguale all'impedenza caratteristica  $Z_0$ . La linea di trasmissione ha un'impedenza caratteristica di 50  $\Omega$ . Un capo della linea è collegato al generatore di tensione tramite un interruttore che si chiude al tempo  $t = 0$ , l'altro capo è collegato al carico adattato di 50  $\Omega$ . In questo paragrafo si analizzano tensioni e correnti nei punti A, B e C, ovvero all'inizio della linea, a un terzo della sua lunghezza e alla fine della linea.

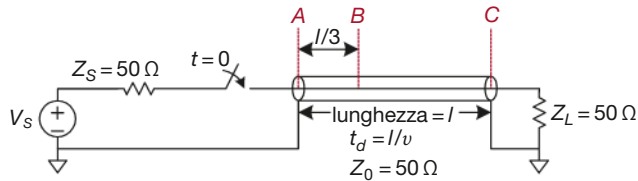
La **Figura A.17** mostra l'andamento nel tempo della tensione nei punti A, B e C. Inizialmente non c'è tensione né corrente nella linea perché l'interruttore è aperto. Al tempo  $t = 0$  l'interruttore si chiude e il generatore di tensione lancia un'onda di tensione  $V = V_s$  – detta **onda incidente** – lungo la linea. Dal

momento che l'impedenza caratteristica è  $Z_0$ , l'onda ha una corrente pari a  $I = V_S/Z_0$ . La tensione raggiunge immediatamente il punto A all'inizio della linea, come mostra la Figura A.17(a). L'onda si propaga lungo la linea alla velocità della luce e al tempo  $t_d/3$  raggiunge il punto B, causando un brusco salto di tensione da 0 a  $V_S$ , come mostrato nella Figura A.17(b). Al tempo  $t_d$  l'onda incidente raggiunge il punto C alla fine della linea, causando anche in questo caso un brusco salto di tensione da 0 a  $V_S$ , come mostrato nella Figura A.17(c). Tutta la corrente  $I$  che arriva alla fine della linea fluisce nell'impedenza  $Z_L$  causando una caduta di tensione ai capi della resistenza stessa pari a  $Z_L I = Z_L (V_S/Z_0) = V_S$  perché  $Z_L = Z_0$ . Questa tensione è consistente con l'onda propagatasi nella linea di trasmissione, quindi l'onda viene **assorbita** dall'impedenza del carico e la linea di trasmissione raggiunge il suo **stato stazionario**.

Allo stato stazionario, la linea di trasmissione si comporta come un filo ideale equipotenziale: la tensione in tutti i punti lungo il filo ha lo stesso valore. La **Figura A.18** mostra il circuito equivalente a quello della Figura A.16 in regime stazionario: la tensione è pari a  $V_S$  lungo tutto il filo.

### ESEMPIO A.2

**Linea di trasmissione con terminazioni di sorgente e di carico adatte.** La **Figura A.19** mostra una linea di trasmissione con impedenze della sorgente e del carico  $Z_S$  e  $Z_L$  entrambe adatte. Tracciare l'andamento nel tempo della tensione nei punti A, B e C. Quando il sistema raggiunge lo stato stazionario, e qual è il circuito equivalente allo stato stazionario?



**Figura A.19** Linea di trasmissione con terminazioni di sorgente e di carico adatte.

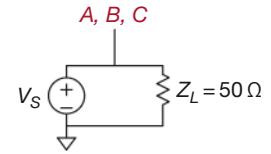
**Soluzione** Quando il generatore di tensione ha un'impedenza della sorgente  $Z_S$  in serie alla linea di trasmissione, parte della tensione cade ai capi di  $Z_S$  e il resto si propaga lungo la linea. Inizialmente la linea si comporta come un'impedenza  $Z_0$  perché il carico alla fine della linea non può influenzare il comportamento della linea finché non sia passato un tempo pari al ritardo di propagazione alla velocità della luce nella linea. Quindi la tensione dell'onda incidente che si propaga nella linea è data dall'espressione del partitore di tensione ovvero:

$$V = V_S \left( \frac{Z_0}{Z_0 + Z_S} \right) = \frac{V_S}{2} \quad (\text{A.6})$$

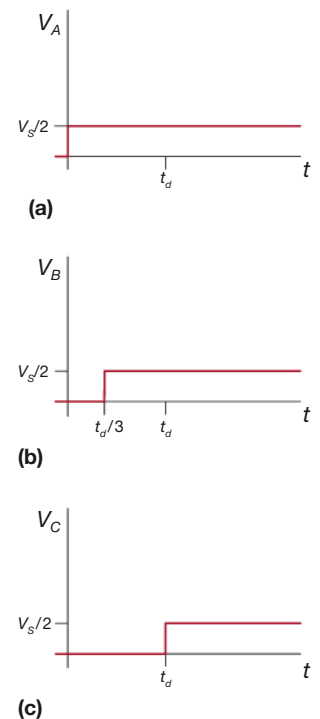
Quindi, al tempo  $t = 0$  un'onda di tensione  $V = \frac{V_S}{2}$  viene inviata nella linea a partire dal punto A, raggiunge il punto B dopo un tempo  $t_d/3$  e il punto C dopo un tempo  $t_d$ , come mostrato nella **Figura A.20**. Tutta la corrente viene assorbita dall'impedenza del carico  $Z_L$ , quindi il circuito raggiunge lo stato stazionario al tempo  $t = t_d$ . In questo stato, l'intera linea si trova alla tensione  $V_S/2$ , e risulta dallo schema equivalente del circuito allo stato stazionario mostrato nella **Figura A.21**.

### A.8.2 Terminazione aperta

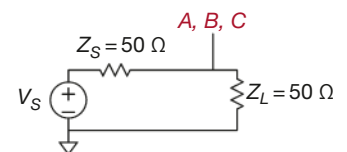
Quando l'impedenza del carico non è pari a  $Z_0$ , la terminazione non può assorbire tutta la corrente e una parte dell'onda viene riflessa. La **Figura A.22** mostra una linea di trasmissione con terminazione di carico aperta. Poiché



**Figura A.18** Circuito equivalente della Figura A.16 allo stato stazionario.



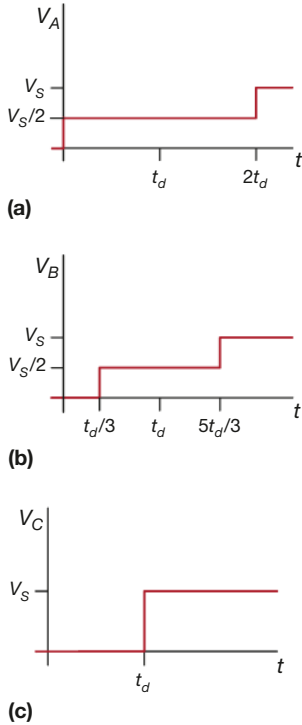
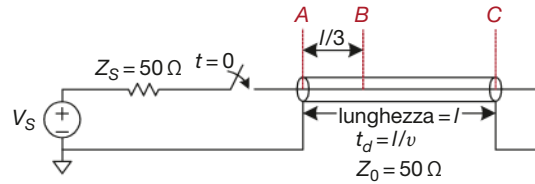
**Figura A.20** Forme d'onda della tensione ai punti A, B e C della Figura A.19.



**Figura A.21** Circuito equivalente della Figura A.19 allo stato stazionario.

**Figura A.22**

Linea di trasmissione con terminazione di carico aperta.

**Figura A.23**

Forme d'onda della tensione ai punti A, B e C della Figura A.22.

non può fluire corrente in una terminazione aperta, la corrente al punto C deve per forza essere sempre 0.

La tensione nella linea è inizialmente zero. Al tempo  $t = 0$  l'interruttore si chiude, e un'onda di tensione  $V = V_S \left( \frac{Z_0}{Z_0 + Z_S} \right) = \frac{V_S}{2}$  inizia a propagarsi lungo la linea. Si noti che l'onda iniziale è la stessa dell'Esempio A.2 ed è indipendente dalla terminazione, poiché il carico alla fine della linea non può influenzare il comportamento all'inizio della stessa finché non sia passato un tempo pari a  $2t_d$ . L'onda raggiunge il punto B dopo un tempo  $t_d/3$  e il punto C dopo un tempo  $t_d$ , come mostrato nella **Figura A.23**.

Quando l'onda incidente raggiunge il punto C non può proseguire perché il circuito è aperto, quindi deve riflettersi all'indietro verso la sorgente; anche l'onda riflessa ha una tensione  $V = V_S/2$  perché la terminazione aperta riflette l'intera onda.

La tensione in ogni punto è quindi la somma dell'onda incidente e di quella riflessa. Al tempo  $t = t_d$  la tensione al punto C è  $V = \frac{V_S}{2} + \frac{V_S}{2} = V_S$ . L'onda riflessa raggiunge il punto B dopo un tempo  $5t_d/3$  e il punto A dopo un tempo  $2t_d$ . Quando arriva in A, l'onda riflessa viene assorbita dall'impedenza della terminazione di sorgente, che ha lo stesso valore dell'impedenza caratteristica della linea, quindi il circuito raggiunge lo stato stazionario al tempo  $t = 2t_d$ , e la linea di trasmissione diventa equivalente a un filo equipotenziale con tensione  $V_S$  e corrente  $I = 0$ .

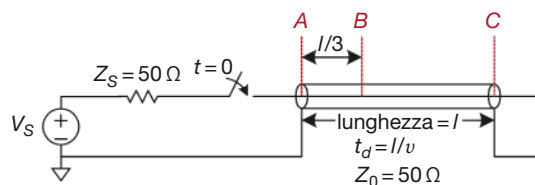
### A.8.3 Terminazione cortocircuitata

La **Figura A.24** mostra una linea di trasmissione con terminazione cortocircuitata a terra: la tensione al punto C deve per forza essere sempre 0.

Come nei casi precedenti, la tensione nella linea è inizialmente zero. Quando l'interruttore si chiude, un'onda di tensione  $V = \frac{V_S}{2}$  inizia a propagarsi lungo la linea (**Figura A.25**). Quando arriva alla fine della linea, l'onda deve riflettersi con polarità opposta. L'onda riflessa di tensione  $V = -\frac{V_S}{2}$  si somma all'onda incidente garantendo che la tensione al punto C sia 0. L'onda riflessa raggiunge il punto A dopo un tempo  $2t_d$  e viene assorbita dall'impedenza della terminazione di sorgente. A questo punto il circuito raggiunge lo stato stazionario, e la linea di trasmissione diventa equivalente a un filo equipotenziale con tensione  $V = 0$ .

**Figura A.24**

Linea di trasmissione con terminazione di carico cortocircuitata.



### A.8.4 Terminazione non adattata

L'impedenza della terminazione si definisce **non adattata** quando il suo valore non corrisponde a quello dell'impedenza caratteristica della linea. In ge-



nerale, quando un'onda incidente raggiunge una terminazione non adattata viene in parte assorbita e in parte riflessa. Il coefficiente di riflessione  $k_r$  indica la frazione dell'onda incidente  $V_i$  che viene riflessa:  $V_r = k_r V_i$ .

Il paragrafo A.8.8 ricava il coefficiente di riflessione utilizzando il criterio della conservazione della corrente, e dimostra che quando un'onda incidente che fluisce lungo una linea di impedenza caratteristica  $Z_0$  raggiunge una terminazione con impedenza  $Z_T$  il coefficiente di riflessione risulta essere:

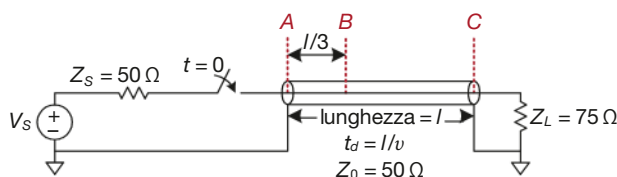
$$k_r = \frac{Z_T - Z_0}{Z_T + Z_0} \quad (\text{A.7})$$

Si possono notare alcune situazioni particolari. Se la terminazione è un circuito aperto ( $Z_T = \infty$ ) risulta  $k_r = 1$  perché l'onda incidente viene completamente riflessa (in modo che la corrente in uscita dalla linea rimanga sempre zero). Se la terminazione è un corto circuito ( $Z_T = 0$ ) risulta  $k_r = -1$  perché l'onda incidente viene riflessa con polarità inversa (in modo che la tensione alla fine della linea rimanga sempre zero). Se la terminazione è un carico adattato ( $Z_T = Z_0$ ) risulta  $k_r = 0$  perché l'onda incidente viene completamente assorbita.

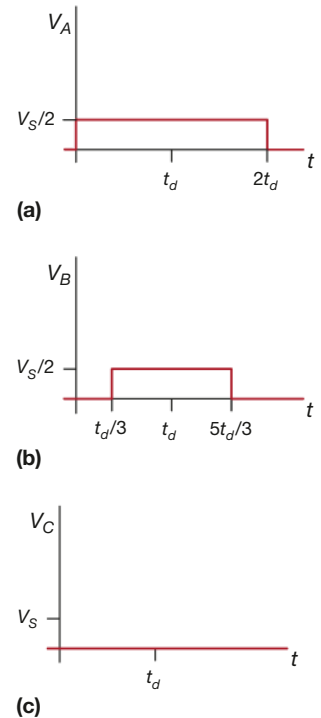
La **Figura A.26** mostra la riflessione in una linea di trasmissione con **terminazione di carico non adattata** pari a  $75 \Omega$ :  $Z_T = Z_L = 75 \Omega$  e  $Z_0 = 50 \Omega$ , quindi  $k_r = 1/5$ . Come nei casi precedenti, la tensione nella linea è inizialmente zero. Quando l'interruttore si chiude, un'onda di tensione  $V = \frac{V_s}{2}$  inizia a propagarsi lungo la linea e arriva alla fine al tempo  $t = t_d$ . Quando l'onda incidente arriva alla fine della linea, un quinto dell'onda viene riflesso e i restanti quattro quindi fluiscono nel carico. L'onda riflessa ha dunque una tensione pari a  $V = \frac{V_s}{2} \times \frac{1}{5} = \frac{V_s}{10}$ . La tensione totale al punto C è la somma di onda incidente e onda riflessa:  $V_C = \frac{V_s}{2} + \frac{V_s}{10} = \frac{3V_s}{5}$ . Al tempo  $t = 2t_d$  l'onda riflessa raggiunge il punto A dove viene assorbita dalla terminazione adattata  $Z_S = 50 \Omega$ . La **Figura A.27** mostra i valori di tensione e corrente lungo la linea. Di nuovo, si può notare che nello stato stazionario (raggiunto quando  $t > 2t_d$ ) la linea di trasmissione è equivalente a un filo equipotenziale, come mostrato nella **Figura A.28**. Allo stato stazionario il sistema si comporta da partitore di tensione, quindi:

$$V_A = V_B = V_C = V_s \left( \frac{Z_L}{Z_L + Z_S} \right) = V_s \left( \frac{75 \Omega}{75 \Omega + 50 \Omega} \right) = \frac{3V_s}{5}$$

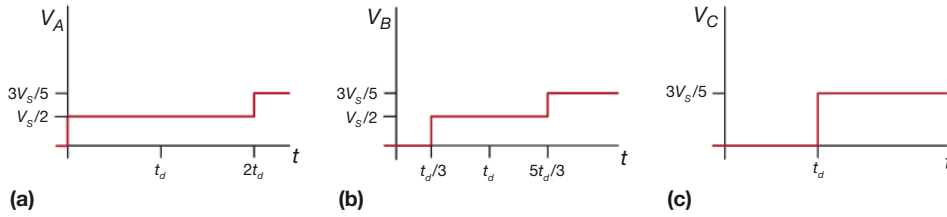
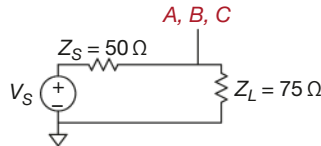
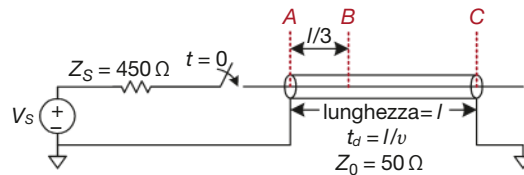
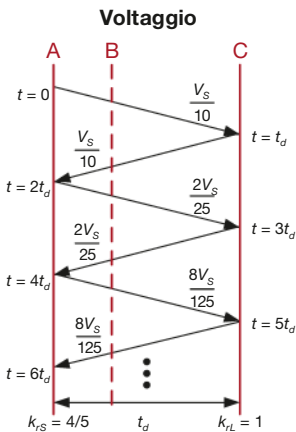
Le riflessioni si possono verificare a entrambi i capi della linea. La **Figura A.29** mostra una linea di trasmissione con un'impedenza di sorgente  $Z_S$  pari a  $450 \Omega$  e una terminazione di carico aperta. I coefficienti di riflessione lato carico e lato sorgente,  $k_{rL}$  e  $k_{rS}$ , sono rispettivamente 1 e  $4/5$ . In questo caso l'onda si riflette a entrambi i capi della linea finché non viene raggiunto uno stato stazionario.



**Figura A.26**  
Linea di trasmissione con terminazione non adattata.



**Figura A.25**  
Forme d'onda della tensione ai punti A, B e C della Figura A.24.

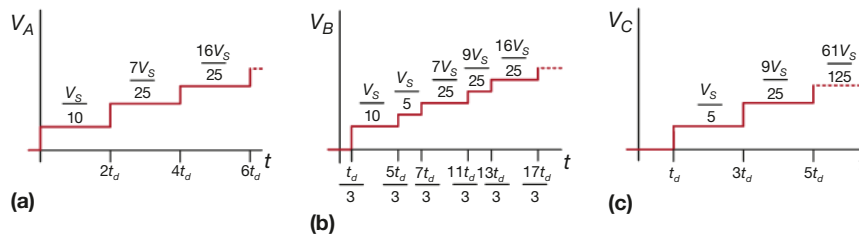
**Figura A.27** Forme d'onda della tensione ai punti A, B e C della Figura A.26.**Figura A.28** Circuito equivalente della Figura A.26 allo stato stazionario.**Figura A.29** Linea di trasmissione con terminazioni di sorgente e di carico non adatte.**Figura A.30** Diagramma dei rimbalzi relativo alla linea della Figura A.29.

Il “diagramma dei rimbalzi” mostrato nella **Figura A.30** aiuta a visualizzare le riflessioni: l’asse orizzontale rappresenta la distanza lungo la linea di trasmissione, e l’asse verticale rappresenta il tempo, crescente verso il basso. I due lati del diagramma sono i due capi sorgente e carico della linea, ovvero i punti A e C. Le onde incidenti e riflesse sono indicate come linee oblique tra i due punti A e C. Al tempo  $t = 0$ , l’impedenza di sorgente e la linea si comportano da partitore di tensione, inviando un’onda di tensione  $V_S/10$  dal punto A verso il punto C. Al tempo  $t = t_d$ , l’onda raggiunge il punto C e viene completamente riflessa ( $k_{rL} = 1$ ). Al tempo  $t = 2t_d$  l’onda riflessa di tensione  $V_S/10$  raggiunge il punto A e viene nuovamente riflessa con coefficiente di riflessione  $k_{rS} = 4/5$ , che produce un’onda di tensione  $\frac{2V_S}{25}$  verso il punto C, e così via.

La tensione in un certo istante in un punto qualsiasi della linea è la somma di tutte le onde incidenti e riflesse. Quindi al tempo  $t = 1.1 t_d$  la tensione al punto C è  $\frac{V_S}{10} + \frac{V_S}{10} = \frac{V_S}{5}$ , al tempo  $t = 3.1 t_d$  la tensione al punto C è  $\frac{V_S}{10} + \frac{V_S}{10} + \frac{2V_S}{25} + \frac{2V_S}{25} = \frac{9V_S}{25}$ , e così via. La **Figura A.31** mostra l’andamento nel tempo delle tensioni: quando  $t$  tende a infinito, le tensioni tendono allo stato stazionario  $V_A = V_B = V_C = V_S$ .

### A.8.5 Quando serve usare i modelli delle linee di trasmissione

I modelli delle linee di trasmissione servono quando il ritardo di propagazione nella linea  $t_d$  è maggiore di una frazione non trascurabile (per es. il 20%) dei tempi di commutazione (fronti di salita e di discesa) dei segnali. Se il ritardo di propagazione è molto più breve, esso ha un effetto trascurabile sulla propagazione del segnale, e le riflessioni fanno in tempo a dissiparsi durante le transizioni stesse del segnale. Se invece il ritardo di propagazione è maggiore, va tenuto in considerazione per predire accuratamente la forma d’onda del segnale: le riflessioni possono infatti distorcere le caratteristiche del segnale e dare luogo a operazioni logiche errate.



**Figura A.31** Forme d'onda di tensione e corrente per la linea della Figura A.29.

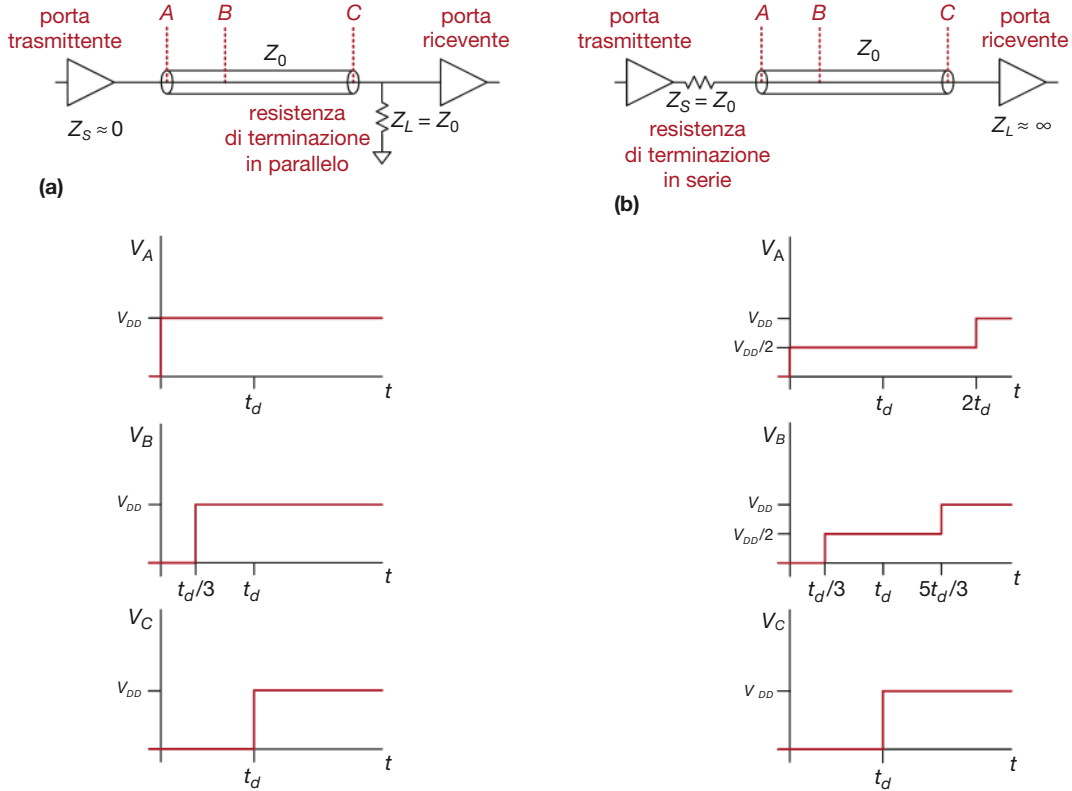
Va ricordato che i segnali si propagano su una scheda di circuito stampato a circa 15 cm/ns. Per la logica TTL, che ha tempi di commutazione di 10 ns, i collegamenti devono essere modellizzati come linee di trasmissione quando sono più lunghi di 30 cm ( $10 \text{ ns} \times 15 \text{ cm/ns} \times 20\%$ ). Le piste conduttrici sulle schede di circuito stampato sono generalmente più corte di 30 cm, quindi possono essere trattate come linee di trasmissione equipotenziali. Chip moderni hanno però tempi di commutazione di 2 ns o anche meno, quindi le piste più lunghe di 6 cm devono essere modellizzate come linee di trasmissione. Vale la pena sottolineare che l'uso di circuiti con fronti di salita e discesa più ripidi (quindi più brevi) del necessario causa solamente complessità inutili per il progettista.

Le schede prototipali non hanno un piano di massa, quindi i campi elettromagnetici di ogni segnale sono non uniformi e difficili da modellizzare. Inoltre interagiscono vicendevolmente. Questo può causare strane riflessioni e accoppiamenti tra segnali, quindi le schede prototipali diventano poco affidabili a frequenze superiori a qualche megahertz.

Le schede di circuito stampato, invece, hanno buone linee di trasmissione con caratteristiche di impedenza e velocità costanti lungo l'intera linea. Se vengono terminate con impedenze di sorgente o di carico adattate all'impedenza della linea, non danno luogo a fenomeni di riflessione.

### A.8.6 Corrette terminazioni delle linee di trasmissione

Ci sono due modi per terminare correttamente una linea di trasmissione, mostrati nella **Figura A.32**. Nella **terminazione in parallelo**, la sorgente (porta trasmittente) ha una bassa impedenza ( $Z_S \approx 0$ ) e una resistenza di carico  $Z_L = Z_0$  viene collegata in parallelo al carico (porta ricevente) tra l'ingresso e la massa. Quando la sorgente commuta da 0 a  $V_{DD}$ , trasmette un'onda di tensione  $V_{DD}$  lungo la linea. L'onda viene assorbita dalla terminazione di carico adattata, e non si verificano riflessioni. Nella **terminazione in serie**, una resistenza di sorgente  $Z_S$  viene collegata in serie alla sorgente per portare l'impedenza di sorgente al valore  $Z_0$ . Il carico ha un'impedenza molto elevata ( $Z_L \approx \infty$ ). Quando la sorgente commuta da 0 a  $V_{DD}$ , trasmette un'onda di tensione  $V_{DD}/2$  lungo la linea. L'onda viene riflessa dal carico corrispondente a un circuito aperto, portando la tensione lungo la linea a  $V_{DD}$ , e successivamente assorbita dalla sorgente. Entrambi gli schemi sono simili nel senso che la tensione all'ingresso del ricevitore passa come desiderato da 0 a  $V_{DD}$  al tempo  $t = t_d$ . Le differenze sono nella potenza dissipata e nelle forme d'onda che si propagano lungo la linea. La terminazione in parallelo continua a dissipare potenza nella resistenza di carico quando la linea è alla tensione  $V_{DD}$ , mentre la terminazione in serie non dissipa potenza continua perché il carico è un circuito aperto. Però in una linea terminata in serie i punti intermedi vedono dapprima un valore di tensione  $V_{DD}/2$  finché l'onda riflessa non torna indietro. Se si collegano altre porte logiche lungo la linea, queste ricevono per un certo tempo un valore di tensione illegale. Dunque, le terminazioni in serie funzionano bene per connessioni **punto-a-punto**, con un singolo trasmetti-



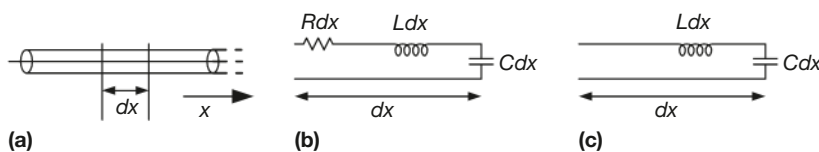
**Figura A.32** Schemi di terminazione: (a) in parallelo, (b) in serie.

tore e un singolo ricevitore, mentre le terminazioni in parallelo sono migliori per connessioni a **bus** con più ricevitori, perché anche i ricevitori posti lungo la linea non ricevono mai valori di tensione illegali.

### A.8.7 Espressione di $Z_0^*$

$Z_0$  è il rapporto tra tensione e corrente di un'onda che si propaga lungo una linea di trasmissione. In questo paragrafo si ricava l'espressione di  $Z_0$ , assumendo che il lettore conosca le basi dell'analisi dei circuiti RLC (Resistenza-Induttanza-Capacità).

Si supponga di applicare un gradino di tensione a una linea di trasmissione semi-infinita (quindi priva di riflessioni). La **Figura A.33** mostra la linea semi-infinita e il modello di un segmento di tale linea di lunghezza  $dx$ .  $R$ ,  $L$  e  $C$  sono i valori di resistenza, induttanza e capacità per unità di lunghezza. La **Figura A.33(b)** mostra il modello della linea di trasmissione con una componente resistiva  $R$ : questo modello viene definito **a perdita** (*lossy*) perché c'è dissipazione (quindi perdita) di energia nella resistenza della linea. Si tratta tuttavia di una perdita spesso trascurabile, per cui l'analisi può essere semplificata ignorando la componente resistiva e trattando la linea come **ideale**, secondo lo schema della **Figura A.33(c)**.



**Figura A.33** Modelli di una linea di trasmissione: (a) semi-infinita, (b) a perdita, (c) ideale.

Tensione e corrente sono funzioni di tempo e spazio lungo la linea di trasmissione, secondo le Espressioni A.8 e A.9.

$$\frac{\partial}{\partial x} V(x, t) = L \frac{\partial}{\partial t} I(x, t) \quad (\text{A.8})$$

$$\frac{\partial}{\partial x} I(x, t) = C \frac{\partial}{\partial t} V(x, t) \quad (\text{A.9})$$

Facendo la derivata nello spazio dell'Espressione A.8 e la derivata nel tempo dell'Espressione A.9 e sostituendo, si ottiene l'espressione d'onda A.10.

$$\frac{\partial^2}{\partial x^2} V(x, t) = LC \frac{\partial^2}{\partial t^2} V(x, t) \quad (\text{A.10})$$

$Z_0$  è il rapporto tra tensione e corrente lungo una linea di trasmissione, come mostrato nella **Figura A.34(a)**.  $Z_0$  deve essere indipendente dalla lunghezza della linea, perché il comportamento dell'onda non può dipendere da aspetti relativi a posizioni distanti dal punto considerato, quindi l'impedenza  $Z_0$  deve essere ancora la stessa anche dopo l'aggiunta di un piccolo tratto  $dx$  lungo la linea di trasmissione, come mostrato nella **Figura A.34(b)**. Si può quindi riscrivere la relazione della Figura A.34 in forma di espressione con i valori di impedenza dell'induttanza e della capacità:

$$Z_0 = j\omega L dx + [Z_0 \parallel (1/(j\omega C dx))] \quad (\text{A.11})$$

Risistemandolo:

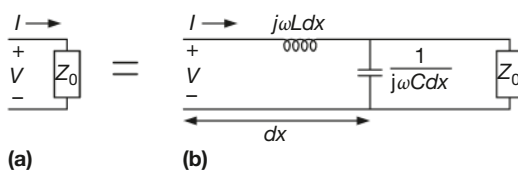
$$Z_0^2(j\omega C) - j\omega L + \omega^2 Z_0 LC dx = 0 \quad (\text{A.12})$$

Se si calcola il limite per  $dx$  che tende a 0, il terzo termine sparisce e si ottiene:

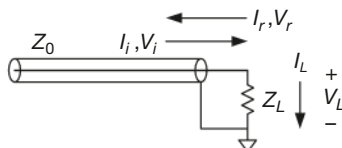
$$Z_0 = \sqrt{\frac{L}{C}} \quad (\text{A.13})$$

### A.8.8 Espressione del coefficiente di riflessione\*

Il coefficiente di riflessione  $k_r$  si ricava applicando la conservazione della corrente. La **Figura A.35** mostra una linea di trasmissione con impedenza caratteristica  $Z_0$  e impedenza di carico  $Z_L$ . Quando un'onda incidente di tensione  $V_i$  e corrente  $I_i$  raggiunge la terminazione, una parte di corrente  $I_L$  fluisce nell'impedenza di carico causando una caduta di tensione  $V_L$ . Il resto della corrente si riflette nella linea di trasmissione come onda di tensione  $V_r$  e corrente  $I_r$ .  $Z_0$  è il rapporto tra tensione e corrente di ogni onda che si propaga lungo la linea, quindi  $\frac{V_i}{I_i} = \frac{V_r}{I_r} = Z_0$ .



**Figura A.34**  
Modello della linea di trasmissione:  
(a) intera linea, (b) con tratto  
aggiuntivo di lunghezza  $dx$ .



**Figura A.35**  
Linea di trasmissione con indicate  
tensioni e correnti incidenti, riflesse  
e sul carico.

La tensione lungo la linea è la somma della tensione dell'onda incidente e di quella dell'onda riflessa, mentre la corrente che fluisce nella direzione positiva della linea è la differenza fra la corrente dell'onda incidente e quella dell'onda riflessa.

$$V_L = V_i + V_r \quad (\text{A.14})$$

$$I_L = I_i - I_r \quad (\text{A.15})$$

Usando la legge di Ohm per sostituire  $I_L$ ,  $I_i$  e  $I_r$  nell'Espressione A.15 si ottiene:

$$\frac{V_i + V_r}{Z_L} = \frac{V_i}{Z_0} - \frac{V_r}{Z_0} \quad (\text{A.16})$$

da cui si può esprimere il coefficiente di riflessione  $k_r$ :

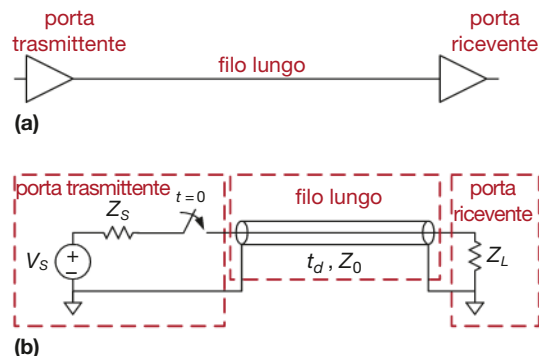
$$\frac{V_r}{V_i} = \frac{Z_L - Z_0}{Z_L + Z_0} = k_r \quad (\text{A.17})$$

### A.8.9 Riassumendo

Le linee di trasmissione modellizzano il fatto che i segnali richiedono tempo per propagarsi lungo le interconnessioni elettriche perché la velocità della luce è finita. Una linea di trasmissione ideale ha induttanza  $L$  e capacità  $C$  per unità di lunghezza costanti, e resistenza zero; è caratterizzata dalla propria impedenza caratteristica  $Z_0$  e dal ritardo di propagazione  $t_d$ , che possono essere derivati dai valori di capacità, induttanza e lunghezza della linea. La linea di trasmissione ha ritardi significativi ed effetti di rumore solo su segnali i cui tempi di salita e discesa sono inferiori a circa  $5t_d$ . Questo significa che per sistemi con tempi di salita e discesa di 2 ns le piste sulle schede di circuito stampato più lunghe di 6 cm devono essere analizzate come linee di trasmissione per comprenderne il comportamento in modo accurato.

Un sistema digitale costituito da una porta logica che pilota un lungo filo collegato all'ingresso di una seconda porta logica può essere modellizzato come linea di trasmissione, come mostrato nella **Figura A.36**. Il generatore di tensione, l'impedenza di sorgente  $Z_S$  e l'interruttore modellizzano la prima porta logica mentre commuta da 0 a 1 al tempo 0. La porta sorgente non può erogare corrente infinita: questo aspetto è modellizzato da  $Z_S$ .  $Z_S$  è usualmente piccola per una porta logica, ma il progettista può aggiungere una resistenza in serie alla porta per incrementare  $Z_S$  e adattarla all'impedenza della linea. L'ingresso della seconda porta è modellizzato da  $Z_L$ . I circuiti CMOS hanno in genere correnti di ingresso molto piccole, per cui  $Z_L$  tende a infinito. Il progettista può aggiungere una seconda resistenza in parallelo alla seconda porta, tra ingresso e massa, per adattare  $Z_L$  all'impedenza della linea.

**Figura A.36**  
Sistema digitale modellizzato come  
linea di trasmissione.



Quando la prima porta commuta, un'onda di tensione viene inviata nella linea. L'impedenza di sorgente e la linea formano un partitore di tensione, quindi la tensione dell'onda incidente risulta essere:

$$V_i = V_s \frac{Z_0}{Z_0 + Z_s} \quad (\text{A.18})$$

Al tempo  $t_d$  l'onda raggiunge la fine della linea: parte viene assorbita e parte viene riflessa. Il coefficiente di riflessione  $k_r$  indica la frazione di onda che viene riflessa:  $k_r = V_r/V_i$ , dove  $V_r$  è la tensione dell'onda riflessa e  $V_i$  la tensione di quella incidente.

$$k_r = \frac{Z_L - Z_0}{Z_L + Z_0} \quad (\text{A.19})$$

L'onda riflessa si somma alla tensione presente sulla linea, e raggiunge la sorgente al tempo  $2t_d$  dove di nuovo viene in parte assorbita e in parte riflessa. La riflessione continua ad avvenire avanti e indietro fino a quando la tensione sulla linea tende al valore che si sarebbe ottenuto se la linea stessa fosse stata un semplice filo equipotenziale.

## A.9 ■ ASPETTI ECONOMICI

Non si deve naturalmente dimenticare che progettisti e ditte produttrici di sistemi digitali non lavorano gratis, quindi gli aspetti economici sono fondamentali nelle scelte di progetto.

Il costo di un sistema digitale può essere suddiviso in una componente **non ricorrente** (NRE, *NonRecurring Engineering cost*) e in una parte **ricorrente**. La parte NRE riguarda il progetto del sistema, e comprende gli stipendi dei progettisti, i costi dei calcolatori e del software utilizzati, il costo per produrre il primo esemplare funzionante. Negli Stati Uniti, facendo riferimento alle tariffe del 2015, il costo per l'azienda di un progettista (stipendio, assicurazioni, fondo pensione, calcolatore con software per la progettazione) era di circa 200 000 dollari all'anno, quindi la parte NRE può essere molto consistente. I costi ricorrenti sono il costo di ogni ulteriore unità del prodotto sviluppato: includono i componenti, la produzione, il marketing, il supporto tecnico e la spedizione.

Il prezzo di vendita non deve coprire solo i costi del sistema, ma anche altri costi come affitto della sede, tasse, salari di altro personale non direttamente coinvolto nella progettazione (dai custodi all'amministratore delegato). E naturalmente, oltre a coprire tutte le spese la produzione, deve garantire un adeguato profitto.

### ESEMPIO A.3

**Ben cerca di fare soldi.** Ben Imbrogliabit ha progettato un circuito per contare le gocce di pioggia... e decide di provare a venderlo per fare un po' di soldi, ma ha bisogno di aiuto per decidere che tipo di realizzazione adottare, tra una FPGA e un ASIC. Il sistema di sviluppo e collaudo per le FPGA costa 1500 dollari e ogni FPGA costa 17 dollari. Lo sviluppo di un ASIC costa 600 000 dollari per realizzare le maschere di integrazione e 4 dollari per ogni chip prodotto.

Indipendentemente dal tipo di chip scelto, Ben deve poi montarlo su una scheda di circuito stampato che gli costa 1.5 dollari per scheda. È convinto di poter vendere 1000 dispositivi al mese, e siccome ha schiavizzato i migliori studenti del corso di laurea facendo loro sviluppare il progetto per superare il suo esame non ha costi di progettazione.

Se il prezzo di vendita deve essere il doppio del costo (100% di margine di profitto) e la vita stimata del prodotto è di 2 anni, quale tecnologia è meglio scegliere?



**Soluzione** Ben calcola il costo totale di entrambe le soluzioni nell’arco dei 2 anni come riportato nella **Tabella A.4**: pensa di vendere 24 000 dispositivi, e come si può vedere dalla Tabella A.4 la soluzione migliore per una vita del prodotto di 2 anni è l’uso di FPGA. Infatti il costo per unità nel caso delle FPGA è pari a  $445\,500\$/24\,000 = 18.56\%$ , e il prezzo di vendita è di 37.13\$, mentre l’opzione ASIC avrebbe un costo per unità di  $732\,000\$/24\,000 = 30.50\%$  e un prezzo di vendita di 61\$.

**Tabella A.4** Confronto dei costi di ASIC e FPGA.

| Costo                       | ASIC                                                     | FPGA                                                  |
|-----------------------------|----------------------------------------------------------|-------------------------------------------------------|
| NRE                         | 600 000 \$                                               | 1500\$                                                |
| chip                        | 4\$                                                      | 17\$                                                  |
| scheda di circuito stampato | 1.50\$                                                   | 1.50\$                                                |
| TOTALE                      | $600\,000\$ + (24\,000 \times 5.50\$)$<br>$= 732\,000\$$ | $1500\$ + (24\,000 \times 18.50\$)$<br>$= 445\,500\$$ |
| per unità                   | 30.50\$                                                  | 18.56\$                                               |

**ESEMPIO A.4**

**Ben diventa avido.** Dopo aver visto la pubblicità del suo prodotto, Ben ritiene che potrà vendere più dispositivi al mese di quanto stimato inizialmente. Se scegliesse l’opzione ASIC, quanti dispositivi dovrebbe vendere ogni mese perché tale opzione risulti più redditizia?

**Soluzione** Ben calcola il minimo numero *N* di dispositivi che deve vendere in due anni:

$$600\,000\$ + (N \times 5.50\$) = 1500\$ + (N \times 18.50\$)$$

Risolvendo l’equazione si ottiene *N* = 46 039 unità, ovvero 1919 unità al mese: Ben deve quasi raddoppiare il volume di vendite per avere vantaggi dall’opzione ASIC.

**ESEMPIO A.5**

**Ben diventa meno avido.** Ben si rende conto che ha esagerato, e che difficilmente potrà vendere più di 1000 dispositivi al mese. Però ha la sensazione che il suo prodotto possa avere una vita superiore ai 2 anni. Con un volume di vendite di 1000 unità al mese, quanto deve durare la vita del prodotto perché valga la pena adottare l’opzione ASIC?

**Soluzione** Se Ben vende più di 46 039 unità, l’opzione ASIC diventa la scelta migliore. Quindi, arrotondando, serve vendere 1000 pezzi al mese per almeno 47 mesi, cioè quasi 4 anni, quando il prodotto di Ben sarà quasi certamente obsoleto.

Solitamente i chip non si acquistano dal produttore ma da un distributore (a meno che non si debbano acquistare decine di migliaia di chip identici). Negli Stati Uniti, Digikey ([www.digikey.com](http://www.digikey.com)) è uno dei maggiori distributori, per un’ampia gamma di dispositivi elettronici, mentre Jameco ([www.jameco.com](http://www.jameco.com)) e All Electronics ([www.allelectronics.com](http://www.allelectronics.com)) hanno ampi cataloghi a prezzi contenuti, ideali per gli hobbisti.

# Istruzioni ARM

# B

- B.1** Istruzioni di elaborazione dati
- B.2** Istruzioni di accesso a memoria
- B.3** Istruzioni di salto
- B.4** Istruzioni varie
- B.5** Flag di condizione

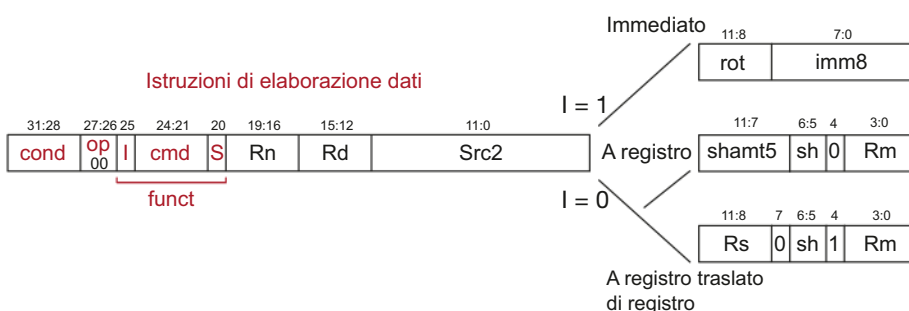
Questa appendice riassume le istruzioni di ARMv4 usate nel libro. Le codifiche delle condizioni sono riportate nella Tabella 6.3.

## B.1 ■ ISTRUZIONI DI ELABORAZIONE DATI

Le istruzioni di elaborazione dati standard usano la codifica riportata nella **Figura B.1**. Il campo *cmd* a 4 bit specifica il tipo di istruzione, come riportato nella **Tabella B.1**. Quando il bit *S* vale 1, il registro di stato viene aggiornato con i valori delle flag prodotti dall'istruzione. Il bit *I* e i bit 4 e 7 specificano una delle tre modalità di codifica del secondo operando, *Src2*, come descritto nel paragrafo 6.4.2. Il campo *cond* specifica quale condizione verificare, come indicato nel paragrafo 6.3.2.

### B.1.1 Istruzioni di moltiplicazione

Le istruzioni di moltiplicazione usano la codifica riportata nella **Figura B.2**. Il campo *cmd* a 3 bit specifica il tipo di istruzione, come riportato nella **Tabella B.2**.



**Figura B.1** Codifica delle istruzioni di elaborazione dati.

|                      |  |
|----------------------|--|
| Software applicativo |  |
| Sistema operativo    |  |
| Architettura         |  |
| Micro-architettura   |  |
| Blocchi logici       |  |
| Reti digitali        |  |
| Reti analogiche      |  |
| Dispositivi          |  |
| Fisica               |  |

Tabella B.1 Istruzioni di elaborazione dati.

| cmd                                                    | Nome                         | Descrizione                       | Operazione                               |
|--------------------------------------------------------|------------------------------|-----------------------------------|------------------------------------------|
| 0000                                                   | AND Rd, Rn, Src2             | AND bit a bit                     | $Rd \leftarrow Rn \& Src2$               |
| 0001                                                   | EOR Rd, Rn, Src2             | XOR bit a bit                     | $Rd \leftarrow Rn \wedge Src2$           |
| 0010                                                   | SUB Rd, Rn, Src2             | Sottrazione                       | $Rd \leftarrow Rn - Src2$                |
| 0011                                                   | RSB Rd, Rn, Src2             | Sottrazione rovesciata            | $Rd \leftarrow Src2 - Rn$                |
| 0100                                                   | ADD Rd, Rn, Src2             | Somma                             | $Rd \leftarrow Rn + Src2$                |
| 0101                                                   | ADC Rd, Rn, Src2             | Somma con riporto                 | $Rd \leftarrow Rn + Src2 + C$            |
| 0110                                                   | SBC Rd, Rn, Src2             | Sottrazione con riporto           | $Rd \leftarrow Rn - Src2 - \overline{C}$ |
| 0111                                                   | RSC Rd, Rn, Src2             | Somma con riporto rovesciata      | $Rd \leftarrow Src2 - Rn - \overline{C}$ |
| 1000 (S = 1)                                           | TST Rn, Src2                 | Controllo                         | Set flags based on $Rn \& Src2$          |
| 1001 (S = 1)                                           | TEQ Rn, Src2                 | Controllo di equivalenza          | Set flags based on $Rn \wedge Src2$      |
| 1010 (S = 1)                                           | CMP Rn, Src2                 | Confronto                         | Set flags based on $Rn - Src2$           |
| 1011 (S = 1)                                           | CMN Rn, Src2                 | Confronto con negativo            | Set flags based on $Rn + Src2$           |
| 1100                                                   | ORR Rd, Rn, Src2             | OR bit a bit                      | $Rd \leftarrow Rn   Src2$                |
| 1101<br>I = 1 OR<br>(instr <sub>11:4</sub> = 0)        | Traslazioni:<br>MOV Rd, Src2 | Copia                             | $Rd \leftarrow Src2$                     |
| I = 0 AND<br>(sh = 00;<br>instr <sub>11:4</sub> ≠ 0)   | LSL Rd, Rm, Rs/shamt5        | Traslazione logica a sinistra     | $Rd \leftarrow Rm \ll Src2$              |
| I = 0 AND<br>(sh = 01)                                 | LSR Rd, Rm, Rs/shamt5        | Traslazione logica a destra       | $Rd \leftarrow Rm \gg Src2$              |
| I = 0 AND<br>(sh = 10)                                 | ASR Rd, Rm, Rs/shamt5        | Traslazione aritmetica a destra   | $Rd \leftarrow Rm \ggg Src2$             |
| I = 0 AND<br>(sh = 11;<br>instr <sub>11:7,4</sub> = 0) | RRX Rd, Rm, Rs/shamt5        | Rotazione a destra con estensione | $\{Rd, C\} \leftarrow \{C, Rd\}$         |
| I = 0 AND<br>(sh = 11;<br>instr <sub>11:7</sub> ≠ 0)   | ROR Rd, Rm, Rs/shamt5        | Rotazione a destra                | $Rd \leftarrow Rn \text{ ror } Src2$     |
| 1110                                                   | BIC Rd, Rn, Src2             | Cancellazione bit a bit           | $Rd \leftarrow Rn \& \sim Src2$          |
| 1111                                                   | MVN Rd, Rn, Src2             | NOT bit a bit                     | $Rd \leftarrow \sim Rn$                  |

NOP (NO Operation) viene generalmente codificata come 0xE1A000 che equivale a MOV R0, R0.

Istruzioni di moltiplicazione

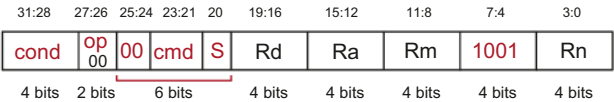


Figura B.2 Codifica delle istruzioni di moltiplicazione.

Tabella B.2 Istruzioni di moltiplicazione.

| cmd | Nome                 | Descrizione                                      | Operazione                                                                           |
|-----|----------------------|--------------------------------------------------|--------------------------------------------------------------------------------------|
| 000 | MUL Rd, Rn, Rm       | Moltiplicazione                                  | $Rd \leftarrow Rn \times Rm$ (low 32 bits)                                           |
| 001 | MLA Rd, Rn, Rm, Ra   | Moltiplicazione con accumulo                     | $Rd \leftarrow (Rn \times Rm) + Ra$ (low 32 bits)                                    |
| 100 | UMULL Rd, Rn, Rm, Ra | Moltiplicazione di long senza segno              | $\{Rd, Ra\} \leftarrow Rn \times Rm$<br>(all 64 bits, Rm/Rn unsigned)                |
| 101 | UMLAL Rd, Rn, Rm, Ra | Moltiplicazione di long senza segno con accumulo | $\{Rd, Ra\} \leftarrow (Rn \times Rm) + \{Rd, Ra\}$<br>(all 64 bits, Rm/Rn unsigned) |
| 110 | SMULL Rd, Rn, Rm, Ra | Moltiplicazione di long con segno                | $\{Rd, Ra\} \leftarrow Rn \times Rm$<br>(all 64 bits, Rm/Rn signed)                  |
| 111 | SMLAL Rd, Rn, Rm, Ra | Moltiplicazione di long con segno con accumulo   | $\{Rd, Ra\} \leftarrow (Rn \times Rm) + \{Rd, Ra\}$<br>(all 64 bits, Rm/Rn signed)   |

## B.2 ■ ISTRUZIONI DI ACCESSO A MEMORIA

Le istruzioni di accesso a memoria più comuni (LDR, STR, LDRB e STRB) operano su word o byte e sono codificate con  $op = 01$ . Altre istruzioni che operano su mezze parole (*halfword*) o su byte con segno sono codificate con  $op = 00$  e hanno meno flessibilità riguardo alla costruzione di  $Src2$ : lo spiazzamento immediato è solo di 8 bit e lo spiazzamento a registro non può essere ruotato. LDRSB e LDRSH estendono il segno. Vedi anche i modi di accesso a memoria tramite indici nel paragrafo 6.3.6.

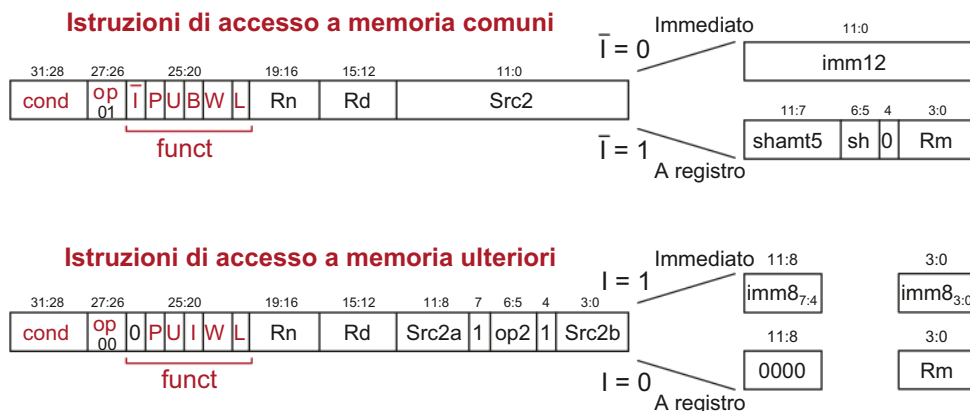


Figura B.3 Codifica delle istruzioni di accesso a memoria.

Tabella B.3 Istruzioni di accesso a memoria.

| op | B   | op2 | L | Nome                  | Descrizione                   | Operazione        |
|----|-----|-----|---|-----------------------|-------------------------------|-------------------|
| 01 | 0   | N/A | 0 | STR Rd, [Rn, ±Src2]   | Memorizza registro            | Mem[Adr] ← Rd     |
| 01 | 0   | N/A | 1 | LDR Rd, [Rn, ±Src2]   | Carica registro               | Rd ← Mem[Adr]     |
| 01 | 1   | N/A | 0 | STRB Rd, [Rn, ±Src2]  | Memorizza byte                | Mem[Adr] ← Rd7:0  |
| 01 | 1   | N/A | 1 | LDRB Rd, [Rn, ±Src2]  | Carica byte                   | Rd ← Mem[Adr]7:0  |
| 00 | N/A | 01  | 0 | STRH Rd, [Rn, ±Src2]  | Memorizza halfword            | Mem[Adr] ← Rd15:0 |
| 00 | N/A | 01  | 1 | LDRH Rd, [Rn, ±Src2]  | Carica mezza parola           | Rd ← Mem[Adr]15:0 |
| 00 | N/A | 10  | 1 | LDRSB Rd, [Rn, ±Src2] | Carica byte con segno         | Rd ← Mem[Adr]7:0  |
| 00 | N/A | 11  | 1 | LDRSH Rd, [Rn, ±Src2] | Carica mezza parola con segno | Rd ← Mem[Adr]15:0 |

## B.3 ■ ISTRUZIONI DI SALTO

La **Figura B.4** mostra la codifica delle istruzioni di salto e la **Tabella B.4** ne descrive le operazioni.

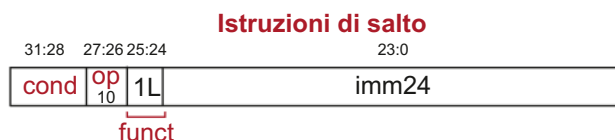


Figura B.4 Codifica delle istruzioni di salto.

**Tabella B.4** Istruzioni di salto.

| L | Nome     | Descrizione            | Operazione                                                            |
|---|----------|------------------------|-----------------------------------------------------------------------|
| 0 | B label  | Salto                  | $PC \leftarrow (PC+8) + \text{imm24} \ll 2$                           |
| 1 | BL label | Salto con collegamento | $LR \leftarrow (PC+8) - 4; PC \leftarrow (PC+8) + \text{imm24} \ll 2$ |

## B.4 ■ ISTRUZIONI VARIE

Il set di istruzioni ARMv4 include le seguenti istruzioni varie. Si rimanda ad ARM Architecture Reference Manual per i dettagli.

| Istruzioni               | Descrizione                                  | Scopo                                                                                               |
|--------------------------|----------------------------------------------|-----------------------------------------------------------------------------------------------------|
| LDM, STM                 | Carica/memorizza multipla                    | Salva e recupera registri nelle chiamate a sottoprogramma                                           |
| SWP/SWPB                 | Scambia (byte)                               | Azione atomica di caricamento e memorizzazione per sincronizzazione di processi                     |
| LDRT, LDRBT, STRT, STRBT | Carica/memorizza parola/byte con traslazione | Consente al sistema operativo di accedere alla memoria nello spazio di memoria virtuale dell'utente |
| SWI <sup>1</sup>         | Interrupt software                           | Genera un'eccezione, spesso usata per le chiamate a sistema operativo                               |
| CDP, LDC, MCR, MRC, STC  | Accesso al coprocessore                      | Comunica con il coprocessore opzionale                                                              |
| MRS, MSR                 | Copia da/verso il registro di stato          | Salva il registro di stato nella gestione delle eccezioni                                           |

<sup>1</sup> SWI è stata rinominata SVC (*SuperVisor Call*, chiamata al supervisore) in ARMv7.

## B.5 ■ FLAG DI CONDIZIONE

Le flag di condizione sono modificate dalle istruzioni di elaborazione dati con  $S = 1$  nel codice macchina. Tutte le istruzioni tranne CMP, CMN, TEQ e TST devono avere una "S" in fondo al codice mnemonico dell'istruzione per forzare  $S = 1$ . La **Tabella B.5** mostra quali flag sono modificate da ciascuna istruzione.

**Tabella B.5** Istruzioni che modificano le flag di condizione.

| Tipo            | Istruzioni                                 | Flag di condizione |
|-----------------|--------------------------------------------|--------------------|
| Somma           | ADDS, ADCS                                 | N, Z, C, V         |
| Sottrazione     | SUBS, SBCS, RSBS, RSCS                     | N, Z, C, V         |
| Confronto       | CMP, CMN                                   | N, Z, C, V         |
| Traslazione     | ASRS, LSLs, LSRs, RORS, RRXS               | N, Z, C            |
| Logiche         | ANDS, ORRS, EORS, BICS                     | N, Z, C            |
| Controllo       | TEQ, TST                                   | N, Z, C            |
| Copia           | MOVS, MVNS                                 | N, Z, C            |
| Moltiplicazione | MULS, MLAS, SMLALS, SMULLS, UMLALS, UMULLS | N, Z               |

# Indice analitico

La lettera "f" dopo i numeri indica i riferimenti alle figure, la lettera "t" le tabelle.  
La denominazione "e." indica i numeri di pagina del Capitolo 9; la denominazione "c."  
indica invece i numeri di pagina dell'Appendice C.

#define, c.5  
#include, c.5-c.6

## A

- ABI. *Vedi Application Binary Interface (ABI)*
- Acceleratori per elaborazioni grafiche, 354
- Acceso, 20, 23f
- Accumulatore, 274
- Acorn Computer, 261, 355
- Acorn RISC Machine, 261
- ADD, 219, 321
- Advanced High-performance Bus (AHB)*, e.43
- Advanced Micro Devices (AMD)*, 218
- Advanced Microcontroller Bus Architecture (AMBA)*, e.43
- Advanced RISC Machines*, 355
- AHB. *Vedi Advanced High-performance Bus (AHB)*
- AHB-Lite bus, e.43-e.44
- Albero dei prefissi, 180
- Alee, 65
- Algebra booleana, 40-45
  - postulati, 40
  - semplificazione delle espressioni, 43-44
  - teoremi, 41
- Allocazione dinamica della memoria (malloc, free),  
c.26-c.27
  - nella mappa di memoria di ARM, 253
- ALTO, 17
- ALU. *Vedi Unità logica/aritmetica (ALU)*
- ALUControl, 289-291, 312
- ALUOp, 295
- ALUResult, 289-293
- ALUSrc, 292
- AMAT. *Vedi Tempo di accesso a memoria (AMAT)*
- AMBA. *Vedi Advanced Microcontroller Bus Architecture (AMBA)*
- AMD. *Vedi Advanced Micro Devices (AMD)*
- AMD64, 275
- Amdahl, Gene, 370
- American Standard Code for Information Interchange (ASCII), 233, 234t, c.22-c.23
- An Investigation of the Laws of Thought* (Boole), 6
- Analisi delle prestazioni, 286-287,
  - confronto tra processori, 315
  - processore ARM a ciclo singolo, 296
  - processore ARM pipeline, 316-318
  - processore multi ciclo ARM, 313-316
- Analisi temporale, 102
  - processore a ciclo singolo, 299
  - processore multi ciclo, 315
  - processore pipeline, 328
- Analog-to-digital converter (ADC)*, e.19, e.24, e.25
- AND, 224
- Anodo, 21
- Antidipendenza, 350
- Application Binary Interface (ABI)*, 237
- Architettura, 217
  - architettura x86, 269
  - codifica delle istruzioni, 272-275
  - flag di stato, 272
  - istruzioni, 272
  - operandi, 271
  - particolarità, 275
  - quadro generale, 275
  - registri, 270
- compilare, assemblare e caricare, 252
- assemblaggio, 254-255

- caricamento, 256-257
  - collegamento, 255-256
  - compilazione, 253-254
  - mappa di memoria, 252-253
  - evoluzione dell'architettura ARM, 261
    - architettura a 64 bit, 269
    - istruzioni in virgola mobile, 233
    - istruzioni per il risparmio di potenza e per la sicurezza, 267
    - istruzioni SIMD, 268
    - processore di segnale (DSP), 263
    - set di istruzioni *Thumb*, 262
  - linguaggio assembly, 218
    - istruzioni, 218-219
    - operandi, 220-221
  - linguaggio macchina, 244
    - interpretazione, 250
    - istruzioni di accesso a memoria, 247
    - istruzioni di elaborazione dati, 244-247
    - istruzioni di salto, 248-249
    - modi di indirizzamento, 250
    - programma scritto in memoria, 251
  - programmare, 224
    - chiamate a sottoprogrammi, 235-244
    - cicli, 230-231
    - costrutti di selezione, 229-230
    - flag di condizione, 226-227
    - istruzioni di elaborazione dati, 224-226
    - memoria, 231-235
    - salti, 227-229
  - qualche dettaglio, 257
    - caricamento di *literal*, 257
    - eccezioni, 258-259
    - NOP, 258
  - Architettura a 64 bit, 269
  - Architettura ARM, evoluzione della, 218, 261
    - architettura a 64 bit, 269
    - istruzioni in virgola mobile, 233
    - istruzioni per il risparmio di potenza e per la sicurezza, 267
    - istruzioni SIMD, 268
    - processore di segnale (DSP), 263
  - Architettura ARMv3, 355
  - Architettura RISC, 219-220, 344-345
  - Architettura x86, 269-275
  - Architetture CISC (*complex instruction set computer*), 220, 270, 344
  - Argomenti nella riga di comando, c.36
  - Aritmetica
    - circuiti, 175-189
    - istruzioni ARM, 224-226
    - operatori C, c.10-c.11
    - operatori HDL, 131
  - Aritmetica a saturazione, 263
  - ARM Microcontroller Development Kit (MDK-ARM), 218
  - ARM7, 355t, 356
  - ARM9, 355t
  - ARM9E, 356
  - Array, 232, c.18-c.22
    - accesso, 232
    - bidimensionale, c.21-c.22
    - byte e caratteri, 233-235, c.22
    - come argomento di ingresso, c.20
    - confronto, c.23
    - dichiarazione di, 232-235, c.19
    - indirizzamento, 233, c.18-c.22
    - inizializzazione, c.21
  - Arrotondamento, 192
  - Aspetti economici, 425
  - ASR, 225
  - Assegnamenti bloccanti e non bloccanti, 144, 147-153
  - Assegnamento con selezione di segnale, 129
  - Assegnamento condizionale, 128-129
  - Assemblaggio, 254-255
  - Assembler, 252
  - Associatività
    - nell'algebra booleana, 41
    - nella cache, 370, 374-375
  - Astrazione, 2-3
    - digitale. *Vedi* Astrazione digitale
  - Astrazione digitale, 2-3, 5-6, 16-20
  - AT *attachment* (ATA), e.50
  - Atomi droganti, 21
- B**
- B, 228, 248-249, 292-293
  - Babbage, Charles, 5
  - Baco, 123
    - nel codice C, c.36-c.39
  - Baco *Floating Point Division* (FDIV), 123, 192
  - Banchi di registri, 259-260
  - BASSO, 17. *Vedi anche* FALSO
  - BCM2835, e.3-e.4, e.5f, e.22, e.44
    - timer, e.18
  - Benchmark, 286
  - BEQ, 228
  - BIC (*bit clear*), 224
  - big.LITTLE, 354-358
  - Biossido di silicio (SiO<sub>2</sub>), 22
  - Bit, 5
    - di modifica M (*dirty bit*), 381
    - di segno, 12
    - di set, 372
    - di utilizzo, 378
    - di validità, 373
    - meno significativo, 9, 10f
    - più significativo, 9
  - BL (*branch and link*), 235
  - Blocchi logici configurabili (CLB), 206, 402
  - Blocco di matrice logica (LAB), 207
  - Block, 493
  - BNE, 229
  - Bolla, 42
    - spingere una, 42-43, 49-50
  - Bolle, nella pipeline, 324



Boole, George, 6  
 BTA. *Vedi* Indirizzo di destinazione del salto (BTA)  
 Buffer, 15  
     assenza di, 80  
     tristate, 51  
 Buffer delle destinazioni di salto, 346  
 Buffer tristate, 51, 133  
     HDL, 133  
     multiplexer realizzato con, 59f, 64  
 Bus, 36  
     tristate, 51  
 Bus *ReadData*, 289  
 Byte 9-10, 233. *Vedi anche* Caratteri  
     meno significativo, 10, 10f  
     più significativo, 10  
 Byte *ModR/M*, 273

**C**

Cache, 370-382  
     a mappatura diretta, 371-372, 372f  
     bit di stato  
         bit di modifica (*M*), 381  
         bit di utilizzo (*U*), 378  
         bit di validità (*V*), 373  
     campo di indirizzo  
         bit di set, 372  
         bit di tag, 372  
         spiazzamento di blocco, 376  
         spiazzamento di byte, 372  
     completamente associative, 375  
     evoluzione delle, di ARM, 382  
     multi livello, 379  
     organizzazione, 378t  
         a mappatura diretta, 371  
         completamente associativa, 375-376  
         parzialmente associativa a molte vie, 374-375  
     parametri  
         blocco, 370  
         capacità (*C*), 370  
         dimensioni del blocco, 370, 376-377  
         grado di associatività (*N*), 370, 374  
         numero di set (*S*), 370  
     politiche di scrittura, 381-382  
         *write-back*, 381-382  
         *write-through*, 381-382  
     politiche di sostituzione, 378  
     prestazioni  
         hit, 369  
         miss, 369  
         tasso di hit, 369  
         tasso di miss, 369  
         tasso di miss vs. parametri della cache, 380f, 381f  
     progetto avanzato, 378-381  
 Calcolo del complemento a due, 12  
 Calcolo dell'indirizzo di memoria, 310f  
     flusso dei dati durante, 310f  
 Cambio di contesto, 352  
 Campionamento, 98

Campo *cmd*, 245, 427, 428t  
 Campo *cond* (condizione), 226, 227t, 244, 427  
 Campo *funct*, 245, 247  
 Campo *Rd*, 244-245  
 Campo *Rm*, 245  
 Campo *Rn*, 245  
 Campo *rot*, 245  
 Campo *Src2*, 245  
 Capacità, della cache, 370  
 Capacità produttiva, 112-114, 316, 353  
 Caratteri, 233, c.7t, c.22  
     array, c.23. *Vedi anche* Stringhe  
     di tipo C, c.23  
 Caratteristica di trasferimento DC, 18-19  
 Caratteristica di trasferimento in corrente continua (DC),  
     18-19, 18f  
 Caricamento, 256-257  
     indirizzamento base di, 250  
*casez*, *case?*, in HDL, 150  
 Catodo, 21  
 Celle di bit, 197  
     DRAM, 199  
     ROM, 198-199  
     SRAM, 199  
 Chiamante, 235  
 Chiamate a sottoprogrammi, 235, c.12-c.13  
     convenzioni sui nomi, c.13  
     foglia, 240  
     non-foglia, 240-241  
     parametri, 236, c.12  
     parametri aggiuntivi e variabili locali, 243  
     prototipi, c.13  
     registri preservati, 238-239  
     ricorsive, 241  
     ritorno, 236  
     salvataggio e ripristino di registri multipli, 238  
     senza ingressi né uscite, 235-236, c.12  
     stack, uso dello, 237. *Vedi anche* Stack  
 Chiamate di funzioni non-foglia, 240  
 Chiamate di funzioni ricorsive, 241-243  
 Chiamato, 235  
 Chip, 22  
     multiprocessori, 353  
 Chip *Medium-Scale Integration* (MSI), 398  
 Chip *Small-Scale Integration* (SSI), 398  
 Cicli, 230-231, c.15-c.16  
     in ARM assembly  
         *for*, 231  
         *while*, 231  
     in C  
         *do/while*, c.15-c.16  
         *for*, c.16  
         *while*, c.15  
 Cicli *for*, 231-232  
 Cicli per istruzione (CPI), 287, 314  
 Ciclo *do/while*, in C, 15-16  
 Ciclo *while*, 231, c.15  
 Circuiti

- a uscite multiple, 46
- alee, 65-67
- asincroni, 82, 84
- astabili, 81
- definizione di, 35
- di priorità, 46
- integrati (IC), 412
- integrati specifici per un'applicazione (ASIC), 404
- ritardo, 62-65
- sequenziali sincroni, 82-83
- sincroni, 84
- sintesi, 124, 126,f 128f
- temporizzazioni, 62-67, 98-104
- Circuiti integrati specifici per un'applicazione (ASIC), 404
- CMOS. *Vedi Complementary Metal-Oxide-Semiconductor Logic* (CMOS)
- cmp, 297
- Codice Gray, 53
- Codifica a singolo 0, 91
- Codifica a singolo 1, 90
- Codifica binaria, 87-88, 90
  - per il contatore a modulo 3, 90, 91t
  - per semafori stradali, 85-86
- Codifica decimale in binario (BCD), 191
- Codifica degli stati, FSM, 89
- Codifica delle istruzioni, x86, 272-274, 274f
- Collaudo *ad hoc*, 339
- Collegamenti USB, e.51-e.52
  - FTDI, e.51
  - modulo UM232H, e.52
- Collegamento, 252, 255
- Commenti
  - in SystemVerilog, 127
  - in VHDL, 127
  - nel codice assembly ARM, 218-219
  - nel codice C, 218-219, c.4-c.5
- Comparatore di uguaglianza, 182
- Comparatore di valore, 182
- Comparatori, 182-183
- Compilare, in C, 252-257
- Complementary Metal-Oxide-Semiconductor Logic* (CMOS), 20-26
- Componenti di memoria, 196-203. *Vedi anche* Memoria
  - celle di bit, 197
  - HDL, 203, 204, 342-343
  - organizzazione, 197
  - uso di funzioni logiche, 202
- Composizione combinatoria, 36
- Computer Newton, 355
- Comunicazione senza cavi Bluetooth, e.33-e.35
  - classi, e.34t
  - modulo BlueSMiRF, e.34
- Comunicazione seriale con un PC, e.16
- Concatenazione di bit, 134
- Condensatori, 21
- Condizioni di corsa, 82
- Conflitto (x), 50-51
- Confronto
  - prestazioni del processore, 315-316
  - usando ALU, 185
- Confronto delle prestazioni dei processori, 329-330
  - processore a ciclo singolo, 299
  - processore ARM multi ciclo, 315
  - processore ARM pipeline, 329-330
- Connettore DE-9, e.14f
- Consumo di potenza, 26-27
- Contatore modulo 3 (divisore-per-3)
  - HDL, 154
  - progetto di un, 90-91
- Contatori, 193-194
  - modulo 3, 90
- Contentitore a due file parallele di piedini (*dual-inline packages*, DIP), 22, e.20
- Controllo con H-bridge, e.36f
- Conversione
  - A/D, e.24-e.25
  - D/A, e.20
  - da binario a decimale, 7, 7f, 7t
  - da binario a esadecimale, 9
  - da decimale a binario, 7-8
  - da decimale a esadecimale, 9
  - da esadecimale a binario e decimale, 8, 8t
  - di formato (atoi, atol, atof), c.34
- Conversione numerica
  - da binario a decimale, 7
  - da binario a esadecimale, 9
  - da decimale a binario, 7-8
  - da decimale a esadecimale, 9
  - da esadecimale a binario e decimale, 8
  - in complemento a due, 12
- Convertitori digitali-analogici (DAC), e.19
- CoreMark, 286
- Corrente di alimentazione quiescente, 27
- Corrente di dispersione, 27
- Cortex-A7 e -A15, 355t, 359
- Cortex-A9, 355t
- Costanti
  - in ARM assembly, 221-222. *Vedi anche* Immediati
  - in C, c.4-c.5
- Costrutti di selezione, 229
  - if, 229
  - if/else, 229-230
  - switch/case, 230
  - in ARM assembly
  - in C, c.13-c.14
  - if, c.13-c.14
  - if/else, c.14
  - switch/case, c.14
  - in HDL, 145-147
  - case, 145
  - casez, case?, 150
  - if, if/else, 145-147
- Costrutti if/else annidati, 230
- Costrutto/istruzione if
  - in ARM assembly, 229

in C, c.13  
 in HDL, 145-146  
 Costrutto/istruzione *switch/case*  
 in C, c.14-c.15  
 nel linguaggio ARM assembly, 230  
 Costrutto *if/else*, 229, c.14  
 in ARM assembly, 229-230  
 in C, c.14  
 in HDL, 145-147

**D**

Data sheet, 404-408  
 Datapath  
 processore ARM a ciclo singolo, 287  
 istruzione B, 292-293  
 istruzione LDR, 288-290  
 istruzione STR, 290  
 processore ARM multi ciclo, 300-305  
 istruzione B, 305  
 istruzione LDR, 301-303  
 istruzione STR, 304  
 processore ARM pipeline, 318-319  
 De Morgan, Augustus, 42  
 Decoder,  
 definizione di, 61-62  
 HDL per,  
 comportamento, 147  
 parametrico, 162  
 logica a decoder, 62  
 Decoder dell'ALU, 293-295  
 Decoder principale, 293, 295t, 298  
 Dennard, Robert, 198  
 Descrittore del segmento, 274  
 Dettagli, 257  
 caricamento dei literal, 257  
 eccezioni, 258  
 NOP, 258  
*Device under test* (DUT), 163  
 Dhrystone, 286  
 Dice, 22  
 Dielettrico, 22  
 Dimensione del blocco (*b*), 376  
 Diodi, 21  
 giunzione p-n, 21f  
 Dipendenza di controllo, 322, 325-327  
 Dipendenza di tipo RAW (*Read After Write*), 321, 350.  
 Dipendenza di tipo WAR, 350. *Vedi anche* Dipendenze  
 Dipendenza di tipo WAW, 350. *Vedi anche* Dipendenze  
 Dipendenza di uscita, 350  
 Dipendenze, 321-330  
 dipendenza di controllo, 322, 325-327  
 dipendenza di dato, 322  
 gestione  
 delle dipendenze di controllo, 325-327  
 inoltro, 322  
 stalli, 324  
 processore pipeline, 321  
*read after write* (RAW), 321, 349

*write after read* (WAR), 349  
*write after write* (WAW), 349  
 Dipendenze di dato, 322-324  
 HDL, 342  
 Dischi a stato solido (SSD), 368. *Vedi anche* Dischi rigidi  
 Dischi rigidi (piatti), 382  
 Disciplina  
 dinamica, 99-100. *Vedi anche* Analisi temporale  
 stabile, 99-100  
 Disciplina statica, 19-20  
 Disco rigido, 368, 382  
 Display di caratteri a cristalli liquidi (LCD), e.26-e.27  
 Dispositivi logici programmabili semplici (SPLD), 205  
 Divisione  
 circuiti, 188-189  
 Divisione in virgola mobile (*Floating-Point Division*,  
 FDIV), 192  
 Divisore, 189f  
 Double, tipo C, c.7  
*Double-data rate memory* (DDR), 200, e.47, e.49  
 DRAM. *Vedi* Memoria ad accesso casuale dinamica  
 (DRAM)  
 Driver di dispositivo, e.4-e.6

**E**

EasyPIO, e.5  
 Eccezioni, 258-261  
 accensione, 261  
 banchi di registri, 259  
 gestori delle, 253, 260-261  
 istruzioni relative alla eccezioni, 261  
 modi di esecuzione e livelli di privilegio, 259  
 tabella del vettore delle eccezioni, 259  
 Elaborazione di segnali digitali (DSP), 187, 261, 263  
*Electrically erasable programmable read only memory*  
 (EEPROM), 202  
 Elementi di ingresso/uscita (IOE), 206  
 Elementi logici (LE), 206-209  
 di Cyclone IV, 206-208, 207f  
 funzioni costruite utilizzando, 208  
 Elemento bistabile, 73  
 Encoder ad albero, e.35, e.38, e.38f  
 EOR (XOR), 224  
*Erasable programmable read only memory* (EPROM), 202,  
 399  
 Errori frequenti in C, c.36-c.39  
 Esecuzione fuori ordine, 351f  
 Esponente con codifica a eccesso, 191  
 Espressioni booleane, 37-40  
 forma prodotto di somme, 39  
 forma somma di prodotti, 38-39  
 Estensione  
 del segno, 13  
 di un immediato, 338  
 Ethernet, e.49  
 Exit, c.31  
*Extended instruction pointer* (EIP), 270  
*ExtImm*, 288

## F

FALSO, 6, 15, 16, 27, 39, 74, 78, 141  
 Famiglie logiche, 19, 408-411, 409f, 410t  
     compatibilità delle, 19  
     livelli logici delle, 19  
     parametri, 409f, 410t  
 FFT. *Vedi* Trasformata veloce di Fourier (FFT)  
 Fili, 45  
 Flag, 184  
 Flag di condizione, 227-229, 227t  
     istruzioni ARM, 430, 430t  
 Flag di stato, 272. *Vedi anche* Flag condizionale  
 Flip-flop, 77-80, 138-142. *Vedi anche* Registri  
     a livello di transistori, 79  
     attivato sui fronti di salita, 77  
     collegato direttamente, 102f, 108-112, 142f. *Vedi anche*  
         Sincronizzatori  
     con abilitazione, 78  
     confronto con latch, 80  
     conteggio dei transistori, 77, 79  
     HDL, 338. *Vedi anche* Registri  
     registri a scorrimento, 194-195  
     registro, 78  
     resettabile, 78-79  
     resettabile sincrono, 79  
     scansionabile, 196, 196f  
 Flip-flop asincrono resettabile, 78-79  
     HDL, 140-141  
 Float, in C, 5, 7t  
     Codici di format printf, 30t  
*Floating-Point Unit* (FPU), 192  
 Forma prodotto di somme, 43  
 Forma somma di prodotti, 38-39  
 Formati a precisione doppia, 191  
 Formati a precisione singola, 191-192. *Vedi anche* Numeri  
     in virgola mobile  
 Formati di istruzioni, ARM, 244-252  
     interpretazione, 250  
     istruzioni di accesso a memoria, 247  
     istruzioni di elaborazione dati, 244-245  
     istruzioni di salto, 248-249  
     modi di indirizzamento, 250  
     programma scritto in memoria, 251  
 Forme d'onda di simulazione, 124f  
     con ritardi, 135f  
 FPGA, 204-210  
 FPGA Cyclone IV, 206  
 FPGA Xilinx, 207. *Vedi anche* Registro di stato e controllo  
     della virgola mobile (FPSCR)  
 FPU. *Vedi* Floating-Point Unit (FPU)  
 Frequenza di campionamento, e.19  
*Front porch*, e.29  
 Fronte di salita, 62  
 FSK. *Vedi* Modulazione numerica a traslazione di  
     frequenza normale (FSK)  
 FSM. *Vedi* Macchine a stati finiti (FSM)  
 FSM Principale, 303-313  
*Full adder* (sommatore completo), 36, 145, 176, 181, 211

    facendo uso di always/process, 145

## Funzione

    cancellaU, C.27  
 Delaymicros, e.19  
 foglia, 240  
 Genwaves, e.21  
 main in C, c.3  
 malloc, c.26  
 niente, 235  
 Funzione di chiamata fattoriale, 241-242  
     durante lo stack, 242f  
 Furber, Steve, 356, 357

## G

Gerarchia, 3  
 Gerarchia di memoria, 368, 368f  
 Gestione di file, in C, c.31  
 Gestire la complessità, 2-5  
     astrazione digitale, 2-3  
     disciplina, 3  
     gerarchia, 3  
     modularità, 4  
     regolarità, 4  
 Gray, Frank, 53

## H

*Half adder* (semisommatore), 176, 176f  
*handshaking* a livello hardware, e.14  
 HDL. *Vedi* Linguaggi di descrizione dell'hardware (HDL)  
 Heap, 253  
 Hit, 367  
 Hopper, Grace, 253

## I

I/O analogici, e.19-e.25,  
     conversione A/D, e.24-e.25  
     conversione D/A, e.20  
     pulse-width modulation (PWM), e.20  
 I/O digitali di uso generale (GPIO), e.7-e.9  
     esempio di LED e switch, e.7f  
 I/O mappato in memoria, e.1-e.3, e.6f  
     comunicazione con i sistemi di I/O, e.2  
     hardware, e.2, e.2f  
     segnale address, e.2, e.2f  
 I/O nei sistemi embedded, e.3-e.25  
     I/O analogici, e.19-e.25  
         conversione A/D, e.24-e.25  
         conversione D/A, e.20-e.24  
     I/O digitali, e.7-e.9  
     I/O digitali di uso generale (GPIO), e.7-e.9  
     I/O seriale, e.9-e.18. *Vedi anche* I/O seriale  
     interrupt, e.25  
     periferiche di microcontrollori, e.26-e.27  
     timer, e.18-e.19  
 I/O parallelo, e.9  
 I/O seriale, e.9-e.18  
 IA-64, 275  
 Idiomi, 124

- Immediati, 221, 244-245. *Vedi anche* Costanti
  - Implicante primo, 44, 54
  - Indifferenza (X), 47, 57, 150
  - Indirizzamento
    - a offset, ARM, 233
    - base, 250
    - immediato, 250
    - post-indicizzato, ARM, 233
    - pre-indicizzato, ARM, 233
    - relativo al PC, 249-250
    - spazio di, 388
  - Indirizzo. *Vedi anche* Memoria
    - di destinazione del salto (BTA), 248
    - fisico, 383-384
    - traduzione, 384
    - virtuale, 383. *Vedi anche* Memoria virtuale
  - Induzione matematica perfetta, dimostrazione dei teoremi usando, 43
  - Informazione, quantità di, 5
  - Inizializzazione
    - delle variabili in C, c.9
    - di array in C, c.18-c.19
  - Inoltro, 322-324
  - Institute of Electrical and Electronics Engineers (IEEE), 191
  - Intel. *Vedi* x86
  - Intel x86. *Vedi* x86
  - Interfaccia a memoria, 365
  - Interfaccia a memoria e periferica, e.44-e.47
  - Interfacce a bus, c.43-c.47
    - AHB-Lite, c.43-c.44
    - esempio di interfaccia a memoria periferica, c.44-c.47
  - Interrupt, 258, e.25
  - IRWrite, 301, 308
  - Istruzione
    - a 32 bit, 244
    - case, in HDL, 145
    - di assegnamento continuo, 126, 139, 144, 151
    - di assegnamento di segnale concorrente, 126, 130-131
    - di salto se minore (BLT), 248
    - imm12, 247
    - imm24, 248
    - imm8, 245, 246
    - op, 245
    - supervisor call (SVC), 261
    - WFE, 267
    - WFI, 267
  - Istruzioni, ARM, 217-269, 427-430
    - di accesso a memoria, 222-224, 232-235, 429
    - di elaborazione dati, 427
    - di moltiplicazione, 225, 427
    - di salto, 227-228, 429
    - di traslazione, 225
    - flag di condizione, 226, 430
    - flag di condizione, 226-227, 430
    - formati
      - interpretazione, 250-251
      - istruzioni di accesso a memoria, 247-248
      - istruzioni di elaborazione dati, 244-245
      - istruzioni di salto, 248-250
      - modi di indirizzamento, 250
      - programma scritto in memoria, 251-251
    - istruzioni di elaborazione dati, 224-226
    - istruzioni di moltiplicazione, 225-226
    - istruzioni di traslazione, 225
    - istruzioni logiche, 224-225
    - istruzioni varie, 430
    - logiche, 224
    - memoria, 222-223
    - salto, 227-229
    - set di istruzioni, 217
    - varie, 430
  - Istruzioni, x86, 272, 274f
  - Istruzioni di accesso a memoria, 222-223, 232, 247-248, 288-290
  - Istruzioni di elaborazione dati, 428t
    - codifiche, 427f, 428f
    - istruzioni ARM, 244-247, 291-292, 427-428
    - codifica, 247-248, 429f
  - Istruzioni di moltiplicazione, 225-226, 427, 428t
  - Istruzioni di moltiplicazione e moltiplicazione con accumulo, 265t
  - Istruzioni di salto, 227-229
    - istruzioni ARM, 429
  - Istruzioni di traslazione, 225, 225f
  - Istruzioni in virgola mobile, ARM, 266
  - Istruzioni logiche, 224
  - Istruzioni per il risparmio di potenza e per la sicurezza, 267
  - Istruzioni relative alle eccezioni, 261
  - Istruzioni SIMD, 268-269
- J**
- Java, 224
- K**
- Karnaugh, Maurice, 52
  - Kilobit (Kb/Kbit), 10
  - Kilobyte (KB), 10
- L**
- Land grid array, e.47
  - Last-in-first-out (LIFO), 237-253
  - Latch, 75-76
    - a livello di transistori, 79
    - D, 76, 77f
    - e flip-flop, 73, 80
    - SR, 75, 75f
  - Latch slave, 77. *Vedi anche* Flip-flop
  - Latch SR, 75-76
  - Latenza, 112-115, 316, 324
  - Latenza di due cicli per LDR, 324
  - LCD. *Vedi* Display di caratteri a cristalli liquidi (LCD)
  - LDR, 222-223, 244, 256-258, 288-293, 429
    - percorso critico per, 299f
  - LE. *Vedi* Elementi logici (LE)

- Legge di Amdahl, 370
- Legge di Moore, 24
- Letterali (literal), 38, 68
  - caricamento, 257
- Libreria `stdlib.h`, C, c.29. *Vedi anche* Librerie standard
- Librerie standard, c.29-c.35
  - `math`, c.34
  - `stdio`, c.29
    - gestione di file, c.31
    - `printf`, c.29
    - `scanf`, c.31
  - `stdlib`, c.33
    - conversione di formato (`atoi`, `atol`, `atof`), c.34
    - `exit`, c.33
    - `rand`, `srand`, c.33
  - `string`, c.35
- Linea di bit, 197
- Linea di parola, 197
- Linee di trasmissione, 414-425
  - coefficiente di riflessione ( $k_r$ ), 423-424
    - derivazione del, 423-424
  - impedenza caratteristica ( $Z_0$ ), 416
    - derivazione del, 423-424
  - terminazione adattata, 416-417
  - terminazione aperta, 417-418
  - terminazione cortocircuitata, 418
  - terminazione non adattata, 418-419
  - terminazioni in serie e in parallelo, 421-422
- Linguaggi di descrizione dell'hardware (HDL), 121, 330-343,
  - blocchi costruttivi del processore, 337-339
  - blocco di registri, 337
  - capacità, 380
  - estensione di un immediato, 338
  - flip-flop resettabile, 338
  - flip-flop resettabile con segnale di abilitazione, 339
  - istruzione `if`, 145-147
    - numeri, 132
    - operatori e precedenza, 127, 131
    - operatori di riduzione, 127
    - variabili interne, 129-131
  - logica combinatoria, 122, 143
    - operatori a singolo bit, 125-126
    - assegnamenti bloccanti e non bloccanti, 147
    - istruzione `case`, 145
    - assegnamento condizionale, 128
    - ritardi, 134
  - logica sequenziale, 138-143, 153-156
  - memoria dati, 342
  - memoria istruzioni, 342
  - modellazione strutturale, 135-138
  - moduli, 121-122
  - moduli parametrici, 160
  - modulo di primo livello, 341
  - multiplexer 2:1, 339
  - origine dei, 122-123
  - processore ARM a ciclo singolo, 331-336
  - simulazione e sintesi, 123
    - sommatore, 337
    - testbench, 163-167, 339
    - tipi di dati, 157
- Linguaggi di programmazione di alto livello, 224, c.2
  - compilare, assemblare e caricare, 252-257
  - tradurre in linguaggio assembly, 221
- Linguaggio. *Vedi anche* Istruzioni
  - assembly, 218-224
  - macchina, 244-252
  - mnemonico, 219
- Linguaggio assembly, ARM, 217-261
  - istruzioni, 218-261, 427-430
  - operandi, 220-224
  - traduzione da codice ad alto livello a, 252
  - traduzione dal linguaggio macchina a, 250
- Linguaggio macchina, 244
  - interpretare il, 250
  - istruzioni di accesso a memoria, 247
  - istruzioni di elaborazione dati, 244-247
  - istruzioni di salto, 248-249
  - modi di indirizzamento, 250
  - programma scritto in memoria, 251, 252f
  - tradurre dal linguaggio macchina ad assembly, 250
- Linker, 252, 255
- Linux, e.18
- Liste collegate, c.27-c.28
- Livelli logici, 17
- Livello più alto di tensione ( $V_{DD}$ ), 17
- Load register instruction* (LDR), 222-223
- Località, 366
- Località spaziale, 366, 377
- Località temporale, 366, 370-371, 376, 387
- Logica
  - a due livelli, 47
  - famiglie, 19, 408-411, 409f, 410t
  - porte. *Vedi* Porte
  - programmabile, 398-404
  - riduzione dell'hardware, 48-49
  - spingere le bolle, 49-50
- Logica 74xx, 397-398
  - parti
    - 2:1 mux (74157), 400f
    - 3:8 decoder (74138), 400f
    - 4:1 mux (74153), 400f
    - AND (7408), 399f
    - AND3 (7411), 399f
    - AND4 (7421), 399f
    - counter (74161, 74163), 400f
    - FLOP (7474), 399f
    - NAND (7400), 399f
    - NOR (7402), 399f
    - NOT (7404), 399f
    - OR (7432), 399f
    - register (74377), 400f
    - tristate buffer (74244), 400f
    - XOR (7486), 399f
- Logica a due livelli, 47
- Logica booleana, 6. *Vedi anche* Algebra booleana



- Logica CMOS a bassa tensione (LVCMOS), 19
  - Logica combinatoria, 122
    - progetto di, 35-72
      - a due livelli, 47
      - algebra booleana, 40-45
      - blocchi costruttivi, 58-62, 175-189
      - espressioni booleane, 37-40
      - indifferenze, 57
      - mappe di Karnaugh, 52-58
      - multi livello, 47-50
      - precedenza, 38
      - ritardi, 62-65
      - temporizzazione, 62-67
  - Logica combinatoria a più livelli, 47-50. *Vedi anche* Logica
  - Logica condizionale, 295-296, 306-308
  - Logica del PC, 295
  - Logica pseudo-nMOS, 26, 26f
    - porta NOR, 26f
    - ROM e PLA, 209-210
  - Logica sequenziale, 73-120, 193-196
    - contatori, 193
    - flip-flop, 77-80. *Vedi anche* Registri
    - latch, 75-77
      - D, 76
      - SR, 75
    - registri. *Vedi* Registri
    - registri a scorrimento, 194
  - Logica Transistore-Transistore (TTL), 19, 408
  - Logica TTL a bassa tensione (LVTTTL), 19
  - Lookup table, 202, 206-207
  - Lovelace, Ada, 251
  - lsb. *Vedi* Bit meno significativo (lsb)
  - LSB. *Vedi* Byte meno significativo (LSB)
  - LSL, 225
  - LSR, 225
- M**
- MAC. *Vedi* Moltiplicazione con accumulo (MAC)
  - Macchine a stati con fattorizzazione, 94-95
  - Macchine a stati finiti (FSM), 84-98, 153-160, 403
    - codifica degli stati, 89-91
    - configurazione degli LE per, 208-209
    - contatore a modulo 3 (divisore-per-3), 90, 154
    - derivazione dallo schema circuitale, 96
    - diagramma degli stati, 86
    - fattorizzazione, 94
    - FSM alla Mealy, 92-94
    - FSM alla Moore, 92-94
    - FSM per semafori per il traffico, 85-89
    - in HDL, 153-157
    - unità di controllo multi ciclo, 314f
  - Macchine alla Mealy, 84, 85f, 92-94
    - diagramma degli stati, 93f
    - diagramma dei tempi per, 94
    - tabella delle transizioni e delle uscite, 93t
  - Macchine alla Moore, 84, 92
    - diagramma degli stati, 93f
    - diagrammi dei tempi, 94f
    - tabelle delle transizioni e delle uscite, 93t
  - Mancanza di pagina, 384
  - Mantissa, 190
  - Mappa di memoria, ARM, 252-253, e.2
  - Mappe di Karnaugh, 52-58
    - implicante primo, 44, 54-56, 66
    - minimizzazione logica con le, 54-55
    - transcodificatore per display a sette segmenti, 55-56
      - con indifferenze, 57-58
  - Margini di rumore, 17-18, 17f
    - calcolo, 18
  - Masuoka, Fujio, 202
  - math.h, libreria C, c.34
  - Matrici logiche, 204-210
    - implementazione a livello di transistori, 209-210
  - Matrici logiche programmabili (PLA), 46, 204-205, 398-402
  - Maxtermine, 38
  - Memoria, 232,
    - area e ritardo, 199-200
    - big-endian, 223
    - byte e caratteri, 233
    - dati, 285
    - fisica, 383
    - gerarchia, 368f
    - HDL, 203, 204, 342-343
    - indirizzabile a byte, 222
    - istruzioni (IM), 317
    - little-endian, 223
    - modi di indirizzamento, 271t
    - operandi in, 222
    - porte di, 198
    - principale, 367
    - protezione della, 388. *Vedi anche* Memoria virtuale
    - tempo di accesso, 369
    - tipi di, 198
      - banco di registri, 200
      - DDR, 200
      - DRAM, 199
      - flash, 202
      - ROM, 198
      - SRAM, 199
    - uso di funzioni logiche, 202
    - virtuale, 368. *Vedi anche* Memoria virtuale
  - Memoria a sola lettura (ROM), 196, 198, 200
  - Memoria ad accesso casuale (RAM), 199, 202-203, 203f
  - Memoria ad accesso casuale dinamica (DRAM), 196, 199, 365, e.47, e.49
  - Memoria ad accesso casuale statica (SRAM), 196, 199, 366
  - Memoria indirizzabile a byte, 222
    - big-endian, 223
    - little-endian, 223
  - Memoria virtuale, 382-391
    - mancanza di pagina, 384
    - numero di pagina, 384
    - pagine, 383-384
    - politiche di scrittura, 381
    - politiche di sostituzione, 389



- protezione della memoria, 388-389
- spiazzamento di pagina, 384
- tabella delle pagine, 386
- tabelle delle pagine multi livello, 389
- traduzione dell'indirizzo, 384
- translation lookaside buffer* (TLB), 387
- MemtoReg*, 291
- MemWrite*, 290, 293
- Metastabilità, 107-108
  - sincronizzatori, 108-110
  - stato metastabile, 74, 107
  - tempo di stabilizzazione, 107, 110-112
- Micro Controller Unit* (MCU), e.3
- Micro operazioni, 344-345
  - ad alte prestazioni, 343
  - progettisti, 343
- Microarchitettura, 218, 285-286. *Vedi anche* Architettura
  - analisi delle prestazioni, 286. *Vedi anche* Analisi delle prestazioni
  - descrizione della, 283-286
  - evoluzione della, 355-359
  - progettazione, 284-285
  - rappresentazione HDL, 330, 343
    - altri blocchi costruttivi, 337-339
    - processore a ciclo singolo, 331-337
    - testbench, 339-343
  - uno sguardo al mondo reale, 355
- Microarchitettura a ciclo singolo, 285
- Microarchitettura avanzata, 343-354
  - micro operazioni. *Vedi* micro operazioni
  - multiprocessori. *Vedi* Multiprocessori
  - multiprocessori eterogenei. *Vedi* Multiprocessori eterogenei
  - multiprocessori simmetrici. *Vedi* Multiprocessori simmetrici
  - multithreading*. *Vedi* Multithreading
  - pipeline lunghe. *Vedi* Pipeline lunghe
  - previsione dei salti. *Vedi* Previsione dei salti
  - processore *out-of-order*. *Vedi* Processore *out-of-order*
  - processore superscalare. *Vedi* Processore superscalare
  - ridenominazione dei registri. *Vedi* Ridenominazione dei registri
  - single instruction multiple data*. *Vedi* Single instruction multiple data (SIMD)
- Microarchitettura multi ciclo, 286
- Microarchitettura pipeline. *Vedi* Processore ARM pipeline
- Microcontrollori, e.3, e.20
- Microprocessore ARM, 283
  - a ciclo singolo, 287-299
  - memoria dati, 283-285
  - memoria istruzioni, 283-285
  - multi ciclo, 300-315
  - pipeline, 316-330
  - program counter, 283-285
  - register file, 283-285
  - stati degli elementi di, 283-285
- Microprocessori, 1, 9, 217
  - stato di esecuzione dei, 252
- Microprocessori ad alte prestazioni, 343
- Milioni di istruzioni per secondo, 316
- Mintermine, 38
- Miss, 367
  - di capacità, 380
  - di conflitto, 380
  - inevitabile, 380
- Mnemonici di condizione, 227t
- Modo di indirizzamento, ARM, 250
  - a registro, 250t
  - base, 250t
  - immediato, 250t
  - relativo al PC, 250t
- Modularità, 4
- Modulazione di ampiezza impulsi (PWM), e.20
  - duty cycle*, e.22, e.22f
  - segnale, e.22f
  - uscita analogica con, e.23-e.24
- Modulazione numerica a traslazione di frequenza normale (FSK), e.34
  - e forme d'onda GFSK, e.34f
- Moduli, in HDL
  - comportamentali e strutturali, 121-122
  - moduli parametrici, 160
- Modulo BlueSMiRF, e.34, 3.34f
- Modulo comportamentale, 121-122
- Modulo di memoria *dual inline* (DIMM), e.49
- Modulo strutturale, 121-122, 135-138
- Moltiplicatore, 186
  - con segno, 160
  - senza segno, 160, 186-187
- Moltiplicazione, 186
- Moltiplicazione con accumulo (MAC), 263, 266t
- Monitor VGA (*Video Graphics Array*), e.29-e.33
  - contatti del connettore, e.30f
  - driver per, e.32-e.33
- Moore, Gordon, 24
- Motore analitico, 5
- Motore passo-passo bipolare, e.41f-e.42f
  - a pilotaggio diretto, 41f
  - AIRPAX LB82773-M1, e.41, e.41f
- Motori
  - DC, e.35-e.38, e.35f
  - H-bridge, e.35, e.36f, e.37t
  - passo-passo, e.35, e.40-e.43, e.41f
  - servo, e.38-39
- Motori in corrente continua (DC), e.35-e.38, e.35f
  - encoder ad albero, e.35
  - H-bridge, e.35-e.36, e.36f
- Motori passo-passo, e.40-e.43
  - bipolari, e.40, e.40f
  - pilotaggio a due fasi, e.40f, e.41
  - pilotaggio a mezzo passo, e.40f, e.41
  - pilotaggio a onda, e.40f, e.41
- MOV, 222
- msb. *Vedi* Bit più significativo (msb)
- MSB. *Vedi* Byte più significativo (MSB)

Multiplexer, 58-61  
 definizione di, 58-59  
 HDL  
   a *N* bit parametrici, 161  
   modellizzazione strutturale, 135-138  
   modello di comportamento, 128-129  
   simbolo e tabella delle verità, 58f  
 Multiprocessori, 353  
   chip, 353  
   eterogenei, 353-354  
   simmetrici, 353  
 Multiprocessori  
   a cluster, 354  
   eterogenei, 353, 354  
   simmetrici, 353  
   simmetrici, 353  
*Multi-Protocol Synchronous Serial Engine* (MPSSE), e.51, e.51f  
*Multithreading*, 352-353  
 Mux. *Vedi* Multiplexer  
 myDAQ, e.51, e.51f

## N

NAND (7400), 399f  
 Negatori (inverter), 15, 73, 81f, 125. *Vedi anche* Porte NOT  
   collegati a croce, 73, 74f  
   comportamento bistabile di, 74f  
   in HDL, 125, 144  
 Nibble, 9  
 Nodo di uscita fluttuante, 79  
 NOP, 258  
*Not a number* (NaN), 191  
 Notazione a trattino, 36f  
 Noyce, Robert, 20  
 Numeri binari,  
   e numeri decimali, 7-8  
   relativi, 11-15  
 Numeri  
   binari relativi, 11-14  
   decimali, 6  
   esadecimali, 8-9  
   in complemento a due, 12  
   in modulo e segno, 11-12, 189  
   in virgola fissa, 189-190  
   non relativi, 13  
 Numeri in virgola mobile, 190-192  
   addizione, 192  
   arrotondamento, 192  
   casi speciali,  
     infinito, 191  
     NaN, 191  
   formati a precisione singola e doppia, 191-192  
 Numero di cicli per istruzione (CPI), 287  
 Numero di istruzioni per ciclo (IPC), 287  
 Numero di pagina, 384  
   fisica, 384  
   virtuale (NPV), 386

## O

Offset (spiazzamento), 223, 288, 301, 301f  
*One-time programmable* (OTP), 398  
 Operandi  
   ARM, 220  
     registri, 220  
     registri ARM, 221  
   x86, 270, 270f  
 Operatore condizionale, 128  
 Operatore ternario, 128, c.10t  
 Operatori  
   in C, c.10-c.12  
   in HDL, 125-135  
     a singolo bit, 125  
     di riduzione, 127  
     precedenza, 131, 131t  
     tabella degli, 131t  
     ternari, 128  
 ORR (OR), 224  
 Oscillatore ad anello, 81  
 Ossido, 20

## P

Package, chip, 411  
 Package *Thin small outline* (TSOP), 412  
 Pagina fisica, 383  
 Pagine, 383  
 Pagine virtuali, 383  
*Paging*, 389  
 Parallelismo, 112-115  
 Parallelismo a livello di istruzioni (ILP), 351  
 Parallelismo spaziale, 112-113  
 Parallelismo temporale, 113-114  
 Parametri/chiamate, c.12-c.13  
   passaggio per riferimento, c.18  
   passaggio per valore, c.18  
 Passaggio per riferimento, c.18  
 Passaggio per valore, c.18  
 PCI express (PCIe), e.49  
*PCSrc*, 290, 291f, 327  
*PCWrite*, 303  
 Penalizzazione di miss, 376  
 Penalizzazione per salto mal previsto, 325, 345  
 Percorsi ciclici, 82  
 Percorso critico, 63  
 Percorso dati a 32 bit, 284  
 Percorso minimo, 63-64, 63f  
 Periferiche di microcontrollori, e.26-e.27  
   comunicazioni wireless Bluetooth, e.33  
   controllo di motori, e.35-e.43  
   display di caratteri a cristalli liquidi, e.26  
   controllo, e.27  
   interfaccia parallela, e.26  
   monitor VGA, e.29-e.33  
 Periodo di clock, 100, 287  
 Peripheral Component Interconnect (PCI), e.48  
*Phase Locked Loop* (PLL), e.31  
 Pilotaggio a tensione diretta, e.41

- Pilotaggio con modulatore a corrente costante, e.42
- Pilotaggio del motore bipolare, e.40
- Pipeline lunghe, 344
- Pipelining*, 113
- Plastic Leaded Chip Carriers* (PLCC), 412
- Politica *Least recently used* (LRU), 378
  - in una cache associative a due vie, 378
- Politiche di scrittura, 381
  - write-back*, 381
  - write-through*, 381
- Politiche di sostituzione, 389
- Porta AND, 15, 15f, 127
  - chip (7408, 7411, 7421), 397, 399f
  - tabella delle verità, 14
  - uso dei transistori CMOS, 25
- Porta NAND, 16
  - CMOS, 24-25
- Porta NOR, 16, 42f, 399f
  - chip (7402), 399f
  - CMOS, 25
  - logica pseudo-nMOS, 26f
  - tabella delle verità, 17f
- Porta ricevitore, 17
- Porte
  - a ingressi multipli, 16
  - AND, 15, 16, 88
  - AND-OR (AO), 32
  - buffer, 15
  - NAND, 16, 24
  - NOR, 16, 75, 88
  - NOT, 15
    - chip (7404), 399f
    - CMOS, 24
  - OR, 15
  - OR-AND-INVERT (OAI), 33, 33f
  - XNOR, 16
  - XOR, 15
- Porte di trasmissione, 25-26
- Porte logiche programmabili sul campo (FPGA), 204-208,  
398, 402-404, e.10-e.11, e.30
  - di pilotaggio di un monitor VGA, e.30f
  - in interfaccia SPI, e.10
- Potenza dinamica, 26
- Potenza statica, 26
- Predittore dinamico a un bit, 346
- Predittore dinamico dei salti a due bit, 346
- Previsione dei salti, 346-347
- Previsione dinamica dei salti, 346
- Previsione statica dei salti, 346
- `printf`, c.29-c.30
  - priority encoder, 46
  - rete logica di priorità, 46
- Processore ARM a ciclo singolo, 287-299, 331
  - controller, 332
  - decoder, 333
  - istruzioni, 296-297
  - logica condizionale, 334-335
  - percorso dati (datapath), 287, 335
  - istruzione B, 292
  - istruzioni di elaborazione dati, 291
  - istruzione LDR, 288-290
  - istruzione STR, 290
  - prestazioni, 298
  - unità di controllo, 293
- Processore ARM a ciclo singolo HDL, 331-343
  - blocchi costitutivi, 337-339
  - controller, 330
  - datapath, 330
  - testbench, 339-343
- Processore ARM pipeline, 316-330
  - analisi delle prestazioni, 328-330
  - capacità di lavoro, 316
  - descrizione, 316-318
  - dipendenze, 321-328
  - percorso dati, 318
  - rappresentazione schematica del, 318f
  - unità di controllo, 319
- Processore Intel, 269
- Processore multi ciclo ARM, 300
  - percorso dati, 300-306
    - istruzione B, 305
    - istruzione LDR, 301-303
    - istruzione STR, 304
    - istruzioni di elaborazione dati, 304
  - prestazioni, 313
  - unità di controllo, 306-313
- Processore *multithread*, 353
- Processore *out-of-order*, 349-350
- Processore superscalare, 347-349, 347f, 348f
- Processore vettoriale, 347
- Processori ARM, 355
- Prodotti parziali, 186
- Progettazione assistita del calcolatore (*computer-aided design*, CAD), 49, 89
- Program counter (PC), 228, 251, 284
- Programma scritto in memoria, 251-252
- Programmable logic devices* (PLD), 402
- Programmable read only memories* (PROM), 201-202,  
398-401
- Programmare
  - costrutti di selezione, 229-232
  - flag di condizione, 226-227
  - in ARM, 224
  - istruzioni aritmetiche e logiche, 224-226
  - istruzioni di traslazione, 225
  - memoria, 232-235
  - salti, 230-231
- Programmazione in C, c.1-c.39
  - esecuzione di un programma, c.3-c.4
  - un semplice programma, c.3
- Protocollo TCP/IP, e.49
- Pseudo-istruzione, 258
- Pull-up debole, 26
- Puntatori, c.17-c.18, c.24, c.28
- Punti di guadagno unitario, 19

- R**
- RAM. *Vedi* Memoria ad accesso casuale
  - Rand, c.33
  - Raspberry Pi, e.3-e.4, e.4f, e.5f, e.34f
  - Realizzazione dei sistemi digitali, 397-426
    - aspetti economici, 425-426
    - assemblaggio, 411-414
    - breadboard, 412-413
    - circuiti integrati specifici per un'applicazione (ASIC), 404
    - data sheet, 404-408
    - famiglie logiche, 408-411
    - logica programmabile, 398-401
    - packaging, 411-414
    - schede di circuito stampato, 413-414
  - Register file
    - descrizione dei registri ARM, 220
    - HDL, 337
    - nei processori ARM pipeline, 318
    - schematico, 200
    - utilizzo nei processori ARM, 284
  - Registri. *Vedi anche* Registri ARM, Flip-flop
    - a scorrimento, 194
    - con abilitazione, 141-142. *Vedi anche* Flip-flop
    - non preservati, 238-239
    - per dati, 221
    - preservati, 238-240, 240t
    - resettabili, 140
    - salvataggio e ripristino, 238
  - Registri a salvataggio di chiamante, 239
  - Registri a salvataggio di chiamato, 239
  - Registri ARM, 220-221
    - program counter, 228, 284-285
    - register file, 284-285
    - set di registro, 220
  - Registro destinazione (rdort), 289, 302
  - Registro di stato corrente nel programma (CPSR), 226, 261
  - Registro di stato e controllo della virgola mobile (FPSCR), 267
  - Registro EFLAGS, 272
  - Registro istruzioni (IR), 308
  - Regola di salvataggio del chiamante, 240
  - Regola di salvataggio del chiamato, 240
  - Regolarità, 4
  - RegSrc, 292
  - RegWrite, 289, 295, 322
  - Reti asincrone, 84
  - Reti logiche sincrone, progetto, 81-84
  - Reti sequenziali sincrone, 82-83. *Vedi anche* Macchine a stati finiti (FSM)
  - Reti sincrone, 84
  - Reti stabili, 81
  - Reticolo, silicio, 20-21, 21f
  - Ridenominazione dei registri, 351-352
  - Riduzione dell'hardware, 48
  - Riga di comando, compilatore e argomenti, 253-254, c.35-c.36
  - Ritardi, porte logiche. *Vedi anche* Ritardo di propagazione in HDL (solo simulazione), 134-135
  - Ritardi di propagazione, 62-64.
  - Ritardo di contaminazione, 62
  - ROM. *Vedi* Memoria a sola lettura
  - ROM programmabile a fusibili, 201
  - ROR, 225
  - Rotatori, 185-186
  - Rotazioni al secondo (RPM), e.35
  - RS-232, e.13
- S**
- Salto, 227-229, 248-249
    - condizionato, 227, 228
    - incondizionato, 228
  - scanf, c.31
  - Scheda di circuito stampato (PCB), 413
  - Scheda prototipale (breadboard), 412
  - SDRAM. *Vedi Synchronous dynamic random access memory* (SDRAM)
  - Segmentazione, 275
  - Segmento dati, 253
  - Segmento dati dinamici, 253
    - disciplina dinamica, 99-100. *Vedi anche* Analisi temporale
  - Segmento dati globali, 253
  - Segmento testo, 252
  - Segnalazione bipolare, e.14
  - Segnale di abilitazione attivo alto, 51
  - Segnale di propagazione, 177
  - Segnali di controllo, 64, 184
  - Segnali di generazione, 177, 178
  - Semiconduttore, 20
    - vendite delle industrie, 1
  - Semplificare le espressioni
    - usando l'algebra booleana, 43-45
    - usando le mappe di Karnaugh. *Vedi* Mappe di Karnaugh
  - Serial ATA (SATA), e.50
  - Serial Peripheral Interface (SPI), e.9-e.12
    - campi dei registri, e.10t
    - connessioni tra il Pi e un dispositivo FPGA, e.11f
    - forme d'onda, e.9f
    - porte
      - Serial Clock (SCK), e.9
      - Serial Data In (SDI), e.9
      - Serial Data Out (SDO), e.9
    - struttura circuitale e temporizzazioni dello slave, e.12f
  - Servomotore, e.35, e.38-e.39
  - Set di istruzioni, 217
    - ARM, 284
    - ARMv4, 261, 430
    - ARMv7, 357
    - Thumb, 262
  - Sfasamento del clock, 104-107, 105f, 106f
  - Simulazione logica, 123-124
  - Sincronizzatori, 108-109, 108f, 109f
  - Single instruction multiple data (SIMD), 268-269

- Sintesi logica, 123-124
- Sistemi di acquisizione dati (DAQ), e.50-e.51
  - myDAQ, e.51
- Sistemi di ingresso/uscita (I/O), e.1-e.52
  - driver del dispositivo, e.3, e.4-e.6
  - I/O mappato in memoria, e.1-e.2
  - registri di I/O, e.2
- Sistemi di memoria, 365
  - analisi delle prestazioni, 368
  - ARM, 382
- Sistemi I/O del PC, e.47-e.52
  - interconnessione in rete, e.49
  - memoria DDR3, e.49
  - PCI, e.48-e.49
  - SATA, e.50
  - sistemi di acquisizione dati, e.50-e.51
  - USB, e.51-e.52
- Sistemi numerici, 6-14
  - binari, 7-8
  - calcolare le potenze di due, 10
  - confronto tra, 14
  - esadecimali, 8-9, 8t
  - in complemento a due, 12-14
  - in modulo e segno, 11
  - in virgola fissa, 189, 189f
  - in virgola mobile, 190-193
    - addizione, 192
    - casi speciali, 191
    - non relativi, 11
    - positivi e negativi, 11
    - relativi, 11-14
- Somma, 10-11, 11-13, 175-181. *Vedi anche* Sommatori
  - binaria, 10-11
  - binaria relativa, 11-13
  - in virgola mobile, 192
- Sommatori, 175-181
  - a prefissi, 179
  - a propagazione di riporto a onda, 176
  - completo, 176
  - HDL per, 130, 145, 337
  - semisommatore, 176
  - sommatore a propagazione di riporto, 176
  - sommatore ad anticipazione di riporto, 177
- Sottrattore, 182
- Sottrazione, 13, 181-182, 219
- Sovraccarico di sequenziamento, 101, 105, 114, 330
- Spazio vuoto, 127
- SPEC, 286
- SPECINT2000, 315-328
- Spento, 20, 23f
- Spiazzamento
  - di blocco, 376
  - di byte, 372
  - di pagina, 384
- Squashing*, 350
- SRAM. *Vedi* Memoria ad accesso casuale statica
- srand, c.33
- Stack, 237
  - parametrici aggiuntivi, 243
  - registri preservati, 238
  - stack frame, 238, 244
  - stack pointer, 237
  - variabili locali, 243
- Stadio Decode, 316
- Stalli, 324. *Vedi anche* Dipendenze
- Stato di esecuzione/architetturale, 252
  - per ARM, 283-284
- Stato non architetturale, 283-284
- STR, 290
- string.h, libreria C, c.35
- Stringa, 235, c.23-c.24. *Vedi anche* Caratteri
- Strutture, c.24-c.25
- SUB, 219
- Substrato, 22
- Swap area, 389
- Symbol table, 255-256
- Synchronous dynamic random access memory (SDRAM), 200
  - DDR, 200
- SystemVerilog, 121-167,
  - accesso a parti di bus, 138
  - assegnamenti bloccanti e non bloccanti, 144
  - assegnamento condizionale, 128
  - buffer tristate, 133
  - circuito a priorità, 149
    - usando indifferenze, 147
  - commenti, 127
  - concentrazione di bit, 134
  - decoder, 147-148
  - FSM divisore-per-3, 154
  - full adder (sommatore completo), 130
    - facendo uso di always/process, 145
    - usando assegnamenti non bloccanti, 151
  - istruzione case, 145, 150
  - istruzione if, 145-146
  - latch, 143
  - logica combinatoria, 125-135, 143-153
  - macchine a stati finiti (FSM), 153-157
    - FSM alla Mealy, 156
    - FSM alla Moore, 155
  - modellazione strutturale, 135-138
  - moduli parametrici, 160-163
    - decoder N:2N, 162
    - multiplexer a N bit, 161
    - porta AND a N ingressi, 163
  - moltiplicatore, 160
  - multiplexer, 128-129, 136-137, 161-162
  - negatori, 125, 144
  - numeri, 132
  - operatori, 131, 131t
  - operatori di riduzione, 127
  - porte logiche, 126
  - registri, 138-143
    - con abilitazione, 142
    - resettabili, 140
  - ritardi (in simulazione), 134-135

- simulazione e sintesi, 123-125
  - sincronizzatore, 142
  - sincronizzatore con uso di assegnamenti bloccanti, 153
  - tabelle delle verità con ingressi non definiti o fluttuanti, 133-134
  - testbench, 163-167
    - con autoverifica, 165
    - con file di vettori di test, 166-167
    - semplice, 164
  - tipi di dati, 157-160
  - transcodificatore per display a sette segmenti, 146
  - usando la logica sequenziale, 138-143, 152-153
  - variabili interne, 129-131
  - Z e X, 132-134
- T**
- Tabella delle pagine, 386
  - Tabella delle verità, 14
    - con indifferenze, 47f, 57, 147
    - con ingressi non definiti o fluttuanti, 133
    - decoder dell'ALU, 294f, 298t
    - latch SR, 75, 76f
    - multiplexer, 58
    - transcodificatore per display a sette segmenti, 55
  - Tabelle delle pagine multi livello, 389-390
  - Tag, 372
  - Tasso di baud, e.13
  - Tasso di hit, 369
  - Tasso di miss, 369
    - e tempo di accesso, 369
  - Tempo di accesso a memoria (AMAT), 369
  - Tempo di esecuzione, 287
  - Tempo di stabilizzazione (risoluzione), 107. *Vedi anche* Metastabilità
    - formulazione del, 110
  - Tempo medio tra errori (MTBF), 109
  - Temporizzazioni
    - della logica sequenziale, 98-112
    - disciplina dinamica, 99
    - delle reti logiche combinatorie, 62-67
  - Tensione di soglia, 23
  - Teorema
    - dei complementi, 41
    - del consenso, 42, 43
    - dell'associatività, 41
    - dell'assorbimento, 42
    - dell'elemento nullo, 41
    - dell'idempotenza, 41
    - dell'identità, 41
    - dell'involuzione, 41
    - della combinazione, 42
    - della commutatività, 41
    - della distributività, 42
    - di De Morgan, 42
  - Terra/massa (GND), 17
    - simbolo per, 24
  - Testbench, 339-343
  - Testbench, HDL, 163-167
    - con autoverifica, 165
    - con file di vettori di test, 166-167
    - semplice, 164
  - Testina di lettura/scrittura, 382
  - Thread level parallelism* (TLP), 352
  - Timer, e.18
  - Tipi di dati, c.17-c.28
    - array. *Vedi* Array
    - caratteri. *Vedi* Caratteri
    - typedef, c.25-c.26
  - Traboccamento
    - con addizione, 11
    - identificazione, 185
  - Tradurre e lanciare un programma, 252f
  - Transistor a gate sommerso, 202. *Vedi anche* Memoria flash
  - Transistori, 20-26
    - a giunzione bipolare, 20
    - CMOS, 20-25
    - latch e flip-flop, 79-80
    - MOSFET, 20
    - nMOS, 22-24, 22f, 23f
    - pMOS, 22-24, 22f
      - porte di trasmissione, 25-26
      - pseudo-nMOS, 26, 26f
      - ROM e PLA, 209
      - porte fatte da, 24-26
  - Transistori a giunzione bipolare, 20
  - Transistori metallo-ossido-semiconduttore a effetto di campo (MOSFET), 20
    - modelli a interruttori, 23, 23f
  - Translation lookaside buffer* (TLB), 384, 387
  - Trasformata veloce di Fourier (FFT), 263
  - Traslatore logico, 185
  - Traslatori, 185-186
  - Tubo a raggi catodici (CRT), e.29. *Vedi anche* Monitor VGA (*Video Graphic Array*),
    - impulso di sincronizzazione orizzontale, e.29
    - impulso di sincronizzazione verticale, e.30
  - typedef, c.25-c.26
- U**
- Unicode, 233
  - Unit under test* (UUT), 163
  - Unità aritmetico/logica (ALU), 183-185
    - implementazione dell', 184
    - nei processori, 288-316
  - Unità di controllo, 284. *Vedi anche* Decoder ALU
    - del processore a ciclo singolo ARM, 293-297
    - del processore multi ciclo ARM, 306-313
    - del processore pipeline ARM, 320
  - Unità di memoria flash, 202
  - Universal Asynchronous Receiver Transmitter* (UART), e.13-e.17
    - Handshaking* a livello hardware, e.14
  - Universal Serial Bus* (USB), 202, e.9, e.14, e.48
    - USB 1.0, e.48
    - USB 2.0, e.48



USB 3.0, e.48  
 Uno più significativo implicito, 191  
 Upton, Eben, e.4  
 USB. *Vedi Universal Serial Bus (USB)*

## V

Valore di ritorno, 235  
 Valore fluttuante (Z), 51  
     in HDL, 132-134  
 Valore logico non valido, 132  
 Variabili dal valore discreto, 5  
 Variabili di stato, 73  
 Variabili in C, c.6-c.9  
     globali e locali, c.8  
     inizializzazione, c.9  
     tipi di dati primitivi, c.6  
 Variabili locali, 243  
 $V_{CC}$ , 17  
 $V_{DD}$ , 17  
 VERO, 6, 15-16, 27, 37-38, 48, 74, 78-79, 86, 124, 127, c.25  
*Very High Speed Integrated Circuits (VHSIC)*, 123  
*VHSIC Hardware Description Language (VHDL)*, 123  
     accesso a parti di bus, 138  
     assegnamenti bloccanti e non bloccanti, 144, 147-153  
     assegnamento condizionale, 128  
     buffer tristate, 133  
     circuito a priorità, 149  
         operatori di riduzione, 127  
         con indifferenze, 150  
         operatori di riduzione, 127  
     commenti, 127  
     concatenazione di bit, 134  
     decoder, 147-148, 162  
     FSM divisore-per-3, 154  
     *full adder* (sommatore completo), 130  
         usando *always/process*, 145  
         usando assegnamenti non bloccanti, 151  
     istruzione *case*, 145, 150  
     istruzione *if*, 145  
     latch, 143  
     macchine a stati finiti (FSM), 153-157  
         FSM alla Mealy, 156  
         FSM alla Moore, 155  
     modellazione strutturale, 135-138  
     moduli parametrici, 160-163  
         decoder  $N:2^N$ , 162  
         multiplexer a  $N$  bit, 161  
         porta AND a  $N$  ingressi, 163  
     moltiplicatore, 160  
     multiplexer, 128-129, 136-137, 161  
     negatori, 125, 144  
     numeri, 132

operatori, 131  
 porte logiche, 126  
 registri, 138-142  
     con abilitazione, 141  
     resettabili, 140  
 ritardi (in simulazione), 135  
 simulazione e sintesi, 123-125  
 sincronizzatore, 142  
 sincronizzatore con uso di assegnamenti bloccanti, 153  
 storia del, 123  
 tabelle delle verità con ingressi non definiti o fluttuanti, 133  
 testbench, 163-167  
     con autoverifica, 165  
     con file dei vettori di test, 166  
     semplice, 164  
 tipi di dati, 157-160  
 transcodificatore per display a sette segmenti, 146  
 usando la logica combinatoria, 125-135, 143-153, 160-162  
 usando la logica sequenziale, 138-143, 153-157  
 variabili interne, 129-131  
 Z e X, 132-133

Vincolo sul tempo di hold, 100-104  
     sfasamento del clock, 104-107  
 Vincolo sul tempo di setup, 100-104  
     con sfasamento del clock, 104-105  
     HDL, 146  
     indifferenze, 57  
     transcodificatore per display a sette segmenti, 55-57  
 Violazioni del tempo di hold, 102, 103f, 104, 106-107  
 $V_{SS}$ , 18

## W

Wafer, 22  
 Wall, Larry, 15  
 Whitmore, Georgiana, 5  
 Wi-Fi, e.50  
 Wilson, Sophie, 356

## X

x86  
     architettura, 270-275  
         codifica delle istruzioni, 272-274  
         condizioni di salto, 274t  
         flag di stato, 272  
         istruzioni, 272-274  
         modi di indirizzamento a memoria, 271t  
         operandi, 271  
         particolarità, 275  
         quadro generale, 275  
         registri, 270